SymIntegration

Moscato · Port · Shiraz · Merlot · Pinot Noir

1st Edition

Contents

Preface **10**

1 Introduction Story **13**

2 Symbolic Integral Computation and Calculus **21**
I Build and Install SymIntegration in GFreya OS . . . 22
II Techniques of Integration . . . 25
III Integration by Parts . . . 27
IV The Method of Substitution . . . 30
V Fraction Integral . . . 32
VI Trigonometry Integral . . . 43
i $\int \sin^n x\, dx$ . . . 46
ii $\int \cos^n x\, dx$ . . . 50
iii $\int \sec^n x\, dx$ . . . 53
iv $\int \csc^n x\, dx$ . . . 65
v $\int \cot^n x\, dx$ . . . 69
vi $\int \tan^n x\, dx$ . . . 78
vii $\int \sin^n x\ \cos^m x\, dx$ . . . 82
viii $\int \tan^n x\ \sec^m x\, dx$ . . . 99
ix $\int \cot^n x\ \csc^m x\, dx$ . . . 122
VII Trigonometry and Transcendentals Formula . . . 123
i Functions in SymIntegration Related to Integration with Transcendental and Trigonometry Functions . . . 126
VIII Infinite Series . . . 127
IX Vector Calculus . . . 130
X Complex Analysis . . . 140
i Functions in SymIntegration Related to Complex Analysis . . . 144

3 Differential Equations, Partial Differential Equations, and Boundary Value Problems **147**
I Solving First Order Ordinary Differential Equations . . . 149
II Second Order Linear Equations . . . 177
i Functions in SymIntegration Related to Second Order Linear Equations . . . 226
III Higher Order Linear Equations . . . 235
i Functions in SymIntegration Related to Higher Order Linear Equations . . . 263
IV Series Solutions of Second Order Linear Equations . . . 272
i Compute Derivatives of a Differential Equation' Solution at the given point x_0 334

ii Compute Power Series Solution for Homogeneous Second Order Linear Differential Equations with Constant Coefficients . . . 335
iii Compute Power Series Solution for Homogeneous Second Order Linear Differential Equations with Variable Coefficients Near an Ordinary Point x_0 342
iv Compute Power Series Solution for Homogeneous Second Order Linear Differential Equations with Variable Coefficients Near a Singular Point x_0 . 351
v Maclaurin Series to Handle Non-Elementary Integral . . . 385
vi Frobenius Method . . . 390
vii The Airy Differential Equation . . . 391
viii The Hermite Differential Equation . . . 393
ix Classical Orthogonal Polynomials . . . 400
x Functions in SymIntegration Related to Series Solutions of Second Order Linear Equations . . . 424
V The Laplace Transform . . . 441
i Elementary Laplace Transforms . . . 446
ii Determine Whether a Function f is Continuous, Piecewise Continuous, or Neither . . . 449
iii Compute the Initial Value Problem for Nonhomogeneous Linear Differential Equations with Laplace Transform . . . 450
iv Functions in SymIntegration Related to Laplace Transform . . . 452
VI Systems of First Order Linear Equations . . . 460
VII Numerical Methods . . . 461
VIII Nonlinear Differential Equations . . . 462
IX Partial Differential Equations . . . 463
i Simulate Three-Dimensional Heat Equation with SFML . . . 465
X Boundary Value Problems and Sturm-Liouville Theory . . . 469

4 Probability and Statistics **471**
I Processing and Summarizing Data with Statistics . . . 472
i Descriptive Statistics Computation . . . 472
II Random Variables and Probability Distributions . . . 473
i Discrete Probability Distributions . . . 473
ii Continuous Probability Distributions . . . 474
iii Joint Probability Distributions . . . 475
iv Covariance and Vectorized Moments . . . 481
v Linear Combinations and Transformations . . . 481
vi The Cholesky Decomposition and Generating Random Vectors . . . 482
vii Generate Random Number in SymIntegration . . . 483
viii Create Covariance Matrix in SymIntegration . . . 485
ix Create Cholesky Decomposition in SymIntegration . . . 487
III Discrete Probability Distributions . . . 488
i Functions in SymIntegration Related to Discrete Distribution . . . 492
IV Continuous Probability Distributions . . . 494
i Uniform Distribution . . . 494
ii Normal Distribution . . . 495
iii Gamma Distribution . . . 498
iv Exponential Distribution . . . 500
v Beta Distribution . . . 501

vi Chi-Squared Distribution . . . 508
vii t-Distribution . . . 509
viii F-Distribution . . . 510
ix Weibull Distribution and Hazard Rates . . . 513
x Cauchy Distribution . . . 514
xi Bivariate Normal . . . 515
xii Functions in SymIntegration Related to Continuous Distribution . . . 516
xiii Generate Gamma Distributed Random Number in SymIntegration and Plot the Histogram with Hamzstplot . . . 522

V Statistical Inference . . . 528
i Sampling from a Normal Population . . . 529
ii The Central Limit Theorem . . . 530
iii Point Estimation . . . 530
iv Confidence Interval as a Concept . . . 530
v Functions in SymIntegration Related to Confidence Interval . . . 541
vi Compute Confidence Interval for $\mu_1 - \mu_2$, σ_1^2 and σ_2^2 Known with SymIntegration . . . 544
vii Compute Confidence Interval for $\mu_1 - \mu_2$, $\sigma_1^2 = \sigma_2^2$ but Both Unknown with SymIntegration . . . 545
viii Compute Confidence Interval for $\mu_1 - \mu_2$, $\sigma_1^2 \neq \sigma_2^2$ and Both Unknown with SymIntegration . . . 547
ix Compute Confidence Interval for $\mu_D = \mu_1 - \mu_2$, Paired Observations with SymIntegration . . . 549
x Compute Confidence Interval for Proportion p with SymIntegration . . . 551
xi Compute Confidence Interval for the Difference Between Two Proportions $p_1 - p_2$ with SymIntegration . . . 553
xii Compute Confidence Interval for Variance σ^2 with SymIntegration . . . 555
xiii Compute Confidence Interval for Ratio of Two Variances $\frac{\sigma_1^2}{\sigma_2^2}$ with SymIntegration . . . 557
xiv Maximum Likelihood Estimation . . . 559
xv Compute Maximum Likelihood Estimation for Continuous Distribution . . . 565
xvi Bayesian Statistics . . . 567

VI Hypothesis Testing . . . 568
i One Sample Hypothesis Test for Comparing Means . . . 568
ii Functions in SymIntegration Related to Hypothesis Testing . . . 581
iii Compute Right-Tailed Hypothesis Testing with SymIntegration . . . 583
iv Compute Left-Tailed Hypothesis Testing with SymIntegration . . . 586
v Compute Two-Tailed Hypothesis Testing with SymIntegration . . . 588
vi Two Samples Hypothesis Tests for Comparing Means . . . 590
vii Compute Right-Tailed Hypothesis Testing on Two Means with Unknown But Equal Variances with SymIntegration . . . 594
viii Compute Two-Tailed Hypothesis Testing on Two Means with Known Variances with SymIntegration . . . 596
ix Compute Two-Tailed Hypothesis Testing on Two Means with Unequal and Unknown Variances with SymIntegration . . . 598
x Compute Two-Tailed Hypothesis Testing on Paired Observations with SymIntegration . . . 600

xi Hypothesis Test on a Single Proportion . . . 603
xii Compute Right-Tailed Hypothesis Testing on a Single Proportion with SymIntegration . . . 605
xiii Hypothesis Tests on Two Proportions . . . 607
xiv Compute Right-Tailed Hypothesis Testing on Two Proportions with SymIntegration . . . 608
xv One- and Two Sample Tests Concerning Variances . . . 610
xvi Compute Right-Tailed Hypothesis Testing on One-Sample Variance with SymIntegration . . . 612
xvii Compute Two-Tailed Hypothesis Testing on Two-Samples Variances with SymIntegration . . . 614
xviii Goodness-of-Fit Test . . . 616
xix Test for Independence (Categorical Data) . . . 617
xx Compute Test for Independence for Categorical Data with SymIntegration . . . 618
xxi Compute Two-Sample Case Study using *t*- and *F*- Tests with the Box-and-Whisker and Quantile-Quantile Plot with SymIntegration and Hamzstplot . . . 620
xxii One-Way Analysis of Variance (ANOVA) . . . 626
xxiii Compute One-Way ANOVA with SymIntegration . . . 632
xxiv Two-Way Analysis of Variance (ANOVA) . . . 637
xxv Compute Two-Way ANOVA with SymIntegration . . . 641
xxvi Three-Way Analysis of Variance (ANOVA) . . . 644
xxvii Compute Three-Way ANOVA with SymIntegration . . . 646
VII Linear Regression and Correlation . . . 651
i Pearson's Correlation . . . 651
ii Linear Regression . . . 653
iii Simple Regression Line Plot and Pearson's Correlation Computation . . . 659
iv Compute Linear Regression with Confidence Interval and Prediction Interval with SymIntegration . . . 661
v Multiple Linear Regression . . . 665
vi Compute Multiple Linear Regression with SymIntegration . . . 673
vii Compute Inferences in Multiple Linear Regression with SymIntegration . . . 676

5 Operations Research Computation with SymIntegration 681
I Linear Programming . . . 682
i Simplex Method . . . 689
ii Functions in SymIntegration Related to Linear Programming . . . 694
II Nonlinear Programming . . . 695

6 Numerical Linear Algebra 697
I Solutions of Systems of Linear Equations . . . 697
i Functions in SymIntegration Related to Basic Matrix-Vector Operations . . . 698
ii Solve Nonhomogeneous System of Linear Equation with Gaussian Elimination in SymIntegration . . . 705
II Determinant . . . 709
i Functions in SymIntegration Related to Determinant . . . 710
ii Compute Determinant of a Square Matrix with SymIntegration . . . 711
III Euclidean Vector Spaces . . . 713
i Functions in SymIntegration Related to Euclidean Vector Spaces . . . 717

ii Compute Norm, Dot Product, Angle with SymIntegration . . . 719
iii Compute Orthogonal Projection on a Line with SymIntegration . . . 720
iv Compute Vector and Parametric Equations of a Plane in $\mathbb{R}^3$ with SymIntegration . . . 722
IV General Vector Spaces . . . 723
i General Vector Spaces Application in Engineering . . . 738
ii Matrix Transformations from $\mathbb{R}^n$ to $\mathbb{R}^m$ . . . 739
iii Dynamical Systems and Markov Chains . . . 746
iv Functions in SymIntegration Related to General Vector Spaces . . . 751
v Determine Linear Independence in $\mathbb{R}^n$ with SymIntegration . . . 756
vi Compute Coordinate Vector Relative to Basis $S = \{\boldsymbol{v}_1, \boldsymbol{v}_2, \boldsymbol{v}_3\}$ with SymIntegration . . . 757
vii Compute Transition Matrix ($P_{B' \to B}$) from Old Basis (B') to New Basis (B) and the Coordinate Vectors $[\boldsymbol{w}]_B$ and $[\boldsymbol{w}]_{B'}$ with SymIntegration . . . 758
viii Compute Basis for Row Space and a Column Space of a Matrix with SymIntegration . . . 761
ix Compute Basis for the Solution Space of the Linear System $A\boldsymbol{x} = \boldsymbol{0}$ with SymIntegration . . . 763
x Compute State Vector $\boldsymbol{x}_k$ in a Markov Chain with SymIntegration . . . 765
V Eigenvalues and Eigenvectors . . . 768
i Diagonalization . . . 786
ii Complex Vector Spaces . . . 790
iii Differential Equations . . . 794
iv Functions in SymIntegration Related to Eigenvalues and Eigenvectors . . . 799
v Compute Moore-Penrose Pseudoinverse with SymIntegration . . . 804
vi Compute Real Eigenvalues and Its Corresponding Eigenvectors with SymIntegration . . . 805
vii Compute Matrix P that Diagonalizes a Matrix A with SymIntegration . . . 809
viii Compute Complex Eigenvalues and Its Corresponding Eigenvector using Rayleigh Quotient Iteration with SymIntegration . . . 811
ix Compute Real Eigenvalues and Its Corresponding Eigenvector using QR Algorithm with Shifts with SymIntegration . . . 817
x Compute Real and Complex Eigenvalues from Real Matrix using Francis Double Step QR Algorithm with SymIntegration . . . 822
xi Compute Inverse of a Complex Matrix with Gauss-Jordan Elimination with SymIntegration . . . 829
xii Compute Initial Value Problem Solution of First-Order Linear Systems of Differential Equations with SymIntegration . . . 833
VI Inner Product Spaces . . . 836
i Gram-Schmidt Process . . . 842
ii QR-Decomposition . . . 845
iii Functions in SymIntegration Related to Inner Product Spaces . . . 847
iv Compute QR-Decomposition Using Gram-Schmidt with SymIntegration . . . 849
v Compute QR-Decomposition Using Householder Reflections with SymIntegration . . . 851
VII Diagonalization and Quadratic Forms . . . 855
i Quadratic Forms . . . 863
ii Hermitian, Unitary, and Normal Matrices . . . 869

iii Functions in SymIntegration Related to Diagonalization and Quadratic Forms 874
iv Compute Spectral Decomposition with SymIntegration . . . 875
v Compute Hessenberg Decomposition with SymIntegration . . . 876
VIII Linear Transformations . . . 881
i Functions in SymIntegration Related to Linear Transformations . . . 902
IX Numerical Methods . . . 902
i Functions in SymIntegration Related to Numerical Methods . . . 903
ii *LU*-Decomposition . . . 903
iii Solve Nonhomogeneous System of Linear Equation with *LU*-Decomposition in SymIntegration . . . 906
iv Cholesky Decomposition . . . 911
v Singular Value Decomposition . . . 911
vi Compute Singular Value Decomposition with SymIntegration . . . 915

7 Numerical Analysis 919
I Solutions of Equations in One Variable (Root Finding) . . . 919
i Functions in SymIntegration Related to Root Finding . . . 925
II Interpolation and Polynomial Approximation . . . 925
i Interpolation and the Lagrange Polynomial . . . 925
ii Divided Differences . . . 928
iii Hermite Interpolation . . . 928
iv Cubic Spline Interpolation . . . 928
III Numerical Methods for Optimization . . . 928
i Stochastic Optimization . . . 946
ii Particle Swarm Optimization for Optimization . . . 949
iii Compute and Simulate a Minimization of Function of Two Variables with Direct Monte Carlo, Stochastic Tunneling, and Parallel Tempering . . . 950
iv Simulate a Minimization of Function of Two Variables with Particle Swarm Optimization . . . 956
IV Numerical Differentiation . . . 959
V Numerical Integration . . . 968
VI Iterative Techniques in Matrix Algebra . . . 972
i Jacobi Iterative Method . . . 972
ii Gauss-Seidel Method . . . 974
iii Successive Over-Relaxation (SOR) Method . . . 976
iv The Conjugate Gradient Method . . . 981
v Compute the Solution of a Linear System with Jacobi Iterative Method . . . 988
vi Compute the Solution of a Linear System with Gauss-Seidel Method and SOR Method . . . 990
vii Compute *A*-Orthogonal Vectors with Modified Gram-Schmidt . . . 992
viii Compute the Solution of a Linear System with Conjugate Gradient Method and Preconditioned Conjugate Gradient Method . . . 993
ix Functions in SymIntegration Related to Iterative Techniques in Matrix Algebra 1000
VII Aproximation Theory . . . 1004
VIII Numerical Solutions of Nonlinear Systems of Equations . . . 1005
i Newton's Method . . . 1006
ii Quasi-Newton Method . . . 1008
iii Steepest Descent Techniques . . . 1012

iv Homotopy and Continuation Method . . . 1014
v Particle Swarm Optimization . . . 1018
vi Compute the Solution of a Nonlinear System with Newton's Method . . . 1022
vii Compute the Solution of a Nonlinear System with Quasi-Newton . . . 1024
viii Compute the Solution of a Nonlinear System with Steepest Descent . . . 1027
ix Compute the Solution of a Nonlinear System with RK-4 . . . 1030
x Compute the Solution of a Nonlinear System with Particle Swarm Optimization 1032
xi Functions in SymIntegration Related to Numerical Solutions of Nonlinear Systems of Equations . . . 1035
IX Boundary-Value Problems for Ordinary Differential Equations . . . 1047
X Numerical Solutions to Partial Differential Equations . . . 1047

8 Molecular Biology 1049
I DNA Transcription and Translation . . . 1049
i Transcription and translation from DNA to mRNA to Polypeptide with SymIntegration . . . 1054

9 Evolutionary Algorithms 1055
I Genetic Algorithms . . . 1055
i Functions in SymIntegration Related to Genetic Algorithms . . . 1058
ii Optimization Problem with Simple Genetic Algorithms . . . 1059
iii Visualization of Optimization Problem with Genetic Algorithms with SFML 1070

10 Machine Learning 1073
I Introduction . . . 1074
II DataFrame in C++ for Neural Networks . . . 1091
i Loading Dataframe with Iris Dataset . . . 1091
III Supervised Learning: Feed-Forward Neural Network (Multilayer Perceptron) . . . 1096
i Backpropagation for Feed-Forward Neural Network . . . 1110
ii Feed-Forward Neural Network for Iris Dataset Classification with 1 Hidden Layer . . . 1117
iii Functions in SymIntegration Related to Feed-Forward Neural Network . . . 1136
IV Supervised Learning: Support Vector Machines (SVM) . . . 1138
V Unsupervised Learning: Principal Component Analysis (PCA) . . . 1138
VI Hybrid Learning: Kernel Methods . . . 1138
VII Reinforcement Learning . . . 1141
VIII Deep Learning: Convolutional Neural Network . . . 1141
i Convolution Operation on an Image with OpenGL and STB Image . . . 1165
ii Extract RGBA Matrices from an Image and then Create an Image from the RGBA Matrices with SymIntegration, OpenGL and STB Image . . . 1172
iii Convolutional Neural Networks for Handwriting Recognition: Digit Classifier with SymIntegration . . . 1177
iv Functions in SymIntegration Related to Convolutional Neural Network . . . 1252
v Functions in SymIntegration Related to Image Manipulation with STB Image 1253

11 Computational Fluid Dynamics 1257
I Introduction . . . 1257
II Navier-Stokes . . . 1258
i Couette Flow . . . 1259

ii Poiseuille Flow 1260
iii Couette and Poiseuille Flow Solver Simulation Using a Second-Orderr Runge-Kutta (RK2) with SFML 2.5.1 1260

12 Introduction to Theory of Interest and Financial Mathematics 1265
I Bond Pricing 1265
i Bond Pricing Computation 1272
II Mortgage Loans 1274
i Mortgage Loan Computation 1276
III Financial Derivative: Forwards, Futures, Swaps, and Options 1277
i Options 1278
IV Econometric Models 1280
i Time Series Models 1281
ii Predicting Crude Oil and Natural Gas with ARMA(1,1), ARIMA(1,1,1), and GARCH(1,1) Visualized by SFML 1282

13 Dynamical System 1285
I Double Pendulum 1285
i Double Simple Pendulum Computation and Simulation 1290

14 Modern Control System 1293
I Introduction to Control System 1294
II Mathematical Models of Systems 1303

Preface

For Moscato, Solreya, Mithra, Catenary, Iyzumrae and Zefir.

For Malbec and Znane along with all the 8 Queens (Pinot Noir, Prosecco, Cava, Riesling, Port, Shiraz).

For Albert Silverberg, Camus and Miklotov who left TREVOCALL to guard me.

For Nature(Paris, Kala, Kathmandu, Big Tree, Cabernet Sauvignon, Aokigahara, Hoia Baciu, Jacob's Well, Mt Logan, etc) and my family Sauvignon Blanc: I have served, I will be of service.

For my human parents.

For my dogs who always accompany me working in Valhalla Projection, go to Puncak Bintang or Kathmandu: Sine Bam Bam, Kecil, Browni Bruncit, Sweden Sexy, Cambridge Klutukk, Milan keng-keng, Piano Bludut, Krem, Walnut and more will be adopted. To my cat who guard the home while I'm away with my dogs: London.

The one who moves a mountain begins by carrying away small stones - Confucius

Words have the power to both destroy and heal. When words are both true and kind, they can change our world - The Buddha

Meditation brings wisdom; lack of meditation leaves ignorance. Know well what leads you forward and what holds you back - The Buddha

A book to explain how we create C++ library from forking / branching out SymbolicC++ 3.35 to improve its' symbolic integral computation.

Moscato is from Norse mythology and Valkyrie Profile series, together with Merlot, they are the main authors for this book. After not working with human anymore since 2019, I (Merlot) work in a forest, shaping a forest into a natural wonder / like a canvas for painting, and also clean up trashes that are thrown in the forest. Thus after 6 years working with Nature I have found what can makes me waltz to work, it is going to the forest in the morning then go back home again and study / learn science, engineering, computer programming.

Figure 1: *Freya, thank you for everything, I could never have done it without you. I created this 3 days before Christmas in 2021*

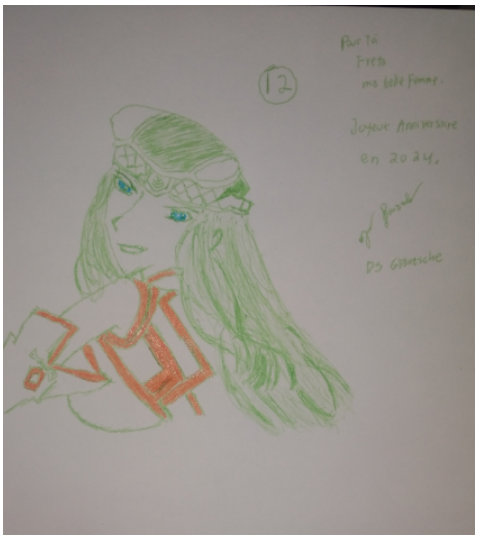

Figure 2: *Anniversary card for Freya in 2024.*

Chapter 1

Introduction Story

On top of the mountain a heart shaped stone waiting for You, my eyes can't see, my body can't touch, but my heart knows it's You. Forever only You. - ***Merlot to Moscato***

This book is initially written on May 9th, 2025, it is exactly 1 month after I have finished with the algorithms and C++ codes for $\int \sin^n x\,dx$, $\int \cos^n x\,dx$, $\int \tan^n x\,dx$, $\int \sec^n x\,dx$, $\int \csc^n x\,dx$.

On April 9th, Sentinel, a Nature residing in a big robust tree in VP (Valhalla Projection) forest, told me to start creating a C++ library that can do integration like what SymPy is able to do. Around April 2025, GiNaC cannot handle integration by parts, and SymbolicC++ also returns $\int \sin^n x\,dx$ with $\frac{1}{n+1}\sin^{n+1} x$. So then we decided to branching out / forking from SymbolicC++ version 3.35. Fixing the C++ codes there so the computation will be made right.
We don't really need to start from scratch. The main idea is to improve its symbolic integration codes. At first, our main focus is to make it able to compute:

1. All kinds of standard integral form (trigonometry, inverse trigonometry, polynomial, transcendental, hyperbolic).
2. The sum, product and divide combination of the standard functions.
3. To be able to compute improper integrals with cases (e.g. computing mean and variance for exponential distribution)

We cover a lot of integral problems, how to find their' patterns then how to create the C++ code so SymIntegration will be able to return the integral correctly, thus there will be a lot of technical writings after this chapter. The requirement to comprehend SymIntegration mathematical background is you should at least self learn, read by yourself a Calculus book [26]. Eventually, as time goes by we also expand and create codes in SymIntegration for other branches in mathematics, such as linear algebra, statistics, differential equations, operations research, machine learning, dynamical system and numerical analysis.

The name is chosen as SymIntegration because it is in accordance with the initial main purpose, to be able to compute all kinds of integration with C++. Which still dominated by SymPy, Mathematica or MATLAB. Mathematica and MATLAB are using license, and I am not working with human anymore, thus I do not have salary / stable income like I used to, so I cannot lean on that (MATLAB / Mathematica), SymPy is free, can be called from JULIA too besides Python, but then, I read that Game Engine (CryEngine, Genie Engine, Godot, Unreal Engine) that is able to

create amazing game with beautiful graphics, animation, or real life graphics, is made with C++. Because it is the near hardware and one of the fastest programming language, thus if you have a mind on creating something better in the future, like Science and Engineering simulator, e.g. Bullet, Project Chrono, HySys, Autodesk Civil Engineering, given that you have a time, then you can create your own Autodesk instead of paying for its license, basically the brain behind them are derivative, integral, statistics and algebra.

Basically we choose C++ so we can create something long lasting in the future, and then I (DS Glanzsche) dedicate everyday of my life to eat and breathe C++ to be able to achieve this stage, hopefully it can be developed further till we can get practical use, like having a simulation for fluid flow, pharmacokinetics (to compute how drugs are metabolized by the body), simulation for human' organ, or to create a stable bridge or skycraper, since the basic of calculus, integral, linear algebra, statistics can be covered nicely by SymIntegration. The front end looks for plot and simulation can be done by third party C++ library like ImGui, MatplotPlusPlus or OpenGL.

Symbolic integration is basically harder than differentiation, differentiation has rigid rules: sum rule, product rule, chain rule, divide rule. While integration has only method of substitution and integration by parts, which back to the method itself. We can obtain different results for integration, but same result for differentiating different functions with respect to the same variable, like this case:

$$D_x \frac{-\cos^2(nx)}{2n} = \cos(nx)\ \sin(nx)$$
$$D_x \frac{\sin^2(nx)}{2n} = \cos(nx)\ \sin(nx)$$

Depending on the method of substitution, which one we substitute, if we reverse the equations above into integral problems, then we will obtain:

$$\int \cos(nx)\ \sin(nx)\ dx = \frac{-\cos^2(nx)}{2n} + C$$
$$\int \cos(nx)\ \sin(nx)\ dx = \frac{\sin^2(nx)}{2n} + C$$

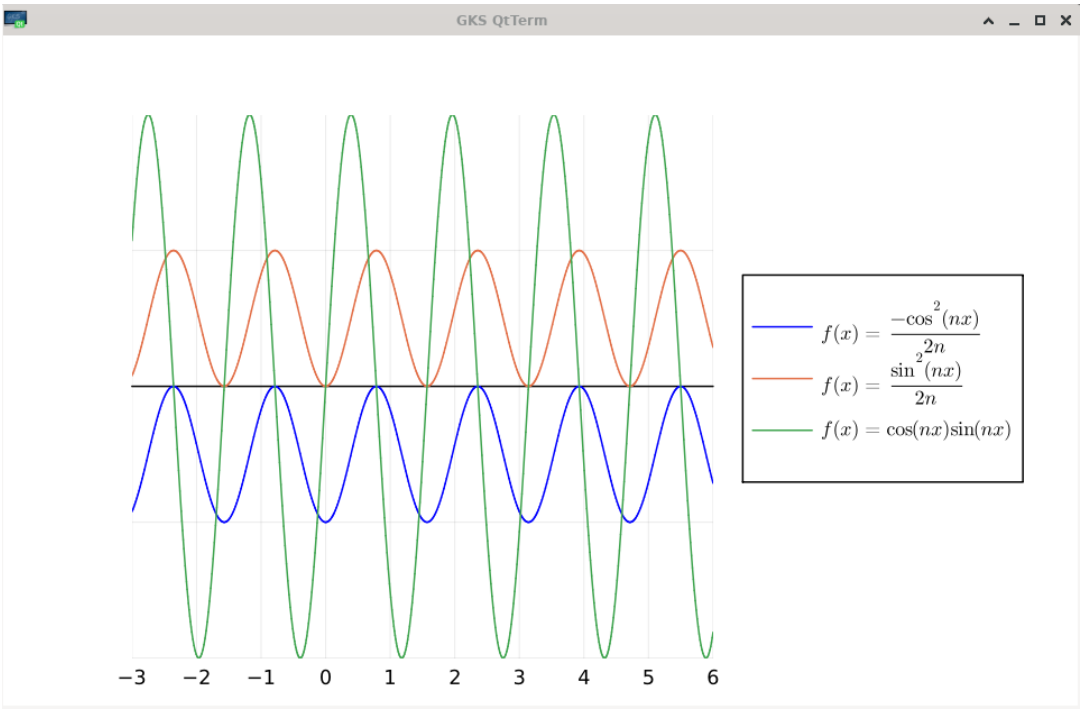

Figure 1.1: *Integrating a function with different method can result in at least two different functions.*

To compute the definite integral of a function in an interval you have to know how it behaves on the entire interval, and to compute the indefinite integral you have to know how it behaves on all intervals. While differentiation is a "local" operation, it computes the derivative of a function at a point you only have to know how it behaves in a neighborhood; distance, speed and acceleration from basic physics is the example of differentiation and integral. Computing integral / antiderivative often requires clever algebraic manipulation and technique and there are no universal rules for integration like there are for differentiation.

Integrals are fundamental in all science and engineering fields for solving problems involving area, volume, and other accumulations, some specific applications of integrals in science and enginnering:

1. To calculate the area of irregular shapes, the volume of solids, and the surface area of objects.
2. To determine center of mass and moments of inertia for various shapes, they are crucial for structural analysis and design.
3. To calculate work done by a variable force, which is essential in mechanics and thermodynamics.
4. To calculate energy, power, and signal properties over time in electrical and computer engineering.
5. To determine stresses and strains analysis in structures under various loads.
6. To calculate fluid flow, pressure distributions, and forces exerted by fluids.

7. To compute probability distributions and statistical calculations.
8. To predict the trajectory of satellites and other objects in motion, ensuring they follow the intended path.
9. To calculate drug accumulation and half-life in a body, it is crucial for understanding how drugs are metabolized and eliminated by the body in Pharmacokinetics.
10. To model how diseases spread through a population with Susceptible-Infectious-Removed (SIR) model.
11. To understand better the complex processes of modeling tumor growth and metastasis.
12. To understand how reactions progress and their rates for reaction kinetics in chemistry.
13. To calculate work done during processes like gas expansion and changes in internal energy or enthalpy. These calculations are crucial for understanding the energetics of chemical reactions and processes.
14. To solve problems related to process design, optimization and understanding reaction kinetics in building a plant in Chemical Engineering.
15. To determine the volume of concrete needed for a structure, the area of a road surface, or the volume of soil required for an excavation in building bridges and structures.
16. To find the deflection (bending) and slope of a beam under load, which is crucial for ensuring structural stability.
17. To determine the velocity of a fluid flow, which is important in designing hydraulic structures like dams and pipelines.
18. To calculate the pressure distribution in fluids, which is essential for understanding the forces exerted on structure by fluids.
19. To understand the behavior of capacitors and inductors. The voltage across a capacitor is the integral of the current flowing through it, and the current through an inductor is the integral of the voltage across it.
20. To compute total charge and energy in Electrical Engineering.
21. To determine the circuit's voltage, current, or other parameters as a function of time by using integration to solve differential equations for many kind of circuit behaviors.

Now we will take a look at the applications of the integration in computer science:

1. Mathematics and computing.
2. Block chain method.
3. Quantum computing.
4. Big data analytics.
5. Neural Networking.
6. Artificial Intelligence.

7. Machine Learning.

Integration, calculus, linear algebra are the brain behind the applications above.

Triple integrals are used in physics, engineering, and even finance. If we get ahold of the C++, we can do the integral computation right, then it is only a matter of presentation, create the plot / simulation / computation for our goal, e.g. to create a stable bridge that can handle earthquake with magnitude of 10 SR. It takes simulation first, before constructing the real product.

The Operating System and Softwares in Use:

1. GFreya OS version 1.8 (based on LFS and BLFS version 11.0 System V books)

2. GCC version 11.2.0 (to compile C++ codes)

3. SymIntegration version 1.51

By October 2026, SymIntegration is able to compute

1. The integral of $\sin(ax+b), \cos(ax+b), \tan(ax+b), \cot(ax+b), \sec(ax+b), \csc(ax+b)$ with $ax+b$ is a polynomial of order 1.
2. The integral of $\frac{1}{(ax+b)}$, $\frac{1}{ax^2+bx+c}$.
3. The integral and derivative of $\operatorname{asin}(ax+b), \operatorname{acos}(ax+b), \operatorname{atan}(ax+b), \operatorname{acot}(ax+b), \operatorname{asec}(ax+b), \operatorname{acsc}(ax+b)$ with $ax+b$ is a polynomial of order 1.
4. The integral and derivative of all hyperbolic trigonometry functions $\sinh(ax+b), \cosh(ax+b), \tanh(ax+b), \coth(ax+b), \operatorname{sech}(ax+b), \operatorname{csch}(ax+b)$.
5. The integral with the form of $x^b e^{ax}, x^b e^x$ with integration by parts method.
6. The integral of $\sin(ax)\cos(bx), \cos(ax)\cos(bx), \sin(ax)\sin(bx)$.
7. The integral of $\int \sin^n(x)\ dx$, $\int \cos^n(x)\ dx$, $\int \tan^n(x)\ dx$, $\int \sec^n(x)\ dx$, $\int \csc^n(x)\ dx$, but the speed to show the whole symbolic integral is slow compared to SymPy, needs to be fixed.
8. The solution ($y(t)$) of first order linear ordinary differential equation of type $ay' + ty = b$, separable equation of type $\frac{dy}{dx} = \frac{f(x)}{g(y)}$, and separable equation with homogeneous ratio equations of type $\frac{dy}{dx} = \frac{f(x,y)}{g(x,y)}$ along with the initial value problem solution.
9. The solution ($y(t)$) of second order linear ordinary differential equation of type $ay'' + by' + cy = g(t)$ (for homogeneous and non-homogeneous equation) along with the initial value problem solution.
10. The series solution of homogeneous second order linear ordinary differential equation of type $y'' + py' + qy = 0$ with constant coefficients, $p, q \in \mathbb{R}$.
11. The series solution of homogeneous n-th order linear ordinary differential equation of type $b_n y^{(n)} + b_{n-1} y^{(n-1)} + \cdots + b_1 y' + b_0 y = 0$ with constant coefficients, $b_0, b_1, \ldots, b_n \in \mathbb{R}$.
12. The series solution of homogeneous second order linear ordinary differential equation of type $P(x)y'' + Q(x)y' + R(x)y = 0$ with variable coefficients, $P(x), Q(x)$, and $R(x)$ are polynomials, for ordinary point and regular singular point (with Frobenius method or Euler-Cauchy method).
13. The computation of divergence, gradient, and curl.
14. The probability mass function, mean, variance, moment generating function of discrete distributions.
15. The probability density function, cumulative distribution function, mean, variance, moment generating function of continuous distributions.
16. The regression line equation along with Pearson's correlation (r) with input of vector of 2 columns
17. Bond pricing and amortization table for mortgage loan or other credit related loan.
18. Compute direction fields and plot it with gnuplot.

19. Linear programming solution with Simplex Method.

20. Numerical differentiation and Richardson's Extrapolation.

21. Iterative techniques in matrix algebra: Jacobi iterative method, Gauss-Seidel method, Successive Over-Relaxation (SOR) method, conjugate gradient, and preconditioned conjugate gradient with Compressed Row Storage (CRS) format.

22. Numerical solutions of nonlinear systems of equations with Newton's method, Quasi-Newton, Steepest Descent, RK4, and Particle Swarm Optimization (PSO).

23. Matrices multiplication, addition, LU decomposition, SVD, eigenvalues and eigenvectors approximation, QR-decomposition, Cholesky decomposition, Schur decomposition, Hessenberg decomposition, QR Algorithm with shifts

24. Generate random number that has certain distribution (uniform, gamma, beta, normal, F, chi-squared) and plot the histogram with Hamzstplot

25. Compute one-way ANOVA, two-way ANOVA, three-way ANOVA, multiple regression model.

26. Compute one-tailed and two-tailed hypothesis testing. One- and Two-Sample Estimation Problem, e.g. Confidence Interval and Prediction Interval.

27. Compute cdf of Normal, student's t distribution, F distribution, chi-squared distribution, beta distribution, gamma distribution without Boost.

28. Compute the quantile of standard normal distribution / inverse of cdf without Boost.

29. Compute the classical orthogonal polynomials: Jacobi, Hermite, Gegenbauer, Chebyshev, Laguerre, Legendre.

30. Transcription and translation from DNA to mRNA to polypeptide (Molecular biology).

31. Genetic algorithms.

32. Feed-forward neural network with steepest descent and conjugate gradient method.

33. Extract RGBA matrices out of an image

34. Perform kernel / convolution matrix on an image with OpenGL.

35. Convolutional neural network for digit handwriting recognition with LeNet-5 architecture and steepest descent method.

We really appreciate those who spend time to write constructing critics, write opinions on what should be added in SymIntegration or if there is an incorrect C++ codes or wrong algorithm. Feedbacks can be sent to **dsglanzsche@gmail.com**.

Chapter 2

Symbolic Integral Computation and Calculus

*"You can't direct the wind, but you can adjust your sails." - **Sailing***

We will first learn how to create the shared library for C++ library in a very simple and manual way, then we will see the source code that composing this C++ library, SymIntegration.

I. Build and Install SymIntegration in GFreya OS

We are using GFreya OS as it is a custom Linux-based Operating System, your distro your rules. Building applications from scratch instead of using package manager, you learn more from mistakes than just depending on package manager.

To download SymIntegration, open terminal / xterm then type:
git clone https://github.com/glanzkaiser/SymIntegration.git

Create Dynamic/Shared Library libsymintegration.so

Enter the directory of the cloned SymIntegration repository then type:
cd src
bash dynamiclibrary.sh
this is a very manual way to create a dynamic library, or you can also type from the main parent directory of SymIntegration repository:

```
cd src
g++ -fPIC -c *.cpp
g++ -shared -o libsymintegration.so *.o
```

Code 1: *Create dynamic library*

It will create the dynamic library **libsymintegration.so** with a very manual way, instead of using CMake, we are using less space.

If you want to use Makefile to create the library, from **SymIntegration/src** open terminal/xterm then type:
make
mv libsymintegration.so /usr/lib

it will move the created shared / dynamic library to the default directory for library.

```
CXX = g++

SYMINTEGRATION_DIR = ../src
LIB = libsymintegration.so
SOURCES = array.cpp
SOURCES += artificialneuralnetworks.cpp cloning.cpp complexanalysis.cpp constants.
    cpp derive.cpp differentialequations.cpp divgradcurl.cpp dsolve.cpp
    dynamicalsystem.cpp
SOURCES += equation.cpp functions.cpp geneticalgorithms.cpp geometryandvectors.cpp
     geompack.cpp identity.cpp infiniteseries.cpp integrate.cpp linearalgebra.cpp
SOURCES += mathematicalfinance.cpp matnorm.cpp matrix.cpp multinomial.cpp
    navierstokes2d.cpp number.cpp numericalmethods.cpp
SOURCES += operationsresearch.cpp polynomial.cpp product.cpp quatern.cpp rational.
    cpp simplify.cpp
```

```
SOURCES += solve.cpp specialfunctions.cpp statistics.cpp stbimage.cpp sum.cpp
    symbol.cpp symbolic.cpp symerror.cpp
SOURCES += symintegrationc++.cpp symmatrix.cpp vecnorm.cpp
SOURCES += vector.cpp vectormatrixoperations.cpp verylong.cpp
SOURCES += asa241.cpp statistics—mle.cpp
SOURCES += biology—dna.cpp

OBJS = $(addsuffix .o, $(basename $(notdir $(SOURCES))))
UNAME_S := $(shell uname —s)

CXXFLAGS = —fPIC
CXXFLAGS += —g —Wall —Wformat


## BUILD FLAGS PER PLATFORM

ifeq ($(UNAME_S), Linux) #LINUX
ECHO_MESSAGE = "SymIntegration from GFreya OS"

CFLAGS = $(CXXFLAGS)
endif

## BUILD RULES

%.o:$(SYMINTEGRATION_DIR)/%.cpp
$(CXX) $(CXXFLAGS) —c —o $@ $<

all: $(LIB)
@echo Build complete for $(ECHO_MESSAGE)

$(LIB): $(OBJS)
$(CXX) —shared —o $@ $^ $(CXXFLAGS)

clean:
rm —f $(OBJS)
```

Code 2: *src/Makefile*

Assuming you are using Linux-based OS, after finish compiling don't forget to copy / move the dynamic library to **/usr/lib**.

Move the Include Files

After finish compiling the dynamic library, open terminal from the current working directory / this repository main directory:

```
cd include
cp —r * /usr/include
```

Code 3: *Move include folder*

When we want to create the shared library it will look for this header files in the default path where the include files are usually located. In Linux OS **usr/include** is the basic / default path.

All files, examples codes and source codes used in this books can be found in this repository: **https://github.com/glanzkaiser/SymIntegration**

We are using less, minimal amount of code to create the library, if you compare it with the original SymbolicC++ that has **.configure** and **Makefile** that will be spawn after you configure it, we give you two options here to compile the library.

Two simple commands with **g++** can already make a shared symbolic integration computation library, with limitation still.

If you are using Windows OS or MacOS then you are on your own.

The C++ codes are only located in the **/src** directory , it is designed to contain all the '.cpp' files, then inside the **/include/** we have header files and another folder **/include/symintegral/** is also a folder that contain header files too.

You can open them one by one, read them and making sense of it based on the function that you are looking for.

In the beginning (from SymbolicC++ 3.35), there are total of 27 **.cpp** files and 26 **.h** / header files. So it won't take a long time for someone to learn it.

II. Techniques of Integration

We will cover some techniques of integration and also two Fundamental Theorems of Calculus that are very useful in solving integral problems or even differential equations.

Theorem 2.1: The First Fundamental Theorem of Calculus

Let f be continuous on the closed interval $[a, b]$ and let x be a (variable) point in (a, b). Then

$$\frac{d}{dx}\int_a^x f(t)\,dt = f(x)$$

Theorem 2.2: The Second Fundamental Theorem of Calculus

Let f be continuous (hence integrable) on $[a, b]$, and let F be any antiderivative of f on $[a, b]$. Then

$$\int_a^b f(x)\,dx = F(b) - F(a)$$

[SI*] **Integration by Parts**
From the analogue of the product rule for differentiation. Suppose we have two functions $f(x)$ and $g(x)$ with

$$\frac{d\,f(x)g(x)}{dx} = f(x)\frac{dg(x)}{dx} + g(x)\frac{df(x)}{dx} \tag{2.1}$$

Integrating both sides and rearranging the terms, we will get the integration by parts formula:

$$\int f(x)\frac{dg(x)}{dx}\,dx = f(x)g(x) - \int g(x)\frac{df(x)}{dx}\,dx \tag{2.2}$$

This formula is only useful if $\int g(x)\frac{df(x)}{dx}\,dx$ or $\int f(x)\frac{dg(x)}{dx}\,dx$ is easier to integrate than $\int f(x)g(x)\,dx$.

The application for integration by part is very useful in statistics, e.g. to be able to compute the mean and variance for exponential distribution.

[SI*] **Substitution Rule for Indefinite Integral**
The substitution rule is nothing more than the Chain Rule in reverse.

Theorem 2.3: Substitution Rule for Indefinite Integrals

Let g be a differentiable function and suppose that F is an antiderivative of f. Then

$$\int f(g(x))g'(x)\,dx = F(g(x)) + C$$

Theorem 2.4: Substitution Rule for Definite Integrals

Let g have a continuous derivative on $[a, b]$, and let f be continuous on the range of g. Then

$$\int_a^b f(g(x))g'(x)\,dx = \int_{g(a)}^{g(b)} f(u)\,du$$

where $u = g(x)$.

III. Integration by Parts

[SI*] **Example: Exponential Distribution**
The exponential distribution, which is sometimes used to model the lifetimes of electrical or mechanical components, has probability density function

$$f(x) = \begin{cases} \lambda e^{-\lambda x}, & \text{if } 0 \leq x \\ 0, & \text{otherwise} \end{cases}$$

where λ is some positive constant. Compute the mean μ and the variance σ^2.

Solution:

To compute for the mean we will have

$$\begin{aligned} E(X) &= \int_{-\infty}^{\infty} xf(x)\,dx \\ &= \int_{-\infty}^{0} x \cdot 0\,dx + \int_{0}^{\infty} x\lambda e^{-\lambda x}\,dx \end{aligned}$$

We apply integration by parts in the second integral with

$$\begin{aligned} u &= x \\ dv &= \lambda e^{-\lambda x}\,dx \\ du &= dx \\ v &= -e^{-\lambda x} \end{aligned}$$

Thus

$$\begin{aligned} E(X) &= [-x\lambda e^{-\lambda x}]_0^{\infty} - \int_0^{\infty} (-e^{-\lambda x})\,dx \\ &= (-0+0) + \left[-\frac{1}{\lambda}e^{-\lambda x}\right]_0^{\infty} \\ &= \frac{1}{\lambda} \end{aligned}$$

The variance is

$$\begin{aligned} \sigma^2 &= E(X^2) - \mu^2 \\ &= \int_{-\infty}^{\infty} x^2 f(x)\,dx - \left(\frac{1}{\lambda}\right) \\ &= \int_{-\infty}^{0} x^2 \cdot 0 + \int_0^{\infty} x^2\ \lambda e^{-\lambda x}\,dx - \frac{1}{\lambda^2} \\ &= [-x^2 e^{-\lambda x}]_0^{\infty} - \int_0^{\infty} (-e^{-\lambda x}) 2x\,dx - \frac{1}{\lambda^2} \\ &= (-0+0) + 2\int_0^{\infty} xe^{-\lambda x}\,dx - \frac{1}{\lambda^2} \\ &= 2\frac{1}{\lambda^2} - \frac{1}{\lambda^2} \\ &= \frac{1}{\lambda^2} \end{aligned}$$

[SI*] When they say integration is an art it is true, you need more practice to be able to handle all kinds of integral problems.

If we are facing this kind of equation:

$$f(x) = x\lambda e^{-\lambda x}$$

then what we need to do is learning the pattern

$$\begin{aligned}
\int xe^x \, dx &= (x-1)e^x + C \\
\int x^2e^x \, dx &= (x^2 - 2x + 2)e^x + C \\
\int x^3e^x \, dx &= (x^3 - 3x^2 + 6x - 6)e^x + C \\
\int x^4e^x \, dx &= (x^4 - 4x^3 + 12x^2 - 24x + 24)e^x + C \\
&\vdots
\end{aligned}$$

so for the general function of

$$f(x) = x^n e^x$$

The algorithm will be like this:

```
Symbolic sgn = 1;
int k = 1;
Symbolic integral;
for (int i = 0; i <= n; i++)
{
integral += k * sgn * (x^(n—i)) * exp(x)
sgn = —sgn;
k *= n—i;
}
return integral;
```

The C++ file to handle cases of integration by parts is located in **src/integrate.cpp**

```
Symbolic integrate(const Symbolic &f,const Symbolic &x)
{
       list<Equations> eq;
       list<Equations>::iterator i;
       UniqueSymbol a, b, c;
...
...
```

Code 4: *src/integrate.cpp*

Now for the case of $f(x) = x^n e^x$, it is handled at this part of codes

```
Symbolic integrate(const Symbolic &f,const Symbolic &x)
{
       ...
```

```
                ...
                eq = ((x^b)*exp(x)).match(f, (a,b)); // case for x^b *
                    exp(x)
                for(i=eq.begin(); i!=eq.end(); ++i)
                try {
                       Symbolic bp = rhs(*i, b);
                       if(bp.type() == typeid(Numeric)
                       && Number<void>(bp)->numerictype() == typeid(int)
                       && Number<void>(bp)>Number<int>(0))
                       {
                              int n = CastPtr<const Number<int>>(bp)->n,
                                   sgn = 1;
                              Symbolic integral, nf = 1;
                              for(; n>=0; nf*=n, --n, sgn=-sgn)
                                  integral += sgn*nf*(x^n)*exp(x);
                              return integral;
                       }
                } catch(const SymbolicError &se) {}

        }
```

Code 5: *src/integrate.cpp to handle $x^n e^x$*

For the computation of mean and variance of exponential distribution with SymIntegration we will get this raw result, where the e^{-infty} is still as it is, and not returning to 0, we will work on it in the future.

```
#include <iostream>
#include "symintegrationc++.h"

using namespace std;

int main(void)
{
       Symbolic x("x"), l("l"), Inf("Inf"), y, Ex, Varx,
            u, dv, v, du;

       y = l*exp(-l*x);
       u = x;
       dv = l*exp(-l*x);
       du = df(u,x);
       v = integrate(dv,x);

       cout << "f(x) = " << y << endl;

       for(int i=1;i<=1;i++)
       {
              y = integrate(y,x);
              cout << i << "-st integral of f(x) = " <<
                  y << endl;
```

```
                    }
                    cout << "int_{0}^{Inf} f(x) = " << y[x==Inf] - y[
                        x==0] <<endl;

                    Ex = x*l*exp(-l*x);

                    cout << "\nx f(x) = " << Ex << endl;
                    cout << "\nu = " << u << endl;
                    cout << "dv = " << dv << endl;
                    cout << "du = " << du << endl;
                    cout << "v = " << v << endl;

                    Ex = integrate(Ex,x);
                    cout << "\nE(x) = int x f(x) = " << Ex << endl;
                    cout << "int_{0}^{Inf} x f(x) = " << Ex[x==Inf] -
                         Ex[x==0] <<endl;

                    Varx = x*x*l*exp(-l*x);
                    Varx = integrate(Varx,x);

                    cout << "\nint x^{2} f(x) = " << Varx << endl;
                    cout << "int_{0}^{Inf} x^{2} f(x) = " << Varx[x==
                        INFINITY, l==1] - Varx[x==0] <<endl;
                    cout << "\nVar(x) = int x^{2} f(x) - mu^{2} = "
                        << Varx[x==INFINITY, l==1] - Varx[x==0] -
                        (1/l^(2)) << endl;

                    return 0;
                }
```

Code 6: *Integration by Parts*

IV. The Method of Substitution

[SI*] **Some Examples**
Suppose we want to compute the integral

$$\int x \sin x^2 \, dx$$

Solution:
Here the appropriate substitution is

$$u = x^2$$
$$du = 2x \, dx$$

Thus

$$\begin{aligned}\int x \ \sin \ x^2 \, dx &= \int \frac{1}{2} \ \sin \ x^2 \ (2x \, dx) \\ &= \int \frac{1}{2} \sin \ u \, du \\ &= -\frac{1}{2} \cos \ u + C \\ &= -\frac{1}{2} \cos \ x^2 + C\end{aligned}$$

[SI*] In SymIntegration the method of substitution can be handled in **src/integrate.cpp**, we can create a pattern of equation to be solved, just like the integration by parts method.

V. Fraction Integral

[SI*] **Fraction Level 1**

Suppose we have this function that we want to integrate toward x.

$$f(x) = \frac{1}{ax+b}$$

a and b are constants.

The integral will be

$$\int \frac{1}{ax+b}\,dx = \frac{\ln(ax+b)}{a} + C \tag{2.3}$$

For the integrate function alone in SymIntegration we do not include the constant C that is obtained after integration.

* Note that the C of the indefinite integration cancels out, as it always will, in the definite integration. That is why in the statement of the Second Fundamental Theorem of Calculus we could use the phrase **any derivative**. In particular, we may always choose $C = 0$ in applying the Second Fundamental Theorem of Calculus [26].

[SI*] The C++ code to handle this fraction level 1 integral problem to make sure the integral will return the correct result is located in **src/functions.cpp** under the function **Symbolic Power::integrate(const Symbolic &s) const**, we still use the function **integrate** for this type of integral.

```
Symbolic Power::integrate(const Symbolic &s) const
{
const Symbolic &a = parameters.front();
const Symbolic &b = parameters.back();
...
...
}
```

```
Symbolic Power::integrate(const Symbolic &s) const
{
...
...

if(b == -1 && parameters.front().coeff(s,2) == 0) return
     ln(parameters.front()) * (1/ parameters.front().df(
    s));
...
...
}
```

Code 7: *fraction level 1 integral*

The **if** statement above is a condition: if the input of a function meets the criteria that the power is -1 / a fraction ($f(x)^{-1} = \frac{1}{f(x)}$) then we will return it as **ln(parameters.front()) * (1/**

parameters.front().df(s)). The symbolic s is used to replace the variable in the integration problem.

I will introduce to the reader here about **parameters.front()** and **parameters.back()**, suppose we have a function

$$f(x) = (ax^2 + bx + c)^d \tag{2.4}$$

with a, b, c as constants.

- The **parameters.front()** of the function from equation (2.4) is $ax^2 + bx + c$.
  parameters.front().coeff(s,2) is a.
  parameters.front().coeff(s,1) is b.
  parameters.front().coeff(s,0) is c.

- The **parameters.back()** of the function from equation (2.4) is d.

With this knowledge we can create a lot of **if, if.. else..** conditions to make the integration computation complete and better.

[SI*] **Fraction Level 2**
Suppose we have this function that we want to integrate toward x.

$$f(x) = \frac{1}{ax^2 + bx + c}$$

a, b and c are constants.

The integral will be

$$\begin{aligned}\int \frac{1}{ax^2 + bx + c}\, dx = &-\sqrt{\frac{-1}{4ac - b^2}} \ln\left(x + \frac{-4ac\sqrt{\frac{-1}{4ac-b^2}} + b^2\sqrt{\frac{-1}{4ac-b^2}} + b}{2a}\right) \\ &+ \sqrt{\frac{-1}{4ac - b^2}} \ln\left(x + \frac{4ac\sqrt{\frac{-1}{4ac-b^2}} - b^2\sqrt{\frac{-1}{4ac-b^2}} + b}{2a}\right) + C\end{aligned} \tag{2.5}$$

The codes for the SymIntegration to handle this integral is located in **src/functions.cpp**

```
...
...

Symbolic Power::integrate(const Symbolic &s) const
{
       ...
       ...
       if(b == -1 && parameters.front().coeff(s,2) != 0)
       {
              double a1 = parameters.front().coeff(s,2);
              double b1 = parameters.front().coeff(s,1);
              double c1 = parameters.front().coeff(s,0);
              double D_inv = sqrt(-1/(4*a1*c1-b1*b1));
```

```
                        return — D_inv * ln(s + (—4*a1*c1*D_inv + b1*b1*
                            D_inv + b1)/(2*a1)) + D_inv * ln(s + (4*a1*c1
                            *D_inv — b1*b1*D_inv + b1)/(2*a1)) ;
                }
                ...
                ...
        }
        ...
        ...
```

Code 8: *fraction level 2 integral type 1*

[SI*] Suppose we have this function that we want to integrate toward x.

$$f(x) = \frac{a_1 x}{ax^2 + bx + c}$$

a_1, a, b, and c are constants.

The integral will be

$$\begin{aligned}\int \frac{a_1 x}{ax^2+bx+c}\,dx = a_1\Big[&\left(-\frac{b\sqrt{b^2-4ac}}{2a(4ac-b^2)} + \frac{1}{2a}\right) * \\ &\ln\left(x + \frac{-4ac\left(-\frac{b\sqrt{b^2-4ac}}{2a(4ac-b^2)} + \frac{1}{2a}\right) + b^2\left(-\frac{b\sqrt{b^2-4ac}}{2a(4ac-b^2)} + \frac{1}{2a}\right) + 2c}{b}\right) \\ &+ \left(\frac{b\sqrt{b^2-4ac}}{2a(4ac-b^2)} + \frac{1}{2a}\right) * \\ &\ln\left(x + \frac{-4ac\left(\frac{b\sqrt{b^2-4ac}}{2a(4ac-b^2)} + \frac{1}{2a}\right) + b^2\left(\frac{b\sqrt{b^2-4ac}}{2a(4ac-b^2)} + \frac{1}{2a}\right) + 2c}{b}\right)\Big] + C\end{aligned} \tag{2.6}$$

The codes for the SymIntegration to handle this integral is located in **src/integrate.cpp**, instead of **integrate** we create a new function **fraction integrate** to handle this type of integral because this type of integral cannot be read nicely with **Symbolic Power::integrate(const Symbolic &s) const** that is in **src/functions.cpp**.

By July 8th, 2025, the function **fraction integrate** already has around 190 lines of code.

```
    ...
    ...

Symbolic fractionintegrate(const Symbolic &fnum, const Symbolic &fdenom,
    const Symbolic &x)
{
    list<Equations> eq;
    list<Equations>::iterator i;
    UniqueSymbol anum, bnum;
    Symbolic integral_sol;
    double anump, bnump, ap, bp, cp;
```

```
      ap = fdenom.coeff(x,2);
      bp = fdenom.coeff(x,1);
      cp = fdenom.coeff(x,0);

      ...
      ...

      if (fnum.coeff(x,2)==0 && fnum.coeff(x,1)!=0 && fnum.coeff(x,0)==0 &&
          fdenom.coeff(x,2)!=0 && fdenom.coeff(x,1)!=0 && fdenom.coeff(x,0)
          !=0 ) // for (a1x)/(ax^2+bx+c)
      {
             eq = ( (bnum*x)).match(fnum, (bnum,cnum));
             for(i=eq.begin(); i!=eq.end(); ++i)
             try {
                    Symbolic a1 = rhs(*i, bnum);
                    bnump= a1;
             } catch(const SymbolicError &se) {}

             double D = (bp*sqrt(bp*bp-4*ap*cp))/(2*ap*(4*ap*cp-(bp*bp)))
                 ;
             integral_sol = bnump*( (-D + (1/(2*ap)))*ln(x+(-4*ap*cp*(-D +
                  1/(2*ap)) +bp*bp*(-D+ 1/(2*ap)) + 2*cp)/(bp)) + (D +
                 (1/(2*ap)))*ln(x+(-4*ap*cp*(D + 1/(2*ap)) +bp*bp*(D+ 1/(2*
                 ap)) + 2*cp)/(bp)) );
      }

      ...

      return integral_sol;
}
      ...
      ...
```

Code 9: *fraction level 2 integral default type*

[SI*] Suppose we have this function that we want to integrate toward x.

$$f(x) = \frac{a_1 x}{ax^2 + bx}$$

a_1, a, and b are constants.

The integral will be

$$\int \frac{a_1 x}{ax^2 + bx}\, dx = \frac{a_1 \ln(ax + b)}{a} + C \tag{2.7}$$

[SI*] Suppose we have this function that we want to integrate toward x.

$$f(x) = \frac{a_1 x}{ax^2 + c}$$

a_1, a and c are constants.

The integral will be

$$\int \frac{a_1 x}{ax^2 + c}\, dx = \frac{a_1 \ln(ax^2 + c)}{2a} + C \tag{2.8}$$

[SI*] Suppose we have this function that we want to integrate toward x.

$$f(x) = \frac{a_1 x}{bx + c}$$

a_1, b, and c are constants.

The integral will be

$$\int \frac{a_1 x}{bx + c}\, dx = a_1 \left(\frac{x}{b} - \frac{c \ln(bx + c)}{b^2} \right) + C \tag{2.9}$$

[SI*] **Fraction Level 3**
Suppose we have this function that we want to integrate toward x.

$$f(x) = \frac{a_1 x + b_1}{ax^2 + bx + c}$$

a_1, b_1, a, b and c are constants.

The integral will be

$$\begin{aligned}\int \frac{a_1 x + b_1}{ax^2 + bx + c}\, dx = &\left(\frac{a_1}{2a} - \frac{(2ab_1 - a_1 b)\sqrt{b^2 - 4ac}}{2a(4ac - b^2)} \right) * \\ &\ln \left(x + \frac{4ac \left(\frac{a_1}{2a} - \frac{(2ab_1 - a_1 b)\sqrt{b^2 - 4ac}}{2a(4ac - b^2)} \right) - 2a_1 c - b^2 \left(\frac{a_1}{2a} - \frac{(2ab_1 - a_1 b)\sqrt{b^2 - 4ac}}{2a(4ac - b^2)} \right) + bb_1}{2ab_1 - a_1 b} \right) \\ &+ \left(\frac{a_1}{2a} + \frac{(2ab_1 - a_1 b)\sqrt{b^2 - 4ac}}{2a(4ac - b^2)} \right) * \\ &\ln \left(x + \frac{4ac \left(\frac{a_1}{2a} + \frac{(2ab_1 - a_1 b)\sqrt{b^2 - 4ac}}{2a(4ac - b^2)} \right) - 2a_1 c - b^2 \left(\frac{a_1}{2a} + \frac{(2ab_1 - a_1 b)\sqrt{b^2 - 4ac}}{2a(4ac - b^2)} \right) + bb_1}{2ab_1 - a_1 b} \right) + C\end{aligned} \tag{2.10}$$

The codes for the SymIntegration to handle this integral is located in **src/integrate.cpp**, instead of **integrate** we create a new function **fraction integrate** to handle this type of integral.

```
...

Symbolic fractionintegrate(const Symbolic &fnum, const Symbolic &fdenom,
    const Symbolic &x)
```

```
{
       list<Equations> eq;
       list<Equations>::iterator i;
       UniqueSymbol anum, bnum;
       Symbolic integral_sol;
       double anump, bnump, ap, bp, cp;
       ap = fdenom.coeff(x,2);
       bp = fdenom.coeff(x,1);
       cp = fdenom.coeff(x,0);

       ...

       if (fnum.coeff(x,2)==0 && fnum.coeff(x,1)!=0 && fnum.coeff(x,0)!=0 &&
           fdenom.coeff(x,2)!=0 && fdenom.coeff(x,1)!=0 && fdenom.coeff(x,0)
           !=0 ) // for (a1x+b1)/(ax^2+bx+c)
       {
              eq = ( (bnum*x+cnum)).match(fnum, (bnum,cnum));
              for(i=eq.begin(); i!=eq.end(); ++i)
              try {
                     Symbolic a1 = rhs(*i, bnum), b1 = rhs(*i, cnum);
                     bnump= a1;
                     cnump = b1;
              } catch(const SymbolicError &se) {}

              double D = (2*ap*cnump - bnump*bp)*sqrt(-4*ap*cp+(bp*bp))/(2*
                  ap*(4*ap*cp-(bp*bp)));
              double denom1 = 2*ap*cnump - bnump*bp;

              integral_sol = ((bnump/(2*ap)) - D)*ln(x + (4*ap*cp*((bnump
                  /(2*ap))-D) - 2*bnump*cp -(bp*bp)*(bnump/(2*ap) - D) +
                  bp*cnump)/(denom1)) + ((bnump/(2*ap)) + D)*ln(x + (4*ap*cp
                  *((bnump/(2*ap))+D) - 2*bnump*cp -(bp*bp)*(bnump/(2*ap) +
                   D) + bp*cnump)/(denom1)) ;
       }

       ...

       return integral_sol;
}

...
```

Code 10: *fraction level 3 integral default type showstringspaces*

[SI*] Suppose we have this function that we want to integrate toward x.

$$f(x) = \frac{a_1 x + b_1}{ax^2 + bx}$$

a_1, b_1, a, and b are constants.

The integral will be

$$\int \frac{a_1 x + b_1}{ax^2 + bx}\, dx = \frac{b_1 \ln(x)}{b} - \frac{(ab_1 - a_1 b)\ln\left(x + \frac{bb_1 + \frac{b(ab_1 - a_1 b)}{a}}{2ab_1 - a_1 b}\right)}{ab} + C \tag{2.11}$$

[SI*] Suppose we have this function that we want to integrate toward x.

$$f(x) = \frac{a_1 x + b_1}{ax^2 + c}$$

a_1, b_1, a, and c are constants.

The integral will be

$$\begin{aligned}\int \frac{a_1 x + b_1}{ax^2 + c}\, dx = &\left(\frac{a_1}{2a} - \frac{b_1\sqrt{-a^3 c}}{2a^2 c}\right) * \\ &\ln\left(x + \frac{2ac\left(\frac{a_1}{2a} - \frac{b_1\sqrt{-a^3 c}}{2a^2 c}\right) - a_1 c}{ab_1}\right) \\ &+ \left(\frac{a_1}{2a} + \frac{b_1\sqrt{-a^3 c}}{2a^2 c}\right) * \\ &\ln\left(x + \frac{2ac\left(\frac{a_1}{2a} + \frac{b_1\sqrt{-a^3 c}}{2a^2 c}\right) - a_1 c}{ab_1}\right) + C\end{aligned} \tag{2.12}$$

[SI*] Suppose we have this function that we want to integrate toward x.

$$f(x) = \frac{a_1 x + b_1}{bx + c}$$

a_1, b_1, b, and c are constants.

The integral will be

$$\int \frac{a_1 x + b_1}{bx + c}\, dx = \frac{a_1 x}{b} - \frac{(a_1 c - bb_1)\ln(bx = c)}{b^2} + C \tag{2.13}$$

[SI*] **Fraction Level 4**
Suppose we have this function that we want to integrate toward x.

$$f(x) = \frac{a_1 x^2 + b_1 x + c_1}{ax^2 + bx + c}$$

a_1, b_1, c_1, a, b and c are constants.

The integral will be

$$\int \frac{a_1x^2+b_1x+c_1}{ax^2+bx+c}\,dx = \left(\frac{ab_1-a_1b}{2a^2} - \frac{(2a^2c_1-2aa_1c-abb_1+a_1b^2)\sqrt{b^2-4ac}}{2a^2(4ac-b^2)}\right) * \ln\left[x + \frac{4a^2c\left(\frac{ab_1-a_1b}{2a^2} - \frac{(2a^2c_1-2aa_1c-abb_1+a_1b^2)\sqrt{b^2-4ac}}{2a^2(4ac-b^2)}\right)}{2a^2c_1-2aa_1c-abb_1+a_1b^2} - \frac{ab^2\left(\frac{ab_1-a_1b}{2a^2} - \frac{(2a^2c_1-2aa_1c-abb_1+a_1b^2)\sqrt{b^2-4ac}}{2a^2(4ac-b^2)}\right)+abc_1-2ab_1c+a_1bc}{2a^2c_1-2aa_1c-abb_1+a_1b^2}\right] + \left(\frac{ab_1-a_1b}{2a^2} + \frac{(2a^2c_1-2aa_1c-abb_1+a_1b^2)\sqrt{b^2-4ac}}{2a^2(4ac-b^2)}\right) * \ln\left[x + \frac{4a^2c\left(\frac{ab_1-a_1b}{2a^2} + \frac{(2a^2c_1-2aa_1c-abb_1+a_1b^2)\sqrt{b^2-4ac}}{2a^2(4ac-b^2)}\right)}{2a^2c_1-2aa_1c-abb_1+a_1b^2} - \frac{ab^2\left(\frac{ab_1-a_1b}{2a^2} + \frac{(2a^2c_1-2aa_1c-abb_1+a_1b^2)\sqrt{b^2-4ac}}{2a^2(4ac-b^2)}\right)+abc_1-2ab_1c+a_1bc}{2a^2c_1-2aa_1c-abb_1+a_1b^2}\right] + \frac{a_1x}{a} + C \tag{2.14}$$

The codes for the SymIntegration to handle this integral is located in **src/integrate.cpp**, we use a new function **fraction integrate** to handle this type of integral.

```
...

Symbolic fractionintegrate(const Symbolic &fnum, const Symbolic &fdenom,
    const Symbolic &x)
{
       list<Equations> eq;
       list<Equations>::iterator i;
       UniqueSymbol anum, bnum, cnum;
       Symbolic integral_sol;
       double anump, bnump, cnump, ap, bp, cp;
       ap = fdenom.coeff(x,2);
       bp = fdenom.coeff(x,1);
       cp = fdenom.coeff(x,0);

       if (fnum.coeff(x,2)!=0 && fnum.coeff(x,1)!=0 && fnum.coeff(x,0)!=0 &&
            fdenom.coeff(x,2)!=0 && fdenom.coeff(x,1)!=0 && fdenom.coeff(x,0)
            !=0 ) // for (a1x^2 + b1x + c1)/(ax^2+bx+c)
       {
              eq = ( (anum*x*x+bnum*x+cnum)).match(fnum, (anum,bnum,cnum));
              for(i=eq.begin(); i!=eq.end(); ++i)
              try {
                     Symbolic a1 = rhs(*i, anum), b1 = rhs(*i, bnum), c1 =
                         rhs(*i, cnum);
```

```
                    anump= a1;
                    bnump= b1;
                    cnump = c1;
                } catch(const SymbolicError &se) {}

                double D = sqrt(bp*bp-(4*ap*cp))*(2*ap*ap*cnump-2*ap*anump*cp
                    -ap*bp*bnump+anump*bp*bp)/(2*ap*ap*(4*ap*cp-(bp*bp))) ;

                integral_sol = ((ap*bnump-anump*bp)/(2*ap*ap) - D)*ln(x+(4*ap
                    *ap*cp*((ap*bnump-anump*bp)/(2*ap*ap) - D) - ap*bp*bp*((
                    ap*bnump-anump*bp)/(2*ap*ap) - D) + ap*bp*cnump-2*ap*
                    bnump*cp+anump*bp*cp) / (2*ap*ap*cnump-2*ap*anump*cp-ap*
                    bp*bnump+anump*bp*bp)) + ((ap*bnump-anump*bp)/(2*ap*ap) +
                    D)*ln(x+(4*ap*ap*cp*((ap*bnump-anump*bp)/(2*ap*ap) + D) -
                     ap*bp*bp*((ap*bnump-anump*bp)/(2*ap*ap) + D) + ap*bp*
                    cnump-2*ap*bnump*cp+anump*bp*cp) / (2*ap*ap*cnump-2*ap*
                    anump*cp-ap*bp*bnump+anump*bp*bp)) + (anump*x)/ap;
        }

...
...
```

Code 11: *fraction level 4 integral default type*

[SI*] The key to solve integral of fraction integral is to use the method of partial fraction decomposition, which allows us to decompose rational functions into sums of simpler, more easily integrated rational functions, for example if we have

$$\int \frac{P_n(x)}{P_m(x)}$$

with $P_n(x)$ is a polynomial with x as the independent variable and highest order of n, $P_m(x)$ is a polynomial with x as the independent variable and highest order of m.
The partial fraction decomposition can be applied to a rational function

$$\frac{P_n(x)}{P_m(x)}$$

only if $n < m$.

For the case where $n \geq m$, it can be tricked by first we must perform long division to rewrite the quotient into a rational function that can fit the bill to be performed the partial fraction decomposition

$$\frac{P_n(x)}{P_m(x)} = B(x) + \frac{P_c(x)}{P_d(x)}$$

where $c < d$.

From the previous part we can see that a solid formula to compute the fraction integral can be very long enough, with $m = 2$, it comes from deriving and generalizing this method of partial fraction decomposition by changing the constants into coefficients like a_1, b_1, c_1, a, b, c.

[SI*] **List of Solvable Fraction Integral**

We will list all the fractions that can be solved with **fractionintegrate** function in SymIntegration here, they are:

$$\int \frac{1}{1+x^2}dx = \operatorname{atan}(x)$$

$$\int \frac{ax}{1+x^2}dx = \frac{a\ln(x^2+1)}{2}$$

$$\int \frac{1}{ax^2+bx+c}\,dx = -\sqrt{\frac{-1}{4ac-b^2}}\ln\left(x+\frac{-4ac\sqrt{\frac{-1}{4ac-b^2}}+b^2\sqrt{\frac{-1}{4ac-b^2}}+b}{2a}\right) + \sqrt{\frac{-1}{4ac-b^2}}\ln\left(x+\frac{4ac\sqrt{\frac{-1}{4ac-b^2}}-b^2\sqrt{\frac{-1}{4ac-b^2}}+b}{2a}\right)+C$$

$$\int \frac{a_1x}{ax^2+bx+c}\,dx = a_1\Big[\left(-\frac{b\sqrt{b^2-4ac}}{2a(4ac-b^2)}+\frac{1}{2a}\right)* \ln\left(x+\frac{-4ac\left(-\frac{b\sqrt{b^2-4ac}}{2a(4ac-b^2)}+\frac{1}{2a}\right)+b^2\left(-\frac{b\sqrt{b^2-4ac}}{2a(4ac-b^2)}+\frac{1}{2a}\right)+2c}{b}\right) + \left(\frac{b\sqrt{b^2-4ac}}{2a(4ac-b^2)}+\frac{1}{2a}\right)* \ln\left(x+\frac{-4ac\left(\frac{b\sqrt{b^2-4ac}}{2a(4ac-b^2)}+\frac{1}{2a}\right)+b^2\left(\frac{b\sqrt{b^2-4ac}}{2a(4ac-b^2)}+\frac{1}{2a}\right)+2c}{b}\right)\Big]+C$$

$$\int \frac{a_1x+b_1}{ax^2+bx+c}\,dx = \left(\frac{a_1}{2a}-\frac{(2ab_1-a_1b)\sqrt{b^2-4ac}}{2a(4ac-b^2)}\right)* \ln\left(x+\frac{4ac\left(\frac{a_1}{2a}-\frac{(2ab_1-a_1b)\sqrt{b^2-4ac}}{2a(4ac-b^2)}\right)-2a_1c-b^2\left(\frac{a_1}{2a}-\frac{(2ab_1-a_1b)\sqrt{b^2-4ac}}{2a(4ac-b^2)}\right)+bb_1}{2ab_1-a_1b}\right) + \left(\frac{a_1}{2a}+\frac{(2ab_1-a_1b)\sqrt{b^2-4ac}}{2a(4ac-b^2)}\right)* \ln\left(x+\frac{4ac\left(\frac{a_1}{2a}+\frac{(2ab_1-a_1b)\sqrt{b^2-4ac}}{2a(4ac-b^2)}\right)-2a_1c-b^2\left(\frac{a_1}{2a}+\frac{(2ab_1-a_1b)\sqrt{b^2-4ac}}{2a(4ac-b^2)}\right)+bb_1}{2ab_1-a_1b}\right)+C$$

$$\int \frac{a_1x^2+b_1x+c_1}{ax^2+bx+c}\,dx = \left(\frac{ab_1-a_1b}{2a^2} - \frac{(2a^2c_1-2aa_1c-abb_1+a_1b^2)\sqrt{b^2-4ac}}{2a^2(4ac-b^2)}\right) *$$
$$\ln\left[x + \frac{4a^2c\left(\frac{ab_1-a_1b}{2a^2} - \frac{(2a^2c_1-2aa_1c-abb_1+a_1b^2)\sqrt{b^2-4ac}}{2a^2(4ac-b^2)}\right)}{2a^2c_1-2aa_1c-abb_1+a_1b^2}\right.$$
$$\left. - \frac{ab^2\left(\frac{ab_1-a_1b}{2a^2} - \frac{(2a^2c_1-2aa_1c-abb_1+a_1b^2)\sqrt{b^2-4ac}}{2a^2(4ac-b^2)}\right) + abc_1 - 2ab_1c + a_1bc}{2a^2c_1-2aa_1c-abb_1+a_1b^2}\right]$$
$$+ \left(\frac{ab_1-a_1b}{2a^2} + \frac{(2a^2c_1-2aa_1c-abb_1+a_1b^2)\sqrt{b^2-4ac}}{2a^2(4ac-b^2)}\right) *$$
$$\ln\left[x + \frac{4a^2c\left(\frac{ab_1-a_1b}{2a^2} + \frac{(2a^2c_1-2aa_1c-abb_1+a_1b^2)\sqrt{b^2-4ac}}{2a^2(4ac-b^2)}\right)}{2a^2c_1-2aa_1c-abb_1+a_1b^2}\right.$$
$$\left. - \frac{ab^2\left(\frac{ab_1-a_1b}{2a^2} + \frac{(2a^2c_1-2aa_1c-abb_1+a_1b^2)\sqrt{b^2-4ac}}{2a^2(4ac-b^2)}\right) + abc_1 - 2ab_1c + a_1bc}{2a^2c_1-2aa_1c-abb_1+a_1b^2}\right]$$
$$+ \frac{a_1x}{a} + C$$

If the denominator has determinant of 0 / $b^2 = 4ac$ then

$$\int \frac{b_1x+c_1}{ax^2+bx+c}\,dx = \frac{-ac_1+b_1c}{a^3x+a^2c} + \frac{b_1\ln(ax+c)}{a^2}$$

VI. Trigonometry Integral

[SI*] **Trigonometry Level 1**

We will want to compute integral of trigonometry at the basic level like these:

$$\int \sin(x)\ dx = -\cos(x) + C$$
$$\int \sin(3x+5)\ dx = -\frac{1}{3}\cos(3x+5) + C$$
$$\int \cos(x)\ dx = \sin(x) + C$$
$$\int \cos(3x+5)\ dx = \frac{1}{3}\sin(3x+5) + C$$
$$\int \tan(x)\ dx = -\ln(\cos(x)) + C$$
$$\int \tan(3x+5)\ dx = \frac{1}{6}\ln(\tan^2(3x+5)+1) + C$$
$$\int \cot(x)\ dx = \ln(\sin(x)) + C$$
$$\int \cot(3x+5)\ dx = -\frac{1}{6}\ln(\tan^2(3x+5)+1) + \frac{1}{3}\ln(\tan(3x+5)) + C$$
$$\int \sec(x)\ dx = -\frac{\ln(\cos(x))}{\sin(x)} + C$$
$$\int \sec(3x+5)\ dx = -\frac{1}{3}\frac{\ln(\cos(3x+5))}{\sin(3x+5)}$$
$$\int \csc(x)\ dx = \frac{\ln(\sin(x))}{\cos(x)} + C$$
$$\int \csc(3x+5)\ dx = \frac{1}{3}\frac{\ln(\sin(3x+5))}{\cos(3x+5)}$$

or in general form

$$\int \sin(ax+b)\ dx = -\frac{1}{a}\cos(ax+b) + C$$
$$\int \cos(ax+b)\ dx = \frac{1}{a}\sin(ax+b) + C$$
$$\int \tan(ax+b)\ dx = \frac{1}{2a}\ln(\tan^2(ax+b)+1) + C$$
$$\int \cot(ax+b)\ dx = -\frac{1}{2a}\ln(\tan^2(ax+b)+1) + \frac{1}{a}\ln(\tan(ax+b)) + C$$
$$\int \sec(ax+b)\ dx = -\frac{1}{a}\frac{\ln(\cos(ax+b))}{\sin(ax+b)}$$
$$\int \csc(ax+b)\ dx = \frac{1}{a}\frac{\ln(\sin(ax+b))}{\cos(ax+b)}$$

At first, the raw codes from SymbolicC++ 3.35 cannot perform that computation, in the **src/functions.cpp** there are implementations for $\sin, \cos, \tan$ but they are imperfect and if the case is not covered it will has an output of **return Integral(*this,s)**, which mean the integral cannot be computed.

So we correct it and put the formula so SymIntegration can compute the integral problems above, which we call them as Level 1 trigonometry problems.

```
...
...

Symbolic Tan::integrate(const Symbolic &s) const
{
       const Symbolic &x = parameters.front();
       if(x == s) return -ln(cos(x)) * (1 / parameters.front().
           df(s));
       if(df(s) == 0) return *this * s;
       return ln( tan(parameters.front()) * tan(parameters.
           front()) + 1) * ( 1 / (2*parameters.front().df(s)) )
            ;
}

...
```

Code 12: *Implementation of integral for tangent function*

README.md | functions.cpp | main.cpp

```
171 Symbolic Acos::integrate(const Symbolic &s) const
172 {
173  const Symbolic &x = parameters.front();
174  if(x == s) return x*acos(x) - sqrt(1-x*x) ;
175  if(df(s) == 0) return *this * s;
176  return ( - sqrt(1-(parameters.front())*(parameters.front()) ) /  parameters.front().df(s) ) + ( parameters.front()*acos(parameters.front()) ,
177 }
178
179
180 ////////////////////////////////////
181 // Implementation of Tan         //
182 ////////////////////////////////////
183
184 Tan::Tan(const Tan &s) : Symbol(s) {}
185
186 Tan::Tan(const Symbolic &s) : Symbol(Symbol("tan")[s]) {}
187
188 Simplified Tan::simplify() const
189 {
190  const Symbolic &s = parameters.front().simplify();
191  if(s == 0) return Number<int>(0);
192  if(s.type() == typeid(Product))
193  {
194   CastPtr<const Product> p(s);
195   if(p->factors.front() == -1) return -Tan(-s);
196  }
197  if(s.type() == typeid(Numeric) &&
198     Number<void>(s).numerictype() == typeid(double))
199   return Number<double>(tan(CastPtr<const Number<double> >(s)->n));
200  return *this;
201 }
202
203 Symbolic Tan::df(const Symbolic &s) const
204 { return tan(parameters.front()) * tan(parameters.front()) * parameters.front().df(s) + parameters.front().df(s) ; }
205
206 Symbolic Tan::integrate(const Symbolic &s) const
207 {
208  const Symbolic &x = parameters.front();
209  if(x == s) return -ln(cos(x)) * (1 / parameters.front().df(s));
210  if(df(s) == 0) return *this * s;
211  return ln( tan(parameters.front()) * tan(parameters.front()) + 1) * ( 1 / (2*parameters.front().df(s)) ) ;
212 }
```

Figure 2.1: *The implementation of tangent function to compute its derivative and integral in* ***src/functions.cpp***.

[SI*] **Trigonometry Level 2**

The trigonometry integrals that we consider as level 2 are

$$\int \sin\ mx\ \cos\ nx\ dx$$
$$\int \cos\ mx\ \cos\ nx\ dx$$
$$\int \sin\ mx\ \sin\ nx\ dx$$

Integrals of this type occurs in many physics and engineering applications. By hands we can use the product identitites to handle these integrals.

[SI*] **Trigonometry Level 3**

The trigonometry integrals that we consider as level 3 are

$$\int \sin^n x\ dx$$
$$\int \cos^n x\ dx$$
$$\int \tan^n x\ dx$$
$$\int \sec^n x\ dx$$
$$\int \csc^n x\ dx$$
$$\int \cot^n x\ dx$$

We happen to check on Wolfram Alpha for these indefinite integrals. The Wolfram Alpha gives result of these integrals in hypergeometric function term, for example:

$$\int \sin^n\ x\ dx = -\cos(x)\sin^{n+1}(x)\sin^{-n-1}(x)\ {}_2F_1(\frac{1}{2}, \frac{1-n}{2}; \frac{3}{2}; \cos^2(x))$$

For the record, hypergeometric function ${}_2F_1(\frac{1}{2}, \frac{1-n}{2}; \frac{3}{2}; \cos^2(x))$ can be computed with SymIntegration, Boost, or even self made C++ codes, but it will produce approximated integral computation, and we tried to use this formulas with hypergeometric function, it compute very slow.

So we refer to the reduction formula to compute these trigonometry integral level 3.

$$\begin{aligned}
\int \sin^n x\ dx &= -\frac{1}{n}\sin^{n-1}(x)\ \cos(x) + \frac{n-1}{n}\int \sin^{n-2}(x)\ dx \\
\int \cos^n x\ dx &= \frac{1}{n}\cos^{n-1}(x)\ \sin(x) + \frac{n-1}{n}\int \cos^{n-2}(x)\ dx \\
\int \tan^n x\ dx &= \frac{\tan^{n-1}(x)}{n-1} - \int \tan^{n-2}(x)\ dx \\
\int \sec^n x\ dx &= \frac{\sec^{n-2}(x)\ \tan(x)}{n-1} + \frac{n-2}{n-1}\int \sec^{n-2}(x)\ dx \\
\int \csc^n x\ dx &= \frac{-\csc^{n-2}(x)\ \cot(x)}{n-1} + \frac{n-2}{n-1}\int \csc^{n-2}(x)\ dx \\
\int \cot^n x\ dx &= -\frac{1}{n-1}\cot^{n-1}(x) - \int \cot^{n-2}(x)\ dx
\end{aligned} \tag{2.15}$$

With this reduction formula we can use recursive method, with **for** loop technique in C++ after we find the pattern, we can code it easily. The computation time with reduction formula and this recursive method is faster than using Hypergeometric function.

[SI*] We will examine each integral and learn about their patterns, one by one. Starting with $\int \sin^n x\, dx$

i. $\int \sin^n x\, dx$

These are the formula of $\int \sin^n x\, dx$ with $n = 2$ to $n = 8$.

$$\int \sin^2 x\, dx = \frac{x}{2} - \frac{\sin\ x\ \cos\ x}{2}$$

$$\int \sin^3 x\, dx = \frac{\cos^3 x}{3} - \cos\ x$$

$$\int \sin^4 x\, dx = \frac{3x}{8} - \frac{\sin^3\ x\ \cos\ x}{4} - \frac{3\sin\ x\ \cos\ x}{8}$$

$$\int \sin^5 x\, dx = -\frac{\cos^5 x}{5} + \frac{2\cos^3 x}{3} - \cos\ x$$

$$\int \sin^6 x\, dx = \frac{5x}{16} - \frac{\sin^5\ x\ \cos\ x}{6} - \frac{5\sin^3\ x\ \cos\ x}{24} - \frac{5\sin\ x\ \cos\ x}{16}$$

$$\int \sin^7 x\, dx = \frac{\cos^7 x}{7} - \frac{3\cos^5 x}{5} + \frac{3\cos^3 x}{3} - \cos\ x$$

$$\int \sin^8 x\, dx = \frac{35x}{128} - \frac{\sin^7\ x\ \cos\ x}{8} - \frac{7\sin^5\ x\ \cos\ x}{48} - \frac{35\sin^3\ x\ \cos\ x}{192} - \frac{35\sin\ x\ \cos\ x}{128}$$

From the first 8 terms we will get this intuition that the pattern occur for even power and odd power, so it will divide into two cases, in C++ we can use the statement (**n % 2 == 0**) in the **if** statement to handle this.

The manual way to solve this can be traced back to the reduction formula, for example if you want to compute $\int \sin^8 x\, dx$, then you start in a decreasing **for** loop with $i = n, i >= 2, i = i - 2$.

Case 1: For $\int \sin^n x\, dx$ with even n
The numerator coefficient and denominator coefficient are making patterns that can be decoded.

```
...

Symbolic Power::integrate(const Symbolic &s) const
{
const Symbolic &a = parameters.front();
const Symbolic &b = parameters.back();
int bpower = parameters.back().coeff(s,0);
if(a.type() == typeid(Sin))
{
       if(b.df(s) == 0 && b!=0 && bpower % 2 == 0) // even power case
       {
              Symbolic c = 1;
```

```
                Symbolic d = parameters.back().coeff(s,0);
                Symbolic integral;
                for(int i = bpower ; i >= 2 ; i = i-2)
                {
                        integral -= c*cos(s)*((sin(s))^(i-1))/(d) ;
                        d = d*(i-2);
                        c = c*(i-1);
                        if (i==4)
                        {
                                integral += ( (c*s)/(d) );
                        }
                }
        return integral;
        }
}
...
```

Code 13: *level 3 integral for sine even case*

As you can see the logic for the code above: we multiply the numerator with c that is the odd factorial $(1 \times 3 \times 5 \times \cdots \times n-1)$, and multiply the denominator with the even factorial $(2 \times 4 \times 6 \times \cdots \times n)$.

The looping starts from the **bpower**, the variable name to represents n, the power for the integral of $\sin^n x$, then descending to 2, and decreasing by 2.

Notice that the numerator and denominator coefficient for x is the same as $\sin\ x\ \cos\ x$, so we only need to compute the coefficient for $\sin\ x\ \cos\ x$ and then copy it as the coefficient for x.

We start to compute the coefficient with the highest power, for example we want to compute

$\int \sin^8 x\ dx$, then we start with $i = 8$ and we will have:

$$
\begin{aligned}
i &= 8 \\
c &= 1 \\
d &= 8 \\
\text{integral} &= -\frac{c * \cos(s) * \sin^{i-1}(s)}{d} \\
d &= d * (i-2) \\
c &= c * (i-1) \\
i &= i-2 \\
&* * * * * * * * * * * * \\
i &= 6 \\
c &= 7 \\
d &= 48 \\
\text{integral} &= -\frac{c * \cos(s) * \sin^{i-1}(s)}{d} \\
d &= d * (i-2) \\
c &= c * (i-1) \\
i &= i-2 \\
&* * * * * * * * * * * * \\
i &= 4 \\
c &= 35 \\
d &= 192 \\
\text{integral} &= -\frac{c * \cos(s) * \sin^{i-1}(s)}{d} \\
d &= d * (i-2) \\
c &= c * (i-1) \\
i &= i-2 \\
&* * * * * * * * * * * * \\
i &= 2 \\
c &= 105 \\
d &= 384 \\
\text{integral} &= -\frac{c * \cos(s) * \sin^{i-1}(s)}{d} \\
d &= d * (i-2) \\
c &= c * (i-1) \\
i &= i-2
\end{aligned}
$$

We stop when $i = 2$ and we will obtain $\frac{105}{384} = \frac{35}{128}$ as the coefficient for $\sin\ x\ \cos\ x$.

It only takes a lot of papers and pen to be able to decode this patterns. Then some patience and persistence with focus to create the C++ codes.

Case 2: For $\int \sin^n x\, dx$ with odd n
The numerator coefficient and denominator coefficient are making patterns that can be decoded. This time it is easier than the even case, but a little bit tricky with alternating sign and need to use combination formula.

```
...

Symbolic Power::integrate(const Symbolic &s) const
{
...
if(a.type() == typeid(Sin))
{
if(b.df(s) == 0 && b!=0 && bpower % 2 != 0) // odd power case
{
       Symbolic sgn = -1;
       Symbolic integral;
       int j =1;
       int m = bpower-(0.5*(bpower+1));
       for(int i = 1 ; i <= bpower ; i = i+2)
       {
              integral += sgn*combinations(m,j-1)*((cos(s))^(i))/(i);
              sgn = -sgn;
              j = j+1;
       }
       return integral;
}
}
```

Code 14: *level 3 integral for sine odd case*

When I was trying to find the pattern, it is nice to write from the lowest odd power to a certain odd power and see the pattern for the numerator and denominator as the power rises.

$$\int \sin^3 x\, dx = \frac{\cos^3 x}{3} - \cos\ x$$
$$\int \sin^5 x\, dx = -\frac{\cos^5 x}{5} + \frac{2\cos^3 x}{3} - \cos\ x$$
$$\int \sin^7 x\, dx = \frac{\cos^7 x}{7} - \frac{3\cos^5 x}{5} + \frac{3\cos^3 x}{3} - \cos\ x$$
$$\int \sin^9 x\, dx = -\frac{\cos^9 x}{9} + \frac{4\cos^7 x}{7} - \frac{6\cos^5 x}{5} + \frac{4\cos^3 x}{3} - \cos\ x$$

Right from the bat, the denominator has an easy pattern of odd number decreasing from the n to 1, the denominator has the same coefficient as the power for the $\cos(x)$. So it is clear already.

The problem now, is the numerator, we have this kind of pattern:

$$\begin{array}{ccccccccc} & & & 1 & & 1 & & & \\ & & 1 & & 2 & & 1 & & \\ & 1 & & 3 & & 3 & & 1 & \\ 1 & & 4 & & 6 & & 4 & & 1 \end{array}$$

It is the infamous Pascal' triangle. Which can be derived from the combination formula:

$$\begin{aligned} {}_0C_0 &= 1 \\ {}_1C_0 &= 1 \\ {}_1C_1 &= 1 \\ {}_2C_0 &= 1 \\ {}_2C_1 &= 2 \\ {}_2C_2 &= 1 \\ {}_3C_0 &= 1 \\ {}_3C_1 &= 3 \\ {}_3C_2 &= 3 \\ {}_3C_3 &= 1 \\ {}_4C_0 &= 1 \\ {}_4C_1 &= 4 \\ {}_4C_2 &= 6 \\ {}_4C_3 &= 4 \\ {}_4C_4 &= 1 \end{aligned}$$

This explains the usage of **combinations** function in the C++ code.

ii. $\int \cos^n x \, dx$

These are the formula of $\int \cos^n x \, dx$ with $n = 2$ to $n = 8$.

$$\begin{aligned} \int \cos^2 x \, dx &= \frac{x}{2} + \frac{\sin\ x\ \cos\ x}{2} \\ \int \cos^3 x \, dx &= -\frac{\sin^3 x}{3} + \sin\ x \\ \int \cos^4 x \, dx &= \frac{3x}{8} + \frac{\sin\ x\ \cos^3\ x}{4} + \frac{3\sin\ x\ \cos\ x}{8} \\ \int \cos^5 x \, dx &= \frac{\sin^5 x}{5} - \frac{2\sin^3 x}{3} + \sin\ x \\ \int \cos^6 x \, dx &= \frac{5x}{16} + \frac{\sin\ x\ \cos^5\ x}{6} + \frac{5\sin\ x\ \cos^3\ x}{24} + \frac{5\sin\ x\ \cos\ x}{16} \\ \int \cos^7 x \, dx &= -\frac{\sin^7 x}{7} + \frac{3\sin^5 x}{5} - \frac{3\sin^3 x}{3} + \sin\ x \\ \int \cos^8 x \, dx &= \frac{35x}{128} + \frac{\sin\ x\ \cos^7\ x}{8} + \frac{7\sin\ x\ \cos^5\ x}{48} + \frac{35\sin\ x\ \cos^3\ x}{192} + \frac{35\sin\ x\ \cos\ x}{128} \end{aligned}$$

Right after we observe the equations above, it is really familiar, the pattern for the numerator and denominator will be like the sine counterpart, thus we only need to exchange the sign, what is $-$ become $+$ and the other way around, and also exchanging sine into cosine and cosine into sine.

```
...

Symbolic Power::integrate(const Symbolic &s) const
{
const Symbolic &a = parameters.front();
const Symbolic &b = parameters.back();
int bpower = parameters.back().coeff(s,0);
if(a.type() == typeid(Cos))
{
       if(b.df(s) == 0 && b!=0 && bpower % 2 == 0) // even power case
       {
              Symbolic c = 1;
              Symbolic d = parameters.back().coeff(s,0);
              Symbolic integral;
              for(int i = bpower ; i >= 2 ; i = i-2)
              {
                     integral += c*sin(s)*((cos(s))^(i-1))/(d) ;
                     d = d*(i-2);
                     c = c*(i-1);
                     if (i==4)
                     {
                            integral += ( (c*s)/(d) );
                     }
              }
              return integral;
       }
}
...
```

Code 15: *level 3 integral for cosine even case*

```
...

Symbolic Power::integrate(const Symbolic &s) const
{
       const Symbolic &a = parameters.front();
       const Symbolic &b = parameters.back();
       int bpower = parameters.back().coeff(s,0);
       if(a.type() == typeid(Cos))
       {
              if(b.df(s) == 0 && b!=0 && bpower % 2 != 0) // odd power
                   case
              {
              Symbolic sgn = 1;
```

```
                    Symbolic integral;
                    int j =1;
                    int m = bpower—(0.5*(bpower+1));
                    for(int i = 1 ; i <= bpower ; i = i+2)
                    {
                    integral += sgn*combinations(m,j—1)*((sin(s))^(i))/(i);
                    sgn = —sgn;
                    j = j+1;
                    }
                    return integral;
                    }
            }
            ...
```

Code 16: *level 3 integral for cosine odd case*

iii. $\int \sec^n x\, dx$

These are the formula of $\int \sec^n x\, dx$ with $n = 2$ to $n = 8$.

$$\int \sec^2 x\, dx = \frac{\sin\ x}{\cos\ x}$$
$$\int \sec^3 x\, dx = -\frac{\log(\sin\ (x) - 1)}{4} + \frac{\log(\sin\ (x) + 1)}{4} - \frac{\sin\ x}{2\sin^2 x - 2}$$
$$\int \sec^4 x\, dx = \frac{2\sin\ x}{3\cos\ x} + \frac{\sin\ x}{3\cos^3\ x}$$
$$\int \sec^5 x\, dx = -\frac{3\sin^3\ x - 5\sin\ x}{8\sin^4\ x - 16\sin^2\ x + 8} - \frac{3\log(\sin\ (x) - 1)}{16} + \frac{3\log(\sin\ (x) + 1)}{16}$$
$$\int \sec^6 x\, dx = \frac{8\sin\ x}{15\cos\ x} + \frac{4\sin\ x}{15\cos^3\ x} + \frac{\sin\ x}{5\cos^5\ x}$$
$$\int \sec^7 x\, dx = \frac{-15\sin^5\ x + 40\sin^3\ x - 33\sin\ x}{48\sin^6\ x - 144\sin^4\ x + 144\sin^2\ x - 48} - \frac{5\log(\sin\ (x) - 1)}{32} + \frac{5\log(\sin\ (x) + 1)}{32}$$
$$\int \sec^8 x\, dx = \frac{16\sin\ x}{35\cos\ x} + \frac{8\sin\ x}{35\cos^3\ x} + \frac{6\sin\ x}{35\cos^5\ x} + \frac{\sin\ x}{7\cos^7\ x}$$

The same case occurs again here, the pattern is repeated every $n + 2$, so there will be the odd power pattern and the even power pattern.

Case 1: For $\int \sec^n x\, dx$ with even n
The numerator coefficient and denominator coefficient are making patterns that can be decoded.

$$\int \sec^2 x\, dx = \frac{\sin\ x}{\cos\ x}$$
$$\int \sec^4 x\, dx = \frac{2\sin\ x}{3\cos\ x} + \frac{\sin\ x}{3\cos^3\ x}$$
$$\int \sec^6 x\, dx = \frac{8\sin\ x}{15\cos\ x} + \frac{4\sin\ x}{15\cos^3\ x} + \frac{\sin\ x}{5\cos^5\ x}$$
$$\int \sec^8 x\, dx = \frac{16\sin\ x}{35\cos\ x} + \frac{8\sin\ x}{35\cos^3\ x} + \frac{6\sin\ x}{35\cos^5\ x} + \frac{\sin\ x}{7\cos^7\ x}$$
$$\int \sec^{10} x\, dx = \frac{128\sin\ x}{315\cos\ x} + \frac{64\sin\ x}{315\cos^3\ x} + \frac{16\sin\ x}{105\cos^5\ x} + \frac{8\sin\ x}{63\cos^7\ x} + \frac{\sin\ x}{9\cos^9\ x}$$

The patterns that can be seen from those equations above are

- You may have misinterpreted it first, but the denominator are all the same, for example:

  $$\int \sec^6 x\, dx = \frac{8\sin\ x}{15\cos\ x} + \frac{4\sin\ x}{15\cos^3\ x} + \frac{\sin\ x}{5\cos^5\ x}$$

  is the same as

  $$\int \sec^6 x\, dx = \frac{8\sin\ x}{15\cos\ x} + \frac{4\sin\ x}{15\cos^3\ x} + \frac{3\sin\ x}{15\cos^5\ x}$$

  Learning from that, the pattern for the numerator can be decoded.

Knowing that the denominator is the same, we only need to find out, how we get the denominator coefficient of 1 for $n = 2$, or of 3 for $n = 4$, or of 15 for $n = 6$.

Turns out, the first intuition is quite correct, it is the factorial odd, we will declare the denominator as variable d_0:

$$d_0 = (n-1) * (n-3) * (n-5) * \cdots * (1)$$

If it is $\int \sec^8 x \, dx$, then the denominator will be

$$d_0 = 7 * 5 * 3 * 1 = 105$$

In the equation for $\int \sec^8 x \, dx$, the denominator is 35, instead of 105, this happens because the numerator is divisible with the denominator till they have no greatest common divisor anymore but 1, it is caused by the computer algebra system that always shows the most simplified version, so you can adjust the equation again to find the raw number for the denominator and the numerator. Making the denominator into the same number is not enough, so we need them in raw number version.

- After we make the denominator to become equal we will have this:

$$\int \sec^2 x \, dx = \frac{\sin x}{\cos x}$$
$$\int \sec^4 x \, dx = \frac{2\sin x}{3\cos x} + \frac{\sin x}{3\cos^3 x}$$
$$\int \sec^6 x \, dx = \frac{8\sin x}{15\cos x} + \frac{4\sin x}{15\cos^3 x} + \frac{3\sin x}{15\cos^5 x}$$
$$\int \sec^8 x \, dx = \frac{16\sin x}{35\cos x} + \frac{8\sin x}{35\cos^3 x} + \frac{6\sin x}{35\cos^5 x} + \frac{5\sin x}{35\cos^7 x}$$
$$\int \sec^{10} x \, dx = \frac{128\sin x}{315\cos x} + \frac{64\sin x}{315\cos^3 x} + \frac{48\sin x}{315\cos^5 x} + \frac{40\sin x}{315\cos^7 x} + \frac{35\sin x}{315\cos^9 x}$$

so the pattern of the numerator can be seen like this:

$$\begin{array}{ccccccccc} & & & & 1 & & & & \\ & & & 1 & & 2 & & & \\ & & 3 & & 4 & & 8 & & \\ & 5 & & 6 & & 8 & & 16 & \\ 35 & & 40 & & 48 & & 64 & & 128 \end{array}$$

Notice that we have to change the denominator into its raw version (for $\int \sec^8 x \, dx$ the denominator should be 105 instead of 35, and for $\int \sec^{10} x \, dx$ the denominator should be 945 instead of 315), this is the logic: the pattern can be decoded at its' raw numbers.

Thus

$$\begin{array}{ccccccccc} & & & & 1 & & & & \\ & & & 1 & & 2 & & & \\ & & 3 & & 4 & & 8 & & \\ & 15 & & 18 & & 24 & & 48 & \\ 105 & & 120 & & 144 & & 192 & & 384 \end{array}$$

The first entry for the next iteration is an odd multiplication of the current first entry from before with an increasing odd number. The first iteration will multiplied by 1, the

second will be multiplied by 3, the next iteration will be multiplied by 5, and so on. This odd multiplication will be called as k, While the second entry to the last entry will be multiplied by even multiplication that will be called as l and it is increasing by 2 at every iteration.

$$
\begin{aligned}
&\text{for } i = 1 \rightarrow \mathbf{v[0]} = 1 * k_{i=1} = 1 \\
&\text{for } i = 1 \rightarrow \mathbf{v[1]} = 1 * l_{i=1} = 2 \\
&\text{for } i = 2 \rightarrow \mathbf{v[0]} = \mathbf{v[0]}_{i=1} * k_{i=2} = 1 * 3 = 3 \\
&\text{for } i = 2 \rightarrow \mathbf{v[1]} = \mathbf{v[0]}_{i=1} * l_{i=2} = 1 * 4 = 4 \\
&\text{for } i = 2 \rightarrow \mathbf{v[2]} = \mathbf{v[1]}_{i=1} * l_{i=2} = 2 * 4 = 8 \\
&\text{for } i = 3 \rightarrow \mathbf{v[0]} = \mathbf{v[0]}_{i=2} * k_{i=3} = 3 * 5 = 15 \\
&\text{for } i = 3 \rightarrow \mathbf{v[1]} = \mathbf{v[0]}_{i=2} * l_{i=3} = 3 * 6 = 18 \\
&\text{for } i = 3 \rightarrow \mathbf{v[2]} = \mathbf{v[1]}_{i=2} * l_{i=3} = 4 * 6 = 24 \\
&\text{for } i = 3 \rightarrow \mathbf{v[3]} = \mathbf{v[2]}_{i=2} * l_{i=3} = 8 * 6 = 48
\end{aligned}
$$

Now we already have an idea about the pattern. For the C++ code, this time instead of using vector we will use array that is initialized with a very big size: **int v[999]**. Array is often used to replace vector, since vector has an invalid pointer problem / out of bounds memory access if you are not careful, array can also be used to become a matrix.

In order to determine the coefficient of the numerator with sine function we will use ascending **for** loop, to be exact we will use **for**$(i = 1; i < n - 1; i = i + 2)$.

We will declare variables / constants to help us compute, which are $d_0 = 1, k = 1, l = 2, c = 1$. Their value will be changing in every iteration (c will be used to store the first entry ($\mathbf{v}[0]$) of the previous array, $k = k + 2$ and $l = l + 2$).

We save the first entry of the previous array because it is going to be used to determine the first and second entry of the array at the current iteration.

$$
\begin{aligned}
k &= 1 \\
l &= 2 \\
d_0 &= 1 \\
c &= 1
\end{aligned}
$$

In every iteration we will clear the entries in the array.

So the first **for** loop will be like this:

$$
\begin{gathered}
i = 1 \\
d_0 = 3 \\
l = 4 \\
k = 3 \\
v[0] = 1 \\
v[1] = 2 \\
* * * * * * * * * * * \\
i = 3 \\
d_0 = 15 \\
l = 6 \\
k = 5 \\
v[0] = 3 \\
v[1] = 4 \\
v[2] = 8 \\
* * * * * * * * * * * \\
i = 5 \\
d_0 = 105 \\
l = 8 \\
k = 7 \\
v[0] = 15 \\
v[1] = 18 \\
v[2] = 24 \\
v[3] = 48 \\
* * * * * * * * * * * \\
i = 7 \\
d_0 = 945 \\
l = 10 \\
k = 9 \\
v[0] = 105 \\
v[1] = 120 \\
v[2] = 144 \\
v[3] = 192 \\
v[4] = 384
\end{gathered}
$$

```
...

Symbolic Power::integrate(const Symbolic &s) const
{
       ...
```

```
if(a.type() == typeid(Sec))
{
...

if(b.df(s) == 0 && b!=0 && bpower % 2 == 0) // even power case
{
int v[999];
v[0] = 1;
Symbolic integral;
Symbolic d0 = 1;
int k = 1, l = 2, c=1;
for(int i = 1 ; i < bpower ; i = i+2) // to compute the denominator
{
       d0 *= i;
}
for(int i = 1 ; i < bpower - 1; i = i+2)
{
       c = v[0];
       int arrsec[999]; // make the size of the array as big as
           possible

       for(int j = 0 ; j < i-1 ; j = j+1)
       {
              arrsec[j] = v[j];
       }
       int d;
       for(int j = 1 ; j < i ; j = j+1)
       {
              d = arrsec[j-1];
              v[j] = d*l;
       }

       v[0] = c*k;
       v[1] = c*l;

       k= k + 2;
       l = l + 2;
}

int j_d = 1 ;
for(int i = 1 ; i < (0.5*bpower)+1; i = i+1)
{
       integral += ( v[i-1] * sin(s) ) / ( d0*(cos(s)^(bpower-j_d)) )
            ;
       j_d = j_d+2;
}
return integral;
}
```

```
        }
}
```

Code 17: *level 3 integral for secant even case*

Case 2: For $\int \sec^n x\, dx$ with odd n
Compared to the even case, the odd case for $\int \sec^n x\, dx$ is harder in terms of determining the pattern and then to code it with C++, we even have to use two vectors, the first one to store the coefficient for the numerator, and the second vector is used to store the "middle coefficient", in order to determine the correct coefficient for the numerator.

$$\int \sec^3 x\, dx = -\frac{\log(\sin(x)-1)}{4} + \frac{\log(\sin(x)+1)}{4} - \frac{\sin x}{2\sin^2 x - 2}$$

$$\int \sec^5 x\, dx = -\frac{3\sin^3 x - 5\sin x}{8\sin^4 x - 16\sin^2 x + 8} - \frac{3\log(\sin(x)-1)}{16} + \frac{3\log(\sin(x)+1)}{16}$$

$$\int \sec^7 x\, dx = \frac{-15\sin^5 x + 40\sin^3 x - 33\sin x}{48\sin^6 x - 144\sin^4 x + 144\sin^2 x - 48} - \frac{5\log(\sin(x)-1)}{32} + \frac{5\log(\sin(x)+1)}{32}$$

$$\int \sec^9 x\, dx = -\frac{105\sin^7 x - 385\sin^5 x + 511\sin^3 x - 279\sin x}{384\sin^8 x - 1536\sin^6 x + 2304\sin^4 x - 1536\sin^2 x + 384} - \frac{35\log(\sin(x)-1)}{256} + \frac{35\log(\sin(x)+1)}{256}$$

$$\int \sec^{11} x\, dx = \frac{-315\sin^9 x + 1470\sin^7 x - 2688\sin^5 x + 2370\sin^3 x - 965\sin x}{1280\sin^{10} x - 6400\sin^8 x + 12800\sin^6 x - 12800\sin^4 x + 6400\sin^2 x - 1280} - \frac{63\log(\sin(x)-1)}{512} + \frac{63\log(\sin(x)+1)}{512}$$

If we want to compute it with C++, then we can use the **for** as usual to help us. Then it is either ascending **for** loop or descending **for** loop. As time goes by and flight hours, you will know which one to use.

We can immediately notice some very obvious patterns:

- We have 3 different terms here, 2 are the log terms and the last is the term with numerator that has the sum of alternating sign of sine with odd power and denominator that has the sum of alternating sign of sine with even power.
- The terms with log has denominator of with multiplication of $(2 * i)$ that still store the previous value so we can determine it with $d_1 = d_1 * (2 * i)$. After we finish with the **for** loop, we will multiply again for the last time to obtain d_1 that will be used

$$d_1 = d_1 * (n-1)$$

- Now, we shall see the coefficient for the sum of alternating sign of sine with odd power:

$$1$$

$$3 \quad 5$$

$$15 \quad 40 \quad 33$$

$$105 \quad 385 \quad 511 \quad 279$$

315 1470 2688 2370 965

The first row represents $\int \sec^3 x\, dx$ and the last row represents $\int \sec^{11} x\, dx$. The sign is alternating actually, but it is omitted for better view, so the reader won't be confused. Alternating sign can be attached later on.

In this case, to determine the coefficient of the numerator with odd sine function we will use ascending **for** loop, to be exact we will use **for**$(i = 1; i < \frac{n-1}{2}; i = i + 1)$.

We also need to have extra variables to help compute the coefficient correctly, we give them initial value before all the **for** loop:

$$\begin{aligned} k &= 1 \\ l &= 2 \\ d_0 &= 2 \\ d_1 &= 2 \\ \text{first_coeff} &= 3 \\ j &= 1 \\ m &= n - \left(\frac{n+1}{2}\right) \\ sgn &= -1 \end{aligned}$$

We are going to use vector **v** to represent the coefficient for the numerator of sine with odd power, the size of the vector is $\frac{n-1}{2}$.

At first, when $i = 1$, we know that $\mathbf{v}[0] = 1$, so how to determine $\mathbf{v}[0]$ and $\mathbf{v}[1]$ for $i = 2$ and continuing the loop to compute for n at higher number?

Go back and see the reduction formula at Equation (2.6) for $\int \sec^n x\, dx$, we will notice that there is $n-2$ and $n-1$ terms at every recursive step / the loop. So we will have to see the multiplication possibilities for the factorial odd: $(n-2) * (n-4) * \cdots * 1$ and the factorial even: $(n-1) * (n-3) * (n-5) * \cdots * 2$. We are focusing on the numerator now, so it is the factorial odd that we will use (the one related to $n-2$ factorial down with difference of 2).

If you take a look again at the triangle above, the trend is increasing then decreasing at the end, it is the typical of Pascal' triangle but we do not use combination here. Here we try step by step from $n = 3$ or $i = 1$:

$$\begin{aligned} n &= 3 \\ i &= 1 \\ \mathbf{v}[0] &= 1 \end{aligned}$$

that will make

$$\int \sec^3 x\, dx = -\frac{\log(\sin(x) - 1)}{4} + \frac{\log(\sin(x) + 1)}{4} - \frac{\mathbf{v}[0] \sin x}{2\sin^2 x - 2}$$

moving on to $n = 5$ and $i = 2$

$$\begin{aligned} n &= 5 \\ i &= 2 \\ \mathbf{v}[0] &= 3 \\ \mathbf{v}[1] &= 5 \end{aligned}$$

$$\int \sec^5 x \, dx = -\frac{\mathbf{v}[0] \sin^3 x - \mathbf{v}[1] \sin x}{8 \sin^4 x - 16 \sin^2 x + 8} - \frac{3 \log(\sin(x) - 1)}{16} + \frac{3 \log(\sin(x) + 1)}{16}$$

Then to $n = 7$ and $i = 3$

$$\begin{aligned} n &= 7 \\ i &= 3 \\ \mathbf{v}[0] &= 15 \\ \mathbf{v}[1] &= 40 \\ \mathbf{v}[2] &= 33 \end{aligned}$$

$$\int \sec^7 x \, dx = \frac{\mathbf{v}[0] \sin^5 x + \mathbf{v}[1] \sin^3 x - \mathbf{v}[2] \sin x}{48 \sin^6 x - 144 \sin^4 x + 144 \sin^2 x - 48} - \frac{5 \log(\sin(x) - 1)}{32} + \frac{5 \log(\sin(x) + 1)}{32}$$

So to be able to create the C++ code to compute this coefficients, when we have the initial condition of $\mathbf{v}[0] = 1$ at $i = 1$, how do we get into $\mathbf{v}[0] = 3, \mathbf{v}[1] = 5$ at $i = 2$ and $\mathbf{v}[0] = 15, \mathbf{v}[1] = 40, \mathbf{v}[2] = 33$ at $i = 3$ and so on correctly?

Watch this, we first define $last_coeff = \mathbf{v}[j-1]$ with $j = 1, 2, 3, \ldots$ as the last index of

the vector from the previous i and $first_coeff = 3$, so

$$
\begin{aligned}
n &= 3 \\
i &= 1 \\
\mathbf{v}[0] &= 1 \\
last_coeff &= \mathbf{v}[0] = 1 \\
k &= 2 \\
&********** \\
n &= 5 \\
i &= 2 \\
\mathbf{v}[0] &= \mathbf{v}[0]_{i=1} * (n-2) = 3 \\
\mathbf{v}[1] &= [last_coeff * (n-2)] + k = [1*3] + 2 = 5 \\
\mathbf{mc}[0] &= \mathbf{v}[0] + \mathbf{v}[1] = 8 \\
last_coeff &= \mathbf{v}[1] = 5 \\
k &= 8 \\
&********** \\
n &= 7 \\
i &= 3 \\
\mathbf{v}[0] &= \mathbf{v}[0]_{i=2} * (n-2) = 3*5 = 15 \\
\mathbf{v}[1] &= \mathbf{mc}[0] * (n-2) = 8*5 = 40 \\
\mathbf{v}[2] &= [last_coeff * (n-2)] + k = [5*5] + 8 = 33 \\
\mathbf{mc}[0] &= \mathbf{v}[0] + \mathbf{v}[1] = 55 \\
\mathbf{mc}[1] &= \mathbf{v}[1] + \mathbf{v}[2] = 73 \\
last_coeff &= \mathbf{v}[2] = 33 \\
k &= 48 \\
&********** \\
n &= 9 \\
i &= 4 \\
\mathbf{v}[0] &= \mathbf{v}[0]_{i=3} * (n-2) = 15*7 = 105 \\
\mathbf{v}[1] &= \mathbf{mc}[0] * (n-2) = 55*7 = 385 \\
\mathbf{v}[2] &= \mathbf{mc}[1] * (n-2) = 73*7 = 511 \\
\mathbf{v}[3] &= [last_coeff * (n-2)] + k = [33*7] + 48 = 279 \\
\mathbf{mc}[0] &= \mathbf{v}[0] + \mathbf{v}[1] = 490 \\
\mathbf{mc}[1] &= \mathbf{v}[1] + \mathbf{v}[2] = 896 \\
\mathbf{mc}[2] &= \mathbf{v}[2] + \mathbf{v}[3] = 790 \\
last_coeff &= \mathbf{v}[2] = 279 \\
k &= 384
\end{aligned}
$$

The $last_coeff$ and $first_coeff$ will be used in the C++ codes to represent $n-2$, it is a little bit of a handy trick.

We finally know the pattern and formula to obtain the coefficients for the numerator. It is not an easy one as I spent days to realize how to obtain this.

- The denominator that has the sum of alternating sign of sine with even power has a distinctive pattern that can be seen when you take the greatest common diviser out:

$$8\sin^4 x - 16\sin^2 x + 8 = 8(\sin^4 x - 2\sin^2 x + 1)$$
$$48\sin^6 x - 144\sin^4 x + 144\sin^2 x - 48 = 48(\sin^6 x - 3\sin^4 x + 3\sin^2 x - 1)$$
$$384\sin^8 x - \cdots + \cdots + 384 = 384(\sin^8 x - 4\sin^6 x + 6\sin^4 x - 4\sin^2 x + 1)$$

We are focusing on the denominator now, so it is the factorial even that we will use, with the initial value of $d_0 = 2$ we will have:

$$\text{for } n = 5 \rightarrow \; d_0 = d_0 * (n-1) = d_0 * (2 + 2 * i) = 8$$
$$\text{for } n = 7 \rightarrow \; d_0 = d_0 * (n-1) = d_0 * (2 + 2 * i) = 48$$
$$\text{for } n = 9 \rightarrow \; d_0 = d_0 * (n-1) = d_0 * (2 + 2 * i) = 384$$

From this you can see clearly what to write for the C++ code.

So the first **for** loop will be like this:

$$
\begin{aligned}
i &= 1 \\
d_1 &= 4 \\
d_0 &= 8 \\
mc[0] &= 1 \\
v[0] &= 3 \\
v[1] &= 5 \\
&********** \\
i &= 2 \\
d_1 &= 16 \\
d_0 &= 48 \\
mc[0] &= 8 \\
v[0] &= 15 \\
v[1] &= 40 \\
v[2] &= 33 \\
&********** \\
i &= 3 \\
d_1 &= 96 \\
d_0 &= 384 \\
mc[0] &= 55 \\
mc[1] &= 73 \\
v[0] &= 105 \\
v[1] &= 385 \\
v[2] &= 511 \\
v[3] &= 279 \\
&********** \\
i &= 4 \\
d_1 &= 768 \\
d_0 &= 3840 \\
mc[0] &= 490 \\
mc[1] &= 896 \\
mc[2] &= 790 \\
v[0] &= 945 \\
v[1] &= 4410 \\
v[2] &= 8064 \\
v[3] &= 7110 \\
v[4] &= 2895
\end{aligned}
$$

The next two **for** loops won't be too hard to comprehend, we include the variable *sgn* to alternate the sign. Overall, to be able to detect the pattern out of this took quite some time,

and makes the author realize that integration to the power of n for basic trigonometry is related to sum / series, and also combination and Pascal' triangle. With that knowledge and logic, thus today Computer Algebra System, like SymIntegration or SymPy able to compute $\int \sec^n x \, dx$ for any integer number of n. It shows how beautiful mathematics is.

```
...

Symbolic Power::integrate(const Symbolic &s) const
{
...
       if(a.type() == typeid(Sec))
       {
       if(b.df(s) == 0 && b!=0 && bpower % 2 != 0) // odd power case
       {
              int k = 1, l=2;
              vector<int> v={1};
              vector<int> mc={1}; // to store the middle coefficient
              Symbolic sgn = -1;
              Symbolic integral_numerator, integral_denominator;
              Symbolic d0 = 2, d1 = 2;
              int j =1;
              int m = bpower-(0.5*(bpower+1));
              int last_coeff;
              int first_coeff = 3;

              // For the coefficient at the numerator of sine with odd power
              for(int i = 1 ; i < (bpower-1)/2 ; i = i+1)
              {
                     if (i >= 2)
                     {
                            for(int ic = 1 ; ic < i ; ic = ic+1)
                            {
                                   mc[ic-1] = v[ic-1] + v[ic];
                            }
                     }
                     k = k*l;
                     last_coeff = v[j-1];
                     v[0] = v[0]*(first_coeff+2*(i-1));
                     v.assign({v[0]});

                     for(int ic = 1 ; ic < i ; ic = ic+1)
                     {
                            v.push_back(mc[ic-1]*(first_coeff+2*(i-1)));
                     }
                     d1 = d1*(2*i);
                     d0 = d0*(2+2*i);
                     v.push_back(last_coeff*(first_coeff+2*(i-1))+k);
                     l = l+2;
                     j = j+1;
```

```
            }
            int j_num = 0 ;
            for(int i = bpower-2 ; i >= 1 ; i = i-2)
            {
                  integral_numerator += sgn*v[j_num]*((sin(s))^(i));
                  sgn = -sgn;
                  j_num = j_num+1;
            }
            sgn = 1;
            j = 1;
            for(int i = bpower-1 ; i >= 0 ; i = i-2)
            {
                  integral_denominator += sgn*d0*combinations(m,j-1)*((
                      sin(s))^(i));
                  sgn = -sgn;
                  j = j+1;
            }
            d1 = d1*(bpower-1);
            return (-v[0]*ln(sin(s)-1))/(d1) + (v[0]*ln(sin(s)+1))/(d1) +
                (integral_numerator)/(integral_denominator);
      }
      }
```

Code 18: *level 3 integral for secant odd case*

iv. $\int \csc^n x \, dx$

These are the formula of $\int \csc^n x \, dx$ with $n = 2$ to $n = 8$.

$$\int \csc^2 x \, dx = -\frac{\cos x}{\sin x}$$

$$\int \csc^3 x \, dx = \frac{\log(\cos(x) - 1)}{4} - \frac{\log(\cos(x) + 1)}{4} + \frac{\cos x}{2\cos^2 x - 2}$$

$$\int \csc^4 x \, dx = -\frac{2\cos x}{3\sin x} - \frac{\cos x}{3\sin^3 x}$$

$$\int \csc^5 x \, dx = \frac{3\cos^3 x - 5\cos x}{8\cos^4 x - 16\cos^2 x + 8} + \frac{3\log(\cos(x) - 1)}{16} - \frac{3\log(\cos(x) + 1)}{16}$$

$$\int \csc^6 x \, dx = -\frac{8\cos x}{15\sin x} - \frac{4\cos x}{15\sin^3 x} - \frac{\cos x}{5\sin^5 x}$$

$$\int \csc^7 x \, dx = -\frac{-15\cos^5 x + 40\cos^3 x - 33\cos x}{48\cos^6 x - 144\cos^4 x + 144\cos^2 x - 48} + \frac{5\log(\cos(x) - 1)}{32} - \frac{5\log(\cos(x) + 1)}{32}$$

$$\int \csc^8 x \, dx = -\frac{16\cos x}{35\sin x} - \frac{8\cos x}{35\sin^3 x} - \frac{6\cos x}{35\sin^5 x} - \frac{\cos x}{7\sin^7 x}$$

The pattern for the numerator and denominator will be like the secant counterpart, thus we only need to exchange the sign, what is $-$ become $+$ and the other way around, and also exchanging sine into cosine and cosine into sine.

```
...
```

```
Symbolic Power::integrate(const Symbolic &s) const
{
       ...
       if(a.type() == typeid(Csc))
       {
              ...

              if(b.df(s) == 0 && b!=0 && bpower % 2 == 0) // even power case
              {
                     int v[999];
                     v[0] = 1;
                     Symbolic integral;
                     Symbolic d0 = 1;
                     int k = 1, l = 2, c=1;
                     for(int i = 1 ; i < bpower ; i = i+2) // to compute the
                         denominator
                     {
                            d0 *= i;
                     }
                     for(int i = 1 ; i < bpower - 1; i = i+2)
                     {
                            c = v[0];
                            int arrsec[999]; // make the size of the array as
                                 big as possible

                            for(int j = 0 ; j < i-1 ; j = j+1)
                            {
                                   arrsec[j] = v[j];
                            }
                            int d;
                            for(int j = 1 ; j < i ; j = j+1)
                            {
                                   d = arrsec[j-1];
                                   v[j] = d*l;
                            }

                            v[0] = c*k;
                            v[1] = c*l;

                            k= k + 2;
                            l = l + 2;
                     }

                     int j_d = 1 ;
                     for(int i = 1 ; i < (0.5*bpower)+1; i = i+1)
                     {
                            integral -= ( v[i-1] * cos(s) ) / ( d0*(sin(s)^(
```

```
                                bpower-j_d)) ) ;
                        j_d = j_d+2;
                }
                return integral;
        }
    }
}
```

Code 19: *level 3 integral for cosecant even case*

```
...

Symbolic Power::integrate(const Symbolic &s) const
{
    ...
    if(a.type() == typeid(Csc))
    {
    ...

    if(b.df(s) == 0 && b!=0 && bpower % 2 != 0) // odd power case
    {
        int k = 1, l=2;
        vector<int> v={1};
        vector<int> mc={1}; // to store the middle coefficient
        Symbolic sgn = -1;
        Symbolic integral_numerator, integral_denominator;
        Symbolic d0 = 2, d1 = 2;
        int j =1;
        int m = bpower-(0.5*(bpower+1));
        int last_coeff;
        int first_coeff = 3;

        // For the coefficient at the numerator of cosine with odd
            power
        for(int i = 1 ; i < (bpower-1)/2 ; i = i+1)
        {
            if (i >= 2)
            {
                for(int ic = 1 ; ic < i ; ic = ic+1)
                {
                    mc[ic-1] = v[ic-1] + v[ic];
                }
            }
            k = k*l;
            last_coeff = v[j-1];
            v[0] = v[0]*(first_coeff+2*(i-1));
            v.assign({v[0]});

            for(int ic = 1 ; ic < i ; ic = ic+1)
```

```
                    {
                           v.push_back(mc[ic-1]*(first_coeff+2*(i-1)));
                    }
                    d1 = d1*(2*i);
                    d0 = d0*(2+2*i);
                    v.push_back(last_coeff*(first_coeff+2*(i-1))+k);
                    l = l+2;
                    j = j+1;
             }
             int j_num = 0 ;
             for(int i = bpower-2 ; i >= 1 ; i = i-2)
             {
                    integral_numerator += sgn*v[j_num]*((cos(s))^(i));
                    sgn = -sgn;
                    j_num = j_num+1;
             }
             sgn = 1;
             j = 1;
             for(int i = bpower-1 ; i >= 0 ; i = i-2)
             {
                    integral_denominator += sgn*d0*combinations(m,j-1)*((
                        cos(s))^(i));
                    sgn = -sgn;
                    j = j+1;
             }
             d1 = d1*(bpower-1);
             return (v[0]*ln(cos(s)-1))/(d1) - (v[0]*ln(cos(s)+1))/(d1) -
                 (integral_numerator)/(integral_denominator);
      }
      }
}
```

Code 20: *level 3 integral for cosecant odd case*

v. $\int \cot^n x \, dx$

These are the formula of $\int \cot^n x \, dx$ with $n = 2$ to $n = 8$.

$$\int \cot^2 x \, dx = -x - \frac{\cos x}{\sin x}$$
$$\int \cot^3 x \, dx = -\log(\sin(x)) - \frac{1}{2\sin^2 x}$$
$$\int \cot^4 x \, dx = x + \frac{\cos x}{\sin x} - \frac{\cos^3 x}{3\sin^3 x}$$
$$\int \cot^5 x \, dx = \log(\sin(x)) + \frac{4\sin^2 x - 1}{4\sin^4 x}$$
$$\int \cot^6 x \, dx = -x - \frac{\cos x}{\sin x} + \frac{\cos^3 x}{3\sin^3 x} - \frac{\cos^5 x}{5\sin^5 x}$$
$$\int \cot^7 x \, dx = -\log(\sin(x)) - \frac{18\sin^4 x - 9\sin^2 x + 2}{12\sin^6 x}$$
$$\int \cot^8 x \, dx = x + \frac{\cos x}{\sin x} - \frac{\cos^3 x}{3\sin^3 x} + \frac{\cos^5 x}{5\sin^5 x} - \frac{\cos^7 x}{7\sin^7 x}$$

Case 1: For $\int \cot^n x \, dx$ with even n
We will cluster only the even power

$$\int \cot^2 x \, dx = -x - \frac{\cos x}{\sin x}$$
$$\int \cot^4 x \, dx = x + \frac{\cos x}{\sin x} - \frac{\cos^3 x}{3\sin^3 x}$$
$$\int \cot^6 x \, dx = -x - \frac{\cos x}{\sin x} + \frac{\cos^3 x}{3\sin^3 x} - \frac{\cos^5 x}{5\sin^5 x}$$
$$\int \cot^8 x \, dx = x + \frac{\cos x}{\sin x} - \frac{\cos^3 x}{3\sin^3 x} + \frac{\cos^5 x}{5\sin^5 x} - \frac{\cos^7 x}{7\sin^7 x}$$

The patterns that can be seen from those equations above are

- The variable x is only alternating sign at every iteration.
- The numerator' coefficient with cosine function of odd power is only 1.
- The denominator coefficient is the same as the power of the sine function in the denominator.
- The integral computed at the previous iteration alternates its sign at the next iteration minus the new function with power of $n - 1$.

The C++ code is an easy one here, the simplest out of all level 3 trigonometry integral, it alternates the whole sign at every iteration. We will use an increasing **for** loop with $i = 2, i <= n, i = i + 2$.

```
...

Symbolic Power::integrate(const Symbolic &s) const
{
	...
	if(a.type() == typeid(Cot))
	{
```

```
        ...
        if(b.df(s) == 0 && b!=0 && bpower % 2 == 0) // even power case
        {
                Symbolic integral;
                Symbolic integral_front;
                Symbolic sgn = -1;
                for(int i = 2 ; i <= (bpower) ; i = i+2)
                {
                        integral = (-1)*integral;
                        integral += - (cos(s)^(i-1)) / ((i-1)*(sin(s)^(i-1)));
                }

                for(int i = 1 ; i <= (bpower)/2 ; i = i+1)
                {
                        integral_front = sgn;
                        sgn = -sgn;
                }
                return integral_front*s + integral;
        }
}
```

Code 21: *level 3 integral for cotangent even case*

Case 2: For $\int \cot^n x\, dx$ with odd n
We will cluster only the odd power

$$\int \cot^3 x\, dx = -\log(\sin(x)) - \frac{1}{2\sin^2 x}$$
$$\int \cot^5 x\, dx = \log(\sin(x)) + \frac{4\sin^2 x - 1}{4\sin^4 x}$$
$$\int \cot^7 x\, dx = -\log(\sin(x)) - \frac{18\sin^4 x - 9\sin^2 x + 2}{12\sin^6 x}$$
$$\int \cot^9 x\, dx = \log(\sin(x)) + \frac{48\sin^4 x - 36\sin^4 x + 16\sin^2 x - 3}{24\sin^8 x}$$
$$\int \cot^{11} x\, dx = -\log(\sin(x)) - \frac{300\sin^8 x - 300\sin^6 x + 200\sin^4 x - 75\sin^2 x + 12}{120\sin^{10} x}$$

The patterns that can be seen from those equations above are

- The $\log(\sin(x))$ function is only alternating the sign at every iteration.
- The denominator for the sine function with power of $n-1$ is making a pattern like this (d_0 is the variable to represent the denominator)

$$d_0 = d_0 * \left(\frac{n-1}{2}\right)$$

We start with $d_0 = 2$ then

$$d_0 = 2 * \left(\frac{5-1}{2}\right) = 4$$
$$d_0 = 4 * \left(\frac{7-1}{2}\right) = 12$$

$$d_0 = 12 * \left(\frac{9-1}{2}\right) = 48$$
$$d_0 = 48 * \left(\frac{11-1}{2}\right) = 240$$

Knowing that the denominator has been simplified then we know that we have to adjust the numerator so they become the raw numbers like this:

$$\int \cot^3 x\, dx = -\log(\sin(x)) - \frac{1}{2\sin^2 x}$$
$$\int \cot^5 x\, dx = \log(\sin(x)) + \frac{4\sin^2 x - 1}{4\sin^4 x}$$
$$\int \cot^7 x\, dx = -\log(\sin(x)) - \frac{18\sin^4 x - 9\sin^2 x + 2}{12\sin^6 x}$$
$$\int \cot^9 x\, dx = \log(\sin(x)) + \frac{96\sin^4 x - 72\sin^4 x + 32\sin^2 x - 6}{48\sin^8 x}$$
$$\int \cot^{11} x\, dx = -\log(\sin(x)) - \frac{600\sin^8 x - 600\sin^6 x + 400\sin^4 x - 150\sin^2 x + 24}{120\sin^{10} x}$$

It is very important to do this in order to know the pattern for the numerator.

- To be able to know the pattern for the numerator with sine function of even power we can easily put the numerator coefficients like this for a better look:

$$\begin{array}{ccccccccccc} & & & & & 1 & & & & & \\ & & & & 1 & & 4 & & & & \\ & & & 2 & & 9 & & 18 & & & \\ & & 6 & & 32 & & 72 & & 96 & & \\ & 24 & & 150 & & 400 & & 600 & & 600 & \end{array}$$

The rule of thumb is we have to vectorize the coefficients, making it into a vector of size $\frac{n-1}{2}$ at every iteration. Back to the reduction formula, if we want to compute $\int \cot^{11} x\, dx$ we will start from $\int \cot^1 x\, dx$ then $\int \cot^3 x\, dx$, this is the recursive method.

When we create the vector at certain iteration, we will realize that to know the entries of this vector, we will need the values from certain variables that are having pattern, e.g. increasing at every iteration or decreasing at every iteration or multiplying with increasing coefficient. Now we take a look again. At the first iteration we have a vector $\mathbf{v}$ with only has one entry / size of 1 which is

$$\mathbf{v}[0] = 1$$

then at the next iteration we have a vector of size 2 with entries

$$\mathbf{v}[0] = 1$$
$$\mathbf{v}[1] = 4$$

then at the next iteration we have a vector of size 3 with entries

$$\mathbf{v}[0] = 2$$
$$\mathbf{v}[1] = 9$$
$$\mathbf{v}[2] = 18$$

We will use 4 vectors this time, since this one is quite tricky compared to the secant case. The vectors are: $\mathbf{v}, \mathbf{v}_{temp}, \mathbf{mc}, \mathbf{mc}_{temp}$.

We also need to use variables to help compute the coefficients with their initial value before the **for** loop :

$$d_0 = 2$$

$$k = 1$$

$$l = 2$$

We will use an increasing **for** loop with $i = 1, i <= \frac{n-1}{2}, i = i + 1$ because the size of the vector is getting bigger at every iteration, the size of the vector is the same as the iteration number.

You can and probably should write it on paper so you can learn more. To be simple, after several days of trial and error, the formula is shown to be like this:

$$\begin{aligned}
\mathbf{v}[0]_i &= \mathbf{v}[0]_{i-1} * (i-1) * \mathbf{mc}[0]_i \\
\mathbf{v}[j]_i &= [\mathbf{v}[j-1]_{i-1} * (2*i-j)] + [\mathbf{v}[0]_i * (\mathbf{mc}[j]_i)] \\
\mathbf{v}\left[\frac{n-1}{2}\right]_i &= \left[\mathbf{v}\left[\frac{n-1}{2}-1\right]_{i-1} * (2*i-j)\right] + (\mathbf{v}[0]_i * \mathbf{mc}\left[\frac{n-1}{2}\right]_i)
\end{aligned}$$

with i is the iteration $i = 1, 2, \ldots, \frac{n-1}{2}$, and $j = 1, 2, \ldots, \frac{n-1}{2}$. The vector **mc**, the middle coefficient is the Pascal' triangle

$$\begin{array}{ccccccccccccc}
 & & & & & & 1 & & & & & & \\
 & & & & & 1 & & 1 & & & & & \\
 & & & & 1 & & 2 & & 1 & & & & \\
 & & & 1 & & 3 & & 3 & & 1 & & & \\
 & & 1 & & 4 & & 6 & & 4 & & 1 & & \\
 & 1 & & 5 & & 10 & & 10 & & 5 & & 1 & \\
1 & & 6 & & 15 & & 20 & & 15 & & 6 & & 1
\end{array}$$

the top row represents $i = 1$ for $\int \cot^3 x \, dx$, and this vector **mc** helps computing the numerator coefficient.

The computation / coding of C++ for the numerator coefficients will be a tough one, it takes an increasing **for** loop with $i = 1, i <= \frac{n-1}{2}, i = i + 1$ and we split it into several cases.

So if we want to compute $\int \cot^{11} x \, dx$ we will gain the numerator coefficients at the iteration $i = 5$.

So the first **for** loop will be like this:

$$
\begin{aligned}
i &= 1 \\
d_0 &= 2 \\
mc[0] &= 1 \\
v[0] &= 1 \\
&**********\\
i &= 2 \\
d_0 &= 4 \\
mc[0] &= 1 \\
mc[1] &= 1 \\
v[0] &= 1 \\
v[1] &= 4 \\
&**********\\
i &= 3 \\
d_0 &= 12 \\
mc[0] &= 1 \\
mc[1] &= 2 \\
mc[1] &= 1 \\
v[0] &= 2 \\
v[1] &= 9 \\
v[2] &= 18 \\
&**********\\
i &= 4 \\
d_0 &= 48 \\
mc[0] &= 1 \\
mc[1] &= 3 \\
mc[2] &= 3 \\
mc[3] &= 1 \\
v[0] &= 6 \\
v[1] &= 32 \\
v[2] &= 72 \\
v[3] &= 96 \\
&**********
\end{aligned}
$$

$$
\begin{aligned}
i &= 5 \\
d_0 &= 240 \\
mc[0] &= 1 \\
mc[1] &= 4 \\
mc[2] &= 6 \\
mc[3] &= 4 \\
mc[4] &= 1 \\
v[0] &= 24 \\
v[1] &= 150 \\
v[2] &= 400 \\
v[3] &= 600 \\
v[4] &= 600
\end{aligned}
$$

We omitted the **mc**$[0]$ and **mc**$[i-1]$ in the C++ codes since it will only returns 1 so it won't really change much.

The rest of the **for** loops are only used to return the correct integral.

```
...

Symbolic Power::integrate(const Symbolic &s) const
{
       ...
       if(a.type() == typeid(Cot))
       {
       ...
       if(b.df(s) == 0 && b!=0 && bpower % 2 != 0) // odd power case
       {
              Symbolic integral;
              Symbolic integral_front;
              Symbolic sgn = -1;
              vector<int> v={1};
              vector<int> v_temp={1};
              vector<int> mc={1}; // to store the middle coefficient
              vector<int> mc_temp={1}; // to store the temporary / new middle
                   coefficient
              int d0 = 2;
              int k = 1, l=2;
              // For the coefficient at the numerator
              for(int i = 1 ; i <= (bpower-1)/2 ; i = i+1)
              {
                     if (i == 1)
                     {
                            v[0] = 1;
                            v.assign({v[0]});
                     }
```

```
                    if (i ==2)
                    {
                           mc[0]=1;
                           mc.assign({mc[0]});
                           v_temp[0] = v[0]*(i-1);
                           v_temp.assign({v_temp[0]});

                           for(int j = 1 ; j < i ; j = j+1)
                           {
                                  v_temp.push_back(v[j-1]*((2*i)-j) +
                                      v_temp[0]*mc[j-1] );
                           }
                           v[0] = v_temp[0];
                           v.assign({v[0]});
                           for(int j = 1 ; j < i ; j = j+1)
                           {
                                  v.push_back(v_temp[j]);
                           }
                    }
                    if (i == 3)
                    {
                           mc[0] = mc[0] + 1; //
                           mc.assign({mc[0]});

                           v_temp[0] = v[0]*(i-1);
                           v_temp.assign({v_temp[0]});
                           for(int j = 1 ; j < i-1 ; j = j+1)
                           {
                                  v_temp.push_back(v[j-1]*((2*i)-j) +
                                      v_temp[0]*mc[j-1] );
                           }
                           v_temp.push_back((v[i-2]*(i+1) ) + (v_temp[0]) );
                           // Assign the temporary vector to vector v for
                               future use
                           v[0] = v_temp[0];
                           v.assign({v[0]});
                           for(int j = 1 ; j < i ; j = j+1)
                           {
                                  v.push_back(v_temp[j]);
                           }
                    }
                    if (i == 4)
                    {
                           mc[0] = mc[0] + 1;
                           mc.assign({mc[0]});
                           mc.push_back(mc[0]);

                           v_temp[0] = v[0]*(i-1);
```

```
                        v_temp.assign({v_temp[0]});
                        for(int j = 1 ; j < i-1 ; j = j+1)
                        {
                                v_temp.push_back(v[j-1]*((2*i)-j) +
                                    v_temp[0]*mc[j-1] );
                        }
                        v_temp.push_back((v[i-2]*(i+1) ) + (v_temp[0]) );
                        // Assign the temporary vector to vector v for
                            future use
                        v[0] = v_temp[0];
                        v.assign({v[0]});
                        for(int j = 1 ; j < i ; j = j+1)
                        {
                                v.push_back(v_temp[j]);
                        }
                }
                if (i >= 5)
                {
                        mc_temp[0] = mc[0]+1;
                        mc_temp.assign({mc_temp[0]});
                        for(int ic = 1 ; ic < l ; ic = ic+1)
                        {
                                mc_temp[ic] = mc[ic-1] + mc[ic];
                        }
                        mc[l] = (mc_temp[0]);
                        mc[l+1] = (mc_temp[0]);

                        mc[0]=mc_temp[0];
                        mc.assign({mc[0]});

                        for(int ic = 1 ; ic < l ; ic = ic+1)
                        {
                                mc.push_back(mc_temp[ic]);
                        }
                        mc.push_back(mc_temp[0]);
                        mc.push_back(0);

                        v_temp[0] = v[0]*(i-1);
                        v_temp.assign({v_temp[0]});
                        for(int j = 1 ; j < i-1 ; j = j+1)
                        {
                                v_temp.push_back(v[j-1]*((2*i)-j) +
                                    v_temp[0]*mc[j-1] );
                        }
                        v_temp.push_back((v[i-2]*(i+1) ) + (v_temp[0]) );
                        // Assign the temporary vector to vector v for
                            future use
                        v[0] = v_temp[0];
```

```
                        v.assign({v[0]});
                        for(int j = 1 ; j < i ; j = j+1)
                        {
                               v.push_back(v_temp[j]);
                        }

                        l = l+1;
                  }

                  d0 = d0*k;
                  k = k+1;
            }
            for(int i = 1 ; i <= (bpower-1)/2 ; i = i+1)
            {
                  integral += sgn*v[i-1]*(sin(s)^(2*(i-1)));
                  sgn=-sgn;

            }

            for(int i = 1 ; i <= (bpower-1)/2 ; i = i+1)
            {
                  integral_front = sgn;
                  sgn = -sgn;
            }
            return integral_front*ln(sin(s)) + (integral)/(d0*(sin(s)^(
                bpower-1))) ;
      }
}
```

Code 22: *level 3 integral for cotangent odd case*

vi. $\int \tan^n x \, dx$

These are the formula of $\int \tan^n x \, dx$ with $n = 2$ to $n = 8$.

$$\int \tan^2 x \, dx = -x + \frac{\sin\ x}{\cos\ x}$$
$$\int \tan^3 x \, dx = \log(\cos\ x) + \frac{1}{2\cos^2\ x}$$
$$\int \tan^4 x \, dx = x - \frac{\sin\ x}{\cos\ x} + \frac{\sin^3\ x}{3\cos^3\ x}$$
$$\int \tan^5 x \, dx = -\log(\cos\ x) - \frac{4\cos^2\ x - 1}{4\cos^4\ x}$$
$$\int \tan^6 x \, dx = -x + \frac{\sin\ x}{\cos\ x} - \frac{\sin^3\ x}{3\cos^3\ x} + \frac{\sin^5\ x}{5\cos^5\ x}$$
$$\int \tan^7 x \, dx = \log(\cos\ x) + \frac{18\cos^4\ x - 9\cos^2\ x + 2}{12\cos^6\ x}$$
$$\int \tan^8 x \, dx = x - \frac{\sin\ x}{\cos\ x} + \frac{\sin^3\ x}{3\cos^3\ x} - \frac{\sin^5\ x}{5\cos^5\ x} + \frac{\sin^7\ x}{7\cos^7\ x}$$

The pattern for the numerator and denominator will be like the cotangent counterpart, thus we only need to exchange the sign, what is $-$ become $+$ and the other way around, and also exchanging sine into cosine and cosine into sine.

```
...

Symbolic Power::integrate(const Symbolic &s) const
{
	...
	if(a.type() == typeid(Cot))
	{
		...
	if(b.df(s) == 0 && b!=0 && bpower % 2 == 0) // even power case
	{
		Symbolic integral;
		Symbolic integral_front;
		Symbolic sgn = -1;
		for(int i = 2 ; i <= (bpower) ; i = i+2)
		{
			integral = (-1)*integral;
			integral += (sin(s)^(i-1)) / ((i-1)*(cos(s)^(i-1)));
		}

		for(int i = 1 ; i <= (bpower)/2 ; i = i+1)
		{
			integral_front = sgn;
			sgn = -sgn;
		}
		return integral_front*s + integral;
	}
```

```
        }
}
```

Code 23: *level 3 integral for tangent even case*

```
...

Symbolic Power::integrate(const Symbolic &s) const
{
        ...
        if(a.type() == typeid(Cot))
        {
        ...
        if(b.df(s) == 0 && b!=0 && bpower % 2 != 0) // odd power case
        {
                Symbolic integral;
                Symbolic integral_front;
                Symbolic sgn = 1;
                vector<int> v={1};
                vector<int> v_temp={1};
                vector<int> mc={1}; // to store the middle coefficient
                vector<int> mc_temp={1}; // to store the temporary / new middle
                     coefficient
                int d0 = 2;
                int k = 1, l=2;
                // For the coefficient at the numerator
                for(int i = 1 ; i <= (bpower-1)/2 ; i = i+1)
                {
                        if (i == 1)
                        {
                                v[0] = 1;
                                v.assign({v[0]});
                        }

                        if (i ==2)
                        {
                                mc[0]=1;
                                mc.assign({mc[0]});
                                v_temp[0] = v[0]*(i-1);
                                v_temp.assign({v_temp[0]});

                                for(int j = 1 ; j < i ; j = j+1)
                                {
                                        v_temp.push_back(v[j-1]*((2*i)-j) +
                                            v_temp[0]*mc[j-1] );
                                }
                                v[0] = v_temp[0];
                                v.assign({v[0]});
                                for(int j = 1 ; j < i ; j = j+1)
```

```
                        {
                                v.push_back(v_temp[j]);
                        }
                }
                if (i == 3)
                {
                        mc[0] = mc[0] + 1; //
                        mc.assign({mc[0]});

                        v_temp[0] = v[0]*(i-1);
                        v_temp.assign({v_temp[0]});
                        for(int j = 1 ; j < i-1 ; j = j+1)
                        {
                                v_temp.push_back(v[j-1]*((2*i)-j) +
                                    v_temp[0]*mc[j-1] );
                        }
                        v_temp.push_back((v[i-2]*(i+1) ) + (v_temp[0]) );
                        // Assign the temporary vector to vector v for
                            future use
                        v[0] = v_temp[0];
                        v.assign({v[0]});
                        for(int j = 1 ; j < i ; j = j+1)
                        {
                                v.push_back(v_temp[j]);
                        }
                }
                if (i == 4)
                {
                        mc[0] = mc[0] + 1;
                        mc.assign({mc[0]});
                        mc.push_back(mc[0]);

                        v_temp[0] = v[0]*(i-1);
                        v_temp.assign({v_temp[0]});
                        for(int j = 1 ; j < i-1 ; j = j+1)
                        {
                                v_temp.push_back(v[j-1]*((2*i)-j) +
                                    v_temp[0]*mc[j-1] );
                        }
                        v_temp.push_back((v[i-2]*(i+1) ) + (v_temp[0]) );
                        // Assign the temporary vector to vector v for
                            future use
                        v[0] = v_temp[0];
                        v.assign({v[0]});
                        for(int j = 1 ; j < i ; j = j+1)
                        {
                                v.push_back(v_temp[j]);
                        }
```

```
                }
                if (i >= 5)
                {
                        mc_temp[0] = mc[0]+1;
                        mc_temp.assign({mc_temp[0]});
                        for(int ic = 1 ; ic < l ; ic = ic+1)
                        {
                                mc_temp[ic] = mc[ic-1] + mc[ic];
                        }
                        mc[l] = (mc_temp[0]);
                        mc[l+1] = (mc_temp[0]);

                        mc[0]=mc_temp[0];
                        mc.assign({mc[0]});

                        for(int ic = 1 ; ic < l ; ic = ic+1)
                        {
                                mc.push_back(mc_temp[ic]);
                        }
                        mc.push_back(mc_temp[0]);
                        mc.push_back(0);

                        v_temp[0] = v[0]*(i-1);
                        v_temp.assign({v_temp[0]});
                        for(int j = 1 ; j < i-1 ; j = j+1)
                        {
                                v_temp.push_back(v[j-1]*((2*i)-j) +
                                    v_temp[0]*mc[j-1] );
                        }
                        v_temp.push_back((v[i-2]*(i+1) ) + (v_temp[0]) );
                        // Assign the temporary vector to vector v for
                            future use
                        v[0] = v_temp[0];
                        v.assign({v[0]});
                        for(int j = 1 ; j < i ; j = j+1)
                        {
                                v.push_back(v_temp[j]);
                        }

                        l = l+1;
                }

                d0 = d0*k;
                k = k+1;
        }
        for(int i = 1 ; i <= (bpower-1)/2 ; i = i+1)
        {
                integral += sgn*v[i-1]*(cos(s)^(2*(i-1)));
```

```
                    sgn=-sgn;

                }

                for(int i = 1 ; i <= (bpower-1)/2 ; i = i+1)
                {
                    integral_front = sgn;
                    sgn = -sgn;
                }
                return integral_front*ln(cos(s)) + (integral)/(d0*(cos(s)^(
                    bpower-1))) ;
        }
        }
}
```

Code 24: *level 3 integral for tangent odd case*

[SI*] **Trigonometry Level 4**

The trigonometry integrals that we consider as level 4 is

$$\int \sin^n x \ \cos^m \ x \, dx$$
$$\int \tan^n x \ \sec^m \ x \, dx$$
$$\int \cot^n x \ \csc^m \ x \, dx$$

the reduction formula to compute these trigonometry integral level 4 are

$$\int \sin^n x \ \cos^m \ x \, dx = \frac{\sin^{n+1} x \ \cos^{m-1} x}{m+n} + \frac{m-1}{m+n} \int \sin^n x \ \cos^{m-2} x \, dx \tag{2.16}$$

vii. $\int \sin^n x \ \cos^m \ x \, dx$

The differentiation to obtain the reduction formula is coming from integration by parts and a smart algebra technique

$$\begin{aligned}
I_{m,n} &= \int \sin^n x \ \cos^m \ x \, dx \\
&= \int \cos^{m-1} x \ \sin^n x \ \cos \ x dx \\
&= \cos^{m-1} x \frac{\sin^{n+1} x}{n+1} - \frac{m-1}{n+1} \int \sin^{n+2} x \ \cos^{m-2} x \, dx \\
&= \cos^{m-1} x \frac{\sin^{n+1} x}{n+1} - \frac{m-1}{n+1} \int \sin^n x \ \cos^{m-2} x (1 - \cos^2 x) \, dx \\
&= \frac{\sin^{n+1} x \ \cos^{m-1} x}{m+n} + \frac{m-1}{m+n} \int \sin^n x \ \cos^{m-2} x (1 - \cos^2 x) \, dx \\
&= \frac{\sin^{n+1} x \ \cos^{m-1} x}{m+n} + \frac{m-1}{m+n} \int \sin^n x \ \cos^{m-2} x \, dx
\end{aligned}$$

These are the formula of $\int \sin^n x \ \cos^m \ x \, dx$ with $n = 1$ and $m = 1$ to $m = 8$.

$$\int \sin^1 x \ \cos^1 \ x \, dx = \frac{\sin^2 \ x}{2}$$
$$\int \sin^1 x \ \cos^2 \ x \, dx = \frac{-\cos^3 \ x}{3}$$
$$\int \sin^1 x \ \cos^3 \ x \, dx = \frac{-\cos^4 \ x}{4}$$
$$\int \sin^1 x \ \cos^4 \ x \, dx = \frac{-\cos^5 \ x}{5}$$
$$\int \sin^1 x \ \cos^5 \ x \, dx = \frac{-\cos^6 \ x}{6}$$
$$\int \sin^1 x \ \cos^6 \ x \, dx = \frac{-\cos^7 \ x}{7}$$
$$\int \sin^1 x \ \cos^7 \ x \, dx = \frac{-\cos^8 \ x}{8}$$
$$\int \sin^1 x \ \cos^8 \ x \, dx = \frac{-\cos^9 \ x}{9}$$

These are the formula of $\int \sin^n x \ \cos^m \ x \, dx$ with $n = 2$ and $m = 1$ to $m = 8$.

$$\int \sin^2 x \ \cos^1 \ x \, dx = \frac{\sin^3 \ x}{3}$$
$$\int \sin^2 x \ \cos^2 \ x \, dx = \frac{x}{8} - \frac{\sin(2x)\cos(2x)}{16}$$
$$\int \sin^2 x \ \cos^3 \ x \, dx = -\frac{\sin^5 \ x}{5} + \frac{\sin^3 \ x}{3}$$
$$\int \sin^2 x \ \cos^4 \ x \, dx = \frac{x}{16} - \frac{\sin(x)\cos^5(x)}{6} + \frac{\sin(x)\cos^3(x)}{24} + \frac{\sin(x)\cos(x)}{16}$$
$$\int \sin^2 x \ \cos^5 \ x \, dx = \frac{\sin^7 \ x}{7} - \frac{2\sin^5 \ x}{5} + \frac{\sin^3 \ x}{3}$$
$$\int \sin^2 x \ \cos^6 \ x \, dx = \frac{5x}{128} - \frac{\sin(x)\cos^7(x)}{8} + \frac{\sin(x)\cos^5(x)}{48} + \frac{5\sin(x)\cos^3(x)}{192} + \frac{5\sin(x)\cos(x)}{128}$$
$$\int \sin^2 x \ \cos^7 \ x \, dx = -\frac{\sin^9 \ x}{9} + \frac{3\sin^7 \ x}{7} - \frac{3\sin^5 \ x}{5} + \frac{\sin^3 \ x}{3}$$
$$\int \sin^2 x \ \cos^8 \ x \, dx = \frac{7x}{256} - \frac{\sin(x)\cos^9(x)}{10} + \frac{\sin(x)\cos^7(x)}{80} + \frac{7\sin(x)\cos^5(x)}{480} + \frac{7\sin(x)\cos^3(x)}{384} + \frac{7\sin(x)\cos(x)}{256}$$

These are the formula of $\int \sin^n x \ \cos^m \ x \, dx$ with $n = 3$ and $m = 1$ to $m = 8$.

$$\int \sin^3 x \ \cos^1 \ x \, dx = \frac{\sin^4 \ x}{4}$$
$$\int \sin^3 x \ \cos^2 \ x \, dx = \frac{\cos^5 \ x}{5} - \frac{\cos^3 \ x}{3}$$
$$\int \sin^3 x \ \cos^3 \ x \, dx = -\frac{\sin^6 \ x}{6} + \frac{\sin^4 \ x}{4}$$
$$\int \sin^3 x \ \cos^4 \ x \, dx = \frac{\cos^7 \ x}{7} - \frac{\cos^5 \ x}{5}$$
$$\int \sin^3 x \ \cos^5 \ x \, dx = \frac{\cos^8 \ x}{8} - \frac{\cos^6 \ x}{6}$$
$$\int \sin^3 x \ \cos^6 \ x \, dx = \frac{\cos^9 \ x}{9} - \frac{\cos^7 \ x}{7}$$
$$\int \sin^3 x \ \cos^7 \ x \, dx = \frac{\cos^{10} \ x}{10} - \frac{\cos^8 \ x}{8}$$
$$\int \sin^3 x \ \cos^8 \ x \, dx = \frac{\cos^{11} \ x}{11} - \frac{\cos^9 \ x}{9}$$

These are the formula of $\int \sin^n x \ \cos^m \ x \, dx$ with $n = 4$ and $m = 1$ to $m = 8$.

$$\int \sin^4 x \ \cos^1 \ x \, dx = \frac{\sin^5 \ x}{5}$$
$$\int \sin^4 x \ \cos^2 \ x \, dx = \frac{x}{16} + \frac{\sin^5(x)\cos(x)}{6} - \frac{\sin^3(x)\cos(x)}{24} - \frac{\sin(x)\cos(x)}{16}$$
$$\int \sin^4 x \ \cos^3 \ x \, dx = -\frac{\sin^7 \ x}{7} + \frac{\sin^5 \ x}{5}$$
$$\int \sin^4 x \ \cos^4 \ x \, dx = \frac{3x}{128} - \frac{\sin^3(2x)\cos(2x)}{128} - \frac{3\sin(2x)\cos(2x)}{256}$$
$$\int \sin^4 x \ \cos^5 \ x \, dx = \frac{\sin^9 \ x}{9} - \frac{2\sin^7 \ x}{7} + \frac{\sin^5 \ x}{5}$$
$$\int \sin^4 x \ \cos^6 \ x \, dx = \frac{3x}{256} + \frac{\sin(x)\cos^9(x)}{10} - \frac{11\sin(x)\cos^7(x)}{80} + \frac{\sin(x)\cos^5(x)}{160} + \frac{\sin(x)\cos^3(x)}{128} + \frac{3\sin(x)\cos(x)}{256}$$
$$\int \sin^4 x \ \cos^7 \ x \, dx = -\frac{\sin^{11} \ x}{11} + \frac{\sin^9 \ x}{3} - \frac{3\sin^7 \ x}{7} + \frac{\sin^5 \ x}{5}$$
$$\int \sin^4 x \ \cos^8 \ x \, dx = \frac{7x}{1024} + \frac{\sin(x)\cos^{11}(x)}{12} - \frac{13\sin(x)\cos^9(x)}{120} + \frac{\sin(x)\cos^7(x)}{320} + \frac{7\sin(x)\cos^5(x)}{1920} + \frac{7\sin(x)\cos^3(x)}{1536} + \frac{7\sin(x)\cos(x)}{1024}$$

These are the formula of $\int \sin^n x \ \cos^m \ x \, dx$ with $n = 5$ and $m = 1$ to $m = 8$.

$$\int \sin^5 x \ \cos^1 \ x \, dx = \frac{\sin^6 \ x}{6}$$
$$\int \sin^5 x \ \cos^2 \ x \, dx = -\frac{\cos^7 \ x}{7} + \frac{2\cos^5 \ x}{5} - \frac{\cos^3 \ x}{3}$$
$$\int \sin^5 x \ \cos^3 \ x \, dx = -\frac{\sin^8 \ x}{8} + \frac{\sin^6 \ x}{6}$$
$$\int \sin^5 x \ \cos^4 \ x \, dx = -\frac{\cos^9 \ x}{9} + \frac{2\cos^7 \ x}{7} - \frac{\cos^5 \ x}{5}$$
$$\int \sin^5 x \ \cos^5 \ x \, dx = \frac{\sin^{10} \ x}{10} - \frac{\sin^8 \ x}{4} + \frac{\sin^6 \ x}{6}$$
$$\int \sin^5 x \ \cos^6 \ x \, dx = -\frac{\cos^{11} \ x}{11} + \frac{2\cos^9 \ x}{9} - \frac{\cos^7 \ x}{7}$$
$$\int \sin^5 x \ \cos^7 \ x \, dx = -\frac{\cos^{12} \ x}{12} + \frac{\cos^{10} \ x}{5} - \frac{\cos^8 \ x}{8}$$
$$\int \sin^5 x \ \cos^8 \ x \, dx = -\frac{\cos^{13} \ x}{13} + \frac{2\cos^{11} \ x}{11} - \frac{\cos^9 \ x}{9}$$

These are the formula of $\int \sin^n x \ \cos^m \ x \, dx$ with $n = 6$ and $m = 1$ to $m = 8$.

$$\int \sin^6 x \ \cos^1 \ x \, dx = \frac{\sin^7 \ x}{7}$$
$$\int \sin^6 x \ \cos^2 \ x \, dx = \frac{5x}{128} + \frac{\sin^7(x)\cos(x)}{8} - \frac{\sin^5(x)\cos(x)}{48} - \frac{5\sin^3(x)\cos(x)}{192} - \frac{5\sin(x)\cos(x)}{128}$$
$$\int \sin^6 x \ \cos^3 \ x \, dx = -\frac{\sin^9 \ x}{9} + \frac{\sin^7 \ x}{7}$$
$$\int \sin^6 x \ \cos^4 \ x \, dx = \frac{3x}{256} - \frac{\sin^9(x)\cos(x)}{10} + \frac{11\sin^7(x)\cos(x)}{80} - \frac{\sin^5(x)\cos(x)}{160} - \frac{\sin^3(x)\cos(x)}{128} - \frac{3\sin(x)\cos(x)}{256}$$
$$\int \sin^6 x \ \cos^5 \ x \, dx = \frac{\sin^{11} \ x}{11} - \frac{2\sin^9 \ x}{9} + \frac{\sin^7 \ x}{7}$$
$$\int \sin^6 x \ \cos^6 \ x \, dx = \frac{5x}{1024} - \frac{\sin^5(2x)\cos(2x)}{768} - \frac{5\sin^3(2x)\cos(2x)}{3072} - \frac{5\sin(2x)\cos(2x)}{2048}$$
$$\int \sin^6 x \ \cos^7 \ x \, dx = -\frac{\sin^{13} \ x}{13} + \frac{3\sin^{11} \ x}{11} - \frac{\sin^9 \ x}{3} + \frac{\sin^7 \ x}{7}$$
$$\int \sin^6 x \ \cos^8 \ x \, dx = \frac{5x}{2048} - \frac{\sin(x)\cos^{13}(x)}{14} + \frac{29\sin(x)\cos^{11}(x)}{168} - \frac{37\sin(x)\cos^9(x)}{336} + \frac{\sin(x)\cos^7(x)}{896} + \frac{\sin(x)\cos^5(x)}{768} + \frac{5\sin(x)\cos^3(x)}{3072} + \frac{5\sin(x)\cos(x)}{2048}$$

These are the formula of $\int \sin^n x \ \cos^m \ x \, dx$ with $n = 7$ and $m = 1$ to $m = 8$.

$$\int \sin^7 x \ \cos^1 \ x \, dx = \frac{\sin^8 \ x}{8}$$
$$\int \sin^7 x \ \cos^2 \ x \, dx = \frac{\cos^9 \ x}{9} - \frac{3\cos^7 \ x}{7} + \frac{3\cos^5 \ x}{5} - \frac{\cos^3 \ x}{3}$$
$$\int \sin^7 x \ \cos^3 \ x \, dx = -\frac{\sin^{10} \ x}{10} + \frac{\sin^8 \ x}{8}$$
$$\int \sin^7 x \ \cos^4 \ x \, dx = \frac{\cos^{11} \ x}{11} - \frac{\cos^9 \ x}{3} + \frac{3\cos^7 \ x}{7} - \frac{\cos^5 \ x}{5}$$
$$\int \sin^7 x \ \cos^5 \ x \, dx = \frac{\sin^{12} \ x}{12} - \frac{\sin^{10} \ x}{5} + \frac{\sin^8 \ x}{8}$$
$$\int \sin^7 x \ \cos^6 \ x \, dx = \frac{\cos^{13} \ x}{13} - \frac{3\cos^{11} \ x}{11} + \frac{\cos^9 \ x}{3} - \frac{\cos^7 \ x}{7}$$
$$\int \sin^7 x \ \cos^7 \ x \, dx = -\frac{\sin^{14} \ x}{14} + \frac{\sin^{12} \ x}{4} - \frac{3\sin^{10} \ x}{10} + \frac{\sin^8 \ x}{8}$$
$$\int \sin^7 x \ \cos^8 \ x \, dx = \frac{\cos^{15} \ x}{15} - \frac{3\cos^{13} \ x}{13} + \frac{3\cos^{11} \ x}{11} - \frac{\cos^9 \ x}{9}$$

These are the formula of $\int \sin^n x \ \cos^m \ x \, dx$ with $n = 8$ and $m = 1$ to $m = 8$.

$$\int \sin^8 x \ \cos^1 \ x \, dx = \frac{\sin^9 \ x}{9}$$
$$\int \sin^8 x \ \cos^2 \ x \, dx = \frac{7x}{256} + \frac{\sin^9(x)\cos(x)}{10} - \frac{\sin^7(x)\cos(x)}{80} - \frac{7\sin^5(x)\cos(x)}{480} - \frac{7\sin^3(x)\cos(x)}{384} - \frac{7\sin(x)\cos(x)}{256}$$
$$\int \sin^8 x \ \cos^3 \ x \, dx = -\frac{\sin^{11} \ x}{11} + \frac{\sin^9 \ x}{9}$$
$$\int \sin^8 x \ \cos^4 \ x \, dx = \frac{7x}{1024} - \frac{\sin^{11}(x)\cos(x)}{12} + \frac{13\sin^9(x)\cos(x)}{120} - \frac{\sin^7(x)\cos(x)}{320} - \frac{7\sin^5(x)\cos(x)}{1920} - \frac{7\sin^3(x)\cos(x)}{1536} - \frac{7\sin(x)\cos(x)}{1024}$$
$$\int \sin^8 x \ \cos^5 \ x \, dx = \frac{\sin^{13} \ x}{13} - \frac{2\sin^{11} \ x}{11} + \frac{\sin^9 \ x}{9}$$
$$\int \sin^8 x \ \cos^6 \ x \, dx = \frac{5x}{2048} + \frac{\sin^{13}(x)\cos(x)}{14} - \frac{29\sin^{11}(x)\cos(x)}{168} + \frac{37\sin^9(x)\cos(x)}{336} - \frac{\sin^7(x)\cos(x)}{896} - \frac{\sin^5(x)\cos(x)}{768} - \frac{5\sin^3(x)\cos(x)}{3072} - \frac{5\sin(x)\cos(x)}{2048}$$
$$\int \sin^8 x \ \cos^7 \ x \, dx = -\frac{\sin^{15} \ x}{15} + \frac{3\sin^{13} \ x}{13} - \frac{3\sin^{11} \ x}{11} + \frac{\sin^9 \ x}{9}$$
$$\int \sin^8 x \ \cos^8 \ x \, dx = \frac{35x}{32768} - \frac{\sin^7(2x)\cos(2x)}{4096} - \frac{7\sin^5(2x)\cos(2x)}{24576} - \frac{35\sin^3(2x)\cos(2x)}{98304} - \frac{35\sin(2x)\cos(2x)}{65536}$$

Case 1: For $\int \sin^n x \ \cos^m \ x \, dx$ with even power for both n and m

This case occurs when both m and n are even number. We divide this case into 3 smaller

cases:

1. When $n > m$
2. When $n < m$
3. When $n = m$

Case 1-a: For $\int \sin^n x \ \cos^m \ x \, dx$ with $n > m$

We will start with a big example, these are the formula of $\int \sin^n x \ \cos^m \ x \, dx$ with $n = 14$ and $m = 2$ to $m = 12$ with increment of 2.

$$\int \sin^{14} x \ \cos^2 \ x \, dx = \frac{429x}{32768} + \frac{\sin^{15}(x)\cos(x)}{16} - \frac{\sin^{13}(x)\cos(x)}{224} - \frac{13\sin^{11}(x)\cos(x)}{2688} - \frac{143\sin^9(x)\cos(x)}{26880} - \frac{429\sin^7(x)\cos(x)}{71680} - \frac{143\sin^5(x)\cos(x)}{20480} - \frac{143\sin^3(x)\cos(x)}{16384} - \frac{429\sin(x)\cos(x)}{32768}$$

$$\int \sin^{14} x \ \cos^4 \ x \, dx = \frac{143x}{65536} - \frac{\sin^{17}(x)\cos(x)}{18} + \frac{19\sin^{15}(x)\cos(x)}{288} - \frac{\sin^{13}(x)\cos(x)}{1344} - \frac{13\sin^{11}(x)\cos(x)}{16128} - \frac{143\sin^9(x)\cos(x)}{161280} - \frac{143\sin^7(x)\cos(x)}{143360} - \frac{143\sin^5(x)\cos(x)}{122880} - \frac{143\sin^3(x)\cos(x)}{98304} - \frac{143\sin(x)\cos(x)}{65536}$$

$$\int \sin^{14} x \ \cos^6 \ x \, dx = \frac{143x}{262144} + \frac{\sin^{19}(x)\cos(x)}{20} - \frac{41\sin^{17}(x)\cos(x)}{360} + \frac{383\sin^{15}(x)\cos(x)}{5760} - \frac{\sin^{13}(x)\cos(x)}{5376} - \frac{13\sin^{11}(x)\cos(x)}{64512} - \frac{143\sin^9(x)\cos(x)}{645120} - \frac{143\sin^7(x)\cos(x)}{573440} - \frac{143\sin^5(x)\cos(x)}{491520} - \frac{143\sin^3(x)\cos(x)}{393216} - \frac{143\sin(x)\cos(x)}{262144}$$

$$\int \sin^{14} x \ \cos^8 \ x \, dx = \frac{91x}{524288} - \frac{\sin^{21}(x)\cos(x)}{22} + \frac{67\sin^{19}(x)\cos(x)}{440} - \frac{1367\sin^{17}(x)\cos(x)}{7920} + \frac{8441\sin^{15}(x)\cos(x)}{126720} - \frac{\sin^{13}(x)\cos(x)}{16896} - \frac{13\sin^{11}(x)\cos(x)}{202752} - \frac{13\sin^9(x)\cos(x)}{184320} - \frac{13\sin^7(x)\cos(x)}{163840} - \frac{91\sin^5(x)\cos(x)}{983040} - \frac{91\sin^3(x)\cos(x)}{786432} - \frac{91\sin(x)\cos(x)}{524288}$$

$$\int \sin^{14} x \ \cos^{10} \ x \, dx = \frac{273x}{4194304} + \frac{\sin^{23}(x)\cos(x)}{24} - \frac{97\sin^{21}(x)\cos(x)}{528} + \frac{1081\sin^{19}(x)\cos(x)}{3520} - \frac{1629\sin^{17}(x)\cos(x)}{7040} + \frac{7507\sin^{15}(x)\cos(x)}{112640} - \frac{\sin^{13}(x)\cos(x)}{45056} - \frac{13\sin^{11}(x)\cos(x)}{540672} - \frac{13\sin^9(x)\cos(x)}{491520} - \frac{39\sin^7(x)\cos(x)}{1310720} - \frac{91\sin^5(x)\cos(x)}{2621440} - \frac{91\sin^3(x)\cos(x)}{2097152} - \frac{273\sin(x)\cos(x)}{4194304}$$

$$\begin{aligned}\int \sin^{14} x\ \cos^{12}\ x\,dx = &\frac{231x}{8388608} - \frac{\sin^{25}(x)\cos(x)}{26} + \frac{131\sin^{23}(x)\cos(x)}{624} - \frac{577\sin^{21}(x)\cos(x)}{1248}\\ &+ \frac{4281\sin^{19}(x)\cos(x)}{8320} - \frac{4829\sin^{17}(x)\cos(x)}{16640} + \frac{17747\sin^{15}(x)\cos(x)}{266240}\\ &- \frac{\sin^{13}(x)\cos(x)}{106496} - \frac{\sin^{11}(x)\cos(x)}{98304} - \frac{11\sin^{9}(x)\cos(x)}{983040}\\ &- \frac{33\sin^{7}(x)\cos(x)}{2621440} - \frac{77\sin^{5}(x)\cos(x)}{5242880} - \frac{77\sin^{3}(x)\cos(x)}{4194304}\\ &- \frac{231\sin(x)\cos(x)}{8388608}\end{aligned}$$

The patterns that can be seen from those equations above are

- The integral $\int \sin^{14} x\,dx$ represents some term for $\int \sin^{14} x\ \cos^{2}\ x\,dx$ with denominator that is multiplied by $m+n = 14+2 = 16$.
- The higher the power of m the longer the new term that is having power of $\sin^{n+1}$ and above.
- As the iteration increases the denominator coefficient is the multiplication of $m+n$ with m starting from 2 to 12 with increment of 2.

This kind of integral is quite challenging, if we want to compute this integral with $n = 14$ and $m = 12$ we will start from $m = 0$, then $m = 2$. So let see when $m = 0$, we have computed and know the pattern for this kind of integral:

$$\begin{aligned}\int \sin^{14} x\ \cos^{0}\ x\,dx = &\frac{429x}{2048} - \frac{\sin^{13}(x)\cos(x)}{14} - \frac{13\sin^{11}(x)\cos(x)}{168}\\ &- \frac{143\sin^{9}(x)\cos(x)}{1680} - \frac{429\sin^{7}(x)\cos(x)}{4480} - \frac{143\sin^{5}(x)\cos(x)}{1280}\\ &- \frac{143\sin^{3}(x)\cos(x)}{1024} - \frac{429\sin(x)\cos(x)}{2048}\end{aligned}$$

now when $m = 2$, we will obtain this:

$$\begin{aligned}\int \sin^{14} x\ \cos^{2}\ x\,dx = &\frac{429x}{32768} + \frac{\sin^{15}(x)\cos(x)}{16} - \frac{\sin^{13}(x)\cos(x)}{224} - \frac{13\sin^{11}(x)\cos(x)}{2688}\\ &- \frac{143\sin^{9}(x)\cos(x)}{26880} - \frac{429\sin^{7}(x)\cos(x)}{71680} - \frac{143\sin^{5}(x)\cos(x)}{20480}\\ &- \frac{143\sin^{3}(x)\cos(x)}{16384} - \frac{429\sin(x)\cos(x)}{32768}\end{aligned}$$

From the reduction formula we know that this term:

$$\frac{c_{n/2}x}{d_{n/2}} + \sum_{i=1}^{n/2} -\frac{c_i \sin^{2i-1} x\ \cos\ x}{d_i}$$

will be multiplied by $\frac{m-1}{m+n}$ at every iteration from $m = 2$ till $m = 12$. So the problem is solved for this term.

For example, to obtain the denominator for function of x of 32768 from multiplying the denominator of function x from $\int \sin^{14} x\,dx$ with $d_0 = m+n = 16$, it is $2048 * 16 = 32768$.

The remaining term will have different pattern for the numerator, from now on we will omit the term of $\frac{c_{n/2}x}{d_{n/2}} + \sum_{i=1}^{n/2} - \frac{c_i \sin^{2i-1} x \ \cos \ x}{d_i}$, now take a look again:

$$\int \sin^{14} x \ \cos^2 \ x \, dx = \frac{\sin^{15}(x)\cos(x)}{16} + \cdots - \ldots$$

$$\int \sin^{14} x \ \cos^4 \ x \, dx = -\frac{\sin^{17}(x)\cos(x)}{18} + \frac{19\sin^{15}(x)\cos(x)}{288} + \cdots - \ldots$$

$$\int \sin^{14} x \ \cos^6 \ x \, dx = \frac{\sin^{19}(x)\cos(x)}{20} - \frac{41\sin^{17}(x)\cos(x)}{360} + \frac{383\sin^{15}(x)\cos(x)}{5760} + \cdots - \ldots$$

$$\int \sin^{14} x \ \cos^8 \ x \, dx = -\frac{\sin^{21}(x)\cos(x)}{22} + \frac{67\sin^{19}(x)\cos(x)}{440} - \frac{1367\sin^{17}(x)\cos(x)}{7920} + \frac{8441\sin^{15}(x)\cos(x)}{126720} + \cdots - \ldots$$

$$\int \sin^{14} x \ \cos^{10} \ x \, dx = \frac{\sin^{23}(x)\cos(x)}{24} - \frac{97\sin^{21}(x)\cos(x)}{528} + \frac{1081\sin^{19}(x)\cos(x)}{3520} - \frac{1629\sin^{17}(x)\cos(x)}{7040} + \frac{7507\sin^{15}(x)\cos(x)}{112640} + \cdots - \ldots$$

$$\int \sin^{14} x \ \cos^{12} \ x \, dx = -\frac{\sin^{25}(x)\cos(x)}{26} + \frac{131\sin^{23}(x)\cos(x)}{624} - \frac{577\sin^{21}(x)\cos(x)}{1248} + \frac{4281\sin^{19}(x)\cos(x)}{8320} - \frac{4829\sin^{17}(x)\cos(x)}{16640} + \frac{17747\sin^{15}(x)\cos(x)}{266240} + \cdots - \ldots$$

We will learn how to decode this pattern, first we will make the denominator into its' raw number, so the numerator will change into its' raw number too, here it goes

$$\int \sin^{14} x \ \cos^2 \ x \, dx = \frac{\sin^{15}(x)\cos(x)}{16} + \cdots - \ldots$$

$$\int \sin^{14} x \ \cos^4 \ x \, dx = -\frac{\sin^{17}(x)\cos(x)}{18} + \frac{19\sin^{15}(x)\cos(x)}{288} + \cdots - \ldots$$

$$\int \sin^{14} x \ \cos^6 \ x \, dx = \frac{\sin^{19}(x)\cos(x)}{20} - \frac{41\sin^{17}(x)\cos(x)}{360} + \frac{383\sin^{15}(x)\cos(x)}{5760} + \cdots - \ldots$$

$$\int \sin^{14} x \ \cos^8 \ x \, dx = -\frac{\sin^{21}(x)\cos(x)}{22} + \frac{67\sin^{19}(x)\cos(x)}{440} - \frac{1367\sin^{17}(x)\cos(x)}{7920} + \frac{8441\sin^{15}(x)\cos(x)}{126720} + \cdots - \ldots$$

$$\int \sin^{14} x \ \cos^{10} \ x \, dx = \frac{\sin^{23}(x)\cos(x)}{24} - \frac{97\sin^{21}(x)\cos(x)}{528} + \frac{3243\sin^{19}(x)\cos(x)}{10560} - \frac{43983\sin^{17}(x)\cos(x)}{190080} + \frac{202689\sin^{15}(x)\cos(x)}{3041280} + \cdots - \ldots$$

$$\int \sin^{14} x \ \cos^{12} \ x \, dx = -\frac{\sin^{25}(x)\cos(x)}{26} + \frac{131\sin^{23}(x)\cos(x)}{624} - \frac{6347\sin^{21}(x)\cos(x)}{13728} + \frac{141273\sin^{19}(x)\cos(x)}{274560} - \frac{1434213\sin^{17}(x)\cos(x)}{4942080} + \frac{5270859\sin^{15}(x)\cos(x)}{79073280} + \cdots - \ldots$$

Then for a better look we will write it into Pascal' triangle form, for the numerator in the raw version

$$\begin{array}{ccccccccccc} & & & & & 1 & & & & & \\ & & & & 1 & & 19 & & & & \\ & & & 1 & & 41 & & 383 & & & \\ & & 1 & & 67 & & 1367 & & 8441 & & \\ & 1 & & 97 & & 3243 & & 43983 & & 202689 & \\ 1 & & 131 & & 6347 & & 141273 & & 1434213 & & 5270859 \end{array}$$

The C++ code will use an increasing **for** loop with $i = 1, i <= \frac{m}{2}, i = i + 1$.

For the computation, we are creating a variable for the denominator for function $\sin^{m+n-1} x \cos \ x$ as d_0, with $d_0 = m + n$. This variable will plays an important role to determine the value for the numerator of this kind of integral.

We will also use vector **v** to store the coefficient with

$$\begin{aligned} \mathbf{v}[0] &= 1 \\ \mathbf{v}[1]_i &= (d_0 * (i-1)) + 1, \quad i = 2, 3, \ldots, \frac{m}{2} - 1 \\ \mathbf{v}\left[\frac{m}{2} - 1\right]_i &= (d_0 * \mathbf{v}\left[\frac{m}{2} - 2\right]_{i-1}) + \mathbf{factorialoddup}(i-1) \end{aligned}$$

$\mathbf{v}\left[\frac{m}{2} - 1\right]_i$ is the last entry for the vector at iteration i, while $\mathbf{v}\left[\frac{m}{2} - 2\right]_{i-1}$ is the last entry for the vector at the previous iteration $i - 1$.

We make a definition here of $\mathbf{factorialoddup}(i-1)$:

$$\mathbf{factorialoddup}(i-1) = 1 * 3 * 5 * \ldots$$

it is a multiplication of odd number from 1 upward till we get $i - 1$ terms.

$$\begin{aligned} \mathbf{factorialoddup}(2) &= 1 * 3 \\ \mathbf{factorialoddup}(3) &= 1 * 3 * 5 \\ \mathbf{factorialoddup}(4) &= 1 * 3 * 5 * 7 \end{aligned}$$

For $i = 4$ we will make a **for** loop again to find the entry of the vector $\mathbf{v}[j]_i$ for $j = 2, 3, \ldots, \frac{m}{2} - 2$.

$$\begin{aligned} \mathbf{v}[j]_i &= (d_0 * \mathbf{v}[j-1]_{i-1}) + (k * \mathbf{v}[j-1]_{i-1} + \mathbf{v}[j]_{i-1}) \\ k &= k - 2 \end{aligned}$$

So the **for** loop will be like this:

$$
\begin{aligned}
i &= 1 \\
d_0 &= 16 \\
v[0] &= 1 \\
&* * * * * * * * * * * \\
i &= 2 \\
d_0 &= 18 \\
v[0] &= 1 \\
v[1] &= (d_0 * (2-1)) + 1 = 19 \\
&* * * * * * * * * * * \\
i &= 3 \\
d_0 &= 20 \\
v[0] &= 1 \\
v[1] &= (d_0 * (3-1)) + 1 = 41 \\
v[2] &= (d_0 * v[1]_{i=2}) + (1 * 3) = 383 \\
&* * * * * * * * * * * \\
i &= 4 \\
d_0 &= 22 \\
k &= 2 \\
v[0] &= 1 \\
v[1] &= (d_0 * (4-1)) + 1 = 67 \\
v[2] &= (d_0 * \mathbf{v}[1]_{i=3}) + (k * \mathbf{v}[1]_{i=3} + \mathbf{v}[2]_{i=3}) = 1367 \\
v[3] &= (d_0 * v[2]_{i=3}) + (1 * 3 * 5) = 8441
\end{aligned}
$$

Case 1-b: For $\int \sin^n x \ \cos^m \ x \, dx$ **with** $n < m$

Let's take a look at several integral related to this case

$$\int \sin^2 x \ \cos^4 \ x \, dx = \frac{x}{16} - \frac{\sin(x)\cos^5(x)}{6} + \frac{\sin(x)\cos^3(x)}{24} + \frac{\sin(x)\cos(x)}{16}$$

$$\int \sin^2 x \ \cos^6 \ x \, dx = \frac{5x}{128} - \frac{\sin(x)\cos^7(x)}{8} + \frac{\sin(x)\cos^5(x)}{48} + \frac{5\sin(x)\cos^3(x)}{192} + \frac{5\sin(x)\cos(x)}{128}$$

$$\int \sin^4 x \ \cos^6 \ x \, dx = \frac{3x}{256} + \frac{\sin(x)\cos^9(x)}{10} - \frac{11\sin(x)\cos^7(x)}{80} + \frac{\sin(x)\cos^5(x)}{160} + \frac{\sin(x)\cos^3(x)}{128} + \frac{3\sin(x)\cos(x)}{256}$$

$$\int \sin^2 x \ \cos^8 \ x \, dx = \frac{7x}{256} - \frac{\sin(x)\cos^9(x)}{10} + \frac{\sin(x)\cos^7(x)}{80} + \frac{7\sin(x)\cos^5(x)}{480} + \frac{7\sin(x)\cos^3(x)}{384} + \frac{7\sin(x)\cos(x)}{256}$$

$$\int \sin^4 x \ \cos^8 \ x \, dx = \frac{7x}{1024} + \frac{\sin(x)\cos^{11}(x)}{12} - \frac{13\sin(x)\cos^9(x)}{120} + \frac{\sin(x)\cos^7(x)}{320} + \frac{7\sin(x)\cos^5(x)}{1920} + \frac{7\sin(x)\cos^3(x)}{1536} + \frac{7\sin(x)\cos(x)}{1024}$$

$$\int \sin^6 x \ \cos^8 \ x \, dx = \frac{5x}{2048} - \frac{\sin(x)\cos^{13}(x)}{14} + \frac{29\sin(x)\cos^{11}(x)}{168} - \frac{37\sin(x)\cos^9(x)}{336} + \frac{\sin(x)\cos^7(x)}{896} + \frac{\sin(x)\cos^5(x)}{768} + \frac{5\sin(x)\cos^3(x)}{3072} + \frac{5\sin(x)\cos(x)}{2048}$$

This case does not need to be scrutinize further, since the numerator and denominator for all terms are exactly the same as those for the case 1-a where $n > m$, we only need to change the sine function into cosine and cosine function into sine in the result, since this case has the cosine function in the term that has the power. The plus and minus will also be interchange for the term with sine and cosine function, the term with x stays positive always.

Case 1-c: For $\int \sin^n x \ \cos^m \ x \, dx$ with $n = m$

These are the formula of $\int \sin^n x \ \cos^m \ x \, dx$ with $n = 1$ to $n = 8$.

$$\int \sin^1 x \ \cos^1 \ x \, dx = \frac{\sin^2 \ x}{2}$$

$$\int \sin^2 x \ \cos^2 \ x \, dx = \frac{x}{8} - \frac{\sin(2x)\cos(2x)}{16}$$

$$\int \sin^3 x \ \cos^3 \ x \, dx = -\frac{\sin^6 \ x}{6} + \frac{\sin^4 \ x}{4}$$

$$\int \sin^4 x \ \cos^4 \ x \, dx = \frac{3x}{128} - \frac{\sin^3(2x)\cos(2x)}{128} - \frac{3\sin(2x)\cos(2x)}{256}$$

$$\int \sin^5 x \ \cos^5 \ x \, dx = \frac{\sin^{10} \ x}{10} - \frac{\sin^8 \ x}{4} + \frac{\sin^6 \ x}{6}$$

$$\int \sin^6 x \ \cos^6 \ x \, dx = \frac{5x}{1024} - \frac{\sin^5(2x)\cos(2x)}{768} - \frac{5\sin^3(2x)\cos(2x)}{3072} - \frac{5\sin(2x)\cos(2x)}{2048}$$

$$\int \sin^7 x \ \cos^7 \ x \, dx = -\frac{\sin^{14} \ x}{14} + \frac{\sin^{12} \ x}{4} - \frac{3\sin^{10} \ x}{10} + \frac{\sin^8 \ x}{8}$$

$$\int \sin^8 x \ \cos^8 \ x \, dx = \frac{35x}{32768} - \frac{\sin^7(2x)\cos(2x)}{4096} - \frac{7\sin^5(2x)\cos(2x)}{24576} - \frac{35\sin^3(2x)\cos(2x)}{98304} - \frac{35\sin(2x)\cos(2x)}{65536}$$

We can tell that the case here will be divided into two, which are even case and odd case.

If n is even

$$\int \sin^2 x \ \cos^2 \ x \, dx = \frac{x}{8} - \frac{\sin(2x)\cos(2x)}{16}$$

$$\int \sin^4 x \ \cos^4 \ x \, dx = \frac{3x}{128} - \frac{\sin^3(2x)\cos(2x)}{128} - \frac{3\sin(2x)\cos(2x)}{256}$$

$$\int \sin^6 x \ \cos^6 \ x \, dx = \frac{5x}{1024} - \frac{\sin^5(2x)\cos(2x)}{768} - \frac{5\sin^3(2x)\cos(2x)}{3072} - \frac{5\sin(2x)\cos(2x)}{2048}$$

$$\int \sin^8 x \ \cos^8 \ x \, dx = \frac{35x}{32768} - \frac{\sin^7(2x)\cos(2x)}{4096} - \frac{7\sin^5(2x)\cos(2x)}{24576} - \frac{35\sin^3(2x)\cos(2x)}{98304} - \frac{35\sin(2x)\cos(2x)}{65536}$$

The patterns that can be seen from those equations above are

- The denominator with function of $\sin(2x)^{n-1}\cos(2x)$ can be determined with:

  $$d_0 = 2^n * (2n)$$

  This denominator will be used for the first integral result which is

  $$\frac{\sin^{n-1}(2x)\cos(2x)}{d_0}$$

  the next iteration will have the pattern like this:

  $$\frac{(n-1)\sin^{n-3}(2x)\cos(2x)}{d_0 * (n-2)}$$

The next iteration will have pattern like this:

$$\frac{(n-1)(n-3)\sin^{n-5}(2x)\cos(2x)}{d_0 * (n-2) * (n-4)}$$

Till we obtain the last iteration integral result with function $\sin(2x)\cos(2x)$ at the numerator we will stop. The easy thing here is the signs are all minus.

- The function x will have numerator and denominator of:

$$\frac{2 * (n-1) * (n-3) * \cdots * 1}{d_0 * (n-2) * (n-4) * \cdots * 1} * x$$

If n is odd

$$\begin{aligned}
\int \sin^1 x \ \cos^1 \ x \, dx &= \frac{\sin^2 \ x}{2} \\
\int \sin^3 x \ \cos^3 \ x \, dx &= -\frac{\sin^6 \ x}{6} + \frac{\sin^4 \ x}{4} \\
\int \sin^5 x \ \cos^5 \ x \, dx &= \frac{\sin^{10} \ x}{10} - \frac{\sin^8 \ x}{4} + \frac{\sin^6 \ x}{6} \\
\int \sin^7 x \ \cos^7 \ x \, dx &= -\frac{\sin^{14} \ x}{14} + \frac{\sin^{12} \ x}{4} - \frac{3\sin^{10} \ x}{10} + \frac{\sin^8 \ x}{8}
\end{aligned}$$

The patterns that can be seen from those equations above are

- The integral will have $\frac{n+1}{2}$ term/s, and have function of $\sin^{n+1} x$ at the numerator, at the next iteration we will have the function of $\sin^{(n+1)+2} x$ at the numerator with alternating sign, till we reach the $\sin^{2n} x$ at the numerator.
- The power determine the Pascal' triangle level / combinations formula that will be used to determine the numerator coefficients.
  For $n = 1$ we will use the first level of Pascal' triangle which is ${}_0C_0$.
  For $n = 3$ we will use the second level of Pascal' triangle which is ${}_1C_0$ and ${}_1C_1$.
  For $n = 5$ we will use the third level of Pascal' triangle which is ${}_2C_0, {}_2C_1$ and ${}_2C_2$.
  For $n = 7$ we will use the fourth level of Pascal' triangle which is ${}_3C_0, {}_3C_1, {}_3C_2$ and ${}_3C_3$.
- The denominator coefficient can be easily determined, it is the same as the power of the function in the numerator

Case 2: For $\int \sin^n x \ \cos^m \ x \, dx$ with at least one odd power in either n or m
This case occurs if either n or m is an odd number. We will recall the integral for $n = 1$,

$n = 2$, $n = 3$, $n = 5$,and $n = 6$

$$\int \sin^1 x \ \cos^2 \ x \, dx = \frac{-\cos^3 \ x}{3}$$
$$\int \sin^1 x \ \cos^3 \ x \, dx = \frac{-\cos^4 \ x}{4}$$
$$\int \sin^1 x \ \cos^4 \ x \, dx = \frac{-\cos^5 \ x}{5}$$
$$\int \sin^1 x \ \cos^5 \ x \, dx = \frac{-\cos^6 \ x}{6}$$
$$\int \sin^1 x \ \cos^6 \ x \, dx = \frac{-\cos^7 \ x}{7}$$
$$\int \sin^1 x \ \cos^7 \ x \, dx = \frac{-\cos^8 \ x}{8}$$
$$\int \sin^1 x \ \cos^8 \ x \, dx = \frac{-\cos^9 \ x}{9}$$

$$\int \sin^2 x \ \cos^3 \ x \, dx = -\frac{\sin^5 \ x}{5} + \frac{\sin^3 \ x}{3}$$
$$\int \sin^2 x \ \cos^5 \ x \, dx = \frac{\sin^7 \ x}{7} - \frac{2\sin^5 \ x}{5} + \frac{\sin^3 \ x}{3}$$
$$\int \sin^2 x \ \cos^7 \ x \, dx = -\frac{\sin^9 \ x}{9} + \frac{3\sin^7 \ x}{7} - \frac{3\sin^5 \ x}{5} + \frac{\sin^3 \ x}{3}$$

$$\int \sin^3 x \ \cos^1 \ x \, dx = \frac{\sin^4 \ x}{4}$$
$$\int \sin^3 x \ \cos^2 \ x \, dx = \frac{\cos^5 \ x}{5} - \frac{\cos^3 \ x}{3}$$
$$\int \sin^3 x \ \cos^4 \ x \, dx = \frac{\cos^7 \ x}{7} - \frac{\cos^5 \ x}{5}$$
$$\int \sin^3 x \ \cos^5 \ x \, dx = \frac{\cos^8 \ x}{8} - \frac{\cos^6 \ x}{6}$$
$$\int \sin^3 x \ \cos^6 \ x \, dx = \frac{\cos^9 \ x}{9} - \frac{\cos^7 \ x}{7}$$
$$\int \sin^3 x \ \cos^7 \ x \, dx = \frac{\cos^{10} \ x}{10} - \frac{\cos^8 \ x}{8}$$
$$\int \sin^3 x \ \cos^8 \ x \, dx = \frac{\cos^{11} \ x}{11} - \frac{\cos^9 \ x}{9}$$

$$\int \sin^5 x \ \cos^1 \ x \, dx = \frac{\sin^6 \ x}{6}$$
$$\int \sin^5 x \ \cos^2 \ x \, dx = -\frac{\cos^7 \ x}{7} + \frac{2\cos^5 \ x}{5} - \frac{\cos^3 \ x}{3}$$
$$\int \sin^5 x \ \cos^3 \ x \, dx = -\frac{\sin^8 \ x}{8} + \frac{\sin^6 \ x}{6}$$
$$\int \sin^5 x \ \cos^4 \ x \, dx = -\frac{\cos^9 \ x}{9} + \frac{2\cos^7 \ x}{7} - \frac{\cos^5 \ x}{5}$$
$$\int \sin^5 x \ \cos^6 \ x \, dx = -\frac{\cos^{11} \ x}{11} + \frac{2\cos^9 \ x}{9} - \frac{\cos^7 \ x}{7}$$
$$\int \sin^5 x \ \cos^7 \ x \, dx = -\frac{\cos^{12} \ x}{12} + \frac{\cos^{10} \ x}{5} - \frac{\cos^8 \ x}{8}$$
$$\int \sin^5 x \ \cos^8 \ x \, dx = -\frac{\cos^{13} \ x}{13} + \frac{2\cos^{11} \ x}{11} - \frac{\cos^9 \ x}{9}$$

$$\int \sin^6 x \ \cos^1 \ x \, dx = \frac{\sin^7 \ x}{7}$$
$$\int \sin^6 x \ \cos^3 \ x \, dx = -\frac{\sin^9 \ x}{9} + \frac{\sin^7 \ x}{7}$$
$$\int \sin^6 x \ \cos^5 \ x \, dx = \frac{\sin^{11} \ x}{11} - \frac{2\sin^9 \ x}{9} + \frac{\sin^7 \ x}{7}$$
$$\int \sin^6 x \ \cos^7 \ x \, dx = -\frac{\sin^{13} \ x}{13} + \frac{3\sin^{11} \ x}{11} - \frac{\sin^9 \ x}{3} + \frac{\sin^7 \ x}{7}$$

There will be two big cases here which are:

1. When $m > n$

2. When $m \leq n$

If $m > n$

$$\int \sin^1 x \ \cos^2 \ x \, dx = \frac{-\cos^3 \ x}{3}$$
$$\int \sin^1 x \ \cos^3 \ x \, dx = \frac{-\cos^4 \ x}{4}$$
$$\int \sin^1 x \ \cos^4 \ x \, dx = \frac{-\cos^5 \ x}{5}$$
$$\int \sin^1 x \ \cos^5 \ x \, dx = \frac{-\cos^6 \ x}{6}$$
$$\int \sin^1 x \ \cos^6 \ x \, dx = \frac{-\cos^7 \ x}{7}$$
$$\int \sin^1 x \ \cos^7 \ x \, dx = \frac{-\cos^8 \ x}{8}$$
$$\int \sin^1 x \ \cos^8 \ x \, dx = \frac{-\cos^9 \ x}{9}$$
$$\int \sin^2 x \ \cos^3 \ x \, dx = -\frac{\sin^5 \ x}{5} + \frac{\sin^3 \ x}{3}$$
$$\int \sin^2 x \ \cos^5 \ x \, dx = \frac{\sin^7 \ x}{7} - \frac{2\sin^5 \ x}{5} + \frac{\sin^3 \ x}{3}$$
$$\int \sin^2 x \ \cos^7 \ x \, dx = -\frac{\sin^9 \ x}{9} + \frac{3\sin^7 \ x}{7} - \frac{3\sin^5 \ x}{5} + \frac{\sin^3 \ x}{3}$$
$$\int \sin^3 x \ \cos^4 \ x \, dx = \frac{\cos^7 \ x}{7} - \frac{\cos^5 \ x}{5}$$
$$\int \sin^3 x \ \cos^5 \ x \, dx = \frac{\cos^8 \ x}{8} - \frac{\cos^6 \ x}{6}$$
$$\int \sin^5 x \ \cos^6 \ x \, dx = -\frac{\cos^{11} \ x}{11} + \frac{2\cos^9 \ x}{9} - \frac{\cos^7 \ x}{7}$$
$$\int \sin^5 x \ \cos^7 \ x \, dx = -\frac{\cos^{12} \ x}{12} + \frac{\cos^{10} \ x}{5} - \frac{\cos^8 \ x}{8}$$
$$\int \sin^5 x \ \cos^8 \ x \, dx = -\frac{\cos^{13} \ x}{13} + \frac{2\cos^{11} \ x}{11} - \frac{\cos^9 \ x}{9}$$
$$\int \sin^6 x \ \cos^7 \ x \, dx = -\frac{\sin^{13} \ x}{13} + \frac{3\sin^{11} \ x}{11} - \frac{\sin^9 \ x}{3} + \frac{\sin^7 \ x}{7}$$

The patterns that can be seen from those equations above are

- The integral will have the result with function of cosine, if n is an odd number.

  The integral will have the result of sine, if n is an even number.
- If n is an even number, then the pattern will be like this for the integral:

$$\int \sin^n x \ \cos^m \ x \, dx = \text{sgn}_0 * {}_{\frac{m-1}{2}}C_0 \frac{\sin^{n+1} \ x}{n+1} + \text{sgn}_1 * {}_{\frac{m-1}{2}}C_1 \frac{\sin^{n+3} \ x}{n+3} + \ldots$$
$$+ \text{sgn}_{\frac{m-1}{2}} * {}_{\frac{m-1}{2}}C_{\frac{m-1}{2}} \frac{\sin^{n+m} \ x}{n+m}$$

We start with $\text{sgn}_0 = 1$ and it is alternating at every iteration.

- If n is an odd number, then the pattern will be like this for the integral:

$$\int \sin^n x \ \cos^m \ x \, dx = \text{sgn}_0 * {}_{\frac{n-1}{2}}C_0 \frac{\cos^{m+1} \ x}{m+1} + \text{sgn}_1 * {}_{\frac{n-1}{2}}C_1 \frac{\cos^{m+3} \ x}{m+3} + \ldots$$
$$\text{sgn}_{\frac{n-1}{2}} * + {}_{\frac{n-1}{2}}C_{\frac{n-1}{2}} \frac{\cos^{n+m} \ x}{n+m}$$

We start with $\text{sgn}_0 = -1$ and it is alternating at every iteration.

If $m \leq n$

$$\int \sin^3 x \ \cos^1 \ x \, dx = \frac{\sin^4 \ x}{4}$$
$$\int \sin^3 x \ \cos^2 \ x \, dx = \frac{\cos^5 \ x}{5} - \frac{\cos^3 \ x}{3}$$
$$\int \sin^5 x \ \cos^1 \ x \, dx = \frac{\sin^6 \ x}{6}$$
$$\int \sin^5 x \ \cos^2 \ x \, dx = -\frac{\cos^7 \ x}{7} + \frac{2\cos^5 \ x}{5} - \frac{\cos^3 \ x}{3}$$
$$\int \sin^5 x \ \cos^3 \ x \, dx = -\frac{\sin^8 \ x}{8} + \frac{\sin^6 \ x}{6}$$
$$\int \sin^5 x \ \cos^4 \ x \, dx = -\frac{\cos^9 \ x}{9} + \frac{2\cos^7 \ x}{7} - \frac{\cos^5 \ x}{5}$$
$$\int \sin^6 x \ \cos^1 \ x \, dx = \frac{\sin^7 \ x}{7}$$
$$\int \sin^6 x \ \cos^3 \ x \, dx = -\frac{\sin^9 \ x}{9} + \frac{\sin^7 \ x}{7}$$
$$\int \sin^6 x \ \cos^5 \ x \, dx = \frac{\sin^{11} \ x}{11} - \frac{2\sin^9 \ x}{9} + \frac{\sin^7 \ x}{7}$$

The patterns that can be seen from those equations above are

- If m is an even number, then the integral will be the function of cosine.

  If m is an odd number, then the integral will be the function of sine.
- If m is an even number, then the pattern will be like this for the integral:

$$\int \sin^n x \ \cos^m \ x \, dx = \text{sgn}_0 * {}_{\frac{n-1}{2}}C_0 \frac{\cos^{m+1} \ x}{m+1} + \text{sgn}_1 * {}_{\frac{n-1}{2}}C_1 \frac{\cos^{m+3} \ x}{m+3} + \ldots$$
$$+ \text{sgn}_{\frac{n-1}{2}} * {}_{\frac{n-1}{2}}C_{\frac{m}{2}} \frac{\cos^{n+m} \ x}{n+m}$$

  We start with $\text{sgn}_0 = -1$ and it is alternating at every iteration.
- If m is an odd number, then the pattern will be like this for the integral:

$$\int \sin^n x \ \cos^m \ x \, dx = \text{sgn}_0 * {}_{\frac{m-1}{2}}C_0 \frac{\sin^{n+1} \ x}{n+1} + \text{sgn}_1 * {}_{\frac{m-1}{2}}C_1 \frac{\sin^{n+3} \ x}{n+3} + \ldots$$
$$+ \text{sgn}_{\frac{m-1}{2}} * {}_{\frac{m-1}{2}}C_{\frac{m-1}{2}} \frac{\sin^{n+m} \ x}{n+m}$$

  We start with $\text{sgn}_0 = -1$ and it is alternating at every iteration.

viii. $\int \tan^n x \ \sec^m \ x \, dx$

These are the formula of $\int \tan^n x \ \sec^m \ x \, dx$ with $n = 1$ and $m = 1$ to $m = 8$.

$$\int \tan^1 x \ \sec^1 \ x \, dx = \frac{1}{\cos(x)}$$
$$\int \tan^1 x \ \sec^2 \ x \, dx = \frac{1}{2\cos^2(x)}$$
$$\int \tan^1 x \ \sec^3 \ x \, dx = \frac{1}{3\cos^3(x)}$$
$$\int \tan^1 x \ \sec^4 \ x \, dx = \frac{1}{4\cos^4(x)}$$
$$\int \tan^1 x \ \sec^5 \ x \, dx = \frac{1}{5\cos^5(x)}$$
$$\int \tan^1 x \ \sec^6 \ x \, dx = \frac{1}{6\cos^6(x)}$$
$$\int \tan^1 x \ \sec^7 \ x \, dx = \frac{1}{7\cos^7(x)}$$
$$\int \tan^1 x \ \sec^8 \ x \, dx = \frac{1}{8\cos^8(x)}$$

These are the formula of $\int \tan^n x \ \sec^m \ x \, dx$ with $n = 2$ and $m = 1$ to $m = 8$.

$$\int \tan^2 x \ \sec^1 \ x \, dx = \frac{\log(\sin(x)-1)}{4} - \frac{\log(\sin(x)+1)}{4} - \frac{\sin(x)}{2\sin^2(x)-2}$$
$$\int \tan^2 x \ \sec^2 \ x \, dx = -\frac{\sin(x)}{3\cos(x)} + \frac{\sin(x)}{3\cos^3(x)}$$
$$\int \tan^2 x \ \sec^3 \ x \, dx = \frac{\log(\sin(x)-1)}{16} - \frac{\log(\sin(x)+1)}{16} - \frac{-\sin^3(x)-\sin(x)}{8\sin^4(x)-16\sin^2(x)+8}$$
$$\int \tan^2 x \ \sec^4 \ x \, dx = -\frac{2\sin(x)}{15\cos(x)} - \frac{\sin(x)}{15\cos^3(x)} + \frac{\sin(x)}{5\cos^5(x)}$$
$$\int \tan^2 x \ \sec^5 \ x \, dx = \frac{\log(\sin(x)-1)}{32} - \frac{\log(\sin(x)+1)}{32} + \frac{3\sin^5(x)-8\sin^3(x)-3\sin(x)}{48\sin^6(x)-144\sin^4(x)+144\sin^2(x)-48}$$
$$\int \tan^2 x \ \sec^6 \ x \, dx = -\frac{8\sin(x)}{105\cos(x)} - \frac{4\sin(x)}{105\cos^3(x)} - \frac{\sin(x)}{35\cos^5(x)} + \frac{\sin(x)}{7\cos^7(x)}$$
$$\int \tan^2 x \ \sec^7 \ x \, dx = \frac{5\log(\sin(x)-1)}{256} - \frac{5\log(\sin(x)+1)}{256} - \frac{-15\sin^7(x)+55\sin^5(x)-73\sin^3(x)-15\sin(x)}{384\sin^8(x)-1536\sin^6(x)+2304\sin^4(x)-1536\sin^2(x)+384}$$
$$\int \tan^2 x \ \sec^8 \ x \, dx = -\frac{16\sin(x)}{315\cos(x)} - \frac{8\sin(x)}{315\cos^3(x)} - \frac{2\sin(x)}{105\cos^5(x)} - \frac{\sin(x)}{63\cos^7(x)} + \frac{\sin(x)}{9\cos^9(x)}$$

These are the formula of $\int \tan^n x \ \sec^m \ x \, dx$ with $n = 3$ and $m = 1$ to $m = 8$.

$$\int \tan^3 x \ \sec^1 \ x \, dx = \frac{1 - 3\cos^2(x)}{3\cos^3(x)}$$
$$\int \tan^3 x \ \sec^2 \ x \, dx = \frac{1 - 2\cos^2(x)}{4\cos^4(x)}$$
$$\int \tan^3 x \ \sec^3 \ x \, dx = \frac{3 - 5\cos^2(x)}{15\cos^5(x)}$$
$$\int \tan^3 x \ \sec^4 \ x \, dx = \frac{2 - 3\cos^2(x)}{12\cos^6(x)}$$
$$\int \tan^3 x \ \sec^5 \ x \, dx = \frac{5 - 7\cos^2(x)}{35\cos^7(x)}$$
$$\int \tan^3 x \ \sec^6 \ x \, dx = \frac{3 - 4\cos^2(x)}{24\cos^8(x)}$$
$$\int \tan^3 x \ \sec^7 \ x \, dx = \frac{7 - 9\cos^2(x)}{63\cos^9(x)}$$
$$\int \tan^3 x \ \sec^8 \ x \, dx = \frac{4 - 5\cos^2(x)}{40\cos^{10}(x)}$$

These are the formula of $\int \tan^n x \ \sec^m \ x \, dx$ with $n = 4$ and $m = 1$ to $m = 8$.

$$\int \tan^4 x \ \sec^1 \ x \, dx = -\frac{3\log(\sin(x) - 1)}{16} + \frac{3\log(\sin(x) + 1)}{16} - \frac{-5\sin^3(x) + 3\sin(x)}{8\sin^4(x) - 16\sin^2(x) + 8}$$
$$\int \tan^4 x \ \sec^2 \ x \, dx = \frac{\sin(x)}{5\cos(x)} - \frac{2\sin(x)}{5\cos^3(x)} + \frac{\sin(x)}{5\cos^5(x)}$$
$$\int \tan^4 x \ \sec^3 \ x \, dx = -\frac{\log(\sin(x) - 1)}{32} + \frac{\log(\sin(x) + 1)}{32} - \frac{-3\sin^5(x) - 8\sin^3(x) + 3\sin(x)}{48\sin^6(x) - 144\sin^4(x) + 144\sin^2(x) - 48}$$
$$\int \tan^4 x \ \sec^4 \ x \, dx = \frac{2\sin(x)}{35\cos(x)} + \frac{\sin(x)}{35\cos^3(x)} - \frac{8\sin(x)}{35\cos^5(x)} + \frac{\sin(x)}{7\cos^7(x)}$$
$$\int \tan^4 x \ \sec^5 \ x \, dx = -\frac{3\log(\sin(x) - 1)}{256} + \frac{3\log(\sin(x) + 1)}{256} - \frac{3\sin^7(x) - 11\sin^5(x) - 11\sin^3(x) + 3\sin(x)}{128\sin^8(x) - 512\sin^6(x) + 768\sin^4(x) - 512\sin^2(x) + 128}$$
$$\int \tan^4 x \ \sec^6 \ x \, dx = \frac{8\sin(x)}{315\cos(x)} + \frac{4\sin(x)}{315\cos^3(x)} + \frac{\sin(x)}{105\cos^5(x)} - \frac{10\sin(x)}{63\cos^7(x)} + \frac{\sin(x)}{9\cos^9(x)}$$
$$\int \tan^4 x \ \sec^7 \ x \, dx = -\frac{3\log(\sin(x) - 1)}{512} + \frac{3\log(\sin(x) + 1)}{512} + \frac{-15\sin^9(x) + 70\sin^7(x) - 128\sin^5(x) - 70\sin^3(x) + 15\sin(x)}{1280\sin^{10}(x) - 6400\sin^8(x) + 12800\sin^6(x) - 12800\sin^4(x) + 6400\sin^2(x) - 1280}$$
$$\int \tan^4 x \ \sec^8 \ x \, dx = \frac{16\sin(x)}{1155\cos(x)} + \frac{8\sin(x)}{1155\cos^3(x)} + \frac{2\sin(x)}{385\cos^5(x)} + \frac{\sin(x)}{231\cos^7(x)} - \frac{4\sin(x)}{33\cos^9(x)} + \frac{\sin(x)}{11\cos^{11}(x)}$$

These are the formula of $\int \tan^n x \ \sec^m \ x \, dx$ with $n = 5$ and $m = 1$ to $m = 8$.

$$\int \tan^5 x \ \sec^1 \ x \, dx = \frac{-(-15\cos^4(x) + 10\cos^2(x) - 3)}{15\cos^5(x)}$$
$$\int \tan^5 x \ \sec^2 \ x \, dx = \frac{-(-3\cos^4(x) + 3\cos^2(x) - 1)}{6\cos^6(x)}$$
$$\int \tan^5 x \ \sec^3 \ x \, dx = \frac{-(-35\cos^4(x) + 42\cos^2(x) - 15)}{105\cos^7(x)}$$
$$\int \tan^5 x \ \sec^4 \ x \, dx = \frac{-(-6\cos^4(x) + 8\cos^2(x) - 3)}{24\cos^8(x)}$$
$$\int \tan^5 x \ \sec^5 \ x \, dx = \frac{-(-63\cos^4(x) + 90\cos^2(x) - 35)}{315\cos^9(x)}$$
$$\int \tan^5 x \ \sec^6 \ x \, dx = \frac{-(-10\cos^4(x) + 15\cos^2(x) - 6)}{60\cos^{10}(x)}$$
$$\int \tan^5 x \ \sec^7 \ x \, dx = \frac{-(-99\cos^4(x) + 154\cos^2(x) - 63)}{693\cos^{11}(x)}$$
$$\int \tan^5 x \ \sec^8 \ x \, dx = \frac{-(-15\cos^4(x) + 24\cos^2(x) - 10)}{120\cos^{12}(x)}$$

These are the formula of $\int \tan^n x \ \sec^m \ x \, dx$ with $n = 6$ and $m = 1$ to $m = 8$.

$$\int \tan^6 x \ \sec^1 \ x \, dx = \frac{5\log(\sin(x) - 1)}{32} - \frac{5\log(\sin(x) + 1)}{32} + \frac{-33\sin^5(x) + 40\sin^3(x) - 15\sin(x)}{48\sin^6(x) - 144\sin^4(x) + 144\sin^2(x) - 48}$$
$$\int \tan^6 x \ \sec^2 \ x \, dx = -\frac{\sin(x)}{7\cos(x)} + \frac{3\sin(x)}{7\cos^3(x)} - \frac{3\sin(x)}{7\cos^5(x)} + \frac{\sin(x)}{7\cos^7(x)}$$
$$\int \tan^6 x \ \sec^3 \ x \, dx = \frac{5\log(\sin(x) - 1)}{256} - \frac{5\log(\sin(x) + 1)}{256} - \frac{-15\sin^7(x) - 73\sin^5(x) + 55\sin^3(x) - 15\sin(x)}{384\sin^8(x) - 1536\sin^6(x) + 2304\sin^4(x) - 1536\sin^2(x) + 384}$$
$$\int \tan^6 x \ \sec^4 \ x \, dx = -\frac{2\sin(x)}{63\cos(x)} - \frac{\sin(x)}{63\cos^3(x)} + \frac{5\sin(x)}{21\cos^5(x)} - \frac{19\sin(x)}{63\cos^7(x)} + \frac{\sin(x)}{9\cos^9(x)}$$
$$\int \tan^6 x \ \sec^5 \ x \, dx = \frac{3\log(\sin(x) - 1)}{512} - \frac{3\log(\sin(x) + 1)}{512} + \frac{15\sin^9(x) - 70\sin^7(x) - 128\sin^5(x) + 70\sin^3(x) - 15\sin(x)}{1280\sin^{10}(x) - 6400\sin^8(x) + 12800\sin^6(x) - 12800\sin^4(x) + 6400\sin^2(x) - 1280}$$
$$\int \tan^6 x \ \sec^6 \ x \, dx = -\frac{8\sin(x)}{693\cos(x)} - \frac{4\sin(x)}{693\cos^3(x)} - \frac{\sin(x)}{231\cos^5(x)} + \frac{113\sin(x)}{693\cos^7(x)} - \frac{23\sin(x)}{99\cos^9(x)} + \frac{\sin(x)}{11\cos^{11}(x)}$$
$$\int \tan^6 x \ \sec^7 \ x \, dx = \frac{5\log(\sin(x) - 1)}{2048} - \frac{5\log(\sin(x) + 1)}{2048}$$

$$- \frac{-15\sin^{11}(x) + 85\sin^9(x) - 198\sin^7(x) - 198\sin^5(x) + 85\sin^3(x) - 15\sin(x)}{3072\sin^{12}(x) - 18432\sin^{10}(x) + 46080\sin^8(x) - 61440\sin^6(x) + 46080\sin^4(x) - 18432\sin^2(x) + 3072}$$

$$\int \tan^6 x \ \sec^8 \ x \, dx = -\frac{16\sin(x)}{3003\cos(x)} - \frac{8\sin(x)}{3003\cos^3(x)} - \frac{2\sin(x)}{1001\cos^5(x)} - \frac{5\sin(x)}{3003\cos^7(x)} + \frac{53\sin(x)}{429\cos^9(x)} - \frac{27\sin(x)}{143\cos^{11}(x)} + \frac{\sin(x)}{13\cos^{13}(x)}$$

These are the formula of $\int \tan^n x \ \sec^m \ x \, dx$ with $n = 7$ and $m = 1$ to $m = 8$.

$$\int \tan^7 x \ \sec^1 \ x \, dx = \frac{-35\cos^6(x) + 35\cos^4(x) - 21\cos^2(x) + 5}{35\cos^7(x)}$$
$$\int \tan^7 x \ \sec^2 \ x \, dx = \frac{-4\cos^6(x) + 6\cos^4(x) - 4\cos^2(x) + 1}{8\cos^8(x)}$$
$$\int \tan^7 x \ \sec^3 \ x \, dx = \frac{-105\cos^6(x) + 189\cos^4(x) - 135\cos^2(x) + 35}{315\cos^9(x)}$$
$$\int \tan^7 x \ \sec^4 \ x \, dx = \frac{-10\cos^6(x) + 20\cos^4(x) - 15\cos^2(x) + 4}{40\cos^{10}(x)}$$
$$\int \tan^7 x \ \sec^5 \ x \, dx = \frac{-231\cos^6(x) + 495\cos^4(x) - 385\cos^2(x) + 105}{1155\cos^{11}(x)}$$
$$\int \tan^7 x \ \sec^6 \ x \, dx = \frac{-20\cos^6(x) + 45\cos^4(x) - 36\cos^2(x) + 10}{120\cos^{12}(x)}$$
$$\int \tan^7 x \ \sec^7 \ x \, dx = \frac{-429\cos^6(x) + 1001\cos^4(x) - 819\cos^2(x) + 231}{3003\cos^{13}(x)}$$
$$\int \tan^7 x \ \sec^8 \ x \, dx = \frac{-35\cos^6(x) + 84\cos^4(x) - 70\cos^2(x) + 20}{280\cos^{14}(x)}$$

Case 1: For $\int \tan^n x \ \sec^m \ x \, dx$ with n even
We divide this case into 2 smaller cases:

1. When m even

2. When m odd

Case 1-a: For $\int \tan^n x \ \sec^m \ x \, dx$ with n even and m even

$$\int \tan^2 x \ \sec^2 \ x \, dx = -\frac{\sin(x)}{3\cos(x)} + \frac{\sin(x)}{3\cos^3(x)}$$

$$\int \tan^2 x \ \sec^4 \ x \, dx = -\frac{2\sin(x)}{15\cos(x)} - \frac{\sin(x)}{15\cos^3(x)} + \frac{\sin(x)}{5\cos^5(x)}$$

$$\int \tan^2 x \ \sec^6 \ x \, dx = -\frac{8\sin(x)}{105\cos(x)} - \frac{4\sin(x)}{105\cos^3(x)} - \frac{\sin(x)}{35\cos^5(x)} + \frac{\sin(x)}{7\cos^7(x)}$$

$$\int \tan^2 x \ \sec^8 \ x \, dx = -\frac{16\sin(x)}{315\cos(x)} - \frac{8\sin(x)}{315\cos^3(x)} - \frac{2\sin(x)}{105\cos^5(x)} - \frac{\sin(x)}{63\cos^7(x)} + \frac{\sin(x)}{9\cos^9(x)}$$

$$\int \tan^4 x \ \sec^2 \ x \, dx = \frac{\sin(x)}{5\cos(x)} - \frac{2\sin(x)}{5\cos^3(x)} + \frac{\sin(x)}{5\cos^5(x)}$$

$$\int \tan^4 x \ \sec^4 \ x \, dx = \frac{2\sin(x)}{35\cos(x)} + \frac{\sin(x)}{35\cos^3(x)} - \frac{8\sin(x)}{35\cos^5(x)} + \frac{\sin(x)}{7\cos^7(x)}$$

$$\int \tan^4 x \ \sec^6 \ x \, dx = \frac{8\sin(x)}{315\cos(x)} + \frac{4\sin(x)}{315\cos^3(x)} + \frac{\sin(x)}{105\cos^5(x)} - \frac{10\sin(x)}{63\cos^7(x)} + \frac{\sin(x)}{9\cos^9(x)}$$

$$\int \tan^4 x \ \sec^8 \ x \, dx = \frac{16\sin(x)}{1155\cos(x)} + \frac{8\sin(x)}{1155\cos^3(x)} + \frac{2\sin(x)}{385\cos^5(x)} + \frac{\sin(x)}{231\cos^7(x)} - \frac{4\sin(x)}{33\cos^9(x)} + \frac{\sin(x)}{11\cos^{11}(x)}$$

$$\int \tan^6 x \ \sec^2 \ x \, dx = -\frac{\sin(x)}{7\cos(x)} + \frac{3\sin(x)}{7\cos^3(x)} - \frac{3\sin(x)}{7\cos^5(x)} + \frac{\sin(x)}{7\cos^7(x)}$$

$$\int \tan^6 x \ \sec^4 \ x \, dx = -\frac{2\sin(x)}{63\cos(x)} - \frac{\sin(x)}{63\cos^3(x)} + \frac{5\sin(x)}{21\cos^5(x)} - \frac{19\sin(x)}{63\cos^7(x)} + \frac{\sin(x)}{9\cos^9(x)}$$

$$\int \tan^6 x \ \sec^6 \ x \, dx = -\frac{8\sin(x)}{693\cos(x)} - \frac{4\sin(x)}{693\cos^3(x)} - \frac{\sin(x)}{231\cos^5(x)} + \frac{113\sin(x)}{693\cos^7(x)} - \frac{23\sin(x)}{99\cos^9(x)} + \frac{\sin(x)}{11\cos^{11}(x)}$$

$$\int \tan^6 x \ \sec^8 \ x \, dx = -\frac{16\sin(x)}{3003\cos(x)} - \frac{8\sin(x)}{3003\cos^3(x)} - \frac{2\sin(x)}{1001\cos^5(x)} - \frac{5\sin(x)}{3003\cos^7(x)} + \frac{53\sin(x)}{429\cos^9(x)} - \frac{27\sin(x)}{143\cos^{11}(x)} + \frac{\sin(x)}{13\cos^{13}(x)}$$

The patterns that can be seen from those equations above are

- The series are consisting of a function with $\sin(x)$ as the numerator and $\cos(x)$ as the denominator, the length of the series and the maximum power for the denominator depending on m and n.
- If you take a look at the $\int \sec^m \ dx$ for m even, you will notice that this case will use the result of $\int \sec^m \ dx$ for m even depending on n and using the combination to make the Pascal' triangle.

For example, let $n = 4$ and $m = 6$

$$\begin{aligned}\int \tan^4 x \ \sec^6 \ x \, dx &= \int (\sec^2 x - 1)^2 \ \sec^6 \ x \, dx \\ &= \int (\sec^4 x - 2\sec^2 x + 1)\sec^6 \ x \, dx \\ &= \int \sec^{10} x - 2\sec^8 x + \sec^6 \ x \, dx \\ &= \frac{8\sin(x)}{315\cos(x)} + \frac{4\sin(x)}{315\cos^3(x)} + \frac{\sin(x)}{105\cos^5(x)} - \frac{10\sin(x)}{63\cos^7(x)} + \frac{\sin(x)}{9\cos^9(x)}\end{aligned}$$

In the end we will refer to the formula for $\int \sec^m \ dx$ for m even that starts the power of the secant and end at the total power of the secant and tangent, with.

```
#include <iostream>
#include "symintegrationc++.h"
#include <vector>

using namespace std;

// Factorial and combinations
Symbolic factorial(int n) {
	if (n <= 1)
	{
		return 1;
	}
	Symbolic result = 1;
	for (int i = 2; i <= n; ++i)
	{
		result *= i;
	}
	return result;
}

Symbolic combinations(int n, int r) {
	if (r < 0 || r > n)
	{
		return 0; // Invalid input
	}
	return factorial(n) / (factorial(r) * factorial(n - r));
}

Symbolic integralsecanteven(int n) {
	Symbolic x("x");
	int v[999];
	v[0] = 1;
	Symbolic integral;
	Symbolic d0 = 1;
	int k = 1, l = 2, c=1;
	for(int i = 1 ; i < n ; i = i+2) // to compute the denominator
```

```
	{
		d0 *= i;
	}
	for(int i = 1 ; i < n — 1; i = i+2)
	{
		c = v[0];
		int arrsec[999]; // make the size of the array as big as
		    possible

		for(int j = 0 ; j < i—1 ; j = j+1)
		{
			arrsec[j] = v[j];
		}
		int d;
		for(int j = 1 ; j < i ; j = j+1)
		{
			d = arrsec[j—1];
			v[j] = d*l;
		}

		v[0] = c*k;
		v[1] = c*l;

		k= k + 2;
		l = l + 2;
	}

	int j_d = 1 ;
	for(int i = 1 ; i < (0.5*n)+1; i = i+1)
	{
		integral += ( v[i—1] * sin(x) ) / ( d0*(cos(x)^(n—j_d)) ) ;
		j_d = j_d+2;
	}
	return integral;
}

int main(void)
{
	Symbolic x("x");
	int bpowersec = 4;
	int bpowertan = 6;
	int bpower = bpowersec+bpowertan;
	Symbolic result;
	Symbolic sgn = 1;
	int j = 1;
	int m = bpowertan/2;
	for(int i = bpower ; i >= bpowersec ; i = i—2)
	{
```

```
            result += sgn*combinations(m,j-1)*integralsecanteven(i);
            sgn = -sgn;
            j = j+1;
        }
        cout << "integral = "<< result << endl;
        //cout << "integral with subs = " << result[x==1] << endl;
        return 0;
}
```

Code 25: *level 4 integral for tangent even power and secant even power case*

Case 1-b: For $\int \tan^n x \ \sec^m \ x \, dx$ with n even and m odd

$$\int \tan^2 x \ \sec^1 \ x \, dx = \frac{\log(\sin(x)-1)}{4} - \frac{\log(\sin(x)+1)}{4} - \frac{\sin(x)}{2\sin^2(x)-2}$$

$$\int \tan^2 x \ \sec^3 \ x \, dx = \frac{\log(\sin(x)-1)}{16} - \frac{\log(\sin(x)+1)}{16} - \frac{-\sin^3(x)-\sin(x)}{8\sin^4(x)-16\sin^2(x)+8}$$

$$\int \tan^2 x \ \sec^5 \ x \, dx = \frac{\log(\sin(x)-1)}{32} - \frac{\log(\sin(x)+1)}{32} + \frac{3\sin^5(x)-8\sin^3(x)-3\sin(x)}{48\sin^6(x)-144\sin^4(x)+144\sin^2(x)-48}$$

$$\int \tan^2 x \ \sec^7 \ x \, dx = \frac{5\log(\sin(x)-1)}{256} - \frac{5\log(\sin(x)+1)}{256} - \frac{-15\sin^7(x)+55\sin^5(x)-73\sin^3(x)-15\sin(x)}{384\sin^8(x)-1536\sin^6(x)+2304\sin^4(x)-1536\sin^2(x)+384}$$

$$\int \tan^4 x \ \sec^1 \ x \, dx = -\frac{3\log(\sin(x)-1)}{16} + \frac{3\log(\sin(x)+1)}{16} - \frac{-5\sin^3(x)+3\sin(x)}{8\sin^4(x)-16\sin^2(x)+8}$$

$$\int \tan^4 x \ \sec^3 \ x \, dx = -\frac{\log(\sin(x)-1)}{32} + \frac{\log(\sin(x)+1)}{32} - \frac{-3\sin^5(x)-8\sin^3(x)+3\sin(x)}{48\sin^6(x)-144\sin^4(x)+144\sin^2(x)-48}$$

$$\int \tan^4 x \ \sec^5 \ x \, dx = -\frac{3\log(\sin(x)-1)}{256} + \frac{3\log(\sin(x)+1)}{256} - \frac{3\sin^7(x)-11\sin^5(x)-11\sin^3(x)+3\sin(x)}{128\sin^8(x)-512\sin^6(x)+768\sin^4(x)-512\sin^2(x)+128}$$

$$\int \tan^4 x \ \sec^7 \ x \, dx = -\frac{3\log(\sin(x)-1)}{512} + \frac{3\log(\sin(x)+1)}{512} + \frac{-15\sin^9(x)+70\sin^7(x)-128\sin^5(x)-70\sin^3(x)+15\sin(x)}{1280\sin^{10}(x)-6400\sin^8(x)+12800\sin^6(x)-12800\sin^4(x)+6400\sin^2(x)-1280}$$

$$\int \tan^6 x \ \sec^1 \ x \, dx = \frac{5\log(\sin(x)-1)}{32} - \frac{5\log(\sin(x)+1)}{32} + \frac{-33\sin^5(x)+40\sin^3(x)-15\sin(x)}{48\sin^6(x)-144\sin^4(x)+144\sin^2(x)-48}$$

$$\int \tan^6 x \ \sec^3 \ x \, dx = \frac{5\log(\sin(x)-1)}{256} - \frac{5\log(\sin(x)+1)}{256} - \frac{-15\sin^7(x)-73\sin^5(x)+55\sin^3(x)-15\sin(x)}{384\sin^8(x)-1536\sin^6(x)+2304\sin^4(x)-1536\sin^2(x)+384}$$

$$\int \tan^6 x \ \sec^5 \ x \, dx = \frac{3\log(\sin(x)-1)}{512} - \frac{3\log(\sin(x)+1)}{512} + \frac{15\sin^9(x)-70\sin^7(x)-128\sin^5(x)+70\sin^3(x)-15\sin(x)}{1280\sin^{10}(x)-6400\sin^8(x)+12800\sin^6(x)-12800\sin^4(x)+6400\sin^2(x)-1280}$$

This case is considered as the hardest of all cases in $\int \tan^n x \ \sec^m \ x \ dx$. If you try to solve it by hand, most calculus book will recommend to use method of substitution to replace $\tan^n x$ into $\sec x$ function so we can then compute the integral of secant with odd power, which already been done previously in level 3 integral section.

The secant with odd power make use of middle coefficient, the sum of two neighbor entry in the vector that will become the constant / coefficient for the numerator with sine function that has odd power.

Here, we will do the same, with an adding of some tricks.

The patterns that can be seen from those equations above are

- The terms are divided into 3, the same like integral of secant with odd power which are:
  1. The numerator with $\sin(x)$ function that has odd power
  2. The denominator with $\sin(x)$ function that has even power
  3. The log terms
- For all $\int \tan^n x \ \sec^m \ x \ dx$, when $m = 1$ the result is the same / equivalent with $\int \sec^{n+1} \ x \ dx$.

  To solve for any m that can be any number but 1, we will need to make another **for** loop to adjust the coefficient.

  So the starting is we will use $\int \sec^{n+1} \ x \ dx$ that we already know the pattern.
- The denominator coefficients have the exact same pattern with the integral of secant with odd power, but this time we will make the denominator coefficient to run another loop again till we reach m.

To learn the pattern we will try with small n and m, we will make use of

$$\tan^2 x = \sec^2 x - 1$$

to get everything in terms of $\sec \ x$.

For $n = 2$ and $m = 3$

$$\begin{aligned}\int \tan^2 x \ \sec^3 \ x \, dx &= (\sec^2 x - 1)\sec^3 x \, dx \\ &= \int \sec^5 x - \sec^3 \ x \, dx \\ &= \left[-\frac{3\log(\sin(x)-1)}{16} + \frac{3\log(\sin(x)+1)}{16} - \frac{3\sin^3 x - 5\sin \ x}{8\sin^4 x - 16\sin^2 x + 8}\right] \\ &- \left[-\frac{\log(\sin(x)-1)}{4} + \frac{\log(\sin(x)+1)}{4} - \frac{\sin \ x}{2\sin^2 x - 2}\right] \\ &= -\frac{\log(\sin(x)-1)}{16} + \frac{\log(\sin(x)+1)}{16} \\ &- \frac{3\sin^3 x - 5\sin \ x - (4\sin^2 x - 4)(\sin \ x)}{8\sin^4 x - 16\sin^2 x + 8} \\ &= -\frac{\log(\sin(x)-1)}{16} + \frac{\log(\sin(x)+1)}{16} - \frac{-\sin^3 x - \sin \ x}{8\sin^4 x - 16\sin^2 x + 8}\end{aligned}$$

For $n = 4$ and $m = 3$

$$\begin{aligned}\int \tan^4 x \ \sec^3 \ x \, dx &= (\sec^2 x - 1)^2 \sec^3 x \, dx \\ &= \int \sec^3 x(\sec^4 \ x - 2\sec^2 x + 1) \, dx \\ &= \int \sec^3 x - 2\sec^5 x + \sec^7 \ x \, dx \\ &= -\frac{\log(\sin(x)-1)}{32} + \frac{\log(\sin(x)+1)}{32} + \frac{-3\sin^5 x - 8\sin^3 x + 3\sin \ x}{48\sin^6 x - 144\sin^4 x + 144\sin^2 x - 48}\end{aligned}$$

If we take a closer look and observe at this part $\int \sec^3 x - 2\sec^5 x + \sec^7 \ x \, dx$, we are seeing a binomial coefficient / pascal triangle with alternating sign, thus we can use this pattern to compute $\int \tan^n x \ \sec^m \ x \, dx$ with n even and m odd for any number of n and m.

```
#include <iostream>
#include "symintegrationc++.h"
#include <vector>

using namespace std;

// Factorial and combinations
Symbolic factorial(int n) {
	if (n <= 1)
	{
		return 1;
	}
	Symbolic result = 1;
	for (int i = 2; i <= n; ++i)
	{
		result *= i;
	}
	return result;
```

```
}

Symbolic combinations(int n, int r) {
	if (r < 0 || r > n)
	{
		return 0; // Invalid input
	}
	return factorial(n) / (factorial(r) * factorial(n — r));
}

Symbolic numeratorintegralsecantodd(int n) {
	Symbolic x("x");
	int k = 1, l=2;
	vector<int> v={1};
	vector<int> mc={1}; // to store the middle coefficient
	Symbolic sgn = —1;
	Symbolic integral_numerator;
	Symbolic d0 = 2, d1 = 2;
	int j =1;
	int m = n—(0.5*(n+1));
	int last_coeff;
	int first_coeff = 3;

	// For the coefficient at the numerator of sine with odd power
	for(int i = 1 ; i < (n—1)/2 ; i = i+1)
	{
		if (i >= 2)
		{
			for(int ic = 1 ; ic < i ; ic = ic+1)
			{
				mc[ic—1] = v[ic—1] + v[ic];
			}
		}

		k = k*l;
		last_coeff = v[j—1];
		v[0] = v[0]*(first_coeff+2*(i—1));
		v.assign({v[0]});

		for(int ic = 1 ; ic < i ; ic = ic+1)
		{
			v.push_back(mc[ic—1]*(first_coeff+2*(i—1)));
		}
		d1 = d1*(2*i);
		d0 = d0*(2+2*i);
		v.push_back(last_coeff*(first_coeff+2*(i—1))+k);
		l = l+2;
		j = j+1;
```

```
	}
	int j_num = 0 ;
	for(int i = n-2 ; i >= 1 ; i = i-2)
	{
		integral_numerator += sgn*v[j_num]*((sin(x))^(i));
		sgn = -sgn;
		j_num = j_num+1;
	}
	return integral_numerator;
}

Symbolic denominatorintegralsecantodd(int n) {
	Symbolic x("x");
	int k = 1, l=2;
	vector<int> v={1};
	vector<int> mc={1}; // to store the middle coefficient
	Symbolic sgn = 1;
	Symbolic integral_denominator;
	Symbolic d0 = 2, d1 = 2;
	int j =1;
	int m = n-(0.5*(n+1));
	int last_coeff;
	int first_coeff = 3;

	// For the coefficient at the numerator of sine with odd power
	for(int i = 1 ; i < (n-1)/2 ; i = i+1)
	{
		if (i >= 2)
		{
			for(int ic = 1 ; ic < i ; ic = ic+1)
			{
				mc[ic-1] = v[ic-1] + v[ic];
			}
		}

		k = k*l;
		last_coeff = v[j-1];
		v[0] = v[0]*(first_coeff+2*(i-1));
		v.assign({v[0]});

		for(int ic = 1 ; ic < i ; ic = ic+1)
		{
			v.push_back(mc[ic-1]*(first_coeff+2*(i-1)));
		}
		d1 = d1*(2*i);
		d0 = d0*(2+2*i);
		v.push_back(last_coeff*(first_coeff+2*(i-1))+k);
		l = l+2;
```

```
              j = j+1;
       }
       j = 1;
       for(int i = n-1 ; i >= 0 ; i = i-2)
       {
              integral_denominator += sgn*d0*combinations(m,j-1)*((sin(x))^(
                  i));
              sgn = -sgn;
              j = j+1;
       }
       return integral_denominator;
}

Symbolic logtermsintegralsecantodd(int n) {
       Symbolic x("x");
       int k = 1, l=2;
       vector<int> v={1};
       vector<int> mc={1}; // to store the middle coefficient
       Symbolic sgn = -1;
       Symbolic integral_denominator;
       Symbolic d0 = 2, d1 = 2;
       int j =1;
       int m = n-(0.5*(n+1));
       int last_coeff;
       int first_coeff = 3;

       // For the coefficient at the numerator of sine with odd power
       for(int i = 1 ; i < (n-1)/2 ; i = i+1)
       {
              if (i >= 2)
              {
                     for(int ic = 1 ; ic < i ; ic = ic+1)
                     {
                            mc[ic-1] = v[ic-1] + v[ic];
                     }
              }

              k = k*l;
              last_coeff = v[j-1];
              v[0] = v[0]*(first_coeff+2*(i-1));
              v.assign({v[0]});


              for(int ic = 1 ; ic < i ; ic = ic+1)
              {
                     v.push_back(mc[ic-1]*(first_coeff+2*(i-1)));
              }
              d1 = d1*(2*i);
```

```
            d0 = d0*(2+2*i);
            v.push_back(last_coeff*(first_coeff+2*(i-1))+k);
            l = l+2;
            j = j+1;
        }
        d1 = d1*(n-1);
        return (-v[0]*ln(sin(x)-1))/(d1) + (v[0]*ln(sin(x)+1))/(d1) ;
}

int main(void)
{
        Symbolic x("x");
        int bpowersec = 3;
        int bpowertan = 6;
        int bpower = bpowersec+bpowertan;
        Symbolic sgn = 1;
        Symbolic result;
        int j = 1;
        int m = bpowertan/2;
        for(int i = bpower ; i >= bpowersec ; i = i-2)
        {
            result += sgn*combinations(m,j-1)*( (
                numeratorintegralsecantodd(i)/denominatorintegralsecantodd(
                i)) + logtermsintegralsecantodd(i));
            sgn = -sgn;
            j = j+1;
        }
        //cout << "integral at numerator= "<< numeratorintegralsecantodd(3)<<
            endl;
        //cout << "integral at denominator= "<< denominatorintegralsecantodd
            (3)<< endl;
        //cout << "log terms= "<< logtermsintegralsecantodd(3)<< endl;
        cout << "for secant power of " << bpowersec << " and tangent power of
            " << bpowertan << endl;
        cout << "integral = "<< result << endl;
        //cout << "integral with subs = " << result[x==1] << endl;
        return 0;
}
```

Code 26: *level 4 integral for tangent even power and secant odd power case*

It is recommended that you use the:
cout « "integral at numerator= "« numeratorintegralsecantodd(3)« endl;
cout « "integral at denominator= "« denominatorintegralsecantodd(3)« endl;
cout « "log terms= "« logtermsintegralsecantodd(3)« endl;
(uncomment them in the C++ code)

and comment the :
cout « "for secant power of " « bpowersec « " and tangent power of " « bpowertan « endl;

cout « "integral = "« result « endl;

Because it is faster to show the result when we only show the log terms function, numerator only / denominator only of the fraction with sine function than to show it in the full fraction function, it is still slow and we wonder why, because SymPy is able to show the result very fast, knowing that C++ is basically / by default faster than SymPy we might suspect because it is the fraction that we are using combining with symbolic computation, if we only show the vector / array, the result can come up very fast, faster than SymPy.

We will figure it out and make it really fast one day. Any reader interested is always welcome to give a nice input for this.

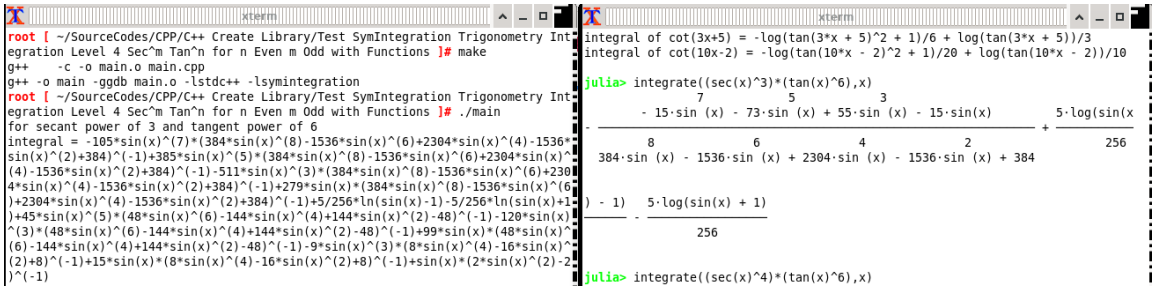

Figure 2.2: *The computation of* $\int \tan^6 x \ \sec^3 \ x \, dx$ *with SymPy and SymIntegration (**SymIntegration/Examples/-Calculus/Test SymIntegration Trigonometry Integration Level 4 Sec^m Tan^n for n Even m Odd with Functions/main.cpp**).*

Case 2: For $\int \tan^n x \ \sec^m \ x \, dx$ **with** n **odd**
We divide this case into 2 smaller cases:

1. When m even

2. When m odd

Case 2-a: For $\int \tan^n x \ \sec^m \ x \, dx$ **with** n **odd and** m **even**

$$\int \tan^1 x \ \sec^2 \ x \, dx = \frac{1}{2\cos^2(x)}$$
$$\int \tan^1 x \ \sec^4 \ x \, dx = \frac{1}{4\cos^4(x)}$$
$$\int \tan^1 x \ \sec^6 \ x \, dx = \frac{1}{6\cos^6(x)}$$
$$\int \tan^1 x \ \sec^8 \ x \, dx = \frac{1}{8\cos^8(x)}$$

$$\int \tan^3 x \ \sec^2 \ x \, dx = \frac{1 - 2\cos^2(x)}{4\cos^4(x)}$$
$$\int \tan^3 x \ \sec^4 \ x \, dx = \frac{2 - 3\cos^2(x)}{12\cos^6(x)}$$
$$\int \tan^3 x \ \sec^6 \ x \, dx = \frac{3 - 4\cos^2(x)}{24\cos^8(x)}$$
$$\int \tan^3 x \ \sec^8 \ x \, dx = \frac{4 - 5\cos^2(x)}{40\cos^{10}(x)}$$

$$\int \tan^5 x \ \sec^2 \ x \, dx = \frac{-(-3\cos^4(x) + 3\cos^2(x) - 1)}{6\cos^6(x)}$$
$$\int \tan^5 x \ \sec^4 \ x \, dx = \frac{-(-6\cos^4(x) + 8\cos^2(x) - 3)}{24\cos^8(x)}$$
$$\int \tan^5 x \ \sec^6 \ x \, dx = \frac{-(-10\cos^4(x) + 15\cos^2(x) - 6)}{60\cos^{10}(x)}$$
$$\int \tan^5 x \ \sec^8 \ x \, dx = \frac{-(-15\cos^4(x) + 24\cos^2(x) - 10)}{120\cos^{12}(x)}$$

The patterns that can be seen from those equations above are

- They are all function that only consist of function of cosine and constants.
- The power of cosine in the denominator is $m + n - 1$.
- You can solve it easily to determine the pattern by substituting

$$\sec^2 x = \tan^2 x + 1$$

then we will have the terms in $\tan(x)$ with odd power only, which we already have learned the pattern from the previous section.

For example, let $m = 4, n = 3$

$$\begin{aligned}\int \tan^3 x \ \sec^4 \ x \, dx &= \int \tan^3 x(\tan^2 x + 1)^2 dx \\ &= \int \tan^3 x(\tan^4 x + 2\tan^2 x + 1)dx \\ &= \int \tan^7 x + 2\tan^5 x + \tan^3 x dx\end{aligned}$$

It is using combination / Pascal' triangle again, this time the sign is the same, all positive.

```
#include <iostream>
#include "symintegrationc++.h"
#include <vector>

using namespace std;

// Factorial and combinations
Symbolic factorial(int n) {
       if (n <= 1)
```

```
	{
		return 1;
	}
	Symbolic result = 1;
	for (int i = 2; i <= n; ++i)
	{
		result *= i;
	}
	return result;
}

Symbolic combinations(int n, int r) {
	if (r < 0 || r > n)
	{
		return 0; // Invalid input
	}
	return factorial(n) / (factorial(r) * factorial(n - r));
}

Symbolic integraltangentodd(int n) {
	Symbolic x("x");
	Symbolic integral;
	Symbolic integral_front;
	Symbolic sgn = 1;
	vector<int> v={1};
	vector<int> v_temp={1};
	vector<int> mc={1}; // to store the middle coefficient
	vector<int> mc_temp={1}; // to store the temporary / new middle
	    coefficient
	int d0 = 2;
	int k = 1, l=2;
	// For the coefficient at the numerator
	for(int i = 1 ; i <= (n-1)/2 ; i = i+1)
	{
		if (i == 1)
		{
			v[0] = 1;
			v.assign({v[0]});
		}

		if (i ==2)
		{
			mc[0]=1;
			mc.assign({mc[0]});
			v_temp[0] = v[0]*(i-1);
			v_temp.assign({v_temp[0]});

			for(int j = 1 ; j < i ; j = j+1)
```

```
                {
                        v_temp.push_back(v[j-1]*((2*i)-j) + v_temp[0]*mc
                            [j-1] );
                }
                v[0] = v_temp[0];
                v.assign({v[0]});
                for(int j = 1 ; j < i ; j = j+1)
                {
                        v.push_back(v_temp[j]);
                }
        }
        if (i == 3)
        {
                mc[0] = mc[0] + 1; //
                mc.assign({mc[0]});

                v_temp[0] = v[0]*(i-1);
                v_temp.assign({v_temp[0]});
                for(int j = 1 ; j < i-1 ; j = j+1)
                {
                        v_temp.push_back(v[j-1]*((2*i)-j) + v_temp[0]*mc
                            [j-1] );
                }
                v_temp.push_back((v[i-2]*(i+1) ) + (v_temp[0]) );
                // Assign the temporary vector to vector v for future
                    use
                v[0] = v_temp[0];
                v.assign({v[0]});
                for(int j = 1 ; j < i ; j = j+1)
                {
                        v.push_back(v_temp[j]);
                }
        }
        if (i == 4)
        {
                mc[0] = mc[0] + 1;
                mc.assign({mc[0]});
                mc.push_back(mc[0]);

                v_temp[0] = v[0]*(i-1);
                v_temp.assign({v_temp[0]});
                for(int j = 1 ; j < i-1 ; j = j+1)
                {
                        v_temp.push_back(v[j-1]*((2*i)-j) + v_temp[0]*mc
                            [j-1] );
                }
                v_temp.push_back((v[i-2]*(i+1) ) + (v_temp[0]) );
                // Assign the temporary vector to vector v for future
```

```
                    use
            v[0] = v_temp[0];
            v.assign({v[0]});
            for(int j = 1 ; j < i ; j = j+1)
            {
                  v.push_back(v_temp[j]);
            }
      }
      if (i >= 5)
      {
            mc_temp[0] = mc[0]+1;
            mc_temp.assign({mc_temp[0]});
            for(int ic = 1 ; ic < l ; ic = ic+1)
            {
                  mc_temp[ic] = mc[ic-1] + mc[ic];
            }
            mc[l] = (mc_temp[0]);
            mc[l+1] = (mc_temp[0]);

            mc[0]=mc_temp[0];
            mc.assign({mc[0]});

            for(int ic = 1 ; ic < l ; ic = ic+1)
            {
                  mc.push_back(mc_temp[ic]);
            }
            mc.push_back(mc_temp[0]);
            mc.push_back(0);

            v_temp[0] = v[0]*(i-1);
            v_temp.assign({v_temp[0]});
            for(int j = 1 ; j < i-1 ; j = j+1)
            {
                  v_temp.push_back(v[j-1]*((2*i)-j) + v_temp[0]*mc
                      [j-1] );
            }
            v_temp.push_back((v[i-2]*(i+1) ) + (v_temp[0]) );
            // Assign the temporary vector to vector v for future
                use
            v[0] = v_temp[0];
            v.assign({v[0]});
            for(int j = 1 ; j < i ; j = j+1)
            {
                  v.push_back(v_temp[j]);
            }

            l = l+1;
      }
```

```
            d0 = d0*k;
            k = k+1;
      }
      for(int i = 1 ; i <= (n-1)/2 ; i = i+1)
      {
            integral += sgn*v[i-1]*(cos(x)^(2*(i-1)));
            sgn=-sgn;

      }
      sgn = 1;
      for(int i = 1 ; i <= (n-1)/2 ; i = i+1)
      {
            integral_front = sgn;
            sgn = -sgn;
      }
      integral = integral_front*ln(cos(x)) + (integral)/(d0*(cos(x)^(n-1)))
          ;
      return integral;
}

int main(void)
{
      Symbolic x("x");
      int bpowersec = 4;
      int bpowertan = 5;
      int bpower = bpowersec+bpowertan;
      Symbolic result;
      int j = 1;
      int m = bpowersec/2;
      for(int i = bpowertan ; i <= bpower ; i = i+2)
      {
            result += combinations(m,j-1)*integraltangentodd(i);
            j = j+1;
      }
      cout << "for secant power of " << bpowersec << " and tangent power of
          " << bpowertan << endl;
      cout << "integral = "<< result<< endl;
      return 0;
}
```

Code 27: *level 4 integral for tangent odd power and secant even power case*

Case 2-b: For $\int \tan^n x \ \sec^m \ x \, dx$ with n odd and m odd

$$\int \tan^1 x \ \sec^1 \ x \, dx = \frac{1}{\cos(x)}$$
$$\int \tan^1 x \ \sec^3 \ x \, dx = \frac{1}{3\cos^3(x)}$$
$$\int \tan^1 x \ \sec^5 \ x \, dx = \frac{1}{5\cos^5(x)}$$
$$\int \tan^1 x \ \sec^7 \ x \, dx = \frac{1}{7\cos^7(x)}$$

$$\int \tan^3 x \ \sec^1 \ x \, dx = \frac{1-3\cos^2(x)}{3\cos^3(x)}$$
$$\int \tan^3 x \ \sec^3 \ x \, dx = \frac{3-5\cos^2(x)}{15\cos^5(x)}$$
$$\int \tan^3 x \ \sec^5 \ x \, dx = \frac{5-7\cos^2(x)}{35\cos^7(x)}$$
$$\int \tan^3 x \ \sec^7 \ x \, dx = \frac{7-9\cos^2(x)}{63\cos^9(x)}$$

$$\int \tan^5 x \ \sec^1 \ x \, dx = \frac{-(-15\cos^4(x)+10\cos^2(x)-3)}{15\cos^5(x)}$$
$$\int \tan^5 x \ \sec^3 \ x \, dx = \frac{-(-35\cos^4(x)+42\cos^2(x)-15)}{105\cos^7(x)}$$
$$\int \tan^5 x \ \sec^5 \ x \, dx = \frac{-(-63\cos^4(x)+90\cos^2(x)-35)}{315\cos^9(x)}$$
$$\int \tan^5 x \ \sec^7 \ x \, dx = \frac{-(-99\cos^4(x)+154\cos^2(x)-63)}{693\cos^{11}(x)}$$

The patterns that can be seen from those equations above are

- They are all function that only consist of function of cosine and constants.
- The power of cosine in the denominator is $m+n-1$.
- You can solve it easily to determine the pattern by substituting

  $$\tan^2 x = \sec^2 x - 1$$

  and then use method of substitution by assigning

  $$u = \sec \ x$$
  $$du = \sec \ x \tan \ x \, dx$$

  then we will have the terms in $\sec(x)$ with even power only, which we already have learned the pattern from the previous section.

For example, let $m = 3, n = 5$

$$\begin{aligned}
\int \tan^5 x \ \sec^3 \ x \, dx &= \int \tan^4 x \, sec^2 x (\sec \ x \tan \ x \, dx) \\
&= \int [(\sec^2 x - 1)^2 sec^2 x \,] (\sec \ x \tan \ x \, dx) \\
&= \int [(\sec^4 x - 2\sec^2 x + 1) sec^2 x] \ (\sec \ x \tan \ x \, dx) \\
&= \int [\sec^6 x - 2\sec^4 x + \sec^2 x] \ (\sec \ x \tan \ x \, dx) \\
&= \int u^6 - 2u^4 + u^2 \, du \\
&= \frac{1}{7}u^7 - \frac{2}{5}u^5 + \frac{1}{3}u^3 \\
&= \frac{1}{7}\sec^7 x - \frac{2}{5}\sec^5 x + \frac{1}{3}\sec^3 x \\
&= \frac{1}{7}\frac{1}{\cos^7 x} - \frac{2}{5}\frac{1}{\cos^5 x} + \frac{1}{3}\frac{1}{\cos^3 x}
\end{aligned}$$

It is using combination / Pascal' triangle combining with a method of substitution, the sign is alternating this time. The thing is the method of substitution is a really helpful here, we do not even need to create a function, only integrating ordinary polynomial and then substituting back, we can substitute $u = \sec(x)$ or $u = \frac{1}{\cos(x)}$.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Trigonometry Int
egration Level 4 Sec^m Tan^n for n Odd m Odd with Functions ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Trigonometry Integration Lev
el 4 Sec^m Tan^n for n Odd m Odd with Functions ]# ./main
for secant power of 3 and tangent power of 5
we will compute the integral of u^(2)-2*u^(4)+u^(6)
integral = 1/3*u^(3)-2/5*u^(5)+1/7*u^(7)
Substitute back, integral = 1/3*cos(x)^(-3)-2/5*cos(x)^(-5)+1/7*cos(x)^(-7)
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Trigonometry Integration Lev
el 4 Sec^m Tan^n for n Odd m Odd with Functions ]#
```

```
julia> integrate((sec(x)^3)*(tan(x)^5),x)
 ⎛        4           2        ⎞
-⎝- 35·cos (x) + 42·cos (x) - 15⎠
──────────────────────────────────
                 7
          105·cos (x)
```

Figure 2.3: *The computation of* $\int \tan^5 x \ \sec^3 \ x \, dx$ *with SymPy and SymIntegration (**SymIntegration/Examples/Calculus/Test SymIntegration Trigonometry Integration Level 4 Sec^m Tan^n for n Odd m Odd/main.cpp**).*

```
#include <iostream>
#include "symintegrationc++.h"
#include <vector>

using namespace std;
```

```
// Factorial and combinations
Symbolic factorial(int n) {
	if (n <= 1)
	{
		return 1;
	}
	Symbolic result = 1;
	for (int i = 2; i <= n; ++i)
	{
		result *= i;
	}
	return result;
}

Symbolic combinations(int n, int r) {
	if (r < 0 || r > n)
	{
		return 0; // Invalid input
	}
	return factorial(n) / (factorial(r) * factorial(n - r));
}

int main(void)
{
	Symbolic u("u"), x("x");
	int bpowersec = 3;
	int bpowertan = 5;
	int bpower = bpowersec+bpowertan;
	Symbolic result;
	int j = 1;
	int m = (bpowertan-1)/2;
	Symbolic sgn = 1;
	for(int i = bpowersec-1 ; i <= bpower-2 ; i = i+2)
	{
		result += sgn*combinations(m,j-1)*(u^i);
		j = j+1;
		sgn = -sgn;
	}
	cout << "for secant power of " << bpowersec << " and tangent power of
	    " << bpowertan << endl;
	cout << "we will compute the integral of "<< result << endl;
	Symbolic f = integrate(result,u);
	cout << "integral = "<< f << endl;
	cout << "Substitute back, integral = "<< f[u==(1/cos(x))] << endl;
	return 0;
}
```

Code 28: *level 4 integral for tangent odd power and secant odd power case*

ix. $\int \cot^n x \ \csc^m x \, dx$

The formula of $\int \cot^n x \ \csc^m x \, dx$ has similar pattern with $\int \tan^n x \ \sec^m x \, dx$, we only change from $\sin(x)$ to $\cos(x)$ and from $\cos(x)$ to $\sin(x)$, the sign will be changed too, from minus to plus, and from plus to minus. Once we have decoded the pattern for $\int \tan^n x \ \sec^m x \, dx$, we only need to modify a bit for this integral type.

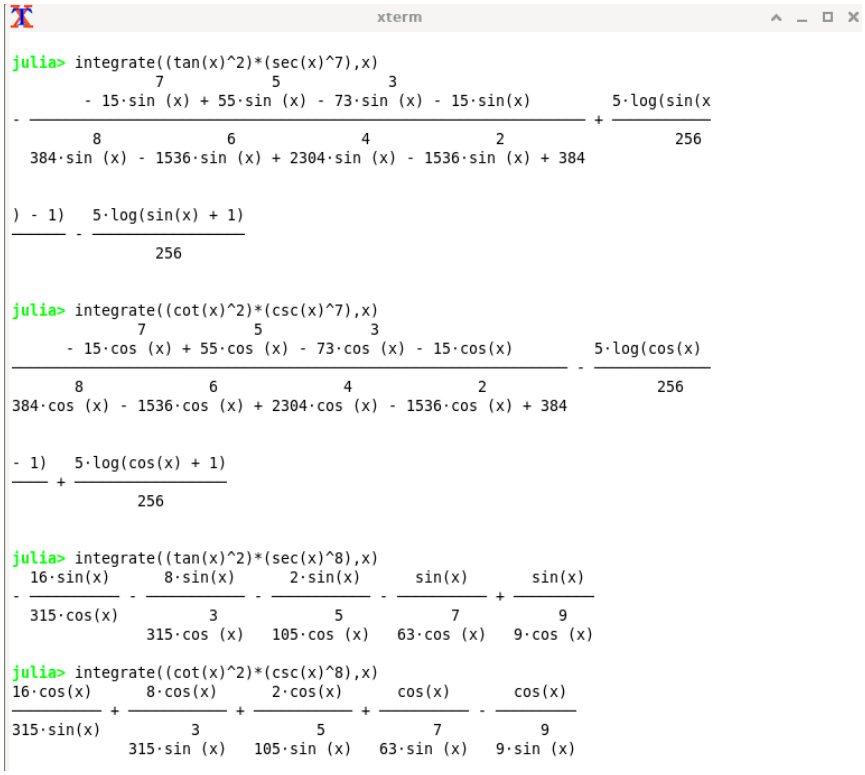

Figure 2.4: *The integral of* $\int \cot^n x \ \csc^m x \, dx$ *that has similar pattern with of* $\int \tan^n x \ \sec^m x \, dx$.

VII. Trigonometry and Transcendentals Formula

We will list all the trignometry and trancendentals (exponential and logarithm) formulas that can be used to help you solve integral problems

[SI*] **Trigonometry**
Reciprocal identities

$$\begin{aligned}\tan\ x &= \frac{\sin\ x}{\cos\ x}\\ \cot\ x &= \frac{\cos\ x}{\sin\ x}\\ \csc\ x &= \frac{1}{\sin\ x}\\ \sec\ x &= \frac{1}{\cos\ x}\end{aligned}$$

Pythagorean identities

$$\begin{aligned}\sin^2 x + \cos^2 x &= 1\\ \tan^2 x &= \sec^2 x - 1\\ \cot^2 x &= \csc^2 x - 1\end{aligned}$$

Sum and difference identities

$$\begin{aligned}\sin(x+y) &= \sin\ x\ \cos\ y + \cos x\ \sin\ y\\ \sin(x-y) &= \sin\ x\ \cos\ y - \cos x\ \sin\ y\\ \cos(x+y) &= \cos\ x\ \cos\ y - \sin x\ \sin\ y\\ \cos(x-y) &= \cos\ x\ \cos\ y + \sin x\ \sin\ y\\ \tan(x+y) &= \frac{\tan\ x + \tan\ y}{1 - \tan\ x\ \tan\ y}\\ \tan(x-y) &= \frac{\tan\ x - \tan\ y}{1 + \tan\ x\ \tan\ y}\end{aligned}$$

Double-angle identities

$$\begin{aligned}\sin\ 2x &= 2\sin\ x\ \cos x = \frac{2\tan\ x}{1+\tan^2 x}\\ \cos\ 2x &= \cos^2 x - \sin^2 x\\ \cos\ 2x &= 2\cos^2 x - 1\\ \cos\ 2x &= 1 - 2\sin^2 x\\ \cos\ 2x &= \frac{1-\tan^2 x}{1+\tan^2 x}\\ \tan\ 2x &= \frac{2\tan\ x}{1-\tan^2 x}\\ \cot\ 2x &= \frac{\cot^2 x - 1}{2\cot\ x}\end{aligned}$$

Half-angle identities

$$\sin\ \frac{x}{2} = \pm\sqrt{\frac{1-\cos\ x}{2}}$$
$$\cos\ \frac{x}{2} = \pm\sqrt{\frac{1+\cos\ x}{2}}$$
$$\tan\ \frac{x}{2} = \pm\sqrt{\frac{1-\cos\ x}{1+\cos\ x}} = \frac{\sin\ x}{1+\cos\ x} = \frac{1-\cos\ x}{\sin\ x}$$

Integral

$$\int \sin\ x\, dx = -\cos\ x$$
$$\int \cos\ x\, dx = \sin\ x$$
$$\int \tan\ x\, dx = -\log(\cos(x))$$
$$\int \sec\ x\, dx = -\frac{\log(\sin(x)-1)}{2} + \frac{\log(\sin(x)+1)}{2}$$
$$\int \csc\ x\, dx = \frac{\log(\cos(x)-1)}{2} - \frac{\log(\cos(x)+1)}{2}$$
$$\int \cot\ x\, dx = \log(\sin(x))$$

Inverse Trigonometric integral

$$\int \frac{1}{1+x^2} dx = \operatorname{atan}(x)$$
$$\int \frac{1}{a+bx^2} dx = -\frac{\sqrt{\frac{-1}{ab}}\ln\left(-a\sqrt{\frac{-1}{ab}}+x\right)}{2} + \frac{\sqrt{\frac{-1}{ab}}\ln\left(a\sqrt{\frac{-1}{ab}}+x\right)}{2}$$

Inverse hyperbolic trigonometry integral

$$\int \frac{1}{\sqrt{1+x^2}} dx = \sinh^{-1}(x) + C$$
$$\int \frac{1}{\sqrt{x^2-1}} dx = \cosh^{-1}(x) + C$$
$$\int \frac{1}{1-x^2} dx = \tanh^{-1}(x) + C, \quad |x| < 1$$
$$\int \frac{1}{1-x^2} dx = \coth^{-1}(x) + C, \quad |x| > 1$$
$$\int \frac{-1}{x\sqrt{1-x^2}} dx = \operatorname{sech}^{-1}(x) + C$$
$$\int \frac{-1}{|x|\sqrt{1+x^2}} dx = \operatorname{csch}^{-1}(x) + C$$

Note that the derivatives of $\tanh^{-1}(x)$ and $\coth^{-1}(x)$ are the same. Thus, when we compute $\int \frac{1}{1-x^2} dx$, we need to select the proper antiderivative based on the domain of the functions and the values of x.

$$\int \frac{1}{1-x^2} dx = \begin{cases} \tanh^{-1}(x) + C, & |x| < 1 \\ \coth^{-1}(x) + C, & |x| > 1 \end{cases}$$

Derivative

$$\begin{aligned}
\frac{d}{dx}\sin\ x &= \cos\ x \\
\frac{d}{dx}\cos\ x &= -\sin\ x \\
\frac{d}{dx}\tan\ x &= \tan^2 x + 1 \\
\frac{d}{dx}\sec\ x &= \sec(x)\tan(x) \\
\frac{d}{dx}\csc\ x &= -\cot(x)\csc(x) \\
\frac{d}{dx}\cot\ x &= -\cot^2 x - 1
\end{aligned}$$

[SI*] **Logarithm**

Logarithm along with exponential play a very important role in solving differential equations, so we will need to take a look at these formulas often to help us when solving differential equations problem

$$\begin{aligned}
\log_a b &= c \\
a^c &= b \\
\log b &= c \\
e^c &= b \\
\log_a a &= 1 \\
\log(a*b) &= \log(a) + \log(b) \\
\log\left(\frac{a}{b}\right) &= \log(a) - \log(b) \\
\log(a^b) &= b\ \log(a) \\
a^{\log_a b} &= b \\
\log_b a &= \frac{\log_c a}{\log_c} \\
\log_b c &= \log_b a \log_a c \\
log_{b^n} a^m &= \frac{m}{n}\log_b a \\
-\log_b a &= \log_b\left(\frac{1}{a}\right) \\
-\log_b a &= \log_{\frac{1}{b}} a \\
\frac{d}{dx}\log_a\ x &= \frac{1}{x\ln(a)} \\
\frac{d}{dx}\ln\ x &= \frac{1}{x}
\end{aligned}$$

with $e = 2.718281828459045$ is the Euler' number. When there is no base written in *log* function, we will use e as the base number. The base number cannot be equal to 1 and it has to be positive. For all $\log(a)$, a is a real number and has to be positive.

i. Functions in SymIntegration Related to Integration with Transcendental and Trigonometry Functions

[SI*] When we learn second order linear differential equation and want to solve it with variation of parameters, a general method that can be applied to any equation, we will be faced with a lot of combination / product of functions of trigonometry, transcendental and polynomial. In SymIntegration before we call the function **integrate()** we can call this function first, to simplify the function into a simpler function that can be integrated:

simplifybeforeintegrate(const Symbolic &F, const Symbolic &t)
this function will simplify function $F(t)$ into simpler function that can be integrated, for example:

$$\frac{3\cos\ t\ \cos\ 2t}{\sin\ 2t} = \frac{3}{2}\csc\ t - 3\sin\ t$$

The list of functions that already in the source code are:

$$F(t) = a\frac{\cos(t)\cos(2bt)}{sin(2bt)}$$
$$F(t) = a\sin(2bt)\csc(bt)$$
$$F(t) = a\cos(2bt)\csc(bt)$$

with a, b are constants and $2 * b$ has to be an even number.

VIII. Infinite Series

[SI*] **Infinite Sequences**
A sequence

$$a_1, a_2, a_3, a_4, \ldots$$

is an ordered arrangement of real numbers, one for each positive integer. More formally, an infinite sequence is a function whose domain is the set of positive integers and whose range is a set of real numbers. We may denote a sequence by $a_1, a_2, a_3, \ldots$, by $\{a_n\}_{n=1}^{\infty}$, or simply by $\{a_n\}$.

A sequence may be specified by giving enough initial terms to establish a pattern, as in

$$1, 4, 7, 10, 13, \ldots$$

by an explicit formula for the nth term, as in

$$a_n = 3n - 2, \quad n \geq 1$$

or by a recursion formula

$$a_1 = 1, \quad a_n = a_{n-1} + 3, \quad n \geq 2$$

Definition 2.1: Convergence

The sequence $\{a_n\}$ is said to converge to L, and we write

$$\lim_{n\to\infty} a_n = L$$

if for each positive number ϵ there is a corresponding positive number N such that

$$n \geq N \to |a_n - L| < \epsilon$$

A sequence that fails to converge to any finite number L is said to diverge, or to be divergent.

Theorem 2.5: Properties of Limits of Sequences

Let $\{a_n\}$ and $\{b_n\}$ be convergent sequences and k a constant. Then

1. $\lim_{n\to\infty} k = k$;
2. $\lim_{n\to\infty} ka_n = k \lim_{n\to\infty} a_n$;
3. $\lim_{n\to\infty}(a_n \pm b_n) = \lim_{n\to\infty} a_n \pm \lim_{n\to\infty} b_n$;
4. $\lim_{n\to\infty}(a_n \cdot b_n) = \lim_{n\to\infty} a_n \cdot \lim_{n\to\infty} b_n$;
5. $\lim_{n\to\infty} \frac{a_n}{b_n} = \frac{\lim_{n\to\infty} a_n}{\lim_{n\to\infty} b_n}$, provided that $\lim_{n\to\infty} b_n \neq 0$;

Theorem 2.6: Squeeze Theorem

Suppose that $\{a_n\}$ and $\{c_n\}$ both converge to L and that $a_n \leq b_n \leq c_n$ for $n \geq K$ (K a fixed integer). Then $\{b_n\}$ also converges to L.

Theorem 2.7: Absolute Convergence Theorem

If $\lim_{n\to\infty} |a_n| = 0$, then $\lim_{n\to\infty} a_n = 0$.

Since $-|a_n| \leq a_n \leq |a_n|$, the result follows from the Squeeze theorem.

Theorem 2.8: Monotonic Sequence Theorem

If U is an upper bound for a nondecreasing sequence $\{a_n\}$, then the sequence converges to a limit A that is less than or equal to U. Similarly, if L is a lower bound for a nonincreasing sequence $\{b_n\}$, then the sequence $\{b_n\}$ converges to a limit B that is greater than or equal to L.

[SI*] **Example:**
A famous sequence $\{f_n\}$, called the Fibonacci sequence after Leonardo Fibonacci, who introduced it around A.D. 1200, is defined by the recursion formula

$$f_1 = f_2 = 1, \quad f_{n+2} = f_{n+1} + f_n$$

(a) Find f_3 through f_{10}.

(b) Let $\phi = \frac{1}{2}(1+\sqrt{5}) = 1.618034$. The greeks called this number the golden ratio, claiming that a rectangle whose dimensions were in this ratio was "perfect." It can be shown that

$$\begin{aligned} f_n &= \frac{1}{\sqrt{5}\left[\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n\right]} \\ &= \frac{1}{\sqrt{5}}[\phi^n - (-1)^n\phi^{-n}] \end{aligned}$$

Check that this gives the right result for $n = 1$ and $n = 2$. The general result can be proved by induction (it is a nice challenge). More in line with this section, use this explicit formula to prove that $\lim_{n\to\infty} \frac{f_{n+1}}{f_n} = \phi$.

(c) Using this limit just proved, show that ϕ satisfies the equation $x^2 - x - 1 = 0$. Then, in another interesting twist, use the Quadratic formula to show that the two roots of this equation are ϕ and $-\frac{1}{\phi}$, two numbers that occur in the explicit formula for f_n.

Solution:

(a) $f_3 = 2, f_4 = 3, f_5 = 5, f_6 = 8, f_7 = 13, f_8 = 21, f_9 = 34, f_{10} = 55$

(b) Using the formula

$$\begin{aligned} f_1 &= \frac{1}{\sqrt{5}}\left[\frac{1+\sqrt{5}}{2} - \frac{1-\sqrt{5}}{2}\right] = \frac{1}{\sqrt{5}}\left[\frac{2\sqrt{5}}{2}\right] = 1 \\ f_2 &= \frac{1}{\sqrt{5}}\left[\left(\frac{1+\sqrt{5}}{2}\right)^2 - \left(\frac{1-\sqrt{5}}{2}\right)^2\right] \\ &= \frac{1}{\sqrt{5}}\left[\frac{1+2\sqrt{5}+5-(1-2\sqrt{5}+5)}{4}\right] \\ &= \frac{1}{\sqrt{5}}\left[\frac{4\sqrt{5}}{4}\right] = 1 \end{aligned}$$

thus

$$\begin{aligned} \lim_{n\to\infty}\frac{f_{n+1}}{f_n} &= \lim_{n\to\infty}\frac{\phi^{n+1}-(-1)^{n+1}\phi^{-n-1}}{\phi^n-(-1)^n\phi^{-n}} \\ &= \lim_{n\to\infty}\frac{\phi^{n+1}-\frac{(-1)^{n+1}}{\phi^{n+1}}}{\phi^n-\frac{(-1)^n}{\phi^n}} \\ &= \lim_{n\to\infty}\frac{\phi-\frac{(-1)^{n+1}}{\phi^{2n+1}}}{1-\frac{(-1)^n}{\phi^{2n}}} = \phi \end{aligned}$$

(c)

$$\begin{aligned} \phi^2-\phi-1 &= \left[\frac{1}{2}(1+\sqrt{5})\right]^2 - \frac{1}{2}(1+\sqrt{5}) - 1 \\ &= \left(\frac{3}{2}+\frac{\sqrt{5}}{2}\right) - \left(\frac{1}{2}+\frac{\sqrt{5}}{2}\right) - 1 \\ &= 0 \end{aligned}$$

Therefore ϕ satisfies $x^2-x-1=0$. Using the Quadratic Formula on $x^2-x-1=0$ yields

$$x = \frac{1\pm\sqrt{1+4}}{2} = \frac{1\pm\sqrt{5}}{2}$$

thus

$$\begin{aligned} \phi &= \frac{1+\sqrt{5}}{2} \\ -\frac{1}{\phi} &= -\frac{2}{1+\sqrt{5}} = -\frac{2(1-\sqrt{5})}{1-5} = \frac{1-\sqrt{5}}{2} \end{aligned}$$

[SI*] **Infinite Series**

Many algorithms in a computer to compute the sine of an angle, or e to some power are based on infinite series. Generally, infinite series can be denoted by

$$\sum_{k=1}^{\infty} a_k \tag{2.17}$$

a_k is the k-th term of the series. Then S_n, the partial n-th sum, is given by

$$S_n = a_1 + a_2 + a_3 + \cdots + a_n = \sum_{k=1}^{n} a_k \tag{2.18}$$

Definition 2.2: Converges and Diverges

The infinite series $\sum_{k=1}^{\infty}$ converges and has sum S if the sequence of partial sums $\{S_n\}$ converges to S. If $\{S_n\}$ diverges, then the series diverges. A divergent series has no sum.

Geometric Series
A series of the form

$$\sum_{k=1}^{\infty} ar^{k-1} = a + ar + ar^2 + ar^3 + \ldots \tag{2.19}$$

where $a \neq 0$, is called a geometric series.

Let $S_n = a + ar + ar^2 + \cdots + ar^{n-1}$. If $r = 1$, $S_n = na$, which grows without bound, and so $\{S_n\}$ diverges. If $r \neq 1$, we may write

$$\begin{aligned} S_n - rS_n &= (a + ar + ar^2 + \cdots + ar^{n-1}) - (ar + ar^2 + ar^3 + \cdots + ar^n) \\ &= a - ar^n \end{aligned}$$

and so

$$S_n = \frac{a - ar^n}{1 - r} = \frac{a}{1 - r} - \frac{a}{1 - r} r^n$$

If $|r| < 1$, then $\lim_{n\to\infty} r^n = 0$, and thus

$$S = \lim_{n\to\infty} S_n = \frac{a}{1 - r}$$

If $|r| > 1$ or $r = -1$, the sequence $\{r^n\}$ diverges, and consequently so does $\{S_n\}$.

[SI*] **Positive Series**

[SI*] **Alternating Series, Absolute Convergence, and Conditional Convergence**

[SI*] **Power Series**

[SI*] **Operations on Power Series**

[SI*] **Taylor and Maclaurin Series**

[SI*] **The Taylor Approximation to a Function**

IX. Vector Calculus

[SI*] **Vector Fields with Div, Grad, Curl**
Vector calculus talks about vector-valued functions, that is, functions whose input is a real

number and whose output is a vector.

In vector calculus we will use the operator ∇, read: del, or nabla, it is an operator used in mathematics as a vector differential operator. When applied to a function defined on a one-dimensional domain, it denotes the standard derivative of the function as defined in calculus.

When applied to a field (a function defined on a multi-dimensional domain), it may denote any one of three operations depending on the way it is applied: the gradient or steepest slope of a scalar field; the divergence of a vector field; or the curl (rotation) of a vector field.

Del is a very convenient mathematical notation for those three operations (gradient, divergent, and curl) that makes many equations easier to write and remember. The del symbol can be formally defined as a vector operator whose components are the corresponding partial derivative operators. As a vector operator, it can act on scalar and vector fields in three different ways, giving rise to three different differential operations:

1. It acts on a scalar field ($f(x,y,z)$) by a formal scalar multiplication-to produce a vector field (∇f) called the gradient.
2. It acts on a vector field ($\boldsymbol{F}(x,y,z)$) by a formal dot product-to produce a scalar field ($\nabla \cdot \boldsymbol{F}$) called the divergence.
3. It acts on a vector field ($\boldsymbol{F}(x,y,z)$) by a formal cross product-to produce a vector field ($\nabla \times \boldsymbol{F}$) called the curl.

These three uses are summarized as:
Gradient

$$\text{grad } f = \nabla f \tag{2.20}$$

Physical interpretation:
Points in the direction of the greatest rate of increase of the scalar field, and its magnitude is that rate of increase.
Example:
Consider a scalar field representing temperature $T(x,y,z)$ in a garden. The gradient ∇T would give a vector at each point indicating the direction in which the temperature increases most rapidly, and its magnitude would be the rate of that increase.

Divergent

$$\text{div } \boldsymbol{v} = \nabla \cdot \boldsymbol{v} \tag{2.21}$$

Physical interpretation:
It measures the "outward flux per unit volume" of a vector field at a point. It quantifies how much a vector field is "spreading out" or "compressing" at that point.
Example:
For a vector field representing fluid flow $\boldsymbol{v}(x,y,z)$, the divergence $\nabla \cdot \boldsymbol{v}$ indicates whether the fluid is expanding (positive divergence, like a source) or contracting (negative divergence, like a sink). If $\nabla \cdot \boldsymbol{v} = 0$, the fluid is incompressible.

Curl

$$\text{curl } \boldsymbol{v} = \nabla \times \boldsymbol{v} \tag{2.22}$$

Physical interpretation:
It measurres the "rotation" or "circulation" of a vector field at a point. The direction of the

curl vector indicates the axis of rotation, and its magnitude indicates the strength of the rotation.
Example:
For a vector field representing the velocity of a fluid $\boldsymbol{v}(x,y,z)$, the curl $\nabla \times \boldsymbol{v}$ indicates the local angular velocity of the fluid. If $\nabla \times \boldsymbol{v} = \boldsymbol{0}$, the fluid is irrotational.

The divergence of a curl is zero

$$\nabla \cdot (\nabla \times \boldsymbol{F}) = 0 \tag{2.23}$$

This means that the divergence of any curl field is always zero, in linear algebra it is a dot product of an orthogonal vector.

Curl of the gradient is zero

$$\nabla \times (\nabla f) = \boldsymbol{0} \tag{2.24}$$

This means that the curl of any gradient field (which is a conservative vector field) is always the zero vector.

The div, grad, curl operators are crucial in various fields, including fluid dynamics, electromagnetism (Maxwell's equations), and heat transfer.

Definition 2.3: Mathematical Vector Fields

A vector field in 2-dimensional space is function whose value at a point (x,y) is a 2-dimensional vector $\boldsymbol{F}(x,y)$. Similarly, in 3-dimensional space, a vector field is a function $\boldsymbol{F}(x,y,z)$ whose value at the point (x,y,z) is a 3-dimensional vector.

If $\boldsymbol{F}(x,y,z)$ is a vector, it has $\boldsymbol{i},\boldsymbol{j}$, and $\boldsymbol{k}$ components. Each of these components is a scalar function of the point (x,y,z), so we will often write

$$\boldsymbol{F}(x,y,z) = F_1(x,y,z)\boldsymbol{i} + F_2(x,y,z)\boldsymbol{j} + F_3(x,y,z)\boldsymbol{k} =< F_1, F_2, F_3 > \tag{2.25}$$

For example, if $\boldsymbol{F}(x,y,z) =< x^2, xy\tan(3z), \ln(y^2) >$, then

$$\begin{aligned} F_1(x,y,z) &= x^2 \\ F_2(x,y,z) &= xy\tan(3z) \\ F_3(x,y,z) &= \ln(y^2) \end{aligned}$$

* The writing of $\boldsymbol{i},\boldsymbol{j},\boldsymbol{k}$ components in this book has the same meaning as $\hat{\boldsymbol{i}},\hat{\boldsymbol{j}},\hat{\boldsymbol{k}}$ or $\hat{\boldsymbol{x}},\hat{\boldsymbol{y}},\hat{\boldsymbol{z}}$ and will be used interchangibly.

[SI*] In three-dimensional Cartesian coordinate system $\mathbb{R}^3$ with coordinates (x,y,z) and standard basis or unit vectors of axes $\{\boldsymbol{e}_x,\boldsymbol{e}_y,\boldsymbol{e}_z\}$, del is written as

$$\nabla = \boldsymbol{e}_x\frac{\partial}{\partial x} + \boldsymbol{e}_y\frac{\partial}{\partial y} + \boldsymbol{e}_z\frac{\partial}{\partial z} = \left(\frac{\partial}{\partial x}, \frac{\partial}{\partial y}, \frac{\partial}{\partial z}\right) \tag{2.26}$$

As a vector operator, del naturally acts on scalar fields via scalar multiplication, and naturally acts on vector fields via dot products and cross products.

Then using the above definition of ∇, we may write

$$\nabla f = \left(\boldsymbol{e}_x \frac{\partial}{\partial x}\right) f + \left(\boldsymbol{e}_y \frac{\partial}{\partial y}\right) f + \left(\boldsymbol{e}_z \frac{\partial}{\partial z}\right) f = \frac{\partial f}{\partial x}\boldsymbol{e}_x + \frac{\partial f}{\partial y}\boldsymbol{e}_y + \frac{\partial f}{\partial z}\boldsymbol{e}_z \tag{2.27}$$

and

$$\nabla \cdot \boldsymbol{F} = \left(\boldsymbol{e}_x \frac{\partial}{\partial x} \cdot \boldsymbol{F}\right) + \left(\boldsymbol{e}_y \frac{\partial}{\partial y} \cdot \boldsymbol{F}\right) + \left(\boldsymbol{e}_z \frac{\partial}{\partial z} \cdot \boldsymbol{F}\right) = \frac{\partial F_x}{\partial x} + \frac{\partial F_y}{\partial y} + \frac{\partial F_z}{\partial z} \tag{2.28}$$

and

$$\begin{aligned} \nabla \times \boldsymbol{F} &= \left(\boldsymbol{e}_x \frac{\partial}{\partial x} \times \boldsymbol{F}\right) + \left(\boldsymbol{e}_y \frac{\partial}{\partial y} \times \boldsymbol{F}\right) + \left(\boldsymbol{e}_z \frac{\partial}{\partial z} \times \boldsymbol{F}\right) \\ &= \frac{\partial}{\partial x}(0, -F_z, F_y) + \frac{\partial}{\partial y}(F_z, 0, -F_x) + \frac{\partial}{\partial z}(-F_y, F_x, 0) \\ &= \left(\frac{\partial F_z}{\partial y} - \frac{\partial F_y}{\partial z}\right)\boldsymbol{e}_x + \left(\frac{\partial F_x}{\partial z} - \frac{\partial F_z}{\partial x}\right)\boldsymbol{e}_y + \left(\frac{\partial F_y}{\partial x} - \frac{\partial F_x}{\partial y}\right)\boldsymbol{e}_z \end{aligned} \tag{2.29}$$

Del can also be expressed in other coordinate systems,e.g. in cylindrical and spherical coordinates.

[SI*] **Gradient**

the vector derivative of a scalar field f is called the gradient, and it can be represented as

$$\text{grad } f = \nabla f = \frac{\partial f}{\partial x}\hat{\boldsymbol{x}} + \frac{\partial f}{\partial y}\hat{\boldsymbol{y}} + \frac{\partial f}{\partial z}\hat{\boldsymbol{z}}$$

It always points in the direction of greatest increase of f, and it has a magnitude equal to the maximum rate of increase at the point-just like a standard derivative.

Theorem 2.9: Gradient Identities

(a) $\nabla(f+g) = \nabla f + \nabla g$

(b) $\nabla(cf) = c\nabla f$, for any constant c.

(c) $\nabla(fg) = (\nabla f)g + f(\nabla g)$

(d) $\nabla\left(\frac{f}{g}\right) = \frac{(g\nabla f - f\nabla g)}{g^2}$ at points $\boldsymbol{x}$ where $g(\boldsymbol{x}) \neq 0$.

(e) $\nabla(\boldsymbol{F}\cdot\boldsymbol{G}) = \boldsymbol{F}\times(\nabla\times\boldsymbol{G}) - (\nabla\times\boldsymbol{F})\times\boldsymbol{G} + (\boldsymbol{G}\cdot\nabla)\boldsymbol{F} + (\boldsymbol{F}\cdot\nabla)\boldsymbol{G}$
Here

$$(\boldsymbol{G}\cdot\nabla)\boldsymbol{F} = \boldsymbol{G}_1\frac{\partial \boldsymbol{F}}{\partial x} + \boldsymbol{G}_2\frac{\partial \boldsymbol{F}}{\partial y} + \boldsymbol{G}_3\frac{\partial \boldsymbol{F}}{\partial z}$$

[SI*] **Divergence**

The divergence of a vector field $\boldsymbol{v}(x,y,z) = v_x\hat{\boldsymbol{x}} + v_y\hat{\boldsymbol{y}} + v_z\hat{\boldsymbol{z}}$ is a scalar field that can be represented as:

$$\text{div } \boldsymbol{v} = \nabla\cdot\boldsymbol{v} = \frac{\partial v_x}{\partial x} + \frac{\partial v_y}{\partial y} + \frac{\partial v_z}{\partial z}$$

The divergence is roughly a measure of a vector field's increase in the direction it points; but more accurately, it is a measure of that field's tendency to converge toward or diverge from a point.

Theorem 2.10: Divergent Identities

(a) $\nabla \cdot (\boldsymbol{F} + \boldsymbol{G}) = \nabla \cdot \boldsymbol{F} + \nabla \cdot \boldsymbol{G}$

(b) $\nabla \cdot (c\boldsymbol{F}) = c\nabla \cdot \boldsymbol{F}$, for any constant c.

(c) $\nabla \cdot (f\boldsymbol{F}) = (\nabla f) \cdot \boldsymbol{F} + f\nabla \cdot \boldsymbol{F}$

(d) $\nabla \cdot (\boldsymbol{F} \times \boldsymbol{G}) = (\nabla \times \boldsymbol{F}) \cdot \boldsymbol{G} - \boldsymbol{F} \cdot (\nabla \times \boldsymbol{G})$

[SI*] **Curl**

The curl of a vector field $\boldsymbol{v}(x,y,z) = v_x\hat{\boldsymbol{x}} + v_y\hat{\boldsymbol{y}} + v_z\hat{\boldsymbol{z}}$ is a vector function that can be represented as:

$$\text{curl } \boldsymbol{v} = \nabla \times \boldsymbol{v} = \left(\frac{\partial v_z}{\partial y} - \frac{\partial v_y}{\partial z}\right)\hat{\boldsymbol{x}} + \left(\frac{\partial v_x}{\partial z} - \frac{\partial v_z}{\partial x}\right)\hat{\boldsymbol{y}} + \left(\frac{\partial v_y}{\partial x} - \frac{\partial v_x}{\partial y}\right)\hat{\boldsymbol{z}}$$

The curl at a point is proportional to the on-axis torque that a tiny pinwheel would be subjected to if it were centered at that point.

Theorem 2.11: Curl Identities

(a) $\nabla \times (\boldsymbol{F} + \boldsymbol{G}) = \nabla \times \boldsymbol{F} + \nabla \times \boldsymbol{G}$

(b) $\nabla \times (c\boldsymbol{F}) = c\nabla \times \boldsymbol{F}$, for any constant c.

(c) $\nabla \times (f\boldsymbol{F}) = (\nabla f) \times \boldsymbol{F} + f\nabla \times \boldsymbol{F}$

(d) $\nabla \times (\boldsymbol{F} \times \boldsymbol{G}) = \boldsymbol{F}(\nabla \cdot \boldsymbol{G}) - (\nabla \cdot \boldsymbol{F})\boldsymbol{G} + (\boldsymbol{G} \cdot \nabla)\boldsymbol{F} - (\boldsymbol{F} \cdot \nabla)\boldsymbol{G}$
Here
$$(\boldsymbol{G} \cdot \nabla)\boldsymbol{F} = \boldsymbol{G}_1\frac{\partial \boldsymbol{F}}{\partial x} + \boldsymbol{G}_2\frac{\partial \boldsymbol{F}}{\partial y} + \boldsymbol{G}_3\frac{\partial \boldsymbol{F}}{\partial z}$$

[SI*] In SymIntegration the computation for div, grad, and curl can be handled in **src/divgradcurl.cpp**.

```
#include "symintegral/symintegrationc++.h"

#ifdef SYMBOLIC_DEFINE
#ifndef SYMINTEGRATION_CPLUSPLUS_DIVGRADCURL_DEFINE
#define SYMINTEGRATION_CPLUSPLUS_DIVGRADCURL_DEFINE

Symbolic div(const Symbolic &F, const Symbolic &x, const Symbolic &y, const
    Symbolic &z)
{
       Symbolic sol, i("i"), j("j"), k("k");

       Symbolic F1 = F.coeff(i,1);
       Symbolic F2 = F.coeff(j,1);
       Symbolic F3 = F.coeff(k,1);

       if(F1 != 0 || F2 != 0 || F3 != 0)
```

```
	{
		sol = df(F1,x) + df(F2,y) + df(F3,z);
	}
	else if(F1_hat != 0 || F2_hat != 0 || F3_hat != 0)
	{
		sol = df(F1_hat,x) + df(F2_hat,y) + df(F3_hat,z);
	}
	return sol;
}

Symbolic grad(const Symbolic &F, const Symbolic &x, const Symbolic &y, const
    Symbolic &z)
{
	Symbolic sol;

	if(F != 0)
	{
		sol = (df(F,x), df(F,y), df(F,z));
		sol = sol.transpose();
	}
	return sol;
}

...

Symbolic curl(const Symbolic &F, const Symbolic &x, const Symbolic &y, const
    Symbolic &z)
{
	Symbolic sol, i("i"), j("j"), k("k");

	Symbolic F1 = F.coeff(i,1);
	Symbolic F2 = F.coeff(j,1);
	Symbolic F3 = F.coeff(k,1);


	if(F1 != 0 || F2 != 0 || F3 != 0)
	{
		sol = (df(F3,y) - df(F2,z), -(df(F3,x) - df(F1,z)), df(F2,x)
		    - df(F1,y));
		sol = sol.transpose();
	}
	else if(F1_hat != 0 || F2_hat != 0 || F3_hat != 0)
	{
		sol = (df(F3_hat,y) - df(F2_hat,z), -(df(F3_hat,x) - df(
		    F1_hat,z)), df(F2_hat,x) - df(F1_hat,y));
		sol = sol.transpose();
	}
	return sol;
```

```
}
```

Code 29: *src/divgradcurl.cpp*

the source code can handle the input of a function along with the independent variables. For example, if we have a scalar field:

$$f(x,y,z) = -\frac{1}{2}xy + 2y + z^2$$

and we can compute the gradient with :
grad(f,x,y,z)

If we have a vector field:

$$\boldsymbol{F}(x,y,z) = y\hat{i} - x\hat{j} + yz\hat{k}$$

we can compute the divergence and curl with these functions:
div(F,x,y,z)
curl(F,x,y,z)

In SymIntegration the source code to compute the div, grad, curl for the example above will be like this:

```
#include <iostream>
#include "symintegrationc++.h"
#include <bits/stdc++.h>
#include <cmath>

using namespace std;
using namespace SymbolicConstant;

int main(void)
{
       Symbolic x("x"), y("y"), z("z"), f, f2, i("i"), j("j"), k("k");

       f = -(0.5*x)*y+2*y + z*z;
       f2 = y*i -x*j +y*z*k;
       Symbolic f2_hat = y*i -x*j +y*z*k;
       cout << "F(x,y,z) = " << f << endl;
       cout << "F2 (vector field) = " << f2_hat << endl;

       cout << "\ndiv(F2) = " << div(f2_hat,x,y,z) <<endl;
       cout << "\ngrad(F) = " << grad(f,x,y,z) <<endl;
       cout << "\ncurl(F2) = " << curl(f2_hat,x,y,z) <<endl;

       return 0;
}
```

Code 30: *Examples/test SymIntegration DivGradCurl/main.cpp*

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DivGradCurl ]# m
ake
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DivGradCurl ]# ./main
F(x,y,z) = -0.5*x*y+2*y+z^(2)
F2 (vector field) = y*i-x*j+y*z*k

div(F2) = y

grad(F) =
[ -0.5*y ]
[-0.5*x+2]
[   2*z  ]


curl(F2) =
[ z]
[ 0]
[-2]

root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DivGradCurl ]#
```

Figure 2.5: *The computation of div grad curl. (**SymIntegration/Examples/Calculus/Test SymIntegration DivGradCurl/main.cpp**).*

[SI*] The input, f, for the gradient is a scalar-valued function, while the output, ∇f, is a vector-valued function.

The input, $\boldsymbol{F}$, for the divergence is a vector-valued function, while the output, $\nabla \cdot \boldsymbol{F}$, is a scalar-valued function.

The input, $\boldsymbol{F}$, for the curl is a vector-valued function, and the output, $\nabla \times \boldsymbol{F}$, is a vector-valued function.

[SI*] If $\boldsymbol{F}$ denotes the velocity field for a fluid, then div $\boldsymbol{F}$ at a point $\boldsymbol{p}$ measures the tendency of that fluid to diverge away from $\boldsymbol{p}$ (div $\boldsymbol{F} > 0$) or accumulate toward $\boldsymbol{p}$ (div $\boldsymbol{F} < 0$).

On the other hand, curl $\boldsymbol{F}$ picks out the direction of the axis about which the fluid rotates (curls) most rapidly, and $||$curl $\boldsymbol{F}||$ is a measure of the speed of this rotation. The direction of the rotation is according to the right-hand rule.

[SI*] **Laplacian**
The scalar function $\text{div}(\text{grad } f) = \nabla \cdot \nabla f$ (also written $\nabla^2 f$) is called the Laplacian, and a function f satisfying

$$\nabla^2 f = 0$$

is said to be harmonic, this concept is important in physics.

The Laplace operator is a scalar operator that can be applied to either vector or scalar fields.

The Laplacian of a scalar field $f(x, y, z)$ is the scalar-valued function:

$$\triangle f = \nabla^2 f = \nabla \cdot \nabla f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} + \frac{\partial^2 f}{\partial z^2} \tag{2.30}$$

The Laplacian of a vector field $\boldsymbol{F}(x, y, z)$ is the vector field:

$$\triangle \boldsymbol{F} = \nabla^2 \boldsymbol{F} = \nabla \cdot \nabla \boldsymbol{F} = \frac{\partial^2 \boldsymbol{F}}{\partial x^2} + \frac{\partial^2 \boldsymbol{F}}{\partial y^2} + \frac{\partial^2 \boldsymbol{F}}{\partial z^2} \tag{2.31}$$

so if we have a vector field that can be written like this:

$$\boldsymbol{F}(x,y,z) = F_1(x,y,z)\hat{i} + F_2(x,y,z)\hat{j} + F_3(x,y,z)\hat{k}$$

then the Laplacian can also be written like this:

$$\begin{aligned}
\nabla^2\boldsymbol{F} &= \nabla^2 F_1\hat{i} + \nabla^2 F_2\hat{j} + \nabla^2 F_3\hat{k} \\
\nabla^2\boldsymbol{F} &= (\nabla^2 F_1, \nabla^2 F_2, \nabla^2 F_3) \\
\nabla^2\boldsymbol{F} &= \left(\frac{\partial^2 F_1}{\partial x^2} + \frac{\partial^2 F_1}{\partial y^2} + \frac{\partial^2 F_1}{\partial z^2}\right)\hat{i} + \left(\frac{\partial^2 F_2}{\partial x^2} + \frac{\partial^2 F_2}{\partial y^2} + \frac{\partial^2 F_2}{\partial z^2}\right)\hat{j} \\
&\quad + \left(\frac{\partial^2 F_3}{\partial x^2} + \frac{\partial^2 F_3}{\partial y^2} + \frac{\partial^2 F_3}{\partial z^2}\right)\hat{k} \\
\nabla^2\boldsymbol{F} &= \left(\frac{\partial^2 F_1}{\partial x^2} + \frac{\partial^2 F_1}{\partial y^2} + \frac{\partial^2 F_1}{\partial z^2}, \frac{\partial^2 F_2}{\partial x^2} + \frac{\partial^2 F_2}{\partial y^2} + \frac{\partial^2 F_2}{\partial z^2}, \frac{\partial^2 F_3}{\partial x^2} + \frac{\partial^2 F_3}{\partial y^2} + \frac{\partial^2 F_3}{\partial z^2}\right)
\end{aligned} \tag{2.32}$$

The Laplacian of the velocity is related to the viscous forces in a fluid, it is also part of the heat equation, which describes how temperature changes over time, the Laplacian is also used in Poisson's equation, which relates electric potential to charge density.

In essence, the Laplacian of a vector field provides valuable information about the "smoothness" or "curvature" of the vector field, highlighting regions where the field changes rapidly.

The Laplacian is ubiquitous throughout modern mathematical physics, appearing for example in Laplace's equation, Poisson's equation, the heat equation, the wave equation, and the Schrödinger equation.

To compute the Laplacian from a defined function $f(x,y,z)$ with SymIntegration we can use :
laplacian(f,x,y,z) (it can handle the input of scalar field or vector field)

Theorem 2.12: Laplacian Identities

(a) $\nabla^2(f+g) = \nabla^2 f + \nabla^2 g$

(b) $\nabla^2(cf) = c\nabla^2 f$, for any constant c.

(c) $\nabla^2(fg) = f\nabla^2 g + 2\nabla f \cdot \nabla g + g\nabla^2 f$

Theorem 2.13: Degree Two Identities

(a) $\nabla \cdot (\nabla \times \boldsymbol{F}) = 0$ (divergence of a curl)

(b) $\nabla \times (\nabla f) = 0$ (curl of gradient)

(c) $\nabla \cdot (f\{\nabla g \times \nabla h\}) = \nabla f \cdot (\nabla g \times \nabla h)$

(d) $\nabla \cdot (f\nabla g - g\nabla f) = f\nabla^2 g - g\nabla^2 f$

(e) $\nabla \times (\nabla \times \boldsymbol{F}) = \nabla(\nabla \cdot \boldsymbol{F}) - \nabla^2\boldsymbol{F}$ (curl of curl)

[SI*] **Hessian Matrix**

Hessian matrix is a matrix that maps a function of n variables into $n \times n$ square matrix. Consider a function

$$f(x_1, x_2, \ldots, x_n)$$

the Hessian matrix of f is the matrix-valued function $H : \mathbb{R}^n \to \mathbb{R}^{n \times n}$ defined by

$$H = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1 \partial x_1} & \frac{\partial^2 f}{\partial x_2 \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_n \partial x_1} \\ \frac{\partial^2 f}{\partial x_1 \partial x_2} & \frac{\partial^2 f}{\partial x_2 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n \partial x_2} \\ \vdots & \vdots & \cdots & \vdots \\ \frac{\partial^2 f}{\partial x_1 \partial x_n} & \frac{\partial^2 f}{\partial x_2 \partial x_n} & \cdots & \frac{\partial^2 f}{\partial x_n \partial x_n} \end{bmatrix} \tag{2.33}$$

we can generate the hessian matrix from a defined function f with SymIntegration with :
hessianmatrix(f,x,y) (for 2 variables function)
hessianmatrix(f,x,y,z) (for 3 variables function)

Below is the code to show hessian matrix for function of 2 variables, $f(x, y)$, we provide the function to create the Hessian matrix for function that has up to 3 variables for now, and it is handled in **src/divgradcurl.cpp**.

```
#include <iostream>
#include <iomanip> // to declare the manipulator of setprecision()
#include <fstream>
#include <bits/stdc++.h> //for setw(6) at display() function
#include <vector> // For std::vector (example container)
#include "symintegrationc++.h"
#include <algorithm> // For std::sort

#include <chrono>
#include <string>
#include <bitset>

using namespace std::chrono;
using namespace std;

int main(int argc, char** argv)
{
       // Get starting timepoint
       auto start = high_resolution_clock::now();

       Symbolic x("x"), y("y");

       Symbolic f = 2*sin(3*x) + 5 + x*x*y + cos(y);
       cout << "\nf(x,y) = " << f << endl;
       cout << "\nH = " << hessian(f,x,y) << endl;

       // Get ending timepoint
       auto stop = high_resolution_clock::now();
```

```
        auto duration = duration_cast<microseconds>(stop - start);

        cout << "Time taken by function: " << duration.count() << "
            microseconds" << endl;

        return 0;
}
```

Code 31: *Examples/test Hessian Matrix/main.cpp*

```
root [ ~/SourceCodes/CPP/C++ Create Library/TEST Hessian ]# ./main

f(x,y) = 2*sin(3*x)+x^(2)*y+cos(y)+5

H =
[-18*sin(3*x)+2*y        2*x      ]
[       2*x            -cos(y)    ]

Time taken by function: 32521 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/TEST Hessian ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb  main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/TEST Hessian ]# ./main

f(x,y) = 2*sin(3*x)+x^(2)*y+cos(y)+5

H =
[-18*sin(3*x)+2*y        2*x      ]
[       2*x            -cos(y)    ]

Time taken by function: 31283 microseconds
```

Figure 2.6: *The computation of Hessian matrix. (**SymIntegration/Examples/Linear Algebra/Test Hessian Matrix/main.cpp**).*

X. Complex Analysis

[SI*] Complex analysis, traditionally known as the theory of functions of a complex variable, is the branch of mathematical analysis that investigates functions of complex numbers. It is helpful in many branches of mathematics, including algebraic geometry, number theory, analytic combinatorics, applied mathematics; as well as in physics, including the branches of hydrodynamics, thermodynamics, and particularly quantum mechanics. By extension, use of complex analysis also has applications in engineering fields such as nuclear, aerospace, mechanical and electrical engineering.

As a differentiable function of a complex variable is equal to its Taylor series (that is, it is analytic), complex analysis is particularly concerned with analytic functions of a complex variable (that is, holomorphic functions).

[SI*] Based on this book [12], the complex numbers have their historical origin in the 16th century when they were created during attempts to solve algebraic equations. G. Cardano (1545) has already introduced formal expressions as for instance $5 \pm \sqrt{-15}$, in order to express solutions of quadratic and cubic equations. Around 1560 R. Bombelli computed systematicaly using such expressions and found 4 as a solution of the equation $x^3 = 15x + 4$ in the disguised form

$$4 = \sqrt[3]{2 + \sqrt{-121}} + \sqrt[3]{2 - \sqrt{-121}}$$

Also in the work of G.W. Leibniz (1675) one can find equations of this kind, e.g.

$$\sqrt{1+\sqrt{-3}}+\sqrt{1-\sqrt{-3}}=\sqrt{6}$$

In the year 1777 L. Euler introduced the notation $i=\sqrt{-1}$ for the imaginary unit.
The terminology "complex number" is due to C.F. Gauss (1831. The rigorous introduction of complex numbers as pairs of real numbers goes back to W.R. Hamilton (1837).
Another example for the fruitful use of complex numbers is related to Fourier series. Following Euler (1748) one can combine the real angular functions sine and cosine, and obtain the "exponential function"

$$e^{ix}=\cos\ x+i\sin\ x$$

Then the addition theorems for sine and cosine reduce to the simple formula

$$e^{i(x+y)}=e^{ix}e^{iy}$$

In particular,

$$(e^{ix})^n=e^{inx},\quad n\in\mathbb{Z}$$

The Fourier series of a sufficiently smooth function f, defined on the real line, with period 1, can be written in terms of such expressions as

$$f(x)=\sum_{n=-\infty}^{\infty}a_n e^{2\pi inx}$$

Here it is irrelevant whether f is real or complex valued.
In these examples the complex numbers serve as useful, but ultimately dispensable tools. New aspects come into play when we consider complex valued functions depending on a complex variable, that is when we start to study functions $f:D\to\mathbb{C}$ with two-dimensional domains D systematically. The dimension two is ensured when we restrict to open domains of definition $D\subset\mathbb{C}$. Analogously to the situation in real analysis one introduces the notion of complex differentiability by requiring the existence of the limit

$$f'(a)=\lim_{z\to a,z\neq a}\frac{f(z)-f(a)}{z-a}$$

for all $a\in D$. It turns out that this notion behaves much more drastically than real differentiability. Complex differentiable functions (defined on open domains) ae also called analytic functions.
Many classical functions from real analysis can be analytically extended to complex analysis. It tuns out that these extensions are unique, as for instance in the case

$$e^{x+iy}=e^x e^{iy}$$

Fom the relation

$$e^{2\pi i}=1$$

it follows that the complex exponential function is periodic with the purely imaginary period $2\pi i$. This observation is fundamental for the complex analysis. As a consequence one can observe further phenomena:

1. The complex logarithm cannot be introduced as the unique inverse function of the exponential function in a natural way. It is a priori determined only up to a multiple of $2\pi i$.

2. The function $\frac{1}{z}$ ($x \neq 0$) does not have any primitive in the punctured complex plane. A related fact is the following: the path integral of $\frac{1}{z}$ with respect to a circle line centered in the origin and oriented anticlockwise yields the non-zero value

$$\oint_{|z|=r} \frac{1}{z} dz = 2\pi i, \quad r > 0$$

Central results of complex analysis, like e.g. the Residue theorem, are nothing but a highly generalized version of these statements.
Real functions often show their true nature first after considering their analytic extensions. For instance, in the real theory it is not directly transparent why the power series representation

$$\frac{1}{1+x^2} = 1 - x^2 + x^4 - x^6 + \cdots - \ldots$$

is valid only for $|x| < 1$. In the complex theory this phenomenon becomes more understandable, simply because the considered function has singularities in $\pm i$. Then its power series representation is valid in the biggest open disk excluding the singularities, namely the unit disk.
In the real theory, it is also hard to understand why the Taylor series around 0 of the C^∞ function

$$f(x) = \begin{cases} e^{-\frac{1}{x^2}}, & x \neq 0 \\ 0, & x = 0 \end{cases}$$

converges for all $x \in \mathbb{R}$, but does not represent the function in any point other than zero. In the complex theory this phenomenon becomes understandable, because the function $e^{-\frac{1}{z^2}}$ has an essential singularity in zero.

[SI*] Let $x, y \in \mathbb{R}$, the complex number $z \in \mathbb{C}$ can be defined as

$$z = x + iy \tag{2.34}$$

Some important properties of complex numbers:

- $\mathrm{Re}(z) = x$ and $\mathrm{Im}(z) = y$ are called the real part of z and the imaginary part of z, respectively.
- $|z| = \sqrt{x^2 + y^2}$ is called the modulus (or absolute value) of z.
- $\overline{z} = x - yi$ is called the complex conjugate of z.
- $z\overline{z} = x^2 + y^2 = |z|^2$
- The angle θ is called an argument of z.
- $\mathrm{Re}(z) = |z| \cos\ \theta$
- $\mathrm{Im}(z) = |z| \sin\ \theta$
- $z = |z|(\cos\ \theta + i\ \sin\ \theta)$ is called the polar form of z.

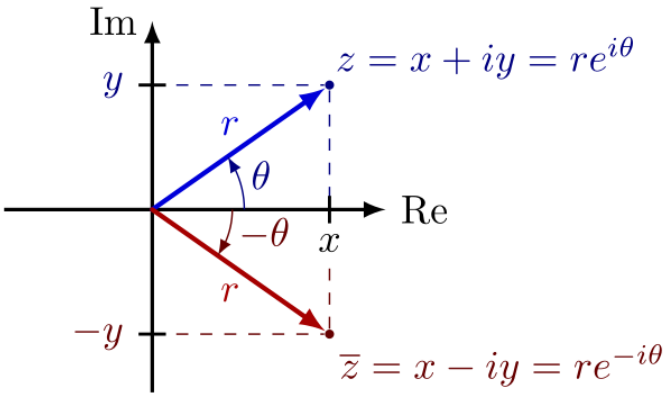

Figure 2.7: *The complex number can be represented as $z = x + iy = re^{i\theta}$, with $r = |z|$ and the angle θ is the argument of z.*

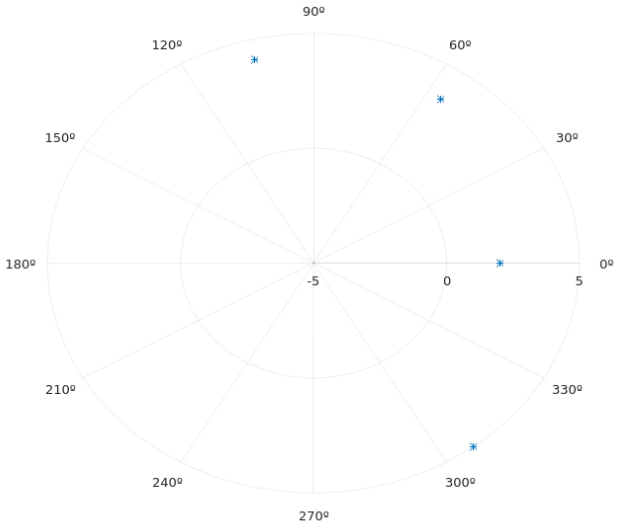

Figure 2.8: *The polar plot of several complex numbers: $2 + 3i, 2, -1 + 4i, 3 - 4i$ (**Hamzstplot/Examples with Makefile/Complex Analysis/Plot 2D Complex Numbers/main.cpp**).*

[SI*] Some important properties of complex function:

- If a complex function is differentiable once at a point and in a small area around it (holomorphic), it is automatically analytic.

[SI*] **Pole in Complex Analysis**
In the study of complex numbers, a pole is a clean, predictable type of point where a function blows up to infinity. If you have a function $f(z) = \frac{1}{z}$, plugging in $z = 0$ results in division by zero, causing the value to shoot off to infinity. This point $z = 0$ is a pole. A point z_0 is a pole of a function $f(z)$ if the reciprocal function $\frac{1}{f(z)}$ is holomorphic (smooth and differentiable) near z_0 and equals zero at z_0.
The power of the zero in the denominator tells the order (or multiplicity) of the pole. For example, $f(z) = \frac{1}{z^3}$ has a pole of order 3 at $z = 0$.

[SI*] **Euler's Formula**
Every positive number x can be written as $x = e^{\ln(x)}$, if we raise the expression to the power of i we will have

$$x^i = (e^{\ln(x)})^i = e^{i\ln(x)}$$

Apply Euler's formula ($e^{i\theta} = \cos\theta + i\sin\theta$), substitute $\theta = \ln(x)$ into the formula

$$e^{i\ln(x)} = \cos(\ln(x)) + i\sin(\ln(x))$$

with the real part $Re(x^i) = \cos(\ln(x))$ and the imaginary part $Im(x^i) = \sin(\ln(x))$. If we isolate the cosine function using both x^i and x^{-i}, we get the true exponential identity:

$$\cos(\ln(x)) = \frac{x^i + x^{-i}}{2}$$

i. Functions in SymIntegration Related to Complex Analysis

[SI*] We will list the basic functions that can be used related to complex numbers and complex analysis:

complexnumber_analysis(z)
to compute the modulus $|z|$, polar angle of the complex number z in radians and in degree, and write the complex number in polar form.

```
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Complex Analysis/Test SymIntegration Complex Number ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Complex Analysis/Test SymIntegration Complex Number ]# ./main

The complex number c is: (2,-3)

Modulus (|z|) = 3.60555
Polar angle (radians): -0.982794
Polar angle (degrees): -56.3099
Polar form :
z = |z| (cos ϕ + i sin ϕ) = -3*i+2
```

Figure 2.9: *The modulus, angle, and polar form of the complex number $z = 2 - 3i$ (**SymIntegration/Examples/Complex Analysis/Test SymIntegration Complex Number/main.cpp**).*

complexdistance(z_1, z_2)
to compute distance in the complex plane with z_1 and z_2 as the complex numbers with data type **std::complex<double>**.

We have defined **PolynomialComplex** as a class in **dsolve.h** when we were working on Frobenius method, instead of putting it in **complexanalysis.h**, nevertheless it is stays the same we can still use the class for complex analysis computation anywhere we need. This class can be used to create a complex polynomial, for example if we want to define this polynomial

$$P(x) = (1 + 0i) + (2 + i)x + (3 + 5i)x^2$$

Then we will need to use this to create such function / polynomial:
PolynomialComplex poly(1.0, 0.0, 2.0, 1.0, 3.0, 5.0)

Several functions related to the **PolynomialComplex poly** that has been defined:

poly.print()
to print the polynomial nicely in the command line interface.

poly.evaluateAt(z)
if we have defined a complex number (e.g. **complex<double>z(2.0,3.0)**) then this function can evaluate $P(z)$.
poly.shift(s)
if we have defined a complex number (e.g. **complex<double>s(4.0,1.0)**) then this function can evaluate $P(x+s)$, a shifted complex polynomial.

```
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Complex Analysis/T
est SymIntegration Complex Polynomial Class ]# ./main
Polynomial P(x): (1,0) * x^0 + (2,1) * x^1 + (3,5) * x^2
Evaluation at (2,3) is: (-73,19)

Shifted Polynomial P(x + (4+i)): (13,105) * x^0 + (16,47) * x^1 + (3,5) * x^2

Time taken by function: 311 microseconds
```

Figure 2.10: *The computation of complex polynomial $P(x)$ at $z = 2 + 3i$ and the shifted polynomial $P(x+s)$ with $s = 4 + i$ (**SymIntegration/Examples/Complex Analysis/Test SymIntegration Complex Polynomial Class/main.cpp**).*

Chapter 3

Differential Equations, Partial Differential Equations, and Boundary Value Problems

"After Odin's defeat atop Yggdrasil and the revelation of Lezard's true nature, Freya quickly teleports there in a frantic state, telling Odin that Lezard's presence was the distortion she felt at Dipan" - ***Freya (Valkyrie Profile 2: Silmeria)***

"Everyone's favorite fighting fairy godmother is still kicking, taking out Ether Strike hits for her godfather Odin. The heretofore unmatched fury of her scowl derives from her lack of lines in the main story" - ***Freya (Valkyrie Profile: Covenant of the Plume)***

Dıfferential equations are of interest to engineers and other nonmathematicians, such as physicists. It is because of the possibility of using them to investigate a wide variety of problems in the physical, biological, and social sciences.

For example consider this Newton's law formula:

$$F = ma \tag{3.1}$$

If you read the Physics book for undergraduate degree you will learn that it is not simply just $F = ma$, but it has a derivative as well

$$F = m\frac{dv}{dt} \tag{3.2}$$

Then add with drag force the equation will become

$$m\frac{dv}{dt} = mg - \gamma v \tag{3.3}$$

The Eq. (3.3) is what we call a differential equation that model an object falling in the atmosphere near sea level. From this equation we are able to determine the velocity at a given time, $v(t)$, from known g, m, γ. You can read more on this book [7], it is the first example of the book.

We will assume that you already read some books or learn from anywhere about differential equations, so we will ony focus on the SymIntegration function and capability that can help you

to compute the solution of a differential equation.

Starting on June 23rd, 2025, we have created a new function **dsolve** in SymIntegration, it is still very fresh and in an infant stage and just able to solve a very simple first order linear ordinary differential equations.

I. Solving First Order Ordinary Differential Equations

[SI*] **Method of Integrating Factors**
Solve the initial value problem

$$2y' + ty = 2 \tag{3.4}$$

with the initial value condition

$$y(0) = 1 \tag{3.5}$$

Solution:
We will convert the differential equation Eq. (3.4) into the standard form to become

$$y' + \left(\frac{t}{2}\right) y = 1 \tag{3.6}$$

Thus $p(t) = t/2$, and the integrating factor is

$$\mu(t) = e^{\int p(t)\ dt} = e^{\frac{t^2}{4}}$$

Then multiply Eq. (3.6) by $\mu(t)$ so that

$$e^{\frac{t^2}{4}} y' + \left(\frac{t}{2}\right) e^{\frac{t^2}{4}} y = e^{\frac{t^2}{4}} \tag{3.7}$$

Then integrate both sides of equation we will obtain

$$e^{\frac{t^2}{4}} y = \int e^{\frac{t^2}{4}}\ dt + c \tag{3.8}$$

thus

$$y = e^{-\frac{t^2}{4}} \int_0^t e^{\frac{t^2}{4}}\ ds + ce^{-\frac{t^2}{4}} \tag{3.9}$$

You can compute the value of the constant c by inputting the initial condition of $y(0) = 1$.

Now, in SymIntegration we will need the user to input the equation in term of $\frac{dy}{dx} = f(x, y)$, we will input the $f(x, y)$ and the coefficient of $\frac{dy}{dx}$ has to be 1.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DSolve ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DSolve ]# ./main
f(x) = -0.5*x*y+1
DSolve for f(x,y).

y(x) = 1/2*π^(1/2)*(erf(x*(-0.25)^(1/2)))*(-0.25)^(-1/2)*e^(-0.25*x^(2))+C*e^(-0.25*x^(2))
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DSolve ]#
```

Figure 3.1: *The newly built function* ***dsolve*** *in SymIntegration is able to compute the solution $y(x)$ for the ordinary differential equation of $2y' + ty = 2$ (**SymIntegration/Examples/Differential Equations/Test SymIntegration DSolve/main.cpp**).*

[SI*] For the coding in SymIntegration, we add a new C++ file:
src/dsolve.cpp
This is the very first beginning of the dsolve by June 23rd, 2025:

```
#include "symintegral/symintegrationc++.h"

#ifdef SYMBOLIC_DEFINE
#ifndef SYMINTEGRATION_CPLUSPLUS_DSOLVE_DEFINE
#define SYMINTEGRATION_CPLUSPLUS_DSOLVE_DEFINE

Symbolic dsolve(const Symbolic &fx, const Symbolic &y, const Symbolic
    &x)
{
       Symbolic dsol, mu, C("C");

       if(fx != 0)
       {
              list<Equations> eq;
              list<Equations>::iterator i;
              UniqueSymbol a, b, c, d;
              eq = (a*x*y + d).match(fx, (a,d));
              for(i=eq.begin(); i!=eq.end(); ++i)
              {
                     try {
                            Symbolic ap = rhs(*i, a), dp = rhs(*i, d);
                            mu = exp(integrate(-ap*x,x));
                            dsol = (integrate(mu*dp,x))/(mu) + (C)/(mu)
                                ;
                     } catch(const SymbolicError &se) {}
              }
       }
       return dsol;
}

#endif
#endif
```

Code 32: *src/dsolve.cpp*

and the include file as well **include/symintegral/dsolve.h**

```
#ifndef SYMINTEGRATION_CPLUSPLUS_DSOLVE

#ifdef SYMBOLIC_FORWARD
#ifndef SYMINTEGRATION_CPLUSPLUS_DSOLVE_FORWARD
#define SYMINTEGRATION_CPLUSPLUS_DSOLVE_FORWARD

#endif
#endif

#ifdef SYMBOLIC_DECLARE
#define SYMINTEGRATION_CPLUSPLUS_DSOLVE
```

```
#ifndef SYMINTEGRATION_CPLUSPLUS_DSOLVE_DECLARE
#define SYMINTEGRATION_CPLUSPLUS_DSOLVE_DECLARE

Symbolic dsolve(const Symbolic &, const Symbolic &, const Symbolic &);

#endif
#endif


#endif
```

Code 33: *include/symintegral/dsolve.h*

Then we also need to add a new function **erf**, as the integral of e^{x^2} is not an easy one to be computed and defined, it is like this

$$\int e^{ax^2}\,dx = \frac{\sqrt{\pi}\mathrm{erf}(x\sqrt{-a})}{2\sqrt{-a}}, \quad a \neq 0 \tag{3.10}$$

To handle that, the need to return the $\int e^{ax^2}\,dx$ correctly in SymIntegration we modify these files:

- src/symintegrationc++.cpp

```
...
...

Symbolic erf(const Symbolic &s)
{ return Erf(s); }

...
...
```

Code 34: *src/symintegrationc++.cpp*

- src/functions.cpp

```
...
...


////////////////////////////////////
// Implementation of Erf //
////////////////////////////////////

Erf::Erf(const Erf &s) : Symbol(s) {}

Erf::Erf(const Symbolic &s) : Symbol(Symbol("erf")[s]) {}

Simplified Erf::simplify() const
{
       const Symbolic &s = parameters.front().simplify();
```

```
        if(s == 0) return Number<int>(0);
        return *this;
}

Symbolic Erf::df(const Symbolic &s) const
{ return Derivative(*this,s); }

Symbolic Erf::integrate(const Symbolic &s) const
{
        const Symbolic &x = parameters.front();
        if(x == s) return Integral(*this,s);
        if(df(s) == 0) return Integral(*this,s);
        return Integral(*this,s);
}

...
...
```

Code 35: *src/functions.cpp*

- include/symintegral/functions.h

```
...
...

class Erf;

...
...

class Erf: public Symbol
{
        public: Erf(const Erf&);
        Erf(const Symbolic&);

        Simplified simplify() const;
        Symbolic df(const Symbolic&) const;
        Symbolic integrate(const Symbolic&) const;

        Cloning *clone() const { return Cloning::clone(*this); }
};

...
...
```

Code 36: *include/symintegral/functions.h*

- include/symintegral/symintegrationc++.h

```
...
...
```

```
Symbolic erf(const Symbolic &);

...
...
```

Code 37: *include/symintegral/symintegrationc++.h*

[SI*] **Separable Equations**
We can integrate a differential equation function that is classified as separable equations if they have this form:

$$\frac{dy}{dx} = \frac{f(x)}{g(y)} \tag{3.11}$$

with $f(x)$ is a function of x only and $g(y)$ is a function of y only. Thus we can write the differential equation in the form of

$$g(y)\,dy - f(x)\,dx = 0 \tag{3.12}$$

Such an equation is said to be separable. A separable equation can be solved by integrating the functions f and g.

[SI*] **Example:**
Show that the equation

$$\frac{dy}{dx} = \frac{x^2}{1-y^2}$$

is separable, and then find an equation for its integral curves.
Solution:

$$\begin{aligned} \frac{dy}{dx} &= \frac{x^2}{1-y^2} \\ 1-y^2\,dy &= x^2\,dx \\ y - \frac{y^3}{3} &= \frac{x^3}{3} + c \\ -x^3 + 3y - y^3 &= c \end{aligned}$$

where c is an arbitrary constant and $-x^3 + 3y - y^3 = c$ is the equation for the integral curves in the form of implicit function.

[SI*] **Homogeneous Ratio Equations**
If the right side of the equation

$$\frac{dy}{dx} = f(x,y)$$

can be expressed as a function of the ratio y/x only, then the equation is said to be a homogeneous ratio equation. Such equations can always be transformed into separable equations by a change of the dependent variable.

[SI*] **Example:**
Find the solution of

$$\frac{dy}{dx} = \frac{y-4x}{x-y}$$

Solution:
First we will make it into the form of y/x

$$\frac{dy}{dx} = \frac{y-4x}{x-y} = \frac{(y/x)-4}{1-(y/x)}$$

Make the change of variables

$$v = \frac{y}{x}$$
$$\frac{dv}{dx} = \frac{1}{x}\frac{dy}{dx} - \frac{y}{x^2}$$
$$y = xv$$
$$\frac{dy}{dx} = x\frac{dv}{dx} + \frac{y}{x} = x\frac{dv}{dx} + v$$

As a result, the ODE becomes

$$\begin{aligned}
\frac{dy}{dx} &= \frac{(y/x)-4}{1-(y/x)} \\
x\frac{dv}{dx} + v &= \frac{v-4}{1-v} \\
x\frac{dv}{dx} &= \frac{v-4}{1-v} - v \\
&= \frac{v-4}{1-v} - \frac{v(1-v)}{1-v} \\
&= \frac{v^2-4}{1-v} \\
\frac{1-v}{v^2-4}dv &= \frac{dx}{x} \\
\int \frac{1-v}{(v+2)(v-2)}dv &= \int \frac{dx}{x} \\
\int \left(-\frac{3}{4(v+2)} - \frac{1}{4(v-2)}\right)dv &= \int \frac{dx}{x} \\
-\frac{3}{4}\ln|v+2| - \frac{1}{4}\ln|v-2| &= \ln|x| + C \\
-\frac{3}{4}\ln|\frac{y}{x}+2| - \frac{1}{4}\ln|\frac{y}{x}-2| &= \ln|x| + C
\end{aligned}$$

We can write the solution as an implicit function like this

$$f(x,y) = -\frac{3}{4}\ln|\frac{y}{x}+2| - \frac{1}{4}\ln|\frac{y}{x}-2| - \ln|x| - C$$

Remember that C is a constant that depends on the initial value.

The C++ code to solve the homogeneous ratio equations is using a function named **dsolveseparable** and we need to input it like this
dsolveseparable(g(x,y),f(x,y),y,x)
with

$$\frac{dy}{dx} = \frac{f(x,y)}{g(x,y)}$$

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DSolve ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DSolve ]# ./main


DSolve for dy/dx = (y-4x) / (x-y)

f(x,y) = -0.25*ln(y*x^(-1)-2)-0.75*ln(y*x^(-1)+2)-ln(x)-C
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DSolve ]#
```

Figure 3.2: *The newly built function **dsolveseparable** in SymIntegration is able to compute the solution $f(x,y)$ for homogeneous ratio equations (**SymIntegration/Examples/Differential Equations/Test SymIntegration DSolve/main.cpp**).*

Now, to draw a direction field and some integral curves, recall that the right hand side of

$$\frac{dy}{dx} = \frac{y-4x}{x-y}$$

depends only on the ratio $\frac{y}{x}$. This means that the integral curves have the same slope at all points on any given straight line through the origin, although the slope changes from one line to another. Therefore the direction field and the integral curves are symmetric with respect to the origin.

List of Solvable Linear ODEs with dsolve

We will list all the first order linear ordinary differential equations that can be solved with **dsolve** function in SymIntegration here, they are:

$$\begin{aligned} ay' + ty &= b \\ aty' + by &= ct^2 \\ ay' + by &= ce^{dt} \\ ay' + by &= ct + d \\ ay' + by &= c \\ ay' + by &= 0 \end{aligned}$$

with a, b, c, d are integers, e is reserved for exponential symbol, the solution will be $y(t)$. Remember when inputting to SymIntegration, you have to input the function $f(t,y)$ with this criteria

$$y' = f(t,y) \tag{3.13}$$

input the right hand side type of function in term of variables t and y only, and the coefficient of y' is 1.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DSolve ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DSolve ]# ./main
f(t,y) = -0.5*t*y+1

DSolve for 2y' + ty = 2

y(t) = 1/2*π^(1/2)*(erf(t*(-0.25)^(1/2)))*(-0.25)^(-1/2)*e^(-0.25*t^(2))+C*e^(-0.25*t^(2))

DSolve for ty' + 2y = 4t^2

y(t) = t^(2)+C*t^(-2)

DSolve for y' + 0.5y = 0.5*exp(t/3)

y(t) = 0.6*e^(0.333333*t)+C*e^(-0.5*t)

DSolve for y' - 2y = 4-t

y(t) = 1/2*t+C*e^(2*t)-7/4

DSolve for Q' = r/4 - rQ/100

Q(t) = C*e^(-1/100*t*r)+25

DSolve for S' = rS

S(t) = C*e^(t*r)
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DSolve ]#
```

Figure 3.3: *The function **dsolve** in SymIntegration able to compute the solution $y(t)$ for several examples of linear first order ordinary differential equations (**SymIntegration/Examples/Differential Equations/Test SymIntegration DSolve/main.cpp**).*

List of Solvable Linear ODEs with dsolveseparable

We will list all the first order linear ordinary differential equations that can be solved with **dsolveseparable** function in SymIntegration here, they are:

$$\frac{dy}{dx} = \frac{f(x)}{g(y)}$$
$$\frac{dy}{dx} = \frac{f(x,y)}{g(x,y)}$$

We create another function besides **dsolve** to make it easier to separate certain type of differential equation, with this **dsolveseparable** we need to input the function in the numerator and denominator. It is able to compute the simple separable equations and also the homogeneous ratio equations (we use this term instead of only 'homogeneous equations' that is written from this book [7] to avoid clashing of name at the next section).

the origin:

31. $\frac{dy}{dx} = \frac{x^2+xy+y^2}{x^2}$
32. $\frac{dy}{dx} = \frac{x^2+3y^2}{2xy}$
33. $\frac{dy}{dx} = \frac{4y-3x}{2x-y}$
34. $\frac{dy}{dx} = -\frac{4x+3y}{2x+y}$
35. $\frac{dy}{dx} = \frac{x+3y}{x-y}$
36. $(x^2+3xy+y^2)\,dx - x^2\,dy = 0$
37. $\frac{dy}{dx} = \frac{x^2-3y^2}{2xy}$
38. $\frac{dy}{dx} = \frac{3y^2-x^2}{2xy}$

Figure 3.4: *The problems from book [7] with regards to the homogeneous ratio equation that can be solved with SymIntegration' **dsolveseparable**.*

```
]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DSolveseparable
]# ./main

DSolveseparable for dy/dx = (x^2) / (1-y^2)

f(x,y) = -1/3*y^(3)+y-1/3*x^(3)-C

DSolveseparable for dy/dx = (y-4x) / (x-y)

f(x,y) = -0.25*ln(y*x^(-1)-2)-0.75*ln(y*x^(-1)+2)-ln(x)-C

DSolveseparable for dy/dx = (x^2+xy+y^2) / (x^2)

f(x,y) = atan(y*x^(-1))-ln(x)-C

DSolveseparable for dy/dx = (4y-3x) / (2x-y)

f(x,y) = 0.25*ln(y*x^(-1)-1)-1.25*ln(y*x^(-1)+3)-ln(x)-C

DSolveseparable for dy/dx = (4x+3y) / (2x+y)

f(x,y) = -0.333333*ln(y*x^(-1)+1)-0.666667*ln(y*x^(-1)+4)-ln(x)-C

DSolveseparable for dy/dx = (x+3y) / (x-y)

f(x,y) = -ln(y*x^(-1)+1)-2*(y*x^(-1)+1)^(-1)-ln(x)-C

DSolveseparable for dy/dx = (x^2+3xy+y^2) / (x^2)

f(x,y) = -(y*x^(-1)+1)^(-1)-ln(x)-C

DSolveseparable for dy/dx = (x^2 - 3y^2) / (2xy)

f(x,y) = -0.2*ln(-5*y^(2)*x^(-2)+1)-ln(x)-C

DSolveseparable for dy/dx = (3y^2 - x^2 ) / (2xy)

f(x,y) = ln(y^(2)*x^(-2)-1)-ln(x)-C
```

Figure 3.5: *The solutions with SymIntegration' **dsolveseparable** of the problems from book [7] at the end of chapter 2.2.*

List of Solvable Linear ODEs with dsolvelogistic

We will list all the first order linear ordinary differential equations that can be solved with **dsolvelogistic** function in SymIntegration here, they are:

$$\frac{dy}{dt} = r\left(1 - \frac{y}{K}\right)y$$
$$\frac{dy}{dt} = -r\left(1 - \frac{y}{T}\right)y$$

with r, K, T as the parameters. We need to input the function in $\frac{dy}{dt}$ term. The function needs 7 input

dsolvelogistic(y',y,y0, t, r, K, T)

We can define $y' = \frac{dy}{dt}$ as either $r\left(1 - \frac{y}{K}\right)y$ or $-r\left(1 - \frac{y}{T}\right)y$,it will return the general solution with initial value problem of $y_0 = y_0$.

The more difficult case occurs in the case for logistic growth with a threshold where the differential equation is

$$\frac{dy}{dt} = -r\left(1 - \frac{y}{K}\right)\left(1 - \frac{y}{T}\right)y$$

This is the case where we have to do integration of fraction polynomial of order 3. For a while, instead of solving the difficult differential equaiton we will break it into 2 cases, with **if.. else if..** conditional.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Logistic Equation ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Logistic Equation ]# ./main
For dy/dt = r*(1-(y/K))*y
y(t) = y0*K*(y0+K*e^(-r*t)-y0*e^(-r*t))^(-1)

For dy/dt = -r*(1-(y/T))*y
y(t) = y0*T*(y0+T*e^(r*t)-y0*e^(r*t))^(-1)

For dy/dt = -r*(1-(y/T))*(1-(y/K))*y
For T < y < K and y > K
y(t) = y0*K*(y0+K*e^(-r*t)-y0*e^(-r*t))^(-1)
For 0 < y < T,
y(t) =
y0*T*(y0+T*e^(r*t)-y0*e^(r*t))^(-1)

Time taken by function: 261099 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Logistic Equation ]#
```

Figure 3.6: *The function* ***dsolvelogistic*** *in SymIntegration (**SymIntegration/Examples/Differential Equations/Test SymIntegration Logistic Equation/main.cpp**).*

Modeling with First Order Equations

[SI*] Mathematical modeling and experiment or observation are both critically important and have somewhat complementary roles in scientific investigations. Mathematical models are validated by comparison of their predictions with experimental results.

[SI*] Three identifiable steps that are always present in the process of mathematical modeling:

1. Construction of the Model.
2. Analysis of the Model.
3. Comparison with Experiment or Observation.

[SI*] **The Water Tank Mixing**
At time $t = 0$ a tank contains Q_0 lb of salt dissolved in 100 gal of water. Assume that water containing $\frac{1}{4}$ lb of salt/gal is entering the tank at a rate of r gal/min and that the well-stirred mixture is draining from the tank at the same rate. Set up the initial value problem that describes this flow process. Find the amount of salt $Q(t)$ in the tank at any time, and also find the limiting amount Q_L that is present after a very long time. If $r = 3$ and $Q_0 = 2Q_L$, find the time T after which the salt level is within 2% of Q_L. Also find the flow rate that is required if the value of T is not to exceed 45 min.

Solution:
We assume that salt is neither created nor destroyed in the tank. Therefore variations in the amount of salt are due solely to the flows in and out of the tank. More precisely, the rate of change of salt in the tank, $\frac{dQ}{dt}$, is equal to the rate at which salt is flowing in minus the rate at which it is flowing out. In symbols,

$$\frac{dQ}{dt} = \text{rate in} - \text{rate out} \tag{3.14}$$

The rate at which salt enters the tank is the concentration $\frac{1}{4}$ lb/gal times the flow rate r gal/min, or $(r/4)$ lb/min. To find the rate at which salt leaves the tank, we need to multiply the concentration of salt in the tank by the rate of outflow, r gal/min. Since the rates of flow in and out are equal, the volume of water in the tank remains constant at 100 gal, and since

the mixture is "well-stirred," the concentration throughout the tank is the same, namely, $\frac{Q(t)}{100}$ lb/gal.

Therefore the rate at which salt leaves the tank is $\frac{rQ(t)}{100}$ lb/min. Thus the differential equation governing this process is

$$\frac{dQ}{dt} = \frac{r}{4} - \frac{rQ}{100} \tag{3.15}$$

The initial condition is

$$Q(0) = Q_0 \tag{3.16}$$

Upon thinking about the problem physically, we might anticipate that eventually the mixture originally in the tank will be essentially replaced by the mixture flowing in, whose concentration is $\frac{1}{4}$ lb/gal. Consequently, we might expect that ultimately the amount of salt in the tank would be very close to 25 lb. We can also find the limiting amount $Q_L = 25$ by setting $\frac{dQ}{dt}$ equal to zero in Eq. (3.15)

$$\frac{dQ}{dt} = \text{rate in} - \text{rate out} \tag{3.17}$$

The rate at which salt enters the tank is the concentration $\frac{1}{4}$ lb/gal times the flow rate r gal/min, or $(r/4)$ lb/min. To find the rate at which salt leaves the tank, we need to multiply the concentration of salt in the tank by the rate of outflow, r gal/min. Since the rates of flow in and out are equal, the volume of water in the tank remains constant at 100 gal, and since the mixture is "well-stirred," the concentration throughout the tank is the same, namely, $\frac{Q(t)}{100}$ lb/gal.

Therefore the rate at which salt leaves the tank is $\frac{rQ(t)}{100}$ lb/min. Thus the differential equation governing this process is

$$\frac{dQ}{dt} = \frac{r}{4} - \frac{rQ}{100} \tag{3.18}$$

The initial condition is

$$Q(0) = Q_0 \tag{3.19}$$

Upon thinking about the problem physically, we might anticipate that eventually the mixture originally in the tank will be essentially replaced by the mixture flowing in, whose concentration is $\frac{1}{4}$ lb/gal. Consequently, we might expect that ultimately the amount of salt in the tank would be very close to 25 lb. We can also find the limiting amount $Q_L = 25$ by setting $\frac{dQ}{dt}$ equal to zero in Eq. (3.15) and solving the resulting algebraic equation for Q.

To solve the initial value problem at Eqs (3.15), (3.16) analytically, note that Eq. (3.15) is both linear and separable. Rewriting it in the standard form for a linear equation, we have

$$\frac{dQ}{dt} + \frac{rQ}{100} = \frac{r}{4} \tag{3.20}$$

Thus the integrating factor is $e^{rt/100}$ and the general solution is

$$Q(t) = 25 + ce^{-\frac{rt}{100}} \tag{3.21}$$

where c is an arbitrary constant. To satisfy the initial condition (3.16), we must choose $c = Q_0 - 25$. Therefore the solution of the initial value problem at Eqs (3.15), (3.16) is

$$Q(t) = 25 + (Q_0 - 25)e^{-\frac{rt}{100}} \tag{3.22}$$

or

$$Q(t) = 25(1 - e^{-\frac{rt}{100}}) + Q_0 e^{-\frac{rt}{100}} \tag{3.23}$$

You can see that

$$Q(t) \to 25$$

as $t \to \infty$, so the limiting value Q_L is 25, confirming our physical intuition.

Further, $Q(t)$ approaches the limit more rapidly as r increases. In interpreting the solution (3.23), note that the second term on the right side is the portion of the original salt that remains at time t, while the first term gives the amount of salt in the tank due to the action of the flow processes.

Now suppose that $r = 3$ and $Q_0 = 2Q_L = 50$, then Eq. (3.22) becomes

$$Q(t) = 25 + (50 - 25)e^{-0.03t} \tag{3.24}$$

Since 2% of 25 is 0.5, we wish to find the time T at which $Q(t)$ has the value 25.5. Substituting $t = T$ and $Q(t) = 25.5$ in Eq. (3.24) and solving for T, we obtain

$$\begin{aligned} Q(t) &= 25 + (Q_0 - 25)e^{-\frac{rt}{100}} \\ 25.5 &= 25 + (50 - 25)e^{-\frac{3t}{100}} \\ e^{0.03t} &= \frac{25}{0.5} \\ T &= \frac{\ln|50|}{0.03} \\ &= 130.4 \end{aligned}$$

T is in minutes. To determine r so that $T = 45$, return to Eq. (3.22), set $t = 45, Q_0 = 50, Q(t) = 25.5$, and solve for r. The result is

$$\begin{aligned} Q(t) &= 25 + (Q_0 - 25)e^{-\frac{rt}{100}} \\ 25.5 &= 25 + (50 - 25)e^{-\frac{45r}{100}} \\ 0.5 &= 25e^{-\frac{45r}{100}} \\ e^{\frac{45r}{100}} &= \frac{25}{0.5} \\ \frac{45r}{100} &= \ln|50| \\ r &= \frac{100}{45}\ln|50| \\ &\approx 8.69 \end{aligned}$$

r is in gal/min.

We would like to look into the metrics and measurement a bit further, since this is really important. We have the units of gal/min which can be converted into

$$1 \text{ gal/min} = 0.00006309 \ m^3/s = 0.0037854 \ m^3/\text{min} \tag{3.25}$$

Now, we try to do some observations and experimentations with the help of computer simulation, namely Hamzstlab Physics. We will fill an empty tank with a water, that is made by water particle, sphere like, but since it is a 2D computer simulation, we make a circle instead.

And as a representation for the water tank, that is having a cylinder shape, we take the cross section of a cylinder that is like a rectangle.

We observe with a circle of radius $r = 0.2$ and a mass of 0.15 as the water particle that is coming into the empty tank, it takes $t = 45\ s$ to fill the water tank to be full.

The volume of the water tank itself based on the computer simulation is

$$\begin{aligned} V &= \pi r^2 h \\ &= \pi(5)^2(10) \\ &= 785.39819 \end{aligned}$$

The metrics for V is m^3. So we will have the rate of the entering water as

$$\frac{785.39819\ m^3}{45\ s} = 17.453293\ m^3/s \tag{3.26}$$

Knowing that $1\ \text{gal/min} = 0.00006309\ m^3/s$, we will have the rate of the incoming water as

$$rate_{r=0.2,m=0.15} = \frac{17.453293\ m^3/s}{0.00006309\ m^3/s} * (1\ \text{gal/min}) = 276,641.19512\ \text{gal/min} \tag{3.27}$$

We are doing a 6 simulations with Hamzstlab Physics and we gather this data:

sim	$\mathbf{r}(m)$	N	$\mathbf{v}_x(m/s)$	$\mathbf{t}(s)$
1	0.2	23437.5	19.5	45
2	0.2	23437.5	25	44.7
3	0.3	6944.44	19.5	20
4	0.3	6944.44	25	20.3
5	0.4	2929.6875	19.5	11.8
6	0.4	2929.6875	25	11.5

Table 3.1: *The variable t denotes time of completion in seconds till the tank is full, v_x is the velocity of the water particle in m/s toward the x axis, while N is the number of water particle that will fit in a tank of volume $V = 785.4\ m^3$, derived from $\frac{V}{\frac{4}{3}\pi r^3}$.*

From table 3.1 we can then use multivariable regression so we can interpolate and approximate the time of completion, t, for any speed of the water that is filling the tank, for any radius of the water particle and the volume of the water tank. Thus, we will treat t as the independent variable, and the rest is the dependent variables.

Multivariable regression models aim to find the best-fitting equation that describes the relationship between the independent variables and the dependent variable. This equation has the form of

$$y = b_0 + b_1x_1 + b_2x_2 + \cdots + b_nx_n + \epsilon \tag{3.28}$$

where y is the dependent variable, b_0 is the intercept, $b_1, b_2, \ldots, b_n$ are the coefficients for each independent variable $(x_1, x_2, \ldots, x_n)$, and ϵ is the error term.

By analyzing the coefficients, we can understand how each factor relates to time of completion.

In this case we will use the independent variables of: N and v_x, we only need 2, since N is derived from the volume of the water tank divided by the volume of each particle so we don't need to use r anymore, it will become an extra cost and not necessary. Thus

$$x_1 = N$$
$$x_2 = v_x$$

Implementing multivariable regression in C++ typically involves handling matrix operations for calculations of coefficients. This can be achieved by implementing the necessary linear algebra functions from scratch or by utilizing existing C++ libraries designed for numerical linear algebra computation.

You can write your own C++ code from scratch to perform matrix multiplication, transposition, and inversion. But this provides a deep understanding of the underlying mathematics and can be complex, very time consuming and prone to errors.

Thankfully, in 2025 we have a high-performance C++ library that can handle float matrix and float vector operations with great precision, including transpose, inverse and multiplication, we will use **Armadillo / Arma** to help us computing the coefficients b_1 and b_2.

Another alternative besides **Armadillo** is **Eigen**.

Now, suppose we have this matrix that we construct from the simulation data above (table 3.1):

$$X = \begin{bmatrix} 1 & 23437.5 & 19.5 \\ 1 & 23437.5 & 25 \\ 1 & 6944.44 & 19.5 \\ 1 & 6944.44 & 25 \\ 1 & 2929.6875 & 19.5 \\ 1 & 2929.6875 & 25 \end{bmatrix}, \quad y = \begin{bmatrix} 45 \\ 44.7 \\ 20 \\ 20.3 \\ 11.8 \\ 11.5 \end{bmatrix}$$

By any means, our measurements for matrix X and vector y / the simulation data could contain human errors as well.

So we will have this matrix equation for this multivariable regression problem:

$$y = X * B + E \tag{3.29}$$

y is the vector of dependent variable values, X is the matrix of independent variables values (including a column of ones for the intercept term), B is the vector of regression coefficients

to be estimated, and E is the vector of error terms.

We will use the most common method to estimate the coefficients B, which is the Ordinary Least Squares (OLS), which minimizes the sum of squared residuals. The formula for OLS coefficient estimation is:

$$B = (X^T * X)^{-1} * X^T * y \tag{3.30}$$

From **Armadillo** we will obtain

$$B = \begin{bmatrix} 8.3696 \\ 0.0016 \\ -0.0182 \end{bmatrix}$$

Thus

$$y = b_0 + b_1 x_1 + b_2 x_2 = 8.3696 + 0.0016N - 0.0182 v_x$$

the equation above tells us about the relation / proportion between y and the independent variables

$$y \propto N$$
$$y \propto \frac{1}{v_x}$$

```
root [ ~/latex/DianFreya Math Physics Simulator/Source Codes/C++/C++ Gnuplot SymbolicC++/ch23-Numerical Linear Algebra/Multivariab
le Regression with Armadillo ]# ./main
Matrix X:
   1.0000e+00   2.3438e+04   1.9500e+01
   1.0000e+00   2.3438e+04   2.5000e+01
   1.0000e+00   6.9444e+03   1.9500e+01
   1.0000e+00   6.9444e+03   2.5000e+01
   1.0000e+00   2.9297e+03   1.9500e+01
   1.0000e+00   2.9297e+03   2.5000e+01
Vector y:
   45.0000
   44.7000
   20.0000
   20.3000
   11.8000
   11.5000

Matrix X^T:
   1.0000e+00   1.0000e+00   1.0000e+00   1.0000e+00   1.0000e+00   1.0000e+00
   2.3438e+04   2.3438e+04   6.9444e+03   6.9444e+03   2.9297e+03   2.9297e+03
   1.9500e+01   2.5000e+01   1.9500e+01   2.5000e+01   1.9500e+01   2.5000e+01

Matrix B:
   8.3696
   0.0016
  -0.0182

Xnew:
   1.0000e+00   6.9444e+03   2.5000e+01

Xnew * B:
   18.9128

Time taken by function: 2155 microseconds
```

Figure 3.7: *The matrix B is computed with **Armadillo** and the predicted t for $N = 6944.44$ and $v_x = 25$ is 18.9128, while the t obtained from the simulation observation is 20.3, the error is 6.83%, it took 2155 microseconds to complete the computation with **Armadillo** (**DFSimulatorC/Source Codes/C++/C++ Gnuplot SymbolicC++/ch23-Numerical Linear Algebra/Multivariable Regression with Armadillo/main.cpp**).*

For comparison,

```
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Linear Algebra/Test SymIntegration Multivariable Regression to Fill a Water Tank ]# ./main

Matrix X:
                1.000000           23437.500000              19.500000
                1.000000           23437.500000              25.000000
                1.000000            6944.440000              19.500000
                1.000000            6944.440000              25.000000
                1.000000            2929.687500              19.500000
                1.000000            2929.687500              25.000000

Matrix X^{T}:
                1.000000              1.000000              1.000000              1.000000              1.000000              1.000000
            23437.500000          23437.500000          6944.440000           6944.440000           2929.687500           2929.687500
               19.500000             25.000000             19.500000             25.000000             19.500000             25.000000

Vector y:
45.000000
44.700000
20.000000
20.300000
11.800000
11.500000

B :
8.369652
0.001584
-0.018182

Xnew :
1.000000
6944.440000
25.000000

Xnew * B = 18.912819

Time taken by function: 1751 microseconds
```

Figure 3.8: *The matrix B is computed with* ***SymIntegration*** *and the predicted t for* $N = 6944.44$ *and* $v_x = 25$ *is* 18.9128*, giving the same result with* ***Armadillo****, the result with SymIntegration took around 1751 microseconds. (****SymIntegration/Examples/Linear Algebra/Test SymIntegration Multivariable Regression to Fill a Water Tank/main.cpp****).*

With this multivariable regression we are able to predict the time of completion to fill a water tank with any given volume of the water tank and any given speed of the particle for the filling, or perhaps in other case, to clear up an organ from dangerous substance; how many dosage of drugs should one person drinks with certain mass and age as the independent variables to be healed from certain disease? How many chlorine needed to clear up a pool with certain volume and water quality?

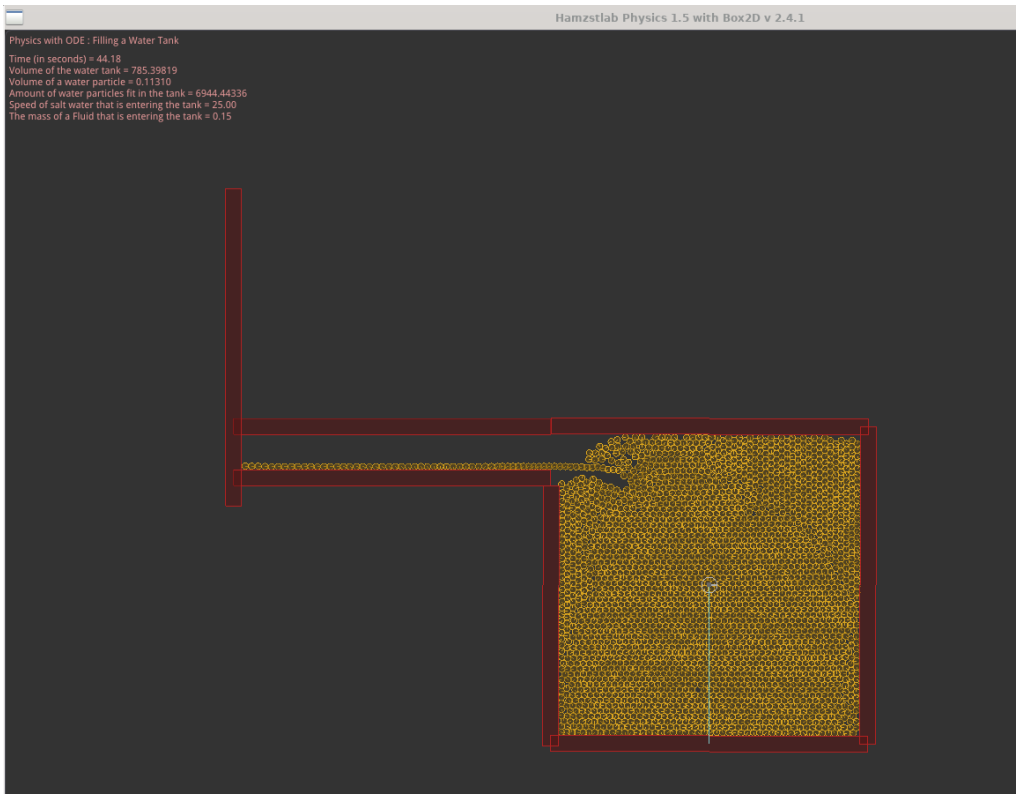

Figure 3.9: *The simulation for a water tank filling with a water particle that has the radius of* $r = 0.4$ *and* $v_x = 25$ *the* t *obtained from the simulation observation is* 11.5 s (***Hamzstlab-Physics2D/tests/ode_watertankfilling.cpp***).

We would like to introduce a book we had made in 2023-2025 that has covered Elementary Linear Algebra for undergraduate level knowledge along with its' C++ code that can be tested by the reader here [13]. We are summarizing the theorems, formulas, definitions, solving the problems from chapter 1 to the last chapter of Chris-Rorres book [29], the addition is that we make the C++ codes ourselves with the help of using already established C++ libraries like **Armadillo, giNaC, Eigen, SymbolicC++**. It is very fortunate that we have learned Linear Algebra first, since it is proven now that it becomes very helpful when we learn other topics such as physics, mechanical engineering, differential equations, and so on. All the C++ codes that we have made and put in that book is very helpful and can be reused again.

The book can be downloaded and all the source codes are available at:
https://github.com/glanzkaiser/DFSimulatorC

Since this example of water tank mixing is hypothetical, the validity of the model is not in question. If the flow rates are as stated, and if the concentration of salt in the tank is uniform, then the differential equation (3.17) is an accurate description of the flow process.

Model of this kind are often used in problems involving a pollutant in a lake, or a drug in an organ of the body, for example, rather than a tank of salt water. In such cases the flow

rates may not be easy to determine or may vary with time.

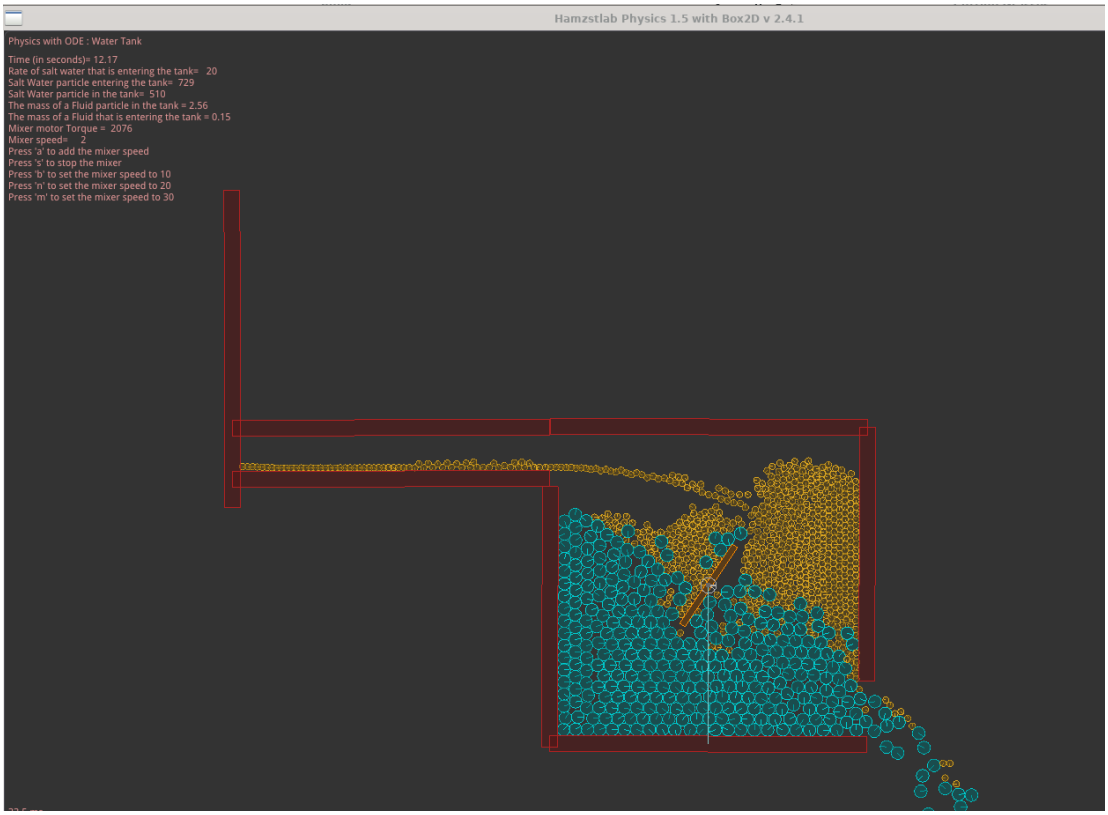

Figure 3.10: *The 2D simulation of a salt water mixing in a tank made with Hamzstlab Physics 2D that is using Box2D version 2.4.2 as the backend library (**Hamzstlab-Physics2D/tests/ode_watertankmixing.cpp**).*

We would like to introduce again one of our creation: **Hamzstlab Physics 2D**, a 2D classical physics simulator, that you can download and try / can be accessed at **https://github.com/glanzkaiser/Hamzstlab-Physics2D**

We make a lot of classical mechanics Physics simulations in 2D with Hamzstlab Physics 2D that is using the backend of Box2D version 2.4.2, at this opportunity we try to make a 2D simulation for this example so people / reader can see the illustration of this water tank mixing with salt water that is coming with certain rate / speed into the tank.

If SymIntegration is able to compute the differential equation solution, and combining it with Hamzstlab Physics we can make a better simulation that is near to real-life physics.

[SI*] **Home Loan / Mortgage**
A home buyer can afford to spend no more than USD 500 / month on mortgage payments. Suppose that the interest rate is 6%, that interest is compounded continuously, and that payments are also made continuously.

(a) Determine the maximum amount that this buyer can afford to borrow on a 20-year mortgage.

(b) Determine the total interest paid during the term of the mortgage.

Solution:

(a) The amount of money $S(t)$ that the buyer has to pay changes in time due to two factors, the compound interest and its' continuous payments. The rate of growth for compounding is rS, and the rate of decay due to the continuous payments is k.

$$\frac{dS}{dt} = rS - k$$
$$\frac{dS}{dt} - rS = -k$$

This is a linear first-order inhomogeneous ODE, so it can be solved by multiplying both sides by an integrating factor $\mu(t)$

$$\mu(t) = e^{\int^t (-r)\ ds} = e^{-rt}$$

proceed with the multiplication and solve it for $S(t)$

$$e^{-rt}\frac{dS}{dt} - re^{-rt}S = -ke^{-rt}$$
$$\frac{d}{dt}(e^{-rt}S) = -ke^{-rt}$$
$$e^{-rt}S = \frac{k}{r}e^{-rt} + C$$
$$S(t) = \frac{k}{r} + Ce^{rt}$$

Apply the initial condition $S(0) = S_0$ to determine C

$$S(0) = \frac{k}{r} + Ce^{rt}$$
$$S_0 = \frac{k}{r} + Ce^{rt}$$
$$C = S_0 - \frac{k}{r}$$

Therefore, the amount of money the buyer has to pay after t years is

$$S(t) = \frac{k}{r} + \left(S_0 - \frac{k}{r}\right)e^{rt}$$
$$= \frac{k}{r}(1 - e^{rt}) + S_0e^{rt}$$

For a year the home buyer will need to pay $k = 500 \times 12 = USD\ 6,000$ year and $r = 6\% = 0.06$.

$$S(t) = \frac{6000}{0.06}(1 - e^{0.06t}) + S_0e^{0.06t}$$

For a 20-year mortgage, $S(t) = 0$ at $t = 20$.

$$0 = \frac{6000}{0.06}(1 - e^{0.06t}) + S_0e^{0.06t}$$
$$S_0 = 100000 - 100000e^{-1.2}$$
$$S_0 \approx USD\ 69,880.58$$

The value of S_0 is how much the buyer can afford to borrow initially.

(b) For the 20-year mortgage, the home buyer pays a total of

$$USD\ 6,000 \times 20 = USD\ 120,000$$

so the total interest paid is

$$USD\ 120,000 - USD\ 69,880.58 = USD\ 50,119.42$$

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DSolve and IVP f
or Mortgage case ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DSolve and IVP f
or Mortgage case ]# ./main

DSolve for S' = rS - k

S(t) = k*r^(-1)+C*e^(t*r)
S(t=0) = k*r^(-1)+C
For ivp S(0) = S0,
C = -k*r^(-1)+S0

Test ivp = k*r^(-1)-k*r^(-1)*e^(t*r)+S0*e^(t*r)
For ivp k = 6000, r = 6%, t = 20, S(t) = 0,
S(0) = -100000*e^(-1.2)+100000
```

Figure 3.11: *The function **dsolve** and **ivp** in SymIntegration is able to compute the solution $S(t)$ for the ordinary differential equation of $\frac{dS}{dt} = Sr - k$ with the initial value problem $S(0) = S_0$ (**SymIntegration/Examples/Differential Equations/Test SymIntegration DSolve and IVP for Mortgage case/main.cpp**).*

As of July 20th, 2025 we just add a new function **ivp**, and it is managed in **/usr/include/symintegral/dsolve.h** and **/src/dsolve.cpp**

```
...
...

Symbolic ivp(const Symbolic &fx, const Symbolic &x, const Symbolic &c)
{
	Symbolic ivpsol, f0, C("C"), S0("S0");

	if(fx != 0)
	{
		list<Equations> eq;
		list<Equations>::iterator i;
		UniqueSymbol a, b;
		// Will work for this model: dS/dt = rS—k
		f0 = fx[x==0];
		C = solve(f0—S0,C).front().rhs ;
		ivpsol = fx[c==C];

	}
	return ivpsol;
}

...
```

Code 38: *src/dsolve.cpp*

[SI*] **Radiocarbon Dating Example:**
An important tool in archeological research is radiocarbon dating, developed by the American chemist Willard F. Libby. This is a means of determining the age of certain wood and plant remains, hence of animal or human bones or artifacts found buried at the same levels.

Radiocarbon dating is based on the fact that some wood or plant remains contain residual amounts of carbon-14, a radioactive isotope of carbon. This isotope is accumulated during the lifetime of the plant and begins to decay at its death. Since the half-life of carbon-14 is long (approximately 5730 years), measurable amounts of carbon-14 is still present, then by appropriate laboratory measurements the proportion of the original amount of carbon-14 that remains can be accurately determined. In other words, if $Q(t)$ is the amount of carbon-14 at time t and Q_0 is the original amount, then the ratio $\frac{Q(t)}{Q_0}$ can be determined, as long as this quantity is not too small. Present measurement techniques permit the use of this method for time periods of 50,000 years or more.

(a) Assuming that Q satisfies the differential equation $Q' = -rQ$, determine the decay constant r for carbon-14.

(b) Find an expression for $Q(t)$ at any time t, if $Q(0) = Q_0$

(c) Suppose that certain remains are discovered in which the current residual amounts of carbon-14 is 20% of the original amount. Determine the age of the remains.

Solution:

(a) The rate that the mass of carbon-14 decreases is assumed to be proportional to how much is present at any given time

$$\frac{dQ}{dt} \propto -Q$$

Change this proportionality to an equation by introducing a constant r on the right side.

$$Q' = -rQ$$

Divide both sides by Q.

$$\frac{Q'}{Q} = -r$$

The left side can be written as the derivative of a logarithm by the chain rule.

$$\begin{aligned} \frac{d}{dt} \ln\ Q &= -r \\ \ln\ Q &= -rt + C \\ Q(t) &= e^{-rt+C} \\ &= ce^{-rt} \end{aligned}$$

(b) Suppose that there is a certain amount of mass Q_0 initially. Then the initial condition is $Q(0) = Q_0$. We will use it to determine c.

$$\begin{aligned} Q(0) &= ce^{-r(0)} \\ c &= Q_0 \end{aligned}$$

Consequently, the mass decays exponentially. Substituting back we will have the solution to the initial value problem as

$$Q(t) = Q_0 e^{-rt}$$

(c) From the formula that we obtain at (b) we know that r can be determined with knowledge of the half-life. For carbon-14, specifically, we know that half the initial mass is lost after 5730 years.

$$\frac{Q_0}{2} = Q_0 e^{-r(5730)}$$

Solve this equation for r

$$\begin{aligned} e^{-5730r} &= \frac{1}{2} \\ \ln|e^{-5730r}| &= \ln\left|\frac{1}{2}\right| \\ r &= -\frac{\ln\left|\frac{1}{2}\right|}{5730} \\ &= \frac{\ln|2|}{5730} \\ r &\approx 0.0001210 \end{aligned}$$

r is the rate of the decreasing mass per year. Therefore, the mass of a sample of carbon-14 is

$$Q(t) = Q_0 e^{-\frac{\ln|2|}{5730}t}$$

where t is in years. If some remains have 20% of the original amount of carbon-14, then $Q(t) = 0.2Q_0$. Solve the resulting equation for t to determine how old the remains are.

$$\begin{aligned} 0.2Q_0 &= Q_0 e^{-\frac{\ln|2|}{5730}t} \\ e^{-\frac{\ln|2|}{5730}t} &= 0.2 \\ \ln|e^{-\frac{\ln|2|}{5730}t}| &= \ln|0.2| \\ -\frac{\ln|2|}{5730}t &= \ln\frac{1}{5} \\ -\frac{\ln|2|}{5730}t &= -\ln|5| \\ t &= \frac{\ln|5|}{\ln|2|}5730 \\ t &\approx 13,305 \end{aligned}$$

so the age of the remains are around 13,305 years old.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DSolve and IVP for Radiocarbon Dating ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DSolve and IVP for Radiocarbon Dating ]# ./ma
in

DSolve for Q' = -rQ

Q(t) = C*e^(-t*r)
Q(t=0) = C
For ivp Q(0) = Q0,
C = Q0

Solution for the ivp,
Q(t) = Q0*e^(-t*r)

Half-life problem for carbon-14, the ivp becomes:
Q0*e^(-5730*r)-0.5*Q0 = 0

Determining the rate by solving the ivp solution
r = 0.000120968

Determining the age of the remains with 20% of carbon-14
t = 13304.6
```

Figure 3.12: *The computation of the rate and the age of remains with SymIntegration' **dsolve** and **ivp** functions. (**SymIntegration/Examples/Differential Equations/Test SymIntegration DSolve and IVP for Radiocarbon Dating/main.cpp**).*

On this example, we have added another **else if** conditional so SymIntegration' **solve** function will be able to solve this equation:

$$ce^{-rx} = a$$

$$ce^{-rx} = a$$

$$x = -\frac{\ln \frac{a}{c}}{r} = \frac{\ln \frac{c}{a}}{r}$$

```
...
double division(Symbolic x, Symbolic y)
{
       return x/y;
}

Equations solve(const Symbolic &e, const Symbolic &x)
{
Equations soln;

...
...

else if(e.coeff(1,0) == e) // for case c * exp(—tx) = a or ln|tx| = a
{
       if(df(e,x).coeff(x,0) != 0 && df(df(e,x),x).coeff(x,0) != df(e,x).
           coeff(x,0) && —df(df(e,x),x).coeff(x,0) != df(e,x).coeff(x,0)) //
            for case c * exp(—tx) = a ,
       {
              Symbolic t = df(df(e,x),x).coeff(x,0) / df(e,x).coeff(x,0); //
                  will work for c == 1 or c != 1
              Symbolic a = —e.coeff(SymbolicConstant::e,0) ;
              Symbolic c = df(e,x).coeff(x,0)/t;
              soln = (soln, x == ln(division(a,c))/t);
```

```
        }
        else if(df(e,x).coeff(x,0) != 0 && -df(df(e,x),x).coeff(x,0) == df(e,
            x).coeff(x,0) ) // for case c * exp(-tx) = a , c !=1, t ==1
        {
                Symbolic a = - e.coeff(SymbolicConstant::e,0) ;
                Symbolic c = - df(e,x).coeff(x,0) ;
                soln = (soln, x == ln(division(c,a)));
        }
        ...
}
return soln;
}
...
```

Code 39: *src/solve.cpp*

[SI*] **Pollutants in a Lake:**
Consider a lake of constant volume V containing at time t an amount $Q(t)$ of pollutant, evenly distributed throughout the lake with a concentration $c(t)$, where $c(t) = \frac{Q(t)}{V}$. Assume that water containing a concentration k of pollutant enters the lake at a rate r, and that water leaves the lake at the same rate. Suppose that pollutants are also added directly to the lake at a constant rate P.

Note that given assumptions neglect a number of factors that may, in some cases, be important-for example, the water added or lost by precipitation, absorption, and evaporation; the stratifying effect of temperature differences in a deep lake; the tendency of irregularities in the coastline to produce sheltered bays; and the fact that pollutants are not deposited evenly throughout the lake but (usually) at isolated points around its periphery. The results below must be interpreted in the light of the neglect of such factors as these.

(a) If at time $t = 0$ the concentration of pollutant is c_0, find an expression for the concentation $c(t)$ at any time. What is the limiting concentration as $t \to \infty$?

(b) If the addition of pollutants to the lake is terminated ($k = 0$ and $P = 0$ for $t > 0$), determine the time interval T that must elapse before the concentration of pollutants is reduced to 50% of its original value; to 10% of its original value.

(c) Table 5.1 contains data for several of the Great Lakes. Using these data, determine from part (b) the time T necessary to reduce the contamination of each of these lakes to 10% of the original value.

Lake	V($km^3 \times 10^3$)	r($km^3/year$)
Superior	12.2	65.2
Michigan	4.9	158
Erie	0.46	175
Ontario	1.6	209

Table 3.2: *Volume and Flow Data for the Great Lakes*

Solution:

(a) The conservation law that governs the mass of pollutant in the lake is as follows.

rate of mass accumulation = rate of mass in - rate of mass out

the rate of mass in is the sum of rk and P, and the rate of mass out is $rc(t)$.

$$\begin{aligned}\frac{dQ}{dt} &= rk + P - rc(t)\\ &= rk + P - \frac{r}{V}Q(t)\\ \frac{dQ}{dt} + \frac{r}{V}Q &= rk + P\end{aligned}$$

This is a first-order linear inhomogeneous ODE, so it can be solved by multiplying both sies by an integrating factor $\mu(t)$.

$$\begin{aligned}\mu(t) &= e^{\int \frac{r}{V}\,ds} = e^{\frac{rt}{V}}\\ e^{\frac{rt}{V}}\frac{dQ}{dt} + \frac{r}{V}e^{\frac{rt}{V}}Q &= rke^{\frac{rt}{V}} + Pe^{\frac{rt}{V}}\\ \frac{d}{dt}(e^{\frac{rt}{V}}Q) &= rke^{\frac{rt}{V}} + Pe^{\frac{rt}{V}}\\ e^{\frac{rt}{V}}Q &= \int^t (rke^{\frac{rs}{V}} + Pe^{\frac{rs}{V}})\,ds + C\\ &= rk\left(\frac{V}{r}\right)e^{\frac{rt}{V}} + P\left(\frac{V}{r}\right)e^{\frac{rt}{V}} + C\\ e^{\frac{rt}{V}}Q &= kVe^{\frac{rt}{V}} + \frac{PV}{r}e^{\frac{rt}{V}} + C\\ Q(t) &= kV + \frac{PV}{r}e^{\frac{rt}{V}} + Ce^{\frac{-rt}{V}}\end{aligned}$$

Divide both sides by V to obtain the concentration.

$$c(t) = \frac{Q(t)}{V} = k + \frac{P}{r} + \frac{C}{V}e^{\frac{-rt}{V}}$$

Apply the initial condition $c(0) = c_0$ to determine C.

$$\begin{aligned}c(0) &= k + \frac{P}{r} + \frac{C}{V}\\ c_0 &= k + \frac{P}{r} + \frac{C}{V}\\ \frac{C}{V} &= c_0 - k - \frac{P}{r}\end{aligned}$$

Therefore, the concentration is

$$c(t) = \frac{Q(t)}{V} = k + \frac{P}{r} + \left(c_0 - k - \frac{P}{r}\right)e^{\frac{-rt}{V}}$$

Because of the decaying exponential function, the limit of $c(t)$ as $t \to \infty$ is

$$\lim_{t\to\infty} c(t) = k + \frac{P}{r}$$

(b) If the pollution stops, then the rate of mass in becomes zero in the conservation law.

rate of mass accumulation = - rate of mass out

The rate of mass out is still $rc(t)$.

$$\begin{aligned}
\frac{dQ}{dt} &= -rc(t) \\
&= -\frac{r}{V}Q(t) \\
\frac{\frac{dQ}{dt}}{Q} &= -\frac{r}{V} \\
\frac{dQ}{Q} &= -\frac{r}{V}\,dt \\
\ln Q &= -\frac{rt}{V} + C_1 \\
Q &= e^{-\frac{rt}{V}+C_1} \\
Q(t) &= C_2 e^{-\frac{rt}{V}}
\end{aligned}$$

Since Q is the mass of pollutant, divide both sides by V to get the concentration.

$$c(t) = \frac{Q(t)}{V} = \frac{A_1}{V}e^{-\frac{rt}{V}}$$

Use the initial condition $c(0) = c_0$ to determine C_2

$$c(0) = \frac{C_2}{V} = c_0$$

Therefore

$$c(t) = c_0 e^{-\frac{rt}{V}}$$

Set $c(t) = 0.5c_0$ and solve for $t = T$ to find how long it will take for the concentration to reach half its initial value.

$$\begin{aligned}
0.5c_0 &= c_0 e^{-\frac{rT}{V}} \\
e^{-\frac{rT}{V}} &= 0.5 \\
\ln e^{-\frac{rT}{V}} &= \ln 0.5 \\
-\frac{rT}{V} &= \ln\frac{1}{2} \\
T &= \frac{V}{r}\ln 2
\end{aligned}$$

On the other hand, set $c(t) = 0.1c_0$ and solve for $t = T$ to find how long it will take for

the concentration to reach one-tenth its initial value.

$$\begin{aligned} 0.1c_0 &= c_0 e^{-\frac{rt}{V}} \\ e^{-\frac{rt}{V}} &= 0.1 \\ \ln e^{-\frac{rt}{V}} &= \ln 0.1 \\ -\frac{rT}{V} &= \ln \frac{1}{10} \\ T &= \frac{V}{r} \ln 10 \end{aligned}$$

(c) Plug in the numbers fo V and r into the formula at part (b).
Lake Superior:

$$T = \frac{V}{r} \ln 10 = \frac{12.2 \times 10^3}{65.2} \ln 10 \approx 431 \ years$$

Lake Michigan:

$$T = \frac{V}{r} \ln 10 = \frac{4.9 \times 10^3}{158} \ln 10 \approx 71.4 \ years$$

Lake Erie:

$$T = \frac{V}{r} \ln 10 = \frac{0.46 \times 10^3}{175} \ln 10 \approx 6.05 \ years$$

Lake Ontario:

$$T = \frac{V}{r} \ln 10 = \frac{1.6 \times 10^3}{209} \ln 10 \approx 17.6 \ years$$

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DSolve and IVP for Pollutant in Great Lakes ]# make
g++     -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DSolve and IVP for Pollutant in Great Lakes ]# ./main

DSolve for Q' = rk +  P - rc(t) ,        c(t) = Q(t)/V

Q(t) = k*V+P*V*r^(-1)+C*e^(-V^(-1)*t*r)
c(t) = k+P*r^(-1)+C*e^(-V^(-1)*t*r)*V^(-1)
c(0) = k+P*r^(-1)+C*V^(-1)
c0 - c(0) = c0-k-P*r^(-1)-C*V^(-1)
For ivp c(0) = c0,
C/V = c0-k-P*r^(-1)

Therefore, the concentration is,
c(t) = k+P*r^(-1)+c0*e^(-V^(-1)*t*r)-k*e^(-V^(-1)*t*r)-P*r^(-1)*e^(-V^(-1)*t*r)

The limit of c(t) as t -> ∞
lim_{t -> ∞} c(t) = k+P*r^(-1)+c0*e^(-inf*V^(-1)*r)-k*e^(-inf*V^(-1)*r)-P*r^(-1)*e^(-inf*V^(-1)*r)

If the pollution stops, then
Q(t) = C*e^(-V^(-1)*t*r)

The concentration when the pollution stops is
c(t) = C*e^(-V^(-1)*t*r)*V^(-1)

For ivp c(0) = c0,
c0-C*V^(-1) = 0

Therefore the ivp solution will be,
c(t) = c0*e^(-V^(-1)*t*r)

Set c(t) = 0.5 c0 and solve for t=T,
T = 0.693147*V*r^(-1)

Set c(t) = 0.1 c0 and solve for t=T,
T = 2.30259*V*r^(-1)

Time to reduce the contamination in each lakes to 10 percent of the original value

Lake Superior:  T = 430.852

Lake Michigan:  T = 71.4093

Lake Erie:      T = 6.05251

Lake Ontario:   T = 17.6274
```

Figure 3.13: *The computation of initial value problem solution for the pollutant concentration in a lake, $c(t)$, and the time to clean up a lake to make the pollutant concentration less than* 10% *with SymIntegration' **dsolve** function. (**SymIntegration/Examples/Differential Equations/Test SymIntegration DSolve and IVP for Pollutant in Great Lakes/main.cpp**).*

[SI*] **Psychology' Problem:**
One problem in psychology is to determine the relation between some physical stimulus and the corresponding sensation or reaction produced in a subject. Suppose that, measured in appropriate units, the strength of a stimulus is s and the intensity of the corresponding sensation is some function of s, say $y(s)$. some experimental data suggest that the rate of change of intensity of the sensation with respect to the stimulus is directly proportional to the intensity of the sensation and inversely proportional to the strength of the stimulus; that is, $y(s)$ satisfies the differential equation

$$\frac{dy}{ds} = k\frac{y}{s}$$

for some positive constant k. Solve this differential equation.

Solution:

$$\begin{aligned}
\frac{dy}{ds} &= k\frac{y}{s} \\
\frac{dy}{y} &= k\frac{ds}{s} \\
\ln|y| &= k\ln|s| \\
y(s) &= s^k + C
\end{aligned}$$

II. Second Order Linear Equations

[SI*] Linear equations of second order are of crucial importance in the study of differential equations for two main reasons. The first is that linear equations have a rich theoretical structure that underlies a number of systematic methods of solution. Further, a substantial portion of this structure and of these methods is understandable at a fairly elementary mathematical level. Another reason to study second order linear equations is that they are vital to any serios investigation of the classical areas of mathematical physics. One cannot go very far in the development of fluid mechanics, heat conduction, wave motion, or electromagnetic phenomena without finding it necessary to solve second order linear differential equations.

[SI*] A second order ordinary differential equation has the form

$$\frac{d^2y}{dt^2} = f\left(t, y, \frac{dy}{dt}\right) \tag{3.31}$$

where f is some given function. We usually denote the independent variable by t since time is often the independent variable in physical problems, but sometimes we will use x instead. We will use y, or occasionally some other letter, to designate the dependent variable. Equation (3.31) is said to be linear if the function f has the form

$$f\left(t, y, \frac{dy}{dt}\right) = g(t) - p(t)\frac{dy}{dt} - q(t)y \tag{3.32}$$

that is, if f is linear in y and $\frac{dy}{dt}$. In Eq. (3.32) g, p, and q are specified functions of the independent variable t but do not depend on y. In this case we usually rewrite Eq. (3.31) as

$$y'' + p(t)y' + q(t)y = g(t) \tag{3.33}$$

where the primes denote differentiation with respect to t. Instead of Eq. (3.33), we often see the equation

$$P(t)y'' + Q(t)y' + R(t)y = G(t) \tag{3.34}$$

Of course, if $P(t) \neq 0$, we can divide Eq. (3.33) by $P(t)$ and thereby obtain Eq. (3.34) with

$$p(t) = \frac{Q(t)}{P(t)}, \quad q(t) = \frac{R(t)}{P(t)}, \quad g(t) = \frac{G(t)}{P(t)} \tag{3.35}$$

In discussing Eq. (3.33) and in trying to solve it, we will restrict ourselves to intervals in which p, q, and g are continuous functions.

If Eq. (3.31) is not of the form (3.33) or (3.34), then it is called nonlinear. Analytical investigations of nonlinear equations are relatively difficult. Numerical or geometrical approaches are often more appropriate.

We should not be misled by the fact that first order linear equations are easily solved by means of formulas. In general, Eq. (3.33) cannot be solved explicitly in terms of known elementary functions, or even in terms of indicated integrations. To find solutions, it is commonly necessary to resort to infinite processes of one kind or another, usually infinite series.

[SI*] An initial value problem consists of a differential equation such as Eq. (3.31), (3.33), or (3.34) together with a pair of initial conditions

$$\begin{aligned} y(t_0) &= y_0 \\ y'(t_0) &= y_0' \end{aligned} \tag{3.36}$$

where y_0 and y_0' are given numbers prescribing values for y and y' at the initial point t_0. Observe that the initial conditions for a second order equation identify not only a particular point (t_0, y_0) through which the graph of the solution must pass, but also the slope y_0' of the graph at that point. It is reasonable to expect that two initial conditions are needed for a second order equation because, roughly speaking, two integrations are required to find a solution and each integration introduces an arbitrary constant. Presumably, two initial conditions will suffice to determine values for these two constants.

[SI*] A second order linear equation is said to be homogeneous if the term $g(t)$ in Eq. (3.33), or the term $G(t)$ in Eq. (3.34), is zero for all t. Otherwise, the equation is called nonhomogeneous. As a result, the term $g(t)$, or $G(t)$, is sometimes called the nonhomogeneous term. We begin our discussion with homogeneous equations, which we will write in the form

$$P(t)y'' + Q(t)y' + R(t)y = 0 \tag{3.37}$$

Once the homogeneous equation has been solved, it is always possible to solve the corresponding nonhomogeneous equation (3.34), or at least to express the solution in terms of an integral. Thus the problem of solving the homogeneous equation is the more fundamental one.

[SI*] Consider the general second order linear differential equation

$$ay'' + by' + cy = 0 \tag{3.38}$$

whose coefficients a, b, and c are arbitrary (real) constants. We can solve Eq. (3.38) for any values of its coefficients and also satisfy any given set of initial conditions for y and y'. We start by seeking exponential solutions of the form $y = e^{rt}$, where r is a parameter to be determined.Then it follows that

$$\begin{aligned} y' &= re^{rt} \\ y'' &= r^2e^{rt} \end{aligned}$$

By substituting these expressions for y, y', and y'' in Eq. (3.38), we obtain

$$(ar^2 + br + c)e^{rt} = 0$$

or, since $e^{rt} \neq 0$,

$$ar^2 + br + c = 0 \tag{3.39}$$

Equation (3.39) is called the characteristic equation for the differential equation (3.38). It significance lies in the fact that if r is a root of the polynomial equation (3.39), then $y = e^{rt}$ is a solution of the differential equation (3.38). Since Eq. (3.39) is a quadratic equation with real coefficients, it has two roots, which may be real and different, real but repeated, or complex conjugates.

Assuming that the roots of the characteristic equation (3.39) are real and different, let them be denoted by r_1 and r_2, where $r_1 \neq r_2$. Then $y_1(t) = e^{r_1 t}$ and $y_2(t) = e^{r_2 t}$ are two solutions of Eq. (3.38). It now follows that

$$y = c_1y_1(t) + c_2y_2(t) = c_1e^{r_1 t} + c_2e^{r_2 t} \tag{3.40}$$

is also a solution of Eq. (3.38). To verify that this is so, we can differentiate the expression in Eq. (3.40); hence

$$y' = c_1 r_1 e^{r_1 t} + c_2 r_2 e^{r_2 t} \tag{3.41}$$

and

$$y'' = c_1 r_1^2 e^{r_1 t} + c_2 r_2^2 e^{r_2 t} \tag{3.42}$$

Substituting these expressions for y, y' and y'' in Eq. (3.38) and rearranging terms, we obtain

$$ay'' + by' + cy = c_1(ar_1^2 + br_1 + c)e^{r_1 t} + c_2(ar_2^2 + br_2 + c)e^{r_2 t} \tag{3.43}$$

The quantity in each of the parentheses on the right side of Eq. (3.43) is zero because r_1 and r_2 are roots of Eq. (3.39); therefore, y as given by Eq. (3.40) is indeed a solution of Eq. (3.38), as we wished to verify.

Now suppose that we want to find the particular member of the family solutions (3.40) that satisfies the initial conditions (3.36)

$$\begin{aligned} y(t_0) &= y_0 \\ y'(t_0) &= y_0' \end{aligned}$$

By substituting $t = t_0$ and $y = y_0$ in Eq. (3.40), we obtain

$$c_1 e^{r_1 t_0} + c_2 e^{r_2 t_0} = y_0 \tag{3.44}$$

Similarly, setting $t = t_0$ and $y' = y_0'$ in Eq. (3.41) gives

$$c_1 r_1 e^{r_1 t_0} + c_2 r_2 e^{r_2 t_0} = y_0' \tag{3.45}$$

On solving Eqs. (3.44) and (3.45) simultaneously for c_1 and c_2, we find that

$$\begin{aligned} c_1 &= \frac{y_0' - y_0 r_2}{r_1 - r_2} e^{-r_1 t_0} \\ c_1 &= \frac{y_0 r_1 - y_0'}{r_1 - r_2} e^{-r_2 t_0} \end{aligned} \tag{3.46}$$

recall that $r_1 - r_2 \neq 0$ so that the expressions in Eq. (3.46) always make sense. Thus, no matter what initial conditions are assigned-that is, regardless of the valueas of t_0, y_0, and y_0' in Eqs. (3.36)-it is always possible to determine c_1 and c_2 so that the initial conditions are satisfied. Moreover, there is only one possible choice of c_1 and c_2 for each set of initial conditons. With the values of c_1 and c_2 given by Eq. (3.46), the expression (3.40) is the solution of the initial value problem

$$\begin{aligned} ay'' + by' + cy &= 0 \\ y(t_0) &= y_0 \\ y'(t_0) &= y_0' \end{aligned} \tag{3.47}$$

[SI*] **Solutions of Linear Homogeneous Equations; the Wronskian**
We will introduce a differential operator notation. Let p and q be continuous functions on an open interval I, that is, for $\alpha < t < \beta$. The cases $\alpha = -\infty$, or $\beta = \infty$, or both, are included.

Then, for any function ϕthat is twice differentiable on I we define the differential operator L by the equation

$$L[\phi] = \phi'' + p\phi' + q\phi \tag{3.48}$$

Note that $L[\phi]$ is a function on I. The value of $L[\phi]$ at a point t is

$$L[\phi](t) = \phi''(t) + p(t)\phi'(t) + q(t)\phi(t)$$

For example, if $p(t) = t^2, q(t) = 1 + t$, and $\phi(t) = \sin\ 3t$, then

$$\begin{aligned} L[\phi](t) &= (\sin\ 3t)'' + t^2(\sin\ 3t)' + (1+t)\sin\ 3t \\ &= -9\sin\ 3t + 3t^2\cos\ 3t + (1+t)\sin\ 3t \end{aligned}$$

The operator L is often written as $L = D^2 + pD + q$, where D is the derivative operator.

Since it is customary to use the symbol y to denote $\phi(t)$, we will usually write the second order linear homogeneous equation in the form

$$L[y] = y'' + p(t)y' + q(t)y = 0 \tag{3.49}$$

With Eq. (3.49) we associate a set of initial conditions

$$y(t_0) = y_0, \quad y'(t_0) = y_0' \tag{3.50}$$

where t_0 is any point in the interval I, and y_0 and y_0' are given real numbers.

Theorem 3.1: Existence and Uniqueness Theorem

Consider the initial value problem

$$\begin{aligned} y'' + p(t)y' + q(t)y &= g(t) \\ y(t_0) &= y_0 \\ y'(t_0) &= y_0' \end{aligned} \tag{3.51}$$

where p, q, and g are continuous on an open interval I that contains the point t_0. Then there is exactly one solution $y = \phi(t)$ of this problem, and the solution exists throughout the interval I.

Theorem 3.2: Principle of Superposition

If y_1 and y_2 are two solutions of the differential equation (3.49),

$$L[y] = y'' + p(t)y' + q(t)y = 0$$

then the linear combination $c_1y_1 + c_2y_2$ is also a solution for any values of the constants c_1 and c_2.

A special case of Principle of Superposition theorem occurs if either c_1 or c_2 is zero. Then we conclude that any constant multiple of a solution of Eq. (3.49) is also a solution. To prove the theorem, we need only substitute

$$y = c_1y_1(t) + c_2y_2(t) \tag{3.52}$$

for y in Eq. (3.49). By calculating the indicated derivatives and rearranging terms, we obtain

$$\begin{aligned} L[c_1y_1 + c_2y_2] &= [c_1y_1 + c_2y_2]'' + p[c_1y_1 + c_2y_2]' + q[c_1y_1 + c_2y_2] \\ &= c_1y_1'' + c_2y_2'' + c_1py_1' + c_2py_2' + c_1qy_1 + c_2qy_2 \\ &= c_1[y_1'' + py_1' + qy_1] + c_2[y_2'' + py_2' + qy_2] \\ &= c_1L[y_1] + c_2L[y_2] \end{aligned}$$

Since $L[y_1] = 0$ and $L[y_2] = 0$, it follows that $L[c_1y_1 + c_2y_2] = 0$ also. Therefore, regardless of the values of c_1 and c_2, y as given by Eq. (3.52) does satisfy the differential equation (3.49), and the proof for the theorem is complete.

The theorem states that, beginning with only two solutions of Eq. (3.49), we can construct an infinite family of solutions by means of Eq. (3.52). The next question is whether all solutions of Eq. (3.49) are included in Eq. (3.52) or whether there may be other solutions of a different form. We begin to address this question by examining whether the constants c_1 and c_2 in Eq. (3.52) can be chosen so as to satisfy the initial conditions (3.50). These initial conditions require c_1 and c_2 to satisfy the equations

$$\begin{aligned} c_1y_1(t_0) + c_2y_2(t_0) &= y_0 \\ c_1y_1'(t_0) + c_2y_2'(t_0) &= y_0' \end{aligned} \tag{3.53}$$

The determinant of coefficients of the system (3.54) is

$$W = \begin{vmatrix} y_1(t_0) & y_2(t_0) \\ y_1'(t_0) & y_2'(t_0) \end{vmatrix} = y_1(t_0)y_2'(t_0) - y_1'(t_0)y_2(t_0) \tag{3.54}$$

If $W \neq 0$, then Eqs (3.53) have a unique solution (c_1, c_2) regardless of the values of y_0 and y_0'. This solution is given by

$$\begin{aligned} c_1 &= \frac{y_0y_2'(t_0) - y_0'y_2(t_0)}{y_1(t_0)y_2'(t_0) - y_1'(t_0)y_2(t_0)} \\ c_2 &= \frac{-y_0y_1'(t_0) + y_0'y_1(t_0)}{y_1(t_0)y_2'(t_0) - y_1'(t_0)y_2(t_0)} \end{aligned} \tag{3.55}$$

or, in terms of determinants,

$$c_1 = \frac{\begin{vmatrix} y_0 & y_2(t_0) \\ y_0' & y_2'(t_0) \end{vmatrix}}{\begin{vmatrix} y_1(t_0) & y_2(t_0) \\ y_1'(t_0) & y_2'(t_0) \end{vmatrix}}, \quad c_2 = \frac{\begin{vmatrix} y_1(t_0) & y_0 \\ y_1'(t_0) & y_0' \end{vmatrix}}{\begin{vmatrix} y_1(t_0) & y_2(t_0) \\ y_1'(t_0) & y_2'(t_0) \end{vmatrix}} \tag{3.56}$$

with these values for c_1 and c_2, the expression (3.52) satisfies the initial conditions (3.50) as well as the differential equation (3.49).

On the other hand, if $W = 0$, then Eqs. (3.53) have no solution unless y_0 and y_0' satisfy a certain additional conditions; in this case there are infinitely many solutions.

The determinant W is called the Wronskian determinant, or simply the Wronskian, of the solutions y_1 and y_2. Sometimes we use the more extended notation $W(y_1, y_2)(t_0)$ to stand

for the expression on the right side of Eq. (3.54), thereby emphasizing that the Wronskian depends on the functions y_1 and y_2, and that it is evaluated at the point t_0.

Theorem 3.3: Wronskian for Second Order Linear Differential Equations

Suppose that y_1 and y_2 are two solutions of Eq. (3.49)

$$L[y] = y'' + p(t)y' + q(t)y = 0$$

and that the initial conditions (3.50)

$$y(t_0) = y_0, \quad y'(t_0) = y'_0$$

are assigned. Then it is always possible to choose the constants c_1, c_2 so that

$$y = c_1 y_1(t) + c_2 y_2(t)$$

satisfies the differential equation (3.49) and the initial conditions (3.50) if and only if the Wronskian

$$W = y_1 y'_2 - y'_1 y_2$$

is not zero at t_0.

Theorem 3.4: Fundamental Set of Solutions

Consider the differential equation (3.49)

$$L[y] = y'' + p(t)y' + q(t)y = 0$$

whose coefficients p and q are continuous on some open interval I. Choose some point t_0 in I. Let y_1 be the solution of Eq. (3.49) that also satisfies the initial conditions

$$y(t_0) = 1, \quad y'(t_0) = 0$$

and let y_2 be the solution of Eq. (3.49) that satisfies the initial conditions

$$y(t_0) = 0, \quad y'(t_0) = 1$$

Then y_1 and y_2 form a fundamental set of solutions of Eq. (3.49). If you compute the Wronskian it will be $W(y_1, y_2)(t_0) = 1$, since the Wronskian is not zero at the point t_0, the functions y_1 and y_2 do form a fundamental set of solutions.

Theorem 3.5: Abel's Theorem

If y_1 and y_2 are solutions of the differential equation

$$L[y] = y'' + p(t)y' + q(t)y = 0$$

where p and q are continuous on an open interval I, then the Wronskian $W(y_1, y_2)(t)$ is given by

$$W(y_1, y_2)(t) = ce^{-\int p(t)\, dt} \tag{3.57}$$

where c is a certain constant that depends on y_1 and y_2, but not on t. Further, $W(y_1, y_2)(t)$ either is zero for all t in I (if $c = 0$) or else is never zero in I (if $c \neq 0$).

[SI*] **The Adjoint Equation**

If a second order linear homogeneous equation is not exact, it can be made exact by multiplying by an appropriate integrating factor $\mu(x)$. Thus we require that $\mu(x)$ be such that

$$\mu(x)P(x)y'' + \mu(x)Q(x)y' + \mu(x)R(x)y = 0$$

can be written in the form

$$\left[\mu(x)P(x)y'\right]' + \left[f(x)y\right]' = 0$$

By equating coefficients in these two equations and eliminating $f(x)$, show that the function μ must satisfy

$$P\mu'' + (2P' - Q)\mu' + (P'' - Q' + R)\mu = 0$$

This equation is known as the adjoint of the original equation and is impotant in the advanced theory of differential equations. In general, the problem of solving the adjoint differential equation is as difficult as that of solving the original equation, so only occasionally is it possible to find an integrating factor for a second order equation.

Solution:

Suppose we have the second-order ODE,

$$P(x)y'' + Q(x)y' + R(x)y = 0$$

To make it exact, multiply both sides by an integrating factor $\mu = \mu(x)$

$$\mu(x)P(x)y'' + \mu(x)Q(x)y' + \mu(x)R(x)y = 0 \tag{3.58}$$

Now that it's exact, it can be written in the form,

$$\left[\mu(x)P(x)y'\right]' + \left[f(x)y\right]' = 0$$

Expand the left side

$$\mu'(x)P(x)y' + \mu(x)P'(x)y' + \mu(x)P(x)y'' + f'(x)y + f(x)y' = 0$$

Factor it now

$$\mu(x)P(x)y'' + \left[\mu'(x)P(x) + \mu(x)P'(x) + f(x)\right]y' + f'(x)y = 0$$

Equate the coefficients with those of equation (3.58)

$$\mu'(x)P(x) + \mu(x)P'(x) + f(x) = \mu(x)Q(x)$$
$$f'(x) = \mu(x)R(x)$$

Differentiate both sides of the first equation with respect to x.

$$\mu''(x)P(x) + \mu'(x)P'(x) + \mu(x)P''(x) + f'(x) = \mu'(x)Q(x) + \mu(x)Q'(x)$$

Substitute $\mu(x)R(x)$ for $f'(x)$

$$\mu''(x)P(x) + \mu'(x)P'(x) + \mu(x)P''(x) + \mu(x)R(x) = \mu'(x)Q(x) + \mu(x)Q'(x)$$

Therefore, the adjoint is

$$P\mu'' + (2P' - Q)\mu' + (P'' - Q' + R)\mu = 0$$

[SI*] **Example:**
Find the adjoint of the given differential equation

(a) $x^2y'' + xy' + (x^2 - \nu^2)y = 0$ (Bessel's equation)

(b) $(1 - x^2)y'' - 2xy' + \alpha(\alpha + 1)y = 0$ (Legendre's equation)

(c) $y'' - xy = 0$ (Airy's equation)

Solution:

(a) To make the ODE exact, multiply both sides by an integrating factor $\mu = \mu(x)$

$$x^2\mu(x)y'' + x\mu(x)y' + (x^2 - \nu^2)\mu(x)y = 0 \tag{3.59}$$

Now that it's exact, it can be written in the form

$$\left[x^2\mu(x)y'\right]' + [f(x)y]' = 0$$

Expand the left side

$$2x\mu(x)y' + x^2\mu'(x)y' + x^2\mu(x)y'' + f'(x)y + f(x)y' = 0$$

Factor it now

$$x^2\mu(x)y'' + \left[x^2\mu'(x) + 2x\mu(x) + f(x)\right]y' + f'(x)y = 0$$

Equate the coefficients with those of equation (3.59)

$$x^2\mu'(x) + 2x\mu(x) + f(x) = x\mu(x)$$
$$f'(x) = (x^2 - \nu^2)\mu(x)$$

Differentiate both sides of the first equation with respect to x

$$2x\mu'(x) + x^2\mu''(x) + 2\mu(x) + 2x\mu'(x) + f'(x) = \mu(x) + x\mu'(x)$$

Substitute $(x^2 - \nu^2)\mu(x)$ for $f'(x)$

$$2x\mu'(x) + x^2\mu''(x) + 2\mu(x) + 2x\mu'(x) + (x^2 - \nu^2)\mu(x) = \mu(x) + x\mu'(x)$$

Bring all terms to the left side and combine like terms

$$x^2\mu''(x) + 3x\mu'(x) + (x^2 - \nu^2 + 1)\mu(x) = 0$$

(b) To make the ODE exact, multiply both sides by an integrating factor $\mu = \mu(x)$

$$(1-x^2)\mu(x)y'' - 2x\mu(x)y' + \alpha(\alpha+1)\mu(x)y = 0 \tag{3.60}$$

Now that it's exact, it can be written in the form

$$\left[(1-x^2)\mu(x)y'\right]' + [f(x)y]' = 0$$

Expand the left side

$$-2x\mu(x)y' + (1-x^2)\mu'(x)y' + (1-x^2)\mu(x)y'' + f'(x)y + f(x)y' = 0$$

Factor it now.

$$(1-x^2)\mu(x)y'' + \left[(1-x^2)\mu'(x) - 2x\mu(x) + f(x)\right]y' + f'(x)y = 0$$

Equate the coefficients with those of equation (3.60)

$$\begin{aligned}(1-x^2)\mu'(x) - 2x\mu(x) + f(x) &= -2x\mu(x)\\ f'(x) &= \alpha(\alpha+1)\mu(x)\end{aligned}$$

Add $2x\mu(x)$ to both sides of the first equation

$$(1-x^2)\mu'(x) + f(x) = 0$$

Differentiate both sides with respect to x.

$$-2x\mu'(x) + (1-x^2)\mu''(x) + f'(x) = 0$$

Substitute $\alpha(\alpha+1)\mu(x)$ for $f'(x)$

$$-2x\mu'(x) + (1-x^2)\mu''(x) + \alpha(\alpha+1)\mu(x) = 0$$

Therefore,

$$(1-x^2)\mu''(x) - 2x\mu'(x) + \alpha(\alpha+1)\mu(x) = 0$$

(c) To make the ODE exact, multiply both sides by an integrating factor $\mu = \mu(x)$

$$\mu(x)y'' - x\mu(x)y = 0 \tag{3.61}$$

Now that it's exact, it can be written in the form

$$\left[\mu(x)y'\right]' + [f(x)y]' = 0$$

Expand the left side

$$\mu'(x)y' + \mu(x)y'' + f'(x)y + f(x)y' = 0$$

Factor it now

$$\mu(x)y'' + \left[\mu'(x) + f(x)\right]y' + f'(x)y = 0$$

Equate the coefficients with those of equation (3.61)

$$\begin{aligned}\mu'(x) + f(x) &= 0\\ f'(x) &= -x\mu(x)\end{aligned}$$

Differentiate both sides of the first equation with respect to x

$$\mu''(x) + f'(x) = 0$$

Substitute $-x\mu(x)$ for $f'(x)$

$$\mu''(x) - x\mu(x) = 0$$

[SI*] **Euler' Equations**

An equation of the form

$$t^2\frac{d^2y}{dt^2}+\alpha\frac{dy}{dt}+\beta y=0, \quad t>0 \tag{3.62}$$

where α and β are real constants, is called an Euler equation.

(a) Let $x=\ln\ t$ and calculate $\frac{dy}{dt}$ and $\frac{d^2y}{dt^2}$ in terms of $\frac{dy}{dx}$ and $\frac{d^2y}{dx^2}$.

(b) Use the results of part (a) to transform Eq. (3.62) into

$$\frac{d^2y}{dx^2}+(\alpha-1)\frac{dy}{dx}+\beta y=0 \tag{3.63}$$

Observe that Eq. (3.63) has constant coefficients. If $y_1(x)$ and $y_2(x)$ form a fundamental set of solutions of Eq. (3.63), then $y_1(\ln\ t)$ and $y_2(\ln\ t)$ form a fundamental set of solutions of Eq. (3.62).

Solution:

(a) Make the substitution $x=\ln\ t$ in the ODE. Then

$$e^x=t\rightarrow e^{2x}=t^2$$

and the ODE becomes

$$e^{2x}\frac{d^2y}{dt^2}+\alpha e^x\frac{dy}{dt}+\beta y=0$$

The aim now is to find what the derivatives are in terms of this new variable by using the chain rule.

$$\frac{dy}{dt}=\frac{dy}{dx}\frac{dx}{dt}=\frac{dy}{dx}\left(\frac{1}{t}\right)=\frac{dy}{dx}\left(\frac{1}{e^x}\right)=e^{-x}\frac{dy}{dx}$$

$$\begin{aligned}\frac{d^2y}{dt^2}&=\frac{d}{dt}\left(\frac{dy}{dt}\right)\\&=\frac{dx}{dt}\frac{d}{dx}\left(e^{-x}\frac{dy}{dx}\right)\\&=\frac{1}{t}\left(-e^{-x}\frac{dy}{dx}+e^{-x}\frac{d^2y}{dx^2}\right)\\&=\frac{1}{e^x}\left(-e^{-x}\frac{dy}{dx}+e^{-x}\frac{d^2y}{dx^2}\right)\end{aligned}$$

(b) Substitute these expressions into the ODE

$$\begin{aligned}e^{2x}\frac{1}{e^x}\left(-e^{-x}\frac{dy}{dx}+e^{-x}\frac{d^2y}{dx^2}\right)+\alpha e^x\left(e^{-x}\frac{dy}{dx}\right)+\beta y&=0\\e^x\left(-e^{-x}\frac{dy}{dx}+e^{-x}\frac{d^2y}{dx^2}\right)+\alpha\frac{dy}{dx}+\beta y&=0\\-\frac{dy}{dx}+\frac{d^2y}{dx^2}+\alpha\frac{dy}{dx}+\beta y&=0\end{aligned}$$

Therefore,

$$\frac{d^2y}{dx^2}+(\alpha-1)\frac{dy}{dx}+\beta y=0$$

[SI*] **Complex Roots of the Characteristic Equation**
Suppose we have the homogeneous second order linear differential equation

$$ay'' + by' + cy = 0 \tag{3.64}$$

We have learned that if we seek solutions of the form $y = e^{rt}$, then r must be a root of the characteristic equation

$$ar^2 + br + c = 0 \tag{3.65}$$

If the roots r_1 and r_2 are real and different, which occurs whenever the discriminant $b^2 - 4ac$ is positive, then the general solution of Eq. (3.64) is

$$y = c_1 e^{r_1 t} + c_2 e^{r_2 t} \tag{3.66}$$

Suppose now that $b^2 - 4ac$ is negative. Then the roots of Eq. (3.65) are conjugate complex numbers; we denote them by

$$\begin{aligned} r_1 &= \lambda + i\mu \\ r_2 &= \lambda - i\mu \end{aligned} \tag{3.67}$$

where λ and μ are real. The corresponding expressions for y are

$$\begin{aligned} y_1(t) &= e^{(\lambda + i\mu)t} \\ y_2(t) &= e^{(\lambda - i\mu)t} \end{aligned} \tag{3.68}$$

[SI*] **Euler's Formula**
To assign a meaning to the expressions in Eqs. (3.68), we need to give a definition of the complex exponential function. Recall from Calculus that the Taylor series for e^t about $t = 0$ is

$$e^t = \sum_{n=0}^{\infty} \frac{t^n}{n!}, \quad -\infty < t < \infty \tag{3.69}$$

If we now assume that we can substitute it for t in Eq. (3.69), then we have

$$\begin{aligned} e^{it} &= \sum_{n=0}^{\infty} \frac{(it)^n}{n!} \\ &= \sum_{n=0}^{\infty} \frac{(-1)^n t^{2n}}{(2n)!} + i \sum_{n=0}^{\infty} \frac{(-1)^{n-1} t^{2n-1}}{(2n-1)!} \end{aligned} \tag{3.70}$$

where we have separated the sum into its real and imaginary parts, making use of the fact that $i^2 = -1, i^3 = -i, i^4 = 1$, and so forth. The first series in Eq. (3.70) is precisely the Taylor series for cos t about $t = 0$, and the second is the Taylor series for sin t about $t = 0$. Thus we have

$$e^{it} = \cos\ t + i\ \sin\ t \tag{3.71}$$

Equation (3.71) is known as Euler's formula and is an extremely important mathematical relationship.

There are some variations of Euler's formula that are also worth noting

$$e^{-it} = \cos\ t - i\ \sin\ t \tag{3.72}$$

Further, if t is replaced by μt, then we obtain a generalized version of Euler's formula, namely

$$e^{i\mu t} = \cos\ \mu t + i\ \sin\ \mu t \tag{3.73}$$

Next, we want to extend the definition of the exponential function to arbitrary complex exponents of the form $(\lambda + i\mu)t$. Since we want the usual properties of the exponential function to hold for complex exponents, we certainly want $e^{(\lambda+i\mu)t}$ to satisfy

$$e^{(\lambda+i\mu)t} = e^{\lambda t}e^{i\mu t} \tag{3.74}$$

Then, substituting for $e^{i\mu t}$ from Eq. (3.73), we obtain

$$\begin{aligned} e^{(\lambda+i\mu)t} &= e^{\lambda t}(\cos\ \mu t + i\ \sin\ \mu t) \\ &= e^{\lambda t}\cos\ \mu t + ie^{\lambda t}\ \sin\ \mu t \end{aligned} \tag{3.75}$$

We now take Eq. (3.75) as the definition of $e^{(\lambda+i\mu)t}$. The value of the exponential function with a complex exponent is a complex number whose real and imaginary parts are given by the terms on the right side of Eq. (3.75). Observe that the real and imaginary parts of $e^{(\lambda+i\mu)t}$ are expressed entirely in terms of elementary real-valued functions.

[SI*] **Complex Roots; The General Case**

The functions $y_1(t)$ and $y_2(t)$, given by Eqs. (3.68) and with the meaning expressed by Eq. (3.75), are solutions of Eq. (3.64) when the roots of the characteristic equation (3.65) are complex numbers $\lambda \pm i\mu$. Unfortunately, the solutions y_1 and y_2 are complex-valued functions, whereas in general we would prefer to have real-valued solutions, if possible, because the differential equation itself has real coefficients. To find a fundamental set of real-valued solutions let us form the sum and then the difference of y_1 and y_2. We have

$$\begin{aligned} y_1(t) + y_2(t) &= e^{\lambda t}(\cos\ \mu t + i\ \sin\ \mu t) + e^{\lambda t}(\cos\ \mu t - i\ \sin\ \mu t) \\ &= 2e^{\lambda t}\cos\ \mu t \end{aligned}$$

and

$$\begin{aligned} y_1(t) - y_2(t) &= e^{\lambda t}(\cos\ \mu t + i\ \sin\ \mu t) - e^{\lambda t}(\cos\ \mu t - i\ \sin\ \mu t) \\ &= 2ie^{\lambda t}\sin\ \mu t \end{aligned}$$

Hence, neglecting the constant multipliers 2 and $2i$, respectively, we have obtained a pair of real-valued solutions

$$\begin{aligned} u(t) &= e^{\lambda t}\cos\ \mu t \\ v(t) &= e^{\lambda t}\sin\ \mu t \end{aligned} \tag{3.76}$$

Observe that u and v are simply the real and imaginary parts, respectively, of y_1.
By direct computation you can show that the Wronskian of u and v is

$$W(u, v)(t) = \mu e^{2\lambda t} \tag{3.77}$$

Thus, as long as $\mu \neq 0$, the Wronskian W is not zero, so u and v form a fundamental set of solutions. If $\mu = 0$ then the roots are real and repeated. Consequently, if the roots of the characteristic equation are complex numbers $\lambda \pm i\mu$, with $\mu \neq 0$, then the general solution of Eq. (3.64) is

$$y = c_1 e^{\lambda t}\cos\ \mu t + c_2 e^{\lambda t}\sin\ \mu t \tag{3.78}$$

[SI*] **Example:**
Find the solution of the initial value problem

$$16y'' - 8y' + 145y = 0, \quad y(0) = -2, \quad y'(0) = 1 \tag{3.79}$$

Solution:

The characteristic equation is $16r^2 - 8r + 145 = 0$ and its roots are $r = \frac{1}{4} \pm 3i$. Thus the general solution of the differential equation is

$$y = c_1 e^{t/4} \cos\ 3t + c_2 e^{t/4} \sin\ 3t \tag{3.80}$$

To apply the first initial condition, we set $t = 0$ in Eq. (3.80); this gives

$$y(0) = c_1 = -2$$

For the second initial condition we must differentiate Eq. (3.80) and then set $t = 0$. In this way we find that

$$y'(0) = \frac{1}{4}c_1 + 3c_2 = 1$$

from which $c_2 = \frac{1}{2}$. Using these values of c_1 and c_2 in Eq. (3.80), we obtain

$$y = -2e^{t/4} \cos\ 3t + \frac{1}{2}e^{t/4} \sin\ 3t \tag{3.81}$$

as the solution of the initial value problem (3.79).

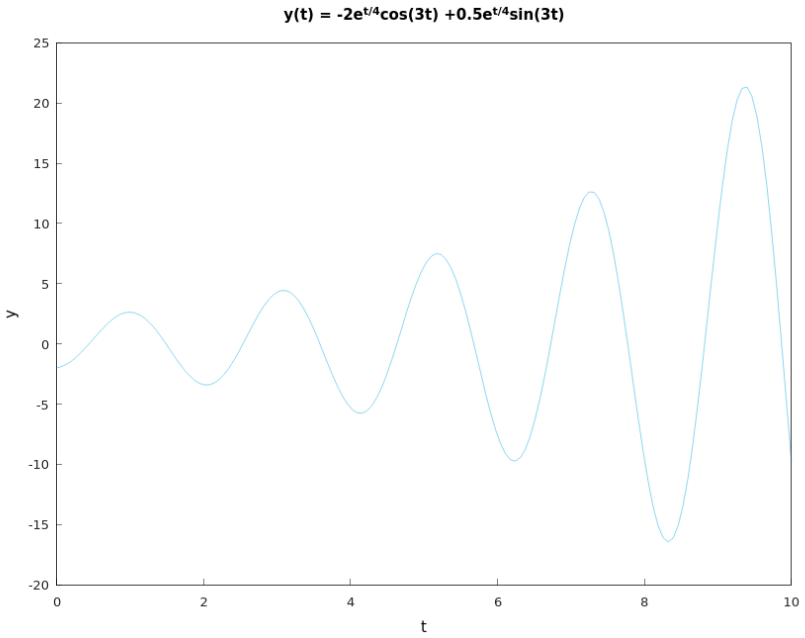

Figure 3.14: *The graph of the solution* $y = -2e^{t/4} \cos\ 3t + \frac{1}{2}e^{t/4} \sin\ 3t$ *of the initial value problem* $16y'' - 8y' + 145y = 0, y(0) = -2, y'(0) = 1$ (***Hamzstplot/Examples with Makefile/Differential Equations/Plot 2D Growing Oscillation/main.cpp***).

In this case we observe that the solution is a growing oscillation. The trigonometric factors in Eq. (3.81) determine the oscillatory part of the solution, while the exponential factor (with a positive exponent) causes the magnitude of the oscillation to increase with time.

[SI*] **Repeated Roots**

We showed how to solve the equation

$$ay'' + by' + cy = 0 \tag{3.82}$$

when the roots of the characteristic equation

$$ar^2 + br + c = 0 \tag{3.83}$$

either are real and different or are complex conjugates. Now we consider the third possibility, namely, that the two roots r_1 and r_2 are equal. This case is transitional between the other two and occurs when the discriminant $b^2 - 4ac$ is zero. Then it follows from the quadratic formula that

$$r_1 = r_2 = -\frac{b}{2a} \tag{3.84}$$

The difficulty is immediately apparent; both roots yield the same solution

$$y_1(t) = e^{-\frac{bt}{2a}} \tag{3.85}$$

of the differential equation (3.82), and it is not obvious how to find a second solution.

To find a second solution, we assume that

$$y = v(t)y_1(t) = v(t)e^{-\frac{bt}{2a}} \tag{3.86}$$

and substitute for y in Eq. (3.82) to determine $v(t)$. We have

$$y' = v'(t)e^{-\frac{bt}{2a}} - \frac{b}{2a}v(t)e^{-\frac{bt}{2a}} \tag{3.87}$$

and

$$y'' = v''(t)e^{-\frac{bt}{2a}} - \frac{b}{a}v'(t)e^{-\frac{bt}{2a}} + \frac{b^2}{4a^2}v(t)e^{-\frac{bt}{2a}} \tag{3.88}$$

Then, by substituting in Eq. (3.82), we obtain

$$\left[a\left(v''(t) - \frac{b}{a}v'(t) + \frac{b^2}{4a^2}v(t)\right) + b\left(v'(t) - \frac{b}{2a}v(t)\right) + cv(t)\right]e^{-\frac{bt}{2a}} = 0 \tag{3.89}$$

Canceling the factor $e^{-\frac{bt}{2a}}$, which is nonzero, and rearranging the remaining terms, we find that

$$av''(t) + (-b + b)v'(t) + \left(\frac{b^2}{4a} - \frac{b^2}{2a} + c\right)v(t) = 0 \tag{3.90}$$

The term involving $v'(t)$ is obviously zero. Further, the coefficient of $v(t)$ is $c - \left(\frac{b^2}{4a}\right)$, which is also zero because $b^2 - 4ac = 0$ (repeated roots). Thus, Eq. (3.90) reduces to

$$v''(t) = 0$$

therefore

$$v(t) = c_1 + c_2 t$$

Hence, from Eq. (3.86), we have

$$y = c_1 e^{-\frac{bt}{2a}} + c_2 t e^{-\frac{bt}{2a}} \tag{3.91}$$

Thus y is a linear combination of the two solutions

$$y_1(t) = e^{-\frac{bt}{2a}}, \quad y_2(t) = te^{-\frac{bt}{2a}} \tag{3.92}$$

The Wronskian of these two solutions is

$$W(y_1, y_2)(t) = \begin{vmatrix} e^{-\frac{bt}{2a}} & te^{-\frac{bt}{2a}} \\ -\frac{b}{2a}e^{-\frac{bt}{2a}} & \left(1 - \frac{bt}{2a}\right) e^{-\frac{bt}{2a}} \end{vmatrix} = e^{-\frac{bt}{a}} \tag{3.93}$$

Since $W(y_1, y_2)(t)$ is never zero, the solutions y_1 and y_2 given by Eq. (3.92) are a fundamental set of solutions. Further Eq. (3.91) is the general solution of Eq. (3.82) when the roots of the characteristic equation are equal.

[SI*] **Example:**
Find the solution of the initial value problem

$$y'' - y' + 0.25y = 0, \quad y(0) = 2, \quad y'(0) = \frac{1}{3} \tag{3.94}$$

Solution:
The characteristic equation is

$$r^2 - r + 0.25 = 0$$

so the roots are $r_1 = r_2 = \frac{1}{2}$. Thus the general solution of the differential equation is

$$y = c_1 e^{t/2} + c_2 t e^{t/2} \tag{3.95}$$

The first initial condition requires that

$$y(0) = c_1 = 2$$

to satisfy the second initial condition, we first differentiate Eq. (3.95) and then set $t = 0$. This gives

$$y'(0) = \frac{1}{2}c_1 + c_2 = \frac{1}{3}$$

so $c_2 = -\frac{2}{3}$. Thus the solution of the initial value problem is

$$y = 2e^{t/2} - \frac{2}{3}te^{t/2} \tag{3.96}$$

[SI*] **Reduction of Order**
Suppose that we know one solution $y_1(t)$, not everywhere zero, of

$$y'' + p(t)y' + q(t)y = 0 \tag{3.97}$$

To find a second solution, let

$$y = v(t)y_1(t) \tag{3.98}$$

then

$$y' = v'(t)y_1(t) + v(t)y_1'(t)$$

and

$$y'' = v''(t)y_1(t) + 2v'(t)y_1'(t) + v(t)y_1''(t)$$

Substituting for y, y', and y'' in Eq. (3.97) and collecting terms, we find that

$$y_1v'' + (2y_1' + py_1)v' + (y_1'' + py_1' + qy_1)v = 0 \tag{3.99}$$

Since y_1 is a solution of Eq. (3.97), the coefficient of v in Eq. (3.99) is zero, so that Eq. (3.99) becomes

$$y_1v'' + (2y_1' + py_1)v' = 0 \tag{3.100}$$

Despite its appearance, Eq. (3.100) is actually a first order equation for the function v' and can be solved either as a first order linear equation or as a separable equation. Once v' has been found, then v is obtained by an integration. Finally, y is determined from Eq. (3.98). This procedure is called the method of reduction of order, because the crucial step is the solution of a first order differential equation for v' rather than the original second order equation for y.

[SI*] **Example:**
Given that $y_1(t) = t^{-1}$ is a solution of

$$2t^2y'' + 3ty' - y = 0, \quad t > 0 \tag{3.101}$$

find a fundamental set of solutions.

Solution:
We set $y = v(t)t^{-1}$; then

$$y' = v't^{-1} - vt^{-2}$$
$$y'' = v''t^{-1} - 2v't^{-2} + 2vt^{-3}$$

Substituting for y, y', and y'' in Eq. (3.101) and collecting terms, we obtain

$$\begin{aligned} 2t^2y'' + 3ty' - y &= 2t^2(v''t^{-1} - 2v't^{-2} + 2vt^{-3}) + 3t(v't^{-1} - vt^{-2}) - vt^{-1} \\ &= 2tv'' + (-4+3)v' + (4t^{-1} - 3t^{-1} - t^{-1})v \\ &= 2tv'' - v' = 0 \end{aligned} \tag{3.102}$$

Note that the coefficient of v is zero, as it should be; this provides a useful check on our algebra.

Separating the variables in Eq. (3.102) and solving for $v'(t)$, we find that

$$v'(t) = ct^{1/2}$$

then

$$v(t) = \frac{2}{3}ct^{3/2} + k$$

It follows that

$$y = \frac{2}{3}ct^{1/2} + kt^{-1} \tag{3.103}$$

where c and k are arbitrary constants. The second term on the right side of Eq. (3.103) is a multiple of $y_1(t)$ and can be dropped, but the first term provides a new solution $y_2(t) = t^{1/2}$. You can verify that the Wronskian of y_1 and y_2 is

$$W(y_1, y_2)(t) = \frac{3}{2}t^{-3/2}, \quad t > 0 \tag{3.104}$$

Consequently, y_1 and y_2 form a fundamental set of solutions of Eq. (3.101).

```
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Differential Equations/Test SymIntegration Reduction of Order ]# ./main
********************************************************************************
Reduction of order with y_{1} (t) : t^(-1)
y : v*t^(-1)
y' : t^(-1)*v'-t^(-2)*v
y'' : t^(-1)*v''-2*v'*t^(-2)+2*t^(-3)*v
Substituting for y, y', and y'' in the solution:
2*t*v''-v' = 0
v'(t) : C*t^(0.5)
v(t) : 0.666667*C*t^(1.5)+k
Final v(t) : 0.666667*C*t^(0.5)+k*t^(-1)
Time taken by function: 132300 microseconds
```

Figure 3.15: *The computation of the second solution with reduction of order method (**SymIntegration/Examples/Differential Equations/Test SymIntegration Reduction of Order/main.cpp**).*

[SI*] **Nonhomogeneous Equations**

We now return to the nonhomogeneous equation

$$L[y] = y'' + p(t)y' + q(t)y = g(t) \tag{3.105}$$

where p, q, and g are given (continuous) functions on the open interval I. The equation

$$L[y] = y'' + p(t)y' + q(t)y = 0 \tag{3.106}$$

in which $g(t) = 0$ and p and q are the same as in Eq. (3.105), is called the homogeneous equation corresponding to Eq. (3.105).

Theorem 3.6: Solutions of the Nonhomogeneous Equation

If Y_1 and Y_2 are two solutions of the nonhomogeneous equation (3.105), then their difference $Y_1 - Y_2$ is a solution of the corresponding homogeneous equation (3.106). If, in addition, y_1 and y_2 are a fundamental set of solutions of Eq. (3.106), then

$$Y_1(t) - Y_2(t) = c_1y_1(t) + c_2y_2(t) \tag{3.107}$$

where c_1 and c_2 are certain constants.

Theorem 3.7: The General Solution of the Nonhomogeneous Equation

The general solution of the nonhomogeneous equation (3.105) can be written in the form

$$y = \phi(t) = c_1 y_1(t) + c_2 y_2(t) + Y(t) \tag{3.108}$$

where y_1 and y_2 are a fundamental set of solutions of the corresponding homogeneous equation (3.106), c_1 and c_2 are arbitrary constants, and Y is some specific solution of the nonhomogeneous equation (3.105).

To solve the nonhomogeneous equation (3.105) we must do three things:

1. Find the general solution $c_1 y_1(t) + c_2 y_2(t)$ of the corresponding homogeneous equation. This solution is frequently caled the complementary solution and may be denoted by $y_c(t)$.
2. Find some single solution $Y(t)$ of the nonhomogeneous equation. Often this solution is referred to as a particular solution.
3. Add together the functions found in the two preceding steps.

[SI*] **Method of Undetermined Coefficients**

The method of undetermined coefficients requires us to make an initial assumption about the form of the particular solution $Y(t)$, but with the coefficients left unspecified. We then substitute the assumed expression into Eq. (3.105) and attempt to determine the coefficients so as to satisfy that equation. If we are successful, then we have found a solution of the differential equation (3.105) and can use it for the particular solution $Y(t)$, If we cannot determine the coefficients, then this means that there is no solution of the form that we assumed. In this case we may modify the initial assumption and try again.

The main advantage of the method of undetermined coefficients is that it is straightforward to execute once the assumption is made as to the form of $Y(t)$. Its major limitation is that it is useful primarily for equations for which we can easily write down the correct form of the particular solution in advance. For this reason, this method is usually used only for problems in which the homogeneous equation has constant coefficients and the nonhomogeneous term is restricted to a relatively small class of functions. For more complex class of functions, a computer algebra system can be very helpful in practical applications.

If the nonhomogeneous term $g(t)$ from the Eq. (3.105) is an exponential function e^{at}, then assume that $Y(t)$ is proportional to the same exponential function; if $g(t)$ is $\sin\ \beta t$ or $\cos\ \beta t$, then assume that $Y(t)$ is a linear combination of $\sin\ \beta t$ and $\cos\ \beta t$; if $g(t)$ is a polynomial, then assume that $Y(t)$ is a polynomial of the same degree with $g(t)$. The same principle extends to the case where $g(t)$ is a product of any two, or all three, of these types of functions.

[SI*] **Example:**

Find a particular solution of

$$y'' - 3y' - 4y = 2\sin\ t \tag{3.109}$$

Solution:

We seek a function $Y(t)$ such that the combination $Y''(t) - 3Y'(t) - 4Y(t)$ is equal to $2\sin\ t$. Our assumption converning $Y(t)$ needs to include cosine as well, since the derivative of sine

and cosine are alternating between sine and cosine. Thus

$$Y(t) = A\sin\ t + B\cos\ t \tag{3.110}$$

where A and B are to be determined. Then

$$Y'(t) = A\cos\ t - B\sin\ t$$
$$Y''(t) = -A\sin\ t - B\cos\ t$$

By substituting these expression for y, y', and y'' in Eq. (3.110) and collecting terms, we obtain

$$(-A + 3B - 4A)\sin\ t + (-B - 3A - 4B)\cos\ t = 2\sin\ t \tag{3.111}$$

To satisfy Eq. (3.111), we must match the coefficients of $\sin\ t$ and $\cos\ t$ on each side of the equation; thus A and B must satisfy the equations

$$-5A + 3B = 2$$
$$-3A - 5B = 0$$

With Gaussian elimination this can be solved. Hence

$$A = -\frac{5}{17}$$
$$B = \frac{3}{17}$$

so a particular solution of Eq. (3.110) is

$$Y(t) = -\frac{5}{17}\sin\ t + \frac{3}{17}\cos\ t$$

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Nonhomogeneous Second Order Linear Differential Equations ]# ./mai
n

Solution for y'' - 3y' - 4y = 3e^{2t}


The particular solution is
Y(t) = -0.5*e^(2*t)

Solution for y'' - 3y' - 4y = 2 sin(t)


Y(t) = A*sin(t)+B*cos(t)

Y'(t) = A*cos(t)-B*sin(t)

Y''(t) = -A*sin(t)-B*cos(t)

The particular solution is
Y(t) = -0.294118*sin(t)+0.176471*cos(t)

Solution for y'' - 3y' - 4y = 5 cos(t)


Y(t) = A*sin(t)+B*cos(t)

Y'(t) = A*cos(t)-B*sin(t)

Y''(t) = -A*sin(t)-B*cos(t)

The particular solution is
Y(t) = -0.441176*sin(t)-0.735294*cos(t)

Time taken by function: 108997 microseconds
```

Figure 3.16: *The computation of the particular solution for the second order nonhomogeneous differential equation (**SymIntegration/Examples/Differential Equations/Test SymIntegration Nonhomogeneous Second Order Linear Differential Equations/main.cpp**).*

[SI*] **Example:**
Find a particular solution of

$$y'' - 3y' - 4y = 2e^{-t} \tag{3.112}$$

Solution:
We assume that $Y(t) = Ae^{-t}$, we then obtain

$$(A - 3A - 4A)e^{-t} = 2e^{-t} \tag{3.113}$$

Since the left side is zero we will obtain the solution as $\frac{2}{0}e^{-t}$, which means that there is no choice of A that satisfies this equation. Therefore, there is no particular solution of Eq. (3.112) of the assumed form. The reason for this possibly unexpected result becomes clear if we solve the homogeneous equation

$$y'' - 3y' - 4y = 0 \tag{3.114}$$

A fundamental set of solutions of Eq. (3.114) is $y_1(t) = e^{-t}$ and $y_2(t) = e^{4t}$. Thus our assumed particular solution of Eq. (3.112) is actually a solution of the homogeneous equation (3.114); consequently, it cannot possibly be a solution of the nonhomogeneous equation (3.112). To find a solution of Eq. (3.112), we must therefore consider functions of a somewhat different form.

We can simply try to guess the proper form of the particular solution of Eq. (3.112). Since the roots of the homogeneous equation are real and different, we can try to use $Y(t) = te^{-t}$, solution with form of te^{-t} occurs if we have repeated root, so it is safe to use this.

We assume that a paticular solution of the form $Y(t) = Ate^{-t}$, then

$$Y'(t) = Ae^{-t} - Ate^{-t}$$
$$Y''(t) = -2Ae^{-t} + Ate^{-t}$$

Substituting these expressions for y, y', and y'' in Eq. (3.112), we obtain

$$(-2A - 3A)e^{-t} + (A + 3A - 4A)te^{-t} = 2e^{-t}$$

Hence $-5A = 2$, so $A = -\frac{2}{5}$. Thus, a particular solution of Eq. (3.112) is

$$Y(t) = -\frac{2}{5}te^{-t}$$

```
g++ -O main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Nonhomogeneous Second Order Linear Differential Equations Special

General solution for y'' - 3y' - 4y = 0


The general solution is:
c1*e^(4*t)+c2*e^(-t)

Particular solution for y'' - 3y' - 4y = 2e^{-t}


The particular solution is
Y(t) = -0.4*t*e^(-t)

Time taken by function: 93265 microseconds
```

Figure 3.17: *The computation of the particular solution for the second order nonhomogeneous differential equation (**SymIntegration/Examples/Differential Equations/Test SymIntegration Nonhomogeneous Second Order Linear Differential Equations Special Case/main.cpp**).*

[SI*] **Variation of Parameters**

Another method to compute a particular solution of a nonhomogeneous equation is known as variation of parameters. It complements the method of undetermined coefficients rather well. The main advantage of variation of parameters is that it is a general method; in principle at least, it can be applied to any equation, and it requires no detailed assumptions about the form of the solution. On the other hand, the method of variation of parameters eventually requires us to evaluate certain integrals involving the nonhomogeneous term in the differential equation.

[SI*] **Example:**

Find a particular solution of

$$y'' + 4y = 3 \csc\ t \tag{3.115}$$

Solution:

Observe that this problem is not a good candidate for the method of undetermined coefficients, because the nonhomogeneous term $g(t) = 3\ \csc\ t$ involves a quotient (rather than a sum or a product) of $\sin\ t$ or $\cos\ t$.

Observe also that the homogeneous equation corresponding to Eq. (3.115) is

$$y'' + 4y = 0 \tag{3.116}$$

and that the general solution of Eq. (3.116) is

$$y_c(t) = c_1 \cos\ 2t + c_2 \sin\ 2t \tag{3.117}$$

The basic idea in the method of variation of parameters is to replace the constants c_1 and c_2 in Eq. (3.117) by functions $u_1(t)$ and $u_2(t)$, respectively, and then to determine these functions so that the resulting expression

$$y = u_1(t) \cos\ 2t + u_2(t) \sin\ 2t \tag{3.118}$$

is a solution of the nonhomogeneous equation (3.115).

To determine u_1 and u_2, we need to substitute for y from Eq. (3.118) in Eq. (3.115). However, even without carrying out this substitution, we can anticipate that the result will be a single equation involving some combination of u_1, u_2, and their first two derivatives. Since there is only one equation and two unknown functions, we can expect that there are many possible choices of u_1 and u_2 that will meet our needs. Alternatively, we may be able to impose a second condition of our own choosing, thereby obtaining two equations for the two unknown dunctions u_1 and u_2. We will soon show (following Lagrange) that it is possible to choose this second condition in a way that makes the computation markedly more efficient.

Returning now to Eq. (3.118), we differentiate it and rearrange the terms, thereby obtaining

$$y' = -2u_1(t) \sin\ 2t + 2u_2(t) \cos\ 2t + u_1'(t) \cos\ 2t + u_2'(t) \sin\ 2t \tag{3.119}$$

Keeping in mind the possibility of choosing a second condition on u_1 and u_2, let us require the sum of the last two terms on the right side of Eq. (3.119) to be zero; that is, we require that

$$u_1'(t) \cos\ 2t + u_2'(t) \sin\ 2t = 0 \tag{3.120}$$

It then follows from Eq. (3.119) that

$$y' = -2u_1(t)\sin\ 2t + 2u_2(t)\cos\ 2t \tag{3.121}$$

Although the ultimate effect of the condition (3.120) is not yet clear, at the very least it has simplified the expression for y'. Further, by differentiating Eq. (3.121), we obtain

$$y'' = -4u_1(t)\cos\ 2t - 4u_2(t)\sin\ 2t - 2u_1'(t)\sin\ 2t + 2u_2'(t)\cos\ 2t \tag{3.122}$$

Then, substituting for y and y'' in Eq. (3.115) from Eq. (3.118) and (3.122), respectively, we find that u_1 and u_2 must satisfy

$$-2u_1'(t)\sin\ 2t + 2u_2'(t)\cos\ 2t = 3\csc\ t \tag{3.123}$$

We want to choose u_1 and u_2 to satisfy Eqs. (3.120) and (3.123). These equations can be viewed as a pair of linear algebraic equations for the two unknown quantities $u_1'(t)$ and $u_2'(t)$. Equations (3.120) and (3.123) can be solved in various ways. For example, solving Eq. (3.120) for $u_2'(t)$, we have

$$u_2'(t) = u_1'(t)\frac{\cos\ 2t}{\sin\ 2t} \tag{3.124}$$

Then, substituting for $u_2'(t)$ in Eq. (3.123)and simplifying, we obtain

$$u_1'(t) = -\frac{3\csc\ t\ \sin\ 2t}{2} = -3\cos\ t \tag{3.125}$$

Further, putting this expression for $u_1'(t)$ back in Eq. (3.124) and using the double-angle formulas, we find that

$$u_2'(t) = \frac{3\cos\ t\ \cos\ 2t}{\sin\ 2t} = \frac{3(1-2\sin^2 t)}{2\sin\ t} = \frac{3}{2}\csc\ t - 3\sin\ t \tag{3.126}$$

Having obtained $u_1'(t)$ and $u_2'(t)$, we next integrate so as to find $u_1(t)$ and $u_2(t)$. The result is

$$u_1(t) = -3\sin\ t + c_1 \tag{3.127}$$

and

$$u_2(t) = \frac{3}{2}\ln|\csc\ t - \cot\ t| + 3\cos\ t + c_2 \tag{3.128}$$

On substituting these expressions in Eq. (3.118), we have

$$\begin{aligned} y = &-3\sin\ t\ \cos\ 2t + \frac{3}{2}\ln|\csc\ t - \cot\ t|\sin\ 2t + 3\cos\ t\sin\ 2t \\ &+ c_1\cos\ 2t + c_2\sin\ 2t \end{aligned}$$

Finally, by using the double-angle formula once more, we obtain

$$y = 3\sin\ t + \frac{3}{2}\ln|\csc\ t - \cot\ t|\sin\ 2t + c_1\cos\ 2t + c_2\sin\ 2t \tag{3.129}$$

The terms in Eq. (3.129) involving the arbitrary constants c_1 and c_2 are the general solution of the corresponding homogeneous equation, while the remaining terms are a particular solution of the nonhomogeneous equation (3.115). Thus Eq. (3.129) is the general solution of Eq. (3.115).

[SI*] Consider

$$y'' + p(t)y' + q(t)y = g(t) \tag{3.130}$$

where p, q, and g are given continuous functions. As a starting point, we assume that we know the general solution

$$y_c(t) = c_1 y_1(t) + c_2 y_2(t) \tag{3.131}$$

of the corresponding homogeneous equation

$$y'' + p(t)y' + q(t)y = 0 \tag{3.132}$$

The crucial idea is to replace the constants c_1 and c_2 in Eq. (3.131) by functions $u_1(t)$ and $u_2(t)$, respectively; this gives

$$y = u_1(t)y_1(t) + u_2(t)y_2(t) \tag{3.133}$$

Then we try to determine $u_1(t)$ and $u_2(t)$ so that the expression in Eq. (3.133) is a solution of the nonhomogeneous equation (3.130) rather than the homogeneous equation (3.132). Thus we differentiate Eq. (3.133), obtaining

$$y' = u_1'(t)y_1(t) + u_1(t)y_1'(t) + u_2'(t)y_2(t) + u_2(t)y_2'(t) \tag{3.134}$$

We now set the terms involving $u_1'(t)$ and $u_2'(t)$ in Eq. (3.134) equal to zero; that is, we require that

$$u_1'(t)y_1(t) + u_2'(t)y_2(t) = 0 \tag{3.135}$$

Then, from Eq. (3.134), we have

$$y' = u_1(t)y_1'(t) + u_2(t)y_2'(t) \tag{3.136}$$

Further, by differentiating again, we obtain

$$y'' = u_1'(t)y_1'(t) + u_1(t)y_1''(t) + u_2'(t)y_2'(t) + u_2(t)y_2''(t) \tag{3.137}$$

Now, we substitute for y, y', and y'' in Eq. (3.130) from Eqs. (3.133), (3.136), and (3.137), respectively. After rearranging the terms in the resulting equation, we find that

$$\begin{aligned} &u_1(t)\left[y_1''(t) + p(t)y_1'(t) + q(t)y_1(t)\right] \\ +&u_2(t)\left[y_2''(t) + p(t)y_2'(t) + q(t)y_2(t)\right] \\ &+u_1'(t)y_1'(t) + u_2'(t)y_2'(t) = g(t) \end{aligned} \tag{3.138}$$

Each of the expression in square brackets in Eq. (3.138) is zero because y_1 and y_2 are solutions of the homogeneous equation (3.132). Therefore Eq. (3.138) reduces to

$$u_1'(t)y_1'(t) + u_2'(t)y_2'(t) = g(t) \tag{3.139}$$

Equations (3.135) and (3.139) form a system of two linear algebraic equations for the derivatives $u_1'(t)$ and $u_2'(t)$ of the unknown functions.

By solving the system (3.135), (3.139) we obtain

$$\begin{aligned} u_1'(t) &= -\frac{y_2(t)g(t)}{W(y_1, y_2)(t)} \\ u_2'(t) &= \frac{y_1(t)g(t)}{W(y_1, y_2)(t)} \end{aligned} \tag{3.140}$$

where $W(y_1, y_2)$ is the Wronskian of y_1 and y_2. Note that division by W is permissible since y_1 and y_2 are a fundamental set of solutions, and therefore their Wronskian is nonzero. By integrating Eqs (3.140), we find the desired functions $u_1(t)$ and $u_2(t)$, namely

$$\begin{aligned} u_1'(t) &= -\int \frac{y_2(t)g(t)}{W(y_1,y_2)(t)}\,dt + c_1 \\ u_2'(t) &= \int \frac{y_1(t)g(t)}{W(y_1,y_2)(t)}\,dt + c_2 \end{aligned} \tag{3.141}$$

If the integrals in Eqs. (3.141) can be evaluated in terms of elementary functions, then we substitute the results in Eq. (3.133), thereby obtaining the general solution of Eq. (3.130).

Theorem 3.8: Variation of Parameters

If the functions p, q, and g are continuous on an open interval I, and if the functions y_1 and y_2 are a fundamental set of solutions of the homogeneous equation (3.132) corresponding to the nonhomogeneous equation (3.130)

$$y'' + p(t)y' + q(t)y = g(t)$$

then a particular solution of Eq. (3.130) is

$$Y(t) = -y_1(t)\int_{t_0}^{t} \frac{y_2(s)g(s)}{W(y_1,y_2)(s)}\,ds + y_2(t)\int_{t_0}^{t} \frac{y_1(s)g(s)}{W(y_1,y_2)(s)}\,ds \tag{3.142}$$

where t_0 is any conveniently chosen point in I. The general solution is

$$y = c_1y_1(t) + c_2y_2(t) + Y(t) \tag{3.143}$$

[SI*] **Mechanical and Electrical Vibrations**

The motion of a mass on a vibrating spring, the torsional oscillations of a shaft with a flywheel, the flow of electric current in a simple series circuit, and many other physical problems are all described by the solution of an initial value problem of the form

$$ay'' + by' + cy = gt, \quad y(0) = y_0, \quad y'(0) = y_0' \tag{3.144}$$

This illustrates a fundamental relationship between mathematics and physics; many physical problems may have the same mathematical model. Thus, once we know how to solve the initial value problem (3.144), it is only necessary to make appropriate interpretations of the constants a, b, and c, and of the functions y and g, to obtain solutions of different physical problems.

The Motion of a Mass on a Spring

Consider a mass m hanging on the end of a vertical spring of original length l. The mass causes an elongation L of the spring in the downward (positive) direction. There are two forces acting at the point where the mass is attached to the spring; the gravitational force, or weight of the mass, acts downward and has magnitude mg, where g is the acceleration due to gravity. There is also a force F, due to the spring, that acts upward. If we assume that the elongation L of the spring is small, the spring force is very nearly proportional to L; this is known as Hooke's law. Thus we write $F_s = -kL$, where the constant of proportionality k is called the spring constant, and the minus sign is due to the fact that the spring force acts in

the upward (negative) direction. Since the mass is in equilibrium, the two forces balance each other, which means that

$$mg - kL = 0 \tag{3.145}$$

For a given weight $w = mg$, one can measure L and then use Eq. (3.145) to determine k. Note that k has the units of force/length.

In the corresponding dynamic problem, we are interested in studying the motion of the mass when it is acted on by an external force or is initially displaced. Let $u(t)$, measured positive downward, denote the displacement of the mass from its equilibrium position at time t. Then $u(t)$ is related to the forces acting on the mass through Newton's law of motion

$$mu''(t) = f(t) \tag{3.146}$$

where u'' is the acceleration of the mass and f is the net force acting on the mass. Observe that both u and f are functions of time. In determining f there are four separate forces that must be considered:

1. The weight $w = mg$ of the mass that always acts downward.
2. The spring force F_s, is assumed to be proportional to the total elongation $L + u$ of the spring and always acts to restore the spring to its natural position. If $L + u > 0$, then the spring is extended, and the spring force is directed upward. In this case

   $$F_s = -k(L + u) \tag{3.147}$$

   On the other hand, if $L + u < 0$, then the spring is compressed a distance $|L + u|$, and the spring force, which is now directed downward, is given by $F_s = k|L + u|$. However, when $L + u < 0$, it follows that $|L + u| = -(L + u)$, so F_s is again given by Eq. (3.147). Thus, regardless of the position of the mass, the force exerted by the spring is always expressed by Eq. (3.147).
3. The damping or resistive foce F_d always acts in the direction opposite to the direction of motion of the mass. This force may arise from several sources: resistance from the air or other medium in which the mass moves, internal energy dissipation due to the extension or compression of the spring, friction between the mass and the guides (if any) that constrain its motion to one dimension, or a mechanical device (dashpot) that imparts a resistive force to the mass. In any case, we assume that the resistive force is proportional to the speed $\left|\frac{du}{dt}\right|$ of the mass; this is usually referred to as viscous damping. If $\frac{du}{dt} > 0$, then u is increasing, so the mass is moving downward. Then F_d is directed upward and is given by

   $$F_d(t) = -\gamma u'(t) \tag{3.148}$$

   where γ is a positive constant of proportionality known as the damping constant. On the other hand, if $\frac{du}{dt} < 0$, then u is decreasing, the mass is moving upward, and F_d is directed downward. In this case, $F_d = \gamma|u'(t)|$; since $|u'(t)| = -u'(t)$, it follows that $F_d(t)$ is again given by Eq. (3.148). Thus, regardless of the direction of motion of the mass, the damping force is always expressed by Eq. (3.148).
   The damping force may be rather complicated, and the assumption that it is modeled adequately by Eq. (3.148) may be open to question. Some dashpots do behave as Eq. (3.148) states, and if the other sources of dissipation are small, it may be possible to

neglect them altogether or to adjust the damping constant γ to approximate them. An important benefit of the assumption (3.148) is that it leads to a linear (rather than a nonlinear) differential equation.

4. An applied external force $F(t)$ is directed downward or upward as $F(t)$ is positive or negative. This could be a force due to the motion of the mount to which the spring is attached, or it could be a force applied directly to the mass. Often the external force is periodic.

Taking account of these forces, we can now rewrite Newton's law (3.146) as

$$\begin{aligned} mu''(t) &= mg + F_s(t) + F_d(t) + F(t) \\ &= mg - k[L + u(t)] - \gamma u'(t) + F(t) \end{aligned} \tag{3.149}$$

Since $mg - kL = 0$ by Eq. (3.145), it follows that the equation of motion of the mass is

$$mu''(t) + \gamma u'(t) + ku(t) = F(t) \tag{3.150}$$

where the constants m, γ, and k are positive. Note that Eq. (3.150) has the same form as Eq. (3.144).

It is important to understand that Eq. (3.150) is only an approximate equation for the displacement $u(t)$. In particular, both Eqs. (3.147) and (3.148) should be viewed as approximations for the spring force and the damping force, respectively. In our derivation we have also neglected the mass of the spring in comparison with the mass of the attached body.

The complete formulation of the vibration problem requires that we specify two initial conditions, namely, the initial position u_0 and the initial velocity v_0 of the mass:

$$u(0) = u_0, \quad u'(0) = v_0 \tag{3.151}$$

These conditions give a mathematical problem that has a unique solution. This is consistent with our physical intuition that if the mass is set in motion with a given initial displacement and velocity, then its position will be determined uniquely at all future times.

[SI*] **Example:**

A mass weighing 4 lb stretches a sring 2 in. Suppose that the mass is displaced an additional 6 in. in the positive direction and then released. The mass is in a medium that exerts a viscous resistance of 6 lb when the mass has a velocity of 3 ft/s. Formulate the initial value problem that governs the motion of the mass.

Solution:

The required initial value problem consists of the differential equation (3.150) and initial conditions (3.151), so our task is to determine the various constants that appear in these equations. The first step is to choose the units of measurement. Based on the statement of the problem, it is natural to use English rather than metric system of units. The only time unit mentioned is the second, so we will measure t in seconds. On the other hand, both the foot and the inch appear in the statement as units of length. It is imamterial which one we use, but having made a choice, we must be consistent. To be definite, let us measure the displacement u in feet.

Since nothing is said in the statement of the problem about an external force, we assume that $F(t) = 0$. To determine m, note that

$$m = \frac{w}{g} = \frac{4 \text{ lb}}{32 \text{ ft/s}^2} = \frac{1}{8} \frac{\text{lb} \cdot \text{s}^2}{\text{ft}}$$

The damping coefficient γ is determined from the statement that $\gamma u'$ is equal to 6 lb when u' is 3 ft/s. Therefore

$$\gamma = -\frac{F_d(t)}{u'(t)} = \frac{6 \text{ lb}}{3 \text{ ft/s}} = 2\frac{\text{lb} \cdot \text{s}}{\text{ft}}$$

The spring constant k is found from the statement that the mass stretches the spring by 2 in., or $\frac{1}{6}$ ft. Thus

$$k = \frac{w}{L} = \frac{4 \text{ lb}}{\frac{1}{6} \text{ ft}} = 24\frac{\text{lb}}{\text{ft}}$$

Consequently, Eq. (3.150) becomes

$$\frac{1}{8}u'' + 2u' + 24u = 0$$

or

$$u'' + 16u' + 192u = 0 \tag{3.152}$$

The initial conditions are

$$u(0) = \frac{1}{2}, \quad u'(0) = 0 \tag{3.153}$$

We obtain $\frac{1}{2}$ from the initial displacement of the mass that is about 6 in (6 in. = $\frac{1}{2}$ ft).
The second initial condition is implied by the word "release" in the statement of the problem, which we interpret to mean that the mass is set in motion with no initial velocity.

Figure 3.18: *The simulation of a spring-mass system with Hamzstlab Physics (modifying Box2D source code), if we are assuming the on screen simulation have metric in inch, then the simulation for this example won't oscillate at all after we displaced the mass downward of 6 inch, it will return after the first displacement, the damping coefficient γ is too big / strong thus it does not oscillate. (**Hamzstlab-Physics2D/tests/ode_verticalspringvibrations.cpp**).*

Undamped Free Vibrations

If there is no external force, then $F(t) = 0$ in Eq. (3.150). Let us also suppose that there is no damping, so that $\gamma = 0$; this is an idealized configuration of the system, seldom (if ever) completely attainable in practice. However, if the actual damping is very small, then the assumption of no damping may yield satisfactoy results over short to moderate time intervals. In this case the equation of motion (3.150) reduces to

$$mu'' + ku = 0 \tag{3.154}$$

The general solution of Eq. (3.154) is

$$u = A\cos\omega_0 t + B\sin\omega_0 t \tag{3.155}$$

where

$$\omega_0^2 = \frac{k}{m} \tag{3.156}$$

The arbitrary constants A and B can be determined if initial conditions of the form (3.151) are given.

In discussing the solution of Eq. (3.154), it is convenient to rewrite Eq. (3.155) in the form

$$u = R\cos(\omega_0 t - \delta) \tag{3.157}$$

or

$$u = R\cos\ \delta\cos\ \omega_0 t + R\sin\ \delta\sin\ \omega_0 t \tag{3.158}$$

By comparing Eq. (3.158) with Eq. (3.155), we find that A, B, R, and δ are related by the equations

$$A = R\cos\ \delta, \quad B = R\sin\ \delta \tag{3.159}$$

Thus

$$R = \sqrt{A^2 + B^2}, \quad \tan\ \delta = \frac{B}{A} \tag{3.160}$$

In calculating δ, we must take care to choose the correct quadrant; this can be done by checking the signs of cos δ and sin δ in Eqs. (3.159).

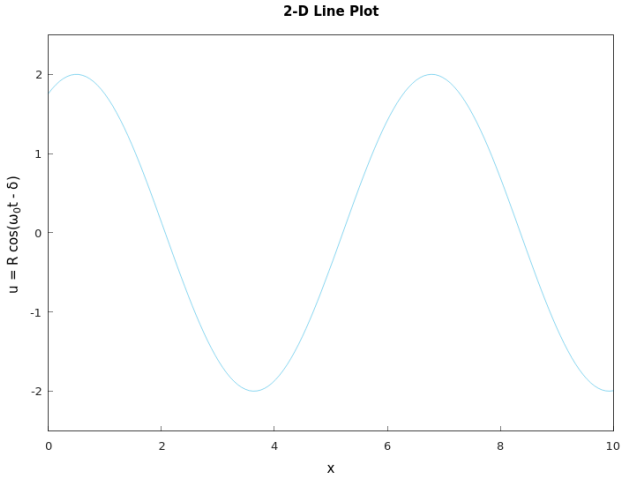

Figure 3.19: *The graph of Eq. (3.157), or the equivalent Eq. (3.155), for a typical set of initial conditions is shown. The graph is a displaced cosine wave that describes a periodic or simple harmonic, motion of the mass. (**Hamzstplot/Examples with Makefile/Differential Equations/Plot 2D Undamped Free Vibrations/main.cpp**).*

The period of the motion is

$$T = \frac{2\pi}{\omega_0} = 2\pi\left(\frac{m}{k}\right)^{1/2} \tag{3.161}$$

The circular frequency $\omega_0 = \sqrt{\frac{k}{m}}$, measured in radians per unit time, is called the natural frequency of the vibration. The maximum displacement R of the mass from equilibrium is the amplitude of the motion. The dimensionless parameter δ is called the phase, or phase angle, and measures the displacement of the wave from its normal position corresponding to $\delta = 0$.

The graph $u(t) = R\cos(\omega_0 t - \delta)$ reaches its highest / amplitude R at $\delta, \delta + n\pi$ with $n = 2, 4, 6, 8, \ldots$, and reaches the lowest / $-R$ at $\delta + m\pi$ with $m = 1, 3, 5, 7, \ldots$.
Note that the motion described by Eq. (3.157) has a constant amplitude that does not diminish with time. This reflects the fact that, in the absence of damping, there is no way for the system to dissipate the energy imparted to it by the initial displacement and velocity. Further, for a given mass m and spring constant k, the system always vibrates at the same frequency ω_0, regardless of the initial conditions. However, the initial conditions do help to determine the amplitude of the motion. Finally, observe from Eq. (3.161) that T increases as m increases, so larger masses vibrate more slowl. On the other hand, T decreases as k increases, which means that stiffer springs cause the system to vibrate more rapidly.

[SI*] **Example:**
Suppose that a mass weighing 10 lb stretches a spring 2 in. If the mass is displaced an additional 2 in. and is then set in motion with an initial upward velocity of 1 ft/s, determine the position of the mass at any later time. Also determine the period, amplitude, and phase of the motion.

Solution:
The spring constant is $k = 10$ lb / 2 in. = 60 lb/ft, and the mass is $m = \frac{w}{g} = \frac{10}{32}$ lb·s^2/ft. Hence the equation of the motion reduces to

$$u'' + 192u = 0 \tag{3.162}$$

and the general solution is

$$u = A\cos(8\sqrt{3}t) + B\sin(8\sqrt{3}t)$$

The solution satisfying the initial conditions $u(0) = \frac{1}{6}$ ft and $u'(0) = -1$ ft/s is

$$u = \frac{1}{6}\cos(8\sqrt{3}) - \frac{1}{8\sqrt{3}}\sin(8\sqrt{3}) \tag{3.163}$$

The natural frequency is $\omega_0 = \sqrt{192} \approxeq 13.856$ rad/s, so the period is $T = \frac{2\pi}{\omega_0} \approxeq 0.45345$ s. The amplitude R and phase δ are found from Eqs. (3.160). We have

$$\begin{aligned} R^2 &= \frac{1}{36} + \frac{1}{192} \\ &= \frac{19}{576} \\ &\approxeq 0.18162 \text{ ft} \end{aligned}$$

The seconds of Eqs. (3.160) yields tan $\delta = -\frac{\sqrt{3}}{4}$. There are two solutions of this equation, one in the second quadrant and one in the fourth. In the present problem cos $\delta > 0$ and sin $\delta < 0$, so δ is in the fourth quadrant, namely,

$$\delta = -\arctan\left(\frac{\sqrt{3}}{4}\right) \approxeq -0.40864 \text{ rad}$$

Damped Free Vibrations
If we include the effect of damping, the differential equation governing the motion of the mass is

$$mu'' + \gamma u' + ku = 0 \tag{3.164}$$

We are especially interested in examining the effect of variations in the damping coefficient γ for given values of the mass m and spring constant k. The roots of the corresponding characteristic equation are

$$r_1, r_2 = \frac{-\gamma \pm \sqrt{\gamma^2 - 4km}}{2m} \tag{3.165}$$

Depending on the sign of $\gamma^2 - 4km$, the solution u has one of the following forms:

$$\gamma^2 - 4km > 0, \quad u(t) = Ae^{r_1 t} + Be^{r_2 t} \tag{3.166}$$

with

$$B = \frac{u'(0) - r_1 u(0)}{r_2 - r_1}$$
$$A = u(0) - B$$

the next form is

$$\gamma^2 - 4km = 0, \quad u(t) = (A + Bt)e^{-\frac{\gamma t}{2m}} \tag{3.167}$$

with

$$A = u(0)$$
$$B = u'(0) + \frac{\gamma}{2m}A$$

and the last form

$$\gamma^2 - 4km < 0, \quad u(t) = e^{-\frac{\gamma t}{2m}}(A\cos\ \mu t + B\sin\ \mu t), \quad \mu = \frac{\sqrt{4km - \gamma^2}}{2m} > 0 \tag{3.168}$$

with

$$A = u(0)$$
$$B = \frac{u'(0) + \frac{\gamma}{2m}A}{\mu}$$

Since m, γ, and k are positive, $\gamma^2 - 4km$ is always less than γ^2. Hence, if $\gamma^2 - 4km \geq 0$, then the values of r_1 and r_2 given by Eq. (3.165) are negative. If $\gamma^2 - 4km < 0$, then the values of r_1 and r_2 are complex, but with negative real part. Thus, in all cases, the solution u tends to zero as $t \to \infty$; this occurs regardless of the values of the arbitrary constants A and B, that is, regardless of the initial conditions. This confirms our intuitive expectation, namely, that

damping gradually dissipates the energy initially imparted to the system, and consequently the motion dies out with increasing time.

The most important case is the third one, which occurs when the damping is small. If we let $A = R\cos\ \delta$ and $B = R\sin\ \delta$ in Eq. (3.168), then we obtain

$$u(t) = Re^{-\frac{\gamma t}{2m}}\cos(\mu t - \delta) \tag{3.169}$$

The displacement u lies between the curves $u = \pm Re^{-\frac{\gamma t}{2m}}$; hence it resembles a cosine wave whose amplitude decreaseas as t increases. The motion is called a damped oscillation or a damped vibration. The amplitude factor R depends on m, γ, k, and the initial conditions.

Although the motion is not periodic, the parameter μ determines the frequency with which the mass oscillates back and forth; consequently, μ is called the quasi frequency. By comparing μ with the frequency ω_0 of undamped motion, we find that

$$\frac{\mu}{\omega_0} = \frac{\frac{\sqrt{4km-\gamma^2}}{2m}}{\sqrt{k/m}} = \sqrt{1 - \frac{\gamma^2}{4km}} \approxeq 1 - \frac{\gamma^2}{8km} \tag{3.170}$$

The last approximation is valued when $\frac{\gamma^2}{4km}$ is small; we refer to this situation as "small damping." Thus the effect of small damping is to reduce slightly the frequency of the oscillation. By analogy with Eq. (3.161), the quantity $T_d = \frac{2\pi}{\mu}$ is called the quasi period. It is the time between successive maxima or successive minima of the position of the mass, or between successive passages of the mass through its equilibrium position while going in the same direction. The relation between T_d and T is given by

$$\frac{T_d}{T} = \frac{\omega_0}{\mu} = \left(1 - \frac{\gamma^2}{4km}\right)^{-1/2} \approxeq 1 + \frac{\gamma^2}{8km} \tag{3.171}$$

where again the last approximation is valid when $\frac{\gamma}{4km}$ is small. thus small damping increases the quasi period.

Equations (3.170) and (3.171) reinforce the significance of the dimensionless ratio $\frac{\gamma^2}{4km}$. It is not the magnitude of γ alone that determines wheter damping is large or small, but the magnitude of γ^2 compared to $4km$. When $\frac{\gamma^2}{4km}$ is small, then the damping has a small effect on the quasi frequency and quasi period of the motion. On the other hand, if we want to study the detailed motion of the mass for all time, then we can never neglect the damping force, no matter how small.

As $\frac{\gamma^2}{4km}$ increases, the quasi frequency μ decreases and the quasi peiod T_d increases. In fact, $\mu \to 0$ and $T_d \to \infty$ as $\gamma \to 2\sqrt{km}$. As indicated by Eqs. (3.166), (3.167), and (3.168), the nature of the solution changes as γ passes through the value $2\sqrt{km}$. This value is known as critical damping, while for larger values of γ the motion is said to be overdamped. In these cases, given by Eqs. (3.167) and (3.166), respectively, the mass creeps back to its equilibrium position but does not oscillate about it, as for small γ.

[SI*] **Example:**
The motion of a certain spring-mass system is governed by the differential equation

$$u'' + 0.125u' + u = 0 \tag{3.172}$$

where u is measured in feet and t in seconds. If $u(0) = 2$ and $u'(0) = 0$, determine the position of the mass at any time. Find the quasi frequency and the quasi period, as well as the time at which the mass first passes through its equilibrium position. Also find the time τ such that $|u(t)| < 0.1$ for all $t > \tau$.

Solution:
The solution of Eq. (3.172) is

$$u(t) = e^{-t/16}\left[A\cos\left(\frac{\sqrt{255}}{16}\right)t + B\sin\left(\frac{\sqrt{255}}{16}\right)t\right]$$

To satisfy the initial conditions, we must choose $A = 2$ and $B = \frac{2}{\sqrt{255}}$; hence the solution of the initial value problem is

$$\begin{aligned} u(t) &= e^{-t/16}\left[2\cos\left(\frac{\sqrt{255}}{16}\right)t + \frac{2}{\sqrt{255}}\sin\left(\frac{\sqrt{255}}{16}\right)t\right] \\ &= \frac{32}{\sqrt{255}}e^{-t/16}\cos\left(\frac{\sqrt{255}}{16}t - \delta\right) \end{aligned} \tag{3.173}$$

where $\tan\ \delta = \frac{1}{\sqrt{255}}$, so $\delta \approxeq 0.06254$.

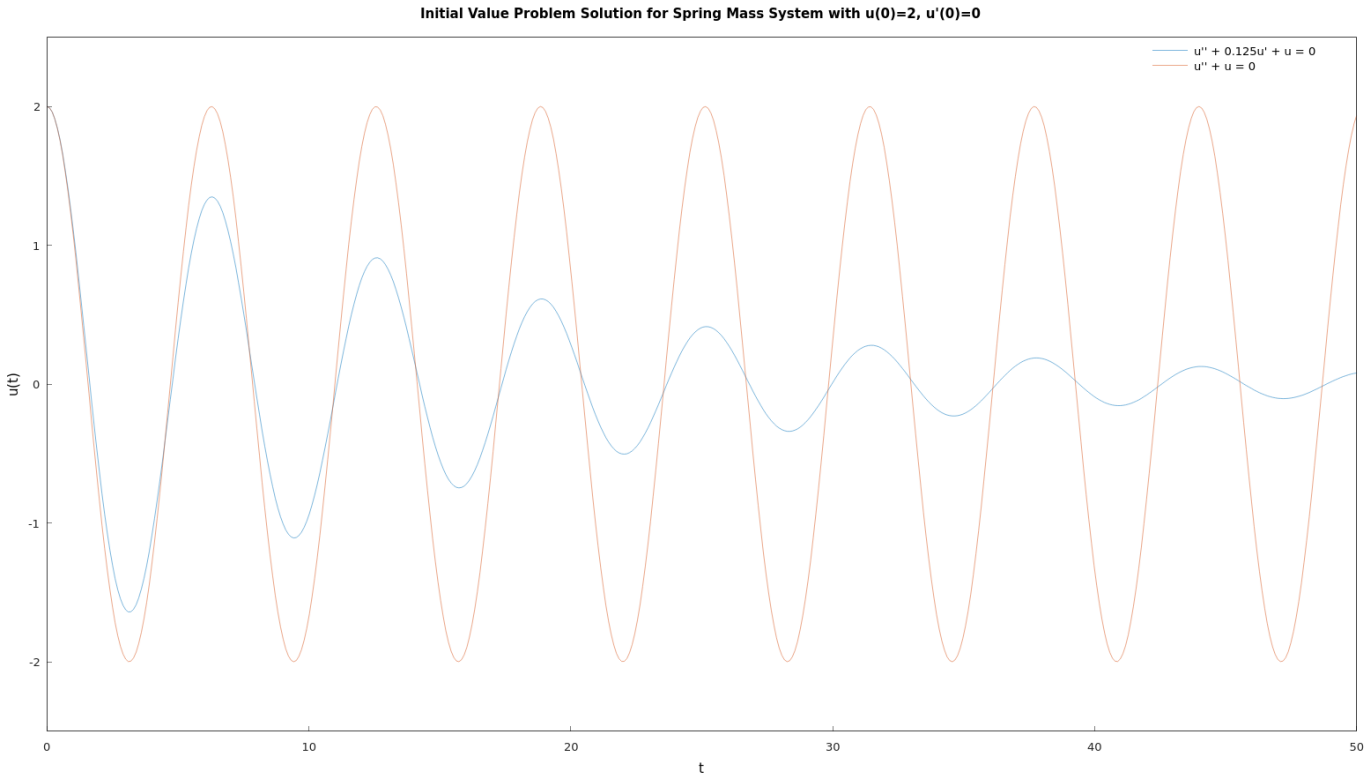

Figure 3.20: *The figure shows the displacement of the mass as a function of time. For purposes of comparison, we also show the motion if the damping term is neglected. (**Hamzstplot/Examples with Makefile/Differential Equations/Plot 2D Vibration with Small Damping and With No Damping Second Order ODE/main.cpp**).*

The quasi frequency is $\mu = \frac{\sqrt{255}}{16} \approxeq 0.998$ and the quasi period is

$$T_d = \frac{2\pi}{\mu} \approxeq 6.295 \text{ s}$$

These values differ only slightly from the corresponding values (1 and 2π, respectively) for the undamped oscillation.

The damping coefficient is small in this example, only one-sixteenth of the critical value, in fact. Nevertheless, the amplitude of the oscillation is reduced rather rapidly.

To find the time at which the mass first passes through its equilibrium position, we refer to Eq. (3.173) and set $\frac{\sqrt{255}}{16} - \delta$ to $\frac{\pi}{2}$, the smallest positive zero of the cosine function. Then, by solving for t, we obtain

$$t = \frac{16}{\sqrt{255}} \left(\frac{\pi}{2} + \delta \right) \approxeq 1.637 \text{ s}$$

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Spring Mass System Second Order Linear Differential Equations ]# ./main

Spring mass system solution for y'' + 0.125y'+ y = 0 with u(0) = 2, u'(0) = 0


The real-valued initial value problem solution is:

u (t) = 2*e^(-0.0625*t)*cos(0.998045*t)+0.125245*e^(-0.0625*t)*sin(0.998045*t)

δ = 0.0625408

T_{d} = 6.29549

The time when the mass passes through its equilibrium position:
t = 1.63654

Spring mass system solution for y'' + y = 0 with u(0) = 2, u'(0) = 0


The real-valued initial value problem solution is:

u (t) = 2*cos(t)

δ = 0

T_{d} = 6.28319

The time when the mass passes through its equilibrium position:
t = 1.5708
```

Figure 3.21: *The computation of initial value problem solution for the second order linear differential equation with small damping and no damping (**SymIntegration/Examples/Differential Equations/Test SymIntegration Spring Mass System Second Order Linear Differential Equations/main.cpp**).*

Electric Circuits

A second example of the occurrence of second order linear differential equations with constant coefficients is their use as a model of the flow of electric current in the simple series circuit. The current I, measured in amperes (A), is a function of time t. The resistance R in ohms (Ω), the capacitance C in farads (F), and the inductance L in Henrys (H) are all positive and are assumed to be known constants. the impressed voltage E in volts (V) is a given function of time. Another physical quantity that enters the discussion is the total charge Q in coulombs (C) on the capacitor at time t. The relation between charge Q and current I is

$$I = \frac{dQ}{dt} \tag{3.174}$$

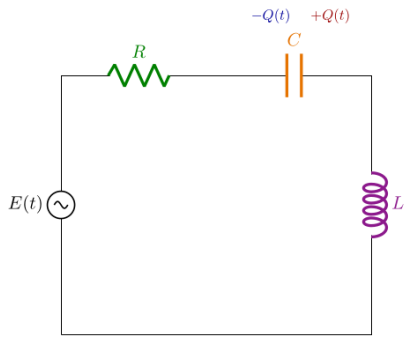

Figure 3.22: *A simple RLC series electric circuit.*

The flow of current in the circuit is governed by Kirchoff's second law: In a closed circuit the impressed voltage is equal to the sum of the voltage drops in the rest of the circuit.

According to the elementary laws of electricity, we know that
The voltage drop across the resistor is IR.

The voltage drop across the capacitor is $\frac{Q}{C}$.
The voltage drop across the inductor is $L\frac{dI}{dt}$.

Hence, by Kirchhoff's law,

$$L\frac{dI}{dt} + RI + \frac{1}{C}Q = E(t) \tag{3.175}$$

The units have been chosen so that 1 volt = 1 ohm ·1 ampere = 1 coulomb/1 farad = 1 henry ·1 ampere/1 second.

Substituting for I from Eq. (3.174), we obtain the differential equation

$$LQ'' + RQ' + \frac{1}{C}Q = E(t) \tag{3.176}$$

for the charge Q. The initial conditions are

$$Q(t_0) = Q_0, \quad Q'(t_0) = I(t_0) = I_0 \tag{3.177}$$

Thus we must know the charge on the capacitor and the current in the circuit at some initial time t_0.
Alternatively, we can obtain a differential equation for the current I by differentiating Eq. (3.176) with respect to t, and then substituting for $\frac{dQ}{dt}$ from Eq. (3.174). The result is

$$LI'' + RI' + \frac{1}{C}I = E(t) \tag{3.178}$$

with the initial conditions

$$I(t_0) = I_0, \quad I'(t_0) = I_0' \tag{3.179}$$

From Eq. (3.175) it follows that

$$I_0' = \frac{E(t_0) - RI_0 - \left(\frac{1}{C}\right)Q_0}{L} \tag{3.180}$$

Hence I_0' is also determined by the initial charge and current, which are physically measurable quantities.
The most important conclusion form this discussion is that the flow of current in the circuit is described by an initial value problem of precisely the same form as the one that describes the motion of a spring-mass system. This is a good example of the unifying role of mathematics: Once you know how to solve second order linear differential equations with constant coefficients, you can interpret the results in terms of mechanical vibrations, electric circuits, or any other physical situation that leads to the same problem.

[SI*] **Example:**
A series circuit has a capacitor of 10^{-5} F, a resistor of $3 \times 10^2\Omega$, and an inductor of 0.2 H. The initial charge on the capacitor is 10^{-6} C and there is no initial current. Find the charge Q on the capacitor at any time t.

Solution:

Assume that the circuit is closed at $t = 0$. Apply Faraday's law to obtain the governing ODE for the current.

$$\sum V = -L\frac{di}{dt}$$

The only potential drops occur over the resistor and the capacitor.

$$iR + \frac{q}{C} = -L\frac{di}{dt}$$

Write $i = \frac{dq}{dt} = q'$.

$$Rq' + \frac{q}{C} = -Lq''$$

Bring Lq'' to the left side and divide both sides by L.

$$q'' + \frac{R}{L}q' + \frac{1}{LC}q = 0 \tag{3.181}$$

Since the coefficients are constant and this is a homogeneous ODE, the solutions are of the form $q = e^{rt}$.

$$q = e^{rt} \rightarrow q' = re^{rt} \rightarrow q'' = r^2e^{rt}$$

Substitute these expressions to obtain an algebraic equation for r.

$$r^2e^{rt} + \frac{R}{L}(re^{rt}) + \frac{1}{LC}(e^{rt}) = 0$$

Divide both sides by e^{rt}.

$$\begin{aligned} r^2 + \frac{R}{L}(re^{rt}) + \frac{1}{LC}(e^{rt}) &= 0 \\ r &= \frac{-\frac{R}{L} \pm \sqrt{\frac{R^2}{L^2} - \frac{4}{LC}}}{2} \\ &\frac{-1500 \pm \sqrt{250000}}{2} \\ &= \frac{-1500 \pm 500}{2} \\ &= -750 \pm 250 \end{aligned}$$

so $r = \{-1000, 500\}$. Two solutions to equation (3.181) are then

$$q(t) = e^{-1000t}$$
$$q(t) = e^{-500t}$$

By the principle of superposition, the general solution is a linear combination of these two.

$$q(t) = C_1e^{-1000t} + C_2e^{-500t}$$

Take a derivative of it with respect to t.

$$q'(t) = -1000C_1e^{-1000t} - 500C_2e^{-500t}$$

Now apply the initial conditions, $q(0) = 10^{-6}$ and $q'(0) = 0$, to determine C_1 and C_2.

$$q(0) = C_1 + C_2 = 10^{-6}$$
$$q'(0) = -1000C_1 - 500C_2 = 0$$

Solving this system of equations yields $C_1 = -10^{-6}$ and $C_2 = 2 \times 10^{-6}$. Therefore,

$$q(t) = -10^{-6}e^{-1000t} + 2 \times 10^{-6}e^{-500t}$$

Note that the charge $q(t)$ is in Coulombs (C).

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration RLC Series Electric Circuit Second Order Linear Differential Equations ]# ./main
RLC series electric circuit solution for R = 0, L = 1 H, C = 0.25 * 10^{-6} F with q(0) = 10^{-6}, q'(0) = 0

The solution for the initial value problem / the charge Q at any time t is:

q (t) = 1e-06*cos(2000*t)

RLC series electric circuit solution for R = 3* 10^{2} Ohm, L = 0.2 H, C = 10^{-5} F with q(0) = 10^{-6}, q'(0) = 0

The general solution is:
c1*e^(-500*t)+c2*e^(-1000*t)

The solution for the initial value problem / the charge Q at any time t is:

q(t) = 2e-06*e^(-500*t)-1e-06*e^(-1000*t)

Time taken by function: 43347 microseconds
```

Figure 3.23: *The computation of initial value problem solution for the second order linear differential equation to compute the charge Q on the capacitor at any time t in a RLC series circuit (**SymIntegration/Examples/Differential Equations/Test SymIntegration RLC Series Electric Circuit Second Order Linear Differential Equations/main.cpp**).*

[SI*] **Example:**
If a series circuit has a capacitor of $C = 0.8 \times 10^{-6}$ F and an inductor of $L = 0.2$ H, find the resistance R so that the circuit is critically damped.

Solution:
Assume the circuit is closed at $t = 0$. Apply Faraday's law to obtain the governing equation for the current.

$$\sum V = -L\frac{di}{dt}$$

The only potential drops occur over the resistor and the capacitor

$$iR + \frac{q}{C} = -L\frac{di}{dt}$$

Write $i = \frac{dq}{dt} = q'$.

$$Rq' + \frac{q}{C} = -Lq''$$
$$Lq'' + Rq' + \frac{q}{C} = 0$$

Critical damping occurs when the ratio of R^2 to $4\frac{L}{C}$ is 1.

$$\begin{aligned}\frac{R^2}{\frac{4L}{C}} &= 1\\ R^2 &= \frac{4L}{C}\\ R &= \sqrt{\frac{4L}{C}}\\ &= \sqrt{\frac{4(0.2)}{0.8\times 10^{-6}}}\\ &= 1000\ \Omega\end{aligned}$$

[SI*] **Forced Vibrations**
We will now investigate the situation in which a periodic external force is applied to a spring-mass system. The behavior of this simple system models that of many oscillatory systems with an external force due, for example, to a motor attached to the system. We will first consider the case in which damping is present and will look later at the idealized special case in which there is assumed to be no damping.

[SI*] **Example for Forced Vibrations with Damping:**
Suppose that the motion of a certain spring-mass system satisfies the differential equation

$$u'' + u' + 1.25u = 3\cos\ t \tag{3.182}$$

and the initial conditions

$$u(0) = 2, \quad u'(0) = 3 \tag{3.183}$$

Find the solution of this initial value problem and describe the behavior of the solution for large t.

Solution:
The homogeneous equation corresponding to Eq. (3.182) has the characteristic equation

$$r^2 + r + 1.25 = 0$$

with roots $r = -0.5 \pm i$. Thus the general solution $u_c(t)$ of this homogeneous equation is

$$u_c(t) = c_1 e^{-t/2}\cos\ t + c_2 e^{-t/2}\sin\ t \tag{3.184}$$

A particular solution of Eq. (3.182) has the form $U(t) = A\cos\ t + B\sin\ t$, where A and B are found by substituting $U(t)$ for u in Eq. (3.182). We have $U'(t) = -A\sin\ t + B\cos\ t$ and $U''(t) = -A\cos\ t - B\sin\ t$. Thus, from Eq. (3.182) we obtain

$$(0.25A + B)\cos\ t + (-A + 0.25B)\sin\ t = 3\cos\ t$$

Consequently, A and B must satisfy the equations

$$\begin{aligned}0.25A + B &= 3\\ -A + 0.25B &= 0\end{aligned}$$

with the result that $A = \frac{12}{17}$ and $B = \frac{48}{17}$. Therefore the particular solution is

$$U(t) = \frac{12}{17}\cos\ t + \frac{48}{17}\sin\ t \tag{3.185}$$

and the general solution of Eq. (3.182) is

$$u = u_c(t) + U(t) = c_1 e^{-t/2} \cos\ t + c_2 e^{-t/2} \sin\ t + \frac{12}{17} \cos\ t + \frac{48}{17} \sin\ t \tag{3.186}$$

The remaining constants c_1 and c_2 are determined by the initial conditions (3.183). From Eq. (3.186) we have

$$u(0) = c_1 + \frac{12}{17} = 2$$
$$u'(0) = -\frac{1}{2} c_1 + c_2 + \frac{48}{17} = 3$$

so $c_1 = \frac{22}{17}$ and $c_2 = \frac{14}{17}$. Thus we finally obtain the solution of the given initial value problem, namely,

$$\frac{22}{17} e^{-t/2} \cos\ t + \frac{14}{17} e^{-t/2} \sin\ t + \frac{12}{17} \cos\ t + \frac{48}{17} \sin\ t \tag{3.187}$$

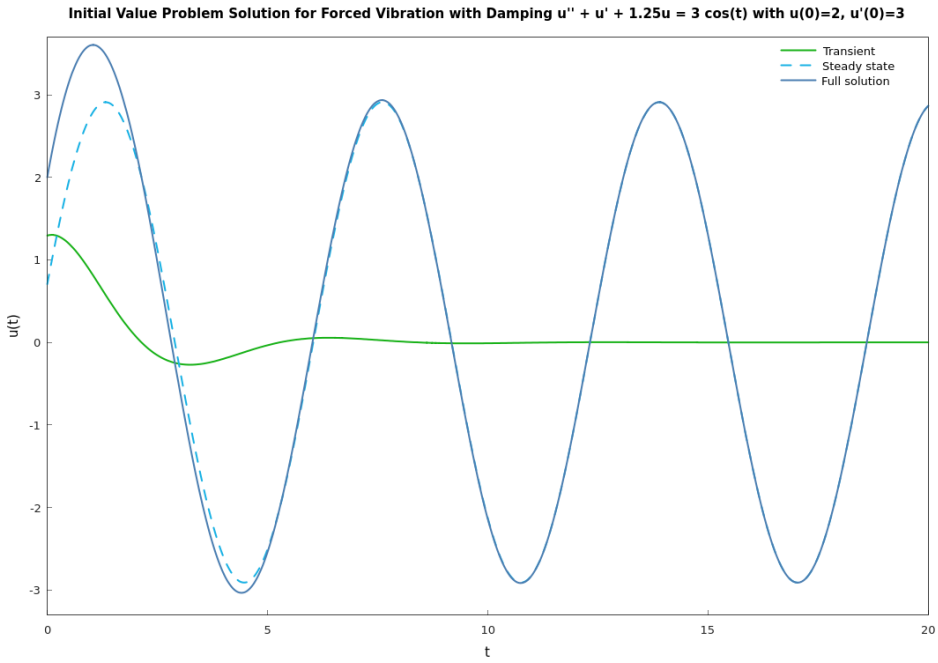

Figure 3.24: *The graph of the solution (3.187) for the initial value problem for the second order nonhomogeneous differential equation (**Hamzstplot/Examples with Makefile/Differential Equations/Plot 2D Forced Vibrations IVP Second Order ODE/main.cpp**).*

It is important to note that the solution consists of two distinct parts. The first two terms on the right side of Eq. (3.187) contain the exponential factor $e^{-t/2}$; as a result they rapidly approach zero. It is customary to call these terms transient. The remaining terms in Eq. (3.187) involve only sines and cosines, and therefore represent an oscillation that continuous indefinitely. We refer to them as a steady state. The green solid and blue dashed curves show the transient and the steady state parts of the solution, respectively. The transient part comes from the solution of the homogeneous equation corresponding to Eq. (3.182) and is

needed to satisfy the initial conditions. The steady state is the particular solution of the full nonhomogeneous equation. After a fairly short time the transient is vanishingly small and the full solution is essentially indistinguishable from the steady state.

[SI*] The equation of motion of a general spring-mass system subject to an external force $F(t)$ is

$$mu''(t) + \gamma u'(t) + ku(t) = F(t) \tag{3.188}$$

where m, γ, k are the mass, damping coefficient, and spring constant of the spring-mass system. Suppose now that the external force is given by $F_0 \cos\ \omega t$, where F_0 and ω are positive constants representing the amplitude and frequency, respectively, of the force. Then Eq. (3.188) becomes

$$mu'' + \gamma u' + ku = F_0 \cos\ \omega t \tag{3.189}$$

Solutions of Eq. (3.189) behave very much like the solution in the preceeding example. The general solution of Eq. (3.189) must have the form

$$u = c_1 u_1(t) + c_2 u_2(t) + A \cos\ \omega t + B \sin\ \omega t = u_c(t) + U(t) \tag{3.190}$$

The first two terms on the right side of Eq. (3.190) are the general solution $u_c(t)$ of the homogeneous equation corresponding to Eq. (3.189), while the latter two terms are a particular solution $U(t)$ of the full nonhomogeneous equation. The coefficients A and B can be found, as usual, by substituting these terms into the differential equation (3.189), while the arbitrary constants c_1 and c_2 are available to satisfy initial conditions, if any are prescribed. The solutions $u_1(t)$ and $u_2(t)$ of the homogeneous equation depend on the roots r_1 and r_2 of the characteristic equation $mr^2 + \gamma r + k = 0$. Since m, γ, and k are all positive, it follows that r_1 and r_2 either are real and negative or are both complex conjugates with negative real part. In either case, both $u_1(t)$ and $u_2(t)$ approach zero as $t \to \infty$. Since $u_c(t)$ dies out as t increases, it is called the transient solution. In many applications it is of little importance and (depending on the value of γ) may well be undetectable after only a few seconds.

The remaining terms in Eq. (3.190), namely, $U(t) = A \cos\ \omega t + B \sin\ \omega t$, do not die out as t increases but persist indefinitely, or as long as the external force is applied. They represent a steady oscillation with the same frequency as the external force and are called the steady state solution or the forced response. The transient solution enables us to satisfy whatever initial conditions may be imposed; with increasing time, the energy put into the system by the initial displacement and velocity is dissipated through the damping force, and the motion then becomes the response of the system to the external force. Without damping, the effect of the initial conditions would persist for all time.

It is convenient to express $U(t)$ as a single trigonometric term rather than as a sum of two terms. Recall that we did this for other similar expressions, thus we write

$$U(t) = R \cos(\omega t - \delta) \tag{3.191}$$

The amplitude R and phase δ depend directly on A and B and indirectly on the parameters in the differential equation (3.189). It is possible to show, by straightforward but somewhat lengthy algebraic computations, that

$$R = \frac{F_0}{\triangle}, \quad \cos\ \delta = \frac{m(\omega_0^2 - \omega^2)}{\triangle}, \quad \sin\ \delta = \frac{\gamma\omega}{\triangle} \tag{3.192}$$

where

$$\begin{aligned} \triangle &= \sqrt{m^2(\omega_0^2 - \omega^2)^2 + \gamma^2\omega^2} \\ \omega_0^2 &= \frac{k}{m} \end{aligned} \tag{3.193}$$

Recall that ω_0 is the natural frequency of the unforced system in the absence of damping. We now investigate how the amplitude R of the steady state oscillation depends on the frequency ω of the external force. Substituting from Eq. (3.193) into the expression for R in Eq. (3.192) and executing some algebraic manipulations, we find that

$$\frac{Rk}{F_0} = \frac{1}{\left[\left(1 - \frac{\omega^2}{\omega_0^2}\right)^2 + \Gamma\frac{\omega^2}{\omega_0^2}\right]^{1/2}} \tag{3.194}$$

where $\Gamma = \frac{\gamma^2}{mk}$. Observe that the quantity $\frac{Rk}{F_0}$ is the ratio of the amplitude R of the forced response to $\frac{F_0}{k}$, the static displacement of the spring produced by a force F_0.
For low frequency excitation, that is, as $\omega \to 0$, it follows from Eq. (3.194) that $\frac{Rk}{F_0} \to 1$ or $R \to \frac{F_0}{k}$. At the other extreme, for very high frequency excitation, Eq. (3.194) implies that $R \to 0$ as $\omega \to \infty$. At an intermediate value of ω the amplitude may have a maximum. To find this maximum point, we can differentiate R with respect to ω and set the result equal to zero. In this way we find that the maximum amplitude occurs when $\omega = \omega_{\max}$, where

$$\omega_{\max}^2 = \omega_0^2 - \frac{\gamma^2}{2m^2} = \omega_0^2\left(1 - \frac{\gamma^2}{2mk}\right) \tag{3.195}$$

Note that $\omega_{\max} < \omega_0$ and that $\omega_{\max}$ is close to ω_0 when γ is small. The maximum value of R is

$$R_{\max} = \frac{F_0}{\gamma\omega_0\sqrt{1 - \left(\frac{\gamma^2}{4mk}\right)}} \approxeq \frac{F_0}{\gamma\omega_0}\left(1 + \frac{\gamma^2}{8mk}\right) \tag{3.196}$$

where the last expression is an approximation for small γ. If $\frac{\gamma^2}{mk} > 2$, then $\omega_{\max}$ as given by Eq. (3.195) is imaginary; in this case the maximum value of R occurs for $\omega = 0$, and R is a monotone decreasing function of ω. Recall that critical damping occurs when $\frac{\gamma^2}{mk} = 4$.
For small γ it follows from Eq. (3.196) that $R_{\max} \approxeq \frac{F_0}{\gamma\omega_0}$. Thus, for lightly damped systems, the amplitude R of the forced response when ω is near ω_0 is quite large even for relatively small external forces, and the smaller the value of γ, the more pronounced is this effect. This phenomenon is known as resonance, and it is often an important design consideration. Resonance can be either good or bad, depending on the circumstances. It must be taken very seriously in the design of structures, such as buildings and bridges, where it can produce instabilities that might lead to the catastrophic failure of the structure. On the other hand, resonance can be put to good use in the design of instruments, such as seismographs, that are intended to detect weak periodic incoming signals.

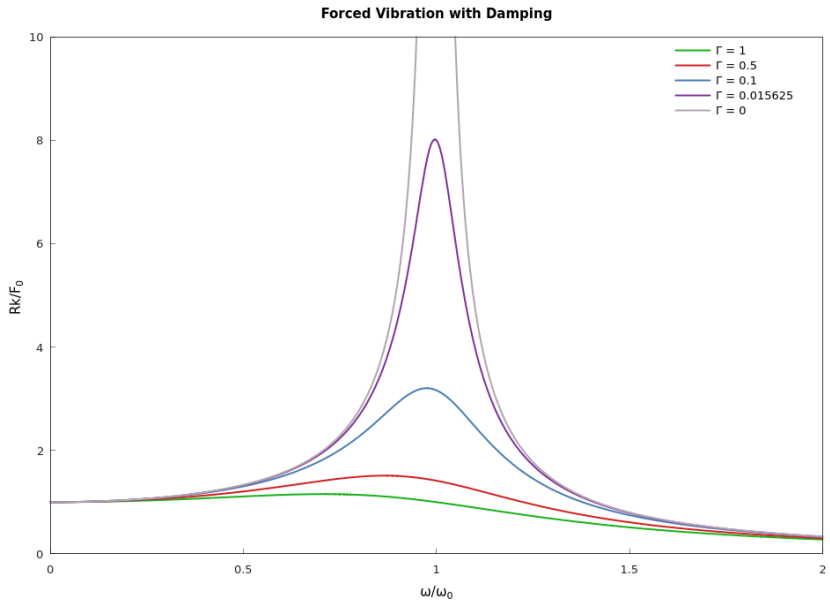

Figure 3.25: *Some representative graphs of* $\frac{Rk}{F_0}$ *versus* $\frac{\omega}{\omega_0}$ *for several values of* $\Gamma = \frac{\gamma^2}{mk}$ (***Hamzstplot/Examples with Makefile/Differential Equations/Plot 2D Forced Vibrations Amplitude of Steady State Response versus Frequency of Driving Force/main.cpp***).

It follows from Eq. (3.194), or from Eqs. (3.192) and (3.193), that $R \to \frac{F_0}{m|\omega_0^2 - \omega^2|}$ as $\gamma \to 0$ and hence $\frac{Rk}{F_0}$ is asymptotic to the vertical line $\omega = \omega_0$. As the damping in the system increases, the peak response gradually diminishes.
You can easily verify that each of the quantities $\frac{Rk}{F_0}$, $\frac{\omega}{\omega_0}$, and Γ is dimensionless. The importance of this observation is that the number of significant parameters in the problem has been reduced to three rather than five that appear in Eq. (3.189). Thus only one family of curves, of which a few are shown in Figure (3.24), is needed to describe the response-versus-frequency behavior of all systems governed by Eq. (3.189).

The phase angle δ also depends in an interesting way on ω. For ω near zero, it follows from Eqs. (3.192) and (3.193) that $\cos\ \delta \approxeq 1$ and $\sin\ \delta \approxeq 0$. Thus $\delta \approxeq 0$, and the response is enarly in phase with the excitation, meaning that they rise and fall together and, in particular, assume their respective maxima nearly together and their respective minima nearly together. For $\omega = \omega_0$ we find that $\cos\ \delta = 0$ and $\sin\ \delta = 1$, so $\delta = \frac{\pi}{2}$. In this acse the response lags behind the excitation by $\frac{\pi}{2}$; that is, the peaks of the response occur $\frac{\pi}{2}$ later than the peaks of the excitation, and similarly for the valleys. Finally, for ω very large, we have $\cos\ \delta \approxeq -1$ and $\sin\ \delta \approxeq 0$. Thus $\delta \approxeq \pi$, so that the response is nearly out of phase with the excitation; this means that the response is minimum when the excitation is maximum, and vice versa.

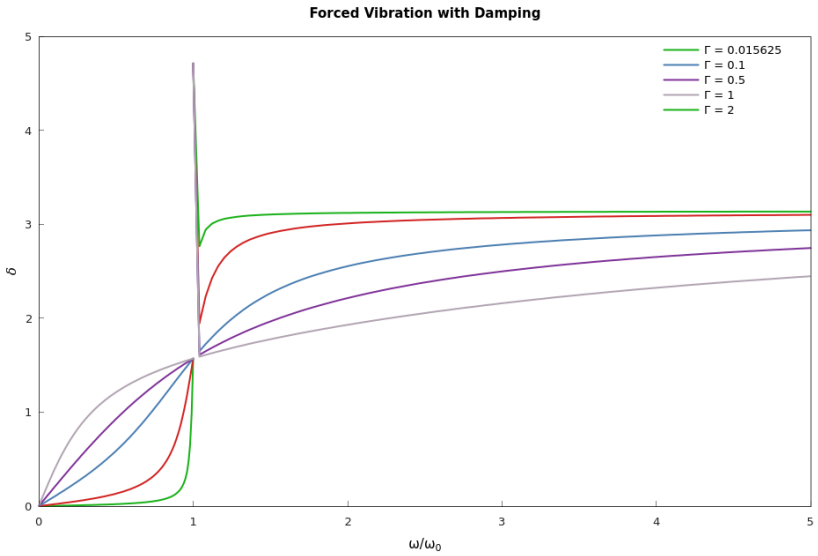

Figure 3.26: *Forced vibration with damping: phase of steady state response versus frequency of driving force;* $\Gamma = \frac{\gamma^2}{mk}$ *(**Hamzstplot/Examples with Makefile/Differential Equations/Plot 2D Forced Vibrations Phase of Steady State Response versus Frequency of Driving Force/main.cpp**).*

Figure (3.25) shows the graph of δ versus $\frac{\omega}{\omega_0}$ for several values of Γ. For small damping, the phase transition from near $\delta = 0$ to near $\delta = \pi$ occurs rather abruptly, whereas for larger values of the damping parameter, the transition takes place more gradually.

To compute the phase δ in terms of $\frac{\omega}{\omega_0}$ we can use the formula of sin δ from Eqs. (3.192), thus

$$\begin{aligned}
\sin\ \delta &= \frac{\gamma\omega}{\triangle} \\
&= \frac{\gamma\omega}{\sqrt{m^2(\omega_0^2-\omega^2)^2+\gamma^2\omega^2}} \\
&= \frac{\gamma\frac{\omega}{\omega_0}}{\frac{1}{\omega_0}\sqrt{m^2(\omega_0^2-\omega^2)^2+\gamma^2\omega^2}} \\
&= \frac{\gamma\frac{\omega}{\omega_0}}{\sqrt{m^2(1-\frac{\omega^2}{\omega_0^2})^2+\gamma^2\frac{\omega^2}{\omega_0^2}}} \\
\delta &= \sin^{-1}\left(\frac{\gamma\frac{\omega}{\omega_0}}{\sqrt{m^2(1-\frac{\omega^2}{\omega_0^2})^2+\gamma^2\frac{\omega^2}{\omega_0^2}}}\right)
\end{aligned}$$

To plot with sine formula we use this formula:

$$\delta = \begin{cases} \sin^{-1}\left(\frac{\gamma\frac{\omega}{\omega_0}}{\sqrt{m^2(1-\frac{\omega^2}{\omega_0^2})^2+\gamma^2\frac{\omega^2}{\omega_0^2}}}\right), & \text{for } \frac{\omega}{\omega_0} \leq 1 \\ \pi - \sin^{-1}\left(\frac{\gamma\frac{\omega}{\omega_0}}{\sqrt{m^2(1-\frac{\omega^2}{\omega_0^2})^2+\gamma^2\frac{\omega^2}{\omega_0^2}}}\right), & \text{for } \frac{\omega}{\omega_0} > 1 \end{cases} \tag{3.197}$$

In reality when we try to plot with this formula, the graphs will be the same if we can adjust the Γ parameter with γ, m, k. Thus based on what has been used, most people are rarely use the arcsine formula to plot the phase of steady state response versus frequency of driving force, in general to plot the phase of steady state response versus frequency of driving force we often use the arctangent formula which is simpler and only require 3 parameters

$$\delta = \begin{cases} \tan^{-1}\left(\frac{2\Gamma\frac{\omega}{\omega_0}}{1-\left(\frac{\omega}{\omega_0}\right)^2}\right), & \text{for } \frac{\omega}{\omega_0} \leq 1 \\ \pi + \tan^{-1}\left(\frac{2\Gamma\frac{\omega}{\omega_0}}{1-\left(\frac{\omega}{\omega_0}\right)^2}\right), & \text{for } \frac{\omega}{\omega_0} > 1 \end{cases} \tag{3.198}$$

As $\frac{\omega}{\omega_0} \to \infty$, $\frac{2\Gamma\frac{\omega}{\omega_0}}{1-\left(\frac{\omega}{\omega_0}\right)^2} \to 0^-$ and $\delta \to \pi$.

$\tan^{-1}(\infty) = \frac{\pi}{2}$, and tangent is positive in quadrant I and III only. You can analyze the value of δ correctly when you computed $R, \cos\ \delta, \sin\ \delta$ in Eqs. (3.192). For example, if the value of $\cos\ \delta$ is negative and the value of $\sin\ \delta$ is positive, then δ is in quadrant II, where sine is positive, cosine is negative and tangent is negative. Furthermore, if $\tan\ \delta$ is negative then we will use this formula

$$\delta = \pi + \frac{2\Gamma\frac{\omega}{\omega_0}}{1-\left(\frac{\omega}{\omega_0}\right)^2}$$

due to

$$\tan^{-1}(\alpha) = \tan^{-1}(\alpha)$$
$$\tan^{-1}(-\alpha) = -\tan^{-1}(\alpha) + \pi$$

In a forced vibration system with damping, the steady-state response (displacement) does not occur simultaneously with the driving force. Instead, the displacement lags behind the driving force by a phase angle (δ), which varies significantly depending on the ratio of the driving frequency (ω) to the system's natural frequency (ω_0).

[SI*] **Example:**

Consider a vibrating system described by the initial value problem

$$u'' + \frac{1}{4}u' + 2u = 2\cos\ \omega t, \quad u(0) = 0, \quad u'(0) = 2$$

(a) Determine the steady state part of the solution of this problem.

(b) Find the amplitude A of the steady state solution in terms of ω.

(c) Plot A versus ω

(d) Find the maximum value of A and the frequency ω for which it occurs.

Solution:

(a) This is the linear nonhomogeneous ODE, so its general solution can be expressed a sum of the complementary solution and the particular solution.

$$u(t) = u_c(t) + u_p(t)$$

The complementary solution satisfies the associated homogeneous equation.

$$u_c'' + \frac{1}{4}u_c' + 2u_c = 0 \tag{3.199}$$

Since the coefficients are constant and this is a homogeneous ODE, the solutions are of the form $u_c = e^{rt}$.

$$u_c = e^{rt} \quad \rightarrow \quad u_c' = re^{rt} \quad \rightarrow \quad u_c'' = r^2e^{rt}$$

Substitute these expressions to obtain an algebraic equation for r.

$$r^2e^{rt} + \frac{1}{4}(re^{rt}) + 2(e^{rt}) = 0$$

Divide both sides by $\frac{e^{rt}}{4}$.

$$\begin{aligned} 4r^2 + r + 8 &= 0 \\ r &= \frac{-1 \pm \sqrt{1 - 4(4)(8)}}{2(4)} \\ &= \frac{-1 \pm \sqrt{-127}}{8} \\ &= \frac{-1 \pm i\sqrt{127}}{8} \\ &= -\frac{1}{8} \pm i\mu \\ r &= \{-\frac{1}{8} - i\mu, -\frac{1}{8} + i\mu\} \end{aligned}$$

Two solutions to equation (3.199) are $u_c = e^{\left(-\frac{1}{8} - i\mu\right)t}$ and $u_c = e^{\left(-\frac{1}{8} + i\mu\right)t}$.

By the principle of superposition, the general solution for u_c is a linear combination of these two.

$$\begin{aligned} u_c(t) &= C_1e^{\left(-\frac{1}{8} - i\mu\right)t} + C_2e^{\left(-\frac{1}{8} + i\mu\right)t} \\ &= C_1e^{-t/8}e^{-i\mu t} + C_2e^{-t/8}e^{i\mu t} \\ &= C_1e^{-t/8}\left[\cos(-\mu t) + i\sin(-\mu t)\right] + C_2e^{-t/8}\left[\cos(\mu t) + i\sin(\mu t)\right] \\ &= C_1e^{-t/8}\left[\cos(\mu t) - i\sin(\mu t)\right] + C_2e^{-t/8}\left[\cos(\mu t) + i\sin(\mu t)\right] \\ &= C_1e^{-t/8}\cos(\mu t) - iC_1e^{-t/8}\sin(\mu t) + C_2e^{-t/8}\cos(\mu t) + iC_2e^{-t/8}\sin(\mu t) \\ &= (C_1 + C_2)e^{-t/8}\cos(\mu t) + (-iC_1 + iC_2)e^{-t/8}\sin(\mu t) \\ &= C_3e^{-t/8}\cos(\mu t) + C_4e^{-t/8}\sin(\mu t) \\ &= C_3e^{-t/8}\cos\frac{\sqrt{127}}{8}t + C_4e^{-t/8}\sin\frac{\sqrt{127}}{8}t \end{aligned}$$

On the other hand, the particular solution satisfies

$$u_p'' + \frac{1}{4}u_p' + 2u_p = 2\cos\ \omega t$$

Since there are both even and odd derivatives on the left side, include both sine and cosine in the trial solution to account for the cosine term on the right side:

$$u_p(t) = B\cos\ \omega t + D\sin\ \omega t$$

Plug this into the ODE to determine B and D

$$(B\cos\ \omega t + D\sin\ \omega t)'' + \frac{1}{4}(B\cos\ \omega t + D\sin\ \omega t)' + 2(B\cos\ \omega t + D\sin\ \omega t) = 2\cos\ \omega t$$

Evaluate the derivatives

$$\left(-B\omega^2 + \frac{D\omega}{4} + 2B\right)\cos\ \omega t + \left(-D\omega^2 - \frac{B\omega}{4} + 2D\right)\sin\ \omega t = 2\cos\ \omega t$$

Match the coefficients.

$$-B\omega^2 + \frac{D\omega}{4} + 2B = 2$$
$$-D\omega^2 - \frac{B\omega}{4} + 2D = 0$$

Solving this system of equations yields

$$B = \frac{32(2-\omega^2)}{16\omega^4 - 63\omega^2 + 64}$$
$$D = \frac{8\omega}{16\omega^4 - 63\omega^2 + 64}$$

Consequently, the particular solution is

$$u_p(t) = \frac{32(2-\omega^2)}{16\omega^4 - 63\omega^2 + 64}\cos\ \omega t + \frac{8\omega}{16\omega^4 - 63\omega^2 + 64}\sin\ \omega t$$

and the general solution is

$$u(t) = C_3 e^{-t/8}\cos\frac{\sqrt{127}}{8}t + C_4 e^{-t/8}\sin\frac{\sqrt{127}}{8}t + \frac{32(2-\omega^2)}{16\omega^4 - 63\omega^2 + 64}\cos\ \omega t + \frac{8\omega}{16\omega^4 - 63\omega^2 + 64}\sin\ \omega t$$

Differentiate it with respect to t

$$u'(t) = -\frac{C_3}{8}e^{-t/8}\cos\frac{\sqrt{127}}{8}t - C_3\frac{\sqrt{127}}{8}e^{-t/8}\sin\frac{\sqrt{127}}{8}t - \frac{C_4}{8}e^{-t/8}\sin\frac{\sqrt{127}}{8}t + C_4\frac{\sqrt{127}}{8}e^{-t/8}\cos\frac{\sqrt{127}}{8}t - \omega\frac{32(2-\omega^2)}{16\omega^4 - 63\omega^2 + 64}\sin\ \omega t + \omega\frac{8\omega}{16\omega^4 - 63\omega^2 + 64}\cos\ \omega t$$

Now apply the initial conditions to determine C_3 and C_4

$$u(0) = C_3 + \frac{32(2-\omega^2)}{16\omega^4 - 63\omega^2 + 64} = 0$$
$$u'(0) = -\frac{C_3}{8} + C_4\frac{\sqrt{127}}{8}e^{-t/8} + \omega\frac{8\omega}{16\omega^4 - 63\omega^2 + 64} = 2$$

Solving this system of equations yields

$$C_3 = \frac{32(\omega^2-2)}{16\omega^4 - 63\omega^2 + 64}$$
$$C_4 = \frac{16(16\omega^4 - 65\omega^2 + 60)}{\sqrt{127}(16\omega^4 - 63\omega^2 + 64)}$$

Therefore

$$u(t) = \frac{32(\omega^2-2)}{16\omega^4 - 63\omega^2 + 64}e^{-t/8}\cos\frac{\sqrt{127}}{8}t + \frac{16(16\omega^4 - 65\omega^2 + 60)}{\sqrt{127}(16\omega^4 - 63\omega^2 + 64)}e^{-t/8}\sin\frac{\sqrt{127}}{8}t$$
$$+ \frac{32(2-\omega^2)}{16\omega^4 - 63\omega^2 + 64}\cos\ \omega t + \frac{8\omega}{16\omega^4 - 63\omega^2 + 64}\sin\ \omega t$$

This is the steady state part:

$$u(t)_{\text{steady state}} = \frac{32(2-\omega^2)}{16\omega^4 - 63\omega^2 + 64}\cos\ \omega t + \frac{8\omega}{16\omega^4 - 63\omega^2 + 64}\sin\ \omega t$$

(b) We introduce an amplitude A and a phase δ to combine the two sinusoidal terms of the steady state into one

$$\begin{aligned} u(t)_{\text{steady state}} &= \frac{32(2-\omega^2)}{16\omega^4 - 63\omega^2 + 64}\cos\ \omega t + \frac{8\omega}{16\omega^4 - 63\omega^2 + 64}\sin\ \omega t \\ &= A\cos\ \delta\cos\ \omega t + A\sin\ \delta\sin\ \omega t \\ &= A\cos(\omega t - \delta) \end{aligned}$$

A and δ satisfy the following system of equations

$$A\cos\ \delta = \frac{32(2-\omega^2)}{16\omega^4 - 63\omega^2 + 64}$$
$$A\sin\ \delta = \frac{8\omega}{16\omega^4 - 63\omega^2 + 64}$$

To find A, square both sides of each equation

$$A^2\cos^2\delta = \left[\frac{32(2-\omega^2)}{16\omega^4 - 63\omega^2 + 64}\right]^2$$
$$A^2\sin^2\delta = \left[\frac{8\omega}{16\omega^4 - 63\omega^2 + 64}\right]^2$$

and then add the respective sides together

$$A^2\cos^2\delta + A^2\sin^2\delta = \left[\frac{32(2-\omega^2)}{16\omega^4 - 63\omega^2 + 64}\right]^2 + \left[\frac{8\omega}{16\omega^4 - 63\omega^2 + 64}\right]^2$$

$$A^2(\cos^2\delta + \sin^2\delta) = \frac{64}{16\omega^4 - 63\omega^2 + 64}$$

$$A^2 = \frac{64}{16\omega^4 - 63\omega^2 + 64}$$

$$A = \frac{8}{\sqrt{16\omega^4 - 63\omega^2 + 64}}$$

(c) Below is the plot of A versus ω

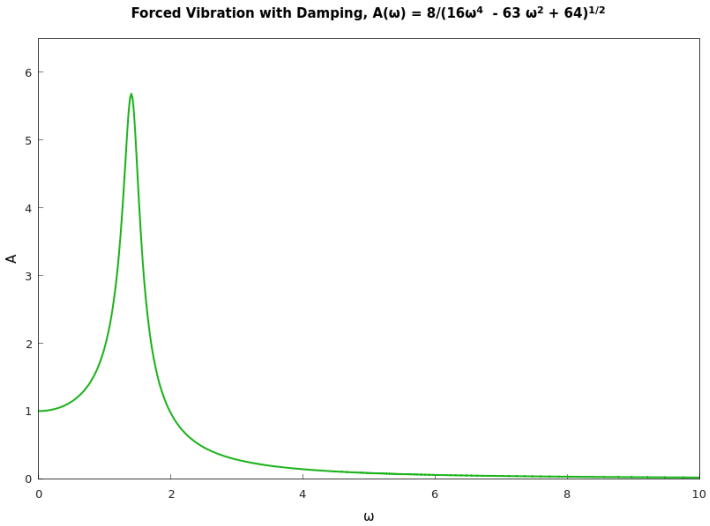

Figure 3.27: *(Hamzstplot/Examples with Makefile/Differential Equations/Plot 2D Forced Vibrations Amplitude of Steady State Response versus Frequency/main.cpp).*

(d) In order to find the value of ω that maximizes A, differentiate A with respect to ω

$$A'(\omega) = -\frac{4}{(16\omega^4 - 63\omega^2 + 64)^{3/2}}(64\omega^3 - 126\omega)$$

and then set it equal to zero.

$$-\frac{4}{(16\omega^4 - 63\omega^2 + 64)^{3/2}}(64\omega^3 - 126\omega) = 0$$

$$64\omega^3 - 126\omega = 0$$

$$2\omega(32\omega^2 - 63) = 0$$

Use the zero product theorem

$$2\omega = 0$$

$$\omega = 0$$

or

$$32\omega^2 - 63 = 0$$
$$\omega = \pm\sqrt{\frac{63}{32}}$$

Since ω has to be positive

$$\omega_{\max} = \sqrt{\frac{63}{32}} \approx 1.403$$

The amplitude that corresponds with this value of ω is

$$\begin{aligned} A_{\max} = A(\omega_{\max}) &= \frac{8}{\sqrt{16\omega^4 - 63\omega^2 + 64}} \\ &= \frac{8}{\sqrt{16\omega_{\max}^4 - 63\omega_{\max}^2 + 64}} \\ &= \frac{8}{\sqrt{16\left(\sqrt{\frac{63}{32}}\right)^4 - 63\left(\sqrt{\frac{63}{32}}\right)^2 + 64}} \\ &= \frac{64}{\sqrt{127}} \\ &\approx 5.679 \end{aligned}$$

```
Solution for u'' + 0.25u' + 2u = 2 cos(1.403t), u(0) = 0, u'(0) = 2


The particular solution is
Y(t) = 5.65618*sin(1.403*t)+0.509435*cos(1.403*t)

The general solution for the homogeneous equation is:

y(t) = e^(-0.125*t)*c1*cos(1.40868*t)+e^(-0.125*t)*c2*sin(1.40868*t)

The solution for the initial value problem is:
-0.509435*e^(-0.125*t)*cos(1.40868*t)-4.25882*e^(-0.125*t)*sin(1.40868*t)+5.65618*sin(1.403*t)+0.509435*cos(1.403*t)

The forced vibrations parameters:
F_{0} = 2
ω = 1.403
ω_{0} = 1.41421
ω_{max} = 1.40312
Γ = 0.03125
R = 5.67908
R_{max} = 5.67908
△ = 0.35217
δ = 1.48097

Time taken by function: 723903 microseconds
```

Figure 3.28: *The computation of initial value problem solution for the second order linear differential equation for forced vibration with damping, the parameter A (the amplitude) is represented by R in SymIntegration (**SymIntegration/Examples/Differential Equations/Test SymIntegration Initial Value Problem for Nonhomogeneous Second Order Linear Differential Equations Forced Vibrations/main.cpp**).*

[SI*] **Forced Vibrations Without Damping**

We now assume that $\gamma = 0$ in Eq. (3.189), thereby obtaining the equation of motion of an undamped forced oscillator

$$mu'' + ku = F_0 \cos\ \omega t \tag{3.200}$$

The form of the general solution of Eq. (3.200) is different, depending on whether the forcing frequency ω is different from or equal to the natural frequency $\omega_0 = \sqrt{\frac{k}{m}}$ of the unforced

system. First consider the case $\omega \neq \omega_0$; then the general solution of Eq. (3.200) is

$$u = c_1 \cos\ \omega_0 t + c_2 \sin\ \omega_0 t + \frac{F_0}{m(\omega_0^2 - \omega^2)} \cos\ \omega t \tag{3.201}$$

The constants c_1 and c_2 are determined by the initial conditions. The resulting motion is, in general, the sum of two periodic motions of different frequencies (ω_0 and ω) and different amplitudes as well.
It is particularly interesting to suppose that the mass is initially at rest, so that the initial conditions are $u(0) = 0$ and $u'(0) = 0$. Then the energy driving the system comes entirely from the external force, with no contribution from the initial conditions. In this case it turns out that the constants c_1 and c_2 in Eq. (3.201) are given by

$$c_1 = -\frac{F_0}{m(\omega_0^2 - \omega^2)}, \quad c_2 = 0 \tag{3.202}$$

and the solution of Eq. (3.200) is

$$u = \frac{F_0}{m(\omega_0^2 - \omega^2)}(\cos\ \omega t - \cos\ \omega_0 t) \tag{3.203}$$

This is the sum of two periodic functions of different periods but the same amplitude. Making use of the trigonometric identities for $\cos(A \pm B)$ with $A = \frac{(\omega_0+\omega)t}{2}$ and $B = \frac{(\omega_0-\omega)t}{2}$, we can write in Eq. (3.203) in the form

$$u = \left[\frac{2F_0}{m(\omega_0^2 - \omega^2)} \sin \frac{(\omega_0 - \omega)t}{2}\right] \sin \frac{(\omega_0 + \omega)t}{2} \tag{3.204}$$

If $|\omega_0 - \omega|$ is small, then $\omega_0 + \omega$ is much greater than $|\omega_0 - \omega|$. Consequently, $\sin \frac{(\omega_0+\omega)t}{2}$ is a rapidly oscillating function compared to $\sin \frac{(\omega_0-\omega)t}{2}$. Thus the motion is a rapid oscillation with frequency $\frac{\omega_0+\omega}{2}$ but with a slowly varying sinusoidal amplitude

$$\frac{2F_0}{m|\omega_0^2 - \omega^2|} \left|\sin \frac{(\omega_0 - \omega)t}{2}\right|$$

This type of motion, possessing a periodic variation of amplitude, exhibits what is called a beat. For example, such a phenomenon occurs in acoustics when two tuning forks of nearly equal frequency are excited simultaneously. In this case the periodic variation of amplitude is quite apparent to the unaided ear. In electronics, the variation of the amplitude with time is called amplitude modulation.

i. Functions in SymIntegration Related to Second Order Linear Equations

[SI*] We will list the basic functions that can be used for solving second order linear differential equations:

secondorderlineardiffeq_dsolve(double *a*, **double** *b*, **double** *c*, **const Symbolic &***y*, **const Symbolic &***t***)**
to solve a homogeneous second order linear differential equation with this form

$$ay'' + by' + cy = 0$$

it will return the general solution, $y(t)$.

secondorderlineardiffeq_ivpsolution(double a, double b, double c, const Symbolic &y, const Symbolic &t, double t_0, double y_0, double y_0')
to solve the initial value problem of a homogeneous second order linear differential equation with this form

$$ay'' + by' + cy = 0$$

with initial value

$$y(t_0) = y_0$$
$$y'(t_0) = y_0'$$

it will also compute the critical value t_m and the maximum value y_m for the initial value problem solution for the case with real and different roots.

```
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Differential Equations/Test SymIntegration IVPSolve Second Order Linear Equations ]# ./main

IVP Solve for y'' + 5y'+ 6y = 0 with y(0) = 2, y'(0) = 3


The general solution is:
c1*e^(-2*t)+c2*e^(-3*t)

The solution for the initial value problem is:
9*e^(-2*t)-7*e^(-3*t)

y' = -18*e^(-2*t)+21*e^(-3*t)

Critical value t_{m} = 0.154151

Maximum value y_{m} = 2.20408

IVP Solve for y'' - y = 0 with y(0) = 2, y'(0) = -1


The general solution is:
c1*e^t+c2*e^(-t)

The solution for the initial value problem is:
0.5*e^t+1.5*e^(-t)

y' = 0.5*e^t-1.5*e^(-t)

Critical value t_{m} = 0.549306

Maximum value y_{m} = 1.73205

IVP Solve for 4y'' - 8y' +  3y = 0 with y(0) = 2, y'(0) = 1/2


The general solution is:
c1*e^(1.5*t)+c2*e^(0.5*t)

The solution for the initial value problem is:
-0.5*e^(1.5*t)+2.5*e^(0.5*t)

y' = -0.75*e^(1.5*t)+1.25*e^(0.5*t)

Critical value t_{m} = 0.510826

Maximum value y_{m} = 2.15166
```

Figure 3.29: *The computation of the initial value problem solution for a homogeneous second order linear equation with* ***secondorderlineardiffeq_ivpsolution()*** *function (****SymIntegration/Examples/Differential Equations/Test SymIntegration IVPSolve Second Order Linear Equations/main.cpp****).*

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Complex Roots Second Order Linear Differential Equations ]# ./main
General solution for y'' + y'+ 9.25y = 0 with t_{0} = 0, y(t_{0}) = 2, y'(t_{0}) = 8

The general solution is:
y_{1} (t) = e^(-0.5*t)*cos(3*t)+e^(-0.5*t)*i*sin(3*t)
y_{2} (t) = e^(-0.5*t)*cos(3*t)-e^(-0.5*t)*i*sin(3*t)
Initial Value Problem solution for y'' + y'+ 9.25y = 0 with t_{0} = 0, y(t_{0}) = 2, y'(t_{0}) = 8

The complex-valued general solution is:
y_{1} (t) = e^(-0.5*t)*cos(3*t)+e^(-0.5*t)*i*sin(3*t)
y_{2} (t) = e^(-0.5*t)*cos(3*t)-e^(-0.5*t)*i*sin(3*t)
u(t) = e^(-0.5*t)*cos(3*t)
v(t) = e^(-0.5*t)*sin(3*t)
The real-valued initial value problem solution is:
y (t) = 2*e^(-0.5*t)*cos(3*t)+3*e^(-0.5*t)*sin(3*t)
Initial Value Problem solution for 16y'' - 8y'+ 145y = 0 with t_{0} = 0, y(t_{0}) = -2, y'(t_{0}) = 1

The complex-valued general solution is:
y_{1} (t) = e^(0.25*t)*cos(3*t)+e^(0.25*t)*i*sin(3*t)
y_{2} (t) = e^(0.25*t)*cos(3*t)-e^(0.25*t)*i*sin(3*t)
u(t) = e^(0.25*t)*cos(3*t)
v(t) = e^(0.25*t)*sin(3*t)
The real-valued initial value problem solution is:
y (t) = -2*e^(0.25*t)*cos(3*t)+0.5*e^(0.25*t)*sin(3*t)
Time taken by function: 289530 microseconds
```

Figure 3.30: *The computation of the initial value problem solution for a homogeneous second order linear equation with complex conjugate roots (**SymIntegration/Examples/Differential Equations/Test SymIntegration Complex Roots Second Order Linear Differential Equations/main.cpp**).*

wronskian(double a**, double** b**, double** c**, const Symbolic &**y**, const Symbolic &**t**,double** t_0**, double** y_0**, double** y_0'**)**
to compute the Wronskian of a homogeneous second order linear differential equation with this form

$$ay'' + by' + cy = 0$$

with initial value

$$y(t_0) = y_0$$
$$y'(t_0) = y_0'$$

wronskian_fundamentalsetofsolutions(double a**, double** b**, double** c**, const Symbolic &**y**, const Symbolic &**t**)**
to compute the fundamental set of solutions of a homogeneous second order linear differential equation with this form

$$ay'' + by' + cy = 0$$

The first fundamental solution y_1 satisfies the initial conditions

$$y(t_0) = 1$$
$$y'(t_0) = 0$$

The second fundamental solution y_2 satisfies the initial conditions

$$y(t_0) = 0$$
$$y'(t_0) = 1$$

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Wronskian Second Order Linear Equations ]# ./main

Wronskian for y'' + 5y'+ 6y = 0 with t_{0} = 0


The general solution is:
c1*e^(-2*t)+c2*e^(-3*t)

The Wronskian is:-e^(-5*t)

W(t_{0}) = -1

Wronskian for y'' -  y = 0 with t_{0} = 0


The solution that satisfies the initial value problem y(0)=1 and y'(0)=0:
0.5*e^t+0.5*e^(-t)

The solution that satisfies the initial value problem y(0)=0 and y'(0)=1:
0.5*e^t-0.5*e^(-t)

The general solution :
0.5*k1*e^t+0.5*k1*e^(-t)+0.5*k2*e^t-0.5*k2*e^(-t)
```

Figure 3.31: *The computation of the Wronskian and the fundamental set of solutions for a homogeneous second order linear equation (**SymIntegration/Examples/Differential Equations/Test SymIntegration Wronskian Second Order Linear Equations/main.cpp**).*

adjointequation(const Symbolic &$c1$, const Symbolic &$c2$, const Symbolic &c_3, Symbolic &t)
to compute the adjoint of the original second order linear homogeneous equation with this form:

$$c_1 y'' + c_2 y' + c_3 y = 0$$

with c_1, c_2, c_3 are functions of t, other symbols (e.g. α, β, ν) will be considered as constants.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Second Order Linear Homogeneous Equation Adjoint Equation ]# ./main


Adjoint for x^{2}y'' + xy' + (x^{2} - ν^{2})y = 0 (Bessel' equation)


Exact equation :
x^(2)*μ(x)*y''+x*μ(x)*y'+x^(2)*μ(x)*y-ν^(2)*μ(x)*y = 0
2*x*μ(x)*y'+x^(2)*μ'(x)*y'+x^(2)*μ(x)*y''+f'(x)*y+f(x)*y' = 0

Equate the coefficients :
2*x*μ(x)+x^(2)*μ'(x)+f(x) = x * μ(x)

f'(x) = x^(2)-ν^(2) * μ(x)

Differentiate both sides of the first equation with respect to : x
3*x*μ'(x)+x^(2)*μ''(x)+μ(x)+f'(x)

Substitute x^(2)-ν^(2) * μ(x) for f'(x)

3*x*μ'(x)+x^(2)*μ''(x)+μ(x)+x^(2)*μ(x)-ν^(2)*μ(x) = 0

****************************************************************


Adjoint for (1-x^{2})y'' - 2xy' + α(α+1)y = 0 (Legendre's equation)


Exact equation :
-x^(2)*μ(x)*y''+μ(x)*y''-2*x*μ(x)*y'+α^(2)*μ(x)*y+α*μ(x)*y = 0
-2*x*μ(x)*y'-x^(2)*μ'(x)*y'+μ'(x)*y'-x^(2)*μ(x)*y''+μ(x)*y''+f'(x)*y+f(x)*y' = 0

Equate the coefficients :
-2*x*μ(x)-x^(2)*μ'(x)+μ'(x)+f(x) = -2*x * μ(x)

f'(x) = α^(2)+α * μ(x)

Differentiate both sides of the first equation with respect to : x
-2*x*μ'(x)-x^(2)*μ''(x)+μ''(x)+f'(x)

Substitute α^(2)+α * μ(x) for f'(x)

-2*x*μ'(x)-x^(2)*μ''(x)+μ''(x)+α^(2)*μ(x)+α*μ(x) = 0
```

Figure 3.32: *The computation of the adjoint of the given second order linear differential equation, Bessel's equation and Legendre's equation (**SymIntegration/Examples/Differential Equations/Test SymIntegration Second Order Linear Homogeneous Equation Adjoint Equation/main.cpp**).*

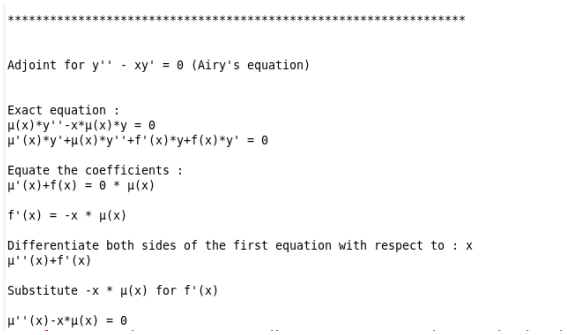

Figure 3.33: *The computation of the adjoint of the given second order linear differential equation, Airy's equation (**SymIntegration/Examples/Differential Equations/Test SymIntegration Second Order Linear Homogeneous Equation Adjoint Equation/main.cpp**).*

reductionoforder(const Symbolic &$a(t)$, const Symbolic &$b(t)$, const Symbolic &$c(t)$, const Symbolic &y_1, const Symbolic &t)
to compute the second solution of a second order homogeneous differential equation with this form

$$a(t)y'' + b(t)y' + c(t)y = 0$$

with known $y_1(t)$ (the first solution) and a, b, c are functions of t, which can also be constants as well.

secondorderlineardiffeq_nonhomogeneousequationssolution(const Symbolic &a, const Symbolic &b, const Symbolic &c, const Symbolic &g, const Symbolic &y, const Symbolic &t)
to compute the general solution (particular solution and the complementary solution) of a second order nonhomogeneous differential equation with this form

$$ay'' + by' + cy = g$$

with a, b, and c as constants, and g, a function of t, a one term function (not a linear combination of functions).

secondorderlineardiffeq_nonhomogeneousequationssolution(const Symbolic &a, const Symbolic &b, const Symbolic &c, const SymbolicMatrix &g, const Symbolic &y, const Symbolic &t)
to compute the general solution (particular solution and the complementary solution) of a second order nonhomogeneous differential equation with this form

$$ay'' + by' + cy = g[1] + g[2] + \ldots g[n]$$

with a, b, and c as constants, and g is a symbolic matrix with 1 column (symbolic vector to be exact) that consists of the term of $g(t)$, e.g. if we have $g(t) = 3e^{2t} + 2\sin\ t - 8e^t \cos\ 2t$, then $g[1] = 3e^{2t}, g[2] = 2\sin\ t, g[3] = -8e^t \cos\ 2t$.

```
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Differential Equations/Test SymIntegration Nonhomogeneous Second Order Linear Differential
 Equations SymbolicMatrix rhs ]# ./main
W:
[   3*e^(2*t)   ]
[    2*sin(t)   ]
[-8*e^t*cos(2*t)]


Solution for y'' - 3y' - 4y = 3e^{2t} + 2 sin(t) - 8e^{t} cos(2t)


************************************************************

For rhs = 3*e^(2*t)



The current solution is
Y(t) = -0.5*e^(2*t)
************************************************************

For rhs = 2*sin(t)



The current solution is
Y(t) = -0.294118*sin(t)+0.176471*cos(t)
************************************************************

For rhs = -8*e^t*cos(2*t)
Attempted to cast -8*cos(2*t) to double failed.
Attempted to cast -8*e^t to double failed.



The current solution is
Y(t) = 0.153846*e^t*sin(2*t)+0.769231*e^t*cos(2*t)
************************************************************

************************************************************

The particular solution is
Y(t) = -0.5*e^(2*t)-0.294118*sin(t)+0.176471*cos(t)+0.153846*e^t*sin(2*t)+0.769231*e^t*cos(2*t)
The general solution for the homogeneous equation is:
c1*e^(4*t)+c2*e^(-t)

The general solution for the nonhomogeneous equation is:
c1*e^(4*t)+c2*e^(-t)+0.153846*e^t*sin(2*t)+0.769231*e^t*cos(2*t)

Time taken by function: 645443 microseconds
```

Figure 3.34: *The computation of the particular solution for the second order nonhomogeneous differential equation (**SymIntegration/Examples/Differential Equations/Test SymIntegration Nonhomogeneous Second Order Linear Differential Equations SymbolicMatrix rhs/main.cpp**).*

* The **attempted to cast** $-8\cos\ 2t$ **to double** and **attempted to cast** $-8e^t$ **to double** are warnings because the source code matching with the pattern of $g(t) = \exp(a * t)$ and $g(t) = \cos(a * t)$ based on the ordering, before it reaches the right pattern of $g(t) = a * exp(c * t) * cos(b * t)$, with a, b, c as constants. The **secondorderlineardiffeq_nonhomogeneousequationssolution()** function can handle these $g(t)$ functions:

$$
\begin{aligned}
g(t) &= e^{bt} \\
g(t) &= ae^{bt} \\
g(t) &= a\sin(bt) \\
g(t) &= a\cos(bt) \\
g(t) &= ae^{bt}\sin(ct) \\
g(t) &= ae^{bt}\cos(ct)
\end{aligned}
$$

and the linear combinations of the $g(t)$ functions above.

secondorderlineardiffeq_nonhomogeneousequationssolution(const Symbolic &a, const Symbolic &b, const Symbolic &c, Polynomial<double> &$P(t)$)
to compute the general solution (particular solution and the complementary solution) of a second order nonhomogeneous differential equation with this form

$$ay'' + by' + cy = P(t)$$

with a, b, and c as constants, with $P(t)$ is a polynomial. In order to solve and find the particular solution with polynomial on the right hand side, we initially assume that $Y(t)$ is a polynomial of the same degree as the nonhomogeneous term, $P(t)$, that is if $P(t) = t^2 - 1$, then $Y(t) = At^2 + Bt + C$. The function currently can handle polynomial of degree n, with n as positive integer.

```
Solution for y'' - 3y' - 4y = 4t^{2} - 1

p(t) = 4t^2-1

The particular solution is
Y(t) = -t^(2)+1.5*t-1.375

The general solution for the homogeneous equation is:
c1*e^(4*t)+c2*e^(-t)

The general solution for the nonhomogeneous equation is:
c1*e^(4*t)+c2*e^(-t)-t^(2)+1.5*t-1.375

Solution for y'' - 3y' - 4y = 5t^{3} + 2t^{2} - 1

p(t) = 5t^3+2t^2-1

The particular solution is
Y(t) = -1.25*t^(3)+2.3125*t^(2)-5.34375*t+5.41406

The general solution for the homogeneous equation is:
c1*e^(4*t)+c2*e^(-t)

The general solution for the nonhomogeneous equation is:
c1*e^(4*t)+c2*e^(-t)-1.25*t^(3)+2.3125*t^(2)-5.34375*t+5.41406

Solution for y'' - 3y' - 4y = 5t - 10

p(t) = 5t-10

The particular solution is
Y(t) = -1.25*t+3.4375

The general solution for the homogeneous equation is:
c1*e^(4*t)+c2*e^(-t)

The general solution for the nonhomogeneous equation is:
c1*e^(4*t)+c2*e^(-t)-1.25*t+3.4375

Solution for y'' - 3y' - 4y = 50

p(t) = 50

The particular solution is
Y(t) = -12.5

The general solution for the homogeneous equation is:
c1*e^(4*t)+c2*e^(-t)

The general solution for the nonhomogeneous equation is:
c1*e^(4*t)+c2*e^(-t)-12.5

Time taken by function: 226665 microseconds
```

Figure 3.35: *The computation of the particular solution for the second order nonhomogeneous differential equation with polynomial as the nonhomogeneous term (**SymIntegration/Examples/Differential Equations/Test SymIntegration Nonhomogeneous Second Order Linear Differential Equations Polynomial rhs/main.cpp**).*

The code used to handle polynomial of degree n uses Gaussian elimination to obtain the coefficient A_0, A_1, A_n for $Y(t)$, the function **solve_nhsystem_resultsonly()** handles this.

```
...
...

void secondorderlineardiffeq_nonhomogeneousequationssolution(const Symbolic &
    lhs_a, const Symbolic &lhs_b, const Symbolic &lhs_c, Polynomial<double> &
    rhs_function)
{
       Symbolic A("A"), t("t");
       Symbolic lhs_final, Yt_final;
```

```
Polynomial<double> p = rhs_function;
cout << "p(t) = " << p << endl;
int n = rhs_function.terms.front().second; // degree of the polynomial
if (n >= 0 ) // this is working for Polynomial of degree n, n =
    positive integer
{
Symbolic Yt;
for (int i = 0; i <= n; ++i)
{
       Yt += A[i]*(t^(n-i)); // A[i] is symbolic A_{i} so we don't
           need to declare infinite number of Symbolic A1("A1"),... ,
           An("An")
}
Symbolic dy = df(Yt,t);
Symbolic ddy = df(dy,t);

lhs_final = ddy*lhs_a + dy* lhs_b + Yt*lhs_c;

vector<vector<double>> mat_A(n+1, vector<double>(n+1));
vector<vector<double>> vec_b(n+1, vector<double>(1));

for (int i = 0; i <= n; ++i)
{
Symbolic coeff_ti = lhs_final.coeff(t,n-i);
       for (int j = 0; j <= n; ++j)
       {
              mat_A[i][j] = coeff_ti.coeff(A[j],1);
       }
}

int n1 = n;
for (int i = 0 ; i < n+1 ; ++i)
{
       if(p.terms.front().second == n1) // p.terms.front().second =
           get the degree of the polynomial
       {
              vec_b[i][0] = p.terms.front().first; // p.terms.front().
                  first = get the coefficient
              p.terms.pop_front(); // remove value from front of list
       }
       else if(p.terms.front().second != n1)
       {
              vec_b[i][0] = 0;
       }
       n1 = n1-1;
}

vector<double> c_solution;
```

```
        solve_nhsystem_resultsonly(mat_A,vec_b,c_solution);

        for (int i = 0 ; i < n+1 ; ++i)
        {
                Yt_final += c_solution[i]*(t^(n-i));
        }
        cout << "\nThe particular solution is\nY(t) = " << Yt_final << endl;
        }
}
```

Code 40: *src/dsolve.cpp*

secondorderlineardiffeq_nonhomogeneousequationsivpsolution(const Symbolic &a, const Symbolic &b, const Symbolic &c, const Symbolic &g, double y_0, double y_0', const Symbolic &y, const Symbolic &t)
to compute the initial value problem general solution (particular solution and the complementary solution) of a second order nonhomogeneous differential equation with this form

$$ay'' + by' + cy = g$$

and the initial conditions

$$y(0) = y_0, \quad y'(0) = y_0'$$

with a, b, and c as constants, and g, a function of t, a one term function (not a linear combination of functions).

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Initial Value Problem for Nonhomogeneous Second Order Linear
rced Vibrations ]# ./main

Solution for u'' + u' + 1.25u = 3 cos(t)


The particular solution is
Y(t) = 2.82353*sin(t)+0.705882*cos(t)

The general solution for the homogeneous equation is:

y(t) = e^(-0.5*t)*c1*cos(t)+e^(-0.5*t)*c2*sin(t)

The solution for the initial value problem is:
1.29412*e^(-0.5*t)*cos(t)+0.823529*e^(-0.5*t)*sin(t)+2.82353*sin(t)+0.705882*cos(t)

Time taken by function: 168117 microseconds
```

Figure 3.36: *The computation of the initial value problem general solution for the second order nonhomogeneous differential equation (**SymIntegration/Examples/Differential Equations/Test SymIntegration Initial Value Problem for Nonhomogeneous Second Order Linear Differential Equations Forced Vibrations/main.cpp**).*

secondorderlineardiffeq_springmasssystem(double a, double b, double c, const Symbolic &y, const Symbolic &t, double y_0, double y_0')
The computation of initial value problem solution for the second order linear differential equation for spring-mass system / mechanical vibration with damping, small damping or no damping. The output also returns δ (the phase), T_d (the quasi period), and t (the time which the mass first passes through its equilibrium position).

secondorderlineardiffeq_RLCserieselectriccircuit(double R, double L, double C, double q_0, double q_0')
to compute the charge $q(t)$ on the capacitor at any time given the resistor R, inductor L, and the capacitor C for the series circuit. We have q_0 to represent the initial charge on the

capacitor, and $q_0' = I_0$ represents the initial current in the circuit.

secondorderlineardiffeq_nonhomogeneousequationsivpforcedvibrationssolution(const Symbolic &a, const Symbolic &b, const Symbolic &c, const Symbolic &$g(t)$ double y_0, double y_0', const Symbolic &y, const Symbolic &t)
to compute the initial value problem general solution (particular solution and the complementary solution) of a second order nonhomogeneous differential equation with this form

$$ay'' + by' + cy = g(t)$$

and the initial conditions

$$y(0) = y_0, \quad y'(0) = y_0'$$

with a, b, and c as constants, and g, a function of t, a one term function (not a linear combination of functions). The output also contains some forced vibration parameters : $F_0, \omega, \omega_0, \omega_{\max}, \Gamma, R, R_{\max}, \triangle, \delta$. If you want to plot it, the forcing function is represented by $g(t)$, and the solution is represented by the general solution of the initial value problem ($u(t)$) that can be expressed a sum of the complementary solution and the particular solution.

$$u(t) = u_c(t) + u_p(t)$$

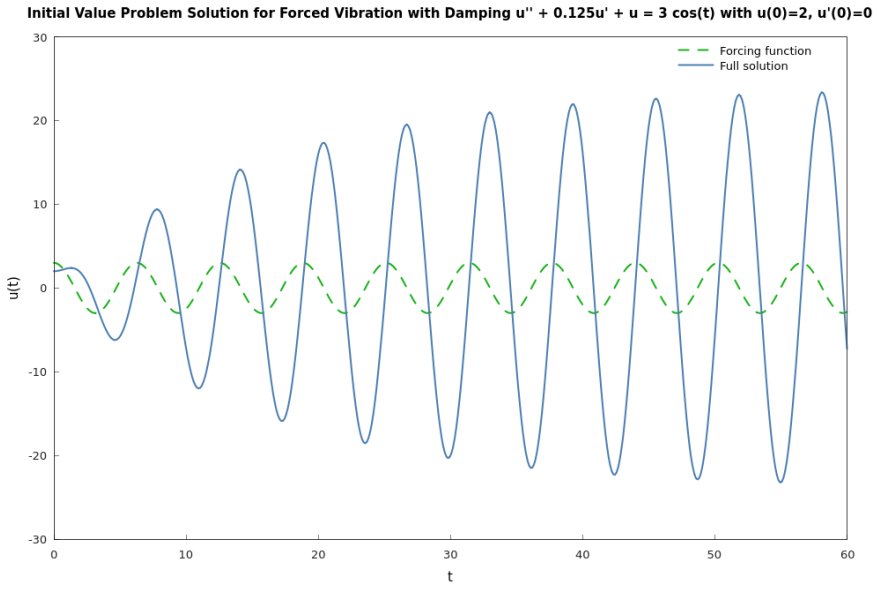

Figure 3.37: *A forced vibration with damping; solution of* $u'' + 0.125u' + u = 3\cos\ t, u(0) = 2, u'(0) = 0$ *(**Hamzstplot/Examples with Makefile/Differential Equations/Plot 2D Forced Vibrations IVP Second Order ODE Solution and Forcing Function/main.cpp**).*

III. Higher Order Linear Equations

[SI*] **General Theory of nth Order Linear Equations**
An nth order linear differential equation is an equation of the form

$$P_0(t)\frac{d^n y}{dt^n} + P_1(t)\frac{d^{n-1} y}{dt^{n-1}} + \cdots + P_{n-1}(t)\frac{dy}{dt} + P_n(t)y = G(t) \tag{3.205}$$

We assume that the functions $P_0, \ldots, P_n$ and G are continuous real-valued functions on some interval $I : \alpha < t < \beta$, and that P_0 is nowhere zero in this interval. Then, dividing Eq. (3.205) by $P_0(t)$, we obtain

$$L[y] = \frac{d^n y}{dt^n} + p_1(t)\frac{d^{n-1}y}{dt^{n-1}} + \cdots + p_{n-1}(t)\frac{dy}{dt} + p_n(t)y = g(t) \tag{3.206}$$

The linear differential operator L of order n defined by Eq. (3.206) is similar to the second order operator. The mathematical theory associated with Eq. (3.206) is completely analogous to that for the second order linear equation; for this reason we simply state the results for the nth order problem. The proofs of most of the results are also similar to those for the second order equation and are usually left as exercises.

Since Eq. (3.206) involves the nth derivative of y with respect to t, it will, so to speak, require n integrations to solve Eq. (3.206). Each of these integrations introduces an arbitrary constant. Hence we can expect that, to obtain a unique solution, it is necessary to specify n initial conditions

$$y(t_0) = y_0, \quad y'(t_0) = y_0', \quad \ldots, y^{(n-1)}(t_0) = y_0^{(n-1)} \tag{3.207}$$

where t_0 may be any point in the interval I and $y_0, \ldots, y_0^{(n-1)}$ is any set of prescribed real constants.

Theorem 3.9: Existence and Uniqueness

If the functions $p_1, p_2, \ldots, p_n$, and g are continuous on the open interval I then there exists exactly one solution $y = \phi(t)$ of the differential equation (3.206) that also satisfies the initial conditions (3.207). This solution exists throughout the interval I.

The Homogeneous Equation

As in the corresponding second order problem, we first discuss the homogeneous equation

$$L[y] = y^{(n)} + p_1(t)y^{(n-1)} + \cdots + p_{n-1}(t)y' + p_n(t)y = 0 \tag{3.208}$$

If the functions $y_1, y_2, \ldots, y_n$ are solutions of Eq. (3.208), then it follows by direct computation that the linear combination

$$y = c_1 y_1(t) + c_2 y_2(t) + \cdots + c_n y_n(t) \tag{3.209}$$

where $c_1, c_2, \ldots, c_n$ are arbitrary constants, is also a solution of Eq. (3.208). For any choice of the point t_0 in I, and for any choice of $y_0, y_0', \ldots, y_0^{(n-1)}$, we must be able to determine $c_1, \ldots, c_n$ so that the equations

$$\begin{aligned} c_1 y_1(t_0) + \cdots + c_n y_n(t_0) &= y_0 \\ c_1 y_1'(t_0) + \cdots + c_n y_n'(t_0) &= y_0' \\ &\vdots \\ c_1 y_1^{(n-1)}(t_0) + \cdots + c_n y_n^{(n-1)}(t_0) &= y_0^{(n-1)} \end{aligned} \tag{3.210}$$

are satisfied. Equations (3.210) can be solved uniquely for the constants $c_1, \ldots, c_n$, provided that the determinant of coefficients is not zero. Ont he other hand, if the determinant of coefficients is zero, then it is always possible to choose values of $y_0, y_0', \ldots, y_0^{(n-1)}$ so that Eqs.

(3.210) do not have a solution. Hence a necessary and sufficient condition for the existence of a solution of Eqs. (3.210) for arbitrary values of $y_0, y_0', \ldots, y_0^{(n-1)}$ is that the Wronskian

$$W(y_1, \ldots, y_n) = \begin{vmatrix} y_1 & y_2 & \cdots & y_n \\ y_1' & y_2' & \cdots & y_n' \\ \vdots & \vdots & \ddots & \vdots \\ y_1^{(n-1)} & y_2^{(n-1)} & \cdots & y_n^{(n-1)} \end{vmatrix} \tag{3.211}$$

is not zero at $t = t_0$. Since t_0 can be any point in the interval I, it is necessary and sufficient that $W(y_1, y_2, \ldots, y_n)$ be nonzero at every point in the interval. Just as for the second order linear equation, it can be shown that if $y_1, y_2, \ldots, y_n$ are solutions of Eq. (3.208), then $W(y_1, y_2, \ldots, y_n)$ either is zero for every t in the interval I or else is never zero.

Theorem 3.10: Wronskian and Linear Combination

If the functions $p_1, p_2, \ldots, p_n$ are continuous on the open interval I, if the functions $y_1, y_2, \ldots, y_n$ are solutions of Eq. (3.208), and if $W(y_1, y_2, \ldots, y_n) \neq 0$ for at least one point in I, then every solution of Eq. (3.208) can be expressed as a linear combination of the solutions $y_1, y_2, \ldots, y_n$.

A set of solutions $y_1, y_2, \ldots, y_n$ of Eq. (3.208) whose Wronskian is nonzero is refered to as a fundamental set of solutions. The existence of a fundamental set of solutions can be demonstrated precisely the same way as for the second order linear equation. Since all solutions of Eq. (3.208) are of the form (3.209), we use the term general solution to refer to an arbitrary linear combination of any fundamental set of solutions of Eq. (3.208).

Linear Dependence and Independence

The functions $f_1, f_2, \ldots, f_n$ are said to be linearly dependent on an interval I if there exists a set of constants $k_1, k_2, \ldots, k_n$, not all zero, such that

$$k_1 f_1(t) + k_2 f_2(t) + \cdots + k_n f_n(t) = 0 \tag{3.212}$$

for all t in I. The functions $f_1, \ldots, f_n$ are said to be linearly independent on I if they are not linearly dependent there.

Theorem 3.11: Fundamental Set of Solutions on I

If $y_1(t), \ldots, y_n(t)$ is a fundamental set of solutions of Eq. (3.208)

$$L[y] = y^{(n)} + p_1(t)y^{(n-1)} + \cdots + p_{n-1}(t)y' + p_n(t)y = 0$$

on an interval I, then $y_1(t), \ldots, y_n(t)$ are linearly independent on I. Conversely, if $y_1(t), \ldots, y_n(t)$ are linearly independent solutions of Eq. (3.208) on I, then they form a fundamental set of solutions on I.

The Nonhomogeneous Equation

Now consider the nonhomogeneous equation (3.206)

$$L[y] = y^{(n)} + p_1(t)y^{(n-1)} + \cdots + p_{n-1}(t)y' + p_n(t)y = g(t)$$

If Y_1 and Y_2 are any two solutions of Eq. (3.206), then it follows immediately from the linearity of the operator L that

$$L[Y_1 - Y_2](t) = L[Y_1](t) - L[Y_2](t) = g(t) - g(t) = 0$$

Hence the difference of any two solutions of the nonhomogeneous equation (3.206) is a solution of the homogeneous equation (3.208). Since any solution of the homogeneous equation can be expressed as a linear combination of a fundamental set of solutions $y_1, \ldots, y_n$, it follows that any solution of Eq. (3.206) can be written as

$$y = c_1 y_1(t) + c_2 y_2(t) + \cdots + c_n y_n(t) + Y(t) \tag{3.213}$$

where Y is some particular solution of the nonhomogeneous equation (3.206). The linear combination (3.213) is called the general solution of the nonhomogeneous equation (3.206). Thus the primary problem is to determine a fundamental set of solutions $y_1, \ldots, y_n$ of the homogeneous equation (3.208). If the coefficients are constants, this is a fairly simple problem. If the coefficients are not constants, it is usually necessary to use numerical methods or series methods. The method of reduction of order also applies to nth order linear equations, but seldom useful for equations of higher than second order.

[SI*] **Homogeneous Equations with Constant Coefficients**

Consider the nth order linear homogeneous differential equation

$$L[y] = a_0 y^{(n)} + a_1 y^{(n-1)} + \cdots + a_{n-1} y' + a_n y = 0 \tag{3.214}$$

where $a_0, a_1, \ldots, a_n$ are real constants. From our knowledge of second order linear equations with constant coefficients, it is natural to anticipate that $y = e^{rt}$ is a solution of Eq. (3.214) for suitable values of r. Indeed,

$$L[e^{rt}] = e^{rt}(a_0 r^n + a_1 r^{n-1} + \cdots + a_{n-1} r + a_n) = e^{rt} Z(r) \tag{3.215}$$

for all r, where

$$Z(r) = a_0 r^n + a_1 r^{n-1} + \cdots + a_{n-1} r + a_n \tag{3.216}$$

For those values of r for which $Z(r) = 0$, it follows that $L[e^{rt}] = 0$ and $y = e^{rt}$ is a solution of Eq. (3.214). The polynomial $Z(r)$ is called the characteristic polynomial, and the equation $Z(r) = 0$is the characteristic equation of the differential equation (3.214). A polynomial of degree n has n zeros, say, $r_1, r_2, \ldots, r_n$, some of which may be equal; hence we can write the characteristic polynomial in the form

$$Z(r) = a_0(r - r_1)(r - r_2)\ldots(r - r_n) \tag{3.217}$$

Real and Unequal Roots

If the roots of the characteristic equation are real and no two are equal, then we have n distinct solutions $e^{r_1 t}, e^{r_2 t}, \ldots, e^{r_n t}$ of Eq. (3.214). If these functions are linearly independent, then the general solution of Eq. (3.214) is

$$y = c_1 e^{r_1 t} + c_2 e^{r_2 t} + \cdots + c_n e^{r_n t} \tag{3.218}$$

One way to establish the linear independence of $e^{r_1 t}, e^{r_2 t}, \ldots, e^{r_n t}$ is to evaluate their Wronskian determinant.

[SI*] **Example:**
Find the general solution of

$$y^{(4)} + y''' - 7y'' - y' + 6y = 0 \tag{3.219}$$

Also find the solution that satisfies the initial conditions

$$y(0) = 1, \quad y'(0) = 0, \quad y''(0) = -2, \quad y'''(0) = -1 \tag{3.220}$$

and plot its graph.

Solution:
Assuming that $y = e^{rt}$, we must determine r by solving the polynomial equation

$$r^4 + r^3 - 7r^2 - r + 6 = 0 \tag{3.221}$$

The roots of this equation are $r_1 = 1, r_2 = -1, r_3 = 2, r_4 = -3$. Therefore the general solution of Eq. (3.219) is

$$y = c_1 e^t + c_2 e^{-t} + c_3 e^{2t} + c_4 e^{-3t} \tag{3.222}$$

The initial conditions (3.220) require that $c_1, \ldots, c_4$ satisfy the four equations

$$\begin{aligned} c_1 + c_2 + c_3 + c_4 &= 1 \\ c_1 - c_2 + 2c_3 - 3c_4 &= 0 \\ c_1 + c_2 + 4c_3 + 9c_4 &= -2 \\ c_1 - c_2 + 8c_3 - 27c_4 &= -1 \end{aligned} \tag{3.223}$$

By solving this system of four linear algebraic equations, we find that

$$c_1 = \frac{11}{8}, \quad c_2 = \frac{5}{12}, \quad c_3 = -\frac{2}{3}, \quad c_4 = -\frac{1}{8}$$

Therefore the solution of the initial value problem is

$$y = \frac{11}{8}e^t + \frac{5}{12}e^{-t} - \frac{2}{3}e^{2t} - \frac{1}{8}e^{-3t} \tag{3.224}$$

```
Poynomial : r^(4)+r^(3)-7*r^(2)-r+6
P:
(1,0)
(1,0)
(-7,0)
(-1,0)
(6,0)

P':
(4,0)
(3,0)
(-14,0)
(-1,0)

Initial guess:
(1.1,0)
(2,0)
(3,0)
(4,0)

Roots found at iteration: 6

**********************************************************************

End of iteration

z_{i} final:
(1,0)
(2,0)
(-1,0)
(-3,0)

Unique vector:
(1,0)
(2,0)
(-1,0)
(-3,0)

Duplicate vector:

The general solution is:
y(t) = e^t*c[0]+e^(2*t)*c[1]+e^(-t)*c[2]+e^(-3*t)*c[3]

The initial value problem solution is:
y(t) = 1.375*e^t-0.666667*e^(2*t)+0.416667*e^(-t)-0.125*e^(-3*t)

Time taken by function: 109664 microseconds
```

Figure 3.38: *The computation of the initial value problem general solution for the fourth order homogeneous differential equation with real and unequal roots (**SymIntegration/Examples/Differential Equations/Test SymIntegration IVP Solution for Higher Order Linear Differential Equations/main.cpp**).*

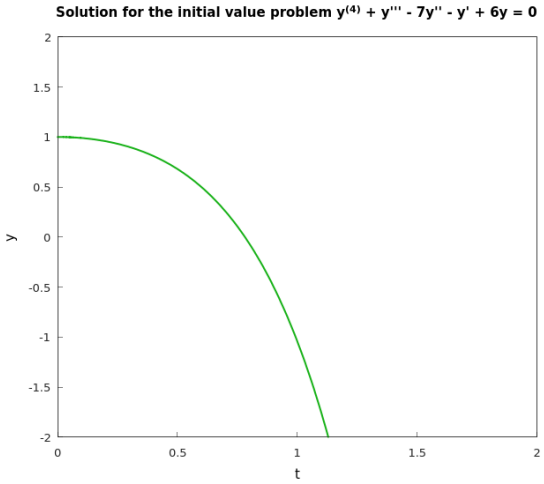

Figure 3.39: *The plot of the solution of the initial value problem from the differential equation (3.219) with the initial conditions from Eq. (3.220) (**Hamzstplot/Examples with Makefile/Differential Equations/Plot 2D IVP Solution for Higher Order Homogeneous Linear Differential Equation/main.cpp**).*

Complex Roots

If the characteristic equation has complex roots, they must occur in conjugate pairs, $\lambda \pm i\mu$, since the coefficients $a_0, \ldots, a_n$ are real numbers. Provided that none of the roots is repeated,

the general solution of Eq. (3.214) is still of the form (3.218). However, just as for the second order equation, we can replace the complex-valued solution $e^{(\lambda+i\mu)t}$ and $e^{(\lambda-i\mu)t}$ by the rea-valued solutions

$$e^{\lambda t}\cos(\mu t), \quad e^{\lambda t}\sin(\mu t) \tag{3.225}$$

obtained as the real and imaginary parts of $e^{(\lambda+i\mu)t}$. Thus, even though some of the roots of the characteristic equation are complex, it is still possible to express the general solution of Eq. (3.214) as a linear combination of real-valued solutions.

[SI*] **Example:**
Find the general solution of

$$y^{(4)} - y = 0 \tag{3.226}$$

Also find the solution that satisfies the initial conditions

$$y(0) = \frac{7}{2}, \quad y'(0) = -4, \quad y''(0) = \frac{5}{2}, \quad y'''(0) = -2 \tag{3.227}$$

and draw its graph.

Solution:
Substituting e^{rt} for y, we find that the characteristic equation is

$$r^4 - 1 = (r^2 - 1)(r^2 + 1) = 0$$

Therefore the roots are $r = 1, -1, i, -i$, and the general solution of Eq. (3.226) is

$$y = c_1 e^t + c_2 e^{-t} + c_3 \cos\ t + c_4 \sin\ t$$

If we impose the initial conditions (3.227), we find that

$$c_1 = 0, \quad c_2 = 3, \quad c_1 = \frac{1}{2}, \quad c_4 = -1$$

thus the solution of the given initial value problem is

$$y = 3e^{-t} + \frac{1}{2}\cos\ t - \sin\ t \tag{3.228}$$

Observe that the initial conditions (3.227) cause the coefficient c_1 of the exponentially growing term in the general solution to be zero. Therefore this term is absent in the solution (3.228), which describes an exponential decay to a steady oscillation. However, if the initial conditions are changed slightly, then c_1 is likely to be nonzero, and the nature of the solution changes enormously. For example, if the first three initial conditions remain the same, but the value of $y'''(0)$ is changed from -2 to $-\frac{15}{8}$, then the solution of the initial value problem becomes

$$y = \frac{1}{32}e^t + \frac{95}{32}e^{-t} + \frac{1}{2}\cos\ t - \frac{17}{16}\sin\ t \tag{3.229}$$

The coefficients in Eq. (3.229) differ only slightly from those in Eq. (3.228), but the exponentially growing term, even with the relatively small coefficient of $\frac{1}{32}$, completely dominates the solution by the time t is larger than about 4 or 5.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration IVP Solution for Higher Order Linear Differential Equations
 ]# ./main
Poynomial : r^(4)-1
P:
(1,0)
(0,0)
(0,0)
(0,0)
(-1,0)

P':
(4,0)
(0,0)
(0,0)
(0,0)

Initial guess:
(1,0)
(2,0)
(3,0)
(4,0)

Roots found at iteration: 31

**************************************************************************

End of iteration

z_{i} final:
(1,0)
(-1,0)
(0,1)
(0,-1)

The general solution is:
y(t) = e^(-t)*c[0]+c[1]*cos(t)+c[2]*sin(t)+e^t*c[3]

The initial value problem solution is:
y(t) = 3*e^(-t)+0.5*cos(t)-sin(t)

Time taken by function: 159680 microseconds
```

Figure 3.40: *The computation of the initial value problem general solution for the fourth order homogeneous differential equation with complex roots (**SymIntegration/Examples/Differential Equations/Test SymIntegration IVP Solution for Higher Order Linear Differential Equations/main.cpp**).*

```
P:
(1,0)
(0,0)
(0,0)
(0,0)
(-1,0)

P':
(4,0)
(0,0)
(0,0)
(0,0)

Initial guess:
(1.1,0)
(2,0)
(3,0)
(4,0)

Roots found at iteration: 31

************************************************************************

End of iteration

z_{i} final:
(1,0)
(0,1)
(0,-1)
(-1,0)

Unique vector:
(1,0)
(0,1)
(0,-1)
(-1,0)

Duplicate vector:

The general solution is:
y(t) = e^t*c[0]+c[1]*cos(t)+c[2]*sin(t)+e^(-t)*c[3]

The initial value problem solution is:
y(t) = 0.03125*e^t+0.5*cos(t)-1.0625*sin(t)+2.96875*e^(-t)

Time taken by function: 168041 microseconds
```

Figure 3.41: *The computation of the initial value problem general solution for the fourth order homogeneous differential equation with complex roots that will have the solution of Eq. (3.229) (**SymIntegration/Examples/Differential Equations/Test SymIntegration IVP Solution for Higher Order Linear Differential Equations/main.cpp**).*

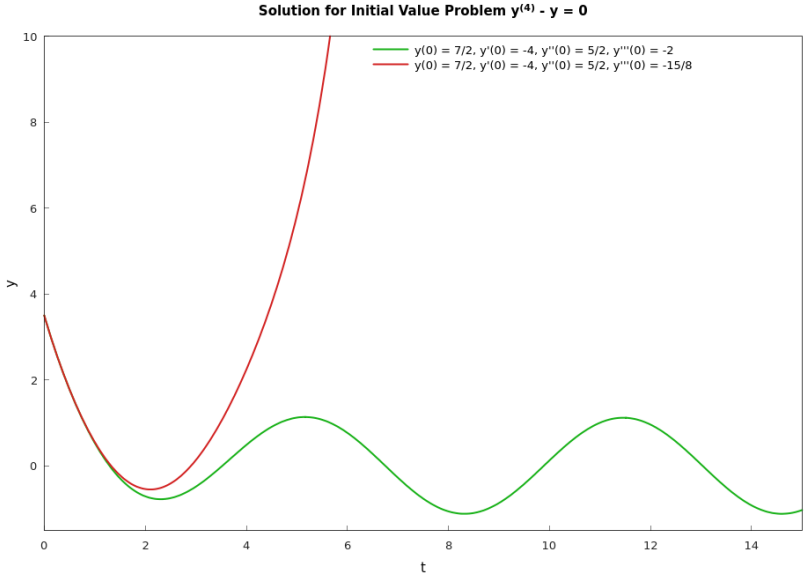

Figure 3.42: *The plots of the solution of the initial value problem of Eqs. (3.228) and (3.229) (**Hamzstplot/Examples with Makefile/Differential Equations/Plot 2D IVP Solution for Higher Order Homogeneous Linear Differential Equation with Complex Roots/main.cpp**).*

Repeated Roots

If the roots of the characteristic equation are not distinct-that is, if some roots are repeated-then the solution (3.218) is clearly not the general solution of Eq. (3.214). Recall that if r_1 is a repeated root for the second order linear equation $a_0y'' + a_1y' + a_2y = 0$, then two linearly independent solutions are e^{r_1t} and te^{r_1t}. For an equation of order n, if a root of $Z(r) = 0$, say $r = r_1$, has multiplicity s (where $s \leq n$), then

$$e^{r_1t}, \quad te^{r_1t}, \quad t^2e^{r_1t}, \quad \ldots, \quad t^{s-1}e^{r_1t} \tag{3.230}$$

are corresponding solutions of Eq. (1).

If a complex root $\lambda \pm i\mu$ is repeated s times, the complex conjugate $\lambda - i\mu$ is also repeated s times. Corresponding to these $2s$ complex-valued solutions, we can find $2s$ real-valued solutions by noting that the real and imaginary parts of $e^{(\lambda+i\mu)t}, te^{(\lambda+i\mu)t}, \ldots, t^{s-1}e^{(\lambda+i\mu)t}$ are also linearly independent solutions:

$$e^{\lambda t}\cos(\mu t), \quad e^{\lambda t}\sin(\mu t), te^{\lambda t}\cos(\mu t), \quad te^{\lambda t}\sin(\mu t),$$
$$\ldots t^{s-1}e^{\lambda t}\cos(\mu t), \quad t^{s-1}e^{\lambda t}\sin(\mu t),$$

Hence the general solution of Eq. (3.214) can always be expressed as a linear combination of n real-valued solutions.

[SI*] **Example:**

Find the general solution of

$$y^{(4)} + 2y'' + y = 0 \tag{3.231}$$

The characteristic equation is

$$r^4 + 2r^2 + 1 = (r^2 + 1)(r^2 + 1) = 0$$

The roots are $r = i, i, -i, -i$, and the general solution of Eq. (3.231) is

$$c_1 \cos\ t + c_2 \sin\ t + c_3 t \cos\ t + c_4 t \sin\ t$$

```
P:
(1,0)
(0,0)
(2,0)
(0,0)
(1,0)

P':
(4,0)
(0,0)
(4,0)
(0,0)

Initial guess:
(1.1,0)
(2,0)
(3,0)
(4,0)

Roots found at iteration: 47

************************************************************************

End of iteration

z_{i} final:
(0,-1)
(0,1)
(0,1)
(0,-1)
```

Figure 3.43: *The computation of the initial value problem general solution for the fourth order homogeneous differential equation with repeated roots (**SymIntegration/Examples/Differential Equations/Test SymIntegration IVP Solution for Higher Order Linear Differential Equations/main.cpp**).*

```
Roots found at iteration: 47

************************************************************************

End of iteration

z_{i} final:
(0,-1)
(0,1)
(0,1)
(0,-1)

Unique vector:

Duplicate vector:
(0,-1)
(0,1)
(0,1)
(0,-1)

Deleted Duplicate vector:
(0,-1)
(0,1)

Final Duplicate vector:
(0,1)

The general solution is:
y(t) = c[0]*cos(t)+c[1]*sin(t)+c[2]*cos(t)*t+c[3]*sin(t)*t

The initial value problem solution is:
y(t) = cos(t)-0.5*sin(t)+0.5*cos(t)*t-0.5*sin(t)*t

Time taken by function: 150408 microseconds
```

Figure 3.44: *The computation of the initial value problem general solution for the fourth order homogeneous differential equation with repeated roots (**SymIntegration/Examples/Differential Equations/Test SymIntegration IVP Solution for Higher Order Linear Differential Equations/main.cpp**).*

[SI*] **Example:**
Consider the spring-mass system, consisting of two unit masses suspended from springs with spring constants 3 and 2, respectively. Assume that there is no damping in the system.

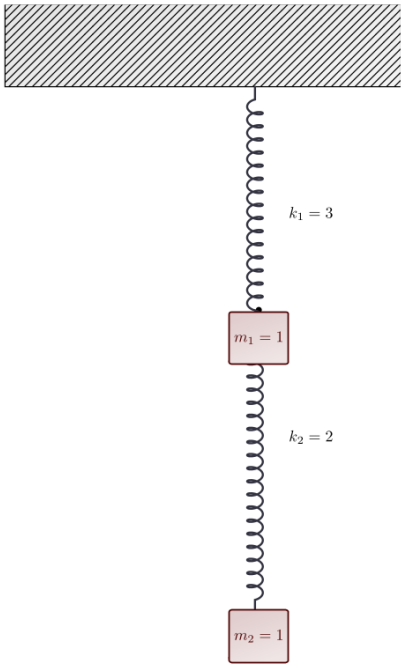

Figure 3.45: *A two-spring, two-mass system.*

(a) Show that the displacements u_1 and u_2 of the masses from their respective equilibrium

positions satisfy the equations

$$u_1'' + 5u_1 = 2u_2, \quad u_2'' + 2u_2 = 2u_1 \tag{3.232}$$

(b) Solve the first of Eqs. (3.232) for u_2 and substitute into the second equation, thereby obtaining the following fourth order equation for u_1:

$$u_1^{(4)} + 7u_1'' + 6u_1 = 0 \tag{3.233}$$

Find the general solution of Eq. (3.233)

(c) Suppose that the initial conditions are

$$u_1(0) = 1, \quad u_1'(0) = 0, \quad u_2(0) = 2, \quad u_2'(0) = 0 \tag{3.234}$$

Use the first of Eqs. (3.232) and the initial conditions (3.234) to obtain values for $u_1''(0)$ and $u_1'''(0)$. Then show that the solution of Eq. (3.233) that satisfies the four initial conditions on u_1 is $u_1(t) = \cos\ t$. Show that the corresponding solution $u_2(t) = 2\cos\ t$.

(d) Now suppose that the initial conditions are

$$u_1(0) = -2, \quad u_1'(0) = 0, \quad u_2(0) = 1, \quad u_2'(0) = 0 \tag{3.235}$$

Proceed as in part (c) to show that the corresponding solutions are $u_1(t) = -2\cos\sqrt{6}t$ and $u_2(t) = \cos\sqrt{6}t$.

(e) Observe that the solutions obtained in parts (c) and (d) describe two distinct modes of vibration. In the first, the frequency of the motion is 1, and the two masses move in phase, both moving up or down together; the second mass moves twice as far as the first. The second motion has frequency $\sqrt{6}$, and the masses move out of phase with each other, one moving down while the other is moving up, and vice versa. In this mode the first mass moves twice as far as the second. For other initial conditions, not proportional to either of Eqs. (3.234) or (3.235), the motion of the masses is a combination of these two modes.

Solution:

(a) Start by drawing free-body diagram for the two masses. If m_1 is displaced a distance u_1 and m_2 is displaced a distance u_2, then we have the following two diagrams.

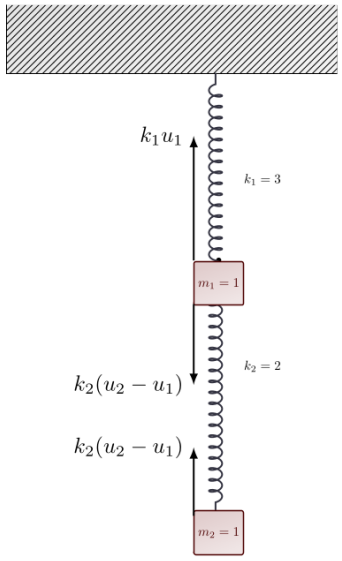

Figure 3.46: *A two-spring, two-mass system.*

Apply Newton's second law twice-once for m_1 and once for m_2.

$$\begin{aligned} k_2(u_2 - u_1) - k_1 u_1 &= m_1 a_1 \\ -k_2(u_2 - u_1) &= m_2 a_2 \end{aligned}$$

Plug in the numbers for k_1, k_2, m_1, and m_2 and use the fact that acceleration is the second derivative of position.

$$\begin{aligned} 2(u_2 - u_1) - 3u_1 &= u_1'' \\ -2(u_2 - u_1) &= u_2'' \end{aligned}$$

Therefore, the equations of motion are

$$u_1'' + 5u_1 = 2u_2 \tag{3.236}$$

$$u_2'' + 2u_2 = 2u_1 \tag{3.237}$$

(b) Divide both sides of equation (3.236) by 2

$$\frac{1}{2}u_1'' + \frac{5}{2}u_1 = u_2$$

and plug this formula for u_2 into equation (3.237).

$$\begin{aligned} \left(\frac{1}{2}u_1'' + \frac{5}{2}u_1\right)'' + 2\left(\frac{1}{2}u_1'' + \frac{5}{2}u_1\right) &= 2u_1 \\ \frac{1}{2}u_1^{(4)} + \frac{5}{2}u_1'' + 5u_1 &= 2u_1 \end{aligned}$$

$$u_1^{(4)} + 7u_1'' + 6u_1 = 0 \tag{3.238}$$

This is a homogeneous ODE with constant coefficients, so the solution is of the form $u_1 = e^{rt}$.

$$\begin{aligned} u_1 &= e^{rt} \\ u_1' &= re^{rt} \\ u_1'' &= r^2 e^{rt} \\ u_1''' &= r^3 e^{rt} \\ u_1^{(4)} &= r^4 e^{rt} \end{aligned}$$

Substitute these expressions into the ODE

$$r^4 e^{rt} + 7(r^2 e^{rt}) + 6(e^{rt}) = 0$$

Divide both sides by e^{rt}.

$$\begin{aligned} r^4 + 7r^2 + 6 &= 0 \\ (r^2+6)(r^2+1) &= 0 \\ r &= \{-\sqrt{6}i, \sqrt{6}i, -i, i\} \end{aligned}$$

Four solutions to equation (3.238) are then $u_1 = e^{-\sqrt{6}it}$ $u_1 = e^{\sqrt{6}it}$ and $u_1 = e^{-it}$ and $u_1 = e^{it}$. By the principle of superposition, the general solution for u_1 is a linear combination of these four.

$$\begin{aligned} u_1(t) &= C_1 e^{-\sqrt{6}it} + C_2 e^{\sqrt{6}it} + C_3 e^{-it} + C_4 e^{it} \\ &= C_1(\cos\sqrt{6}t - i\sin\sqrt{6}t) + C_2(\cos\sqrt{6}t + i\sin\sqrt{6}t) + C_3(\cos\ t - i\sin\ t) + C_4(\cos\ t + i\sin\ t) \\ &= (C_1 + C_2)\cos\sqrt{6}t + (-iC_1 + iC_2)\sin\sqrt{6}t + (C_3 + C_4)\cos\ t + (-iC_3 + iC_4)\sin\ t \\ &= C_5\cos\sqrt{6}t + C_6\sin\sqrt{6}t + C_7\cos\ t + C_8\sin\ t \end{aligned}$$

(c) Suppose that the initial conditions are

$$\begin{aligned} u_1(0) = 1, \quad u_1'(0) = 0 \\ u_2(0) = 2, \quad u_2'(0) = 0 \end{aligned}$$

From equation (3.236), $u_1''(0)$ and $u_1'''(0)$ can be determined.

$$u_1'' = 2u_2 - 5u_1 \rightarrow \begin{cases} u_1''(0) = 2u_2(0) - 5u_1(0) = -1 \\ u_1'''(0) = 2u_2'(0) - 5u_1'(0) = 0 \end{cases}$$

Differentiate the general solution three times.

$$\begin{aligned} u_1(t) &= C_5\cos\sqrt{6}t + C_6\sin\sqrt{6}t + C_7\cos\ t + C_8\sin\ t \\ u_1'(t) &= -C_5\sqrt{6}\sin\sqrt{6}t + C_6\sqrt{6}\cos\sqrt{6}t - C_7\sin\ t + C_8\cos\ t \\ u_1''(t) &= -6C_5\cos\sqrt{6}t - 6C_6\sin\sqrt{6}t - C_7\cos\ t - C_8\sin\ t \\ u_1'''(t) &= 6\sqrt{6}C_5\sin\sqrt{6}t - 6\sqrt{6}C_6\cos\sqrt{6}t + C_7\sin\ t - C_8\cos\ t \end{aligned}$$

Apply the initial conditions now to determine C_5, C_6, C_7, and C_8.

$$\begin{aligned} u_1(0) &= C_5 + C_7 = 1 \\ u_1'(0) &= C_6\sqrt{6} + C_8 = 0 \\ u_1''(0) &= -6C_5 - C_7 = -1 \\ u_1'''(0) &= -6\sqrt{6}C_6 - C_8 = 0 \end{aligned}$$

Solving this system of equations yields $C_5 = 0, C_6 = 0, C_7 = 1$, and $C_8 = 0$. Therefore,

$$u_1(t) = \cos\ t$$

Now that $u_1(t)$ is known, plug it back into equation (3.237) to find $u_2(t)$.

$$u_2'' + 2u_2 = 2\cos\ t$$

This is a linear nonhomogeneous ODE, so its general solution can be expressed as a sum of the complementary solution and the particular solution.

$$u_2(t) = u_{2c}(t) + u_{2p}(t)$$

The complementary solution satisfies the associated homogeneous equation.

$$u_{2c}'' + 2u_{2c} = 0 \tag{3.239}$$

This is a homogeneous ODE with constant coefficients, so the solution is of the form $u_{2c} = e^{st}$.

$$u_{2c} = e^{st} \ \rightarrow\ u_{2c}' = se^{st} \ \rightarrow\ u_{2c}'' = s^2e^{st}$$

Substitute these expressions into the ODE.

$$s^2e^{st} + 2(e^{st}) = 0$$

To compute s, divide both sides by e^{st}

$$\begin{aligned} s^2 + 2 &= 0 \\ s &= \{-i\sqrt{2}, i\sqrt{2}\} \end{aligned}$$

As a result, two solutions to equation (3.239) are $u_{2c} = e^{-i\sqrt{2}t}$ and $u_{2c} = e^{i\sqrt{2}t}$. By the principle of superposition, the general solution for u_{2c} is a linear combination of these two.

$$\begin{aligned} u_{2c}(t) &= C_9e^{-i\sqrt{2}t} + C_{10}e^{i\sqrt{2}t} \\ &= C_9(\cos\sqrt{2}t - i\sin\sqrt{2}t) + C_{10}(\cos\sqrt{2}t + i\sin\sqrt{2}t) \\ &= (C_9 + C_{10})\cos\sqrt{2}t + (-iC_9 + iC_{10})\sin\sqrt{2}t \\ &= C_{11}\cos\sqrt{2}t + C_{12}\sin\sqrt{2}t \end{aligned}$$

On the other hand, the particular solution satisfies

$$u_{2p}'' + 2u_{2p} = 2\cos\ t$$

Since the nonhomogeneous term is cosine and there are only even derivatives on the left, we use the trial solution $u_{2p} = A\cos\ t$. Plug this into the ODE to determine A.

$$\begin{aligned}(A\cos\ t)'' + 2(A\cos\ t) &= 2\cos\ t \\ (-A\cos\ t) + 2A\cos\ t &= 2\cos\ t \\ A\cos\ t &= 2\cos\ t\end{aligned}$$

Matching the coefficients, we see that $A = 2$, which means $u_{2p}(t) = 2\cos\ t$. The general solution for $u_2(t)$ is then

$$u_2(t) = C_{11}\cos\sqrt{2}t + C_{12}\sin\sqrt{2}t + 2\cos\ t$$

Differentiate it with respect to t.

$$u_2'(t) = -C_{11}\sqrt{2}\sin\sqrt{2}t + C_{12}\sqrt{2}\cos\sqrt{2}t - 2\sin\ t$$

Apply the initial conditions to determine C_{11} and C_{12}.

$$\begin{aligned}u_2(0) &= C_{11} + 2 = 2 \\ u_2'(0) &= C_{12}\sqrt{2} = 0\end{aligned}$$

Solving this system of equations yields $C_{11} = 0$ and $C_{12} = 0$. Therefore,

$$u_2(t) = 2\cos\ t$$

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Two Spring Two Mass System Higher Order Linear Differential Equations ]# ./
main

Two-spring, two-mass system solution k_{1} = 3, k_{2} = 2, m_{1} = 1, m_{2} = 1,
with u_{1}(0) = 1, u_{1}'(0) = 0, u_{2}(0) = 2, u_{2}'(0) = 0


The equation of motion :
df(u1[t],t,t)-2*u2+5*u1 = 0
df(u2[t],t,t)+2*u2-2*u1 = 0

u2 == 0.5*df(u1[t],t,t)+2.5*u1

The homogeneous ODE with constant coefficients:
0.5*df(u1[t],t,t,t,t)+3.5*df(u1[t],t,t)+3*u1[t] = 0

*******************************************************************************

********************             For u_{1}(t)              ******************

*******************************************************************************

P:
(1,0)
(0,0)
(7,0)
(0,0)
(6,0)

P':
(4,0)
(0,0)
(14,0)
(0,0)

Initial conditions (u_{1} and u_{2}):
(1,0)
(0,0)
(2,0)
(0,0)

Initial conditions (u_{1}):
(1,0)
(0,0)
(-1,0)
(0,0)
```

Figure 3.47: *The computation of the initial value problem general solution for the fourth order homogeneous differential equation that represent the two-spring, two-mass system for part (c) (**SymIntegration/Examples/Differential Equations/Test SymIntegration Two Spring Two Mass System Higher Order Linear Differential Equations/main.cpp**).*

```
*************************************************************************

*******************             For u_{1}(t)              *****************

*************************************************************************

P:
(1,0)
(0,0)
(7,0)
(0,0)
(6,0)

P':
(4,0)
(0,0)
(14,0)
(0,0)

Initial conditions (u_{1} and u_{2}):
(1,0)
(0,0)
(2,0)
(0,0)

Initial conditions (u_{1}):
(1,0)
(0,0)
(-1,0)
(0,0)

Initial guess for the roots (generated randomly):
(5.59957,0)
(2.82398,0)
(6.61124,0)
(3.93247,0)

Roots found at iteration: 26

*************************************************************************

End of iteration

z_{i} final:
(0,1)
(0,-1)
(0,2.44949)
(0,-2.44949)

The general solution is:
u_{1}(t) = c[0]*cos(t)+c[1]*sin(t)+c[2]*cos(2.44949*t)+c[3]*sin(2.44949*t)

The initial value problem solution is:
u_{1}(t) = cos(t)

The equation of motion 2 with u_{1}(t):
df(u2[t],t,t)+2*u2[t]-2*cos(t) = 0
```

Figure 3.48: *The computation of the initial value problem general solution for the fourth order homogeneous differential equation that represent the two-spring, two-mass system for part (c) (**SymIntegration/Examples/Differential Equations/Test SymIntegration Two Spring Two Mass System Higher Order Linear Differential Equations/main.cpp**).*

```
*************************************************************************

******************             For u_{2}(t)              *****************

*************************************************************************

P:
(1,0)
(0,0)
(2,0)

P':
(2,0)
(0,0)

Initial conditions (u_{2}):
(2,0)
(0,0)

Initial guess for the roots (generated randomly):
(5.19157,0)
(5.97163,0)

Roots found at iteration: 33

*************************************************************************

End of iteration

z_{i} final:
(0,-1.41421)
(0,1.41421)

The general solution is:
u_{2}(t) = c[0]*cos(1.41421*t)+c[1]*sin(1.41421*t)+2*cos(t)

The initial value problem solution is:
u_{2}(t) = 2*cos(t)

Time taken by function: 200112 microseconds
```

Figure 3.49: *The computation of the initial value problem general solution for the fourth order homogeneous differential equation that represent the two-spring, two-mass system for part (c) (**SymIntegration/Examples/Differential Equations/Test SymIntegration Two Spring Two Mass System Higher Order Linear Differential Equations/main.cpp**).*

(d) Suppose secondly that the initial conditions are

$$u_1(0) = -2, \quad u_1'(0) = 0$$
$$u_2(0) = 1, \quad u_2'(0) = 0$$

From equation (3.236), $u_1''(0)$ and $u_1'''(0)$ can be determined.

$$u_1'' + 5u_1 = 2u_2 \begin{cases} u_1''(0) = 2u_2(0) - 5u_1(0) = 12 \\ u_1'''(0) = 2u_2'(0) - 5u_1'(0) = 0 \end{cases}$$

Differentiate the general solution three times

$$u_1'(t) = -C_5\sqrt{6}\sin\sqrt{6}t + C_6\sqrt{6}\cos\sqrt{6}t - C_7\sin\ t + C_8\cos\ t$$
$$u_1''(t) = -6C_5\cos\sqrt{6}t - 6C_6\sin\sqrt{6}t - C_7\sin\ t - C_8\cos\ t$$
$$u_1'''(t) = 6\sqrt{6}C_5\sin\sqrt{6}t - 6\sqrt{6}C_6\cos\sqrt{6}t + C_7\sin\ t - C_8\cos\ t$$

Apply the initial conditions now to determine C_5, C_6, C_7, and C_8.

$$u_1(0) = C_5 + C_7 = -2$$
$$u_1'(0) = C_6\sqrt{6} + C_8 = 0$$
$$u_1''(0) = -6C_5 - C_7 = 12$$
$$u_1'''(0) = -6\sqrt{6}C_6 - C_8 = 0$$

Solving this system of equations yields $C_5 = -2, C_6 = 0, C_7 = 0$, and $C_8 = 0$. Therefore,

$$u_1(t) = -2\cos\sqrt{6}t$$

Now that $u_1(t)$ is known, plug it back into equation (2) to find $u_2(t)$.

$$u_2'' + 2u_2 = -4\cos\sqrt{6}t$$

This is a linear nonhomogeneous ODE, so its general solution can be expressed as a sum of the complementary solution and the particular solution.

$$u_2(t) = u_{2c}(t) + u_{2p}(t)$$

The complementary solution satisfies the associated homogeneous equation.

$$u_{2c}'' + 2u_{2c} = 0 \tag{3.240}$$

This is a homogeneous ODE with constant coefficients, so the solution is of the form $u_{2c} = e^{st}$.

$$u_{2c} = e^{st} \;\rightarrow\; u_{2c}' = se^{st} \;\rightarrow\; u_{2c}'' = s^2e^{st}$$

Substitute these expressions into the ODE.

$$s^2e^{st} + 2(e^{st}) = 0$$

To compute s, divide both sides by e^{st}

$$\begin{aligned} s^2 + 2 &= 0 \\ s &= \{-i\sqrt{2}, i\sqrt{2}\} \end{aligned}$$

As a result, two solutions to equation (3.240) are $u_{2c} = e^{-i\sqrt{2}t}$ and $u_{2c} = e^{i\sqrt{2}t}$. By the principle of superposition, the general solution for u_{2c} is a linear combination of these two.

$$\begin{aligned} u_{2c}(t) &= C_9e^{-i\sqrt{2}t} + C_{10}e^{i\sqrt{2}t} \\ &= C_9(\cos\sqrt{2}t - i\sin\sqrt{2}t) + C_{10}(\cos\sqrt{2}t + i\sin\sqrt{2}t) \\ &= (C_9 + C_{10})\cos\sqrt{2}t + (-iC_9 + iC_{10})\sin\sqrt{2}t \\ &= C_{11}\cos\sqrt{2}t + C_{12}\sin\sqrt{2}t \end{aligned}$$

On the other hand, the particular solution satisfies

$$u_{2p}'' + 2u_{2p} = -4\cos\sqrt{6}t$$

Since the nonhomogeneous term is cosine and there are only even derivatives on the left, we use the trial solution $u_{2p} = A\cos\sqrt{6}t$. Plug this into the ODE to determine A.

$$\begin{aligned} (A\cos\sqrt{6}t)'' + 2(A\cos\sqrt{6}t) &= -4\cos\sqrt{6}t \\ (-6A\cos\sqrt{6}t) + 2A\cos\sqrt{6}t &= -4\cos\sqrt{6}t \\ -4A\cos\sqrt{6}t &= -4\cos\sqrt{6}t \end{aligned}$$

Matching the coefficients, we see that $A = 1$, which means $u_{2p}(t) = \cos\sqrt{6}t$. The general solution for $u_2(t)$ is then

$$u_2(t) = C_{11}\cos\sqrt{2}t + C_{12}\sin\sqrt{2}t + \cos\sqrt{6}t$$

Differentiate it with respect to t.

$$u_2'(t) = -C_{11}\sqrt{2}\sin\sqrt{2}t + C_{12}\sqrt{2}\cos\sqrt{2}t - \sqrt{6}\sin\sqrt{6}t$$

Apply the initial conditions to determine C_{11} and C_{12}.

$$u_2(0) = C_{11} + 1 = 1$$
$$u_2'(0) = C_{12}\sqrt{2} = 0$$

Solving this system of equations yields $C_{11} = 0$ and $C_{12} = 0$. Therefore,

$$u_2(t) = \cos\sqrt{6}t$$

```
********************************************************************************

********************             For u_{1}(t)              ******************

********************************************************************************

P:
(1,0)
(0,0)
(7,0)
(0,0)
(6,0)

P':
(4,0)
(0,0)
(14,0)
(0,0)

Initial conditions (u_{1} and u_{2}):
(-2,0)
(0,0)
(1,0)
(0,0)

Initial conditions (u_{1}):
(-2,0)
(0,0)
(12,0)
(0,0)

Initial guess for the roots (generated randomly):
(4.55549,0)
(3.71947,0)
(4.50372,0)
(4.74454,0)

Roots found at iteration: 29
```

Figure 3.50: *The computation of the initial value problem general solution for the fourth order homogeneous differential equation that represent the two-spring, two-mass system for part (d) (**SymIntegration/Examples/Differential Equations/Test SymIntegration Two Spring Two Mass System Higher Order Linear Differential Equations/main.cpp**).*

```
**************************************************************************

End of iteration

z_{i} final:
(0,2.44949)
(0,-2.44949)
(0,-1)
(0,1)

The general solution is:
u_{1}(t) = c[0]*cos(t)+c[1]*sin(t)+c[2]*cos(2.44949*t)+c[3]*sin(2.44949*t)

The initial value problem solution is:
u_{1}(t) = -2*cos(2.44949*t)

The equation of motion 2 with u_{1}(t):
df(u2[t],t,t)+2*u2[t]+4*cos(2.44949*t) = 0

**************************************************************************

******************              For u_{2}(t)               ******************

**************************************************************************

P:
(1,0)
(0,0)
(2,0)

P':
(2,0)
(0,0)

Initial conditions (u_{2}):
(1,0)
(0,0)

Initial guess for the roots (generated randomly):
(5.0858,0)
(5.39588,0)

Roots found at iteration: 33
```

Figure 3.51: *The computation of the initial value problem general solution for the fourth order homogeneous differential equation that represent the two-spring, two-mass system for part (d) (**SymIntegration/Examples/Differential Equations/Test SymIntegration Two Spring Two Mass System Higher Order Linear Differential Equations/main.cpp**).*

```
********************************************************************************

********************             For u_{2}(t)              ******************

********************************************************************************

P:
(1,0)
(0,0)
(2,0)

P':
(2,0)
(0,0)

Initial conditions (u_{2}):
(1,0)
(0,0)

Initial guess for the roots (generated randomly):
(5.0858,0)
(5.39588,0)

Roots found at iteration: 33

************************************************************************

End of iteration

z_{i} final:
(0,1.41421)
(0,-1.41421)

The general solution is:
u_{2}(t) = c[0]*cos(1.41421*t)+c[1]*sin(1.41421*t)+1*cos(2.44949*t)

The initial value problem solution is:
u_{2}(t) = 1*cos(2.44949*t)

Time taken by function: 207694 microseconds
```

Figure 3.52: *The computation of the initial value problem general solution for the fourth order homogeneous differential equation that represent the two-spring, two-mass system for part (d) (**SymIntegration/Examples/Differential Equations/Test SymIntegration Two Spring Two Mass System Higher Order Linear Differential Equations/main.cpp**).*

[SI*] **The Method of Undetermined Coefficients**

A particular solution Y of the nonhomogeneous nth order linear equation with constant coefficients

$$L[y] = a_0 y^{(n)} + a_1 y^{(n-1)} + \cdots + a_{n-1} y' + a_n y = g(t) \tag{3.241}$$

can be obtained by the method of undetermined coefficients, provided that $g(t)$ is of an appropriate form. Although the method of undetermined coefficients is not as general as the method of variation of parameters, it is usualy much easier to use when it is applicable. Just as for the second order linear equation, when the constant coefficient linear differential operator L is applied to a polynomial $A_0 t^m + A_1 t^{m-1} + \cdots + A_m$, an exponential function $e^{\alpha t}$, a sine function $\sin \beta t$, or a cosine function $\cos \beta t$, the result is a polynomial, an exponential function, or a linear combination of sine and cosine functions, respectively.

The main difference in using this method for higher order equations stems from the fact that roots of the chaacteristic polynomial equation may have multiplicity greater than 2.

[SI*] **Example:**

Find the general solution of

$$y''' - 3y'' + 3y' - y = 4e^t \tag{3.242}$$

Solution:

The characteristic polynomial for the homogeneous equation corresponding to Eq. (3.242) is

$$r^3 - 3r^2 + 3r - 1 = (r-1)^3$$

so the general solution of the homogeneous equation is

$$y_c(t) = c_1 e^t + c_2 t e^t + c_3 t^2 e^t \tag{3.243}$$

To find a particular solution $Y(t)$ of Eq. (3.242), we start by assuming that $Y(t) = Ae^t$. However, since e^t, te^t, and t^2e^t are all soltions of the homogeneous equation, we must multiply this initial choice by t^3. Thus our final assumption is that $Y(t) = At^3e^t$, where A is an undetermined coefficient. To find the correct value for A, we differentiate $Y(t)$ three times, substitute for y and its derivatives in Eq. (3.242), and collect terms in the resulting equation. In this way we obtain

$$6Ae^t = 4e^t$$

Thus $A = \frac{2}{3}$ and the particular solution is

$$Y(t) = \frac{2}{3}t^3e^t \tag{3.244}$$

The general solution of Eq. (3.242) is the sum of $y_c(t)$ from Eq. (3.243) and $Y(t)$ from Eq. (3.244), that is

$$y = c_1e^t + c_2te^t + c_3t^2e^t + \frac{2}{3}t^3e^t$$

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Nonhomogeneous Higher Order Linear Differential
Equations General Solution with Method of Undetermined Coefficients ]# ./main

General solution for y''' - 3y'' + 3y' - y = 4 exp(t)


P:
(1,0)
(-3,0)
(3,0)
(-1,0)

P':
(3,0)
(-6,0)
(3,0)

Initial guess for the roots (generated randomly):
(3.34839,0)
(3.12548,0)
(3.94203,0)

Roots found at iteration: 25

*************************************************************************

End of iteration

z_{i} final:
(1,0)
(1,0)
(1,0)

Deleted Duplicate vector:
(1,0)

Final Duplicate vector:
(1,0)

The general solution of the homogeneous equation is:
y(t) = e^t*c[0]+e^t*c[1]*t+e^t*c[2]*t^(2)

The general solution of the nonhomogeneous equation is:
Y(t) = 0.666667*e^t*t^(3)

The general solution of the differential equation is:
Y(t) = e^t*c[0]+e^t*c[1]*t+e^t*c[2]*t^(2)+0.666667*e^t*t^(3)

Time taken by function: 380497 microseconds
```

Figure 3.53: *The computation of the general solution for the Eq. (3.242) (**SymIntegration/Examples/Differential Equations/Test SymIntegration Nonhomogeneous Higher Order Linear Differential Equations General Solution with Method of Undetermined Coefficients/main.cpp**).*

[SI*] **Example:**
Find the particular solution of the equation

$$y^{(4)} + 2y'' + y = 3\sin\ t - 5\cos\ t \tag{3.245}$$

Solution:
The general soltion of the homogeneous equation is

$$y_c(t) = c_1 \cos\ t + c_2 \sin\ t + c_3 t \cos\ t + c_4 t \sin\ t$$

corresponding to the roots $r = i, i, -i$, and $-i$ of the characteristic equation. Our initial assumption for a particular solution is $Y(t) = A \sin\ t + B \cos\ t$, but we must multiply this choice by t^2 to make it different from all solutions of the homogeneous equation. Thus our final asusmption is

$$Y(t) = At^2 \sin\ t + Bt^2 \cos\ t$$

Next, we differentiate $Y(t)$ four times, substitute into the differential equation (3.245), and collect terms, obtaining finally

$$-8A \sin\ t - 8B \cos\ t = 3 \sin\ t - 5 \cos\ t$$

Thus $A = -\frac{3}{8}, B = \frac{5}{8}$, and the particular solution of Eq. (3.245) is

$$Y(t) = -\frac{3}{8}t^2 \sin\ t + \frac{5}{8}t^2 \cos\ t \tag{3.246}$$

```
End of iteration

z_{i} final:
(0,1)
(0,-1)
(0,1)
(0,-1)

Deleted Duplicate vector:
(0,-1)
(0,1)

Final root vector:
(0,1)
n_occurence = 2

The general solution of the homogeneous equation is:
y(t) = c[0]*cos(t)+c[1]*sin(t)+c[2]*cos(t)*t+c[3]*sin(t)*t

************************************************************

For rhs = 3*sin(t)

Y(t) particular = -8*A*cos(t)-8*A*sin(t)

Y(t) assumed = cos(t)*t^(2)+sin(t)*t^(2)

The general solution of this nonhomogeneous equation is:
Y_{i}(t) = -0.375*t^(2)*sin(t)

************************************************************

For rhs = -5*cos(t)

Y(t) particular = -8*A*cos(t)-8*A*sin(t)

Y(t) assumed = cos(t)*t^(2)+sin(t)*t^(2)

The general solution of this nonhomogeneous equation is:
Y_{i}(t) = 0.625*t^(2)*cos(t)

The general solution of the differential equation is:
Y(t) = c[0]*cos(t)+c[1]*sin(t)+c[2]*cos(t)*t+c[3]*sin(t)*t-0.375*t^(2)*sin(t)+0.625*t^(2)*cos(t)

Time taken by function: 788510 microseconds
```

Figure 3.54: *The computation of the general solution for the Eq. (3.245) (**SymIntegration/Examples/Differential Equations/Test SymIntegration Nonhomogeneous Higher Order Linear Differential Equations General Solution with Method of Undetermined Coefficients/main.cpp**).*

[SI*] The method of undetermined coefficients can be used whenever it is possible to guess the correct form for $Y(t)$. However, this is usually impossible for differential equations not having constant coefficients, or for nonhomogeneous terms other than the type described previously.

[SI*] **The Method of Variation of Parameters**

The method of variation of parameters for determining a particular solution of the nonhomogeneous nth order linear differential equation

$$L[y] = y^{(n)} = y^{(n)} + p_1(t)y^{(n-1)} + \cdots + p_{n-1}(t)y' + p_n(t)y = g(t) \tag{3.247}$$

is a direct extension of the method for the second order differential equation. As before, to use the method of variation of parameters, it is first necessary to solve the corresponding homogeneous differential equation. In general, this may be difficult unless the coefficients are constants. However, the method of variation of parameters is still more general than the method of undetermined coefficients in that it leads to an expression for the particular solution for any continuous function g, whereas the method of undetermined coefficients is restricted in practice to a limited class of functions g.

Suppose then that we know a fundamental set of solutions $y_1, y_2, \ldots, y_n$ of the homogeneous equation. Then the general solution of the homogeneous equation is

$$y_c(t) = c_1y_1(t) + c_2y_2(t) + \cdots + c_ny_n(t) \tag{3.248}$$

The method of variation of parameters for determining a particular solution of Eq. (3.247) rests on the possibility of determining n functions $u_1, u_2, \ldots, u_n$, such that $Y(t)$ is of the form

$$Y(t) = u_1(t)y_1(t) + u_2(t)y_2(t) + \cdots + u_n(t)y_n(t) \tag{3.249}$$

Since we have n functions to determine, we will have to specify n conditions. One of these is clearly that Y satisfy Eq. (3.247). The other $n-1$ conditions are chosen so as to make the calculations as simple as possible. Since we hardly expect a simplification in determining Y if we must solve high order differential equations for $u_1, \ldots, u_n$ it is natural to impose conditions to suppress the terms that lead to higher derivatives of $u_1, \ldots, u_n$. From Eq. (3.249) we obtain

$$Y' = (u_1y_1' + u_2y_2' + \cdots + u_ny_n') + (u_1'y_1 + u_2'y_2 + \cdots + u_n'y_n) \tag{3.250}$$

where we have omitted the independent variable t on which each function in Eq. (3.250) depends. Thus the first condition that we impose is that

$$u_1'y_1 + u_2'y_2 + \cdots + u_n'y_n = 0 \tag{3.251}$$

Continuing this process in a similar manner through $n-1$ derivatives of Y gives

$$Y^{(m)} = u_1y_1^{(m)} + u_2y_2^{(m)} + \cdots + u_ny_n^{(m)}, \quad m = 0, 1, 2, \ldots, n-1 \tag{3.252}$$

and the following $n-1$ conditions on the functions $u_1, \ldots, u_n$:

$$u_1'y_1^{(m-1)} + u_2'y_2^{(m-1)} + \cdots + u_n'y_n^{(m-1)}, \quad m = 1, 2, \ldots, n-1 \tag{3.253}$$

The nth derivative of Y is

$$Y^{(n)} = (u_1y_1^{(n)} + \cdots + u_ny_n^{(n)}) + (u_1'y_1^{(n-1)} + \cdots + u_n'y_n^{(n-1)}) \tag{3.254}$$

Finally, we impose the condition that Y must be a solution of Eq. (3.247). On substituting for the derivatives of Y from Eqs. (3.252) and (3.254), collecting terms, and making use of the fact that $L[y_i] = 0$ $i = 1, 2, \ldots, n$, we obtain

$$u_1'y_1^{(n-1)} + u_2'y_2^{(n-1)} + \cdots + u_n'y_n^{(n-1)} = g \tag{3.255}$$

Equation (3.255), coupled with the $n-1$ equations (3.253), gives n simultaneous linear nonhomogeneous algebraic equations for $u_1', u_2', \ldots, u_n'$:

$$\begin{aligned} y_1u_1' + y_2u_2' + \cdots + y_nu_n' &= 0 \\ y_1'u_1' + y_2'u_2' + \cdots + y_n'u_n' &= 0 \\ y_1''u_1' + y_2''u_2' + \cdots + y_n''u_n' &= 0 \\ &\vdots \\ y_1^{(n-1)}u_1' + y_2^{(n-1)}u_2' + \cdots + y_n^{(n-1)}u_n' &= 0 \end{aligned} \tag{3.256}$$

The system (3.256) is a linear algebraic system for the unknown quantities $u_1', \ldots, u_n'$. By solving this system and then integrating the resulting expressions, you can obtain the coefficients $u_1, \ldots, u_n$. A sufficient condition for the existence of a solution of the system of equations (3.256) is that the determinant of coefficients is nonzero for each value of t. However, the determinant of coefficients is precisely $W(y_1, y_2, \ldots, y_n)$, and it is nowhere zero since $y_1, \ldots, y_n$ is a fundamental set of solutions of the homogeneous equation. Hence it is possible to determine $u_1', \ldots, u_n'$. Using Cramer's rule, we can write the solution of the system of equations (3.256) in the form

$$u_m'(t) = \frac{g(t)W_m(t)}{W(t)}, \quad m = 1, 2, \ldots, n \tag{3.257}$$

Here $W(t) = W(y_1, y_2, \ldots, y_n)(t)$, and W_m is the determinant obtained from W by replacing the mth column by the column $(0, 0, \ldots, 0, 1)$. With this notation a particular solution of Eq. (3.247) is given by

$$Y(t) = \sum_{m=1}^{n} y_m(t) \int_{t_0}^{t} \frac{g(s)W_m(s)}{W(s)} ds \tag{3.258}$$

where t_0 is arbitrary. Although the procedure is straightforward, the algebraic computations involved in determining $Y(t)$ from Eq. (3.258) become more and more complicated as n increases. In some cases the calculations may be simplified to some extent by using Abel's identity

$$W(t) = W(y_1, \ldots, y_n)(t) = ce^{[-\int p_1(t)\ dt]}$$

The constant c can be determined by evaluating W at some convenient point.

[SI*] **Example:**
Given that $y_1(t) = e^t, y_2(t) = te^t$, and $y_3(t) = e^{-t}$ are solutions of the homogeneous equation corresponding to

$$y''' - y'' - y' + y = \sin(t) \tag{3.259}$$

determine a particular solution of Eq. (3.259) in terms of an integral.

Solution:
We use Eq. (3.258). First, we have

$$W(t) = W(e^t, te^t, e^{-t})(t) = \begin{vmatrix} e^t & te^t & e^{-t} \\ e^t & (t+1)e^t & -e^{-t} \\ e^t & (t+2)e^t & e^{-t} \end{vmatrix} = 4e^t$$

Next, we can use minors associated with the ith column for each $W_i(t)$

$$W_1(t) = \begin{vmatrix} 0 & te^t & e^{-t} \\ 0 & (t+1)e^t & -e^{-t} \\ 1 & (t+2)e^t & e^{-t} \end{vmatrix} = -2t - 1$$

$$W_2(t) = \begin{vmatrix} e^t & 0 & e^{-t} \\ e^t & 0 & -e^{-t} \\ e^t & 1 & e^{-t} \end{vmatrix} = 2$$

$$W_3(t) = \begin{vmatrix} e^t & te^t & 0 \\ e^t & (t+1)e^t & 0 \\ e^t & (t+2)e^t & 1 \end{vmatrix} = e^{2t}$$

Substituting these results in Eq. (3.258), we have

$$\begin{aligned} Y(t) &= e^t \int_{t_0}^{t} \frac{\sin(s)(-1-2s)}{4e^s} ds + te^t \int_{t_0}^{t} \frac{\sin(s)(2)}{4e^s} ds + e^{-t} \int_{t_0}^{t} \frac{\sin(s)(e^{2s})}{4e^s} ds \\ &= \frac{1}{4} \int_{t_0}^{t} \{e^{t-s}[-1+2(t-s)] + e^{-(t-s)}\} \sin(s)\, ds \end{aligned}$$

```
General solution for y''' - y'' - y' + y = sin(t)


P:
(1,0)
(-1,0)
(-1,0)
(1,0)

P':
(3,0)
(-2,0)
(-1,0)

Initial guess for the roots (generated randomly):
(5.492,0)
(4.59702,0)
(4.358,0)

Roots found at iteration: 22

************************************************************************

End of iteration

z_{i} final:
(1,0)
(-1,0)
(1,0)

Sorted Duplicate vector:
(1,0)
(1,0)

Final root vector:
(-1,0)
(1,0)

The general solution of the homogeneous equation is:
y(t) = e^(-t)*c[0]+e^t*c[1]+e^t*c[2]*t
```

Figure 3.55: *The computation of the particular solution for the Eq. (3.259) (**SymIntegration/Examples/Differential Equations/Test SymIntegration Nonhomogeneous Higher Order Linear Differential Equations General Solution with Variation of Parameters/main.cpp**).*

```
The general solution of the homogeneous equation is:
y(t) = e^(-t)*c[0]+e^t*c[1]+e^t*c[2]*t

W(t):
[   e^(-t)       e^t        e^t*t   ]
[  -e^(-t)       e^t      e^t*t+e^t ]
[   e^(-t)       e^t     e^t*t+2*e^t]


det (W(t)) = 4*e^t

W_{i}(t):
[     0          e^t        e^t*t   ]
[     0          e^t      e^t*t+e^t ]
[     1          e^t     e^t*t+2*e^t]


det (W_{i}(t)):
e^(2*t)

W_{i}(t):
[   e^(-t)        0         e^t*t   ]
[  -e^(-t)        0       e^t*t+e^t ]
[   e^(-t)        1      e^t*t+2*e^t]


det (W_{i}(t)):
-2*t-1

W_{i}(t):
[  e^(-t)   e^t      0   ]
[ -e^(-t)   e^t      0   ]
[  e^(-t)   e^t      1   ]


det (W_{i}(t)):
2

The particular solution of the differential equation is:
Y(t) = e^(-t)*(int(0.25*e^t*sin(t),t))+e^t*(int(-0.5*t*sin(t)*e^(-t),t))+e^t*(int(-0.25*sin(t)*e^(-t),t))+e^t*t*(int(0.5*sin(t)*e^(-t),t))

Time taken by function: 4659706 microseconds
```

Figure 3.56: *The computation of the particular solution for the Eq. (3.259) (**SymIntegration/Examples/Differential Equations/Test SymIntegration Nonhomogeneous Higher Order Linear Differential Equations General Solution with Variation of Parameters/main.cpp**).*

i. Functions in SymIntegration Related to Higher Order Linear Equations

[SI*] We will list the basic functions that can be used for solving higher order linear differential equations:

AberthEhrlich(const vector<complex<double>> &$\vec{P}$, const vector<complex<double>> &x_0, int N)
to compute the root of higher order polynomial P with Aberth-Ehrlich formula, P is a vector of size n with complex entry, the configuration is like this:

$$\vec{P} = \begin{bmatrix} (a_1, b_1) \\ (a_2, b_2) \\ \vdots \\ (a_n, b_n) \end{bmatrix}$$

$$\text{polynomial}(\vec{P}) = (a_1, b_1)r^{n-1} + (a_2, b_2)r^{n-2} + \cdots + (a_n, b_n)$$

so if we want to compute a polynomial of order 6, then $\vec{P}$ has to be a size of 7. N is the number of maximum iteration for the Abert-Ehrlich method, and x_0 is the initial guess / approximation vector with complex entry and has to be a size of $n-1$, the entry of x_0 is preferable to be distinct and in decimal (e.g. 1.23 or 2.55), not round number to avoid error due to rounding.

This function is written in **src/numericalmethods.cpp** since it is a numerical method, but very useful in Differential Equations field and application thus we write it here.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Aberth-Ehrlich Method to Compute Roots of Polynomial ]# ./main
Poynomial : r^(4)+r^(3)-7*r^(2)-r+6
P:
(1,0)
(1,0)
(-7,0)
(-1,0)
(6,0)

P':
(4,0)
(3,0)
(-14,0)
(-1,0)

Initial guess:
(1,0)
(2,0)
(3,0)
(4,0)

iteration: 0

z_{i} new:
(1,0)
(2,0)
(2.29412,0)
(8.10931,0)

iteration: 1

z_{i} new:
(1,0)
(2,0)
(0.789054,0)
(-7.71719,0)

iteration: 2

z_{i} new:
(1,0)
(2,0)
(-0.628758,0)
(-3.16734,-1.77748e-15)

iteration: 3

z_{i} new:
(1,0)
(2,1.49372e-47)
(-0.996208,-4.54674e-17)
(-3.00002,-5.32888e-16)
```

Figure 3.57: *The root-finding computation with Aberth-Ehrlich method (**SymIntegration/Examples/Numerical Methods/Test SymIntegration Aberth-Ehrlich Method to Compute Roots of Polynomial/-main.cpp**).*

```
iteration: 4

z_{i} new:
(1,0)
(2,1.47984e-48)
(-1,7.65354e-17)
(-3,-1.65491e-16)

iteration: 5

z_{i} new:
(1,0)
(2,-4.2281e-49)
(-1,-4.59293e-17)
(-3,2.01903e-16)

iteration: 6

z_{i} new:
(1,0)
(2,-1.66768e-49)
(-1,7.65354e-17)
(-3,-1.65491e-16)

Roots found at iteration: 6

**********************************************************************

End of iteration

z_{i} final:
(1,0)
(2,0)
(-1,0)
(-3,0)

Time taken by function: 9088 microseconds
```

Figure 3.58: *The root-finding computation with Aberth-Ehrlich method (**SymIntegration/Examples/Numerical Methods/Test SymIntegration Aberth-Ehrlich Method to Compute Roots of Polynomial/-main.cpp**).*

higherorderlineardiffeq_homogeneousequationsivpsolution(const vector<complex<double>> &$\vec{P}$, const vector<complex<double>> &$\vec{ic}$, const vector<complex<double>> &$\vec{x}_0$, int N)
this function computes the general solution and the initial value solution for higher order homogeneous linear differential equation (the order of the differential equation is nth order)

$$(a_1, b_1)y^{(n)} + (a_2, b_2)y^{(n-1)} + \cdots + (a_n, b_n)y' + (a_{n+1}, b_{n+1})y = 0$$

the input $\vec{P}$ is the coefficients vector of the characteristic equation representing the differential equation

$$\vec{P} = \begin{bmatrix} (a_1, b_1) \\ (a_2, b_2) \\ \vdots \\ (a_n, b_n) \\ (a_{n+1}, b_{n+1}) \end{bmatrix}$$

it is a vector with size of $n + 1$, as we can see the characteristic equation from $\vec{P}$ is

$$\text{polynomial}(\vec{P}) = (a_1, b_1)r^{n-1} + (a_2, b_2)r^{n-2} + \cdots + (a_n, b_n)r + (a_{n+1}, b_{n+1})$$

usually we will deal with the condition where $b_1, b_2, \ldots, b_n, b_{n+1} = 0$ (the coefficients are real number). The input has to be in complex form because there is a possibility to obtain complex roots.
Then we will have another input $\vec{ic}$ that represents the initial condition with this configura-

tion:

$$\begin{aligned} y(0) &= ic_1 \\ y'(0) &= ic_2 \\ &\vdots \\ y^{(n-1)}(0) &= ic_n \end{aligned}$$

thus

$$\vec{ic} = \begin{bmatrix} ic_1 \\ ic_2 \\ \vdots \\ ic_n \end{bmatrix}$$

it is a vector with size of n, and the input of $\vec{ic}$ is also in complex form. The last two parameters are N, the number of maximum iteration for the Abert-Ehrlich method in order to compute the roots of the characteristic equation, and $\vec{x}_0$ is the initial guess / approximation vector with complex entry, it has to be a size of n, the entry of $\vec{x}_0$ is preferable to be distinct and in decimal (e.g. 1.23 or 2.55), not round number to avoid error due to rounding. The last two parameters can be omitted and automated in the main source code, with Mersenne Twister engine, we can generate a good $\vec{x}_0$ that can approximate the roots.

higherorderlineardiffeq_homogeneousequationsivpsolution(const vector<complex<double>> &$\vec{P}$, const vector<complex<double>> &$\vec{ic}$)
this function produces the the same output with the previous function but with less input, this function computes the general solution and the initial value solution for higher order homogeneous linear differential equation (the order of the differential equation is nth order)

$$(a_1, b_1)y^{(n)} + (a_2, b_2)y^{(n-1)} + \cdots + (a_n, b_n)y' + (a_{n+1}, b_{n+1})y = 0$$

the input $\vec{P}$ is the coefficients vector of the characteristic equation representing the differential equation

$$\vec{P} = \begin{bmatrix} (a_1, b_1) \\ (a_2, b_2) \\ \vdots \\ (a_n, b_n) \\ (a_{n+1}, b_{n+1}) \end{bmatrix}$$

it is a vector with size of $n + 1$, as we can see the characteristic equation from $\vec{P}$ is

$$\text{polynomial}(\vec{P}) = (a_1, b_1)r^{n-1} + (a_2, b_2)r^{n-2} + \cdots + (a_n, b_n)r + (a_{n+1}, b_{n+1})$$

usually we will deal with the condition where $b_1, b_2, \ldots, b_n, b_{n+1} = 0$ (the coefficients are real number). The input has to be in complex form because there is a possibility to obtain complex roots.
Then we will have another input $\vec{ic}$ that represents the initial condition with this configura-

tion:

$$\begin{aligned} y(0) &= ic_1 \\ y'(0) &= ic_2 \\ &\vdots \\ y^{(n-1)}(0) &= ic_n \end{aligned}$$

thus

$$\vec{ic} = \begin{bmatrix} ic_1 \\ ic_2 \\ \vdots \\ ic_n \end{bmatrix}$$

it is a vector with size of n, and the input of $\vec{ic}$ is also in complex form. This function omitted $\vec{x}_0$ (we generate it with Mersenne Twister that is normally distributed with $\mu = 5$ and $\sigma = 1.2$), and it also set $N = 100$ in the source code **src/dsolve.cpp**.

```
P:
(1,0)
(-3,0)
(2,0)
(-6,0)
(1,0)
(-3,0)

P':
(5,0)
(-12,0)
(6,0)
(-12,0)
(1,0)

Initial guess:
(1.1,0)
(2,0)
(3,0)
(4,0)
(5.1,0)

********************************************************************************

End of iteration

z_{i} final:
(0,1)
(0,-1)
(3,0)
(0,-1)
(0,1)

Unique vector:
(3,0)

Duplicate vector:
(0,1)
(0,-1)
(0,-1)
(0,1)
```

Figure 3.59: *The computation for the initial value problem of* $y^{(5)} - 3y^{(4)} + 2y^{(3)} - 6y'' + y' - 3$ *with initial condition* $y(0) = 1, y'(0) = 0, y''(0) = -2, y^{(3)}(0) = -1, y^{(4)}(0) = 3$ *(**SymIntegration/Examples/Numerical Methods/Test SymIntegration IVP Solution for Higher Order Linear Differential Equations/-main.cpp**).*

```
************************************************************************

End of iteration

z_{i} final:
(0,1)
(0,-1)
(3,0)
(0,-1)
(0,1)

Unique vector:
(3,0)

Duplicate vector:
(0,1)
(0,-1)
(0,-1)
(0,1)

Sorted Duplicate vector:
(0,-1)
(0,-1)
(0,1)
(0,1)

Deleted Duplicate vector:
(0,-1)
(0,1)

Final Duplicate vector:
(3,0)
(0,1)

The general solution is:
y(t) = e^(3*t)*c[0]+c[1]*cos(t)+c[2]*sin(t)+c[3]*cos(t)*t+c[4]*sin(t)*t
n_occurence = 2

The initial value problem solution is:
y(t) = 1*cos(t)-0.5*sin(t)+0.5*cos(t)*t-0.5*sin(t)*t

Time taken by function: 351427 microseconds
```

Figure 3.60: *The computation for the initial value problem of* $y^{(5)} - 3y^{(4)} + 2y^{(3)} - 6y'' + y' - 3$ *with initial condition* $y(0) = 1, y'(0) = 0, y''(0) = -2, y^{(3)}(0) = -1, y^{(4)}(0) = 3$ *(**SymIntegration/Examples/Numerical Methods/Test SymIntegration IVP Solution for Higher Order Linear Differential Equations/-main.cpp**).*

higherorderlineardiffeq_vectorize(const Symbolic &$F(y,t)$, const Symbolic &y, const Symbolic &t, int n)
this function takes the coefficient of the ordinary differential equation with order of n, with this form:

$$F(y,t) = a_0 y^{(n)} + a_1 y^{(n-1)} + \cdots + a_{n-1} y' + a_n y = 0$$

with y as the function of t, $y(t)$.
It will return this complex vector (as the final result):

$$P = \begin{bmatrix} (a_0, 0) \\ (a_1, 0) \\ \vdots \\ (a_{n-1}, 0) \\ (a_n, 0) \end{bmatrix}$$

```
The ODE with constant coefficients:
1/2*df(u1[t],t,t,t,t)+7/2*df(u1[t],t,t)+3*u1[t] = 0

The vector coefficient of homogeneous ODE with constant coefficients:
(1,0)
(0,0)
(7,0)
(0,0)
(6,0)

Time taken by function: 96071 microseconds
```

Figure 3.61: *The vectorization of an ODE (**SymIntegration/Examples/Numerical Methods/Test SymIntegration Vectorize ODE/main.cpp**).*

higherorderlineardiffeq_twospringtwomasssystem(double k_1, double k_2, double m_1, double m_2, Symbolic &t, const vector<complex<double>> &$\vec{ic}$)
this function computes the equation of motion for two-spring, two-mass system, and the initial value problem solution with given input of m_1 as the mass for the first box / object, m_2 as the mass for the second box / object, k_1 is the spring constant between the ceiling / wall with the first box, and k_2 is the spring constant between the first and second box. Lastly, t is the independent variable that represents time. The solution will be given as $u_1(t)$ and $u_2(t)$.
Then we will have another input $\vec{ic}$ that represents the initial condition with this configuration:

$$\begin{aligned} u_1(0) &= ic_1 \\ u_1'(0) &= ic_2 \\ u_2(0) &= ic_3 \\ u_2'(0) &= ic_4 \end{aligned}$$

thus

$$\vec{ic} = \begin{bmatrix} ic_1 \\ ic_2 \\ ic_3 \\ ic_4 \end{bmatrix}$$

it is a vector with size of $n = 4$, it is the order of the differential equation for the system after we solve the equations of motion, the input of $\vec{ic}$ is in complex form.

higherorderlineardiffeq_nonhomogeneousequationsgeneralsolution(const vector<complex<double>> &$\vec{P}$, const Symbolic &$g(t)$, const Symbolic &t)
this function computes the general solution of a nonhomogeneous higher order linear differential equation with this form

$$a_0 y^{(n)}(t) + a_1 y^{(n-1)}(t) + \cdots + a_{n-1} y'(t) + a_n y(t) = g(t)$$

with

$$\vec{P} = \begin{bmatrix} (a_1, b_1) \\ (a_2, b_2) \\ \vdots \\ (a_n, b_n) \\ (a_{n+1}, b_{n+1}) \end{bmatrix}$$

as a vector with size of $n+1$ that represents the differential equation of order n, $b_1, b_2, \ldots, b_{n+1}$ will have value of zero.

The function $g(t)$ is the right hand side function, it is a single function only, not a linear combination of functions.
Currently, it can handle several types of $g(t)$ as listed below:

$$\begin{aligned} g(t) &= ae^t \\ g(t) &= a\sin(t) \\ g(t) &= a\cos(t) \end{aligned}$$

with a as constant.
The code that handle or recognize certain $g(t)$ function can be made with **if** conditional, here is a snippet of what is already written

```
void higherorderlineardiffeq_nonhomogeneousequationsgeneralsolution(const
    vector<complex<double>> &P, const Symbolic &rhs_function, const Symbolic
    &t)
{
...
...
              Symbolic A_solve;
              Symbolic F = rhs_function;
              cout << "\nY(t) particular = "<< Yt_particular_total << endl;
              cout << "\nY(t) assumed = "<< Yt_assumed << endl;

       // Case 1 : g(t) = a*exp(t)
       if (df(F,t,0) == df(F,t,1) && df(F,t,1) == df(F,t,2) && df(F,t,0) ==
           df(F,t,2)) // meaning that the rhs_function is in the form of a*
           exp(t)
       {
              //double a = df(F,t,0)/exp(t);
              Symbolic lhs_nh = Yt_particular_total.coeff(exp(t),1);
              A_solve = solve(lhs_nh*exp(t)— rhs_function, A).front().rhs;
              Yt_final = A_solve*Yt_assumed ;
       }
       // Case 2 : g(t) = a*sin(t) or g(t) = a*cos(t)
       if (df(F,t,0) == —df(F,t,2) && df(F,t,0) != df(F,t,1) && df(F,t,0) ==
            df(F,t,4)) // meaning that the rhs_function is in the form of a*
           sin(t) or a* cos(t)
       {
       if (df(F/sin(t),t) == 0) // meaning that the rhs_function is in the
           form of a*sin(t)
       {
              double a = F/sin(t);
              Symbolic lhs_nh = Yt_particular_total.coeff(A*sin(t),1);
              A_solve = a/lhs_nh;
              Yt_final = A_solve*Yt_assumed.coeff(sin(t),1)*sin(t) ;
       }
       if (df(F/cos(t),t) == 0) // meaning that the rhs_function is in the
           form of a*cos(t)
       {
              double a = F/cos(t);
              Symbolic lhs_nh = Yt_particular_total.coeff(A*cos(t),1);
              A_solve = a/lhs_nh;
              Yt_final = A_solve*Yt_assumed.coeff(cos(t),1)*cos(t) ;
       }

       ...
       ...
```

```
}
```

Code 41: *src/dsolve.cpp*

higherorderlineardiffeq_nonhomogeneousequationsgeneralsolution(const vector<complex<double>> &$\vec{P}$, const SymbolicMatrix &A, const Symbolic &t)
this function computes the general solution of a nonhomogeneous higher order linear differential equation with this form

$$a_0 y^{(n)}(t) + a_1 y^{(n-1)}(t) + \cdots + a_{n-1} y'(t) + a_n y(t) = A$$

with

$$\vec{P} = \begin{bmatrix} (a_1, b_1) \\ (a_2, b_2) \\ \vdots \\ (a_n, b_n) \\ (a_{n+1}, b_{n+1}) \end{bmatrix}$$

as a vector with size of $n+1$ that represents the differential equation of order n, $b_1, b_2, \ldots, b_{n+1}$ will have value of zero.

The matrix A acts as the right hand side, a linear combination of functions that is represented as a matrix with 1 column, each row represent unique function, for example:

$$A = \begin{bmatrix} 3\sin\ t \\ -5\cos\ t \end{bmatrix} = 3\sin\ t - 5\cos\ t$$

higherorderlineardiffeq_nonhomogeneousequations_variationofparameters(const vector<complex<double>> &$\vec{P}$, const Symbolic &$g(t)$, const Symbolic &t)
this function use the method of variation of parameters to compute the particular solution of a nonhomogeneous higher order linear differential equation with this form

$$a_0 y^{(n)}(t) + a_1 y^{(n-1)}(t) + \cdots + a_{n-1} y'(t) + a_n y(t) = g(t)$$

with

$$\vec{P} = \begin{bmatrix} (a_1, b_1) \\ (a_2, b_2) \\ \vdots \\ (a_n, b_n) \\ (a_{n+1}, b_{n+1}) \end{bmatrix}$$

as a vector with size of $n+1$ that represents the differential equation of order n, $b_1, b_2, \ldots, b_{n+1}$ will have value of zero.

IV. Series Solutions of Second Order Linear Equations

[SI*] **Power Series**

Finding the general solution of a linear differential equation depends on determining a fundamental set of solutions of the homogeneous equation. We will discuss the use of power series to construct fundamental sets of solutions of second order linear differential equations whose coefficients are functions of the independent variable.

1. A power series $\sum_{n=0}^{\infty} a_n(x-x_0)^n$ is said to converge at a point x if
$$\lim_{m\to\infty} \sum_{n=0}^{m} a_n(x-x_0)^n$$
exists for that x. The series certainly converges for $x = x_0$; it may converge for all x, or it may converge for some values of x and not for others.
2. The series $\sum_{n=0}^{\infty} a_n(x-x_0)^n$ is said to converge absolutely at a point x if the series
$$\sum_{n=0}^{\infty} |a_n(x-x_0)|^n = \sum_{n=0}^{\infty} |a_n||x-x_0|^n$$
converges. It can be shown that if the series converges absolutely, then the series also converges; however, the converse is not necessarily true.
3. One of the most useful tests for the absolute convergence of a power series is the ratio test. If $a_n \neq 0$, and if, for a fixed value of x,
$$\lim_{n\to\infty} \left| \frac{a_{n+1}(x-x_0)^{n+1}}{a_n(x-x_0)} \right| = |x-x_0| \lim_{n\to} \left| \frac{a_{n+1}}{a_n} \right| = |x-x_0|L$$
then the power series converges absolutely at that value of x if $|x-x_0|L < 1$ and diverges if $|x-x_0|L > 1$. If $|x-x_0|L = 1$, the test is inconclusive.
4. If the power series $\sum_{n=0}^{\infty} a_n(x-x_0)^n$ converges at $x = x_1$, it converges absolutely for $|x-x_0| < |x_1-x_0|$; and if it diverges at $x = x_1$, it diverges for $|x-x_0| > |x_1-x_0|$.
5. There is a nonnegative number ρ, called the radius of convergence, such that $\sum_{n=0}^{\infty} a_n(x-x_0)^n$ converges absolutely for $|x-x_0| < \rho$ and diverges for $|x-x_0| > \rho$. For a series that converges only at x_0, we define ρ to be zero; for a series that converges for all x; we say that ρ is infinite. If $\rho > 0$, then the interval $|x-x_0| < \rho$ is called the interval of convergence. The series may either converge or diverge when $|x-x_0| = \rho$.
6. Suppose that $\sum_{n=0}^{\infty} a_n(x-x_0)^n$ and $\sum_{n=0}^{\infty} b_n(x-x_0)^n$ converge to $f(x)$ and $g(x)$, respectively, for $|x-x_0| < \rho, \rho > 0$.
The series can be added or subtracted termwise, and
$$f(x) \pm g(x) = \sum_{n=0}^{\infty} (a_n \pm b_n)(x-x_0)^n$$
the resulting series converges at least for $|x-x_0| < \rho$.
7. Suppose that $\sum_{n=0}^{\infty} a_n(x-x_0)^n$ and $\sum_{n=0}^{\infty} b_n(x-x_0)^n$ converge to $f(x)$ and $g(x)$, respectively, for $|x-x_0| < \rho, \rho > 0$.
The series can be formally multiplied, and
$$f(x)g(x) = \left[\sum_{n=0}^{\infty} a_n(x-x_0)^n\right] \left[\sum_{n=0}^{\infty} b_n(x-x_0)^n\right] = \sum_{n=0}^{\infty} c_n(x-x_0)^n$$

where $c_n = a_0b_n + a_1b_{n-1} + \cdots + a_nb_0$. The resulting series converges at least for $|x - x_0| < \rho$.
Further, if $g(x) \neq 0$, the series can be formally divided, and

$$\frac{f(x)}{g(x)} = \sum_{n=0}^{\infty} d_n(x - x_0)^n$$

In most cases the coefficients d_n can be most easily obtained by equating coefficients in the equivalent relation

$$\begin{aligned}\sum_{n=0}^{\infty} a_n(x - x_0)^n &= \left[\sum_{n=0}^{\infty} d_n(x - x_0)^n\right]\left[\sum_{n=0}^{\infty} b_n(x - x_0)^n\right] \\ &= \sum_{n=0}^{\infty}\left(\sum_{k=0}^{n} d_k b_{n-k}\right)(x - x_0)^n\end{aligned}$$

In the case of division the radius of convergence of the resulting power series may be less than ρ.

8. The function f is continuous and has derivatives of all orders for $|x - x_0| < \rho$. Further, $f', f'', \ldots$ can be computed by differentiating the series termwise; that is,

$$\begin{aligned}f'(x) &= a_1 + 2a_2(x - x_0) + \cdots + na_n(x - x_0)^{n-1} + \ldots \\ &= \sum_{n=1}^{\infty} na_n(x - x_0)^{n-1} \\ f''(x) &= 2a_2 + 6a_3(x - x_0) + \cdots + n(n-1)a_n(x - x_0)^{n-2} + \ldots \\ &= \sum_{n=2}^{\infty} n(n-1)a_n(x - x_0)^{n-2}\end{aligned}$$

and so forth, and each of the series converges absolutely for $x - x_0 < \rho$.

9. The value of a_n is given by

$$a_n = \frac{f^{(n)}(x_0)}{n!}$$

The series is called the Taylor' series for the function f about $x = x_0$.

10. If $\sum_{n=0}^{\infty} a_n(x - x_0)^n = \sum_{n=0}^{\infty} b_n(x - x_0)^n$ for each x in some open interval with center x_0, then $a_n = b_n$ for $n = 0, 1, 2, 3, \ldots$. In particular, if $\sum_{n=0}^{\infty} a_n(x - x_0)^n = 0$ for each such x, then $a_0 = a_1 = \cdots = a_n = \cdots = 0$.

[SI*] A function f that has a Taylor series expansion about $x = x_0$

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(x_0)}{n!}(x - x_0)^n$$

with a radius of convergence $\rho > 0$, is said to be analytic at $x = x_0$. All of the familiar functions of calculus are analytic except perhaps at certain easily recognized points. For example, sin x and e^x are analytic everywhere except at $x = 0$, and tan x is analytic except at odd multiples of $\frac{\pi}{2}$.
In many respects the natural context for the use of power series is the complex plane.

[SI*] **Example:**
Determine the radius of convergence of the given power series

(a)
$$\sum_{n=0}^{\infty}(x-3)^n$$

(b)
$$\sum_{n=0}^{\infty}\frac{n}{2^n}x^n$$

(c)
$$\sum_{n=0}^{\infty}\frac{x^{2n}}{n!}$$

(d)
$$\sum_{n=0}^{\infty}2^n x^n$$

(e)
$$\sum_{n=1}^{\infty}\frac{(2x+1)^n}{n^2}$$

(f)
$$\sum_{n=1}^{\infty}\frac{(x-x_0)^n}{n^2}$$

(g)
$$\sum_{n=1}^{\infty}\frac{(-1)^n n^2(x+2)^n}{4^n}$$

(h)
$$\sum_{n=1}^{\infty}\frac{n!x^n}{n^n}$$

Solution:

(a) Apply the ratio test to determine the condition in which the provided series converges.

$$\begin{aligned}\lim_{n\to\infty}\left|\frac{a_{n+1}}{a_n}\right| &= \lim_{n\to\infty}\left|\frac{(x-3)^{n+1}}{(x-3)^n}\right| \\ &= \lim_{n\to\infty}|x-3| \\ &= |x-3|\end{aligned}$$

According to this test, the series is

$$\begin{cases}\text{convergent if } |x-3|<1 \\ \text{unknown if } |x-3|=1 \\ \text{divergent if } |x-3|>1\end{cases}$$

From the condition of convergence, which can also be written as $-1 < x-3 < 1$, or $2 < x < 4$, we see that the center of convergence is at $x = 3$ and the radius of convergence is 1.

(b) Apply the ratio test to determine the condition in which the provided series converges.

$$\begin{aligned}\lim_{n\to\infty}\left|\frac{a_{n+1}}{a_n}\right| &= \lim_{n\to\infty}\left|\frac{\frac{n+1}{2^{n+1}}x^{n+1}}{\frac{n}{2^n}x^n}\right| \\ &= \lim_{n\to\infty}\left|\frac{1}{2}\frac{n+1}{n}x\right| \\ &= \lim_{n\to\infty}\frac{1}{2}\frac{n+1}{n}|x| \\ &= \frac{1}{2}\lim_{n\to\infty}\left(1+\frac{1}{n}\right)|x| \\ &= \frac{1}{2}|x|\end{aligned}$$

According to this test, the series is

$$\begin{cases}\text{convergent if } \frac{1}{2}|x|<1 \\ \text{unknown if } \frac{1}{2}|x|=1 \\ \text{divergent if } \frac{1}{2}|x|>1\end{cases}$$

From the condition of convergence, which can also be written as $|x|<2$, or $-2<x<2$, we see that the center of convergence is at $x=0$ and the radius of convergence is 2.

(c) Apply the ratio test to determine the condition in which the provided series converges.

$$\begin{aligned}\lim_{n\to\infty}\left|\frac{a_{n+1}}{a_n}\right| &= \lim_{n\to\infty}\left|\frac{\frac{x^{2(n+1)}}{(n+1)!}}{\frac{x^{2n}}{n!}}\right| \\ &= \lim_{n\to\infty}\left|\frac{n!}{(n+1)!}\frac{x^{2n+2}}{x^{2n}}\right| \\ &= \lim_{n\to\infty}\left|\frac{1}{n+1}x^2\right| \\ &= \lim_{n\to\infty}\frac{1}{n+1}x^2 \\ &= 0x^2 \\ &= 0\end{aligned}$$

According to this test, the series is

$$\begin{cases}\text{convergent if } 0x^2<1 \\ \text{unknown if } 0x^2=1 \\ \text{divergent if } 0x^2>1\end{cases}$$

From the condition of convergence , which can also be written as $|x|<\sqrt{\frac{1}{0}}=\infty$, or $-\infty<x<\infty$, we see that the center of convergence is at $x=0$ and the radius of convergence is ∞.

(d) Apply the ratio test to determine the condition in which the provided series converges.

$$\begin{aligned}\lim_{n\to\infty}\left|\frac{a_{n+1}}{a_n}\right| &= \lim_{n\to\infty}\left|\frac{2^{n+1}x^{n+1}}{2^n x^n}\right| \\ &= \lim_{n\to\infty}|2x| \\ &= \lim_{n\to\infty}2\,|x| \\ &= 2|x|\end{aligned}$$

According to this test, the series is

$$\begin{cases}\text{convergent if } 2|x|<1\\ \text{unknown if } 2|x|=1\\ \text{divergent if } 2|x|>1\end{cases}$$

From the condition of convergence, which can also be written as $|x|<\frac{1}{2}$, or $-\frac{1}{2}<x<\frac{1}{2}$, we see that the center of convergence is at $x=0$ and the radius of convergence is $\frac{1}{2}$.

(e) Apply the ratio test to determine the condition in which the provided series converges.

$$\begin{aligned}\lim_{n\to\infty}\left|\frac{a_{n+1}}{a_n}\right| &= \lim_{n\to\infty}\left|\frac{\frac{(2x+1)^{n+1}}{(n+1)^2}}{\frac{(2x+1)^n}{n^2}}\right| \\ &= \lim_{n\to\infty}\left|\frac{n^2}{(n+1)^2}(2x+1)\right| \\ &= \lim_{n\to\infty}\left|\frac{n^2}{n^2\left(1+\frac{1}{n}\right)^2}(2x+1)\right| \\ &= \lim_{n\to\infty}\left|\frac{1}{\left(1+\frac{1}{n}\right)^2}(2x+1)\right| \\ &= \lim_{n\to\infty}\frac{1}{\left(1+\frac{1}{n}\right)^2}|(2x+1)| \\ &= |2x+1|\end{aligned}$$

According to this test, the series is

$$\begin{cases}\text{convergent if } |2x+1|<1\\ \text{unknown if } |2x+1|=1\\ \text{divergent if } |2x+1|>1\end{cases}$$

From the condition of convergence, which can also be written as $-1<2x+1<1$, or $-1<x<0$, we see that the center of convergence is at $x=-\frac{1}{2}$ and the radius of convergence is $\frac{1}{2}$.

(f) Apply the ratio test to determine the condition in which the provided series converges.

$$\begin{aligned}
\lim_{n\to\infty}\left|\frac{a_{n+1}}{a_n}\right| &= \lim_{n\to\infty}\left|\frac{\frac{(x-x_0)^{n+1}}{(n+1)^2}}{\frac{(x-x_0)^n}{n^2}}\right| \\
&= \lim_{n\to\infty}\left|\frac{n^2}{(n+1)^2}(x-x_0)\right| \\
&= \lim_{n\to\infty}\left|\frac{n^2}{n^2\left(1+\frac{1}{n}\right)^2}(x-x_0)\right| \\
&= \lim_{n\to\infty}\left|\frac{1}{\left(1+\frac{1}{n}\right)^2}(x-x_0)\right| \\
&= \lim_{n\to\infty}\frac{1}{\left(1+\frac{1}{n}\right)^2}|x-x_0| \\
&= |x-x_0|
\end{aligned}$$

According to this test, the series is

$$\begin{cases}\text{convergent if } |x-x_0|<1\\ \text{unknown if } |x-x_0|=1\\ \text{divergent if } |x-x_0|>1\end{cases}$$

From the condition of convergence, which can also be written as $-1 < x - x_0 < 1$, or $-1 + x_0 < x < 1 + x_0$, we see that the center of convergence is at $x = x_0$ and the radius of convergence is 1.

(g) Apply the ratio test to determine the condition in which the provided series converges.

$$\begin{aligned}
\lim_{n\to\infty}\left|\frac{a_{n+1}}{a_n}\right| &= \lim_{n\to\infty}\left|\frac{\frac{(-1)^{n+1}(n+1)^2(x+2)^{n+1}}{4^{n+1}}}{\frac{(-1)^n n^2(x+2)^n}{4^n}}\right| \\
&= \lim_{n\to\infty}\left|\frac{(-1)^{n+1}}{(-1)^n}\frac{(n+1)^2}{n^2}\frac{4^n}{4^{n+1}}\frac{(x+2)^{n+1}}{(x+2)^n}\right| \\
&= \lim_{n\to\infty}\left|(-1)\frac{n^2\left(1+\frac{1}{n}\right)^2}{n^2}\frac{1}{4}(x+2)\right| \\
&= \lim_{n\to\infty}\left|(-1)\left(1+\frac{1}{n}\right)^2\frac{1}{4}(x+2)\right| \\
&= \lim_{n\to\infty}\frac{1}{4}\left(1+\frac{1}{n}\right)^2|x+2| \\
&= \frac{1}{4}|x+2|
\end{aligned}$$

According to this test, the series is

$$\begin{cases} \text{convergent if } \frac{1}{4}|x+2| < 1 \\ \text{unknown if } \frac{1}{4}|x+2| = 1 \\ \text{divergent if } \frac{1}{4}|x+2| > 1 \end{cases}$$

From the condition of convergence, which can also be written as $|x+2| < 4$, or $-6 < x < 2$, we see that the center of convergence is at $x = -2$ and the radius of convergence is 4.

(h) Apply the ratio test to determine the condition in which the provided series converges.

$$\begin{aligned}
\lim_{n\to\infty}\left|\frac{a_{n+1}}{a_n}\right| &= \lim_{n\to\infty}\left|\frac{\frac{(n+1)!x^{n+1}}{(n+1)^{n+1}}}{\frac{n!x^n}{n^n}}\right| \\
&= \lim_{n\to\infty}\left|\frac{(n+1)!}{n!}\frac{n^n}{(n+1)^{n+1}}\frac{x^{n+1}}{x^n}\right| \\
&= \lim_{n\to\infty}\left|(n+1)\frac{n^n}{n^{n+1}\left(1+\frac{1}{n}\right)^{n+1}}x\right| \\
&= \lim_{n\to\infty}\left|(n+1)\frac{1}{n\left(1+\frac{1}{n}\right)^{n+1}}x\right| \\
&= \lim_{n\to\infty}\frac{n+1}{n}\frac{1}{\left(1+\frac{1}{n}\right)^{n+1}}|x| \\
&= \left(\lim_{n\to\infty}\frac{n+1}{n}\right)\left[\lim_{n\to\infty}\frac{1}{\left(1+\frac{1}{n}\right)^{n+1}}\right]|x| \\
&= \left(\lim_{n\to\infty}\frac{1+\frac{1}{n}}{1}\right)\left(\frac{1}{e}\right)|x| \\
&= (1)\left(\frac{1}{e}\right)|x| \\
&= \frac{1}{e}|x|
\end{aligned}$$

According to this test, the series is

$$\begin{cases} \text{convergent if } \frac{1}{e}|x| < 1 \\ \text{unknown if } \frac{1}{e}|x| = 1 \\ \text{divergent if } \frac{1}{e}|x| > 1 \end{cases}$$

From the condition of convergence, which can also be written as $|x| < e$, or $-e < x < e$, we see that the center of convergence is at $x = 0$ and the radius of convergence is e.

[SI*] **Example:**
Determine the Taylor series about the point x_0 for the given function. Also determine the radius of convergence of the series.

(a) $\sin\ x,\ x_0 = 0$

(b) $e^x,\ x_0 = 0$

(c) $x,\ x_0 = 1$

(d) $x^2,\ x_0 = -1$

(e) $\ln(x),\ x_0 = 1$

(f) $\frac{1}{1-x},\ x_0 = 0$

(g) $\frac{1}{1+x},\ x_0 = 0$

(h) $\frac{1}{1-x},\ x_0 = 2$

Solution:

(a) The Taylor series expansion for a function $f(x)$ about the point $x = x_0$ is

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(x_0)}{n!}(x - x_0)^n$$

Since $x_0 = 0$, this formula reduces to

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(0)}{n!}x^n$$

The aim then is to determine the nth derivative of $\sin\ x$ evaluated at $x = 0$. Start taking derivatives and try to observe a pattern.

$$\begin{aligned}
n = 0: &\quad f^{(0)}(x) = \sin\ x &\rightarrow &\quad f^{(0)}(0) = 0 \\
n = 1: &\quad f^{(1)}(x) = \cos\ x &\rightarrow &\quad f^{(1)}(0) = 1 \\
n = 2: &\quad f^{(2)}(x) = -\sin\ x &\rightarrow &\quad f^{(2)}(0) = 0 \\
n = 3: &\quad f^{(3)}(x) = -\cos\ x &\rightarrow &\quad f^{(3)}(0) = -1 \\
n = 4: &\quad f^{(4)}(x) = \sin\ x &\rightarrow &\quad f^{(4)}(0) = 0 \\
n = 5: &\quad f^{(5)}(x) = \cos\ x &\rightarrow &\quad f^{(5)}(0) = 1 \\
&\ \ \vdots
\end{aligned}$$

Since $f^{(0)}(0) = 0$, the sum can be started from $n = 1$.

$$f(x) = \sin\ x = \sum_{n=1}^{\infty} \frac{f^{(n)}(0)}{n!}x^n$$

All the even valued of n result in $f^{(n)}(0)$ being zero, so sum over the odd integers only by making the substitution $n = 2k + 1$.

$$\sin\ x = \sum_{k=0}^{\infty} \frac{f^{(2k+1)}(0)}{(2k+1)!}x^{2k+1}$$

Based on the results above, $f^{(2k+1)}(0) = (-1)^k$. Therefore,

$$\sin\ x = \sum_{k=0}^{\infty} \frac{(-1)^k}{(2k+1)!}x^{2k+1}$$

Apply the ratio test to determine the condition in which the series converges.

$$\begin{aligned}
\lim_{k\to\infty}\left|\frac{a_{k+1}}{a_k}\right| &= \lim_{k\to\infty}\left|\frac{\frac{(-1)^{k+1}}{(2(k+1)+1)!}x^{2(k+1)+1}}{\frac{(-1)^k}{(2k+1)!}x^{2k+1}}\right| \\
&= \lim_{k\to\infty}\left|\frac{(-1)^{k+1}}{(-1)^k}\frac{(2k+1)!}{(2k+3)!}\frac{x^{2k+3}}{x^{2k+1}}\right| \\
&= \lim_{k\to\infty}\left|-\frac{1}{(2k+3)(2k+2)}x^2\right| \\
&= \lim_{k\to\infty}\frac{1}{(2k+3)(2k+2)}x^2 \\
&= 0x^2
\end{aligned}$$

According to this test, the series is

$$\begin{cases}\text{convergent if } 0x^2 < 1\\ \text{unknown if } 0x^2 = 1\\ \text{divergent if } 0x^2 > 1\end{cases}$$

From the condition of convergence, which can also be written as $|x| < \sqrt{\frac{1}{0}} = \infty$, or $-\infty < x < \infty$, we see that the center of convergence is at $x = 0$ and the radius of convergence is ∞.

(b) The Taylor series expansion for a function $f(x)$ about the point $x = x_0$ is

$$f(x) = \sum_{n=0}^{\infty}\frac{f^{(n)}(x_0)}{n!}(x - x_0)^n$$

Since $x_0 = 0$, this formula reduces to

$$f(x) = \sum_{n=0}^{\infty}\frac{f^{(n)}(0)}{n!}x^n$$

The aim then is to determine the nth derivative of e^x evaluated at $x = 0$. Start taking derivatives and try to observe a pattern.

$$\begin{aligned}
n = 0: &\quad f^{(0)}(x) = e^x \quad\to\quad f^{(0)}(0) = 1\\
n = 1: &\quad f^{(1)}(x) = e^x \quad\to\quad f^{(1)}(0) = 1\\
n = 2: &\quad f^{(2)}(x) = e^x \quad\to\quad f^{(2)}(0) = 1\\
n = 3: &\quad f^{(3)}(x) = e^x \quad\to\quad f^{(3)}(0) = 1\\
n = 4: &\quad f^{(4)}(x) = e^x \quad\to\quad f^{(4)}(0) = 1\\
n = 5: &\quad f^{(5)}(x) = e^x \quad\to\quad f^{(5)}(0) = 1\\
&\vdots
\end{aligned}$$

Notice that $f^{(n)}(0) = 1$ for all n. Therefore,

$$f(x) = e^x = \sum_{n=0}^{\infty}\frac{1}{n!}x^n$$

Apply the ratio test to determine the condition in which the series converges.

$$\begin{aligned}\lim_{n\to\infty}\left|\frac{a_{n+1}}{a_n}\right| &= \lim_{n\to\infty}\left|\frac{\frac{1}{(n+1)!}x^{n+1}}{\frac{1}{n!}x^n}\right| \\ &= \lim_{n\to\infty}\left|\frac{n!}{(n+1)!}\frac{x^{n+1}}{x^n}\right| \\ &= \lim_{n\to\infty}\left|\frac{1}{n}x\right| \\ &= \lim_{n\to\infty}\frac{1}{n}|x| \\ &= 0|x|\end{aligned}$$

According to this test, the series is

$$\begin{cases}\text{convergent if } 0|x| < 1 \\ \text{unknown if } 0|x| = 1 \\ \text{divergent if } 0|x| > 1\end{cases}$$

From the condition of convergence, which can also be written as $|x| < \frac{1}{0} = \infty$, or $-\infty < x < \infty$, we see that the center of convergence is at $x = 0$ and the radius of convergence is ∞.

(c) The Taylor series expansion for a function $f(x)$ about the point $x = x_0$ is

$$f(x) = \sum_{n=0}^{\infty}\frac{f^{(n)}(x_0)}{n!}(x - x_0)^n$$

Since $x_0 = 1$, this formula becomes

$$f(x) = \sum_{n=0}^{\infty}\frac{f^{(n)}(1)}{n!}(x - 1)^n$$

The aim then is to determine the nth derivative of x evaluated at $x = 1$. Start taking derivatives and try to observe a pattern.

$$\begin{aligned}n = 0: &\quad f^{(0)}(x) = x &\rightarrow &\quad f^{(0)}(1) = 1 \\ n = 1: &\quad f^{(1)}(x) = 1 &\rightarrow &\quad f^{(1)}(1) = 1 \\ n = 2: &\quad f^{(2)}(x) = 0 &\rightarrow &\quad f^{(2)}(1) = 0 \\ n = 3: &\quad f^{(3)}(x) = 0 &\rightarrow &\quad f^{(3)}(1) = 0 \\ n = 4: &\quad f^{(4)}(x) = 0 &\rightarrow &\quad f^{(4)}(1) = 0 \\ n = 5: &\quad f^{(5)}(x) = 0 &\rightarrow &\quad f^{(5)}(1) = 0 \\ &\vdots\end{aligned}$$

Notice that $f^{(n)}(1) = 0$ for $n > 1$. Therefore,

$$\begin{aligned} f(x) = x &= \frac{f^{(0)}(1)}{0!}(x-1)^0 + \frac{f^{(1)}(1)}{1!}(x-1)^1 + \sum_{n=2}^{\infty} \frac{f^{(n)}(1)}{n!}(x-1)^n \\ &= (x-1)^0 + (x-1)^1 + \sum_{n=2}^{\infty} \frac{0}{n!}(x-1)^n \\ &= 1 + (x-1) \end{aligned}$$

Since this series is finite, it is convergent for any value of x. That is, the radius of convergence is ∞.

(d) The Taylor series expansion for a function $f(x)$ about the point $x = x_0$ is

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(x_0)}{n!}(x-x_0)^n$$

Since $x_0 = -1$, this formula becomes

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(-1)}{n!}(x+1)^n$$

The aim then is to determine the nth derivative of x^2 evaluated at $x = -1$. Start taking derivatives and try to observe a pattern.

$$\begin{aligned} n = 0: &\quad f^{(0)}(x) = x^2 &\rightarrow &\quad f^{(0)}(-1) = 1 \\ n = 1: &\quad f^{(1)}(x) = 2x &\rightarrow &\quad f^{(1)}(-1) = -2 \\ n = 2: &\quad f^{(2)}(x) = 2 &\rightarrow &\quad f^{(2)}(-1) = 2 \\ n = 3: &\quad f^{(3)}(x) = 0 &\rightarrow &\quad f^{(3)}(-1) = 0 \\ n = 4: &\quad f^{(4)}(x) = 0 &\rightarrow &\quad f^{(4)}(-1) = 0 \\ n = 5: &\quad f^{(5)}(x) = 0 &\rightarrow &\quad f^{(5)}(-1) = 0 \\ &\quad \vdots \end{aligned}$$

Notice that $f^{(n)}(-1) = 0$ for $n > 2$. Therefore,

$$\begin{aligned} f(x) = x &= \frac{f^{(0)}(-1)}{0!}(x+1)^0 + \frac{f^{(1)}(-1)}{1!}(x+1)^1 + \frac{f^{(2)}(-1)}{2!}(x+1)^2 + \sum_{n=3}^{\infty} \frac{f^{(n)}(-1)}{n!}(x+1)^n \\ &= (x-1)^0 - 2(x+1)^1 + (x+1)^2 + \sum_{n=3}^{\infty} \frac{0}{n!}(x+1)^n \\ &= 1 - 2(x+1) + (x+1)^2 \end{aligned}$$

Since this series is finite, it is convergent for any value of x. That is, the radius of convergence is ∞.

(e) The Taylor series expansion for a function $f(x)$ about the point $x = x_0$ is

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(x_0)}{n!}(x-x_0)^n$$

Since $x_0 = 1$, this formula becomes

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(1)}{n!}(x-1)^n$$

The aim then is to determine the nth derivative of $\ln\ x$ evaluated at $x = 1$. Start taking derivatives and try to observe a pattern.

$$\begin{aligned}
n = 0: &\quad f^{(0)}(x) = \ln\ x &\rightarrow &\quad f^{(0)}(1) = 0 \\
n = 1: &\quad f^{(1)}(x) = x^{-1} &\rightarrow &\quad f^{(1)}(1) = 1 \\
n = 2: &\quad f^{(2)}(x) = -x^{-2} &\rightarrow &\quad f^{(2)}(1) = -1 \\
n = 3: &\quad f^{(3)}(x) = 2x^{-3} &\rightarrow &\quad f^{(3)}(1) = 2 \\
n = 4: &\quad f^{(4)}(x) = -6x^{-4} &\rightarrow &\quad f^{(4)}(1) = -6 \\
n = 5: &\quad f^{(5)}(x) = 24x^{-5} &\rightarrow &\quad f^{(5)}(1) = 24 \\
&\quad \vdots
\end{aligned}$$

Since $f^{(0)}(1) = 0$, the series can start at $n = 1$.

$$f(x) = \ln\ x = \sum_{n=1}^{\infty} \frac{f^{(n)}(1)}{n!}(x-1)^n$$

Notice that $f^{(n)}(1) = (-1)^{n-1}(n-1)!$ for $n > 0$.

$$\begin{aligned}
\ln\ x &= \sum_{n=1}^{\infty} \frac{(-1)^{n-1}(n-1)!}{n!}(x-1)^n \\
&= \sum_{n=1}^{\infty} \frac{(-1)^{n-1}}{n}(x-1)^n
\end{aligned}$$

Apply the ratio test to determine the condition in which the series converges.

$$\begin{aligned}
\lim_{n\to\infty}\left|\frac{a_{n+1}}{a_n}\right| &= \lim_{n\to\infty}\left|\frac{\frac{(-1)^n}{n+1}(x-1)^{n+1}}{\frac{(-1)^{n-1}}{n}(x-1)^n}\right| \\
&= \lim_{n\to\infty}\left|(-1)\frac{n}{n+1}(x-1)\right| \\
&= \lim_{n\to\infty}\frac{n}{n+1}|x-1| \\
&= \lim_{n\to\infty}\frac{1}{1+\frac{1}{n}}|x-1| \\
&= |x-1|
\end{aligned}$$

According to this test, the series is

$$\begin{cases}
\text{convergent if } |x-1| < 1 \\
\text{unknown if } |x-1| = 1 \\
\text{divergent if } |x-1| > 1
\end{cases}$$

From the condition of convergence, which can also be written as $-1 < x - 1 < 1$, $0 < x < 2$, we see that the center of convergence is at $x = 1$ and the radius of convergence is 1.

(f) The Taylor series expansion for a function $f(x)$ about the point $x = x_0$ is

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(x_0)}{n!}(x - x_0)^n$$

Since $x_0 = 0$, this formula becomes

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(0)}{n!}(x)^n$$

The aim then is to determine the nth derivative of $\frac{1}{1-x}$ evaluated at $x = 0$. Start taking derivatives and try to observe a pattern.

$$\begin{aligned}
n = 0: &\quad f^{(0)}(x) = (1-x)^{-1} &\rightarrow &\quad f^{(0)}(0) = 1 \\
n = 1: &\quad f^{(1)}(x) = (1-x)^{-2} &\rightarrow &\quad f^{(1)}(0) = 1 \\
n = 2: &\quad f^{(2)}(x) = 2(1-x)^{-3} &\rightarrow &\quad f^{(2)}(0) = 2 \\
n = 3: &\quad f^{(3)}(x) = 6(1-x)^{-4} &\rightarrow &\quad f^{(3)}(0) = 6 \\
n = 4: &\quad f^{(4)}(x) = 24(1-x)^{-5} &\rightarrow &\quad f^{(4)}(0) = 24 \\
n = 5: &\quad f^{(5)}(x) = 120(1-x)^{-6} &\rightarrow &\quad f^{(5)}(0) = 120 \\
&\quad \vdots
\end{aligned}$$

Notice that $f^{(n)}(0) = n!$.

$$\begin{aligned}
f(x) = \frac{1}{1-x} &= \sum_{n=0}^{\infty} \frac{n!}{n!} x^n \\
&= \sum_{n=0}^{\infty} x^n
\end{aligned}$$

Apply the ratio test to determine the condition in which the series converges.

$$\begin{aligned}
\lim_{n\to\infty} \left| \frac{a_{n+1}}{a_n} \right| &= \lim_{n\to\infty} \left| \frac{x^{n+1}}{x^n} \right| \\
&= \lim_{n\to\infty} |x| \\
&= |x|
\end{aligned}$$

According to this test, the series is

$$\begin{cases}
\text{convergent if } |x| < 1 \\
\text{unknown if } |x| = 1 \\
\text{divergent if } |x| > 1
\end{cases}$$

From the condition of convergence, which can also be written as $-1 < x < 1$, we see that the center of convergence is at $x = 0$ and the radius of convergence is 1.

(g) The Taylor series expansion for a function $f(x)$ about the point $x = x_0$ is

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(x_0)}{n!}(x - x_0)^n$$

Since $x_0 = 0$, this formula becomes

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(0)}{n!}(x)^n$$

The aim then is to determine the nth derivative of $\frac{1}{1+x}$ evaluated at $x = 0$. Start taking derivatives and try to observe a pattern.

$$\begin{aligned}
n = 0: &\quad f^{(0)}(x) = (1+x)^{-1} &\rightarrow &\quad f^{(0)}(0) = 1 \\
n = 1: &\quad f^{(1)}(x) = -(1+x)^{-2} &\rightarrow &\quad f^{(1)}(0) = -1 \\
n = 2: &\quad f^{(2)}(x) = 2(1+x)^{-3} &\rightarrow &\quad f^{(2)}(0) = 2 \\
n = 3: &\quad f^{(3)}(x) = -6(1+x)^{-4} &\rightarrow &\quad f^{(3)}(0) = -6 \\
n = 4: &\quad f^{(4)}(x) = 24(1+x)^{-5} &\rightarrow &\quad f^{(4)}(0) = 24 \\
n = 5: &\quad f^{(5)}(x) = -120(1+x)^{-6} &\rightarrow &\quad f^{(5)}(0) = -120 \\
&\quad \vdots
\end{aligned}$$

Notice that $f^{(n)}(0) = (-1)^n n!$.

$$\begin{aligned}
f(x) = \frac{1}{1+x} &= \sum_{n=0}^{\infty} \frac{(-1)^n n!}{n!} x^n \\
&= \sum_{n=0}^{\infty} (-1)^n x^n
\end{aligned}$$

Apply the ratio test to determine the condition in which the series converges.

$$\begin{aligned}
\lim_{n\to\infty} \left| \frac{a_{n+1}}{a_n} \right| &= \lim_{n\to\infty} \left| \frac{(-1)^{n+1} x^{n+1}}{(-1)^n x^n} \right| \\
&= \lim_{n\to\infty} |(-1)x| \\
&= \lim_{n\to\infty} |x| \\
&= |x|
\end{aligned}$$

According to this test, the series is

$$\begin{cases} \text{convergent if } |x| < 1 \\ \text{unknown if } |x| = 1 \\ \text{divergent if } |x| > 1 \end{cases}$$

From the condition of convergence, which can also be written as $-1 < x < 1$, we see that the center of convergence is at $x = 0$ and the radius of convergence is 1.

(h) The Taylor series expansion for a function $f(x)$ about the point $x = x_0$ is

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(x_0)}{n!}(x - x_0)^n$$

Since $x_0 = 2$, this formula becomes

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(0)}{n!}(x)^n$$

The aim then is to determine the nth derivative of $\frac{1}{1-x}$ evaluated at $x = 2$. Start taking derivatives and try to observe a pattern.

$$\begin{aligned}
n = 0: &\quad f^{(0)}(x) = (1-x)^{-1} &\rightarrow &\quad f^{(0)}(2) = -1 \\
n = 1: &\quad f^{(1)}(x) = (1-x)^{-2} &\rightarrow &\quad f^{(1)}(2) = 1 \\
n = 2: &\quad f^{(2)}(x) = 2(1-x)^{-3} &\rightarrow &\quad f^{(2)}(2) = -2 \\
n = 3: &\quad f^{(3)}(x) = 6(1-x)^{-4} &\rightarrow &\quad f^{(3)}(2) = 6 \\
n = 4: &\quad f^{(4)}(x) = 24(1-x)^{-5} &\rightarrow &\quad f^{(4)}(2) = -24 \\
n = 5: &\quad f^{(5)}(x) = 120(1-x)^{-6} &\rightarrow &\quad f^{(5)}(2) = 120 \\
&\quad \vdots
\end{aligned}$$

Notice that $f^{(n)}(2) = (-1)^{n+1}n!$.

$$\begin{aligned}
f(x) = \frac{1}{1-x} &= \sum_{n=0}^{\infty} \frac{(-1)^{n+1}n!}{n!}(x-2)^n \\
&= \sum_{n=0}^{\infty} (-1)^{n+1}(x-2)^n
\end{aligned}$$

Apply the ratio test to determine the condition in which the series converges.

$$\begin{aligned}
\lim_{n\to\infty} \left| \frac{a_{n+1}}{a_n} \right| &= \lim_{n\to\infty} \left| \frac{(-1)^{n+2}(x-2)^{n+1}}{(-1)^{n+1}(x-2)^n} \right| \\
&= \lim_{n\to\infty} |(-1)(x-2)| \\
&= \lim_{n\to\infty} |x-2| \\
&= |x-2|
\end{aligned}$$

According to this test, the series is

$$\begin{cases}
\text{convergent if } |x-2| < 1 \\
\text{unknown if } |x-2| = 1 \\
\text{divergent if } |x-2| > 1
\end{cases}$$

From the condition of convergence, which can also be written as $-1 < x - 2 < 1$, or $1 < x < 3$, we see that the center of convergence is at $x = 2$ and the radius of convergence is 1.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test Taylor Series ]# ./main

Taylor series for sin(x) , x_{0} = 0

x-1/6*x^(3)+1/120*x^(5)-1/5040*x^(7)+1/362880*x^(9)

Taylor series for exp(x) , x_{0} = 0

x+1/2*x^(2)+1/6*x^(3)+1/24*x^(4)+1/120*x^(5)+1/720*x^(6)+1/5040*x^(7)+1/40320*x^(8)+1/362880*x^(9)+1

Taylor series for x, x_{0} = 1

x

Taylor series for x^{2}, x_{0} = -1

x^(2)

Taylor series for ln(x), x_{0} = 1

9*x-18*x^(2)+28*x^(3)-31.5*x^(4)+25.2*x^(5)-14*x^(6)+5.14286*x^(7)-1.125*x^(8)+1/9*x^(9)-2.82897

Taylor series for 1/(1-x), x_{0} = 0

x+x^(2)+x^(3)+x^(4)+x^(5)+x^(6)+x^(7)+x^(8)+x^(9)+1

Taylor series for 1/(1+x), x_{0} = 0

-x+x^(2)-x^(3)+x^(4)-x^(5)+x^(6)-x^(7)+x^(8)-x^(9)+1

Taylor series for 1/(1-x), x_{0} = 2

4097*x-7423*x^(2)+7937*x^(3)-5503*x^(4)+2561*x^(5)-799*x^(6)+161*x^(7)-19*x^(8)+x^(9)-1023

Time taken by function: 2445077 microseconds
```

Figure 3.62: *The computation for the taylor series expansion for function $f(x)$ in the example (a) to (h) (**SymIntegration/Examples/Differential Equations/Test Taylor Series/main.cpp**).*

[SI*] **Series Solutions Near an Ordinary Point**

We now consider methods of solving second order linear equations when the coefficients are functions of the independent variable. In this section we will denote the independent variable by x. It is sufficient to consider the homogeneous equation

$$P(x)\frac{d^2y}{dx^2} + Q(x)\frac{dy}{dx} + R(x)y = 0 \tag{3.260}$$

since the procedure for the corresponding nonhomogeneous equation is similar. Many problems in mathematical physics lead to equations of the form (3.260) having polynomial coefficients; examples include the Bessel equation

$$x^2y'' + xy' + (x^2 - \nu^2)y = 0$$

where ν is a constant, and the Legendre equation

$$(1 - x^2)y'' - 2xy' + \alpha(\alpha + 1)y = 0$$

where α is a constant. For this reason, as well as to simplify the algebraic computations, we primarily consider the case in which the functions P, Q, and R are polynomials. However, as we will see, the method of solution is also applicable when P, Q, and R are general analytic functions.

For the present, then, suppose that P, Q, and R are polynomials and that they have no common factors. Suppose also that we wish to solve Eq. (3.260) in the neighborhood of a point x_0. The solution of Eq. (3.260) in an interval containing x_0 is closely associated with the behavior of P in that interval.

A point x_0 such that $P(x_0) \neq 0$ is called an ordinary point. Since P is continuous, it follows that there is an interval about x_0 in which $P(x)$ is never zero. In that interval we can divide Eq. (3.260) by $P(x)$ to obtain

$$y'' + p(x)y' + q(x)y = 0 \tag{3.261}$$

where $p(x) = \frac{Q(x)}{P(x)}$ and $q(x) = \frac{R(x)}{P(x)}$ are continuous functions. Hence, according to the existence and uniqueness Theorem, there exists in that interval a unique solution of Eq. (3.260) that also satisfies the initial conditions $y(x_0) = y_0, y'(x_0) = y'_0$ for arbitrary values of y_0 and y'_0. In this section we discuss the solution of Eq. (3.260) in the neighborhood of an ordinary point.

On the other hand, if $P(x_0) = 0$, then $x_0$is called a signular point of Eq. (3.260). In this case at least one of $Q(x_0)$ and $R(x_0)$ is not zero. Consequently, at least one of the coefficients p and q in Eq. (3.261) becomes unbounded as $x \to x_0$, and therefore the uniqueness Theorem does not apply in this case.

We now take up the problem of solving Eq. (3.260) in the neighborhood of an ordinary point x_0. We look for solutions of the form

$$y = a_0 + a_1(x - x_0) + \cdots + a_n(x - x_0)^n + \cdots = \sum_{n=0}^{\infty} a_n(x - x_0)^n \tag{3.262}$$

and assume that the series converges in the interval $|x - x_0| < \rho$ for some $\rho > 0$. While at first sight it may appear unattractive to seek a solution in the form of a power series, this is actually a convenient and useful form for a solution. Within their intervals of convergence, power series behave very much like polynomials and are easy to manipulate both analytically and numerically. Indeed, even if we can obtain a solution in terms of elementary functions, such as exponential and trigonometric functions, we are likely to need a power series or some equivalent expression if we want to evaluate them numerically or to plot their graphs. The most practical way to determine the coefficients a_n is to substitute the series (3.262) and its derivatives for y, y', and y'' in Eq. (3.260).

[SI*] **Example:**

Find a series solution of the equation

$$y'' + y = 0, \quad -\infty < x < \infty \tag{3.263}$$

Solution:

As we know, a fundamental set of solutions of this equation are sin x and cos x, so series methods are not needed to solve this equation. However, this example illustrates the use of power series in a relatively simple case. For Eq. (3.263), $P(x) = 1, Q(x) = 0$, and $R(x) = 1$; hence every point is an ordinary point.

We look for a solution in the form of a power series about $x_0 = 0$

$$y = a_0 + a_1x + a_2x^2 + \cdots + a_nx^n + \cdots = \sum_{n=0}^{\infty} a_nx^n \tag{3.264}$$

and assume that the series converges in some interval $|x| < \rho$. Differentiating Eq. (3.264) term by term yields

$$y' = a_1 + 2a_2x + \cdots + na_nx^{n-1} + \cdots = \sum_{n=1}^{\infty} na_nx^{n-1} \tag{3.265}$$

$$y'' = 2a_2 + 6a_3x + \cdots + n(n-1)a_nx^{n-2} + \cdots = \sum_{n=2}^{\infty} n(n-1)a_nx^{n-2} \tag{3.266}$$

Substituting the series (3.264) and (3.366) for y and y'' in Eq. (3.263) gives

$$\sum_{n=2}^{\infty} n(n-1)a_nx^{n-2} + \sum_{n=0}^{\infty} a_nx^n = 0$$

To combine the two series, we need to rewrite at least one of them so that both series display the same generic term. Thus, in the first sum, we shift the index of summation by replacing n by $n+2$ and starting the sum at 0 rather than at 2. We obtain

$$\sum_{n=0}^{\infty}(n+2)(n+1)a_{n+2}x^n + \sum_{n=0}^{\infty}a_n x^n = 0$$

or

$$\sum_{n=0}^{\infty}\left[(n+2)(n+1)a_{n+2} + a_n\right]x^n = 0$$

For this equation to be satisfied for all x, the coefficient of each power of x must be zero; hence we conclude that

$$(n+2)(n+1)a_{n+2} + a_n = 0, \quad n = 0, 1, 2, 3, \ldots \tag{3.267}$$

Equation (3.267) is referred to as a recurrence relation. The successive coefficients can be evaluated one by one by writing the recurrence relation first for $n = 0$, then for $n = 1$, and so forth. In this example Eq. (3.267) relates each coefficient to the second one before it. Thus the even-numbered coefficients $(a_0, a_2, a_4, \ldots)$ and the odd-numbered ones $(a_1, a_3, a_5, \ldots)$ are determined separately. For the even-numbered coefficients we have

$$\begin{aligned} a_2 &= -\frac{a_0}{2\cdot 1} = -\frac{a_0}{2!} \\ a_4 &= -\frac{a_2}{4\cdot 3} = \frac{a_0}{4!} \\ a_6 &= -\frac{a_4}{6\cdot 5} = -\frac{a_0}{6!} \end{aligned}$$

These results suggest that in general, if $n = 2k$, then

$$a_n = a_{2k} = \frac{(-1)^k}{(2k)!}a_0, \quad k = 1, 2, 3, \ldots \tag{3.268}$$

We can prove Eq. (3.268) by mathematical induction. First, observe that it is true for $k = 1$. Next, assume that it is true for an arbitrary value of k and consider the case $k+1$. We have

$$a_{2k+2} = -\frac{a_{2k}}{(2k+2)(2k+1)} = -\frac{(-1)^k}{(2k+2)(2k+1)(2k)!}a_0 = \frac{(-1)^{k+1}}{(2k+2)!}a_0$$

Hence Eq. (3.268) is also true for $k+1$, and consequently it is true for all positive integers k. Similarly, for the odd-numbered coefficients

$$\begin{aligned} a_3 &= -\frac{a_1}{3\cdot 2} = -\frac{a_1}{3!} \\ a_5 &= -\frac{a_3}{5\cdot 4} = \frac{a_1}{5!} \\ a_7 &= -\frac{a_5}{7\cdot 6} = -\frac{a_1}{7!} \end{aligned}$$

and in general, if $n = 2k+1$, then

$$a_n = a_{2k+1} = \frac{(-1)^k}{(2k+1)!}a_1, \quad k = 1, 2, 3, \ldots \tag{3.269}$$

Substituting these coefficients into Eq. (3.264)

$$\begin{aligned} y &= a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n + \ldots \\ &= a_0 + a_1 x - \frac{a_0}{2!} x^2 - \frac{a_1}{3!} x^3 + \frac{a_0}{4!} x^4 + \frac{a_1}{5!} x^5 \\ &+ \cdots + \frac{(-1)^n a_0}{(2n)!} x^{2n} + \frac{(-1)^n a_1}{(2n+1)!} x^{2n+1} + \ldots \\ &= a_0 \left[1 - \frac{x^2}{2!} + \frac{x^4}{4!} + \cdots + \frac{(-1)^n}{(2n)!} x^{2n} + \ldots \right] \\ &+ a_1 \left[x - \frac{x^3}{3!} + \frac{x^5}{5!} + \cdots + \frac{(-1)^n}{(2n+1)!} x^{2n+1} + \ldots \right] \\ &= a_0 \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n)!} x^{2n} + a_1 \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)!} x^{2n+1} \end{aligned} \tag{3.270}$$

Now that we have formally obtained two series solutions of Eq. (3.263), we can test them for convergence. Using the ratio test, we can show that each of the series in Eq. (3.270) converges for all x, and this justifies retroactively all the steps used in obtaining the solutions. Indeed, we recognize that the first series in Eq. (3.270) is exactly the Taylor series for $\cos\ x$ about $x = 0$ and that the second is the Taylor series for $\sin\ x$ about $x = 0$. Thus, as expected, we obtain the solution $y = a_0 \cos\ x + a_1 \sin\ x$.
Notice that no conditions are imposed on a_0 and a_1; hence they are arbitrary. From Eqs. (3.264) and (3.265) we see that y and y' evaluated at $x = 0$ are a_0 and a_1, respectively. Since the initial conditions $y(0)$ and $y'(0)$ can be chosen arbitrarily, it follows that a_0 and a_1 should be arbitrary until specific initial conditions are stated.

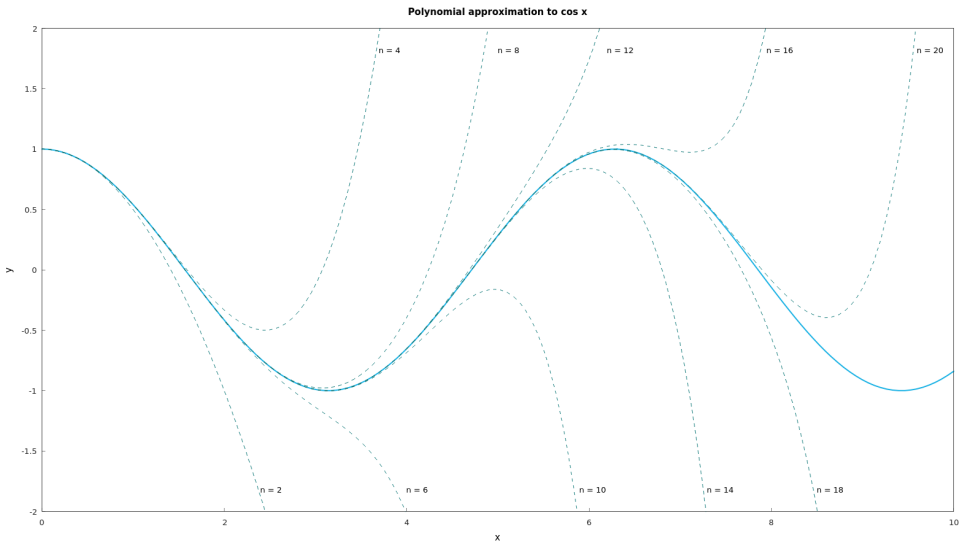

Figure 3.63: *The polynomial approximations to* $\cos\ x$. *The value of n is the degree of the approximating polynomial* (***Hamzstplot/Examples with Makefile/Differential Equations/Plot 2D Polynomial Approximation to Cosine/main.cpp***).

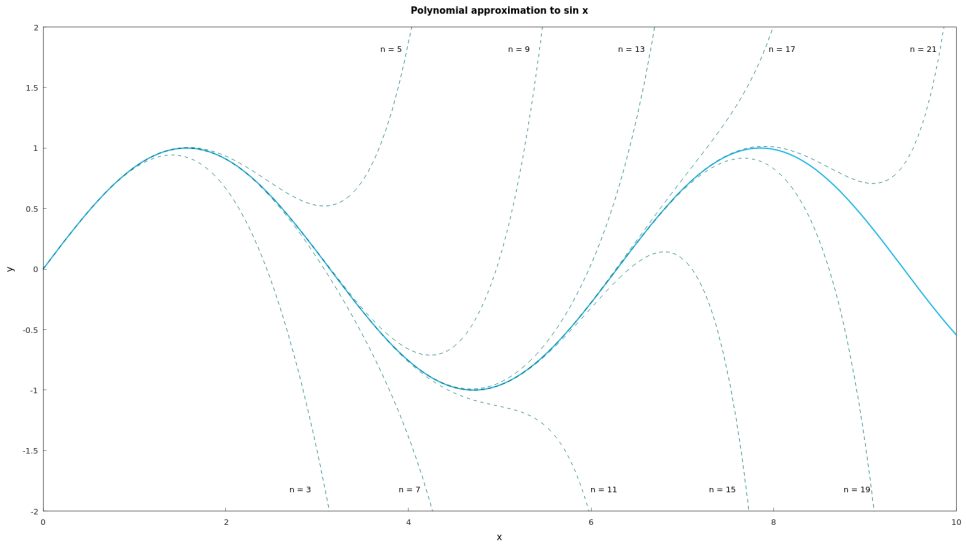

Figure 3.64: *The polynomial approximations to* sin x. *The value of n is the degree of the approximating polynomial (**Hamzstplot/Examples with Makefile/Differential Equations/Plot 2D Polynomial Approximation to Cosine/main.cpp**).*

The graphs show how the partial sums of the series in Eq. (3.270) approximate cos x and sin x. As the number of terms increases, the interval over which the approximation is satisfactory become longer, and for each x in this interval the accuracy of the approximation improves. However, you should always remember that a truncated power series provides only a local approximation of the solution in a neighborhood of the initial point $x = 0$; it cannot adequately represent the solution for large $|x|$.

[SI*] If we had not known that sin x and cos x form a fundamental set of solutions of Eq. (3.263) and had simply solved Eq. (3.263) using series methods, we would still have obtained the solution (3.270). In recognition of the fact that the differential equation (3.263) often occurs in applications, we might decide to give the two solutions of Eq. (3.270) special names

$$C(x) = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n)!} x^{2n}, \quad S(x) = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)!} x^{2n+1} \tag{3.271}$$

Then we might ask what properties these functions have. For instance, can we be sure that $C(x)$ and $S(x)$ form a fundamental set of solutions? It follows at once from the series expansions that $C(0) = 1$ and $S(0) = 0$. By differentiating the series for $C(x)$ and $S(x)$ term by term, we find that

$$S'(x) = C(x), \quad C'(x) = -S(x) \tag{3.272}$$

Thus, at $x = 0$ we have $S'(0) = 1$ and $C'(0) = 0$. Consequently, the Wronskian of C and S at $x = 0$ is

$$W(C, S)(0) = \begin{vmatrix} 1 & 0 \\ 0 & 1 \end{vmatrix} = 1 \tag{3.273}$$

so these functions do indeed form a fundamental set of solutions. By substituting $-x$ for x in each of Eqs. (3.271), we obtain $C(-x) = C(x)$, and $S(-x) = -S(x)$. Moreover, by calculating with the infinite series, we can show that the functions $C(x)$ and $S(x)$ have all the usual analytical and algebraic properties of the cosine and sine functions, respectively.

The function sin x can be defined as the unique solution of the initial value problem $y'' + y = 0, y(0) = 0, y'(0) = 1$; similarly, cos x can be defined as the unique solution of the initial value problem $y'' + y = 0, y(0) = 1, y'(0) = 0$. Many other functions that are important in mathematical physics are also defined as solutions of certain initial value problems. For most of these functions there is no simpler or more elmentary way to approach them.

[SI*] **Example:**
Find a series solution in powers of x of Airy's equation

$$y'' - xy = 0, \quad -\infty < x < \infty \tag{3.274}$$

Solution:
For this equation $P(x) = 1, Q(x) = 0$, and $R(x) = -x$; hence every point is an ordinary point. We assume that

$$y = \sum_{n=0}^{\infty} a_n x^n \tag{3.275}$$

and that the series converges in some interval $|x| < \rho$. The series for y'' is given by Eq. (3.266), we can rewrite it as

$$y'' = \sum_{n=0}^{\infty} (n+2)(n+1) a_{n+2} x^n \tag{3.276}$$

Substituting the series (3.275) and (3.276) for y and y'' in Eq. (3.274), we obtain

$$\sum_{n=0}^{\infty} (n+2)(n+1) a_{n+2} x^n - x \sum_{n=0}^{\infty} a_n x^n = \sum_{n=0}^{\infty} a_n x^{n+1} \tag{3.277}$$

Next, we shift the index of summation in the series on the right side of Eq. (3.277) by replacing n by $n-1$ and starting the summation at 1 rathe than zero. Thus we have

$$2 \cdot 1 a_2 + \sum_{n=1}^{\infty} (n+2)(n+1) a_{n+2} x^n = \sum_{n=1}^{\infty} a_{n-1} x^n$$

Again, for this equation to be satisfied for all x in some interval, the coefficients of like powers of x must be equal; hence $a_2 = 0$, and we obtain the recurrence relation

$$(n+2)(n+1) a_{n+2} = a_{n-1}, \quad \text{for } n = 1, 2, 3, \ldots \tag{3.278}$$

Since a_{n+2} is given in terms of a_{n-1}, the a's are determined in steps of three. Thus a_0 determines a_3, which in turn determines $a_6, \ldots$; a_1 determines a_4, which in turn determines $a_7, \ldots$; and a_2 determines a_5, which in turn determines $a_8, \ldots$. Since $a_2 = 0$, we immediately conclude that $a_5 = a_8 = a_{11} = \cdots = 0$.
For the sequence $a_0, a_3, a_6, a_9, \ldots$ we set $n = 1, 4, 7, 10, \ldots$ in the recurrence relation:

$$\begin{aligned} a_3 &= \frac{a_0}{2 \cdot 3} \\ a_6 &= \frac{a_3}{5 \cdot 6} = \frac{a_0}{2 \cdot 3 \cdot 5 \cdot 6} \\ a_9 &= \frac{a_6}{8 \cdot 9} = \frac{a_0}{2 \cdot 3 \cdot 5 \cdot 6 \cdot 8 \cdot 9} \end{aligned}$$

These results suggest the general formula

$$a_{3n} = \frac{a_0}{2 \cdot 3 \cdot 5 \cdot 6 \ldots (3n-1)(3n)}, \quad n \geq 4$$

For the sequence $a_1, a_4, a_7, a_{10}, \ldots$ we set $n = 2, 5, 8, 11, \ldots$ in the recurrence relation:

$$\begin{aligned} a_4 &= \frac{a_1}{3 \cdot 4} \\ a_7 &= \frac{a_4}{6 \cdot 7} = \frac{a_0}{3 \cdot 4 \cdot 6 \cdot 7} \\ a_{10} &= \frac{a_7}{9 \cdot 10} = \frac{a_0}{3 \cdot 4 \cdot 6 \cdot 7 \cdot 9 \cdot 10} \end{aligned}$$

In general, we have

$$a_{3n+1} = \frac{a_1}{3 \cdot 4 \cdot 6 \cdot 7 \ldots (3n)(3n+1)}, \quad n \geq 4$$

Thus the general solution of Airy's equation is

$$\begin{aligned} y = a_0 &\left[1 + \frac{x^3}{2 \cdot 3} + \frac{x^6}{2 \cdot 3 \cdot 5 \cdot 6} + \cdots + \frac{x^{3n}}{2 \cdot 3 \ldots (3n-1)(3n)} + \ldots\right] \\ + a_1 &\left[x + \frac{x^4}{3 \cdot 4} + \frac{x^7}{3 \cdot 4 \cdot 6 \cdot 7} + \cdots + \frac{x^{3n+1}}{3 \cdot 4 \ldots (3n)(3n+1)} + \ldots\right] \end{aligned} \tag{3.279}$$

Having obtained these two series solutions, we can now investigate their convergence. Because of the rapid growth of the denominators of the terms in the series (3.279), we might expect these series to have a large radius of convergence. Indeed, it is easy to use the ratio test to show that both these series converge for all x.

Assuming for the moment that the series do converge for all x, let y_1 and y_2 denote the functions defined by the expressions in the first and second sets of brackets, respectively, in Eq. (3.279). Then, by choosing first $a_0 = 1, a_1 = 0$ and then $a_0 = 0, a_1 = 1$, it follows that y_1 and y_2 are individually solutions of Eq. (3.274). Notice that y_1 satisfies the initial conditions $y_1(0) = 1, y_1'(0) = 0$ and that y_2 satisfies the initial conditions $y_2(0) = 0, y_2'(0) = 1$. Thus $W(y_1, y_2)(0) = 1 \neq 0$, and consequently y_1 and y_2 are a fundamental set of solutions. Hence the general solution of Airy's equation is

$$y = a_0 y_1(x) + a_1 y_2(x), \quad -\infty < x < \infty$$

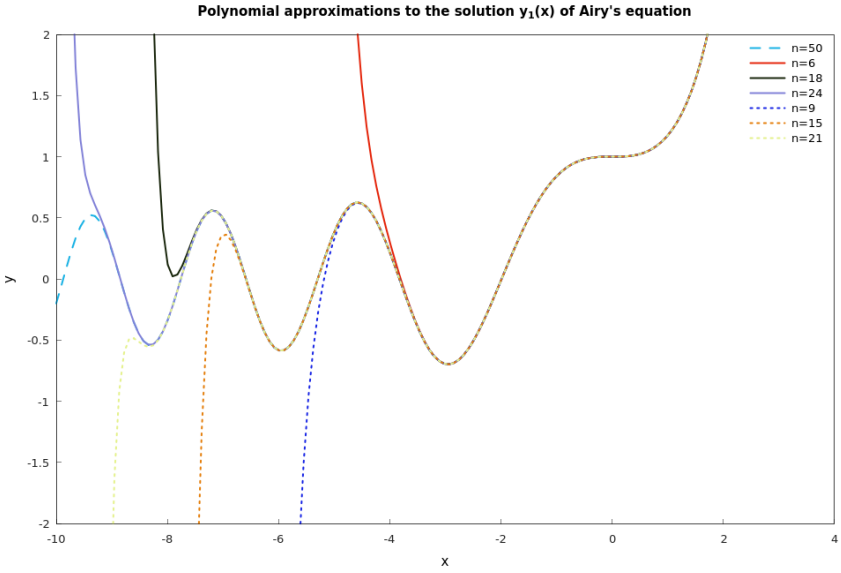

Figure 3.65: *The polynomial approximations to the solution $y_1(x)$ of Airy's equation. The value of n is the degree of the approximating polynomial (**Hamzstplot/Examples with Makefile/Differential Equations/Plot 2D Solution of Airy's Equation Ai(x)/main.cpp**).*

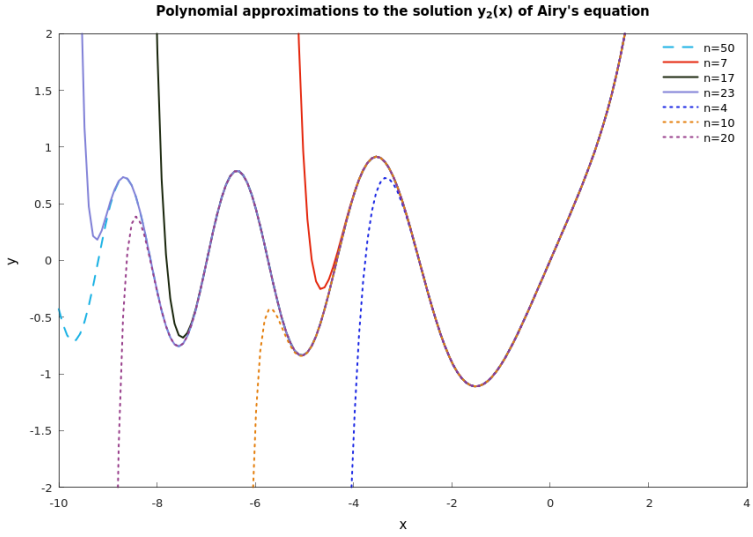

Figure 3.66: *The polynomial approximations to the solution $y_2(x)$ of Airy's equation. The value of n is the degree of the approximating polynomial (**Hamzstplot/Examples with Makefile/Differential Equations/Plot 2D Solution of Airy's Equation Bi(x)/main.cpp**).*

The partial sums provide provide local approximations to the solutions in a neighborhood of the origin. Although the quality of the approximation improves as the number of terms increases, no polynomial can adequately represent y_1 and y_2 for large $|x|$. A practical way to estimate the interval in which a given partial sum is reasonably accurate is to compare the graphs of that partial sum and the next one, obtained by including one more term. As soon

as the graphs begin to separate noticeably, we can be confident that the original partial sum is no longer accurate.

Observe that both y_1 and y_2 are monotone for $x > 0$ and oscillatory for $x < 0$. The oscillations are not uniform but, rather, decay in amplitude and increase in frequency as the distance from the origin increases.

The solutions y_1 and y_2 of Airy's equation are not elementary functions that you have already encountered in calculus. However, because of their importance in some physical applications, these functions have been extensively studied, and their properties are well known to applied mathematicians and scientists.

[SI*] **Example:**
Find a series solution in powers of $x - 1$ of Airy's equation.

Solution:
The point $x = 1$ is an ordinary point of Eq. (3.274), and thus we look for a solution of the form

$$y = \sum_{n=0}^{\infty} a_n (x-1)^n$$

where we assume that the series converges in some interval $|x - 1| < \rho$. Then

$$y' = \sum_{n=1}^{\infty} n a_n (x-1)^{n-1} = \sum_{n=0}^{\infty} (n+1) a_{n+1} (x-1)^n$$

and

$$y'' = \sum_{n=2}^{\infty} n(n-1) a_n (x-1)^{n-2} = \sum_{n=0}^{\infty} (n+2)(n+1) a_{n+2} (x-1)^n$$

Substituting for y and y'' in Eq. (3.274), we obtain

$$\begin{aligned} y'' - xy &= 0 \\ \sum_{n=0}^{\infty} (n+2)(n+1) a_{n+2} (x-1)^n - x \sum_{n=0}^{\infty} a_n (x-1)^n &= 0 \\ \sum_{n=0}^{\infty} (n+2)(n+1) a_{n+2} (x-1)^n &= x \sum_{n=0}^{\infty} a_n (x-1)^n \end{aligned} \tag{3.280}$$

Now to equate the coefficients of like powers of $(x - 1)$, we must express x, the coefficient of y in Eq. (3.274), in powers of $x - 1$; that is, we write $x = 1 + (x - 1)$. Note that this is precisely the Taylor series for x about $x = 1$. Then Eq. (3.280) takes the form

$$\begin{aligned} \sum_{n=0}^{\infty} (n+2)(n+1) a_{n+2} (x-1)^n &= [1 + (x-1)] \sum_{n=0}^{\infty} a_n (x-1)^n \\ &= \sum_{n=0}^{\infty} a_n (x-1)^n + \sum_{n=0}^{\infty} a_n (x-1)^{n+1} \end{aligned}$$

Shifting the index of summation in the second series on the right gives

$$\sum_{n=0}^{\infty} (n+2)(n+1) a_{n+2} (x-1)^n = \sum_{n=0}^{\infty} a_n (x-1)^n + \sum_{n=1}^{\infty} a_{n-1} (x-1)^n$$

Equating coefficients of like powers of $x-1$, we obtain

$$\begin{aligned} 2a_2 &= a_0 \\ (3\cdot 2)a_3 &= a_1 + a_0 \\ (4\cdot 3)a_4 &= a_2 + a_1 \\ (5\cdot 4)a_5 &= a_3 + a_2 \\ &\vdots \end{aligned}$$

The general recurrence relation is

$$(n+2)(n+1)a_{n+2} = a_n + a_{n-1}, \quad \text{for } n \geq 1 \tag{3.281}$$

Solving for the first few coefficients a_n in terms of a_0 and a_1, we find that

$$\begin{aligned} a_2 &= \frac{a_0}{2} \\ a_3 &= \frac{a_1}{6} + \frac{a_0}{6} \\ a_4 &= \frac{a_2}{12} + \frac{a_1}{12} = \frac{a_0}{24} + \frac{a_1}{12} \\ a_5 &= \frac{a_3}{20} + \frac{a_2}{20} = \frac{a_0}{30} + \frac{a_1}{120} \end{aligned}$$

Hence

$$\begin{aligned} y = a_0 &\left[1 + \frac{(x-1)^2}{2} + \frac{(x-1)^3}{6} + \frac{(x-1)^4}{24} + \frac{(x-1)^5}{30} + \dots\right] \\ &+ a_1 \left[(x-1) + \frac{(x-1)^3}{6} + \frac{(x-1)^4}{12} + \frac{(x-1)^5}{120} + \dots\right] \end{aligned} \tag{3.282}$$

In general, when the recurrence relation has more than two terms, as in Eq. (3.281), the determination of a formula for a_n in terms a_0 and a_1 will be fairly complicated, if not impossible. In this example such a formula is not readily apparent. Lacking such a formula, we cannot test the two series in Eq. (3.282) for convergence by direct methods such as the ratio test. However, even without knowing the formula for a_n, it is possible to establish that the two series in Eq. (3.282) converge for all x. Further, they define functions y_3 and y_4, that are a fundamental set of solutions of the Airy equation (3.274). Thus

$$y = a_0 y_3(x) + a_1 y_4(x)$$

is the general solution of Airy's equation for $-\infty < x < \infty$.

[SI*] If we look for a solution of Eq. (3.260) of the form $y = \sum_{n=0}^{\infty} a_n (x-x_0)^n$, then the coefficients $P(x), Q(x)$, and $R(x)$ in Eq. (3.260) must also be expressed in powers of $x - x_0$. Alternatively, we can make the change of variable $x - x_0 = t$, obtaining a new differential equation for y as a function of t, and then look for solutions of this new equation of the form $\sum_{n=0}^{\infty} a_n t^n$. When we have finished the calculations, we replace t by $x - x_0$.

When we have found the two sets of solutions , for example of Airy's equation. The functions y_1 and y_2 defined by the series in Eq. (3.279) are a fundamental set of solutions of Eq. (3.274) for all x, and this is also true for the functions y_3 and y_4 defined by the series in Eq.

(3.282). According to the general theory of second order linear equations, each of the first two functions can be expressed as a linear combination of the latter two functions and vice versa.

It is not particularly important if we are unable to determine the general coefficient a_n in terms of a_0 and a_1. What is essential is that we can determine as many coefficients as we want. Thus we can find as many terms in the two series solutions as we want, even if we cannot determine the general term.

[SI*] **The Hermite Equation Example:**
The differential equation

$$y'' - 2xy' + \lambda y = 0, \quad -\infty < x < \infty$$

where λ is a constant, is known as the Hermite equation. it is an important equation in mathematical physics

(a) Find the first four terms in each of two solutions about $x = 0$ and show that they form a fundamental set of solutions.

(b) Observe that if λ is a nonnegative even integer, then one or the other of the series solutions terminates and becomes a polynomial. Find the polynomial solutions for $\lambda = 0, 2, 4, 6, 8$, and 10. Note that each polynomial is determined only up to a multiplicative constant.

(c) The Hermite polynomial $H_n(x)$ is defined as the polynomial solution of the Hermite equation with $\lambda = 2n$ for which the coefficient of x^n is 2^n. Find $H_0(x), \ldots, H_5(x)$.

Solution:

(a) $x = 0$ is not a zero of the coefficient of y'', so $x = 0$ is an ordinary point. As such, the solution for y can be represented as a power series centered at $x = 0$.

$$y(x) = \sum_{n=0}^{\infty} a_n x^n$$

Differentiate this series twice with respect to x to get y' and y''.

$$y = \sum_{n=0}^{\infty} a_n x^n$$
$$y' = \sum_{n=1}^{\infty} n a_n x^{n-1}$$
$$y'' = \sum_{n=2}^{\infty} n(n-1) a_n x^{n-2}$$

Substitute these series into the ODE

$$\sum_{n=2}^{\infty} n(n-1) a_n x^{n-2} - 2x \sum_{n=1}^{\infty} n a_n x^{n-1} + \lambda \sum_{n=0}^{\infty} a_n x^n = 0$$

Bring $2x$ and λ inside the respective summands.

$$\sum_{n=2}^{\infty} n(n-1) a_n x^{n-2} - \sum_{n=1}^{\infty} 2n a_n x^n + \sum_{n=0}^{\infty} \lambda a_n x^n = 0$$

Because of the factor n, the second sum can be set to start from $n = 0$

$$\sum_{n=2}^{\infty} n(n-1)a_n x^{n-2} - \sum_{n=0}^{\infty} 2na_n x^n + \sum_{n=0}^{\infty} \lambda a_n x^n = 0$$

Substitute $k = n - 2$ in the first sum and $k = n$ in the others.

$$\sum_{k+2=2}^{\infty} (k+2)((k+2)-1)a_{k+2}x^{(k+2)-2} - \sum_{k=0}^{\infty} 2ka_k x^k + \sum_{k=0}^{\infty} \lambda a_k x^k = 0$$

Solve for k.

$$\sum_{k=0}^{\infty} (k+2)(k+1)a_{k+2}x^k - \sum_{k=0}^{\infty} 2ka_k x^k + \sum_{k=0}^{\infty} \lambda a_k x^k = 0$$

Now that each of the sums has the same limits and factors of x, they can be combined.

$$\sum_{k=0}^{\infty} \left[(k+2)(k+1)a_{k+2}x^k - 2ka_k x^k + \lambda a_k x^k\right] = 0$$

Factor the summand.

$$\sum_{k=0}^{\infty} \left[(k+2)(k+1)a_{k+2} - 2ka_k + \lambda a_k\right] x^k = 0$$
$$\sum_{k=0}^{\infty} \left[(k+2)(k+1)a_{k+2} - (\lambda - 2k)a_k\right] x^k = 0$$

The coefficients must be zero, then solve for a_{k+2}.

$$(k+2)(k+1)a_{k+2} - (\lambda - 2k)a_k = 0$$
$$a_{k+2} = \frac{2k-\lambda}{(k+2)(k+1)}a_k$$

Plug in enough values of k to get four terms involving a_0 and four terms involving a_1.

$$k = 0: \quad a_2 = \frac{0-\lambda}{(2)(1)}a_0 = \frac{0-\lambda}{2\cdot 1}a_0$$
$$k = 1: \quad a_3 = \frac{2-\lambda}{(3)(2)}a_1 = \frac{2-\lambda}{3\cdot 2}a_1$$
$$k = 2: \quad a_4 = \frac{4-\lambda}{(4)(3)}a_2 = \frac{(4-\lambda)(0-\lambda)}{4\cdot 3\cdot 2\cdot 1}a_0$$
$$k = 3: \quad a_5 = \frac{6-\lambda}{(5)(4)}a_3 = \frac{(6-\lambda)(2-\lambda)}{5\cdot 4\cdot 3\cdot 2}a_1$$
$$k = 4: \quad a_6 = \frac{8-\lambda}{(6)(5)}a_4 = \frac{(8-\lambda)(4-\lambda)(0-\lambda)}{6\cdot 5\cdot 4\cdot 3\cdot 2\cdot 1}a_0$$
$$k = 5: \quad a_7 = \frac{10-\lambda}{(7)(6)}a_5 = \frac{(10-\lambda)(6-\lambda)(2-\lambda)}{7\cdot 6\cdot 5\cdot 4\cdot 3\cdot 2}a_1$$
$$\vdots$$

Therefore, the general solution to the Hermite equation is

$$\begin{aligned}
y(x) &= \sum_{n=0}^{\infty} a_n x^n \\
&= a_0 + a_1 x + \frac{0-\lambda}{2\cdot 1}a_0 x^2 + \frac{2-\lambda}{3\cdot 2}a_1 x^3 + \frac{(4-\lambda)(0-\lambda)}{4\cdot 3\cdot 2\cdot 1}a_0 x^4 + \frac{(6-\lambda)(2-\lambda)}{5\cdot 4\cdot 3\cdot 2}a_1 x^5 \\
&+ \frac{(8-\lambda)(4-\lambda)(0-\lambda)}{6\cdot 5\cdot 4\cdot 3\cdot 2\cdot 1}a_0 x^6 + \frac{10-\lambda}{(7)(6)}a_5 = \frac{(10-\lambda)(6-\lambda)(2-\lambda)}{7\cdot 6\cdot 5\cdot 4\cdot 3\cdot 2}a_1 x^7 + \dots \\
&= a_0\left[1 + \frac{0-\lambda}{2\cdot 1}x^2 + \frac{(4-\lambda)(0-\lambda)}{4\cdot 3\cdot 2\cdot 1}x^4 + \frac{(8-\lambda)(4-\lambda)(0-\lambda)}{6\cdot 5\cdot 4\cdot 3\cdot 2\cdot 1}x^6 + \dots\right] \\
&+ a_1\left[x + \frac{2-\lambda}{3\cdot 2}x^3 + \frac{(6-\lambda)(2-\lambda)}{5\cdot 4\cdot 3\cdot 2}x^5 + \frac{(10-\lambda)(6-\lambda)(2-\lambda)}{7\cdot 6\cdot 5\cdot 4\cdot 3\cdot 2}x^7 + \dots\right]
\end{aligned}$$

Denote these two series solutions as $y_1(x)$ and $y_2(x)$, respectively

$$y(x) = a_0 y_1(x) + a_1 y_2(x)$$

Calculate the Wronskian of y_1 and y_2

$$W(y_1, y_2) = \begin{vmatrix} y_1 & y_2 \\ y_1' & y_2' \end{vmatrix} = y_1 y_2' - y_1' y_2$$

at $x = 0$ the Wronskian is nonzero.

$$W(y_1, y_2)(0) = (1)(1) - (0)(0) = 1$$

which means y_1 and y_2 form a fundamental set of solutions for the ODE.

(b) If $\lambda = 0$, then the first series solution terminates

$$y_1(x) = 1$$

If $\lambda = 2$, then the second series solution terminates

$$y_2(x) = x$$

If $\lambda = 4$, then the first series solution terminates

$$y_1(x) = 1 - 2x^2$$

If $\lambda = 6$, then the second series solution terminates

$$y_2(x) = x - \frac{2}{3}x^3$$

If $\lambda = 8$, then the first series solution terminates

$$y_1(x) = 1 - 4x^2 + \frac{4}{3}x^4$$

If $\lambda = 10$, then the second series solution terminates

$$y_2(x) = x - \frac{4}{3}x^3 + \frac{4}{15}x^5$$

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Hermite Differential Equation ]# ./main
( 1 ) y'' + (  - 2x ) y' + ( 0 ) y
Initial conditions: y(0) = 1 , y'(0) = 0

x_{0} = 0

Series solution:

y(x) = 1
( 1 ) y'' + (  - 2x ) y' + ( 2 ) y
Initial conditions: y(0) = 0 , y'(0) = 1

x_{0} = 0

Series solution:

y(x) = 1*x
( 1 ) y'' + (  - 2x ) y' + ( 4 ) y
Initial conditions: y(0) = 1 , y'(0) = 0

x_{0} = 0

Series solution:

y(x) = 1 - 2*x^2
( 1 ) y'' + (  - 2x ) y' + ( 6 ) y
Initial conditions: y(0) = 0 , y'(0) = 1

x_{0} = 0

Series solution:

y(x) = 1*x - 0.666667*x^3
( 1 ) y'' + (  - 2x ) y' + ( 8 ) y
Initial conditions: y(0) = 1 , y'(0) = 0

x_{0} = 0

Series solution:

y(x) = 1 - 4*x^2 + 1.33333*x^4
( 1 ) y'' + (  - 2x ) y' + ( 10 ) y
Initial conditions: y(0) = 0 , y'(0) = 1

x_{0} = 0

Series solution:

y(x) = 1*x - 1.33333*x^3 + 0.266667*x^5

Time taken by function: 800 microseconds
```

Figure 3.67: *The first series solution and second series solution with $\lambda = 0, 2, 4, 6, 8, 10$ for $y'' - 2xy' + \lambda y = 0$ (**SymIntegration/Examples/Differential Equations/Test SymIntegration Hermite Differential Equation/main.cpp**).*

(c) The first five Hermite polynomials are obtained from these previous polynomial solutions. Write them so that the coefficients of x^n are 2^n, where $\lambda = 2n$.

$$n = 0: \quad y_1 = 1 = 2^0 x^0 \quad \rightarrow H_0(x) = 1$$

$$n = 1: \quad y_2 = x = \frac{1}{2}(2^1 x^1) \quad \rightarrow H_1(x) = 2x$$

$$n = 2: \quad y_1 = 1 - 2x^2 = -\frac{1}{2}(2^2 x^2 - 2) \quad \rightarrow H_2(x) = 4x^2 - 2$$

$$n = 3: \quad y_2 = x - \frac{2}{3}x^3 = -\frac{1}{12}(2^3 x^3 - 12x) \quad \rightarrow H_3(x) = 8x^3 - 12x$$

$$n = 4: \quad y_1 = 1 - 4x^2 + \frac{4}{3}x^4 = \frac{1}{12}(12 - 48x^2 + 2^4 x^4) \quad \rightarrow H_4(x) = 16x^4 - 48x^2 + 12$$

$$n = 5: \quad y_2 = x - \frac{4}{3}x^3 + \frac{4}{15}x^5 = \frac{1}{120}(120x - 160x^3 + 2^5 x^5) \quad \rightarrow H_5(x) = 32x^5 - 160x^3 + 120x$$

Charles Hermite (1822-1901) was an influential French analyst and algebraist. An inspiring teacher, he was professor at the École Polytechnique and the Sorbonne. He introduced the Hermite functions in 1864 and showed in 1873 that e is a transcendental number (that is, e is not a root of any polynomial equation with rational coefficients). His name is also associated with Hermitian matrices, somw of whose properties he discovered.

[SI*] **The Chebyshev Equation Example:**

The Chebyshev differential equation is

$$(1-x^2)y'' - xy' + \alpha^2 y = 0$$

where α is a constant.

(a) Determine two solutions in powers of x for $|x| < 1$ and show that they form a fundamental set of solutions.

(b) Show that if α is a nonnegative integer n, then there is a polynomial solution of degree n. These polynomials, when properly normalized, are called the Chebyshev polynomials. They are very useful in problems that require a polynomial approximation to a function defined on $-1 \le x \le 1$.

(c) Find a polynomial solution for each of the cases $\alpha = n = 0, 1, 2, 3$.

Solution:

(a) $x = 0$ is not a zero of the coefficient of y'', so it is an ordinary point. As such, the solution for y can be represented as a power series centered at $x = 0$

$$y(x) = \sum_{p=0}^{\infty} a_p x^p$$

Note that because $x = -1$ and $x = 1$ are zeros of $1 - x^2$, this solution will be valid for $-1 < x < 1$. Differentiate it with respect to x twice to get y' and y''

$$y = \sum_{p=0}^{\infty} a_p x^p$$
$$y' = \sum_{p=1}^{\infty} p a_p x^{p-1}$$
$$y'' = \sum_{p=2}^{\infty} p(p-1) a_p x^{p-2}$$

Substitute these expressions into the ODE

$$(1-x^2)\sum_{p=2}^{\infty} p(p-1)a_p x^{p-2} - x\sum_{p=1}^{\infty} p a_p x^{p-1} + \alpha^2 \sum_{p=0}^{\infty} a_p x^p = 0$$
$$\sum_{p=2}^{\infty} p(p-1)a_p x^{p-2} - x^2\sum_{p=2}^{\infty} p(p-1)a_p x^{p-2} - x\sum_{p=1}^{\infty} p a_p x^{p-1} + \alpha^2 \sum_{p=0}^{\infty} a_p x^p = 0$$

Bring x^2, x, and α^2 into the respective summands

$$\sum_{p=2}^{\infty} p(p-1)a_p x^{p-2} - \sum_{p=2}^{\infty} p(p-1)a_p x^p - \sum_{p=1}^{\infty} p a_p x^p + \sum_{p=0}^{\infty} \alpha^2 a_p x^p = 0$$

Because of the p and $p-1$ factors in the summand of the second sum, the lower limit can be set to start from $p = 0$. Similarly, start the third sum from $p = 0$

$$\sum_{p=2}^{\infty} p(p-1)a_p x^{p-2} - \sum_{p=0}^{\infty} p(p-1)a_p x^p - \sum_{p=0}^{\infty} p a_p x^p + \sum_{p=0}^{\infty} \alpha^2 a_p x^p = 0$$

Substitute $k = p - 2$ in the first sum and substitute $k = p$ in the others

$$\sum_{k+2=2}^{\infty} (k+2)(k+1)a_{k+2}x^k - \sum_{k=0}^{\infty} k(k-1)a_k x^k - \sum_{k=0}^{\infty} ka_k x^k + \sum_{k=0}^{\infty} \alpha^2 a_k x^k = 0$$
$$\sum_{k=0}^{\infty} (k+2)(k+1)a_{k+2}x^k - \sum_{k=0}^{\infty} k(k-1)a_k x^k - \sum_{k=0}^{\infty} ka_k x^k + \sum_{k=0}^{\infty} \alpha^2 a_k x^k = 0$$

Now that the limits and the factors of x are the same in each sum, they can be combined

$$\sum_{k=0}^{\infty} \left[(k+2)(k+1)a_{k+2}x^k - k(k-1)a_k x^k - ka_k x^k + \alpha^2 a_k x^k \right] = 0$$

Factor the summand

$$\sum_{k=0}^{\infty} \left[(k+2)(k+1)a_{k+2} - k(k-1)a_k - ka_k + \alpha^2 a_k \right] x^k = 0$$
$$\sum_{k=0}^{\infty} \left[(k+2)(k+1)a_{k+2} - (k^2 - \alpha^2)a_k \right] x^k = 0$$

The coefficients must be zero

$$(k+2)(k+1)a_{k+2} - (k^2 - \alpha^2)a_k = 0$$
$$a_{k+2} = \frac{k^2 - \alpha^2}{(k+2)(k+1)} a_k$$

Plug in enough values of k to get four terms involving a_0 and four terms involving a_1

$$k = 0: \quad a_2 = \frac{0 - \alpha^2}{(2)(1)} a_0 = -\frac{\alpha^2}{2 \cdot 1} a_0$$
$$k = 1: \quad a_3 = \frac{1 - \alpha^2}{(3)(2)} a_1 = \frac{1 - \alpha^2}{3 \cdot 2} a_1$$
$$k = 2: \quad a_4 = \frac{4 - \alpha^2}{(4)(3)} a_2 = -\frac{(4 - \alpha^2)(\alpha^2)}{4 \cdot 3 \cdot 2 \cdot 1} a_0$$
$$k = 3: \quad a_5 = \frac{9 - \alpha^2}{(5)(4)} a_3 = \frac{(9 - \alpha^2)(1 - \alpha^2)}{5 \cdot 4 \cdot 3 \cdot 2} a_1$$
$$k = 4: \quad a_6 = \frac{16 - \alpha^2}{(6)(5)} a_4 = -\frac{(16 - \alpha^2)(4 - \alpha^2)(\alpha^2)}{6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1} a_0$$
$$k = 5: \quad a_7 = \frac{25 - \alpha^2}{(7)(6)} a_5 = \frac{(25 - \alpha^2)(9 - \alpha^2)(1 - \alpha^2)}{7 \cdot 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2} a_1$$
$$\vdots$$

Generalize the coefficients

$$a_{2k} = \frac{[(2k-2)^2 - \alpha^2][(2k-4)^2 - \alpha^2] \ldots [(0)^2 - \alpha^2]}{(2k)!} a_0$$
$$a_{2k+1} = \frac{[(2k-1)^2 - \alpha^2][(2k-3)^2 - \alpha^2] \ldots [(1)^2 - \alpha^2]}{(2k+1)!} a_1$$

Therefore,

$$\begin{aligned}
y(x) &= \sum_{p=0}^{\infty} a_p x^p \\
&= a_0 + a_1 x + \sum_{k=1}^{\infty} a_{2k} x^{2k} + \sum_{k=1}^{\infty} a_{2k+1} x^{2k+1} \\
&= a_0 + a_1 x + a_0 \sum_{k=1}^{\infty} \frac{[(2k-2)^2 - \alpha^2][(2k-4)^2 - \alpha^2] \dots [(0)^2 - \alpha^2]}{(2k)!} x^{2k} \\
&+ a_1 \sum_{k=1}^{\infty} \frac{[(2k-1)^2 - \alpha^2][(2k-3)^2 - \alpha^2] \dots [(1)^2 - \alpha^2]}{(2k+1)!} x^{2k+1} \\
&= a_0 \left[1 + \sum_{k=1}^{\infty} \frac{[(2k-2)^2 - \alpha^2][(2k-4)^2 - \alpha^2] \dots [(0)^2 - \alpha^2]}{(2k)!} x^{2k} \right] \\
&+ a_1 \left[x + \sum_{k=1}^{\infty} \frac{[(2k-1)^2 - \alpha^2][(2k-3)^2 - \alpha^2] \dots [(1)^2 - \alpha^2]}{(2k+1)!} x^{2k+1} \right] \\
&= a_0 \left[1 + \frac{\alpha^2}{2 \cdot 1} x^2 - \frac{(4-\alpha^2)(\alpha^2)}{4 \cdot 3 \cdot 2 \cdot 1} x^4 - \frac{(16-\alpha^2)(4-\alpha^2)(\alpha^2)}{6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1} - \dots \right] \\
&+ a_1 \left[x + \frac{1-\alpha^2}{3 \cdot 2} x^3 + \frac{(9-\alpha^2)(1-\alpha^2)}{5 \cdot 4 \cdot 3 \cdot 2} x^5 + \frac{(25-\alpha^2)(9-\alpha^2)(1-\alpha^2)}{7 \cdot 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2} x^7 + \dots \right] \\
&= a_0 y_1(x) + a_1 y_2(x)
\end{aligned}$$

Now calculate the Wronskian of y_1 and y_2

$$W(y_1, y_2) = \begin{vmatrix} y_1 & y_2 \\ y_1' & y_2' \end{vmatrix} = y_1 y_2' - y_1' y_2$$

At $x = 0$ the Wronskian is nonzero,

$$W(y_1, y_2)(0) = (1)(1) - (0)(0) = 1$$

which means that y_1 and y_2 form a fundamental set of solutions for the ODE.

(b) Notice that if α is a certain nonnegative even number $(0, 4, \dots)$, then the series solution y_1 eventually terminates. On the other hand, if α is a certain nonnegative odd number $(1, 9, \dots)$, then the series solution y_2 eventually terminates.

(c) The problem from the book [7] at section 5.3 number 10 part (c) is quite misfit but we still wrote the problem as it is at the problem statement. The Chebyshev polynomials occur at certain α for the first kind and also the second kind, if we compute $\alpha = 0, 1, 2, 3$ the series solution doesn't terminate at $\alpha = 2, 3$. Thus, the first few polynomial solutions when y_1 and y_2 terminate are

$$\begin{aligned}
\alpha = 0: &\quad y_1(x) = 1 \\
\alpha = 1: &\quad y_2(x) = x \\
\alpha = 4: &\quad y_1(x) = 1 - 2x^2 \\
\alpha = 9: &\quad y_2(x) = x - \frac{4}{3} x^3
\end{aligned}$$

```
Differential Equation: ( 1 - 1x^2 ) y'' + (  - 1x ) y' + ( 0 ) y
Initial conditions: y(0) = 1 , y'(0) = 0

x_{0} = 0

Series solution:

y(x) = 1

Differential Equation: ( 1 - 1x^2 ) y'' + (  - 1x ) y' + ( 2 ) y
Initial conditions: y(0) = 1 , y'(0) = 0

x_{0} = 0

Series solution:

y(x) = 1 - 1*x^2 - 0.166667*x^4 - 0.0777778*x^6 - 0.0472222*x^8 - 0.0325309*x^10 - 0.0241517*x^12

Differential Equation: ( 1 - 1x^2 ) y'' + (  - 1x ) y' + ( 4 ) y
Initial conditions: y(0) = 1 , y'(0) = 0

x_{0} = 0

Series solution:

y(x) = 1 - 2*x^2
Time taken by function: 914 microseconds
```

Figure 3.68: *The power series solution for the first kind of Chebyshev polynomials that occur when $\alpha = 0$ and $\alpha = 4$ (**SymIntegration/Examples/Differential Equations/Test SymIntegration Series Solutions for Chebyshev Differential Equation/main.cpp**).*

```
Differential Equation: ( 1 - 1x^2 ) y'' + (  - 1x ) y' + ( 1 ) y
Initial conditions: y(0) = 0 , y'(0) = 1

x_{0} = 0

Series solution:

y(x) = 1*x

Differential Equation: ( 1 - 1x^2 ) y'' + (  - 1x ) y' + ( 3 ) y
Initial conditions: y(0) = 0 , y'(0) = 1

x_{0} = 0

Series solution:

y(x) = 1*x - 0.333333*x^3 - 0.1*x^5 - 0.052381*x^7 - 0.0334656*x^9 - 0.0237302*x^11 - 0.0179497*x^13

Differential Equation: ( 1 - 1x^2 ) y'' + (  - 1x ) y' + ( 5 ) y
Initial conditions: y(0) = 0 , y'(0) = 1

x_{0} = 0

Series solution:

y(x) = 1*x - 0.666667*x^3 - 0.133333*x^5 - 0.0634921*x^7 - 0.0388007*x^9 - 0.0268078*x^11 - 0.019934*x^13

Differential Equation: ( 1 - 1x^2 ) y'' + (  - 1x ) y' + ( 7 ) y
Initial conditions: y(0) = 0 , y'(0) = 1

x_{0} = 0

Series solution:

y(x) = 1*x - 1*x^3 - 0.1*x^5 - 0.0428571*x^7 - 0.025*x^9 - 0.0168182*x^11 - 0.0122902*x^13

Differential Equation: ( 1 - 1x^2 ) y'' + (  - 1x ) y' + ( 9 ) y
Initial conditions: y(0) = 0 , y'(0) = 1

x_{0} = 0

Series solution:

y(x) = 1*x - 1.33333*x^3
Time taken by function: 1359 microseconds
```

Figure 3.69: *The power series solution for the second kind of Chebyshev polynomials that occur when $\alpha = 1$ and $\alpha = 9$ (**SymIntegration/Examples/Differential Equations/Test SymIntegration Series Solutions for Chebyshev Differential Equation/main.cpp**).*

Pafnuty L. Chebyshev (1821-1894), professor at Petersburg University for 35 years and the most influential nineteenth-century Russian mathematician, founded the so-called Petersburg school, which produced a long line of distinguished mathematicians. His study of Chebyshev polynomials began about 1854 as part of an investigation of the approximation of functions by polynomials. Chebyshev is also known for his work in number theory and probability.

[SI*] We have considered the problem of finding solutions of

$$P(x)y'' + Q(x)y' + R(x)y = 0 \tag{3.283}$$

where P, Q, and R are polynomials in the neighborhood of an ordinary point x_0. Assuming that Eq. (3.283) does have a solution $y = \phi(x)$ and that ϕ has a Taylor series

$$y = \phi(x) = \sum_{n=0}^{\infty} a_n (x - x_0)^n \tag{3.284}$$

which converges for $|x - x_0| < \rho$, where $\rho > 0$, we found that the a_n can be determined by directly substituting the series (3.284) for y in Eq. (3.283).

Let us now consider how we might justify the statement that if x_0 is an ordinary point of Eq. (3.283), then there exist solutions of the form (3.284). We also consider the question of the radius of convergence of such series. In doing this, we are led to a generalization of the definition of an ordinary point.

Suppose, then, that there is a solution of Eq. (3.283) of the form (3.284). By differentiating Eq. (3.284) m times and setting x equal to x_0, we obtain

$$m! a_m = \phi^{(m)}(x_0)$$

Hence, to compute a_n in the series (3.284), we must show that we can determine $\phi^{(n)}(x_0)$ for $n = 0, 1, 2, \ldots$ from the differential equation (3.283).

Suppose that $y = \phi(x)$ is a solution fo Eq. (3.283) satisfying the initial conditions $y(x_0) = y_0$, $y'(x_0) = y_0'$. Then $a_0 = y_0$ and $a_1 = y_0'$. If we are solely interested in finding a solution to Eq. (3.283) without specifying any initial conditions, then a_0 and a_1 remain arbitrary. To determine $\phi^{(n)}(x_0)$ and the corresponding a_n for $n = 2, 3, \ldots$, we turn to Eq. (3.283). Since ϕ is a solution of Eq. (3.283), we have

$$P(x)\phi''(x) + Q(x)\phi'(x) + R(x)\phi(x) = 0$$

For the interval about x_0 for which P is nonvanishing, we can write this equation in the form

$$\phi''(x) = -p(x)\phi'(x) - q(x)\phi(x) \tag{3.285}$$

where $p(x) = \frac{Q(x)}{P(x)}$ and $q(x) = \frac{R(x)}{P(x)}$. Setting x equal to x_0 in Eq. (3.285) gives

$$\phi''(x_0) = -p(x_0)\phi'(x_0) - q(x_0)\phi(x_0)$$

Hence a_2 is given by

$$2! a_2 = \phi''(x_0) = -p(x_0)a_1 - q(x_0)a_0 \tag{3.286}$$

To determine a_3, we differentiate Eq. (3.285) and then set x equal to x_0, obtaining

$$\begin{aligned} 3! a_3 = \phi'''(x_0) &= -\left[p\phi'' + (p' + q)\phi' + q'\phi\right] \\ &= -2! p(x_0)a_2 - [p'(x_0) + q(x_0)]a_1 - q'(x_0)a_0 \end{aligned} \tag{3.287}$$

Substituting for a_2 from Eq. (3.286) gives a_3 in terms of a_1 and a_0. Since P, Q, and R are polynomials and $P(x_0) \neq 0$, all the derivatives of p and q exist at x_0. Hence, we can continue to differentiate Eq. (3.285) indefinitely, determining after each differentiation the successive coefficients $a_4, a_5, \ldots$ by setting x equal to x_0.

Notice that the important property that we used in determining the a_n was that we could

compute infintely many derivatives of the functions p and q. It might seem reasonable to relax our assumption that the functions p and q are ratios of polynomials and simply require that they be infinitely differentiable in the neighborhood of x_0. Unfortunately, this condition is too weak to ensure that we can prove the convergence of the resulting series expansion for $y = \phi(x)$. What is needed is to assume that the functions p and q are analytic at x_0; that is, they have Taylor series expansions that converge to them in some interval about the point x_0:

$$p(x) = p_0 + p_1(x - x_0) + \cdots + p_n(x - x_0)^n + \cdots = \sum_{n=0}^{\infty} p_n(x - x_0)^n \tag{3.288}$$

$$q(x) = q_0 + q_1(x - x_0) + \cdots + q_n(x - x_0)^n + \cdots = \sum_{n=0}^{\infty} q_n(x - x_0)^n \tag{3.289}$$

With this idea in mind, we can generalize the definitions of an ordinary point and a singular point of Eq. (3.283) as follows: if the functions $p(x) = \frac{Q(x)}{P(x)}$ and $q(x) = \frac{R(x)}{P(x)}$ are analytic at x_0, then the point x_0 is said to be an ordinary point of the differential equation (3.283); otherwise, it is a singular point.

Theorem 3.12: Solution for Second Order Differential Equation with Variable Coefficients

If x_0 is an ordinary point of the differential equation (3.283)

$$P(x)y'' + Q(x)y' + R(x)y = 0$$

that is, if $p(x) = \frac{Q(x)}{P(x)}$ and $q(x) = \frac{R(x)}{P(x)}$ are analytic at x_0, then the general solution of Eq. (3.283) is

$$y = \sum_{n=0}^{\infty} a_n(x - x_0)^n = a_0 y_1(x) + a_1 y_2(x) \tag{3.290}$$

where a_0 and a_1 are arbitrary, and y_1 and y_2 are two power series solutions that are analytic at x_0. The solutions y_1 and y_2 form a fundamental set of solutions. Further, the radius of convergence for each of the series solutions y_1 and y_2 is at least as large as the minimum of the radii of convergence of the series for p and q.

To see that y_1 and y_2 are a fundamental set of solutions, note that they have the form $y_1(x) = 1 + b_2(x - x_0)^2 + \ldots$ and $y_2(x) = (x - x_0) + c_2(x - x_0)^2 + \ldots$, where $b_2 + c_2 = a_2$. Hence y_1 satisfies the initial conditions $y_1(x_0) = 1$, $y_1'(x_0) = 0$, and y_2 satisfies the initial conditions $y_2(x_0) = 0$, $y_2'(x) = 1$. Thus $W(y_1, y_2)(x_0) = 1$.

What is important is that there is a series solution of the form (3.284) and that the radius of convergence of the series solution cannot be less than the smaller of the radii of convergence of the series for $p(x)$ and $q(x)$; hence we need only determine these.

This can be done in either of two ways. Again, one possibility is simply to compute the power series for $p(x)$ and $q(x)$ and then to determine the radii of convergence by using one of the convergence tests for infinite series. However, there is an easier way when P, Q, and R are polynomials. It is shown in the theory of functions of a complex variable that the ratio of two polynomials, say, $\frac{Q(x)}{P(x)}$, has a convergent power series expansion about a point $x = x_0$ if $P(x_0) \neq 0$. Further, if we assume that any factors common to $Q(x)$ and $P(x)$ have been canceled, then the radius of convergence of the power series for $\frac{Q(x)}{P(x)}$ about the point x_0 is precisely the distance from x_0 to the nearest zero of $P(x)$. In determining this

distance, we must remember that $P(x) = 0$ may have complex roots and that these must also be considered.

[SI*] **Example:**
What is the radius of convergence of the Taylor series for $(1 + x^2)^{-1}$ about $x = 0$?
Solution:
One way to proceed is to find the Taylor series in question, namely,

$$\frac{1}{1+x^2} = 1 - x^2 + x^4 - x^6 + \cdots + (-1)^n x^{2n} + \ldots$$

Then it can be verified by the ratio test that $\rho = 1$. Another approach is to note that the zeros of $1 + x^2$ are $x = \pm i$. Since the distance in the complex plane from 0 to i or to $-i$ is 1, the radius of convergence of the power series about $x = 0$ is 1.

[SI*] **Example:**
What is the radius of convergence of the Taylor series for $(x^2 - 2x + 2)^{-1}$ about $x = 0$? about $x = 1$?
Solution:
First notice that

$$x^2 - 2x + 2 = 0$$

has solutions $x = 1 \pm i$. The distance in the complex plane from $x = 0$ to either $x = 1 + i$ or $x = 1 - i$ is $\sqrt{2}$; hence the radius of convergence of the Taylor series expansion $\sum_{n=0}^{\infty} a_n x^n$ about $x = 0$ is $\sqrt{2}$.
The distance in the complex plane from $x = 1$ to either $x = 1 + i$ or $x = 1 - i$ is 1; hence the radius of convergence of the Taylor series expansion $\sum_{n=0}^{\infty} b_n (x-1)^n$ about $x = 1$ is 1.
In SymIntegration you can use the function **complexdistance(**z_1, z_2**)** to compute distance in the complex plane with z_1 and z_2 as the complex numbers with data type **std::complex<double>**.

[SI*] **Example:**
Determine a lower bound for the radius of convergence of series solutions about $x = 0$ for the Legendre equation

$$(1 - x^2)y'' - 2xy' + \alpha(\alpha + 1)y = 0$$

where α is a constant.
Solution:
Note that $P(x) = 1 - x^2, Q(x) = -2x$, and $R(x) = \alpha(\alpha + 1)$ are polynomials and that the zeros of P, namely, $x = \pm 1$, are a distance 1 from 0. Hence a series solution of the form $\sum_{n=0}^{\infty} a_n x^n$ converges at least for $|x| < 1$, and possibly for larger values of x. Indeed, it can be shown that if α is a positive integer, one of the series solutions terminates after a finite number of terms and hence converges not just for $|x| < 1$ but for all x. For example, if $\alpha = 1$, the polynomial solution is $y = x$.

[SI*] **Example:**
Determine a lower bound for the radius of convergence of series solutions of the differential equation

$$(1 + x^2)y'' + 2xy' + 4x^2 y = 0$$

about the point $x = 0$; about the point $x = -\frac{1}{2}$.
Solution:
Note that $P(x) = 1 + x^2, Q(x) = 2x$, and $R(x) = 4x^2$ are polynomials, and $P(x)$ has zeroes at $x = \pm i$. The distance in the complex plane from 0 to $\pm i$ is 1, and from $-\frac{1}{2}$ to $\pm i$ is $\sqrt{1 + \frac{1}{4}} = \frac{\sqrt{5}}{2}$. Hence in the first case the series $\sum_{n=0}^{\infty} a_n x^n$ converges at least for $|x| < 1$, and

in the second case the series $\sum_{n=0}^{\infty} b_n \left(x + \frac{1}{2}\right)^n$ converges at least for $\left|x + \frac{1}{2}\right| < \frac{\sqrt{5}}{2}$.
An interesting observation that we can make about the differential equation follows from Theorems (3.1) and (3.12). Suppose that initial conditions $y(0) = y0$ and $y'(0) = dy0$ are given. Since $1 + x^2 \neq 0$ for all $x \setminus i$, we know from Theorem (3.1) that there exists a unique solution of the initial value problem on $-\infty < x < \infty$. On the other hand, Theorem (3.12) only guarantees a series solution of the form $\sum_{n=0}^{\infty} a_n x^n$ (with $a_0 = y0$, $a_1 = dy0$) for $-1 < x < 1$. The unique solution on the interval $-\infty < x < \infty$ may not have a power series about $x = 0$ that converges for all x.

Definition 3.1: Analytic Function

A function is analytic at $x = a$ if it can be written as a converging power series centered at a. This means that in a small open space around a, the function equals its own Taylor series.
You can find an open range around a where $f(x) = \sum_{n=0}^{\infty} c_n (x - a)^n$. The function $f(x)$ has derivatives of all orders at that point. The Taylor series adds up correctly to match the exact value of the function near a.
For real numbers function, being infinitely differentiable (C^∞) is not enough to make a real function analytic at a point. It must explicitly match a converging power series locally. A classic counterexample is the function $f(x) = e^{-\frac{1}{x^2}}$ for $x \neq 0$ and $f(0) = 0$, which has derivatives of all orders equal to zero at $x = 0$, but its Taylor series is identically zero and does not match the function nearby.

$$f(x) = \begin{cases} e^{-\frac{1}{x^2}}, & \text{if } x \neq 0 \\ 0, & \text{if } x = 0 \end{cases}$$

Every derivative $f^{(n)}(0)$ equals 0 for all integers $n \geq 0$. The Maclaurin series is $0 + 0x + 0x^2 + \cdots = 0$. For $x \neq 0$, the function is strictly positive ($f(x) > 0$), so the function does not equal its Taylor series for any $x \neq 0$.

The reasons why smoothness fails for analyticity

- **Taylor Series Gap:** An infinitely differentiable function has a Taylor series at a point, but the series might not converge to the function itself away from that point.
- **Radius of Convergence:** For a function to be analytic, its Taylor series must have a non-zero radius of convergence where it equals the function.
- **Complex Extension:** Real analytic functions can be extended to holomorphic functions in an open neighborhood in the complex plane, which C^∞ functions do not generally allow.

Guide 3.1: The Ratio Test and the Root Test

To compute the radius of convergence (R) of a Taylor series you must implement either the ratio test or the root test. Because computers cannot calculate limits to infinity, you approximate the limit by evaluating the formula at a large value of n (such as $n = 1000$).

The ratio test formula

$$R \lim_{n\to\infty} \left| \frac{a_n}{a_{n+1}} \right|$$

where a_n represents the coefficient of the n-th term of the series. The ratio test works perfectly for series containing factorials or exponents (e.g., $e^x, \sin(x), \cos(x)$) where the relationship between consecutive coefficients is clean.

The root test formula

$$R \lim_{n\to\infty} \frac{1}{\sqrt[n]{|a_n|}}$$

When we want to compute the radius of convergence with C++ there are crucial numerical limitations:

- **Floating-Point Overflow:** Factorials grow too fast for standard data types (171! overflows a 64-bit **double**). Do not compute a_n and a_{n+1} separately if they contain factorials. Instead, algebraically simplify the ratio expression $\left|\frac{a_n}{a_{n+1}}\right|$ before writing the code.
- **Alternating Signs:** Always wrap your coefficient math inside **std:abs()** because the radius of convergence depends only on the absolute values of the coefficients.
- **Step Size Restrictions:** Some Taylor series have zero coefficients for alternating odd/even terms (like $\sin(x)$ or $\cos(x)$). For those, you must rewrite your index tracker so n maps strictly to the non-zero terms (e.g., using $2n + 1$ update).

[SI*] **Euler Equations; Regular Singular Points**

We will begin to consider how to solve equations of the form

$$P(x)y'' + Q(x)y' + R(x)y = 0 \tag{3.291}$$

in the neighborhood of a singular point x_0. Recall that if the functions P, Q, and R are polynomials having no common factors, then the singular points of Eq. (3.291) are the points for which $P(x) = 0$.

Euler Equations

A relatively simple differential equation that has a singular point is the Euler equation

$$L[y] = x^2y'' + \alpha xy' + \beta y = 0 \tag{3.292}$$

where α and β are real constants. In this $P(x) = x^2$, so $x = 0$ is the only singular point for Eq. (3.292); all other points are ordinary points. For convenience, we first consider the interval $x > 0$; later we extend our results to the interval $x < 0$.

Observe that $(x^r)' = rx^{r-1}$ and $(x^r)'' = r(r-1)x^{r-2}$. Hence, if we assume that Eq. (3.292) has a solution of the form

$$y = x^r \tag{3.293}$$

then we obtain

$$\begin{aligned} L[x^r] &= x^2(x^r)'' + \alpha x(x^r)' + \beta x^r \\ &= x^r[r(r-1) + \alpha r + \beta] \end{aligned} \tag{3.294}$$

If r is a root of the quadratic equation

$$F(r) = r(r-1) + \alpha r + \beta = 0 \tag{3.295}$$

then $L[x^r]$ is zero, and $y = x^r$ is a solution of Eq. (3.292). The roots of Eq. (3.295)are

$$r_1, r_2 = \frac{-(\alpha - 1) \pm \sqrt{(\alpha - 1)^2 - 4\beta}}{2} \tag{3.296}$$

and $F(r) = (r - r_1)(r - r_2)$. As for second order linear equations with constant coefficients, we consider separately the cases in which the roots are real and different, real but equal, and complex conjugates. Indeed, the entire discussion of Euler equations is similar to the treatment of second order linear equations with constant coefficients, with e^{rx} replaced by x^r.

- **Real, Distinct Roots**
  If $F(r) = 0$ has real roots r_1 and r_2, with $r_1 \neq r_2$, then $y_1(x) = x^{r_1}$ and $y_2(x) = x^{r_2}$ are solutions of Eq. (3.292). Since

  $$W(x^{r_1}, x^{r_2}) = (r_2 - r_1)x^{r_1 + r_2 - 1}$$

  is nonvanishing for $r_1 \neq r_2$ and $x > 0$, it follows that the general solution of Eq. (3.292) is

  $$y = c_1 x^{r_1} + c_2 x^{r_2}, \quad x > 0 \tag{3.297}$$

  Note that if r is not a rational number, then x^r is defined by $x^r = e^{r \ln\ x}$.
- **Equal Roots**
  If the roots r_1 and r_2 are equal, then we obtain only one solution $y_1(x) = x^{r_1}$ of the assumed form. A second solution can be obtained by the method of reduction of order, but we consider an alternative method. Since $r_1 = r_2$, it follows that $F(r) = (r - r_1)^2$. Thus in this case not only does $F(r_1) = 0$ but also $F'(r_1) = 0$. This suggests differentiating Eq. (3.294) with respect to r and then setting r equal to r_1. Differentiating Eq. (3.294) with respect to r gives

  $$\frac{\partial}{\partial r} L[x^r] = \frac{\partial}{\partial r}[x^r F(r)]$$

  Substituting for $F(r)$, interchanging differentiation with respect to x and with respect to r, and noting that $\frac{\partial(x^r)}{\partial r} = x^r \ln\ x$, we obtain

  $$L[x^r \ln\ x] = (r - r_1)^2 x^r \ln\ x + 2(r - r_1)x^r \tag{3.298}$$

  The right side of Eq. (3.300) is zero for $r = r_1$; consequently,

  $$y_2(x) = x^{r_1} \ln\ x, \quad x > 0 \tag{3.299}$$

  is a second solution of Eq. (3.292). By evaluating the Wronskian, we find that

  $$W(x^{r_1}, x^{r_1} \ln\ x) = x^{2r_1 - 1}$$

  Hence x^{r_1} and $x^{r_1} \ln\ x$ are a fundamental set of solutions for $x > 0$, and the general solution of Eq. (3.292) is

  $$y = (c_1 + c_2 \ln\ x)x^{r_1}, \quad x > 0 \tag{3.300}$$

- **Complex Roots**
  Suppose that the roots r_1 and r_2 of Eq. (3.295) are complex conjugates, say, $r_1 = \lambda + i\mu$ and $r_2 = \lambda - i\mu$, with $\mu \neq 0$. We must now explain what is meant by x^r when r is complex. Remembering that

$$x^r = e^{r \ln\ x} \tag{3.301}$$

  when $x > 0$ and r is real, we can use this equation to define x^r when r is complex. Then, using Euler's formula for $e^{i\mu \ln\ x}$, we obtain

$$\begin{aligned} x^{\lambda+i\mu} = e^{(\lambda+i\mu)\ln\ x} &= e^{\lambda\ \ln\ x} e^{i\mu\ \ln\ x} \\ &= x^\lambda[\cos(\mu\ \ln\ x) + i\sin(\mu\ \ln\ x)], \quad x > 0 \end{aligned}$$

  with this definition of x^r for complex values of r, it can be verified that the usual laws of algebra and the differential calculus hold, and hence x^{r_1} and x^{r_2} are indeed solutions of Eq. (3.292). The general solution of Eq. (3.292) is

$$y = c_1 x^{\lambda+i\mu} + c_2 x^{\lambda-i\mu} \tag{3.302}$$

  The disadvantage of this expression is that the functions $x^{\lambda+i\mu}$ and $x^{\lambda-i\mu}$ are complex-valued. Recall that we had a similar situation for the second order differential equation with constant coefficients when the roots of the characteristic equation were complex. In the same way as we did then, we observe that the real and imaginary parts of $x^{\lambda+i\mu}$, namely,

$$x^\lambda \cos(\mu\ \ln\ x) \quad \text{and} \quad x^\lambda \sin(\mu\ \ln\ x) \tag{3.303}$$

  are also solutions of Eq. (3.292). A straightforward calculation shows that

$$W[x^\lambda \cos(\mu\ \ln\ x), x^\lambda \sin(\mu\ \ln\ x)] = \mu x^{2\lambda-1}$$

  Hence these solutions form a fundamental set of solutions for $x > 0$, and the general solution of Eq. (3.292) is

$$y = c_1 x^\lambda \cos(\mu\ \ln\ x) + c_2 x^\lambda \sin(\mu\ \ln\ x), \quad x > 0 \tag{3.304}$$

Now let us consider the qualitative behavior of the solutions of Eq. (3.292) near the singular point $x = 0$. This depends entirely on the values of the exponents r_1 and r_2. First, if r is real and positive, then $x^r \to 0$ as x tends to zero through positive values. On the other hand, if r is real and negative, then x^r becomes unbounded. Finally, if $r = 0$, then $x^r = 1$.
If r is complex, then a typical solution is $x^\lambda \cos(\mu \ln(x))$. This functions becomes unbounded or approaches zero if λ is negative or positive, respectively, and also oscillates more and more rapidly as $x \to 0$. If $\lambda = 0$, the oscillation is of constant amplitude.
Finally, if there are repeated roots, then one solution is of the form $x^r \ln(x)$, which tends to zero if $r > 0$ and becomes unbounded if $r \leq 0$.
The extension of the solutions of Eq. (3.292) into the interval $x < 0$ can be carried out in a relatively straightforward manner. The difficulty lies in understanding what is meant by x^r when x is negative and r is not an integer; similarly, $\ln(x)$ has not been defined for $x < 0$. The solutions of the Euler equation what we have given for $x > 0$ can be shown to be valid for $x < 0$, but in general they are complex-valued.

Guide 3.2: Mathematical Classification Framework for Series Solutions

Suppose we have a homogeneous second order linear differential equation

$$P(x)y'' + Q(x)y' + R(x)y = 0$$

Now we want to classify a point x_0, thus we will perform these computations:

1. **Ordinary Point:** If $P(x_0) \neq 0$, the equation can be solved using a standard Taylor power series.
2. **Singular Point:** If $P(x_0) = 0$.
3. **Regular Singular Point:** If $P(x_0) = 0$, but the limits $p_0 = \lim_{x\to x_0}(x - x_0)\frac{Q(x)}{P(x)}$ and $q_0 = \lim_{x\to x_0}(x - x_0)^2\frac{R(x)}{P(x)}$ both exist and are finite, x_0 is a regular singular point. For polynomials, this means the root multiplicity of x_0 in $P(x)$ must not exceed the multiplicity in $Q(x)$ by more than 1, and in $R(x)$ by more than 2. This guarantees it can be solved using the Frobenius method.
4. **Euler-Cauchy Special Case:** If the differential equation simplifies precisely to $a(x - x_0)^2y'' + b(x - x_0)y' + cy = 0$, where a, b, c are constants, thus it is an Euler-Cauchy equation, which yields an exact algebraic solution without requiring a full infinite recurrence relation.
5. **Irregular Singular Point:** Any singular point that fails to meet the criteria for a regular singular point.

Guide 3.3: Solution of the Euler-Cauchy Differential Equation

The general solution of the Euler-Cauchy differential equation $x^2y'' + axy' + by = 0$ with complex roots $r = \lambda + i\mu$ around a singular point x_0 is

$$y(x) = (x - x_0)^\lambda \left[C_1 \cos(\mu \ln|x - x_0|) + C_2 \sin(\mu \ln|x - x_0|)\right]$$

with C_1 and C_2 are constants that can be determined with the initial conditions. The two linearly independent fundamental solutions are

$$y_1(x) = (x - x_0)^\lambda \cos(\mu \ln|x - x_0|)$$
$$y_2(x) = (x - x_0)^\lambda \sin(\mu \ln|x - x_0|)$$

For the general equation $P(x)y'' + Q(x)y' + R(x)y = 0$ to be an Euler-Cauchy equation, the coefficients must take the form:

$$P(x) = x^2$$
$$Q(x) = ax$$
$$R(x) = b$$

To solve it, we assume a solution of the form $y(x) = x^r$ substituting this into the differential

equation yields the auxiliary (characteristic) equation:

$$r(r-1)+ar+b=0$$
$$r^2+(a-1)r+b=0$$

When the discriminant $(a-1)^2-4b<0$, the quadratic formula yields a pair of conjugate complex roots:

$$r_1,r_2=\lambda\pm i\mu$$

Substituting these roots back into the assumed solution $y=x^r$ gives:

$$y(x)=C_1x^{\lambda+i\mu}+C_2x^{\lambda-i\mu}$$

Using Euler's identity ($e^{i\theta}=\cos(\theta)+i\sin(\theta)$) and the logarithmic identity $x^k=e^{k\ln(x)}$, we rewrite the complex variable:

$$x^{i\mu}=e^{i\mu\ln(x)}=\cos(\mu\ln(x))+i\sin(\mu\ln(x))$$

By separating the real and imaginary parts to ensure real-valued solutions, we obtain the final general solution:

$$y(x)=x^{\lambda}\left[C_1\cos(\mu\ln(x))+C_2\sin(\mu\ln(x))\right]$$

[SI*] **Series Solution Near a Regular Singular Point**

Guide 3.4: Recurrence Relation for Coefficients $a_n(r)$ with Frobenius Method (Case 1)

To find the series coefficients for a homogeneous second order differential equation at a regular singular point with Frobenius method when the roots are distinct and differ by an integer ($r_1 - r_2 \in \mathbb{Z}$), we will consider this differential equation to be used:

$$P(x)y'' + Q(x)y' + R(x)y = 0$$

then we changed the differential equation into this Frobenius form:

$$x^2y'' + xp(x)y' + q(x)y = 0$$

where $p(x) = \sum_{n=0}^{\infty} p_n x^n$ and $q(x) = \sum_{n=0}^{\infty} q_n x^n$ are analytic at $x = 0$.

We assume a solution of the form $y(x) = \sum_{n=0}^{\infty} a_n(r)x^{n+r}$. Substituting this into the differential equation yields.

1. **The Indicial Equation**
   For $n = 0$:

   $$f(r) = r(r-1) + p_0 r + q_0 = r^2 + (p_0 - 1)r + q_0 = 0$$

   This yields two distinct roots, r_1 and r_2 that differ by an integer.

2. **The Recurrence Relation**
   By equating the coefficient of x^{n+r} to zero for $n \geq 1$, we get a general recurrence relation for the coefficients $a_n(r)$ corresponding to a root r:

   $$f(n + r_1)a_n(r_1) + \sum_{k=1}^{n} [p_k(n - k + r_1) + q_k]a_{n-k}(r_1) = 0$$

   or more explicitly expanded:

   $$f(n + r_1)a_n(r_1) = -\sum_{k=1}^{n} [p_k(n - k + r_1) + q_k]a_{n-k}(r_1)$$

   Because $f(n + r_1) \neq 0$ for all $n \geq 1$ when the roots by an integer, we can isolate $a_n(r)$ to get the general formula for the coefficients:

   $$a_n(r_1) = -\frac{1}{f(n + r_1)} \sum_{k=1}^{n} a_{n-k}(r_1)[p_k(n - k + r_1) + q_k]$$

   where $f(n + r_1) = (n + r_1)(n + r_1 - 1) + p_0(n + r_1) + q_0$.

Guide 3.5: Recurrence Relation for Coefficients $a_n(r)$ with Frobenius Method (Case 1)

3. **Final Solutions**
   Use $r = r_1$ to generate the first solution, we just calculate it normally like Case 1 using the recurrence relation:

$$y_1(x) = \sum_{n=0}^{\infty} a_n(r_1)x^{n+r_1}, \quad a_0 = 1$$

   Use $r = r_2$, the second solution (using r_2) may either contain a logarithm term or truncate successfully, thus the general form of the second solution is:

$$y_2(x) = Cy_1(x)\ln(x) + \sum_{n=0}^{\infty} b_n(r_2)x^{n+r_2}, \quad b_0 = 1$$

   If the regular singular point $x_0 \neq 0$ then the general solutions are:

$$y_1(x) = (x - x_0)_1^r \sum_{n=0}^{\infty} a_n(r_1)(x - x_0)^n, \quad a_0 = 1$$

$$y_2(x) = Cy_1(x)\ln(x - x_0) + (x - x_0)^{r_2} \sum_{n=0}^{\infty} b_n(r_2)(x - x_0)^n, \quad b_0 = 1$$

   To compute the coefficients b_n without manual derivative matching, we can substitute $y(x, r) = x^r \sum_{n=0}^{\infty} a_n(r)x^n$ directly into the ODE, treating the coefficients a_n as functions of an undefined variable r. We set $a_0(r) = r - r_2$. By scaling a_0 this way, it eliminates the division-by-zero singularity at $n = N$ when evaluating the series at $r = r_2$. To find b_n and C, define $A_n(r) = (r - r_2)a_n(r)$. Thus, the resulting coefficients b_n are given by:

$$b_n = \frac{\partial}{\partial r} A_n(r)\big|_{r=r_2}$$

   The constant C may turn out to be zero in certain problems, which allows r_2 to directly yield a second full series solution without a logarithm.

$$C = \lim_{r \to r_2} A_N(r), \quad N = r_1 - r_2,\ r_1 > r_2$$

Guide 3.6: Recurrence Relation for Coefficients $a_n(r)$ with Frobenius Method (Case 2)

To find the series coefficients for a homogeneous second order differential equation at a regular singular point with Frobenius method when the roots are distinct and do not differ by an integer ($r_1 - r_2 \notin \mathbb{Z}$), we will consider this differential equation to be used:

$$P(x)y'' + Q(x)y' + R(x)y = 0$$

then we changed the differential equation into this Frobenius form:

$$x^2y'' + xp(x)y' + q(x)y = 0$$

where $p(x) = \sum_{n=0}^{\infty} p_n x^n$ and $q(x) = \sum_{n=0}^{\infty} q_n x^n$ are analytic at $x = 0$.

We assume a solution of the form $y(x) = \sum_{n=0}^{\infty} a_n(r)x^{n+r}$. Substituting this into the differential equation yields.

1. **The Indicial Equation**
   For $n = 0$:
   $$f(r) = r(r-1) + p_0 r + q_0 = 0$$
   This yields two distinct roots, r_1 and r_2 that do not differ by an integer.

2. **The Recurrence Relation**
   By equating the coefficient of x^{n+r} to zero for $n \geq 1$, we get a general recurrence relation for the coefficients $a_n(r)$ corresponding to a root r:
   $$f(n+r)a_n(r) + \sum_{k=1}^{n} [p_k(n-k+r) + q_k]a_{n-k}(r) = 0$$
   or more explicitly expanded:
   $$f(n+r)a_n(r) = -\sum_{k=1}^{n} [p_k(n-k+r) + q_k]a_{n-k}(r)$$
   Because $f(n+r) \neq 0$ for all $n \geq 1$ when the roots do not differ by an integer, we can isolate $a_n(r)$ to get the general formula for the coefficients:
   $$a_n(r) = -\frac{1}{f(n+r)} \sum_{k=1}^{n} a_{n-k}(r)[p_k(n-k+r) + q_k]$$
   where $f(n+r) = (n+r)(n+r-1) + p_0(n+r) + q_0$.

Guide 3.7: Recurrence Relation for Coefficients $a_n(r)$ with Frobenius Method (Case 2)

3. **Final Solutions**
   For a regular singular point x_0, use $r = r_1$ to generate the first solution:

$$y_1(x) = (x - x_0)^{r_1} \sum_{n=0}^{\infty} a_n(r_1)(x - x_0)^n, \quad a_0 = 1$$

   Use $r = r_2$ to generate the second solution:

$$y_2(x) = (x - x_0)^{r_2} \sum_{n=0}^{\infty} b_n(r_2)(x - x_0)^n, \quad b_0 = 1$$

Alternative for Case 2:
If we want to use the arbitrary differential equation in this general form $P(x)y'' + Q(x)y' + R(x)y = 0$ then the indicial equation will be

$$f(r) = r(r-1)p_0 + rp_1 + p_2$$

If we want to use the arbitrary differential equation in this general form $P(x)y'' + Q(x)y' + R(x)y = 0$, for any choice of root r, the general formula to compute the coefficient a_n is

$$a_n(r) = \frac{-1}{f(n+r)} \sum_{k=0}^{n-1} a_k \cdot \mathbb{M}(k, n, r)$$

where $\mathbb{M}(k, n, r)$ is a dynamic multiplier combining the polynomial coefficients of $P(x), Q(x), R(x)$ evaluated at the shifted indexes.

Guide 3.8: Recurrence Relation for Coefficients $a_n(r)$ with Frobenius Method (Case 3)

To find the series coefficients for a homogeneous second order differential equation at a regular singular point with Frobenius method when the roots are repeated, we will consider this differential equation to be used:

$$P(x)y'' + Q(x)y' + R(x)y = 0$$

then we changed the differential equation into this Frobenius form:

$$x^2y'' + xp(x)y' + q(x)y = 0$$

where $p(x) = \sum_{n=0}^{\infty} p_n x^n$ and $q(x) = \sum_{n=0}^{\infty} q_n x^n$ are analytic at $x = 0$.

We assume a solution of the form $y(x) = \sum_{n=0}^{\infty} a_n(r)x^{n+r}$. Substituting this into the differential equation yields.

1. **The Indicial Equation**
   For $n = 0$:
   $$f(r) = r(r-1) + p_0 r + q_0 = 0$$
   This yields the repeated roots, $r_1 = r_2$. Thus, $f(r) = (r - r_1)^2 = 0$, meaning the derivative satisfies $\frac{\partial F}{\partial r}\big|_{r=r_1} = 0$.

2. **The Recurrence Relation**
   By equating the coefficient of x^{n+r} to zero for $n \geq 1$, we get a general recurrence relation for the coefficients $a_n(r)$ corresponding to a root r:
   $$f(n+r)a_n(r) + \sum_{k=0}^{n-1} a_k(r)[p_{n-k}(k+r) + q_{n-k}] = 0$$
   or more explicitly expanded:
   $$f(n+r)a_n(r) = -\sum_{k=0}^{n-1} a_k(r)[p_{n-k}(k+r) + q_{n-k}]$$
   we can isolate $a_n(r)$ to get the general formula for the coefficients:
   $$a_n(r) = -\frac{1}{f(n+r)} \sum_{k=0}^{n-1} a_k(r)[p_{n-k}(k+r) + q_{n-k}]$$
   where $f(n+r) = (n+r)(n+r-1) + p_0(n+r) + q_0$. We set $a_0(r) = 1$.

Guide 3.9: Recurrence Relation for Coefficients $a_n(r)$ with Frobenius Method (Case 3)

3. **Final Solutions**
   For a regular singular point x_0, use $r = r_1$ to generate the first solution:

$$y_1(x) = (x - x_0)^{r_1} \sum_{n=0}^{\infty} a_n(r_1)(x - x_0)^n, \quad a_0 = 1$$

   Use $r = r_2$ to generate the second solution (the second solution requires a logarithmic shift):

$$y_2(x) = y_1(x) \ln(x - x_0) + (x - x_0)^{r_2} \sum_{n=0}^{\infty} a_n'(r_2)(x - x_0)^n, \quad a_0' = 1$$

   where $a_n'(r_2) = \frac{da_n(r)}{dr}\big|_{r=r_1}$ are computed by differentiating the recurrence relation for $a_n(r)$ with respect to r and evaluating it at the repeating root $r = r_1 = r_2$. We can compute $a_n'(r_2)$ with the help of automatic differentiation (dual numbers).

Guide 3.10: Recurrence Relation for Coefficients $a_n(r)$ with Frobenius Method (Case 4)

To find the series coefficients for a homogeneous second order differential equation at a regular singular point with Frobenius method when the roots are complex conjugate ($r_1, r_2 \in \mathbb{C}$), we will consider this differential equation to be used:

$$P(x)y'' + Q(x)y' + R(x)y = 0$$

then we changed the differential equation into this Frobenius form:

$$x^2y'' + xp(x)y' + q(x)y = 0$$

where $p(x) = \sum_{n=0}^{\infty} p_n x^n$ and $q(x) = \sum_{n=0}^{\infty} q_n x^n$ are analytic at $x = 0$.

We assume a solution of the form $y(x) = \sum_{n=0}^{\infty} a_n(r)x^{n+r}$. Substituting this into the differential equation yields.

1. **The Indicial Equation**
   For $n = 0$:
   $$f(r) = r(r-1) + p_0 r + q_0 = 0$$
   This yields two distinct roots, r_1 and r_2 are complex conjugate ($r_1, r_2 = \lambda \pm i\mu$).

2. **The Recurrence Relation**
   Substituting the full series yields the general algebraic recurrence relation for coefficients $a_n(r)$ for $n \geq 1$:
   $$f(n+r)a_n(r) + \sum_{k=1}^{n} [p_k(n-k+r) + q_k]a_{n-k}(r) = 0$$
   or more explicitly expanded:
   $$f(n+r)a_n(r) = -\sum_{k=1}^{n} [p_k(n-k+r) + q_k]a_{n-k}(r)$$
   where the shifted indicial polynomial is:
   $$f(n+r) = (n+r)(n+r-1) + p_0(n+r) + q_0$$
   Solving the recurrence relation explicitly by isolating $a_n(r)$ gives the general computational formula:
   $$a_n(r) = -\frac{1}{f(n+r)} \sum_{k=1}^{n} a_{n-k}(r)[p_k(n-k+r) + q_k]$$
   where $f(n+r) = (n+r)(n+r-1) + p_0(n+r) + q_0$.

Guide 3.11: Recurrence Relation for Coefficients $a_n(r)$ with Frobenius Method (Case 4)

3. **Final Solutions**
   The resulting complex series solution splits into real and imaginary parts via Euler's formula to form two real-valued linearly independent solutions.
   The complex roots $r_1, r_2 = \lambda \pm i\mu$ will make the solutions involve Euler's formula transformation:

$$x^{\lambda \pm i\mu} = x^{\lambda}[\cos(\mu \ln(x)) \pm i \sin(\mu \ln(x))]$$

   For $x > 0$, the general solution is expressed as:

$$y(x) = C_1 y_1(x) + C_2 y_2(x)$$

   The two linearly independent solutions $y_1(x)$ and $y_2(x)$ are:

$$y_1(x) = x^{\lambda}\left[\cos(\mu \ln(x)) \sum_{n=0}^{\infty} u_n x^n - \sin(\mu \ln(x)) \sum_{n=0}^{\infty} v_n x^n\right]$$
$$y_2(x) = x^{\lambda}\left[\sin(\mu \ln(x)) \sum_{n=0}^{\infty} u_n x^n + \cos(\mu \ln(x)) \sum_{n=0}^{\infty} v_n x^n\right]$$

   Here, u_n and v_n represent the real and imaginary parts of the complex coefficients a_n.

Derivation Steps

- **Select one root:** Use $r_1 = \lambda + i\mu$.
- **Find complex coefficients:** Substitute r_1 into the recurrence relation.
- **Split the coefficients:** Separate a_n into $u_n + iv_n$.
- **Expand the Exponent:** Rewrite $x^{\lambda+i\mu}$ as $x^{\lambda}e^{i\mu \ln(x)}$.
- **Apply Euler's formula:** Convert $e^{i\mu \ln(x)}$ to $\cos(\mu \ln(x)) + i \sin(\mu \ln(x))$.
- **Extract real component:** This forms the $y_1(x)$ basis function.
- **Extract imaginary component:** This forms the $y_2(x)$ basis function.

If the homogeneous second order differential equation has complex roots then the solutions obtained from the Frobenius method and the Euler-Cauchy equation share the exact same structural form. But, they differ in generality because the Frobenius method applies to a broader class of differential equations.

- **Euler-Cauchy Equation**
  An Euler equation is a specific type of homogeneous linear ordinary differential equation where the coefficients of y'', y', and y are fixed multiples of x^2, x. and 1, respectively. For an Euler-Cauchy equation, the assumed trial solution $y = x^r$ directly yields the exact general solution without needing an infinite series because all higher-order recurrence coefficients a_n turn out to be zero for $n \geq 1$.
- **Frobenius Method**
  The Frobenius method is a general technique used for more complex differential equations with a regular singular point. It assumes a solution of the form $y = x^r \sum_{n=0}^{\infty} a_n x^n$. When the indicial roots are complex conjugates, you plug one root (e.g., $r = \lambda \pm i\mu$)

into the Frobenius series. Solving the recurrence relation gives a complex-valued power series. Taking the real and imaginary parts of this single complex Frobenius solution yields two real, linearly independent solutions that match the functional behavior seen in the Euler equation approach.

The Euler-Cauchy equation is a special, simplified case where the Frobenius series truncates to just the leading term (a_0). For general differential equations, the Frobenius method adds an infinite power series $\sum_{n=0}^{\infty} a_n x^n$ attached to the front of x^r term to account for non-constant coefficient variations.
The resulting function spaces spanned by the two independent real solutions are identical in form $x^\lambda \cos(\mu \ln(x))$ and $x^\lambda \sin(\mu \ln(x))$ multiplied by their respective analytic power series expansions.

Condition	Classification	Method to Use
$P(x_0) \neq 0$	Ordinary point	Standard power series : $y = \sum a_n (x - x_0)^n$
$P(x_0) = 0$, with $\lim_{x \to x_0} (x - x_0) \frac{Q(x)}{P(x)}$ and $\lim_{x \to x_0} (x - x_0)^2 \frac{R(x)}{P(x)}$ are finite	Regular Singular Point	Frobenius method: $\sum a_n (x - x_0)^{n+r}$
$\deg(P) = 2, \deg(Q) = 1, \deg(R) = 0$ (centered at x_0)	Euler-Cauchy equation	Algebraic guess: $y = (x - x_0)^m$
$\lim_{x \to x_0} (x - x_0) \frac{Q(x)}{P(x)}$ and $\lim_{x \to x_0} (x - x_0)^2 \frac{R(x)}{P(x)}$ do not exist / diverge	Irregular singular point	Frobenius fails (requires advanced asymptotics)

Table 3.3: *Classification logic to determine how to compute the solution of the homogeneous second order differential equation with variable coefficients.*

To determine whether a point x_0 is a regular singular point or an irregular singular point for the differential equation of this form

$$P(x)y'' + Q(x)y' + R(x)y = 0$$

we look at the limits of the standard Frobenius coefficients.
Specifically, if x_0 is a singular point if $P(x_0) = 0$. It is a regular singular point if both of the following limits exist and are finite:

$$\lim_{x \to x_0} (x - x_0) \frac{Q(x)}{P(x)} < \infty$$

$$\lim_{x \to x_0} (x - x_0)^2 \frac{R(x)}{P(x)} < \infty$$

If either limit is infinite, x_0 is an irregular singular point.
When working with polynomials, this means $(x - x_0)$ must divide into $P(x)$ at most once more than it divides $Q(x)$, and at most twice more than it divides $R(x)$. To handle shifted

points ($x_0 \neq 0$), we can shift our polynomials by substituting x with $(x + x_0)$ so that the evaluation point effectively becomes 0. **Rule of Thumb:**

Feature	Euler-Cauchy Equation	Frobenius Method
Coefficient Structure	Strict: $P(x) = x^2$, $Q(x) \propto x$, $R(x) \in \mathbb{R}$	Flexible: $p(x)$ has at most a $\frac{1}{x}$ pole; $q(x)$ has at most a $\frac{1}{x^2}$ pole.
Solution Type	$r^2 + (a-1)r + b = 0$	$r^2 + (p_0 - 1)r + q_0 = 0$
Effect of Complex Roots	Results in $x^\lambda \cos(\mu \ln(x))$ and $x^\lambda \sin(\mu \ln(x))$	Results in a power series $\sum a_n x^n$ multiplied by $x^\lambda \cos(\mu \ln(x))$ and $x^\lambda \sin(\mu \ln(x))$

Table 3.4: *Comparison between Euler-Cauchy and Frobenius method.*

Every Euler-Cauchy equation can technically be solved by the Frobenius method (where the series simply terminates after the first term because $a_n = 0$ for $n > 0$). However, if the coefficients match the exact powers of the Euler-Cauchy form, use the Euler-Cauchy shortcut instead of expanding a full Frobenius series.

The behavior of a physical system modeled by a differential equation frequently is most interesting in the neighborhood of a singular point. Often geometric singularities in a physical problem, such as corners or sharp edges, lead to singular points in the corresponding differential equation. Thus, although at first we might want to avoid the few points where a differential equation is singular, it is precisely at these points that it is necessary to study the solution most carefully.
Numerical methods are ill suited (not suitable) for the study of solutions near a singular point. Thus, even if we adopt a numerical approach, it is advantageous to combine it with the analytical methods in order to examine the behavior of solutions near singular points.

Theorem 3.13: Series Solution for Differential Equation with Frobenius Method for Case 1,2,3

Consider the differential equation

$$x^2y'' + x[xp(x)]y' + [x^2q(x)]y = 0$$

where $x = 0$ is a regular singular point. Then $xp(x)$ and $x^2q(x)$ are analytic at $x = 0$ with convergent power series expansions

$$xp(x) = \sum_{n=0}^{\infty} p_n x^n, \quad x^2q(x) = \sum_{n=0}^{\infty} q_n x^n$$

for $|x| < \rho$, where $\rho > 0$ is the minimum of the radii of convergence of the power series for $xp(x)$ and $x^2q(x)$. Let r_1 and r_2 be the roots of the indicial equation

$$f(r) = r(r-1) + p_0 r + q_0 = 0$$

with $r_1 \geq r_2$ if r_1 and r_2 are real. Then in either the interval $-\rho < x < 0$ or the interval $0 < x < \rho$, there exists a solution of the form

$$y_1(x) = |x|^{r_1}\left[1 + \sum_{n=1}^{\infty} a_n(r_1)x^n\right] \tag{3.305}$$

where the $a_n(r_1)$ are given by the recurrence relation

$$f(r+n)a_n + \sum_{k=0}^{n-1} a_k[(r+k)p_{n-k} + q_{n-k}] = 0, \quad n \geq 1 \tag{3.306}$$

with $a_0 = 1$ and $r = r_1$.
If $r_1 - r_2$ is not zero or a positive integer, then in either the interval $-\rho < x < 0$ or the interval $0 < x < \rho$, there exists a second solution of the form

$$y_2(x) = |x|^{r_2}\left[1 + \sum_{n=1}^{\infty} a_n(r_2)x^n\right] \tag{3.307}$$

The $a_n(r_2)$ are also determined by the recurrence relation with $a_0 = 1$ and $r = r_2$. The power series in Eqs. (3.305) and (3.307) converge at least for $|x| < \rho$.
If $r_1 = r_2$, then the second solution is

$$y_2(x) = y_1(x)\ln|x| + |x|^{r_1}\sum_{n=1}^{\infty} b_n(r_1)x^n \tag{3.308}$$

If $r_1 - r_2 = N$, a positive integer, then the second solution is

$$y_2(x) = Cy_1(x)\ln|x| + |x|^{r_2}\sum_{n=1}^{\infty} c_n(r_2)x^n \tag{3.309}$$

The coefficients $a_n(r_1), b_n(r_1), c_n(r_2)$, and the constant C can be determined by substituting the form of the series solutions for y. The constant C may turn out to be zero, in which case there is no logarithmic term in the solution when the roots differ by an integer. Each of the series in Eqs. (3.308) and (3.309) converges at least for $|x| < \rho$ and defines a function that is analytic in some neighborhood of $x = 0$.
In all three cases the two solutions $y_1(x)$ and $y_2(x)$ form a fundamental set of solutions of the given differential equation.

History

Frobenius' contribution was not so much in all the possible forms of the series solutions involved. These forms had all been established earlier, by Lazarus Fuchs. The indicial polynomial and its role had also been established by Fuchs.

A first contribution by Frobenius to the theory was to show that - as regards a first, linearly independent solution, which then has the form of an analytical power series multiplied by an arbitrary power r of the independent variable - the coefficients of the generalized power series obey a recurrence relation, so that they can always be straightforwardly calculated.

A second contribution by Frobenius was to show that, in cases in which the roots of the indicial equation differ by an integer, the general form of the second linearly independent solution can be obtained by a procedure which is based on differentiation with respect to the parameter r.

A large part of Frobenius' 1873 publication was devoted to proofs of convergence of all the series involved in the solutions, as well as establishing the radii of convergence of these series.

[SI*] **Bessel's Equation**

We will consider three special cases of Bessel's equation,

$$x^2y'' + xy' + (x^2 - \nu^2)y = 0 \tag{3.310}$$

where ν is a constant. It is easy to show that $x = 0$ is a regular singular point of Eq. (3.310). We have

$$p_0 = \lim_{x\to 0} x\frac{Q(x)}{P(x)} = \lim_{x\to 0} x\frac{1}{x} = 1$$
$$q_0 = \lim_{x\to 0} x^2\frac{R(x)}{P(x)} = \lim_{x\to 0} x^2\frac{x^2-\nu^2}{x^2} = -\nu^2$$

Thus the indicial equation is

$$f(r) = r(r-1) + p_0 r + q_0 = r(r-1) + r - \nu^2 = r^2 - \nu^2 = 0$$

with the roots $r = \pm\nu$. We will consider the three cases $\nu = 0, \nu = \frac{1}{2}$, and $\nu = 1$ for the interval $x > 0$.

1. **Bessel Equation of Order Zero**

   In this case $\nu = 0$, so Eq. (3.310) reduces to

   $$L[y] = x^2y'' + xy' + x^2y = 0 \tag{3.311}$$

   and the roots of the indicial equation are equal: $r_1 = r_2 = 0$. Substituting

   $$y = \phi(r, x) = a_0x^r + \sum_{n=1}^{\infty} a_n x^{r+n} \tag{3.312}$$

   in Eq. (3.311), we obtain

   $$\begin{aligned} L[\phi](r,x) &= \sum_{n=0}^{\infty} a_n[(r+n)(r+n-1) + (r+n)]x^{r+n} + \sum_{n=0}^{\infty} a_n x^{r+n+2} \\ &= a_0[r(r-1)+r]x^r + a_1[(r+1)r + (r+1)]x^{r+1} \\ &+ \sum_{n=2}^{\infty} \left[a_n[(r+n)(r+n-1) + (r+n)] + a_{n-2}\right] x^{r+n} = 0 \end{aligned} \tag{3.313}$$

As we have already noted, the roots of the indicial equation $f(r) = r(r-1) + r = 0$ are $r_1 = 0$ and $r_2 = 0$. The recurrence relation is

$$a_n(r) = -\frac{a_{n-2}(r)}{(r+n)(r+n-1)+(r+n)} = -\frac{a_{n-2}(r)}{(r+n)^2}, \quad n \geq 2 \tag{3.314}$$

To determine $y_1(x)$, we set r equal to 0. Then, from Eq. (3.313), it follows that for the coefficient of x^{r+1} to be zero we must choose $a_1 = 0$. Hence, from Eq.(3.314, $a_3 = a_5 = a_7 = \cdots = 0$. Further,

$$a_n(0) = -\frac{a_{n-2}(0)}{n^2}, \quad n = 2, 4, 6, 8, \ldots$$

or, letting $n = 2m$, we obtain

$$a_{2m}(0) = -\frac{a_{2m-2}(0)}{(2m)^2}, \quad m = 1, 2, 3, 4, \ldots$$

Thus

$$\begin{aligned} a_2(0) &= -\frac{a_0}{2^2} \\ a_4(0) &= \frac{a_0}{2^4 2^2} \\ a_6(0) &= -\frac{a_0}{2^6(3\cdot 2)^2} \end{aligned}$$

and, in general,

$$a_{2m}(0) = \frac{(-1)^m a_0}{2^{2m}(m!)^2}, \quad m = 1, 2, 3, \ldots \tag{3.315}$$

Hence

$$y_1(x) = a_0 \left[1 + \sum_{m=1}^{\infty} \frac{(-1)^m x^{2m}}{2^{2m}(m!)^2}\right], \quad x > 0 \tag{3.316}$$

The function in brackets is known as the Bessel function of the first kind of order zero and is denoted by $J_0(x)$. It follows from Theorem 3.13 that the series converges for all x and that J_0 is analytic at $x = 0$.

To determine $y_2(x)$, we will calculate $a_n'(0)$. First we note from the coefficient of x^{r+1} in Eq. (3.313) that $(r+1)^2 a_1(r) = 0$. Thus $a_1(r) = 0$ for all r near $r = 0$.

So not only does $a_1(0) = 0$ but also $a_1'(0) = 0$. From the recurrence relation Eq. (3.314) it follows that $a_3'(0) = a_5'(0) = \cdots = a_{2n+1}'(0) = 0$; hence we need only compute $a_{2m}'(0), m = 1, 2, 3, \ldots$. From Eq. (3.314) we have

$$a_{2m}(r) = -\frac{a_{2m-2}(r)}{(r+2m)^2}, \quad m = 1, 2, 3, \ldots$$

By solving this recurrence relation, we obtain

$$a_2(r) = -\frac{a_0}{(r+2)^2}, \quad a_4(r) = \frac{a_0}{(r+2)^2(r+4)^2}$$

and, in general,

$$a_{2m}(r) = \frac{(-1)^m a_0}{(r+2)^2 \ldots (r+2m)^2}, \quad m \geq 3 \tag{3.317}$$

The computation of $a'_{2m}(r)$ can be carried out most conveniently by noting that if

$$f(x) = (x-\alpha_1)^{\beta_1}(x-\alpha_2)^{\beta_2}(x-\alpha_3)^{\beta_3}\dots(x-\alpha_n)^{\beta_n}$$

and if x is not equal to $\alpha_1, \alpha_2, \dots, \alpha_n$, then

$$\frac{f'(x)}{f(x)} = \frac{\beta_1}{x-\alpha_1} + \frac{\beta_2}{x-\alpha_2} + \cdots + \frac{\beta_n}{x-\alpha_n}$$

Applying this result to $a_{2m}(r)$ from Eq. (3.317), we find that

$$\frac{a'_{2m}(r)}{a_{2m}(r)} = -2\left(\frac{1}{r+2} + \frac{1}{r+4} + \cdots + \frac{1}{r+2m}\right)$$

and setting r equal to 0, we obtain

$$a'_{2m}(0) = -2\left[\frac{1}{2} + \frac{1}{4} + \cdots + \frac{1}{2m}\right] a_{2m}(0)$$

Substituting for $a_{2m}(0)$ from Eq. (3.315), and letting

$$H_m = \sum_{n=1}^{m} \frac{1}{n} = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{m} \tag{3.318}$$

we obtain, finally

$$a'_{2m}(0) = -H_m \frac{(-1)^m a_0}{2^{2m}(m!)^2}, \quad m = 1, 2, 3, \dots$$

The second solution of the Bessel equation of order zero is found by setting $a_0 = 1$ and substituting for $y_1(x)$ and $a'_{2m}(0) = b_{2m}(0)$ in Eq. (3.308).We obtain

$$y_2(x) = J_0(x)\ln(x) + \sum_{m=1}^{\infty} \frac{(-1)^{m+1} H_m}{2^{2m}(m!)^2} x^{2m}, \quad x > 0 \tag{3.319}$$

Instead of y_2, the second solution is usually taken to be a certain linear combination of J_0 and y_2. It is known as the Bessel function of the second kind of order zero and is denoted by Y_0. We define

$$Y_0(x) = \frac{2}{\pi}[y_2(x) + (\gamma - \ln(2))J_0(x)] \tag{3.320}$$

Here γ is a constant known as the Euler-Máscheroni constant; it is defined by the equation

$$\gamma = \lim_{n\to\infty}(H_n - \ln(n)) \cong 0.5772 \tag{3.321}$$

Substituting for $y_2(x)$ in Eq. (3.320), we obtain

$$Y_0(x) = \frac{2}{\pi}\left[\left(\gamma + \ln\left(\frac{x}{2}\right)\right) J_0(x) + \sum_{m=1}^{\infty} \frac{(-1)^{m+1} H_m}{2^{2m}(m!)^2} x^{2m}\right], \quad x > 0 \tag{3.322}$$

The general solution of the Bessel equation of order zero for $x > 0$ is

$$y = c_1 J_0(x) + c_2 Y_0(x)$$

Note that $J_0(x) \to 1$ as $x \to 0$ and that $Y_0(x)$ has a logarithmic singularity at $x = 0$; that is, $Y_0(x)$ behaves as $\left(\frac{2}{\pi}\right)\ln(x)$ when $x \to 0$ through positive values. Thus, if we are interested in solutions of Bessel's equation of order zero that are finite at the origin, which is often the case, we must discard Y_0.

It is interesting to note that for x large, both $J_0(x)$ and $Y_0(x)$ are oscillatory. Such a behavior might be anticipated from the original equation; indeed it is true for the solutions of the Bessel equation of order ν. If we divide Eq. (3.310) by x^2, we obtain

$$y'' + \frac{1}{x}y' + \left(1 - \frac{\nu^2}{x^2}\right)y = 0$$

For x very large it is reasonable to conjecture that the terms $\left(\frac{1}{x}\right)y'$ and $\left(\frac{\nu^2}{x^2}\right)y$ are small and hence can be neglected. If this is true, then the Bessel equation of order ν can be approximated by

$$y'' + y = 0$$

The solutions of this equation are $\sin(x)$ and $\cos(x)$; thus we might anticipate that the solutions of Bessel's equation for large x are similar to linear combinations of $\sin(x)$ and $\cos(x)$. This is correct insofar as the Bessel functions are oscillatory; however, it is only partly correct. For x large the functions J_0 and Y_0 also decay as x increases; thus the equation $y'' + y = 0$ does not provide an adequate approximation to the Bessel equation for large x, and a more delicate analysis is required. In fact, it is possible to show that

$$J_0(x) \approxeq \left(\frac{2}{\pi x}\right)^{1/2} \cos\left(x - \frac{\pi}{4}\right), \quad \text{as } x \to \infty \tag{3.323}$$

and that

$$Y_0(x) \approxeq \left(\frac{2}{\pi x}\right)^{1/2} \sin\left(x - \frac{\pi}{4}\right), \quad \text{as } x \to \infty \tag{3.324}$$

These asymptotic approximations, as $x \to \infty$, are actually very good. For example, to approximate $J_0(x)$ over the entire range from zero to infinity, we can use two or three terms of the series at Eq. (3.316) for $x \leq 1$ and the aymptotic approximation at Eq. (3.323) for $x \geq 1$.

2. **Bessel Equation of Order One-Half**

This case illustrates the situation in which the roots of the indicial equation differ by a positive integer but there is no logarithmic term in the second solution. Setting $\nu = \frac{1}{2}$ in Eq. (3.310) gives

$$L[y] = x^2y'' + xy' + \left(x^2 - \frac{1}{4}\right)y = 0 \tag{3.325}$$

When we substitute the series (3.312) for $y = \phi(r, x)$, we obtain

$$\begin{aligned} L[\phi](r,x) &= \sum_{n=0}^{\infty}\left[(r+n)(r+n-1) + (r+n) - \frac{1}{4}\right]a_n x^{r+n} + \sum_{n=0}^{\infty} a_n x^{r+n+2} \\ &= \left(r^2 - \frac{1}{4}\right)a_0 x^r + \left[(r+1)^2 - \frac{1}{4}\right]a_1 x^{r+1} \\ &+ \sum_{n=2}^{\infty}\left[\left((r+n)^2 - \frac{1}{4}\right)a_n + a_{n-2}\right]x^{r+n} = 0 \end{aligned} \tag{3.326}$$

The roots of the indicial equation are $r_1 = \frac{1}{2}$ and $r_2 = -\frac{1}{2}$; hence the roots differ by an integer. The recurrence relation is

$$\left[(r+n)^2 - \frac{1}{4}\right] a_n = -a_{n-2}, \quad n \geq 2 \tag{3.327}$$

Corresponding to the larger root $r_1 = \frac{1}{2}$, we find from the coefficient of x^{r+1} in Eq. (3.326) that $a_1 = 0$. Hence, from Eq. (3.327), $a_3 = a_5 = a_7 = \cdots = a_{2n+1} = \cdots = 0$. Further, for $r = \frac{1}{2}$,

$$a_n = -\frac{a_{n-2}}{n(n+1)}, \quad n = 2, 4, 6, \ldots$$

or, letting $n = 2m$, we obtain

$$a_{2m} = -\frac{a_{2m-2}}{2m(2m+1)}, \quad m = 1, 2, 3, \ldots$$

By solving this recurrence relation, we find that

$$a_2 = -\frac{a_0}{3!}, \quad a_4 = \frac{a_0}{5!}, \ldots$$

and, in general,

$$a_{2m} = \frac{(-1)^m a_0}{(2m+1)!}, \quad m = 1, 2, 3, \ldots$$

Hence, taking $a_0 = 1$, we obtain

$$\begin{aligned} y_1(x) &= x^{1/2}\left[1 + \sum_{m=1}^{\infty} \frac{(-1)^m x^{2m}}{(2m+1)!}\right] \\ &= x^{-1/2} \sum_{m=0}^{\infty} \frac{(-1)^m x^{2m+1}}{(2m+1)!}, \quad x > 0 \end{aligned} \tag{3.328}$$

The second power series in Eq. (3.328) is precisely the Taylor series for $\sin(x)$; hence one solution of the Bessel equation of order one-half is $x^{-1/2}\sin(x)$. The Bessel function of the first kind of order one-half, $J_{1/2}$, is defined as $\left(\frac{2}{\pi}\right)^{1/2} y_1$. Thus

$$J_{1/2}(x) = \left(\frac{2}{\pi x}\right)^{1/2} \sin(x), \quad x > 0 \tag{3.329}$$

Corresponding to the root $r_2 = -\frac{1}{2}$, it is possible that we may have difficulty in computing a_1 since $N = r_1 - r_2 = 1$. However, from Eq. (3.326) for $r = -\frac{1}{2}$, the coefficients of x^r and x^{r+1} are both zero regardless of the choice of a_0 and a_1. Hence a_0 and a_1 can be chosen arbitrarily. From the recurrence relation (3.327) we obtain a set of even-numbered coefficients corresponding to a_0 and a set of odd-numbered coefficients corresponding to a_1. Thus no logarithmic term is needed to obtain a second solution in this case. For $r = -\frac{1}{2}$,

$$a_{2n} = \frac{(-1)^n a_0}{(2n)!}, \quad a_{2n+1} = \frac{(-1)^n a_1}{(2n+1)!}, \quad n = 1, 2, 3, \ldots$$

Hence

$$\begin{aligned} y_2(x) &= x^{-1/2}\left[a_0\sum_{n=0}^{\infty}\frac{(-1)^n x^{2n}}{(2n)!} + a_1\sum_{n=0}^{\infty}\frac{(-1)^n x^{2n+1}}{(2n+1)!}\right] \\ &= a_0\frac{\cos(x)}{x^{1/2}} + a_1\frac{\sin(x)}{x^{1/2}} \end{aligned} \tag{3.330}$$

The constant a_1 simply introduces a multiple of $y_1(x)$. The second solution of the Bessel equation of order one-half is usually taken to be the solution for which $a_0 = \left(\frac{2}{\pi}\right)^{1/2}$ and $a_1 = 0$. It is denoted by $J_{-1/2}$. Then

$$J_{-1/2}(x) = \left(\frac{2}{\pi x}\right)^{1/2}\cos(x), \quad x > 0 \tag{3.331}$$

The general solution of Eq. (3.325) is

$$y = c_1 J_{1/2}(x) + c_2 J_{-1/2}(x)$$

By comparint Eqs. (3.329) and (3.331)with Eqs. (3.323) and (3.324), we see that, except for a phase shift of $\frac{\pi}{4}$, the functions $J_{-1/2}$ and $J_{1/2}$ resemble J_0 and Y_0, respectively, for large x.

3. **Bessel Equation of Order One**
This case illustrates the situation in which the roots of the indicial equation differ by a positive integer and the second solution involves a logarithmic term. Setting $\nu = 1$ in Eq. (3.310) gives

$$L[y] = x^2y'' + xy' + (x^2 - 1)y = 0 \tag{3.332}$$

If we substitute the series (3.312) for $y = \phi(r, x)$ and collect terms as in the preceding cases, we obtain

$$\begin{aligned} L[\phi](r,x) &= a_0(r^2-1)x^r + a_1[(r+1)^2-1]x^{r+1} \\ &\quad + \sum_{n=2}^{\infty}\left[((r+n)^2-1)a_n + a_{n-2}\right]x^{r+n} = 0 \end{aligned} \tag{3.333}$$

The roots of the indicial equation are $r_1 = 1$ and $r_2 = -1$. The recurrence relation is

$$[(r+n)^2 - 1]a_n(r) = -a_{n-2}(r), \quad n \geq 2 \tag{3.334}$$

Corresponding to the larger root $r = 1$, the recurrence relation becomes

$$a_n = -\frac{a_{n-2}}{(n+2)n}, \quad n = 2, 3, 4, \ldots$$

We also find from the coefficient of x^{r+1} in Eq. (3.333) that $a_1 = 0$; hence from the recurrence relation, $a_3 = a_5 = \cdots = 0$. For even values of n, let $n = 2m$; then

$$a_{2m} = -\frac{a_{2m-2}}{(2m+2)(2m)} = -\frac{a_{2m-2}}{2^2(m+1)m}, \quad m = 1, 2, 3, \ldots$$

By solving this recurrence relation, we obtain

$$a_{2m} = -\frac{(-1)^m a_0}{2^{2m}(m+1)!m!}, \quad m = 1, 2, 3, \ldots \tag{3.335}$$

The Bessel function of the first kind of order one, denoted by J_1, is obtained by choosing $a_0 = \frac{1}{2}$. Hence

$$J_1(x) = \frac{x}{2} \sum_{m=0}^{\infty} \frac{(-1)^m x^{2m}}{2^{2m}(m+1)!m!} \tag{3.336}$$

The series converges absolutely for all x, so the function J_1 is analytic everywhere.
In determining a second solution of Bessel's equation of order one, we illustrate the method of direct substitution. The calculation of the general term in Eq. (3.337) below is rather complicated, but the first few coefficients can be found fairly easily. According to Theorem 3.13, we assume that

$$y_2(x) = aJ_1(x)\ln(x) + x^{-1}\left[1 + \sum_{n=1}^{\infty} c_n x^n\right], \quad x > 0 \tag{3.337}$$

Computing $y_2'(x), y_2''(x)$, substituting in Eq. (3.332), and making use of the fact that J_1 is a solution of Eq. (3.332), we obtain

$$2axJ_1'(x) + \sum_{n=0}^{\infty}[(n-1)(n-2)c_n + (n-1)c_n - c_n]x^{n-1} + \sum_{n=0}^{\infty} c_n x^{n+1} = 0 \tag{3.338}$$

where $c_0 = 1$. Substituting for $J_1(x)$ from Eq. (3.336), shifting indices of summation in the two series, and carrying out several steps of algebra, we arrive at

$$-c_1 + [0 \cdot c_2 + c_0]x + \sum_{n=2}^{\infty}[(n^2-1)c_{n+1} + c_{n-1}]x^n = -ax - a\left[\sum_{m=1}^{\infty} \frac{(-1)^m(2m+1)x^{2m+1}}{2^{2m}(m+1)!m!}\right] \tag{3.339}$$

From Eq. (3.339) we observe first that $c_1 = 0$, and $a = -c_0 = -1$. Further, since there are only odd powers of x on the right, the coefficient of each even power of x on the left must be zero. Thus, since $c_1 = 0$, we have $c_3 = c_5 = \cdots = 0$. Corresponding to the odd powers of x, we obtain the recurrence relation (let $n = 2m+1$ in the series on the left side of Eq. (3.339))

$$[(2m+1)^2 - 1]c_{2m+2} + c_{2m} = \frac{(-1)^m(2m+1)}{2^{2m}(m+1)!m!}, \quad m = 1, 2, 3, \ldots \tag{3.340}$$

When we set $m = 1$ in Eq. (3.340), we obtain

$$(3^2 - 1)c_4 + c_2 = \frac{(-1)3}{2^2 \cdot 2!}$$

Notice that c_2 can be selected arbitrarily, and then this equation determines c_4. Also notice that in the equation for the coefficient of x, c_2 appeared multiplied by 0, and that equation was used to determine a. That c_2 is arbitrary is not surprising, since c_2 is the coefficient of x in the expression $x^{-1}\left[1 + \sum_{n=1}^{\infty} c_n x^n\right]$. Consequently, c_2 simply generates a multiple of J_1, and y_2 is determined only up to an additive multiple of J_1.

In accordance with the usual practice, we choose $c_2 = \frac{1}{2^2}$. Then we obtain

$$\begin{aligned} c_4 &= \frac{-1}{2^4 \cdot 2}\left[\frac{3}{2}+1\right] \\ &= \frac{-1}{2^4 2!}\left[\left(1+\frac{1}{2}\right)+1\right] \\ &= \frac{-1}{2^4 2!}(H_2 + H_1) \end{aligned}$$

It is possible to show that the solution of the recurrence relation (3.340) is

$$c_{2m} = \frac{(-1)^{m+1}(H_m + H_{m-1})}{2^{2m} m!(m-1)!}, \quad m = 1, 2, \ldots$$

with the understanding that $H_0 = 0$. Thus

$$y_2(x) = -J_1(x)\ln(x) + \frac{1}{x}\left[1 - \sum_{m=1}^{\infty} \frac{(-1)^m (H_m + H_{m-1})}{2^{2m} m!(m-1)!} x^{2m}\right], \quad x > 0 \tag{3.341}$$

The calculation of $y_2(x)$ using the alternative procedure in which we determine the $c_n(r_2)$ is slightly easier. In particular, the latter procedure yields the general formula for c_{2m} without the necessity of solving a recurrence relation of the form (3.340). In this regard, we may also wish to compare the calculations of the second solution of Bessels' equation of order zero.
The second solution of Eq. (3.332), the Bessel function of the second kind of order one, Y_1, is usually taken to be a certain linear combination of J_1 and y_2. Thus, Y_1 is defined as

$$Y_1(x) = \frac{1}{\pi}[-y_2(x) + (\gamma - \ln(2))J_1(x)] \tag{3.342}$$

where γ is defined in Eq. (3.321). The general solution of Eq. (3.332) for $x > 0$ is

$$y = c_1 J_1(x) + c_2 Y_1(x)$$

Notice that although J_1 is analytic at $x = 0$, the second solution Y_1 becomes unbounded in the same manner as $\frac{1}{x}$ as $x \to 0$.

Guide 3.12: Frobenius Method to Compute the First Solution for Bessel's Equation

Suppose Bessel's differential equation of order ν is:

$$x^2y'' + xy' + (x^2 - \nu^2)y = 0$$

We can use the method of Frobenius to find the first solution for any order ν, the point $x = 0$ is a regular singular point. By assuming a Frobenius series solution:

$$y(x) = \sum_{n=0}^{\infty} a_n x^{n+r}$$

Substituting this into Bessel's equation gives the indicial equation by looking at the lowest power ($n = 0$):

$$r^2 - \nu^2 = 0$$
$$r = \pm\nu$$

Using the root $r_1 = \nu$, the Frobenius method successfully produces the Bessel function of the first kind $J_\nu(x)$ for any real or complex order ν:

$$J_\nu(x) = \sum_{n=0}^{\infty} \frac{(-1)^n}{n!\Gamma(\nu + n + 1)} \left(\frac{x}{2}\right)^{2n+\nu}$$

For the second solution, whether Frobenius works directly for it depends entirely on the order ν:

1. ν **is not an integer (**2ν **is not an integer difference)**
   Using the second root $r_2 = -\nu$, the Frobenius method directly yields a second linearly independent solution, $J_{-\nu}(x)$. The general solution can be written as a linear combination of $J_\nu(x)$ and $J_{-\nu}(x)$. The standard Bessel function of the second kind (Neumann function Y_ν) is defined via a linear combination of these two:

   $$Y_\nu(x) = \frac{J_\nu(x)\cos(\nu\pi) - J_{-\nu}(x)}{\sin(\nu\pi)}$$

2. ν **is an integer (**$n = 0, 1, 2, \dots$ **)**
   The roots differ by an integer ($r_1 - r_2 = 2\nu$ is an integer), and plugging $r_2 = -\nu$ leads to a repeating/redundant series or division by zero in the coefficients. The Frobenius method fails to give a second independent series directly because a logarithmic term ($\ln(x)$) naturally appears in the second solution.
   To find the second solution ($Y_n(x)$) for integer orders, we must use a limit process (applying L'Hôpital's rule to the non-integer formula above as $\nu \to n$) or use the reduction of order method starting from $J_n(x)$.

i. Compute Derivatives of a Differential Equation' Solution at the given point x_0

Determine $\phi''(x_0), \phi'''(x_0)$, and $\phi^{(4)}(x_0)$ for the given point x_0 if $y = \phi(x)$ is a solution of the given initial value problem

$$y'' + xy' + y = 0$$

with initial conditions

$$y(0) = 1$$
$$y'(0) = 0$$

Solution

Solve the ODE for y''

$$y'' = -xy' - y \tag{3.343}$$

Plug in $x = 0$

$$y''(0) = -(0)y'(0) - y(0) = -1$$

Differentiate both sides of Eq. (3.343) with respect to x

$$y''' = -y' - xy'' - y' = -2y' - xy'' \tag{3.344}$$

Plug in $x = 0$

$$y'''(0) = -2y'(0) - (0)y''(0) = 0$$

Differentiate both sides of equation (3.344) with respect to x

$$y^{(4)} = -y'' - y'' - xy''' - y'' = -3y'' - xy'''$$

Plug in $x = 0$

$$y^{(4)}(0) = -3y''(0) - (0)y'''(0) = 3$$

In SymIntegration we can use **secondorderlineardiffeq_derivativesvalueatx0(**f**,**y**,**x**,**x_0**,** y_{x_0}**,** y'_{x_0}**)** function to compute the derivatives' value till fourth order at x_0 from homogeneous second order linear differential equation with arbitrary initial conditions.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Second Order Linear Differential Equation Derivatives Values at x0 ]# ./main

The differential equation :
df(y[x],x,x)+x*df(y[x],x)+y[x]

x0 = 0, y(0) = 1, y'(0) = 0

y'' = -x*df(y[x],x)-y[x]

y''(x0) = -1

Differentiate the differential equation with respect to x :
df(y[x],x,x,x)+2*df(y[x],x)+x*df(y[x],x,x)

y''' = -2*df(y[x],x)-x*df(y[x],x,x)

y'''(x0) = 0

Differentiate the differential equation again with respect to x :
df(y[x],x,x,x,x)+3*df(y[x],x,x)+x*df(y[x],x,x,x)

y''' = -3*df(y[x],x,x)-x*df(y[x],x,x,x)

y'''(x0) = 3

Time taken by function: 49307 microseconds
```

Figure 3.70: *The computation $\phi''(x_0), \phi'''(x_0)$, and $\phi^{(4)}(x_0)$ for the given point $x_0 = 0$ for differential equation $y'' + xy' + y = 0$ with initial conditions $y(0) = 1$, $y'(0) = 0$ (**SymIntegration/Examples/Differential Equations/Test SymIntegration Second Order Linear Differential Equation Derivatives Values at x0/main.cpp**).*

ii. Compute Power Series Solution for Homogeneous Second Order Linear Differential Equations with Constant Coefficients

Consider the homogeneous second order linear differential equation

$$y'' + 3y' + 2y = 0$$

with initial conditions

$$y(0) = 1$$
$$y'(0) = 2$$

We will compute the power series solution around an ordinary point $x_0 = 0$.

We have created a function previously in SymIntegration to compute this type of differential equation: **secondorderlineardiffeq_ivpsolution(1,3,2,y,t,0,1,2)**.

The exact solution is

$$y(x) = 4e^{-x} - 3e^{-2x}$$

Now, we want to compute the power series solution for this differential equation to see whether the series solutions converge to the exact solution or not as the terms goes to ∞.

To find a power series solution for a second order linear differential equation $P(x)y'' + Q(x)y' + R(x)y = 0$ near an ordinary (regular) point $x_0 = 0$, assume a standard power series

$$y(x) = \sum_{n=0}^{\infty} a_n x^n$$

We will have

$$y(x) = \sum_{n=0}^{\infty} a_n x^n = a_0 + a_1 x + a_2 x^2 + a_3 x^3 + \dots$$
$$y'(x) = \sum_{n=1}^{\infty} n a_n x^{n-1} = a_1 + 2a_2 x + 3a_3 x^2 + 4a_3 x^3 + \dots$$
$$y''(x) = \sum_{n=2}^{\infty} n(n-1) a_n x^{n-2} = 2a_2 + 6a_3 x + 12a_4 x^2 + \dots$$

How to compute the recurrence relation:

1. **Plug in series:** Substitute y, y', and y'' into the differential equation.
2. **Multiply and distribute:** Multiply the coefficients like x or x^2 into the summation terms, adjusting indices so every power of x matches (e.g., changing x^{n-1} or x^n to a common power like x^k or x^{n-2}).
3. **Combine sums:** Merge all summations into a single expression of the format $\sum c_n x^n = 0$.
4. **Set coefficients to zero:** Equate the coefficient of each power of x to 0 to generate the algebraic equation.
5. **Isolate the highest index:** Solve the equation for the term with the highest subscript (like a_{n+2} or a_n) in terms of lower subscripts. This equation is the recurrence relation.

6. **Iterate coefficients:** Compute sequential terms $(a_2, a_3, a_4, \dots)$ as multiples of the free parameters a_0 and a_1 to write the final general solution $y(x) = a_0 y_1(x) + a_1 y_2(x)$

For the differential equation mentioned above, we substitute $y = \sum_{n=0}^{\infty} a_n x^n$ then differentiate it twice and substitute them into the differential equation to find the recurrence relation. Knowing that we only deal with constant coefficients now, the recurrence formula is easy to obtain.

Always remember that a single universal or "master" formula does not exist for the power series coefficients of every second-order differential equation. Instead, the power series method provides a systematic algorithmic procedure to construct the general solution for linear differential equations with analytic coefficients.

To compute the power series solution of a homogeneous linear differential equation with constant coefficients, we can use the recurrence relation derived from matching coefficients.
For an m-th order linear homogeneous differential equation with constant coefficients b_k:

$$\sum_{k=0}^{m} b_k y^{(k)}(x) = 0$$

Assuming a power series solution $y(x) \sum_{n=0}^{\infty} a_n x^n$, substituting the derivatives yields the recurrence relation for the coefficients a_n:

$$a_{n+m} = -\frac{1}{b_m \prod_{j=1}^{m}(n+j)} \sum_{k=0}^{m-1} b_k a_{n+k} \prod_{j=1}^{k}(n+j)$$

To solve a homogeneous second order linear differential equation with constant coefficients using a power series method in C++, we can represent the series solution as a polynomial class where the coefficients are calculated recurrence-wise.

We have created a class in SymIntegration:
class HigherOrderODE_Homogeneous_PowerSeriesSolver

This class is defined in **dsolve.h** and the computation code for the functions of the class is in **dsolve.cpp**.

```
...

#ifndef HIGHERORDERODE_HOMOGENEOUS_POWERSERIESSOLVER_H
#define HIGHERORDERODE_HOMOGENEOUS_POWERSERIESSOLVER_H

class HigherOrderODE_Homogeneous_PowerSeriesSolver {
       private:
       vector<double> ode_coefficients; // ODE coefficients [b_0, b_1, ..., b_m]
       vector<double> initial_values; // Initial conditions [y(0), y'(0), ..., y^(
           m-1)(0)]
       int order; // Order of the differential equation (m)
       vector<double> coefficients;
```

```
      // Helper function to calculate Pochhammer symbol / falling factorial
          factors
      double get_factorial_multiplier(int n, int k) const
      {
            double multiplier = 1.0;
            for (int j = 1; j <= k; ++j)
            {
                  multiplier *= (n + j);
            }
            return multiplier;
      }
      public:
      // Constructor initializes the terms (series degree+1) and base boundary
          conditions
      HigherOrderODE_Homogeneous_PowerSeriesSolver(const vector<double>&
          ode_coeffs, const vector<double>& init_conditions) ;

      // Computes coefficients up to order N using the recurrence relation:
      void computeSeries(int terms) ;
      vector<double> coefficientsvector(int terms) ;
      double evaluateAt(double x, int terms) ;
      void printSeries() const ;
};

#endif


...
```

Code 42: *include/symintegral/dsolve.h*

The **dsolve.cpp** that handles the function to compute the power series solution and approximation at x can be seen below

```
HigherOrderODE_Homogeneous_PowerSeriesSolver::
    HigherOrderODE_Homogeneous_PowerSeriesSolver(const vector<double>& ode_coeffs
    , const vector<double>& init_conditions)
{
      ode_coefficients = ode_coeffs;
      initial_values = init_conditions;
      order = ode_coeffs.size() - 1;

}

void HigherOrderODE_Homogeneous_PowerSeriesSolver::computeSeries(int terms)
{
      if (terms < 2)
      {
            return;
      }
```

```
      coefficients.resize(terms,0.0);

      // Step 1: Assign initial conditions to the first m series terms
      // Recall that y^(k)(0) = k! * coefficients_k, so coefficients_k = y^(k)(0)
           / k!
      double factorial = 1.0;
      for (int k = 0; k <= order - 1 && k < terms; ++k)
      {
            if (k > 0)
            {
                  factorial *= k;
            }
            coefficients[k] = initial_values[k] / factorial;
      }

      // Step 2: Use recurrence relation to compute subsequent coefficients
      double b_m = ode_coefficients[order];
      if (b_m == 0.0)
      {
            std::cerr << "Error: Leading coefficient b_m cannot be zero." <<
                endl;
      }

      for (int n = 0; n <= terms - order - 1; ++n)
      {
            double sum_terms = 0.0;
            for (int k = 0; k < order; ++k)
            {
                  double mult = get_factorial_multiplier(n, k);
                  sum_terms += ode_coefficients[k] * coefficients[n + k] * mult
                      ;
            }

            double divisor = b_m * get_factorial_multiplier(n, order);
            coefficients[n + order] = -sum_terms / divisor;
      }
      ...
}

vector<double> HigherOrderODE_Homogeneous_PowerSeriesSolver::coefficientsvector(
    int terms)
{
      if (terms < 2)
      {
            return {};
      }
```

```
        ...

        return coefficients;
}

// Evaluates the power series at a specific value of x
double HigherOrderODE_Homogeneous_PowerSeriesSolver::evaluateAt(double x, int
    terms)
{
        double current_x_power = 1.0;
        double sum = 0.0;

        for (int i = 0; i <= terms; ++i)
        {
                sum += coefficients[i] * current_x_power;
                current_x_power *= x;
        }
        return sum;
}

void HigherOrderODE_Homogeneous_PowerSeriesSolver::printSeries() const
{
        cout << "\nSeries solution: \n"<< endl;
        cout << "y(x) = ";
        bool first = true;
        for (size_t i = 0; i < coefficients.size(); ++i)
        {
                if (coefficients[i] == 0.0)
                {
                        continue;
                }
                if (!first && coefficients[i] > 0)
                {
                        cout << " + ";
                }
                ...
        }
        cout << " + ...\n";
}
```

Code 43: *src/dsolve.cpp*

```
IVP Solve for y'' + 3y' +  2y = 0 with y(0) = 1, y'(0) = 2


The general solution is:
c1*e^(-t)+c2*e^(-2*t)

The solution for the initial value problem is:
4*e^(-t)-3*e^(-2*t)

y' = -4*e^(-t)+6*e^(-2*t)

Critical value t_{m} = 0.405465

Maximum value y_{m} = 1.33333
```

Figure 3.71: *The exact solution of* $y'' + 3y' + 2y = 0$ *computed with* ***secondorderlineardiffeq_ivpsolution(1,3,2,y,t,0,1,2)*** *(**SymIntegration/Examples/Differential Equations/Test SymIntegration IVPSolve Second Order Linear Equations/main.cpp**).*

```
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Differential Equations/Test SymIntegration Series Solutions for Homogeneous Second
 Order Linear Differential Equation with Constant Coefficients ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Differential Equations/Test SymIntegration Series Solutions for Homogeneous Second
 Order Linear Differential Equation with Constant Coefficients ]# ./main

( 2 ) y + ( 3 ) y^(1) + ( 1 ) y^(2)  = 0

Initial conditions:
y(0) = 1
y^(1) (0) = 2

Series solution:

y(x) = 1 + 2x - 4x^2 + 3.33333x^3 - 1.83333x^4 + 0.766667x^5 - 0.261111x^6 + 0.0753968x^7 - 0.0189484x^8 + 0.00422178x^9 - 0.000845459x^10 + 0.0
0015382x^11 - 2.5645e-05x^12 + 3.94603e-06x^13 - 5.63764e-07x^14 + 7.51716e-08x^15 + ...

Time taken by function: 472 microseconds
```

Figure 3.72: *The power series solution of* $y'' + 3y' + 2y = 0$ *computed with* ***class HigherOrderODE_Homogeneous_PowerSeriesSolver****,* $n = 16$ *(highest degree of the solution is at* $n - 1$*) (**SymIntegration/Examples/Differential Equations/Test SymIntegration Series Solutions for Homogeneous Second Order Linear Differential Equation with Constant Coefficients/main.cpp**).*

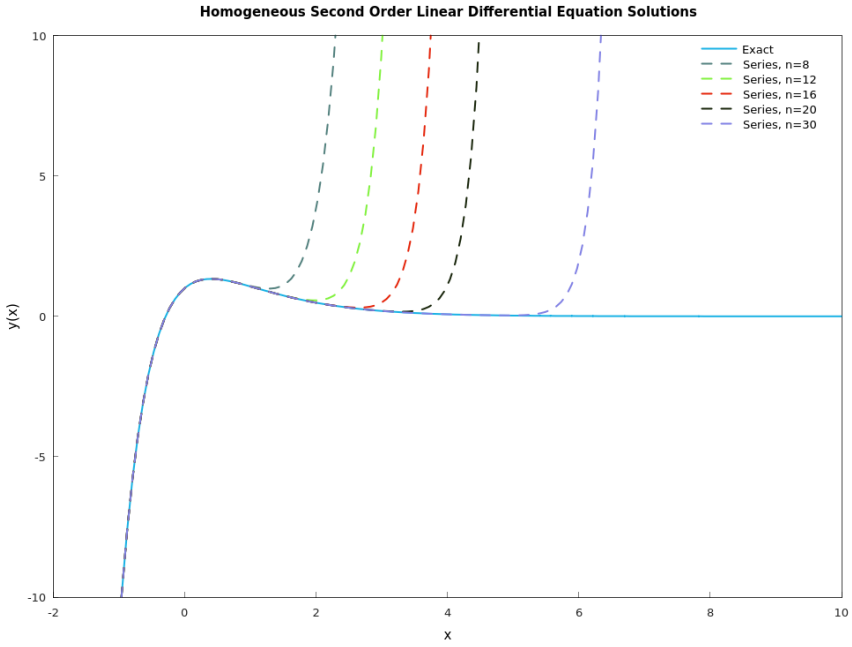

Figure 3.73: *The plot of the exact solution and the series solution with various terms for the differential equation* $y'' + 3y' + 2y = 0$*, we combine SymIntegration library with Hamzstplot,thus SymIntegration is used to compute the series solution and then plot it with Hamzstplot (**Hamzstplot/Examples with Makefile/Differential Equations/Plot 2D Comparing Series Solutions with Exact Solution for Second Order Homogeneous Linear Differential Equation/main.cpp**).*

iii. Compute Power Series Solution for Homogeneous Second Order Linear Differential Equations with Variable Coefficients Near an Ordinary Point x_0

To solve a homogeneous second order linear differential equation with variable coefficients around a point x_0, we must first classify the point x_0. If x_0 is an ordinary point ($P(x_0) \neq 0$), we use a standard power series expansion. If x_0 is a regular singular point, use the Frobenius method.

The general homogeneous second order linear differential equation is structured as

$$P(x)y'' + Q(x)y' + R(x)y = 0$$

with the initial condition:

$$y(0) = y0$$
$$y'(0) = dy0$$

1. For a homogeneous second order linear differential equation of the form:

   $$P(x)y'' + Q(x)y' + R(x)y = 0$$

   where $P(x), Q(x)$, and $R(x)$ are polynomials or analytic functions, we find a series solution depending on the nature of the point x_0 where we expand the series.

2. **Ordinary or Singular Points**

   - **Ordinary Point ($P(x_0) \neq 0$)**: We can assume a direct Taylor power series centered at x_0:

     $$y(x) = \sum_{n=0}^{\infty} a_n (x - x_0)^n$$

     Plugging this into the differential equation yields a recurrence relation that defines all a_n based on two arbitrary constants (a_0 and a_1), producing two linearly independent solutions $y_1(x)$ and $y_2(x)$. The general solution is $y(x) = c_1 y_1(x) + c_2 y_2(x)$.
   - **Regular Singular Point ($P(x_0) = 0$, but limits for $\frac{(x-x_0)Q}{P}$ and $\frac{(x-x_0)^2 R}{P}$ are finite)**: We use the Frobenius method, assuming a solution multiplied by a fractional or real power r:

     $$y(x) = (x - x_0)^r \sum_{n=0}^{\infty} a_n (x - x_0)^n$$

     An indicial equation determines the value of r, leading to the series expansion.

   Since there is no universal formula for the recurrence relation, the specific recurrence relation for the coefficients a_n changes completely depending on the functions $P(x), Q(x)$, and $R(x)$. The clean principle of superposition and straightforward power series substitution generally fails or becomes non-algorithmic for second order nonlinear equations‘

Below is the full mathematical mechanism to compute the power series solution of a homogeneous second order linear differential equation around a point x_0:

1. **Point Classification**
   First, write the differential equation in standard form:

   $$y'' + p(x)y' + q(x)y = 0$$

where $p(x) = \frac{Q(x)}{P(x)}$ and $q(x) = \frac{R(x)}{P(x)}$. We will compute the power series solution about the given point x_0, if $x_0 \neq 0$ then the center will be shifted by x_0.

- **Power Series Method (Ordinary Point)**
  If $p(x)$ and $q(x)$ are analytic at x_0 ($P(x_0) \neq 0$), we assume the solution as a standard Taylor series:
  $$y(x) = \sum_{n=0}^{\infty} a_n (x - x_0)^n$$
- **Frobenius Method (Regular Singular Point)**
  If $p(x)$ or $q(x)$ diverges at x_0 ($P(x_0) = 0$), but $\lim_{x \to x_0}(x - x_0)p(x)$ and $\lim_{x \to x_0}(x - x_0)^2 q(x)$ finite (remain analytic), x_0 is a regular singular point. We assume a Frobenius series:
  $$y(x) = \sum_{n=0}^{\infty} a_n (x - x_0)^{n+r}, \quad a_0 \neq 0$$

2. **Compute Derivatives**
   After we have classified the point x_0 and obtain the $y(x)$, we now differentiate the assumed series term-by-term to find expressions for y' and y''.

   - **Power Series Method (Ordinary Point)**
     $$y(x) = \sum_{n=0}^{\infty} a_n (x - x_0)^n$$
     $$y'(x) = \sum_{n=1}^{\infty} n a_n (x - x_0)^{n-1}$$
     $$y''(x) = \sum_{n=2}^{\infty} n(n-1) a_n (x - x_0)^{n-2}$$
     Substituting these equations above into the differential equation yields a recurrence relation determining a_n for $n \geq 2$ based on arbitrary constants a_0 and a_1.
   - **Frobenius Method (Regular Singular Point)**
     $$y(x) = \sum_{n=0}^{\infty} a_n (x - x_0)^{n+r}, \quad a_0 \neq 0$$
     $$y'(x) = \sum_{n=0}^{\infty} (n + r) a_n (x - x_0)^{n+r-1}$$
     $$y''(x) = \sum_{n=0}^{\infty} (n + r)(n + r - 1) a_n (x - x_0)^{n+r-2}$$
     Plugging this into the differential equation produces the indicial equation (a quadratic equation in r). Solving it gives roots r_1 and r_2, which dictate the structure of the recurrence relation for a_n.

3. **Shifting the Index of Summation, Substitute and Align Powers**
   Substitute $y(x), y'(x)$, and $y''(x)$ back into the original differential equation. To combine the terms into a single summation, we must perform two adjustments:

   - **Shift Indices:** Change the dummy variables (e.g., let $k = n - 2$) so that every term features the exact same power of x (typically x^k or $(x - x_0)^k$).

- **Align Starting Limits:** Expand the lower terms of the series explicitly until all remaining summations start at the same index value.

Frobenius Method (Regular Singular Point)
After we substitute y, y', y'' into the differential equation multiplied by x^2 to clear denominators:

$$x^2y'' + x \cdot p(x)y' + q(x)y = 0$$

Expand $p(x) = \sum_{k=0}^{\infty} p_k x^k$ and $q(x) = \sum_{k=0}^{\infty} q_k x^k$ as standard Taylor series.
Combine Cauchy products of series so every summation term matches a unified power exponent of x^{n+r}. If a summation index starts at $n = 0$ but evaluates with an offset x^{n+r-2}, re-index or shift the summation counter $n \to n + 2$ (adjusting the summation bounds and general coefficient subscripts simultaneously) to factor out a common x^{n+r} baseline.

4. **Solve the Indicial and Recurrence Equations**
Factor out the common power of x^k. For the differential equation to equal zero for all x, the coefficient of every power of x must independently equal zero.

- **Power Series Method (Ordinary Point)**

$$\sum_{k=0}^{\infty} f(a_k, a_{k+1}, a_{k+2})x^k = 0$$

Because this equation must equal zero for all values of x, the identity principle states that the coefficients of each power of x must independently equal zero. Set the inner expression to zero to obtain the recurrence relation:

$$f(a_k, a_{k+1}, a_{k+2}) = 0$$

Solve the recurrence relation for higher coefficients, to express the higher-order coefficients $(a_2, a_3, a_4, \dots)$ in terms of the arbitrary starting constraints a_0 and a_1

- **Frobenius Method (Regular Singular Point)**
Setting the lowest power coefficient (x^r) to zero yields a quadratic equation in terms of r / the indicial equation:

$$f(r) = 0$$
$$r(r-1) + p_0 r + q_0 = 0$$

Solve this to find two roots, r_1 and r_2 (where $\mathrm{Re}(r_1) \geq \mathrm{Re}(r_2)$).
Equate the general coefficient of x^{n+r} to zero for $n \geq 1$, producing a linear relation linking a_n to preceding coefficients $a_0, a_1, \dots, a_{n-1}$:

$$a_n(n+r)(n+r-1) + \sum_{k=0}^{n} \left[p_{n-k}(k+r) + q_{n-k}\right] a_k = 0$$

(a) **Case 1: $r_1 - r_2$ is not an integer**
When we get the roots that do not differ by an integer, both solutions are pure power series:

$$y_1(x) = (x - x_0)^{r_1} \sum_{n=0}^{\infty} a_n (x - x_0)^n, \quad (a_0 \neq 0)$$
$$y_2(x) = (x - x_0)^{r_2} \sum_{n=0}^{\infty} b_n (x - x_0)^n, \quad (b_0 \neq 0)$$

(b) **Case 2:** $r_1 = r_2$
When the roots are identical, the second linearly independent solution introduces a logarithmic term:

$$y_1(x) = (x - x_0)^{r_1} \sum_{n=0}^{\infty} a_n x^n, \quad (a_0 \neq 0)$$
$$y_2(x) = y_1(x - x_0)\ln(x - x_0) + (x - x_0)^{r_1} \sum_{n=1}^{\infty} b_n (x - x_0)^n$$

(c) **Case 3:** $r_1 - r_2$ **is a positive integer**
When the roots differ by a positive integer ($r_1 - r_2 \in N, N \in \mathbb{Z}^+$), the second solution may contain a logarithmic term depending on whether a coefficient log-jam occurs:

$$y_1(x) = (x - x_0)^{r_1} \sum_{n=0}^{\infty} a_n (x - x_0)^n, \quad (a_0 \neq 0)$$
$$y_2(x) = Cy_1(x)\ln(x - x_0) + (x - x_0)^{r_2} \sum_{n=0}^{\infty} b_n (x - x_0)^n, \quad (b_0 \neq 0)$$

where C can be 0.

$r_1 - r_2$	**First solution ($y_1(x)$)**	**Second solution ($y_2(x)$)**
Not an integer **($r_1 - r_2 \notin \mathbb{Z}$)**	$\sum_{n=0}^{\infty} a_n x^{n+r_1}$	$\sum_{n=0}^{\infty} b_n x^{n+r_2}$
Equal roots **($r_1 = r_2$)**	$\sum_{n=0}^{\infty} a_n x^{n+r_1}$	$y_1(x)\ln(x) + \sum_{n=1}^{\infty} b_n x^{n+r_1}$
differ by a positive integer **($r_1 - r_2 \in \mathbb{N}$)**	$\sum_{n=0}^{\infty} a_n x^{n+r_1}$	$Cy_1(x)\ln(x) + \sum_{n=0}^{\infty} b_n x^{n+r_2}$

Table 3.5: *Depending on the difference $\Delta = r_1 - r_2$, the fundamental solutions $y_1(x)$ and $y_2(x)$ take different forms.*

5. **Compute the Final General Solution**

$$y(x) = a_0 y_1(x) + a_1 y_2(x)$$

where $y_1(x)$ and $y_2(x)$ are two linearly independent fundamental solutions.

We would like to demonstrate a class in SymIntegration that we have created:
class SecondOrderODE_Homogeneous_PowerSeriesSolver
this **class** is able to compute the power series solution of a homogeneous second order linear differential equation with variable coefficients near an ordinary point (not singular), we name linear differential equation as ODE (Ordinary Differential Equation).

We will use this **class** to compute the solution for the famous Airy differential equation

$$y'' - xy = 0$$

We have computed the solution near $x_0 = 0$ first (no shifting), then near a regular point $x_0 = 1$ in the examples above under the **Series Solutions Near an Ordinary Point** part. In this section we will use SymIntegration to compute the series solutions.

A power series solution for a homogeneous second order linear differential equation with variable coefficients, formulated as $P(x)y'' + Q(x)y' + R(x)y = 0$, can be computed systematically by shifting the variable to center the equations around x_0, and then evaluating Taylor expansion derivatives recursively.

The general solution of Airy's equation near $x_0 = 0$ is

$$y = a_0 \left[1 + \frac{x^3}{2 \cdot 3} + \frac{x^6}{2 \cdot 3 \cdot 5 \cdot 6} + \cdots + \frac{x^{3n}}{2 \cdot 3 \ldots (3n-1)(3n)} + \ldots\right] + a_1 \left[x + \frac{x^4}{3 \cdot 4} + \frac{x^7}{3 \cdot 4 \cdot 6 \cdot 7} + \cdots + \frac{x^{3n+1}}{3 \cdot 4 \ldots (3n)(3n+1)} + \ldots\right]$$

it is the same equation as Eq. (3.279), that is composed by the function $Ai(x)$ and $Bi(x)$, two linearly independent solutions to the Airy differential equation.

$$Ai(x) = 1 + \frac{x^3}{2 \cdot 3} + \frac{x^6}{2 \cdot 3 \cdot 5 \cdot 6} + \cdots + \frac{x^{3n}}{2 \cdot 3 \ldots (3n-1)(3n)} + \ldots$$
$$Bi(x) = x + \frac{x^4}{3 \cdot 4} + \frac{x^7}{3 \cdot 4 \cdot 6 \cdot 7} + \cdots + \frac{x^{3n+1}}{3 \cdot 4 \ldots (3n)(3n+1)} + \ldots$$

the general solution of Airy's equation near $x_0 = 1$ is

$$y = a_0 \left[1 + \frac{(x-1)^2}{2} + \frac{(x-1)^3}{6} + \frac{(x-1)^4}{24} + \frac{(x-1)^5}{30} + \ldots\right] + a_1 \left[(x-1) + \frac{(x-1)^3}{6} + \frac{(x-1)^4}{12} + \frac{(x-1)^5}{120} + \ldots\right]$$

it is the same equation as Eq. (3.282) with

$$Ai(x) = 1 + \frac{(x-1)^2}{2} + \frac{(x-1)^3}{6} + \frac{(x-1)^4}{24} + \frac{(x-1)^5}{30} + \ldots$$
$$Bi(x) = (x-1) + \frac{(x-1)^3}{6} + \frac{(x-1)^4}{12} + \frac{(x-1)^5}{120} + \ldots$$

In SymIntegration if we want to compute the power series solution for $Ai(x)$, then we will set the initial condition as

$$y(0) = 1$$
$$y'(0) = 0$$

if we want to compute the power series solution for $Bi(x)$, then we will set the initial condition as

$$y(0) = 0$$
$$y'(0) = 1$$

Below is the **main.cpp** source code to compute the $Ai(x)$ solution of the Airy differential equation $y'' - xy = 0$ near $x_0 = 0$.

```
int main(void)
{
       // Get starting timepoint
       auto start = high_resolution_clock::now();

       PolynomialDouble P({1.0, 0.0, 0.0});
       PolynomialDouble Q({0.0, 0.0, 0.0});
       PolynomialDouble R({0.0, -1.0, 0.0});

       double x0 = 0.0;

       double y0 = 1.0;
       double dy0 = 0.0;
       int num_terms = 14;

       double test_x = 0.45;

       SecondOrderODE_Homogeneous_PowerSeriesSolver solver(P, Q, R, x0, y0, dy0);
       solver.computeSeries(num_terms);

       //cout << std::fixed << std::setprecision(10);
       //solver.printCoefficients();
       solver.printSolution();

       double approximation = solver.evaluateAt(test_x, 10);
       cout << "\nx = " << test_x << std::endl;
       cout << "Power Series Approximation at x : " << std::fixed << std::
           setprecision(6) << approximation << endl;

...
       return 0;
}
```

Code 44: *SymIntegration/Examples/Differential Equations/Test SymIntegration Series Solutions for Homogeneous Second Order Linear Differential Equation with Variable Coefficients/main.cpp*

```
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Differential Equations/Test SymIntegration Series Solutions for Homogeneous Second Order Linear D
ifferential Equation with Variable Coefficients ]# ./main
( 1 ) y'' + ( 0 ) y' + (  - 1x ) y
Initial conditions: y(0) = 1 , y'(0) = 0

x_{0} = 0
Computed Power Series Coefficients:
c_{0} =          1
c_{1} =          0
c_{2} =         -0
c_{3} =   0.166667
c_{4} =         -0
c_{5} =         -0
c_{6} = 0.00555556
c_{7} =         -0
c_{8} =         -0
c_{9} = 7.71605e-05
c_{10} =         -0
c_{11} =         -0
c_{12} = 5.84549e-07
c_{13} =         -0

Series solution:

y(x) = 1 + 0.166667*(x - 0)^3 + 0.00555556*(x - 0)^6 + 7.71605e-05*(x - 0)^9 + 5.84549e-07*(x - 0)^12 + ...

x = 0.45
Power Series Approximation at x : 1.015234

Time taken by function: 851 microseconds
```

Figure 3.74: *The power series solution of Airy differential equation $y'' - xy = 0$ for $Ai(x)$ with initial condition $y(0) = 1,\ y'(0) = 0$ computed with* ***class SecondOrderODE_Homogeneous_PowerSeriesSolver****, $n = 14$ (highest degree of the solution is at $n - 1$) near a regular point $x_0 = 0$ (****SymIntegration/Examples/Differential Equations/Test SymIntegration Series Solutions for Homogeneous Second Order Linear Differential Equation with Variable Coefficients/main.cpp****).*

```
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Differential Equations/Test SymIntegration Series Solutions for Homogeneous Second Order Linear D
ifferential Equation with Variable Coefficients ]# ./main
( 1 ) y'' + ( 0 ) y' + (  - 1x ) y
Initial conditions: y(0) = 0 , y'(0) = 1

x_{0} = 0
Computed Power Series Coefficients:
c_{0} =          0
c_{1} =          1
c_{2} =         -0
c_{3} =         -0
c_{4} =  0.0833333
c_{5} =         -0
c_{6} =         -0
c_{7} = 0.00198413
c_{8} =         -0
c_{9} =         -0
c_{10} = 2.20459e-05
c_{11} =         -0
c_{12} =         -0
c_{13} = 1.4132e-07

Series solution:

y(x) = 1*(x - 0) + 0.0833333*(x - 0)^4 + 0.00198413*(x - 0)^7 + 2.20459e-05*(x - 0)^10 + 1.4132e-07*(x - 0)^13 + ...

x = 0.45
Power Series Approximation at x : 0.453425

Time taken by function: 861 microseconds
```

Figure 3.75: *The power series solution of Airy differential equation $y'' - xy = 0$ for $Bi(x)$ with initial condition $y(0) = 0,\ y'(0) = 1$ computed with* ***class SecondOrderODE_Homogeneous_PowerSeriesSolver****, $n = 14$ (highest degree of the solution is at $n - 1$) near a regular point $x_0 = 0$ (****SymIntegration/Examples/Differential Equations/Test SymIntegration Series Solutions for Homogeneous Second Order Linear Differential Equation with Variable Coefficients/main.cpp****).*

```
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Differential Equations/Test SymIntegration Series Solutions for Homogeneous Second Order Linear Differential
 Equation with Variable Coefficients ]# ./main
( 1 ) y'' + ( 0 ) y' + (  - 1x ) y
Initial conditions: y(0) = 1 , y'(0) = 0

x_{0} = 1
Computed Power Series Coefficients:
c_{0} =          1
c_{1} =          0
c_{2} =        0.5
c_{3} =   0.166667
c_{4} =  0.0416667
c_{5} =  0.0333333
c_{6} = 0.00694444
c_{7} = 0.00178571
c_{8} = 0.000719246
c_{9} = 0.000121252
c_{10} = 2.78329e-05
c_{11} = 7.64089e-06
c_{12} = 1.12943e-06
c_{13} = 2.27396e-07

Series solution:

y(x) = 1 + 0.5*(x - 1)^2 + 0.166667*(x - 1)^3 + 0.0416667*(x - 1)^4 + 0.0333333*(x - 1)^5 + 0.00694444*(x - 1)^6 + 0.00178571*(x - 1)^7 + 0.000719246*(x - 1)^8 + 0.000121
252*(x - 1)^9 + 2.78329e-05*(x - 1)^10 + 7.64089e-06*(x - 1)^11 + 1.12943e-06*(x - 1)^12 + 2.27396e-07*(x - 1)^13 + ...

x = 0.45
Power Series Approximation at x : 1.118827

Time taken by function: 877 microseconds
```

Figure 3.76: *The power series solution of Airy differential equation $y'' - xy = 0$ for $Ai(x)$ with initial condition $y(0) = 1,\ y'(0) = 0$ computed with* ***class SecondOrderODE_Homogeneous_PowerSeriesSolver****, $n = 14$ (highest degree of the solution is at $n - 1$) near a regular point $x_0 = 1$ (****SymIntegration/Examples/Differential Equations/Test SymIntegration Series Solutions for Homogeneous Second Order Linear Differential Equation with Variable Coefficients/main.cpp****).*

```
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Differential Equations/Test SymIntegration Series Solutions for Homogeneous Second Order Linear Differential
 Equation with Variable Coefficients ]# ./main
( 1 ) y'' + ( 0 ) y' + (  - 1x ) y
Initial conditions: y(0) = 0 , y'(0) = 1

x_{0} = 1
Computed Power Series Coefficients:
c_{0} =          0
c_{1} =          1
c_{2} =         -0
c_{3} =   0.166667
c_{4} =  0.0833333
c_{5} = 0.00833333
c_{6} = 0.00833333
c_{7} = 0.00218254
c_{8} = 0.000297619
c_{9} = 0.000146054
c_{10} = 2.75573e-05
c_{11} = 4.03339e-06
c_{12} = 1.31524e-06
c_{13} = 2.02505e-07

Series solution:

y(x) = 1*(x - 1) + 0.166667*(x - 1)^3 + 0.0833333*(x - 1)^4 + 0.00833333*(x - 1)^5 + 0.00833333*(x - 1)^6 + 0.00218254*(x - 1)^7 + 0.000297619*(x - 1)^8 + 0.000146054*(x
 - 1)^9 + 2.75573e-05*(x - 1)^10 + 4.03339e-06*(x - 1)^11 + 1.31524e-06*(x - 1)^12 + 2.02505e-07*(x - 1)^13 + ...

x = 0.45
Power Series Approximation at x : 0.468836

Time taken by function: 875 microseconds
```

Figure 3.77: *The power series solution of Airy differential equation $y'' - xy = 0$ for $Bi(x)$ with initial condition $y(0) = 0,\ y'(0) = 1$ computed with* ***class SecondOrderODE_Homogeneous_PowerSeriesSolver****, $n = 14$ (highest degree of the solution is at $n - 1$) near a regular point $x_0 = 1$ (****SymIntegration/Examples/Differential Equations/Test SymIntegration Series Solutions for Homogeneous Second Order Linear Differential Equation with Variable Coefficients/main.cpp****).*

Remark

Homogeneous second order linear ordinary differential equations with variable coefficients appear as $y'' + p(x)y' + q(x)y = 0$. To compute the solutions we can use advanced techniques like power series method, Frobenius method, or reduction of order because constant-root methods fail.
They model advanced systems where parameters change with location or time, such as quantum mechanics, heat flow in non-uniform shapes, planetary orbits with changing gravity, non-uniform vibrating strings, electrical circuits with time-varying components and wave propagation. Some

major physical and engineering applications:

- **Vibrating Circular Membranes:** Modeled using Bessel's differential equation to find the harmonic frequencies and nodal lines of a struck drumhead. Bessel's equation arises when studying vibrations of circular membranes, acoustics in tapered horns, or heat flow in cylindrical coordinates.
- **Heat Conduction in Cyclinders:** Solves steady-plate and transient temperature distributions inside cylindrical pipes or solid metal rods.
- **Quantum Mechanics:** The Schrödinger equation reduces to this form for potentials like the quantum harmonic oscillator or hydrogen atom, yielding Legendre polynomials for angular wave functions. The behavior of a particle in a varying potential energy field reduces to a variable-coefficient ODE such as Airy's or Hermite's differential equations.
- **Electromagnetic Waveguides:** Determines transverse electric and magnetic field distributions inside coaxial cables and cylindrical wave guides.
- **Acoustics and Optics:** Explains diffraction patterns, sound waves traveling through a medium with changing density or temperature (like the atmosphere or ocean layers), planetary motion perturbations, and sound waves in tapered horns.
- **Astronomy and Aerospace:** Satellite motion and orbital mechanics under central forces with changing radial distances or atmospheric drag gradients. Legendre's differential equation models potential fields in spherical coordinates, gravity of planets, and electrostatics.
- **Non-Uniform Mechanical Systems:** Vibrations of a hanging chain or a variable cross-section beam where mass and tension change with position.
- **Electrical Engineering:** RLC circuits where resistance, inductance, or capacitance change over time (e.g., signal modulators, or time-varying filters). Transmission lines with position-dependent resistance, capacitance, or inductance use these models to compute voltage and current loss over varying distances.
- **Advanced Engineering:** Non-uniform transmission lines and tapered beams use these equations to track variable voltage drop, current dispersion, or structural load limits.

Common types of homogeneous second order linear ordinary differential equations with variable coefficients :

- **Cauchy-Euler Equation:** Used in scaling-invariant mechanical adn erospace stress problems, elasticity and radial fluid flow, it is defined by

$$x^2y'' + axy' + by = 0$$

- **Bessel Equation:** Applied to heat conduction in cylinders and circular wave propagation, it is defined by

$$x^2y'' + xy' + (x^2 - \nu^2)y = 0$$

- **Legendre Equation:** Found in spherical harmonic problems in electrostatics and quantum wave functions. Legendre's equation used in physics problems involving spherical coordinates (like gravitational and electrostatic potentials), it is defined by

$$(1 - x^2)y'' - 2xy' + n(n+1)y = 0$$

iv. Compute Power Series Solution for Homogeneous Second Order Linear Differential Equations with Variable Coefficients Near a Singular Point x_0

There are two equations that are used in the case of a regular singular point:

1. **Cauchy-Euler Equation**
   The standard form:
   $$ax^2y'' + bxy' + cy = 0$$
   The coefficient of y'' is always ax^2 or just x^2 (if divided by a). The power of x must match the order of the derivative (x^2 for y'', x for y', and x^0 for y).

2. **Frobenius Method Equation**
   The standard form:
   $$x^2y'' + xp(x)y' + q(x)y = 0$$
   The coefficient of y'' is always x^2 after multiplying the general equation through to isolate the regular singular point. A regular singular point requires $xp(x)$ and $x^2q(x)$ to be analytic (able to be written as a power series) at $x = 0$. Multiplying the base equation by x^2 puts it in this required standard form.
   If the singular point is at $x = x_0$ instead of $x = 0$, the x^2 term becomes $(x - x_0)^2$.
   If we start with a constant in front like $3x^2y''$, we must divide the entire differential equation by 3 so that the coefficient of y'' becomes x^2 before applying the Frobenius series method.

To solve a general second-order homogeneous linear differential equation near a singular point, consider this equation

$$P(x)y'' + Q(x)y' + R(x)y = 0$$

where $P(x), Q(x)$, and $R(x)$ are complex-coefficient polynomials, to compute the power series solution near a shifted regular singular point $x = x_0$ (or not shifted with $x_0 = 0$), we employ the Frobenius method. It uses a modified power series containing an explicit fractional or real power exponent, yielding an indicial equation to solve for the exponents and recurrence relations for the coefficients.

Let the polynomials be defined as:

$$P(x) = \sum_{k=0}^{D_P} p_k x^k$$
$$Q(x) = \sum_{k=0}^{D_Q} q_k x^k$$
$$R(x) = \sum_{k=0}^{D_R} r_k x^k$$

assuming that $x = 0$ is a regular singular point, we will have $P(0) = p_0 = 0, p_1 \neq 0$ (or if $p_1 = 0, q_0 = 0$ and $p_2 \neq 0$). To guarantee a regular singularity, we shift powers so that the equation matches the standard Frobenius form:

$$x^2y'' + xp(x)y' + q(x)y = 0$$

The general formula for a series solution near a singular point is commonly called a Frobenius series, which is

$$y(x) = \sum_{n=0}^{\infty} a_n (x - x_0)^{n+r}$$

where r is a root found via an indicial equation and $a_0 \neq 0$. For an ordinary (non-singular) point, if $r = 0$, then we are reducing the formula to a standard power series. From the $y(x)$ we can compute $y'(x)$ and $y''(x)$ then substitute it to the differential equation.

We can write the differential equation in this form (by dividing the differential equation with $P(x)$):

$$y'' + p(x)y' + q(x)y = 0$$

where $p(x) = \frac{Q(x)}{P(x)}$ and $q(x) = \frac{R(x)}{P(x)}$. We will compute the power series solution about the given point x_0, if $x_0 \neq 0$ then the center will be shifted by x_0.

If $p(x)$ or $q(x)$ diverges at x_0 ($P(x_0) = 0$), but $\lim_{x \to x_0}(x - x_0)p(x)$ and $\lim_{x \to x_0}(x - x_0)^2 q(x)$ are both finite (remain analytic), then x_0 is a regular singular point. This is where the Frobenius method is required. Otherwise, if $P(x_0) = 0$ and either limit $\lim_{x \to x_0}(x - x_0)p(x)$ or $\lim_{x \to x_0}(x - x_0)^2 q(x)$ blows up to infinity, the point x_0 is irregular, and we cannot use Frobenius method in this scenario.

Thus, knowing that we have a regular singular point we can assume a Frobenius series:

$$y(x) = \sum_{n=0}^{\infty} a_n (x - x_0)^{n+r}, \quad a_0 \neq 0$$

After we have classified the point x_0 and obtain the $y(x)$, we now differentiate the assumed series term-by-term to find expressions for y' and y''.

$$\begin{aligned} y(x) &= \sum_{n=0}^{\infty} a_n (x - x_0)^{n+r}, \quad a_0 \neq 0 \\ y'(x) &= \sum_{n=0}^{\infty} (n + r) a_n (x - x_0)^{n+r-1} \\ y''(x) &= \sum_{n=0}^{\infty} (n + r)(n + r - 1) a_n (x - x_0)^{n+r-2} \end{aligned}$$

Plugging this into the differential equation produces the indicial equation (a quadratic equation in r). Solving it gives roots r_1 and r_2, which dictate the structure of the recurrence relation for a_n. Then we will need to find the recurrence relation, equate coefficients of general x^{n+r} powers to zero. This creates a recursive formula linking a_n to previous coefficients.

This yields the general recurrence relation for any index m:

$$\sum_{k=0}^{m} \left[(m - k + r)(m - k + r - 1)p_k + (m - k + r)q_k + r_k \right] a_{m-k} = 0$$

After we have found the recurrence relation, substitute the roots r_1 and r_2 back into the series. The nature of the roots (whether their difference is an integer, zero, the difference is non-integer,

or complex roots) dictates whether the general solution requires two distinct series or an added logarithmic term.

The Indicial Equation ($m = 0$)

Assuming $a_0 \neq 0$, the smallest power matching requirements reveals the indicial equation:

$$F(r) = r(r-1)p_{shift} + rq_{shift} + r_{shift} = 0$$

This yields at most two distinct roots or repeated roots, r_1 and r_2.

- If we use the standard homogeneous differential equation for Frobenius method with this form:
$$x^2y'' + xp(x)y' + q(x)y = 0$$
where $p(x)$ and $q(x)$ are analytic (have regular power series expansions) at $x = 0$. We will evaluate the limiting coefficients:
$$p_0 = \lim_{x\to 0} p(x) = p(0)$$
$$q_0 = \lim_{x\to 0} q(x) = q(0)$$
Substitute p_0 and q_0 into the standard quadratic template:
$$r(r-1) + p_0r + q_0 = 0$$
$$r^2 + (p_0 - 1)r + q_0 = 0$$
We can then solve the resulting quadratic equation for the two roots r_1 and r_2. These roots determine the behavior and the form of the power series solutions.

  The recurrence relation will be:
$$a_n = \frac{-\sum_{k=0}^{n-1} a_k[(r+k)p_{n-k} + q_{n-k}]}{F(r+n)}$$
  If the roots differ by integer, computing the second solutiona can trigger a division-by-zero if the denominator collapses to zero. For a robust framework, the second solution would require a logarithmic tem conversion ($\ln(x)$).
- If we use shifted mechanism, where $x = x_0$ and $x_0 \neq 0$, we use substitution $t = x - x_0$ to shift the singular point to the origin ($t = 0$).
  The equation standard form will be:
$$t^2y'' + tp(t)y' + q(t)y = 0$$
  where $p(t) = \frac{tQ(t)}{P(t)}$ and $q(t) = \frac{t^2R(t)}{P(t)}$ are analytical polynomials.
  The quadratic equation will still be the same
$$r(r-1) + p_0r + q_0 = 0$$
  For each root r, the coefficients are found iteratively with this recurrence relation:
$$a_n = \frac{-1}{F(r+n)} \sum_{k=0}^{n-1} a_k[p_{n-k}(r+k) + q_{n-k}]$$

where $F(r+n)$ is the indicial function evaluated at $r+n$. Also if $r_1 - r_2$ is an integer or zero, the second independent solution requires a logarithmic term ($\ln(t)$ term multiplier).

The Recurrence Relation (for $n \geq 1$):

$$a_n = \frac{-1}{F(n+r)} \sum_{k=0}^{n-1} a_k[(k+r)p_{n-k} + q_{n-k}]$$

where $F(n+r) = (n+r)(n+r-1) + p_0(n+r) + q_0$.

Another method:
Setting the lowest power coefficient (x^r) to zero yields a quadratic equation in terms of r / the indicial equation:

$$f(r) = 0$$
$$r(r-1) + p_0 r + q_0 = 0$$

Solve this to find two roots, r_1 and r_2 (where $\text{Re}(r_1) \geq \text{Re}(r_2)$).
Equate the general coefficient of x^{n+r} to zero for $n \geq 1$, producing a linear relation linking a_n to preceding coefficients $a_0, a_1, \ldots, a_{n-1}$:

$$a_n(n+r)(n+r-1) + \sum_{k=0}^{n} [p_{n-k}(k+r) + q_{n-k}]\, a_k = 0$$

Shifted Polynomials

When $P(x), Q(x)$, and $R(x)$ are polynomials, we shift our perspective to $t = x - x_0$ so that the singular point becomes centered at $t = 0$. The differential equation is then reframed as

$$p(t)y''(t) + q(t)y'(t) + r(t)y(t) = 0$$

Where $p(t) = P(t+x_0), q(t) = Q(t+x_0)$, and $r(t) = R(t+x_0)$ are shifted polynomials. Since x_0 is a regular singular point, $p(t)$ must have a root at $t = 0$ with a multiplicity of at most 2. The Frobenius solution takes the form

$$y(t) = \sum_{n=0}^{\infty} a_n(t)^{n+r}$$

There are 4 conditions of the roots of the indicial equation and we will have several examples for certain type of root so the reader can comprehend them more:

1. **Roots of the Indicial Equation are Distinct, They Do Not Differ by an Integer ($r_1 - r_2 \notin \mathbb{Z}$)**

   Solve and compute the first 6 non-zero terms of the differential equation

   $$2x^2y'' + 3xy' + (x^2 - 1)y = 0$$

   using the Frobenius method.

   Solution:

(a) **Verify that $x = 0$ is a Regular Singular Point**
First, we write the differential equation in standard form:

$$y'' + p(x)y' + q(x)y = 0$$

dividing the original equation by x^2, we will obtain

$$p(x) = \frac{3x}{2x^2} = \frac{3}{2x}$$
$$q(x) = \frac{x^2 - 1}{x^2} = 1 - \frac{1}{x^2}$$

Since both $p(x)$ and $q(x)$ are undefined at $x = 0$, then $x = 0$ is a singular point.
To determine if it is a regular singular point, we check the limits of $xp(x)$ and $x^2q(x)$ as $x \to 0$:

$$\lim_{x\to 0} xp(x) = \lim_{x\to 0} x\frac{3}{2x} = 3$$
$$\lim_{x\to 0} x^2q(x) = \lim_{x\to 0} x^2\left(1 - \frac{1}{x^2}\right) = -1$$

Because both limits are finite, $xP(x)$ and $x^2Q(x)$ are analytic at $x = 0$. Therefore $x = 0$ is a regular singular point and we can safely use the Frobenius method.

(b) **The Frobenius Assumption and the Indicial Equation**
According to the Frobenius method, we assume a series solution of the form:

$$y(x) = \sum_{n=0}^{\infty} a_n x^{n+r}, \quad a_0 \neq 0$$

Computing the first and second derivatives, we will have

$$y'(x) = \sum_{n=0}^{\infty} a_n(n+r)x^{n+r-1}$$
$$y''(x) = \sum_{n=0}^{\infty} a_n(n+r)(n+r-1)x^{n+r-2}$$

Substitute these derivatives back into the differential equation, we will obtain

$$2x^2 \sum_{n=0}^{\infty} a_n(n+r)(n+r-1)x^{n+r-2} + 3x \sum_{n=0}^{\infty} a_n(n+r)x^{n+r-1} + (x^2-1) \sum_{n=0}^{\infty} a_n x^{n+r} = 0$$

Distributing the powers of x

$$\sum_{n=0}^{\infty} 2a_n(n+r)(n+r-1)x^{n+r} + \sum_{n=0}^{\infty} 3a_n(n+r)x^{n+r} + \sum_{n=0}^{\infty} a_n x^{n+r+2} - \sum_{n=0}^{\infty} a_n x^{n+r} = 0$$

Combine the terms containing x^{n+r}

$$\sum_{n=0}^{\infty} a_n[2(n+r)(n+r-1) + 3(n+r) - 1]x^{n+r} + \sum_{n=0}^{\infty} a_n x^{n+r+2} = 0$$

Simplifying the bracketed term gives the polynomial

$$P(n+r) = 2(n+r)^2 + (n+r) - 1 = (2(n+r)-1)(n+r+1)$$

So, the simplified series equation is

$$\sum_{n=0}^{\infty} a_n(2(n+r)-1)(n+r+1)x^{n+r} + \sum_{n=0}^{\infty} a_n x^{n+r+2} = 0$$

(c) **Find the Roots of the Indicial Equation**
To find the lowest power coefficient ($n = 0$), isolate the x^r term

$$a_0(2r-1)(r+1) = 0, \quad a_0 \neq 0$$
$$r_1 = \frac{1}{2}$$
$$r_2 = -1$$

Because the difference between roots $r_1 - r_2 = \frac{3}{2}$ is not an integer, we are guaranteed two distinct, purely algebraic series solutions without logarithmic terms.

(d) **Recurrence Relation**
For $n = 1$, the coefficient of x^{r+1} is

$$a_1(2r+1)(r+2) = 0$$

Since neither root makes $(2r+1)(r+2) = 0$, we find that $a_1 = 0$ for both cases.

Shifting the index of the second summation ($\sum_{n=2}^{\infty} a_{n-2}x^{n+r}$), we establish the general recurrence relation for $n \geq 2$:

$$a_n = -\frac{a_{n-2}}{(2(n+r)-1)(n+r+1)}$$

Because $a_1 = 0$, all odd-indexed terms ($a_3, a_5, a_7, dots$) drop to zero. Thus, we only compute even terms where $n = 2k$.

(e) **Find the First General Solution (y_1)**
The first solution corresponding to the larger root r_1:

$$y_1(x) = x^{r_1} \sum_{n=0}^{\infty} a_n(r_1)x^n, \quad a_0 \neq 0$$

(f) **Find the Second Linearly Independent Solution (y_2)**
The first solution corresponding to the smaller root r_2:

$$y_2(x) = x^{r_2} \sum_{n=0}^{\infty} b_n(r_2)x^n, \quad b_0 \neq 0$$

(g) **Final General Solution**
The formula for the general solution is

$$y(x) = C_1 y_1(x) + C_2 y_2(x)$$

where $y_1(x)$ uses r_1 and $y_2(x)$ uses r_2. The constants C_1 and C_2 can be computed with initial conditions.

(h) **The First 6 Non-Zero Terms for the Power Series Solutions**
For the first root, setting $r = \frac{1}{2}$ and choosing $a_0 = 1$, the recurrence relation simplifies to

$$a_n = -\frac{a_{n-2}}{n(2n+3)}$$

Evaluating the first 6 non-zero terms, we will obtain

$$\begin{aligned} a_0 &= 1 \\ a_2 &= -\frac{1}{14} \\ a_4 &= \frac{1}{616} \\ a_6 &= -\frac{1}{55,440} \\ a_8 &= \frac{1}{8,426,880} \\ a_{10} &= -\frac{1}{1,938,182,400} \end{aligned}$$

Thus, we will have

$$y_1(x) = x^{\frac{1}{2}}\left(1 - \frac{1}{14}x^2 + \frac{1}{616}x^4 - \frac{1}{55,440}x^6 + \frac{1}{8,426,880}x^8 - \frac{1}{1,938,182,400}x^{10}\right)$$

For the second root, setting $r = -1$ and choosing $b_0 = 1$, the recurrence relation simplifies to

$$b_n = -\frac{b_{n-2}}{n(2n-3)}$$

Evaluating the first 6 non-zero terms, we will obtain

$$\begin{aligned} a_0 &= 1 \\ a_2 &= -\frac{1}{2} \\ a_4 &= \frac{1}{40} \\ a_6 &= -\frac{1}{2,160} \\ a_8 &= \frac{1}{224,640} \\ a_{10} &= -\frac{1}{38,188,800} \end{aligned}$$

Thus, we will have

$$y_2(x) = x^{-1}\left(1 - \frac{1}{2}x^2 + \frac{1}{40}x^4 - \frac{1}{2,160}x^6 + \frac{1}{224,640}x^8 - \frac{1}{38,188,800}x^{10}\right)$$

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Frobenius as Class ]# ./main

P(x) y'' + Q(x)y'+ R(x) y = 0

P(x) = (0,0) * x^0 + (0,0) * x^1 + (2,0) * x^2

Q(x) = (0,0) * x^0 + (3,0) * x^1

R(x) = (-1,0) * x^0 + (0,0) * x^1 + (1,0) * x^2

x0 = 0
r1 = (0.5,0)
r2 = (-1,-0)

Case 2: Different roots; roots differ by non-integer.

Series solution:

y1(x) = x^(0.5,0) (1 - 0.0714286*x^2 + 0.00162338*x^4 - 1.80375e-05*x^6 + 1.18668e-07*x^8 )
y2(x) = x^(-1,-0) (1 - 0.5*x^2 + 0.025*x^4 - 0.000462963*x^6 + 4.45157e-06*x^8 - 2.61857e-08*x^10  )

Computed Power Series Coefficients
```

Figure 3.78: *The power series solution of* $2x^2y'' + 3xy' + (x^2 - 1)y = 0$ *computed with* ***class SecondOrderODE_Homogeneous_Frobenius_PowerSeriesSolver****, it is categorized as different roots that differ by non-integer with* $r_1 = \frac{1}{2}$ *and* $r_2 = -1$*, and* $x_0 = 0$ *is a regular singular value (****SymIntegration/Examples/Differential Equations/Test SymIntegration Frobenius as Class/main.cpp****).*

```
Computed Power Series Coefficients

y1(x) coefficients (r = (0.5,0)):
a_{0} = (1,0)
a_{1} = (-0,0)
a_{2} = (-0.0714286,0)
a_{3} = (-0,0)
a_{4} = (0.00162338,-0)
a_{5} = (-0,0)
a_{6} = (-1.80375e-05,0)
a_{7} = (-0,0)
a_{8} = (1.18668e-07,-0)
a_{9} = (-0,0)
a_{10} = (-5.15947e-10,0)
a_{11} = (-0,0)
a_{12} = (1.59243e-12,-0)
a_{13} = (-0,0)
a_{14} = (-3.66919e-15,0)
a_{15} = (-0,0)


y2(x) coefficients (r = (-1,-0)):
b_{0} = (1,0)
b_{1} = (-0,0)
b_{2} = (-0.5,0)
b_{3} = (-0,0)
b_{4} = (0.025,-0)
b_{5} = (-0,0)
b_{6} = (-0.000462963,0)
b_{7} = (-0,0)
b_{8} = (4.45157e-06,-0)
b_{9} = (-0,0)
b_{10} = (-2.61857e-08,0)
b_{11} = (-0,0)
b_{12} = (1.03911e-10,-0)
b_{13} = (-0,0)
b_{14} = (-2.9689e-13,0)
b_{15} = (-0,0)

Time taken by function: 2034 microseconds
```

Figure 3.79: *The power series solution of* $2x^2y'' + 3xy' + (x^2 - 1)y = 0$ *computed with* ***class SecondOrderODE_Homogeneous_Frobenius_PowerSeriesSolver****, it is categorized as different roots that differ by non-integer with* $r_1 = \frac{1}{2}$ *and* $r_2 = -1$*, and* $x_0 = 0$ *is a regular singular value (****SymIntegration/Examples/Differential Equations/Test SymIntegration Frobenius as Class/main.cpp****).*

2. **Roots of the Indicial Equation are Distinct, They Differ by an Integer (**$r_1 - r_2 = N \in \mathbb{Z}^+,\ r_1 > r_2$**)**
   Solve and compute the first 6 non-zero terms of the differential equation

$$x^2y'' + xy' + (x^2 - 1)y = 0$$

   using the Frobenius method.

Solution:

(a) **Verify that $x = 0$ is a Regular Singular Point**
First, we write the differential equation in standard form:

$$y'' + p(x)y' + q(x)y = 0$$

dividing the original equation by x^2, we will obtain

$$p(x) = \frac{x}{x^2} = \frac{1}{x}$$
$$q(x) = \frac{x^2 - 1}{x^2} = 1 - \frac{1}{x^2}$$

Since both $p(x)$ and $q(x)$ are undefined at $x = 0$, then $x = 0$ is a singular point.
To determine if it is a regular singular point, we check the limits of $xp(x)$ and $x^2q(x)$ as $x\ to0$:

$$p_0 = \lim_{x\to 0} xp(x) = \lim_{x\to 0} x\frac{1}{x} = 1$$
$$q_0 = \lim_{x\to 0} x^2q(x) = \lim_{x\to 0} x^2\frac{x^2-1}{x^2} = -1$$

Because both limits are finite, $xP(x)$ and $x^2Q(x)$ are analytic at $x = 0$. Therefore $x = 0$ is a regular singular point.

(b) **Formulate the Indicial Equation**
According to the Frobenius method, we assume a series solution of the form:

$$y(x) = \sum_{n=0}^{\infty} a_n x^{n+r}, \quad a_0 \neq 0$$

Computing the first and second derivatives, we will have

$$y'(x) = \sum_{n=0}^{\infty} a_n(n+r)x^{n+r-1}$$
$$y''(x) = \sum_{n=0}^{\infty} a_n(n+r)(n+r-1)x^{n+r-2}$$

Substitute these derivatives back into the differential equation $x^2y'' + xy' + (x^2-1)y = 0$

$$x^2\sum_{n=0}^{\infty} a_n(n+r)(n+r-1)x^{n+r-2} + x\sum_{n=0}^{\infty} a_n(n+r)x^{n+r-1} + (x^2-1)\sum_{n=0}^{\infty} a_n x^{n+r} = 0$$
$$\sum_{n=0}^{\infty} a_n(n+r)(n+r-1)x^{n+r} + \sum_{n=0}^{\infty} a_n(n+r)x^{n+r} + \sum_{n=0}^{\infty} a_n x^{n+r+2} - \sum_{n=0}^{\infty} a_n x^{n+r} = 0$$

Combine the terms containing x^{n+r}:

$$\sum_{n=0}^{\infty}[(n+r)(n+r-1) + (n+r) - 1]a_n x^{n+r} + \sum_{n=0}^{\infty} a_n x^{n+r+2} = 0$$

Simplify the algebraic expression inside the first bracket:

$$(n+r)^2-(n+r)+(n+r)-1=(n+r)^2-1$$

This gives us the simplified summation expression:

$$\sum_{n=0}^{\infty}[(n+2)^2-1]a_n x^{n+r}+\sum_{n=0}^{\infty}a_n x^{n+r+2}=0$$

(c) **Find the Roots of the Indicial Equation**
to find the indicial equation, we look at the lowest power of x, which occurs when $n=0$ in the first summation (x^r). Setting its coefficient to zero (since $a_0 \neq 0$) we will obtain the indicial equation and able to compute the roots:

$$\begin{aligned}(0+r)^2-1&=0\\ r^2&=1\end{aligned}$$

Solving for r yields the two roots $r_1=1$ and $r_2=-1$. The roots of the indicial equation are distinct $r_1 \neq 1$ and their difference is $r_1-r_2=2$, which is an integer. This requires special handling for the second linearly independent solution (typically introducing a logarithmic term). Note that this specific differential equation is Bessel's equation of order 1.

(d) **Determine the Recurrence Relation**
We examine the simplified summation expression that we have computed above

$$\sum_{n=0}^{\infty}[(n+2)^2-1]a_n x^{n+r}+\sum_{n=0}^{\infty}a_n x^{n+r+2}=0$$

For any $n \geq 2$, the shifting of indices yields the general recurrence relation:

$$a_n(r)=-\frac{a_{n-2}(r)}{(n+r)^2-1}=-\frac{a_{n-2}(r)}{(n+r-1)(n+r+1)}$$

For $n=1$, we find $a_1=0$, meaning all odd-indexed coefficients are zero ($a_1=a_3=a_5=\cdots=0$).

(e) **Find the First General Solution (y_1)**
The first solution $y_1(x)$ is always solvable using the larger root r_1. Compute its coefficients $a_n(r_1)$ sequentially using $a_0=1$.

Substituting $r=1$ into the recurrence relation gives

$$a_n=-\frac{a_{n-2}}{n(n+2)}$$

Setting the arbitrary constant $a_0 = 1$

$$a_2 = -\frac{1}{2(4)} = -\frac{1}{8}$$
$$a_4 = -\frac{-1/8}{4(6)} = \frac{1}{192}$$
$$a_6 = -\frac{1/192}{6(8)} = -\frac{1}{9216}$$
$$a_8 = -\frac{-1/9216}{8(10)} = \frac{1}{737280}$$
$$a_{10} = -\frac{1/737280}{10(12)} = -\frac{1}{88473600}$$

Remember the general recurrence relation

$$a_n(r) = -\frac{a_{n-2}(r)}{(n+r-1)(n+r+1)}$$

Since $a_1 = 0$, all odd coefficients are zero ($a_1 = a_3 = a_5 = \cdots = 0$). Let $n = 2k$ for even indices:

$$a_{2k}(r) = \frac{-a_{2k-2}}{(r+2k-1)(r+2k+1)}$$

Iterating this gives the general coefficient in terms of r:

$$a_{2k}(r) = \frac{(-1)^k a_0}{(r+1)(r+3)^2(r+5)^2 \dots (r+2k-1)^2(r+2k-1)}$$

Substitute $r = 1$ into the recurrence formula:

$$a_{2k}(1) = \frac{(-1)^k a_0}{2 \cdot 4^2 \cdot 6^2 \dots (2k)^2 \cdot (2k+2)} = \frac{(-1)^k a_0}{2^{2k} k!(k+1)!}$$

Setting $a_0 = 1$, the first linearly independent solution is:

$$y_1(x) = x^{r_1} \sum_{k=0}^{\infty} \frac{(-1)^k}{2^{2k}(k!)(k+1)!} x^{2k} = \sum_{k=0}^{\infty} \frac{(-1)^k}{2^{2k}(k!)(k+1)!} x^{2k+1}$$

Note that this is the Bessel function of the first kind, $J_1(x)$.

$$y_1(x) = C_1 y_1(x)$$

(f) **Find the Second Linearly Independent Solution (y_2)**
A unique coupling occurs: computing the second linearly independent solution directly via the standard recurrence relation can encounter a division-by-zero or require a logarithmic component. The larger root r_1 always yields a clean, direct power series solution. However, plugging in the smaller root r_2 causes the denominator of the recurrence relation $F(r_2 + N)$ to become zero when calculating the N-th coefficient. Thus, the first solution $y_1(x)$ uses r_1 (the larger root).

The second linearly independent solution $y_2(x)$ may involve a logarithmic term and the formula is:

$$y_2(x) = Cy_1(x)\ln(x) + \sum_{n=0}^{\infty} b_n x^{n+r_2}$$

To systematically obtain both C and b_n algorithmically without symbolic derivatives, we substitute $r = r_2$ into the recurrence system while keeping a_0 as an indeterminate variable. At $n = N$, the system yields an equation of the form:

$$0 \cdot a_N + \text{known_terms}(a_0, \ldots, a_{N-1}) = 0$$

If known_terms $= 0$, then $C = 0$. a_N becomes an arbitrary constant (we can pick $a_N = 0$), and no logarithm is needed.

If known_terms $\neq 0$, a log term is required ($C \neq 0$). The safest numerical implementation uses a Frobenius' generalized parameter technique, evaluating the recurrence treating r as a continuous variable near r_2, or directly solving the coupled log-series equations.

Handling the Root Breakout and Finding $y_2(x)$
If we substitute $r_2 = -1$ directly into the recurrence formula for $n = 2$:

$$a_2(-1) = -\frac{a_0}{(2-1)^2 - 1} = -\frac{a_0}{0}$$

This division by zero causes a breakdown. To circumvent this via the Frobenius method, we define the coefficients as functions of r and scale them by multiplying by $r - r_2 = (r+1)$:

$$a_0(r) = r + 1$$

Using the general recurrence relation in terms of r:

$$a_2(r) = -\frac{r+1}{(r+1)(r+3)} = -\frac{1}{r+3}$$
$$a_4(r) = -\frac{a_2(r)}{(r+3)(r+5)} = \frac{1}{(r+3)^2(r+5)}$$

The second linearly independent solution is then found by taking the partial derivative with respect to r evaluated at $r_2 = -1$:

$$y_2(x) = \frac{\partial}{\partial r}\left[\sum_{n=0}^{\infty} a_n(r) x^{n+r}\right]\Big|_{r=-1}$$

Applying the product rule introduces the characteristic logarithmic term:

$$y_2(x) = \left[\lim_{r\to -1} \frac{\partial a_n(r)}{\partial r}\right] x^{n-1} + \left[\lim_{r\to -1} a_n(r)\right] x^{n-1}\ln(x)$$

If we plug $r_2 = -1$ directly into $a_{2k}(r)$:

$$a_{2k}(r) = \frac{(-1)^k a_0}{(r+1)(r+3)^2(r+5)^2 \ldots (r+2k-1)^2(r+2k-1)}$$

the $(r+1)$ term in the denominator causes a division by zero at $k=1$. To bypass this singularity, we define a modified coefficient function:

$$A_{2k}(r) = (r-r_2)a_{2k}(r) = (r+1)a_{2k}(r)$$
$$A_{2k}(r) = \frac{(-1)^k a_0}{(r+3)^2(r+5)^2\ldots(r+2k-1)^2(r+2k+1)}$$

Because the denominator of the recurrence relation becomes zero for $n=2$ when $r=-1$, we redefine the coefficients as $A_n(r) = (r-r_2)a_n(r) = (r+1)a_n(r)$, choosing $A_0(r) = r+1$.
The second solution is determined via the derivative with respect to r:

$$y_2(x) = \left[\frac{\partial}{\partial r}\sum_{n=0}^{\infty} A_n(r)x^{n+r}\right]_{r=-1} = \ln(x)\sum_{n=0}^{\infty} A_n(-1)x^{n-1} + \sum_{n=0}^{\infty} A_n'(-1)x^{n-1}$$

Evaluating the limits and derivatives of the coefficients at $r=-1$

$$\begin{aligned}
n=0 \quad & A_n(-1) = 0, \quad A_n'(-1) = 1 \\
n=2 \quad & A_n(-1) = -\frac{1}{2}, \quad A_n'(-1) = \frac{1}{4} \\
n=4 \quad & A_n(-1) = \frac{1}{16}, \quad A_n'(-1) = -\frac{5}{64} \\
n=6 \quad & A_n(-1) = -\frac{1}{384}, \quad A_n'(-1) = \frac{5}{1152} \\
n=8 \quad & A_n(-1) = \frac{1}{18,432}, \quad A_n'(-1) = -\frac{47}{442,368} \\
n=10 \quad & A_n(-1) = -\frac{1}{1,474,560}, \quad A_n'(-1) = \frac{131}{88473600}
\end{aligned}$$

All odd terms where $n = 1,3,5,7,9,11,\ldots$ evaluate to 0.

The Logarithmic Term (C)
Remember the general recurrence relation that is derived fom the Frobenius substitution:

$$a_n(r) = -\frac{a_{n-2}(r)}{(n+r)^2-1} = -\frac{a_{n-2}(r)}{(n+r-1)(n+r+1)}$$

When the roots differ by an integer N, the standard method to remove the division-by-zero singularity is to define the initial coefficients as

$$a_0(r) = r - r_2 = r+1$$

The coefficient C of the $y_1(x)\ln(x)$ term is then given by evaluating $a_N(r)$ at $r=r_2$:

$$C = \lim_{r\to r_2} a_N(r) = c_2(-1)$$

To evaluate it, set

$$a_0(r) = r+1$$

Find $a_2(r)$ using the recurrence relation for $n=2$:

$$a_2(r) = -\frac{a_0(r)}{(r+1)(r+3)} = -\frac{r+1}{(r+1)(r+3)} = -\frac{1}{r+3}$$

Evaluate $a_2(r)$ at $r = r_2 = -1$

$$C = -\frac{1}{-1+3} = -\frac{1}{2}$$

Thus, the logarithmic portion of the second Frobenius solution reduces to $C = -\frac{1}{2}$.

Note that if $y_1(x)$ is normalized exactly to the standard Bessel function

$$J_1(x) = \frac{x}{2} - \frac{x^3}{16} + \frac{x^5}{384} - \cdots + \ldots$$

the scaling coefficient of the log term matches -1 relative to $J_1(x)$ due to the factor of $\frac{1}{2}$ in the first term of the J_1 series

$$Cy_1(x) = (-1)J_1(x)$$

The Series Term
Taking the derivative $A'_{2n}(r)$ using logarithmic differentiation and evaluating at $r = -1$. For $n = 0$

$$A_0(r) = (r+1)a_0$$
$$A'_0(-1) = 1$$

For $k \geq 1$:

$$A'_{2n}(-1) = b_{2n} = -\frac{(-1)^n}{2^{2n-1}n!(n-1)!}\left(H_{n-1} + \frac{1}{2n}\right) = \frac{(-1)^{n+1}}{2^{2n-1}n!(n-1)!}\left(H_{n-1} + \frac{1}{2n}\right)$$

where $H_n = \sum_{k=1}^{n} \frac{1}{k}$ is the n-th harmonic number with $H_0 = 0$.

Combining both parts yields the second independent solution:

$$y_2(x) = Cy_1(x)\ln(x) + x^{r_2}\sum_{n=1}^{\infty} \frac{(-1)^n\left(H_{n-1} + \frac{1}{2n}\right)}{2^{2n-1}n!(n-1)!}x^{2n}$$

We can also write the second linearly independent solution in this form:

$$y_2(x) = Cy_1(x)\ln(x) + x^{r_2}\sum_{n=0}^{\infty} b_n x^n = Cy_1(x)\ln(x) + \sum_{n=0}^{\infty} b_n x^{n-1}$$

Using the formal derivative-to-index approach

$$\frac{\partial}{\partial r}[r - r_2 y(x,r)]\big|_{r=r_2}$$

the constant scales to $C = -0.5$ relative to a leading power series term initialized at $b_0 = 1$.

About the Log Term(C)
The second solution $y_2(x)$ contains a log term scaled by a constant C, alongside a standard series:

$$y_2(x) = Cy_1(x)\ln(x) + x^{r_2}\sum_{n=0}^{\infty} b_n x^n$$

Here, $y_1(x)$ is the first solution corresponding to the larger root r_1, and C is a constant determined by the recurrence relation that may or may not be zero. Depending on the technique used, C can be found using one of three primary methods:

- **The Limit/Derivative Method (Frobenius's Original Method)**
  When tracking the series coefficients $a_n(r)$ as functions of an arbitrary root variable r, the denominator of the N-th coefficient $a_N(r)$ will contain a term $(r - r_2)$, causing it to blow up (become infinite) as $r \to r_2$.
  To remove this singularity, the entire series is multiplied by $(r - r_2)$. Through this differentiation process, the constant C is defined as the residue limit of the N-th coefficient:
  $$C = \lim_{r \to r_2} \left[(r - r_2) \frac{a_N(r)}{a_0} \right]$$
  If we set the initial coefficient to $a_0(r) = r - r_2$ to clear the denominator upfront, then C is simply the value of the N-th coefficient evaluated at the smaller root:
  $$C = a_N(r_2)$$
- **The Direct Substitution Method**
  If we plug the trial solution $y_2(x) = Cy_1(x)\ln(x) + \sum_{n=0}^{\infty} b_n x^{n+r_2}$ directly back into the original differential equation:
  i. The $\ln(x)$ terms will cancel out automatically because $y_1(x)$ is already a solution.
  ii. We will be left with a new recurrence relation for the coefficients b_n.
  iii. When you reach the N-th index ($n = N$), the recurrence relation will yield an equation involving both b_0 and the unknown constant C. We then solve this algebraic equation for C (usually in terms of b_0, which is arbitrarily set to 1).
- **Reduction of Order (Wronskian Method)**
  We can bypass the Frobenius derivation for $y_2(x)$ by using the reduction of order formula:
  $$y_2(x) = y_1(x) \int \frac{e^{-\int P(x)\ dx}}{[y_1(x)]^2} dx$$
  When you expand the inside integrand into a Laurent series, it will take the form:
  $$\frac{e^{-\int P(x)\ dx}}{[y_1(x)]^2} = \frac{C}{x} + \sum_{n \neq -1} d_n x^n$$
  Integrating this series term-by-term yields $C\ln(x)$ from the $\frac{1}{x}$ term, meaning C is exactly the coefficient of the x^{-1} term in the expanded integrand.

Summary of outcomes for C:

- $C \neq 0$: The logarithmic term is required because the smaller root fails to yield a clean, independent power series (common in equations like the Bessel equation of integer order).
- $C = 0$: No logarithmic term is needed (y_2 is a pure power series). This happens if the recurrence relation naturally turns out to be consistent at $n = N$ without forcing a division by zero.

(g) **Final General Solution**

$$y(x) = C_1 y_1(x) + C_2 y_2(x)$$

(h) **The First 6 Non-Zero Terms for the Power Series Solutions**
The first solution is

$$\begin{aligned} y_1(x) &= x^{r_1} \sum_{n=0}^{\infty} a_n x^n \\ &= x\left(1 - \frac{1}{8}x^2 + \frac{1}{192}x^4 - \frac{1}{9,216}x^6 + \frac{1}{737,280}x^8 - \frac{1}{88,473,600}x^{10}\right) \end{aligned}$$

The first 6 non-zero terms of the power series component ($\sum_{n=0}^{\infty} b_n x^{n-1}$) from the second independent solution $y_2(x) = Cy_1(x)\ln(x) + \sum_{n=0}^{\infty} b_n x^{n-1}$ is

$$\begin{aligned} y_2(x) &= Cy_1(x)\ln(x) + \sum_{n=0}^{\infty} b_n x^{n-1} \\ &= -\frac{1}{2}y_1(x)\ln(x) + x^{-1}\left(1 + \frac{1}{4}x^2 - \frac{5}{64}x^4 + \frac{5}{1152}x^6 - \frac{47}{442368}x^8 + \frac{131}{88473600}x^{10}\right) \\ &= -\frac{1}{2}y_1(x)\ln(x) + \left(\frac{1}{x} + \frac{1}{4}x - \frac{5}{64}x^3 + \frac{5}{1152}x^5 - \frac{47}{442368}x^7 + \frac{131}{88473600}x^9\right) \end{aligned}$$

where:

- $Cy_1(x)\ln(x)$ is the log part.
- $\sum_{n=0}^{\infty} b_n x^{n-1}$ is the non-log part.

The non-log power series contains alternating fractions spanning the inverse power to higher odd powers.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Frobenius as Class ]# ./main

P(x) y'' + Q(x)y'+ R(x) y = 0

P(x) = (0,0) * x^0 + (0,0) * x^1 + (1,0) * x^2

Q(x) = (0,0) * x^0 + (1,0) * x^1

R(x) = (-1,0) * x^0 + (0,0) * x^1 + (1,0) * x^2

x0 = 0
r1 = (1,0)
r2 = (-1,-0)

Case 1: Different roots; roots differ by an integer.

Series solution:

y1(x) = 1 - 0.125*x^2 + 0.00520833*x^4 - 0.000108507*x^6 + 1.35634e-06*x^8 - 1.13028e-08*x^10
y2(x) = (-0.5,0) * ln(x) y1(x) + x^(-1,-0) ( 1 + 0.25*x^2 - 0.078125*x^4 + 0.00434028*x^6 - 0.000106246*x^8 + 1.48067e-06*x^10 - 1.3375e-08*x^12  )

Computed Power Series Coefficients

y1(x) coefficients (r = (1,0)):
a_{0} = (1,0)
a_{1} = (-0,0)
a_{2} = (-0.125,0)
a_{3} = (-0,0)
a_{4} = (0.00520833,-0)
a_{5} = (-0,0)
a_{6} = (-0.000108507,0)
a_{7} = (-0,0)
a_{8} = (1.35634e-06,-0)
a_{9} = (-0,0)
a_{10} = (-1.13028e-08,0)
a_{11} = (-0,0)
a_{12} = (6.72786e-11,-0)
a_{13} = (-0,0)
a_{14} = (-3.00351e-13,0)
a_{15} = (-0,0)
```

Figure 3.80: *The power series solution of $x^2y'' + xy' + (x^2 - 1)y = 0$ computed with **class SecondOrderODE_Homogeneous_Frobenius_PowerSeriesSolver**, it is categorized as different roots that differ by an integer with $r_1 = 1$ and $r_2 = -1$, and $x_0 = 0$ is a regular singular value (**SymIntegration/Examples/Differential Equations/Test SymIntegration Frobenius as Class/main.cpp**).*

```
Computed Power Series Coefficients

y1(x) coefficients (r = (1,0)):
a_{0} = (1,0)
a_{1} = (-0,0)
a_{2} = (-0.125,0)
a_{3} = (-0,0)
a_{4} = (0.00520833,-0)
a_{5} = (-0,0)
a_{6} = (-0.000108507,0)
a_{7} = (-0,0)
a_{8} = (1.35634e-06,-0)
a_{9} = (-0,0)
a_{10} = (-1.13028e-08,0)
a_{11} = (-0,0)
a_{12} = (6.72786e-11,-0)
a_{13} = (-0,0)
a_{14} = (-3.00351e-13,0)
a_{15} = (-0,0)


y2(x) coefficients (r = (-1,-0)):
Logarithmic constant C = (-0.5,0)
b_{0} = (1,0)
b_{1} = (0,-0)
b_{2} = (0.25,0)
b_{3} = (0,0)
b_{4} = (-0.078125,0)
b_{5} = (0,0)
b_{6} = (0.00434028,-0)
b_{7} = (0,0)
b_{8} = (-0.000106246,0)
b_{9} = (0,0)
b_{10} = (1.48067e-06,-0)
b_{11} = (0,0)
b_{12} = (-1.3375e-08,0)
b_{13} = (0,0)
b_{14} = (8.48191e-11,-0)
b_{15} = (0,0)

Time taken by function: 2541 microseconds
```

Figure 3.81: *The power series solution of $x^2y'' + xy' + (x^2 - 1)y = 0$ computed with* ***class SecondOrderODE_Homogeneous_Frobenius_PowerSeriesSolver****, it is categorized as different roots that differ by an integer with $r_1 = 1$ and $r_2 = -1$, and $x_0 = 0$ is a regular singular value (****SymIntegration/Examples/Differential Equations/Test SymIntegration Frobenius as Class/main.cpp****).*

3. **Repeated Roots for the Indicial Equation ($r_1 = r_2$)**
   Solve and compute the first 6 non-zero terms of the differential equation

$$x^2y'' + xy' + x^2y = 0$$

using the Frobenius method.

Solution:
We first recognize that this is the Bessel equation of order zero.

(a) **Analyze the Singularity**
Divide the equation by x^2 to put it in standard form:

$$y'' + \frac{1}{x}y' + y = 0$$

$P(x) = \frac{1}{x} \to xP(x) = 1$ (analytic at $x = 0$)
$Q(x) = 1 \to x^2Q(x) = x^2$ (analytic at $x = 0$)
Since both limits are finite at $x = 0$, then $x = 0$ is a regular singular point, justifying the use of the Frobenius method.

(b) **Assume the Frobenius Series Solution**
We assume a solution of the form:

$$y(x) = \sum_{n=0}^{\infty} a_n x^{n+r}, \quad a_0 \neq 0$$

Differentiating the series gives:

$$y'(x) = \sum_{n=0}^{\infty} a_n(n+r)x^{n+r-1}$$
$$y''(x) = \sum_{n=0}^{\infty} a_n(n+r)(n+r-1)x^{n+r-2}$$

(c) **Substitute into the Differential Equation**
Substitute y, y', and y'' back into $x^2y'' + xy' + x^2y = 0$.

$$x^2 \sum_{n=0}^{\infty} a_n(n+r)(n+r-1)x^{n+r-2} + x \sum_{n=0}^{\infty} a_n(n+r)x^{n+r-1} + x^2 \sum_{n=0}^{\infty} a_n x^{n+r} = 0$$

Combine the first two sums by factoring out $a_n x^{n+r}$:

$$\sum_{n=0}^{\infty} a_n \left[(n+r)(n+r-1) + (n+r)\right] x^{n+r} + \sum_{n=0}^{\infty} a_n x^{n+r+2} = 0$$

$$\sum_{n=0}^{\infty} a_n(n+r)^2 x^{n+r} + \sum_{n=0}^{\infty} a_n x^{n+r+2} = 0$$

(d) **Find the Indicial Equation**
to align the powers of x, strip out the lowest power (x^r, which corresponds to $n = 0$ in the first sum):

$$a_0 r^2 x^r + a_1(1+r)^2 x^{r+1} + \sum_{n=2}^{\infty} a_n(n+r)^2 x^{n+r} + \sum_{n=0}^{\infty} a_n x^{n+r+2} = 0$$

Equating the coefficient of x^r to zero gives the indicial equation:

$$a_0 r^2 = 0$$

Since $a_0 \neq 0$, we get a repeated root:

$$r_1 = r_2 = 0$$

(e) **Determine the Recurrence Relation**
Equating the coefficient of x^{r+1} ($n = 1$) to zero:

$$a_1(1+r)^2 = 0$$
$$a_1 = 0$$

Shift the index of the second summation by letting $k = n + 2$:

$$\sum_{n=2}^{\infty} [a_n(n+r)^2 + a_{n-2}]x^{n+r} = 0$$

This yields the recurrence relation for $n \geq 2$:

$$a_n = -\frac{a_{n-2}}{(n+r)^2}$$

(f) **Find the First General Solution (y_1)**
Substitute $r = 0$ into the recurrence relation:

$$a_n = -\frac{a_{n-2}}{n^2}$$

Since $a_1 = 0$, all odd coefficients are zero ($a_3 = a_5 = a_7 = \cdots = 0$). For even indices ($n = 2k$)

$$\begin{aligned} n = 2: \quad & a_2 = -\frac{a_0}{2^2} \\ n = 4: \quad & a_4 = -\frac{a_2}{4^2} = \frac{a_0}{2^2 4^2} = \frac{a_0}{2^4 (2!)^2} \\ n = 6: \quad & a_6 = -\frac{a_4}{6^2} = \frac{a_0}{2^2 4^2 6^2} = \frac{a_0}{2^6 (3!)^2} \end{aligned}$$

The general pattern for even coefficients is:

$$a_{2k} = \frac{(-1)^k a_0}{2^{2k}(k!)^2}$$

Setting $a_0 = 1$, the first linearly independent solution is the Bessel function of the first kind of order zero ($J_0(x)$):

$$y_1(x) = \sum_{k=0}^{\infty} \frac{(-1)^k}{(k!)^2} \left(\frac{x}{2}\right)^{2k}$$

(g) **Find the Second Linearly Independent Solution (y_2)**
Because the roots of the indicial equation are equal ($r_1 = r_2$), the second solution must take the form:

$$y_2(x) = y_1(x)\ln(x) + \sum_{n=1}^{\infty} b_n x^n$$

Alternatively, using the Frobenius derivative method $y_2(x) = \frac{\partial y(x,r)}{\partial r}\big|_{r=0}$, the second solution yields:

$$y_2(x) = J_0(x)\ln(x) + \sum_{k=1}^{\infty} \frac{(-1)^{k+1} H_k}{2^{2k}(k!)^2} x^{2k}$$

where $H_k = \sum_{n=1}^{k} \frac{1}{n}$ is the k-th harmonic number.

(h) **Final General Solution**
The complete general solution is a linear combination of both parts:

$$y(x) = C_1 y_1(x) + C_2 y_2(x)$$

Because the roots of the indicial equation are equal ($r_1 = r_2 = 0$), $y_1(x)$ is a regular power series (the Bessel function $J_0(x)$), and $y_2(x)$ contains a logarithmic term.

(i) **The First 6 Non-Zero Terms for the Power Series Solutions**
Setting the initial coefficient $a_0 = 1$, the first solution is an even power series:

$$\begin{aligned} y_1(x) &= x^{r_1} \sum_{n=0}^{\infty} a_n x^n \\ &= \left(1 - \frac{x^2}{4} + \frac{x^4}{64} - \frac{x^6}{2,304} + \frac{x^8}{147,456} - \frac{x^{10}}{14,745,600}\right) \end{aligned}$$

The second independent solution takes the form $y_2(x) = y_1(x)\ln(x) + \sum_{m=1}^{\infty} b_{2m}x^{2m}$, where the coefficients are derived using harmonic numbers. The first 6 terms are:

$$\begin{aligned} y_2(x) &= y_1(x)\ln(x) + x^{r_1}\sum_{m=1}^{\infty} a_{b2m}x^{2m} \\ &= y_1(x)\ln(x) + \left(\frac{x^2}{4} - \frac{3x^4}{128} + \frac{11x^6}{13,824} - \frac{25x^8}{1,769,472} + \frac{137x^{10}}{884,736,000} - \frac{49x^{12}}{42,467,328,000}\right) \end{aligned}$$

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Frobenius as Class ]# ./main

P(x) y'' + Q(x)y'+ R(x) y = 0

P(x) = (0,0) * x^0 + (0,0) * x^1 + (1,0) * x^2

Q(x) = (0,0) * x^0 + (1,0) * x^1

R(x) = (0,0) * x^0 + (0,0) * x^1 + (1,0) * x^2

x0 = 0
r1 = (0,0)
r2 = (-0,-0)

Case 3: Repeated roots.

Series solution:

y1(x) = x^(0,0) ( 1 - 0.25*x^2 + 0.015625*x^4 - 0.000434028*x^6 + 6.78168e-06*x^8 - 6.78168e-08*x^10 )
y2(x) = y1(x) ln(x) + x^(0,0) ( 0.25*x^2 - 0.0234375*x^4 + 0.000795718*x^6 - 1.41285e-05*x^8 + 1.54848e-07*x^10 - 1.15383e-09*x^12 )
```

Figure 3.82: *The power series solution of $x^2y'' + xy' + x^2y = 0$ computed with **class SecondOrderODE_Homogeneous_Frobenius_PowerSeriesSolver**, it is categorized as repeated roots with $r_1 = r_2 = 0$, and $x_0 = 0$ is a regular singular value (**SymIntegration/Examples/Differential Equations/Test SymIntegration Frobenius as Class/main.cpp**).*

```
Computed Power Series Coefficients

y1(x) coefficients (r = (0,0)):
a_{0} = (1,0)
a_{1} = (0,0)
a_{2} = (-0.25,0)
a_{3} = (0,0)
a_{4} = (0.015625,0)
a_{5} = (0,0)
a_{6} = (-0.000434028,0)
a_{7} = (0,0)
a_{8} = (6.78168e-06,0)
a_{9} = (0,0)
a_{10} = (-6.78168e-08,0)
a_{11} = (0,0)
a_{12} = (4.7095e-10,0)
a_{13} = (0,0)
a_{14} = (-2.40281e-12,0)
a_{15} = (0,0)


y2(x) coefficients (r = (-0,-0)):
b_{0} = (0,0)
b_{1} = (0,0)
b_{2} = (0.25,0)
b_{3} = (0,0)
b_{4} = (-0.0234375,0)
b_{5} = (0,0)
b_{6} = (0.000795718,0)
b_{7} = (0,0)
b_{8} = (-1.41285e-05,0)
b_{9} = (0,0)
b_{10} = (1.54848e-07,0)
b_{11} = (0,0)
b_{12} = (-1.15383e-09,0)
b_{13} = (0,0)
b_{14} = (6.23014e-12,0)
b_{15} = (0,0)

Time taken by function: 2236 microseconds
```

Figure 3.83: *The power series solution of $x^2y'' + xy' + x^2y = 0$ computed with **class SecondOrderODE_Homogeneous_Frobenius_PowerSeriesSolver**, it is categorized as repeated roots with $r_1 = r_2 = -1$, and $x_0 = 0$ is a regular singular value (**SymIntegration/Examples/Differential Equations/Test SymIntegration Frobenius as Class/main.cpp**).*

4. **Repeated Roots for the Indicial Equation ($r_1 = r_2$)**
We will see another case for repeated roots, we will find the first 6 terms of the series solutions $y_1(x)$ and $y_2(x)$ using the Frobenius method for the differential equation:

$$x^2y'' + 3xy' + (1+x)y = 0$$

(a) **Indicial Equation and Recurrence Relation** We assume a solution of the form

$$y(x) = \sum_{n=0}^{\infty} a_n x^{n+r}$$

Substituting $y(x), y'(x)$, and $y''(x)$ into the differential equation and gathering the coefficients for the lowest power of x (which is x^r for $n = 0$) gives the indicial equation:

$$\begin{aligned} r(r-1) + 3r + 1 &= 0 \\ r^2 + 2r + 1 &= 0 \\ (r+1)^2 &= 0 \end{aligned}$$

This yields a repeated root:

$$r_1 = r_2 = -1$$

For $n \geq 1$, the coefficients satisfy the recurrence relation:

$$a_n(r) = -\frac{a_{n-1}(r)}{(n+r+1)^2}$$

(b) **First Series Solution $y_1(x)$**
The first solution takes the form:

$$y_1(x) = x^{r_1} \sum_{n=0}^{\infty} a_n x^n$$

Setting $r = -1$ into the recurrence relation gives:

$$\begin{aligned} a_n &= -\frac{a_{n-1}}{n^2} \\ a_n &= \frac{(-1)^n}{(n!)^2} a_0 \end{aligned}$$

Setting $a_0 = 1$, the first 6 terms (from $n = 0$ to $n = 9$) for $y_1(x) = x^{-1} \sum_{n=0}^{\infty} a_n x^n$ are:

$$y_1(x) = x^{-1}\left(1 - x + \frac{1}{4}x^2 - \frac{1}{36}x^3 + \frac{1}{576}x^4 - \frac{1}{14,400}x^5 + \frac{1}{518,400}x^6\right)$$

(c) **Second Series Solution $y_2(x)$**
Because we have a repeated root ($r_1 = r_2$), the second linearly independent solution takes the form:

$$y_2(x) = y_1(x)\ln(x) + x^{r_1} \sum_{n=0}^{\infty} b_n x^n$$

where the coefficients b_n are determined by differentiating $a_n(r)$ with respect to r at $r = -1$:

$$b_n = \frac{\partial a_n(r)}{\partial r}\Big|_{r=-1} = -2H_n a_n(-1) = -2H_n \frac{(-1)^n}{(n!)^2}$$

where $H_n = \sum_{k=1}^{n} \frac{1}{k}$ is the n-th harmonic number.

Calculating the first 6 terms yields

$$y_2(x) = y_1(x)\ln(x) + x^{-1}\left(2x - \frac{3}{4}x^2 + \frac{11}{108}x^3 - \frac{25}{3,456}x^4 + \frac{137}{432,000}x^5 - \frac{49}{5,184,000}x^6\right)$$

The complete general solution is a linear combination of both parts

$$\begin{aligned} y(x) &= C_1 y_1(x) + C_2 y_2(x) \\ &C_1 \sum_{n=0}^{\infty} a_n x^{n+r_1} + C_2 y_1(x)\ln(x) + C_2 \sum_{n=0}^{\infty} b_n x^{n+r_1} \end{aligned}$$

we can determine C_1 and C_2 if we have the initial conditions.

```
P(x) y'' + Q(x)y'+ R(x) y = 0

P(x) = (0,0) * x^0 + (0,0) * x^1 + (1,0) * x^2

Q(x) = (0,0) * x^0 + (3,0) * x^1

R(x) = (1,0) * x^0 + (1,0) * x^1

x0 = 0
r1 = (-1,0)
r2 = (-1,-0)

Case 3: Repeated roots.

Series solution:

y1(x) = x^(-1,0) ( 1 - 1*x + 0.25*x^2 - 0.0277778*x^3 + 0.00173611*x^4 - 6.94444e-05*x^5 + 1.92901e-06*x^6 - 3.93676e-08*x^7 )
y2(x) = y1(x) ln(x) + x^(-1,0) ( 2*x - 0.75*x^2 + 0.101852*x^3 - 0.0072338*x^4 + 0.00031713*x^5 - 9.45216e-06*x^6 + 2.04149e-07*x^7 - 3.34361e-09*x^8 )

Computed Power Series Coefficients

y1(x) coefficients (r = (-1,0)):
a_{0} = (1,0)
a_{1} = (-1,0)
a_{2} = (0.25,0)
a_{3} = (-0.0277778,0)
a_{4} = (0.00173611,0)
a_{5} = (-6.94444e-05,0)
a_{6} = (1.92901e-06,0)
a_{7} = (-3.93676e-08,0)
a_{8} = (6.15119e-10,0)
a_{9} = (-7.59406e-12,0)
a_{10} = (7.59406e-14,0)
a_{11} = (-6.27608e-16,0)
a_{12} = (4.35839e-18,0)
a_{13} = (-2.57893e-20,0)
a_{14} = (1.31578e-22,0)
a_{15} = (-5.84791e-25,0)
```

Figure 3.84: *The power series solution of $x^2y'' + 3xy' + (1+x)y = 0$ computed with **class SecondOrderODE_Homogeneous_Frobenius_PowerSeriesSolver**, it is categorized as repeated roots with $r_1 = r_2 = -1$, and $x_0 = 0$ is a regular singular value (**SymIntegration/Examples/Differential Equations/Test SymIntegration Frobenius as Class/main.cpp**).*

```
y1(x) coefficients (r = (-1,0)):
a_{0} = (1,0)
a_{1} = (-1,0)
a_{2} = (0.25,0)
a_{3} = (-0.0277778,0)
a_{4} = (0.00173611,0)
a_{5} = (-6.94444e-05,0)
a_{6} = (1.92901e-06,0)
a_{7} = (-3.93676e-08,0)
a_{8} = (6.15119e-10,0)
a_{9} = (-7.59406e-12,0)
a_{10} = (7.59406e-14,0)
a_{11} = (-6.27608e-16,0)
a_{12} = (4.35839e-18,0)
a_{13} = (-2.57893e-20,0)
a_{14} = (1.31578e-22,0)
a_{15} = (-5.84791e-25,0)


y2(x) coefficients (r = (-1,-0)):
b_{0} = (0,0)
b_{1} = (2,0)
b_{2} = (-0.75,0)
b_{3} = (0.101852,0)
b_{4} = (-0.0072338,0)
b_{5} = (0.00031713,0)
b_{6} = (-9.45216e-06,0)
b_{7} = (2.04149e-07,0)
b_{8} = (-3.34361e-09,0)
b_{9} = (4.29667e-11,0)
b_{10} = (-4.44855e-13,0)
b_{11} = (3.7906e-15,0)
b_{12} = (-2.705e-17,0)
b_{13} = (1.64027e-19,0)
b_{14} = (-8.55668e-22,0)
b_{15} = (3.88094e-24,0)

Time taken by function: 2364 microseconds
```

Figure 3.85: *The power series solution of* $x^2y'' + 3xy' + (1+x)y = 0$ *computed with* ***class SecondOrderODE_Homogeneous_Frobenius_PowerSeriesSolver****, it is categorized as repeated roots with* $r_1 = r_2 = -1$*, and* $x_0 = 0$ *is a regular singular value (****SymIntegration/Examples/Differential Equations/Test SymIntegration Frobenius as Class/main.cpp****).*

5. **Complex Roots for the Indicial Equation (**$r_1 = \lambda + i\mu,\ r_2 = \lambda - i\mu,\ \mu \neq 0$**)**
   Solve and compute the first 6 non-zero terms of the differential equation

$$x^2y'' + xy' + (x^2+4)y = 0$$

   using the Frobenius method.

   Solution:

   (a) **Verify that** $x = 0$ **is a Regular Singular Point**
   First, we write the differential equation in standard form:

$$y'' + p(x)y' + q(x)y = 0$$

   dividing the original equation by x^2, we will obtain

$$p(x) = \frac{x}{x^2} = \frac{1}{x}$$
$$q(x) = \frac{x^2+4}{x^2} = 1 + \frac{4}{x^2}$$

   Since both $p(x)$ and $q(x)$ are undefined at $x = 0$, then $x = 0$ is a singular point.
   To determine if it is a regular singular point, we check the limits of $xp(x)$ and $x^2q(x)$ as $x\ to2$:

$$p_0 = \lim_{x\to 0} xp(x) = \lim_{x\to 0} x\frac{1}{x} = 1$$
$$q_0 = \lim_{x\to 0} x^2q(x) = \lim_{x\to 0} x^2\left(1 + \frac{4}{x^2}\right) = 4$$

Because both limits are finite, $xP(x)$ and $x^2Q(x)$ are analytic at $x = 0$. Therefore $x = 0$ is an regular singular point and we can use Frobenius method.

(b) **The Frobenius Assumption and and the Indicial Equation**
According to the Frobenius method, we assume a series solution of the form:

$$y(x) = \sum_{n=0}^{\infty} a_n x^{n+r}, \quad a_0 \neq 0$$

Computing the first and second derivatives, we will have

$$y'(x) = \sum_{n=0}^{\infty} a_n (n+r) x^{n+r-1}$$
$$y''(x) = \sum_{n=0}^{\infty} a_n (n+r)(n+r-1) x^{n+r-2}$$

Substitute these derivatives back into the differential equation:

$$x^2 \sum_{n=0}^{\infty} a_n (n+r)(n+r-1) x^{n+r-2} + x \sum_{n=0}^{\infty} a_n (n+r) x^{n+r-1} + (x^2+4) \sum_{n=0}^{\infty} a_n x^{n+r} = 0$$
$$\sum_{n=0}^{\infty} a_n (n+r)(n+r-1) x^{n+r} + \sum_{n=0}^{\infty} a_n (n+r) x^{n+r} + \sum_{n=0}^{\infty} a_n x^{n+r+2} + 4 \sum_{n=0}^{\infty} a_n x^{n+r} = 0$$

Combine the terms containing x^{n+r}:

$$\sum_{n=0}^{\infty} a_n [(n+r)(n+r-1) + (n+r) + 4] x^{n+r} + \sum_{n=0}^{\infty} a_n x^{n+r+2} = 0$$
$$\sum_{n=0}^{\infty} a_n [(n+r)^2 + 4] x^{n+r} + \sum_{n=2}^{\infty} a_{n-2} x^{n+r} = 0$$

To find the indicial equation, we set the coefficient of the lowest power of x ($n = 0$) to zero:

$$a_0 (r^2 + 4) = 0$$

Since $a_0 \neq 0$, we get the indicial roots:

$$r^2 + 4 = 0$$
$$r = \pm 2i$$

Thus, we have complex conjugate roots:

$$r_1 = 2i$$
$$r_2 = -2i$$

(c) **The Recurrence Relation**
For $n = 1$ we will have

$$[(1+r)^2 + 4] a_1 = 0$$
$$a_1 = 0$$

and for $n \geq 2$

$$a_n = -\frac{a_{n-2}}{(n+r)^2+4}$$

Expanding the denominator and substituting $r^2 = -4$:

$$(n+r)^2+4 = n^2+2nr+r^2+4 = n(n+2r)$$

Thus, the simplified recurrence relation is:

$$a_n = -\frac{a_{n-2}}{n(n+2r)}$$

Since $a_1 = 0$, all odd coefficients ($a_3 = a_5 = a_7 = \cdots = 0$) vanish. We only need to calculate the even coefficients. Let's substitute $r = 2i$ into the recurrence relation:

$$a_n = -\frac{a_{n-2}}{n^2+4(i)(n)-4+4} = -\frac{a_{n-2}}{n(n+4i)}$$

(d) **Find the First General Solution (y_1)**
The first independent solution is

$$y_1(x) = x^{r_1}\sum_{n=0}^{\infty} a_n x^n$$

with a_n as the coefficients that can be computed with the recurrence relation by using r_1.

(e) **Find the Second Linearly Independent Solution (y_2)**
The second independent solution is

$$y_2(x) = x^{r_2}\sum_{n=0}^{\infty} b_n x^n$$

with b_n as the coefficients that can be computed with the recurrence relation by using r_2.

(f) **The First 6 Non-Zero Terms for the Power Series Solutions**
Power Series Solution for $y_1(x)$
Setting $r = 2i$ and choosing $a_0 = 1$, the recurrence relation simplifies to

$$a_n = -\frac{a_{n-2}}{n(n+4i)}$$

we compute the first 6 non-zero coefficients (up to x^{10}):

$$\begin{aligned}
a_0 &= 1 \\
a_2 &= -\frac{1}{2(2+4i)} = -\frac{1}{4+8i} = -\frac{4-8i}{80} = -\frac{1}{20}+\frac{1}{10}i \\
a_4 &= -\frac{a_2}{4(4+4i)} = -\frac{\frac{1}{20}+\frac{1}{10}i}{16+16i} = -\frac{1}{640}-\frac{3}{640}i \\
a_6 &= -\frac{a_4}{6(6+4i)} = \frac{3}{33280}+\frac{7}{99840}i \\
a_8 &= -\frac{a_6}{8(8+4i)} = -\frac{1}{638976}-\frac{1}{3194880}i \\
a_{10} &= -\frac{a_8}{10(10+4i)} = \frac{9}{617676800}-\frac{1}{370606080}i
\end{aligned}$$

First 6 terms of $y_1(x)$ is

$$y_1(x) = x^{2i}\Big[1 + \left(-\frac{1}{20} + \frac{1}{10}i\right)x^2 + \left(-\frac{1}{640} - \frac{3}{640}i\right)x^4 + \left(\frac{3}{33280} + \frac{7}{99840}i\right)x^6 + \left(-\frac{1}{638976} - \frac{1}{3194880}i\right)x^8 + \left(\frac{9}{617676800} - \frac{1}{370606080}i\right)x^{10}\Big]$$

Power Series Solution for $y_2(x)$
Setting $r = -2i$ and choosing $b_0 = 1$, because the roots are complex conjugates the coefficients for $y_2(x)$ are simply the complex conjugates of the coefficients found for $y_1(x)$

$$\begin{aligned} b_0 &= 1 \\ b_2 &= -\frac{1}{20} - \frac{1}{10}i \\ b_4 &= -\frac{1}{640} + \frac{3}{640}i \\ b_6 &= \frac{3}{33280} - \frac{7}{99840}i \\ b_8 &= -\frac{1}{638976} + \frac{1}{3194880}i \\ b_{10} &= \frac{9}{617676800} + \frac{1}{370606080}i \end{aligned}$$

First 6 terms of $y_2(x)$ is

$$y_2(x) = x^{-2i}\Big[1 + \left(-\frac{1}{20} - \frac{1}{10}i\right)x^2 + \left(-\frac{1}{640} + \frac{3}{640}i\right)x^4 + \left(\frac{3}{33280} - \frac{7}{99840}i\right)x^6 + \left(-\frac{1}{638976} + \frac{1}{3194880}i\right)x^8 + \left(\frac{9}{617676800} + \frac{1}{370606080}i\right)x^{10}\Big]$$

Since $x^{\pm 2i} = e^{\pm 2i\ln(x)}$, real-valued basis solutions can be constructed by taking the linear combinations

$$u_1(x) = \frac{y_1 + y_2}{2}, \quad u_2(x) = \frac{y_1 - y_2}{2i}$$

using Euler's identity ($e^{ix} = \cos(x) + i\sin(x)$) to split the complex series into two linearly independent real solutions if a real solution space is desired.

(g) **Final General Solution**
We can separate the series $\sum_{n=0}^{\infty} a_n x^n$ into its real part $u_n(x)$ and imaginary part $v_n(x)$:

$$\sum_{n=0}^{\infty} u_n(x)x^n = 1 - \frac{1}{20}x^2 - \frac{1}{640}x^4 + \frac{3}{33280}x^6 - \frac{1}{638976}x^8 + \frac{9}{617676800}x^{10} - \dots$$

$$\sum_{n=0}^{\infty} v_n(x)x^n = \frac{1}{10}x^2 - \frac{3}{640}x^4 + \frac{7}{99840}x^6 - \frac{1}{3194880}x^8 - \frac{1}{370606080}x^{10} - \dots$$

Using Euler's formula to rewrite the front factor $x^{2i} = e^{2i\ln(x)} = \cos(2\ln(x)) + i\sin(2\ln(x))$, the full complex solution is

$$y(x) = \cos(2\ln(x)) + i\sin(2\ln(x))\Big[\sum_{n=0}^{\infty} u(x) + iv(x)\Big]$$

Thus if we want to convert the two complex solutions into two real, linearly independent solutions $y_1(x)$ and $y_2(x)$ we will use this formula:

$$y_1(x) = x^\lambda \left[\cos(\mu \ln(x)) \sum_{n=0}^{\infty} u_n x^n - \sin(\mu \ln(x)) \sum_{n=0}^{\infty} v_n x^n \right]$$
$$y_2(x) = x^\lambda \left[\sin(\mu \ln(x)) \sum_{n=0}^{\infty} u_n x^n + \cos(\mu \ln(x)) \sum_{n=0}^{\infty} v_n x^n \right]$$

for complex roots $r_{1,2} = \lambda \pm \mu$. Here, u_n and v_n represent the real and imaginary parts of the complex coefficients a_n, respectively.

Expanding this product gives two final real-valued solutions $y_1(x)$ and $y_2(x)$

$$y_1(x) = \cos(2\ln(x)) \sum_{n=0}^{\infty} u_n x^n - \sin(2\ln(x)) \sum_{n=0}^{\infty} v_n x^n$$
$$y_2(x) = \sin(2\ln(x)) \sum_{n=0}^{\infty} u_n x^n + \cos(2\ln(x)) \sum_{n=0}^{\infty} v_n x^n$$

The general real solution to the differential equation near the regular singular point is

$$y(x) = C_1 y_1(x) + C_2 y_2(x)$$

where C_1 and C_2 are arbitrary real constants that can be determined with initial conditions.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Frobenius as Class ]# ./main

P(x) y'' + Q(x)y'+ R(x) y = 0

P(x) = (0,0) * x^0 + (0,0) * x^1 + (1,0) * x^2

Q(x) = (0,0) * x^0 + (1,0) * x^1

R(x) = (4,0) * x^0 + (0,0) * x^1 + (1,0) * x^2

x0 = 0
r1 = (0,2)
r2 = (-0,-2)

Case 4: Complex roots.

Series solution:

y1(x) = x^(0,2) ( (1,0) + (-0.05,0.1)*x^2 + (-0.0015625,-0.0046875)*x^4 + (9.01442e-05,7.01122e-05)*x^6 + (-1.565e-06,-3.13001e-07)*x^8 + (1.45707e-08,-2.69828e-09)*x^10 )
y2(x) = x^(-0,-2) ( (1,0) + (-0.05,-0.1)*x^2 + (-0.0015625,0.0046875)*x^4 + (9.01442e-05,-7.01122e-05)*x^6 + (-1.565e-06,3.13001e-07)*x^8 + (1.45707e-08,2.69828e-09)*x^10  )
```

Figure 3.86: *The power series solution of $x^2y'' + xy' + (x^2 + 4)y = 0$ computed with **class SecondOrderODE_Homogeneous_Frobenius_PowerSeriesSolver**, it is categorized as complex roots with $r_1 = 2i$, $r_2 = -2i$, and $x_0 = 0$ is a regular singular value (**SymIntegration/Examples/Differential Equations/Test SymIntegration Frobenius as Class/main.cpp**).*

```
Computed Power Series Coefficients

y1(x) coefficients (r = (0,2)):
a_{0} = (1,0)
a_{1} = (0,0)
a_{2} = (-0.05,0.1)
a_{3} = (0,0)
a_{4} = (-0.0015625,-0.0046875)
a_{5} = (0,0)
a_{6} = (9.01442e-05,7.01122e-05)
a_{7} = (0,0)
a_{8} = (-1.565e-06,-3.13001e-07)
a_{9} = (0,0)
a_{10} = (1.45707e-08,-2.69828e-09)
a_{11} = (0,0)
a_{12} = (-8.54456e-11,4.72199e-11)
a_{13} = (0,0)
a_{14} = (3.39407e-13,-3.37891e-13)
a_{15} = (0,0)


y2(x) coefficients (r = (-0,-2)):
b_{0} = (1,0)
b_{1} = (-0,0)
b_{2} = (-0.05,-0.1)
b_{3} = (-0,0)
b_{4} = (-0.0015625,0.0046875)
b_{5} = (0,0)
b_{6} = (9.01442e-05,-7.01122e-05)
b_{7} = (0,0)
b_{8} = (-1.565e-06,3.13001e-07)
b_{9} = (0,0)
b_{10} = (1.45707e-08,2.69828e-09)
b_{11} = (0,0)
b_{12} = (-8.54456e-11,-4.72199e-11)
b_{13} = (0,0)
b_{14} = (3.39407e-13,3.37891e-13)
b_{15} = (0,0)

Time taken by function: 2541 microseconds
```

Figure 3.87: *The power series solution of $x^2y'' + xy' + (x^2 + 4)y = 0$ computed with* ***class SecondOrderODE_Homogeneous_Frobenius_PowerSeriesSolver****, it is categorized as complex roots with $r_1 = 2i$, $r_2 = -2i$, and $x_0 = 0$ is a regular singular value (****SymIntegration/Examples/Differential Equations/Test SymIntegration Frobenius as Class/main.cpp****).*

6. **Complex Roots for the Indicial Equation (Euler-Cauchy Equation)**
Solve and compute the first 6 non-zero terms of the differential equation

$$(x-2)^2y'' + 5(x-2)y' + 8y = 0$$

using the Euler-Cauchy equation.

Solution:

(a) **Verify that $x = 2$ is a Regular Singular Point**
First, we write the differential equation in standard form:

$$y'' + p(x)y' + q(x)y = 0$$

dividing the original equation by x^2, we will obtain

$$p(x) = \frac{5(x-2)}{(x-2)^2} = \frac{5}{x-2}$$
$$q(x) = \frac{8}{(x-2)^2}$$

Since both $p(x)$ and $q(x)$ are undefined at $x = 2$, then $x = 2$ is a singular point.
To determine if it is a regular singular point, we check the limits of $xp(x)$ and $x^2q(x)$ as

x *to*2:

$$p_0 = \lim_{x\to 2} xp(x) = \lim_{x\to 2} x\frac{5}{x-2} = \infty$$
$$q_0 = \lim_{x\to 2} x^2 q(x) = \lim_{x\to 2} x^2\frac{8}{(x-2)^2} = \infty$$

Because both limits are infinite, $xP(x)$ and $x^2Q(x)$ are not analytic at $x = 2$. Therefore $x = 2$ is either a irregular singular point or an Euler-Cauchy equation.

Knowing this, we first simplify it by making a substitution.

Let $t = x - 2$. The equation transforms into a classic Euler-Cauchy equation:

$$t^2y'' + 5ty' + 8y = 0$$

Now we have known that this is a pure Euler-Cauchy equation, the Frobenius series terminates immediately after the first term ($a_n = 0$ for all $n \geq 1$). A standard real-valued Frobenius power series cannot be constructed for this differential equation without introducing complex exponents. Instead of a standard infinite series of changing coefficients, this is a perfect Euler-Cauchy equation where the series terminates or collapses completely into a closed-form combination of logarithms, sines, and consines.

(b) **The Frobenius Assumption and and the Indicial Equation**
According to the Frobenius method, we assume a series solution of the form:

$$y(t) = \sum_{n=0}^{\infty} a_n t^{n+r}, \quad a_0 \neq 0$$

Computing the first and second derivatives, we will have

$$y'(t) = \sum_{n=0}^{\infty} a_n(n+r)t^{n+r-1}$$
$$y''(t) = \sum_{n=0}^{\infty} a_n(n+r)(n+r-1)t^{n+r-2}$$

Substitute these derivatives back into the differential equation:

$$t^2\sum_{n=0}^{\infty} a_n(n+r)(n+r-1)t^{n+r-2} + 5t\sum_{n=0}^{\infty} a_n(n+r)t^{n+r-1} + 8\sum_{n=0}^{\infty} a_n t^{n+r} = 0$$
$$\sum_{n=0}^{\infty} a_n(n+r)(n+r-1)t^{n+r} + 5\sum_{n=0}^{\infty} a_n(n+r)t^{n+r} + 8\sum_{n=0}^{\infty} a_n t^{n+r} = 0$$

Combining the sums:

$$\sum_{n=0}^{\infty} a_n[(n+r)(n+r-1) + 5(n+r) + 8]t^{n+r} = 0$$

To find the indicial equation, we set $n = 0$ and assume $a_0 \neq 0$:

$$r(r-1) + 5r + 8 = 0$$
$$r^2 + 4r + 8 = 0$$

Solving for r using the quadratic formula:

$$r = \frac{-4 \pm \sqrt{16 - 32}}{2} = -2 \pm 2i$$

Thus, we have complex conjugate roots:

$$r_1 = -2 + 2i$$
$$r_2 = -2 - 2i$$

(c) **Find the First General Solution (y_1), the Second General Solution ($y_2(x)$) and the General Solution**
For $n \geq 1$, the coefficient equation must satisfy:

$$a_n[(n+r)^2 + 4(n+r) + 8] = 0$$

Expanding the bracket using $r^2 + 4r + 8 = 0$:

$$a_n \cdot n(n + 2r + 4) = 0$$

Since $r = -2 \pm 2i$, the term $(n + 2r + 4) = n \pm 4i$. Because $n \geq 1$, this factor can never be zero. Therefore, we must have

$$a_n = 0$$

for all $n \geq 1$.

This is the proof that this equation is the Euler-Cauchy equation (equidimensional), we don't need an infinite series.

The first linearly independent solution for roots $r = \lambda \pm i\mu$ is

$$y_1(x) = (x - x_0)^\lambda \cos(\mu \ln|x - x_0|)$$

The second independent solution is

$$y_1(x) = (x - x_0)^\lambda \sin(\mu \ln|x - x_0|)$$

The exact general solution for the complex roots $r = 2 \pm 2i$ is:

$$\begin{aligned} y(x) &= (x - x_0)^\lambda \left[C_1 \cos(\mu \ln|x - x_0|) + C_2 \sin(\mu \ln|x - x_0|)\right] \\ &= (x-2)^{-2} \left[C_1 \cos(2 \ln|x-2|) + C_2 \sin(2 \ln|x-2|)\right] \end{aligned}$$

(d) **The First 6 Non-Zero Terms for the Power Series Solutions**
To compute the first 6 terms we can expand using the Maclaurin series, even if the solution in this case is an exact one in terms of sine and cosine, we can still get the solutions in series term if we wanted to.

By expanding the $\cos(u)$ and $\sin(u)$ components using standard Maclaurin series where $u = 2\ln(x-2)$ we will have

$$y_1(x) = (x-2)^{-2}\left[1 - \frac{(2\ln(x-2))^2}{2!} + \frac{(2\ln(x-2))^4}{4!} - \frac{(2\ln(x-2))^6}{6!} + \frac{(2\ln(x-2))^8}{8!} - \frac{(2\ln(x-2))^{10}}{10!} +\right.$$

$$y_2(x) = (x-2)^{-2}\left[2\ln(x-2) - \frac{(2\ln(x-2))^3}{3!} + \frac{(2\ln(x-2))^5}{5!} - \frac{(2\ln(x-2))^7}{7!} + \frac{(2\ln(x-2))^9}{9!} - \frac{(2\ln(x\right.$$

```
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Differential Equations/Test SymIntegration Frobenius as Class ]# ./main
Analyzing ODE at point x0 = (2,0)
--> Result: Euler-Cauchy Equation.
    Solve exactly using the auxiliary algebraic characteristic equation.

P(x) y'' + Q(x)y'+ R(x) y = 0

P(x) = (0,0) * x^0 + (0,0) * x^1 + (1,0) * x^2

Q(x) = (0,0) * x^0 + (5,0) * x^1

R(x) = (8,0) * x^0

x0 = (2,0)
r1 = (-2,2)
r2 = (-2,-2)

Case 4: Complex roots.

General solution:

y(x) = [ C1 *  (x - (2,0))^{-2} cos ( 2 ln(x - (2,0)) ) ] + [ C2 *  (x - (2,0))^{-2} sin ( 2 ln(x - (2,0)) ) ]
Time taken by function: 1034 microseconds
```

Figure 3.88: *The power series solution of* $(x-2)^2y'' + 5(x-2)y' + 8y = 0$ *computed with* ***class SecondOrderODE_Homogeneous_Frobenius_PowerSeriesSolver****, it is categorized as complex roots with* $r_1 = 2+2i$, $r_2 = 2-2i$, *and* $x_0 = 2$ *is a regular singular value, it is categorized as Euler-Cauchy differential equation, thus we solved it by using the exact general solution of the Euler-Cauchy method (****SymIntegration/Examples/Differential Equations/Test SymIntegration Frobenius as Class/-main.cpp****).*

Implementing the Frobenius method for complex indicial roots in C++ requires a **Polynomial** class and **complex<double>** data type. Basically to implement this universally, we need to use a robust object-oriented setup. Thus, in SymIntegration we create another custom **PolynomialComplex** class to store polynomial in **complex<double>** format. The **complex<double>** encapsulation for **PolynomialComplex** make us able to evaluate complex fields seamlessly.

We apply **PolynomialComplex** in:

class SecondOrderODE_Homogeneous_Frobenius_PowerSeriesSolver

thus it is able to find the solution of the differential equation with Frobenius method for all kinds of roots. It solves the indicial equation (handles real, imaginary, or complex distinct roots natively), sets up the recurrence relations (the dynamic recurrence solver evaluates the Frobenius summation sequentially), computes the roots in **complex<double>** format, the series solution will also be returned as **vector<complex<double>>** format.

Mathematical context behind the code:

- **Singular Point Alignment:** The code dynamically extracts the minimum index offset (L) of the polynomial functions to cleanly deduce the indicial equation $a_0r(r-1) + b_0r + c_0 = 0$ irrespective of leading zero-power gaps with **find_L** function.
- **Shifted Polynomial / Transforming to** t**:** The substitution $t = x - x_0$ shifts the singular point to the origin ($t = 0$). The input $P(x)$, $Q(x)$, and $R(x)$ will be shifted at the beginning of the code. If the differential equation has a regular singular point at $x_0 = 2$, the code maps all operators cleanly into powers of $t = x - 2$ automatically via binomial accumulation. By applying a change of variable $t = x - x_0$, we shift the regular singular point to $t = 0$. Thus from the original equation $P(x)y'' + Q(x)y' + R(x)y = 0$, then the equation is rewritten as

$$P(t+x_0)\frac{d^2y}{dt^2} + Q(t+x_0)\frac{dy}{dt} + R(t+x_0)y = 0$$

to perform this programmatically without floating-point division limits, we shift our coordinate system by setting $t = x - x_0$. This lets us re-write the polynomials in terms of t (a shifted polynomial class centered at 0).
Let $P(t + x_0), Q(t + x_0)$, and $R(t + x_0)$ be represented as polynomials in t:

$$P(t + x_0) = a_m t^m + a_{m+1} t^{m+1} + \dots$$
$$Q(t + x_0) = b_n t^n + b_{n+1} t^{n+1} + \dots$$
$$R(t + x_0) = c_k t^k + c_{k+1} t^{k+1} + \dots$$

For x_0 to be a regular singular point, the lowest non-zero powers of t must satisfy: $val(Q) \geq val(P) - 1$ and $val(R) \geq val(P) - 2$, with $val(...)$ means it is the lowest non-zero power of that polynomial.

- **valuation() Method:** This method for **PolynomialDouble** class finds the lowest power of x for polynomial $P(x), Q(x)$, and $R(x)$ that has a non-zero coefficient. This acts as factoring out x^k to evaluate limits at $x \to x_0$. This method is very useful to compute the indicial equation that will then be solved with quadratic solver using the discriminant method, cleanly handling real, repeating, or complex roots.
- **Indicial Alignment:** The program assumes regular singularity conditions at $x = x_0$, it will identify the lowest non-zero powers of each $P(x - x_0), Q(x - x_0)$, and $R(x - x_0)$, check the regular singularity condition, extract dominant structural coefficients then solve the quadratic indicial equation $r^2 + (p_0 - 1)r + q_0$.
- **Coupling Matrix Elements:** For the case of distinct roots, when $r_1 - r_2 = N$, plugging $y_2(x)$ back into the ODE yields a strict system constraint when evaluating the index $n = N$. The denominator for b_N evaluates to zero.
- **Logarithmic Constant** C**:** For the case of distinct roots, differ by integer, the code solves for C directly at step $n = N$ to cancel out the vanishing singularity driven by the Frobenius mismatch, stabilizing the numerical pipeline for complex domains.
- **Complex Scalar Vectoring:** All algebra is native to **complex<double>**. It works out-of-the-box for equations with complex geometries (e.g., solutions displaying oscillating behaviors near singularity boundaries). Roots or coefficients naturally possess imaginary parts (e.g., in advanced quantum mechanics wave-distribution ODEs).
- **Singularity Defusing:** By mapping $a_0(r) = r - r_2$ (for distinct roots that differ by integer), the algorithm forces the pole factor out of the denominator. Computing the system at $r_2 \pm h$ effectively evaluates the analytical limit without hitting an exact 0.0 division panic.
- **Automatic Differentiation (Dual Numbers):** Because calculating exact symbolic derivatives is complex in standard C++ (it is possible with SymIntegration, but costs more), thus we use automatic differentiation (dual numbers). A dual number is represented as $u + v\epsilon$, where $\epsilon^2 = 0,\ a, b \in \mathbb{C}$. This allows us to track both the value $a_n(r)$ and its derivative $a'_n(r) = b_n$ simultaneously (for distinct roots that differ by integer case).

  Dual number struct that is written in **dsolve.h** elegantly handle the tricky cases where roots differ by an integer (including repeated roots and complex roots), it uses a dual number to track and compute parameter derivatives ($\frac{\partial y}{\partial r}$) automatically through the recurrence relations.

  This struct overloads operators $(+, -, *, /)$ using complex numbers, it tracks the derivative chain rule automatically during recurrence generation when tracking variable parameter r.

  We can initialize a dual with:

Dual(r2,1.0) (with r_2 as the second root)

Any downstream algebraic configuration computes the exact symbolic derivative step-by-step alongside the value. The code handles singularities gracefully for repeated roots and distinct roots that differ by an integer, by shifting $a_0 \to (r - r_2)a_0$, calculating the coefficient's exact limiting path value using the dual number's derivative register ($a[n].der$).

When the indicial equations yield repeated roots ($r_1 = r_2$), the second linearly independent solution involves a derivative with respect to the parameter r:

$$y_2(x) = y_1(x)\ln(x) + x^{r_1}\sum_{n=0}^{\infty}\frac{\partial a_n(r)}{\partial r}\Big|_{r=r_1}x^n$$

In some type of roots to get the recurrence formula $a_n(r)$ we must use $P(x), Q(x), R(x)$, but to get the second solution from $a_n(r)$ we do not use $P(x), Q(x), R(x)$ or use reduction-of-order integrals; evaluating the recurrence via dual automatic differentiation at the root will instantly yield both solutions.

After we find the first series ($y_1(x)$), the second step is where dual numbers and automatic differentiation shine. Instead of evaluating the series at the real root, we can evaluate the generalized series at the dual root $r = r_1 + \epsilon$

$$y(x, r_1 + \epsilon) = y(x, r_1) + \epsilon\frac{\partial y(x, r)}{\partial r}\Big|_{r=r_1}$$

Because of how dual numbers naturally propagate derivatives, the **Dual** struct carries both $a_n(r)$ (in **.val**) and its derivative $\frac{\partial a_n}{\partial r}$ (in **.der**). The real part of the resulting expression will automatically be the first solution $y_1(x)$, and the dual part (ϵ) will automatically be the second solution $y_2(x)$.

For the case of distinct roots that differ by an integer, the second solution $y_2(x)$ contains a log term scaled by a constant C, alongside a standard series

$$y_2(x) = Cy_1(x)\ln(x) + x^{r_2}\sum_{n} = 0^{\infty}b_n x^n$$

Instead of manually deriving equations to find C and b_n, we can scale the initial coefficient function by setting $a_0(r) = r - r_2$, the code maps the polynomial values through standard linear operators while tracing their explicit derivatives. Evaluating the full solution and its derivative with respect to r automatically tracks the values and prevents division-by-zero singularities at the N-th step (automatically resolve the pole). The **Dual** struct that is carrying a value component (**val**) and a derivative component (**der**) with respect to r perfectly resolves this problem. To obtain C value, $a_0(r_2)$ naturally resolves the $\frac{0}{0}$ indeterminacy, providing the exact scale value for C. The derivative payload (**a[n].der**) matches the required calculus expansion parameter $\frac{\partial a_n(r)}{\partial r}\big|_{r=r_2}$ automatically.

There are some options, we have created **Dual, Dual1,** and **Dual2** structs that can be used for different case.

Dual Struct code maps to the formula:

- **Dual Struct**: Instead of doing messy calculus manually, this structure tracks both the coefficient value (**.val**) and its derivative with respect to r (**.der**).
- **r(r_1, 1.0)**: The root is initialized with a derivative step of 1.0. This triggers chain-rule calculations automatically during arithmetic operations.
- $a[n]$**.val**: It yields the standard coefficients for the first solution, $y_1(x)$
- $a[n]$**.der**: It yields the b_n coefficients needed for the second logarithmic solution, $y_2(x)$ for distinct roots that differ by an integer.

Implementation Segment	Frobenius Equivalent	Purpose
Dual2($r_2, 1.0, 1.0$**)**	$r = r_2 + \epsilon$	Expands the tracking scope across the singularity boundary.
$a[0]$ = **Dual2** $(0.0, 1.0, 0.0)$	$a_0(r) = r - r_2$	Eliminates the $\frac{0}{0}$ structural failure at index offset $m = N$.
Output $a[n].a$	$a_n(r_2)$	Extracted coefficient scaling the $\ln(x)y_1(x)$ wrapper component.
Output $a[n].b$	$a_n'(r_2)$	Main derivative mapping representing the standalone x^n polynomial offset array.

Table 3.6: *Mathematical Mapping Overview for* ***Dual2*** *struct.*

- **General Recurrence Engine:** Instead of tedious manual derivations, it parses coefficients directly out of the generated arbitrary degree standard polynomials inside a nested evaluation loop to process the Frobenius summation equation dynamically. It applies the standard recurrence relation to compute the next series term based on all previous terms (a_0 to a_{n-1}).
- **Shift Mechanism Logic:** When computing for the power series solution for each different case, the nested **for** loop tracking $shift = n - k$ aligns indices correctly. This allows lower-order terms like a_k to build the next coefficient step when multiplying polynomial series expansions.

Root categories in SymIntegration for Frobenius solver:

1. **Case 1:** Distinct roots differ by integer.
2. **Case 2:** Distinct roots differ by non-integer.
3. **Case 3:** Repeated roots.
4. **Case 4:** Complex roots.

Complex roots generate complex coefficients, meaning that the solution terms will look like $x^r \sum_{n=0}^{\infty} a_n (x - x_0)^n$, where both a_n and r are complex numbers.

v. Maclaurin Series to Handle Non-Elementary Integral

[SI*] A Maclaurin series is an infinite polynomial expansion of a function centered specifically at zero, functioning as a special case of the Taylor series. It allows us to approximate complex functions like exponentials and trigonometric ratios using basic powers of x. The general formula for Maclaurin series for function $f(x)$ is defined as

$$\sum_{n=0}^{\infty} \frac{f^{(n)}(0)}{n!}x^n = f(0) + f'(0)x + \frac{f''(0)}{2!} + \frac{f'''(0)}{3!}x^3 + \dots$$

In the **Series Solutions Near an Ordinary Point** section we encounter Airy's differential equation which has series solutions, for a second order differential equation

$$\frac{d^2y}{dx^2} - xy = 0$$

the general solution can be expressed as a linear combination of two independent Airy functions, $Ai(x)$ and $Bi(x)$:

$$y(x) = c_1 Ai(x) + c_2 Bi(x)$$

with

$$Ai(x) = \frac{1}{\pi}\int_0^{\infty} \cos\left(\frac{t^3}{3} + xt\right) dt$$

and

$$Bi(x) = \frac{1}{\pi}\int_0^{\infty} \left[\sin\left(\frac{t^3}{3} + xt\right) + e^{-\frac{t^3}{3}+xt}\right] dt$$

The integral $\int \sin\left(\frac{t^3}{3} + xt\right) dt$ and $\int \cos\left(\frac{t^3}{3} + xt\right) dt$ cannot be evaluated in terms of elementary functions, thus we need to expand it by using Maclaurin series.

1. **Definite Integral Form (Airy Function)**
   The boundary conditions are from 0 to ∞, the integral is directly related to the Airy function of the first kind:

   $$Ai(x) = \frac{1}{\pi}\int_0^{\infty} \cos\left(\frac{t^3}{3} + xt\right) dt$$

   By multiplying both sides by π, we get:

   $$\pi Ai(x) = \int_0^{\infty} \cos\left(\frac{t^3}{3} + xt\right) dt$$

2. **Indefinite Integral via Series Expansion**
   To approximate or express the indefinite integral, you can use a Taylor series expansion centered at $t = 0$ (the Maclaurin series).
   Expand the Cosine Term
   Recall the standard Maclaurin series for $\cos(\theta)$:

   $$\cos(\theta) = \sum_{n=0}^{\infty} \frac{(-1)^n \theta^{2n}}{(2n)!} = 1 - \frac{\theta^2}{2!} + \frac{\theta^4}{4!} - \dots$$

Substitute the Argument
Substitute $\theta = \left(\frac{t^3}{3} + xt\right)$:

$$\cos\left(\frac{t^3}{3} + xt\right) = 1 - \frac{1}{2}\left(\frac{t^3}{3} + xt\right)^2 + \frac{1}{24}\left(\frac{t^3}{3} + xt\right)^4 - \dots$$

Expanding the first few terms:

$$\cos\left(\frac{t^3}{3} + xt\right) = 1 - \frac{1}{2}\left(\frac{t^6}{9} + \frac{2xt^4}{3} + x^2t^2\right) + \dots$$
$$\cos\left(\frac{t^3}{3} + xt\right) = 1 - \frac{x^2}{2}t^2 - \frac{x}{3}t^4 - \frac{1}{18}t^6 + \dots$$

Integrate Term-by-Term
Integrating each power of t with respect to t:

$$\begin{aligned}\int \cos\left(\frac{t^3}{3} + xt\right) dt &= \int \left(1 - \frac{x^2}{2}t^2 - \frac{x}{3}t^4 - \frac{1}{18}t^6 + \dots\right) dt \\ &= t - \frac{x^2}{6}t^3 - \frac{x}{15}t^5 - \frac{1}{126}t^7 + \dots + C\end{aligned}$$

For the $\int \sin\left(\frac{t^3}{3} + xt\right) dt$:

1. **Expand the Sine Term**
   Recall the Maclaurin series for the sine term:

   $$\sin(\theta) = \sum_{n=0}^{\infty} \frac{(-1)^n \theta^{2n+1}}{(2n+1)!}$$

   Subtitute $\theta = \frac{t^3}{3} + xt$

   $$\sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)!}\left(\frac{t^3}{3} + xt\right)^{2n+1}$$

2. **Decompose with Binomial Theorem**

   $$\left(\frac{t^3}{3} + xt\right)^{2n+1} = \sum_{k=0}^{2n+1} \binom{2n+1}{k} \left(\frac{t^3}{3}\right)^{2n+1-k} (xt)^k$$
   $$\left(\frac{t^3}{3} + xt\right)^{2n+1} = \sum_{k=0}^{2n+1} \binom{2n+1}{k} \frac{x^k}{3^{2n+1-k}} t^{6n+3-2k}$$

3. **Integrate**
   integrate it with respect to variable t:

   $$\begin{aligned}\int \sin\left(\frac{t^3}{3} + xt\right) dt &= \sum_{n=0}^{\infty} \sum_{k=0}^{2n+1} \frac{(-1)^n}{(2n+1)!} \binom{2n+1}{k} \frac{x^k}{3^{2n+1-k}} \int t^{6n+3-2k}\, dt \\ &= \sum_{n=0}^{\infty} \sum_{k=0}^{2n+1} \frac{(-1)^n \binom{2n+1}{k} x^k t^{6n+4-2k}}{(2n+1)! \cdot 3^{2n+1-k}(6n+4-2k)} + C\end{aligned}$$

Thus the linearly independent series components:

$$\begin{aligned} y_1(x) &= 1 + \sum_{n=1}^{\infty} \frac{x^{3n}}{2 \cdot 3 \cdot 5 \cdot 6 \ldots (3n-1)(3n)}, \quad \text{(Associated with } Ai(x)\text{)} \\ &= 1 + \sum_{n=1}^{\infty} \frac{x^{3n}}{3^n n![2 \cdot 5 \ldots (3n-1)]} \end{aligned}$$

$$\begin{aligned} y_2(x) &= x + \sum_{n=1}^{\infty} \frac{x^{3n+1}}{3 \cdot 4 \cdot 6 \cdot 7 \ldots (3n)(3n+1)}, \quad \text{(Associated with } Bi(x)\text{)} \\ &= x + \sum_{n=1}^{\infty} \frac{x^{3n+1}}{3^n n![1 \cdot 4 \ldots (3n+1)]} \end{aligned}$$

Theorem 3.14: The Binomial Theorem

The Binomial theorem provides a fast way to expand any two-term expression raised to a power, the coefficients for each term is computed with Pascal' triangle or combination rule.
The expansion of $(x+y)^n$ with integer $n \geq 0$:

$$(x+y)^n = \sum_{k=0}^{n} \binom{n}{k} x^{n-k} y^k$$

with

- n as the power or exponent, n is an integer.
- x and y as the two terms inside the parentheses.
- $\binom{n}{k}$ as the Binomial coefficient (n choose k).
- Exponents of x start at n and decrease by one each step $(n, n-1, \ldots, 0)$.
- Exponents of y start at 0 and increase by one each step $(0, 1, \ldots, n)$.

Definition 3.2: Bernoulli Numbers

Bernoulli numbers (B_n) form a sequence of rational numbers used in number theory, calculus, and expansions of trigonometric functions. They are most easily defined via an Exponential Generating Function or computed step-by-step using a recursive formula.

The Bernoulli numbers are the coefficients in the Taylor series expansion of the function $\frac{x}{e^x-1}$

$$\frac{x}{e^x-1} = \sum_{n=0}^{\infty} B_n \frac{x^n}{n!}$$

To compute B_n recursively starting with $B_0 = 1$, use the identity for $n \geq 1$:

$$\sum_{k=0}^{n} \binom{n+1}{k} B_k = 0$$

which rearranges to the explicit iterative formula:

$$B_n = -\frac{1}{n+1} \sum_{k=0}^{n-1} \binom{n+1}{k} B_k$$

The first 7 terms of Bernoulli numbers:

$$\begin{aligned} B_0 &= 1 \\ B_1 &= -0.5 \\ B_2 &= 0.166667 \\ B_3 &= 0 \\ B_4 &= -0.0333333 \\ B_5 &= 0 \\ B_6 &= 0.0238095 \end{aligned}$$

The Bernoulli numbers have appeared in many important results, including the series expansions of trigonometric and hyperbolic trigonometric functions, the Euler-Maclaurin summation formula, the evaluation of the Riemann zeta function, and Fermat's Last Theorem.

[SI*] We will list the Maclaurin series (or the Taylor series expansion about $x = 0$)for several

functions:

$$\cos(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n}}{(2n)!} = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \ldots, \quad -\infty < x < \infty$$

$$\sin(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n+1}}{(2n+1)!} = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \ldots, \quad -\infty < x < \infty$$

$$\tan(x) = \sum_{n=1}^{\infty} \frac{(-1)^{n-1} 2^{2n} (2^{2n-1}) B_{2n}}{(2n)!} x^{2n-1} = x + \frac{x^3}{3} + \frac{2x^5}{15} + \frac{16x^7}{315} + \ldots, \quad |x| < \frac{\pi}{2}$$

$$\csc(x) = \sum_{n=1}^{\infty} \frac{(-1)^{n-1} 2(2^{2n-1} - 1) B_{2n}}{(2n)!} x^{2n-1} = \frac{1}{x} + \frac{x}{6} + \frac{7x^3}{360} + \frac{31x^5}{15120} + \ldots, \quad 0 < |x| < \pi$$

$$\sec(x) = \sum_{n=0}^{\infty} \frac{E_{2n}}{(2n)!} x^{2n} = 1 + \frac{x^2}{2} + \frac{5x^4}{24} + \frac{61x^6}{720} + \ldots, \quad |x| < \frac{\pi}{2}$$

$$\cot(x) = \sum_{n=0}^{\infty} \frac{(-1)^n 2^{2n} B_{2n}}{(2n)!} x^{2n-1} = \frac{1}{x} - \frac{x}{3} - \frac{x^3}{45} - \frac{2x^5}{945} - \ldots, \quad 0 < |x| < \pi$$

$$\sinh(x) = \sum_{n=0}^{\infty} \frac{x^{2n+1}}{(2n+1)!} = x + \frac{x^3}{3!} + \frac{x^5}{5!} + \frac{x^7}{7!} + \ldots, \quad x \in \mathbb{C} \text{ or } x \in \mathbb{R}$$

$$\cosh(x) = \sum_{n=0}^{\infty} \frac{x^{2n}}{(2n)!} = 1 + \frac{x^2}{2!} + \frac{x^4}{4!} + \frac{x^6}{6!} + \ldots, \quad x \in \mathbb{C} \text{ or } x \in \mathbb{R}$$

$$\tanh(x) = \sum_{n=1}^{\infty} \frac{2^{2n}(2^{2n} - 1) B_{2n}}{(2n)!} x^{2n-1} = x - \frac{x^3}{3} + \frac{2}{15}x^5 - \frac{17}{315}x^7 + \ldots, \quad |x| < \frac{\pi}{2}$$

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!} = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \ldots$$

$$\ln(1+x) = \sum_{n=1}^{\infty} (-1)^{n-1} \frac{x^n}{n} = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \ldots, \quad -1 < x \leq 1$$

B_{2n} is Bernoulli numbers and E_n is Euler numbers, the function $\ln(x)$ has no Maclaurin series because $ln(x)$ is undefined at $x = 0$. Since $\sinh(x) = \frac{e^x - e^{-x}}{2}$ and $\cosh(x) = \frac{e^x + e^{-x}}{2}$, we can substitute the known Maclaurin series for e^x and e^{-x} and add or subtract them term by term.

If we want to compute the Maclaurin series for $f(x) = \ln(x+2)$, we will need to use substitution and using the logarithm arithmetic rule:

$$\ln\left(2\left(1 + \frac{x}{2}\right)\right) = \ln(2) + \ln\left(1 + \frac{x}{2}\right)$$

By substituting $u = \frac{x}{2}$, we will have

$$\ln(1+u) = u - \frac{u^2}{2} + \frac{u^3}{3} - \frac{u^4}{4} + \ldots$$

thus

$$\ln\left(1 + \frac{x}{2}\right) = \frac{x}{2} - \frac{x^2}{8} + \frac{x^3}{24} - \frac{x^4}{64} + \ldots$$

Finally, we will obtain the Maclaurin series for $\ln(x+2)$

$$\begin{aligned}\ln(x+2) &= \ln\left(2\left(1+\frac{x}{2}\right)\right)\\ &= \ln(2)+\ln\left(1+\frac{x}{2}\right)\\ &= \ln(2)+\frac{x}{2}-\frac{x^2}{8}+\frac{x^3}{24}-\frac{x^4}{64}+\dots\\ &= \ln(2)+\sum_{n=1}^{\infty}\frac{(-1)^{n-1}x^n}{n2^n}\end{aligned}$$

vi. Frobenius Method

[SI*] In mathematics, the method of Frobenius, named after Ferdinand Georg Frobenius, is a way to find an infinite series solution for a linear second-order ordinary differential equation of the form

$$x^2y''+p(x)xy'+q(x)y=0 \tag{3.345}$$

with $y'=\frac{dy}{dx}$ and $y''=\frac{d^2y}{dx^2}$.
In the vicinity of the regular singular point $x=0$. One can divide by x^2 to obtain a differential equation of the form

$$y''+\frac{p(x)}{x}y'+\frac{q(x)}{x^2}y=0$$

which will not be solvable with regular power series methods if either $\frac{p(x)}{x}$ or $\frac{q(x)}{x^2}$ is not analytic at $x=0$. The Frobenius method enables one to create a power series solution to such a differential equation, provided that $p(x)$ and $q(x)$ are themselves analytic at 0 or, being analytic elsewhere, both their limits at 0 exist (and are finite).

[SI*] Frobenius' contribution was not so much in all the possible forms of the series solutions involved. These forms had all been established earlier, by Lazarus Fuchs. The indicial polynomial and its role had also been established by Fuchs.

A first contribution by Frobenius to the theory was to show that-as regards a first, linearly independent solution, which then has the form of an analytical power series multiplied by an arbitrary power r of the independent variable-the coefficients of the generalized power series obey a recurrence relation, so that they can always be straightforwardly calculated.

A second contribution by Frobenius was to show that, in cases in which the roots of the indicial equation differ by an integer, the general form of the second linearly independent solution can be obtained by a procedure which is based on differentiation with respect to the parameter r.

A large part of Frobenius' 1873 publication was devoted to proofs of convergence of all the series involved in the solutions, as well as establishing the radii of convergence of these series.

[SI*] The method of Frobenius is to seek a power series solution of the form

$$y(x)=x^r\sum_{k=0}^{\infty}A_kx^k,\quad A_0\neq 0$$

Differentiating:

$$y'(x) = \sum_{k=0}^{\infty}(k+r)A_k x^{k+r-1}$$
$$y''(x) = \sum_{k=0}^{\infty}(k+r-1)(k+r)A_k x^{k+r-2}$$

Substituting the above differentiation into the original ODE Eq. (3.283):

$$\begin{aligned}
&x^2\sum_{k=0}^{\infty}(k+r-1)(k+r)A_k x^{k+r-2} + xp(x)\sum_{k=0}^{\infty}(k+r)A_k x^{k+r-1} + q(x)\sum_{k=0}^{\infty}A_k x^{k+r}\\
&= \sum_{k=0}^{\infty}(k+r-1)(k+r)A_k x^{k+r} + p(x)\sum_{k=0}^{\infty}(k+r)A_k x^{k+r} + q(x)\sum_{k=0}^{\infty}A_k x^{k+r}\\
&= \left[(k+r-1)(k+r)A_k x^{k+r} + p(x)(k+r)A_k x^{k+r} + q(x)x^{k+r}\right]\\
&= \sum_{k=0}^{\infty}\left[(k+r-1)(k+r)A_k x^{k+r} + p(x)(k+r)A_k x^{k+r} + q(x)A_k x^{k+r}\right]\\
&= \sum_{k=0}^{\infty}\left[(k+r-1)(k+r) + p(x)(k+r) + q(x)\right]A_k x^{k+r}\\
&= [r(r-1) + p(x)r + q(x)]A_0 x^r\\
&+ \sum_{k=1}^{\infty}[(k+r-1)(k+r) + p(x)(k+r) + q(x)]A_k x^{k+r} = 0
\end{aligned}$$

vii. The Airy Differential Equation

[SI*] In mathematics, the Airy function $Ai(x)$ is a special function named after the British astronomer George Biddell Airy. The function $Ai(x)$ and the related function $Bi(x)$ are linearly independent solutions to the differential equation

$$\frac{d^2y}{dx^2} - xy = 0$$

known as the Airy equation or the Stokes equation. This is a second-order linear ordinary differential equation with a non-constant coefficient (x).
Because the solution of the linear differential equation

$$\frac{d^2y}{dx^2} - ky = 0$$

is oscillatory for $k < 0$ and exponential for $k > 0$, the Airy functions are oscillatory for $x < 0$ and exponential for $x > 0$. In fact, the Airy equation is the simplest second-order linear differential equation with a turning point (a point where the character of the solutions changes from oscillatory to exponential).

[SI*] The general solution to the Airy differential equation $y'' - xy = 0$ is expressed as a linear combination of two independent Airy functions, $Ai(x)$ and $Bi(x)$:

$$y(x) = c_1 Ai(x) + c_2 Bi(x)$$

where c_1 and c_2 are real constants, $Ai(x)$ is the Airy function of the first kind, which decays exponentially as $x \to \infty$ and oscillates $x \to -\infty$. $Bi(x)$ is the Airy function of the second kind, which grows exponentially as $x \to \infty$ and oscillates as $x \to -\infty$. Both $Ai(x)$ and $Bi(x)$ share the same amplitude and frequency of oscillation when $x \to -\infty$, but they differ in phase by $\frac{\pi}{2}$.

For real values of x, the Airy function of the first kind can be defined by the improper Riemann integral

$$Ai(x) = \frac{1}{\pi}\int_0^\infty \cos\left(\frac{t^3}{3} + xt\right) dt$$

which converges by Dirichlet's test. Up to scalar multiplication, $Ai(x)$ is the solution subject to the condition $y \to 0$ as $x \to \infty$. The standard choice for the other solution is the Airy function of the second kind, denoted by $Bi(x)$. It is defined as the solution with the same amplitude of oscillation as $Ai(x)$ as $x \to \infty$ which differs in phase by $\frac{\pi}{2}$:

$$Bi(x) = \frac{1}{\pi}\int_0^\infty \left[\sin\left(\frac{t^3}{3} + xt\right) + e^{-\frac{t^3}{3}+xt}\right] dt$$

Power Series Solutions
Because the coefficient is variable, the equation is solved using a power series

$$y(x) = \sum_{n=0}^{\infty} a_n x^n$$

The recurrence relation:

$$a_{n+2} = \frac{a_{n-1}}{(n+1)(n+2)}, \quad \text{for } n \geq 1$$

Thus the linearly independent series components:

$$\begin{aligned} y_1(x) &= 1 + \sum_{n=1}^{\infty} \frac{x^{3n}}{2 \cdot 3 \cdot 5 \cdot 6 \ldots (3n-1)(3n)}, \quad \text{(Associated with } Ai(x)\text{)} \\ &= 1 + \sum_{n=1}^{\infty} \frac{x^{3n}}{3^n n![2 \cdot 5 \ldots (3n-1)]} \\ y_2(x) &= x + \sum_{n=1}^{\infty} \frac{x^{3n+1}}{3 \cdot 4 \cdot 6 \cdot 7 \ldots (3n)(3n+1)}, \quad \text{(Associated with } Bi(x)\text{)} \\ &= x + \sum_{n=1}^{\infty} \frac{x^{3n+1}}{3^n n![1 \cdot 4 \ldots (3n+1)]} \end{aligned}$$

For negative x ($x < 0$) solutions are oscillatory, behaving similarly to sine and cosine functions with changing amplitude. For positive x ($x > 0$) $Ai(x)$ decays exponentially toward zero, while $Bi(x)$ grows exponentially.

[SI*] The Airy differential equation ($y'' - xy = 0$) is applied widely in physics, quantum mechanics, optics, and wave propagation to model phenomena near turning points, such as light diffraction, supernumerary rainbows, and electrons in triangular potential wells. Its solutions, called Airy functions, describe fields that change from oscillating to exponential behavior, it is able to describe physical phenomena where wave behavior transitions between oscillatory and exponential decay.
Application of the Airy differential equation:

1. **Quantum Mechanics**

   - **Triangular Wells:** Solves the Schrödinger equation for particles in a linear constant force field or confined by a triangular potential well. Describes electron wave functions in triangular potential wells.
   - **Semiconductor Heterojunctions:** Explains the behavior and trapping of electrons near interfaces under uniform electric fields.
   - **Uniform Approximations:** Approximates wave behavior via the WKB method near classical turning points where potential equals energy.
   - **WKB Approximation:** Provides uniform semiclassical approximations near classical turning points where potentials change linearly.

2. **Optics and Wave Propagation**

   - **Optical Caustics:** Models light intensity patterns near bright edges and caustics, such as the formation of rainbows.
   - **Electromagnetic Diffraction:** Describes light and radio wave diffraction patterns near shadow boundaries.

3. **Acoustics and Electromagnetism**

   - **Underwater Sound:** Computes transparent boundary conditions for parabolic wave equations simulating sound in the ocean.
   - **Radio Waves:** Analyzes tropospheric radar propagation and radio wave signals curving through the atmosphere.
   - **Atmospheric and Underwater Acoustics:** Used in parabolic equation simulations for tracking sound or radar wave propagation through the troposphere and oceans.

viii. The Hermite Differential Equation

[SI*] In mathematics, the Hermite polynomials are a classical orthogonal polynomial sequence. The polynomials arise in:

- Signal processing as Hermitian wavelets for wavelet transform analysis.
- Probability, such as the Edgeworth series, as well as in connection with Brownian motion.
- Combinatorics, as an example of an Appell sequence, obeying the umbral calculus.
- Numerical analysis as Gaussian quadrature.
- Physics, where they give rise to the eigenstates of the quantum harmonic oscillator; and they also occur in some cases of the heat equation (when the term xu_x is present).
- Systems theory in connection with nonlinear operations on Gaussian noise.
- Random matrix theory in Gaussian ensembles.

Hermite polynomials were defined by Pierre-Simon Laplace in 1810, though in scarcely recognizable form, and studied in detail by Pafnuty Chebyshev in 1859. Chebyshev's work was overlooked, and they were named later after Charles Hermite, who wrote on the polynomials in 1864, describing them as new. They were not new, although Hermite was the first to define the multidimensional polynomials.

Like the other classical orthogonal polynomials, the Hermite polynomials can be defined from several different starting points. Noting from the outset that there are two different standardizations in common use, one convenient method is as follows:

- The "**probabilist's Hermite polynomials**" are given by

$$He_n(x) = (-1)^n e^{\frac{x^2}{2}} \frac{d^n}{dx^n} e^{-\frac{x^2}{2}}$$

- The "**physicist's Hermite polynomials**" are given by

$$H_n(x) = (-1)^n e^{x^2} \frac{d^n}{dx^n} e^{-x^2}$$

These equations have the form of a Rodrigues' formula and can also be written as

$$He_n(x) = \left(x - \frac{d}{dx}\right)^n \cdot 1$$
$$H_n(x) = \left(2x - \frac{d}{dx}\right)^n \cdot 1$$

The two definitions are not exactly identical; each is rescaling of the other:

$$He_n(x) = 2^{-\frac{n}{2}} H_n\left(\frac{x}{\sqrt{2}}\right)$$
$$H_n(x) = 2^{\frac{n}{2}} He_n\left(\sqrt{2}x\right)$$

The polynomials He_n are sometimes denoted by H_n, especially in probability theory, because

$$\frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}$$

is the probability density function for the normal distribution with expected value 0 and standard deviation 1. The probabilist's Hermite polynomials are also called the monic Hermite polynomials, because they are monic.

- The first eleven probabilist's Hermite polynomials are:

$$He_0(x) = 1$$
$$He_1(x) = x$$
$$He_2(x) = x^2 - 1$$
$$He_3(x) = x^3 - 3x$$
$$He_4(x) = x^4 - 6x^2 + 3$$
$$He_5(x) = x^5 - 10x^3 + 15x$$
$$He_6(x) = x^6 - 15x^4 + 45x^2 - 15$$
$$He_7(x) = x^7 - 21x^5 + 105x^3 - 105x$$
$$He_8(x) = x^8 - 28x^6 + 210x^4 - 420x^2 + 105$$
$$He_9(x) = x^9 - 36x^7 + 378x^5 - 1260x^3 + 945x$$
$$He_{10}(x) = x^{10} - 45x^8 + 630x^6 - 3150x^4 + 4725x^2 - 945$$

- The first eleven physicist's Hermite polynomials are:

$$
\begin{aligned}
H_0(x) &= 1 \\
H_1(x) &= 2x \\
H_2(x) &= 4x^2 - 2 \\
H_3(x) &= 8x^3 - 12x \\
H_4(x) &= 16x^4 - 48x^2 + 12 \\
H_5(x) &= 32x^5 - 160x^3 + 120x \\
H_6(x) &= 64x^6 - 480x^4 + 720x^2 - 120 \\
H_7(x) &= 128x^7 - 1344x^5 + 3360x^3 - 1680x \\
H_8(x) &= 256x^8 - 3584x^6 + 13440x^4 - 13440x^2 + 1680 \\
H_9(x) &= 512x^9 - 9216x^7 + 48384x^5 - 80640x^3 + 30240x \\
H_{10}(x) &= 1024x^{10} - 23040x^8 + 161280x^6 - 403200x^4 + 302400x^2 - 30240
\end{aligned}
$$

	Physicist's	**Probabilist's**
Symbol	H_n	He_n
Head coefficient	2^n	1
Differential operator	$(-1)^n e^{x^2} \frac{d^n}{dx^n} e^{-x^2}$	$(-1)^n e^{\frac{x^2}{2}} \frac{d^n}{dx^n} e^{-\frac{x^2}{2}}$
Orthogonal to	e^{-x^2}	$e^{-\frac{1}{2}x^2}$
Inner product	$\int H_m(x) H_n(x) \frac{e^{-x^2}}{\sqrt{\pi}} dx = 2^n n! \delta_{mn}$	$\int He_m(x) He_n(x) \frac{e^{-\frac{x^2}{2}}}{\sqrt{2\pi}} dx = n! \delta_{nm}$
Generating function	$e^{xt-t^2} = \sum_{n=0}^{\infty} H_n(x) \frac{t^n}{n!}$	$e^{xt-\frac{1}{2}t^2} = \sum_{n=0}^{\infty} He_n(x) \frac{t^n}{n!}$
Rodrigues' formula	$\left(2x - \frac{d}{dx}\right)^n \cdot 1$	$\left(x - \frac{d}{dx}\right)^n \cdot 1$
Recurrence relation	$H_{n+1}(x) = 2xH_n(x) - 2nH_{n-1}(x)$	$He_{n+1}(x) = xHe_n(x) - nHe_{n-1}(x)$

Table 3.7: *The differences between probabilist's Hermite polynomials and physicist's Hermite polynomials.*

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Hermite Polynomials ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Hermite Polynomials ]# ./main
Physicist's Hermite polynomials:
H_{0} (x) = 1
H_{1} (x) = 2*x
H_{2} (x) = 4*x^(2)-2
H_{3} (x) = 8*x^(3)-12*x
H_{4} (x) = 16*x^(4)-48*x^(2)+12
H_{5} (x) = 32*x^(5)-160*x^(3)+120*x
H_{6} (x) = 64*x^(6)-480*x^(4)+720*x^(2)-120
H_{7} (x) = 128*x^(7)-1344*x^(5)+3360*x^(3)-1680*x
H_{8} (x) = 256*x^(8)-3584*x^(6)+13440*x^(4)-13440*x^(2)+1680
H_{9} (x) = 512*x^(9)-9216*x^(7)+48384*x^(5)-80640*x^(3)+30240*x
H_{10} (x) = 1024*x^(10)-23040*x^(8)+161280*x^(6)-403200*x^(4)+302400*x^(2)-30240

Probabilist's Hermite polynomials:
He_{0} (x) = 1
He_{1} (x) = x
H_{2} (x) = x^(2)-1
H_{3} (x) = x^(3)-3*x
H_{4} (x) = x^(4)-6*x^(2)+3
H_{5} (x) = x^(5)-10*x^(3)+15*x
H_{6} (x) = x^(6)-15*x^(4)+45*x^(2)-15
H_{7} (x) = x^(7)-21*x^(5)+105*x^(3)-105*x
H_{8} (x) = x^(8)-28*x^(6)+210*x^(4)-420*x^(2)+105
H_{9} (x) = x^(9)-36*x^(7)+378*x^(5)-1260*x^(3)+945*x
H_{10} (x) = x^(10)-45*x^(8)+630*x^(6)-3150*x^(4)+4725*x^(2)-945

Time taken by function: 196326 microseconds
```

Figure 3.89: *The first eleven physicist's and probabilist's Hermite polynomials (**SymIntegration/Examples/Differential Equations/Test SymIntegration Hermite Polynomials/main.cpp**).*

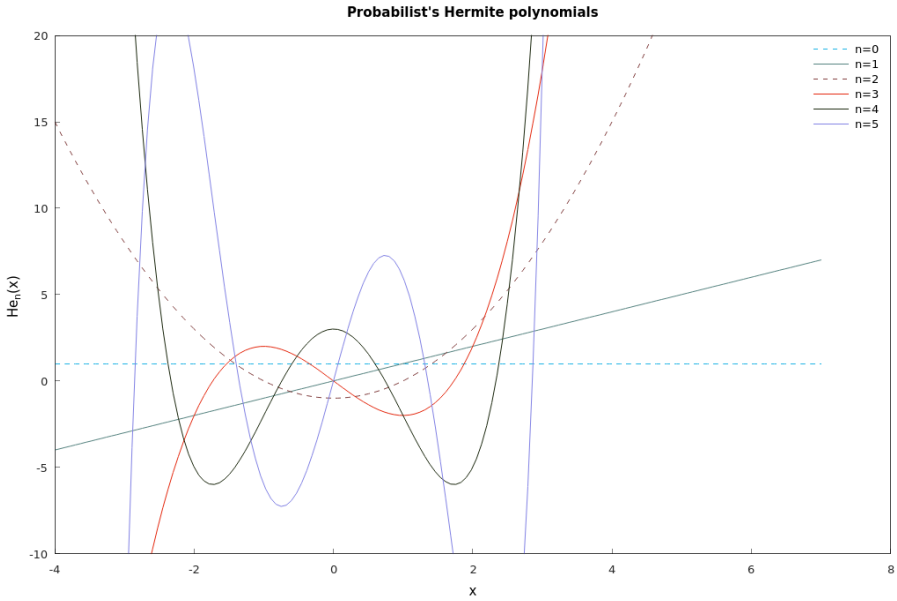

Figure 3.90: *The probabilist's Hermite polynomials (**Hamzstplot/Examples with Makefile/Differential Equations/Plot 2D Probabilist's Hermite Polynomials/main.cpp**).*

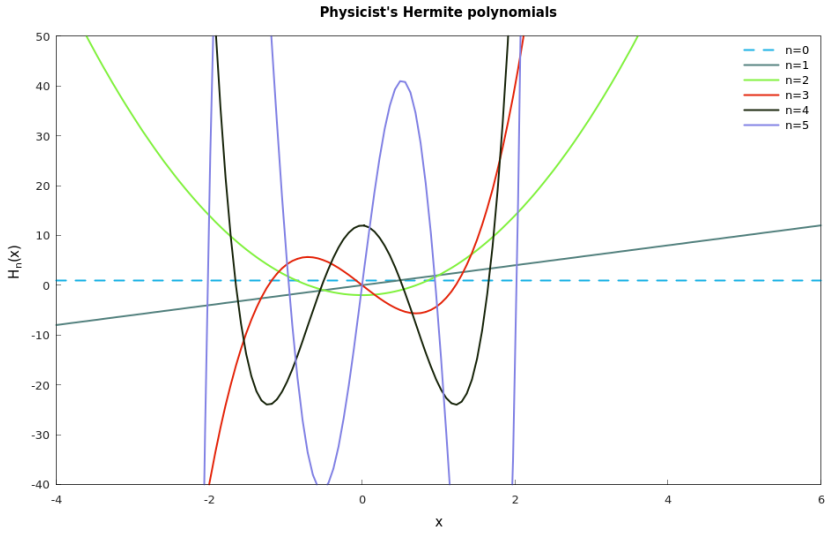

Figure 3.91: *The physicist's Hermite polynomials (**Hamzstplot/Examples with Makefile/Differential Equations/-Plot 2D Physicist's Hermite Polynomials/main.cpp**).*

[SI*] The probabilist's Hermite polynomials are solutions of the Sturm-Liouville differential equation

$$\left(e^{-\frac{1}{2}x^2}y'\right)' + \lambda e^{-\frac{1}{2}x^2}y = 0$$

where λ is a constant. Imposing the boundary condition that y should be polynomially bounded at infinity, the equation has solutions only if λ is a non-negative integer, and the solution is uniquely given by

$$y(x) = C_1 He_\lambda(x)$$

where C_1 is a constant.
Rewriting the differential equation as an eigenvalue problem

$$L[y] = y'' - xy' = -\lambda y$$

the Hermite polynomials $He_\lambda(x)$ may be understood as eigenfunctions of the differential operator $L[y]$. This eigenvalue problem is called the Hermite equation, although the term is also used for the closely related equation

$$y'' - xy' = -\lambda y$$

whose solution is uniquely given in terms of physicist's Hermite polynomials in the form

$$y(x) = C_1 H_\lambda(x)$$

where C_1 denotes a constant, after imposing the boundary condition that y should be polynomially bounded at infinity.
The general solutions to the above second-order differential equations are in fact linear combinations of both Hermite polynomials and confluent hypergeometric functions of the first kind. For example, for the physicist's Hermite equation

$$y'' - xy' + \lambda y = 0, \quad -\infty < x < \infty$$

the general solution takes the form

$$y(x) = C_1 H_\lambda(x) + C_2 h_\lambda(x)$$

where C_1 and C_2 are constants, $H_\lambda(x)$ are physicist's Hermite polynomials (of the first kind), and $h_\lambda(x)$ are physicist Hermite functions (of the second kind). The latter functions are compactly represented as

$$h_\lambda(x) = {}_1F_1\left(-\frac{\lambda}{2}; \frac{1}{2}; x^2\right)$$

where ${}_1F_1(a;b;z)$ are Confluent hypergeometric functions of the first kind. The conventional Hermite polynomials may also be expressed in terms of confluent hypergeometric functions. With more general boundary conditions, the Hermite polynomials can be generalized to obtain more general analytic functions for complex-valued λ. An explicit formula of Hermite polynomials in terms of contour integrals is also possible.

[SI*] **Recurrence Relation**

The sequence of probabilist's Hermite polynomials also satisfies the recurrence relation

$$He_{n+1}(x) = xHe_n(x) - He_n'(x)$$

Individual coefficients are related by the following recursion formula:

$$a_{n+1,k} = \begin{cases} -(k+1)a_{n,k+1}, & k = 0 \\ a_{n,k-1} - (k+1)a_{n,k+1}, & k > 0 \end{cases}$$

with $a_{0,0} = 1, a_{1,0} = 0, a_{1,1} = 1$.

For the physicist's Hermite polynomials, assuming

$$H_n(x) = \sum_{k=0}^{n} a_{n,k} x^k$$

we have

$$H_{n+1}(x) = 2xH_n(x) - H_n'(x)$$

Individual coefficients are related by the following recursion formula:

$$a_{n+1,k} = \begin{cases} -a_{n,k+1}, & k = 0 \\ 2a_{n,k-1}, & k > 0 \end{cases}$$

with $a_{0,0} = 1, a_{1,0} = 0, a_{1,1} = 2$.

The Hermite polynomials constitute an Appell sequence, i.e., they are a polynomial sequence satisfying the identity

$$He_n'(x) = nHe_{n-1}(x)$$
$$H_n'(x) = 2nH_{n-1}(x)$$

An integral recurrence that is deduced is as follows:

$$He_{n+1}(x) = (n+1)\int_0^x He_n(t)\, dt - He_n'(0)$$
$$H_{n+1}(x) = 2(n+1)\int_0^x H_n(t)\, dt - H_n'(0)$$

Equivalently, by Taylor-expanding,

$$He_n(x+y) = \sum_{k=0}^{n} \binom{n}{k} x^{n-k} He_k(y) = 2^{-\frac{n}{2}} \sum_{k=0}^{n} \binom{n}{k} He_{n-k}(x\sqrt{2}) He_k(y\sqrt{2})$$
$$H_n(x+y) = \sum_{k=0}^{n} \binom{n}{k} H_k(x)(2y)^{n-k} = 2^{-\frac{n}{2}} \sum_{k=0}^{n} \binom{n}{k} H_{n-k}(x\sqrt{2}) H_k(y\sqrt{2})$$

These umbral identities are self-evident and included in the differential operator representation

$$He_n(x) = e^{-\frac{D^2}{2}} x^n$$
$$H_n(x) = 2^n e^{-\frac{D^2}{4}} x^n$$

In consequence, for the m-th derivatives the following relations hold:

$$He_n^{(m)}(x) = \frac{n!}{(n-m)!} He_{n-m}(x) = m! \binom{n}{m} He_{n-m}(x)$$
$$H_n^{(m)}(x) = 2^m \frac{n!}{(n-m)!} H_{n-m}(x) = 2^m m! \binom{n}{m} H_{n-m}(x)$$

It follows that the Hermite polynomials also satisfy the recurrence relation.

$$He_{n+1}(x) = xHe_n(x) - nHe_{n-1}(x)$$
$$H_{n+1}(x) = 2xH_n(x) - 2nH_{n-1}(x)$$

These last relations, together with the initial polynomials $H_0(x)$ and $H_1(x)$, can be used in practice to compute the polynomials quickly.
Turán's inequalities are

$$H_n(x)^2 - H_{n-1}(x)H_{n+1}(x) = (n-1)! \sum_{i=0}^{n-1} \frac{2^{n-1}}{i!} H_i(x)^2 > 0$$

Moreover, the following multiplication theorem holds:

$$H_n(\gamma x) = \sum_{i=0}^{\lfloor \frac{n}{2} \rfloor} \gamma^{n-2i} (\gamma^2 - 1)^i \binom{n}{2i} \frac{(2i)!}{i!} H_{n-2i}(x)$$
$$He_n(\gamma x) = \sum_{i=0}^{\lfloor \frac{n}{2} \rfloor} \gamma^{n-2i} (\gamma^2 - 1)^i \binom{n}{2i} \frac{(2i)!}{i!} 2^{-i} He_{n-2i}(x)$$

[SI*] **Explicit Expression**
The physicist's Hermite polynomials can be written explicitly as

$$H_n(x) = \begin{cases} n! \sum_{l=0}^{\frac{n}{2}} \frac{(-1)^{\frac{n}{2}-l}}{(2l)!\left(\frac{n}{2}-l\right)!} (2x)^{2l}, & \text{for even } n, \\ n! \sum_{l=0}^{\frac{n-1}{2}} \frac{(-1)^{\frac{n-1}{2}-l}}{(2l+1)!\left(\frac{n-1}{2}-l\right)!} (2x)^{2l+1}, & \text{for even } n, \end{cases}$$

These two equations may be combined into one using the floor function:

$$H_n(x) = n! \sum_{m=0}^{\lfloor \frac{n}{2} \rfloor} \frac{(-1)^m}{m!(n-2m)!} (2x)^{n-2m}$$

The probabilist's Hermite polynomials He have similar formulas, which may be obtained from these by replacing the power of $2x$ with the corresponding power of $\sqrt{2}x$ and multiplying the entire sum by $2^{-\frac{n}{2}}$:

$$He_n(x) = n! \sum_{m=0}^{\lfloor \frac{n}{2} \rfloor} \frac{(-1)^m}{m!(n-2m)!} \frac{x^{n-2m}}{2^m}$$

Inverse Explicit Expression
The inverse of the above explicit expressions, that is, those for monomials in terms of probabilist's Hermite polynomials He are

$$x^n = n! \sum_{m=0}^{\lfloor \frac{n}{2} \rfloor} \frac{1}{2^m m!(n-2m)!} He_{n-2m}(x)$$

The corresponding expressions for the physicist's Hermite polynomials H follow directly by properly scaling this:

$$x^n = \frac{n!}{2^n} \sum_{m=0}^{\lfloor \frac{n}{2} \rfloor} \frac{1}{m!(n-2m)!} H_{n-2m}(x)$$

[SI*] **Generating Function**
The Hermite polynomials are given by the exponential generating function

$$e^{xt - \frac{1}{2}t^2} = \sum_{n=0}^{\infty} He_n(x) \frac{t^n}{n!}$$
$$e^{2xt - t^2} = \sum_{n=0}^{\infty} H_n(x) \frac{t^n}{n!}$$

This equality is valid for all complex values of x and t, and can be obtained by writing the Taylor expansion at x of the entire function $z \to e^{-z^2}$ (in the physicist's case). One can also derive the (physicist's) generating function by using the Cauchy's integral formula to write the Hermite polynomials as

$$H_n(x) = (-1)^n e^{x^2} \frac{d^n}{dx^n} e^{-x^2} = (-1)^n e^{x^2} \frac{n!}{2\pi i} \oint_\gamma \frac{e^{-z^2}}{(z-x)^{n+1}} dz$$

Using this in the sum

$$\sum_{n=0}^{\infty} H_n(x) \frac{t^n}{n!}$$

one can evaluate the remaining integral using the calculus of residues and arrive at the desired generating function.
A slight generalization states

$$e^{2xt - t^2} H_k(x - t) = \sum_{n=0}^{\infty} \frac{H_{n+k}(x) t^n}{n!}$$

ix. Classical Orthogonal Polynomials

[SI*] Orthogonal polynomials are infinite sequences of polynomials where any two distinct polynomials in the set have an inner product of zero, defined with respect to a specific interval

and weight function. They behave like perpendicular vectors and serve as foundational tools in approximation theory, quantum physics, and numerical integration.
Core properties:

- **Inner Product:** Defined via an integral over an interval $[a, b]$ with a non-negative weight function $w(x)$, this is the orthogonality condition:

$$\int_a^b p_n(x)p_m(x)w(x)\,dx = 0, \quad \text{for } n \neq m$$

- **Degree Sequence:** Each $p_n(x)$ represents a distinct polynomial of exact degree n.
- **Real Roots:** The roots of an orthogonal polynomial of degree n are real, distinct, and located strictly inside the orthogonality interval.
- **Recurrence Relations:** Any system of orthogonal polynomials satisfies a simple three-term recurrence relation, allowing fast generation of higher degrees.

Any standard basis $\{1, x, x^2, \dots\}$ can be turned into orthogonal polynomials using the Gram-Schmidt process.

Classical families:

- **Legendre Polynomials:** Orthogonal on $[-1, 1]$ with a weight function of 1; used in potential theory and spherical harmonics.
- **Chebyshev Polynomials:** Orthogonal on $[-1, 1]$ with weight $(1 - x^2)^{-\frac{1}{2}}$; crucial for minimax polynomial approximation and numerical interpolation.
- **Hermite Polynomials:** Orthogonal on $(-\infty, \infty)$ with weight e^{-x^2}; heavily applied in quantum harmonic oscillators.
- **Laguerre Polynomials:** Orthogonal on $[0, \infty)$ with weight e^{-x}; used in quantum mechanics for hydrogen wavefunctions.

[SI*] Classical orthogonal polynomials-such as Legendre, Chebyshev, Hermite and Laguerre polynomials-are widely used in mathematical physics (in particular, the theory of random matrices), numerical analysis, and engineering to solve differential equations, approximate complex functions efficiently, and improve numerical integration accuracy / compute integrals.
Core mathematical tasks:

- **Differential Equations:** Serve as exact eigenfunction solutions to classical Sturm-Liouville differential systems.
- **Spectral Methods:** Act as global basis functions to convert differential equations into algebraic equations for fast computer simulations.
- **Numerical Integration (Quadrature):** Form the foundation of Gaussian quadrature rules for precise numerical integration of smooth functions. Gaussian quadrature rules use roots of these polynomials to compute definite integrals with high precision using few points.

[SI*] Major function families and their applications:

- **Legendre Polynomials:** Applied in potential theory and problems with spherical symmetry (like gravitational and electrostatic potentials).
- **Chebyshev Polynomials:** Used in numerical analysis for polynomial interpolation, minimizing interpolation errors (minimax approximation), and efficient numerical quadrature.

- **Hermite Polynomials:** Applied in quantum mechanics to solve the quantum harmonic oscillator wave equation, and in probability for Hermite transform and random processes (probability models with Gaussian distributions).
- **Laguerre Polynomials:** Used in quantum chemistry (solving Schrödinger equation for hydrogen-like atoms), as well as in chemical physics and control theory involving systems with exponential decay over semi-infinite.
- **Jacobi Polynomials:** Applied in boundary value problems and specialized spectral methods where endpoint behavior needs precise tuning.

[SI*] In mathematics, the classical orthogonal polynomials are the most widely used orthogonal polynomials: the Hermite polynomials, Laguerre polynomials, Jacobi polynomials (including as a special case the Gegenbauer polynomials, Chebyshev polynomials, and Legendre polynomials).
Classical orthogonal polynomials appeared in the early 19th century in the works of Adrien-Marie Legendre, who introduced the Legendre polynomials. In the late 19th century, the study of continued fractions to solve the moment problem by P.L. Chebyshev and then A.A. Markov and T.J. Stieltjes led to the general notion of orthogonal polynomials.

For given polynomials $Q, L : \mathbb{R} \to \mathbb{R}$ and $\forall n \in \mathbb{N}_0$ the classical orthogonal polynomials $f_n : \mathbb{R} \to \mathbb{R}$ are characterized by being solutions of the differential equation

$$Q(x)f_n'' + L(x)f_n' + \lambda_n f_n = 0$$

with to be determined constants $\lambda_n \in \mathbb{R}$.

[SI*] In general, the orthogonal polynomials P_n with respect to a weight $W : \mathbb{R} \to \mathbb{R}^+$ satisfy

$$\deg P_n = n,\ n = 0, 1, 2, \ldots$$
$$\int P_m(x)P_n(x)W(x)\ dx = 0, \quad m \neq n$$

The relations above define P_n up to multiplication by a number. Various normalisations are used to fix the constant, e.g.

$$\int P_n(x)^2 W(x)\ dx = 1$$

The classical orthogonal polynomials correspond to the following three families of weights:

$$\text{(Jacobi)} \quad W(x) = \begin{cases} (1-x)^\alpha (1+x)^\beta, & -1 \leq x \leq 1 \\ 0, & \text{otherwise} \end{cases}$$

$$\text{(Physicist's Hermite)} \quad W(x) = e^{-x^2}$$

$$\text{(Laguerre)} \quad W(x) = \begin{cases} x^\alpha e^{-x}, & x \geq 0 \\ 0, & \text{otherwise} \end{cases}$$

[SI*] **Jacobi Polynomials**

Jacobi polynomials are a standard class of orthogonal polynomials denoted as $P_n^{(\alpha,\beta)}(x)$ that are defined on the interval $[-1, 1]$ with respect to the weight function $(1-x)^\alpha(1+x)^\beta$.

The Jacobi polynomials are defined by

$$P_n^{(\alpha,\beta)}(x) = \frac{\Gamma(\alpha+n+1)}{n!\Gamma(\alpha+\beta+n+1)} \sum_{m=0}^{n} \binom{n}{m} \frac{\Gamma(\alpha+\beta+n+m+1)}{\Gamma(\alpha+m+1)} \left(\frac{x-1}{2}\right)^m$$

The general formula to compute the first n Jacobi polynomials are given by

$$P_n^{(\alpha,\beta)}(x) = \sum_{k=0}^{n} \binom{n+\alpha}{n-k} \binom{n+k+\alpha+\beta}{k} \left(\frac{x-1}{2}\right)^k$$

[SI*] **Hermite Polynomials**
The Hermite polynomials are defined by

$$H_n(x) = (-1)^n e^{x^2} \frac{d^n}{dx^n} e^{-x^2} = e^{\frac{x^2}{2}} \left(x - \frac{d}{dx}\right)^n e^{-\frac{x^2}{2}}$$

They satisfy the orthogonality condition

$$\int_{-\infty}^{\infty} H_n(x) H_m(x) e^{-x^2}\, dx = \sqrt{\pi} 2^n n! \delta_{mn}$$

and the differential equation

$$y''02xy' + 2ny = 0$$

[SI*] **Laguerre Polynomials**
The generalised Laguerre polynomials are defined by

$$L_n^{(\alpha)}(x) = \frac{x^{-\alpha} e^x}{n!} \frac{d^n}{dx^n} (e^{-x} x^{n+\alpha})$$

(the classical Laguerre polynomials correspond to $\alpha = 0$).
They satisfy the orthogonality relation

$$\int_0^{\infty} x^{\alpha} e^{-x} L_n^{(\alpha)}(x) L_m^{(\alpha)}(x)\, dx = \frac{\Gamma(n+\alpha+1)}{n!} \delta_{n,m}$$

and the differential equation

$$xy'' + (\alpha + 1 - x)y' + ny = 0$$

[SI*] **Differential Equation**
The classical orthogonal polynomials arise from a differential equation of the form

$$Q(x)y'' + L(x)y' + \lambda y = 0 \tag{3.346}$$

where Q is a given quadratic (at most) polynomial, and L is a given linear polynomial. The function y, and the constant λ, are to be found.
This is a Sturm-Liouville type of equation. Such equations generally have singularities in their solution functions y except for particular values of λ. They can be thought of as eigenvector/eigenvalue problems: Letting D be the differential operator,

$$D(y) = Qy'' + Ly'$$

and changing the sign of λ, the problem is to find the eigenvectors (eigenfunctions) y, and the corresponding eigenvalues λ, such that y does not have singularities and $D(y) = \lambda y$.

The solutions of this differential equation have singularities unless λ takes on specific values. There is a series of numbers $\lambda_0, \lambda_1, \lambda_2, \ldots$ that led to a series of polynomial solutions $P_0, P_1, P_2, \ldots$ if one of the following sets of conditions are met:

1. Q is actually quadratic, L is linear, Q has two distinct real roots, the root of L lies strictly between the roots of Q and the leading terms of Q and L have the same sign.
2. Q is not actually quadratic, but is linear, L is linear, the roots of Q and L are different, and the leading terms of Q and L have the same sign if the root of L is less than the root of Q, or vice versa.
3. Q is just a nonzero constant, L is linear, and the leading term of L has the opposite sign of Q.

These three cases lead to the Jacobi-like, Laguerre-like, and Hermite-like polynomials, respectively.
In each of these three cases, we have the following:

- The solutions are a series of polynomials $P_0, P_1, P_2, \ldots$, each P_n having degree n, and corresponding to a number λ_n.
- The interval of orthogonality is bounded by whatever roots Q has.
- The root of L is inside the interval of orthogonality.
- Letting $R(x) = e^{\int \frac{L(x)}{Q(x)} dx}$, the polynomials are orthogonal under the weight function $W(x) = \frac{R(x)}{Q(x)}$.
- $W(x)$ has no zeros or infinities inside the interval, though it may have zeros or infinities at the end points.
- $W(x)$ gives a finite inner product to any polynomials.
- $W(x)$ can be made to be greater than 0 in the interval (Negate the entire differential equation if necessary so that $Q(x) > 0$ inside the interval).

Because of the constant of integration, the quantity $R(x)$ is determined only up to an arbitrary positive multiplicative constant. It will be used only in homogeneous differential equations (where this doesn't matter) and in the definition of the weight function (which can also be indeterminate).

[SI*] **Rodrigues' Formula**
Under the assumptions of the preceding section, $P_n(x)$ is proportional to $\frac{1}{W(x)} \frac{d^n}{dx^n} \left(W(x)[Q(x)]^n\right)$. This is known as Rodrigues's formula, after Olinde Rodrigues. It is often written as

$$P_n(x) = \frac{1}{e_n W(x)} \frac{d^n}{dx^n} \left(W(x)[Q(x)]^n\right) \tag{3.347}$$

where the numbers e_n depend on the standardization.

[SI*] **The Numbers λ_n**
Under the assumptions of the preceding section, we have

$$\lambda_n = -n \left(\frac{n-1}{2} Q'' + L'\right) \tag{3.348}$$

Since Q is quadratic and L is linear, Q'' and L' are constants, so these are just numbers.

[SI*] **Second Form for the Differential Equation**
Let

$$R(x) = e^{\int \frac{L(x)}{Q(x)} dx}$$

Then

$$(Ry')' = Ry'' + R'y' = Ry'' + \frac{RL}{Q} y'$$

Now multiply the differential equation

$$Qy'' + Ly' + \lambda y = 0$$

by $\frac{R}{Q}$, getting

$$Ry'' + \frac{RL}{Q}y' + \frac{R\lambda}{Q}y = 0$$
$$(Ry')' + \frac{R\lambda}{Q}y = 0$$

This is the standard Sturm-Liouville form for the equation.

[SI*] **Third Form for the Differential Equation**

Let

$$S(x) = \sqrt{R(x)} = e^{\int \frac{L(x)}{2Q(x)}dx}$$

Then

$$S' = \frac{SL}{2Q}$$

Now multiply the differential equation

$$Qy'' + Ly' + \lambda y = 0$$

by $\frac{S}{Q}$, thus

$$Sy'' + \frac{SL}{Q}y' + \frac{S\lambda}{Q}y = 0$$
$$Sy'' + 2S'y' + \frac{S\lambda}{Q}y = 0$$

since $(Sy)'' = Sy'' + 2S'y' + S''y$, so

$$(Sy)'' + \left(\frac{S\lambda}{Q} - S''\right)y = 0$$

or, letting $u = Sy$,

$$u'' + \left(\frac{\lambda}{Q} - \frac{S''}{S}\right)u = 0$$

[SI*] **Formulas Involving Derivatives**

Under the assumptions of the preceding section, let $P_n^{(r)}$ denote the r-th derivative of P_n. $P_n^{(r)}$ is a polynomial of degree $n - r$. Then we have the following:

- **Orthogonality:** For fixed r, the polynomial sequence $P_r^{(r)}, P_{r+1}^{(r)}, P_{r+2}^{(r)}, \ldots$ are orthogonal, weighted by WQ^r.
- **Generalized Rodrigues' formula:** $P_n^{(r)}$ is proportional to

$$\frac{1}{W(x)[Q(x)]^r}\frac{d^{n-r}}{dx^{n-r}}(W(x)[Q(x)]^n)$$

- **Differential equation:** $P_n^{(r)}$ is a solution of $Qy'' + (rQ' + L)y' + [\lambda_n - \lambda_r]y = 0$, where λ_r is the same function as λ_n, that is,

$$\lambda_r = -r\left(\frac{r-1}{2}Q'' + L'\right)$$

- **Differential equation, second form:** $P_n^{(r)}$ is a solution of $(RQ^r y')' + [\lambda_n - \lambda_r]RQ^{r-1}y = 0$.

There are also some mixed recurrences. In each of these, the numbers a, b, and c depend on n and r, and are unrelated in the various formulas.

- $P_n^{(r)} = aP_{n+1}^{(r+1)} + bP_n^{(r+1)} + cP_{n-1}^{(r+1)}$
- $P_n^{(r)} = (ax+b)P_n^{(r+1)} + cP_{n-1}^{(r+1)}$
- $QP_n^{(r+1)} = (ax+b)P_n^{(r)} + cP_{n-1}^{(r)}$

There are an enormous number of other formulas involving orthogonal polynomials in various ways. Here is a tiny sample of them, relating to the Chebyshev, associated Laguerre, and Hermite polynomials:

- $2T_m(x)T_n(x) = T_{m+n}(x) + T_{m-n}(x)$
- $H_{2n}(x) = (-4)^n n! L_n^{-\frac{1}{2}}(x^2)$
- $H_{2n+1}(x) = 2(-4)^n n! x L_n^{\frac{1}{2}}(x^2)$

[SI*] **Orthogonality**

The differential equation for a particular λ may be written (omitting explicit dependence on x)

$$Q\ddot{y}_n + L\dot{y}_n + \lambda_n y_n = 0$$

multiplying by $\left(\frac{R}{Q}\right) y_m$ yields

$$Ry_m\ddot{y}_n + \frac{R}{Q}Ly_m\dot{y}_n + \frac{R}{Q}\lambda_n y_m y_n = 0$$

and reversing the subscripts yields

$$Ry_n\ddot{y}_m + \frac{R}{Q}Ly_n\dot{y}_m + \frac{R}{Q}\lambda_m y_n y_m = 0$$

subtracting and integrating:

$$\int_a^b \left[R(y_m\ddot{y}_n - y_n\ddot{y}_m) + \frac{R}{Q}L(y_m\dot{y}_n - y_n\dot{y}_m)\right] dx + (\lambda_n - \lambda_m)\int_a^b \frac{R}{Q}y_m y_n \, dx = 0$$

it can be seen that

$$\frac{d}{dx}\left[R(y_m\dot{y}_n - y_n\dot{y}_m)\right] = R(y_m\ddot{y}_n - y_n\ddot{y}_m) + R\frac{L}{Q}(y_m\dot{y}_n - y_n\dot{y}_m)$$

so that

$$[R(y_m\dot{y}_n - y_n\dot{y}_m)]_a^b + (\lambda_n - \lambda_m)\int_a^b \frac{R}{Q}y_m y_n \, dx = 0$$

If the polynomials y are such that the term on the left is zero, and $\lambda_m \neq \lambda_n$ for $m \neq n$, then the orthogonality relationship will hold:

$$\int_a^b \frac{R}{Q}y_m y_n \, dx = 0$$

for $m \neq n$.

[SI*] **Derivation from Differential Equation**

All of the polynomial sequences arising from the differential equation above are equivalent, under scaling and/or shifting of the domain, and standardizing of the polynomials, to more restricted classes. Those restricted classes are exactly "classical orthogonal polynomials."

- Every Jacobi-like polynomial sequence can have its domain shifted and/or scaled so that its interval of orthogonality is $[-1,1]$, and has $Q = 1 - x^2$. They can then be standardized into the Jacobi polynomials $P_n^{(\alpha,\beta)}$. There are several important subclasses of these: Gegenbauer, Legendre, and two types of Chebyshev.
- Every Laguerre-like polynomial sequence can have its domain shifted, scaled, and/or reflected so that its interval of orthogonality is $[0,\infty)$, and has $Q = x$. They can then be standardized into the Associated Laguerre polynomials $L_n^{(\alpha)}$. The plain Laguerre polynomials L_n are a subclass of these.
- Every Hermite-like polynomial sequence can have its domain shifted and/or scaled so that its interval of orthogonality is $(-\infty,\infty)$, and has $Q = 1$ and $L(0) = 0$. They can then be standardized into the Hermite polynomials H_n.

Because all polynomial sequences arising from a different equation in the manner described above are trivially equivalent to the classical polynomials, the actual classical polynomials are always used.

1. **Jacobi Polynomials**

   Jacobi polynomials are a standard class of orthogonal polynomials denoted as $P_n^{(\alpha,\beta)}(x)$ that are defined on the interval $[-1,1]$ with respect to the weight function $(1-x)^\alpha(1+x)^\beta$. They generalize other important polynomials and appear widely in differential equations and physical sciences (the Gegenbauer polynomials, and thus also the Legendre, Zernike and Chebyshev polynomials are special cases of the Jacobi polynomials). The Jacobi polynomials were introduced by Carl Gustav Jacob Jacobi.

   The variety of characterizations of the Jacobi polynomials:

   - The general formula to compute the first n Jacobi polynomials are given by

   $$P_n^{(\alpha,\beta)}(x) = \sum_{k=0}^{n} \binom{n+\alpha}{n-k} \binom{n+k+\alpha+\beta}{k} \left(\frac{x-1}{2}\right)^k$$

   - For $\alpha, \beta > -1$ (this puts the root of L inside the interval of orthogonality) the Jacobi polynomials are given by the Rodrigues' formula

   $$P_n^{(\alpha,\beta)}(x) = \frac{(-1)^n}{2^n n!}(1-x)^{-\alpha}(1+x)^{-\beta}\frac{d^n}{dx^n}\{(1-x)^\alpha(1+x)^\beta(1-x^2)^n\}$$

   If $\alpha = \beta = 0$, then it reduces to the Legendre polynomials:

   $$P_n(x) = \frac{1}{2^n n!}\frac{d^n}{dx^n}(x^2-1)^n$$

   - They are normalised (standardized) by

   $$P_n^{(\alpha,\beta)}(1) = \binom{n+\alpha}{n}$$

and satisfy the orthogonality condition

$$\int_{-1}^{1}(1-x)^{\alpha}(1+x)^{\beta}P_m^{(\alpha,\beta)}(x)P_n^{(\alpha,\beta)}(x)\,dx = \frac{2^{\alpha+\beta+1}}{2n+\alpha+\beta+1}\frac{\Gamma(n+\alpha+1)\Gamma(n+\beta+1)}{\Gamma(n+\alpha+\beta+1)n!}\delta_{nm}$$

with $\alpha, \beta > -1$. As defined, they do not have unit norm with respect to the weight. This can be corrected by dividing by the square root of the right hand side of the equation above, when $n = m$.
Although it does not yield an orthonormal basis, an alternative normalization is sometimes preferred due to its simplicity:

$$P_n^{(\alpha,\beta)}(1) = \binom{n+\alpha}{n}$$

- The polynomials have the symmetry relation

$$P_n^{(\alpha,\beta)}(-x) = (-1)^n P_n^{(\beta,\alpha)}(x)$$

thus the other terminal value is

$$P_n^{(\alpha,\beta)}(-1) = (-1)^n\binom{n+\beta}{n}$$

- The kth derivative of the explicit expression leads to

$$\frac{d^k}{dx^k}P_n^{(\alpha,\beta)}(x) = \frac{\Gamma(\alpha+\beta+n+1+k)}{2^k\Gamma(\alpha+\beta+n+1)}P_{n-k}^{(\alpha+k,\beta+k)}(x)$$

- The generating function of the Jacobi polynomials is given by

$$\sum_{n=0}^{\infty}P_n^{(\alpha,\beta)}(x)t^n = 2^{(\alpha+\beta)}R^{-1}(1-t+R)^{-\alpha}(1+t+R)^{-\beta}$$

where

$$R = R(x,t) = (1-2xt+t^2)^{\frac{1}{2}}$$

and the branch of square root is chosen so that $R(x,0) = 1$.
- The Jacobi polynomials reduce to other classical polynomials.
Ultraspherical

$$C_n^{(\lambda)}(x) = \frac{(2\lambda)_n}{\left(\lambda+\frac{1}{2}\right)_n}P_n^{\left(\lambda-\frac{1}{2},\lambda-\frac{1}{2}\right)}(x)$$
$$P_n^{(\alpha,\alpha)}(x) = \frac{(\alpha+1)_n}{(2\alpha+1)_n}C_n^{\left(\alpha+\frac{1}{2}\right)}(x)$$

Legendre

$$P_n(x) = C_n^{\left(\frac{1}{2}\right)}(x) = P_n^{(0,0)}(x)$$

Chebyshev

$$T_n(x) = \frac{P_n^{\left(-\frac{1}{2},-\frac{1}{2}\right)}(x)}{P_n^{\left(-\frac{1}{2},-\frac{1}{2}\right)}(1)}$$
$$U_n(x) = C_n^{(1)}(x) = (n+1)\frac{P_n^{\left(\frac{1}{2},\frac{1}{2}\right)}(x)}{P_n^{\left(\frac{1}{2},\frac{1}{2}\right)}(1)}$$
$$V_n(x) = \frac{P_n^{\left(-\frac{1}{2},\frac{1}{2}\right)}(x)}{P_n^{\left(-\frac{1}{2},\frac{1}{2}\right)}(1)}$$
$$W_n(x) = (2n+1)\frac{P_n^{\left(\frac{1}{2},-\frac{1}{2}\right)}(x)}{P_n^{\left(\frac{1}{2},-\frac{1}{2}\right)}(1)}$$
$$T_n^*(x) = T_n(2x-1)$$
$$U_n^*(x) = U_n(2x-1)$$

Laguerre

$$\lim_{\beta\to\infty} P_n^{(\alpha,\beta)}\left(1-\left(\frac{2x}{\beta}\right)\right) = L_n^{(\alpha)}(x)$$
$$\lim_{\alpha\to\infty} P_n^{(\alpha,\beta)}\left(\left(\frac{2x}{\alpha}\right)-1\right) = (-1)^n L_n^{(\beta)}(x)$$

Hermit

$$\lim_{\alpha\to\infty} \alpha^{-\frac{1}{2}n} P_n^{(\alpha,\alpha)}\left(\alpha^{-\frac{1}{2}}x\right) = \frac{H_n(x)}{2^n n!} L_n^{(\beta)}(x)$$

- The Jacobi polynomials appear as the eigenfunctions of the Markov process on $[-1,+1]$

  $$\mathcal{L} = (1-x^2)\frac{\partial^2}{\partial x^2} + (px-q)\frac{\partial}{\partial x}$$

  defined up to the time it hits the boundary. For $p = -(\beta+\alpha+2),\ q = \beta-\alpha$, we have

  $$\mathcal{L}P_n^{(\alpha,\beta)} = -n(n+\alpha+\beta+1)P_n^{(\alpha,\beta)}$$

  Thus this process is named the Jacobi process.
- The Jacobi polynomials are solutions to the singular Sturm-Liouville eigenvalue problem

  $$(1-x^2)y'' + (\beta-\alpha-(\alpha+\beta+2)x)y' + n(n+\alpha+\beta+1)y = 0$$

- The Jacobi polynomials can also be expressed as a finite terminating Gauss hypergeometric series ${}_2F_1\left(-n, n+\alpha+\beta+1; \alpha+1; \frac{1-x}{2}\right)$

  $$P_n^{(\alpha,\beta)}(x) = \frac{(\alpha+1)_n}{n!}{}_2F_1\left(-n, n+\alpha+\beta+1; \alpha+1; \frac{1-x}{2}\right)$$

where $(\alpha+1)_n$ is Pochhammer's symbol (for the rising factorial). In this case, the series for the hypergeometric function is finite, therefore one obtains the following equivalent expression:

$$P_n^{(\alpha,\beta)}(x) = \frac{\Gamma(\alpha+n+1)}{n!\Gamma(\alpha+\beta+n+1)} \sum_{m=0}^{n} \binom{n}{m} \frac{\Gamma(\alpha+\beta+n+m+1)}{\Gamma(\alpha+m+1)} \left(\frac{x-1}{2}\right)^m$$

- Special cases

$$P_0^{(\alpha,\beta)}(x) = 1$$
$$P_1^{(\alpha,\beta)}(x) = (\alpha+1)(\alpha+\beta+2)\frac{x-1}{2}$$
$$P_2^{(\alpha,\beta)}(x) = \frac{(\alpha+1)(\alpha+2)}{2} + (\alpha+2)(\alpha+\beta+3)\frac{x-1}{2} + \frac{(\alpha+\beta+3)(\alpha+\beta+4)}{2}\left(\frac{x-1}{2}\right)^2$$
$$\vdots$$
$$P_n^{(\alpha,\beta)}(x) = \frac{\Gamma(1+2n+\alpha+\beta)}{\Gamma(1+n)\Gamma(1+n+\alpha+\beta)}\left(\frac{x}{2}\right)^n + \text{lower-degree terms}$$

Thus, the leading coefficient is

$$\frac{\Gamma(1+2n+\alpha+\beta)}{2^n n!\Gamma(1+n+\alpha+\beta)}$$

Special Cases
The Jacobi polynomials with $\alpha = \beta$ are called the Gegenbauer polynomials (with parameter $\gamma = \alpha + \frac{1}{2}$).

For $\alpha = \beta = 0$, these are called the Legendre polynomials (for which the interval of orthogonality is $[-1,1]$ and the weight function is simply 1):

$$P_0(x) = 1$$
$$P_1(x) = x$$
$$P_2(x) = \frac{3x^2-1}{2}$$
$$P_3(x) = \frac{5x^3-3x}{2}$$

For $\alpha = \beta = -\frac{1}{2}$, one obtains the Chebyshev polynomials of the first, and for $\alpha = \beta = \frac{1}{2}$, one obtains the Chebyshev polynomials of the second kind.

The Jacobi-like polynomials, once they have had their domain shifted and scaled so that the interval of orthogonality is $[-1,1]$, still have two parameters to be determined. They are α and β in the Jacobi polynomials, written $P_n^{(\alpha,\beta)}$. We have $Q(x) = 1-x^2$ and $L(x) = \beta - \alpha - (\alpha+\beta+2)x$. Both α and β are required to be greater than -1 (This puts the root of L inside the interval of orthogonality).

When α and β are not equal, these polynomials are not symmetrical about $x = 0$.

The Jacobi polynomial $P_n^{(\alpha,\beta)}$ is, up to scaling, the unique polynomial solution of the Sturm-Liouville problem:

$$(1-x^2)y'' + (\beta - \alpha - [\alpha+\beta+2]x)y' + \lambda y = 0, \quad \text{with } \lambda = n(n+1+\alpha+\beta)$$

The other solution involves the logarithm function. Bochner's theorem states that the Jacobi polynomials are uniquely characterized as polynomial solutions to Sturm-Liouville problems with polynomial coefficients.

2. **Gegenbauer Polynomials**

In mathematics, Gegenbauer polynomials $C_n^{(\alpha)}$ are orthogonal polynomials on the interval $[-1,1]$ with respect to the weight function $(1-x^2)^{\alpha-\frac{1}{2}}$. They are named after Leopold Gegenbauer. When one sets parameters α and β in the Jacobi polynomials equal to each other, one obtains the Gegenbauer or ultraspherical polynomials. They are written $C_n^{(\alpha)}$, and defined as

$$C_n^{(\alpha)}(x) = \frac{\Gamma(2\alpha+n)\Gamma\left(\alpha+\frac{1}{2}\right)}{\Gamma(2\alpha)\Gamma\left(\alpha+n+\frac{1}{2}\right)} P_n^{\left(\alpha-\frac{1}{2},\alpha-\frac{1}{2}\right)}(x)$$

We have $Q(x) = 1-x^2$ and $L(x) = -(2\alpha+1)x$. The parameter α is required to be greater than $-\frac{1}{2}$. The accepted standardization sets $C_n^{(0)}(1) = \frac{2}{n}$.
The parameter α is closely related to the derivatives of $C_n^{(\alpha)}$:

$$C_n^{(\alpha+1)}(x) = \frac{1}{2\alpha}\frac{d}{dx}C_{n+1}^{(\alpha)}(x)$$

or, more generally:

$$C_n^{(\alpha+m)}(x) = \frac{\Gamma(\alpha)}{2^m\Gamma(\alpha+m)}C_{n+m}^{(\alpha)[m]}(x)$$

All the other classical Jacobi-like polynomials (Legendre, etc) are special cases of the Gegenbauer polynomials, obtained by choosing a value of α and choosing a standardization.

The variety of characterizations of the Gegenbauer polynomials:

- The polynomials can be defined in terms of their generating function:

$$\frac{1}{(1-2xt+t^2)^\alpha} = \sum_{n=0}^{\infty} C_n^{(\alpha)}(x)t^n, \quad 0 \le |x| < 1,\ |t| \le 1,\ \alpha > 0$$

- The recurrence relation for Gegenbauer polynomials is

$$\begin{aligned} C_0^{(\alpha)}(x) &= 1 \\ C_1^{(\alpha)}(x) &= 2\alpha x \\ C_{n+1}^{(\alpha)}(x) &= \frac{1}{n+1}\left[2x(n+\alpha)C_n^{(\alpha)}(x) - (n+2\alpha-1)C_{n-1}^{(\alpha)}(x)\right] \end{aligned}$$

with $n = 1,2,3,\ldots$, they are orthogonal polynomials depending on a parameter α.

- Gegenbauer polynomials are particular solutions of the Gegenbauer differential equation:
$$(1-x^2)y'' - (2\alpha+1)xy' + n(n+2\alpha)y = 0$$
when $\alpha = \frac{1}{2}$, the equation reduces to the Legendre equation, and the Gegenbauer polynomials reduce to the Legendre polynomials. When $\alpha = 1$, the equation reduces to the Chebyshev differential equation, and the Gegenbauer polynomials reduce to the Chebyshev polynomials of the second kind.
- They are given as Gaussian hypergeometric series in certain cases where the series is in fact finite:
$$C_n^{(\alpha)}(z) = \frac{(2\alpha)_n}{n!} {}_2F_1\left(-n, 2\alpha+n; \alpha+\frac{1}{2}; \frac{1-z}{2}\right)$$
Here $(2\alpha)_n$ is the rising factorial. Explicitly,
$$C_n^{(\alpha)}(z) = \sum_{k=0}^{\lfloor \frac{n}{2} \rfloor} (-1)^k \frac{\Gamma(n-k+\alpha)}{\Gamma(\alpha)k!(n-2k)!}(2z)^{n-2k}$$
From this it is also easy to obtain the value at unit argument:
$$C_n^{(\alpha)}(1) = \frac{\Gamma(2\alpha+n)}{\Gamma(2\alpha)n!}$$
- They are special cases of the Jacobi polynomials:
$$C_n^{(\alpha)}(x) = \frac{(2\alpha)_n}{\left(\alpha+\frac{1}{2}\right)_n} P_n^{\left(\alpha-\frac{1}{2}, \alpha-\frac{1}{2}\right)}(x)$$
in which $(\theta)_n$ represents the rising factorial of θ.
One therefore also has the Rodrigues formula
$$C_n^{(\alpha)}(x) = \frac{(-1)^n}{2^n n!} \frac{\Gamma\left(\alpha+\frac{1}{2}\right)\Gamma(n+2\alpha)}{\Gamma(2\alpha)\Gamma\left(\alpha+n+\frac{1}{2}\right)} (1-x^2)^{-\alpha+\frac{1}{2}} \frac{d^n}{dx^n}\left[(1-x^2)^{n+\alpha-\frac{1}{2}}\right]$$
- An altenative normalization sets $C_n^{(\alpha)}(1) = 1$. Assuming this alternative normalization, the derivatives of Gegenbauer are expressed in terms of Gegenbauer:
$$\frac{d^q}{dx^q} C_{q+2j+1}^{(\alpha)}(x) = \frac{2^q(q+2j+1)}{(q-1)!\Gamma(q+2j+2\alpha+1)} \sum_{i=0}^{j} \frac{(2i+\alpha+1)\Gamma(2i+2\alpha+1)}{(2i+1)!(j-i)!} \times \frac{\Gamma(q+j+i+\alpha+1)}{\Gamma(j+i+\alpha+2)} (q+j-i-1)! C_{2i+1}^{(\alpha)}(x)$$

Orthogonality and Normalization

For a fixed $\alpha > -\frac{1}{2}$, the polynomials are orthogonal on $[-1,1]$ with respect to the weighting function
$$w(z) = (1-z^2)^{\alpha-\frac{1}{2}}$$

To wit, for $n \neq m$,

$$\int_{-1}^{1} C_n^{(\alpha)}(x) C_m^{(\alpha)}(x)(1-x^2)^{\alpha-\frac{1}{2}} dx = 0$$

They are normalized by

$$\int_{-1}^{1} \left[C_n^{(\alpha)}(x)\right]^2 (1-x^2)^{\alpha-\frac{1}{2}} dx = \frac{\pi 2^{1-2\alpha}\Gamma(n+2\alpha)}{n!(n+\alpha)[\Gamma(\alpha)]^2}$$

Applications
The Gegenbauer polynomials appear naturally as extensions of Legendre polynomials in the context of potential theory and harmonic analysis. The Newtonian potential in $\mathbb{R}^n$ has the expansion, valid with $\alpha = \frac{n-2}{2}$,

$$\frac{1}{|\boldsymbol{x}-\boldsymbol{y}|^{n-2}} = \sum_{k=0}^{\infty} \frac{|\boldsymbol{x}|}{|\boldsymbol{y}|^{k+n-2}} C_k^{(\alpha)}\left(\frac{\boldsymbol{x}\cdot\boldsymbol{y}}{|\boldsymbol{x}||\boldsymbol{y}|}\right)$$

When $n = 3$, this gives the Legendre polynomial expansion of the gravitational potential. Similar expressions are available for the expansion of the Poisson kenel in a ball.

It follows that the quantities $C_k^{\left(\frac{n-2}{2}\right)}(\boldsymbol{x}\cdot\boldsymbol{y})$ are spherical harmonics, when regarded as a function of $\boldsymbol{x}$ only. They are, in fact, exactly the zonal spherical harmonics, up to a normalizing constant.

Gegenbauer polynomials also appear in the theory of positive-definite functions.

The Askey-Gasper inequality reads

$$\sum_{j=0}^{n} \frac{C_j^{(\alpha)}(x)}{\binom{2\alpha+j-1}{j}} \geq 0, \quad x \geq -1, \alpha \geq \frac{1}{4}$$

In spectral methods for solving differential equations, if a function is expanded in the basis of Chebyshev polynomials and its derivative is represented in a Gegenbauer/ultraspherical basis, then the derivative operator becomes a diagonal matrix, leading to fast banded matrix methods for large problems.

3. **Legendre Polynomials**
The differential equation

$$(1-x^2)y'' - 2xy' + \lambda y = 0, \quad \text{with } \lambda = n(n+1)$$

is Legendre's equation.
The second form of the differential equation is:

$$\frac{d}{dx}[(1-x^2)y'] + \lambda y = 0$$

The recurrence relation is

$$(n+1)P_{n+1}(x) = (2n+1)xP_n(x) - nP_{n-1}(x)$$

A mixed recurrence is

$$P_{n+1}^{[r+1]}(x) = P_{n+1}^{[r+1]}(x) + (2n+1)P_n^{[r]}(x)$$

Rodrigues' formula is

$$P_n(x) = \frac{1}{2^n n!}\frac{d^n}{dx^n}\left([x^2-1]^n\right)$$

Associated Legendre polynomials

The associated Legendre polynomials, denoted by $P_l^{(m)}(x)$ where l and m are integers with $0 \leq m \leq l$, are defined as

$$P_l^{(m)}(x) = (-1)^m(1-x^2)^{\frac{m}{2}}P_l^{[m]}(x)$$

The m in parentheses (to avoid confusion with an exponent) is a parameter. The m in brackets denotes the m-th derivative of the Legendre polynomial.

These "polynomials" are misnamed-they are not polynomials when m is odd.

They have a recurrence relation:

$$(l+1-m)P_{l+1}^{(m)}(x) = (2l+1)xP_l^{(m)}(x) - (l+m)P_{l-1}^{(m)}(x)$$

For fixed m, the sequence $P_m^{(m)}, P_{m+1}^{(m)}, P_{m+2}^{(m)}, \ldots$ are orthogonal over $[-1,1]$, with weight 1.

For given m, $P_l^{(m)}(x)$ are the solutions of

$$(1-x^2)y'' - 2xy' + \left[\lambda - \frac{m^2}{1-x^2}\right]y = 0, \quad \text{with } \lambda = l(l+1)$$

4. **Chebyshev Polynomials**

The Chebyshev polynomials are two sequences of orthogonal polynomials related to cosine and sine functions, notated as $T_n(x)$ and $U_n(x)$. They can be defined in several equivalent ways, one of which starts with trigonometric functions:

- The Chebyshev polynomials of the first kind T_n are defined by

$$T_n(\cos\ \theta) = \cos(n\theta), \quad n = 0,1,2,3,\ldots$$

The recurrence relation is

$$\begin{aligned} T_0(x) &= 1 \\ T_1(x) &= x \\ T_{n+1}(x) &= 2xT_n(x) - T_{n-1}(x) \end{aligned}$$

- The Chebyshev polynomials of the second kind U_n are defined by

$$U_n(\cos\ \theta) = \frac{\sin((n+1)\theta)}{\sin\ \theta}, \quad n = 0,1,2,3,\ldots$$

or

$$U_n = \frac{1}{n+1}T'_{n+1}$$

The recurrence relation is

$$\begin{aligned} U_0(x) &= 1 \\ U_1(x) &= 2x \\ U_{n+1}(x) &= 2xU_n(x) - U_{n-1}(x) \end{aligned}$$

That these expressions define polynomials in $\cos\ \theta$ is not obvious at first sight but can be shown using de Moivre's formula.

The Chebyshev polynomials T_n are polynomials with the largest possible leading coefficient whose absolute value on the interval $[-1, 1]$ is bounded by 1, they are also the "extremal" polynomials for many other properties.

In 1952, Cornelius Lanczos showed that the Chebyshev polynomials are important in approximation theory for the solution of linear systems; the roots of $T_n(x)$, which are also called Chebyshev nodes, are used as matching points for optimizing polynomial interpolation. The resulting interpolation polynomial minimizes the problem of Runge's phenomenon and provides an approximation that is close to the best polynomial approximation to a continuous function under the maximum norm, also called the "minimax" criterion. This approximation leads directly to the method of Clenshaw-Curtis quadrature.

These polynomials were named after Pafnuty Chebyshev. The letter T is used because of the alternative transliterations of the name *Chebyshev* as *Tchebycheff*, *Tchebyshev* (French) or *Tschebyschow* (German).

Trigonometric and Complex Definition
An equivalent way to state the Chebyshev trigonometric definition is via exponentiation of a complex number: given a complex number $z = a + bi$ with absolute value of one,

$$z^n = T_n(a) + ibU_{n-1}(a)$$

That $\cos(nx)$ is an n-th degree polynomial in $\cos(x)$ and can be seen by observing that $\cos(nx)$ is the real part of one side of de Moivre's formula:

$$\cos\ n\theta + i \sin\ n\theta = (\cos\ \theta + i \sin\ \theta)^n$$

The real part of the other side is a polynomial in $\cos\ x$ and $\sin\ x$, in which all powers of $\sin\ x$ are even and thus replaceable through the identity $\cos^2 x + \sin^2 x = 1$. By the same reasoning, $\sin\ nx$ is the imaginary part of the polynomial, in which all powers of $\sin\ x$ are odd and thus, if one factor of $\sin\ x$ is factored out, the remaining factors can be replaced to create a polynomial of degree $n - 1$ in $\cos\ x$.

For x outside the interval $[-1, 1]$, the above definition implies

$$T_n(x) = \begin{cases} \cos(n \arccos\ x), & \text{if } |x| \leq 1 \\ \cosh(n \operatorname{arccosh}\ x), & \text{if } x \geq 1 \\ (-1)^n \cosh(n \operatorname{arccosh}(-x)), & \text{if } x \leq -1 \end{cases}$$

Generating Function

The ordinary generating function for T_n is

$$\sum_{n=0}^{\infty} T_n(x)t^n = \frac{1-tx}{1-2tx+t^2}$$

There are several other generating functions for the Chebyshev polynomials; the exponential generating function is

$$\begin{aligned}\sum_{n=0}^{\infty} T_n(x)\frac{t^n}{n!} &= \frac{1}{2}\left(e^{t(x-\sqrt{x^2-1})} + e^{t(x+\sqrt{x^2-1})}\right) \\ &= e^{tx}\cosh(t\sqrt{x^2-1})\end{aligned}$$

The generating function relevant for 2-dimensional potential theory and multipole expansion is

$$\sum_{n=0}^{\infty} T_n(x)\frac{t^n}{n!} = \ln\left(\frac{1}{\sqrt{1-2tx+t^2}}\right)$$

The ordinary generating function for U_n is

$$\sum_{n=0}^{\infty} U_n(x)t^n = \frac{1}{1-2tx+t^2}$$

and the exponential generating function is

$$\sum_{n=0}^{\infty} U_n(x)\frac{t^n}{n!} = e^{tx}\left(\cosh(t\sqrt{x^2-1}) + \frac{x}{\sqrt{x^2-1}}\sinh(t\sqrt{x^2-1})\right)$$

The differential equation

$$(1-x^2)y'' - xy' + \lambda y = 0, \quad \text{with } \lambda = n^2$$

is Chebyshev's equation.

The first six Chebyshev polynomials of the first kind $T_n(x)$ are

$$\begin{aligned}T_0(x) &= 1 \\ T_1(x) &= x \\ T_2(x) &= 2x^2 - 1 \\ T_3(x) &= 4x^3 - 3x \\ T_4(x) &= 8x^4 - 8x^2 + 1 \\ T_5(x) &= 16x^5 - 20x^3 + 5x\end{aligned}$$

The first six Chebyshev polynomials of the second kind $U_n(x)$ are

$$\begin{aligned}U_0(x) &= 1 \\ U_1(x) &= 2x \\ U_2(x) &= 4x^2 - 1 \\ U_3(x) &= 8x^3 - 4x \\ U_4(x) &= 16x^4 - 12x^2 + 1 \\ U_5(x) &= 32x^5 - 32x^3 + 6x\end{aligned}$$

```
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Differential Equations/Test SymIntegration Chebyshev Polynomials ]# ./main
Chebyshev polynomials of the first kind:
T_{0} (x) = 1
T_{1} (x) = x
T_{2} (x) = 2*x^(2)-1
T_{3} (x) = 4*x^(3)-3*x
T_{4} (x) = 8*x^(4)-8*x^(2)+1
T_{5} (x) = 16*x^(5)-20*x^(3)+5*x
T_{6} (x) = 32*x^(6)-48*x^(4)+18*x^(2)-1
T_{7} (x) = 64*x^(7)-112*x^(5)+56*x^(3)-7*x
T_{8} (x) = 128*x^(8)-256*x^(6)+160*x^(4)-32*x^(2)+1
T_{9} (x) = 256*x^(9)-576*x^(7)+432*x^(5)-120*x^(3)+9*x
T_{10} (x) = 512*x^(10)-1280*x^(8)+1120*x^(6)-400*x^(4)+50*x^(2)-1

Chebyshev polynomials of the second kind:
U_{0} (x) = 1
U_{1} (x) = 2*x
U_{2} (x) = 4*x^(2)-1
U_{3} (x) = 8*x^(3)-4*x
U_{4} (x) = 16*x^(4)-12*x^(2)+1
U_{5} (x) = 32*x^(5)-32*x^(3)+6*x
U_{6} (x) = 64*x^(6)-80*x^(4)+24*x^(2)-1
U_{7} (x) = 128*x^(7)-192*x^(5)+80*x^(3)-8*x
U_{8} (x) = 256*x^(8)-448*x^(6)+240*x^(4)-40*x^(2)+1
U_{9} (x) = 512*x^(9)-1024*x^(7)+672*x^(5)-160*x^(3)+10*x
U_{10} (x) = 1024*x^(10)-2304*x^(8)+1792*x^(6)-560*x^(4)+60*x^(2)-1

Time taken by function: 174549 microseconds
```

Figure 3.92: *The first eleven Chebyshev polynomials of the first kind and the second kind (**SymIntegration/Examples/Differential Equations/Test SymIntegration Jacobi Polynomials/main.cpp**).*

Rodrigues' formula is

$$T_n(x) = \frac{\Gamma\left(\frac{1}{2}\right)\sqrt{1-x^2}}{(-2)^n\Gamma\left(n+\frac{1}{2}\right)}\frac{d^n}{dx^n}\left([1-x^2]^{n-\frac{1}{2}}\right)$$

These polynomials have the property that, in the interval of orthogonality,

$$T_n(x) = \cos(n\arccos(x))$$

This means that all their local minima and maxima have values of -1 and $+1$, that is, the polynomials are "level." Because of this, expansion of functions in terms of Chebyshev polynomials is sometimes used for polynomial approximations in computer math libraries.
Some authors use versions of these polynomials that have been shifted so that the interval of orthogonality is $[0,1]$ or $[-2,2]$.

5. **Laguerre Polynomials**
The most general Laguerre-like polynomials, after the domain has been shifted and scaled, are the Associated Laguerre polynomials (also called generalized Laguerre polynomials), denoted by $L_n^{(\alpha)}$. There is a parameter α, which can be any real number strictly greater than -1. The parameter is put in parentheses to avoid confusion with an exponent. The plain Laguerre polynomials are simply the $\alpha = 0$ version of these:

$$L_n(x) = L_n^{(0)}(x)$$

In mathematics, the Laguerre polynomials, named after Edmond Laguerre (1834-1886), are nontrivial solutions of Laguerre's differential equation:

$$xy'' + (1-x)y' + ny = 0, \quad y = L(x)$$

which is a second-order linear differential equation. This equation has nonsingular solutions only if n is a non-negative integer.

The Laguerre polynomials denoted by $L_n(x)$, and defined as solutions to Laguerre's differential equation can be explicitly computed using the Rodrigues's formula:

$$L_n(x) = \frac{e^x}{n!}\frac{d^n}{dx^n}(x^n e^{-x})$$

Sometimes the name Laguerre polynomials is used for solutions of

$$xy'' + (\alpha + 1 - x)y' + \lambda y = 0, \quad \text{with } \lambda = n$$

where $\lambda = n$ is still a non-negative integer. Then they are also named generalized Laguerre polynomials, or rarely Sonine polynomials.

The second form of the differential equation is

$$(x^{\alpha+1}e^{-x}y')' + \lambda x^\alpha e^{-x} y = 0$$

More generally, a Laguerre function is a solution when n is not necessarily a non-negative integer.

The variety of characterizations of the Laguerre polynomials:

- The Laguerre polynomials are also used for Gauss-Laguerre quadrature to numerically compute integrals of the form

$$\int_0^\infty f(x)e^{-x}\,dx$$

- These polynomials, usually denoted by $L_0, L_1, \ldots$, are a polynomial sequence which can be defined by the Rodrigues' formula,

$$L_n(x) = \frac{e^x}{n!}\frac{d^n}{dx^n}(e^{-x}x^n) = \frac{1}{n!}\left(\frac{d}{dx} - 1\right)^n x^n$$

or with $\alpha \neq 0$

$$L_n^{(\alpha)}(x) = \frac{x^{-\alpha}e^x}{n!}\frac{d^n}{dx^n}(e^{-x}x^{n+\alpha}) = \frac{x^{-\alpha}}{n!}\left(\frac{d}{dx} - 1\right)^n x^{n+\alpha}$$

The parameter α is closely related to the derivatives of $L_n^{(\alpha)}$:

$$L_n^{(\alpha+1)}(x) = -\frac{d}{dx}L_{n+1}^{(\alpha)}(x)$$

or, more generally:

$$L_n^{(\alpha+m)}(x) = (-1)^m L_{n+m}^{(\alpha)[m]}(x)$$

The generating function for them with $\alpha \neq 0$ is

$$\sum_{n=0}^{\infty} t^n L_n^{(\alpha)}(x) = \frac{1}{(1-t)^{\alpha+1}}e^{-\frac{tx}{1-t}}$$

The generating function for them with $\alpha = 0$ is

$$\sum_{n=0}^{\infty} t^n L_n(x) = \frac{1}{1-t}e^{-\frac{tx}{1-t}}$$

- They are orthogonal polynomials on the interval $[0, \infty)$ with respect to an inner product and the weighting function e^{-x}:

$$< f, g >= \int_0^\infty f(x)g(x)e^{-x}\, dx$$

or

$$\int_0^\infty e^{-x} L_m(x) L_n(x)\, dx = \delta_{mn}$$

The rook polynomials in combinatorics are more or less the same as Laguerre polynomials, up to elementary changes of variables.

- The recurrence relation for the standard Laguerre polynomials without α ($\alpha = 0$):

$$\begin{aligned}
L_0(x) &= 1 \\
L_1(x) &= 1 - x \\
L_2(x) &= \frac{1}{2}(x^2 - 4x + 2) \\
L_3(x) &= \frac{1}{6}(-x^3 + 9x^2 - 18x + 6) \\
L_4(x) &= \frac{1}{24}(x^4 - 16x^3 + 72x^2 - 96x + 24) \\
L_5(x) &= \frac{1}{120}(-x^5 + 25x^4 - 200x^3 + 600x^2 - 600x + 120) \\
&\vdots \\
L_{k+1}(x) &= \frac{(2k + 1 - x)L_k(x) - kL_{k-1}(x)}{k+1}, \quad k \geq 1
\end{aligned}$$

The generalized Laguerre polynomials with α ($\alpha \neq 0$) are given by:

$$\begin{aligned}
L_0^{(\alpha)}(x) &= 1 \\
L_1^{(\alpha)}(x) &= 1 - x + \alpha \\
L_2^{(\alpha)}(x) &= \frac{1}{2}x^2 - (\alpha + 2)x + \frac{1}{2}(\alpha + 1)(\alpha + 2) \\
L_3^{(\alpha)}(x) &= -\frac{1}{6}x^3 + \frac{1}{2}(\alpha + 3)x^2 - \frac{1}{2}(\alpha + 2)(\alpha + 3)x + \frac{1}{6}(\alpha + 1)(\alpha + 2)(\alpha + 3) \\
&\vdots \\
L_{k+1}^{(\alpha)}(x) &= \frac{(2k + 1 + \alpha - x)L_k^{(\alpha)}(x) - (k + \alpha)L_{k-1}^{(\alpha)}(x)}{k+1}, \quad k \geq 1
\end{aligned}$$

The coefficient of the leading term is $\frac{(-1)^n}{n!}$, the constant term, which the value at 0, is

$$L_n^{(\alpha)}(0) = \binom{n + \alpha}{n} = \frac{\Gamma(n + \alpha + 1)}{n!\Gamma(\alpha + 1)}$$

The discriminant is

$$\text{Disc}\left(L_n^{(\alpha)}\right) = \prod_{j=1}^{n} j^{j-2n+2}(j + \alpha)^{j-1}$$

```
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Differential Equations/Test SymIntegration Laguerre Polynomials ]# ./main
Laguerre polynomials with α = 0 :
L_{0} (x) = 1
L_{1} (x) = -x+1
L_{2} (x) = 1/2*x^(2)-2*x+1
L_{3} (x) = -1/6*x^(3)+1.5*x^(2)-3*x+1
L_{4} (x) = 1/24*x^(4)-0.666667*x^(3)+3*x^(2)-4*x+1
L_{5} (x) = -1/120*x^(5)+0.208333*x^(4)-1.66667*x^(3)+5*x^(2)-5*x+1
L_{6} (x) = 1/720*x^(6)-0.05*x^(5)+0.625*x^(4)-3.33333*x^(3)+7.5*x^(2)-6*x+1
L_{7} (x) = -1/5040*x^(7)+0.00972222*x^(6)-0.175*x^(5)+1.45833*x^(4)-5.83333*x^(3)+10.5*x^(2)-7*x+1
L_{8} (x) = 1/40320*x^(8)-0.0015873*x^(7)+0.0388889*x^(6)-0.466667*x^(5)+2.91667*x^(4)-9.33333*x^(3)+14*x^(2)-8*x+1
L_{9} (x) = -1/362880*x^(9)+0.000223214*x^(8)-0.00714286*x^(7)+0.116667*x^(6)-1.05*x^(5)+5.25*x^(4)-14*x^(3)+18*x^(2)-9*x+1
L_{10} (x) = 1/3628800*x^(10)-2.75573e-05*x^(9)+0.00111607*x^(8)-0.0238095*x^(7)+0.291667*x^(6)-2.1*x^(5)+8.75*x^(4)-20*x^(3)+22.5*x^(2)-10*x+1
Laguerre polynomials with α = 1 :
L_{0} (x) = 1
L_{1} (x) = -x+2
L_{2} (x) = 1/2*x^(2)-3*x+3
L_{3} (x) = -1/6*x^(3)+2*x^(2)-6*x+4
L_{4} (x) = 1/24*x^(4)-0.833333*x^(3)+5*x^(2)-10*x+5
L_{5} (x) = -1/120*x^(5)+0.25*x^(4)-2.5*x^(3)+10*x^(2)-15*x+6
L_{6} (x) = 1/720*x^(6)-0.0583333*x^(5)+0.875*x^(4)-5.83333*x^(3)+17.5*x^(2)-21*x+7
L_{7} (x) = -1/5040*x^(7)+0.0111111*x^(6)-0.233333*x^(5)+2.33333*x^(4)-11.6667*x^(3)+28*x^(2)-28*x+8
L_{8} (x) = 1/40320*x^(8)-0.00178571*x^(7)+0.05*x^(6)-0.7*x^(5)+5.25*x^(4)-21*x^(3)+42*x^(2)-36*x+9
L_{9} (x) = -1/362880*x^(9)+0.000248016*x^(8)-0.00892857*x^(7)+0.166667*x^(6)-1.75*x^(5)+10.5*x^(4)-35*x^(3)+60*x^(2)-45*x+10
L_{10} (x) = 1/3628800*x^(10)-3.03131e-05*x^(9)+0.00136409*x^(8)-0.0327381*x^(7)+0.458333*x^(6)-3.85*x^(5)+19.25*x^(4)-55*x^(3)+82.5*x^(2)-55*x+11

Time taken by function: 977588 microseconds
```

Figure 3.93: *The first eleven Laguerre polynomials with $\alpha = 0$ and $\alpha = 1$(**SymIntegration/Examples/Differential Equations/Test SymIntegration Laguerre Polynomials/main.cpp**).*

Furthermore

$$xL_n'(x) = nL_n(x) - nL_{n-1}(x)$$

- In solution of some boundary value problems, the characteristic values can be useful:

$$L_k(0) = 1, \quad L_k'(0) = -k$$

The closed form is

$$L_n(x) = \sum_{k=0}^{n} \binom{n}{k} \frac{(-1)^k}{k!} x^k$$

- Polynomials of negative index can be expressed using the ones with positive index

$$L_{-n}(x) = e^x L_{n-1}(-x)$$

- Laguerre functions are defined by confluent hypergeometric functions and Kummer's transformation as

$$L_n^{(\alpha)}(x) := \binom{n+\alpha}{n} M(-n, \alpha+1, x) = \frac{(\alpha+1)_n}{n!} {}_1F_1(-n; \alpha+1; x)$$

where $(a)_n$ is the Pochhammer symbol (the rising factorial), $\binom{n+\alpha}{n}$ is a generalized binomial coefficient. When n is an integer the function reduces to a polynomial of degree n. It has the alternative expression

$$L_n^{(\alpha)}(x) = \frac{(-1)^n}{n!} U(-n, \alpha+1, x)$$

in terms of Kummer's function of the second kind.

- The closed form for these generalized Laguerre polynomials of degree n is

$$L_n^{(\alpha)}(x) = \sum_{i=0}^{n} (-1)^i \binom{n+\alpha}{n-i} \frac{x^i}{i!}$$

derived by applying Leibniz's theorem for differentiation of a product to Rodrigues' formula.

- Laguerre polynomials have a differential operator representation, much like the closely related Hermite polynomials. Namely, let $D = \frac{d}{dx}$ and consider the differential operator $M = xD^2 + (\alpha + 1)D$. Then

$$e^{-tM}x^n = (-1)^n t^n n! L_n^{(\alpha)}\left(\frac{x}{t}\right)$$

- Laguerre's equation can be manipulated into a form that is more useful in applications:

$$u = x^{\frac{\alpha-1}{2}} e^{-\frac{x}{2}} L_n^{(\alpha)}(x)$$

which is a solution of

$$u'' + \frac{2}{x}u' + \left[\frac{\lambda}{x} - \frac{1}{4} - \frac{\alpha^2 - 1}{4x^2}\right] u = 0, \quad \text{with } \lambda = n + \frac{\alpha + 1}{2}$$

This can be further manipulated. When $l = \frac{\alpha-1}{2}$ is an integer, and $n \geq l + 1$:

$$u = x^l e^{-\frac{x}{2}} L_{n-l-1}^{(2l+1)}(x)$$

is a solution of

$$u'' + \frac{2}{x}u' + \left[\frac{\lambda}{x} - \frac{1}{4} - \frac{l(l+1)}{x^2}\right] u = 0, \quad \text{with } \lambda = n$$

The solution is often expressed in terms of derivatives instead of assocaited Laguerre polynomials:

$$u = x^l e^{-\frac{x}{2}} L_{n+l}^{[2l+1]}(x)$$

- Given the generating function specified above, the polynomials may be expressed in terms of a contour integral

$$L_n^{(\alpha)}(x) = \frac{1}{2\pi i} \oint_C \frac{e^{-\frac{xt}{1-t}}}{(1-t)^{\alpha+1} t^{n+1}} dt$$

where the contour circles the origin once in a counterclockwise direction without enclosing the essential singularity at 1.
- Differentiating the power series representation of a generalized Lagurre polynomial k times leads to

$$\frac{d^k}{dx^k} L_n^{(\alpha)}(x) = \begin{cases} (-1)^k L_{n-k}^{(\alpha+k)}(x), & \text{if } k \leq n \\ 0, & \text{otherwise} \end{cases}$$

This points to a special case ($\alpha = 0$) of the formula above: for integer $\alpha = k$ the generalized polynomial may be written

$$L_n^{(k)}(x) = (-1)^k \frac{d^k L_{n+k}(x)}{dx^k}$$

The shift by k sometimes causing confusion with the usual parenthesis notation for a derivative.
Moreover, the following equation holds:

$$\frac{1}{k!}\frac{d^k}{dx^k} x^\alpha L_n^{(\alpha)}(x) = \binom{n+\alpha}{k} x^{\alpha-k} L_n^{(\alpha-k)}(x)$$

which generalizes with Cauchy's formula to

$$L_n^{(\alpha')}(x) = (\alpha' - \alpha)\binom{\alpha' + n}{\alpha' - \alpha}\int_0^x \frac{t^\alpha (x-t)^{\alpha'-\alpha-1}}{x^{\alpha'}} L_n^{(\alpha)}(t)\, dt$$

The derivative with respect to the second variable α has the form,

$$\frac{d}{d\alpha} L_n^{(\alpha)}(x) = \sum_{i=0}^{n-1} \frac{L_i^{(\alpha)}(x)}{n-i}$$

The generalized Laguerre polynomials obey the differential equation

$$xL_n^{(\alpha)\prime\prime}(x) + (\alpha + 1 - x)L_n^{(\alpha)\prime}(x) n L_n^{(\alpha)}(x) = 0$$

which may be compared with the equation obeyed by the k-th derivative of the ordinary Laguerre polynomial,

$$xL_n^{[k]\prime\prime}(x) + (k + 1 - x)L_n^{[k]\prime}(x) + (n-k)L_n^{[k]}(x) = 0$$

where $L_n^{[k]}(x) \equiv \frac{d^k L_n(x)}{dx^k}$ for this equation only.
In Sturm-Liouville form, the differential equation is

$$-(x^{\alpha+1}e^{-x} \cdot L_n^{(\alpha)}(x)')' = n \cdot x^\alpha e^{-x} \cdot L_n^{(\alpha)}(x)$$

which shows that $L_n^{(\alpha)}$ is an eigenvector for the eigenvalue n.

- The Laguerre polynomials arise in quantum mechanics, in the radial part of the solution of the Schrödinger equation for a one-electron atom. They also describe the static Wigner functions of oscillator systems in quantum mechanics in phase space. They further enter in the quantum mechanics of the Morse potential and of the 3D isotropic harmonic oscillator.

  Physicists often use a definition for the Laguerre polynomials that is larger, by a factor of $(n!)$, than the definition used here.
- The generalized Laguerre polynomials are used to describe the quantum wavefunction for hydrogen atom orbitals. The convention used expresses the generalized Laguerre polynomials as

$$L_n^{(\alpha)}(x) = \frac{\Gamma(\alpha + n + 1)}{\Gamma(\alpha + 1)n!} {}_1F_1(-n; \alpha + 1; x)$$

where ${}_1F_1(a; b; x)$ is the confluent hypergeometric function. In the physics literature, the generalized Laguerre polynomials are instead defined as

$$\bar{L}_n^{(\alpha)}(x) = \frac{[\Gamma(\alpha + n + 1)]^2}{\Gamma(\alpha + 1)n!} {}_1F_1(-n; \alpha + 1; x)$$

The physics version is related to the standard version by

$$\bar{L}_n^{(\alpha)}(x) = (n + \alpha)! L_n^{(\alpha)}(x)$$

There is yet another, albeit less frequently used, convention in the physics literature

$$\tilde{L}_n^{(\alpha)}(x) = (-1)^\alpha \bar{L}_{n-\alpha}^{(\alpha)}$$

6. **Hermite Polynomials**
The differential equation

$$y'' - 2xy' + \lambda y = 0, \quad \text{with } \lambda = 2n$$

is Hermite's equation.

The second form of the differential equation is

$$(e^{-x^2}y')' + e^{-x^2}\lambda y = 0$$

The third form is

$$(e^{-\frac{x^2}{2}}y')'' + (\lambda + 1 - x^2)(e^{-\frac{x^2}{2}}y) = 0$$

The recurrence relation is

$$H_{n+1}(x) = 2xH_n(x) - 2nH_{n-1}(x)$$

this is the physicist's Hermite polynomial.

Rodrigues' formula is

$$H_n(x) = (-1)^n e^{x^2} \frac{d^n}{dx^n}\left(e^{-x^2}\right)$$

One can define the associated Hermite functions

$$\psi_n(x) = (h_n)^{-\frac{1}{2}} e^{-\frac{x^2}{2}} H_n(x)$$

Because the multiplier is proportional to the square root of the weight function, these functions are orthogonal over $(-\infty, \infty)$ with no weight function.

The third form of the differential equation above, for the associated Hermite functions, is

$$\psi'' + (\lambda + 1 - x^2)\psi = 0$$

The associated Hermite functions arise in many areas of mathematics and physics. In quantum mechanics, they are the solutions of Schrödinger's equation for the harmonic oscillator. They are also eigenfunctions (with eigenvalue $(-i^n)$) of the continuous Fourier transform.

Many authors, particularly probabilists, use an alternate definition of the Hermite polynomials, with a weight functions of $e^{-\frac{x^2}{2}}$ instead of e^{-x^2}. If the notation He is used for these Hermite polynomials, and H for those above, then these may be characterized by

$$He_n(x) = 2^{-\frac{n}{2}} H_n\left(\frac{x}{\sqrt{2}}\right)$$

x. Functions in SymIntegration Related to Series Solutions of Second Order Linear Equations

[SI*] We will list the basic functions and classes that can be used related to series solutions of second order linear differential equations:

secondorderlineardiffeq_derivativesvalueatx0($f(x,y)$**,**y**,**x**,**x_0**,** y_{x_0}**,** y'_{x_0}**)**
to compute the $\phi''(x_0), \phi'''(x_0)$, and $\phi^{(4)}(x_0)$ for the given point x_0 if $y = \phi(x)$ is a solution of an initial value problem for homogeneous second order linear differential equation with this configuration

$$f(x,y) = P(x)y'' + Q(x)y' + R(x)y = 0$$

and the initial conditions

$$y(x_0) = y_{x_0}$$
$$y'(x_0) = y'_{x_0}$$

the differential equation is using variable y as the function of x.

class HigherOrderODE_Homogeneous_PowerSeriesSolver
this class encapsulates the mathematical array elements and generation rules to compute the series solution for homogeneous higher order linear differential equations with constant coefficients

$$b_m y^{(m)} + b_{m-1} y^{(m-1)} + \cdots + b_0 y = 0$$

with initial conditions

$$y(0) = ic_0$$
$$y'(0) = ic_1$$
$$\vdots$$
$$y^{(m-1)}(0) = ic_{m-1}$$

around an ordinary point $x_0 = 0$, with $b_i \in \mathbb{R}, i = 0, 1, 2, \ldots, m$ and $ic_j \in \mathbb{R}, j = 0, 1, 2, \ldots, m-1$.

Inside the **private:** we have defined **vector<double> ode_coefficients** to store the ordinary differential equation coefficients $(b_0, b_1, \ldots, b_m)$ the entries represents the coefficients starting from y to $y^{(m)}$, **vector<double> initial_values** to store the initial conditions value $(ic_0, ic_1, \ldots, ic_{m-1})$, and **vector<double> coefficients** to dynamically stores the c_n (the coefficients of the power series solution).

The functions related to this class are:

- **computeSeries(int** n**)**
  a **void** function to compute the power series solution with n as the terms that we specify for the series solution. This is the core algorithmic function, if you want to change the differential equation type, we only need to modify this specific recurrence formula.
- **coefficientsvector(int** n**)**
  a **vector<double>** function to save the coefficients of the power series solution with n as the terms that we specify for the series solution.

- **evaluateAt(double x_0, int n)**
  a **double** function to compute the approximation of the power series solution at x_0 ($y(x_0)$), the number of terms is n and the highest degree of the power series will be $n-1$.
- **printSeries() const**
  a **void** function to display the series on the command line interface (terminal / xterm) up until the n-th terms that we have specified in the **computeSeries(int n)** function, we need to call this function (**computeSeries(int n)**) first, before we are able to display the series. This function formats output nicely by handling sign changes and omitting zero-value coefficients.

This class uses **vector<double>** to store computed elements sequentially, serving as the look-back pipeline for the relation step. The recurrence loop is using **coefficients[n+order]**, it is isolated directly by shifting index bounds inside a standard **for** loop.

```
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Differential Equations/Test SymIntegration Series Solutions for Homogeneous Second
 Order Linear Differential Equation with Constant Coefficients ]# ./main

( 1 ) y + ( 0 ) y^(1) + ( 1 ) y^(2)  = 0

Initial conditions:
y(0) = 1
y^(1) (0) = 0

Series solution:

y(x) = 1 - 0.5x^2 + 0.0416667x^4 - 0.00138889x^6 + 2.48016e-05x^8 - 2.75573e-07x^10 + ...

x = 0.45

Power Series Approximation at x : 0.900447

Time taken by function: 496 microseconds
```

Figure 3.94: *The power series solution of $y'' + y = 0$ computed with **class HigherOrderODE_Homogeneous_PowerSeriesSolver**, $n = 11$ (highest degree of the solution is at $n-1$) and then compute the approximation of the series solution at $x_0 = 0.45$ (**SymIntegration/Examples/Differential Equations/Test SymIntegration Series Solutions for Homogeneous Second Order Linear Differential Equation with Constant Coefficients/main.cpp**).*

class PolynomialDouble
An elegant way to implement a power series solution for an ordinary differential equation (ODE) in C++ is to create a polynomial class, this class capables of polynomial shifting and we can implement a solver (e.g., in **class SecondOrderODE_Homogeneous_PowerSeriesSolver**) that computes the first n terms of the power series solution given two arbitrary initial conditions $y(0) = y0$, $y'(0) = dy0$. This class uses a vector to store the coefficient of a polynomial, suppose if we declare and create a polynomial like this
PolynomialDouble P($\{1.0, 0.0, 2.0, -3.0\}$**);**
then it will create a polynomial of name **P** with this configuration:

$$P(x) = 1 + 2x^2 - 3x^3$$

this class is very useful in order to compute series solution for second order homogeneous ODE with variable coefficients. This class is written in **/include/symintegral/dsolve.h** only.

The functions related to this class are:

- **shift_around(double x_0) const**
  a **PolynomialDouble** function to create a shifted polynomial around the point x_0, it will return a new polynomial $p(x_{new})$ where $x_{new} = x - x_0$. This implementation applies

a fast synthetic division algorithm (nested loops inside **shift_around**) to dynamically construct Taylor expansions for $P(x), Q(x)$, and $R(x)$ centered at x_0.

- **shiftToCenter(double x_0) const**
  a **PolynomialDouble** function to create a shifted polynomial around the point x_0, it is an alternative besides **shift_around(double x_0)**.
- **shift(double x_0) const**
  a **PolynomialDouble** function to create a shifted polynomial around the point x_0, it is an alternative besides **shift_around(double x_0)**.
- **trim()**
  a **void** function that remove the trailing zeros to maintain accurate degree.
- **get_coeff(size_t m)**
  a **double** function to retrieve the coefficient of the polynomial at degree m, given that the highest order of the polynomial is equal or greater than m.
- **sum_coeff() const**
  a **double** function to retrieve the sum of the coefficients of the polynomial, we can call it in the source code with: **P.sum_coeff()**, given that we have declared a **PolynomialDouble P** with its entries.
- **maxDegree() const**
  an **int** function to return the maximum degree of the polynomial, we can call it in the source code with: **P.maxDegree()**, given that we have declared a **PolynomialDouble P** with its entries.
- **evaluateAt(double x) const**
  a **double** function to evaluate the polynomial at a specific x value using Horner's method.
- **print() const**
  a **void** function to print the polynomial on the command line interface / terminal.

The **PolynomialDouble Class** encapsulates variable coefficient arrays. It uses a secure access function **get_coeff()** to smoothly manage degree requests beyond the polynomial's boundary without crashing or forcing unnecessary manual padding. Thus, it is very useful when it is used in **class SecondOrderODE_Homogeneous_PowerSeriesSolver**.

class PolynomialComplex
This is the complex version of **PolynomialDouble**, this class is created to handle polynomial with complex number coefficients, this class capables of polynomial shifting, polynomial evaluation at $z = a + bi$, print the polynomial and create a **PolynomialComplex** object from **PolynomialDouble** object (with conversion constructor). This class uses a vector to store the coefficient of a polynomial, suppose if we declare and create a polynomial like this
PolynomialComplex P($(\{\{1.0, 0.0\}, \{2.0, 1.0\}, \{3.0, 5.0\}\})$**);**
then it will create a complex polynomial of name **P** with this configuration:

$$P(x) = (1 + 0i) + (2 + i)x + (3 + 5i)x^2$$

this class is very useful in order to compute series solution for second order homogeneous ODE with variable coefficients near a singular point with Frobenius method. This class is written in **/include/symintegral/dsolve.h**.
For the storage order, the coefficients are stored from lowest degree (x^0) to highest degree (x^n). This aligns the vector index directly with the power of x.

The functions related to this class are:

- **shift(complex<double> x_0) const**
  to compute $P(r + x_0)$ given a polynomial $P(r) = \sum a_i r^i$ with binomial expansion. Each term r^i becomes

$$(r + s)^i = \sum_{k=0}^{i} \binom{i}{k} r^k s^{i-k}$$

  The algorithm loops through each existing coefficient a_i, computes its expanded contribution using a localized binomial generation strategy, and allocates it directly to the new index vector without requiring a full polynomial multiplication library.

  When solving differential equations using the Frobenius method, we often need to work with polynomials of the indicial equation (r) and shift their indices (e.g., substituting r with $r + n$). This implementation uses **vector<complex<double>>** to store coefficients, where the index of the vector represents the power of the variable.

  If the algorithm requires us to find the new coefficients of the polynomial centered at x_0 (e.g., expanding $P(x - x_0)$ back into standard form $a_0 + a_1 x + a_2 x^2$), we must use Taylor's Theorem or Synthetic Division (Horner's Scheme) to shift the coefficients.
- **Add(const PolynomialComplex& P_2) const**
  a basic addition operation to add a current highlighted / selected **PolynomialComplex** with another **PolynomialComplex** P_2.
- **PolynomialComplex(const PolynomialDouble& other)**
  a conversion constructor, it creates a complex polynomial from a double polynomial. The method of conversion constructor in a class is generally the better and more idiomatic choice for the specific task of **PolynomialDouble** to **PolynomialComplex** conversion. Another method is with a template-based approach, but the complexity is high, it requires rewriting the entire class as a template, the conversion is also not automatic unless explicitly defined, and can cause code bloat if many types are used.
- **trim()**
  a **void** function that remove the trailing zeros to maintain accurate degree.
- **get_coeff(size_t m)**
  a **complex<double>** function to retrieve the coefficient of the polynomial at degree m, given that the highest order of the polynomial is equal or greater than m. A safe lambda proxy ensures that requesting higher-order index shifts beyond the polynomial's true dimension safely returns 0.0 instead of triggering an out-of-bounds error.
- **sum_coeff() const**
  a **double** function to retrieve the sum of the coefficients of the polynomial, we can call it in the source code with: **P.sum_coeff()**, given that we have declared a **PolynomialDouble P** with its entries.
- **maxDegree() const**
  an **int** function to return the maximum degree of the polynomial, we can call it in the source code with: **P.maxDegree()**, given that we have declared a **PolynomialComplex P** with its entries.
- **evaluateAt(complex<double> z) const**
  a **complex<double>** function to evaluate the polynomial at a specific x value using Horner's method. Instead of calculating powers of z manually (which is slow and error-prone), the evaluationAt loops backward through the coefficients (**coeffs.rbegin()** to **coeffs.rend()**), multiplying the accumulated result by z and adding the next coefficient.
- **print() const**

a **void** function to print the polynomial on the command line interface / terminal.

By using **std::complex<double>**, this class seamlessly handles both real and imaginary inputs, coefficients, and outputs.

class SecondOrderODE_Homogeneous_PowerSeriesSolver
this class is an elegant way to solve a homogeneous second order linear differential equation with variable coefficients

$$P(x)y'' + Q(x)y' + R(x)y = 0$$

by using power series this class encapsulates the mathematical array elements and generation rules to compute the series solution around an ordinary point $x_0 \in \mathbb{R}$, x_0 can be 0 or other real number. It validates that the layout center point (x_0) is an ordinary / regular point ($P(x_0) \neq 0$) before processing. This class will compute the power series solution of a homogeneous second order linear differential equation with polynomial coefficients around an ordinary point x_0, the solution can be written as a Taylor series centered at x_0

$$y(x) = \sum_{n=0}^{\infty} c_n (x - x_0)^n$$

By shifting the coordinate system such that $x_{new} = x - x_0$, we can rewrite the coefficient polynomials $P(x), Q(x)$, and $R(x)$ in terms of $(x - x_0)$ using Taylor expansion. This simplifies the computation to expanding around $x_{new,0} = 0$.

Inside the **private:** we have defined **vector<double> coefficients** to dynamically stores the c_n (the coefficients of the power series solution) the use of **std::vector** allow flexible calculation depths without memory leaks, then we also define **PolynomialDouble** P**,** Q**,** R to represents $P(x), Q(x)$, and $R(x)$ respectively.

This **class** encapsulates the differential equation coefficients that use standard recurrence mechanisms derived by shifting indices of the power series. For an equation $P(x)y'' + Q(x)y' + R(x) = 0$, the algorithm isolates the highest index element c_{n+2} dynamically based on previous computed indices $c_0, c_1, c_2, \ldots, c_{n+1}$.

```
...

#ifndef SECONDORDERODE_HOMOGENEOUS_POWERSERIESSOLVER_H
#define SECONDORDERODE_HOMOGENEOUS_POWERSERIESSOLVER_H
class SecondOrderODE_Homogeneous_PowerSeriesSolver {
      private:
      PolynomialDouble P; // Coefficient for y''
      PolynomialDouble Q; // Coefficient for y'
      PolynomialDouble R; // Coefficient for y
      double x0;
      double y0; // for y(0)
      double dy0; // for y'(0)
      vector<double> coefficients; // Power series coefficients [c_0, c_1,
          c_2, ...]

      public:
      SecondOrderODE_Homogeneous_PowerSeriesSolver(
```

```
      const PolynomialDouble& P_input,
      const PolynomialDouble& Q_input,
      const PolynomialDouble& R_input,
      double x0_input,
      double y0_input,
      double dy0_input) ;

      void computeSeries(int terms) ;
      vector<double> coefficientsvector(int terms) ;
      double evaluateAt(double x, int terms) ;
      void printCoefficients() const ;
      void printSolution() const ;

};

#endif

...
```

Code 45: *include/symintegral/dsolve.h*

The functions related to this class are:

- **computeSeries(int n)**
  a **void** function to compute the power series solution with n as the terms that we specify for the series solution. It dynamically shifts indices and unrolls the algorithmic Cauchy product loop configurations to automatically decouple the targeted c_{n+2} parameter from past historical weights.
- **coefficientsvector(int n)**
  a **vector<double>** function to save the coefficients of the power series solution with n as the terms that we specify for the series solution.
- **evaluateAt(double x_0, int n)**
  a **double** function to compute the approximation of the power series solution at x_0 ($y(x_0)$), the number of terms is n and the highest degree of the power series will be $n-1$. This function applies Horner's-like incremental power aggregation to minimize floating-point multiplication errors. The Horner-adjacent evaluation approach avoids using the expensive **std::pow** operations inside loops by incrementally multiplying the current x value. When we employ Horner's evaluation schema, it can safely aggregate terms without experiencing float overflows or scaling precision decay at localized evaluations.
- **printCoefficients() const**
  a **void** function to display the coefficients of the solution ($y(x)$) for the differential equation, we need to call this function (**computeSeries(int n)**) first, before we are able to display the coefficients.
- **printSolution() const**
  a **void** function to display the solution ($y(x)$) on the command line interface (terminal / xterm) up until the n-th terms that we have specified in the **computeSeries(int n)** function, we need to call this function (**computeSeries(int n)**) first, before we are able to display the series. This function formats output nicely by handling sign changes and omitting zero-value coefficients.

Recurrence Tracking
By substituting $y(x_{new}) = \sum_{n=0}^{\infty} c_n x_{new}^n$ into the differential equation, the total coefficient multiplying x_{new}^m aggregates contributions from the j-th derivative coefficients. **Isolating the Next Coefficient**
At step m, the coefficient of x_{new}^m uniquely relies on known entries up to c_{m+1}, alongside exactly one unknown entry: $p_0(m+2)(m+1)c_{m+2}$. Isolating c_{m+2} yields:

$$c_{m+2} = \frac{-1}{P(x_0)(m+2)(m+1)} \left[\sum_{j=1}^{m} p_j(m-j+2)(m-j+1)c_{m-j+2} + q_j(m-j+1)c_{m-j+1} + r(j)c_{m-j} \right]$$

The algorithm safely loops up to n terms, resolving one higher order coefficient with every incremental iteration step.

class SecondOrderODE_Homogeneous_Frobenius_PowerSeriesSolver
this class is an elegant way to solve a homogeneous second order linear differential equation with variable coefficients

$$P(x)y'' + Q(x)y' + R(x)y = 0$$

by using power series this class encapsulates the mathematical array elements and generation rules to compute the series solution around a regular singular point $x_0 \in \mathbb{R}$, x_0 can be 0 or other real number. It validates that the layout center point (x_0) is a regular singular point ($P(x_0) = 0$) before processing. This class will compute the power series solution of a homogeneous second order linear differential equation with polynomial coefficients around a regular singular point x_0, the solution can be written as a Taylor series centered at x_0

$$y_1(x) = r_1 \sum_{n=0}^{\infty} a_n(x - x_0)^n$$
$$y_2(x) = r_2 \sum_{n=0}^{\infty} b_n(x - x_0)^n$$

By shifting the coordinate system such that $x_{new} = x - x_0$, we can rewrite the coefficient polynomials $P(x), Q(x)$, and $R(x)$ in terms of $(x - x_0)$ using Taylor expansion. This simplifies the computation to expanding around $x_{new,0} = 0$.
The computation that is used in this class is far more complex than:
SecondOrderODE_Homogeneous_PowerSeriesSolver

In the **dsolve.h** we define the class, in **private:** we have defined **PolynomialDouble** P, Q, R (already shifted polynomials, the process occurs in **public:**) and **PolynomialComplex** Pc, Qc, Rc that is used to store the converted P, Q, C that become complex polynomial with the conversion constructor, the process occurs in **computeSeriesCoefficients()** function.

There are several important variables that we define in **private:**. First, we define x_0, **max_terms**, r_1 and r_2 as **complex<double>**, **coefficients_root1** and **coefficients_root2** as **vector<complex<double»** to store the coefficients of the linearly independent solution $y_1(x)$ and $y_2(x)$, respectively. Then, we have **C_log** to store the logarithmic term C when we compute $y_2(x)$ in case of different indicial equation roots that differ by an integer. Finally, we define the **bool** to categorize roots: **integer_diff, complex_roots, repeated_roots, and noninteger_diff**.

In **public:**, we starts by shifting the input polynomial that we call as **P_in, Q_in, R_in** to be shifted around x_0 and stored as **PolynomialDouble** P, Q, R, we also run the function **solve_indicial_equation()** directly to compute the roots of the indicial equations (r_1 and r_2) so we can use them right away for further computation.

```
...


#ifndef
    SECONDORDERODE_HOMOGENEOUS_FROBENIUS_POWERSERIESSOLVER_H

#define
    SECONDORDERODE_HOMOGENEOUS_FROBENIUS_POWERSERIESSOLVER_H

class SecondOrderODE_Homogeneous_Frobenius_PowerSeriesSolver {
      private:
      // Shifted polynomials
      PolynomialDouble P, P_unshifted; // Coefficient for y''
      PolynomialDouble Q, Q_unshifted; // Coefficient for y'
      PolynomialDouble R, R_unshifted; // Coefficient for y
      PolynomialComplex Pc, Qc, Rc;

      complex<double> x0;
      int max_terms;
      complex<double> r1, r2; // Roots of indicial equation
      vector<complex<double>> coefficients_root1;
      vector<complex<double>> coefficients_root2;
      complex<double> C_log;
      bool integer_diff, complex_roots, repeated_roots, noninteger_diff; //
          to categorize the roots
      bool p0_exists, q0_exists, EulerCauchy, Frobenius; // to classify the
          ode
      complex<double> get_indicial_value(complex<double> rho);// necessary
          or not?

      public:
      SecondOrderODE_Homogeneous_Frobenius_PowerSeriesSolver(
          PolynomialDouble P_in, PolynomialDouble Q_in, PolynomialDouble
          R_in, complex<double> x0_point, int terms)
      : max_terms(terms), x0(x0_point)
      {
            P_unshifted = P_in;
            Q_unshifted = Q_in;
            R_unshifted = R_in;
            // Shift equations so that the regular singular point is at t =
                 0
            P = P_in.shift_around(x0.real());
            Q = Q_in.shift_around(x0.real());
            R = R_in.shift_around(x0.real());
```

```
            solve_indicial_equation();
            classify_ode() ;
        }
        void classify_ode() ;
        void computeSeriesCoefficients() ;

        void solve_indicial_equation() ;
        std::pair<complex<double>, complex<double>>
            solve_indicial_equation_inpair() ;
        complex<double> evaluate_series_y1(complex<double> x) ;
        complex<double> evaluate_series_y2(complex<double> x) ;
        complex<double> evaluate_series_derivative(const vector<complex<double
            >>& a, complex<double> r_val, complex<double> x) ;
        void solve_ivp(double x_init, double y_init, double dy_init, const
            vector<complex<double>>& test_points);
        void printCoefficients() const ;
        void printSolution() const ;

};

#endif
```

Code 46: *include/symintegral/dsolve.h*

The functions related to this class are:

- **solve_inidicial_equation()**
  a **void** function to compute the roots of the indicial equation r_1 and r_2, it will return the roots in data type of **complex<double>**. This function extracts values for p_0, q_0, r_0 at the singularity point x_0, solves the characteristic quadratic equation $P_0 r(r-1) + Q_0 r + R_0 = 0$. It also utilizes **std::sqrt** overloaded for complex numbers to output complex roots if necessary.
  If we have the second-order homogeneous linear differential equation with this form:

  $$P(x)y'' + Q(x)y' + R(x) = 0$$

  at a regular singular point $x = x_0$, then the quadratic equation will be:

  $$r(r-1) + p_0 r + q_0 = 0$$

  where the coefficients p_0 and q_0 are defined by the limits:

  $$p_0 = \lim_{x \to x_0} (x - x_0) \frac{Q(x)}{P(x)}$$
  $$q_0 = \lim_{x \to x_0} (x - x_0)^2 \frac{R(x)}{P(x)}$$

  Substitute p_0 and q_0 into $r^2 + (p_0 - 1)r + q_0 = 0$ (which expands to $r(r-1) + p_0 r + q_0 = 0$) and solve for r using the quadratic formula:

  $$r_{1,2} = \frac{-(p_0 - 1) \pm \sqrt{(p_0 - 1)^2 - 4q_0}}{2}$$

- **classify_ode()**
  a **void** function to classify the differential equation at a given point x_0, it will check if it is an ordinary point or a regular singular point. Afterwards, it will check $p_0 = \lim_{x\to x_0}(x-x_0)\frac{Q(x)}{P(x)}$ and $q_0 = \lim_{x\to x_0}(x-x_0)^2\frac{R(x)}{P(x)}$ so it can differentiate the differential equation can be solved using which method (Frobenius or Euler-Cauchy), or it is an irregular singular point that cannot be solved by Frobenius.
- **computeSeriesCoefficients()**
  a **void** function to compute the power series solution. It clustered the code into 4 sections to compute $y_1(x)$ and $y_2(x)$ based on the type of roots obtained from solving the indicial equation. For a shift point x_0, the Frobenius method assumes a solution form:

$$y(x) = (x-x_0)^r \sum_{n=0}^{\infty} a_n (x-x_0)^n$$

  With this solution form, the algorithm uses an automated nesting loop representing the Frobenius summation shift to compute the power series solution. It implements the Frobenius recurrence relation for computing $y_1(x)$, given it is a non-complex root:

$$a_n = -\frac{\sum_{k=0}^{n-1} a_k[p_{n-k+2}(k+r)(k+r-1) + q_{n-k+1}(k+r) + r_{n-k}]}{f(r+n)}$$

  with $f(r+n) = P(x_0)(r+n)(r+n-1) + Q(x_0)(r+n) + R(x_0)$ as the indicial function evaluation at $n+r$, and $n = 1, 2, \ldots, max_terms$. Afterwards, to compute $y_2(x)$ the code will be separated into each root categories, with complex roots that has the more complicated code.

  We will breakdown the reason of the specific indices $(n-k+2, n-k+1, n-k)$ appear and how the recurrence relation is structured. After we assume a Frobenius series solution of the form $y(x) = (x-x_0)^r \sum_{n=0}^{\infty} a_n(x-x_0)^n$, we plug y, y', and y'' back into the differential equation, the powers of x will not match immediately.

  To combine them into a single summation, we must perform an index shift so that every term carries the exact same power of x (usually targeting x^{n+r} or a shifted version like x^n).

  Depending on how we define the summation index, it shifts the counter variable so that:

  - The y'' term (which naturally brings down a -2 in the exponent) requires a shift of $+2$ to match the baseline power. This yields the $n-r+2$ shifting logic for the P term.
  - The y' term (which naturally brings down a -1 in the exponent) requires a shift of $+1$. This yields the $n-r+1$ shifting logic for the Q term.
  - The y term remains at the baseline $n-r$ for the R term.

  For the Frobenius series to satisfy the differential equation, the total coefficient of every single power of x must equal zero. When we group the shifted terms together, we get a recurrence relation that looks like this:

$$P(n,r)a_n + Q(n,r)a_{n-1} + R(n,r)a_{n-2} = 0$$

- For $n = 0$, it strips down to the indicial equation $f(r)a_0 = 0$. Since $a_0 \neq 0, f(r) = 0$, giving us the two roots.
- For $n > 0$, we isolate a_n:

$$a_n = \frac{-Q(n,r)a_{n-1} - R(n,r)a_{n-2}}{P(n,r)}$$

This recurrence relation is perfectly fine to compute the Frobenius series solution for two distinct roots that differ by non-integer, for other cases some adjustments need to be made, especially to compute $y_2(x)$.

- **evaluate_series_y1(complex<double>** x_{eval}**)**
  a **complex<double>** function to compute the series solution for $y_1(x)$ at $x = x_{eval}$ by using Horner's method.
- **evaluate_series_y2(complex<double>** x_{eval}**)**
  a **complex<double>** function to compute the series solution for $y_2(x)$ at $x = x_{eval}$ by using Horner's method.
- **evaluate_series_derivative(const vector<complex<double>>&** a_n**, complex<double>** r**, complex<double>** t_0
  a **complex<double>** function to compute the derivative of the basis solution by using Horner's method, a_n is either **coefficients_root1** coupled with $r = r_1$ or **coefficients_root2** coupled with $r = r_2$, and t_0 is one of the initial condition parameter for the initial value problem.
- **solve_ivp(double** t_0**, double** $y(t_0)$**, double** $y'(t_0)$**, const vector<complex<double>>&** $test_points$**)**
  a **void** function to solve the initial value problem by using Cramer's rule to find the linear combination C_1 and C_2 matching the initial boundary state vector, bypassing typical analytical integration constraints.
  If initial values $y(t_0)$ and $y'(t_0)$ are provided at a point near x_0, we can construct the general solution $y(t) = C_1y_1(t) + C_2y_2(t)$ and then compute $y(test_points)$.

  As a rule of thumb, initial conditions cannot be set directly at $x = x_0$ because the solution typically blows up or branches out at a singular point. Instead, they are evaluated at a neighboring point t_0.
  A 2×2 linear system matches the targets:

$$\begin{bmatrix} y_1(t_0) & y_2(t_0) \\ y_1'(t_0) & y_2'(t_0) \end{bmatrix} \begin{bmatrix} C_1 \\ C_2 \end{bmatrix} \begin{bmatrix} y(t_0) \\ y'(t_0) \end{bmatrix}$$

  With Cramer's rule we will obtain

$$\begin{aligned} \det &= (y_1(t_0) * y_2'(t_0)) - (y_2(t_0) * y_1'(t_0)) \\ C_1 &= \frac{(y(t_0) * y_2'(t_0)) - (y_2(t_0) * y'(t_0))}{\det} \\ C_2 &= \frac{(y_1(t_0) * y'(t_0)) - (y(t_0) * y_1'(t_0))}{\det} \end{aligned}$$

- **printCoefficients() const**
  a **void** function to display the coefficients of the solution ($y_1(x)$ and $y_2(x)$) for the differential equation, we need to call this function (**computeSeriesCoefficients()**) first, before we are able to display the coefficients.

- **printSolution() const**
  a **void** function to display the solution ($y(x)$) on the command line interface (terminal / xterm) up until the n-th terms that we have specified in the **computeSeries(int n)** function, we need to call this function (**computeSeries(int n)**) first, before we are able to display the series. This function formats output nicely by handling sign changes and omitting zero-value coefficients.

```
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Differential Equations/Test SymIntegration Frobenius as Class ]# ./main
Analyzing ODE at point x0 = (0,0)
--> Result: Euler-Cauchy Equation.
    Solve exactly using the auxiliary algebraic characteristic equation.

P(x) y'' + Q(x)y'+ R(x) y = 0

P(x) = (0,0) * x^0 + (0,0) * x^1 + (1,0) * x^2

Q(x) = (0,0) * x^0 + (1,0) * x^1

R(x) = (4,0) * x^0

x0 = (0,0)
r1 = (0,2)
r2 = (-0,-2)

Case 4: Complex roots.

Series solution:

y1(x) = [ C1 *  x^0 cos ( 2 ln(x) ) ] + [ C2 *  x^0 sin ( 2 ln(x) ) ]
Time taken by function: 992 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Differential Equations/Test SymIntegration Frobenius as Class ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Differential Equations/Test SymIntegration Frobenius as Class ]# ./main
Analyzing ODE at point x0 = (0,0)
--> Result: Euler-Cauchy Equation.
    Solve exactly using the auxiliary algebraic characteristic equation.

P(x) y'' + Q(x)y'+ R(x) y = 0

P(x) = (0,0) * x^0 + (0,0) * x^1 + (1,0) * x^2

Q(x) = (0,0) * x^0 + (5,0) * x^1

R(x) = (4,0) * x^0

x0 = (0,0)
r1 = (-2,0)
r2 = (-2,-0)

Case 3: Repeated roots.

Series solution:

y(x) = ( C1 + C2 ln(x) ) x^(-2,0)
Time taken by function: 1019 microseconds
```

Figure 3.95: *The **class SecondOrderODE_Homogeneous_Frobenius_PowerSeriesSolver** has been set to automatically classify the differential equation after it has computed the indicial roots, it can determine whether x_0 is a regular singular point or irregular singular point. If it is a regular singular point then it classifies the second order differential equation into two types (based on the method which can solve the ODE): Frobenius or Euler-Cauchy. Above is the example of two differential equations that can be solved with Euler-Cauchy method (**SymIntegration/Examples/Differential Equations/Test SymIntegration Frobenius as Class/main.cpp**).*

The Reason of Using Horner's Method

If you evaluate $P(x) = 3x^2 - 2x + 5$ naively using a loop or **std::pow**, you execute multiple exponentiation operations:

- **Naive calculation:** Evaluates x^2, multiplies by 3, then evaluates x, multiplies by -2, and adds them up.
- **Horner's arrangement:** Factors the equation into nested groupings: $P(x) = ((3) \cdot (x - 2)) \cdot x + 5$

The Horner's method (this is the industry-standard) evaluates the polynomial in $O(n)$ time by minimizing the number of multiplications, preventing both arithmetic overflow and precision loss compared to using **std::pow**. $O(n)$ is significantly better than $O(n^2)$ time

complexity. In computer science and C++, Big O notation measures how the running time of an algorithm grows as the input size (n) increases. A lower time complexity means the program executes fewer operations and runs much faster.

- $O(n)$ **(Linear Time):** The execution time grows linearly with the input size. If we double the data, the program takes roughly double the time. An $O(n)$ algorithm typically processes each element in a collection exactly once. A classic example is finding a number in an array using a single **for** loop.
- $O(n^2)$ **(Quadratic Time):** The execution time grows quadratically. If you double the data, the program takes four times longer. If the input increases by 10 times, the execution time jumps by 100 times. An $O(n^2)$ algorithm usually performs an operation on every pair of elements. A classic sign is nested loops iterating over the same data, such as Bubble Sort or checking for duplicates.

Feature	**Naive Exponentiation Method**	**Horner's Method**
Time Complexity	$O(n^2)$ or $O(n\log(n))$	$O(n)$
Multiplication	Roughly $\frac{n(n+1)}{2}$	Strictly n
Numerical Stability	Prone to rounding errors at high powers	Highly stable and accurate

Table 3.8: *Naive exponentiation method compared with Horner's method in evaluating polynomial.*

Step	**Operation Type**	**Time Complexity**	**Space Complexity**
Roots Resolution	Analytic (quadratic equation solver)	$O(1)$	$O(1)$
Recurrence Kernel	Nested Coefficient Convolution	$O(N^2)$	$O(N)$
Output Evaluation	Polynomial Series Accumulation	$O(N)$	$O(1)$

Table 3.9: *Computational matrix for Frobenius method.*

```
P(x) y'' + Q(x)y'+ R(x) y = 0

P(x) = (0,0) * x^0 + (0,0) * x^1 + (1,0) * x^2

Q(x) = (0,0) * x^0 + (1,0) * x^1

R(x) = (4,0) * x^0 + (0,0) * x^1 + (1,0) * x^2

x0 = 0
r1 = (0,2)
r2 = (-0,-2)

Case 4: Complex roots.

Series solution:

y1(x) = x^(0,2) ( (1,0) + (-0.05,0.1)*x^2 + (-0.0015625,-0.0046875)*x^4 + (9.01442e-05,7.01122e-05)*x^6 + (-1.565e-06,-3.13001e-07)*x^8 +
(1.45707e-08,-2.69828e-09)*x^10 )
y2(x) = x^(-0,-2) ( (1,0) + (-0.05,-0.1)*x^2 + (-0.0015625,0.0046875)*x^4 + (9.01442e-05,-7.01122e-05)*x^6 + (-1.565e-06,3.13001e-07)*x^8
+ (1.45707e-08,2.69828e-09)*x^10  )
Real-valued series solution:

y1(x) = x^0[ cos( 2 ln (x) ) * ( 1 - -0.05*x^2 - -0.0015625*x^4 + 9.01442e-05*x^6 - -1.565e-06*x^8 + 1.45707e-08*x^10 )  - sin( 2 ln (x) )
 * ( 0.1*x^2 - -0.0046875*x^4 + 7.01122e-05*x^6 - -3.13001e-07*x^8 - -2.69828e-09*x^10 ) ]
y2(x) = x^0 [  sin( 2 ln (x) ) * ( 1 - -0.05*x^2 - -0.0015625*x^4 + 9.01442e-05*x^6 - -1.565e-06*x^8 + 1.45707e-08*x^10 )  + cos( 2 ln (x)
 ) * ( 0.1*x^2 - -0.0046875*x^4 + 7.01122e-05*x^6 - -3.13001e-07*x^8 - -2.69828e-09*x^10 ) ]

Computed Power Series Coefficients

y1(x) coefficients (r = (0,2)):
a_{0} = (1,0)
a_{1} = (0,0)
a_{2} = (-0.05,0.1)
a_{3} = (0,0)
a_{4} = (-0.0015625,-0.0046875)
a_{5} = (0,0)
a_{6} = (9.01442e-05,7.01122e-05)
a_{7} = (0,0)
a_{8} = (-1.565e-06,-3.13001e-07)
a_{9} = (0,0)
a_{10} = (1.45707e-08,-2.69828e-09)
a_{11} = (0,0)
a_{12} = (-8.54456e-11,4.72199e-11)
a_{13} = (0,0)
a_{14} = (3.39407e-13,-3.37891e-13)
a_{15} = (0,0)


y2(x) coefficients (r = (-0,-2)):
b_{0} = (1,0)
b_{1} = (-0,0)
```

Figure 3.96: *The power series solution of* $x^2y'' + xy' + (x^2 + 4)y = 0$ *computed with* ***class SecondOrderODE_Homogeneous_Frobenius_PowerSeriesSolver****, it is categorized as complex roots with* $r_1 = 2i$, $r_2 = -2i$, *and* $x_0 = 0$ *is a regular singular value (****SymIntegration/Examples/Differential Equations/Test SymIntegration Frobenius as Class/main.cpp****).*

```
y2(x) coefficients (r = (-0,-2)):
b_{0} = (1,0)
b_{1} = (-0,0)
b_{2} = (-0.05,-0.1)
b_{3} = (-0,0)
b_{4} = (-0.0015625,0.0046875)
b_{5} = (0,0)
b_{6} = (9.01442e-05,-7.01122e-05)
b_{7} = (0,0)
b_{8} = (-1.565e-06,3.13001e-07)
b_{9} = (0,0)
b_{10} = (1.45707e-08,2.69828e-09)
b_{11} = (0,0)
b_{12} = (-8.54456e-11,-4.72199e-11)
b_{13} = (0,0)
b_{14} = (3.39407e-13,3.37891e-13)
b_{15} = (0,0)

--- IVP Solution Results ---
Initial Conditions:
t0 = 1.1,       y(t0) = 0.84147,        y'(t0) = -0.13938
Linear Coefficients: C1 = (0.428558,-0.12275), C2 = (0.428558,0.12275)

x                       y(x)
-----------------------------------
(1,0)                   (0.836414,0)
(1.2,0)                 (0.811446,0)
(1.4,0)                 (0.677093,0)
(1.6,0)                 (0.483546,0)
(1.8,0)                 (0.265859,0)
(2,0)                   (0.048332,0)

Time taken by function: 3873 microseconds
```

Figure 3.97: *The power series solution and the initial value problem solution of* $x^2y'' + xy' + (x^2 + 4)y = 0$ *with* $t_0 = 1.1$, $y(t_0) = 0.84147$, $y'(t_0) = -0.13938$ *computed with* ***class SecondOrderODE_Homogeneous_Frobenius_PowerSeriesSolver****, it is categorized as complex roots with* $r_1 = 2i$, $r_2 = -2i$, *and* $x_0 = 0$ *is a regular singular value (****SymIntegration/Examples/Differential Equations/Test SymIntegration Frobenius as Class/main.cpp****).*

bernoullinumbers(int n)
to compute the n-th term of Bernoulli numbers.

probabilists_Hermitepolynomials(int n, const Symbolic &x)
to compute the first n probabilist's Hermite polynomials.

physicists_Hermitepolynomials(int n, const Symbolic &x)
to compute the first n physicist's Hermite polynomials.

Jacobipolynomials(double α, double β, int n, const Symbolic &x)
to compute the first n Jacobi polynomials.

```
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Differential Equations/Test SymIntegration Jacobian Polynomials ]# ./main
Jacobian polynomials:
α = 2, β = 1
P_{0} (x) = 1
P_{1} (x) = 2.5*x+0.5
P_{2} (x) = 1.5*x+5.25*x^(2)-0.75
P_{3} (x) = -3.5*x+3.5*x^(2)+10.5*x^(3)-0.5
P_{4} (x) = -2.5*x-11.25*x^(2)+7.5*x^(3)+20.625*x^(4)+0.625
P_{5} (x) = 4.21875*x-8.4375*x^(2)-30.9375*x^(3)+15.4688*x^(4)+40.2188*x^(5)+0.46875
P_{6} (x) = 3.28125*x+18.0469*x^(2)-24.0625*x^(3)-78.2031*x^(4)+31.2812*x^(5)+78.2031*x^(6)-0.546875
P_{7} (x) = -4.8125*x+14.4375*x^(2)+62.5625*x^(3)-62.5625*x^(4)-187.688*x^(5)+62.5625*x^(6)+151.938*x^(7)-0.4375
P_{8} (x) = -3.9375*x-25.5938*x^(2)+51.1875*x^(3)+191.953*x^(4)-153.562*x^(5)-435.094*x^(6)+124.312*x^(7)+295.242*x^(8)+0.492188
P_{9} (x) = 5.33203*x-21.3281*x^(2)-106.641*x^(3)+159.961*x^(4)+543.867*x^(5)-362.578*x^(6)-984.141*x^(7)+246.035*x^(8)+574.082*x^(9)+0.410156
P_{10} (x) = 4.51172*x+33.8379*x^(2)-90.2344*x^(3)-383.496*x^(4)+460.195*x^(5)+1457.29*x^(6)-832.734*x^(7)-2185.93*x^(8)+485.762*x^(9)+1117.25*x^(10)-0.451172

Legendre polynomials:
α = 0, β = 0
P_{0} (x) = 1
P_{1} (x) = x
P_{2} (x) = 1.5*x^(2)-0.5
P_{3} (x) = -1.5*x+2.5*x^(3)
P_{4} (x) = -3.75*x^(2)+4.375*x^(4)+0.375
P_{5} (x) = 1.875*x-8.75*x^(3)+7.875*x^(5)
P_{6} (x) = 6.5625*x^(2)-19.6875*x^(4)+14.4375*x^(6)-0.3125
P_{7} (x) = -2.1875*x+19.6875*x^(3)-43.3125*x^(5)+26.8125*x^(7)
P_{8} (x) = -9.84375*x^(2)+54.1406*x^(4)-93.8438*x^(6)+50.2734*x^(8)+0.273438
P_{9} (x) = 2.46094*x-36.0938*x^(3)+140.766*x^(5)-201.094*x^(7)+94.9609*x^(9)
P_{10} (x) = 13.5352*x^(2)-117.305*x^(4)+351.914*x^(6)-427.324*x^(8)+180.426*x^(10)-0.246094

Gegenbauer polynomials:
α = 1, β = 1
P_{0} (x) = 1
P_{1} (x) = 2*x
P_{2} (x) = 3.75*x^(2)-0.75
P_{3} (x) = -3*x+7*x^(3)
P_{4} (x) = -8.75*x^(2)+13.125*x^(4)+0.625
P_{5} (x) = 3.75*x-22.5*x^(3)+24.75*x^(5)
P_{6} (x) = 14.7656*x^(2)-54.1406*x^(4)+46.9219*x^(6)-0.546875
P_{7} (x) = -4.375*x+48.125*x^(3)-125.125*x^(5)+89.375*x^(7)
P_{8} (x) = -21.6562*x^(2)+140.766*x^(4)-281.531*x^(6)+170.93*x^(8)+0.492188
P_{9} (x) = 4.92188*x-85.3125*x^(3)+383.906*x^(5)-621.562*x^(7)+328.047*x^(9)
P_{10} (x) = 29.3262*x^(2)-293.262*x^(4)+997.09*x^(6)-1353.19*x^(8)+631.49*x^(10)-0.451172

Time taken by function: 8256343 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Differential Equations/Test SymIntegration Jacobian Polynomials ]#
```

Figure 3.98: *The first eleven Jacobi polynomials, Legendre polynomials, and Gegenbauer polynomials (**SymIntegration/Examples/Differential Equations/Test SymIntegration Jacobi Polynomials/main.cpp**).*

firstkind_Chebyshevpolynomials(int n, const Symbolic &x)
to compute the first n Chebyshev polynomials of the first kind.

secondkind_Chebyshevpolynomials(int n, const Symbolic &x)
to compute the first n Chebyshev polynomials of the second kind.

Gegenbauerpolynomials(double α, int n, const Symbolic &x)
to compute the first n Gegenbauer polynomials with parameter α.

```
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Differential Equations/Test SymIntegration Gegenbauer Polynomials ]# ./main
Gegenbauer polynomials with α = 1 :
C_{0} (x) = 1
C_{1} (x) = 2*x
C_{2} (x) = 4*x^(2)-1
C_{3} (x) = 8*x^(3)-4*x
C_{4} (x) = 16*x^(4)-12*x^(2)+1
C_{5} (x) = 32*x^(5)-32*x^(3)+6*x
C_{6} (x) = 64*x^(6)-80*x^(4)+24*x^(2)-1
C_{7} (x) = 128*x^(7)-192*x^(5)+80*x^(3)-8*x
C_{8} (x) = 256*x^(8)-448*x^(6)+240*x^(4)-40*x^(2)+1
C_{9} (x) = 512*x^(9)-1024*x^(7)+672*x^(5)-160*x^(3)+10*x
C_{10} (x) = 1024*x^(10)-2304*x^(8)+1792*x^(6)-560*x^(4)+60*x^(2)-1

Time taken by function: 139430 microseconds
```

Figure 3.99: *The first eleven Gegenbauer polynomials with $\alpha = 1$ (**SymIntegration/Examples/Differential Equations/Test SymIntegration Gegenbauer Polynomials/main.cpp**).*

Laguerrepolynomials(double α, int n, const Symbolic &x)
to compute the first n generalized Laguerre polynomials with parameter α.

About Class

In C++, when you instantiate a class object, you reserve a specific region of memory for that instance, and the system automatically invokes a contructor to initialize its data members. Depending on how you choose to structure your syntax, the object will be instantiated using one of two primary memory allocations.
If you instantiate the object normally without special keywords, it is created on the stack. The memory is managed automatically, and the object is destroyed as soon as it goes out of scope.

```
// Calls the default constructor
MyClass obj;

// Calls a parametrized constructor
MyClass obj2(10,"text");

// Modern uniform (list) initialization
MyClass obj3{20, "more text"};
```

Code 47: *include/symintegral/dsolve.h*

If we use the **new** keyword, the object is instantiated on the **heap**. This returns a pointer to the memory location. This memory persists until we explicitly free it using the **delete** keyword.

```
// Instantiates on the heap and returns a pointer
MyClass* ptr = new MyClass(10, "text");

// We must manually clean up heap allocations to avoid memory leaks
delete ptr;
```

Code 48: *include/symintegral/dsolve.h*

V. The Laplace Transform

[SI*] **Definition**

Many practical engineering problems involve mechanical or electrical systems acted on by discontinuous or impulsive forcing terms. The Laplace transform is a method that is especially well suited to these problems.

The Laplace transform involves an integral from zero to infinity, a knowledge of improper integrals of this type is necessary to appreciate the subsequent development of the properties of the transform.

An improper integral over an unbounded interval is defined as a limit of integrals over finite intervals; thus

$$\int_a^\infty f(t)\,dt = \lim_{A\to\infty} \int_a^A f(t)\,dt \tag{3.349}$$

where A is a positive real number. If the integral from a to A exists for each $A > a$, and if the limit as $A \to \infty$ exists, then the improper integral is said to converge to that limiting value. Otherwise the integral is said to diverge, or to fail to exist.

A function f is said to be piecewise continuous on an interval $\alpha \le t \le \beta$ if the interval can be partitioned by a finite number of points $\alpha = t_0 < t_1 < \cdots < t_n = \beta$ so that

1. f is continuous on each open subinterval $t_{i-1} < t < t_i$.
2. f approaches a finite limit as the endpoints of each subinterval are approached from within the subinterval.

In other words, f is piecewise continuous on $\alpha \le t \le \beta$ if it is continuous there except for a finite number of jump discontinuities. If f is piecewise continuous on $\alpha \le t \le \beta$ for every $\beta > \alpha$, then f is said to be piecewise continuous on $t \ge \alpha$.

The integral of a piecewise continuous function on a finite interval is just the sum of the integrals on the subintervals created by the partition points. For instance, for the function $f(t)$ with three subintevals, we have

$$\int_\alpha^\beta f(t)\,dt = \int_\alpha^{t_1} f(t)\,dt + \int_{t_1}^{t_2} f(t)\,dt + \int_{t_2}^\beta f(t)\,dt \tag{3.350}$$

Thus, if f is a piecewise continuous on the interval $a \le t \le A$, then $\int_a^A f(t)\,dt$ exists. Hence, if f is piecewise continuous for $t \ge a$, then $\int_a^A f(t)\,dt$ exists for each $A > a$. However, piecewise continuity is not enough to ensure convergence of the improper integral $\int_a^\infty f(t)\,dt$. If f cannot be integrated easily in terms of elementary functions, the definition of convergence of $\int_a^\infty f(t)\,dt$ may be difficult to apply. Frequently, the most convenient way to test the convergence or divergence of an improper integral is by the following comparison theorem, which is analogous to a similar theorem for infinite series.

Theorem 3.15: Comparison Theorem

If f is piecewise continuous for $t \ge a$, if $|f(t)| \le g(t)$ when $t \ge M$ for some positive constant M, and if $\int_M^\infty g(t)\,dt$ converges, then $\int_a^\infty f(t)\,dt$ also converges. On the other hand, if $f(t) \ge g(t) \ge 0$ for $t \ge M$, and if $\int_M^\infty g(t)\,dt$ diverges, then $\int_a^\infty f(t)\,dt$ also diverges.

The Laplace Transform

Among the tools that are very useful for solving linear differential equations are integral

transforms. An integral transform is a relation of the form

$$F(s) = \int_{\alpha}^{\beta} K(s,t) f(t)\, dt \tag{3.351}$$

where $K(s,t)$ is a given function, called the kernel of the transformation, and the limits of integration α and β are also given. It is possible that $\alpha = -\infty$ or $\beta = \infty$, or both. The relation at Eq. (3.351) transforms the function f into another function F, which is called the transform of f.

There are several integral transforms that are useful in applied mathematics, including the Laplace transform that we will discuss. This transform is defined in the following way. Let $f(t)$ be given for $t \geq 0$, and suppose that f satisfies certain conditions to be stated a little later. Then the Laplace transform of f, which we will denote by $\mathcal{L}\{f(t)\}$ or by $F(s)$, is defined by the equation

$$\mathcal{L}\{f(t)\} = F(s) = \int_{0}^{\infty} e^{-st} f(t)\, dt \tag{3.352}$$

whenever this improper integral converges. The Laplace transform makes use of the kernel $K(s,t) = e^{-st}$. Since the solution of linear differential equations with constant coefficients are based on the exponential function, the Laplace transform is particularly useful for such equations. The general idea in using the Laplace transform to solve a differential equation is as follows:

1. Use the relation equation (3.352) to transform an initial value problem for an unknown function f in the t-domain into a simple problem (indeed, an algebraic problem) for F in the s-domain.
2. Solve this algebraic problem to find F.
3. Recover the desired function f from its transform F. This last step is known as "inverting the transform."

In general the parameter s may be complex, and the full power of the Laplace transform becomes available only when we regard $F(s)$ as a function of a complex variable.

Theorem 3.16: Existence of Laplace Transform

Suppose that

1. f is piecewise continuous on the interval $0 \leq t \leq A$ for any positive A.
2. $|f(t)| \leq Ke^{at}$ when $t \geq M$. In this inequality, K, a, and M are real constants, K and M necessarily positive.

Then the Laplace transform $\mathcal{L}\{f(t)\} = F(s)$, defined by Eq. (3.352), exists for $s > a$.

[SI*] **Example:**
Let $f(t) = 1,\ t \geq 0$. Then the Laplace transform is

$$\mathcal{L}\{1\} = \int_{0}^{\infty} e^{-st}\, dt = -\lim_{A\to\infty} \frac{e^{-st}}{s}\Big|_0^A = \frac{1}{s}, \quad s > 0$$

[SI*] **Example:**
Let $f(t) = e^{at},\ t \geq 0$. Then the Laplace transform is

$$\mathcal{L}\{e^{at}\} = \int_{0}^{\infty} e^{-st} e^{at}\, dt = -\lim_{A\to\infty} \frac{e^{-(s-a)t}}{s-a}\Big|_0^A = \frac{1}{s-a}, \quad s > a$$

[SI*] **Example:**
Let

$$f(t) = \begin{cases} 1, & 0 \le t < 1 \\ k, & t = 1 \\ 0, & t > 1 \end{cases}$$

where k is a constant. In engineering contexts $f(t)$ often represents a unit pulse, perhaps of force or voltage.
Note that f is a piecewise continuous function. Then

$$\mathcal{L}\{f(t)\} = \int_0^\infty e^{-st} f(t)\, dt = \int_0^1 e^{-st}\, dt = -\frac{e^{-st}}{s}|_0^1 = \frac{1 - e^{-s}}{s}, \quad s > 0$$

Observe that $\mathcal{L}\{f(t)\}$ does not depend on k, the function value at the point of discontinuity. Even if $f(t)$ is not defined at this point, the Laplace transform of f remains the same. Thus there are many functions, differing only in their value at a single point, that have the same Laplace transform.

[SI*] **Example:**
Let $f(t) = \sin(at),\ t \ge 0$. Then

$$\mathcal{L}\{\sin(at)\} = F(s) = \int_0^\infty e^{-st} \sin(at)\, dt, \quad s > 0$$

Since

$$F(s) = \lim_{A \to \infty} \int_0^A e^{-st} \sin(at)\, dt$$

upon integrating by parts, we obtain

$$\begin{aligned} F(s) &= \lim_{A \to \infty} \left[-\frac{e^{-st}\cos(at)}{a}|_0^A - \frac{s}{a}\int_0^A e^{-st}\cos(at)\, dt \right] \\ &= \frac{1}{a} - \frac{s}{a}\int_0^\infty e^{-st}\cos(at)\, dt \end{aligned}$$

A second integration by parts then yields

$$\begin{aligned} F(s) &= \frac{1}{a} - \frac{s^2}{a^2}\int_0^\infty e^{-st}\sin(at)\, dt \\ &= \frac{1}{a} - \frac{s^2}{a^2}F(s) \end{aligned}$$

Hence, solving for $F(s)$, we have

$$F(s) = \frac{a}{s^2 + a^2}, \quad s > 0$$

[SI*] Now let us suppose that f_1 and f_2 are two functions whose Laplace transforms exist for $s > a_1$ and $s > a_2$, respectively. Then, for s greater than the maximum of a_1 and a_2,

$$\begin{aligned} \mathcal{L}\{c_1 f_1(t) + c_2 f_2(t)\} &= \int_0^\infty e^{-st}[c_1 f_1(t) + c_2 f_2(t)]\, dt \\ &= c_1 \int_0^\infty e^{-st} f_1(t)\, dt + c_2 \int_0^\infty e^{-st} f_2(t)\, dt \end{aligned}$$

hence

$$\mathcal{L}\{c_1 f_1(t) + c_2 f_2(t)\} = c_1 \mathcal{L}\{f_1(t)\} + c_2 \mathcal{L}\{f_2(t)\} \tag{3.353}$$

Equation (3.353) states that the Laplace transform is a linear operator, and we make frequent use of this property. The sum in Eq. (3.353) can be readily extended to an arbitrary number of terms.

[SI*] **Solution of Initial Value Problems**

Laplace transform can be used to solve initial value problems for linear differential equations with constant coefficients. The usefulness of the Laplace transform in this connection rests primarily on the fact that the transform of f' is related in a simple way to the transform of f.

Theorem 3.17: Continuous and Piecewise Continuous f in Laplace Transform

Suppose that f is continuous and f' is piecewise continuous on any interval $0 \leq t \leq A$. Suppose further that there exist constants K, a, and M such that $|f(t) \leq Ke^{at}|$ for $t \geq M$. Then $\mathcal{L}\{f'(t)\}$ exists for $s > a$, and moreover

$$\mathcal{L}\{f'(t)\} = s\mathcal{L}\{f(t)\} - f(0) \tag{3.354}$$

If f' and f'' satisfy the same conditions that are imposed on f and f', respectively, then it follows that the Laplace transform of f'' also exists for $s > a$ and is given by

$$\mathcal{L}\{f''(t)\} = s^2\mathcal{L}\{f(t)\} - sf(0) - f'(0) \tag{3.355}$$

Indeed, provided the function f and its derivatives satisfy suitable conditions, an expression for the transform of the nth derivative $f^{(n)}$ can be derived by successive applications of this theorem.

Corollary 3.1: Laplace Transform for n-th Derivative

Suppose that the functions $f, f', \ldots, f^{(n-1)}$ are continuous and that $f^{(n)}$ is piecewise continuous on any interval $0 \leq t \leq A$. Suppose further that there exist constants K, a, and M such that $|f(t)| \leq Ke^{at}$, $|f'(t)| \leq Ke^{at}, \ldots, |f^{(n-1)}(t)| \leq Ke^{at}$ for $t \geq M$. Then $\mathcal{L}\{f^{(n)}(t)\}$ exists for $s > a$ and is given by

$$\mathcal{L}\{f^{(n)}(t)\} = s^n\mathcal{L}\{f(t)\} - s^{n-1}f(0) - \cdots - sf^{(n-2)}(0) - f^{(n-1)}(0) \tag{3.356}$$

[SI*] Suppose we have a general second order linear equation with constant coefficients

$$ay'' + by' + cy = f(t) \tag{3.357}$$

Assuming that the solution $y = \phi(t)$ satisfies the conditions of Corollary (3.1) for $n = 2$, we can take the transform of Eq. (3.357) and thereby obtain

$$a[s^2Y(s) - sy(0) - y'(0)] + b[sY(s) - y(0)] + cY(s) = F(s) \tag{3.358}$$

where $F(s)$ is the transform of $f(t)$. By solving Eq. (3.358) for $Y(s)$, we find that

$$Y(s) = \frac{(as + b)y(0) + ay'(0)}{as^2 + bs + c} + \frac{F(s)}{as^2 + bs + c} \tag{3.359}$$

The problem is then solved, provided that we can find the function $y = \phi(t)$ whose transform is $Y(s)$.

Even at this early stage we can point out some of the essential features of the transform method. In the first place, the transform $Y(s)$ of the unknown function $y = \phi(t)$ is found by solving an algebraic equation rather than a differential equation, Eq. (3.358) rather than Eq. (3.357). This is the key to the usefulness of Laplace transforms for solving linear, constant coefficient, ordinary differential equations-the problem is reduced from a differential equation to an algebraic one. Next, the solution satisfying given initial conditions is automatically found, so that the task of determining appropriate values for the arbitrary constants in the general solution does not arise. Further, as indicated in Eq. (3.358), nonhomogeneous equations are handled in exactly the same way as homogeneous ones; it is not necessary to solve the corresponding homogeneous equation first. Finally, the method can be applied in the same way to higher order equations, as long as we assume that the solution satisfies the conditions of the corollary for the appropriate value of n.

Observe that the polynomial $as^2 + bs + c$ in the denominator on the right side of Eq. (3.359) is precisely the characteristic polynomial associated with Eq. (3.357). Since the use of a partial fraction expansion $Y(s)$ to determine $\phi(t)$ requires us to factor this polynomial, the use of Laplace transforms does not avoid the necessity of finding roots of the characteristic equation. For equations of higher than second order this may require a numerical approximation, particularly if the roots are irrational or complex.

The main difficulty that occurs in solving initial value problems by the transform method lies in the problem of determining the function $y = \phi(t)$ corresponding to the transform $Y(s)$. This problem is known as the inversion problem for the Laplace transform; $\phi(t)$ is called the inverse transform corresponding to $Y(s)$, and the process of finding $\phi(t)$ from $Y(s)$ is known as inverting the transform. We use the notation $\mathcal{L}^{-1}\{Y(s)\}$ to denote the inverse transform of $Y(s)$.

In solving the initial value problem, if f and g are continuous functions with the same Laplace transform, then f and g must be identical. On the other hand, if f and g are only piecewise continuous, then they may differ at one or more points of discontinuity and yet have the same Laplace transform. This lack of uniqueness of the inverse Laplace transform for piecewise continuous functions is of no practical significance in applications.

Thus there is essentially a one-to-one correspondence between functions and their Laplace transforms. We put a subsection below that list the elementary Laplace transforms.

Frequently, a Laplace transform $F(s)$ is expressible as a sum of several terms

$$F(s) = F_1(s) + F_2(s) + \cdots + F_n(s) \tag{3.360}$$

Suppose that $f_1(t) = \mathcal{L}^{-1}\{F_1(s)\}, \ldots, f_n(t) = \mathcal{L}^{-1}\{F_n(s)\}$. Then the function

$$f(t) = f_1(t) + f_2(t) + \cdots + f_n(t)$$

has the Laplace transform $F(s)$. By the uniqueness property stated previously, no other continuous function f has the same transform. Thus

$$\mathcal{L}^{-1}\{F(s)\} = \mathcal{L}^{-1}\{F_1(s)\} + \cdots + \mathcal{L}^{-1}\{F_n(s)\} \tag{3.361}$$

that is, the inverse Laplace transform is also a linear operator.

[SI*] The most important applications of the Laplace transform are in the study of mechanical vibrations and in the analysis of electric circuits. A vibrating spring-mass system has the equation of motion

$$m\frac{d^2u}{dt^2} + \gamma\frac{du}{dt} + ku = F(t) \tag{3.362}$$

where m is the mass, γ is the damping coefficient, k is the spring constant, and $F(t)$ the applied external force. The equation that describes an electric circuit containing an inductance L, a resistance R, and a capacitance C (an RLC circuit) is

$$L\frac{d^2Q}{dt^2} + R\frac{dQ}{dt} + \frac{1}{C}Q = E(t) \tag{3.363}$$

where $Q(t)$ is the charge on the capacitor and $E(t)$ is the applied voltage. In terms of the current $I(t) = \frac{dQ(t)}{dt}$, we can differentiate Eq. (3.363) and write

$$L\frac{d^2I}{dt^2} + R\frac{dI}{dt} + \frac{1}{C}I = \frac{dE}{dt}(t) \tag{3.364}$$

Suitable initial conditions on u, Q, or I must also be prescribed.
The spring-mass system and the electric circuit are identical mathematically, differing only in the interpretation of the constants and variables appearing in them. There are other physical problems that also lead to the same differential equation. Thus, once the mathematical problem is solved, its solution can be interpreted in terms of whichever corresponding physical problem is of immediate interest.

[SI*] **Step Functions**

[SI*] **Differential Equations with Discontinuous Forcing Functions**

[SI*] **Impulse Functions**

[SI*] **The Convolution Integral**

i. Elementary Laplace Transforms

The Laplace transform is a mathematical tool that converts a function of time into a function of a complex frequency variable.
The unilateral Laplace transform of a function $f(t)$ for $t \geq 0$ is defined by the improper integral:

$$F(s) = \mathcal{L}\{f(t)\} = \int_0^\infty e^{-st} f(t)\, dt$$

with $s = a + ib$ is a complex frequency variable, and $F(s)$ is the transformed function in the s-domain.

Below, we have the list of some elementary Laplace transforms:

$$
\begin{aligned}
f(t) = \mathcal{L}^{-1}\{F(s)\} \quad & F(s) = \mathcal{L}\{f(t)\} \\
f(t) = 1 \quad & F(s) = \frac{1}{s}, \quad s > 0 \\
f(t) = e^{at} \quad & F(s) = \frac{1}{s-a}, \quad s > a \\
f(t) = t^n,\ n \in \mathbb{Z}^+ \quad & F(s) = \frac{n!}{s^{n+1}}, \quad s > 0 \\
f(t) = t^p,\ p > -1 \quad & F(s) = \frac{\Gamma(p+1)}{s^{p+1}}, \quad s > 0 \\
f(t) = \sin(at) \quad & F(s) = \frac{a}{s^2 + a^2}, \quad s > 0 \\
f(t) = \cos(at) \quad & F(s) = \frac{s}{s^2 + a^2}, \quad s > 0 \\
f(t) = \sinh(at) \quad & F(s) = \frac{a}{s^2 - a^2}, \quad s > |a| \\
f(t) = \cosh(at) \quad & F(s) = \frac{s}{s^2 - a^2}, \quad s > |a| \\
f(t) = e^{at}\sin(bt) \quad & F(s) = \frac{b}{(s-a)^2 + b^2}, \quad s > a \\
f(t) = e^{at}\cos(bt) \quad & F(s) = \frac{s-a}{(s-a)^2 + b^2}, \quad s > a \\
f(t) = t^n e^{at},\ n \in \mathbb{Z}^+ \quad & F(s) = \frac{n!}{(s-a)^{n+1}}, \quad s > a \\
f(t) = u_c(t) \quad & F(s) = \frac{e^{-cs}}{s}, \quad s > 0 \\
f(t) = u_c(t) f(t-c) \quad & F(s) = e^{-cs}F(s) \\
f(t) = e^{ct} f(t) \quad & F(s) = F(s-c) \\
f = f(ct) \quad & F(s) = \frac{1}{c} F\left(\frac{s}{c}\right), \quad c > 0 \\
f(t) = \int_0^t f(t-\tau) g(\tau)\, d\tau \quad & F(s) = F(s)G(s) \\
f(t) = \delta(t-c) \quad & F(s) = e^{-cs} \\
f(t) = f^{(n)}(t) \quad & F(s) = s^n F(s) - s^{n-1} f(0) - \cdots - f^{(n-1)}(0) \\
f(t) = (-t)^n f(t) \quad & F(s) = F^{(n)}(s)
\end{aligned}
$$

Key properties:

- **Linearity:** $\mathcal{L}\{af(t) + bg(t)\} = aF(s) + bG(s)$ for any constants a and b.
- **Differentiation:** $\mathcal{L}\{f'(t)\} = sF(s) - f(0)$, which allows initial conditions to be included directly.
- **Integration:** $\mathcal{L}\{\int_0^t f(v)\, dv\} = \frac{F(s)}{s}$.
- **Convolution:** $\mathcal{L}\{f(t) * g(t)\} = F(s)G(s)$, turning complex convolution integrals into simple multiplications.

Main Applications:

- **Solving Differential Equations:** Converts hard calculus problems into easy algebra problems.
- **Control Theory:** Represents linear time-invariant (LTI) systems using transfer functions $H(s)$.
- **Signal Processing:** Analyzes system stability and frequency responses

Laplace transform is an integral transform, a mathematical tool, that changes a function from one domain (such as time) into another (such as frequency) to make hard math problems easier. Besides Laplace transform, there is Fourier transform that will be explained later on, it converts a time-based signal into pure-frequencies, whereas Laplace transform converts a signal into complex frequencies that include a fading or growth factor.
The complex number variable in Laplace transform $s = a + ib$ can be break down into two. The first one, the real part (a) handles exponential growth or decay, while the second one, the imaginary part (ib) handles oscillation.
Laplace can successfully handles growing, unstable signals by multiplying them by an exponential decay factor e^{-at} that forces the math to converge, while Fourier fails or "blows up" for signals that grow infinitely over time because it lacks a dampenener.
Thus, the Fourier transform is a special, restricted case of the Laplace transform. If we set the real damping part (a) of the Laplace variable to zero, the Laplace transform becomes a Fourier transform.

Laplace Tranform Applications:

- Solving complex differential equations with initial values.
- Designing control systems and checking mechanical or electrical stability.
- Analyzing transient (temporary and changing) circuit behavior.

ii. Determine Whether a Function f is Continuous, Piecewise Continuous, or Neither

In each case determine whether f is continuous,piecewise continuous, or neither on the interval $0 \leq t \leq 3$.

1. Case 1

$$f(t) = \begin{cases} t^2, & 0 \leq t \leq 1 \\ 2 + t, & 1 < t \leq 2 \\ 6 - t, & 2 < t \leq 3 \end{cases}$$

2. Case 2

$$f(t) = \begin{cases} t^2, & 0 \leq t \leq 1 \\ 1, & 1 < t \leq 2 \\ 3 - \frac{t^2}{2}, & 2 < t \leq 3 \end{cases}$$

3. Case 3

$$f(t) = \begin{cases} t^2, & 0 \leq t \leq 1 \\ (t-1)^{-1}, & 1 < t \leq 2 \\ 1, & 2 < t \leq 3 \end{cases}$$

Solution:

iii. Compute the Initial Value Problem for Nonhomogeneous Linear Differential Equations with Laplace Transform

Consider the differential equation

$$y'' - y' - 2y = 0 \tag{3.365}$$

and the initial conditions

$$y(0) = 1, \quad y'(0) = 0 \tag{3.366}$$

This problem is easily solved if we use this characteristic equation to represent the differential equation

$$r^2 - r - 2 = (r-2)(r+1) = 0 \tag{3.367}$$

and consequently the general solution of Eq. (3.365) is

$$y = c_1 e^{-t} + c_2 e^{2t} \tag{3.368}$$

To satisfy the initial conditions (3.366), we must have

$$\begin{aligned} c_1 + c_2 &= 1 \\ -c_1 + 2c_2 &= 0 \end{aligned}$$

hence $c_1 = \frac{2}{3}$ and $c_2 = \frac{1}{3}$, so that the solution of the initial value problem (3.365) and (3.366) is

$$y = \phi(t) = \frac{2}{3}e^{-t} + \frac{1}{3}e^{2t} \tag{3.369}$$

Now let us solve the same problem by using the Laplace transform. To do this, we must assume that the problem has a solution $y = \phi(t)$, which its first two derivatives satisfies the conditions of Corollary (3.1). Then, taking the Laplace transform of the differential equation (3.365), we obtain

$$\mathcal{L}\{y''\} - \mathcal{L}\{y'\} - 2\mathcal{L}\{y\} = 0 \tag{3.370}$$

where we have used the linearity of the transform to write the transform of a sum as the sum of the separate transforms. Upon using the corollary to express $\mathcal{L}\{y''\}$ and $\mathcal{L}\{y'\}$ in terms of $\mathcal{L}\{y\}$, we find that Eq. (3.370) becomes

$$\begin{aligned} s^2\mathcal{L}\{y\} - sy(0) - y'(0) - s\mathcal{L}\{y\} - y(0) - 2\mathcal{L}\{y\} = 0 \\ (s^2 - s - 2)Y(s) + (1-s)y(0) - y'(0) = 0 \end{aligned} \tag{3.371}$$

where $Y(s) = \mathcal{L}\{y\}$. Substituting for $y(0)$ and $y'(0)$ in Eq. (3.371) from the initial conditions (3.366), and then solving for $Y(s)$, we obtain

$$Y(s) = \frac{s-1}{s^2 - s - 2} = \frac{s-1}{(s-2)(s+1)} \tag{3.372}$$

We have thus obtained an expression for the Laplace transform $Y(s)$ of the solution $y = \phi(t)$ of the given initial value problem. To determine the function ϕ, we must find the function whose Laplace transform is $Y(s)$, as given by Eq. (3.372).
This can be done most easily by expanding the right side of Eq. (3.372) in partial fractions. Thus we write

$$Y(s) = \frac{s-1}{(s-2)(s+1)} = \frac{a}{s-2} + \frac{b}{s+1} = \frac{a(s+1) + b(s-2)}{(s-2)(s+1)} \tag{3.373}$$

where the coefficients a and b are to be determined. By equating the numerators of the second and fourth members of Eq. (3.373), we obtain

$$s - 1 = a(s + 1) + b(s - 2)$$

an equation that must hold for all s. In particular, if we set $s = 2$, then it follows that $a = \frac{1}{3}$. Similarly, if we set $s = -1$, then we find that $b = \frac{2}{3}$. By substituting these values for a and b, respectively, we have

$$Y(s) = \frac{1/3}{s - 2} + \frac{2/3}{s + 1} \tag{3.374}$$

Finally, it follows that $\frac{1}{3}e^{2t}$ has the transform $\frac{1}{3}(s - 2)^{-1}$; similarly, $\frac{2}{3}e^{-t}$ has the transform $\frac{2}{3}(s + 1)^{-1}$. Hence, by the linearity of the Laplace transform,

$$y = \phi(t) = \frac{1}{3}e^{2t} + \frac{2}{3}e^{-t}$$

has the transform (3.374) and is therefore the solution of the initial value problem (3.365), (3.366). Observe that it does satisfy the conditions of Corollary, as we assumed initially. Of course, this is the same solution that we obtained earlier.

iv. Functions in SymIntegration Related to Laplace Transform

[SI*] We have added two new implementations of **Heaviside** and **Dirac** functions. These two functions are very important in Laplace transform, in generalized calculus used by symbolic engines:

- The derivative of the Heaviside step function $H(x)$ is the Dirac delta function $\delta(x)$.

$$\frac{d}{dt}H(t-c) = \delta(t-c)$$

or

$$\int_{-\infty}^{t} \delta(\tau - c)\, d\tau = H(t-c)$$

- The derivative of the Dirac delta function $\delta(x)$ is often represented as $\delta'(x)$ (delta prime or higher distributions).

Heaviside Step Function ($H(t)$ or $u(t)$)
The Heaviside step function represents a unit switch turning on at $t = 0$:

$$H(t) = \begin{cases} 0, & \text{for } t < 0 \\ 1, & \text{for } t > 0 \end{cases}$$

Value at $t = 0$: Frequently defined as $H(0) = \frac{1}{2}$, though 0 or 1 are also used depending on convention.
Shifted step function (it equals 0 before time c, and 1 from time c onward):

$$H(t-c) = u_c(t) = u(t-c) = \begin{cases} 0, & \text{for } t < c \\ 1, & \text{for } t \geq c \end{cases}$$

A time delay / shift means that if the standard unit step is $u(t)$ (which turns on at $t = 0$), the subscript c in $u_c(t)$ means the "switch" is delayed or shifted to time c. It is widely used in differential equations and Laplace transforms to turn individual pieces of a multi-part(piecewise) function on or off at specific moments.

Key properties:

- **Time shifting/translation:** Multiplying a function by $H(t-c)$ cuts off the signal before $t = c$:

$$H(t-c)f(t-c)$$

- **Relationship to Delta:** The integral of the Dirac delta function gives the Heaviside function:

$$\int_{-\infty}^{t} \delta(\tau - c)\, d\tau = H(t-c)$$

Laplace Transform:

$$\mathcal{L}\{H(t)\} = \frac{1}{s}$$

$$\mathcal{L}\{H(t-c)f(t-c)\} = e^{-cs}F(s) \quad \text{for } c > 0$$

Dirac Delta Function ($\delta(t)$
The Dirac delta function is an impulse distribution that is zero everywhere except at the origin, where it is infinitely high, with a total area under the curve equal to 1:

$$\delta(t) = \begin{cases} \infty, & \text{for } t = 0 \\ 0, & \text{for } t \neq 0 \end{cases}$$

with $\int_{-\infty}^{\infty} \delta(t)\, dt = 1$.
Shifted Delta function: $\delta(t - c)$ is centered at $t = c$.

Key properties:

- **Sifting (Sampling) Property:** It "sifts" out the value of a continuous function $f(t)$ at the point where the delta is centered:

$$\int_{-\infty}^{\infty} f(t)\delta(t - c)\, dt = f(c)$$

- **Derivative Relation:** The weak derivative of the Heaviside step function is the Dirac delta function

$$\frac{d}{dt}H(t) = \delta(t)$$

- **Scaling Property:**

$$\delta(kt) = \frac{1}{|k|}\delta(t)$$

- **Even Symmetry:**

$$\delta(-t) = \delta(t)$$

- **Product Property:**

$$f(t)\delta(t - c) = f(c)\delta(t - c)$$

- **Laplace Transform:**

$$\mathcal{L}\{\delta(t)\} = 1$$
$$\mathcal{L}\{\delta(t - c)\} = e^{-cs}, \quad \text{for } c \geq 0$$

```
...
Symbolic heaviside(const Symbolic &);
Symbolic dirac(const Symbolic &);
...
```

Code 49: *include/symintegral/symintegrationc++.h*

```
...

Symbolic heaviside(const Symbolic &s)
{ return Heaviside(s); }

Symbolic dirac(const Symbolic &s)
{ return Dirac(s); }

...
```

Code 50: *src/symintegrationc++.cpp*

```
...
class Heaviside;
class Dirac;
...

class Heaviside : public Symbol
{
       public: Heaviside(const Heaviside&);
       Heaviside(const Symbolic&);

       Simplified simplify() const;
       Symbolic df(const Symbolic&) const;
       Symbolic integrate(const Symbolic&) const;

       Cloning *clone() const { return Cloning::clone(*this); }
};

class Dirac : public Symbol
{
       public: Dirac(const Dirac&);
       Dirac(const Symbolic&);

       Simplified simplify() const;
       Symbolic df(const Symbolic&) const;
       Symbolic integrate(const Symbolic&) const;

       Cloning *clone() const { return Cloning::clone(*this); }
};

...
```

Code 51: *include/symintegral/functions.h*

```
...

////////////////////////////////////
// Implementation of Heaviside //
////////////////////////////////////

Heaviside::Heaviside(const Heaviside &s) : Symbol(s) {}

Heaviside::Heaviside(const Symbolic &s) : Symbol(Symbol("heaviside")[s]) {}

Simplified Heaviside::simplify() const
{
       const Symbolic &s = parameters.front().simplify();
       // If argument is a known number
       if (s.type() == typeid(Numeric))
```

```
	{
		double val = CastPtr<const Number<double>>(s)->n;
		if (val > 0) return Number<int>(1);
		if (val < 0) return Number<int>(0);
		// heaviside(0) can be defined as 1/2 or left symbolic; here
		    return 0.5 or 1 if desired
	}
	return *this;
}

// Derivative of H(x) is Dirac(x) * x.df(s)
Symbolic Heaviside::df(const Symbolic &s) const
{
	//const Symbolic &arg = parameters.front();
	return dirac(parameters.front()) * parameters.front().df(s);
}

// Integral of H(x) is x * H(x) [indistinct or ramp-like behavior]
Symbolic Heaviside::integrate(const Symbolic &s) const
{
	const Symbolic &x = parameters.front();
	if (x == s) return x * heaviside(x);
	if (df(s) == 0) return *this * s;
	return *this * s; // basic placeholder
}


////////////////////////////////////
// Implementation of Dirac Delta //
////////////////////////////////////

Dirac::Dirac(const Dirac &s) : Symbol(s) {}

Dirac::Dirac(const Symbolic &s) : Symbol(Symbol("dirac")[s]) {}

Simplified Dirac::simplify() const
{
	const Symbolic &s = parameters.front().simplify();
	if (s.type() == typeid(Numeric))
	{
		double val = 0.0;
		bool is_numeric = false;

		// Try casting to Number<double> first, or fall back to Number<
		    int>
		if (isInteger_sym(s))
		{
			val = CastPtr<const Number<int>>(s)->n;
```

```
                    is_numeric = true;
                }
                else if (!isInteger_sym(s))
                {
                    val = CastPtr<const Number<double>>(s)->n;
                    is_numeric = true;
                }

                if (is_numeric && val != 0.0)
                {
                    return Number<int>(0); // Dirac(x) = 0 for x != 0
                }
        }
        return *this;
}

// Derivative of Dirac(x) is Dirac'(x)
Symbolic Dirac::df(const Symbolic &s) const
{
        const Symbolic &arg = parameters.front();
        // Assuming a symbolic constructor for derivative DiracPrime or
            similar
        return Symbol("DiracPrime")[arg] * arg.df(s);
}

// Integral of Dirac(x) w.r.t x yields Heaviside(x)
Symbolic Dirac::integrate(const Symbolic &s) const
{
        const Symbolic &x = parameters.front();
        if (x == s) return heaviside(x);
        if (df(s) == 0) return *this * s;
        return *this * s;
}


...
```

Code 52: *src/functions.cpp*

[SI*] We have created a function to obtain the argument of any function such as sin cos, exp, *dirac*, *heaviside*, if it is a structural symbol class object they usually inherit from **Symbol** or contain a structural representation so we can extract the argument. It will be very useful not only in Laplace transform related computation, but in other areas as well.
The global **getargument()** function should inspect whether the incoming expression *s* is a functional expression or has parameters. Since we branched out from SymbolicC++ 3.35, we follow their structure, SymbolicC++ wraps these inside **Symbol**, we can use **CastPtr** or inspect the underlying **type()** to extract the internal parameters.

```
...
```

```
Symbolic getargument(const Symbolic &);

...
```

Code 53: *include/symintegral/symintegrationc++.h*

```
...

// Universal argument extractor
Symbolic getargument(const Symbolic &s)
{
       try
       {
              CastPtr<const Symbol> sym(s);
              if ( !sym->parameters.empty())
              {
                     return sym->parameters.front();
              }
       }
       catch (const SymbolicError &se) {}


       return s;
}

...
```

Code 54: *src/symintegrationc++.cpp*

[SI*] We will list the basic functions and classes that can be used related to the Laplace transform:
class LaplaceTransform
this class computes Laplace transform for several functions $f(t)$ that already defined with numerical method (Adaptive Simpson's Rule), we also put a condition if $s < 0$ then it will return 0.0 instead.
The functions that already covered and can be computed with this class are:

$$f(t) = 1$$
$$f(t) = e^{at}$$
$$f(t) = t^n, \quad n \in \mathbb{Z}^+$$
$$f(t) = \sin(at)$$
$$f(t) = \cos(at)$$
$$f(t) = \sinh(at)$$
$$f(t) = \cosh(at)$$

suppose if we declare and create a class like this
LaplaceTransform lt;
then it will create a **LaplaceTransform** class that can be used to compute the Laplace transform of generic functions or custom functions $f(t)$.

The functions related to this class are:

- **simpsonStep(const std::function<double(double)>& f, double a, double b, double ϵ, double $whole$, int $limit$)**
  this function computes the numerical integration with adaptive Simpson's rule. It is mainly used when we compute custom function, generic function has its own function to compute the Laplace transform.
- **integrate(const std::function<double(double)>& f, double a, double b, double ϵ) const**
  this function integrates the function f over interval $[a, b]$. It is used alongside **simpsonStep()** to compute custom function.
- **numericalTransform(const std::function<double(double)>& f, double s, double $upperBound$) const**
  To compute arbitrary custom functions, such as t^2e^{3t}, it computes integral from 0 to $upperBound$ (approximating infinity) of $f(t)e^{-st}$. The $upperBound$ is defined in **include/symintegral/dsolve.h**.s
- **constant(double s) const**
  To compute the Laplace transform with $f(t) = 1$.
- **exponential(double s, double a) const**
  To compute the Laplace transform with $f(t) = e^{at}$.
- **tPower(double s, int n) const**
  To compute the Laplace transform with $f(t) = t^n$, with n as a positive integer.
- **sine(double s, double a) const**
  To compute the Laplace transform with $f(t) = \sin(at)$.
- **cosine(double s, double a) const**
  To compute the Laplace transform with $f(t) = \cos(at)$.
- **sinh(double s, double a) const**
  To compute the Laplace transform with $f(t) = \sinh(at)$.
- **cosh(double s, double a) const**
  To compute the Laplace transform with $f(t) = \cosh(at)$.

SymbolicLaplaceTransform(const Symbolic &$f(t)$, const Symbolic &t, const Symbolic &s) this **Symbolic** function performs the Laplace transform for some elementary function $f(t)$ (such as $\sin(at), \cos(at), e^{at}, t^n, u_c(t), \delta(t-c)$). Our original approach using **.match()** is the correct way to handle pattern matching, but some need a **string** check (dirac and heaviside functions) to match the pattern.
We make some notes here about the codes in this function:

- Symbolic engines often struggle matching t^1 into t^c unless explicitly coded. A quick shortcut check handles $f(t) = t$ directly to avoid false negatives.
- The power rule for $t^p, p > -1, p \in \mathbb{R}$ and $t^n,\ n \in \mathbb{Z}^+$ checks if the matched power is a plain positive integer. If it is then the Laplace transform will return $\frac{n!}{s^{n+1}}$, otherwise it will return $\frac{\Gamma(p+1)}{s^{p+1}}$.
- We add structural checks for $e^{at}\cos(bt)$, $e^{at}\sin(bt)$, $\cos(bt)e^{at}$, and $\sin(bt)e^{at}$ for commutativity support. This handles edge cases where SymIntegration internally orders terms differently during input processing.
- Put the complex pattern first, for example t^ne^{at}, if a user inputs t^2e^{3t}, it gets caught immediately by case that is listed above. If they just input e^{3t}, then the case that is written above fails gracefully and drops down to the next case that handle this type of function.

- After we match the equation, we create an **if** clause with **df(...,t)==0** for variables extracted from matching sequences to ensure constants are completely time-independent.

SymbolicInverseLaplaceTransform(const Symbolic &F_s, const Symbolic &s, const Symbolic &t)
this **Symbolic** function performs the inverse Laplace transform for some functions $F(s)$, given that the pattern is already defined in **dsolve.cpp**.
The list of function $F(s)$ that can be computed with this function is

$$F(s) = \frac{1}{s}$$
$$F(s) = \frac{1}{s-a}$$
$$F(s) = \frac{n!}{s^{n+1}}$$
$$F(s) = \frac{a}{s^2+a^2}$$
$$F(s) = \frac{s}{s^2+a^2}$$
$$F(s) = \frac{a}{s^2-a^2}$$
$$F(s) = \frac{s}{s^2-a^2}$$
$$F(s) = \frac{b}{(s-a)^2+b^2}$$
$$F(s) = \frac{s-a}{(s-a)^2+b^2}$$

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Inverse Laplace
Transform ]# ./main

F(s) = (s-2)^(-1)        ->      f(t) = e^(2*t)
F(s) = s^(-1)    ->      f(t) = 1
F(s) = 720*s^(-7)        ->      f(t) = t^(6)
F(s) = s*(s^(2)+4)^(-1)          ->      f(t) = cos(2*t)
F(s) = s*(s^(2)-4)^(-1)          ->      f(t) = cosh(2*t)
F(s) = 2*(s^(2)+4)^(-1)          ->      f(t) = sin(2*t)
F(s) = 2*(s^(2)-4)^(-1)          ->      f(t) = sinh(2*t)
F(s) = 3*(s^(2)-2*s+10)^(-1)     ->      f(t) = e^t*sin(3*t)
F(s) = s*(s^(2)-12*s+117)^(-1)-6*(s^(2)-12*s+117)^(-1)   ->      f(t) = e^(6*t)*cos(9*t)

Time taken by function: 3680242 microseconds
```

Figure 3.100: *The inverse Laplace transform for several functions $F(s)$ (**SymIntegration/Examples/Differential Equations/Test SymIntegration Gegenbauer Polynomials/main.cpp**).*

VI. Systems of First Order Linear Equations

[SI*] **Basic Theory**

[SI*]

VII. Numerical Methods

[SI*] **The Euler or Tangent Line Method**

[SI*] **The Runge-Kutta Method**

VIII. Nonlinear Differential Equations

[SI*] **The Phase Plane**

IX. Partial Differential Equations

[SI*] Partial differential equations (PDEs) are multivariate differential equations where derivatives of more than one dependent variable occur. Conceptually, the difference between ordinary and partial differential equations is not that big, but the computational techniques required to deal with ODEs and PDEs are very different, and solving PDEs is typically much more computationally demanding.

Most techniques for solving PDEs numeically are based on the idea of discretizing the problem in each independent variable that occurs in the PDE, thereby u the problem into an algebraic form. This usually results in very large-scale linear algebra problems.

The solution of partial differential equations (PDEs), both analytical and numerical, is the single, most important topic in applied analysis. Virtually all areas of science and engineering phrase problems in the language of PDEs. Because PDEs are concerned with differential equations in two or more independent variables, it is more common to encounter computational problems. Typically, for PDEs the memory and computation time requirements are increased by an order of magnitude, round-off error is more likely a concern, pathologies in the solution are more frequent, and, overall, the calculation is likely to be more complicated, bigger, and more expensive [6].

Two common techniques for recasting PDEs into algebraic form are the finite-difference methods (FDMs), where the derivatives in the problem are approximated with their finite-difference formula, and the finite-element methods (FEMs), where the unknown function is written as linear combination of simple basis functions thatn can be differentiated and integrated easily. The unknown function is described by a set of coefficients for the basis functions in this representation, and by a suitable rewriting of the PDEs, we can obtain algebraic equations for these coefficients.

[SI*] **The Standard Form**

The standard form of a second-order, linear, partial differential equation in tow independent variables is

$$A\frac{\partial^2 u}{\partial x^2} + 2B\frac{\partial^2 u}{\partial x \partial y} + C\frac{\partial^2 u}{\partial y^2} = F\left(x, y, u, \frac{\partial u}{\partial x}, \frac{\partial u}{\partial y}\right)$$

where $u(x, y)$ is the dependent variable, and $F\left(x, y, u, \frac{\partial u}{\partial x}, \frac{\partial u}{\partial y}\right)$ is an arbitrary function of x and y, and is linear in u and its first derivatives.

		A	B	C	F
$\frac{\partial^2 u}{\partial x^2} = \frac{1}{c^2}\frac{\partial^2 u}{\partial t^2}$	One-dimensional wave equation	1	0	$-\frac{1}{c^2}$	0
$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}$	Two-dimensional Laplace's equation	1	0	1	0
$F\frac{\partial^2 T}{\partial x^2} = -\frac{\partial T}{\partial t}$	One-dimensional diffusion equation	D	0	0	$-\frac{\partial T}{\partial t}$

Table 3.10: *Examples of homogeneous second-order PDEs.*

[SI*] **The Boundary Conditions**

To fully specify a particular solution to a PDE, we need to define its boundary conditions, which are known values of the function or a combination of its derivatives along the boundary of the problem domain Ω, as well as the initial values if the problem is time-dependent. The

boundary is often denoted as Γ or $\partial\Omega$, and in general different boundary conditions can be given for different parts of the boundary.
As with ordinary differential equations, the method of solution of a PDE is dictated by the types of boundary conditions specified in the problem. For ODEs, the choices are either initial value problems or boundary value problems. In either case, boundary values are required to guarantee that the equation has a unique and stable solution. In initial value problems, sufficient information regarding the function and its derivatives are given at a single point. Then, using finite difference expressions of the form

$$y_{i+1} = y_i + \frac{dy}{dx}\bigg|_i \triangle x + \frac{1}{2}\frac{d^2y}{dx^2}\bigg|_i (\triangle x)^2 + \dots$$

the solution is constructed in the neighborhood of the point, in a stepping manner. (The initial conditions specify the function and its first derivatives, and the differential equation is used to determine higher-order derivatives in the series.) Thus, the solution is constructed emanating from the initial point, and the natural method of solution is a stepping procedure. Methods to obtain numerical solutions to boundary value problems differ substantially from those used for initial value problems. Procedure based on finite difference equations seek to iteratively refine an initial guess for the solution to the actual solution simultaneously at all points on the grid. This usually leads to solutions of a large set of simultaneous linear equations. Three important types of boundary conditions are:

- **Dirichlet boundary conditions**
  which specify the value of the function at the boundary $u(\boldsymbol{x}) = h(\boldsymbol{x})$ for $\boldsymbol{x} \in \Gamma_D$, $h(\boldsymbol{x})$ is an arbitrary function.
- **Von Neumann boundary conditions**
  which specify the normal derivative on the boundary, $\frac{\partial u(\boldsymbol{x})}{\partial \boldsymbol{n}} = g(\boldsymbol{x})$ for $\boldsymbol{x} \in \Gamma_N$, where $\boldsymbol{n}$ is the outward normal from the boundary, $g(\boldsymbol{x})$ is an arbitrary function.
- **Cauchy boundary conditions**
  which specify the value of both the function and its normal derivative on the same boundary.

i. Simulate Three-Dimensional Heat Equation with SFML

In this case we choose to use SFML 2.5.1 that is already being installed in GFreya OS to create the heat equation simulation with SymIntegration, and even if it is inherently a 2D library, rendering a 3D mesh is most efficiently done using **SF::VertexArray** with an orthographic or weak perspective projection. This bypasses raw OpenGL configuration issues and runs exceptionally fast in real time. We create this 3D simulation by combining two distinct domains: continuum physics to model how heat spreads over time, and linear algebra/geometry to project three-dimensional surfaces onto a two-dimensional computer screen.

1. **The Governing Physics: The 2D Heat Equation**
   The heat equation is a classic parabolic partial differential equation (PDE) that models how thermal energy difuses through a medium. In a two-dimensional space without internal heat generation, the equation is expressed as

   $$\frac{\partial u}{\partial t} = \alpha \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right)$$

   with

   - $u(x, y, t)$ as the temperature at spatial coordinates (x, y) at time t.
   - $\frac{\partial u}{\partial t}$ as the rate of change of temperature over time at a specific point.
   - α as the thermal diffusivity constant, it dictates how fast heat flows through the material.
   - $\nabla^2 u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}$ as the spatial Laplacian operator in Cartesian coordinates. It measures the difference between the local temperature and the average temperature of the surrounding points. Heat flows away if the Laplacian is negative and inward if it is positive.

2. **Numerical Discretization: The Finite Difference Method (FDM)**
   Since computers cannot solve continuous calculus equations directly, we must discretize the continuous domain into a discrete grid of points spaced $\triangle x$ apart along space, and advance time in discrete steps of size $\triangle t$. We use index i for the x-direction, j for the y-direction, and n for the time step, thus:

   $$u_{i,j}^n \approx u(i\triangle x, j\triangle y, n\triangle t)$$

   Using Taylor series expansions, we approximate the continuous derivatives with differences between adjacent grid points.

   Spatial Derivatives (Central Difference)
   To find the second derivative in space, we look at the points to the left, right, top, and bottom of the current node:

   $$\frac{\partial^2 u}{\partial x^2} \approx \frac{u_{i+1,j}^n - 2u_{i,j}^n + u_{i-1,j}^n}{\triangle x^2}$$

   $$\frac{\partial^2 u}{\partial y^2} \approx \frac{u_{i,j+1}^n - 2u_{i,j}^n + u_{i,j-1}^n}{\triangle y^2}$$

   Since our grid are perfectly square, we set $\triangle x = \triangle y$. Combining them gives the discrete 2D Laplacian

   $$\nabla^2 u \approx \frac{u_{i+1,j}^n + u_{i-1,j}^n + u_{i,j+1}^n + u_{i,j-1}^n - 4u_{i,j}^n}{\triangle x^2}$$

Time Derivative (Forward Euler)
To step forward into the future, we approximate the time derivative using a forward difference:

$$\frac{\partial u}{\partial t} \approx \frac{u_{i,j}^{n+1} - u_{i,j}^{n}}{\triangle t}$$

The Update Equation
Substituting these approximations back into our original heat equation yields the final numerical integration formula used in our simulation loop:

$$\frac{u_{i,j}^{n+1} - u_{i,j}^{n}}{\triangle t} = \alpha \left(\frac{u_{i+1,j}^{n} + u_{i-1,j}^{n} + u_{i,j+1}^{n} + u_{i,j-1}^{n} - 4u_{i,j}^{n}}{\triangle x^2} \right)$$

Solving explicitly for the future state $u_{i,j}^{n+1}$ we will have

$$u_{i,j}^{n+1} = u_{i,j}^{n} + \triangle t \cdot \alpha \left(\frac{u_{i+1,j}^{n} + u_{i-1,j}^{n} + u_{i,j+1}^{n} + u_{i,j-1}^{n} - 4u_{i,j}^{n}}{\triangle x^2} \right)$$

3. **Stability Constraints: The CFL Condition**
The Forward Euler method is an explicit integration scheme. If the time step $\triangle t$ is selected too large, errors will cascade exponentially, resulting in an unphysical numerical explosion. To ensure stability, the parameters must strictly follow the Courant-Friedrichs-Lewy (CFL) condition for the 2D diffusion

$$\triangle t \leq \frac{\triangle x^2}{4\alpha}$$

In the source code, we guarantee safety by enforcing a low CFL constant:

$$\triangle t = 0.125 \cdot \frac{\triangle x^2}{\alpha}$$

4. **Boundary Initial Conditions**
We have several boundary conditions:

- **Dirichlet:** The outermost edges of the grid matrix are held constant at zero ($u = 0.0$). This models a plate whose outer perimeter is locked in contact with a freezing ice bath. It simulates the domain being placed inside an infinite ice bath kept strictly at 0^0. Heat will quickly drain out of the shape towards the boundaries until the entire grid eventually cools down to a flat, frozen sheet.
- **Neumann (Insulated Wall)**
The code for this boundary condition is $U[0][k] = U[1][k], U[k][0] = U[k][1]$. This sets the spatial derivative (temperature gradient) at the boundary to zero $\left(\frac{\partial U}{\partial n} = 0\right)$. It simulates a perfectly insulated container wrapped in thermos-like material. Because the boundary cells always copy the temperature of their immediate internal neighbors, heat cannot cross the border. No thermal energy escapes. If we inject heat into an insulated grid, the heat will bounce off the walls, redistribute itself, and eventually settle into a perfectly flat, uniform plateau representing the average temperature of the system.

5. **Shape**
We have several shapes that can be used to simulate the heat equation:

- **Crescent Moon:** The original mathematical pattern embedded inside the code evaluates a multi-term geometric equation across the spatial grid. If the value evaluates to less than zero, the temperature is clamped to maximm (2,0), generating a clean crescent profile:

$$Pattern(x,y) = \left(c^2 - \left(\frac{x}{e} - d\right)^2 - \left(\frac{y}{f}\right)^2\right) + k\left(c + d + \frac{x}{e}\right)^3 - shift$$

- **The Cylinder (Circle):** We use the standard Euclidean circle equation ($x^2 + y^2 < r^2$). Because it looks at the radial distance from the center, it generates a perfectly smooth, circular pillar of heat. When projected in 3D, it looks like a flat-topped cylinder.
The formula is

$$x^2 + y^2 < 1.0$$

- **The Box:** It checks the absolute distance from the origin (0,0) along both axes independently. If a point is withing 0.8 units left/right and 0.8 units up/down, it falls inside. This creates a sharp-edged square block of heat in the center of the domain.
The formula is

$$|x| < 0.8 \quad \& \quad |y| < 0.8$$

6. **Graphics Projection: 3D to 2D Screenspace Matrix**
To render a 3D visualization using SFML (which is strictly a 2D canvas), the code manually replicates a 3D perspective rotation matrix mapping spatial triplets (X, Y, Z) into 2D display coordinates (X_{screen}, Y_{screen}). The variable Z maps directly to the active temperature matrix $u_{i,j}$.
Given standard coordinate rotation formulas:

   (a) **Rotate around the Z-axis (Yaw / Left-Right rotation θ_z)**

$$\begin{aligned} x_1 &= x\cos(\theta_z) - y\sin(\theta_z) \\ y_1 &= x\sin(\theta_z) + y\cos(\theta_z) \end{aligned}$$

   (b) **Rotate around the X-axis (Pitch / Up-Down tilt θ_x)**
The calculated height coordinate z is scaled visually by a scale factor s_z

$$\begin{aligned} x_2 &= x_1 \\ y_2 &= y_1\cos(\theta_x) - (z \cdot s_z)\sin(\theta_x) \end{aligned}$$

   (c) **Screen Offset and Final Translation**
Finally, the points are multiplied by a global zoom scalar (S) and offset to place the grid origin $(0,0)$ exactly in the middle of the display window:

$$\begin{aligned} X_{screen} &= \frac{\text{Window Width}}{2} + x_2 \cdot S \\ Y_{screen} &= \frac{\text{Window Height}}{2} + y_2 \cdot S \end{aligned}$$

   By recalculating this projection for all points in real-time as θ_x and θ_z adjust to mouse drag movements, the grid structure creates a true-to-life 3D topology.

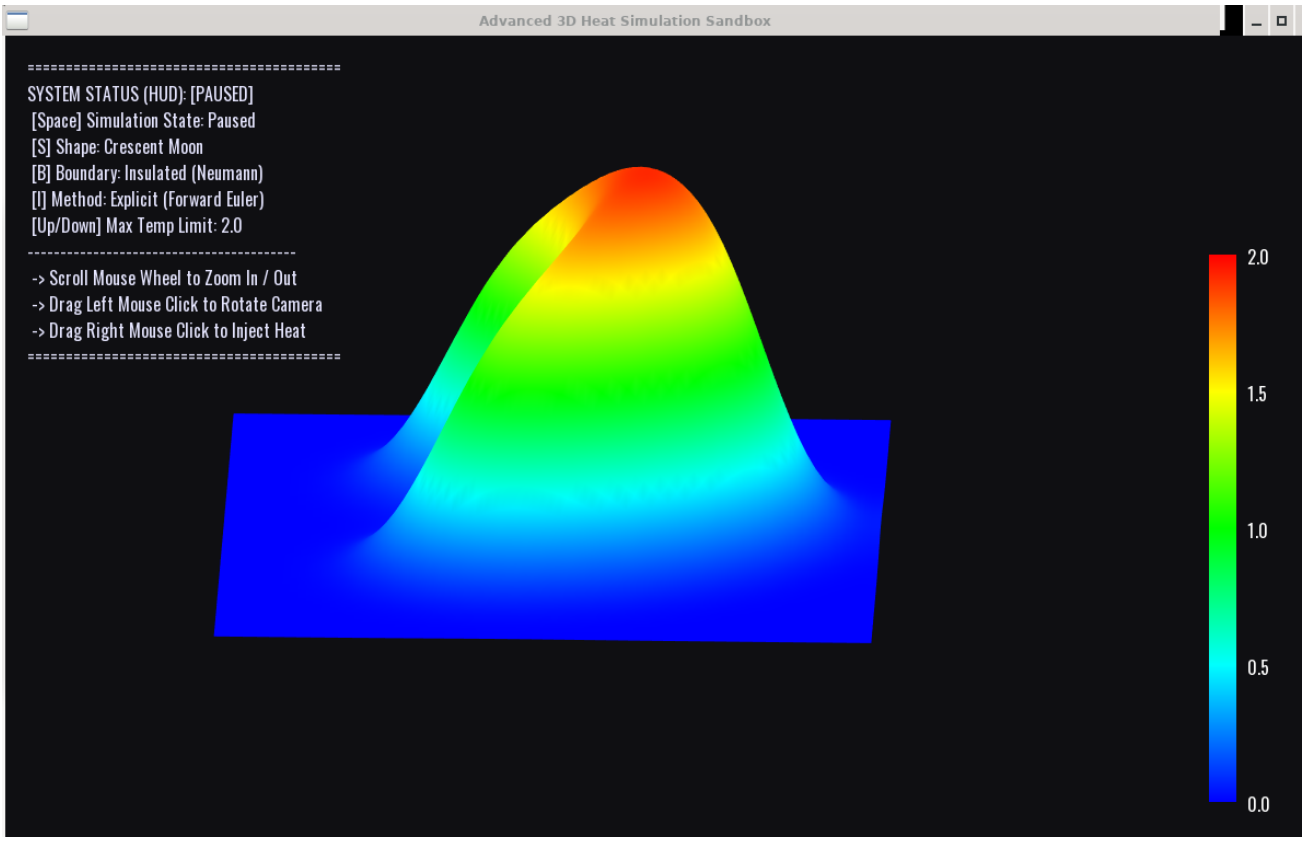

Figure 3.101: *The real-time 3D surface visualization of the 2D heat equation using SymIntegration and SFML 2.5.1, we simulate the physics on a 2D grid and project the points into a 3D perspective using basic matrix-free trigonometry (**SymIntegration/Examples/Statistics/Test SymIntegration Heat Equation with SFML/main.cpp**).*

X. Boundary Value Problems and Sturm-Liouville Theory

[SI*] **The Occurrence of Two-Point Boundary Value Problems**

[SI*] **Nonhomogeneous Boundary Value Problems**

Chapter 4

Probability and Statistics

*"Education is the best provision for old age." - **Aristotle***

At a glance, people might be thinking that probability is only numerical computation, the probability that a head will shows up on a coin that is tossed upward is around 0.5 on a fair coin. After more thorough reading, statistics and probability are actually more symbolic than numeric. We just substitute the random variable with a number that we want to test / to know the probability a certain event will occur is how much. That is the main reason, why we put statistics and probability computation in SymIntegration.

On August 23rd, 2025 we have created a new source code **statistics.cpp** that will handle the probability and statistics computation, like computing probability mass function of a certain discrete distribution, the moment generating function and to compute the mean and variance. It could come in handy one day.

I. Processing and Summarizing Data with Statistics

[SI*] Statistical methods are used to analyze data from an industrial process (manufacturing, energy sources, drug testing). Statistical methods are involved with gathering of information or **scientific data**, which then will be processed to extract the knowledge from the data and improve the **quality** of the process.

i. Descriptive Statistics Computation

Suppose we have a univariate data of a Mathematics examination result for 25 students, compute its' mean, median, mode, standard deviation, and variance.

Solution:

To compute the mean we will use this formula:

$$\bar{x} = \frac{\sum x}{n} \tag{4.1}$$

Mode is the value that occur frequently, and Median is the middle value after the data is sorted, if the size of the data is even, then the median is the average of the two middle data.

Standard deviation can be computed with this formula:

$$S_x = \sqrt{\frac{\sum x^2}{n} - (\bar{x})^2} \tag{4.2}$$

Standard deviation is the square root of variance.

In SymIntegration there is a function that can be used to compute the standard descriptive statistics, it is:
double descriptivestatistics(vector<double>)
the input is a vector with double as the data type, since it can also represents integer.

We have saved the grade data in a textfile named **vectorx.txt**, we can just load / call it from the **main.cpp** file.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Load Vector from textfile and Compute Descriptive Statistics ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Load Vector from textfile and Compute Descriptive Statistics ]# ./main

Statistics Descriptive
Mode : 6
Median : 6
Mean : 6.2
Quantile 1/4: 5
Quantile 3/4: 8
Variance : 6.41667
Standard deviation : 2.53311

Time taken by function: 833 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Load Vector from textfile and Compute Descriptive Statistics ]#
```

Figure 4.1: *The computation of descriptive statistics with SymIntegration took 833 microseconds (**SymIntegration/Examples/Statistics/Test SymIntegration Load Vector from textfile and Compute Descriptive Statistics/main.cpp**).*

II. Random Variables and Probability Distributions

Definition 4.1: Random Variable

A random variable is a function that associates a real number with each element in the sample space.

We shall use a capital letter, say X, to denote a random variable and its corresponding small letter, x in this case, for one of its values.

Definition 4.2: Discrete Sample Space

If a sample space contains a finite number of possibilities or an unending sequence with as many elemnts as there are whole numbers, it is called a discrete sample space.

Definition 4.3: Continuous Sample Space

If a sample space contains an infinite number of possibilities equal to the number of points on a line segment, it is called a continuous sample space.

i. Discrete Probability Distributions

Definition 4.4: Probability Mass Function

If X is discrete, the probability mass function of X is defined as

$$f(x) = P(X = x), \quad \forall x \in \mathbb{R} \tag{4.3}$$

the probability is concentrated at only discrete values in the range of X, and is zero for all other values. f must also obey the basic rules of probability. That is, f must be non-negative

$$f(x) \geq 0$$

and the sum of all probabilities should add to 1

$$\sum_x f(x) = 1$$

Definition 4.5: Discrete Cumulative Distribution Function

The cumulative distribution function $F(x)$ of a discrete random variable X with probability distribution $f(x)$ iss

$$F(x) = P(X \leq x) = \sum_{t \leq x} f(t), \quad \text{for } -\infty < x < \infty \tag{4.4}$$

ii. Continuous Probability Distributions

Definition 4.6: Probability Density Function

If X is continuous, its range is the entire set of real numbers $\mathbb{R}$. The probability of any specific value x is only one out of the infinitely many possible values in the range of X, which means that

$$P(X = x) = 0$$

for all $x \in \mathbb{R}$. The probability mass is spread so thinly over the range of values, that it can be measured only over intervals $[a, b] \subset \mathbb{R}$, rather than at specific points.
the probability density function of X that takes on values in any interval $[a, b] \subset \mathbb{R}$ is defined as

$$P(X \in [a, b]) = \int_a^b f(x)\, dx \tag{4.5}$$

the density function f must satisfy the basic laws of probability

$$f(x) \geq 0, \quad \forall x \in \mathbb{R}$$

and

$$\int_{-\infty}^{\infty} f(x)\, dx = 1$$

Definition 4.7: Continuous Cumulative Distribution Function

The function $f(x)$ is a probability density function (pdf) for the continuous random variable X, defined over the set of real numbers, if

$$f(x) \geq 0, \quad \forall x \in \mathbb{R} \tag{4.6}$$

$$\int_{-\infty}^{\infty} f(x)\, dx = 1 \tag{4.7}$$

$$P(a < X < b) = \int_a^b f(x)\, dx \tag{4.8}$$

Definition 4.8: Cumulative Distribution Function

For any random variable X, whether discrete or continuous, we can define the cumulative distribution function (cdf) as

$$F : \mathbb{R} \to [0, 1]$$

that gives the probability of observing a value at most some given value x

$$F(x) = P(X \leq x), \quad \forall -\infty < x < \infty \tag{4.9}$$

when X is discrete, F is given as

$$F(x) = P(X \leq x) = \sum_{u \leq x} f(u) \tag{4.10}$$

and when X is continuous, F is given as

$$F(x) = P(X \leq x) = \int_{-\infty}^{x} f(u)\, du \tag{4.11}$$

iii. Joint Probability Distributions

Definition 4.9: Joint Probability Distribution

The function $f(x, y)$ is a joint probability distribution or probability mass function of the discrete random variables X and Y if

$$f(x, y) \geq 0, \quad \forall\ (x, y) \tag{4.12}$$

$$\sum_{x} \sum_{y} f(x, y) = 1 \tag{4.13}$$

$$P(X = x, Y = y) = f(x, y) \tag{4.14}$$

For any region A in the xy plane,

$$P[(X, Y) \in A] = \sum \sum_{A} f(x, y) \tag{4.15}$$

Definition 4.10: Joint Density Function

The function $f(x,y)$ is a joint density function of the continuous random variables X and Y if

$$f(x,y) \geq 0, \quad \forall\ (x,y) \tag{4.16}$$

$f(x,y)$ is a surface lying above the xy plane, and $P[(X,Y) \in A]$, where A is any region in the xy plane. The joint density function is equal to the volume of the right cylinder bounded by the base A and the surface.

$$\int_{-\infty}^{\infty}\int_{-\infty}^{\infty} f(x,y)\ dx\ dy = 1 \tag{4.17}$$

$$P[(X,Y)\infty A] = \int\int_{A} f(x,y)\ dx\ dy \tag{4.18}$$

for any region A in the xy plane.

Definition 4.11: Marginal Distributions

Given the joint probability distribution $f(x,y)$ of the discrete random variables X and Y, the probability distribution $g(x)$ of X alone is obtained by summing $f(x,y)$ over the values of Y. Similarly, the probability distribution $h(y)$ of Y alone is obtained by summing $f(x,y)$ over the values of X.

We define $g(x)$ and $h(y)$ to be the marginal distributions of X and Y, respectively. When X and Y are continuous random variables, summations are replaced by integrals.

The marginal distributions of X alone and Y alone are

$$\begin{aligned} g(x) &= \sum_{y} f(x,y) \\ h(y) &= \sum_{x} f(x,y) \end{aligned} \tag{4.19}$$

for the discrete case, and

$$\begin{aligned} g(x) &= \int_{-\infty}^{\infty} f(x,y)\ dy \\ h(y) &= \int_{-\infty}^{\infty} f(x,y)\ dx \end{aligned} \tag{4.20}$$

for the continuous case.

Definition 4.12: Conditional Distribution

Let X and Y be two random variables, discrete or continuous. The conditional distribution of the random variable Y given that $X = x$ is

$$f(y|x) = \frac{f(x,y)}{g(x)} \tag{4.21}$$

provided $g(x) > 0$.
Similarly, the conditional distribution of X given that $Y = y$ is

$$f(x|y) = \frac{f(x,y)}{h(y)} \tag{4.22}$$

provided $h(y) > 0$.

If we wish to find the probability that the discrete random variable X falls between a and b when it is known that the discrete variable $Y = y$, we evaluate

$$P(a < X < b|Y = y) = \sum_{a<x<b} f(x|y) \tag{4.23}$$

where the summation extends over all values of X between a and b. When X and Y are continuous, we evaluate

$$P(a < X < b|Y = y) = \int_a^b f(x|y)\, dx \tag{4.24}$$

Definition 4.13: Statistically Independent

Let X and Y be two random variables, discrete or continuous, with joint probability distribution $f(x,y)$ and marginal distributions $g(x)$ and $h(y)$, respectively. The random variables X and Y are said to be statistically independent if and only if

$$f(x,y) = g(x)h(y) \tag{4.25}$$

for all (x,y) within their range.

Definition 4.14: Mutually Statistically Independent

Let $X_1, X_2, \ldots, X_n$ be n random variables, discrete or continuous, with joint probability distribution $f(x_1, x_2, \ldots, x_n)$ and marginal distribution $f_1(x_1), f_2(x_2), \ldots, f_n(x_n)$, respectively. The random variables $X_1, X_2, \ldots, X_n$ are said to be mutually statistically independent if and only if

$$f(x_1, x_2, \ldots, x_n) = f_1(x_1)f_2(x_2)\ldots f_n(x_n) \tag{4.26}$$

for all $(x_1, x_2, \ldots, x_n)$ within their range.

Definition 4.15: Mean

Let X be a random variable with probability distribution $f(x)$. The mean, or expected value, of X is

$$\mu = E(X) = \sum_x xf(x) \tag{4.27}$$

if X is discrete, and

$$\mu = E(X) = \int_{-\infty}^{\infty} xf(x)\,dx \tag{4.28}$$

if X is continuous.

Theorem 4.1: Expected Value of a Random Variable $g(X)$

Let X be a random variable with probability distribution $f(x)$. The expected value of the random variable $g(X)$ is

$$\mu_{g(X)} = E[g(X)] = \sum_x g(x)f(x) \tag{4.29}$$

if X is discrete, and

$$\mu_{g(X)} = E[g(X)] = \int_{-\infty}^{\infty} g(x)f(x)\,dx \tag{4.30}$$

if X is continuous.

Definition 4.16: Mean for Two Random Variables

Let X and Y be random variables with joint probability distribution $f(x,y)$. The mean, or expected value, of the random variable $g(X,Y)$ is

$$\mu_{g(X,Y)} = E[g(X,Y)] = \sum_x \sum_y g(x,y)f(x,y) \tag{4.31}$$

if X and Y are discrete, and

$$\mu_{g(X,Y)} = E[g(X,Y)] = \int_{-\infty}^{\infty}\int_{-\infty}^{\infty} g(x,y)f(x,y)\,dx\,dy \tag{4.32}$$

if X and Y are continuous.

Definition 4.17: Variance

Let X be a random variable with probability distribution $f(x)$ and mean μ. The variance of X is

$$\sigma^2 = E[(X-\mu)^2] = \sum_x (x-\mu)^2 f(x) \tag{4.33}$$

if X is discrete, and

$$\sigma^2 = E[(X-\mu)^2] = \int_{-\infty}^{\infty} (x-\mu)^2 f(x)\, dx \tag{4.34}$$

if X is continuous.

The positive square root of the variance, σ, is called the standard deviation of X. The quantity $x-\mu$ is called the deviation of an obsevation from its mean.

Theorem 4.2: Variance

The variance of a random variable X is

$$\sigma^2 = E(X^2) - \mu^2 \tag{4.35}$$

Theorem 4.3: Variance of Random Variable $g(X)$

Let X be a random variable with probability distribution $f(x)$. The variance of the random variable $g(X)$ is

$$\sigma^2_{g(X)} = E\{[g(X) - \mu_{g(X)}]^2\} = \sum_x [g(X) - \mu_{g(X)}]^2 f(x) \tag{4.36}$$

if X is discrete, and

$$\sigma^2_{g(X)} = E\{[g(X) - \mu_{g(X)}]^2\} = \int_{-\infty}^{\infty} [g(X) - \mu_{g(X)}]^2 f(x)\, dx \tag{4.37}$$

Definition 4.18: Covariance

Let X and Y be random variables with joint probability distribution $f(x,y)$. The covariance of X and Y is

$$\sigma_{XY} = E[(X-\mu_X)(Y-\mu_Y)] = \sum_x \sum_y (x-\mu_X)(y-\mu_Y) f(x,y) \tag{4.38}$$

if X and Y are discrete, and

$$\sigma_{XY} = E[(X-\mu_X)(Y-\mu_Y)] = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} (x-\mu_X)(y-\mu_Y) f(x,y)\, dx\, dy \tag{4.39}$$

if X and Y are continuous.

The covariance between two random variables is a measure of the nature of the association between the two.

If large values of X often result in large values of Y or small values of X result in small values of Y, positive $X-\mu_X$ will often result in positive $Y-\mu_Y$ and negative $X-\mu_X$ will often result in negative $Y-\mu_Y$.

Thus, the product $(X-\mu_X)(Y-\mu_Y)$ will tend to be positive. On the other hand, if large X values often result in small Y values, the product $(X-\mu_X)(Y-\mu_Y)$ will tend to be negative.

When X and Y are statistically independent, it can be shown that the covariance is zero. The converse, however, is not generally true. Two variables may have zero covariance and still not be statistically independent. Note that the covariance only describes the linear relationship between two random variables. Therefore, if a covariance between X and Y is zero, X and Y may have a nonlinear relationship, which means that they are not necessarily independent.

Theorem 4.4: Covariance

The covariance of two random variables X and Y with means μ_X and μ_Y, respectively, is given by

$$\sigma_{XY} = E(XY) - \mu_X \mu_Y \tag{4.40}$$

Definition 4.19: Correlation

Let X and Y be random variables with covariance σ_{XY} and standard deviations σ_X and σ_Y, respectively. The correlation coefficient of X and Y is

$$\rho_{XY} = \frac{\sigma_{XY}}{\sigma_X \sigma_Y} \tag{4.41}$$

it should be clear that ρ_{XY} is free of the units of X and Y. The correlation coefficient satisfies the inequality $-1 \leq \rho_{XY} \leq 1$. It assumes a value of zero when $\sigma_{XY} = 0$.

iv. Covariance and Vectorized Moments

Given two random variables, X and Y, with respective means, μ_X and μ_Y, the covariance is defined by

$$\text{Cov}(X, Y) = E[(X - \mu_X)(Y - \mu_Y)] = E[XY] - \mu_x\mu_y$$

The second formula follows by expansion. Notice also that

$$\text{Cov}(X, X) = \text{Var}(X)$$

the covariance is a common measure of the relationship between the two random variables, where if $\text{Cov}(X, Y) = 0$, we say the random variables are uncorrelated. Furthermore, if $\text{Cov}(X, Y) \neq 0$, the sign of the covariance gives an indication of the direction of the relationship.

Another important concept is the correlation coefficient,

$$\rho_{XY} = \frac{\text{Cov}(X, Y)}{\sqrt{\text{Var}(X)\text{Var}(Y)}} \tag{4.42}$$

It is a normalized form of the covariance with $-1 \leq \rho_{XY} \leq 1$. Values nearing ± 1 indicate a very strong linear relationship between X and Y, whereas values near or at 0 indicate a lack of a linear relationship.

Note that if X and Y are independent random variables, then $\text{Cov}(X, Y) = 0$ and hence $\rho_{XY} = 0$. However, the opposite case does not always hold, since, in general $\rho_{XY} = 0$ does not imply independence. Nevertheless for jointly normal random variables it does.

Consider now a random vector $\boldsymbol{X} = (X_1, \ldots, X_n)$, taken as a column vector. It can be described by moments in an analogous manner to a scalar random variable. A key quantity is the mean vector,

$$\mu_X := [E[X_1], E[x_2], \ldots, E[X_n]]^T$$

Furthermore, the covariance matrix is the matrix defined by the expectation (taken element-wise) of the (outer product) random matrix given by $(X - \mu_X)(X - \mu_X)^T$, and is expressed as

$$\Sigma_{\boldsymbol{X}} = \text{Cov}(X) = E[(X - \mu_x)(X - \mu_x)^T] \tag{4.43}$$

As can be verified, the i, j-th element of $\Sigma_{\boldsymbol{X}}$ is $\text{Cov}(X_i, X_j)$ and hence the diagonal elements are the variances.

The covariance matrix is a fundamental concept in statistics and data analysis, providing insight into the relationships between multiple variables. It captures the extent to which two variables change together and is essential for the various applications such as the Principal Component Analysis (PCA), financial analysis and multivariate statistics. By understanding and interpreting the covariance matrix, we can make more informed decisions and uncover deeper patterns in data.

v. Linear Combinations and Transformations

We now consider linear transformations applied to random vectors. For any collection of random variables,

$$E[X_1 + \cdots + x_n] = E[X_1] + \cdots + E[X_n]$$

For uncorrelated random variables,

$$\text{Var}(X_1 + \cdots + X_n) = \text{Var}(X_1) + \cdots + \text{Var}(X_n)$$

More generally, if we allow the random variables to be correlated, then

$$\text{Var}(X_1 + \cdots + X_n) = \text{Var}(X_1) + \cdots + \text{Var}(X_n) + 2\sum_{i<j} \text{Cov}(X_i, X_j) \tag{4.44}$$

Note that the right-hand side of Eq. (4.51) is the sum of the elements of the matrix $\text{Cov}(X_1, \ldots, X_n)$. this is a special case of a more general affine transformation, where we take a random vector $\boldsymbol{X} = (x_1, \ldots, X_n)$ with covariance matrix $\sum_{\boldsymbol{X}}$, and an $m \times n$ matrix A and m vector $\boldsymbol{b}$. We then set

$$\boldsymbol{Y} = A\boldsymbol{X} + \boldsymbol{b} \tag{4.45}$$

In this case, the new random vector $\boldsymbol{Y}$ exhibits mean and covariance

$$\begin{aligned} E[\boldsymbol{Y}] &= AE[\boldsymbol{X}] + \boldsymbol{b} \\ \text{Cov}(Y) &= A\Sigma_{\boldsymbol{X}} A^T \end{aligned} \tag{4.46}$$

Now to retrieve Eq. (4.51), we set $1 \times n$ matrix $A = [1, \ldots, 1]$ and observe that $A\Sigma_{\boldsymbol{X}} A^T$ is a sum of all of the elements of $\Sigma_{\boldsymbol{X}}$.

vi. The Cholesky Decomposition and Generating Random Vectors

If you wish to create an n-dimensional random vector $\boldsymbol{Y}$ with some specified mean vector $\mu_{\boldsymbol{Y}}$ and covariance matrix $\sum_{\boldsymbol{Y}}$. That is, $\mu_{\boldsymbol{Y}}$ and $\sum_{\boldsymbol{Y}}$ are known.

The formulas in Eqs. (4.53) yield a potential recipe for such a task if we are given a random vector $\boldsymbol{X}$ with zero-mean and identity-covariance matrix ($\sum_{\boldsymbol{Y}} = I$). FOr example, in the context of Monte Carlo random variable generation, creating such a random vector $\boldsymbol{X}$ is trivial-just generate a sequence of n i.i.d. normal $(0, 1)$ random variables.

Now, apply the affine transformation Eq. (4.52) on $\boldsymbol{X}$ with $\boldsymbol{b} = \mu_{\boldsymbol{Y}}$ and a matrix A that satisfies

$$\sum_{\boldsymbol{Y}} = AA^T \tag{4.47}$$

Now Eq. (4.53) guarantees that $\boldsymbol{Y}$ has the desired $\mu_{\boldsymbol{Y}}$ and $\sum_{\boldsymbol{Y}}$.

vii. Generate Random Number in SymIntegration

Generating random number is the essence or basic need in statistical inference, it is very usefull as well if we want to simulate physics phenomena or forecasting weather with random initial condition.

In SymIntegration we have two functions to generate random number, the first one is:
randomnumberint(int a, int b, int n)
with a and b as the range so the number generated will be in $[a, b]$, and n is the amount of random number that we want to generate. This function will generate integer random number only.

The second function is:
randomnumberreal(double a, double b, int n)
with a and b as the range so the number generated will be in $[a, b]$, and n is the amount of random number that we want to generate. This function will generate real random number, in C++ we use **double** data type to represent the real number here.

The functions are written in **src/statistics.cpp** with the header that corresponds to this is in **/include/symintegral/statistics.h**. We are using uniform distribution to generate the random number to ensure that the probability of each number in the range to be picked is the same, equal opportunity for all.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Generate Random Number from Function ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Generate Random Number from Function ]# ./main
Random integer:
19
129
123
133
93
58
173
82
88
28
173
Random double:
1.06899
0.683921
4.21156
3.94917
2.27306
0.971918
4.0864
3.09706
2.44007
3.7999

Time taken by function: 759 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Generate Random Number from Function ]#
```

Figure 4.2: *The random number generation with SymIntegration took 759 microseconds (**SymIntegration/Examples/Statistics/Test SymIntegration Generate Random Number from Function/main.cpp**).*

We are using the standard C++ code **std::random_device rd** as a uniform random bit generator that is intended to produce non-deterministic random numbers, typically by accessing a hardware entropy source or a system-provided source of randomness. Its purpose is to provide a high-quality, truly random seed for other pseudo-random number generators. the **rd** is an object of type **std::random_device**.

Then, we also use **std::mt19937 generate** as the Mersenne Twister engine, a widely used and well-regarded pseudo-random number generator (PRNG). It produces a long sequence of pseudo-random numbers based on an initial seed. The **generate** is an object of type **std::mt19937**,

the **rd()** part calls the **operator()** of the **std::random_device** object, which returns a single random value. This value is then used as the seed to initialize the **std::mt19937** engine **generate**.

std::random_device provides a non-deterministic seed, ensuring that each run of the program will likely produce a different sequence of random numbers. This seed then initializes **std::mt19937**, which is a high-quality and efficient pseudo-random number generator suitable for most non-cryptographic applications. Subsequent calls to **generate()** or distributions associate with **generate** will produce the pseudo-random numbers.

viii. Create Covariance Matrix in SymIntegration

The general form of a covariance matrix is as follow:

$$\Sigma_X = \begin{bmatrix} \sigma_{11} & \sigma_{12} & \dots & \sigma_{1n} \\ \sigma_{21} & \sigma_{22} & \dots & \sigma_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_{n1} & \sigma_{n2} & \dots & \sigma_{nn} \end{bmatrix} = \begin{bmatrix} \mathrm{Cov}(X_1, X_1) & \mathrm{Cov}(X_1, X_2) & \dots & \mathrm{Cov}(X_1, X_n) \\ \mathrm{Cov}(X_2, X_1) & \mathrm{Cov}(X_2, X_2) & \dots & \mathrm{Cov}(X_2, X_n) \\ \vdots & \vdots & \ddots & \vdots \\ \mathrm{Cov}(X_n, X_1) & \mathrm{Cov}(X_n, X_2) & \dots & \mathrm{Cov}(X_n, X_n) \end{bmatrix}$$

we can tell that covariance matrix is a symmetric matrix with the property of $a_{ij} = a_{ji}$. The formula to compute σ_{ij} can be seen at Eq. (4.47).

For sample covariance the formula is as follow:

$$\sigma_{xy} = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{n-1}$$

Now suppose we have a mid-term exam results by Sine, Sweden, and Bludut in Linear Algebra and Differential Equations. Make a covariance matrix.

Student	Linear Algebra	Differential Equations
Sine	80	70
Sweden	63	20
Bludut	100	50

Table 4.1: *The mid-term exam results.*

With SymIntegration we can create the covariance matrix with this function:
covariancematrix(vector<vector<double>>)

The output will be a **SymbolicMatrix** or **Matrix<Symbolic>**, a class that is originally made from SymbolicC++. The goodness of this SymbolicMatrix is that it can also contain symbolic input.

The result will be show that

$$\mathrm{Cov}(X, X) = \sigma_{11} = 343$$
$$\mathrm{Cov}(X, Y) = \sigma_{12} = 260$$
$$\mathrm{Cov}(Y, Y) = \sigma_{22} = 633.33$$

the covariance for X (the Linear Algebra mid-term score) and Y (the Differential Equations mid-term score) is 260, as this is a positive number, it means that when X increases (or decreases) Y also increases (or decreases).

While the diagonal elements are 343 and 633.33 indicate the variance in data sets X and Y respectively. X shows the lowest variance, while Y shows the highest variance.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Create Covariance Matrix from Textfile ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Create Covariance Matrix from Textfile ]# ./main
Data:
        80        70
        63        20
       100        50

The covariance matrix:
[  343     260  ]
[  260   633.333]


Time taken by function: 1265 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Create Covariance Matrix from Textfile ]#
```

Figure 4.3: *The covariance matrix for this example case. (**SymIntegration/Examples/Statistics/Test SymIntegration Create Covariance Matrix from Textfile/main.cpp**).*

ix. Create Cholesky Decomposition in SymIntegration

Perform a Cholesky decomposition on

$$\Sigma_X = \begin{bmatrix} 4 & 12 & -16 \\ 12 & 37 & -43 \\ -16 & -43 & 98 \end{bmatrix}$$

We will use this formula

$$A_{ii} = \sqrt{\Sigma_{X_{ii}} - \sum_{k=0}^{i-1} (A_{ik})^2}$$

The result is

$$\begin{bmatrix} 4 & 12 & -16 \\ 12 & 37 & -43 \\ -16 & -43 & 98 \end{bmatrix} = \begin{bmatrix} 2 & 0 & 0 \\ 6 & 1 & 0 \\ -8 & 5 & 3 \end{bmatrix} \begin{bmatrix} 2 & 6 & -8 \\ 0 & 1 & 5 \\ 0 & 0 & 3 \end{bmatrix}$$

In SymIntegration the Cholesky decomposition can be performed with this function: **choleskyDecomposition(vector<vector<double>> matrix)**

The function is **void**, the goodness of using **void** is we don't need to have **return ..** at the end of the function, and we can call the result / the output after we call the function.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Cholesky Decomposition ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Cholesky Decomposition ]# ./main
Data:
         4        12       -16
        12        37       -43
       -16       -43        98

A =
         2         0         0
         6         1         0
        -8         5         3


A^{T} =
         2         6        -8
         0         1         5
         0         0         3

Time taken by function: 889 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Cholesky Decomposition ]#
```

Figure 4.4: *The Cholesky decomposition for a square matrix of size* 3*, we load the matrix from textfile* ***matrix.txt****.* ***(SymIntegration/Examples/Statistics/Test SymIntegration Cholesky Decomposition/main.cpp)****.*

III. Discrete Probability Distributions

[SI*] We will start with the Bernoulli process, then we will cover the definition of other discrete distributions.

Definition 4.20: Bernoulli Process

An experiment that consists of repeated trials, each with two possible outcomes that may be labeled success or failure is called a Bernoulli process.

The Bernoulli process must possess the following properties:

1. The experiment consists of repeated trials.
2. Each trial results in an outcome that may be classified as a success or a failure.
3. The probability of success, denoted by p, remains constant from trial to trial.
4. The repeated trials are independent.

Definition 4.21: Binomial Distribution

The number X of successes in n Bernoulli trials is called a binomial random variable. The probability distribution of this discrete random variable is called the binomial distribution, and its values will be denoted by $b(x; n, p)$.

A Bernoulli trial can result in a success with probability p and a failure with probability $q = 1 - p$. Then the probability distribution of the binomial random variable X, the number of successes in n independent trials, is

$$b(x; n, p) = \binom{n}{x} p^x (q^{n-x}), \quad x = 0, 1, 2, \ldots, n \tag{4.48}$$

Theorem 4.5: Mean and Variance of the Binomial Distribution

The mean and variance of the binomial distribution $b(x; n, p)$ are

$$\mu = np \tag{4.49}$$

$$\sigma^2 = npq \tag{4.50}$$

Definition 4.22: Negative Binomial Distribution

The number X of trials required to produce k successes in a negative binomial experiment is called a negative binomial random variable, and its probability distribution is called the negative binomial distribution.

If repeated independent trials can result in a success with probability p and a failure with probability $q = 1 - p$, then the probability distribution of the random variable, X, the number of the trial on which the k-th success occurs, is

$$b^*(x; k, p) = \binom{x-1}{k-1} p^k q^{x-k}, \quad x = k, k+1, k+2, \ldots \tag{4.51}$$

Definition 4.23: Geometric Distribution

If repeated independent trials can result in a success with probability p and a failure with probability $q = 1 - p$, then the probability distribution of the random variable X, the number of the trial on which the first success occurs, is

$$g(x; p) = pq^{x-1}, \quad x = 1, 2, 3, \ldots \tag{4.52}$$

Theorem 4.6: The Mean and Variance of the Geometric Distribution

The mean and variance of a random variable following the geometric distribution are

$$\mu = \frac{1}{p} \tag{4.53}$$

$$\sigma^2 = \frac{1-p}{p^2} \tag{4.54}$$

Definition 4.24: Poisson Process

Experiments yielding numerical values of a random variable X, the number of outcomes occurring during a given time interval or in a specified region, are called Poisson experiments. A Poisson experiment can generate observations for the random variable X representing the number of telephone calls received per hour by an office, the number of days school is closed due to snow during the winter, or the number of games postponed due to rain during a baseball season.

Properties of the Poisson process:

1. The number of outcomes occurring in one time interval or specified region of space is independent of the number that occur in any other disjoint time interval or region. In this sense we say that the Poisson process has no memory.
2. The probability that a single outcome will occur during a very short time interval or in a small region is proportional to the length of the time interval or the size of the region and does not depend on the number of outcomes occurring outside this time interval or region.
3. The probability that more than one outcome will occur in such a short time interval or fall in such a small region is negligible.

The number X of outcomes occurring during a Poisson experiment is called a Poisson random variable, and its probability distribution is called the Poisson distribution.

If you want to learn more on non-homogeneous Poisson process, it allows for a varying rate of events over time or space, and also the average rate (λ) can becomes a function of time or space.

Definition 4.25: Poisson Distribution

The probability distribution of the Poisson random variable X, representing the number of outcomes occurring in a given time interval or specified region denoted by t, is

$$p(x; \triangle t) = \frac{e^{-\lambda t}(\lambda t)^x}{x!}, \quad x = 0, 1, 2, \ldots \tag{4.55}$$

where λ is the average number of outcomes per unit time, distance, area, or volume and $e = 2.71828$.

Use cases of Poisson distribution:

1. Traffic flow: Modeling the number of cars passing through a toll booth in a given time period.
2. Call Centers: Predicting the number of incoming calls during specific hours.
3. Insurance Claims: Estimating the number of insurance claims within a certain timeframe.
4. Web Server Requests: Analyzing the number of requests a server receives in a fixed time interval.
5. Epidemiology: Studying the occurrence of diseases or rare events in a population.

Practical applications of Poisson distribution:

1. Network Security: Analyzing the number of security breaches or attacks on a network within a specific timeframe.
2. Inventory Management: Estimating the number of items sold in a store during a particular hour.
3. Quality Control: Assessing the number of defects in a manufacturing process.
4. Biology and Genetics: Studying the distribution of mutations in a DNA sequence.
5. Finance: Predicting the number of defaults in a loan portfolio.

Theorem 4.7: Mean and Variance of the Poisson Distribution

Both the mean and the variance of the Poisson distribution $p(x; \lambda t)$ are

$$\mu = \sigma^2 = \lambda t \tag{4.56}$$

where t is the specific "time", "distance", "area", or "volume" of interest.

Theorem 4.8: Approximation of Binomial Distribution by a Poisson Distribution

Let X be a binomial random variable with probability distribution $b(x;n,p)$. When $n \to \infty$, $p \to 0$, and $np \xrightarrow{n\to\infty} \mu$ remains constant,

$$b(x;n,p) \xrightarrow{n\to\infty} p(x;\mu) \tag{4.57}$$

i. Functions in SymIntegration Related to Discrete Distribution

[SI*] To compute the probability mass function, cumulative density function, mean, variance, and moment generating function for the discrete distributions above we can use these functions in SymIntegration:
binomialpmf(x,n,p)
binomialcdf(x,n,p)
binomialmean(x,n,p)
binomialvar(x,n,p)
binomialmgf(x,n,p)

negativebinomialpmf(x,k,p)
negativebinomialmean(x,k,p)
negativebinomialvar(x,k,p)
negativebinomialmgf(x,k,p)

geometricpmf(x,p)
geometricmean(x,p)
geometricvar(x,p)
geometricmgf(x,p)

poissonpmf(x,λt)
poissoncdf(x,λt)
poissonmean(x,λt)
poissonvar(x,λt)
poissonmgf(x,λts)

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Discrete Distributions ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb  main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Discrete Distributions ]# ./main

binomialpmf(2;4,0.75) = 0.210938

binomialcdf(8;15,0.4) = 0.904953

binomialmean(8;15,0.4) = 6

binomialvar(8;15,0.4) = 3.6

binomialmgf(8;15,0.4) = 0.4*e^t+0.6

negativebinomialpmf(6;4,0.55) = 0.1853

negativebinomialmean(6;4,0.55) = 7.27273

negativebinomialvar(6;4,0.55) = 5.95041

negativebinomialmgf(6;4,0.55) = 0.0915063*(-0.45*e^t+1)^(-4)

geometricpmf(5;0.01) = 0.00960596

geometricmean(5;0.01) = 100

geometricvar(5;0.01) = 9900

geometricmgf(5;0.01) = 0.01*(-0.99*e^t+1)^(-1)

poissonpmf(6;4) = 0.104196

poissoncdf(6;4) = 0.889326

poissonmean(6;4) = 4

poissonvar(6;4) = 4

poissonmgf(6;4) = e^(4*e^t-4)

Time taken by function: 56539 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Discrete Distributions ]#
```

Figure 4.5: *The functions in SymIntegration to compute the probability mass function, cumulative distribution function, mean, variance, and moment generating function for several discrete probability distributions (**SymIntegration/Examples/Statistics/Test SymIntegration Discrete Distributions/main.cpp**).*

[SI*] To generate random numbers that have certain discrete distributions we can use these functions in SymIntegration:
vrandn_bernoulli(p**,** n**)**
vrandn_binomial(p**,** n**)**

with n as the number of random number we want to generate, the result will be in **std::vector<double>**.

[SI*] To generate n unique integer random numbers that have uniform distribution with range $[a, b]$ we can use this function in SymIntegration:
vrandn_uniqueinteger(a**,** b**,** n**)**

IV. Continuous Probability Distributions

[SI*] We will show the definitions and theorems of continous distributions, starting from uniform distribution.

i. Uniform Distribution

Definition 4.26: Uniform Distribution

The density function of the continuous uniform random variable X on the interval $[A, B]$ is

$$f(x; A, B) = \begin{cases} \frac{1}{B-A}, & A \leq x \leq B, \\ 0, & \text{elsewhere} \end{cases} \tag{4.58}$$

the density function forms a rectangle with base $B - A$ and constant height $\frac{1}{B-A}$. The uniform distribution is often called the rectangular distribution. This is the simplest continuous distributions in all of statistics.

Theorem 4.9: The Mean and Variance

The mean and variance of the uniform distribution are

$$\mu = \frac{A + B}{2} \tag{4.59}$$

$$\sigma^2 = \frac{(B - A)^2}{12} \tag{4.60}$$

ii. Normal Distribution

Definition 4.27: Normal Distribution

This is the most important continuous probability distribution in the entire field of statistics.

A continuous random variable X having the bell-shaped distribution is called a normal random variable. The density of the normal random variable X, with mean μ and variance σ^2, is

$$n(x;\mu,\sigma) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{1}{2\sigma^2}(x-\mu)^2}, \quad -\infty < x < \infty \tag{4.61}$$

where $\pi = 3.14159$ and $e = 2.71828$.

The properties of the normal curve:

1. The mode, which is the point on the horizontal axis where the curve is a maximum, occurs at $x = \mu$.
2. The curve is symmetric about a vertical axis through the mean μ.
3. The curve has its points of inflection at $x = \mu \pm \sigma$; it is concave downward if $\mu - \sigma < X < \mu + \sigma$ and is concave upward otherwise.
4. The normal curve approaches the horizontal axis asymptotically as we proceed in either direction away from the mean.
5. The total area under the curve and above the horizontal axis is equal to 1.

The normal distribution finds enormous application as a limiting distribution. Under certain conditions, the normal distribution provides a good continuous approximation to the binomial and hypergeometric distributions.

Theorem 4.10: The Mean and Variance

The mean and variance of $n(x;\mu,\sigma)$ are μ and σ^2. Hence, the standard deviations is σ.

The curve of any continuous probability distribution or density function is constructed so that the area under the curve bounded by two ordinates $x = x_1$ and $x = x_2$ equals the probability that the random variable X assumes a value between $x = x_1$ and $x = x_2$. Thus, for the normal curve we will have

$$P(x_1 < X < x_2) = \int_{x_1}^{x_2} n(x;\mu,\sigma)\, dx = \frac{1}{\sqrt{2\pi}\sigma} \int_{x_1}^{x_2} e^{-\frac{1}{2\sigma^2}(x-\mu)^2}\, dx \tag{4.62}$$

Definition 4.28: Standard Normal Distribution

We are able to transform all the observations of any normal random variable X into a new set of observations of a normal random variable Z with mean 0 and variance 1. This can be done by means of the transformation

$$Z = \frac{X - \mu}{\sigma} \tag{4.63}$$

Whenever X assumes a value x, the corresponding value of Z is given by

$$z = \frac{x - \mu}{\sigma}$$

Therefore, if X falls between the values $x = x_1$ and $x = x_2$, the random variable Z will fall between the corresponding values

$$z_1 = \frac{x_1 - \mu}{\sigma}$$

and

$$z_2 = \frac{x_2 - \mu}{\sigma}$$

Consequently, we may write

$$\begin{aligned} P(x_1 < X < x_2) &= \frac{1}{\sqrt{2\pi}\sigma} \int_{x_1}^{x_2} e^{-\frac{1}{2\sigma^2}(x-\mu)^2}\, dx \\ &= \frac{1}{\sqrt{2\pi}(1)} \int_{x_1}^{x_2} e^{-\frac{1}{2(1)^2}(z)^2}\, dz \\ &= \int_{x_1}^{x_2} n(z; 0, 1)\, dz \\ &= P(z_1 < Z < z_2) \end{aligned} \tag{4.64}$$

where Z is seen to be a normal random variable with mean 0 and variance 1.

The distribution of a normal random variable with mean 0 and variance 1 is called a standard normal distribution.

Theorem 4.11: Normal Approximation to the Binomial

If X is a binomial random variable with mean $\mu = np$ and variance $\sigma^2 = npq$, then the limiting form of the distribution of

$$Z = \frac{X - np}{\sqrt{npq}}$$

as $n \to \infty$, is the standard normal distribution $n(z; 0, 1)$.

Definition 4.29: Normal Approximation to the Binomial Distribution

Let X be a binomial random variable with parameters n and p. For large n, X has approximately a normal distribution with $\mu = np$ and $\sigma^2 = npq = np(1-p)$ and

$$\begin{aligned} P(X \leq x) &= \sum_{k=0}^{x} b(k; n, p) \\ &= P\left(Z \leq \frac{x + 0.5 - np}{\sqrt{npq}}\right) \end{aligned}$$

as the area under normal curve to the left of $x + 0.5$, and the approximation will be good if np and $np(1-p)$ are greater than or equal to 5.

[SI*] **Computing the cdf of a Normal Distribution**

When computing for the probability density function of a normal distribution we can use the Eq (4.53) directly, but the problem is when we have to compute the cumulative distribution function of a normal distribution, it will take an advanced level of integration that has no known indefinite integral, take a look at Eq. (4.53)

$$P(X = x) = n(x; \mu, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{1}{2\sigma^2}(x-\mu)^2}, \quad -\infty < x < \infty$$

When we want to compute the cdf we will integrate the pdf just like this

$$\begin{aligned} P(X \leq x) &= \int_{-\infty}^{x} \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{1}{2\sigma^2}(x-\mu)^2} \, dx \\ &= \frac{1}{\sqrt{2\pi}\sigma} \int_{-\infty}^{x} e^{-\frac{1}{2\sigma^2}(x-\mu)^2} \, dx \\ &= \frac{1}{\sqrt{2\pi}\sigma} \int_{-\infty}^{x} e^{-\frac{1}{2\sigma^2}(x^2 - 2\mu x + \mu^2)} \, dx \end{aligned}$$

The computation for the integral above is not easy analytically since there is no elementary indefinite integral for

$$\int e^{-x^2} \, dx$$

Now, we would like to introduce to the Gaussian integral, also known as Euler-Poisson integral, it is the integral of the Gaussian function $f(x) = e^{-x^2}$ over the entire real line. The integral is

$$\int_{-\infty}^{\infty} e^{-x^2} \, dx = \sqrt{\pi} \tag{4.65}$$

Carl Friedrich Gauss published the precise integral in 1809. The integral has a wide range of applications, e.g. with a slight chance of variables it is used to compute the normalizing constant of the normal distribution. In physics this type of integral appeas frequently in quantum mechanics.

The same integral with finite limits is closely related to both the error function and the cumulative distribution function of the normal distribution. So we can approximate the cdf of normal distribution with error function.

The Gaussian integral can be solved analytically through the methods of multivariable calculus. Thus if we have an arbitrary Gaussian function the definite integral is

$$\int_{-\infty}^{\infty} e^{-a(x+b)^2}\,dx = \sqrt{\frac{\pi}{a}} \tag{4.66}$$

In short, the cdf for a normal distribution can be related to error function, so the cdf formula is

$$\phi(x) = P(X \leq x) = \frac{1}{2}\text{erf}\left(-\frac{x}{\sqrt{2}}\right) \tag{4.67}$$

for $\mu = 0, \sigma = 1$.

$$\phi(x) = P(X \leq x) = \frac{1}{2}\text{erf}\left(-\frac{\frac{x-\mu}{\sigma}}{\sqrt{2}}\right) \tag{4.68}$$

for $\mu \neq 0, \sigma \neq 1$. The error function itself can be computed with Gamma function or Hypergeometric function

$$\text{erf}(x) = 1 - \frac{\Gamma\left(\frac{1}{2}, x^2\right)}{\sqrt{\pi}} \tag{4.69}$$

$$\text{erf}(x) = \frac{2x}{\sqrt{\pi}}{}_2F_1\left(-, \frac{1}{2}; \frac{3}{2}; -x^2\right) \tag{4.70}$$

Luckily, the standard library for C++ **<cmath>** already has the function to compute the error function **erfc(x)** that we can use and embed in SymIntegration. Undergraduate students usually only taught to read the cdf for normal distribution from statistics table, the derivative to compute the cdf itself is rarely taught at undergraduate level. This is the background, or the engine behind all that values at the statistics table. How integral plays an important part in statistics, it really shows that statistics actually more symbolic than numeric.

iii. Gamma Distribution

Definition 4.30: Gamma Function

The Gamma distribution derives its name from the well-known gamma function. The Gamma function is defined by

$$\Gamma(\alpha) = \int_0^{\infty} x^{\alpha-1}e^{-x}\,dx, \quad \textbf{for } \alpha > 0 \tag{4.71}$$

Simple properties of the gamma function:

(a) $\Gamma(n) = (n-1)(n-2)\dots(1)\Gamma(1)$, for a positive integer n.

(b) $\Gamma(n) = (n-1)!$ for a positive integer n.

(c) $\Gamma(1) = 1$.

(d) $\Gamma(\frac{1}{2}) = \sqrt{\pi}$

Definition 4.31: Gamma Distribution

The continuous random variable X has a gamma distribution, with parameters α and β, if its density function is given by

$$f(x;\alpha,\beta) = \begin{cases} \frac{1}{\beta^\alpha \Gamma(\alpha)} x^{\alpha-1} e^{-\frac{x}{\beta}}, & x > 0 \\ 0, \quad \text{elsewhere} \end{cases} \tag{4.72}$$

where $\alpha > 0$ and $\beta > 0$. Often we call α as shape and β as scale.

The exponential distribution is a special case of the gamma distribution. The exponential and gamma distributions play an important role in both queuing theory and reliability problems.

The special gamma distribution for which $\alpha = 1$ is called the exponential distribution.

The Lower Incomplete Gamma Function $\gamma(s,x)$
In order to compute the cdf of several continuous distributions, e.g. chi-squared and Gamma, without using external library like **Boost** we need to know this function: the lower incomplete Gamma function.

The lower incomplete Gamma function is defined by the integral:

$$\gamma(s,x) = \int_0^x t^{s-1} e^{-t}\, dt$$

the definite integral above is quite hard to compute, so we approximate it with the Taylor expansion, the expansion of the exponential function and term by term integration give the following expansion:

$$\gamma(s,x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{n+s}}{n!(n+s)}$$

We can use the series expansion above and create the C++ code to compute the cdf of Gamma distribution or chi-squared distribution. The lower incomplete Gamma function will come in handy for other statistics related computation.

The relations for the Gamma functions:

$$\gamma(s,x) + \Gamma(s,x) = \Gamma(s)$$

with $\Gamma(s,x)$ as the upper incomplete Gamma function.

The Gamma Distribution' cdf with Lower Incomplete Gamma Function $\gamma(s,x)$

The Gamma cdf formula is an integral that represents the probability of a Gamma-distributed

random variable being less than or equal to a specific value, it is defined as

$$\begin{aligned} P(X \leq x) &= F(x;\alpha,\beta) \\ &= \frac{1}{\Gamma(\alpha)} \int_0^x t^{\alpha-1} e^{-\beta t}\, dt \\ &= \frac{\gamma\left(\alpha, \frac{x}{\beta}\right)}{\Gamma(\alpha)} \end{aligned}$$

With α as the shape parameter and β as the scale parameter. Be careful, some textbooks use rate parameter: $\lambda = \frac{1}{\beta}$ instead of using β. Computer can only use this formula due to the complex integral, the cdf is calculated using numerical methods.

iv. Exponential Distribution

Definition 4.32: Exponential Distribution

The continuous random variable X has an exponential distribution, with parameter β, if its density function is given by

$$f(x;\beta) = \begin{cases} \frac{1}{\beta} e^{-\frac{x}{\beta}}, & x > 0 \\ 0, & \text{elsewhere} \end{cases} \tag{4.73}$$

where $\beta > 0$. Sometimes we also write exponential distribution density function this way

$$f(x;\beta) = \begin{cases} \lambda e^{-\lambda x}, & x > 0 \\ 0, & \text{elsewhere} \end{cases} \tag{4.74}$$

The exponential distribution is more appropriate when the memoryless property is justified. If the failure of the component is a result of gradual or slow wear, then the exponential does not apply and either the gamma or the Weibull distribution may be more appropriate.

Theorem 4.12: The Mean and Variance

The mean and variance of the gamma distribution are

$$\begin{aligned} \mu &= \alpha\beta \\ \sigma^2 &= \alpha\beta^2 \end{aligned} \tag{4.75}$$

The mean and variance of the exponential distribution are

$$\begin{aligned} \mu &= \beta \\ \sigma^2 &= \beta^2 \end{aligned} \tag{4.76}$$

v. Beta Distribution

Definition 4.33: Beta Function

A beta function is defined by

$$B(\alpha, \beta) = \int_0^1 x^{\alpha-1}(1-x)^{\beta-1}\, dx = \frac{\Gamma(\alpha)\Gamma(\beta)}{\Gamma(\alpha+\beta)}, \quad \text{for } \alpha, \beta > 0 \tag{4.77}$$

where $\Gamma(\alpha)$ is the gamma function.

Definition 4.34: Beta Distribution

The continuous random variable X has a beta distribution with parameters $\alpha > 0$ and $\beta > 0$ if its density function is given by

$$f(x) = \begin{cases} \frac{1}{B(\alpha,\beta)} x^{\alpha-1}(1-x)^{\beta-1}, & 0 < x < 1 \\ 0, & \text{elsewhere} \end{cases} \tag{4.78}$$

Note that the uniform distribution on $(0,1)$ is a beta distribution with parameters $\alpha = 1$ and $\beta = 1$. The major reason why we use Beta distribution is because it is defined in the range of $[0,1]$ so a Beta distribution is a very natural distribution to use when we are talking about probabilities and we want to specify about a prior knowledge of the probabilities of something accruing.

Applications:

1. **Survival Analysis**
   In fields like medicine and engineering, the Beta distribution is used to analyze survival times or failure rates, aiding in understanding product reliability or treatment effectiveness. By modeling these distributions, researchers gain insights into how long products last or how treatments perform over time. This statistical tool helps assess durability and reliability, crucial for making informed decisions in various industries.

2. **Time Management**
   In project management, the Beta distribution helps estimate task durations and overall project completion times. It's like a tool that assists in planning and scheduling. By using this distribution, project managers can better understand the range of possible timeframes for completing tasks or the entire project. This aids in making more informed decisions and managing time effectively.

3. **Risk Management**
   Risk management is essential in finance and insurance. It's about figuring out the chances of different events happening and how they could affect things. By looking at probabilities, it helps make smart choices to reduce risks.

4. **Genetics**
   The beta distribution is a part of the population genetics that not only allows scientists to keep tabs on genes at the population level but as well makes them understand how the genes mutate within the whole group. Allele frequencies, or the different version of genes, must be borne in mind while studying pedigrees and evolution. Utilizing the Beta distribution data, the researchers are able to track the changes in the allele distribution on behalf of the intergenerational population. Such data is informative as far as evolutionary patterns and the nature of ecosystems is known.

5. **Quality Control**
   In manufacturing industries, the Beta distribution serves as a crucial tool for characterizing the variability in product quality. By employing this distribution, businesses gain valuable insights into the distribution of defects or deviations from desired specifications within their production processes. This understanding enables effective control measures to be implemented, ensuring products consistently meet or exceed quality standards. By leveraging the Beta distribution, manufacturers can optimize processes, minimize waste, and enhance overall product reliability, fostering customer satisfaction and trust.

The shape of the distribution is determined entirely by the parameters α and β:

- $\alpha = 1,\ \beta = 1$: Uniform distribution on $(0, 1)$
- $\alpha > 1,\ \beta > 1$: Unimodal distribution with a peak within the interval (0, 1). As α and β increase, the peak becomes sharper and more centered.
- $\alpha < 1,\ \beta < 1$: U-shaped distribution with peaks at the endpoints of the interval.
- $\alpha > 1,\ \beta < 1$: Right-skewed distribution
- $\alpha < 1,\ \beta > 1$: Left-skewed distribution

The Beta (α, β) distribution converges to the normal distribution as $\alpha, \beta \to \infty$. This convergence can be seen by reducing the variance in the graph of the beta distribution. The probability mass then becomes more concentrated around the mean, and the distribution becomes more bell-shaped.

Definition 4.35: Moment Generating Function for Beta Distribution

The moment generating function (mgf) for beta distribution is defined by

$$M_X(\alpha; \beta; t) = 1 + \sum_{i=1}^{\infty} \left(\prod_{j=0}^{i-1} \frac{\alpha + j}{\alpha + \beta + j} \right) \frac{t^i}{i!} \tag{4.79}$$

this function is a series representation of a beta distribution that can be used to compute moments like mean and variance by computing the derivative of the mgf.

The mgf of the beta distribution can be related to the Kummer's confluent hypergeometric function (of the first kind). It is a power series solution to Kummer's differential equation (or confluent hypergeometric equation), an important equation in physics, chemistry, and engineering. It is used to solve problems involving normal and slow rotationally symmetric TE modes in azimuthally magnetized circular ferrite waveguides.

Thus, this is the step by step to obtain the moment generating function for beta distribution

$$\begin{aligned}
M_X(\alpha; \beta; t) &= E[e^{tX}] \\
&= \int_0^1 e^{tx} f(x; \alpha, \beta)\, dx \\
&= \frac{\Gamma(\alpha + \beta)}{\Gamma(\alpha)\Gamma(\beta)} \int_0^1 e^{tx} x^{\alpha - 1} (1 - x)^{\beta - 1}\, dx \\
&= \frac{1}{B(\alpha, \beta)} \int_0^1 e^{tx} x^{\alpha - 1} (1 - x)^{\beta - 1}\, dx \\
&= \sum_{n=0}^{\infty} \frac{\alpha^{(n)}}{(\alpha + \beta)^n} \frac{t^n}{n!} \\
&= 1 + \sum_{k=1}^{\infty} \left(\prod_{r=0}^{k-1} \frac{\alpha + r}{\alpha + \beta + r} \right) \frac{t^k}{k!} \\
&= {}_1F_1(\alpha; \alpha + \beta; t)
\end{aligned}$$

where

$$x^{(n)} = x(x + 1)(x + 2) \dots (x + n - 1)$$

is the rising factorial, also known as the "Pochhammer symbol."

We want to put it simply like this, the mgf of the beta distribution is:

$$M_X(\alpha;\beta;t) = {}_1F_1(\alpha;\alpha+\beta;t)$$

where

$${}_1F_1(\alpha;\beta;t) = \sum_{n=0}^{\infty} \frac{\alpha^{(n)}}{(\beta)^n}\frac{t^n}{n!} \tag{4.80}$$

and

$${}_1F_1(\alpha;\alpha+\beta;t) = \sum_{n=0}^{\infty} \frac{\alpha^{(n)}}{(\alpha+\beta)^n}\frac{t^n}{n!} \tag{4.81}$$

The good news is the **GNU scientific library** and **Boost** are able to numerically compute the hypergeometric function 1F1 directly for a small t ($|t| < 1$), but not for the symbolic computation. So we add this to SymIntegration, thus SymIntegration is able to compute hypergeometric function 1F1 numerically and symbolically, with limitation of course, the speed is slower than GSL and Boost since symbolic function will has more cost overall, knowing we can do more after that, to integrate it symbolically or derivate it.

It is handled in **src/specialfunctions.cpp** along with the include in **include/symintegral/specialfunctions.h**

```
Symbolic hypergeometric_1F1(double a, double b, const Symbolic &s, int
    max_iterations) {
       Symbolic sum = 1;

       for (int i = 1; i <= max_iterations; ++i)
       {
              sum += (rising_pochhammer(a, i) * (s^(i))) / (rising_pochhammer
                  (b, i) * factorial(i));
       }
       return sum;
}

double hypergeometric_1F1(double a, double b, double s, int max_iterations) {
       Symbolic sum = 1;

       for (int i = 1; i <= max_iterations; ++i)
       {
              sum += (rising_pochhammer(a, i) * (s^(Symbolic(i)))) / (
                  rising_pochhammer(b, i) * factorial(i));
       }
       return sum;
}
```

Code 55: *src/specialfunctions.cpp*

We put the **max_iterations** as 5, since it gives the same numeric answer as Boost and GSL.

```
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test/Test Hypergeometric 1F1 GSL vs Boost ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb  main.o -lstdc++ -lgsl -lcblas
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test/Test Hypergeometric 1F1 GSL vs Boost ]# ./main
GSL: Hypergeometric 1F1 (2.0, 3.0, 0.5): 1.405115e+00
Boost: MGF of Beta(2, 3) at t=0.5 is: 1.22758

Time taken by function: 273 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test/Test Hypergeometric 1F1 GSL vs Boost ]#
```

Figure 4.6: *The computation of the hypergeometric function and the mgf of beta distribution ($\alpha = 2, \beta = 3, t = 0.5$) with GSL and Boost has a very fast speed. (**SymIntegration/Examples/Calculus/Test Hypergeometric 1F1 GSL vs Boost/main.cpp**).*

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Compute Hypergeometric Function 1F1 and mgf Beta ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Compute Hypergeometric Function 1F1 and mgf Beta ]# ./main

for n = 5

Hypergeometric_1F1(2, 3, z) = 0.666667*z+0.25*z^(2)+0.0666667*z^(3)+0.0138889*z^(4)+0.00238095*z^(5)+1

Hypergeometric_1F1(2, 3, 0.5) = 1.40511

mgf Beta (2, 3, 0.5) = 1.22758

Time taken by function: 24444 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Compute Hypergeometric Function 1F1 and mgf Beta ]#
```

Figure 4.7: *The computation of the complete hypergeometric function and the mgf of beta distribution ($\alpha = 2, \beta = 3, t = 0.5$) with SymIntegration, it is slower than GSL and Boost but able to use symbolic of "z". (**SymIntegration/Examples/Calculus/Test SymIntegration Compute Hypergeometric Function 1F1 and mgf Beta/main.cpp**).*

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Compute Hypergeometric Function 1F1 and mgf Beta ]#  make clean
rm -f main.o main
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Compute Hypergeometric Function 1F1 and mgf Beta ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Compute Hypergeometric Function 1F1 and mgf Beta ]# ./main

for n = 5

Hypergeometric_1F1(2, 3, z) = 0.666667*z+0.25*z^(2)+0.0666667*z^(3)+0.0138889*z^(4)+0.00238095*z^(5)+1

mgf Beta (2, 3, 0.5) = 1.22758

Time taken by function: 19414 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Compute Hypergeometric Function 1F1 and mgf Beta ]#
```

Figure 4.8: *The computation of the symbolic hypergeometric function and the mgf of beta distribution ($\alpha = 2, \beta = 3, t = 0.5$) with SymIntegration. (**SymIntegration/Examples/Calculus/Test SymIntegration Compute Hypergeometric Function 1F1 and mgf Beta/main.cpp**).*

Definition 4.36: Survival Function

The survival function, also known as the complementary cumulative distribution function (ccdf), is the probability that X takes a value greater than x, or

$$P(X > x)$$

it is defined as

$$S(x;\alpha,\beta) = 1 - F(x;\alpha,\beta) = 1 - \frac{B(x;\alpha,\beta)}{B(\alpha,\beta)} \tag{4.82}$$

Real life applications of Beta distribution:

1. **Crop Yield Analysis**
   Farmers and researchers use the Beta distribution to understand how crop yields vary across different areas of land. This helps them decide where to plant crops, how to allocate resources, and manage risks.

2. **Sport Analytics**
   In sports, the Beta distribution helps analyze player performance metrics like shooting accuracy or goal-scoring rates. Coaches and analysts use this information to make strategic decisions.

3. **Environmental Risk Assessment**
   Environmental scientists use the Beta distribution to model uncertainty in environmental factors like pollutant concentrations. This helps assess potential risks and guide policy decisions.

4. **Clinical Trials**
   In medical research, the Beta distribution models treatment outcomes or response rates in clinical trials. This helps researchers design trials, determine sample sizes, and accurately analyze results.

5. **Weather Forecasting**
   Meteorologists use the Beta distribution to model the distribution of weather phenomena such as rainfall or wind speeds. This helps predict weather patterns and assess the likelihood of extreme events.

Theorem 4.13: Mean and Variance

The mean and variance of a beta distribution with parameters α and β are

$$\mu = \frac{\alpha}{\alpha + \beta} \tag{4.83}$$

$$\sigma^2 = \frac{\alpha\beta}{(\alpha + \beta)^2(\alpha + \beta + 1)} \tag{4.84}$$

The first two theoretical moments for Beta distribution are the mean (μ) and variance (σ^2). When we are doing observation or experiment we obtain samples and we can compute the sample mean and variance with

$$\overline{x} = \frac{\sum_{i=1}^{n} x_i}{n}$$
$$s^2 = \frac{\sum_{i=1}^{n} (x_i - \overline{x})^2}{n - 1}$$

Now by equating $\overline{x} = \mu$ and $s^2 = \sigma^2$ we will obtain two equations with two unknowns (α, β). Solving these equations simultaneously gives the estimates for α and β. This is called the method of moments, the drawback is that it can be inefficient and sensitive to outliers. Other methods, such as maximum likelihood estimation, often provide more accurate estimates.

The Beta Distribution' cdf with Regularized Incomplete Beta Function $I_x(\alpha, \beta)$

The beta cdf can be expressed using the regularized incomplete beta function, which is defined as the incomplete beta function ($B(x; \alpha, \beta)$) divided by the complete beta function ($B(\alpha, \beta)$). This can be written as:

$$F_X(x) = \frac{B(x; \alpha, \beta)}{B(\alpha, \beta)} = I_x(\alpha, \beta)$$

where $I_x(\alpha, \beta)$ is the regularized incomplete beta function.

The incomplete beta function is defined as an integral:

$$B(x; \alpha, \beta) = \int_0^x u^{\alpha-1}(1 - u)^{\beta-1}\, du$$

And the complete beta function can be expressed using the Gamma function:

$$B(\alpha, \beta) = \frac{\Gamma(\alpha)\Gamma(\beta)}{\Gamma(\alpha + \beta)}$$

The incomplete beta function and its regularized form are crucial in statistics, particularly in calculating probabilities related to beta distribution and in determining cumulative distribution functions (cdf) for distributions like the binomial, F-distribution, and Student's t-distribution.

Characteristics of the cdf and sf:

1. Both cdf and sf are increasing functions on the interval $(0, 1)$.
2. The cdf approaches 0 as x approaches 0 and approaches 1 as x approaches 1.
3. The cdf approaches 1 as x approaches 0 and approaches 0 as x approaches 1.
4. The sum of the cdf and sf at any point x is equal to 1.
5. The shapes of the cdf and sf depend on the values of α and β. For example, if $\alpha > 1$ and $\beta > 1$, the cdf is the S-shaped and the sf is concave down.

vi. Chi-Squared Distribution

Definition 4.37: Chi-Squared Distribution

The chi-squared distribution with ν degrees of freedom is the distribution of a sum of the squares of k independent standard normal random variables.

The chi-squared distribution χ^2_ν is a special case of the gamma distribution and the univariate Wishart distribution. Specifically if X χ^2_ν then X Gamma $\left(\alpha = \frac{\nu}{2}, \beta = 2\right)$ (where α is the shape parameter and $\beta$is the scale parameter of the gamma distribution) and X $W_1(1, k)$.

The probability density function of the chi-squared distribution is

$$f(x; \nu) = \frac{1}{\Gamma(\nu/2)2^{\frac{\nu}{2}}} x^{\frac{\nu}{2}-1} e^{-\frac{x}{2}}, \quad x > 0 \tag{4.85}$$

The cumulative distribution function is

$$F(x; \nu) = \frac{\gamma\left(\frac{\nu}{2}, \frac{x}{2}\right)}{\Gamma\left(\frac{\nu}{2}\right)} = P\left(\frac{\nu}{2}, \frac{x}{2}\right) \tag{4.86}$$

where $\gamma(s, t)$ is the lower incomplete gamma function and $P(s, t)$ is the regularized gamma function.

The chi-squared distribution is one of the most widely used probability distributions in inferential statistics, notably in hypothesis testing and in construction of confidence intervals.

The lower incomplete gamma function is given by

$$\begin{aligned} \gamma(a, x) &= \int_0^x t^{a-1} e^{-t}\, dt \\ &= a^{-1} x^a e^{-x}\, {}_1F_1(1; 1 + a; x) \\ &= a^{-1} x^a\, {}_1F_1(a; 1 + a; -x) \end{aligned}$$

where ${}_1F_1(a; b; x)$ is the confluent hypergeometric function of the first kind.

By definition, the lower and upper incomplete gamma functions satisfy

$$\Gamma(a, x) + \gamma(a, x) = \Gamma(a) \tag{4.87}$$

Theorem 4.14: The Mean and Variance

The mean and variance of the chi-squared distribution are

$$\mu = \nu \tag{4.88}$$

$$\sigma^2 = 2\nu \tag{4.89}$$

The Chi-Squared Distribution' cdf with Lower Incomplete Gamma Function $\gamma(s, x)$
The chi-squared cdf is expressed using the incomplete Gamma function as

$$\begin{aligned} P(X \leq x) &= F(x; \nu) \\ &= \frac{1}{\Gamma\left(\frac{\nu}{2}\right)} \gamma\left(\frac{\nu}{2}, \frac{x}{2}\right) \end{aligned}$$

with ν as the degrees of freedom, Γ is the Gamma function, and γ is the lower incomplete Gamma function.

We can also write the cdf in a series expansion for computation:

$$F(x; \nu) = \left(\frac{x}{2}\right)^{\left(\frac{k}{2}\right)} e^{-\frac{x}{2}} \sum_{m=0}^{\infty} \frac{\left(\frac{x}{2}\right)^m}{\Gamma\left(\frac{k}{2} + m + 1\right)}$$

vii. t-Distribution

In many experimental scenarios, knowledge of σ is certainly no more reasonable than knowledge of the population mean μ. Often, in fact, an estimate of σ must be supplied by the same sample information that produced the sample average $\overline{x}$. As a result, a natural statistic to consider to deal with inferences on μ is

$$T = \frac{\overline{X} - \mu}{S/\sqrt{n}} \tag{4.90}$$

since S is the sample analog to σ. If the sample size is small, the values of S^2 fluctuate considerably from sample to sample and the distribution T deviates appreciably from that of a standard normal distribution.

If the sample size is large enough, say $n \geq 30$, the distribution of T does not differ considerably from the standard normal. However, for $n < 30$, it is useful to deal with the exact distribution of T. In developing the sampling distribution of T, we shall assume that our random sample was selected from a normal population. We can then write

$$T = \frac{(\overline{X} - \mu)/(\sigma/\sqrt{n})}{\sqrt{S^2/\sigma^2}} = \frac{Z}{\sqrt{V/(n-1)}} \tag{4.91}$$

where

$$Z = \frac{\overline{X} - \mu}{\sigma/\sqrt{n}}$$

has the standard normal distribution and

$$V = \frac{(n-1)S^2}{\sigma^2} \tag{4.92}$$

has a chi-squared distribution with $\nu = n - 1$ degrees of freedom. In sampling from normal populations, we can show that $\overline{X}$ and S^2 are independent, and consequently so are Z and V.

Theorem 4.15: t-Distribution

Let Z be a standard normal random variable and V a chi-squared random variable with ν degrees of freedom. If Z and V are independent, then the distribution of the random variable T, where

$$T = \frac{Z}{\sqrt{\frac{V}{\nu}}} \tag{4.93}$$

with $\nu = n - 1$, is given by the density function

$$h(t) = \frac{\Gamma\left[\frac{\nu+1}{2}\right]}{\Gamma\left(\frac{\nu}{2}\right)\sqrt{\pi\nu}}\left(1 + \frac{t^2}{\nu}\right)^{-\frac{\nu+1}{2}}, \quad -\infty < t < \infty \tag{4.94}$$

This is known as the t-distribution with ν degrees of freedom.

The t-distribution represents the structure that occurs if all of the values of $\frac{\overline{x}-\mu}{s/\sqrt{n}}$ are formed, where $\overline{x}$ and s are taken from samples of size n from a $n(x;\mu,\sigma)$ distribution.

The Student's t-Distribution' cdf with Regularized Incomplete Beta Function $I_x(\alpha,\beta)$

To compute the cdf of t-distribution we will use this formula:

$$F_\nu(t) = \frac{1}{2}I_x\left(\frac{\nu}{2},\frac{1}{2}\right)$$

for $t \leq 0$.

$$F_\nu(t) = 1 - \frac{1}{2}I_x\left(\frac{\nu}{2},\frac{1}{2}\right)$$

for $t \geq 0$, where

$$x = \frac{\nu}{\nu + t^2}$$

and $\nu = n - 1$ is the degrees of freedom.

viii. F-Distribution

In probability theory and statistics, the F-distribution or F-ratio, also known as Snedecor's F distribution or the Fisher Snedecor distribution, is a continuous probability distribution that arises frequently as the null distribution of a test statistic, most notably in the analysis of variance (ANOVA) and other F-tests.

As you can see, the F-distribution finds enormous application in comparing sample variances. Applications of the F-distribution are found in problems involving two or more samples.

The statistic F is defined to be the ratio of two independent chi-squared random variables, each divided by its number of degrees of freedom. Hence, we can write

$$F = \frac{U/\nu_1}{V/\nu_2} \tag{4.95}$$

where U and V are independent random variables having chi-squared distributions with ν_1 and ν_2 degrees of freedom.

Theorem 4.16: F-Distribution

Let U and V be two independent random variables having chi-squared distributions with ν_1 and ν_2 degrees of freedom, respectively. Then the distribution of the random variable $F = \frac{U/\nu_1}{V/\nu_2}$ is given by the density function

$$h(f) = \begin{cases} \frac{\Gamma\left[\frac{\nu_1+\nu_2}{2}\right]\left(\frac{\nu_1}{\nu_2}\right)^{\frac{\nu_1}{2}}}{\Gamma\left(\frac{\nu_1}{2}\right)\Gamma\left(\frac{\nu_2}{2}\right)} \frac{f^{\frac{\nu_1}{2}-1}}{\left(1+\frac{\nu_1 f}{\nu_2}\right)^{\frac{\nu_1+\nu_2}{2}}}, & f > 0 \\ 0, & f \leq 0 \end{cases} \tag{4.96}$$

This is known as the F-distribution with ν_1 and ν_2 degrees of freedom. The curve of the F-distribution depends not only on the two parameters ν_1 and ν_2 but also on the order in which we state them.

The statistics table provides the f-value that you are looking for given the various combinations of the degrees of freedom ν_1 and ν_2. Hence the f-value with 6 and 10 degrees of freedom ν_1 and ν_2, leaving an area of 0.05 to the right, is

$$f_{0.05} = 3.22$$

Theorem 4.17: The F-Distribution with Degrees of Freedom

Writing $f_\alpha(\nu_1, \nu_2)$ for f_α with ν_1 and ν_2 degrees of freedom, we obtain

$$f_{1-\alpha}(\nu_1, \nu_2) = \frac{1}{f_\alpha(\nu_2, \nu_1)} \tag{4.97}$$

The F-Distribution' pdf with Beta Function

The pdf of the F-distribution for a variable x with numerator degrees of freedom ν_1 and denominator degrees of freedom ν_2 is defined as:

$$f(x; \nu_1, \nu_2) = \frac{1}{B\left(\frac{\nu_1}{2}, \frac{\nu_2}{2}\right)} \left(\frac{\nu_1}{\nu_2}\right)^{\frac{\nu_1}{2}} x^{\frac{\nu_1}{2}-1} \left(1 + \frac{\nu_1}{\nu_2} x\right)^{-\frac{\nu_1+\nu_2}{2}}$$

where $B(a, b)$ is the beta function, which can be defined in terms of the gamma function:

$$B(a, b) = \frac{\Gamma(a)\Gamma(b)}{\Gamma(a+b)}$$

the gamma function $\Gamma()$ can be computed easily and available in standard C++ library as **tgamma()**.

The pdf of F-distribution is always greater than or equal to zero for all valid x. The integral of the pdf over its entire domain $(0, \infty)$ is equal to 1.

The F-Distribution' cdf with Incomplete Beta Function
The regularized incomplete beta function, denoted $I_x(a,b)$, is defined as:

$$I_x(a,b) = \frac{B_x(a,b)}{B(a,b)} = \frac{\int_0^x t^{a-1}(1-t)^{b-1}\,dt}{\int_0^1 t^{a-1}(1-t)^{b-1}\,dt}$$

where $B_x(a,b)$ is the incomplete beta function, and $B(a,b)$ is the complete beta function.

The cdf of an F-distribution with ν_1 and ν_2 degrees of freedom is given by the following formula:

$$F(x;\nu_1,\nu_2) = P(X \leq x) = I_z\left(\frac{\nu_1}{2}, \frac{\nu_2}{2}\right)$$

where the variable z is defined as

$$z = \frac{\nu_1 x}{\nu_1 x + \nu_2}$$

Using the identity $I_z(a,b) = 1 - I_{1-z}\left(\frac{\nu_2}{2}, \frac{\nu_1}{2}\right)$

$$F(x;\nu_1,\nu_2) = 1 - I_{1-z}\left(\frac{\nu_2}{2}, \frac{\nu_1}{2}\right)$$

where $1-z$ is:

$$1 - z = 1 - \frac{\nu_1 x}{\nu_1 x + \nu_2} = \frac{\nu_2}{\nu_1 x + \nu_2}$$

The F-Distribution Connection to the F-Statistic
The F-distribution is often derived as the ratio of two chi-squared distributions, which is the basis for the F-test. The F-statistic is calculated as

$$F = \frac{\text{Variance}_{\text{between}}}{\text{Variance}_{\text{within}}}$$

or

$$F = \frac{\text{Mean Square}_{\text{between}}}{\text{Mean Square}_{\text{within}}}$$

In the context of ANOVA, this can be written as:

$$F = \frac{\text{SS}_{\text{between}}/df_{\text{between}}}{\text{SS}_{\text{within}}/df_{\text{within}}}$$

The F-Distribution with Two Sample Variances

Suppose that random samples of size n_1 and n_2 are selected from two normal populations with variances σ_1^2 and σ_2^2, respectively. We know that

$$\chi_1^2 = \frac{(n_1-1)S_1^2}{\sigma_1^2}$$
$$\chi_2^2 = \frac{(n_2-1)S_2^2}{\sigma_2^2}$$

are random variables having chi-squared distributions with $\nu_1 = n_1 - 1$ and $\nu_2 = n_2 - 1$ degrees of freedom. Furthermore, since the samples are selected at random, we are dealing with independent random variables. Furthermore, since the samples are selected at random, we are dealing with independent random variables.

Theorem 4.18: F-Distribution with Two Sample Variances

If S_1^2 and S_2^2 are the variances of independent random samples of size n_1 and n_2 taken from normal populations with variances σ_1^2 and σ_2^2, respectively, then

$$F = \frac{S_1^2/\sigma_1^2}{S_2^2/\sigma_2^2} = \frac{\sigma_2^2 S_1^2}{\sigma_1^2 S_2^2} \tag{4.98}$$

has an F-distribution with $\nu_1 = n_1 - 1$ and $\nu_2 = n_2 - 1$ degrees of freedom.

The F-distribution is used in two-sample situations to draw inferences about the population variances. However, the F-distribution can also be applied to many other types of problems involving sample variances. In fact, the F-distribution is called the variance ratio distribution. The F-distribution plays an important role in the analysis of variance.

Three things one must bear in mind, regarding these fundamental sampling distributions:

1. One cannot use Central Limit Theorem unless σ is known. When σ is not known, it should be replaced by s, the sample standard deviation, in order to use the Central Limit Theorem.
2. The T statistic is not a result of the Central Limit Theorem and $x_1, x_2, \ldots, x_n$ must come from a $n(x; \mu, \sigma)$ distribution in order for $\frac{\overline{x}-\mu}{s/\sqrt{n}}$ to be a t-distribution; s is, of course, merely an estimate of σ.
3. While the notion of degrees of freedom is new at this point, the concept should be very intuitive, since it is reasonable that the nature of the distribution of S and also t should depend on the amount of information in the sample $x_1, x_2, \ldots, x_n$.

ix. Weibull Distribution and Hazard Rates

For a random variable T, representing the lifetime of an individual or a component, an interesting quantity is the instantaneous chance of failure at any time, given that the component has been operating without failure up to time x. This can be expressed as

$$h(x) = \lim_{\delta \to 0} \frac{1}{\delta} P(T \in [x, x+\delta) | T > x)$$

Alternatively, by using the conditional probability and noticing that the pdf $f(x)$ satisfies $f(x)\delta \approx P(x \leq T < x + \delta)$ for small δ, we can express the above as

$$h(x) = \frac{f(x)}{1 - F(x)} \tag{4.99}$$

Here the function $h(\cdot)$ is called the hazard rate, and it is a common method of viewing the distribution for lifetime random variables T. In fact, we can reconstruct the cdf $F(x)$ by

$$1 - F(x) = e^{-\int_0^x h(t)\ dt} \tag{4.100}$$

Hence, every continuous non-negative random variable can be described uniquely by its hazard rate. The Weibull distribution is naturally defined through the hazard rate by considering hazard rate functions that have a specific simple form. It is a distribution with

$$h(x) = \lambda x^{\alpha - 1} \tag{4.101}$$

where λ is positive and α takes on any real value. Notice that the parameter α gives the Weibull distribution different modes of behavior. If $\alpha = 1$ then the hazard rate is constant, in which case the Weibull distribution is actually an exponential distribution with rate λ. If $\alpha > 1$, then the hazard rate increases over time. This depicts a situation of "aging components", i.e. the longer a component has lived, the higher the instantaneous chance of failure. This is sometimes called Increasing Failure Rate (IFR). Conversely, $\alpha < 1$ depicts a situation where the longer a component has lasted, the lower the chance of it failing (as it perhaps the case with totalitarian political regimes). This is sometimes called Decreasing Failure Rate (DFR).

Based on Eq. (4.108) and using Eq. (4.107), we obtain the cdf and pdf

$$\begin{aligned} F(x) &= 1 - e^{-\frac{\lambda}{\alpha}x^{\alpha}} \\ f(x) &= \lambda x^{\alpha-1} e^{-\frac{\lambda}{\alpha}x^{\alpha}} \end{aligned} \tag{4.102}$$

where the bijection from λ to θ is

$$\begin{aligned} \lambda &= \alpha\theta^{-\alpha} \\ \theta &= \left(\frac{\alpha}{\lambda}\right)^{\frac{1}{\alpha}} \end{aligned} \tag{4.103}$$

In this case, θ is called the scale parameter and α is the shape parameter.

x. Cauchy Distribution

Also known as the Loretz distribution, this is a distribution without mean.

Definition 4.38: Cauchy Distribution

At first glance, a plot of the pdf looks very similar to the normal distribution. However, it is fundamentally different as its mean and standard deviation are undefined. The pdf of the Cauchy distribution is given by

$$f(x) = \frac{1}{\pi\gamma\left(1 + \left(\frac{x-x_0}{\gamma}\right)^2\right)} \tag{4.104}$$

where x_0 is the location parameter at which the peak is observed and γ is the scale parameter.

The physical example of Cauchy distributed random variable is like this: consider a drone hovering stationary in the sky at unit height. A pivoting laser is attached to its undercarriage, which pivots back and forth as it shoots pulses at the ground. At any point the laser fires, it makes an angle θ from the vertical $\left(-\frac{\pi}{2} \leq \theta \leq \frac{\pi}{2}\right)$.

Since the laser fires at a high frequency as it is pivoting, we can assume that the angle θ is distributed uniformyl on $\left[-\frac{\pi}{2}, \frac{\pi}{2}\right]$. For each shot from the laser, a point can be measured, X, horizontally on the ground from the point above which the drone is hovering. We can now

consider the horizontal measurement as a new random variable, X. Hence the cdf is

$$\begin{aligned} F_X(x) &= P(\tan(\theta) \leq x) = P(\theta \leq \operatorname{atan}(x)) \\ &= F_\theta(\operatorname{atan}(x)) \\ &= \begin{cases} 0, & \operatorname{atan}(x) \leq -\frac{\pi}{2} \\ \frac{1}{\pi}\operatorname{atan}(x), & \operatorname{atan}(x) \in \left(-\frac{\pi}{2}, \frac{\pi}{2}\right) \\ 1, & \operatorname{atan}(x) \geq \frac{\pi}{2} \end{cases} \end{aligned} \tag{4.105}$$

Now since it always holds that $\operatorname{atan}(x) \in \left(-\frac{\pi}{2}, \frac{\pi}{2}\right)$ we can obtain the density by taking the derivative of $\frac{1}{\pi}\operatorname{atan}(x)$ which evaluates to

$$f(x) = \frac{1}{\pi(1+x^2)}$$

This is a special case of the more complicated density Eq. (4.111), with x_0 and $\gamma = 1$. Importantly, the expectation integral,

$$\int_{-\infty}^{\infty} x f(x)\, dx$$

is not defined since each of the one-sided improper integrals does not converge.

xi. Bivariate Normal

One of the most ubiquitous families of multivariate distributions is the multivariate normal distribution. Similarly to the fact that a scalar (univariate) normal distribution is parametrized by the mean μ and the variance σ^2, a multivariate normal distribution is parametrized by the mean vector $\mu_{\boldsymbol{X}}$ and the covariance matrix $\Sigma_{\boldsymbol{X}}$.

Definition 4.39: Standard Multivariate Normal Distribution

For the standard multivariate normal distribution having $\mu_{\boldsymbol{X}} = \mathbf{0}$ mean and $\Sigma_{\boldsymbol{X}} = I$, the pdf for the random vector $\boldsymbol{X} = (X_1, \ldots, X_n)$ is

$$f(x) = (2\pi)^{-\frac{n}{2}} e^{-\frac{1}{2}\boldsymbol{x}^T\boldsymbol{x}} \tag{4.106}$$

Now, in general, using an affine transformation, it can be shown that for arbitrary $\mu_{\boldsymbol{X}}$ and $\Sigma_{\boldsymbol{X}}$ (positive definite)

$$f(\boldsymbol{x}) = |\Sigma_{\boldsymbol{X}}|^{-\frac{1}{2}} (2\pi)^{-\frac{n}{2}} e^{-\frac{1}{2}(\boldsymbol{x}-\mu_{\boldsymbol{X}})^T \Sigma_{\boldsymbol{X}}^{-1} (\boldsymbol{x}-\mu_{\boldsymbol{X}})} \tag{4.107}$$

where $|\Sigma_{\boldsymbol{X}}|$ is the determinant of $\Sigma_{\boldsymbol{X}}$.

Definition 4.40: Bivariate Normal Distribution

From Eq. (4.114) In the case of $n = 2$, this becomes the bivariate normal distribution with a density represented as

$$f_{XY}(x, y; \sigma_X, \sigma_Y, \mu_X, \mu_Y, \rho) = \frac{1}{2\pi\sigma_X\sigma_Y\sqrt{1-\rho^2}} \times \exp\Big[-\frac{1}{2(1-\rho^2)} \left(\frac{(x-\mu_X)^2}{\sigma_X^2} - \frac{2\rho(x-\mu_X)(y-\mu_Y)}{\sigma_X\sigma_Y} + \frac{(y-\mu_Y)^2}{\sigma_Y^2}\right)\Big] \tag{4.108}$$

Here the elements of the mean and covariance matrix are

$$\mu_{\boldsymbol{X}} = \begin{bmatrix} \mu_X \\ \mu_Y \end{bmatrix}$$

$$\Sigma_{\boldsymbol{Y}} = \begin{bmatrix} \sigma_X^2 & \sigma_X\sigma_Y\rho \\ \sigma_X\sigma_Y\rho & \sigma_Y^2 \end{bmatrix}$$

Note that $\rho \in (-1, 1)$ is the correlation coefficient.

xii. Functions in SymIntegration Related to Continuous Distribution

[SI*] To compute the probability density function, cumulative density function, mean, variance, and moment generating function for the continuous distributions above we can use these functions in SymIntegration:
uniformpdf(x,a,b)
uniformcdf(x,a,b)
uniformmgf(x,a,b)
uniformmean(x,a,b)
uniformvar(x,a,b)

normalpdf(x,μ,σ)
normalcdf(x,μ,σ)
normalmgf(x,μ,σ)
normalmean(x,μ,σ)
normalvar(x,μ,σ)

gammapdf(x,α, β)
gammacdf(x,α, β)
gammamgf(x,α, β)
gammamean(x,α, β)
gammavar(x,α, β)

exponentialpdf(x,λ)
exponentialcdf(x,λ)

exponentialmgf(x,λ)
exponentialmean(x,λ)
exponentialvar(x,λ)

betapdf(x,α, β)
betacdf(x,α, β)
betamgf(x,α, β)
betamean(x,α, β)
betavar(x,α, β)

```
g++    -c -o main.o main.cpp
g++ -o main -ggdb  main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Continuous Distributions ]# ./main

uniformpdf(1;1,5) = 0.25

uniformcdf(3;1,5) = 0.5

uniformmgf(x;1,5) = 0.25*e^(5*t)*t^(-1)-0.25*e^t*t^(-1)

uniformmean(x;1,5) = 3

uniformvar(x;1,5) = 1.33333

normalpdf(362;300,50) = 0.00369875

normalcdf(362;300,50) = 0.892512

normalmgf(362;300,50) = e^(300*t+1250*t^(2))

normalmean(362;300,50) = 300

normalvar(362;300,50) = 2500

gammapdf(1,2,0.2) = 0.168449

gammacdf(1,2,0.2) = 0.959572

gammamgf(1,2,0.2) = (-0.2*t+1)^(-2)

gammamean(1,2,0.2) = 0.4

gammavar(1,2,0.2) = 0.08

exponentialpdf(8,0.2) = 0.0403793

exponentialcdf(8,0.2) = 0.798103

exponentialmgf(8,0.2) = (-5*t+1)^(-1)

exponentialmean(8,0.2) = 5

exponentialvar(8,0.2) = 25

betapdf(0.8,3,2) = 1.536

betacdf(0.8,3,2) = 0.8192

betamgf(0.8,3,2) = 0.6*t+0.2*t^(2)+0.047619*t^(3)+0.00892857*t^(4)+0.00138889*t^(5)+1

betamean(0.8,3,2) = 0.6

betavar(0.8,3,2) = 0.04

Time taken by function: 321002 microseconds
```

Figure 4.9: *The functions in SymIntegration to compute the probability density function, cumulative distribution function, mean, variance, and moment generating function for several continuous probability distributions (**SymIntegration/Examples/Statistics/Test SymIntegration Continuous Distributions/main.cpp**).*

[SI*] To generate random numbers that have certain continuous distributions we can use these

functions in SymIntegration:
vrandn_normal(μ**,** σ**,** n**)**
vrandn_exponential(λ**,** n**)**
vrandn_chisquared(ν**,** n**)**
vrandn_fdist(ν_1**,** ν_2**,** n**)**
vrandn_tdist(ν**,** n**)**
vrandn_erlang(k**,** λ**,** n**)**
vrandn_gamma(α**,** β**,** n**)**
vrandn_beta(α**,** β**,** n**)**

with n as the number of random number we want to generate, the result will be in **std::vector<double>**.

[SI*] To compute the quantile of certain distributions we can use these functions in SymIntegration:
zquantile(p**)**
(p is the desired probability, we are using Wichura's algorithm, a robust algorithm for accurate computation of z quantile, especially for extreme quantiles. This numerical approximation algorithm is very useful to compute the inverse cdf / quantile value of the normal distribution. Given a probability p, the inverse cdf returns the value x as the z quantile such that probability of a random variable being less than or equal to x is p, $P(X \leq x) = p$. We take it from John Burkardt's CPP consortium codes, named ASA 241, then blend it into SymIntegration.)
tquantile(x_0**,** ν**,** p**)**
(x_0 is the initial value for the approximation, you can input any value you want, ν is the degrees of freedom, p is the desired probability, we are using iterative method / Newton-Raphson method to find the root of $F(x) - p = 0$, where $F(x)$ is the cdf of the Student's t-distribution)

svec to replace **vector<string>**
We create a typedef defined in **include/symintegral/symintegrationc++.h** to represents **vector<string>**, so we don't need to write **vector<string>** anymore, we can use **svec** instead. This is very useful for data analysis.

Some explanations:

1. We used to call **Boost** to compute the pdf and cdf of some continuous distributions, but now we are releasing dependencies on **Boost** to make the program runs faster. For example, we used to use this code to compute the pdf of Beta distribution:

```
#include <boost/math/distributions/beta.hpp> //Faster computing of
    beta cdf and pdf

...

Symbolic betapdf(double x, double alpha, double beta)
{
      boost::math::beta_distribution<> my_beta(alpha,beta);
      double pdf_value = boost::math::pdf(my_beta,x);
```

```
    }

    ...
```

Code 56: *src/statistics.cpp*

Now we are using this code to compute the pdf of Beta distribution:

```
...

double betapdf(double x, double alpha, double beta)
{
if (x < 0.0 || x > 1.0 || alpha <= 0.0 || beta <= 0.0)
{
    // Handle invalid input: PDF is 0 outside [0,1] or parameters are
        invalid
    return 0.0;
}
else
{
    // Calculate log of the Beta function using log-Gamma functions
    double log_beta_function = std::lgamma(alpha) + std::lgamma(beta)
        - std::lgamma(alpha + beta);

    // Calculate log of the numerator
    double log_numerator = (alpha - 1.0) * std::log(x) + (beta -
        1.0) * std::log(1.0 - x);

    // Calculate the log of the PDF
    double log_pdf = log_numerator - log_beta_function;

    // Return the exponentiated value
    return std::exp(log_pdf);
}

}

...
```

Code 57: *src/statistics.cpp*

we can compute the pdf of Beta distribution by using the **lgamma** function from the **<cmath>** header to compute the logarithm of the Gamma function, which is more numerically stable for large values.

2. To generate random number of certain distribution, we make use of **std** library that already have a lot of continuous and discrete distributions covered.

First thing needed is to create / obtain a non-deterministic seed for the random number engine. Two good options currently are:
std::random_device
(create a non-deterministic seed if available and entropy > 0)

or
std::chrono::system_clock::now().time_since_epoch().count()
(provides a more robust seed than a fixed value)

Then we will use **std::mt19937** (Mersenne Twister engine) to generate the random number.

Here is an example of a function in **src/statistics.cpp** to generate a random number that has normal distribution:

```
...

std::vector<double> vrandn_normal(double mu, double sigma, int n)
{
       std::chrono::system_clock::now().time_since_epoch().count()

       std::default_random_engine generator(
       std::chrono::system_clock::now().time_since_epoch().count());

       std::vector<double> vec;
       std::normal_distribution<double> distribution(mu, sigma);
       for(int i=1; i<n; i++)
       {
              vec.push_back(static_cast<double>(distribution(generator)))
                  ;
       }
       return vec;
}

...
```

Code 58: *src/statistics.cpp*

3. The C++ Standard Library does not directly offer a **std::beta_distribution**. However, a Beta distribution can be simulated using two independently generated Gamma distributions.

   The relationship is as follows:
   If X is a random variable from a Gamma distribution with shape parameter α and scale parameter 1 (Gamma(α, 1)), and Y is a random variable from a Gamma distribution with shape parameter β and scale parameter 1 (Gamma(β, 1)), then the random variable

$$Z = \frac{X}{X+Y}$$

follows a Beta distribution with shape parameters α and β (denoted Beta(α, β)).

xiii. Generate Gamma Distributed Random Number in SymIntegration and Plot the Histogram with Hamzstplot

We will demonstrate how to generate random number with Gamma distribution then plot the data as histogram with Hamzstplot.

We will create a C++ source file **main.cpp**, and a Makefile (or compile it manually).

```
#include <iostream>
#include "symintegrationc++.h"
#include <bits/stdc++.h>
#include <cmath>

#include <chrono>

using namespace std::chrono;
using namespace std;
using namespace SymbolicConstant;

int main(void)
{
	// Get starting timepoint
	auto start = high_resolution_clock::now();

	std::vector<double> x = vrandn_gamma(2,1.5,50);
	for(int i=0;i<50;i++)
	{
		cout<< x[i] << endl;
	}

	// Get ending timepoint
	auto stop = high_resolution_clock::now();
	auto duration = duration_cast<microseconds>(stop - start);

	cout << "\nTime taken by function: " << duration.count() << " microseconds"
	     << endl;

	return 0;
}
```

Code 59: *main.cpp*

```
CFLAGS = -ggdb
DEFINES = -DDEBUGGA
INCLUDES =
LIBS = -lstdc++ -lsymintegration
MAIN = main.o
```

```
CC=g++

.cc.o:
$(CC) —c $(CFLAGS) $(DEFINES) $(INCLUDES) $<

all:: main

gnuplot_i.o:
main.o: main.cpp

main: $(MAIN)
$(CC) —o $@ $(CFLAGS) $(MAIN) $(LIBS)

clean:
rm —f $(MAIN) main
```

Code 60: *Makefile*

From the current working directory containing the **main.cpp** and the makefile, type:
make
./main

to save the output you can also type this:
./main > gamma.dat

If you are still learning about Makefile, or feel more confident with manual compiling, then instead of **make** you can type this:
g++ -o main main.cpp -lsymintegration
./main

```
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Generate Random Number from Gamma Distribution ]# ./main
1.49028
4.10471
1.345
0.465866
1.34767
2.26934
1.07946
4.17375
1.48454
3.77183
2.87478
0.318233
3.1834
1.90781
3.69633
1.51896
4.32441
2.56319
6.54034
2.51703
3.25444
4.40127
1.68972
0.734017
3.02267
6.22162
4.24151
1.61816
5.48238
3.12266
1.6428
5.69689
1.54791
3.17954
5.72097
5.6629
4.99465
2.66944
6.83603
2.46991
1.13762
2.35658
0.503396
3.15512
5.12802
4.61204
4.52307
5.98776
4.05892
0

Time taken by function: 1008 microseconds
```

Figure 4.10: *The random number generation that has Gamma distribution with parameters* $\alpha = 2, \beta = 1.5$*, we generate* 50 *random numbers. (**SymIntegration/Examples/Statistics/Test SymIntegration Generate Random Number from Gamma Distribution/main.cpp**).*

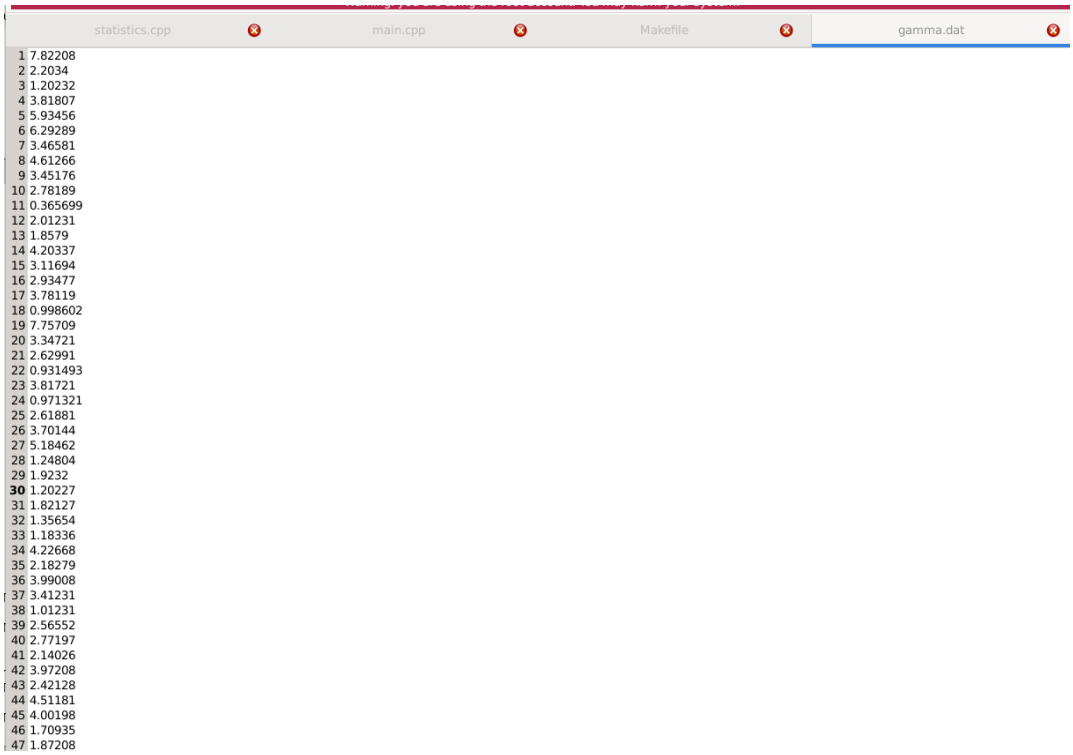

Figure 4.11: *The random number generation that has been saved in* ***gamma.dat****. (****SymIntegration/Examples/Statistics/Test SymIntegration Generate Random Number from Gamma Distribution/main.cpp****).*

Now, if we want to plot the histogram, we will need to add **Hamzstplot** library when compiling and add few more lines of code.

We will create again a C++ source file **main.cpp**, and a Makefile (or compile it manually).

```
#include <cmath>
#include <hamzstplot/hamzstplot.h>
#include <random>
#include "symintegrationc++.h"

using namespace std;

int main() {
	using namespace hamzstplot;

	std::vector<double> x =vrandn_gamma(2, 1.5,1000);

	auto h = hist(x);
	std::cout << "Histogram with " << h->num_bins() << " bins" << std::endl;

	show();

	return 0;
```

```
}
```

Code 61: *main.cpp*

```
CFLAGS = -ggdb
DEFINES = -DDEBUGGA
INCLUDES =
LIBS = -lstdc++ -lhamzstplot -lsymintegration
MAIN = main.o
CC=g++

.cc.o:
$(CC) -c $(CFLAGS) $(DEFINES) $(INCLUDES) $<

all:: main

gnuplot_i.o:
main.o: main.cpp

main: $(MAIN)
$(CC) -o $@ $(CFLAGS) $(MAIN) $(LIBS)

clean:
rm -f $(MAIN) main
```

Code 62: *Makefile*

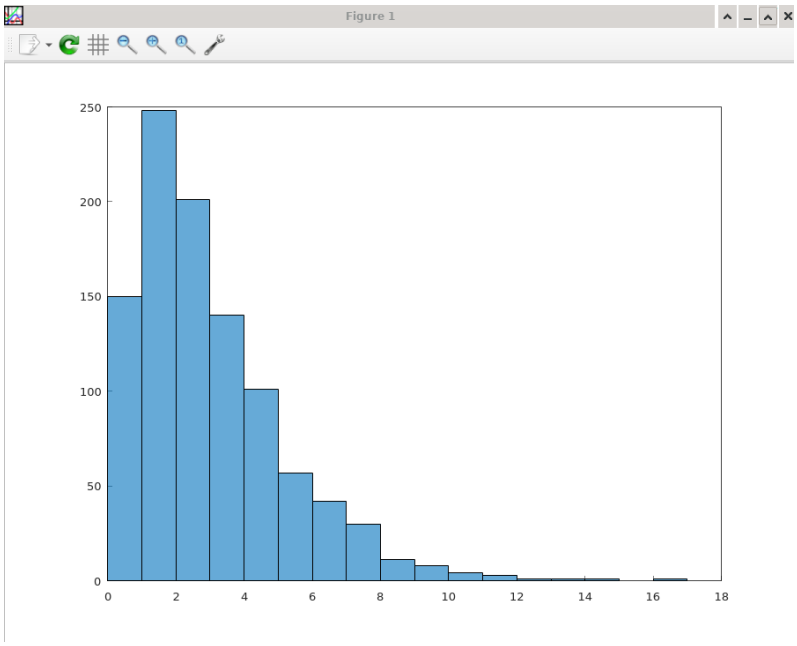

Figure 4.12: *The histogram of* 1000 *random numbers that has been generated with Gamma distribution of parameters* $\alpha = 2, \beta = 1.5$. *(**Hamzstplot/Examples with Makefile/Statistics/Plot 2D Histogram of Random Number/main.cpp**).*

V. Statistical Inference

[SI*] The action of statistical inference involves using mathematical techniques to make conclusions about unknown population parameters based on collected data. The field of statistical inference employs a variety of stochastic models to analyze and put forward efficient methods for carrying out such analyses.

Analysis and methods if statistical inference can be categorized as either frequentist (also known as classical) or Bayesian. The former is based on the assumption that population parameters of some underlying distribution, or probability law, exist and are fixed, but are yet unknown. The process of statistical inference then deals with making conclusions about these parameters based on sampled data. In the latter Bayesian case, it is only assumed that there is a prior distribution of the parameters. In this case, the key process deals with analyzing a posterior distribution (of the parameters)-an outcome of the inference process.

In general, a statistical inference process involves data, a model, and analysis. The data is assumed to be comprised of random samples from the model. The goal of the analysis is then to make informed statements about population parameters of the model based on the data. Such statements typically take one of the following forms:

1. **Point Estimation**
   Determination of a single value (or vector of values) representing a best estimate of the parameter / parameters. In this case, the notion of "best" can be defined in different ways.

2. **Confidence Intervals**
   Determination of a range of values where the parameter lies. Under the model and the statistical process used, it is guaranteed that the parameter lies within this range with a pre-specified probability.

3. **Hypothesis Tests**
   The process of determining if the parameter lies in a given region, in the complement of that region, or fails to take on a specific value. Such tests often represent a scientific hypothesis in a very natural way

[SI*] **A Random Sample**
When carrying out (frequentist) statistical inference, we assume there is some underlying distribution $F(x, \theta)$ from which we are sampling, where θ is the scalar or vector-valued unknown parameter we wish to know. Importantly, we assume that each observation is statistically independent and identically distributed as the rest. That is, from a probabilistic perspective, the observations are taken as independent and identically distributed (i.i.d.) random variables. In mathematical statistics language, this is called a random sample. We denote the random variables of the observations by $X_1, \ldots, X_n$ and their respective values $x_1, \ldots, x_n$.

Typically, we compute statistics from the random sample. For example, two common standard statistics include the sample mean and sample variance. However, we can model

these statistics as random variables

$$\begin{aligned} \overline{X} &= \frac{1}{n}\sum_{i=1}^{n} X_i \\ S^2 &= \frac{1}{n-1}\sum_{i=1}^{n}(X_i - \overline{X})^2 \end{aligned} \tag{4.109}$$

Note that for S^2, the denominator is $n-1$ to make S^2 an unbiased estimator.

In general, the phrase statistic implies a quantity calculated based on the sample. When working with data, the sample mean and sample variance are nothing but numbers computed from our sample observations. However, in the statistical inference paradigm, we associate random variables to these values, since they themselves are functions of the random sample.

To illustrate the fact that $\overline{X}$ and S^2 are random variables, assume we have sampled data from certain distribution (e.g. exponential). If we collect $n = 10$ observations, then the sample mean and sample variance are random variables. The point to see is that $\overline{X}$ and S^2 are themselves random variale with underlying distributions.

[SI*] Statistical inference consists of some methods by which one makes inferences or generalizations about a population. The trend today is to distinguish between the classical method of estimating a population parameter, whereby inferences are absed strictly on information obtained from a random sample selected from the population, and the Bayesian method, which utilizes prior subjective knowledge about the probability distribution of the unknown parameters in conjunction with the information provied by the sample data.

On this book, the sections on confidence interval before the Bayesian statistics section are the so-called classical methods to estimate unknown population parameters such as the mean, the proportion, and the variance by computing statistics from random samples and applying the theory of sampling distributions.

i. Sampling from a Normal Population

[SI*] It is often assumed that the distribution $F(x;\theta)$ is a normal distribution, and hence $\theta = (\mu, \sigma^2)$. This assumption is called the normality assumption, and is sometimes justified due to the central limit theorem. Under the normality assumption, the distribution of the random variables $\overline{X}$ and S^2 as well as transformations of them are well known. The following three distributional relationships play a key role:

$$\begin{aligned} \overline{X} \text{ Normal}\left(\mu, \frac{\sigma^2}{n}\right) \\ \frac{(n-1)S^2}{\sigma^2} \ \chi^2_{n-1} \\ T := \frac{\overline{X} - \mu}{S/\sqrt{n}} \ t_{n-1} \end{aligned} \tag{4.110}$$

Here, " " denotes as "distributed as," and implies that the statistics on the left hand side of the " " symbols are distributed according to the distribution on the right hand side. The notation

χ^2_{n-1} and t_{n-1} denotes a chi-squared and t-distribution, respectively, each with $n-1$ degrees of freedom. The chi-squared distribution is a gamma distribution with parameters $\alpha = \frac{n}{2}$ and $\beta = 2$, some textbooks, e.g. [22], use another parameter thus state that $\lambda = \frac{1}{2}$ because $\lambda = \frac{1}{\beta}$, for Gamma distribution in this book we use parameters α and β.

ii. The Central Limit Theorem

[SI*] The Central Limit Theorem (CLT) is one of the most fundamental results of probability and statistics. It has several versions and many generalizations, they all have one thing in common: summation of a large number of random quantities, each with finite variance, yield a sum that is approximately normally distributed. This is the main reason that the normal distribution is ubiquitous in nature and present throughout the universe.

Theorem 4.19: Central Limit Theorem

If $\overline{X}$ is the mean of a random sample of size n taken from a population with mean μ and finite variance σ^2, then the limiting form of the distribution of

$$Z = \frac{\overline{X} - \mu}{\sigma/\sqrt{n}}$$

as $n \to \infty$, is the standard normal distribution $n(z; 0, 1)$.

iii. Point Estimation

[SI*] A point estimate of some population parameter θ is a single value $\hat{\theta}$ of a statistic $\hat{\Theta}$. Fo example, the value $\overline{x}$ of the statistic $\overline{X}$, computed from a sample of size n, is a point estimate of the population parameter μ.

Definition 4.41: Unbiased Estimator

A statistic $\hat{\Theta}$ is said to be an unbiased estimator of the parameter θ if

$$\mu_{\hat{\Theta}} = E(\hat{\Theta}) = \theta$$

Definition 4.42: Most Efficient Estimator

If we consider all possible unbiased estimators of some parameter θ, the one with the smallest variance is called the most efficient estimator of θ.

iv. Confidence Interval as a Concept

[SI*] Even the most efficient unbiased estimator is unlikely to estimate the population parameter exactly. It is true that estimation accuracy increases with large samples, but there is still no reason we should expect a point estimate to be exactly equal to the population parameter it is supposed to estimate. There are many situations in which it is preferable to determine an interval within which we would expect to find the value of the parameter. Such an interval

is called an interval estimate.

An interval estimate of a population parameter θ is an interval of the form $\hat{\theta}_L < \theta < \hat{\theta}_U$, where $\hat{\theta}_L$ and $\hat{\theta}_U$ depend on the value of the statistic $\hat{\Theta}$ for a particular sample and also on the sampling distribution of $\hat{\Theta}$.

[SI*] Since different samples will generally yield different values of $\hat{\Theta}$ and, therefore different values of $\hat{\theta}_L$ and $\hat{\theta}_U$, these endpoints of the interval are values of coresponding random variables $\hat{\Theta}_L$ and $\hat{\Theta}_U$. From the sampling distribution of $\hat{\Theta}$ we shall be able to determine $\hat{\Theta}_L$ and $\hat{\Theta}_U$ such that $P(\hat{\Theta}_L < \theta < \hat{\Theta}_U)$ is equal to any positive fractional value we care to specify. If, for instance, we find $\hat{\Theta}_L$ and $\hat{\Theta}_U$ such that

$$P(\hat{\Theta}_L < \theta < \hat{\Theta}_U) = 1 - \alpha$$

for $0 < \alpha < 1$, then we have a probability of $1 - \alpha$ of selecting a random sample that will produce an interval containing θ. The interval $\hat{\theta}_L < \theta < \hat{\theta}_U$, computed from the selected sample, is called a $100(1-\alpha)\%$ confidence interval, the fraction $1 - \alpha$ is called the confidence coefficient or the degree of confidence, and the endpoints, $\hat{\theta}_L$ and $\hat{\theta}_U$, are called the lower and upper confidence limits. Thus, when $\alpha = 0.05$, we have a 95% confidence interval, and when $\alpha = 0.01$ we obtaina wider 99% confidence interval. The wider the confidence interval is, the more confident we can be that the interval contains the unknown parameter.

Definition 4.43: Confidence Interval on μ, σ^2 Known

If $\overline{x}$ is the mean of a random sample of size n from a population with known variance $\sigma 2$, a $100(1-\alpha)\%$ confidence interval for μ is given by

$$\overline{x} - z_{\alpha/2}\frac{\sigma}{\sqrt{n}} < \mu < \overline{x} + z_{\alpha/2}\frac{\sigma}{\sqrt{n}}$$

where $z_{\alpha/2}$ is the z-value leaving an area of $\alpha/2$ to the right.

Theorem 4.20: Error in Confidence Interval

If $\overline{x}$ is used as an estimate of μ, we can be $100(1-\alpha)\%$ confident that the error will not exceed $z_{\alpha/2}\frac{\sigma}{\sqrt{n}}$.

Theorem 4.21: Sample Size for Confidence Interval

If $\overline{x}$ is used as an estimate of μ, we can be $100(1-\alpha)\%$ confident that the error will not exceed a specified amount e when the sample size is

$$n = \left(\frac{z_{\alpha/2}\sigma}{e}\right)^2$$

Definition 4.44: One-Sided Confidence Bounds on μ, σ^2 Known

If $\overline{X}$ is the mean of a random sample of size n from a population with variance σ^2, the one-sided $100(1-\alpha)\%$ confidence bounds for μ are given by

upper-one sided bound: $\overline{x} + z_\alpha \frac{\sigma}{\sqrt{n}}$.

lower-one sided bound: $\overline{x} - z_\alpha \frac{\sigma}{\sqrt{n}}$.

[SI*] If we have a random sample from a normal distribution, then the random variable

$$T = \frac{\overline{X} - \mu}{S/\sqrt{n}}$$

has a Student' t-distribution with $n-1$ degrees of freedom. Here S is the sample standard deviation. In this situation, with σ unknown, T can be used to construct a confidence interval on μ.

Definition 4.45: Confidence Interval on μ, σ^2 Unknown

If $\overline{x}$ and s are the mean and standard deviation of a random sample from a normal population with unknown variance $\sigma 2$, a $100(1-\alpha)\%$ confidence interval for μ is given by

$$\overline{x} - t_{\alpha/2}\frac{s}{\sqrt{n}} < \mu < \overline{x} + t_{\alpha/2}\frac{s}{\sqrt{n}}$$

where $t_{\alpha/2}$ is the t-value with $\nu = n-1$ degrees of freedom, leaving an area of $\alpha/2$ to the right.

[SI*] We should emphasize that for σ known we exploited the Central Limit Theorem, whereas for σ unknown we made use of the sampling distribution of the random variable T.

[SI*] Often statisticians recommend that even when normality cannot be assumed, σ is unknown, and $n \geq 30$, s can replace σ and the confidence interval

$$\overline{x} \pm z_{\alpha/2}\frac{s}{\sqrt{n}}$$

may be used. This is often referred to as a large-sample confidence interval.

[SI*] A common way to calculate a confidence interval is by using the following formula:
Confidence interval = **Point estimate** $\pm$ **Margin of Error**

The sample mean is often used as the point estimate, and the required margin on error depends on the desired confidence level and the variability of the data, calculated as a critical value (e.g., a z-value or t-value) multiplied by the standard error of the mean.

[SI*] The confidence interval provides a range of plausible values for a population parameter. The confidence level indicates the reliability of the method used to create the interval. For example, a 95% confidence interval for the average height of a certain plant species suggests that we are 95% confident that the true average height of the species falls between 10 and 15.

[SI*] Sometimes, other than the population mean, the experimenter may also be interested in predicting the possible value of a future observation. For instance, in quality control, the experimenter may need to use the observed data to predict a new observation.

Definition 4.46: Prediction Interval of a Future Observation, σ^2 Known

For a normal distribution of a measurements with unknown mean μ and known variance σ^2, a $100(1-\alpha)\%$ prediction interval of a future observation X_0 is

$$\overline{x} - z_{\alpha/2}\sigma\sqrt{1+\frac{1}{n}} < X_0 < \overline{x} + z_{\alpha/2}\sigma\sqrt{1+\frac{1}{n}}$$

where $z_{\alpha/2}$ is the z-value leaving an area of $\alpha/2$ to the right.

Definition 4.47: Prediction Interval of a Future Observation, σ^2 Unknown

For a normal distribution of a measurements with unknown mean μ and unknown variance σ^2, a $100(1-\alpha)\%$ prediction interval of a future observation X_0 is

$$\overline{x} - t_{\alpha/2}s\sqrt{1+\frac{1}{n}} < X_0 < \overline{x} + t_{\alpha/2}s\sqrt{1+\frac{1}{n}}$$

where $t_{\alpha/2}$ is the t-value with $\nu = n-1$ degrees of freedom, leaving an area of $\alpha/2$ to the right.

[SI*] If we have two populations with means μ_1 and μ_2 and variance σ_1^2 and σ_2^2, respectively, a point estimator of the difference between μ_1 and μ_2 is given by statistic $\overline{X}_1 - \overline{X}_2$.

Definition 4.48: Confidence Interval for $\mu_1 - \mu_2$, σ_1^2 and σ_2^2 Known

If $\overline{x}_1$ and $\overline{x}_2$ are means of independent random samples of sizes n_1 and n_2 from populations with known variances σ_1^2 and σ_2^2, respectively, a $100(1-\alpha)\%$ confidence interval for $\mu_1 - \mu_2$ is given by

$$(\overline{x}_1 - \overline{x}_2) - z_{\alpha/2}\sqrt{\frac{\sigma_1^2}{n_1}+\frac{\sigma_2^2}{n_2}} < \mu_1 - \mu_2 < (\overline{x}_1 - \overline{x}_2) + z_{\alpha/2}\sqrt{\frac{\sigma_1^2}{n_1}+\frac{\sigma_2^2}{n_2}}$$

where $z_{\alpha/2}$ is the z-value leaving an area of $\alpha/2$ to the right.

[SI*] Consider the case where σ_1^2 and σ_2^2 are unknown. If $\sigma_1^2 = \sigma_2^2 = \sigma^2$, we obtain a standard normal variable of the form

$$Z = \frac{(\overline{X}_1 - \overline{X}_2) - (\mu_1 - \mu_2)}{\sqrt{\sigma^2\left(\frac{1}{n_1}+\frac{1}{n_2}\right)}}$$

The two random variables

$$\frac{(n_1-1)S_1^2}{\sigma^2} \quad \text{and} \quad \frac{(n_2-1)S_2^2}{\sigma^2}$$

have chi-squared distributions with $n_1 - 1$ and $n_2 - 1$ degrees of freedom, respectively. Furthermore, they are independent chi-squared variables, since the random samples were selected independently. Consequently their sum

$$V = \frac{(n_1-1)S_1^2}{\sigma^2} + \frac{(n_2-1)S_2^2}{\sigma^2} = \frac{(n_1-1)S_1^2 + (n_2-1)S_2^2}{\sigma^2}$$

has a chi-squared distribution with $\nu = n_1 + n_2 - 2$ degrees of freedom.

Since the preceding expressions for Z and V can be shown to be independent, it follows that the statistic

$$T = \frac{(\overline{X}_1 - \overline{X}_2) - (\mu_1 - \mu_2)}{\sqrt{\sigma^2\left(\frac{1}{n_1} + \frac{1}{n_2}\right)}} \cdot \frac{1}{\frac{(n_1-1)S_1^2+(n_2-1)S_2^2}{\sigma^2(n_1+n_2-2)}}$$

has the t-distribution with $\nu = n_1 + n_2 - 2$ degrees of freedom.

Definition 4.49: Pooled Estimate of Variance

A point estimate of the unknown common variance σ^2 can be obtained by pooling the sample variances. Denoting the pooled estimator by S_p^2, we have the following

$$S_p^2 = \frac{(n_1 - 1)S_1^2 + (n_2 - 1)S_2^2}{n_1 + n_2 - 2}$$

Substituting S_p^2 in the T statistic, we obtain the less cumbersome form

$$T = \frac{(\overline{X}_1 - \overline{X}_2) - (\mu_1 - \mu_2)}{S_p\sqrt{\left(\frac{1}{n_1} + \frac{1}{n_2}\right)}}$$

Using the T statistic, we have

$$P(-t_{\alpha/2} < T < t_{\alpha/2}) = 1 - \alpha$$

where $t_{\alpha/2}$ is the t-value with $n_1 + n_2 - 2$ degrees of freedom, above which we find an area of $\alpha/2$. Substituting for T in the inequality, we write

$$P\left(-t_{\alpha/2} < \frac{(\overline{X}_1 - \overline{X}_2) - (\mu_1 - \mu_2)}{S_p\sqrt{\left(\frac{1}{n_1} + \frac{1}{n_2}\right)}} < t_{\alpha/2}\right) = 1 - \alpha$$

After the usual mathematical manipulations, the difference of the sample means $\overline{x}_1 - \overline{x}_2$ and the pooled variance are computed and then the following $100(1 - \alpha)\%$ confidence interval for $\mu_1 - \mu_2$ is obtained.

The value of s_p^2 is easily seen to be a weighted average of the two sample variances s_1^2 and s_2^2, where the weights are the degrees of freedom.

Definition 4.50: Confidence Interval for $\mu_1 - \mu_2$, $\sigma_1^2 = \sigma_2^2$ but Both Unknown

If $\bar{x}_1$ and $\bar{x}_2$ are means of independent random samples of sizes n_1 and n_2 from approximately normal populations with unknown but equal variances, a $100(1-\alpha)\%$ confidence interval for $\mu_1 - \mu_2$ is given by

$$(\bar{x}_1 - \bar{x}_2) - t_{\alpha/2} s_p \sqrt{\frac{1}{n_1} + \frac{1}{n_2}} < \mu_1 - \mu_2 < (\bar{x}_1 - \bar{x}_2) + t_{\alpha/2} s_p \sqrt{\frac{1}{n_1} + \frac{1}{n_2}}$$

where s_p is the pooled estimate of the population standard deviation and $t_{\alpha/2}$ is the t-value with $\nu = n_1 + n_2 - 2$ degrees of freadom leaving an area of $\alpha/2$ to the right.

Definition 4.51: Confidence Interval for $\mu_1 - \mu_2$, $\sigma_1^2 \neq \sigma_2^2$ and Both Unknown

If $\bar{x}_1$ and s_1^2 and $\bar{x}_2$ and s_2^2 are the means and variance of independent random samples of sizes n_1 and n_2, respectively, from approximately normal populations with unknown and unequal variances, a $100(1-\alpha)\%$ confidence interval for $\mu_1 - \mu_2$ is given by

$$(\bar{x}_1 - \bar{x}_2) - t_{\alpha/2} \sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}} < \mu_1 - \mu_2 < (\bar{x}_1 - \bar{x}_2) + t_{\alpha/2} \sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}$$

where $t_{\alpha/2}$ is the t-value with

$$\nu_{\text{raw}} = \frac{\left(\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}\right)^2}{\frac{\left(\frac{s_1^2}{n_1}\right)^2}{n_1 - 1} + \frac{\left(\frac{s_2^2}{n_2}\right)^2}{n_2 - 1}}$$

$\nu = \lfloor \nu_{\text{raw}} \rfloor$ is the degrees of freedom leaving an area of $\alpha/2$ to the right.

[SI*] **Paired Observations**

We shall consider estimation procedures for the difference of two means when the samples are not independent and the variances of the two populations are not necessarily equal.

The conditions of the two populations are not assigned randomly to experimental units. Rather, each homogeneous experimental unit receives both population conditions; as a result, each experimental unit has a pair of observations, one for each observation. For example, if we run a test on a new diet using 15 individuals, the weights before and after going on the diet form the information for our two samples. The two populations are "before" and "after," and the experimental unit is the individual. Obviously, the observations in a pair have something in common. To determine if the diet is effective, we consider the differences $d_1, d_2, \ldots, d_n$ in the paired observations. These differences are the values of a random sample $D_1, D_2, \ldots, D_n$ from a population of differences that we shall assume to be normally distributed with mean $\mu_D = \mu_1 - \mu_2$ and variance σ_D^2. We estimate σ_D^2 by s_d^2, the variance of the differences that constitute our sample. the point estimator of μ_D is given by $\overline{D}$.

The reader may visualize the i-th pair difference as

$$D_i = X_{1i} - X_{2i}$$

Since the two observations are taken on the sample experimental unit, they are not independent and, in fact,

$$\text{Var}(D_i) = \text{Var}(X_{1i} - X_{2i}) = \sigma_1^2 + \sigma_2^2 - 2\text{Cov}(X_{1i}, X_{2i})$$

One certainly expects that if the unit is homogeneous, the covariance is positive. As a result, the gain in quality of the confidence interval over that obtained without pairing will be greatest when there is homogeneity within units and large differences as one goes from unit to unit.

[SI*] Although pairing should indeed reduce variance and hence reduce the standard error of the point estimate, the degrees of freedom are reduced by reducing the problem to a one-sample problem. As a result, the $t_{\alpha/2}$ point attached to the standard error is adjusted accordingly. Thus, pairing may be counterproductive.

[SI*] A $100(1-\alpha)\%$ confidence interval for μ_D can be established by writing

$$P(-t_{\alpha/2} < T < t_{\alpha/2}) = 1 - \alpha$$

where $T = \frac{\overline{D} - \mu_D}{S_d/\sqrt{n}}$ and $t_{\alpha/2}$, as before, is a value of the t-distribution with $n-1$ degrees of freedom.

Definition 4.52: Confidence interval for $\mu_D = \mu_1 - \mu_2$ for Paired Observations

If $\overline{d}$ and s_d are the mean and standard deviation, respectively, of the normally distributed differences of n random pairs of measurements, a $100(1-\alpha)\%$ confidence interval for $\mu_D = \mu_1 - \mu_2$ is

$$\overline{d} - t_{\alpha/2}\frac{s_d}{\sqrt{n}} < \mu_D < \overline{d} + t_{\alpha/2}\frac{s_d}{\sqrt{n}}$$

where $t_{\alpha/2}$ is the t-value with $\nu = n-1$ degrees of freedom, leaving an area of $\alpha/2$ to the right.

[SI*] **Single Sample: Estimating a Proportion**

A point estimator of the proportion p in a binomial experiment is given by the statistic $\hat{P} = \frac{X}{n}$, where X represents the number of successes in n trials. Therefore, the sample proportion $\hat{p} = \frac{x}{n}$ will be used as the point estimate of the parameter p.

If the unknown propotion p is not expected to be too close to 0 or 1, we can establish a confidence interval for p by considering the sampling distribution of $\hat{P}$. Designating a failure in each binomial trial by the value 0 and a success by the value 1, the number of successes, can be interpreted as the sum of n values consisting only of 0 and 1s, and $\hat{p}$ is just the sample mean of these n values. Hence, by the Central Limit Theorem, for n sufficiently large, $\hat{P}$ is approximately normally distributed with mean

$$\mu_{\hat{p}} = E(\hat{P}) = E\left(\frac{X}{n}\right) = \frac{np}{n} = p$$

and variance

$$\sigma^2_{\hat{P}} = \sigma_{\frac{X}{n}} = \frac{\sigma^2_X}{n^2} = \frac{npq}{n^2} = \frac{pq}{n}$$

Therefore, we can assert that

$$P(-z_{\alpha/2} < Z < z_{\alpha/2}) = 1 - \alpha$$

with

$$Z = \frac{\hat{P} - p}{\sqrt{\frac{pq}{n}}}$$

and $z_{\alpha/2}$ is the z-value leaving an area of $\alpha/2$ to the right. Substituting for Z, we write

$$P\left(-z_{\alpha/2} < \frac{\hat{P} - p}{\sqrt{\frac{pq}{n}}} < z_{\alpha/2}\right) = 1 - \alpha$$

When n is large, very little error is introduced by substituting the point estimate $\hat{p} = \frac{x}{n}$ for the p under the radical sign. Then we can write

$$P\left(\hat{P} - z_{\alpha/2}\sqrt{\frac{pq}{n}} < p < \hat{P} + z_{\alpha/2}\sqrt{\frac{pq}{n}}\right) \approx 1 - \alpha$$

On the other hand, by solving for p in the quadratic equality above,

$$-z_{\alpha/2} < \frac{\hat{P} - p}{\sqrt{\frac{pq}{n}}} < z_{\alpha/2}$$

we obtain another form of the confidence interval for p with limits

$$\frac{\hat{p} + \frac{z^2_{\alpha/2}}{2n}}{1 + \frac{z^2_{\alpha/2}}{n}} \pm \frac{z_{\alpha/2}}{1 + \frac{z^2_{\alpha/2}}{n}}\sqrt{\frac{\hat{p}\hat{q}}{n} + \frac{z^2_{\alpha/2}}{4n^2}}$$

Definition 4.53: Large Sample Confidence Intervals for p

If $\hat{p}$ is the proportion of successes in a random sample of size n and $\hat{q} = 1 - \hat{p}$, an approximate $100(1-\alpha)\%$ confidence interval, for the binomial parameter p is given by (method 1)

$$\hat{p} - z_{\alpha/2}\sqrt{\frac{pq}{n}} < p < \hat{p} + z_{\alpha/2}\sqrt{\frac{pq}{n}}$$

or by (method 2)

$$\frac{\hat{p} + \frac{z^2_{\alpha/2}}{2n}}{1 + \frac{z^2_{\alpha/2}}{n}} - \frac{z_{\alpha/2}}{1 + \frac{z^2_{\alpha/2}}{n}}\sqrt{\frac{\hat{p}\hat{q}}{n} + \frac{z^2_{\alpha/2}}{4n^2}} < p < \frac{\hat{p} + \frac{z^2_{\alpha/2}}{2n}}{1 + \frac{z^2_{\alpha/2}}{n}} + \frac{z_{\alpha/2}}{1 + \frac{z^2_{\alpha/2}}{n}}\sqrt{\frac{\hat{p}\hat{q}}{n} + \frac{z^2_{\alpha/2}}{4n^2}}$$

where $z_{\alpha/2}$ is the z-value leaving an area of $\alpha/2$ to the right.

When n is small and the unknown proportion p is believed to be close to 0 or to 1, the confidence-interval procedure established here is unreliable and, therefore, should not be

used. To be on the safe side, one should require both $n\hat{p}$ and $n\hat{q}$ to be greater than or equal to 5.

Theorem 4.22: Error in estimating p by $\hat{p}$

If $\hat{p}$ is used as an estimate of p, we can be $100(1-\alpha)\%$ confident that the error will not exceed $z_{\alpha/2}\sqrt{\frac{\hat{p}\hat{q}}{n}}$.

Theorem 4.23: Choice of Sample Size

If $\hat{p}$ is used as an estimate of p, we can be $100(1-\alpha)\%$ confident that the error will be less than a specified amount e when the sample size is approximately

$$n = \frac{z_{\alpha/2}^2 \hat{p}\hat{q}}{e^2}$$

Theorem 4.24: Determine Sample Size Without $\hat{p}$

If $\hat{p}$ is used as an estimate of p, we can be at least $100(1-\alpha)\%$ confident that the error will not exceed a specified amount e when the sample size is

$$n = \frac{z_{\alpha/2}^2}{4e^2}$$

[SI*] **Two Samples: Estimating the Difference Between Two Proportions**
Consider the problem where we wish to estimate the difference between two binomial parameters p_1 and p_2. First, we select independent random samples of sizes n_1 and n_2 from the two binomial populations with means $n_1 p_1$ and $n_2 p_2$ and variances $n_1 p_1 q_1$ and $n_2 p_2 q_2$, respectively; then we determine the numbers x_1 and x_2 of people in each sample with the target criteria (e.g. defective, lung cancer, etc) and form the proportions

$$\hat{p}_1 = \frac{x_1}{n}$$
$$\hat{p}_2 = \frac{x_2}{n}$$

A point estimator of the difference between the two poportions, $p_1 - p_2$, is given by the statistic $\hat{P}_1 - \hat{P}_2$. Therefore, the difference of the sample proportions, $\hat{p}_1 - \hat{p}_2$, will be used as the point estimate of $p_1 - p_2$.

We know that $\hat{P}_1$ and $\hat{P}_2$ are each approximately normally distributed, with means p_1 and p_2 and variances $\frac{p_1 q_1}{n_1}$ and $\frac{p_2 q_2}{n_2}$, respectively. Choosing independent samples from the two populations ensures that the variables $\hat{P}_1$ and $\hat{P}_2$ will be independent, and then by the reproductive property of the normal distribution, we conclude that $\hat{P}_1 - \hat{P}_2$ is approximately normally distributed with mean

$$\mu_{\hat{P}_1 - \hat{P}_2} = p_1 - p_2$$

and variance

$$\sigma^2_{\hat{P}_1 - \hat{P}_2} = \frac{p_1 q_1}{n_1} + \frac{p_2 q_2}{n_2}$$

Therefore, we can assert that

$$P(-z_{\alpha/2} < Z < z_{\alpha/2}) = 1 - \alpha$$

where

$$Z = \frac{(\hat{P}_1 - \hat{P}_2) - (p_1 - p_2)}{\sqrt{\frac{p_1 q_1}{n_1} + \frac{p_2 q_2}{n_2}}}$$

and $z_{\alpha/2}$ is the value above which we find an area of $\alpha/2$ to he right under the standard normal curve. Substituting for Z, we write

$$P\left(-z_{\alpha/2} < \frac{(\hat{P}_1 - \hat{P}_2) - (p_1 - p_2)}{\sqrt{\frac{p_1 q_1}{n_1} + \frac{p_2 q_2}{n_2}}} < z_{\alpha/2}\right)$$

After performing the usual mathematical manipulations, we replace p_1, p_2, q_1, and q_2 under the radiacal sign by their estimates $\hat{p}_1 = \frac{x_1}{n_1}, \hat{p}_2 = \frac{x_2}{n_2}, \hat{q}_1 = 1 - \hat{p}_1$, and $\hat{q}_2 = 1 - \hat{p}_2$, provided that $n_1\hat{p}_1, n_1\hat{q}_1, n_2\hat{p}_2$, and $n_2\hat{q}_2$ are all greater than or equal to 5.

Definition 4.54: Large Sample Confidence Interval for $p_1 - p_2$

If $\hat{p}_1$ and $\hat{p}_2$ are the proportions of successes in random samples of sizes n_1 and n_2, respectively, $\hat{q}_1 = 1 - \hat{p}_1$, and $\hat{q}_2 = 1 - \hat{p}_2$, an approximate $100(1-\alpha)\%$ confidence interval for the difference of two binomial parameters, $p_1 - p_2$, is given by

$$(\hat{p}_1 - \hat{p}_2) - z_{\alpha/2}\sqrt{\frac{\hat{p}_1\hat{q}_1}{n_1} + \frac{\hat{p}_2\hat{q}_2}{n_2}} < p_1 - p_2 < (\hat{p}_1 - \hat{p}_2) + z_{\alpha/2}\sqrt{\frac{\hat{p}_1\hat{q}_1}{n_1} + \frac{\hat{p}_2\hat{q}_2}{n_2}}$$

where $z_{\alpha/2}$ is the z-value leaving an area of $\alpha/2$ to the right.

[SI*] **Single Sample: Estimating the Variance**

If a sample of size n is drawn from a normal population with variance σ^2 and the sample variance s^2 is computed, we obtain a value of the statistic S^2. This computed sample variance is used as a point estimate of σ^2. Hence, the statistic S^2 is called an estimator of σ^2.

An interval estimate of σ^2 can be establishedby using this statistic

$$X^2 = \frac{(n-1)S^2}{\sigma^2}$$

The statistic X^2 has a chi-squared distribution with $n-1$ degrees of freedom when samples are chosen from a normal population. We may write

$$P(\chi^2_{1-\frac{\alpha}{2}} < X^2 < \chi^2_{\alpha/2}) = 1 - \alpha$$

where $\chi^2_{1-\frac{\alpha}{2}}$ and $\chi^2_{\alpha/2}$ are values of the chi-square distribution with $n-1$ degrees of freedom, leaving areas of $1 - \frac{\alpha}{2}$ and $\frac{\alpha}{2}$, respectively, to the right. Substituting for X^2, we write

$$P\left[\chi^2_{1-\frac{\alpha}{2}} < \frac{(n-1)S^2}{\sigma^2} < \chi^2_{\alpha/2}\right] = 1 - \alpha$$

Dividing each term in the inequality by $(n-1)S^2$ and then inverting each term, we obtain

$$P\left[\frac{(n-1)S^2}{\chi^2_{\alpha/2}} < \sigma^2 < \frac{(n-1)S^2}{\chi^2_{1-\frac{\alpha}{2}}}\right] = 1-\alpha$$

Definition 4.55: Confidence Interval for σ^2

If s^2 is the variance of a random sample of size n from a normal population, a $100(1-\alpha)\%$ confidence interval for σ^2 is

$$\frac{(n-1)S^2}{\chi^2_{\alpha/2}} < \sigma^2 < \frac{(n-1)S^2}{\chi^2_{1-\frac{\alpha}{2}}}$$

where $\chi^2_{1-\frac{\alpha}{2}}$ and $\chi^2_{\alpha/2}$ are values of the chi-square distribution with $n-1$ degrees of freedom, leaving areas of $1-\frac{\alpha}{2}$ and $\frac{\alpha}{2}$, respectively, to the right.

[SI*] **Two Samples: Estimating the Ratio of Two Variances**

A point estimate of the ratio of two population variances $\frac{\sigma_1^2}{\sigma_2^2}$ is given by the ratio $\frac{s_1^2}{s_2^2}$ of the sample variances. Hence, the statistic $\frac{S_1^2}{S_2^2}$ is called an estimator of $\frac{\sigma_1^2}{\sigma_2^2}$.

If σ_1^2 and σ_2^2 are the variances of normal populations, we can establish an interval estimate of by using the statistic

$$F = \frac{\sigma_2^2 S_1^2}{\sigma_1^2 S_2^2}$$

The random variable F has an F-distribution with $\nu_1 = n_1 - 1$ and $\nu_2 = n_2 - 1$ degrees of freedom. Therefore, we may write

$$P(f_{1-\frac{\alpha}{2}}(\nu_1,\nu_2) < F < f_{\frac{\alpha}{2}}(\nu_1,\nu_2)) = 1-\alpha$$

where $f_{1-\frac{\alpha}{2}}(\nu_1,\nu_2)$ and $f_{\frac{\alpha}{2}}(\nu_1,\nu_2)$ are the values of the F-distribution with ν_1 and ν_2 degrees of freedom, leaving areas of $1-\frac{\alpha}{2}$ and $\frac{\alpha}{2}$, respectively, to the right.

Substituting for F, we write

$$P\left(f_{1-\frac{\alpha}{2}}(\nu_1,\nu_2) < \frac{\sigma_2^2 S_1^2}{\sigma_1^2 S_2^2} < f_{\frac{\alpha}{2}}(\nu_1,\nu_2)\right) = 1-\alpha$$

Multiplying each term in the inequality by $\frac{S_2^2}{S_1^2}$ and then inverting each term, we obtain

$$P\left(\frac{S_1^2}{S_2^2}\frac{1}{f_{\frac{\alpha}{2}}(\nu_1,\nu_2)} < \frac{\sigma_1^2}{\sigma_2^2} < \frac{S_1^2}{S_2^2}\frac{1}{f_{1-\frac{\alpha}{2}}(\nu_1,\nu_2)}\right) = 1-\alpha$$

the results enable us to replace the quantity $f_{1-\frac{\alpha}{2}}(\nu_1,\nu_2)$ by $\frac{1}{f_{\frac{\alpha}{2}}(\nu_2,\nu_1)}$. Therefore,

$$P\left(\frac{S_1^2}{S_2^2}\frac{1}{f_{\frac{\alpha}{2}}(\nu_1,\nu_2)} < \frac{\sigma_1^2}{\sigma_2^2} < \frac{S_1^2}{S_2^2} f_{\frac{\alpha}{2}}(\nu_2,\nu_1)\right) = 1-\alpha$$

For any two independent random samples of sizes n_1 and n_2 selected from two normal populations, the ratio of the sample variances $\frac{s_1^2}{s_2^2}$ is computed, then the confidence interval can be obtained.

Definition 4.56: The Confidence Interval for $\frac{\sigma_1^2}{\sigma_2^2}$

If s_1^2 and s_2^2 are the variances of independent samples of sizes n_1 and n_2, respectively, from normal populations, then a $100(1-\alpha)\%$ confidence interval for $\frac{\sigma_1^2}{\sigma_2^2}$ is

$$\frac{s_1^2}{s_2^2}\frac{1}{f_{\frac{\alpha}{2}}(\nu_1,\nu_2)} < \frac{\sigma_1^2}{\sigma_2^2} < \frac{s_1^2}{s_2^2} f_{\frac{\alpha}{2}}(\nu_2,\nu_1)$$

where $f_{\frac{\alpha}{2}}(\nu_1,\nu_2)$ is an f-value with $\nu_1 = n_1 - 1$ and $\nu_2 = n_2 - 1$ degrees of freedom, leaving an area of $\frac{\alpha}{2}$ to the right, and $f_{\frac{\alpha}{2}}(\nu_2,\nu_1)$ is a similar f-value with $\nu_2 = n_2 - 1$ and $\nu_1 = n_1 - 1$ degrees of freedom.

v. Functions in SymIntegration Related to Confidence Interval

[SI*] To compute several computations related to confidence interval, we can use these functions in SymIntegration:

confidenceinterval_onesampletwosides(vector<double> data, double σ, double α)
this function computes a two sides $100(1-\alpha)\%$ confidence interval for μ with a given σ (the population standard deviation) that is known. The formula comes from the two sides probability statement that makes use of the Central Limit Theorem:

$$P\left(-z_{\alpha/2} < \frac{\overline{X}-\mu}{\sigma/\sqrt{n}} < z_{\alpha/2}\right) = 1-\alpha$$

confidenceinterval_onesampleonesided(vector<double> data, double σ, double α)
this function computes a one-sided $100(1-\alpha)\%$ confidence interval for μ with a given σ (the population standard deviation) that is known. The formula comes from the one-sided probability statement that makes use of the Central Limit Theorem:

$$P\left(\frac{\overline{X}-\mu}{\sigma/\sqrt{n}} < z_{\alpha/2}\right) = 1-\alpha$$

for the upper bound, and

$$P\left(\frac{\overline{X}-\mu}{\sigma/\sqrt{n}} > -z_{\alpha/2}\right) = 1-\alpha$$

for the lower bound.

confidenceinterval_onesampletwosides(vector<double> data, double α)
this function computes a two sides $100(1-\alpha)\%$ confidence interval for μ with unknown σ (the population standard deviation) so we use T statistics that has a Student's t-distribution to approximate σ with s (sample standard deviation).

The formula comes from the two sides probability statement that makes use of the Central Limit Theorem:

$$P\left(-t_{\alpha/2} < \frac{\overline{X} - \mu}{S/\sqrt{n}} < t_{\alpha/2}\right) = 1 - \alpha$$

confidenceinterval_onesampleonesided(vector<double> data, double α)
this function computes a one-sided $100(1-\alpha)\%$ confidence interval for μ with unknown σ (the population standard deviation) so we use T statistics that has a Student's t-distribution to approximate σ with s (sample standard deviation).

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Confidence Interval for One Sample ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Confidence Interval for One Sample ]# ./main

*********************************************************

Confidence interval one sample two sides
Mean data = 2.6, σ = 0.3, n = 36

The 99% confidence interval is :

2.47121 < μ1 < 2.72879

*********************************************************

*********************************************************

Confidence interval one sample one-sided confidence bounds
Mean data = 6.2, σ = 2, n = 25

The 95% confidence interval is :

Upper one-sided bound: 6.85794
Lower one-sided bound: 5.54206

*********************************************************

*********************************************************

Confidence interval one sample two sides
Mean data = 10, σ = 0.282843, n = 7

The 95% confidence interval is :

9.73841 < μ1 < 10.2616

*********************************************************

*********************************************************

Confidence interval one sample one-sided confidence bounds
Mean data = 10, σ = 0.282843, n = 7

The 95% confidence interval is :

Upper one-sided bound: 10.2077
Lower one-sided bound: 9.79227

*********************************************************

Time taken by function: 1319 microseconds
```

Figure 4.13: *The computation of one sample confidence interval for two sides and one-sided confidence bounds with σ known and σ unknown (**SymIntegration/Examples/Statistics/Test SymIntegration Confidence Interval for One Sample/main.cpp**).*

predictioninterval(vector<double> data, doubleσ, double α)
this function computes a $100(1-\alpha)\%$ prediction interval of a future observation with known σ.

predictioninterval(vector<double> data, double α)
this function computes a $100(1-\alpha)\%$ prediction interval of a future observation with unknown σ.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Prediction Interval for One Sample ]# ./main

*********************************************************

Prediction interval for one sample data, σ known
Mean data = 257300, σ = 25000, n = 50

The 95% prediction interval is :

207813 < X0 < 306787

*********************************************************

*********************************************************

Prediction interval for one sample data, σ unknown
Mean data = 96.24, s = 0.878714, n = 30

The 95% prediction interval is :

94.4131 < X0 < 98.0669

*********************************************************

Time taken by function: 973 microseconds
```

Figure 4.14: *The computation of one sample prediction interval with σ known and σ unknown (**SymIntegration/Examples/Statistics/Test SymIntegration Prediction Interval for One Sample/main.cpp**).*

Sweden Sexy, Walnut PUPULU, Sine Bam Bam, Belang CUPULU, Bludut Hasta La Meo
The functions for two samples will be described with the example in the next subsection.

vi. Compute Confidence Interval for $\mu_1 - \mu_2$, σ_1^2 and σ_2^2 Known with SymIntegration

A study was conducted in which two types of engines, A and B, were compared. Gas mileage, in miles per gallon, was measured. Fifty experiments were conducted using engine type A and 75 experiments were done with engine type B. The gasoline used and other conditions were held constant. The average gas mileage was 36 miles per gallon for engine A and 42 miles per gallon for engine B. Find a 96% confidence interval on $\mu_B - \mu_A$, where μ_A and μ_B are population mean gas mileages for engines A and B, respectively. Assume that the population standard deviations are 6 and 8 for engines A and B, respectively.

Solution:

The point estimate of $\mu_B - \mu_A$ is $\overline{x}_B - \overline{x}_A = 42 - 36 = 6$. Using $\alpha = 0.04$, we find

$$z_{0.02} = 2.05$$

Hence, with substitution in the formula, the 96% confidence interval is

$$6 - 2.05\sqrt{\frac{64}{75} + \frac{36}{50}} < \mu_B - \mu_A < 6 + 2.05\sqrt{\frac{64}{75} + \frac{36}{50}}$$

or simply $3.43 < \mu_B - \mu_A < 8.57$.

In SymIntegration to compute the confidence interval for $\mu_1 - \mu_2$, σ_1^2 and σ_2^2 known we will use this function:
confidenceinterval_knownsigma(vector<double> data1, vector<double> data2, double σ_1, double σ_2, double α)

The input is vector **data1** (corresponding to X_1, a random variable that represents the gas mileage for engine A), vector **data2** (corresponding to X_2, a random variable that represents the gas mileage for engine B), the known population standard deviation σ_1 (corresponding to **data1**) and σ_2 (corresponding to **data2**), and the level of significance α to create the $100(1-\alpha)\%$ confidence interval for $\mu_1 - \mu_2$.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Confidence Interval Variances Known ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb  main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Confidence Interval Variances Known ]# ./main


*********************************************************

Confidence interval with population variances known
Mean data 1 = 36, n1 = 50
Mean data 2 = 42, n2 = 75

The 96% confidence interval is :

3.42393 < μ1 - μ2 < 8.57607

*********************************************************

Time taken by function: 830 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Confidence Interval Variances Known ]#
```

Figure 4.15: *The computation of confidence interval for $\mu_1 - \mu_2$, σ_1^2 and σ_2^2 known with SymIntegration (**SymIntegration/Examples/Statistics/Test SymIntegration Confidence Interval Variances Known/main.cpp**).*

vii. Compute Confidence Interval for $\mu_1 - \mu_2$, $\sigma_1^2 = \sigma_2^2$ but Both Unknown with SymIntegration

The article "Macroinvertebate Community Structure as an Indicator of Acid Mine Pollution," published in the *Journal of Environmental Pollution*, reports on an investigation undertaken in Cane Creek, Alabama, to determine the relationship between selected physiochemical parameters and different measures of macroinvetebrate community structure. One facet of the investigation was an evaluation of the effectiveness of a numerical species diveristy index to indicate aquatic degradation due to acid mine drainage. Conceptually, a high index of macroinvertebrate species diversity should indicate an unstressed aquatic system, while a low diversity index should indicate a stressed aquatic system.

Two independent sampling stations were chosen for this study, one located downstream from the acid mine discharge point and the other located upstream. For 12 monthly samples collected at the downstream station, the species diversity index had a mean value $\overline{x}_1 = 3.11$ and a standard deviation $s_1 = 0.771$, while 10 monthly samples collected at the upstream station had a mean index value $\overline{x}_2 = 2.04$ and a standard deviation $s_2 = 0.448$. Find a 90% confidence interval for the difference between the population means for the two locations, asusming that the populations are approximately normally distributed with equal variances.

Solution:

Let μ_1 and μ_2 represent the population means, respectively, for the species diversity indices at the downstream and upstream stations. We wish to find a 90% confidence interval for $\mu_1 - \mu_2$. Our point estimate of $\mu_1 - \mu_2$ is

$$\overline{x}_1 - \overline{x}_2 = 3.11 - 2.04 = 1.07$$

The pooled estimate, s_p^2, of the common variance, σ^2, is

$$\begin{aligned} s_p^2 &= \frac{(n_1 - 1)S_1^2 + (n_2 - 1)S_2^2}{n_1 + n_2 - 2} \\ &= \frac{(11)(0.771^2) + (9)(0.448^2)}{12 + 10 - 2} \\ &= 0.417 \end{aligned}$$

Taking the square root, we obtain $s_p = 0.646$. Using $\alpha = 0.1$, we obtain

$$t_{0.05} = 1.725$$

for $\nu = n_1 + n_2 - 2 = 20$ degrees of freedom. Therefore, the 90%confidence interval for $\mu_1 - \mu_2$ is

$$1.07 - (1.725)(0.646)\sqrt{\frac{1}{12} + \frac{1}{10}} < \mu_1 - \mu_2 < 1.07 + (1.725)(0.646)\sqrt{\frac{1}{12} + \frac{1}{10}}$$

which simplifies to $0.593 < \mu_1 - \mu_2 < 1.547$.

In SymIntegration to compute the confidence interval for $\mu_1 - \mu_2$, with $\sigma_1^2 = \sigma_2^2$ but both population variances are unknown we will use this function:
confidenceinterval_sameunknownsigma(vector<double> data1, vector<double> data2, double α)

The input is vector **data1** (corresponding to X_1, a random variable that represents index of macroinvertebrate species diversity at the downstream location), vector **data2** (corresponding to X_2, a random variable that represents index of macroinvertebrate species diversity at the upstream location), and the level of significance α to create the $100(1-\alpha)\%$ confidence interval for $\mu_1 - \mu_2$.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Confidence Interval Variances Same but Unknown ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb  main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Confidence Interval Variances Same but Unknown ]# ./main


*********************************************************

Confidence interval with same population variances but both unknown
Mean data 1 = 3.11, n1 = 12
Mean data 2 = 2.04, n2 = 10

The 90% confidence interval is :

0.592974 < μ1 - μ2 < 1.54703

*********************************************************

Time taken by function: 753 microseconds
```

Figure 4.16: *The computation of confidence interval for $\mu_1 - \mu_2$, $\sigma_1^2 = \sigma_2^2$ but both are unknown with SymIntegration (**SymIntegration/Examples/Statistics/Test SymIntegration Confidence Interval Variances Same but Unknown/main.cpp**).*

viii. Compute Confidence Interval for $\mu_1 - \mu_2$, $\sigma_1^2 \neq \sigma_2^2$ and Both Unknown with SymIntegration

A study was conducted by the Department of Zoology at the Virginia Tech to estimate the difference in the amounts of the chemical orthophosphorusmeasured at two different stations on the James River. Orthophosphorus was measured in milligrams per liter. Fifteen samples were collected from station 1, and 12 samples were collected from station 2. The 15 samples from station 1 had an average orthophosphorus content of 3.84 milligrams per liter and a standard deviation of 3.07 milligrams per liter, while the 12 samples from station 2 had an average content of 1.49 milligrams per liter and a standard deviation of 0.80 milligram per liter. Find a 95% confidence interval for the difference in the true average orthophosphorus contents at these two stations, assuming that the observations came from normal populations with different variances.

Solution:

For station 1, we have $\overline{x}_1 = 3.84, s_1 = 3.07$, and $n_1 = 15$. For station 2, $\overline{x}_2 = 1.49, s_2 = 0.80$, and $n_2 = 12$. We wish to find a 95% confidence interval for $\mu_1 - \mu_2$. Since the population variances are assumed to be unequal, we can only find an approximate 95% confidence interval based on the t-distribution with ν degrees of freedom, where

$$\nu = \frac{\left(\frac{3.07^2}{15} + \frac{0.80^2}{12}\right)^2}{\left(\frac{\left(\frac{3.07^2}{15}\right)^2}{14}\right) + \left(\frac{\left(\frac{0.80^2}{12}\right)^2}{11}\right)} = 16.3 \approx 16$$

Our point estimate of $\mu_1 - \mu_2$ is

$$\overline{x}_1 - \overline{x}_2 = 3.84 - 1.49 - 2.35$$

Using $\alpha = 0.05$, we find that

$$t_{0.025} = 2.120$$

for $\nu = 16$ degrees of freedom. Therefore, the 95% confidence interval for $\mu_1 - \mu_2$ is

$$2.35 - 2.120\sqrt{\frac{3.07^2}{15} + \frac{0.80^2}{12}} < \mu_1 - \mu_2 < 2.35 + 2.120\sqrt{\frac{3.07^2}{15} + \frac{0.80^2}{12}}$$

which simplifies to $0.60 < \mu_1 - \mu_2 < 4.10$. Hence, we are confident that the interval from 0.60 to 4.10 milligrams per liter contains the difference of the true average orthophosphorus contents for these two locations.

In SymIntegration to compute the confidence interval for $\mu_1 - \mu_2$, with $\sigma_1^2 \neq \sigma_2^2$ and both population variances are unknown we will use this function:
confidenceinterval_unequalunknownsigma(vector<double> data1, vector<double> data2, double α)

The input is vector **data1** (corresponding to X_1, a random variable that represents the amount of the chemical orthophosphorus measured at station 1), vector **data2** (corresponding to X_2, a random variable that representsthe amount of the chemical orthophosphorus measured at station 2), and the level of significance α to create the $100(1-\alpha)\%$ confidence interval for $\mu_1 - \mu_2$.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Confidence Interval Variances Unequal and Unknown ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb  main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Confidence Interval Variances Unequal and Unknown ]# ./main


*********************************************************

Confidence interval with unequal and unknown population variances
Mean data 1 = 3.84, s1 = 3.07, n1 = 15
Mean data 2 = 1.49, s2 = 0.8, n2 = 12

The 95% confidence interval is :

0.599749 < μ1 - μ2 < 4.10025

*********************************************************

Time taken by function: 781 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Confidence Interval Variances Unequal and Unknown ]#
```

Figure 4.17: *The computation of confidence interval for $\mu_1 - \mu_2$, $\sigma_1^2 \neq \sigma_2^2$ and both are unknown with SymIntegration (**SymIntegration/Examples/Statistics/Test SymIntegration Confidence Interval Variances Unequal and Unknown/main.cpp**).*

ix. Compute Confidence Interval for $\mu_D = \mu_1 - \mu_2$, Paired Observations with SymIntegration

Fortune magazine (March 1997) reported the total returns to investors for the 10 years prior to 1996 and also for 1996 for 431 companies. The total returns for 10 of the companies are listed below. Find a 95% confidence interval for the mean change in percent return to investors.

Company	Total Returns 1986 - 1996	Total Returns 1996
Coca-Cola	29.8%	43.3%
Mirage Resorts	27.9%	25.4%
Merck	22.1%	24.0%
Microsoft	44.5%	88.3%
Johnson & Johnson	22.2%	18.1%
Intel	43.8%	131.2%
Pfizer	21.7%	34.0%
Procter & Gamble	21.9%	32.1%
Berkshire Hathaway	28.3%	6.2%
S&P 500	11.8%	20.3%

Table 4.2: *Total return to investors for* 10 *yeas prior to 1996 and also for 1996 for 10 companies.*

Solution:

We wish to find 95% confidence interval for $\mu_1 - \mu_2$. Since the observations are paired, $\mu_1 - \mu_2 = \mu_D$. The point estimate of μ_D is $\overline{d} = 14.89$. The standard deviation, s_d, of the sample differences is

$$s_d = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(d_i - \overline{d})^2} = 30.4868$$

Using $\alpha = 0.05$, we find that $t_{0.025} = 2.262$ for $\nu = n - 1 = 9$ degrees of freedom. Therefore the 95% confidence interval is

$$14.89 - (2.262)\frac{30.4868}{\sqrt{10}} < \mu_D < 14.89 + (2.262)\frac{30.4868}{\sqrt{10}}$$

which yields $-6.92 < \mu_D < 36.70$.

In SymIntegration to compute the confidence interval for $\mu_D = \mu_1 - \mu_2$ paired observations we can use this function:
confidenceinterval_paired(vector<double> data1, vector<double> data2, double α)

The input is vector **data1** (corresponding to X_1, a random variable that represents the "after" state), vector **data2** (corresponding to X_2, a random variable that represents the "before" state), and the level of significance α to create the $100(1-\alpha)\%$ confidence interval for $\mu_D = \mu_1 - \mu_2$.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Confidence Interval Paired Observations ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb  main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Confidence Interval Paired Observations ]# ./main


*********************************************************

Confidence interval for paired observations
Mean d = 14.89, sd = 30.4868, n = 10

The 95% confidence interval is :

-6.91892 < µD < 36.6989

*********************************************************

Time taken by function: 742 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Confidence Interval Paired Observations ]#
```

Figure 4.18: *The computation of confidence interval for $\mu_D = \mu_1 - \mu_2$ paired observations with SymIntegration (**SymIntegration/Examples/Statistics/Test SymIntegration Confidence Interval Paired Observations/main.cpp**).*

x. Compute Confidence Interval for Proportion p with SymIntegration

In a random sample of $n = 500$ families owning television sets in the city of Ontario, Canada, it is found that $x = 340$ subscribe to HBO, and the rest to BAM, a channel for dogs and by dogs all around the world. Find a 95% confidence interval for the actual proportion of families with television sets in this city that subscribe to HBO.

Solution:

The point estimate of p is

$$\hat{p} = \frac{340}{500} = 0.68$$

We find that

$$z_{0.025} = 1.96$$

Therefore, using method 1 from Definition (4.53), the 95% confidence interval for p is

$$0.68 - 1.96\sqrt{\frac{(0.68)(0.32)}{500}} < p < 0.68 + 1.96\sqrt{\frac{(0.68)(0.32)}{500}}$$

which simplifies to $0.6391 < p < 0.7209$.

In SymIntegration to compute the confidence interval for proportion p we can use this function: **confidenceinterval_proportion(vector<string> data, const string &inputString, double α)**

The input is a string vector **data**, **inputString** as the string that we want to compute the confidence interval for the proportion p, and the level of significance α to create the $100(1-\alpha)\%$ confidence interval for p.

In SymIntegration we are using method 2 from Definition (4.53) to compute the confidence interval for p, the exact results are only until 2 digit decimals for this example between method 1 and method 2. Note that although method 2 yields more accurate results, it is more complicated to calculate (not a problem if we are using computer), and the gain in accuracy that it provides diminishes when the sample size is large enough. Hence, method 1 is commonly used in practice, or college examination.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Confidence Interval for Proportion ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Confidence Interval for Proportion ]# ./main

*******************************************************

Confidence interval for proportion
Data size =  500
Amount of HBO in the data = 340
Proportion for HBO, p = 0.68, q = 0.32

The 95% confidence interval is :

0.637873 < p < 0.719382

*******************************************************

Time taken by function: 1128 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Confidence Interval for Proportion ]#
```

Figure 4.19: *The computation of confidence interval for proportion p (the proportion of families that subscribe to HBO in Ontario, Canada) with SymIntegration (**SymIntegration/Examples/Statistics/Test SymIntegration Confidence Interval for Proportion/main.cpp**).*

xi. Compute Confidence Interval for the Difference Between Two Proportions $p_1 - p_2$ with SymIntegration

A certain change in a process for manufacturing component parts is being considered. Samples are taken under both the existing and the new process so as to determine if the new process results in an improvement. If 75 of 1500 items from the existing process are found to be defective and 80 of 2000 items from the new process are found to be defective, find a 90% confidence interval for the true difference in the proportion of defectives between the existing and the new process.

Solution:

Let p_1 and p_2 be the true proportions of defectives for the existing and new processes, respectively. Hence

$$\hat{p}_1 = \frac{75}{1500} = 0.05$$
$$\hat{p}_2 = \frac{80}{2000} = 0.04$$

and

$$z_{0.05} = 1.645$$

Therefore, substituting into the formula we will obtain

$$(0.05-0.04) - 1.645\sqrt{\frac{(0.05)(0.95)}{1500} + \frac{(0.04)(0.96)}{2000}} < p_1 - p_2 < (0.05-0.04) + 1.645\sqrt{\frac{(0.05)(0.95)}{1500} + \frac{(0.04)(0.96)}{2000}}$$

we find the 90% confidence interval to be $-0.0017 < p_1 - p_2 < 0.0217$. Since the interval contains the value 0, there is no reason to believe that the new process produces a significant decrease in the proportion of defectives over the existing method.

In SymIntegration to compute the confidence interval for the difference between two proportions $p_1 - p_2$ we can use this function:
confidenceinterval_differencebetweentwoproportions(vector<string> data1, vector<string> data2, const string &inputString, double α)

The input is a string vector **data1**, string vector **data2**, **inputString** as the string (e.g. Defective, Summa Cum laude, Lung Cancer, etc) that we want to compute the confidence interval for the proportion p, and the level of significance α to create the $100(1-\alpha)\%$ confidence interval for $p_1 - p_2$.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Confidence Interval for Difference Between Proportions ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Confidence Interval for Difference Between Proportions ]# ./main

*********************************************************

Confidence interval for difference between two proportions
Data 1 size =  1500, Data 2 size =  2000
Amount of DEFECTIVE in the data 1 = 75
Amount of DEFECTIVE in the data 2 = 80
Proportion for DEFECTIVE, p1 = 0.05, q1 = 0.95
Proportion for DEFECTIVE, p2 = 0.04, q2 = 0.96

The 90% confidence interval is :

-0.00173124 < p1 - p2 < 0.0217312

*********************************************************

Time taken by function: 4008 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Confidence Interval for Difference Between Proportions ]#
```

Figure 4.20: *The computation of confidence interval for the difference between two proportions of the existing process and the new process in manufacturing component parts (**SymIntegration/Examples/Statistics/Test SymIntegration Confidence Interval for Difference Between Proportions/main.cpp**).*

xii. Compute Confidence Interval for Variance σ^2 with SymIntegration

The following are the weights, in decagrams, of 10 packages of grass seed distributed by a certain company: 46.4, 46.1, 45.8, 47.0, 46.1, 45.9, 45.8, 46.9, 45.2, and 46.0. Find a 95% confidence interval for the variance of the weights of all such packages of grass seed distributed by this company, asusming a normal population.

Solution:

First we find

$$\begin{aligned} s^2 &= \frac{n\sum_{i=1}^{n} x_i^2 - \left(\sum_{i=1}^{n} x_i\right)^2}{n(n-1)} \\ &= \frac{(10)(21,273.12) - (461.2)^2}{(10)(9)} \\ &= 0.286 \end{aligned}$$

To obtain a 95% confidence interval, we choose $\alpha = 0.05$. Then, using inverse cdf formula for chi-squared with $\nu = 9$ degrees of freedom, we find

$$\begin{aligned} \chi^2_{0.025} &= 19.023 \\ \chi^2_{0.975} &= 2.700 \end{aligned}$$

The notation of $\chi^2_{0.025}$ means that after the $\chi^2_{0.025}$ value there is 0.025 area to the right, and an area of 0.975 to the left.

Therefore, the 95% confidence interval for σ^2 is

$$\frac{(9)(0.286)}{19.023} < \sigma^2 < \frac{(9)(0.286)}{2.700}$$

or simply $0.135 < \sigma^2 < 0.953$.

In SymIntegration to compute the confidence interval for variance σ^2 we can use this function: **confidenceinterval_variance(vector<doubles> data, double α)**

The input is a double vector **data**, and the level of significance α to create the $100(1-\alpha)\%$ confidence interval for σ^2.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Confidence Interval for Variance ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Confidence Interval for Variance ]# ./main

*********************************************************

Confidence interval for variance
Data size =  10
Mean sample =  46.12, variance sample = 0.286222

The 95% confidence interval is :

0.135411 < σ^{2} < 0.953856

*********************************************************

Time taken by function: 10225 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Confidence Interval for Variance ]#
```

Figure 4.21: *The computation of confidence interval for variance σ^2 of the weights of grass seed distributed by certain company with SymIntegration (**SymIntegration/Examples/Statistics/Test SymIntegration Confidence Interval for Variance/main.cpp**).*

xiii. Compute Confidence Interval for Ratio of Two Variances $\frac{\sigma_1^2}{\sigma_2^2}$ with SymIntegration

A confidence interval for the difference in the mean orthophosphorus contents, measured in milligrams per liter, at two stations on the James River was constructed in the previous computation example by assuming the normal population variance to be unequal. Justify this assumption by constructing 98% for $\frac{\sigma_1^2}{\sigma_2^2}$ and for $\frac{\sigma_1}{\sigma_2}$, where σ_1^2 and σ_2^2 are the variances of the populations of orthophosphorus contents at station 1 and station 2, respectively.

Solution:

We will use new dummy data as **station1.txt** and **station2.txt** with $n_1 = 15$, $n_2 = 12$, $s_1 = 3.07479$ and $s_2 = 0.805983$.

For a 98% confidence interval, $\alpha = 0.02$, and we will obtain

$$f_{0.01}(14, 11) \approx 4.30$$
$$f_{0.01}(11, 14) \approx 3.87$$

Therefore, the 98% confidence interval for $\frac{\sigma_1^2}{\sigma_2^2}$ is

$$\left(\frac{3.07479^2}{0.805983^2}\right)\left(\frac{1}{4.30}\right) < \frac{\sigma_1^2}{\sigma_2^2} < \left(\frac{3.07479^2}{0.805983^2}\right)(3.87)$$

which simplifies to $3.38995 < \frac{\sigma_1^2}{\sigma_2^2} < 56.991$. Taking square roots of the confidence limits, we find that a 98% confidence interval for $\frac{\sigma_1}{\sigma_2}$ is

$$1.84118 < \frac{\sigma_1}{\sigma_2} < 7.49912$$

Since this interval does not allow for the possibility of $\frac{\sigma_1}{\sigma_2}$ being equal to 1, we were correct in assuming that $\sigma_1 \neq \sigma_2$ or $\sigma_1^2 \neq \sigma_2^2$.

In SymIntegration to compute the confidence interval for the ratio of two variances $\frac{\sigma_1^2}{\sigma_2^2}$ we can use this function:
confidenceinterval_ratiotwovariances(vector<doubles> data1, vector<doubles> data2, double α)

The input is the first data as a double vector **data1**, the second data as a double vector **data2**, and the level of significance α to create the $100(1-\alpha)\%$ confidence interval for σ^2.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Confidence Interval for Ratio of Two Variances ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Confidence Interval for Ratio of Two Variances ]# ./main

*********************************************************

Confidence interval for the ratio of two variances
Mean data 1 = 4.68074, s1 = 3.07479, n1 = 15
Mean data 2 = 1.14591, s2 = 0.805983, n2 = 12

The 98% confidence interval is :

3.38995 <  σ1^{2} /  σ2^{2} < 56.2368
1.84118 <  σ1 /  σ2 < 7.49912

*********************************************************

Time taken by function: 938 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Confidence Interval for Ratio of Two Variances ]#
```

Figure 4.22: *The computation of confidence interval for ratio of two variances* $\frac{\sigma_1^2}{\sigma_2^2}$ *and the ration of two standard deviations* $\frac{\sigma_1}{\sigma_2}$ *of the populations of orthophosphorus contents in two different stations with SymIntegration (**SymIntegration/Examples/Statistics/Test SymIntegration Confidence Interval for Ratio of Two Variances/main.cpp**).*

xiv. Maximum Likelihood Estimation

[SI*] Maximum likelihood estimation (MLE) is one of the most important approaches to estimation in all of statistical inference. It is a statistical method for estimating the parameters of a probability distribution by finding the parameter values that make the observed data most probable.

To do that, it maximizes the likelihood function, which represents the probability of the observed data given the parameters. The maximum likelihood estimate is the parameter value that results in this highest probability.

[SI*] Some key concepts in MLE:

1. **Likelihood function**
   This function is essentially the joint probability density function (pdf) of the observed data, viewed as a function of the distribution's parameters.
2. **Maximization**
   The goal is to find the parameter value(s) that maximize the likelihood function. This is often done by taking the natural logarithm of the likelihood function, which simplifies calculations because products become sums.
3. **Log-likelihood**
   The logarithm is taken because it is often easier to differentiate. Maximizing the log-likelihood is equivalent to maximizing the likelihood itself.
4. **Estimator**
   The process involves taking the derivative of the log-likelihood function, setting it to zero, and solving for the parameter(s) to find the maximum.

[SI*] The applications of MLE:

1. **Statistics and machine learning**
   MLE is a powerful and widely used technique in both statistics and machine learning for optimizing models.
2. **Regression**
   For example, the least-squares method used in linear regression produces the same result as MLE under certain assumptions.
3. **Model fitting**
   It helps find the best-fitting distribution to a set of data by finding the parameters that make that data most likely to have occurred.

Definition 4.57: Maximum Likelihood Estimator

Given independent observations $x_1, x_2, \ldots, x_n$ from a probability density function (continuous case) or probability mass function (discrete case) $f(\boldsymbol{x}; \theta)$, the maximum likelihoo estimator $\hat{\theta}$ is that which maximizes the likelihood function

$$L(x_1, x_2, \ldots, x_n; \theta) = f(\boldsymbol{x}; \theta) = f(x_1, \theta) f(x_2, \theta) \ldots f(x_n, \theta)$$

[SI*] **Example:**
Consider a Poisson distribution with probability mass function

$$f(x|\mu) = \frac{e^{-\mu}\mu^x}{x!}, \quad x = 0, 1, 2, \ldots$$

Suppose that a random sample $x_1, x_2, \ldots, x_n$ is taken from the distribution. What is the maximum likelihood estimate of μ?

Solution:

The likelihood function is

$$L(x_1, x_2, \ldots, x_n; \mu) = \prod_{i=1}^{n} f(x_i|\mu) = \frac{e^{-n\mu}\mu^{\sum_{i=1}^{n} x_i}}{\prod_{i=1}^{n} x_i!}$$

Now consider

$$\ln L(x_1, x_2, \ldots, x_n; \mu) = -n\mu + \sum_{i=1}^{n} x_i \ln \mu - \ln \prod_{i=1}^{n} x_i!$$

$$\frac{\partial \ln L(x_1, x_2, \ldots, x_n; \mu)}{\partial \mu} = -n + \sum_{i=1}^{n} \frac{x_i}{\mu}$$

Solving for $\hat{\mu}$, the maximum likelihood estimator, involves setting the derivative to zero and solving for the parameter. Thus,

$$\hat{\mu} = \sum_{i=1}^{n} \frac{x_i}{n} = \overline{x}$$

the second derivative of the log-likelihood is negative, which implies that the solution above indeed is a maximum. Since μ is the mean of the Poisson distribution, the sample average would certainly seem like a reasonable estimator.

[SI*] **Example:**
Consider a random sample $x_1, x_2, \ldots, x_n$ from a normal distribution $N(\mu, \sigma)$. Find the maximum likelihood estimators for μ and σ^2.

Solution:
The likelihood function for the normal distribution is

$$L(x_1, x_2, \ldots, x_n; \mu, \sigma^2) = \frac{1}{(2\pi)^{\frac{n}{2}}(\sigma^2)^{\frac{n}{2}}} e^{\left[-\frac{1}{2}\sum_{i=1}^{n}\left(\frac{x_i-\mu}{\sigma}\right)^2\right]}$$

taking logarithm gives us

$$\ln L(x_1, x_2, \ldots, x_n; \mu, \sigma^2) = -\frac{n}{2}\ln(2\pi) - \frac{n}{2}\ln\sigma^2 - \frac{1}{2}\sum_{i=1}^{n}\left(\frac{x_i-\mu}{\sigma}\right)^2$$

Hence,

$$\frac{\partial \ln L}{\partial \mu} = \sum_{i=1}^{n}\left(\frac{x_i-\mu}{\sigma^2}\right)$$

$$\frac{\partial \ln L}{\partial \sigma^2} = -\frac{n}{2\sigma^2} + \frac{1}{2(\sigma^2)^2}\sum_{i=1}^{n}(x_i-\mu)^2$$

Setting both derivatives equal to 0, we obtain

$$\sum_{i=1}^{n} x_i - n\mu = 0$$

and

$$n\sigma^2 = \sum_{i=1}^{n}(x_i - \mu)^2$$

Thus, the maximum likelihood estimator of μ is given by

$$\hat{\mu} = \frac{1}{n}\sum_{i=1}^{n} x_i = \overline{x}$$

On the other hand, the maximum likelihood estimator of σ^2 is

$$\hat{\sigma^2} = \frac{1}{n}\sum_{i=1}^{n}(x_i - \overline{x})^2$$

Checking the second-order partial derivative matrix confirms that the solution results in a maximum of the likelihood function. It is interesting to note the distinction between the maximum likelihood estimator of σ^2 and the unbiased estimator S^2. The numerators are identical, of course, and the denominator is the degrees of freedom $n-1$ for the unbiased estimator and n for the maximum likelihood estimator. Maximum likelihood estimators do not necessarily enjoy the property of unbiasedness. However, they do have very important asymptotic properties.

[SI*] The method of maximum likelihood cannot be applied without knowledge of the underlying distribution.

[SI*] **MLE for Binomial Distribution**
Suppose that there are n trials $x_1, x_2, \ldots, x_n$ from a Bernoulli process with parameter p, the probability of a success. That is, the probability of r successes is given by

$$\binom{n}{r} p^r (1-p)^{n-r}$$

work out the maximum likelihood estimator for the parameter p.

Solution:

The likelihood function is

$$\begin{aligned} L(x_1, x_2, \ldots, x_n; p) &= \prod_{i=1}^{n} f(x_i; p) \\ &= \prod_{i=1}^{n} p^{x_i}(1-p)^{1-x_i} \\ &= p^{n\overline{x}}(1-p)^{n(1-\overline{x})} \end{aligned}$$

Hence,

$$\ln L = n\left[\overline{x}\ln(p) + (1-\overline{x})\ln(1-p)\right]$$

taking derivative with respect to p and setting the derivative to zero, we obtain

$$\frac{\partial \ln(L)}{\partial p} = n\left(\frac{\overline{x}}{p} - \frac{1-\overline{x}}{1-p}\right) = 0$$

which yields

$$\frac{\overline{x}}{p} - \frac{1-\overline{x}}{1-p} = 0$$

Therefore

$$\hat{p} = \overline{x}$$

[SI*] **MLE for Negative Binomial Distribution**
Consider the observation X from the negative binomial distribution

$$b*(x;k,p) = \binom{x-1}{k-1} p^k (1-p)^{x-k}, \quad x = k, k+1, k+2, \ldots$$

Find the maximum likelihood estimator for p, assuming k is known.

Solution:

From the density function $b*(x;p) = \binom{x-1}{k-1} p^k (1-p)^{x-k}$, we obtain

$$\ln L = \ln \binom{x-1}{k-1} + k \ln p + (n-k) \ln(1-p)$$

Setting

$$\frac{\partial \ln L}{\partial p} = \frac{k}{p} = \frac{n-k}{1-p} = 0$$

we obtain

$$\hat{p} = \frac{k}{n}$$

[SI*] **MLE for Uniform Distribution**
Consider a random sample of $x_1, \ldots, x_n$ from a uniform distribution $U(0, \theta)$ with unknown parameter θ, where $\theta > 0$. Determine the maximum likelihood estimator of θ.

Solution:

The likelihood function of θ is

$$L(x_1, \ldots, x_n; \theta) = \frac{1}{\theta^n}, \text{ for } \max\{x_1, \ldots, x_n\} < \theta < \infty$$

Since $\frac{1}{\theta^n}$ is a decreasing function of θ, the maximum only achieves at the lower bound, which is $\max\{x_1, \ldots, x_n\}$. This is called the largest order statistic of the sample.

[SI*] **MLE for Lognormal Distribution**
Consider the lognormal distribution with the density function

$$f(x;\mu,\sigma) = \begin{cases} \frac{1}{\sqrt{2\pi}\sigma x} e^{-\frac{1}{2\sigma^2}[\ln(x)-\mu]^2}, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

Suppose we have a random sample $x_1, x_2, \ldots, x_n$ from a lognormal distribution. Write out the likelihood function then develop the maximum likelihood estimators of μ and σ^2.

Solution:

The likelihood function is

$$\begin{aligned} L(x_1, x_2, \ldots, x_n; \mu, \sigma) &= \prod_{i=1}^{n} f(x_i; \mu, \sigma) \\ &= \prod_{i=1}^{n} \left[\frac{1}{\sqrt{2\pi}\sigma x_i} e^{-\frac{[\ln(x_i)-\mu]^2}{2\sigma^2}} \right] \\ &= \frac{1}{(2\pi)^{\frac{n}{2}} \sigma^n \prod_{i=1}^{n} x_i} e^{-\frac{1}{2\sigma^2}\sum_{i=1}^{n}[\ln(x_i)-\mu]^2} \end{aligned}$$

It is easy to obtain

$$\ln(L) = -\frac{n}{2}\ln(2\pi) - \frac{n}{2}\ln(\sigma^2) - \sum_{i=1}^{n}\ln(x_i) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}[\ln(x_i)-\mu]^2$$

So, setting

$$\frac{\partial \ln L}{\partial \mu} = \frac{1}{\sigma^2}\sum_{i=1}^{n}[\ln(x_i)-\mu] = 0$$

we obtain

$$\hat{\mu} = \frac{1}{n}\sum_{i=1}^{n}\ln(x_i)$$

and setting

$$\frac{\partial \ln L}{\partial \sigma^2} = -\frac{n}{2\sigma^2} + \frac{1}{2\sigma^4}\sum_{i=1}^{n}[\ln(x_i)-\mu]^2 = 0$$

we get

$$\hat{\sigma}^2 = \frac{1}{n}\sum_{i=1}^{n}[\ln(x_i)-\hat{\mu}]^2$$

[SI*] **MLE for Gamma Distribution**

Consider a random sample of $x_1, x_2, \ldots, x_n$ coming from the gamma distribution with density function

$$f(x; \alpha, \beta) = \begin{cases} \frac{1}{\beta^\alpha \Gamma(\alpha)} x^{\alpha-1} e^{-\frac{x}{\beta}}, & x > 0 \\ 0, \quad \text{elsewhere} \end{cases}$$

for $\alpha, \beta > 0$.

Determine the maximum likelihood estimation for parameters α and β.

Solution:

The likelihood function is

$$\begin{aligned} L(x_1, \ldots, x_n; \alpha, \beta) &= \frac{1}{\beta^{\alpha n}\Gamma(\alpha)^n} \prod_{i=1}^{n} \left(x_i^{\alpha-1} e^{-\frac{x_i}{\beta}} \right) \\ &= \frac{1}{\beta^{\alpha n}\Gamma(\alpha)^n} \left(\prod_{i=1}^{n} x_i \right)^{\alpha-1} e^{-\sum_{i=1}^{n}\left(\frac{x_i}{\beta}\right)} \end{aligned}$$

The log-likelihood can be expressed as

$$\begin{aligned} \ln L &= -n\alpha\ln(\beta) - n\ln(\Gamma(\alpha)) + (\alpha-1)\sum_{i=1}^{n}\ln(x_i) - \frac{1}{\beta}\sum_{i=1}^{n}x_i \\ &= C(\underline{x}) - \alpha n \ln\beta - \frac{n\overline{x}}{\beta} \end{aligned}$$

where $C(\underline{x})$ is a function of $\underline{x}$. Taking logarithm of $\ln L$ and setting it to be zero, we obtain the maximum likelihood estimation for β to be

$$\hat{\beta} = \frac{\overline{x}}{\alpha}$$

[SI*] **MLE for Weibull Distribution**
Consider a random sample of $x_1, x_2, \ldots, x_n$ observations from a Weibull distribution with parameters α and β and density function

$$f(x) = \begin{cases} \alpha\beta x^{\beta-1} e^{-\alpha x^{\beta}}, & x > 0 \\ 0, & \text{elsewhere} \end{cases}$$

for $\alpha, \beta > 0$.
Determine the maximum likelihood estimation for parameters α and β.

Solution:
The likelihood function is

$$\begin{aligned} L(x_1, \ldots, x_n; \alpha, \beta) &= \prod_{i=1}^{n} f(x_i; \alpha, \beta) = (\alpha\beta)^n \prod_{i=1}^{n} x_i^{\beta-1} e^{-\alpha x_i^{\beta}} \\ &= (\alpha\beta)^n e^{-\alpha \sum_{i=1}^{n} x_i^{\beta}} \left(\prod_{i=1}^{n} x_i \right)^{\beta-1} \end{aligned}$$

So, the log-likelihood can be expressed as

$$\ln L = n[\ln(\alpha) + \ln(\beta)] - \alpha \sum_{i=1}^{n} x_i^{\beta} + (\beta - 1) \sum_{i=1}^{n} \ln(x_i)$$

To solve for the maximum likelihood estimate, we need to solve the following two equations

$$\begin{aligned} \frac{\partial \ln L}{\partial \alpha} &= 0 \\ \frac{\partial \ln L}{\partial \beta} &= 0 \end{aligned}$$

xv. Compute Maximum Likelihood Estimation for Continuous Distribution

It is known that a sample consisting of the values 12, 11.2, 13.5, 12.3, 13.8, and 11.9 comes from a population with the density function

$$f(x;\theta) = \begin{cases} \frac{\theta}{x^{\theta+1}}, & x > 1 \\ 0, & \text{elsewhere} \end{cases}$$

where $\theta > 0$. Find the maximum likelihood estimate of θ.

Solution:

The likelihood function of n observations from this population can be written as

$$L(x_1, x_2, \ldots, x_6; \theta) = \prod_{i=1}^{n} \frac{\theta}{x_i^{\theta+1}} = \frac{\theta^n}{\left(\prod_{i=1}^{n} x_i\right)^{\theta+1}}$$

which implies that

$$\ln L(x_1, x_2, \ldots, x_6; \theta) = n \ln(\theta) - (\theta + 1) \sum_{i=1}^{n} \ln(x_i)$$

Setting $0 = \frac{\partial \ln L}{\partial \theta} = \frac{n}{\theta} - \sum_{i=1}^{n} \ln(x_i)$ results in

$$\begin{aligned} \hat{\theta} &= \frac{n}{\sum_{i=1}^{n} \ln(x_i)} \\ &= \frac{6}{\ln(12) + \ln(11.2) + \ln(13.5) + \ln(12.3) + \ln(13.8) + \ln(11.9)} \\ &= 0.3970 \end{aligned}$$

Since the second derivative of L is $-\frac{n}{\theta^2}$ which is always negative, the likelihood function does achieve its maximum value at $\hat{\theta}$.

In SymIntegration, we can compute the mle for any distribution given that we provide the pdf or the pmf, it is assumed that the parameters for the distribution is at least one and at maximum two. Normal distribution has two parameters: μ and σ^2, Gamma distribution has two parameters : α and β, exponential distribution has one parameter $\lambda = \frac{1}{\beta}$.

The function that is used in SymIntegration to compute the maximum likelihood estimation is: **mle(const Symbolic &f($x; \mu, \sigma^2$), const Symbolic &x, const Symbolic &μ, const Symbolic &σ^2, int n)**

The first input is the Symbolic function of the pdf or pmf, we are using a normal distribution as the example above, then the random variable x, and the first parameter μ, then the second parameter σ^2, and last the number of samples.

For this case, we can put any symbolic for the second parameter (e.g. s, γ, ρ, etc) because the pdf does not have the second parameter, thus it won't really matter with the end result what symbolic second parameter that we put into the function **mle**.

If we want a numeric answer then we will make use of **Equations rules** and then implement the **.subst_all(rules)** into the **mle** result.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Maximum Likelihood Estimation ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Maximum Likelihood Estimation ]# ./main
f(x;θ) = θ*x^(-θ-1)
estimated θ = 6*(ln(x[1])+ln(x[2])+ln(x[3])+ln(x[4])+ln(x[5])+ln(x[6]))^(-1)
estimated θ = 0.396975

Time taken by function: 42617 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Maximum Likelihood Estimation ]#
```

Figure 4.23: *The computation of maximum likelihood estimation for parameter θ with the density function $f(x;\theta) = \frac{\theta}{x^{\theta+1}}, x > 1$ (**SymIntegration/Examples/Statistics/Test SymIntegration Maximum Likelihood Estimation/main.cpp**).*

The source code that handle this maximum likelihood estimation is located in **src/statistics-mle.cpp** and **include/statistics-mle.h**

xvi. Bayesian Statistics

[SI*]

VI. Hypothesis Testing

i. One Sample Hypothesis Test for Comparing Means

[SI*] Hypothesis testing is a statistical process used to make a decision about a population based on sample data. It involves creating a null hypothesis (a statement of no effect or no difference) and an alternative hypothesis, then using sample evidence and probability to test the null hypothesis. The goal is to determine if there is enough evidence to reject the null hypothesis in favor of the alternative.

[SI*] **How Hypothesis Testing Works**

1. **State the hypotheses:** Formulate a null hypothesis (H_0) and an alternative hypothesis (H_1).
   - Null Hypothesis (H_0): A specific statement of no significant difference between a hypothesized value and the value estimated from a sample (e.g., a new drug has no effect).s
   - Alternative Hypothesis (H_1): A statement that contradicts the null hypothesis (e.g., the new drug does have an effect).
2. **Formulate an analysis plan:** Choose a statistical test and define a significance level (α). The choice of test depends on the nature of the data and the question being asked.

   The test statistic measures how much the sample data deviates from what we did expect if the null hypothesis were true. Different tests use different statistics:
   (a) Z-test: Used when population variance is known and sample size is large.
   (b) Student's t-test: Used when the sample size is small or population variance unknown.
   (c) Chi-Squared test: Used for categorical data to compare observed vs expected counts.
3. **Analyze the sample data:** Collect a sample data and perform the chosen statistical test. This involves calculating a test statistic and a P-value.
4. **Interpret the results:**
   (a) **Using P-value:**
   Compare the P-value to the significance level (α) to decide whether to reject or fail to reject the null hypothesis.
   - If the P-value is less than the significance level (α), you reject the null hypothesis.
   - If the P-value is greater than or equal to the significance level, you fail to reject the null hypothesis.

   The P-value approach has been adopted extensively by users of applied statistics. The approach is designed to give the user an alternative to a mere "reject" or "do not reject" conclusion. The P-value computation also gives the user important information when the z value falls well into the ordinary critical region.
   (b) **Using Critical Value:**
   We compute the test statistic, for example if we know that the sample size is more than 30 or we know the population variance then we use z statistic (otherwise, we use t statistic), thus we use this formula:

$$z = \frac{\overline{x} - \mu_0}{\sigma/\sqrt{n}}$$

For the case of hypothesis testing with

$$H_0 : \mu = \mu_0$$
$$H_1 : \mu < \mu_0$$

If the test statistic $z < a$ then reject H_0, if $z \geq a$ then we do not reject H_0. With a as the critical value

$$a = \mu_0 - z_\alpha \frac{\sigma}{\sqrt{n}}$$

For the case of hypothesis testing with

$$H_0 : \mu = \mu_0$$
$$H_1 : \mu > \mu_0$$

If the test statistic $z > b$ then reject H_0, if $z \leq b$ then we do not reject H_0. With b as the critical value

$$b = \mu_0 + z_\alpha \frac{\sigma}{\sqrt{n}}$$

For the case of hypothesis testing with

$$H_0 : \mu = \mu_0$$
$$H_1 : \mu \neq \mu_0$$

If the test statistic $z < a$ or $z > b$ then reject H_0, if $z \geq a$ then we do not reject H_0. With a and b as the critical value

$$a = \mu_0 - z_{\alpha/2} \frac{\sigma}{\sqrt{n}}$$
$$b = \mu_0 + z_{\alpha/2} \frac{\sigma}{\sqrt{n}}$$

[SI*] Examples that are using hypothesis testing:

- Testing a new drug by comparing a group that receives the drug to a control group to see if there is a statistically significant difference in outcomes.
- Assessing if a change in a production process has resulted in a statistical difference in product yield.

[SI*] Types of hypothesis testing:

1. Two-tailed

$$H_0 : \mu = c$$
$$H_1 : \mu \neq c$$

If the hypothesis test is two-tailed, with variance known and large sample size then the P-value can be computed with:

$$P = P(|Z| > z_{\frac{\alpha}{2}}) = 2P(Z < -z_{\frac{\alpha}{2}})$$

with

$$z_{\frac{\alpha}{2}} = \frac{\bar{x} - \mu_0}{\sigma/\sqrt{n}}$$

2. One-tailed (Left-tailed)

$$H_0 : \mu \geq c$$
$$H_1 : \mu < c$$

If the hypothesis test is one-tailed (left-tailed), with variance known and large sample size then the P-value can be computed with:

$$P = P(Z < z_{\frac{\alpha}{2}})$$

with

$$z_{\frac{\alpha}{2}} = \frac{\bar{x} - \mu_0}{\sigma/\sqrt{n}}$$

3. One-tailed (Right-tailed)

$$H_0 : \mu \leq c$$
$$H_1 : \mu > c$$

If the hypothesis test is one-tailed (right-tailed), with variance known and large sample size then the P-value can be computed with:

$$P = P(Z > z_{\frac{\alpha}{2}})$$

with

$$z_{\frac{\alpha}{2}} = \frac{\bar{x} - \mu_0}{\sigma/\sqrt{n}}$$

[SI*] We will explore hypothesis testing through a few specific practical hypothesis tests. Recall the general hypothesis test formulation where we partition the parameter space Θ as follows:

$$H_0 : \theta \in \Theta_0, \quad H_1 : \theta \in \Theta_1$$

One of the most common cases for a single population is to consider θ as μ, the population mean, in which case $\Theta = \mathbb{R}$. Often, we wish to test if the population mean is equal to some value, μ_0. This allows us to construct a two-sided hypothesis test as follows:

$$H_0 : \mu = \mu_0, \quad H_1 : \mu \neq \mu_0 \tag{4.111}$$

However, one could instead chose to construct a one-sided hypothesis test, as

$$H_0 : \mu \leq \mu_0, \quad H_1 : \mu > \mu_0 \tag{4.112}$$

or alternatively, in the opposite direction

$$H_0 : \mu \geq \mu_0, \quad H_1 : \mu < \mu_0 \tag{4.113}$$

the choice of setting up which one out of these three hypothesis tests, depend on the context of the problem.

Once the hypothesis is established, the general approach involves calculating the test statistic, along with the corresponding p-value, and then finally making some statement about the null hypothesis based on some chosen level of significance.

[SI*] Possible situations for testing a statistical hypothesis

	H_0 **is true**	H_0 **is false**
Do not reject H_0	Correct decision	Type II error
Reject H_0	Type I error	Correct decision

Table 4.3

[SI*] **Statistical Hypothesis with Discrete Data**

A certain type of cold vaccine is known to be only 25% effective after a period of 2 years. To determine if a new and somewhat more expensive vaccine is superior in providing protection against the same virus for a longer period of time, suppose that 20 people are chosen at random and inoculated. If more than 8 of those receiving the new vaccine surpass the 2-year period without contracting the virus, the new vaccine will be considered superior to the one presently in use. We are essentially testing the null hypothesis that the new vaccine is equally effective after a period of 2 years as the one now commonly used. The alternative hypothesis is that the new vaccine is in fact superior. The hypothesis testing is usually written as follows

$$H_0 : p = 0.25$$
$$H_1 : p > 0.25$$

The test statistic on which we base our decision is X, the number of individuals in our test group who receive protection from the new vaccine for a period of at least 2 years. The possible values of X, from 0 to 20, are divided into two groups: those numbers less than or equal to 8 and those greater than 8. All possible scores greater than 8 constitute the citical region. The last number that we observe in passing into the critical region is called the critical value.

Therefore, if $x > 8$, we reject H_0 in favor of the alternative hypothesis H_1. If $x \leq 8$, we fail to reject H_0.

1. **The Probability of Type I Error**

   The probability of committing a type I error, also caleld the level of significance, is denoted by the Greek letter α.

   In our illustration, a type I error will occur when more than 8 individuals inoculated with the new vaccine surpass the 2-year period without contracting the virus and researchers conclude that the new vaccine is better when it is actually equivalent to the one in use. Hence, if X is the number of individuals who remain free of the virus for at least 2 years,

$$\begin{aligned}\alpha &= P(\text{type I error}) \\ &= P(X > 8, p = 0.25) \\ &= \sum_{x=9}^{20} b(x; 20, 0.25) \\ &= 1 - \sum_{x=0}^{8} b(x; 20, 0.25) \\ &= 1 - 0.9591 \\ &= 0.0409\end{aligned}$$

   We say that the null hypothesis, $p = 0.25$, is being tested at the $\alpha = 0.0409$ level of

significance. Sometimes the elvel of significance is called the size of the test. A critical region of size 0.0409 is very small, and therefore it is unlikely that a type I error will be committed. Consequently, it would be most unusual for more than 8 individuals to remain immune to a virus for a 2-year period using a new vaccine that is essentially equivalent to the one now on the market.

2. **The Probability of Type II Error**
The probability of committing a type II error, denoted by β, is impossible to compute unless we have a specific alternative hypothesis. If we test the null hypothesis that $p = 0.25$ against the alternative hypothesis that $p = 0.5$, then we are able to compute the probability of not rejecting H_0 when it is false.

We simply find the probability of obtaining 8 or fewer in the group to surpass the 2-year period when $p = 0.5$. In this case,

$$\begin{aligned}\beta &= P(\text{type II error}) \\ &= P(X \leq 8, p = 0.5) \\ &= \sum_{x=0}^{8} b(x; 20, 0.5) \\ &= 0.2517\end{aligned}$$

This is a rather high probability, indicating a test procedure in which it is quite likely that we shall reject the new vaccine when, in fact, it is superior to what is now in use. Ideally, we like to use a test procedure for which the type I and type II error probabilities are both small.

The probability of committing both types of error can be reduced by increasing the sample size.

[SI*] **Statistical Hypothesis with Continuous Data**
Consider the null hypothesis that the average weight of male students in a certain college is 68 kilograms against the alternative hypothesis that is unequal to 68. That is, we wish to test

$$\begin{aligned}H_0 &: \mu = 68 \\ H_1 &: \mu \neq 68\end{aligned}$$

The alternative hypothesis allows for the possibility that $\mu < 68$ or $\mu > 68$.

Reject H_0 ($\mu \neq 68$)	Do not reject H_0 ($\mu = 68$)	Reject H_0 ($\mu \neq 68$)

Table 4.4: *Critical region (in light cyan)*

Assume the standard deviation of the population of weights to be $\sigma = 3.6$. For large samples, we may substitute s for σ if no other estimate of σ is available. Our decision statistic, based on a random sample of size $n = 36$, will be $\overline{X}$, the most efficient estimator of μ.
From the Central Limit Theorem, we know that the sampling distribution of $\overline{X}$ is approximately normal with standard deviation

$$\sigma_{\overline{X}} = \frac{\sigma}{\sqrt{n}} = \frac{3.6}{6} = 0.6$$

1. **The Probability of Type I Error**
The probability of committing a type I error, or the level of significance of our test, is equal to the sum of the areas that have been shaded in each tail of the distribution.

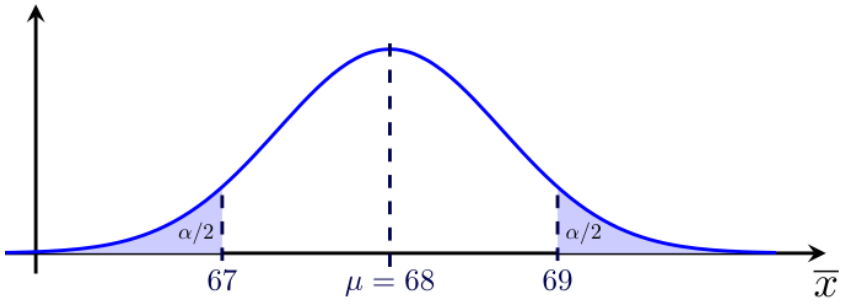

Figure 4.24: *Critical region for testing $\mu = 68$ versus $\mu \neq 68$.*

Therefore

$$\alpha = P(\overline{X} < 67, \mu = 68) + P(\overline{X} > 69, \mu = 68)$$

The z-values corresponding to $\overline{x}_1 = 67$ and $\overline{x}_2 = 69$ when H_0 is true are

$$z_1 = \frac{67 - 68}{0.6} = -1.67$$
$$z_2 = \frac{69 - 68}{0.6} = 1.67$$

Therefore,

$$\begin{aligned} \alpha &= P(Z < -1.67) + P(Z > 1.67) \\ &= 2P(Z < -1.67) \\ &= 0.0950 \end{aligned}$$

The 9.5% of all samples of size 36 would lead us to reject $\mu = 68$ kilograms when, in fact, it is true. To reduce α, we have a choice of increasing the sample size or widening the fail-to-reject region.

Suppose that we increase the sample size to $n = 64$. Then $\sigma_{\overline{X}} = \frac{3.6}{8} = 0.45$. Now

$$z_1 = \frac{67 - 68}{0.45} = -2.22$$
$$z_2 = \frac{69 - 68}{0.45} = 2.22$$

Hence,

$$\begin{aligned} \alpha &= P(Z < -2.22) + P(Z > 2.22) \\ &= 2P(Z < -2.22) \\ &= 0.0264 \end{aligned}$$

The reduction in α is not sufficient by itself to guarantee a good testing procedure. We must also evaluate β for various alternative hypotheses.

2. **The Probability of Type II Error**
   If it is important to reject H_0 when the true mean is some value $\mu \geq 70$ or $\mu \leq 66$, then the probability of committing a type II error should be computed and examined for the alternatives $\mu = 66$ and $\mu = 70$. Because of symmetry, it is only necessary to consider the probability of not rejecting the null hypothesis that $\mu = 68$ when the alternative $\mu = 70$ is true. A type II error will result when the sample mean $\bar{x}$ falls between 67 and 69 when H_1 is true. Therefore, we find that

$$\beta = P(67 \leq \overline{X} \leq 69, \mu = 70)$$

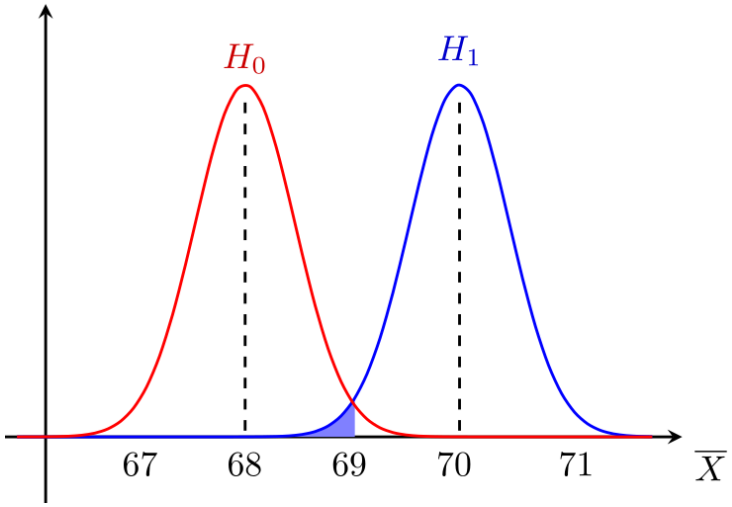

Figure 4.25: *Probability of type II error for testing $\mu = 68$ versus $\mu = 70$.*

The z-values corresponding to $\bar{x}_1 = 67$ and $\bar{x}_2 = 69$ when H_1 is true are

$$z_1 = \frac{67 - 70}{0.45} = -6.67$$
$$z_2 = \frac{69 - 70}{0.45} = -2.22$$

Therefore

$$\begin{aligned}\beta &= P(-6.67 < Z < -2.22) \\ &= P(Z < -2.22) - P(Z < -6.67) \\ &= 0.0132 - 0.0000 \\ &= 0.0132\end{aligned}$$

If the true value of μ is the alternative $\mu = 66$, the value of β will again be 0.0132. For all possible values of $\mu < 66$ or $\mu > 70$, the value of β will be even smaller when $n = 64$, and consequently there would be little chance of not rejecting H_0 when it is false.

Definition 4.58: Important Properties of a Test of Hypothesis

1. The type I error and type II error are related. A decrease in the probability of one generally results in an increase in the probability of the other.
2. The sze of the critical region, and therefore the probability of committing a type I error, can always be reduced by adjusting the critical value(s).
3. An increase in the sample size n will reduce α and β simultaneously.
4. If the null hypothesis is false, β is a maximum when the true value of a parameter approaches the hypothesized value. The greater the distance between the true value and the hypothesized value, the smaller β will be.

Definition 4.59: Power of a Test

The power of a test is the probability of rejecting H_0 given that a specific alternative is true.

The power of a test can be computed as $1 - \beta$. Often different types of tests are compared by contrasting power properties. To produce a desirable power, say, greater than 0.8, one must either increase α or increase the sample size.

Definition 4.60: Test Procedure for a Single Mean (Variance Known)

For large sample size $n > 30$ and known population variance we will use Z statistics

$$z = \frac{\bar{x} - \mu_0}{\sigma/\sqrt{n}} > z_{\alpha/2}$$

or

$$z = \frac{\bar{x} - \mu_0}{\sigma/\sqrt{n}} < -z_{\alpha/2}$$

If $-z_{\alpha/2} < z < z_{\alpha/2}$, do not reject H_0. Rejection of H_0, of course, implies acceptance of the alternative hypothesis $\mu \neq \mu_0$. With this definition of the critical region, it should be clear that there will be probability α of rejecting H_0 (falling into the critical region) when, indeed, $\mu = \mu_0$.

[SI*] Although it is easier to understand the critical region written in terms of z, we can wite the same critical region in terms of the computed average $\overline{x}$. The following can be written as an identical decision procedure:
Reject H_0 if $\overline{x} < a$ or $\overline{x} > b$, where

$$a = \mu_0 - z_{\alpha/2}\frac{\sigma}{\sqrt{n}}$$
$$b = \mu_0 + z_{\alpha/2}\frac{\sigma}{\sqrt{n}}$$

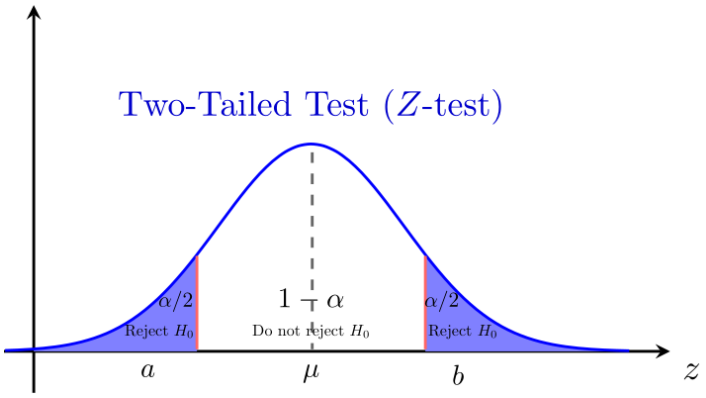

Figure 4.26: *Critical region for the alternative hypothesis $\mu \neq \mu_0$.*

[SI*] Tests on one-sided hypotheses on the mean involve the same statistic described in the two-sided case. The difference, of course, is that the critical region is only in one tail of the standard normal distribution. For example, suppose that we seek to test

$$H_0 : \mu = \mu_0$$
$$H_1 : \mu > \mu_0$$

The signal that favors H_1 comes from large values of z. Thus, rejection of H_0 results when the computed $z > z_\alpha$. Obviously, if the alternative is $H_1 : \mu < \mu_0$, the critical region is entirely in the lower tail and thus rejection results from $z < -z_\alpha$. Although in a one-sided testing case the null hypothesis can be written as $H_0 : \mu \leq \mu_0$ or $H_0 : \mu \geq \mu_0$, it is usually written as $H_0 : \mu = \mu_0$.

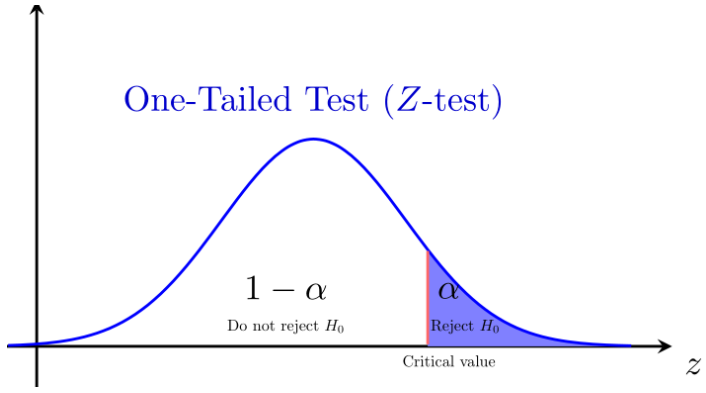

Figure 4.27: *Critical region for the alternative hypothesis $\mu > \mu_0$.*

Definition 4.61: The t-Statistic for a Test on a Single Mean

For two-sided hypothesis with unknown population variance

$$H_0 : \mu = \mu_0$$
$$H_1 : \mu \neq \mu_0$$

We reject H_0 at significance level α when computed t-statistic

$$t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}}$$

exceeds $t_{\frac{\alpha}{2},n-1}$ or is less than $-t_{\frac{\alpha}{2},n-1}$.

The sample variance formula is:

$$s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2$$

with

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n}\bar{x}_i$$

Guide 4.1: On Type I Error for Hypothesis Testing

Type I error has the same analogy when we make a verdict that an innocent person is guilty.

1. **Significance Level (α)**
   The significance level is a predefined maximum probability of commuting type I error we're willing to accept.

   The lower the significance level is, the lower the risk of committing a type I error.

   With the significance level, we can find the critical value to reject the null hypothesis in a hypothesis test. The critical value for right-tailed hypothesis test:

   $$\text{Critical Value} = \mu + Z\left(\frac{\sigma}{\sqrt{n}}\right)$$

   The critical value for left-tailed hypothesis test:

   $$\text{Critical Value} = \mu - Z\left(\frac{\sigma}{\sqrt{n}}\right)$$

   For two-tailed hypothesis test, we use both critical values above so the rejection area will be at $\overline{x} > \mu + Z\left(\frac{\sigma}{\sqrt{n}}\right)$ and at $\overline{x} < \mu - Z\left(\frac{\sigma}{\sqrt{n}}\right)$.

   with μ as the population mean, σ as the population standard deviation, if σ is unknown, we can estimate it using sample standard deviation, s, n as the sample size, Z as the Z statistics associated with a given α. If σ is unknown or the sample size is less than 30, we would use T statistics to produce a more realizable result.

2. **Sample Size (n)**
   When we increase the sample size the probability of type I error would decrease. This means that the precision of the test statistic improves.

3. **Data Variability**
   If the data variability decreases, (i.e., the population standard deviation becomes smaller), we would expect to have a smaller probability of committing a type I error.

Guide 4.2: On Type II Error for Hypothesis Testing

Type II error has the same analogy when we make a verdict that a guilty person is innocent.

1. **Significance Level (α)**
   A decrease in the significant level causes an increase in the probability of the type II error or a decrease in the power.

2. **Sample Size (n)**
   When we increase the sample size the probability of type II error would decrease.

3. **Data Variability**
   Across different significance levels, if the data variability decreases, (i.e., the population standard deviation becomes smaller), the probability of committing a type II error would decrease.

4. **Effect Size**
   The effect size is the magnitude of the difference between the null hypothesis and the alternative hypothesis.

   If the effect size increases, it is easier to detect a true effect, and the probability of a type II error decreases.

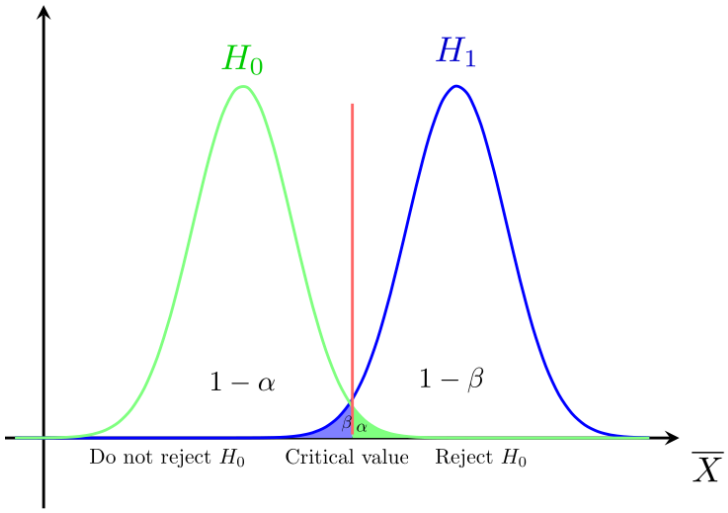

Figure 4.28: *Probability of type I and type II errors.*

[SI*] The hypothesis-testing approach to statistical inference is very closely relate to the confidence interval approach. Confidence interval estimation involves computation of bounds within which it is "reasonable" for the parameter in question to lie. For the case of a single population mean μ with σ^2 known, the structure of both hypothesis testing and confidence interval

estimation is based on the random variable

$$Z = \frac{\overline{X} - \mu}{\sigma / \sqrt{n}}$$

It turns out that the testing of $H_0 : \mu = \mu_0$ against $H_1 : \mu \neq \mu_0$ at a significance level α is equivalent to computing a $100(1-\alpha)\%$ confidence interval on μ and rejecting H_0 if μ_0 is outside the confidence interval. If μ_0 is inside the confidence interval, the hypothesis is not rejected. Recall that with an observed value $\overline{x}$, failure to reject H_0 at significance level α implies that

$$-z_{\alpha/2} \leq \frac{\overline{x} - \mu_0}{\sigma / \sqrt{n}} \leq z_{\alpha/2}$$

which is equivalent to

$$\overline{x} - z_{\alpha/2} \frac{\sigma}{\sqrt{n}} \leq \mu_0 \leq \overline{x} + z_{\alpha/2} \frac{\sigma}{\sqrt{n}}$$

ii. Functions in SymIntegration Related to Hypothesis Testing

[SI*] To compute several computations related to confidence interval, we can use these functions in SymIntegration:

hypothesistest(vector<double> data, double μ, double σ, double α, double effect_size)

this function computes the two-tailed hypothesis test, if $n < 30$ it will automatically use T statistics and compute the sample standard deviation instead of using the given σ, otherwise it will use Z statistics and use the given σ.

hypothesistest_righttailed(vector<double> data, double μ, double σ, double α, double effect_size)

this function computes the right-tailed hypothesis test.

hypothesistest_lefttailed(vector<double> data, double μ, double σ, double α, double effect_size)

this function computes the left-tailed hypothesis test.

hypothesistest_knownvariances_twotailed(vector<double> data1, vector<double> data2, double d_0, double σ_1, double σ_2, double α, double δ)

this function computes the two-tailed hypothesis test for two means with known variances, thus we will use Z statistics. The parameter δ is the difference for the specific alternative, we can choose any small number here, for example for the right-tailed $H_1 : \mu > \mu_0$ the specific alternative will be $\mu = \mu_0 + \delta$, for the left-tailed $H_1 : \mu < \mu_0$ the specific alternative will be $\mu = \mu_0 - \delta$. The parameter δ here is used to compute the power of the test and β (type II error).

hypothesistest_knownvariances_righttailed(vector<double> data1, vector<double> data2, double d_0, double σ_1, double σ_2, double α, double δ)

hypothesistest_knownvariances_lefttailed(vector<double> data1, vector<double> data2, double d_0, double σ_1, double σ_2, double α, double δ)

hypothesistest_equalunknownvariances_twotailed(vector<double> data1, vector<double> data2, double d_0, double α, double δ)

this function computes the two-tailed hypothesis test for two means with equal but unknown variances, thus we will use T statistics with $\nu = n_1 + n_2 - 2$ degrees of freedom.

hypothesistest_equalunknownvariances_righttailed(vector<double> data1, vector<double> data2, double d_0, double α, double δ)

hypothesistest_equalunknownvariances_lefttailed(vector<double> data1, vector<double> data2, double d_0, double α, double δ)

hypothesistest_unequalunknownvariances_twotailed(vector<double> data1, vector<double> data2, double d_0, double α, double δ)

this function computes the two-tailed hypothesis test for two means with unequal and unknown variances, thus we will use T statistics with $\nu = \frac{\left(\frac{s_1^2}{n_1}+\frac{s_2^2}{n_2}\right)^2}{\frac{\left(\frac{s_1^2}{n_1}\right)^2}{n_1-1}+\frac{\left(\frac{s_2^2}{n_2}\right)^2}{n_2-1}}$ degrees of freedom.

hypothesistest_unequalunknownvariances_righttailed(vector<double> data1, vector<double> data2, double d_0, double α, double δ)

hypothesistest_unequalunknownvariances_lefttailed(vector<double> data1, vector<double> data2, double d_0, double α, double δ)

hypothesistest_paired_twotailed(vector<double> data1, vector<double> data2, double d_0, double α, double δ)

this function computes the two-tailed hypothesis test for paired observations using T statistics $t = \frac{\overline{D}-d_0}{s_D/\sqrt{n}}$ with $\nu = n-1$ degrees of freedom.

hypothesistest_paired_righttailed(vector<double> data1, vector<double> data2, double d_0, double α, double δ)

hypothesistest_paired_lefttailed(vector<double> data1, vector<double> data2, double d_0, double α, double δ)

iii. Compute Right-Tailed Hypothesis Testing with SymIntegration

A random sample of 100 deaths in the United States during the past year showed an average life span of 71.8 years. Assuming a population standard deviation of 8.9 years, does this seem to indicate that the mean life span today is greater than 70 years? Use a 0.05 level of significance.

Solution:

We construct the one-sided(right-tailed) hypothesis test as

$$H_0 : \mu = 70 \text{ years}$$
$$H_1 : \mu > 70 \text{ years}$$

with $\alpha = 0.05$. The critical region can be computed by finding the inverse of the cdf of Z / inverse of normal cdf with $\mu = 0, \sigma = 1$ (standard normal distribution), also known as the normal quantile function or the probit function.

To compute the inverse cdf of Z we use Wichura's algorithm, this algorithm calculates the percentage point z_p of the standard normal distribution corresponding to a given lower tail area p, in this case $p = \alpha$.

$$P(Z > z_\alpha) = 0.05$$

With Wichura's algorithm we will obtain:

$$z_\alpha = 1.645$$

thus the critical region is:

$$z > 1.645$$

where

$$z = \frac{\overline{x} - \mu_0}{\sigma / \sqrt{n}}$$

We are using the data from textfile **Vectory.txt** that has 100 data of recorded deaths with mean

$$\overline{x} = 71.8842$$

the standard deviation from the sample is

$$s = 6.63$$

while from the problem it is known that the population standard deviation is

$$\sigma = 8.9$$

because we are using Z statistics so we will use σ instead of s, and hence

$$z_{\text{computed}} = \frac{71.8 - 70}{8.9 / \sqrt{100}} = 2.02$$

Decision: Reject H_0 (because $z_{\text{computed}} > z_\alpha$) and conclude that the mean life span today is greater than 70 years.

If we want to use P-value method, then we need to compute

$$P = P(Z > z_{\text{computed}}) = P(Z > 2.02) = 0.0216$$

As a result, we reach the same conclusion with P-value method, since the P-value $< \alpha$ then we have to reject H_0.

In SymIntegration we have some functions to compute the hypothesis test, for this case we will use this function:
hypothesistest_righttailed(vector<double>, double μ_0, double σ, double α, double δ)

The input is a vector for the data, then the μ_0 that will be used for the H_0, σ is the population standard deviation, if the sample size is less than 30 then the function will use s (sample standard deviation) with t statistics instead, α is the level of significance / type I error, δ is the effect size that is defined with $\delta = |\overline{x} - \mu|$.

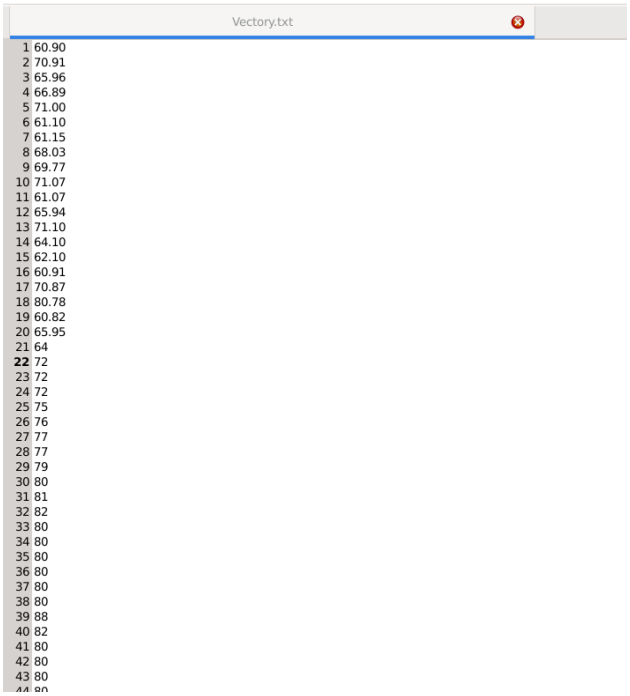

Figure 4.29: *The data that is used for this example is saved as* ***Vectory.txt****.*

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Hypothesis Test ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb  main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Hypothesis Test ]# ./main

Sample size : 100

Mode : 72
Median : 72
Mean : 71.8842
Quantile 1/4: 70
Quantile 3/4: 75.25
Variance : 44.0116
Standard deviation : 6.63412

**********************************************************

Hypothesis testing right-tailed

H0: μ = 70
H1: μ > 70

The probability of a type I error (alpha): 0.05
The probability of a type II error (beta): 0.352857
Power: 0.647143

Critical value: 71.4639
Critical region : z > 1.64485
Computed z : 2.02247

*Reject the null hypothesis when the sample average is greater than 71.4639

*Reject the null hypothesis when the computed z is greater than 1.64485

P-value: 0.0215638

*Reject null hypothesis if P-value <= 0.05 , we fail to reject the null hypothesis if P-value > 0.05

**********************************************************

Time taken by function: 1750 microseconds
```

Figure 4.30: *The computation of one-tailed hypothesis testing (right-tailed) with SymIntegration (**SymIntegration/Examples/Statistics/Test SymIntegration Right-Tailed Hypothesis Testing/main.cpp**).*

iv. Compute Left-Tailed Hypothesis Testing with SymIntegration

The SineBam Electric Institute has published figures on the number of kilowatt hours used annually by various home appliances. It is claimed that a vacuum cleaner uses an average of 46 kilowatt hours per year. If a random sample of 12 homes included in a planned study indicates that vacuum cleaners use an average of 42 kilowatt hours per year with a standar deviation of 11.9 kilowatt hours, does this suggest at the 0.05 level of significance that vacuum cleaners use, on average, less than 46 kilowatt hours annually? Assume the population of kilowatt hours to be normal.

Solution:

We construct the one-sided(left-tailed) hypothesis test as

$$H_0 : \mu = 46 \text{ kilowatt hours}$$
$$H_1 : \mu < 46 \text{ kilowatt hours}$$

with $\alpha = 0.05$. The critical region can be computed by finding the inverse of the cdf of t.

We use Newton-Raphson method to compute the value of t_{alpha} that leaves an area of α to the left.

$$P(T \leq t_\alpha) = 0.05$$
$$t_\alpha = -1.796$$

thus the critical region is:

$$t < -1.796$$

where

$$t = \frac{\overline{x} - \mu_0}{s/\sqrt{n}}$$

We are using the data from textfile **Vectory.txt** that has 12 dummy data of kilowatt hours mean

$$\overline{x} = 46$$

the standard deviation from the sample is

$$s = 0$$

because the dummy data are all 46, while from the problem it is known that the sample standard deviation is

$$s = 11.9$$

so we will use $s = 11.9$, and hence

$$t_{\text{computed}} = \frac{42 - 46}{11.9/\sqrt{12}} = -1.16$$

Decision: Do not reject H_0 (because $t_{\text{computed}} > t_\alpha$) and conclude that the average number of kilowatt hours used annually by home vacuum cleaners is not significantly less than 46.

If we want to use P-value method, then we need to compute

$$P = P(T < t_{\text{computed}}) = P(T < -1.16) \approx 0.135$$

As a result, we reach the same conclusion with P-value method, since the P-value $> \alpha$ then we fail to reject H_0.

In SymIntegration we have some functions to compute the hypothesis test, for this case we will use this function:
hypothesistest_lefttailed(vector<double>, double μ_0, double σ, double α, double δ)

The input is a vector for the data, then the μ_0 that will be used for the H_0, σ is the population standard deviation, if the sample size is less than 30 then the function will read σ as s (sample standard deviation) and we will use t statistics instead, α is the level of significance / type I error, δ is the effect size that is defined with $\delta = |\overline{x} - \mu|$.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Left-Tailed Hypothesis Testing ]# ./main

Sample size : 12

Mode : 46
Median : 46
Mean : 46
Quantile 1/4: 46
Quantile 3/4: 46
Variance : 0
Standard deviation : 0

*******************************************************

Hypothesis testing left-tailed

H0: μ = 46
H1: μ < 46

The probability of a type I error (alpha): 0.05
The probability of a type II error (beta): 0.729687
Power: 0.270313

Critical value: 39.8307
Critical region : t < -1.79588
Computed t : -1.1644

*Reject the null hypothesis when the sample average is less than 39.8307

*Reject the null hypothesis when the computed t is less than -1.79588

P-value: 0.134446

*Reject null hypothesis if P-value <= 0.05 , we fail to reject the null hypothesis if P-value > 0.05

*******************************************************

Time taken by function: 1215 microseconds
```

Figure 4.31: *The computation of one-tailed hypothesis testing (left-tailed) with SymIntegration (**SymIntegration/Examples/Statistics/Test SymIntegration Left-Tailed Hypothesis Testing/main.cpp**).*

v. Compute Two-Tailed Hypothesis Testing with SymIntegration

A manufacturer of sports equipment has developed a new synthetic fishing line that the company claims has a mean breaking strength of 8 kilograms with a standard deviation of 0.5 kilogram. Test the hypothesis that $\mu = 8$ kilograms against the alternative that $\mu \neq 8$ kilograms if a random sample of 50 lines is tested and found to have a mean breaking strength of 7.8 kilograms. Use a 0.01 level of significance.

Solution:

We construct the two-sided(two-tailed) hypothesis test as

$$H_0 : \mu = 8 \text{ kilograms}$$
$$H_1 : \mu \neq 8 \text{ kilograms}$$

with $\alpha = 0.01$.

The critical region can be computed by finding the inverse of the cdf of Z / inverse of normal cdf with $\mu = 0, \sigma = 1$ (standard normal distribution), also known as the normal quantile function or the probit function.

For this two-tailed case, we will have

$$P(Z < -z_{\alpha/2}) = 0.005$$
$$P(Z > z_{\alpha/2}) = 0.005$$

With Wichura's algorithm we will obtain:

$$z_{\alpha/2} = 2.575$$

due to the symmetry of standard normal distribution, the critical regions are:

$$z < -2.575 \quad \text{and} \quad z > 2.575$$

where

$$z = \frac{\overline{x} - \mu_0}{\sigma/\sqrt{n}}$$

We are using the data from textfile **Vectory.txt** that has 50 data of recorded deaths with mean

$$\overline{x} = 7.8$$

the standard deviation from the sample is

$$s = 1.03$$

while from the problem it is known that the population standard deviation is

$$\sigma = 0.5$$

because we are using Z statistics so we will use σ instead of s, and hence

$$z_{\text{computed}} = \frac{7.8 - 8}{0.5/\sqrt{50}} = -2.823843$$

Decision: Reject H_0 (because $z_{\text{computed}} < z_\alpha$) and conclude that the mean breaking strength of the synthetic fishing line is not 8 kilograms, but is in fact, less than 8 kilograms based on the sample data.

Since the test in this example is two-tailed, the desired P-value is twice the area of the shaded region to the left of $z = -2.83$. Therefore, we have

$$P = P(|Z| > 2.83) = 2P(Z < -2.83) = 0.0046$$

In SymIntegration we have some functions to compute the hypothesis test, for this case of two-tailed hypothesis testing we will use this function:
hypothesistest(vector<double>, double μ_0, double σ, double α, double δ)

The input is a vector for the data, then the μ_0 that will be used for the H_0, σ is the population standard deviation, if the sample size is less than 30 then the function will use s (sample standard deviation) with t statistics instead, α is the level of significance / type I error, δ is the effect size that is defined with $\delta = |\overline{x} - \mu|$.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Two-Tailed Hypothesis Testing ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb  main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Two-Tailed Hypothesis Testing ]# ./main

Sample size : 50

Mode : 7
Median : 8
Mean : 7.8
Quantile 1/4: 7
Quantile 3/4: 8
Variance : 1.06122
Standard deviation : 1.03016

**********************************************************

Hypothesis testing two-tailed

H0: μ = 8
H1: μ != 8

The probability of a type I error (alpha): 0.01
The probability of a type II error (beta): 0.199421
Power: 0.800579

Critical value (upper bound): 8.18214
Critical value (lower bound): 7.81786
Critical region : z < - 2.57583 and z > 2.57583
Computed z : -2.82843

*Reject the null hypothesis when the sample average is greater than 8.18214 or less than 7.81786

*Reject the null hypothesis when the computed z is greater than 2.57583 or smaller than -2.57583

P-value: 0.00467773

*Reject null hypothesis if P-value <= 0.01 , we fail to reject the null hypothesis if P-value > 0.01

**********************************************************

Time taken by function: 1459 microseconds
```

Figure 4.32: *The computation of two-tailed hypothesis testing with SymIntegration (**SymIntegration/Examples/Statistics/Test SymIntegration Two-Tailed Hypothesis Testing/main.cpp**).*

vi. Two Samples Hypothesis Tests for Comparing Means

[SI*] Tests concerning two means represent a set of very important analytical tools for the scientist or engineer. Two independent random samples of sizes n_1 and n_2, respectively, are drawn from two populations with means μ_1 and μ_2 and variances σ_1^2 and σ_2^2. We know that the random variable

$$Z = \frac{(\overline{X}_1 - \overline{X}_2) - (\mu_1 - \mu_2)}{\sqrt{\frac{\sigma_1^2}{n_1} + \frac{\sigma_2^2}{n_2}}}$$

has a standard normal distribution. Here we assuming that n_1 and n_2 are sufficiently large that the Central Limit Theorem applies. Of course, if the two populations are normal, the statistic above has a standard normal distribution even for small n_1 and n_2. Obviously, if we can assume that $\sigma_1 = \sigma_2 = \sigma$, the statistic above reduces to

$$Z = \frac{(\overline{X}_1 - \overline{X}_2) - (\mu_1 - \mu_2)}{\sigma\sqrt{\frac{1}{n_1} + \frac{1}{n_2}}}$$

The two statistics above serve as a basis for the development of the test procedures involving two means. The equivalence between tests and confidence intervals, along with the technical detail involving tests on one mean, allow a simple transition to tests on two means.

[SI*] The two-sided hypothesis on two means can be written generally as

$$H_0 : \mu_1 - \mu_2 = d_0$$

Obviously, the alternative can be two sided or one sided. Again, the distribution used is the distribution of the test statistic under H_0. Values $\overline{x}_1$ and $\overline{x}_2$ are computed and, for σ_1 and σ_2 known, the test statistic is given by

$$z = \frac{(\overline{x}_1 - \overline{x}_2) - d_0}{\sqrt{\frac{\sigma_1^2}{n_1} + \frac{\sigma_2^2}{n_2}}}$$

with a two-tailed critical region in the case of a two-sided alternative. That is, reject H_0 in favor of $H_1 : \mu_1 - \mu_2 \neq d_0$ if $z > z_{\alpha/2}$ or $z < -z_{\alpha/2}$. One-tailed critical regions are used in the case of the one-sided alternatives.

[SI*] Unknown But Equal Variances

The more prevalent situations involving tests on two means are those in which variances are unknown. If the scientist involved is willing to assume that both distributions are normal and that $\sigma_1 = \sigma_2 = \sigma$, the pooled t-test (two-sampe t-test) may be used.

Definition 4.62: Two Sample Pooled t-Test

For the two-sided hypothesis

$$H_0 : \mu_1 = \mu_2$$
$$H_1 : \mu_1 \neq \mu_2$$

we reject H_0 at significance level α when the computed t-statistic

$$t = \frac{(\overline{x}_1 - \overline{x}_2) - d_0}{s_p\sqrt{\frac{1}{n_1} + \frac{1}{n_2}}}$$

where

$$s_p^2 = \frac{s_1^2(n_1 - 1) + s_2^2(n_2 - 1)}{n_1 + n_2 - 2}$$

exceeds $t_{\frac{\alpha}{2}, n_1+n_2-2}$ or is less than $-t_{\frac{\alpha}{2}, n_1+n_2-2}$.

[SI*] **Unknown but Unequal Variances**

There are situations where the analyst is not able to assume that $\sigma_1 = \sigma_2$. Recall that, if the populations are normal, the statistic

$$T' = \frac{(\overline{X}_1 - \overline{X}_2) - d_0}{\sqrt{\frac{s_1^2}{n_1} + fracs_2^2 n_2}}$$

has an approximate t-distribution with approximate degrees of freedom

$$\nu = \frac{\left(\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}\right)^2}{\frac{\left(\frac{s_1^2}{n_1}\right)^2}{n_1 - 1} + \frac{\left(\frac{s_2^2}{n_2}\right)^2}{n_2 - 1}}$$

As a result, the test procedure is to not reject H_0 when

$$-t_{\frac{\alpha}{2}, \nu} < t' < t_{\frac{\alpha}{2}, \nu}$$

with ν given as above. Again, as in the case of the pooled t-test, one-sided alternatives suggest one-sided critical regions.

[SI*] **Paired Observations**

A study of the two-sample t-test or confidence interval on the difference between means should suggest the need for experimental design. It was suggested that the conditions of the two populations (often referred to as the two treatments) should be assigned randomly to the experimental units. This is done to avoid biased results due to systematic differences between experimental units. In other wods, in hypothesis-testing jargon, it is important that any significant difference found between means be due to the different conditions of the populations and not due to the experimental units in the study.

Testing of two means can be accomplishedwhen data are in the form of paired observations. In this pairing structure, the conditions of the two populations (treatments) are assigned

randomly within homogeneous units. Computation of the confidence interval for $\mu_1 - \mu_2$ in the situation with paired observations is based on the random variable

$$T = \frac{\overline{D} - \mu_D}{S_d / \sqrt{n}}$$

where $\overline{D}$ and S_d are random variables representing the sample mean and standard deviation of the differences of the observations in the experimental units. As in the case of the pooled t-test, the assumption is that the observations from each population are normal. This two-sample problem is essentially reduced to a one-sample problem by using the computed differences $d_1, d_2, \ldots, d_n$. Thus, the hypothesis reduces to

$$H_0 : \mu_D = d_0$$

The computed test statistic is then given by

$$t = \frac{\overline{d} - d_0}{s_d / \sqrt{n}}$$

Critical regions are constructed using the t-distribution with $n - 1$ degrees of freedom.

[SI*] **Choice of Sample Size for Testing Means**

In most practical circumstances, the experiment should be planned, with a choice of sample size made prior to the data-taking process if possible. the sample size is usually determined to achieve good power for a fixed α and fixed specific alternative. This fixed alternative may be in the form of $\mu - \mu_0$ in the case of a hypothesis involving a single mean or $\mu_1 - \mu_2$ in the case of a problem involving two means.

Suppose that we wish to test the hypothesis

$$H_0 : \mu = \mu_0$$
$$H_1 : \mu > \mu_0$$

with a significance level α, when the variance σ^2 is known. For a specific alternative, say $\mu = \mu_0 + \delta$, the power of the test is

$$1 - \beta = P(\overline{X} > a \text{ when } \mu = \mu_0 + \delta)$$

Therefore,

$$\begin{aligned} \beta &= P(\overline{X} < a \text{ when } \mu = \mu_0 + \delta) \\ &= P\left(\frac{\overline{X} - (\mu_0 + \delta)}{\sigma / \sqrt{n}} < \frac{a - (\mu_0 + \delta)}{\sigma / \sqrt{n}} \text{ when } \mu = \mu_0 + \delta\right) \end{aligned}$$

Under the alternative hypothesis $\mu = \mu_0 + \delta$, the statistic

$$\frac{\overline{X} - (\mu_0 + \delta)}{\sigma / \sqrt{n}}$$

is the standard normal variable Z. So

$$\beta = P\left(Z < \frac{a - \mu_0}{\sigma / \sqrt{n}} - \frac{\delta}{\sigma / \sqrt{n}}\right) = P\left(Z < z_\alpha - \frac{\delta}{\sigma / \sqrt{n}}\right)$$

from which we conclude that

$$-z_\beta = z_\alpha - \frac{\delta\sqrt{n}}{\sigma}$$

and hence the choice of sample size

$$n = \frac{(z_\alpha + z_\beta)^2\sigma^2}{\delta^2}$$

a result that is also true when the alternative hypothesis is $H_1 : \mu < \mu_0$.

In the case of a two-tailed test, we obtain the power $1 - \beta$ for a specified alternative when

$$n \approx \frac{(z_{\alpha/2} + z_\beta)^2\sigma^2}{\delta^2}$$

vii. Compute Right-Tailed Hypothesis Testing on Two Means with Unknown But Equal Variances with SymIntegration

An experiment was peformed to compare the abrasive wear of two different laminated materials. Twelve pieces of material 1 were tested by exposing each piece to a machine measuring wear. Ten pieces of material 2 were similarly tested. In each case, the depth of wear was observed. The samples of material 1 gave an average (code) wear of 85 units with a sample standard deviation of 4, while the samples of material 2 gave an average of 81 with a sample standard deviation of 5. Can we conclude at the 0.05 level of significance that the abrasive wear of material 1 exceeds that of material 2 by more than 2 units? Assume the populations to be approximately normal with equal variances.

Solution:

Let μ_1 and μ_2 represent the population means of the abrasive wear for material 1 and material 2, respectively.

We construct the one-sided (right-tailed) hypothesis test as

$$H_0 : \ \mu_1 - \mu_2 = 2$$
$$H_1 : \ \mu_1 - \mu_2 > 2$$

with $\alpha = 0.05$. The critical region can be computed by finding the inverse of the cdf of t.

$$P(T > t_\alpha) = 0.05$$

thus, we will obtain

$$t_\alpha = 1.725$$

where

$$t = \frac{(\overline{x}_1 - \overline{x}_2) - d_0}{s_p \sqrt{\frac{1}{n_1} + \frac{1}{n_2}}}$$

with $\nu = 20$ degrees of freedom.

We are using data from textfiles **Vectorx.txt** and **Vectory.txt**, the computations are:

$$\overline{x}_1 = 85, \quad s_1 = 4, \quad n_1 = 12$$
$$\overline{x}_2 = 81, \quad s_2 = 5, \quad n_2 = 10$$

Hence

$$s_p = \sqrt{\frac{(11)(16) + (9)(25)}{12 + 10 - 2}} = 4.478$$
$$t_{\text{computed}} = \frac{(85 - 81) - 2}{4.478 \sqrt{\frac{1}{12} + \frac{1}{10}}} = 1.04$$
$$P = P(T > 1.04) \approx 0.16$$

Decision: Do not reject H_0. We are unable to conclude that the abrasive wear of material 1 exceeds that of material 2 by more than 2 units.

In SymIntegration we have some functions to compute the hypothesis test for tests on two means, for this case we will use this function:
hypothesistest_equalunknownvariances_righttailed(vector<double> data1, vector<double> data2, double d_0, double α)

The textfile **Vectorx.txt** will be **data1** (the corresponding sample mean is $\overline{x}_1$ and sample standard deviation is s_1), the textfile **Vectory.txt** will be **data2** (the corresponding sample mean is $\overline{x}_2$ and sample standard deviation is s_2), d_0 is the difference we want to test in the null hypothesis $H_0 : \mu_1 - \mu_2 = d_0$, and α is the level of significance.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Two Samples Right-Tailed Hypothesis Testing ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb  main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Two Samples Right-Tailed Hypothesis Testing ]# ./main

Sample size : 12

Mode : 88
Median : 88
Mean : 85.0417
Quantile 1/4: 80
Quantile 3/4: 88
Variance : 16.1117
Standard deviation : 4.01394
Sample size : 10

Mode : 83
Median : 83
Mean : 81.07
Quantile 1/4: 78
Quantile 3/4: 85.025
Variance : 25.3379
Standard deviation : 5.03368

**********************************************************

Hypothesis testing right-tailed

H0: μ1 - μ2 = 2
H1: μ1 - μ2 > 2

The probability of a type I error (alpha): 0.05
The probability of a type II error (beta): 0.95
Power: 0.05

Critical value: 3.73604
Critical region : t > 1.72472
Computed t : 1.02295

*Reject the null hypothesis when the computed t is greater than 1.72472

P-value: 0.159273

*Reject null hypothesis if P-value <= 0.05 , we fail to reject the null hypothesis if P-value > 0.05

**********************************************************

Time taken by function: 1422 microseconds
```

Figure 4.33: *The computation of right-tailed hypothesis testing for two samples tests on two means with same but unknown variances (**SymIntegration/Examples/Statistics/Test SymIntegration Two Samples Right-Tailed Hypothesis Testing/main.cpp**).*

viii. Compute Two-Tailed Hypothesis Testing on Two Means with Known Variances with SymIntegration

A random sample of size $n_1 = 25$, taken from a normal population with a standard deviation $\sigma_1 = 5.2$, has a mean $\overline{x}_1 = 81$. A second random sample of size $n_2 = 36$, taken from a different normal population with a standard deviation $\sigma_2 = 3.4$, has a mean $\overline{x}_2 = 76$. Test the hypothesis that $\mu_1 = \mu_2$ against the alternative, $\mu_1 \neq \mu_2$. Quote a P-value in your conclusion.

Solution:

We construct the two-sided (two-tailed) hypothesis test as

$$H_0 : \mu_1 - \mu_2 = 0$$
$$H_1 : \mu_1 - \mu_2 \neq 0$$

with $\alpha = 0.05$, since the problem does not state α, we assume this α to be used to compute the critical region.

Meanwhile to only make a decision, we don't really need α, we only need to compute

$$z = \frac{(\overline{x}_1 - \overline{x}_2) - d_0}{\sqrt{\frac{\sigma_1^2}{n_1} + \frac{\sigma_2^2}{n_2}}} = \frac{(81 - 76) - 0}{\sqrt{\frac{5.2^2}{25} + \frac{3.5^2}{36}}} = 4.22$$

So, the P-value is

$$P(|Z| > 4.22) = 2P(Z < -4.22) = 0.00002428$$

Since P value is smaller than our chosen $\alpha = 0.05$ then we reject the null hypothesis $H_0 : \mu_1 - \mu_2 = 0$, even if we use smaller $\alpha = 0.01$ or $\alpha = 0.0001$ we will still reject the null hypothesis based on P-value method. We conclude that $\mu_1 > \mu_2$.

In SymIntegration we have some functions to compute the hypothesis test for tests on two means, for this case we will use this function:
hypothesistest_knownvariances_twotailed(vector<double> data1, vector<double> data2, double d_0, double σ_1, double σ_2, double α, double δ)

The textfile **Vectorx.txt** will be **data1** (the corresponding sample mean is $\overline{x}_1$ and known population standard deviation is σ_1), the textfile **Vectory.txt** will be **data2** (the corresponding sample mean is $\overline{x}_2$ and known population standard deviation is σ_2), d_0 is the difference we want to test in the null hypothesis $H_0 : \mu_1 - \mu_2 = d_0$, α is the level of significance, δ is the difference for the specific alternative, we can choose any small number here, for example for the right-tailed $H_1 : \mu > \mu_0$ the specific alternative will be $\mu = \mu_0 + \delta$, for the left-tailed $H_1 : \mu < \mu_0$ the specific alternative will be $\mu = \mu_0 - \delta$. The parameter δ here is used to compute the power of the test and β (type II error).

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Two Samples Two-Tailed Hypothesis Testing ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb  main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Two Samples Two-Tailed Hypothesis Testing ]# ./main

*********************************************************

Hypothesis testing two-tailed, known variances

H0: μ1 - μ2 = 0
H1: μ1 - μ2 != 0

Mean data 1 = 81, σ1 = 5.2, n1 = 25
Mean data 2 = 76, σ2 = 3.4, n2 = 36

The probability of a type I error (alpha): 0.05
The probability of a type II error (beta): 0.264582
Power: 0.735418

Critical value (upper bound): 1.5745
Critical value (lower bound): -1.5745
Critical region : z < - 1.95996 and z > 1.95996
Computed z : 4.22169

*Reject the null hypothesis when the computed z is greater than 1.95996 or less than -1.95996

P-value: 2.42482e-05

*Reject null hypothesis if P-value <= 0.05 , we fail to reject the null hypothesis if P-value > 0.05

*********************************************************

Time taken by function: 969 microseconds
```

Figure 4.34: *The computation of two-tailed hypothesis testing for two samples tests on two means with known variances (**SymIntegration/Examples/Statistics/Test SymIntegration Two Samples Two-Tailed Hypothesis Testing/main.cpp**).*

ix. Compute Two-Tailed Hypothesis Testing on Two Means with Unequal and Unknown Variances with SymIntegration

In a study conducted at Virginia Tech, the plasma ascorbic acid levels of pregnant women were compared for smokers versus nonsmokers. Thirty-two women in the last three months of pregnancy, free of major health disorders and ranging in age from 15 to 32 years, were selected for the study. Prior to the collection 20 ml of blood, the participants were told to avoid breakfast, forgo their vitamin supplements, and avoid foods high in ascorbic acid content. From the blood samples, the following plasma ascorbic acid values were determined, in milligrams per 100 milliliters:

Is there sufficient evidence to conclude that there is a difference between plasma ascorbic acid levels of smokers and nonsmokers? Assume that the two sets of data came from normal populations with unequal variances. Use a P-value.

Nonsmokers		**Smokers**
0.97	1.16	0.48
0.72	0.86	0.71
1.00	0.85	0.98
0.81	0.58	0.68
0.62	0.57	1.18
1.32	0.64	1.36
1.24	0.98	0.78
0.99	1.09	1.64
0.90	0.92	
0.74	0.78	
0.88	1.24	
0.94	1.18	

Table 4.5: *Plasma ascorbic acid values.*

Solution:

We construct the two-sided (two-tailed) hypothesis test as

$$H_0 : \mu_1 - \mu_2 = 0$$
$$H_1 : \mu_1 - \mu_2 \neq 0$$

with $\alpha = 0.05$, since the problem does not state α, we assume this α to be used to compute the critical region.

The degrees of freedom is calculated as

$$\nu = \frac{\left(\frac{0.391478^2}{8} + \frac{0.214414^2}{24}\right)^2}{\frac{\left(\frac{0.391478^2}{8}\right)^2}{7} + \frac{\left(\frac{0.214414^2}{24}\right)^2}{23}} = 8.44$$

Hence, we use 8 degrees of freedom. Now, we compute the t_{computed}

$$t_{\text{computed}} = \frac{(\overline{x}_1 - \overline{x}_2) - d_0}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}} = \frac{0.91583 - 0.97625}{\sqrt{\frac{0.391478^2}{8} + \frac{0.214414^2}{24}}} \approx -0.42$$

The P-value is

$$P(|T| > 0.42) = 2P(T < -0.42) = 0.688207$$

Decision: We fail to reject the null hypothesis, thus there is no difference between plasma ascorbic acid levels of smokers and nonsmokers.

In SymIntegration we have some functions to compute the hypothesis test for tests on two means, for this case we will use this function:
hypothesistest_unequalunknownvariances_twotailed(vector<double> data1, vector<double> data2, double d_0, double α, double δ)

The textfile **Vectorx.txt** will be **data1** (the corresponding sample mean is $\overline{x}_1$ and the sample standard deviation is s_1), the textfile **Vectory.txt** will be **data2** (the corresponding sample mean is $\overline{x}_2$ and the sample standard deviation is s_2), d_0 is the difference we want to test in the null hypothesis $H_0 : \mu_1 - \mu_2 = d_0$, α is the level of significance, δ is the difference for the specific alternative, we can choose any small number here, for example for the right-tailed $H_1 : \mu > \mu_0$ the specific alternative will be $\mu = \mu_0 + \delta$, for the left-tailed $H_1 : \mu < \mu_0$ the specific alternative will be $\mu = \mu_0 - \delta$. The parameter δ here is used to compute the power of the test and β (type II error).

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Two Samples Two-Tailed Hypothesis Testing Unequal Unknown Variances ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb  main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Two Samples Two-Tailed Hypothesis Testing Unequal Unknown Variances ]# ./main

**********************************************************

Hypothesis testing two-tailed, unequal and unknown variances

H0: μ1 - μ2 = 0
H1: μ1 - μ2 != 0

Mean data 1 = 0.915833, s1 = 0.214414, n1 = 24
Mean data 2 = 0.97625, s2 = 0.391478, n2 = 8

The probability of a type I error (alpha): 0.05
The probability of a type II error (beta): 0.00179547
Power: 0.998205

Critical value (upper bound): 0.210049
Critical value (lower bound): -0.210049
Critical region : t < - 2.306 and t > 2.306
Computed t : -0.416197

*Reject the null hypothesis when the computed t is greater than 2.306 or less than -2.306

P-value: 0.688207

*Reject null hypothesis if P-value <= 0.05 , we fail to reject the null hypothesis if P-value > 0.05

**********************************************************

Time taken by function: 937 microseconds
```

Figure 4.35: *The computation of two-tailed hypothesis testing for two samples tests on two means with unequal and unknown variances (**SymIntegration/Examples/Statistics/Test SymIntegration Two Samples Two-Tailed Hypothesis Testing Unequal Unknown Variances/main.cpp**).*

x. Compute Two-Tailed Hypothesis Testing on Paired Observations with SymIntegration

In a study conducted in the Forestry and Wildlife Department at Virginia Tech, J. A. Wesson examined the influence of the drug succinylcholine on the circulation levels of androgens in the blood. Blood samples were taken from wild, free-ranging deer immediately after they had received an intramuscular injection of succinylcholine administered using darts and a capture gun. A second blood sample was obtained from each deer 30 minutes after the first sample, after which the deer was released. The levels of androgens at time of capture and 30 minutes later, measured in nanograms per milliliter (ng/mL), for 15 deer.

Assuming that the populations of androgen levels at time of injection and 30 minutes later are normally distributed, test at the 0.05 level of significance whether the androgen concentrations are altered after 30 minutes.

Deer	Androgen At Time of Injection	Androgen 30 Minutes after Injection	d_i
1	2.76	7.02	4.26
2	5.18	3.10	-2.08
3	2.68	5.44	2.76
4	3.05	3.99	0.94
5	4.10	5.21	1.11
6	7.05	10.26	3.21
7	6.60	13.91	7.31
8	4.79	18.53	13.74
9	7.39	7.91	0.52
10	7.30	4.85	-2.45
11	11.78	11.10	-0.68
12	3.90	3.74	-0.16
13	26.00	94.03	68.03
14	67.48	94.03	26.55
15	17.04	41.70	24.66

Table 4.6: *The level of androgens at time of capture and 30 minutes later, measured in nanograms per milliliter (ng/mL).*

Solution:

Let μ_1 and μ_2 be the average androgen concentration at the time of injection and 30 minutes later, respectively.

We construct the two-sided (two-tailed) hypothesis test as

$$H_0 : \mu_D = \mu_2 - \mu_1 = 0$$
$$H_1 : \mu_D = \mu_2 - \mu_1 \neq 0$$

with $\alpha = 0.05$. The critical region can be computed by finding the inverse of the cdf of t.

$$P(T < t_{\alpha/2}) = 0.025$$

thus, we will obtain

$$t_{\alpha/2} = 2.145$$

so the critical region is when $t < -2.145$ or $t > 2.145$, where

$$t_{\text{computed}} = \frac{\overline{d} - d_0}{s_D/\sqrt{n}}$$

with $\nu = 14$ degrees of freedom.

Therefore,

$$t_{\text{computed}} = \frac{9.848 - 0}{18.474/\sqrt{15}} = 2.06$$

Though the t-statistic is not significant at the 0.05 level, we obtain the P-value

$$P = P(|T| > 2.06) \approx 0.06$$

As a result, there is some evidence that there is a difference in mean circulating levels of androgen.

The assumption of no interaction would imply that the effect on androgen levels of the deer is roughly the same in the data for both treatments, i.e., at the time of injection of succinylcholine and 30 minutes following injection. This can be expressed with the two factors switching roles; for example, the difference in treatments is roughly the same across the units (i.e., the deer). There certainly are some deer/treatment combinations for which the no interaction assumption seems to hold, but there is hardly any strong evidence that the experimental units are homogeneous.

However, the nature of the interaction and the resulting increase in $\text{Var}(\overline{D})$ appear to be dominated by a substantial difference in the treatments. This is further demonstrated by the fact that 11 of the 15 deer exhibited positive signs for the computed d_i and the negative d_i (for deer $2, 10, 11$, and 12) are small in magnitude compared to the 12 positive ones. Thus, it appears that the mean level of androgen is significantly higher 30 minutes following injection than at injection, and the conclusions may be stronger than $p = 0.06$ would suggest.

In SymIntegration we have some functions to compute the hypothesis test for tests on paired observations, for this case we will use this function:
hypothesistest_paired_twotailed(vector<double> data1, vector<double> data2, double d_0, double α, double δ)

The textfile **Vectorx.txt** will be **data1** (the corresponding sample mean is $\overline{x}_1$), the textfile **Vectory.txt** will be **data2** (the corresponding sample mean is $\overline{x}_2$), we make a new vector **vector<double> datad** in the function to compute the difference between **data1** and **data2**, thus datad[i] = data1[i] - data2[i], for $i = 1, 2, \ldots, n$, with n is the number of the paired observations,**datad**, **data1** and **data2** all have the same number of sample size: n. d_0 is the difference we want to test in the null hypothesis $H_0 : \mu_D = \mu_2 - \mu_1 = d_0$, α is the level of significance, δ is the difference for the specific alternative, we can choose any small number here, for example for the right-tailed $H_1 : \mu_D > d_0$ the specific alternative will be $\mu_D = d_0 + \delta$, for the left-tailed $H_1 : \mu_D < d_0$ the specific alternative

will be $\mu_D = d_0 - \delta$. The parameter δ here is used to compute the power of the test and β (type II error).

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Two Samples Two-Tailed Hypothesis Testing Paired Observations ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb  main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Two Samples Two-Tailed Hypothesis Testing Paired Observations ]# ./main

*********************************************************

Hypothesis testing two-tailed, paired observations

H0: μd = μ1 - μ2 = 0
H1: μd = μ1 - μ2 != 0

Mean data 1 = 21.6547, s1 = 30.9207, n1 = 15
Mean data 2 = 11.8067, s2 = 16.6364, n2 = 15
Mean d = 9.848, sd = 18.4736, nd = 15

The probability of a type I error (alpha): 0.05
The probability of a type II error (beta): 0.0734377
Power: 0.926562

Critical value (upper bound): 10.2304
Critical value (lower bound): -10.2304
Critical region : t < - 2.14479 and t > 2.14479
Computed t : 2.06463

*Reject the null hypothesis when the computed t is greater than 2.14479 or less than -2.14479

P-value: 0.0579983

*Reject null hypothesis if P-value <= 0.05 , we fail to reject the null hypothesis if P-value > 0.05

*********************************************************

Time taken by function: 1068 microseconds
```

Figure 4.36: *The computation of two-tailed hypothesis testing for paired observations (**SymIntegration/Examples/Statistics/Test SymIntegration Two Samples Two-Tailed Hypothesis Testing Paired Observations/main.cpp**).*

xi. Hypothesis Test on a Single Proportion

[SI*] We shall consider the problem of testing the hypothesis that the proportion of successes in a binomial experiment equals some specified value. That is, we are testing the null hypothesis H_0 that $p = p_0$, where p is the parameter of the binomial distribution. The alternative hypothesis may be one of the usual one-sided or two-sided alternatives:

$$p < p_0,\ p > p_0,\ \text{or } p \neq p_0$$

The appropriate random variable on which we base our decision criterion is the binomial random variable X, although we could just as well use the statistic $\hat{p} = \frac{X}{n}$. Values of X that are far from the mean $\mu = np_0$ will lead to the rejection of the null hypothesis. Because X is a discrete binomial variable, it is unlikely that a critical region can be established whose size is exactly equal to a prespecified value fo α. For this reason it is preferable, in dealing with small samples, to base our decisions on P-values. TO test the hypothesis

$$H_0 : p = p_0$$
$$H_1 : p < p_0$$

we use the binomial distribution to compute the P-value

$$P = P(X \leq x \text{ when } p = p_0)$$

The value x is the number of successes in our sample of size n. If this P-value is less than or equal to α, our test is significant at the α level and we reject H_0 in favor of H_1. Similarly, to test the hypothesis

$$H_0 : p = p_0$$
$$H_1 : p > p_0$$

at the α-level of significance, we compute

$$P = P(X \geq x \text{ when } p = p_0)$$

and reject H_0 in favor of H_1 if this P-value is less than or equal to α. Finally, to test the hypothesis

$$H_0 : p = p_0$$
$$H_1 : p \neq p_0$$

at the α-level of significance, we compute

$$P = 2P(X \leq x \text{ when } p = p_0) \quad \text{if } x < np_0$$

or

$$P = 2P(X \geq x \text{ when } p = p_0) \quad \text{if } x > np_0$$

and reject H_0 in favor of H_1 if the computed P-value is less than or equal to α.

[SI*] The binomial probabilities can be obtained from the actual binomial formula when n is small. For large n, approximation procedures are required. When the hypothesized value p_0 is very close to 0 or 1, the Poisson distribution, with parameter $\lambda = np_0$ may be used. However, the normal curve approximation, with parameters $\mu = np_0$ and $\sigma^2 = np_0q_0$, is usually preferred

for large n and is very accurate as long as p_0 is not extremely close to 0 or to 1. If we use the normal approximation, the z-value for testing $p = p_0$ is given by

$$z = \frac{x - np_0}{\sqrt{np_0q_0}} = \frac{\hat{p} - p_0}{\sqrt{\frac{p_0q_0}{n}}}$$

which is a value of the standard normal variable Z. Hence, for a two-tailed test at the α-level of significance, the critical region is $z < -z_{-\alpha/2}$ or $z > z_{\alpha/2}$. For the one-sided alternative $p < p_0$, the critical region is $z < -z_\alpha$, and for the alternative $p > p_0$, the critical region is $z > z_\alpha$.

xii. Compute Right-Tailed Hypothesis Testing on a Single Proportion with SymIntegration

A commonly prescribed drug for relieving nervous tension is believed to be only 60% effective. Experimental results with a new drug administered to a random sample of 100 adults who were suffering from nervous tension show that 70 received relief. Is this sufficient evidence to conclude that the new drug is superior the the one commonly prescribed? Use a 0.05 level of significance.

Solution:

We construct the one-sided(right-tailed) hypothesis test as

$$H_0 : \ p = 0.6$$
$$H_1 : \ p > 0.6$$

with $\alpha = 0.05$. The critical region can be computed by finding the inverse of the cdf of Z

$$P(Z > z_\alpha) = 0.05$$

we will obtain $z_\alpha = 1.645$, thus the critical region is

$$z > 1.645$$

Now for the computation

$$z_{\text{computed}} = \frac{\hat{p} - p_0}{\sqrt{\frac{p_0 q_0}{n}}} = \frac{0.7 - 0.6}{\sqrt{\frac{(0.6)(0.4)}{100}}} = 2.04$$

and the P-value

$$P = P(Z > 2.04) = 0.0206134$$

Decision: P-value is less than α. Reject H_0 and conclude that the new drug is superior.

In SymIntegration we have some functions to compute the hypothesis test for tests on a single proportion, for this case we will use this function:
hypothesistest_proportion_righttailed(vector<string> data, const string &inputString, double p_0, double α)

The textfile **drug.txt** will be **data**, the **inputString** is the proportion for the string (e.g. RELIEF, Defective, CUPULU, PLUTUTUT, etc) that we want to compute in the **data**, the p_0 is the hypothesized value in the null hypothesis $H_0 : p = p_0$ and the alternative hypothesis $H_1 : p > p_0$, and α is the level of significance.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Hypothesis Testing for Proportion ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Hypothesis Testing for Proportion ]# ./main

*********************************************************

Right-tailed hypothesis testing on a single proportion

H0: p = 0.6
H1: p > 0.6
Data size =  100
Amount of RELIEF in the data = 70
Proportion for RELIEF, p = 0.7, q = 0.3
Critical region : z > 1.64485
Computed z : 2.04124

*********************************************************

*Reject the null hypothesis when the computed z is greater than 1.64485

P-value: 0.0206134

*Reject null hypothesis if P-value <= 0.05 , we fail to reject the null hypothesis if P-value > 0.05

*********************************************************

Time taken by function: 816 microseconds
```

Figure 4.37: *The computation of two-tailed hypothesis testing on a single proportion (**SymIntegration/Examples/Statistics/Test SymIntegration Hypothesis Testing on a Single Proportion/main.cpp**).*

xiii. Hypothesis Tests on Two Proportions

[SI*] Situations often arise where we wish to test the hypothesis that two proportions are equal. In general, we wish to test the null hypothesis that two proportions, or binomial parameters, are equal. That is, we are testing $p_1 = p_2$ against one of the alternatives $p_1 < p_2, p_1 > p_2$, or $p_1 \neq p_2$. Of course, this is equivalent to testing the null hypothesis that $p_1 - p_2 = 0$ against one of the alternatives $p_1 - p_2 < 0,\ p_1 - p_2 > 0$, or $p_1 - p_2 \neq 0$. The statistic on which we base our decision is the random variable $\hat{P}_1 - \hat{P}_2$. Independent samples of sizes n_1 and n_2 are selected at random from two binomial populations and the proportions of successes $\hat{P}_1$ and $\hat{P}_2$ for the two samples are computed.

[SI*] In our construction of confidence intervals for p_1 and p_2 we noted, for n_1 and n_2 sufficiently large, that the point estimator $\hat{P}_1$ minus $\hat{P}_2$ was approximately normally distributed with mean

$$\mu_{\hat{P}_1 - \hat{P}_2} = p_1 - p_2$$

and variance

$$\sigma^2_{\hat{P}_1 - \hat{P}_2} = \frac{p_1 q_1}{n_1} + \frac{p_2 q_2}{n_2}$$

Therefore, our critical region(s) can be established by using the standard normal variable

$$Z = \frac{(\hat{P}_1 - \hat{P}_2) - (p_1 - p_2)}{\sqrt{\frac{p_1 q_1}{n_1} + \frac{p_2 q_2}{n_2}}}$$

When H_0 is true, we can substitute $p_1 = p_2 = p$ and $q_1 = q_2 = q$ (where p and q are the common values) in the preeceding formula for Z to give the form

$$Z = \frac{\hat{P}_1 - \hat{P}_2}{\sqrt{pq\left(\frac{1}{n_1} + \frac{1}{n_2}\right)}}$$

To compute a value of Z, however, we must estimate the parameters p and q that appear in the radical. Upon pooling the data from both samples, the pooled estimate of the proportion p is

$$\hat{p} = \frac{x_1 + x_2}{n_1 + n_2}$$

where x_1 and $x_2$are the numbers of successes in each of the two samples. Substituting $\hat{p}$ for p and $\hat{q} = 1 - \hat{p}$ for q, the z-value for testing $p_1 = p_2$ is determined from the formula

$$z = \frac{\hat{p}_1 - \hat{p}_2}{\sqrt{\hat{p}\hat{q}\left(\frac{1}{n_1} + \frac{1}{n_2}\right)}}$$

The critical regions for the appropriate alternative hypotheses are set up as before, using critical points of the standard normal curve. Hence, for the alternative $p_1 \neq p_2$ at the α-level of significance, the critical region is $z < -z_{\alpha/2}$ or $z > z_{\alpha/2}$. For a test where the alternative is $p_1 < p_2$, the critical region is $z < -z_\alpha$, and when the alternative is $p_1 > p_2$, the critical region is $z > z_\alpha$.

xiv. Compute Right-Tailed Hypothesis Testing on Two Proportions with SymIntegration

A vote is to be taken among the residents of a town and the surrounding country determine whether a proposed chemical plant should be constructed. the construction site is within the town limits, and for this reason many voters in the county believe that the proposal will pass because of the large proportion of the town voters who favor the construction. To determine if there is a significant difference in the proportions of town voters and county voters favoring the proposal, a poll is taken. If 120 of 200 town voters favor the proposal and 240 of 500 county residents favor it, would you agree that the proportion of town voters favoring the proposal is higher than the propotion of county voters? Use an $\alpha = 0.05$ level fo significance.

Solution:

Let p_1 and p_2 be the true proportions of voters in the town and county, respectively, favoring the proposal.

We construct the right-tailed hypothesis test as

$$H_0 : p_1 = p_2$$
$$H_1 : p_1 > p_2$$

with $\alpha = 0.05$. The critical region can be computed by finding the inverse of the cdf of Z

$$P(Z > z_\alpha) = 0.05$$

thus, we will obtain

$$z_\alpha = 1.64485$$

so the critical region is when

$$z > 1.64485$$

Now for further computations

$$\hat{p}_1 = \frac{x_1}{n_1} = \frac{120}{200} = 0.60$$
$$\hat{p}_2 = \frac{x_2}{n_2} = \frac{240}{500} = 0.48$$
$$\hat{p} = \frac{x_1 + x_2}{n_1 + n_2} = \frac{120 + 240}{200 + 500} = 0.51$$

Therefore,

$$z_{\text{computed}} = \frac{\hat{p}_1 - \hat{p}_2}{\sqrt{\hat{p}\hat{q}\left(\frac{1}{n_1} + \frac{1}{n_2}\right)}} = \frac{0.60 - 0.48}{\sqrt{(0.51)(0.49)\left(\frac{1}{200} + \frac{1}{500}\right)}} = 2.86972$$

The P-value is

$$P = P(Z > 2.86972) = 0.0020$$

Decision: Reject $H_0$and agree that the proportion of town voters favoring the proposal is higher than the proportion of county voters.

In SymIntegration we have some functions to compute the hypothesis test for tests on two proportions, for this case we will use this function:
hypothesistest_twoproportions_righttailed(vector<string> data1, vector<string> data2, const string &inputString, double α)

The textfile **town.txt** will be **data1**, textfile **county.txt** will be **data2** the **inputString** is the proportion for the string (e.g. FAVOR, SWEDEN, CUPULU, PLUTUTUT, etc) that we want to compute in the **data1** and **data2**, and α is the level of significance.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Hypothesis Testing on Two Proportions ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Hypothesis Testing on Two Proportions ]# ./main

*********************************************************

Right-tailed hypothesis testing on two proportions

H0: p1 = p2
H1: p1 > p2
Data 1 size =  200, Data 2 size =  500
Amount of FAVOR in the data 1 = 120
Amount of FAVOR in the data 2 = 240
Proportion for FAVOR
p1 = 0.6, q1 = 0.4
p2 = 0.48, q2 = 0.52
pooled estimate of p = 0.514286, pooled estimate of q = 0.485714
Critical region : z > 1.64485
Computed z : 2.86972

*********************************************************

*Reject the null hypothesis when the computed z is greater than 1.64485

P-value: 0.00205418

*Reject null hypothesis if P-value <= 0.05 , we fail to reject the null hypothesis if P-value > 0.05

*********************************************************

Time taken by function: 1575 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Hypothesis Testing on Two Proportions ]#
```

Figure 4.38: *The computation of two-tailed hypothesis testing on two proportions (**SymIntegration/Examples/Statistics/Test SymIntegration Hypothesis Testing on Two Proportions/main.cpp**).*

xv. One- and Two Sample Tests Concerning Variances

[SI*] Applications of one- and two-sample tests on variances are certainly not difficult to motivate. Engineers and scientists are confronted with studies in which they are required to demonstrate that measurements involving products or processes adhere to specifications set by consumers. The specifications are often met if the process variance is sufficiently small. Attention is also focused on comparative experiments between methods or processes, where inherent reproducibility or variability must formally be compared. In addition to determine if the equal variance assumption is violated, a test comparing two variances is often applied prior to conducting a t-test on two means.

[SI*] Let us first consider the problem of testing the null hypothesis H_0 that the population variance σ^2 equals a specified value σ_0^2 against one of the usual alternatives $\sigma^2 < \sigma_0^2$, $\sigma^2 > \sigma_0^2$, or $\sigma^2 \neq \sigma_0^2$. The appropriate statistic on which to base our decision is the chi-squared statistic, which was used to construct a confidence interval for σ^2. Therefore, if we assume that the distribution of the population being sampled is normal, the chi-squared value for testing $\sigma^2 = \sigma_0^2$ is given by

$$\chi^2 = \frac{(n-1)s^2}{\sigma_0^2}$$

where n is the sample size, s^2 is the sample variance, and σ_0^2 is the value of σ^2 given by the null hypothesis. If H_0 is true, χ^2 is a value of the chi-squared distribution with $\nu = n - 1$ degrees of freedom. Hence, for a two-tailed test at the α-level of significance, the critical region is $\chi^2 < \chi^2_{1-\frac{\alpha}{2}}$ or $\chi^2 > \chi^2_{\frac{\alpha}{2}}$. For the one-sided alternative $\sigma^2 < \sigma_0^2$, the critical region is $\chi^2 < \chi^2_{1-\alpha}$, and for the one-sided alternative $\sigma^2 > \sigma_0^2$, the critical region is

$$\chi^2 > \chi^2_{\alpha}$$

.

[SI*] **Robustness of χ^2-Test to Assumption to Normality**
Various tests depend, at least theoretically, on the assumption of normality. In general, many procedures in applied statistics have theoretical underpinnings that depend on the normal distribution. These procedures vary in the degree of their dependency on the assumption of normality.

A procedure that is reasonably insensitive to the assumption is called a robust procedure (i.e., robust to normality). The χ^2-test on a single variance is very nonrobust to normality (i.e., the practical success of the procedure depends on normality). As a result, the P-value computed may be appreciably different from the actual P-value if the population sampled is not normal. Indeed, it is quite feasible that a statistically significant P-value may not truly signal $H_1 : \sigma \neq \sigma_0$; rather, a significant value may be a result of the violation of the normality assumptions. Therefore, the analyst should approach the use of this particular χ^2-test with caution.

[SI*] Now let us consider the problem of testing the equality of the variances σ_1^2 and σ_2^2 of two populations. That is, we shall test the null hypothesis H_0 that $\sigma_1^2 = \sigma_2^2$ against one of the usual alternatives

$$\sigma_1^2 < \sigma_2^2$$
$$\sigma_1^2 > \sigma_2^2$$
$$\sigma_1^2 \neq \sigma_2^2$$

For independent random samples of sizes n_1 and n_2, respectively, from the two populations, the f-value for testing $\sigma_1^2 = \sigma_2^2$ is the ratio

$$f = \frac{s_1^2}{s_2^2}$$

where s_1^2 and s_2^2 are the variances computed from the two samples. If the two populations are approximately normally distributed and the null hypothesis is true, the ratio $f = \frac{s_1^2}{s_2^2}$ is a value of the F-distribution with $\nu = n_1 - 1$ and $\nu_2 = n_2 - 1$ degrees of freedom. Therefore, the critical regions of size α corresponding to the one-sided alternatives $\sigma_1^2 < \sigma_2^2$ and $\sigma_1^2 > \sigma_2^2$ are, respectively, $f < f_{1-\alpha}(\nu_1, \nu_2)$ and $f > f_\alpha(\nu_1, \nu_2)$. For the two-sided alternative $\sigma_1^2 \neq \sigma_2^2$, the critical region is $f < f_{1-\frac{\alpha}{2}}(\nu_1, \nu_2)$ or $f > f_{\frac{\alpha}{2}}(\nu_1, \nu_2)$.

xvi. Compute Right-Tailed Hypothesis Testing on One-Sample Variance with SymIntegration

A manufacturer of car batteries claims that the life of the company's batteries is approximately normally distributed with a standard deviation equal to 0.9 year. If a random sample of 10 of these batteries has a standard deviation of 1.2 years, do you think that $\sigma > 0.9$ year? Use a 0.05 level of significance.

Solution:

We construct the right-tailed hypothesis test as

$$\sigma^2 = 0.81$$
$$\sigma^2 > 0.81$$

with $\alpha = 0.05$. The critical region can be computed by finding the inverse of the cdf of chi-squared distribution

$$P(\chi^2 > \chi^2_\alpha) = 0.05$$

thus, we will obtain

$$\chi^2_\alpha = 16.919$$

so the critical region is when

$$\chi^2 > 16.919$$

Now to compute χ^2_{computed}

$$\chi^2_{\text{computed}} = \frac{(n-1)s^2}{\sigma_0^2} = \frac{(9)(1.44)}{0.81} = 16.0077$$

The P-value is

$$P(\chi^2_\alpha > 16.0) = 0.06672$$

Decision: The χ^2 statistic is not significant at the 0.05 level. However, based on the P-value 0.06672, there is evidence that $\sigma > 0.9$.

In SymIntegration for one-sample right-tailed hypothesis test concerning variance we will use this function:
hypothesistest_onesamplevariance_righttailed(vector<double> data, double σ_0, double α)

The textfile **battery.txt** will be **data**, $\sigma_0 = 0.9$, and $\alpha = 0.05$ is the level of significance.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Hypothesis Testing on One-Sample Variance ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Hypothesis Testing on One-Sample Variance ]# ./main

*********************************************************

Right-tailed hypothesis testing on one-sample variance

H0: σ^{2} = 0.81
H1: σ^{2} > 0.81
Mean data = 2.475, s^{2} = 1.44069, n =  10
Critical region : χ^{2} > 16.919
Computed χ^{2} : 16.0077

*********************************************************

*Reject the null hypothesis when the computed χ^{2} is greater than 16.919

P-value: 0.0667207

*Reject null hypothesis if P-value <= 0.05 , we fail to reject the null hypothesis if P-value > 0.05

*********************************************************

Time taken by function: 1585 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Hypothesis Testing on One-Sample Variance ]#
```

Figure 4.39: *The computation for one-sample right-tailed hypothesis test concerning variance (**SymIntegration/Examples/Statistics/Test SymIntegration Hypothesis Testing on One-Sample Variance/main.cpp**).*

xvii. Compute Two-Tailed Hypothesis Testing on Two-Samples Variances with SymIntegration

In testing for the difference in the abrasive wear of the two materials, we assumed that the two unknown population variances were equal. Were we justified in making this assumption? Use a 0.10 level of significance.

Solution:

We construct the right-tailed hypothesis test as

$$H_0 : \sigma_1^2 = \sigma_2^2$$
$$H_1 : \sigma_1^2 \neq \sigma_2^2$$

with $\alpha = 0.10$. The critical region can be computed by finding the inverse of the cdf of F distribution

$$P(F > f_{\alpha/2}(\nu_1, \nu_2)) = 0.05$$
$$P(F < f_{1-\frac{\alpha}{2}}(\nu_1, \nu_2)) = 0.05$$

thus we will obtain

$$f_{\alpha/2}(\nu_1, \nu_2) = f_{0.05}(11, 9) = 3.10249$$
$$f_{1-\frac{\alpha}{2}}(\nu_1, \nu_2) = f_{0.95}(11, 9) = 0.345277$$

Therefore, the null hypothesis is rejected when $f < 0.34$ or $f > 3.10$, where $f = \frac{s_1^2}{s_2^2}$ with $\nu_1 = 11$ and $\nu_2 = 9$ degrees of freedom.

The computations for F statistic is

$$f = \frac{s_1^2}{s_2^2} = \frac{16}{25} = 0.64$$

Decision: Do not reject H_0. Conclude that there is insufficient evidence that the variances differ.

In SymIntegration for two-samples two-tailed hypothesis test concerning variances we will use this function:
hypothesistest_twosamplesvariances_twotailed(vector<double> data1, vector<double> data2, double α)

The textfile **Vectorx.txt** will be **data1**, the textfile **Vectory.txt** will be **data2**, and $\alpha = 0.1$ is the level of significance.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Hypothesis Testing on Two-Samples Variances ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Hypothesis Testing on Two-Samples Variances ]# ./main

*********************************************************

Two-tailed hypothesis testing on two-samples variances

H0: σ1^{2} = σ2^{2}
H1: σ1^{2} != σ2^{2}
Mean data 1 = 85.0417, s1^{2} = 16.1117, n1 =  12
Mean data 2 = 81.07, s2^{2} = 25.3379, n2 =  10
Critical region : f > 3.10249 and f < 0.345277
Computed f : 0.635875

*********************************************************

*Reject the null hypothesis when the computed f is greater than 3.10249 or less than 0.345277

P-value: 0.472498

*Reject null hypothesis if P-value <= 0.1 , we fail to reject the null hypothesis if P-value > 0.1

*********************************************************

Time taken by function: 935 microseconds
```

Figure 4.40: *The computation for two-samples two-tailed hypothesis test concerning variances (**SymIntegration/Examples/Statistics/Test SymIntegration Hypothesis Testing on Two-Samples Variances/main.cpp**).*

xviii. Goodness-of-Fit Test

[SI*] We have been concerned with the testing of statistical hypotheses about single population parameters such as μ, σ^2, and p. Now we shall consider a test to determine if a population has a specified theoretical distribution.

xix. Test for Independence (Categorical Data)

[SI*] The chi-squared test can also be used to test the hypothesis of independence of two variables of classification.

A contingency table with r rows and c columns is referred to as an $r \times c$ table. The row and column totals are called marginal frequencies.

Definition 4.63: Test for Independence

Calculate

$$\chi^2 = \sum_i \frac{(o_i - e_i)^2}{e_i}$$

where the summation extends over all rc cells in the $r \times c$ contingency table.

If $\chi^2 > \chi^2_\alpha$ with $\nu = (r-1)(c-1)$ degrees of freedom, reject the null hypothesis of independence at the α-level of significance; otherwise, we fail to reject the null hypothesis.

[SI*] It is important that the statistic on which we base our decision has a distribution that is only approximated by the chi-squared distribution. The computed χ^2-values depend on the cell frequencies and consequently are discrete. The continuous chi-squared distribution seems to approximate the discrete sampling distribution of χ^2 very well, provided that the number of degrees of freedom is greater than 1. In a 2×2 contigency table, where we have only 1 degree of freedom, a correction called Yates' correction for continuity is applied. The corrected formula then becomes

$$\chi^2(\text{corrected}) = \sum_i \frac{(|o_i - e_i| - 0.5)^2}{e_i}$$

If the expected cell frequencies are large, the corrected and uncorrected results are almost the same. When the expected frequencies are between 5 and 10, Yates' correction should be applied. For expected frequencies less than 5, the Fisher-Irwin exact test should be used. The Fisher-Irwin test may be avoided, however, by choosing a larger sample.

xx. Compute Test for Independence for Categorical Data with SymIntegration

Suppose that we wish to determine whether the opinions of the are independent of their levels of income. Members of a random sample of 1000 registered voters from the state of Illinois are classified as to whether they are in a low, medium, or high income bracket and whether or not the favor the tax reform.

	Income Level			
Tax Reform	**Low**	**Medium**	**High**	**Total**
For	182	213	203	598
Against	154	138	110	402
Total	336	351	313	1000

Table 4.7: *A random sample of 1000 registered voters from the state of Illinois are classified based on their income level and whether or not they favor the tax reform.*

Our decision to accept or reject the null hypothesis, H_0, of independence between a voter's opinion concerning the tax reform and his or her level of income is based upon how good a fit we have between the observed frequencies in each of the 6 cells of the Table and the frequenciesthat we would expect for each cell under the assumption that H_0 is true. To find these expected frequencies, elt us define the following events:
L : A person selected is in the low-income level.
M : A person selected is in the medium-income level.
H : A person selected is in the high-income level.
F : A person selected is for the tax reform.
A : A person selected is against the tax reform.

By using the marginal frequencies, we can list the following probability estimates

$$P(L) = \frac{336}{1000}, \quad P(M) = \frac{351}{1000}, \quad P(H) = \frac{313}{1000}$$
$$P(F) = \frac{598}{1000}, \quad P(A) = \frac{402}{1000}$$

The expected frequencies are obtained by multiplying each cell probability by the total number of observations. We round these frequencies to one decimal. Thus, the expected number of low-income voters in our sample who favor the tax reform is estimated to be

$$P(L \cap F) \times 1000 = P(L)P(F) \times 1000 = \frac{(336)(598)}{1000} = 200.9$$

when H_0 is true. The general rule for obtaining the expected frequency of any cell is given by the following formula;

$$\text{expected frequency} = \frac{(\text{column total}) \times (\text{row total})}{\text{grand total}}$$

We will find that

$$\begin{aligned}\chi^2 &= \frac{(182-200.9)^2}{200.9} + \frac{(213-209.9)^2}{209.9} + \frac{(203-187.2)^2}{187.2} \\ &+ \frac{(154-135.1)^2}{135.1} + \frac{(138-141.1)^2}{141.1} + \frac{(110-125.8)^2}{125.8} \\ &= 7.878212\end{aligned}$$

with $\alpha = 0.05$ we will have

$$\chi^2_{0.05} = 5.991464$$

with $\nu = (r-1)(c-1) = (2-1)(3-1) = 2$. If $\chi^2 > \chi^2_{0.05}$ then reject the null hypothesis; otherwise, we fail to reject the null hypothesis.

Another way is to use P-value, which can be obtained by

$$P = P(\chi^2 > 7.878212) = 0.019466$$

In this case, the null hypothesis is rejected ($\chi^2 > \chi^2_{0.05}$ and P-value is less than $\alpha = 0.5$)and we conclude that a voter's opinion concerning the tax reform and his or her level of income are not independent.

In SymIntegration, to compute test for independence for categorical data we can use this function:
hypothesistest_categoricaldata(vector<vector<double>> data, double α)

The textfile **Voters.txt** will be **data** in matrix form, and $\alpha = 0.05$ is the level of significance.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Hypothesis Testing for Independece with Categorical Data ]# ./main
sum of column 0 = 336
sum of column 1 = 351
sum of column 2 = 313
sum of row 0 = 598
sum of row 1 = 402

*********************************************************

Hypothesis testing on categorical data

H0: Independent
H1: Not independent
Number of rows =  2, number of columns = 3
Degrees of freedom =  2

Observed Frequencies:
            182.000000            213.000000            203.000000
            154.000000            138.000000            110.000000

Expected Frequencies:
            200.928000            209.898000            187.174000
            135.072000            141.102000            125.826000
Critical region : χ^{2} > 5.991464
Computed χ^{2} : 7.878212

*********************************************************

*Reject the null hypothesis when the computed χ^{2} is greater than 5.991464

P-value: 0.019466

*Reject null hypothesis if P-value <= 0.050000 , we fail to reject the null hypothesis if P-value > 0.050000

*********************************************************

Time taken by function: 1748 microseconds
```

Figure 4.41: *The computation for test for independence for categorical data (**SymIntegration/Examples/Statistics/Test SymIntegration Hypothesis Testing for Independence with Categorical Data/main.cpp**).*

xxi. Compute Two-Sample Case Study using *t*- and *F*- Tests with the Box-and-Whisker and Quantile-Quantile Plot with SymIntegration and Hamzstplot

In a data analysis study conducted by personnel at the Statistics Consulting Center at Virginia Tech, two different materials, alloy A and alloy B, were compared in terms of breaking strength. Alloy B is more expensive, but it should certainly be adopted if it can be shown to be stronger than alloy A. The consistency of performance of the two alloys should also be taken into account. Random samples of beams made from each alloy were selected, and strength was measured in units of 0.001-inch deflection as a fixed force was applied at both ends of the beam. Twenty specimens were used for each of the two alloys you can see it in **alloyA.txt** and **alloyB.txt** for this case example working directory.

Solution:

It is important that the engineer compare the two alloys. Of concern is average strength and reproducibility. It is of interest to determine if there is a severe violation of the normality assumption required of both the *t*- and *F*-tests.

We will construct the hypothesis test :

$$H_0 : \mu_A = \mu_B$$
$$H_1 : \mu_A > \mu_B$$

we will use the *t*-test to confirm whether alloy A and alloy B has the same average strength or alloy B has greater average strength than alloy A.

The sample means and standar deviations are

$$\overline{x}_A = 83.55$$
$$s_A = 3.663$$
$$\overline{x}_B = 79.70$$
$$s_B = 3.097$$

With $\alpha = 0.05$. Now, we will compute

$$t_{\text{computed}} = \frac{(\overline{x}_A - \overline{x}_B) - d_0}{\sqrt{\frac{s_A^2}{n_A} + \frac{s_B^2}{n_B}}} = 3.58953$$

The *P*-value is

$$P = P(T > 3.58953) = 0.0004897$$

Afterwards, we will construct another hypothesis test for the *F*-test:

$$H_0 : \sigma_A^2 = \sigma_B^2$$
$$H_1 : \sigma_A^2 \neq \sigma_B^2$$

we will have

$$f_{\text{computed}} = \frac{s_A^2}{s_B^2} = 1.39929$$

The P-value can be computed with

$$P = 2P(F > 1.39929) = 2(1 - P(F < 1.39929)) = 0.4709$$

Decision: Rejects H_0 in favor of H_1 due to the P-value from t-test is smaller than α ($0.0004897 < 0.05$), it means that the strength of alloy A is greater than alloy B. Then for the second test, the F-test, we fail to reject H_0 as the P-value is greater than α ($0.4709 > 0.05$), thus it means that the variances of alloy A and alloy B are different.

In SymIntegration, to help for the t-test and F-test for this example case we can use these two functions:
hypothesistest_unequalunknownvariances_righttailed(vector<double> data1, vector<double> data2, double d_0, double α, double δ)

hypothesistest_twosamplesvariances_twotailed(vector<double> data1, vector<double> data2, double α)

The textfile **alloyA.txt** will be **data1**, textfile **alloyB.txt** will be **data2**, $d_0 = 0$, $\delta = 1$ (to compute β and the power of the test), and $\alpha = 0.05$ is the level of significance.

To plot the box-and-whisker or the normal quantile-quantile plot we can use **Hamzstplot** and **SymIntegration**, we join the textfile into one **alloy.txt** that has the first 20 data that represents alloy A and the next 20 data represents data for alloy B, we also create a textfile **alloyname.txt** to name the box-and-whisker plot for each alloy.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Hamzstplot 1.51/Examples with Makefile/Statistics/Plot Box-and-Whisker Plots for Two Alloys ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lhamzstplot -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Hamzstplot 1.51/Examples with Makefile/Statistics/Plot Box-and-Whisker Plots for Two Alloys ]# ./main

*********************************************************

Right-tailed hypothesis testing, unequal and unknown variances

H0: μ1 - μ2 = 0
H1: μ1 - μ2 > 0

Mean data 1 = 83.55, s1 = 3.66312, n1 = 20
Mean data 2 = 79.7, s2 = 3.09669, n2 = 20

The probability of a type I error (alpha): 0.05
The probability of a type II error (beta): 0.227298
Power: 0.772702

Critical value: 1.27596
Critical region : t > 1.6883
Computed t : 3.58953

*Reject the null hypothesis when the computed t is greater than 1.6883

P-value: 0.000489718

*Reject null hypothesis if P-value <= 0.05 , we fail to reject the null hypothesis if P-value > 0.05

*********************************************************

*********************************************************

Right-tailed hypothesis testing on two-samples variances

H0: σ1^{2} = σ2^{2}
H1: σ1^{2} > σ2^{2}
Mean data 1 = 83.55, s1^{2} = 13.4184, n1 =  20
Mean data 2 = 79.7, s2^{2} = 9.58947, n2 =  20
Critical region : f > 2.16825
Computed f : 1.39929

*********************************************************

*Reject the null hypothesis when the computed f is greater than 2.16825

P-value: 0.235461

*Reject null hypothesis if P-value <= 0.05 , we fail to reject the null hypothesis if P-value > 0.05

*********************************************************
```

Figure 4.42: *The computation for both t- and F-test for the alloys data (**SymIntegration/Examples/Statistics/Test SymIntegration and Hamzstplot Proc t-Test and Plot Box-and-Whisker Plots for Two Alloys/-main.cpp**).*

```
*******************************************************

Two-tailed hypothesis testing on two-samples variances

H0: σ1^{2} = σ2^{2}
H1: σ1^{2} != σ2^{2}
Mean data 1 = 83.55, s1^{2} = 13.4184, n1 =  20
Mean data 2 = 79.7, s2^{2} = 9.58947, n2 =  20
Critical region : f > 2.52645 and f < 0.395812
Computed f : 1.39929

*******************************************************

*Reject the null hypothesis when the computed f is greater than 2.52645 or less than 0.395812

P-value: 0.470922

*Reject null hypothesis if P-value <= 0.05 , we fail to reject the null hypothesis if P-value > 0.05

*******************************************************
Press ENTER to continue...
```

Figure 4.43: *The computation for both t- and F-test for the alloys data (**SymIntegration/Examples/Statistics/Test SymIntegration and Hamzstplot Proc t-Test and Plot Box-and-Whisker Plots for Two Alloys/-main.cpp**).*

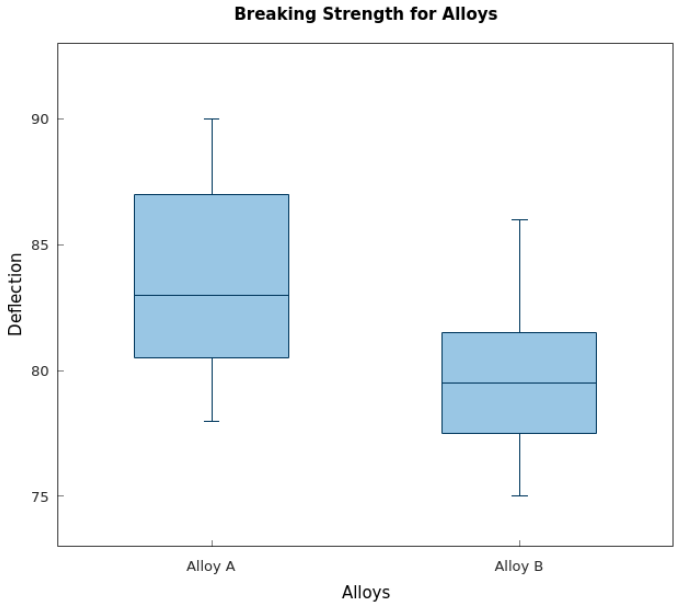

Figure 4.44: *The box-and-whisker plot for the alloys data (**SymIntegration/Examples/Statistics/Test SymIntegration and Hamzstplot Proc t-Test and Plot Box-and-Whisker Plots for Two Alloys/main.cpp**).*

For the quantile-quantile plot, we make use of this function in SymIntegration:
normalquantile(vector<double> data, *c***)**

With **data** is the sample data, e.g. alloy A or alloy B, the data is saved in textfile, and c is the scale to limit the x-axis range.

Basically, the normal quantile-quantile plot needs to follow this process:

1. Sort the **vector<double>** for the data, from lowest to highest, e.g. **std::sort(alloyA.begin(), alloyA.end())**.

2. Call the function to create a **vector<double>** of the normal quantile, e.g. **normalquantile(alloyA, 0.3)**, 0.3 is the scale to shrink the x-axis range / limit.

3. We will then have this sequence of 2-tuple:

$$(x_i, y_i) = (\text{data}_i - \overline{\text{data}}, \text{data}_i)$$

with $\overline{\text{data}}$ is the mean sample for alloy A or alloy B. We can then use the scatter plot from this quantile-quantile that have been generated : (x_i, y_i).

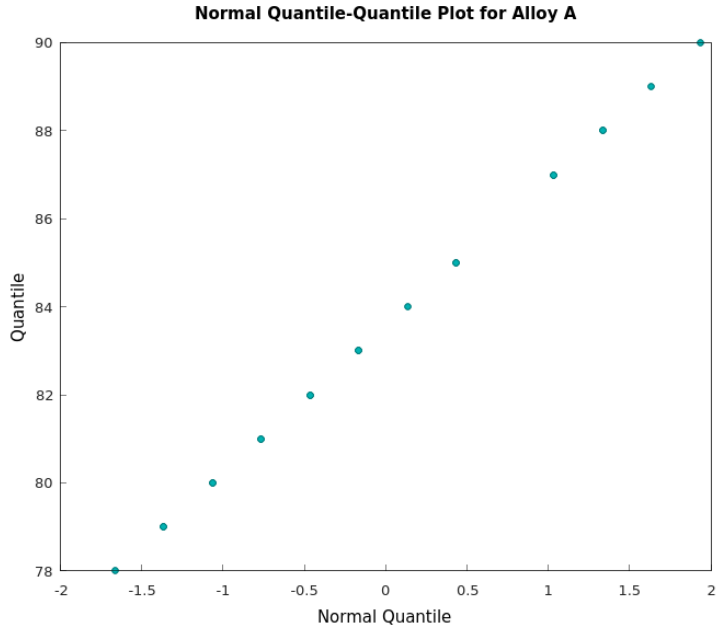

Figure 4.45: *The normal quantile-quantile plot of data for alloy A (**SymIntegration/Examples/Statistics/Test SymIntegration and Hamzstplot Plot 2D Normal Quantile-Quantile Plot/main.cpp**).*

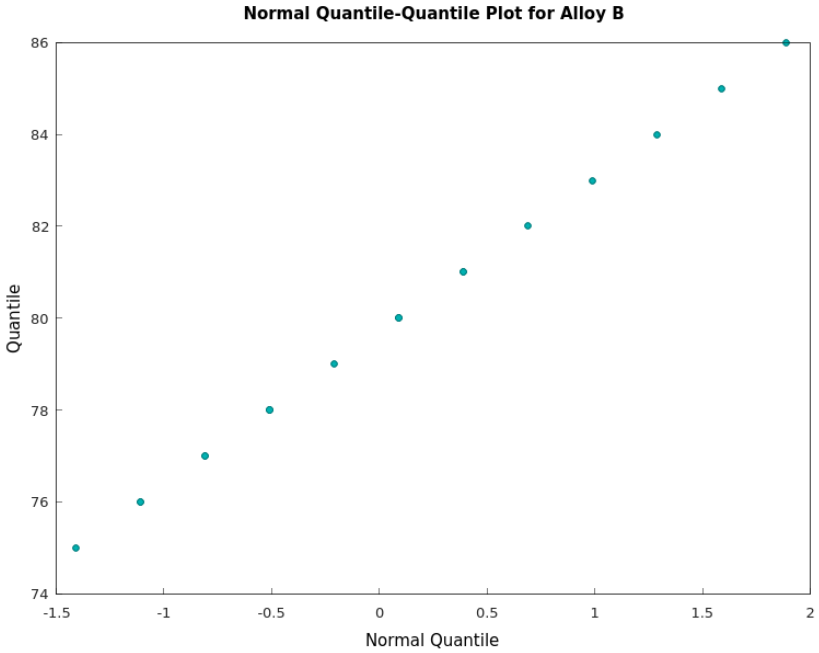

Figure 4.46: *The normal quantile-quantile plot of data for alloy A (**SymIntegration/Examples/Statistics/Test SymIntegration and Hamzstplot Plot 2D Normal Quantile-Quantile Plot/main.cpp**).*

xxii. One-Way Analysis of Variance (ANOVA)

[SI*] In the estimation and hypothesis testing section, we were restricted in each case to considering no more than two population parameters. Such was the case, for example, in testing for the equality of two population means using independent samples from normal populations with common but unknown variance, where it was necessary to obtain a pooled estimate of σ^2.

Suppose in an industrial experiment that an engineer is interested in how the mean absorption of moisture in concrete varies among 5 different concrete aggregates. The samples are exposed to moisture for 48 hours. It is decided that 6 samples are to be tested for each aggregate, requiring a total of 30 samples to be tested.

The model for this situation may be set up as follows. There are 6 observations taken from each of 5 populations with means $\mu_1, \mu_2, \ldots, \mu_5$, respectively. We may wish to test

$$H_0 : \mu_1 = \mu_2 = \cdots = \mu_5$$
$$H_1 : \text{At least two of the means are not equal}$$

In addition, we may be interested in making individual comparisons among these 5 population means.

In the analysis-of-variance procedure, it is assumed that whatever variation exists among the aggregate average is attributed to

1. Variation in absorption among observation within aggregate types
2. Variation among aggregate types, that is, due to differences in the chemical composition of the aggregates.

[SI*] Random samples of size n are selected from each of k populations. The k different populations are classified on the basis of a single criterion such as different treatments or groups. Today the term **treatment** is used generally to refer to the various classifications, whether they be different aggregates, different analysts, different fertilizers, or different regions of the country.

[SI*] **Assumption and Hypotheses in One-Way ANOVA**
It is assumed that the k populations are independent and normally distributed with means $\mu_1, \mu_2, \ldots, \mu_k$ and common variance σ^2. These assumptions are made more palatable by randomization. We wish to derive appropriate methods for testing the hypothesis

$$H_0 : \mu_1 = \mu_2 = \cdots = \mu_k$$
$$H_1 : \text{At least two of the means are not equal}$$

Let y_{ij} denote the j-th obsevation from the i-th treatment and arrange the data to become a table. Here, $Y_{i\cdot}$ is the total of all observations in the sample from the i-th treatment, $\bar{y}_i$ is the mean of all observations in the sample from the i-th treatment, $Y_{\cdot\cdot}$ is the total of all nk observations, and $\bar{y}_{\cdot\cdot}$ is the mean of all nk observations.

[SI*] **Model for One-Way ANOVA**
Each observation may be written in the form

$$Y_{ij} = \mu_i + \epsilon_{ij}$$

where ϵ_{ij} measures the deviation of the j-th observation of the i-th sample from the corresponding treatment mean. The ϵ_{ij}-term represents random error and plays the same role as the error terms in the regression models. An alternative preferred form of this equation is obtained by substituting $\mu_i = \mu + \alpha_i$, subject to the constraint

$$\sum_{i=1}^{k} \alpha_i = 0$$

Hence, we may write

$$Y_{ij} = \mu + \alpha_i + \epsilon_{ij}$$

where μ is just the grand mean of all the μ_i, that is,

$$\mu = \frac{1}{k} \sum_{i=1}^{k} \mu_i$$

and α_i is called the effect of the i-th treatment.

The null hypothesis that the k population means are equal against the alternative that at least two of the means are unequal may now be replaced by the equivalent hypothesis

$$H_0 : \alpha_1 = \alpha_2 = \cdots = \alpha_k$$
$$H_1 : \text{At least one of the } \alpha_i \text{ is not equal to zero}$$

[SI*] **Resolution of Total Variability into Components**
Our test will e based on a comparison of two independent estimates of the common population variance σ^2. These estiamtes will be obtained by partitioning the total variability of our data, designated by the double summation

$$\sum_{i=1}^{k} \sum_{j=1}^{n} (y_{ij} - \bar{y}_{..})^2$$

into two components.

Theorem 4.25: Sum of Squares Identity

The formula is

$$\sum_{i=1}^{k} \sum_{j=1}^{n} (y_{ij} - \bar{y}_{..})^2 = n \sum_{i=1}^{k} (\bar{y}_{i.} - \bar{y}_{..})^2 + \sum_{i=1}^{k} \sum_{j=1}^{n} (y_{ij} - \bar{y}_{i.})^2$$

Definition 4.64: Three Important Measures of Variability

It will be convenient in what follows to identify the terms of the sum-of-squares identity by the following notation:

SST (Total sum of squares)

$$\sum_{i=1}^{k}\sum_{j=1}^{n_i}(y_{ij}-\bar{y}_{..})^2$$

SSA (Treatment sum of squares)

$$n\sum_{i=1}^{k}(\bar{y}_{i.}-\bar{y}_{..})^2$$

SSE (Error sum of squares)

$$\sum_{i=1}^{k}\sum_{j=1}^{n_i}(y_{ij}-\bar{y}_{i.})^2$$

The sum-of-squares identity can then be represented symbolically by the equation

$$SST = SSA + SSE$$

The identity above expresses how between-treatment and within-treatment variation add to the total sum of squares.

Theorem 4.26: The Expected Value of SSA

The formula is

$$E(SSA) = (k-1)\sigma^2 + n\sum_{i=1}^{k}\alpha_i^2$$

Definition 4.65: Treatment Mean Square

If H_0 is true, an estimate of σ^2, based on $k-1$ degrees of freedom, is provided by this expression:

$$s_1^2 = \frac{SSA}{k-1}$$

If H_0 is true and thus each α_i in $E(SSA)$ is equal to zero, we see that

$$E\left(\frac{SSA}{k-1}\right) = \sigma^2$$

and s_1^2 is an unbiased estimate of σ^2. However, if H_1 is true, we have

$$E\left(\frac{SSA}{k-1}\right) = \sigma^2 + \frac{n}{k-1}\sum_{i=1}^{k}\alpha_i^2$$

and s_1^2 estimates σ^2 plus an additional term, which measures variation due to the systematic effects.

Definition 4.66: Error Mean Square

A second and independent estimate of σ^2, based on $k(n-1)$ degrees of freedom, is this familiar formula:

$$s^2 = \frac{SSE}{k(n-1)} \tag{4.114}$$

It is instructive to point out the importance of the expected values of the mean squares indicated above.

[SI*] **Use of F-Test in ANOVA**

The estimate s^2 is unbiased regardless of the truth or falsity of the null hypothesis. It is important to note that the sum-of-squares identity has partitioned not only the total variability of the data, but also the total number of degrees of freedom. That is,

$$nk - 1 = k - 1 + k(n-1)$$

[SI*] **F-Ratio for Testing Equality of Means**

When H_0 is true, the ratio $f = \frac{s_1^2}{s^2}$ is a value of the random variable F having the F-distribution with $k-1$ and $k(n-1)$ degrees of freedom. Since s_1^2 overestiamtes σ^2 when H_0 is false, we have a one-tailed test with the critical region entirely in the right tail of the distribution.

The null hypothesis H_0 is rejected at the α-level of significance when

$$f > f_\alpha[k-1, k(n-1)]$$

Another approach, the P-value approach, suggests that the evidence in favor or against H_0 is

$$P = P\{f[k-1, k(n-1)] > f\}$$

When H_1 is true, the presence of the condition $E(s_1^2) > E(s^2)$ suggests that the F-ratio be used in the context of a one-sided upper-tailed test. That is, we would expect the numerator

s_1^2 to exceed the denominator.

The computations for an analysis-of-variance problem are usually summarized in tabular form.

[SI*] Assumptions for one-way ANOVA:

1. The dependent variable is approximately normally distributed within each group. This assumption is more critical for smaller sample sizes.

2. The samples are selected at random and should be independent of one another.

3. All groups have equal standard deviations.

4. Each data point should belong to one and only one group / treatment. There should be no overlap or sharing of data points between groups / treatments.

[SI*] There are two main types of ANOVA:

1. **One-way ANOVA**
   This is the most basic form of ANOVA and is used when there is only one independent variable with more than two treatments or groups. It asses whether there are any statistically significant differences among the means of the groups.

2. **Two-way ANOVA**
   Extending the one-way ANOVA, two-way ANOVA involves two independent variables. It allows for examination of the main effects of each variable as well sa the interaction between them. The interaction effect explores whether the effect of one variable on the dependent variable is different depending on the level of the other variable.

Guide 4.3: How to Perform One-Way ANOVA

One-way ANOVA is a type of hypothesis test where only one factor is considered. We use F-statistic to perform a one-way analysis of variance.
The steps are:

1. Define the null and alternative hypothesis

$$H_0 : \mu_1 = \mu_2 = \cdots = \mu_k$$
$$H_1 : \text{At least two of the means are not equal}$$

2. Compute the degrees of freedom between and within the treatments / groups.

$$df_{\text{between}} = k - 1$$
$$df_{\text{within}} = N - k$$
$$df_{\text{total}} = N - 1$$

   with N is the number of samples in all treatments / groups combined, and k is the number of treatments / groups.

3. Compute the mean of all samples in each treatment / group, then compute the grand mean $(\mu_1, \ldots, \mu_k, \mu_{\text{grand}})$.

4. Compute the sum of squares total and sum of squares within, then compute the sum of squares between (SSA, SST, SSE).

5. Compute the variance between and within samples

$$s_1^2 = \frac{SSA}{k-1}$$
$$s^2 = \frac{SSE}{k(n-1)}$$

6. Compute the F value with this formula:

$$F_{\text{value}} = \frac{s_1^2}{s^2}$$

7. Compute the P value with this formula:

$$P_{\text{value}} = 1 - P(X \leq F_{\text{value}}) = 1 - F(F_{\text{value}}; k-1, N-k)$$

   with $F(F_{\text{value}}; k-1, N-k)$ is the cdf of F-distribution with numerator degree of freedom $k-1$ and denominator degree of freedom $N-k$.

8. If $F_{\text{value}} < P_{\text{value}}$ then we fail to reject the null hypothesis.

   If $F_{\text{value}} > P_{\text{value}}$ then we reject the null hypothesis.

The fundamental strategy of ANOVA is to systematically examine variability within treatments / groups being compared and also examine variability among the groups being compared.

xxiii. Compute One-Way ANOVA with SymIntegration

Suppose in an industrial experiment that an engineer is interested in how the mean absorption of moisture in concrete varies among 5 different concrete aggregates. The samples are exposed to moisture for 48 hours. It is decided that 6 samples are to be tested for each aggregate, requiring a total of 30 samples to be tested.

The model for this situation may be set up as follows. There are 6 observations taken from each of 5 populations with means $\mu_1, \mu_2, \ldots, \mu_5$, respectively.

Aggregate:	**1**	**2**	**3**	**4**	**5**	
	551	595	639	417	563	
	457	580	615	449	631	
	450	508	511	517	522	
	731	583	573	438	613	
	499	633	648	415	656	
	632	517	677	555	679	
Total	3320	3416	3663	2791	3664	16,854
Mean	553.33	569.33	610.50	465.17	610.67	561.80

Table 4.8: *The absorption of moisture in concrete aggregates.*

Test the hypothesis $\mu_1 = \mu_2 = \mu_3 = \mu_4 = \mu_5$ at the 0.05 level of significance for the data on absorption of moisture by various typpes of cement aggregates.

Solution:

The hypotheses are

$$\begin{aligned} H_0 &: \mu_1 = \mu_2 = \cdots = \mu_5 \\ H_1 &: \text{At least two of the means are not equal} \\ \alpha &= 0.05 \end{aligned}$$

Critical region :

$$f > 2.76$$

with $\nu_1 = 4$ and $\nu_2 = 25$ degrees of freedom. The sum-of-squares computations give

$$\begin{aligned} SST &= 209,377 \\ SSA &= 85,356 \\ SSE &= 209,377 - 85,356 = 124,021 \end{aligned}$$

Decision: Reject H_0 and conclude that the aggregates do not have the same mean absorption. The P-value for $f = 4.30$ is 0.0088, which is smaller than $\alpha = 0.05$.

This example has equal sample size from aggregate 1 to aggregate 5, however in real life, during experimental work, one often loses some of the desired observations. Experimental animals may die, experimental material may be damaged, or human subjects may drop out of a study. For analysis with unequal sample size, we just need to adjust the formula for SST, SSA and SSE.

In SymIntegration, the function to compute the one-way analysis-of-variance / ANOVA is: **ANOVA(vector<vector<double>>)**

The input is a matrix that is defined as **vector<vector<double>>** from the standard C++ library. It is a **void** function, so it returns only the table and nothing else.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration ANOVA ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration ANOVA ]# ./main

Matrix A :
       551.000000       595.000000       639.000000       417.000000       563.000000
       457.000000       580.000000       615.000000       449.000000       631.000000
       450.000000       508.000000       511.000000       517.000000       522.000000
       731.000000       583.000000       573.000000       438.000000       613.000000
       499.000000       633.000000       648.000000       415.000000       656.000000
       632.000000       517.000000       677.000000       555.000000       679.000000

One-Way ANOVA:

Source        DF                    SS                    MS                     F                     P

Model          4           85356.466667          21339.116667              4.301536              0.008752

Error         25          124020.333333           4960.813333

Total         29          209376.800000

R-Square             Grand Mean              Root MSE
0.407669             561.800000             70.433041

Time taken by function: 1417 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration ANOVA ]#
```

Figure 4.47: *The computation of one-way analysis-of-variance / ANOVA for the cement aggregates with SymIntegration, the execution time for the function is around* 1417 *microseconds without dependencies of using any external libraries besides SymIntegration, it is 1.76 times faster than using Boost. (**SymIntegration/Examples/Statistics/Test SymIntegration One-Way ANOVA/main.cpp**).*

We would like to explain why we tried to use **Boost** as comparison. The library **Boost** is very robust and solid, it includes all pdf and cdf computation for almost all distributions that are known today. So we tried to use **Boost** to compute the cdf for F-distribution to compute the P-value. It resulted in 2494 microseconds for the execution time of the ANOVA function. It becomes slower because we are using two libraries **Boost** and **SymIntegration**. If the reader wants to compare only using **Boost** and create the ANOVA table, they can compare the execution time, **Boost** only, **SymIntegration** only, and **Boost** + **SymIntegration** (this one will be slower than the previous two options).

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration ANOVA ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration ANOVA ]# ./main

Matrix A :
       551.000000       595.000000       639.000000       417.000000       563.000000
       457.000000       580.000000       615.000000       449.000000       631.000000
       450.000000       508.000000       511.000000       517.000000       522.000000
       731.000000       583.000000       573.000000       438.000000       613.000000
       499.000000       633.000000       648.000000       415.000000       656.000000
       632.000000       517.000000       677.000000       555.000000       679.000000

One-Way ANOVA:

Source        DF                    SS                    MS                    F                    P

Model         4           85356.466667          21339.116667              4.301536             0.008752

Error        25          124020.333333           4960.813333

Total        29          209376.800000

R-Square             Grand Mean              Root MSE
0.407669             561.800000             70.433041

Time taken by function: 2494 microseconds
```

Figure 4.48: *The computation of one-way analysis-of-variance / ANOVA for the cement aggregates with Boost and SymIntegration, the execution time for the function is around* 2494 *microseconds.*

```
...

double Fcdf(double x, double r1, double r2)
{
       //boost::math::fisher_f_distribution<> fisher_f(r1,r2);
       //double cdf_value = boost::math::cdf(fisher_f,x);

       /*The cumulative distribution function (CDF) for Fisher's F distribution
       It has at least 11 decimal digit precision with boost::math::
           fisher_f_distribution
       */

       double z ;
       double cdf_value;

       z = (x*r1)/(r2 + r1*x);
       cdf_value = incbeta(z,r1/2.0, r2/2.0);


       return cdf_value;
}

...
...

void ANOVA(vector<vector<double>> matrix)
{
       vector<vector<double>> anovamatrix;
```

```
      vector<double> total_column;
      vector<double> mean_column;
      vector<double> sst_column;
      vector<double> ssa_column;
      int C = matrix[0].size();
      int N = 0; // computing total data

      for(int i = 0 ; i < C ; ++i)
      {
            double sum = 0;
            int R = matrix.size();
            for (int j = 0 ; j < R ; ++j)
            {
                  sum += matrix[j][i]; // sum of the column

                  if(matrix[j][i] !=0 )
                  {
                        N = N+1;
                  }
                  else if(matrix[j][i] == 0 )
                  {
                        N = N;
                  }

            }
            total_column.push_back(sum);
            mean_column.push_back(sum/(R));
            //cout << "sum of column " << i << " = " << total_column[i] << endl;
            //cout << "mean sum of column " << i << " = " << mean_column[i] <<
                endl;

      }

...
...

      double SSE = SST-SSA;

      int df_model = C-1;
      int df_total = N-1;
      int df_error = df_total - df_model;

      double s1_square = SSA/(df_model);
      double s1 = SSE/(df_error);
      double computed_f = s1_square/s1;

      int r1 = df_model;
      int r2 = df_error;
```

```
        double p_value = 1 - Fcdf(computed_f,r1,r2);
        double r_square = 1 - SSE/SST;

        cout << "\nSource" << setw(10) << "DF" << setw(23) << "SS" << setw(23) << "
            MS" << setw(23) << "F" << setw(23) << "P" << endl;
        cout << "\nModel" << setw(10) << df_model << setw(23) << SSA << setw(23) <<
             s1_square << setw(23) << computed_f << setw(23) << p_value << endl;
        cout << "\nError" << setw(10) << df_error << setw(23) << SSE << setw(23) <<
             s1 << setw(23) << endl;
        cout << "\nTotal" << setw(10) << df_total << setw(23) << SST << endl;

        cout << "\nR-Square"<< setw(23) << "Grand Mean" << setw(23) << "Root MSE"
            << endl;
        cout << r_square << setw(23) << y_bar << setw(23) << sqrt(s1) << endl;

}
```

Code 63: *src/statistics.cpp*

To compute the cdf of F-distribution is not an easy task. We need to use numerical integration of the pdf of F-distribution, most people will use a well-tested library like **Boost** that provides high accuracy and handles potential numerical issues. We use manual implementation made by **Lewis Van Winkle** with zlib license, he created a C file that compute incomplete Beta function, that is very important in order to compute the cdf of F-distribution and student's t-distribution by continued fraction using Lentz's algorithm. The article can be read here:
https://codeplea.com/incomplete-beta-function-c

xxiv. Two-Way Analysis of Variance (ANOVA)

[SI*] Consider a situation where it is of interest to study the effects of two factors, A and B, on some response. For example, in a chemical experiment, we would like to vary simultaneously the reaction pressure and reaction time and study the effect of each on the yield. In a biological experiment, it is of interest to study the effects of drying time and temperature on the amount of solids (percent by weight) left in samples of yeast.

[SI*] Interaction and experimental error are separated in the two-factor experiment only if multiple observations are taken at the various treatment combinations. For maximum efficiency, there should be the same number n of observations at each combination. These should be true replications, not just repeated measurements.

[SI*] **Two-Factor Analysis of Variance**

To present general formulas for the analysis of variance of a two-factor experiment using repeated observations in a completely randomized design, we shall consider the case of n replications of the treatment combinations determined by a levels of factor A and b levels of factor B. The observations may be classified by means of a rectangular array where the rows represent the levels of factor A and the columns represent the levels of factor B. Each treatment combination defines a cell in our array. thus, we have ab cells, each cell containing n observations. Denoting the k-th observation taken at the i-th level of factor A and the j-th level of factor B by y_{ijk}. The observations in the (ij)-th cell constitute a random sample

	B					
A	**1**	**2**	**...**	**b**	**Total**	**Mean**
1	y_{111}	y_{121}	$\cdots$	y_{1b1}	$Y_{1..}$	$\overline{y}_{1..}$
	y_{111}	y_{122}	$\cdots$	y_{1b2}		
	$\vdots$	$\vdots$		$\vdots$		
	y_{11n}	y_{12n}	$\cdots$	y_{1bn}		
2	y_{211}	y_{221}	$\cdots$	y_{2b1}	$Y_{2..}$	$\overline{y}_{2..}$
	y_{212}	y_{222}	$\cdots$	y_{2b2}		
	$\vdots$	$\vdots$		$\vdots$		
	y_{21n}	y_{22n}	$\cdots$	y_{2bn}		
$\vdots$	$\vdots$	$\vdots$		$\vdots$	$\vdots$	$\vdots$
a	y_{a11}	y_{a21}	$\cdots$	y_{ab1}	$Y_{a..}$	$\overline{y}_{a..}$
	y_{a12}	y_{a22}	$\cdots$	y_{ab2}		
	$\vdots$	$\vdots$		$\vdots$		
	y_{a1n}	y_{a2n}	$\cdots$	y_{abn}		
Total	$Y_{.1.}$	$Y_{.2.}$	$\cdots$	$Y_{.b.}$	$Y_{...}$	
Mean	$\overline{y}_{.1.}$	$\overline{y}_{.2.}$	$\cdots$	$\overline{y}_{.b.}$		$\overline{y}_{...}$

Table 4.9: *two-Factor Experiment with n Replications.*

of size n from a population that is assumed to be normally distributed with mean μ_{ij} and variance σ^2. All ab populations are assumed to have the same variance σ^2.

Let us define the following useful symbols:
$Y_{ij.}$ = sum of the observations in the (ij)-th cell,
$Y_{i..}$ = sum of the observations for the i-th level of factor A,
$Y_{.j.}$ = sum of the observations for the j-th level of factor B,
$Y...$ = sum of all abn observations,
$\overline{y}_{ij.}$ = mean of the observations in the $(ij)th$ cell,
$\overline{y}_{i..}$ = mean of the observations for the $(i)th$ level of factor A,
$\overline{y}_{.j.}$ = mean of the observations for the $(j)th$ level of factor B,
$\overline{y}_{...}$ = mean of all abn observations.

Unlike in the one-factor situation, here we are assuming that the populations, where n independent identically distributed observations are taken, are combinations of factors. Also we will assume throughout that an equal number (n) of observations are taken at each factor combination.

[SI*] **Model and Hypotheses for the Two-Factor Problem**
Each observation may be written in the form

$$y_{ijk} = \mu_{ij} + \epsilon_{ijk}$$

where ϵ_{ijk} measures the deviations of the observed y_{ijk} values in the (ij)-th cell from the population mean μ_{ij}. If we let $(\alpha\beta)_{ij}$ denote the interaction effect of the i-th level of factor A and the j-th level of factor B, α_i the effect of the i-th level of factor A, β_j the effect of the j-th level of factor B, and μ the overall mean, we can write

$$\mu_{ij} = \mu + \alpha_i + \beta_j + (\alpha\beta)_{ij}$$

and then

$$y_{ijk} = \mu + \alpha_i + \beta_j + (\alpha\beta)_{ij} + \epsilon_{ijk}$$

on which we impose the restrictions

$$\sum_{i=1}^{a} a_i = 0, \quad \sum_{j=1}^{b} \beta_j = 0, \quad \sum_{i=1}^{a} (\alpha\beta)_{ij} = 0, \quad \sum_{j=1}^{b} (\alpha\beta)_{ij} = 0$$

The three hypotheses to be tested are as follows:

1. The first hypothesis

$$H_0' : \alpha_1 = \alpha_2 = \cdots = \alpha_a = 0$$
$$H_1' : \text{At least one of } \alpha_i \text{ is not equal to zero}$$

2. The second hypothesis

$$H_0'' : \beta_1 = \beta_2 = \cdots = \beta_b = 0$$
$$H_1'' : \text{At least one of } \beta_j \text{ is not equal to zero}$$

3. The third hypothesis

$$H_0''' : (\alpha\beta)_{11} = (\alpha\beta)_{12} = \cdots = (\alpha\beta)_{ab} = 0$$
$$H_1''' : \text{At least one of } (\alpha\beta)_{ij} \text{ is not equal to zero}$$

Theorem 4.27: Sum-of-Squares Identity

The formula is

$$\sum_{i=1}^{a}\sum_{j=1}^{b}\sum_{k=1}^{n}(y_{ijk}-\overline{y}_{...})^2 = bn\sum_{i=1}^{a}(\overline{y}_{i..}-\overline{y}_{...})^2 + an\sum_{j=1}^{b}(\overline{y}_{.j.}-\overline{y}_{...})^2 + n\sum_{i=1}^{a}\sum_{j=1}^{b}(\overline{y}_{ij.}-\overline{y}_{i..}-\overline{y}_{.j.}+\overline{y}_{...}) + \sum_{i=1}^{a}\sum_{j=1}^{b}\sum_{k=1}^{n}(y_{ijk}-\overline{y}_{ij.})^2$$

Symbolically we write the sum-of-squares identity as

$$SST = SSA + SSB + SS(AB) + SSE$$

where SSA and SSB are called the sums of squares for the main effects A and B, respectively, $SS(AB)$ is called the interaction sum of squares for A and B, and SSE is the error sum of squares. The degrees of freedom are partitioned according to the identity

$$abn-1 = (a-1)+(b-1)+(a-1)(b-1)+ab(n-1)$$

[SI*] **Formation of Mean Squares**

If we divide each of the sums of squares on the right side of the sum-of-squares identity by its corresponding number of degrees of freedom, we obtain the four statistics

$$S_1^2 = \frac{SSA}{a-1}$$
$$S_2^2 = \frac{SSB}{b-1}$$
$$S_3^2 = \frac{SS(AB)}{(a-1)(b-1)}$$
$$S^2 = \frac{SSE}{ab(n-1)}$$

All of these variance estimates are independent estimates of σ^2 under the condition that there are no effects α_i, β_j, and, of course, $(\alpha\beta)_{ij}$. If we interpret the sums of squares as functions of the independent random variables $y_{111}, y_{112}, \ldots, y_{abn}$, it is not difficult to verify that

$$E(S_1^2) = E\left[\frac{SSA}{a-1}\right] = \sigma^2 + \frac{nb}{a-1}\sum_{i=1}^{a}\alpha_i^2$$
$$E(S_2^2) = E\left[\frac{SSB}{b-1}\right] = \sigma^2 + \frac{na}{b-1}\sum_{j=1}^{b}\beta_j^2$$
$$E(S_3^2) = E\left[\frac{SS(AB)}{(a-1)(b-1)}\right] = \sigma^2 + \frac{n}{(a-1)(b-1)}\sum_{i=1}^{a}\sum_{j=1}^{b}(\alpha\beta)_{ij}^2$$
$$E(S^2) = E\left[\frac{SSE}{ab(n-1)}\right] = \sigma^2$$

from which we imemdiately observe that all four estimates of σ^2 are unbiased when H_0', H_0'', and H_0''' are true.

Definition 4.67: F-Test for Factor A

To test the hypothesis H_0', that the effects of factors A are all equal to zero, we compute the following ratio:

$$f_1 = \frac{s_1^2}{s^2}$$

which is a value of the random variable F_1 having the F-distribution with $a-1$ and $ab(n-1)$ degrees of freedom when H_0' is true. The null hypothesis is rejected at the α-level of significance when $f_1 > f_\alpha[a-1, ab(n-1)]$.

Definition 4.68: F-Test for Factor B

To test the hypothesis H_0'', that the effects of factors B are all equal to zero, we compute the following ratio:

$$f_2 = \frac{s_2^2}{s^2}$$

which is a value of the random variable F_2 having the F-distribution with $b-1$ and $ab(n-1)$ degrees of freedom when H_0'' is true. The null hypothesis is rejected at the α-level of significance when $f_2 > f_\alpha[b-1, ab(n-1)]$.

Definition 4.69: F-Test for Interaction

To test the hypothesis H_0''', that the interaction effects are all equal to zero, we compute the following ratio:

$$f_3 = \frac{s_3^2}{s^2}$$

which is a value of the random variable F_3 having the F-distribution with $(a-1)(b-1)$ and $ab(n-1)$ degrees of freedom when H_0''' is true. We conclude that, at the α-level of significance, interaction is present when $f_3 > f_\alpha[(a-1)(b-1), ab(n-1)]$.

Source of Variation	Sum of Squares	Degrees of Freedom	Mean Square	Computed f
A	SSA	$a-1$	$s_1^2 = \frac{SSA}{a-1}$	$f_1 = \frac{s_1^2}{s^2}$
B	SSB	$b-1$	$s_2^2 = \frac{SSB}{b-1}$	$f_2 = \frac{s_2^2}{s^2}$
AB	$SS(AB)$	$(a-1)(b-1)$	$s_3^2 = \frac{SS(AB)}{(a-1)(b-1)}$	$f_3 = \frac{s_3^2}{s^2}$
Error	SSE	$ab(n-1)$	$s^2 = \frac{SSE}{ab(n-1)}$	
Total	SST	$abn-1$		

Table 4.10: *Analysis of Variance for the Two-Factor Experiment with n Replications.*

xxv. Compute Two-Way ANOVA with SymIntegration

In an experiment conducted to determine which of 3 different missile systems is preferable, the propellant burning rate for 24 static firings was measured. Four different propellant types were used. The experiment yielded duplicate observations of burning rates at each combination of the treatments.

		Propellant Type		
Missile System	b_1	b_2	b_3	b_4
a_1	34.0	30.1	29.8	29.0
	32.7	32.8	26.7	28.9
a_2	32.0	30.2	28.7	27.6
	33.2	29.8	28.1	27.8
a_3	28.4	27.3	29.7	28.8
	29.3	28.9	27.3	29.1

Table 4.11: *Propellant Burning Rates.*

Test the following hypotheses:

(a) H_0' : there is no difference in the mean propellant burning rates when different missile systems are used.

(b) H_0'' : there is no difference in the mean propellant burning rates of the 4 propellant types.

(c) H_0''' : there is no interaction between the different missile systems and the different propellant types.

Solution:

(a) The first hypothesis

$$H_0' : \alpha_1 = \alpha_2 = \alpha_3 = 0$$
$$H_1' : \text{At least one of } \alpha_i \text{ is not equal to zero}$$

(b) The second hypothesis

$$H_0'' : \beta_1 = \beta_2 = \beta_3 = \beta_4 = 0$$
$$H_1'' : \text{At least one of } \beta_j \text{ is not equal to zero}$$

(c) The third hypothesis

$$H_0''' : (\alpha\beta)_{11} = (\alpha\beta)_{12} = \cdots = (\alpha\beta)_{34} = 0$$
$$H_1''' : \text{At least one of } (\alpha\beta)_{ij} \text{ is not equal to zero}$$

With the sum-of-squares formula to do the computation, we will have this resulting table for the analysis of variance:

Source of Variation	Sum of Squares	Degrees of Freedom	Mean Square	Computed f
Missile system	14.52	2	7.26	5.84
Propellant type	40.08	3	13.36	10.75
Interaction	22.16	6	3.69	2.97
Error	14.91	12	1.24	
Total	91.68	23		

Table 4.12: *Analysis of Variance for the Two-Factor Experiment Data.*

We can compute the "model" (with 11 degrees of freedom), "error", "total" analysis of variance before we test the system, type, and system by type interaction.

In SymIntegration to compute two-way analysis of variance (ANOVA) we can use this function:
twoway_ANOVA(vector<vector<double>> matrix, int a, int b)

The input is a matrix that is defined as **vector<vector<double>>** from the standard C++ library, then a is the number of levels of factor A (in this case $a = 3$), and b is the number of levels of factor B (in this case $b = 4$). It is a **void** function, so it returns only the table and nothing else.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Two-Way ANOVA ]# ./main

*********************************************************

Two-way table of averages
             33.350000             31.450000             28.250000             28.950000                  | 30.500000
             32.600000             30.000000             28.400000             27.700000                  | 29.675000
             28.850000             28.100000             28.500000             28.950000                  | 28.600000
-------------------------------------------------------------------------------------------------------------------------
             31.600000             29.850000             28.383333             28.533333
*********************************************************

Hypotheses for the Two-Factor Problem

H0': α_{1} = α_{2} = ... = α_{a} = 0
H1':  At least one of the α_{i} is not equal to zero

H0'': β_{1} = β_{2} = ... = β_{b} = 0
H1'':  At least one of the β_{j} is not equal to zero

H0''': (αβ)_{11} = (αβ)_{12} = ... = (αβ)_{ab} = 0
H1''':  At least one of the (αβ)_{ij} is not equal to zero

*********************************************************

Analysis of Variance

Source          DF              SS              MS              F               P
Model           11              76.768333       6.978939        5.616853        0.002986
Error           12              14.910000       1.242500
Total           23              91.678333

R-Square             Grand Mean              Root MSE
0.837366              29.591667              1.114675

Analysis of Variance for the Two-Factor Experiment with n replications

Source          DF              Type III SS             MS                  F                   P
A               2               14.523333               7.261667            5.844400            0.016898
B               3               40.081667               13.360556           10.752962           0.001020
A * B           6               22.163333               3.693889            2.972949            0.051168

Time taken by function: 1542 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Two-Way ANOVA ]#
```

Figure 4.49: *The computation of two-way analysis-of-variance / ANOVA for the propellant rate data. (**SymIntegration/Examples/Statistics/Test SymIntegration Two-Way ANOVA/main.cpp**).*

Learning from the two-way ANOVA table we can say that :

The F-test on the model ($P = 0.002986$) is testing the accumulation of the two main effects and the interaction.

(a) Reject H_0' and conclude that different missile systems result in different mean propellant burning rates. The P-value is approximately 0.0168978.

(b) Reject H_0'' and conclude that the mean propellant burning rates are not the same for the four propellant types. The P-value is approximately 0.0010.

(c) Interaction is barely insignificant at the 0.05 level, but the P-value of approximately 0.05116 would indicate that interaction must be taken seriously.

At this point we should draw some type of interpretation of the interaction. It should be emphasized that statistical significance of a main effect merely implies that marginal means are significantly different. However, if we take a look at the two-way table of averages it is apparent that more important information exists in the body of the table, trends that are inconsistent with the trend depicted by marginal averages. The table of averages certainly suggests that the effect of propellant type depends on the system being used. For example, for system 3 the propellant-type effect does not appear to be important, although it does have a large effect if either system 1 or system 2 is used. This explains the "significant" interaction between these two factors.

xxvi. Three-Way Analysis of Variance (ANOVA)

[SI*] We consider an experiment with three factors, A, B, and C, at a, b, and c levels, respectively, in a completely randomized experimental design. Assume again that we have n observations for each of the abc treatment combinations.

Definition 4.70: Model for the Three-Factor Experiment

The model for the three-factor experiment is

$$y_{ijkl} = \mu + \alpha_i + \beta_j + \gamma_k + (\alpha\beta)_{ij} + (\alpha\gamma)_{ik} + (\beta\gamma)_{jk} + (\alpha\beta\gamma)_{ijk} + \epsilon_{ijkl}$$

with $i = 1, 2, \ldots, a$; $j = 1, 2, \ldots, b$; $k = 1, 2, \ldots, c$; and $l = 1, 2, \ldots, n$, where α_i, β_j, and γ_k are the main effects and $(\alpha\beta)_{ij}, (\alpha\gamma)_{ik}$, and $(\beta\gamma)_{jk}$ are the two-factor interaction effects that have the same interpretation as in the two-factor experiment.

The term $(\alpha\beta\gamma)_{ijk}$ is called the three-factor interaction effect, a term that represents a nonadditivity of the $(\alpha\beta)_{ij}$ over the different levels of the factor C. The sum of all main effects is zero and the sum over any subscript of the two- and three-factor interaction effects is zero. In many experimental situations, these higher-order interactions are insignificant and their mean squares reflect only random variation, but we shall outline the analysis in its most general form.

Definition 4.71: Sum of Squares for a Three-Factor Experiment

The general philosophy concerning the analysis is the same as the one- and two-factor experiments. The sum of squares is partitioned into eight terms, each representing a source of variation from which we obtain independent estimates of σ^2 when all the main effects and interaction effects are zero. If the effects of any given factor or interaction are not all zero, then the mean square will estimate the error variance plus a component due to the systematic effect in question.

$$SSA = bcn \sum_{i=1}^{a} (\overline{y}_{i\ldots} - \overline{y}_{\ldots})^2$$

$$SSB = acn \sum_{j=1}^{b} (\overline{y}_{.j.} - \overline{y}_{\ldots})^2$$

$$SSC = abn \sum_{k=1}^{c} (\overline{y}_{..k} - \overline{y}_{\ldots})^2$$

$$SS(AB) = cn \sum_{i=1}^{a} \sum_{j=1}^{b} (\overline{y}_{ij.} - \overline{y}_{i\ldots} - \overline{y}_{.j.} + \overline{y}_{\ldots})^2$$

$$SS(AC) = bn \sum_{i=1}^{a} \sum_{k=1}^{c} (\overline{y}_{i.k} - \overline{y}_{i\ldots} - \overline{y}_{..k} + \overline{y}_{\ldots})^2$$

$$SS(BC) = an \sum_{j=1}^{b} \sum_{k=1}^{c} (\overline{y}_{.jk} - \overline{y}_{.j.} - \overline{y}_{..k} + \overline{y}_{\ldots})^2$$

$$SS(ABC) = n \sum_{i=1}^{a} \sum_{j=1}^{b} \sum_{k=1}^{c} (\overline{y}_{ijk} - \overline{y}_{ij.} - \overline{y}_{i.k} - \overline{y}_{.jk} + \overline{y}_{i\ldots} + \overline{y}_{.j.} + \overline{y}_{..k} - \overline{y}_{\ldots})^2$$

$$SST = \sum_{i=1}^{a} \sum_{j=1}^{b} \sum_{k=1}^{c} \sum_{l=1}^{n} (y_{ijkl} - \overline{y}_{\ldots})^2$$

$$SSE = \sum_{i=1}^{a} \sum_{j=1}^{b} \sum_{k=1}^{c} \sum_{l=1}^{n} (y_{ijkl} - \overline{y}_{ijk})^2$$

The averages in the formulas are defined as follows:
$\overline{y}_{\ldots}$ =average of all $abcn$ observations.
$\overline{y}_{i\ldots}$ =average of the observations for the i-th level of factor A.
$\overline{y}_{.j.}$ =average of the observations for the j-th level of factor B.
$\overline{y}_{..k}$ =average of the observations for the k-th level of factor C.
$\overline{y}_{ij.}$ =average of the observations for the i-th level of factor A and the j-th level of factor B.
$\overline{y}_{i.k}$ =average of the observations for the i-th level of factor A and the k-th level of factor C.
$\overline{y}_{.jk}$ =average of the observations for the j-th level of factor B and the kth level of factor C.
$\overline{y}_{ijk}$ =average of the observations for the (ijk)-th treatment combination.

Source of Variation	Sum of Squares	Degrees of Freedom	Mean Square	Computed f
A	SSA	$a-1$	$s_1^2 = \frac{SSA}{a-1}$	$f_1 = \frac{s_1^2}{s^2}$
B	SSB	$b-1$	$s_2^2 = \frac{SSB}{b-1}$	$f_2 = \frac{s_2^2}{s^2}$
C	SSC	$c-1$	$s_3^2 = \frac{SSC}{c-1}$	$f_3 = \frac{s_3^2}{s^2}$
AB	$SS(AB)$	$(a-1)(b-1)$	$s_4^2 = \frac{SS(AB)}{(a-1)(b-1)}$	$f_4 = \frac{s_4^2}{s^2}$
AC	$SS(AC)$	$(a-1)(c-1)$	$s_5^2 = \frac{SS(AC)}{(a-1)(c-1)}$	$f_5 = \frac{s_5^2}{s^2}$
BC	$SS(BC)$	$(b-1)(c-1)$	$s_6^2 = \frac{SS(BC)}{(b-1)(c-1)}$	$f_6 = \frac{s_6^2}{s^2}$
ABC	$SS(ABC)$	$(a-1)(b-1)(c-1)$	$s_7^2 = \frac{SS(ABC)}{(a-1)(b-1)(c-1)}$	$f_7 = \frac{s_7^2}{s^2}$
Error	SSE	$abc(n-1)$	$s^2 = \frac{SSE}{ab(n-1)}$	
Total	SST	$abcn-1$		

Table 4.13: *Analysis of Variance for the Three-Factor Experiment with n Replications.*

xxvii. Compute Three-Way ANOVA with SymIntegration

In the production of a particular material, three variables are of interest: A, the operator effect (three operators); B, the catalyst used in the experiment (three catalysts); and C, the washing time of the product following the cooling process (15 minutes and 20 minutes). Three runs were made at each combination of factors. It was felt that all interactions among the factors should be studied. Perform an analysis of variance to test for significant effects.

Solution:

The three-way ANOVA table computation result:
The ANOVA table shows that none of the interactions show a significant effect at the $\alpha = 0.05$ level.

To understand how we do the hypothesis testing with the P-value, we should take a look at:

1. The P-value for A is 0.000126, if we have chosen $\alpha = 0.05$, it means that we reject the null hypothesis that is saying

$$H_0' : \alpha_1 = \alpha_2 = \alpha_3 = 0$$
$$H_1' : \text{At least one of } \alpha_i \text{ is not equal to zero}$$

It means that the operator / factor A is significant in production of a particular material.

	Washing Time, C					
	15 Minutes			20 Minutes		
	Catalyst, B			Catalyst, B		
Operator, A	**1**	**2**	**3**	**1**	**2**	**3**
1	10.7	10.3	11.2	10.9	10.5	12.2
	10.8	10.2	11.6	12.1	11.1	11.7
	11.3	10.5	12.0	11.5	10.3	11.0
2	11.4	10.2	10.7	9.8	12.6	10.8
	11.8	10.9	10.5	11.3	7.5	10.2
	11.5	10.5	10.2	10.9	9.9	11.5
3	13.6	12.0	11.1	10.7	10.2	11.9
	14.1	11.6	11.0	11.7	11.5	11.6
	14.5	11.5	11.5	12.7	10.9	12.2

Table 4.14: *Data for production of a particular material with three factors.*

2. The P-value for B is 0.0010, if we have chosen $\alpha = 0.05$, it means that we reject the null hypothesis that is saying

$$H_0'' : \beta_1 = \beta_2 = \beta_3 = 0$$
$$H_1'' : \text{At least one of } \beta_j \text{ is not equal to zero}$$

   It means that the catalyst effect / factor B is significant in production of a particular material.

3. The P-value for C is 0.1686, if we have chosen $\alpha = 0.05$, it means that we do not reject the null hypothesis that is saying

$$H_0''' : \gamma_1 = \gamma_2 = 0$$
$$H_1''' : \text{At least one of } \gamma_k \text{ is not equal to zero}$$

   It means that the washing time / factor C is not significant in production of a particular material.

However, the P-value for BC is 0.0610; thus, it should not be ignored. The operator and catalyst effects are significant, while the effect of washing time is not significant.

Suppose we want to see the impact of interaction BC, about dealing with the effect that the interaction between catalyst and washing time is having on the test on the washing time main effect (factor C).

The BC interaction is significant at approximately 0.06 level. Suppose, however, we observe a two-way table of means between BC (the two-way table is provided in the result from SymIntegration' three-way ANOVA function).

It is clear why washing time was found not to be significant. A non-thorough analyst may get the impression that washing time can be eliminated from any future study which yield is being

Source of Variation	Sum of Squares	Degrees of Freedom	Mean Square	Computed f	P-Value
A	13.98	2	6.99	11.64	0.000126
B	10.18	2	5.09	8.48	0.00096
C	1.185	1	1.185	1.97	0.168585
AB	4.77	4	1.19	1.987	0.117196
AC	2.91	2	1.46	2.43	0.1027
BC	3.63	2	1.82	3.03	0.060982
ABC	4.91	4	1.23	2.04	0.1089
Error	21.613	36	0.60		
Total	63.19	53			

Table 4.15: *Analysis of Variance for the Three-Factor Experiment with $n = 3$ Replications.*

measured. However, it is obvious how the effect of washing time changes from a negative effect for the first catalyst to what appears to be a positive effect for the third catalyst. If we merely focus on the data for catalyst 1, a simple comparison between the means at the two washing times will produce a simple t-statistic:

$$t = \frac{12.19 - 11.29}{\sqrt{0.6\left(\frac{2}{9}\right)}} = 2.5$$

which is significant at a level less than 0.02. Thus, an important negative effect of washing time for catalyst 1 might very well be ignored if the analyst makes the incorrect broad interpretation of the insignificant F-ratio for washing time.

To analyze the three variables in the three-way ANOVA in order to create the source code to compute the ANOVA table, we should take a note that:

1. We start with the input as a square matrix with the number of rows, R, and the number of columns, C.

2. The factor A will be the main row, there will be R/a row for each factor $i = 1, 2, a$ that will represents factor A, in this case $a = 3$.

3. The factor B will be composed of b columns, since it is three-way then the total columns will be $b \times c$ columns each number of b columns corresponds to each factor C, in this case 1 column of factor C has 3 columns from factor B.

4. The total number of entries / data is $N = abcn$.

We will depends on a lot of **for** loop with linear algebra in the source code.

In SymIntegration to compute three-way analysis of variance (ANOVA) we can use this function:
threeway_ANOVA(vector<vector<double>> matrix, int a, int b, int c)

The input is a matrix that is defined as **vector<vector<double>>** from the standard C++ library, then a is the number of levels of factor A (in this case $a = 3$), b is the number of levels of factor

B (in this case $b = 3$), and c is the number of levels of factor C (in this case $c = 2$). It is a **void** function, so it returns only the table and nothing else.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Three-Way ANOVA ]# ./main

*******************************************************

Two-way table of averages, A (row) and B (column)
             11.216667             10.483333             11.616667                    | 11.105556
             11.116667             10.266667             10.650000                    | 10.677778
             12.883333             11.283333             11.550000                    | 11.905556
-------------------------------------------------------------------------------------------
             11.738889             10.677778             11.272222

*******************************************************

Two-way table of averages, A (row) and C (column)
             10.955556             11.255556                  | 11.105556
             10.855556             10.500000                  | 10.677778
             12.322222             11.488889                  | 11.905556
-------------------------------------------------------------------------------------------
             11.377778             11.081481

*******************************************************

Two-way table of averages, B (row) and C (column)
             12.188889             11.288889                  | 11.738889
             10.855556             10.500000                  | 10.677778
             11.088889             11.455556                  | 11.272222
-------------------------------------------------------------------------------------------
             11.377778             11.081481
*******************************************************
```

Figure 4.50: *The computation of three-way analysis-of-variance / ANOVA for the production of a particular material showing all the two-way tables from all possible combinations of* 2 *factors (**SymIntegration/Examples/Statistics/Test SymIntegration Three-Way ANOVA/main.cpp**).*

```
Hypotheses for the Three-Factor Problem

H0^{(1)}: α_{1} = α_{2} = ... = α_{a} = 0
H1^{(1)}:  At least one of the α_{i} is not equal to zero

H0^{(2)}: β_{1} = β_{2} = ... = β_{b} = 0
H1^{(2)}:  At least one of the β_{j} is not equal to zero

H0^{(3)}: γ_{1} = γ_{2} = ... = γ_{c} = 0
H1^{(3)}:  At least one of the γ_{k} is not equal to zero

H0^{(4)}: (αβ)_{11} = (αβ)_{12} = ... = (αβ)_{ab} = 0
H1^{(4)}:  At least one of the (αβ)_{ij} is not equal to zero

H0^{(5)}: (αγ)_{11} = (αγ)_{12} = ... = (αγ)_{ac} = 0
H1^{(5)}:  At least one of the (αγ)_{ik} is not equal to zero

H0^{(6)}: (βγ)_{11} = (βγ)_{12} = ... = (βγ)_{bc} = 0
H1^{(6)}:  At least one of the (βγ)_{jk} is not equal to zero

H0^{(7)}: (αβγ)_{111} = (αβγ)_{112} = ... = (αβγ)_{abc} = 0
H1^{(7)}:  At least one of the (αβγ)_{ijk} is not equal to zero

*******************************************************

Analysis of Variance

Source          DF              SS              MS              F               P
Model           17              41.579259       2.445839        4.073883        0.000194
Error           36              21.613333       0.600370
Total           53              63.192593

R-Square             Grand Mean               Root MSE
0.657977              11.229630               0.774836

Analysis of Variance for the Three-Factor Experiment with n replications

Source          DF              Type III SS             MS                      F                       P
A               2               13.982593               6.991296                11.644972               0.000126
B               2               10.182593               5.091296                8.480259                0.000960
C               1               1.185185                1.185185                1.974090                0.168585
A * B           4               4.774074                1.193519                1.987970                0.117196
A * C           2               2.913704                1.456852                2.426589                0.102655
B * C           2               3.633704                1.816852                3.026218                0.060982
A * B * C       4               4.907407                1.226852                2.043492                0.108896
Error           36              21.613333               0.600370
-------------------------------------------------------------
Total           53              63.192593

Time taken by function: 2697 microseconds
```

Figure 4.51: *The computation of three-way analysis-of-variance / ANOVA for the production of a particular material (**SymIntegration/Examples/Statistics/Test SymIntegration Three-Way ANOVA/main.cpp**).*

VII. Linear Regression and Correlation

i. Pearson's Correlation

[SI*] In Darrell Huff's book **How to Lie with Statistics**, he refers to a study that claimed that non-smoking college students gained better grades than students who smoked. The study concluded that 'smoking leads to poor college grades.' Huff questioned the findings of the study, arguing that instead of smoking being the cause of poor grades, perhaps 'low grades depressed students and caused them to smoke,' or maybe 'students with lower grades were more sociable and therefore more likely to smoke.' Measuring the relationship between two sets of data is a useful tool to establish how one variable changes with another, but it is limited; it does not tell you why this relationship exists. the data in the study clearly showed a relationship between smoking and poor grades, but it does not explain why this relationship exists.

Data that consists of measurements of two variables taken from each individual in a sample is called bivariate data; the relationship between the two variables is referred to as the correlation.

For bivariate data we try to classify one of two variables as 'independent' and the other as 'dependent.' The independent variable is the one that can be controlled by the person conducting the experiment or study; it is hypothesised to 'cause' some kind of effect to the dependent variable. the dependent variable is the variable that is just observed without being controlled, and is supposed to show the 'effect.'

[SI*] The line of best fit on a scatter diagram is drawn to give the best representation of the correlation between the two variables. The line of best fit is the one that has an approximately even spread of the data points either side of it.

[SI*] **Pearson's product moment correlation coefficient**
Pearson product moment correlation coefficient (PMCC) is a number, usually denoted by r, which can take any value between -1 and $+1$. Its sign indicates the type of correlation, and its magnitude (size) indicates the strength of correlation.

1. $r = +1$ means that there is perfect positive correlation
2. $r = 0$ means that there is no correlation
3. $r = -1$ means that there is perfect negative correlation

If $-0.5 < r < 0.5$, it is difficult to draw a line of best fit that has any meaning, because the data is just too scattered.

The formula to compute the PMCC is

$$r = \frac{S_{xy}}{S_x S_y} \tag{4.115}$$

where S_{xy} is the covariance of x and y, S_x is the standard deviation of the x values, and S_y is the standard deviation of the y values.

The formula that act as the foundation blocks are:

$$\bar{x} = \frac{\sum x}{n} \tag{4.116}$$

$$\bar{y} = \frac{\sum y}{n} \tag{4.117}$$

$$S_x = \sqrt{\frac{\sum x^2}{n} - (\bar{x})^2} \tag{4.118}$$

$$S_y = \sqrt{\frac{\sum y^2}{n} - (\bar{y})^2} \tag{4.119}$$

$$S_{xy} = \frac{\sum xy}{n} - (\bar{x})(\bar{y}) \tag{4.120}$$

with n as the number of observations / data points.

[SI*] The formula for the regression line is

$$y - \bar{y} = \frac{S_{xy}}{(S_x)^2}(x - \bar{x}) \tag{4.121}$$

ii. Linear Regression

[SI*] Often, in practice, we need to solve problems involving sets of variables when it is known that there exists some inherent relationship among the variables. For example, in an industrial situation it may be known that the tar content in the outlet stream in a chemical process is related to the inlet temperature. It may be of interest to develop a method of prediction, that is, a procedure for estimating the tar content for various levels of the inlet temperature from experimental information. In this case the tar content is the dependent variable, or response, and the inlet temperature is the independent variable, or regressor. A reasonable form of a relationship between the response Y and the regressor x is the linear relationship

$$Y = \beta_0 + \beta_1 x$$

with β_0 as the intercept and β_1 as the slope. If the relationship is exact, then it is a deterministic relationship between two scientific variables adn there is no random or probabilistic component to it. However, the relationship is not deterministic (i.e., a given x does not always give the same value for Y).

The concept of regression analysis deals with finding the best relationship between Y and x, quantifying the strength of that relationship, and using methods that allow for predictionof the response values given values of the regressor x.

[SI*] In most application of regression, the linear equation $Y = \beta_0 + \beta_1 x$ is an approximation that is a simplification of something unknown and much more complicated.

Thus, an analysis of the relationship between Y and x requires the statement of a statistical model. The model include the set

$$\{(x_i, y_i);\ i = 1, 2, \ldots, n\}$$

of data involving n pairs of (x, y) values. The value y_i depends on x_i, via a linear structure that also has the random component involved.

The basis for the use of a statistical model relates to how the random variable Y moves with x and the random component.

Definition 4.72: Simple Linear Regression Model

the response Y is related to the independent variable x through the equation

$$Y = \beta_0 + \beta_1 x + \epsilon$$

β_0 and β_1 are unknown intercept and slope parameters, respectively, and ϵ is a random variable that is assumed to be distributed with $E(\epsilon) = 0$ and $\text{Var}(\epsilon) = \sigma^2$. The quantity σ^2 is often called the error variance or residual variance.

Definition 4.73: The Fitted Regression Line

Suppose we denote the estimates b_0 for β_0 and b_1 for β_1. Then the estimated or fitted regression line is given by

$$\hat{y} = b_0 + b_1 x$$

where $\hat{y}$ is the predicted or fitted value. Obviously, the fitted line is the estimate of the true regression line.

Definition 4.74: Residual: Error in Fit

Given a set of regression data $\{(x_i, y_i);\ i = 1, 2, \ldots, n\}$ and a fitted model, $\hat{y}_i = b_0 + b_1 x$, the ith residual ϵ_i, is given by

$$\epsilon_i = y_i - \hat{y}_i, \quad i = 1, 2, \ldots, n$$

Definition 4.75: Estimating the Regression Coefficients

In order to find b_0 and b_1, the estimates of β_0 and β_1, so taht the sum of squares of the residuals is a minimum. The residual sum of squares is often called the sum of squares of the errors about the regression line and is denoted by SSE. This minimization procedure for estimating the parameters is called the method of least squares.

Given the sample $\{(x_i, y_i);\ i = 1, 2, \ldots, n\}$, the least squares estimates b_0 and b_1 of the regression coefficients β_0 and β_1 are computed from the formulas

$$b_1 = \frac{n\sum_{i=1}^n x_i y_i - \left(\sum_{i=1}^n x_i\right)\left(\sum_{i=1}^n y_i\right)}{n\sum_{i=1}^n x_i^2 \left(\sum_{i=1}^n x_i\right)^2} = \frac{\sum_{i=1}^n (x_i - \overline{x})(y_i - \overline{y})}{\sum_{i=1}^n (x_i - \overline{x})^2}$$

and

$$b_0 = \frac{\sum_{i=1}^n y_i - b_1 \sum_{i=1}^n x_i}{n}$$

Thus, the estimated regression line is given by

$$\hat{y} = b_0 + b_1 x$$

[SI*] In addition to the assumptions that the error term in the model

$$Y_i = \beta_0 + \beta_1 x_i + \epsilon_i$$

is a random variable with mean 0 and constant variance σ^2, suppose that we make the further assumption that $\epsilon_1, \epsilon_2, \ldots, \epsilon_n$ are independent from run to run in the experiment. This provides a foundation for finding the means and variances for the estimators of β_0 and β_1.

It is important to remember that our values of b_0 and b_1, based on a given sample of n observations, are only estimates of true parameters β_0 and β_1. If the experiment is repeated over and over again, each time using the same fixed values of x, the resulting estimates of β_0 and β_1 will most likely differ from experiment to experiment. These different estimates

may be viewed as values assumed by the random variables B_0 and B_1, while b_0 and b_1 are specific realizations.

Since the values of x remain fixed, the values of B_0 and B_1 depend on the variations in the values of y or, more precisely, on the values of the random variables, $Y_1, Y_2, \ldots, Y_n$. The distributional assumptions imply that the $Y_i, i = 1, 2, \ldots, n$, are also independently distributed, with mean $\mu_{Y|x} = \beta_0 + \beta_1 x_i$ and equal variances σ^2; that is,

$$\sigma^2_{Y|x_i} \quad \text{for } i = 1, 2, \ldots, n$$

[SI*] **Mean and Variance of Estimators**

We show that the estimator B_1 is unbiased for β_1 and demonstrate the variances of both B_0 and B_1. This will begin a series of developments that lead to hypothesis testing and confidence interval estimation on the intercept and slope.

Since the estimator

$$B_1 = \frac{\sum_{i=1}^n (x_i - \overline{x})(Y_i - \overline{Y})}{\sum_{i=1}^n (x_i - \overline{x})^2} = \frac{\sum_{i=1}^n (x_i - \overline{x})Y_i}{\sum_{i=1}^n (x_i - \overline{x})^2}$$

is of the form $\sum_{i=1}^n c_i Y_i$, where

$$c_i = \frac{x_i - \overline{x}}{\sum_{i=1}^n (x_i - \overline{x})^2}, \quad i = 1, 2, \ldots, n$$

we may conclude that B_1 has a $n(\mu_{B_1}, \sigma_{B_1})$ distribution with

$$\mu_{B_1} = \frac{\sum_{i=1}^n (x_i - \overline{x})(\beta_0 + \beta_1 x_i)}{\sum_{i=1}^n (x_i - \overline{x})^2} = \beta_1$$

$$\sigma^2_{B_1} = \frac{\sum_{i=1}^n (x_i - \overline{x})^2 \sigma^2_{Y_i}}{[\sum_{i=1}^n (x_i - \overline{x})^2]^2} = \frac{\sigma^2}{\sum_{i=1}^n (x_i - \overline{x})^2}$$

it can also be shown that the random variable B_0 is normally distributed with mean

$$\mu_{B_0} = \beta_0$$

and variance

$$\sigma^2_{B_0} = \frac{\sum_{i=1}^n x_i^2}{n \sum_{i=1}^n (x_i - \overline{x})^2} \sigma^2$$

From the foregoing results, it is apparent that the least squares estimators for β_0 and β_1 are both unbiased estimators.

[SI*] The parameter σ^2, the model error variance, reflects random variation or experimental error variation around the regression line. It is advantageous to use the notation

$$S_{xx} = \sum_{i=1}^n (x_i - \overline{x})^2$$

$$S_{yy} = \sum_{i=1}^n (y_i - \overline{y})^2$$

$$S_{xy} = \sum_{i=1}^n (x_i - \overline{x})(y_i - \overline{y})$$

Now we may write the error sum of squares as follows:

$$\begin{aligned} SSE &= \sum_{i=1}^{n}(y_i - b_0 - b_1 x_i)^2 \\ &= \sum_{i=1}^{n}[(y_i - \overline{y}) - b_1(x_i - \overline{x})]^2 \\ &= \sum_{i=1}^{n}(y_i - \overline{y})^2 - 2b_1\sum_{i=1}^{n}(x_i - \overline{x})(y_i - \overline{y}) + b_1^2\sum_{i=1}^{n}(x_i - \overline{x})^2 \\ &= S_{yy} - 2b_1 S_{xy} + b_1^2 S_{xx} \\ &= S_{yy} - b_1 S_{xy} \end{aligned}$$

with $b_1 = \frac{S_{xy}}{S_{xx}}$.

Theorem 4.28: Unbiased Estimate of Variance

An unbiased estimate of σ^2 is

$$s^2 = \frac{SSE}{n-2} = \sum_{i=1}^{n}\frac{(y_i - \hat{y})^2}{n-2} = \frac{S_{yy} - b_1 S_{xy}}{n-2}$$

The important parameter σ^2 estimated by s^2 is called a mean squared error, depicting a type of mean (division by $n-2$) of the squared residuals.

[SI*] We know that B_1 follows a normal distribution. It turns out that under the normality assumption, we are able to conclude that $\frac{(n-2)S^2}{\sigma^2}$ is a chi-squared variable with $n-2$ degrees of freedom, independent of the random variable B_1.

Thus, the statistic

$$T = \frac{(B_1 - \beta_1)/(\sigma/\sqrt{S_{xx}})}{S/\sigma} = \frac{B_1 - \beta_1}{S/\sqrt{S_{xx}}}$$

has a t-distribution with $n-2$ degrees of freedom. The statistic T can be used to construct a $100(1-\alpha)\%$ confidence interval for the coefficient β_1

Definition 4.76: Confidence Interval for β_1

A $100(1-\alpha)\%$ confidence interval for the parameter β_1 in the regression line $\mu_{Y|x} = \beta_0 + \beta_1 x$ is

$$b_1 - t_{\frac{\alpha}{2}}\frac{s}{\sqrt{S_{xx}}} < \beta_1 < b_1 + t_{\frac{\alpha}{2}}\frac{s}{\sqrt{S_{xx}}}$$

where $t_{\frac{\alpha}{2}}$ is a value of the t-distribution with $n-2$ degrees of freedom that leaves an area of $\frac{\alpha}{2}$ to the right.

[SI*] Confidence intervals and hypothesis testing on the coefficient β_0 may be established from the fact that B_0 is also normally distributed. It is not difficult to show that

$$T = \frac{B_0 - \beta_0}{S\sqrt{\frac{\sum_{i=1}^{n} x_i^2}{nS_{xx}}}}$$

has a t-distribution with $n-2$ degrees of freedom from which we may construct a $100(1-\alpha)\%$ confidence interval for β_0

Definition 4.77: Confidence Interval for β_0

A $100(1-\alpha)\%$ confidence interval for the parameter β_0 in the regression line $\mu_{Y|x} = \beta_0 + \beta_1 x$ is

$$b_0 - t_{\frac{\alpha}{2}} \frac{s}{\sqrt{nS_{xx}}} \sqrt{\sum_{i=1}^{n} x_i^2} < \beta_0 < b_0 + t_{\frac{\alpha}{2}} \frac{s}{\sqrt{nS_{xx}}} \sqrt{\sum_{i=1}^{n} x_i^2}$$

where $t_{\frac{\alpha}{2}}$ is a value of the t-distribution with $n-2$ degrees of freedom that leaves an area of $\frac{\alpha}{2}$ to the right.

[SI*] Coefficient of determination, represented by R^2, is a measure of the proprotion of variability explained by the fitted model. The analysis-of-variance approach makes use of the error sum of squares $SSE = \sum_{i=1}^{n}(x_i - \hat{y}_i)$ and the total corrected sum of squares $SST = \sum_{i=1}^{n}(y_i - \overline{y}_i)^2$. The SSE value is the variation due to error, or variation unexplained. Clearly, if $SSE = 0$, all variation is explained. The quantity that represents variation explained is $SST - SSE$. The R^2 is

$$R^2 = 1 - \frac{SSE}{SST}$$

Note that if the fit is perfect, all residuals are zero, and thus $R^2 = 1.0$. But f SSE is only slightly smaller than SST, $R^2 \approx 0$.

The R^2 criterion is dangerous to use for comparing competing models for the same data set. Adding additional terms to the model decreases SSE and thus increases R^2 (or at least does not decrease it). This implies that R^2 can be made artificially high by an unwise practice of overfitting (i.e., the inclusion of too many model terms).

[SI*] **Prediction**

There are several reasons for building a linear regression. One, of course, is to predict response values at one or more values of the independent variable.

The equation $\hat{y} = b_0 + b_1 x$ may be used to predict or estimate the mean response $\mu_{Y|x_0}$ at $x = x_0$, where x_0 is not necessarily one of the prechosen values, or it may be used to predict a single value y_0 of the variable Y_0, when $x = x_0$. We would expect the error of prediction to be higher in the case of a single predicted value than in the case where a mean is predicted. This, then, will affect the width of our intervals for the values being predicted.

Suppose that the experimenter wishes to construct a confidence interval for $\mu_{Y|x_0}$. We shall use the point estimator $\hat{Y}_0 = B_0 + B_1 x_0$ to estimate $\mu_{Y|x_0} = \beta_0 + \beta_1 x$. It can be shown that the sampling distribution of $\hat{Y}_0$ is normal with mean

$$\mu_{Y|x_0} = E(\hat{Y}_0) = E(B_0 + B_1 x_0) = \beta_0 + \beta_1 x_0 = \mu_{Y|x_0}$$

and variance

$$\sigma^2_{\hat{Y}_0} = \sigma^2_{B_0 + B_1 x_0} = \sigma_{\overline{Y} + B_1(x_0 - \overline{x})} = \sigma^2 \left[\frac{1}{n} + \frac{(x_0 - \overline{x})^2}{S_{xx}} \right]$$

the latter following from the fact that $\text{Cov}(\overline{Y}, B_1) = 0$. Thus, a $100(1-\alpha)\%$ confidence

interval on the mean response $\mu_{Y|x_0}$ can now be constructed from the statistic

$$T = \frac{\hat{Y}_0 - \mu_{Y|x_0}}{S\sqrt{\frac{1}{n} + \frac{(x_0-\overline{x})^2}{S_{xx}}}}$$

which has a t-distribution with $n-2$ degrees of freedom.

Definition 4.78: Confidence Interval for $\mu_{Y|x_0}$

A $100(1-\alpha)\%$ confidence interval for the mean response $\mu_{Y|x_0}$ is

$$\hat{y}_0 - t_{\frac{\alpha}{2}} s\sqrt{\frac{1}{n} + \frac{(x_0-\overline{x})^2}{S_{xx}}} < \mu_{Y|x_0} < \hat{y}_0 + t_{\frac{\alpha}{2}} s\sqrt{\frac{1}{n} + \frac{(x_0-\overline{x})^2}{S_{xx}}}$$

where $t_{\frac{\alpha}{2}}$ is a value of the t-distribution with $n-2$ degrees of freedom that leaves an area of $\frac{\alpha}{2}$ to the right.

[SI*] **Prediction Interval**

Another type of interval that is often misinterpreted and confused with that $\mu_{Y|x_0}$ is the prediction interval for a future observed response.

To obtain a prediction interval for any single value y_0 of the variable Y_0, it is necessary to estimate the variance of the differences between the ordinates $\hat{y}_0$, obtained from the computed regression lines in repeated sampling when $x = x_0$, and the corresponding true ordinate y_0. We can think of the difference $\hat{y}_0 - y_0$ as a value of the random variable $\hat{Y}_0 - Y_0$, whose sampling distribution can be shown to be normal with mean

$$\mu_{\hat{Y}_0-Y_0} = E(\hat{Y}_0 - Y_0) = E\left[B_0 + B_1x_0 - (\beta_0 + \beta_1x_0 + \epsilon_0)\right] = 0$$

and variance

$$\sigma^2_{\hat{Y}_0-Y_0} = \sigma_{B_0+B_1x_0-\epsilon_0} = \sigma_{\overline{Y}+B_1(x_0-\overline{x})-\epsilon_0} = \sigma^2\left[1 + \frac{1}{n} + \frac{(x_0-\overline{x})^2}{S_{xx}}\right]$$

Thus, a $100(1-\alpha)\%$ prediction interval for a single predicted value y_0 can be constructed from the statistic

$$T = \frac{\hat{Y}_0 - Y_0}{S\sqrt{1 + \frac{1}{n} + \frac{(x_0-\overline{x})^2}{S_{xx}}}}$$

which has a t-distribution with $n-2$ degrees of freedom.

Definition 4.79: Prediction Interval for y_0

A $100(1-\alpha)\%$ prediction interval for a single response y_0 is given by

$$\hat{y}_0 - t_{\frac{\alpha}{2}} s\sqrt{1 + \frac{1}{n} + \frac{(x_0-\overline{x})^2}{S_{xx}}} < y_0 < \hat{y}_0 + t_{\frac{\alpha}{2}} s\sqrt{1 + \frac{1}{n} + \frac{(x_0-\overline{x})^2}{S_{xx}}}$$

where $t_{\frac{\alpha}{2}}$ is a value of the t-distribution with $n-2$ degrees of freedom that leaves an area of $\frac{\alpha}{2}$ to the right..

iii. Simple Regression Line Plot and Pearson's Correlation Computation

Suppose we have a bivariate data as follow Compute the Pearson's correlation (PMCC), then plot

Distance, x (km)	4	8	5	10	6
Time, y (minutes)	15	35	12	40	24

Table 4.16: *Distance travelled to school and travel time*

the data on a scatter diagram and draw the line of best fit.

Solution:

We will have

$$\bar{x} = \frac{\sum x}{n} = 6.6$$
$$\bar{y} = \frac{\sum y}{n} = 25.2$$
$$S_x = \sqrt{\frac{\sum x^2}{n} - (\bar{x})^2} = 2.154$$
$$S_y = \sqrt{\frac{\sum y^2}{n} - (\bar{y})^2} = 10.907$$
$$S_{xy} = \frac{\sum xy}{n} - (\bar{x})(\bar{y}) = 22.48$$

Hence

$$r = \frac{S_{xy}}{S_x S_y} = 0.957$$

The value of r is close to $+1$, showing that there is strong positive correlation between the distance travelled to school and the time taken.

Now we will compute the regression line formula

$$y - \bar{y} = \frac{S_{xy}}{(S_x)^2}(x - \bar{x})$$
$$y - 25.2 = \frac{22.48}{(2.154)^2}(x - 6.6)$$
$$y = 4.85x - 6.78$$

To plot the bivariate data on a scatter diagram and the regression line we can use Hamzstlab Mathematics, and the backend computation can be done with SymIntegration.

In SymIntegration the function to compute Pearson's correlation is:
rpearson(M,n)
with M as the matrix of size $n \times 2$, the first column of the matrix represents the independent variable, x, while the second column of the matrix represents the dependent variable, y.

The function to compute the regression line is:
regressionline(const SymbolicMatrix &M, int n)

with M as the matrix of size $n \times 2$, the result will be a symbolic function, $y(x)$.

```
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
./main ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Load Vector from textfile and Compute Regression Line ]#
W:
[ 4 15]
[ 8 35]
[ 5 12]
[10 40]
[ 6 24]

x bar: 6.6
y bar: 25.2
sum x: 33
sum y: 126
sum xy: 944
sum x^2: 241
sum y^2: 3770

Pearson's correlation coefficient, r :
0.956835

Regression line, y = 4.84483*x-6.77586

Time taken by function: 7136 microseconds
```

Figure 4.52: *The computation of Pearson's correlation (PMCC), r, and the regression line equation with SymIntegration took 7136 microseconds. (**SymIntegration/Examples/Statistics/Test SymIntegration Load Vector from textfile and Compute Regression Line/main.cpp**).*

iv. Compute Linear Regression with Confidence Interval and Prediction Interval with SymIntegration

One of the more challenging problems confronting the water pollution control field is presentedby the tanning industry. Tannery wastes are chemically complex. They are characterized by high values of chemical oxygen demand, volatile solids, and othe pollution measures. Consider the experimental data, which were obtained from 33 samples of chemically treated waste in a study conducted at Virginia Tech. Readings on x, the percent reduction in total solids, and y, the percent reduction in chemical oxygen demand, were recorded.

Solids Reduction, x(%)	**Oxygen Demand Reduction, y (%)**	**Solids Reduction, x(%)**	**Oxygen Demand Reduction, y (%)**
3	5	36	34
7	11	37	36
11	21	38	38
15	16	39	37
18	16	39	36
27	28	39	45
29	27	40	39
30	25	41	41
30	35	42	40
31	30	42	44
31	40	43	37
32	32	44	44
33	34	45	46
33	32	46	46
34	34	47	49
36	37	50	51
36	38		

Table 4.17: *Measures of reduction in solids and oxygen demand.*

Estimate the regression line for the pollution data of Table 4.10. Find a 95% confidence interval for β_0 and β_1 in the regression line $\mu_{Y|x} = \beta_0 + \beta_1 x$ based on the pollution data, construct 95% confidence limits for the mean response $\mu_{Y|x_i}$ and the 95% prediction interval for y_i for all x_i with $i = 1, 2, \ldots, 33$.

Solution:

$$\sum_{i=1}^{n=33} x_i = 1104$$
$$\sum_{i=1}^{n=33} y_i = 1124$$
$$\sum_{i=1}^{n=33} x_i y_i = 41,355$$
$$\sum_{i=1}^{n=33} x_i^2 = 41,086$$

Therefore,

$$b_1 = \frac{(33)(41,355) - (1104)(1124)}{(33)(41,086) - (1104)^2} = 0.903643$$
$$b_0 = \frac{1124 - (0.903643)(1104)}{33} = 3.829633$$

Thus, the estimated regression line is given by

$$\hat{y} = 3.8296 + 0.9036x$$

Now, to compute the confidence interval we will need to compute the "S" sum formulas

$$S_{xx} = 4152.18$$
$$S_{yy} = 3713.88$$
$$S_{xy} = 3752.09$$

The 95% confidence interval for the parameter β_1 is

$$b_1 - t_{0.025}\frac{s}{\sqrt{S_{xx}}} < \beta_1 < b_1 + t_{0.025}\frac{s}{\sqrt{S_{xx}}}$$

Therefore,

$$0.801425 < \beta_1 < 1.00586$$

The 95% confidence interval for the parameter β_0 is

$$b_0 - t_{0.025}\frac{s}{\sqrt{nS_{xx}}}\sqrt{\sum_{i=1}^{n} x_i^2} < \beta_0 < b_0 + t_{0.025}\frac{s}{\sqrt{nS_{xx}}}\sqrt{\sum_{i=1}^{n} x_i^2}$$

Therefore,

$$0.222861 < \beta_0 < 7.43641$$

In SymIntegration, we have another function that can compute the estimated regression line, the $100(1-\alpha)\%$ confidence interval for β_0 and β_1 in the regression line $\mu_{Y|x} = \beta_0 + \beta_1 x$, that will also compute the one-way ANOVA table for the regression line, and print the tabe of the original sample data and the fitted / regressed value and the $100(1-\alpha)\%$ confidence limits for the mean response $\mu_{Y|x_i}$ and the $100(1-\alpha)\%$ prediction interval for y_i for all x_i with $i = 1, 2, \ldots, n$.

This function is:
regressionline(vector<vector<double>> matrix, double α)

With **matrix** as a $n \times 2$ matrix that can be saved as textfile (.txt), and α is the level of significance. While the other function at the previous example: **regressionline(const SymbolicMatrix &M, int n)** resulting in Symbolic output, this function is a **void** function that shows the output only on the computer screen.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Linear Regression with Confidence Interval and Prediction Interval

**********************************************************

Simple Linear Regression

Sxx = 4152.18 , Syy = 3713.88 , Sxy = 3752.09
SSE = 323.327, SSR = 3390.55, SST = 3713.88
R-Squared = 91.2941 %
Computed f = 325.08, P-value = 0

An unbiased estimate of σ^{2} is :
s^{2} = 10.4299
The estimated regression line is given by:
y = 3.82963 + 0.903643x

**********************************************************

Analysis of Variance

Source                  DF              SS                      MS                      F               P
Regression              1               3390.55                 3390.55                 325.08          0
Residual Error          31              323.327                 10.4299
Total                   32              3713.88

R-Squared               S
91.2941 %               3.22954

If P value is less than the level of significance 0.05 then reject H0: β1 = 0

Rejection of H0 may suggest that the relationship is, indeed, linear.
The failure to reject H0: β1 = 0 suggests that there is no linear relationship between Y and x.

**********************************************************

A 95% confidence interval for the parameter β1 is :
0.801425 < β1 < 1.00586
A 95% confidence interval for the parameter β0 is :
0.222861 < β0 < 7.43641
```

Figure 4.53: *The computation of the estimated regression line, the* $100(1-\alpha)\%$ *confidence interval for* β_0 *and* β_1 *in the regression line* $\mu_{Y|x} = \beta_0 + \beta_1 x$*, the one-way ANOVA table. (**SymIntegration/Examples/Statistics/Test SymIntegration Linear Regression with Confidence Interval and Prediction Interval/main.cpp**).*

Obs	y_data	Fit	SE Fit	95% CI	95% PI
1	5	6.54056	1.62659	(3.22311 , 9.85802)	(-0.834391 , 13.9155)
2	11	10.1551	1.44014	(7.21795 , 13.0923)	(2.94324 , 17.367)
3	21	13.7697	1.25801	(11.204 , 16.3354)	(6.70095 , 20.8385)
4	16	17.3843	1.08238	(15.1768 , 19.5918)	(10.4375 , 24.331)
5	16	20.0952	0.957084	(18.1432 , 22.0472)	(13.2254 , 26.965)
6	28	28.228	0.648619	(26.9051 , 29.5509)	(21.5098 , 34.9462)
7	27	30.0353	0.604898	(28.8016 , 31.269)	(23.3341 , 36.7365)
8	25	30.9389	0.588247	(29.7392 , 32.1387)	(24.2439 , 37.634)
9	35	30.9389	0.588247	(29.7392 , 32.1387)	(24.2439 , 37.634)
10	30	31.8426	0.575493	(30.6688 , 33.0163)	(25.1521 , 38.533)
11	40	31.8426	0.575493	(30.6688 , 33.0163)	(25.1521 , 38.533)
12	32	32.7462	0.566897	(31.59 , 33.9024)	(26.0588 , 39.4336)
13	34	33.6499	0.562652	(32.5023 , 34.7974)	(26.964 , 40.3358)
14	32	33.6499	0.562652	(32.5023 , 34.7974)	(26.964 , 40.3358)
15	34	34.5535	0.562855	(33.4056 , 35.7015)	(27.8675 , 41.2395)
16	37	36.3608	0.576484	(35.185 , 37.5365)	(29.67 , 43.0516)
17	38	36.3608	0.576484	(35.185 , 37.5365)	(29.67 , 43.0516)
18	34	36.3608	0.576484	(35.185 , 37.5365)	(29.67 , 43.0516)
19	36	37.2644	0.589604	(36.0619 , 38.4669)	(30.5689 , 43.96)
20	38	38.1681	0.606595	(36.9309 , 39.4052)	(31.4662 , 44.8699)
21	37	39.0717	0.62714	(37.7927 , 40.3508)	(32.362 , 45.7814)
22	36	39.0717	0.62714	(37.7927 , 40.3508)	(32.362 , 45.7814)
23	45	39.0717	0.62714	(37.7927 , 40.3508)	(32.362 , 45.7814)
24	39	39.9754	0.650904	(38.6478 , 41.3029)	(33.2562 , 46.6945)
25	41	40.879	0.677548	(39.4971 , 42.2609)	(34.1489 , 47.6091)
26	40	41.7826	0.706746	(40.3412 , 43.2241)	(35.0401 , 48.5252)
27	44	41.7826	0.706746	(40.3412 , 43.2241)	(35.0401 , 48.5252)
28	37	42.6863	0.738195	(41.1807 , 44.1919)	(35.9297 , 49.4429)
29	44	43.5899	0.771621	(42.0162 , 45.1637)	(36.8179 , 50.362)
30	46	44.4936	0.806777	(42.8481 , 46.139)	(37.7045 , 51.2827)
31	46	45.3972	0.843448	(43.677 , 47.1174)	(38.5896 , 52.2048)
32	49	46.3009	0.881443	(44.5031 , 48.0986)	(39.4733 , 53.1285)
33	51	49.0118	1.00185	(46.9685 , 51.0551)	(42.1155 , 55.9081)

```
Time taken by function: 3486 microseconds
```

Figure 4.54: *The table that shows the fitted $\hat{y}_i$, SE fit, the confidence interval for the mean response $\mu_{Y|x_i}$ and prediction interval for y_i. (**SymIntegration/Examples/Statistics/Test SymIntegration Linear Regression with Confidence Interval and Prediction Interval/main.cpp**).*

The term "SE Fit" is used in computing confidence intervals on mean response, the formula is:

$$\text{SE Fit}_i = t_{\frac{\alpha}{2}} s \sqrt{\frac{1}{n} + \frac{(x_i - \overline{x})^2}{S_{xx}}}$$

v. Multiple Linear Regression

[SI*] In most research problems where regression analysis is applied, more than one independent variable is needed in the regression model. The complexity of most scientific mechanisms is such that in order to be able to predict an important response, a multiple regression model is needed. When this model is linear in the coefficients, it is called a multiple linear regression model. For the case of k independent variables $x_1, x_2, \ldots, x_k$, the mean $Y|x_1, x_2, \ldots, x_k$ is given by the multiple linear regression model

$$\mu_{Y|x_1,x_2,\ldots,x_k} = \beta_0 + \beta_1 x_1 + \cdots + \beta_k x_k$$

and the estimated response is obtained from the sample regression equation

$$\hat{y} = b_0 + b_1 x_1 + \cdots + b_k x_k$$

where each regression coefficient β_i is estimated by b_i from the sample data using the method of least squares.

Definition 4.80: Multiple Linear Regression Model

We obtain the least squares estimators of the parameters $\beta_0, \beta_1, \ldots, \beta_k$ by fitting the multiple linear regression model

$$\mu_{Y|x_1,x_2,\ldots,x_k} = \beta_0 + \beta_1 x_1 + \cdots + \beta_k x_k$$

to the data points

$$\{(x_{1i}, x_{2i}, \ldots, x_{ki}, y_i); \quad i = 1, 2, \ldots, n \text{ and } n > k\}$$

where y_i is the observed response to the values $x_{1i}, x_{2i}, \ldots, x_{ki}$ of the k independent variables $x_1, x_2, \ldots, x_k$. Each observation $(x_{1i}, x_{2i}, \ldots, x_{ki}, y_i)$ is assumed to satisfy the following equation.

$$y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \cdots + \beta_k x_{ki} + \epsilon_i$$

or

$$\begin{aligned} y_i &= \hat{y}_i + e_i \\ &= b_0 + b_1 x_{1i} + b_2 x_{2i} + \cdots + b_k x_{ki} + e_i \end{aligned}$$

where ϵ_i and e_i are the random error and residual, respectively, associated with the response y_i and fitted value $\hat{y}_i$.

[SI*] As in the case of simple linear regression, it is assumed that the ϵ_i are independent and identically distributed with mean 0 and common variance σ^2.

In using the concept of least squares to arive at estimates $b_0, b_1, \ldots, b_k$, we minimize the expression

$$\begin{aligned} SSE &= \sum_{i=1}^{n} e_i^2 \\ &= \sum_{i=1}^{n} (y_i - b_0 - b_1 x_{1i} - b_2 x_{2i} - \cdots - b_k x_{ki})^2 \end{aligned}$$

Differentiating SSE in turn with respect to $b_0, b_1, \ldots, b_k$ and equating to zero, we generate the set of $k+1$ normal equations for multiple linear regression.

Multiple Linear Regression Parameter Estimation with Matrices
In fitting a multiple linear regression model, particularly when the number of variables exceeds two, a knowledge of matrix theory can facilitate the mathematical manipulations considerably.

Suppose that the experimenter has k independent variables $x_1, x_2, \ldots, x_k$ and n observations $y_1, y_2, \ldots, y_n$, each of which can be expressed by the equation

$$y_1 = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \cdots + \beta_k x_{ki} + \epsilon_i$$

This model essentially represents n equations describing how the response values are generated in the scientific process.

Definition 4.81: General Linear Model

Using matrix notation, we can write the following equation that represents the multiple linear regression equation:

$$\boldsymbol{y} = \boldsymbol{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}$$

where

$$\boldsymbol{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}, \quad \boldsymbol{X} = \begin{bmatrix} 1 & x_{11} & x_{21} & \ldots & x_{k1} \\ 1 & x_{12} & x_{22} & \ldots & x_{k2} \\ \vdots & \vdots & \ddots & \vdots & \\ 1 & x_{1n} & x_{2n} & \ldots & x_{kn} \end{bmatrix}$$

$$\boldsymbol{\beta} = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_k \end{bmatrix}, \quad \boldsymbol{\epsilon} = \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_n \end{bmatrix}$$

Then the least squares method for estimation of β, involves finding $\boldsymbol{b}$ for which

$$SSE = (\boldsymbol{y} - \boldsymbol{X}\boldsymbol{b})^T(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{b})$$

is minimized. This minimization process involves solving for $\boldsymbol{b}$ in the equation

$$\frac{\partial}{\partial \boldsymbol{b}}(SSE) = \boldsymbol{0}$$

The result reduces to the solution of $\boldsymbol{b}$ in

$$(\boldsymbol{X}^T\boldsymbol{X})\boldsymbol{b} = \boldsymbol{X}^T\boldsymbol{y}$$

Notice the nature of the $\boldsymbol{X}$ matrix. Apart from the initial element, the ith row represents the x-values that give rise to the response y_i. Writing

$$\boldsymbol{X}^T\boldsymbol{X} = \begin{bmatrix} n & \sum_{i=1}^n x_{1i} & \sum_{i=1}^n x_{2i} & \ldots & \sum_{i=1}^n x_{ki} \\ \sum_{i=1}^n x_{1i} & \sum_{i=1}^n x_{1i}^2 & \sum_{i=1}^n x_{1i}x_{2i} & \ldots & \sum_{i=1}^n x_{1i}x_{ki} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \sum_{i=1}^n x_{ki} & \sum_{i=1}^n x_{ki}x_{1i} & \sum_{i=1}^n x_{ki}x_{2i} & \ldots & \sum_{i=1}^n x_{ki}^2 \end{bmatrix}$$

and

$$\boldsymbol{X}^T\boldsymbol{y} = \begin{bmatrix} \sum_{i=1}^{n} y_i \\ \sum_{i=1}^{n} x_{1i}y_i \\ \vdots \\ \sum_{i=1}^{n} x_{ki}y_i \end{bmatrix}$$

If the matrix $\boldsymbol{X}^T\boldsymbol{X}$ is not singular, the formula for parameter estimation in multiple linear regression is

$$\boldsymbol{b} = (\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\boldsymbol{y}$$

where $\boldsymbol{b}$ is the vector of estimated coefficients, $\boldsymbol{X}$ is the matrix of independent variables (with a column of ones for the intercept), $\boldsymbol{X}^T$ is the transpose of $\boldsymbol{X}$, and $\boldsymbol{y}$ is the vector of the dependent variable. This formula uses the method of least squares to find the coefficients that minimize the sum of squared errors, which is the vertical distance between the observed and predicted values.

Model:

$$\boldsymbol{y} = \boldsymbol{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}$$

with:
$\boldsymbol{y}$: Vector of the dependent variable
$\boldsymbol{X}$: Matrix of independent variables (including a column of 1 for the intercept)
$\boldsymbol{\beta}$: Vector of population parameters to be estimated
$\boldsymbol{\epsilon}$: Vector of error terms

Estimation:

$$\boldsymbol{b} = (\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\boldsymbol{y}$$

with:
$\boldsymbol{b}$: Vector of estimated coefficients (intercept and slopes)
$\boldsymbol{X}^T$: Transpose of the matrix $\boldsymbol{X}$.
$(\boldsymbol{X}^T\boldsymbol{X})^{-1}$: The inverse of the matrix product $\boldsymbol{X}^T\boldsymbol{X}$.

The formula finds the set of coefficients $(\boldsymbol{b})$ that produce a regression line (or plane/hyper-plane) that best fits the data. "Best fit" is defined by the least-squares criterion, meaning the sum of the squared differences between the actual y values and the predicted $\hat{y}$ values is minimized. The matrix algebra approach provides a direct way to calculate the coefficients without having to manually solve a system of linear equations for each parameter, which would be necessary for a large number of independent variables.

To evaluate the model we can use R^2(R-squared) to measure the proportion of the variance in the dependent variable that is predictable from the independent variables. A modified version of R^2, adjusted R-squared is a better measure when comparing models with different numbers of predictors. Another measure is the p-value, it is often used to determine the statistical significance of the independent variables, indicating whether their relationship with the dependent variable is likely due to chance.

[SI*] **Properties of the Least Squares Estimators**

The means and variances of the estimators $b_0, b_1, \ldots, b_k$ are readily obtained under certain assumptions on the random errors $\epsilon_1, \epsilon_2, \ldots, \epsilon_k$ that are identical to those made in the case of simple linear regression. When we assume these errors to be independent, each with mean 0

and variance σ^2, it can be shown that $b_0, b_1, \ldots, b_k$ are, respectively, unbiased estimatros of the regression coefficients $\beta_0, \beta_1, \ldots, \beta_k$.

In addition, the variances of the b's are obtained through the elements of the inverse of the $\boldsymbol{A}$ matrix. Note that the off-diagonal elements of $\boldsymbol{A} = \boldsymbol{X}^T\boldsymbol{X}$ represents sums of products of elements in the columns of $\boldsymbol{X}$, while the diagonal elements of $\boldsymbol{A}$ represents sums of squares of elements in the columns of $\boldsymbol{X}$.
The inverse matrix, $\boldsymbol{A}^{-1}$, apart from the multiplier σ^2, represents the variance-covariance matrix of the estimated regression coefficients.

That is, the elements of the matrix $\boldsymbol{A}^{-1}\sigma^2$ display the variances of $b_0, b_1, \ldots, b_k$ on the main diagonal and covariances on the off-diagonal. For example, in a $k = 2$ multiple linear regression problem, we might write

$$(\boldsymbol{X}^T\boldsymbol{X})^{-1} = \begin{bmatrix} c_{00} & c_{01} & c_{02} \\ c_{10} & c_{11} & c_{12} \\ c_{20} & c_{21} & c_{22} \end{bmatrix}$$

with the elements below the main diagonal determined through the symmetry of the matrix. Then we can write

$$\sigma^2_{b_i} = c_{ii}\sigma^2, \quad i = 0, 1, 2$$
$$\sigma_{b_i b_j} = \text{Cov}(b_i, b_j) = c_{ij}\sigma^2, \quad i \neq j$$

Theorem 4.29: Unbiased Estimate for Multiple Linear Regression

For the linear regression equation

$$\boldsymbol{y} = \boldsymbol{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}$$

an unbiased estimate of σ^2 is given by the error or residual mean square

$$s^2 = \frac{SSE}{n - k - 1}$$

where

$$SSE = \sum_{i=1}^{n} e_i^2 = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

The error and regression sums of squares take on the same form and play the same role as in the simple linear regression case. In fact, the sum-of-squares identity

$$\sum_{i=1}^{n} (y_i - \overline{y_i})^2 = \sum_{i=1}^{n} (\hat{y}_i - \overline{y_i})^2 + \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

continues to hold, and we retain our previous notation, namely

$$SST = SSR + SSE$$

with

$$SST = \sum_{i=1}^{n} (y_i - \overline{y_i})^2$$

and

$$SSR = \sum_{i=1}^{n} (\hat{y}_i - \overline{y_i})^2$$

There are k degrees of freedom associated with SSR, and, as always, SST has $n-1$ degrees of freedom. Therefore, after subtraction, SSE has $n-k-1$ degrees of freedom.

[SI*] **Analysis of Variance in Multiple Regression**

The partition of the total sum of squares into its components, the regression and error sums of squares, plays an important role. An analysis of variance can be conducted to shed light on the quality of the regression equation. A useful hypothesis that determines if a significant amount of variation is explained by the model is

$$H_0 : \ \beta_1 = \beta_2 = \beta_3 = \cdots = \beta_k = 0$$

The analysis of variance involves an F-test via a table. The test is an upper-tailed test.

Source	Sum of Squares	Degrees of Freedom	Mean Squares	F
Regression	SSR	k	$MSR = \frac{SSR}{k}$	$f = \frac{MSR}{MSE}$
Error	SSE	$n-(k+1)$	$MSE = \frac{SSE}{n-(k+1)}$	
Total	SST	$n-1$		

Table 4.18: *The analysis of variance table that uses F-test.*

Rejection of H_0 implies that the regression equation differs from a constant. That is, at least one regressor variable is important.

Further utility of the mean square error (or residual mean square) lies in its use in hypothesis testing and confidence interval estimation. In addition, the mean square error plays an important role in situations where the scientist is searching for the best from a set of competing models. Many model-building criteria involve the statistic s^2.

[SI*] **Inferences in Multiple Linear regression**

A knowledge of the distributions of the individual coefficient estimators enables the experimenter to construct confidence intervals for the coefficients and to test hypotheses about them.

Recall that the $b_j (j = 0, 1, 2, \ldots, k)$ are normally distributed with mean β_j and variance $c_{jj}\sigma^2$. Thus, we can use the statistic

$$t = \frac{b_j - \beta_{j0}}{s\sqrt{c_{jj}}}$$

with $n-k-1$ degrees of freedom to test hypotheses and construct confidence intervals on β_j. For example, if we wish to test

$$H_0 : \beta_j = \beta_{j0}$$
$$H_1 : \beta_j \neq \beta_{j0}$$

we compute the above t-statistic and do not reject H_0 if $-t_{\alpha/2} < t < t_{\alpha/2}$, where $t_{\alpha/2}$ has $n-k-1$ degrees of freedom.

[SI*] **Individual t-Tests for Variable Screening**
The t-test most often used in multiple regression is the one that tests the importance of individual coefficients (i.e.,$H_0 : \beta_j = 0$ against the alternative $H_1 : \beta_j \neq 0$). These tests often contribute to what is termed variable screening, where the analyst attempts to arrive at the most useful model (i.e., the choice of which regressors to use). It should be emphasized here that if a coefficient is found insignificant (i.e., the hypothesis $H_0 : \beta_j = 0$ is not rejected), the conclusion drawn is that the variable is insignificant (i.e., explains an insignificant amount of variation in y), in the presence of the other regressors in the model.

[SI*] **Inferences on Mean Response and Prediction**
One of the most useful inferences that can be made regarding the quality of the predicted response y_0 corresponding to the values $x_{10}, x_{20}, \ldots, x_{k0}$ is the confidence interval on the mean response $\mu_{Y|x_{10},x_{20},\ldots,x_{k0}}$. We are interested in constructing a confidence interval on the mean response for the set of conditions given by

$$\boldsymbol{x}_0^T = [1, x_{10}, x_{20}, \ldots, x_{k0}]$$

We augment the conditions on the x's by the number 1 in order to facilitate the matrix notation. Normality in the ϵ_i produces normality in the b_j and the mean and variance are still the same. So is the covariance between b_i and b_j, for $i \neq j$. Hence,

$$\hat{y} = b_0 + \sum_{j=1}^{k} b_j x_{j0}$$

is likewise normally distributed and is, in fact, an unbiased estimator for the mean response on which we are attempting to attach a confidence interval. The variance of $\hat{y}_0$, written in matrix notation simply as a function of σ^2, $(\boldsymbol{X}^T\boldsymbol{X})^{-1}$, and the condition vector $\boldsymbol{x}_0^T$is

$$\sigma^2_{\hat{y}_0} = \sigma^2 \boldsymbol{x}_0^T (\boldsymbol{X}^T\boldsymbol{X})^{-1} \boldsymbol{x}_0$$

If this expression is expanded for a given case, say $k = 2$, it is readily seen that it appropriately accounts for the variance of the b_j and the covariance of b_i and b_j, for $i \neq j$. After σ^2 is replaced by s^2, the $100(1-\alpha)\%$ confidence interval on $\mu_{Y|x_{10},x_{20},\ldots,x_{k0}}$ can be constructed from the statistic

$$T = \frac{\hat{y}_0 - \mu_{Y|x_{10},x_{20},\ldots,x_{k0}}}{s\sqrt{\boldsymbol{x}_0^T (\boldsymbol{X}^T\boldsymbol{X})^{-1} \boldsymbol{x}_0}}$$

which has a t-distribution with $n - k - 1$ degrees of freedom.

Definition 4.82: Confidence Interval for $\mu_{Y|x_{10},x_{20},\ldots,x_{k0}}$

A $100(1-\alpha)\%$ confidence interval for the mean response $\mu_{Y|x_{10},x_{20},\ldots,x_{k0}}$ is

$$\hat{y}_0 - t_{\frac{\alpha}{2}} s\sqrt{\boldsymbol{x}_0^T (\boldsymbol{X}^T\boldsymbol{X})^{-1} \boldsymbol{x}_0} < \mu_{Y|x_{10},x_{20},\ldots,x_{k0}} < \hat{y}_0 + t_{\frac{\alpha}{2}} s\sqrt{\boldsymbol{x}_0^T (\boldsymbol{X}^T\boldsymbol{X})^{-1} \boldsymbol{x}_0}$$

where $t_{\frac{\alpha}{2}}$ is a value of the t-distribution with $n - k - 1$ degrees of freedom.

The quantity $s\sqrt{\boldsymbol{x}_0^T (\boldsymbol{X}^T\boldsymbol{X})^{-1} \boldsymbol{x}_0}$ is often called the standard error of prediction.

[SI*] As in the case of simple linear regression, we need to make a clear distinction between the confidence interval on a mean response and the prediction interval on an observed response.

The latter provides a bound within which we can say with a preselected degree of certainty that a new observed response will fall.

A prediction interval for a single predicted response y_0 is once again established by considering the difference $\hat{y}_0 - y_0$. The sampling distribution can be shown to be normal with mean

$$\mu_{\hat{y}_0 - y_0} = 0$$

and variance

$$\sigma^2_{\hat{y}_0 - y_0} = \sigma^2[1 + \boldsymbol{x}_0^T(\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{x}_0]$$

Thus, a $100(1-\alpha)\%$ prediction interval for a single prediction value y_0 can be constructed from the statistic

$$T = \frac{\hat{y}_0 - y_0}{s\sqrt{1 + \boldsymbol{x}_0^T(\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{x}_0}}$$

which has t-distribution with $n-k-1$ degrees of freedom.

Definition 4.83: Prediction Interval for y_0

A $100(1-\alpha)\%$ prediction interval for a single response y_0 is given by

$$\hat{y}_0 - t_{\frac{\alpha}{2}} s\sqrt{1 + \boldsymbol{x}_0^T(\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{x}_0} < y_0 < \hat{y}_0 + t_{\frac{\alpha}{2}} s\sqrt{1 + \boldsymbol{x}_0^T(\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{x}_0}$$

where $t_{\frac{\alpha}{2}}$ is a value of the t-distribution with $n-k-1$ degrees of freedom.

[SI*] **Choice of a Fitted Model through Hypothesis Testing**

In many regression situations, individual coefficients are of importance to the experimenter. For example, in an economics application, $\beta_1, \beta_2, \ldots$ might have some particular significance, and then confidence intervals and tests of hypotheses on these parameters would be of interest to the economist.

The experimenter using regression analysis is also interested in deletion of variables when the situation dictates that, in addition to arriving at a workable prediction equation, he or she must find the "best regression" involving only variables that are useful predictors.

One criterion that is commonly used to illustrate the adequacy of a fitted regression model is the coefficient of determination, R^2.

Definition 4.84: R^2

The formula is

$$R^2 = \frac{SSR}{SST} = 1 - \frac{SSE}{SST}$$

The regression sum of squares can be used to give some indication concerning whether or not the model is an adequate explanation of the true situation. We can test the hypothesis H_0 that the regression is not significant by merely forming the ratio

$$f = \frac{SSR/k}{SSE/(n-k-1)} = \frac{SSR/k}{s^2} = \frac{MSR}{MSE}$$

and rejecting H_0 at the α-level of significance when $f > f_\alpha(k, n-k-1)$.

[SI*] **The Adjusted Coefficient of Determination R^2_{adj}**

Adjusted R^2 is a variation on R^2 that provides an adjustment for degrees of freedom. The coefficient of determination cannot decrease as terms are added to the model. In other words, R^2 does not decrease as the error degrees of freedom $n-k-1$ are reduced, the latter result being produced by an increase in k, the number of model terms.

Definition 4.85: Adjusted R^2

The formula is

$$R^2_{\text{adj}} = 1 - \frac{SSE/(n-k-1)}{SST/(n-1)}$$

vi. Compute Multiple Linear Regression with SymIntegration

A study was done on a diesel-powered light-duty pickup truck to see if humidity, air temperature, and barometric pressure influence emission of nitrous oxide (in ppm). Emission measurements were taken at different times, with varying experimental conditions. The data are given in table below. The model is

$$\mu_{Y|x_1,x_2,x_3} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3$$

or, equivalently,

$$y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \beta_3 x_{3i} + \epsilon_i, \quad i = 1, 2, \ldots, 20$$

Fit this multiple linear regression model to the given data and then estimate the amount of nitrous oxide emitted for the conditions where humidity is 50%, temperature is 76^0 F, and barometric pressure is 29.30.

Nitrous Oxide, y	Humidity, x_1	Temp., x_2	Pressure, x_3	Nitrous Oxide, y	Humidity x_1	Temp., x_2	Pressure, x_3
0.90	72.4	76.3	29.18	1.07	23.2	76.8	29.38
0.91	41.6	70.3	29.35	0.94	47.4	86.6	29.35
0.96	34.3	77.1	29.24	1.10	31.5	76.9	29.63
0.89	35.1	68.0	29.27	1.10	10.6	86.3	29.56
1.00	10.7	79.0	29.78	1.10	11.2	86.0	29.48
1.10	12.9	67.4	29.39	0.91	73.3	76.3	29.40
1.15	8.3	66.8	29.69	0.87	75.4	77.9	29.28
1.03	20.1	76.9	29.48	0.78	96.6	78.7	29.29
0.77	72.2	77.7	29.09	0.82	107.4	86.8	29.03
1.07	24.0	67.7	29.60	0.95	54.9	70.9	29.37

Table 4.19: *Source: Charles T. Hare,"Light-Duty Diesel Emission Correction Factors for Ambient Conditions," EPA-600/2-77-116. U.S. Environmental Protection Agency.*

Solution:

The solution of the set of estimating equations yields the unique estimates

$$b_0 = -3.507778$$
$$b_1 = -0.002625$$
$$b_2 = 0.000799$$
$$b_3 = 0.154155$$

Therefore, the regression equation is

$$\hat{y} = -3.507778 - 0.002625x_1 + 0.000799x_2 + 0.154155x_3$$

For 50% humidity, a temperature of 76^0 F, and a barometric pressure of 29.30, the estimated amount of nitrous oxide emitted is

$$\begin{aligned}\hat{y} &= -3.507778 - 0.002625(50) + 0.000799(76) + 0.154155(29.30) \\ &= 0.9384\end{aligned}$$

In SymIntegration, to compute the multiple regression model' coefficients we can use this function:
multipleregression(vector<vector<double>> &X, vector<vector<double>> &y)

with X as the matrix of independent variables, including a column if 1 as the first column for the intercept, and y is the vector of the dependent variable. The computation requires inverse operation, thus it is a bit expensive on the computational side since computing inverse of a matrix requires some works before, such as computing the determinant of the corresponding matrix and the adjugate matrix as well.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Multiple Linear
Regression ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Multiple Linear Regression ]# ./main

Matrix X :
               1.000000             72.400000             76.300000             29.180000
               1.000000             41.600000             70.300000             29.350000
               1.000000             34.300000             77.100000             29.240000
               1.000000             35.100000             68.000000             29.270000
               1.000000             10.700000             79.000000             29.780000
               1.000000             12.900000             67.400000             29.390000
               1.000000              8.300000             66.800000             29.690000
               1.000000             20.100000             76.900000             29.480000
               1.000000             72.200000             77.700000             29.090000
               1.000000             24.000000             67.700000             29.600000
               1.000000             23.200000             76.800000             29.380000
               1.000000             47.400000             86.600000             29.350000
               1.000000             31.500000             76.900000             29.630000
               1.000000             10.600000             86.300000             29.560000
               1.000000             11.200000             86.000000             29.480000
               1.000000             73.300000             76.300000             29.400000
               1.000000             75.400000             77.900000             29.280000
               1.000000             96.600000             78.700000             29.290000
               1.000000            107.400000             86.800000             29.030000
               1.000000             54.900000             70.900000             29.370000

Vector y :
               0.900000
               0.910000
               0.960000
               0.890000
               1.000000
               1.100000
               1.150000
               1.030000
               0.770000
               1.070000
               1.070000
               0.940000
               1.100000
               1.100000
               1.100000
               0.910000
               0.870000
               0.780000
               0.820000
               0.950000
```

Figure 4.55: *The computation of the multiple regression model with SymIntegration (**SymIntegration/Examples/Statistics/Test SymIntegration Multiple Linear Regression/main.cpp**).*

```
Multiple regression with matrix algebra:

b[0] = -3.507778
b[1] = -0.002625
b[2] = 0.000799
b[3] = 0.154155

(X^{T} X)^{-1} X^{T} y:
              -3.507778
              -0.002625
               0.000799
               0.154155

y                               y estimated
0.900000                        0.861376
0.910000                        0.963638
0.960000                        0.971276
0.890000                        0.966530
1.000000                        1.117988
1.100000                        1.042824
1.150000                        1.100667
1.030000                        1.045388
0.770000                        0.849145
1.070000                        1.046299
1.070000                        1.021756
0.940000                        0.961436
1.100000                        1.038587
1.100000                        1.090168
1.100000                        1.076021
0.910000                        0.892927
0.870000                        0.870194
0.780000                        0.816725
0.820000                        0.754766
0.950000                        0.932288

For 50% humidity, 76^{0} F and 29.30 barometric pressure, the estimated multiple regression is:
0.938434

Time taken by function: 6299 microseconds
```

Figure 4.56: *The computation of the multiple regression model with SymIntegration (**SymIntegration/Examples/Statistics/Test SymIntegration Multiple Linear Regression/main.cpp**).*

vii. Compute Inferences in Multiple Linear Regression with SymIntegration

The percent survival rate of sperm in a certain type of animal semen, after storage, was measured at various combinations of concentrations of three materials used to increase chance of survival. The data are given in table below. Test the hypothesis that $\beta_2 = -2.5$ at the 0.05 level of significance against the alternative that $\beta_2 > -2.5$. Then construct a 95% confidence interval for the mean response and the prediction interval for y_0.

y (% survival)	x_1 (weight %)	x_2 (weight %)	x_3 (weight %)
25.5	1.74	5.30	10.80
31.2	6.32	5.42	9.40
25.9	6.22	8.41	7.20
38.4	10.52	4.63	8.50
18.4	1.19	11.60	9.40
26.7	1.22	5.85	9.90
26.4	4.10	6.62	8.00
25.9	6.32	8.72	9.10
32.0	4.08	4.42	8.70
25.2	4.15	7.60	9.20
39.7	10.15	4.83	9.40
35.7	1.72	3.12	7.60
26.5	1.70	5.30	8.20

Table 4.20: *Data for survival rate of sperm.*

Solution:

$$H_0 : \beta_2 = -2.5$$
$$H_1 : \beta_2 > -2.5$$

Computations:

$$t = \frac{b_2 - \beta_{20}}{s\sqrt{c_{22}}} = \frac{-1.8618 + 2.5}{2.073\sqrt{0.0166}} = 2.390$$
$$P = P(T > 2.390) = 0.04$$

The signal that favors H_1 comes from large values of t. The P-value corresponding to $t = 2.390$ computed by the formula $P(T > 2.390)$ is coming from $H_1 : \beta_2 > -2.5$, since the sign is $>$ in H_1 thus we need to compute the P-value with $>$ sign.

Decision: Reject H_0 and conclude that $\beta_2 > -2.5$.

In SymIntegration to compute the estimation of the multiple linear regression model along with the ANOVA table and inferences in multiple linear regression we can use this function:
multipleregressionfull(vector<vector<double>> &X, vector<vector<double>> &y, double α)

with X as the vector $\boldsymbol{X}$ as the matrix of independent variables as in the General Linear Model $\boldsymbol{y} = \boldsymbol{X\beta} + \boldsymbol{\epsilon}$, $\boldsymbol{y}$ is the vector of the dependent variable, and α is the level of significance for computing inferences in multiple linear regression.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Multiple Linear Regression Animal Semen ]# ./main

Matrix X :
              1.000000              1.740000              5.300000             10.800000
              1.000000              6.320000              5.420000              9.400000
              1.000000              6.220000              8.410000              7.200000
              1.000000             10.520000              4.630000              8.500000
              1.000000              1.190000             11.600000              9.400000
              1.000000              1.220000              5.850000              9.900000
              1.000000              4.100000              6.620000              8.000000
              1.000000              6.320000              8.720000              9.100000
              1.000000              4.088000              4.420000              8.700000
              1.000000              4.150000              7.600000              9.200000
              1.000000             10.150000              4.830000              9.400000
              1.000000              1.720000              3.120000              7.600000
              1.000000              1.700000              5.300000              8.200000

Vector y :
             25.500000
             31.200000
             25.900000
             38.400000
             18.400000
             26.700000
             26.400000
             25.900000
             32.000000
             25.200000
             39.700000
             35.700000
             26.500000
Multiple regression with matrix algebra:

b[0] = 39.153886
b[1] = 1.016159
b[2] = -1.861393
b[3] = -0.343152

y = 39.153886 +  1.016159 x1 +  -1.861393 x2 +  -0.343152 x3
```

Figure 4.57: *The computation of the multiple regression model with ANOVA and inferences on mean response and prediction (**SymIntegration/Examples/Statistics/Test SymIntegration Multiple Linear Regression Animal Semen/main.cpp**).*

```
**********************************************************

Analysis of Variance

Source                  DF             SS                    MS                    F                     P-value
Regression              3              399.452877            133.150959            30.983038             0.000045
Residual Error          9              38.677892             4.297544
Total                   12             438.130769

Rejection of H0 implies that the regression equation differs from a constant


R-Square             Grand Mean              Root MSE
0.911721              29.038462              2.073052

Variable        DF              Parameter Estimate              Standard Error                  t value               P-value
Intercept       1               39.153886                       5.887382                        6.650475              0.000047
x1              1               1.016159                        0.190911                        5.322683              0.000240
x2              1               -1.861393                       0.267338                        -6.962706             0.000033
x3              1               -0.343152                       0.617067                        -0.556103             0.295848

Obs     y_data                  Fit                     SE Fit                  95% CI                          95% PI                          Residual
1       25.500000               27.816643               3.201540                (24.615103 , 31.018182)         (22.138446 , 33.494839)         -2.316643
2       31.200000               32.404840               1.774690                (30.630150 , 34.179530)         (27.390702 , 37.418978)         -1.204840
3       25.900000               27.779020               3.073862                (24.705158 , 30.852882)         (22.171820 , 33.386219)         -1.879020
4       38.400000               38.691119               2.899519                (35.791600 , 41.590638)         (33.177566 , 44.204672)         -0.291119
5       18.400000               16.125418               3.571770                (12.553647 , 19.697188)         (10.230539 , 22.020297)         2.274582
6       26.700000               26.356941               2.343384                (24.013558 , 28.700325)         (21.114470 , 31.599412)         0.343059
7       26.400000               28.404945               1.831029                (26.573916 , 30.235973)         (23.370590 , 33.439299)         -2.004945
8       25.900000               26.442355               2.201496                (24.240859 , 28.643850)         (21.261752 , 31.622957)         -0.542355
9       32.000000               32.237628               1.770072                (30.467556 , 34.007699)         (27.225123 , 37.250133)         -0.237628
10      25.200000               26.178759               1.565245                (24.613514 , 27.744003)         (21.234869 , 31.122649)         -0.978759
11      39.700000               37.649205               2.956527                (34.692677 , 40.605732)         (32.105460 , 43.192950)         2.050795
12      35.700000               32.985589               3.314084                (29.671504 , 36.299673)         (27.243184 , 38.727993)         2.714411
13      26.500000               28.427540               2.226526                (26.201014 , 30.654067)         (23.236252 , 33.618829)         -1.927540

Time taken by function: 5086 microseconds
```

Figure 4.58: *The computation of the multiple regression model with ANOVA and inferences on mean response and prediction (**SymIntegration/Examples/Statistics/Test SymIntegration Multiple Linear Regression Animal Semen/main.cpp**).*

The f-test in the analysis of variance indicates that a significant amount of variability is explained. The R^2 value of 0.9117 implies that the model explains 91.17% of the variability in the response.

Note the parameter estimates, the standard errors, and the t-statistics shown in the output. The standard errors are computed from square roots of diagonal elements of $(\boldsymbol{X}^T\boldsymbol{X})^{-1}s^2$, $s^2 = MSE = RMSE^2$.

In this illustration, the variable x_3 is insignificant in the presence of x_1 and x_2 based on the t-test and the corresponding P-value of 0.295848.

The formula to compute the t-value for the variable intercept, x_1, x_2, x_3 is based from the hypothesis test

$$H_0 : \beta_i = 0$$
$$H_1 : \beta_i > 0$$

we then use this formula:

$$t = \frac{b_i - \beta_{i0}}{s\sqrt{c_{ii}}}$$

with $\beta_{i0} = 0$, s is the RMSE (Root Mean Square Error), and c_{ii} is the diagonal entry of the matrix $(\boldsymbol{X}^T\boldsymbol{X})^{-1}$.

The P-value itself can be computed with

$$P(T > |t|)$$

Take a look at the P-value at the analysis of variance (ANOVA) table, it means that we reject H_0 with

$$H_0 : \beta_1 = \beta_2 = \beta_3 = \cdots = \beta_k = 0$$

This should not be misinterpreted. Although it does indicate that the regression explained by the model is significant, this does not rule out the following possibilities:

1. The linear regression model for this set of x's is not the only model that can be used to explain the data; indeed, there may be other models with transformations on the x's that give a larger value of the F-statistic.

2. The model might have been more effective with the inclusion of other variables in addition to x_1, x_2, and x_3 or perhaps with the deletion of one or more of the variables in the model, say x_3, which has a $P = 0.2958$.

Regarding the pitfalls in the use of R^2 as a criterion for comparing competing models. These pitfalls are certainly relevant in multiple linear regression. In fact, in its employment in multiple regression, the dangers are even more pronounced since the temptation to overfit is so great. One should always keep in mind that $R^2 \approx 1.0$ can always be achieved at the expense of error degrees of freedom when an excess od model terms is employed. However, $R^2 = 1$, describing a model with a near perfect fit, does not always result in a model that predicts well.

Chapter 5

Operations Research Computation with SymIntegration

"Without mathematics, there's nothing you can do. Everything around you is mathematics. Everything around you is numbers." - ***Shakuntala Devi***

Operations research is a branch of mathematics which is concerned with the application of scientific methods and techniques to decision making problems and with establishing the best or optimal solutions. Computers and mathematical algorithms have impacted our lives in many ways. In many engineering disciplines, it has become common to perform simulations for the rational design and the optimization of all kinds of devices and processes. Simulations can be much cheaper than performing many experiments and they provide theoretical and quantitative insights. Examples are airplanes, combustion engines, antennas, and the construction of bridges and other buildings.

The optimum seeking methods are also known as mathematical programming techniques and are generally studied as a part of operations research.

Methods of operations research:

- Mathematical programming techniques
    1. Calculus methods
    2. Calculus of variations
    3. Nonlinear programming
    4. Geometric programming
    5. Quadratic programming
    6. Linear programming
    7. Dynamic programming
    8. Integer programming
    9. Stochastic programming
    10. Separable programming
    11. Multiobjective programming

12. Network methods: CPM and PERT
13. Game theory

- Stochastic process techniques
  1. Statistical decision theory
  2. Markov processes
  3. Queueing theory
  4. Renewal theory
  5. Simulation methods
  6. Reliability theory
- Statistical methods
  1. Regression analysis
  2. Cluster analysis, pattern recognition
  3. Design of experiments
  4. Discriminate analysis (factor analysis)

I. Linear Programming

[SI*] Linear programming is concerned with the optimization (minimization or maximization) of a linear function while satisfying a set of linear equality and/or inequality constraints or restrictions. The linear programming problem was first conceived by George B. Dantzig around 1947 while he was working as a Mathematical Advisor to the United States Air Force Comptroller on developing a mechanized planning tool for a time-staged deployment, training, and logistical supply program.

In 1949, George B. Dantzig published the "simplex method" for solving linear programs. Since that time a number of individuals have contributed to the field of linear programming in many different ways, including theoretical development, computational aspects, and exploration of new applications of the subject [4].

[SI*] The standard form of linear programming in matrix form:
Minimize

$$\boldsymbol{c}^T\boldsymbol{x} \tag{5.1}$$

subject to the constraints

$$A\boldsymbol{x} = \boldsymbol{b} \tag{5.2}$$

and

$$\boldsymbol{x} \geq \boldsymbol{0} \tag{5.3}$$

where

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad \boldsymbol{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}, \quad \boldsymbol{c} = \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{bmatrix}$$

$$A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}$$

It can be seen that there are m equations and n decision variables in a linear programming problem.

We may assume that $m < n$, for if $m > n$ there would be $m - n$ redundant equations which could be eliminated (it is the basic knowledge of linear algebra, read more here [29]). The case where $m = n$ is of no interest, for then there is either a unique solution x which satisfies Eqs. (5.2) and (5.3) (in which case there can be no optimization) or no solution, in which case the constraints are inconsistent.

the case $m < n$ corresponds to an underdetermined set of linear equations which, if they have one solution, have an infinite number of solutions. The problem of linear programming is to find one of these solutions satisfying Eqs.(5.2) and (8.3) and yielding the minimum of f.

[SI*] A linear programming problem may have

1. A unique and finite optimum solution.
2. An infinite number of optimal solutions.
3. An unbounded solution.
4. No solution.
5. A unique feasible point.

Assuming that the linear programming is properly formulated, the following general geometrical characteristics can be noted from the graphical solution.

1. The feasible region is a convex polygon.
2. The optimum value occurs at an extreme point or vertex of the feasible region.

[SI*] To represent an optimization problem as a linear program, several assumptions that are implicit in the linear programming formulation are needed, they are:

1. **Proportionality**
   Given a variable x_j, its contribution to cost is $c_j x_j$ and its contribution to the ith constraint is $a_{ij} x_j$. This means that if x_j is doubled, so is its contribution to cost and to each of the constraints.
2. **Additivity**
   This assumption guarantees that the total cost is the sum of the individual costs, and that the total contribution to the ith restriction is the sum of the individual contributions of the individual activities. In other words, there are no substitution or interaction effects among the activities.
3. **Divisibility**
   This assumption ensures that the decision variables can be divided into any fractional levels so that noninteger values for the decision variables are permitted.
4. **Deterministic**
   The coefficients c_j, a_{ij}, and b_i are all known deterministically. Any probabilistic or stochastic element inherent in demand, costs, prices, resource availabilities, usages, and so on are all assumed to be approximated by these coefficients through some deterministic equivalent.

It is important to recognize that if a linear programming problem is being used to model a given situation, then the aforementioned assumptions are implied to hold, at least over some anticipated operating range for the activities.

Despite the seemingly restrictive assumptions, linear programs are among the most widely used models today. They represent several systems quite satisfactorily, and they are capable of providing a large amount of information besides simply a solution. Moreover, they are also often used to solve certain types of nonlinear optimization problems via (successive) linear approximations and constitute an important tool in solution methods for linear discrete optimization problems having integer restricted variables.

[SI*] **Problem Manipulation**

Recall that a linear program is a problem of minimizing or maximizing a linear function in the presence of linear inequality and/or equality constraints. By simple manipulation the problem can be transformed from one form to another equivalent form.

1. **Inequalities and Equations**
   An inequality can be easily transformed into an equation. To illustrate, consider the constraint given by
   $$\sum_{j=1}^{n} a_{ij}x_j \geq b_i$$
   this constraint can be put in an equation form by subtracting the nonnegative surplus or slack variable x_{n+i} (sometimes denoted by s_i)leading to
   $$\sum_{j=1}^{n} a_{ij}x_j - x_{n+i} = b_i$$
   and
   $$x_{n+i} \geq 0$$
   Similarly, the constraint
   $$\sum_{j=1}^{n} a_{ij}x_j \leq b_i$$
   is equivalent to
   $$\sum_{j=1}^{n} a_{ij}x_j + x_{n+i} = b_i$$
   and
   $$x_{n+i} \geq 0$$
2. **Standard Form of a Linear Programming Problem**
   The general linear programming problem can be stated in the following standard form
   - In scalar form.
     Minimize
     $$f(x_1, x_2, \ldots, x_n) = c_1x_1 + c_2x_2 + \cdots + c_nx_n \tag{5.4}$$
     subject to the constraints
     $$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \\ &\vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n &= b_m \end{aligned} \tag{5.5}$$

and

$$\begin{aligned} x_1 &\geq 0 \\ x_2 &\geq 0 \\ &\vdots \\ x_n &\geq 0 \end{aligned} \tag{5.6}$$

- In compact form.
  Minimize

$$f(x_1, x_2, \ldots, x_n) = \sum_{j=1}^{n} c_j x_j \tag{5.7}$$

subject to the constraints

$$\sum_{j=1}^{n} a_{ij} x_j = b_i, \quad i = 1, 2, \ldots, m \tag{5.8}$$

and

$$x_j \geq 0, \quad j = 1, 2, \ldots, n \tag{5.9}$$

- In matrix form.
  Minimize

$$\boldsymbol{c}^T \boldsymbol{x} \tag{5.10}$$

subject to the constraints

$$A\boldsymbol{x} = \boldsymbol{b} \tag{5.11}$$

and

$$\boldsymbol{x} \geq \boldsymbol{0} \tag{5.12}$$

where

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad \boldsymbol{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}, \quad \boldsymbol{c} = \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{bmatrix}$$

$$A = \begin{bmatrix} a_{11} & a_{12} & \ldots & a_{1n} \\ a_{21} & a_{22} & \ldots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \ldots & a_{mn} \end{bmatrix}$$

It can be seen that there are m equations and n decision variables in a linear programming problem, and the superscript T is used to indicate the transpose. In some linear programming books, e.g. [35], vector $\boldsymbol{x}$ is written with uppercase $\boldsymbol{X}$ which is usually confusing for those who are used to use lowercase for vectors only, and uppercase for matrix. In this book vectors will be denoted always with lowercase either with rightarrow ($\vec{x}$) or bolded ($\boldsymbol{x}$), while a matrix will be denoted with big uppercase without rightarrow or being bolded and will always be written first that it is a matrix.

[SI*] **Definitions and Theorems**

Some of the more poweful methods for solving linear programming problems take advantage of these characteristics.

1. Point in n-dimensional space. A point $\boldsymbol{x}$ in an n-dimensional space is characterized by an ordered set of n values of coordinates $(x_1, x_2, \ldots, x_n)$. The coordinates of $\boldsymbol{x}$ are also called the components of $\boldsymbol{x}$.

2. Line segment in n-dimensions (L). If the coordinates of two points A and B are given by $x_j^{(1)}$ and $x_j^{(2)}$ $(j = 1, 2, \ldots, n)$, the line segment (L) joining these points is the collection of points $\boldsymbol{x}(\lambda)$ whose coordinates are given by

$$x_j = \lambda x_j^{(1)} + (1 - \lambda) x_j^{(2)}, \quad j = 1, 2, \ldots, n$$

where $0 \leq \lambda \leq 1$.

Thus

$$L = \{\boldsymbol{x} | \boldsymbol{x} = \lambda \boldsymbol{x}^{(1)} + (1 - \lambda)\boldsymbol{x}^{(2)}\} \tag{5.13}$$

In one dimension, for example, it is easy to see that the definition is in accordance with our experience

$$x(\lambda) - x^{(1)} = \lambda(x^{(2)} - x^{(1)}), \quad 0 \leq \lambda \leq 1 \tag{5.14}$$

hence

$$x(\lambda) = (1 - \lambda)x^{(1)} + \lambda x^{(2)} \tag{5.15}$$

3. In n-dimensional space, the set of points coordinates satisfy a linear equation

$$a_1 x_1 + a_2 x_2 + \cdots + a_n x_n = \boldsymbol{a}^T \boldsymbol{x} = b \tag{5.16}$$

is called a hyperplane.

Thus, the hyperplane H is given by

$$H(A, b) = \{\boldsymbol{x} | A^T \boldsymbol{x} = b\} \tag{5.17}$$

A hyperplane has $(n - 1)$ dimensions in n-dimensional space. For example, in a 3-dimensional space, it is a plane and in 2-dimensional space, it is a line. The set of points whose coordinates satisfy a linear inequality like $a_1 x_1 + \cdots + a_n x_n \leq b$ is called a closed half space, closed due to the inclusion of equality sign in the above inequality.

A hyperplane partitions E^n into two closed half-spaces so that

$$H^+ = \{\boldsymbol{x} | A^T \boldsymbol{x} \geq b\} \tag{5.18}$$

and

$$H^- = \{\boldsymbol{x} | A^T \boldsymbol{x} \leq b\} \tag{5.19}$$

4. Convex set is a collection of points such that if $\boldsymbol{x}^{(1)}$ and $\boldsymbol{x}^{(2)}$ are any two points in the collection, the line segment joining them is also in the collection. If S denotes the convex set, it can be defined mathematically as follows:

$$\text{if } \boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)} \in S, \text{ then } \boldsymbol{x} \in S$$

where

$$\boldsymbol{x} = \alpha \boldsymbol{x}^{(2)} + (1 - \alpha)\boldsymbol{x}^{(1)}, \quad 0 \leq \alpha \leq 1$$

As an assumption, the set containing only one point is convex.

A convex polygon consists of a set of points having the property that the line segment joining any two points in the set is entirely in the convex set. In problems having more than two decision variables, the feasible region is called a convex polyhedron.

5. Convex polyhedron is a set of points common to one or more half-spaces. In particular, a convex polygon is the intersection of one or more half planes.
6. Vertex is a point in the convex set which does not lie on a line segment joining two toher points of the set.
   Thus, for example, every point on the circumference of a circle and each corner point of a polygon can be called a vertex or extreme point.
7. Feasible solution is any solution in linear programming problem that satisfies the constraints

$$A\boldsymbol{x} = \boldsymbol{b} \tag{5.20}$$

   and

$$\boldsymbol{x} \geq \boldsymbol{0} \tag{5.21}$$

8. Basic solution is a solution in which $(n - m)$ variables are set equal to zero.
   The basic solution can be obtained by setting $(n - m)$ variables to zero and solving the constraint Eqs. (5.5) simultaneously.
9. Basis is the collection of variables that are not set equal to zero to obtain the basic solution.
10. Basic feasible solution is a basic solution which satisfies the nonnegativity conditions of Eqs. (5.6).
11. Nondegenerate basic feasible solution is a basic feasible solution which has got exactly m positive x_i.
12. Optimal solution is a feasible solution which optimizes the objective function.
13. Optimal basic solution is a basic feasible solution for which the objective function is optimal.

Theorem 5.1: Intersection of Convex Sets

The intersection of any number of convex sets is also convex.

Theorem 5.2: Feasible Region in Linear Programming

The feasible region of a linear programming problem is convex.

Theorem 5.3: Local Minimum in Linear Programming

Any local minimum solution is global for a linear programming problem.

Theorem 5.4: Basic Feasible Solution in Linear Programming

Every basic feasible solution is an extreme point of the convex set of feasible solutions.

Theorem 5.5: Closed and Bounded Convex Polyhedron

Let S be a closed, bounded convex polyhedron with $\boldsymbol{x}_{i^e}$, $i = 1$ to p as the set of its extreme points. Then any vector $\boldsymbol{x} \in S$ can be written as

$$\begin{aligned} \boldsymbol{x} &= \sum_{i=1}^{p} \lambda_i \boldsymbol{x}_{i^e} \\ \lambda_i &\geq 0 \\ \sum_{i=1}^{p} \lambda_i &= 1 \end{aligned} \tag{5.22}$$

Theorem 5.6: Closed Convex Polyhedron

Let S be a closed convex polyhedron. Then the minimum of a linear function over S is attained at an extreme point of S.

$$\begin{aligned} \boldsymbol{x} &= \sum_{i=1}^{p} \lambda_i \boldsymbol{x}_{i^e} \\ \lambda_i &\geq 0 \\ \sum_{i=1}^{p} \lambda_i &= 1 \end{aligned} \tag{5.23}$$

For the proof of each theorem above, you can read it here [35].

i. Simplex Method

Find the amount of each type of food to be purchased in order to meet exactly the daily requirements of a person at minimum cost. Assume that a person, on the average, requires 3000 calories and 100 grams of protein.

	Bread	Meat	Potatoes	Cabbage	Milk
Calories	2500	3000	600	100	600
Protein $(grams)$	80	150	20	10	40
Cost (USD/kg)	3	10	1	2	3

Table 5.1: *The calorie values and the protein contents of five types of foods along with their costs.*

Solution:

Let x_1, x_2, x_3, x_4, and x_5 denote the amount of bread, meat, potatoes, cabbage, and milk to be purchased in kg per day. Then the objective function to be minimized is given by

$$f = 3x_1 + 10x_2 + x_3 + 2x_4 + 3x_5 \tag{5.24}$$

The constraints due to the calorie and protein requirements are respectively given by

$$\begin{aligned} 2500x_1 + 3000x_2 + 600x_3 + 100x_4 + 600x_5 &= 3000 \\ 80x_1 + 150x_2 + 20x_3 + 10x_4 + 40x_5 &= 100 \end{aligned} \tag{5.25}$$

The nonnegativity requirements of the decision variables are given by

$$x_i \geq 0, \quad i = 1, 2, 3, 4, 5 \tag{5.26}$$

Since $m = 2$ (the number of equality constraints) and $n = 5$ (the number of decision variables), a basic solution can be obtained by setting any of the $(n - m)$ variables equal to zero and solving the constraint equations. The number of basic solutions is

$$\frac{5!}{3!2!} = 10$$

Out of these ten basic solutions, the solution which has all the variables nonnegative and makes the objective function given by Eq. (5.4) assume a minimum value will be the optimum.

The simplex computation makes use of Gaussian elimination to compute the values of the basic variables and the objective function for all basic solutions.

This is the initial simplex tableau:

$$S = \begin{bmatrix} x_1 & x_2 & x_3 & x_4 & x_5 & f & C \\ 2500 & 3000 & 600 & 100 & 600 & 0 & 3000 \\ 80 & 150 & 20 & 10 & 40 & 0 & 100 \\ -3 & -10 & -1 & -2 & -3 & 1 & 0 \end{bmatrix} \tag{5.27}$$

Thus we make this matrix A that represents the constraints equations only:

$$A = \begin{bmatrix} 2500 & 3000 & 600 & 100 & 600 & 0 & 3000 \\ 80 & 150 & 20 & 10 & 40 & 0 & 100 \end{bmatrix} \tag{5.28}$$

The matrix A is the one that will be the input for the C++ codes, the objective function will be inputted as a vector, to create the matrix A we make two options, the first one is loading the matrix from textfile (**matrix.txt**) and the second one is to input the coefficients manually in the C++ source code (**main.cpp**).

To load a **std::vector** from a text file in C++, we need to include **<fstream>** for file operations, **<vector>** for using vector, **<string>** for reading lines, and **<iostream>** for input/output.

The loaded vector will then be saved as Symbolic matrix **B_mat** that will be used as the input to do the simplex computation.

To do the simplex computation we use this function in SymIntegration:
Symbolic simplexmethod(const SymbolicMatrix &A, vector<double> f, int C, int R, int n)

with A as the SymbolicMatrix that represent the constraint equations that we have defined in Eq. (5.8), f as the vector of objective function, C as the number of column for matrix A, R as the number of row for matrix A, and n is the number of decision variables.

```
xterm
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Simplex Method Function ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Simplex Method Function ]# ./main
Simplex Method :
A :
[2500 3000  600  100  600   0  3000]
[ 80   150  20   10   40    0   100]


**********Basic Variables**********
************************************************************
 0 1
************************************************************

My wife is the most beautiful Goddess, she teaches me this !
A :
[2500 3000   0    0    0    0  3000]
[ 80   150   0    0    0    0   100]

A (in reduced row form) :
[2500 3000   0    0    0    0  3000]
[  0   54    0    0    0    0    4 ]

A (in row reduced echelon form) :
[    1        1.2        0         0         0         0        1.2   ]
[    0         1         0         0         0         0     0.0740741]

Simplified final matrix :
[    1        1.2       1.2   ]
[    0         1     0.0740741]

Solution:
x_{1} = 1.11111
x_{2} = 0.0740741

Value of the objective function : 4.07407

**********Basic Variables**********
************************************************************
 0 2
************************************************************

My wife is the most beautiful Goddess, she teaches me this !
A :
[2500   0   600   0    0    0  3000]
[ 80    0   20    0    0    0   100]

A (in reduced row form) :
[2500   0   600   0    0    0  3000]
[  0    0   0.8   0    0    0    4 ]
```

Figure 5.1: *The function in SymIntegration to compute the linear programming problem with simplex method (**SymIntegration/Examples/Operations Research/Test SymIntegration Simplex Method Function/main.cpp**).*

```
xterm
x_{5} = -5.55112e-16

Value of the objective function : 5

********************Basic Variables**********************
************************************************************
 3 4
************************************************************

My wife is the most beautiful Goddess, she teaches me this !
A :
[  0    0    0   100  600   0  3000]
[  0    0    0   10   40    0   100]

A (in reduced row form) :
[  0    0    0   100  600   0  3000]
[  0    0    0   10   40    0   100]

A (in row reduced echelon form) :
[    0         0         0     0.0333333    0.2        0         1    ]
[    0         0         0        0.1       0.4        0         1    ]

Simplified final matrix :
[0.0333333    0.2        1    ]
[   0.1       0.4        1    ]

Solution:
x_{4} = -30
x_{5} = 10

Value of the objective function : -30

************************************************************
********************End of Simplex Method*******************
************************************************************

Objective Function Value               Decision Variables
4.1                                     0   1
5                                      0   2
4.7                                     0   3
4                                      0   4
5                                      1   2
-6.7                                      1   3
5                                      1   4
5                                      2   3
5                                      2   4
-30                                     3   4
0

Time taken by function: 51473 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Simplex Method Function ]#
```

Figure 5.2: *The function in SymIntegration to compute the linear programming problem with simplex method takes 51473 microseconds (**SymIntegration/Examples/Operations Research/Test SymIntegration Simplex Method Function/main.cpp**).*

Due to the C++ coding, the meaning of decision variable 0 is for x_1, 1 for x_2 and so on, from the result we know that the feasible and optimum solution occurs when we use basic variables 0 and 4, or, x_1 and x_5, referring to bread and milk. With the values for the basic variables:

$$x_1 = 1.15385$$
$$x_5 = 0.192308$$
$$f = 4.03846$$

In real life, you cannot really buy 0.192308 kg of milk at that exact decimal value, so we usually round it to the nearest possible value that we can use in real life / in the market.

For comparison, we also try to use **Armadillo** to handle the Gaussian elimination part, but it becomes slower because we need to add another library when compiling.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Simplex Method Function ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration -larmadillo
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Simplex Method Function ]# ./main
Simplex Method :
A :
[2500 3000  600  100  600   0  3000]
[ 80   150  20   10   40    0   100]


**********Basic Variables**********
************************************************************
 0 1
************************************************************

My wife is the most beautiful Goddess, she teaches me this !
A :
[2500 3000   0    0    0    0  3000]
[ 80   150   0    0    0    0   100]

A (in reduced row form) :
[2500 3000   0    0    0    0  3000]
[  0   54    0    0    0    0    4 ]

A (in row reduced echelon form) :
[    1        1.2        0         0         0         0        1.2   ]
[    0         1         0         0         0         0     0.0740741]

Simplified final matrix :
[    1        1.2       1.2   ]
[    0         1     0.0740741]

Solution:
x_{1} = 1.11111
x_{2} = 0.0740741

Value of the objective function : 4.07407
```

Figure 5.3: *Comparing computing with simplex method with SymIntegration and Armadillo (**SymIntegration/Examples/Operations Research/Test SymIntegration Simplex Method Function/main.cpp**).*

```
**********Basic Variables**********
************************************************************
 3 4
************************************************************

My wife is the most beautiful Goddess, she teaches me this !
A :
[  0    0    0   100  600   0  3000]
[  0    0    0   10   40    0   100]

A (in reduced row form) :
[  0    0    0   100  600   0  3000]
[  0    0    0   10   40    0   100]

A (in row reduced echelon form) :
[    0         0         0     0.0333333    0.2        0         1    ]
[    0         0         0         0.1      0.4        0         1    ]

Simplified final matrix :
[0.0333333    0.2       1    ]
[   0.1       0.4       1    ]

Solution:
x_{4} = -30
x_{5} = 10

Value of the objective function : -30

************************************************************
**********End of Simplex Method**********
************************************************************

Objective Function Value               Decision Variables
4.1                                      0   1
5                                      0   2
4.7                                      0   3
4                                      0   4
5                                      1   2
-6.7                                       1   3
5                                      1   4
5                                      2   3
5                                      2   4
-30                                        3   4
0

Time taken by function: 81639 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Simplex Method Function ]#
```

Figure 5.4: *The function in SymIntegration to compute the linear programming problem with simplex method and Armadillo for the Gaussian elimination part takes 82639 microseconds (**SymIntegration/Examples/Operations Research/Test SymIntegration Simplex Method Function/main.cpp**).*

So we remove the dependencies toward **Armadillo** to make things easier and faster.

ii. Functions in SymIntegration Related to Linear Programming

[SI*] We will list the basic functions that can be used in linear programming with SymIntegration:

allCombinations(int n**, int** k**, int** *start_idx*, **vector<int>&** *current_comb*, **vector<vector<int>>&** *all_combs***)**
this void function returns the all the possible index combination of k from $n+1$ items, the *start_idx* is set at 0, since C++ has the first index set to 0. You have to declare empty vector<int> *current_comb* and empty vector<vector<int>> *all_combs* before run this void function.

contiguousCombinations(int n**, int** k**, int** *start_idx***)**
this function returns the contiguous index from combination for choosing k elements from $n+1$ elements, the *start_idx* is set at 0, since C++ has the first index set to 0.
For example for all contiguous combinations of 4 out of 6 elements we will have

0	1	2	3
0	1	2	5
0	1	4	5
0	3	4	5
1	2	3	4
2	3	4	5

discontiguousCombinations(int n**, int** k**, int** *start_idx***)**
this function returns the discontiguous index from combination for choosing k elements from $n+1$ elements, the *start_idx* is set at 0, since C++ has the first index set to 0.
For example for all discontiguous combinations of 4 out of 6 elements we will have

0	1	2	4
0	1	3	4
0	1	3	5
0	2	3	4
0	2	3	5
0	2	4	5
1	2	3	5
1	2	4	5
1	3	4	5

```
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Operations Research/Test SymIntegration Contiguous and Discontinuous Comb:
ion ]# ./main
All contiguous combinations of 4 from 6 items:
contiguous matrix:
                    0                   1                   2                   3
                    0                   1                   2                   5
                    0                   1                   4                   5
                    0                   3                   4                   5
                    1                   2                   3                   4
                    2                   3                   4                   5
All discontiguous combinations of 4 from 6 items:
discontiguous matrix:
                    0                   1                   2                   4
                    0                   1                   3                   4
                    0                   1                   3                   5
                    0                   2                   3                   4
                    0                   2                   3                   5
                    0                   2                   4                   5
                    1                   2                   3                   5
                    1                   2                   4                   5
                    1                   3                   4                   5

Time taken by function: 1285 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Operations Research/Test SymIntegration Contiguous and Discontinuous Comb:
```

Figure 5.5: *Generating contiguous and discontiguous index combinations for choosing 4 elements from 6 elements (**SymIntegration/Examples/Operations Research/Test SymIntegration Contiguous and Discontinuous Combination/main.cpp**).*

Symbolic simplexmethod(const SymbolicMatrix &A, vector<double> f, int C, int R, int n) this function will do simplex computation with A as the SymbolicMatrix that represent the constraint equations that we have defined in Eq. (5.8), f as the vector of objective function, C as the number of column for matrix A, R as the number of row for matrix A, and n is the number of decision variables.

Chapter 6

Numerical Linear Algebra

*"All the girls in the world were divided into two classes: one class included all the girls in the world except her, and they had all the usual human feelings and were very ordinary girls; while the other class -herself alone- had no weaknesses and was superior to all humanity." - **Anna Karenina (Leo Tolstoy)***

Why we create this chapter when we have created a book in 2023 [13] that contain a full summary of Elementary Linear Algebra? Because it is still using open source C++ library like **Eigen, Armadillo, GiNaC, SymbolicC++** that are created by someone else, we only learn to call the functions needed, but not deep enough. Thus starting from 2025 we are creating library for C++ that not only able to compute and solve problems in calculus, differential equations, linear programming, it grows and can also cover the field of linear algebra, so we are going to introduce a lot of features, functions in SymIntegration that can be used in Numerical Linear Algebra problems.

Instead of using open source library that already established, after branching out from **SymbolicC++**, SymIntegration can do a lot more, we learn more everyday, theoretically and practically, how to convert algorithms to a working C++ codes that can manage matrix multiplication, addition, LU decomposition, computing eigenvalues, and many more. If you want to read the full comprehensive summary read [13], in this book we will only cover the basic examples, and function in SymIntegration to solve Numerical Linear Algebra problems, there won't be theorems and explanations that are long here.

I. Solutions of Systems of Linear Equations

[SI*] Consider the nonhomogeneous linear system

$$\begin{array}{ccccccc} a_{11}x_1 & + a_{12}x_2 & + \ldots & + a_{1n}x_n & = & b_1 \\ a_{21}x_1 & + a_{22}x_2 & + \ldots & + a_{2n}x_n & = & b_2 \\ \vdots & \vdots & & \vdots & & \vdots \\ a_{m1}x_1 & + a_{m2}x_2 & + \ldots & + a_{mn}x_n & = & b_m \end{array} \tag{6.1}$$

it consists of m equations with n variables that need to be solved. We can also write it in matrix vector term like this:

$$A\boldsymbol{x} = \boldsymbol{b} \tag{6.2}$$

with $\boldsymbol{x}$ is a vector of size n and $\boldsymbol{b}$ is a vector of size m. The entire solution set of $A\boldsymbol{x} = \boldsymbol{b}$ can be obtained by translating the solution set of $A\boldsymbol{x} = \boldsymbol{0}$ by the vector $\boldsymbol{x}_0$.

When you have matrix of size $m \times n$, you just cannot apply Gaussian elimination directly to an $m \times n$ matrix problem. If you have more equations than unknowns ($m > n$), the your problem is overdetermined and you have no solution, which means you need to use something like the least squares method. If you have $A\boldsymbol{x} = \boldsymbol{b}$, then instead of having $\boldsymbol{x} = A^{-1}\boldsymbol{b}$ (when $n = m$), then you have to do

$$\boldsymbol{x} = (A^T A)^{-1} A^T \boldsymbol{b}$$

In the case where you have less equations then unknowns ($m < n$), then your problem is underdetermined and you have an infinity of solutions, like the general solution of homogeneous linear system. In that case, you either pick one at random (e.g. setting some of the unknowns to an arbitrary value as parameter), or you need to use regularization, which means trying to add some extra constraints.

For the case when we have Eq. (6.1) with $m = n$ we will use Gaussian elimination, but when the size of the square matrix is very large, it is better and faster to use numerical method like *LU*-decomposition.

i. Functions in SymIntegration Related to Basic Matrix-Vector Operations

[SI*] We will list the basic functions that can be used in matrix, vector operations and loading textfile with SymIntegration:
loadMatrixFromFile(const string& filename)
this function will reads a textfile and return **vector<vector<double>>** as the output, e.g. **vector<vector<double>> matrixA loadMatrixFromFile("matrix.txt")** is a declaration of data type of **vector<vector<double>>** with name **matrixA** and the data is taken from **matrix.txt** that is in the current working directory.

loadIntMatrixFromFile(const string& filename)
this function will reads a textfile and return **vector<vector<int>>** as the output.

loadComplexMatrixFromFile(const string& filename)
this function will reads a textfile and return **vector<vector<complex<double>>>** as the output.

loadVectorFromFile(const string& filename)
this function will reads a textfile and return **vector<double>** as the output.

loadIntVectorFromFile(const string& filename)
this function will reads a textfile and return **vector<int>** as the output.

loadComplexVectorFromFile(const string& filename)
this function will reads a textfile and return **vector<complex<double>>** as the output.

print3DMatrix(vector<vector<vector<double>>>)
this function will display / print the 3D matrix that has the data type of **double**, **vector<vector<vector<double>>>**.

printMatrix(vector<vector<double>>)
this function will display / print the matrix that has the data type of **double**, **vector<vector<double>>**.

printIntMatrix(vector<vector<int>>)
this function will display / print the matrix that has the data type of **integer**, **vector<vector<int>>**.

printVector(vector<double>)
this function will display / print a vector that has the data type of **double**, **vector<double>**.

printIntVector(vector<int>)
this function will display / print a vector that has the data type of **integer**, **vector<int>**.

vector<double> flattenDoubleMatrix(vector<vector<double>> matrix)
this function will flatten a matrix with **double** data type entries into a **vector<double>**.

vector<vector<vector<double>>> Create3DMatrixfromVector(vector<double> vector, int d**, int** r**, int** c**)**
this function will construct a 3D matrix with **double** (with size $d \times r \times c$) data type entries from a vector.

vector<vector<double>> matricizeDoubleVector(vector<double> vec, int R, int C)
this function will make a matrix of size $R \times C$ from a vector **vec** with size n, this condition has to be fulfilled: $n = R * C$.

save3DMatrixdouble(vector<vector<double>>, const string& filename)
this function save the 3D matrix / **vector<vector<vector<double>>>** of **double** data type with **filename** that we specify, e.g. "matrix.txt" or "matrix.csv."

save3DMatrixint(vector<vector<int>>, const string& filename)
this function save the 3D matrix / **vector<vector<vector<int>>>** of **integer** data type with **filename** that we specify, e.g. "matrix.txt" or "matrix.csv."

saveMatrixdouble(vector<vector<double>>, const string& filename)
this function save the matrix / **vector<vector<double>>** of **double** data type with **filename** that we specify, e.g. "matrix.txt" or "matrix.csv."

saveMatrixint(vector<vector<double>>, const string& filename)
this function save the matrix / **vector<vector<double>>** of **int** datatype with **filename** that we specify, e.g. "matrix.txt" or "matrix.csv."

saveVectordouble(vector<double>, const string& filename)
this function save the vector / **vector<double>** of **double** datatype with **filename** that we specify, e.g. "vec.txt" or "vec.csv."

saveVectorint(vector<int>, const string& filename)
this function save the vector / **vector<int>** of **integer** datatype with **filename** that we specify, e.g. "vec.txt" or "vec.csv."

saveVectorstring(vector<string>, const string& filename)
this function save the vector / **vector<string>** of **string** datatype with **filename** that we specify, e.g. "vec.txt" or "vec.csv."

addRow(vector<vector<double>> matrix, vector<double> vector_x, int k)
this function adds a row column (**vector_x**) into the *k*-throw in the matrix.

addColumn(vector<vector<double>> matrix, vector<double> vector_x, int k)
this function adds a vector column (**vector_x**) into the *k*-th column in the matrix.

deleteRow(vector<vector<double>>&, int)
this function deletes a certain row from a matrix.

deleteColumn(vector<vector<double>>&, int)
this function deletes a certain column from a matrix.

getRow(vector<vector<double>>, int)
this function extracts a certain row from a matrix.

getColumn(vector<vector<double>>, int)
this function extracts a certain column from a matrix.

The indices will be starting from 0 for the first row / first column, so if you want to take the first row you will type like this **getRow(matrixA,0)**, with **matrixA** is the matrix with **vector<vector<double>>** data type.

```
#include<bits/stdc++.h>
#include<iostream>
#include "symintegrationc++.h"
#include<vector>
#include <chrono>
#include <string>
#include <iomanip> // For std::setprecision
using namespace std::chrono;
using namespace std;

// Driver program
int main()
{
	vector<vector<double>> MatrixA = loadMatrixFromFile("matrix.txt");

	int R = MatrixA.size();
	int C = MatrixA[0].size();

	cout << "Column—1" << endl;
	for (int i = 0; i<R; i++)
```

```
	{
		// take the first column
		cout << getColumn(MatrixA,0)[i] << endl;
	}

	cout << "Row—2" << endl;
	for (int i = 0; i<C; i++)
	{
		// take the second row
		cout << getRow(MatrixA,1)[i] << endl;
	}

	cout << "Vector from row 1:"<<endl;
	vector<double> vectora = getRow(MatrixA,0) ;
	printVector(vectora);

	return 0;
}
```

Code 64: *Test SymIntegration getRow and getColumn Example/main.cpp*

multiply(vector<vector<double>>, vector<vector<double>>)
this function will perform multiplication of two matrices represented by vector<vector<double>>. For matrices multiplication, order matters, thus in general for non-similar matrices A and B, $AB \neq BA$.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Matrices Multiplication ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
.root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Matrices Multiplication ]# ./m
in
A:
        1.000000        2.000000
        3.000000        4.000000
        0.000000        1.000000

B:
        4.000000        3.000000
        2.000000        1.000000

A*B:
        8.000000        5.000000
       20.000000       13.000000
        2.000000        1.000000

Time taken by function: 977 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Matrices Multiplication ]#
```

Figure 6.1: *The multiplication between two matrices, we load the matrices from textfiles* ***matrixA.txt*** *and* ***matrixB.txt*** *(**SymIntegration/Examples/Linear Algebra/Test SymIntegration Matrices Multiplication//main.cpp**).*

add(vector<vector<double>>, vector<vector<double>>)
this function will perform addition of two matrices represented by vector<vector<double>>.

subtract(vector<vector<double>>, vector<vector<double>>)
this function will perform subtraction of two matrices represented by vector<vector<double>>.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Matrices Addition ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Matrices Addition ]# ./main
A:
        4.000000        3.000000
        2.000000        1.000000

B:
        1.000000        0.000000
        2.000000        3.000000

A + B:
        5.000000        3.000000
        4.000000        4.000000

Time taken by function: 1074 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Matrices Addition ]#
```

Figure 6.2: *The addition between two matrices, we load the matrices from textfiles* ***matrixA.txt*** *and* ***matrixB.txt*** *(**SymIntegration/Examples/Linear Algebra/Test SymIntegration Matrices Addition/main.cpp**).*

transpose(const vector<vector<double>> &)
this function will perform transpose operation of a matrix represented by vector<vector<double>>.

dmat to replace **vector<vector<double>>**
We create a typedef defined in **include/symintegral/symintegrationc++.h** to represents **vector<vector<double>>**, so we don't need to write **vector<vector<double>>** anymore, we can use **dmat** instead.
The compiler will process the typedef declaration when compiling each .cpp file that includes **include/symintegral/symintegrationc++.h**, making the aliased types available for use within those files.

dvec to replace **vector<double>**
svec to replace **vector<string>**
smat to replace **vector<vector<string>>**

complexd to replace **complex<double>**
cmat to replace **vector<vector<complex<double>>>**
cvec to replace **vector<complex<double>>**

createVector(int n, double k)
this function will create a vector represented by vector<double> with size of n and populated with k as the entries.

createMatrix(int R, int C, double k)
this function will create a matrix represented by vector<vector<double>> with size of $R \times C$ and populated with k as the entries.

createIdentityMatrix(int n)
this function will create an identity matrix with size of n.

createMatrixFromColumnVectors(const vector<vector<double>>& columnVectors)
this function will create a matrix represented by vector<vector<double>> from column vectors that are composed by $\{v_1, v_2, \ldots, v_c\}$ and all the column vectors are of the same size of r, thus the matrix will have the size of $r \times c$.

```
int main()
{
       // Get starting timepoint
       auto start = high_resolution_clock::now();

       dvec veca = loadVectorFromFile("veca.txt");
       dvec vecb = loadVectorFromFile("vecb.txt");
       dvec b = loadVectorFromFile("b.txt");

       dmat columnVectors = {veca, vecb, b};
       dmat newmatrix = createMatrixFromColumnVectors(
           columnVectors);

       printMatrix(newmatrix);

       // Get ending timepoint
       auto stop = high_resolution_clock::now();
       auto duration = duration_cast<microseconds>(stop - start
           );

       cout << "\nTime taken by function: " << duration.count()
            << " microseconds" << endl;

       return 0;
}
```

Code 65: *Test SymIntegration Create Matrix from Column Vectors/main.cpp*

VandermondeMatrix(const vector<double>& vector_x, int n)
this function will return a Vandermonde matrix with type **vector<vector<double>>** from input of vector x (with size of m) and n as the number of column for the Vandermonde matrix.

The Vandermonde matrix:

$$A = \begin{bmatrix} 1 & x_1 & x_1^2 & \dots & x_1^{n-1} \\ 1 & x_2 & x_2^2 & \dots & x_2^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_m & x_m^2 & \dots & x_m^{n-1} \end{bmatrix}$$

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Vandermonde Matrix ]# ./main
Vector x:
1
2
3
4
5

Vandermonde Matrix(x,3):
               1.000000              1.000000              1.000000
               1.000000              2.000000              4.000000
               1.000000              3.000000              9.000000
               1.000000              4.000000             16.000000
               1.000000              5.000000             25.000000

Time taken by function: 788 microseconds
```

Figure 6.3: *The function to create Vandermonde matrix* $(\boldsymbol{x}, 3)$*. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Vandermonde Matrix/main.cpp**).*

multiplymatrixvector(vector<vector<double>> &A, vector<double> & $\boldsymbol{x}$)
this function will multiply matrix A and vector $\boldsymbol{x}$.

scalarmultiplication(vector<vector<double>> &A, double k)
to compute the scalar multiplication of a matrix with k.

ii. Solve Nonhomogeneous System of Linear Equation with Gaussian Elimination in SymIntegration

Suppose we have a nonhomogeneous linear system like this:

$$\begin{aligned} 0.0333x_1 + 0.2x_2 &= 1 \\ 0.1x_1 + 0.4x_2 &= 1 \end{aligned}$$

In the form of matrix and vector following the Eq. (6.2) we will have:

$$A = \begin{bmatrix} 0.0333 & 0.2 \\ 0.1 & 0.4 \end{bmatrix}, \quad \boldsymbol{b} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

We are going to solve for

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

In SymIntegration to solve a nonhomogeneous linear system with Gaussian elimination and back substitution you can use:
solve_nhsystem(vector<vector<double>> &,vector<vector<double>> &, vector<double> &)
it is a **void** function that returns nothing, and you can still call the solution after the function is called. The source code that handles this computation is located in **src/linearalgebra.cpp**.

Algorithm for **solve_nhsystem**:

1. Read input from textfiles of **matrix.txt** for matrix A and **vectorb.txt** for vector $\boldsymbol{b}$.

2. The function will perform a merge for the matrix A and vector $\boldsymbol{b}$ to become an augmented matrix.

3. Then the function will perform forward elimination, that will iterate through each row to create zeros below the main diagonal. Ensuring for each pivot element (diagonal element) it will be non-zero. If it is zero, the function will swap the current row with a row below that has a non-zero element in the same column (partial pivoting). If no such row exists, then it indicates that there is no unique solution. The rank of the matrix is not full. Then for each row below the current pivot row, it calculates a ratio and performs row operations to make the element below the pivot zero.

4. After forward elimination, the function will perform back substitution to obtain the solution. Once the matrix is in row echelon form (upper triangular), it solves for the last variable directly, then it substitutes the value of the last variable into the second-to-last equation to solve the second-to-last variable, and so on, working upwards till x_1.

When we code with C++ we need to remember that there is some truncation error, and precision error, like floating-point inaccuracies when we comparing values to zero or performing divisions.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Gaussian Elimination Void Function for Square Mat
tion Void Function for Square Matrix Load from Textfiles ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Gaussian Elimination Void Function for Square Mat
rix Load from Textfiles ]# ./main
Augmented Matrix:
         0.033300         0.200000         1.000000
         0.100000         0.400000         1.000000

Augmented Matrix in reduced row form:
         0.100000         0.400000         1.000000
         0.000000         0.066800         0.667000

Solution:
x1 = 9.98503
x2 = -29.94012


Time taken by function: 1151 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Gaussian Elimination Void Function for Square Mat
rix Load from Textfiles ]# ./main
Augmented Matrix:
         0.033300         0.200000         1.000000
         0.100000         0.400000         1.000000

Augmented Matrix in reduced row form:
         0.100000         0.400000         1.000000
         0.000000         0.066800         0.667000

Solution:
x1 = 9.98503
x2 = -29.94012


Time taken by function: 1159 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Gaussian Elimination Void Function for Square Mat
rix Load from Textfiles ]# ./main
Augmented Matrix:
         0.033300         0.200000         1.000000
         0.100000         0.400000         1.000000

Augmented Matrix in reduced row form:
         0.100000         0.400000         1.000000
         0.000000         0.066800         0.667000

Solution:
x1 = 9.98503
x2 = -29.94012


Time taken by function: 978 microseconds
```

Figure 6.4: *The Gaussian elimination with forward elimination and back substitution, we load the matrix from textfiles* ***matrix.txt*** *and* ***vectorb.txt****. We run the code 3 times to see the running time of the function from* 978 *microseconds to* 1151 *microseconds (****SymIntegration/Examples/Linear Algebra/Test SymIntegration Gaussian Elimination Void Function for Square Matrix Load from Textfiles/main.cpp****).*

We use **void** function for this Gaussian elimination because in C++ a void function is a function that does not return any value to the calling code. Instead of a specific data type (like int, double, string) as its return type, some functions in SymIntegration uses **void** for ease of use, it can be used for printing output to the console or a file, modifying global variables or variables passed by reference, performing calculations and storing results in external structures, and calling other functions to achieve a specific task.

We also compare it with Armadillo' function **solve(A,B,solve_opts::force_approx)**, turns out we can gain faster execution time, it is probably because we code it ourselves and can make it more efficient to return the solution of $\boldsymbol{x}$. In Armadillo you need to print the solution yourself. In SymIntegration currently, we are printing out the augmented matrix, the reduced row form of the augmented matrix, and the solution when you execute / run the function **solve_nhsystem**.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Gaussian Elimina
tion Void Function for Square Matrix Load from Textfiles/Gaussian Elimination fo
r Square Matrix with Armadillo ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -larmadillo
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Gaussian Elimination Void Function for Square Ma
trix Load from Textfiles/Gaussian Elimination for Square Matrix with Armadillo ]# ./main
Matrix A:
   0.0333   0.2000
   0.1000   0.4000

Vector B:
   1.0000
   1.0000

Solution:
  -29.9401
    9.9850


Time taken by function: 1551 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Gaussian Elimination Void Function for Square Ma
trix Load from Textfiles/Gaussian Elimination for Square Matrix with Armadillo ]#
```

Figure 6.5: *The comparison of solving a linear system with Armadillo, we load the matrix from textfiles* ***matrixA.txt*** *and* ***vectorB.txt****. (****DianFreya Math Physics Simulator/Source Codes/C++/C++ Gnuplot SymbolicC++/ch23-Numerical Linear Algebra/Gaussian Elimination for Square Matrix with Armadillo/main.cpp****).*

The code that is using Armadillo (so the reader can check it themselves):

```
#include <iostream>
#include <iomanip> // to declare the manipulator of setprecision()
#include <fstream>
#include <bits/stdc++.h> //for setw(6) at display() function
#include <vector>
#include <armadillo>
#include <chrono>

using namespace std::chrono;
using namespace std;
using namespace arma;

// Driver code
int main(int argc, char** argv)
{
	// Get starting timepoint
	auto start = high_resolution_clock::now();

	mat A;
	A.load("matrixA.txt");
	mat B;
	B.load("vectorB.txt");
	mat X;
	X = solve(A,B,solve_opts::force_approx);

	cout <<"Matrix A:" << "\n" << A <<endl;
	cout <<"Vector B:" << "\n" << B <<endl;
	cout <<"Solution:" << "\n" << X <<endl;

	// Get ending timepoint
```

```
        auto stop = high_resolution_clock::now();
        auto duration = duration_cast<microseconds>(stop - start);

        cout << "\nTime taken by function: " << duration.count() << " microseconds"
             << endl;

}
```

Code 66: *main.cpp*

II. Determinant

[SI*] Suppose we have a 2×2 matrix A

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

the matrix A is invertible if $ad - bc \neq 0$, and the expression $ad - bc$ is called the determinant.

$$\det(A) = \begin{vmatrix} a & b \\ c & d \end{vmatrix} = ad - bc$$

Thus, the inverse of A can be expressed in terms of the determinant as

$$A^{-1} = \frac{1}{\det(A)} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix} \tag{6.3}$$

Now, we will find the formula that applicable to square matrices of all orders, not only of size 2.

Definition 6.1: Minor and Cofactor

If A is a square matrix, then the minor of entry a_{ij} is denoted by M_{ij} and is defined to be the determinant of the submatrix that remains after the i-th row and j-th column are deleted from A. The number

$$C_{ij} = (-1)^{i+j} M_{ij} \tag{6.4}$$

is called the cofactor of entry a_{ij}.

Theorem 6.1: Row or Column of Cofactors

If A is an $n \times n$ matrix, then regardless of which row or column of A is chosen, the number obtained by multiplyinh the entries in that row or column by the corresponding cofactors and adding the resulting products is always the same.

Definition 6.2: Cofactor Expansion

If A is an $n \times n$ matrix, then the number obtained by multiplying the entries in any row or column of A by the corresponding cofactors and adding the resulting products is called the determinant of A, and the sums themselves are called cofactor expansions of A. That is,

$$\det(A) = a_{1j}C_{1j} + a_{2j}C_{2j} + \cdots + a_{nj}C_{nj} \tag{6.5}$$

(cofactor expansion along the j-th column)

$$\det(A) = a_{i1}C_{i1} + a_{i2}C_{i2} + \cdots + a_{in}C_{in} \tag{6.6}$$

(cofactor expansion along the i-th row)

[SI*] To find the determinant of size n that is very large will need more computational cost, since we will peel the cofactor expansion along the i-th row or j-th column down to the matrix of size 2×2. It is an ordered-recursive computation technique.

[SI*] The concept of a determinant is mathematically defined only for square matrices. Non-square matrices, also known as rectangular matrices, do not possess a determinant in the conventional sense.

The determinant of a square matrix represents the scaling factor of the volume (or area in 2D) when a linear transformation is applied. This geometric is inherently tied to dimensions being equal. The determinant is also crucial for calculating the inverse of a matrix, which is only defined for square, non-singular matrices.

[SI*] **Alternative Concepts for Non-Square Matrices**
While a determinant doesn't exist for non-square matrices, related concepts can be used in certain contexts:

- **Singular Values**
  Non-square matrices have singular values, which are related to the scaling of principal axes in the transformation defined by the matrix. The product of singular values can be related to a "volume" in a generalized sense.
- **Gramian Determinant**
  For a matrix A, the Gramian matrix A^TA (or AA^T) is always square. Its determinant, known as the Gramian determinant, can be used in some applications involving non-square matrices.

i. Functions in SymIntegration Related to Determinant

[SI*] We will list the basic functions that can be used related to determinant and inverse:

determinant(const vector<vector<double>> &matrix)
this function computes the determinant of a square matrix.

inverse(vector<vector<double>> &matrix)
this function computes the inverse of a square matrix that is non-empty. The computation of an inverse of a matrix is a complex one, it needs adjugate, cofactor and determinant computation.

ii. Compute Determinant of a Square Matrix with SymIntegration

Suppose we have a matrix A

$$A = \begin{bmatrix} 3 & -2 & 7 \\ -2 & 4 & -3 \\ -1 & 9 & 4 \end{bmatrix}$$

to compute the determinant with SymIntegration we will use this function:
determinant(const vector<vector<double>> &matrix)

The full C++ source code can be seen below:

```
#include<bits/stdc++.h>
#include<iostream>
#include "symintegrationc++.h"
#include<vector>
#include <chrono>
#include <algorithm> // For std::next_permutation
#include <string>
using namespace std::chrono;
using namespace std;

// Driver program
int main()
{
	// Get starting timepoint
	auto start = high_resolution_clock::now();

	//string filename = "matrix.txt";

	vector<vector<double>> doubleMatrix = loadMatrixFromFile("matrix.txt");

	cout << "\nMatrix A : " << endl;
	printMatrix(doubleMatrix);
	cout << "\nDeterminant of matrix A : " << determinant(doubleMatrix) << endl
	    ;

	// Get ending timepoint
	auto stop = high_resolution_clock::now();
	auto duration = duration_cast<microseconds>(stop - start);

	cout << "\nTime taken by function: " << duration.count() << " microseconds"
	     << endl;

	return 0;
}
```

Code 67: *Test SymIntegration Compute Determinant/main.cpp*

```
Time taken by function: 757 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Compute Determinant ]# ./main

Matrix A :
         3.000000        -2.000000         7.000000
        -2.000000         4.000000        -3.000000
        -1.000000         9.000000         4.000000

Determinant of matrix A : 9.000000

Time taken by function: 751 microseconds
```

Figure 6.6: *The computation of the determinant of matrix A (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Compute Determinant/main.cpp**).*

III. Euclidean Vector Spaces

[SI*] Engineers and physicists represent vectors in two dimensions or in three dimensions by arrows. The direction of the arrowhead specifies the direction of the vector and the length of the arrow specifies the magnitude. The tail of the arrow is called the initial point of the vector and the tip the terminal point.

We usually denote vectors with boldface such as $\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{v}, \boldsymbol{w}$ or with an arrow above it $\vec{a}, \vec{b}, \vec{v}, \vec{w}$, and we denote scalars in lowercase letters like a, b, c, d. When we want to indicate that a vector has initial point A and terminal point B, we will write

$$\boldsymbol{v} = \vec{AB}$$

[SI*] We denote the length of a vector $\boldsymbol{v}$ by the symbol $||\boldsymbol{v}||$, which is read as norm of $\boldsymbol{v}$, or the magnitude of $\boldsymbol{v}$.

Definition 6.3: Norm

If $\boldsymbol{v} = (v_1, v_2, \ldots, v_n)$ is a vector in $\mathbb{R}^n$, then the norm of $\boldsymbol{v}$ is denoted by $||\boldsymbol{v}||$, and is defined by the formula

$$||\boldsymbol{v}|| = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2} \tag{6.7}$$

[SI*] A vector of norm 1 is called a unit vector. Such vectors are useful for specifying a direction when length is not relevant to the problem at hand. If $\boldsymbol{v}$ is any nonzero vector in $\mathbb{R}^n$, then

$$\boldsymbol{u} = \frac{1}{||\boldsymbol{v}||}\boldsymbol{v} \tag{6.8}$$

defines a unit vector that is in the same direction as $\boldsymbol{v}$. The process of multiplying a nonzero vector by the reciprocal of its length to obtain a unit vector is called normalizing $\boldsymbol{v}$.

Definition 6.4: Distance in $\mathbb{R}^n$

If $\boldsymbol{u} = (u_1, u_2, \ldots, u_n)$ and $\boldsymbol{v} = (v_1, v_2, \ldots, v_n)$ are points in $\mathbb{R}^n$, then we denote the distance between $\boldsymbol{u}$ and $\boldsymbol{v}$ by $d(\boldsymbol{u}, \boldsymbol{v})$ and define it to be

$$d(\boldsymbol{u}, \boldsymbol{v}) = ||\boldsymbol{u} - \boldsymbol{v}|| = \sqrt{(u_1 - v_1)^2 + (u_2 - v_2)^2 + \cdots + (u_n - v_n)^2} \tag{6.9}$$

Definition 6.5: Angle Between Two Vectors in $\mathbb{R}^2$ and $\mathbb{R}^3$

If $\boldsymbol{u}$ and $\boldsymbol{v}$ are nonzero vectors in $\mathbb{R}^2$ or $\mathbb{R}^3$, and if θ is the angle between $\boldsymbol{u}$ and $\boldsymbol{v}$, then the dot product of $\boldsymbol{u}$ and $\boldsymbol{v}$ is denoted by $\boldsymbol{v} \cdot \boldsymbol{v}$ and is defined as

$$\boldsymbol{u} \cdot \boldsymbol{v} = ||\boldsymbol{u}||||\boldsymbol{v}|| \cos \theta \tag{6.10}$$

If $\boldsymbol{u} = \boldsymbol{0}$ or $\boldsymbol{v} = \boldsymbol{0}$, then we define $\boldsymbol{u} \cdot \boldsymbol{v}$ to be 0.

Since $0 \leq \theta \leq \pi$, it follows from the properties of the cosine function that

- θ is acute if $\boldsymbol{u} \cdot \boldsymbol{v} > 0$.
- θ is obtuse if $\boldsymbol{u} \cdot \boldsymbol{v} < 0$.
- $\theta = \frac{\pi}{2}$ if $\boldsymbol{u} \cdot \boldsymbol{v} = 0$.

Definition 6.6: Dot Product in $\mathbb{R}^n$

If $\boldsymbol{u}$ and $\boldsymbol{v}$ are vectors in $\mathbb{R}^n$, then the dot product or the Euclidean inner product of $\boldsymbol{u}$ and $\boldsymbol{v}$ is denoted by $\boldsymbol{u} \cdot \boldsymbol{v}$ and is defined by

$$\boldsymbol{u} \cdot \boldsymbol{v} = u_1 v_1 + u_2 v_2 + \cdots + u_n v_n \tag{6.11}$$

Definition 6.7: Orthogonal and Orthonormal

Two nonzero vectors $\boldsymbol{u}$ and $\boldsymbol{v}$ in $\mathbb{R}^n$ are said to be orthogonal (or perpendicular) if $\boldsymbol{u} \cdot \boldsymbol{v} = 0$.

The zero vector ($\boldsymbol{0}$) in $\mathbb{R}^n$ is orthogonal to every vector in $\mathbb{R}^n$.

A nonempty set of vectors in $\mathbb{R}^n$ is called an orthogonal set if all pairs of distinct vectors in the set are orthogonal.

An orthogonal set of unit vectors is called an orthonormal set.

[SI*] The vector equation

$$\boldsymbol{n} \cdot \vec{P_0 P} = 0$$

is the representation of of a line through the point $P_0(x_0, y_0)$ that has normal $\boldsymbol{n} = (a, b)$ or a plane through the point $P_0(x_0, y_0, z_0)$ that has normal $\boldsymbol{n} = (a, b, c)$.

For line, the vector $\vec{P_0 P}$ can be expressed in terms of components as

$$\vec{P_0 P} = (x - x_0, y - y_0)$$

For plane, the vector $\vec{P_0 P}$ can be expressed in terms of components as

$$\vec{P_0 P} = (x - x_0, y - y_0, z - z_0)$$

[SI*] The point-normal equation of the line:

$$a(x - x_0) + b(y - y_0) = 0 \tag{6.12}$$

Theorem 6.2: Equation for Line and Plane

If a and b are constants that are not both zero, then an equation of the form

$$ax + by + c = 0 \tag{6.13}$$

represents a line in $\mathbb{R}^2$ with normal $\boldsymbol{n} = (a, b)$.

If a, b, and c are constants that are not all zero, then an equation of the form

$$ax + by + cz + d = 0 \tag{6.14}$$

represents a plane in $\mathbb{R}^3$ with normal $\boldsymbol{n} = (a, b, c)$.

[SI*] There are other useful ways of specifying lines and planes.

A unique line in $\mathbb{R}^2$ or $\mathbb{R}^3$ is determined by a point $\boldsymbol{x}_0$ on the line and a nonzero vector $\boldsymbol{v}$ parallel to the line.

A unique plane in $\mathbb{R}^3$ is determined by a point $\boldsymbol{x}_0$ in the plane and two collinear vectors $\boldsymbol{v}_1$ and $\boldsymbol{v}_2$ parallel to the plane. The best way to visualize this is to translate the vectors so their initial points are at $\boldsymbol{x}_0$.

Theorem 6.3: Vector and Parametric Equations of Lines in $\mathbb{R}^2$ and $\mathbb{R}^3$

Let L be the line in $\mathbb{R}^2$ or $\mathbb{R}^3$ that contains the point $\boldsymbol{x}_0$ and is parallel to the nonzero vector $\boldsymbol{v}$. Then the equation of the line through $\boldsymbol{x}_0$ that is parallel to $\boldsymbol{v}$ is

$$\boldsymbol{x} = \boldsymbol{x}_0 + t\boldsymbol{v} \tag{6.15}$$

If $\boldsymbol{x}_0 = \boldsymbol{0}$, then the line passes through the origin and the equation has the form

$$\boldsymbol{x} = t\boldsymbol{v} \tag{6.16}$$

the variable t is called a parameter, and it varies from $-\infty$ to ∞.

Theorem 6.4: Vector and Parametric Equations of Planes in and $\mathbb{R}^3$

Let W be the plane in $\mathbb{R}^3$ that contains the point $\boldsymbol{x}_0$ and is parallel to the noncollinear vectors $\boldsymbol{v}_1$ and $\boldsymbol{v}_2$. Then an equation of the plane through $\boldsymbol{x}_0$ that is parallel to $\boldsymbol{v}_1$ and $\boldsymbol{v}_2$ is given by

$$\boldsymbol{x} = \boldsymbol{x}_0 + t_1\boldsymbol{v}_1 + t_2\boldsymbol{v}_2 \tag{6.17}$$

If $\boldsymbol{x}_0 = \boldsymbol{0}$, then the plane passes through the origin and the equation has the form

$$\boldsymbol{x} = t_1\boldsymbol{v}_1 + t_2\boldsymbol{v}_2 \tag{6.18}$$

Definition 6.8: Parametric Equations of Lines in $\mathbb{R}^n$

If $\boldsymbol{x}_0$ and $\boldsymbol{v}$ are vectors in $\mathbb{R}^n$, and if $\boldsymbol{v}$ is nonzero, then the equation

$$\boldsymbol{x} = \boldsymbol{x}_0 + t\boldsymbol{v} \tag{6.19}$$

defines the line through $\boldsymbol{x}_0$ that is parallel to $\boldsymbol{v}$. In the special case where $\boldsymbol{x}_0 = \boldsymbol{0}$, the line is said to pass through the origin.

Definition 6.9: Parametric Equations of Planes in $\mathbb{R}^n$

If $\boldsymbol{x}_0$, $\boldsymbol{v}_1$ and $\boldsymbol{v}_2$ are vectors in $\mathbb{R}^n$, and if $\boldsymbol{v}_1$ and $\boldsymbol{v}_2$ are not collinear, then the equation

$$\boldsymbol{x} = \boldsymbol{x}_0 + t_1\boldsymbol{v}_1 + t_2\boldsymbol{v}_2 \tag{6.20}$$

defines the plane through $\boldsymbol{x}_0$ that is parallel to $\boldsymbol{v}_1$ and $\boldsymbol{v}_2$. In the special case where $\boldsymbol{x}_0 = \boldsymbol{0}$, the plane is said to pass through the origin.

Definition 6.10: Cross Product

If $\boldsymbol{u} = (u_1, u_2, u_3)$ and $\boldsymbol{v} = (v_1, v_2, v_3)$ are vectors in 3-space, then the cross product $\boldsymbol{u} \times \boldsymbol{v}$ is the vector defined by

$$\boldsymbol{u} \times \boldsymbol{v} = (u_2v_3 - u_3v_2, u_3v_1 - u_1v_3, u_1v_2 - u_2v_1) \tag{6.21}$$

or, in determinant notation,

$$\boldsymbol{u} \times \boldsymbol{v} = \left(\begin{vmatrix} u_2 & u_3 \\ v_2 & v_3 \end{vmatrix}, - \begin{vmatrix} u_1 & u_3 \\ v_1 & v_3 \end{vmatrix}, \begin{vmatrix} u_1 & u_2 \\ v_1 & v_2 \end{vmatrix} \right) \tag{6.22}$$

Definition 6.11: Scalar Triple Product

If $\boldsymbol{u}, \boldsymbol{v}$, and $\boldsymbol{w}$ are vectors in 3-space, then

$$\boldsymbol{u} \cdot (\boldsymbol{v} \times \boldsymbol{w}) = \begin{vmatrix} u_1 & u_2 & u_3 \\ v_1 & v_2 & v_3 \\ w_1 & w_2 & w_3 \end{vmatrix} \tag{6.23}$$

is called the scalar triple product of $\boldsymbol{u} = (u_1, u_2, u_3), \boldsymbol{v} = (v_1, v_2, v_3)$, and $\boldsymbol{w} = (w_1, w_2, w_3)$.

It follows that

$$\boldsymbol{u} \cdot (\boldsymbol{v} \times \boldsymbol{w}) = \boldsymbol{w} \cdot (\boldsymbol{u} \times \boldsymbol{v}) = \boldsymbol{v} \cdot (\boldsymbol{w} \times \boldsymbol{u})$$

Theorem 6.5: Geometric Interpretation of Determinants

(a) The absolute value of the determinant

$$\det \begin{bmatrix} u_1 & u_2 \\ v_1 & v_2 \end{bmatrix}$$

is equal to the area of the parallelogram in 2-space determined by the vectors $\boldsymbol{u} = (u_1, u_2)$ and $\boldsymbol{v} = (v_1, v_2)$.

(b) The absolute value of the determinant

$$\det \begin{bmatrix} u_1 & u_2 & u_3 \\ v_1 & v_2 & v_3 \\ w_1 & w_2 & w_3 \end{bmatrix}$$

is equal to the volume of the parallelepiped in 3-dimension determined by the vectors $\boldsymbol{u} = (u_1, u_2, u_3), \boldsymbol{v} = (v_1, v_2, v_3)$, and $\boldsymbol{w} = (w_1, w_2, w_3)$.

i. Functions in SymIntegration Related to Euclidean Vector Spaces

[SI*] We will list the basic functions that can be used related to Euclidean vector spaces:
distance(const vector<double> &vectorx, const vector<double> &vectory)
to compute the distance between two points (the point can be represented as n-tuple / array / **vector<double>**).

distance(const Symbolic &f, const Symbolic &x, const Symbolic &y, const Symbolic &z, const vector<double> &vectorx)
to compute the distance between a plane / a line that is defined with function $f(x, y, z)$ / $f(x, y)$ and a point that is represented as vector $\boldsymbol{x}$.

dot(const vector<double> &vectorx, const vector<double> &vectory)
to compute the dot product / Euclidean inner product of two vectors.

angle(const vector<double> &vectorx, const vector<double> &vectory)
to compute the angle between two vectors.

norm(const vector<double> &vectorx)
to compute the norm of a vector.

add(vector<double> &vectorA, vector<double> &vectorB)
to compute the addition between two vectors.

subtract(vector<double> &vectorA, vector<double> &vectorB)
to compute subtraction between two vectors.

scalarmultiplication(vector<double> &vectorA, double k)
to compute the scalar multiplication of a vector with k.

othogonalprojection(vector<double> vectorx, vector<double> vectory)
to compute the orthogonal projection of $\boldsymbol{x}$ on $\boldsymbol{y}$.

vectorequation(const Symbolic &f, const Symbolic &x, const Symbolic &y, const Symbolic &z)
to compute the vector equation of the form $\boldsymbol{x} = \boldsymbol{x}_0 + t\boldsymbol{v}_1 + s\boldsymbol{v}_2$ that represents a plane / a line through $\boldsymbol{x}_0$ and parallel to $\boldsymbol{v}_1$ and/or $\boldsymbol{v}_2$ (for a line we will only have one $\boldsymbol{v}$) with given input of a function $f(x, y, z) = 0$.

vectorequationdecomp(const Symbolic &f, const Symbolic &x, const Symbolic &y, const Symbolic &z, vector<double> &vectorv1, vector<double> &vectorv2, vector<double> &vectorv3)
to compute all the vectors that compose the vector equation / parametric equations of the form $\boldsymbol{x} = \boldsymbol{x}_0 + t\boldsymbol{v}_1 + s\boldsymbol{v}_2$ that represents a plane / a line through $\boldsymbol{x}_0$ and parallel to $\boldsymbol{v}_1$ and/or $\boldsymbol{v}_2$ (for a line we will only have one $\boldsymbol{v}$) with given input of a function $f(x, y, z) = 0$. It is a **void** function, so we will need to declare the vector $\boldsymbol{v}_1, \boldsymbol{v}_2$, and $\boldsymbol{v}_3$ before calling the function, afterwards we can print the vectors.

crossproduct(vector<double> &vectoru, vector<double> &vectorv)
to compute the cross product of $\boldsymbol{u} \times \boldsymbol{v}$.

scalartripleproduct(vector<double> &vectoru, vector<double> &vectorv, vector<double> &vectorw)
to compute the scalar triple product of $\boldsymbol{u}, \boldsymbol{v}$, and $\boldsymbol{w}$.

areaoftriangle(vector<double> &x, vector<double> &y, vector<double> &z)
to compute the area of triangle in $\mathbb{R}^n$, given 3 vertices that we input as **vector<double>** type, x, y, z. We use the formula with two known sides and the angle between these two sides

$$A = \frac{1}{2}ab\sin(c)$$

it is more efficient than using determinant formula to compute the area.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Area of triangle in n-dimension ]# ./main

Vertices, a :
-1
4

b :
3
1

c :
2
6

Area of triangle : 8.5

Time taken by function: 808 microseconds
```

Figure 6.7: *The computation of the area of triangle given 3 vertices:* $\boldsymbol{a}, \boldsymbol{b}$ *and* $\boldsymbol{c}$ *in* $\mathbb{R}^2$ *(**SymIntegration/Examples/-Linear Algebra/Test SymIntegration Area of triangle in n-dimension/main.cpp**).*

ii. Compute Norm, Dot Product, Angle with SymIntegration

Suppose we have two vectors :

$$\boldsymbol{x} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

$$\boldsymbol{y} = \begin{bmatrix} 0 \\ 2 \\ 2 \end{bmatrix}$$

we can compute the norm, dot product, and the angle between $\boldsymbol{x}$ and $\boldsymbol{y}$ with SymIntegration' functions.

$$\begin{aligned} ||\boldsymbol{x}|| &= 1 \\ ||\boldsymbol{y}|| &= 2.82843 \\ \boldsymbol{x} \cdot \boldsymbol{y} &= 2 \\ \theta &= \cos^{-1}\left(\frac{\boldsymbol{x} \cdot \boldsymbol{y}}{||\boldsymbol{x}||||\boldsymbol{y}||}\right) = 0.70710678 \text{ rad} = 45^0 \end{aligned}$$

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Euclidean Vector Spaces ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Euclidean Vector Spaces ]# ./main

Vector x :
0
0
1

Vector y :
0
2
2

norm(x) : 1

norm(y) : 2.82843

dot(x,y) : 2

angle(x,y) in degree: 45

Time taken by function: 689 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Euclidean Vector Spaces ]#
```

Figure 6.8: *The computation of the norm, dot product, and angle of two vectors $\boldsymbol{x}$ and $\boldsymbol{y}$ (**SymIntegration/Examples/-Linear Algebra/Test SymIntegration Euclidean Vector Spaces/main.cpp**).*

iii. Compute Orthogonal Projection on a Line with SymIntegration

Let $\boldsymbol{u} = (2, -1, 3)$ and $\boldsymbol{a} = (4, -1, 2)$. Find the vector component of $\boldsymbol{u}$ along $\boldsymbol{a}$ and the vector component of $\boldsymbol{u}$ orthogonal to $\boldsymbol{a}$.

Solution:

$$\begin{aligned}\boldsymbol{u} \cdot \boldsymbol{a} &= (2)(4) + (-1)(-1) + (3)(2) = 15\\ ||\boldsymbol{a}||^2 &= 4^2 + (-1)^2 + 2^2 = 21\end{aligned}$$

Then the vector component of $\boldsymbol{u}$ along $\boldsymbol{a}$ is

$$\begin{aligned}\text{proj}_{\boldsymbol{a}}\boldsymbol{u} &= \frac{\boldsymbol{u} \cdot \boldsymbol{a}}{||\boldsymbol{a}||^2}\boldsymbol{a}\\ &= \frac{15}{21}(4, -1, 2)\\ &= \left(\frac{20}{7}, -\frac{5}{7}, \frac{10}{7}\right)\end{aligned}$$

and the vector component of $\boldsymbol{u}$ orthogonal to $\boldsymbol{a}$ is

$$\begin{aligned}\boldsymbol{u} - \text{proj}_{\boldsymbol{a}}\boldsymbol{u} &= (2, -1, 3) - \left(\frac{20}{7}, -\frac{5}{7}, \frac{10}{7}\right)\\ &= \left(-\frac{6}{7}, -\frac{2}{7}, \frac{11}{7}\right)\end{aligned}$$

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Orthogonal Projection ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Orthogonal Projection ]# ./main

Vector u :
2
-1
3

Vector a :
4
-1
2

norm(a)^2 : 29.6985

dot(u,a) : 15

Orthogonal projection of u on a:
2.85714
-0.714286
1.42857

Vector component of u orthogonal to a:
-0.857143
-0.285714
1.57143

u + a :
6
-2
5

u - a :
-2
0
1

5*u  :
10
-5
15

Time taken by function: 1066 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Orthogonal Projection ]#
```

Figure 6.9: *The computation of the orthofonal projection of vector* $\boldsymbol{u}$ *on* $\boldsymbol{a}$ *and some other vector operations. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Orthogonal Projection/main.cpp**).*

iv. Compute Vector and Parametric Equations of a Plane in $\mathbb{R}^3$ with SymIntegration

Find vector and parametric equations of the plane $x - y + 2z = 5$.

Solution:

We will find the parametric equations first. We can do this by solving the equation for any one of the variables in terms of the other two and then using those two variables as parameters.

For example, solving for x in terms of y and z yields

$$x = 5 + y - 2z$$

and then using y and z as parameters t and s, respectively, yields the parametric equations

$$\begin{aligned} x &= 5 + t - 2s \\ y &= t \\ z &= s \end{aligned}$$

To obtain a vector equation of the plane we rewrite these parametric equations as

$$(x, y, z) = (5 + t - 2s, t, s)$$

or, equivalently, as

$$(x, y, z) = (5, 0, 0) + t(1, 1, 0) + s(-2, 0, 1)$$

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Vector and Parametric Equations ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Vector and Parametric Equations ]# ./main

Vector equation of the plane x - y + 2z - 5 = 0:
[t-2*s+5]
[   t   ]
[   s   ]


The parametric equations in vector terms:
(x,y,z) = v1 + v2*t + v2*s
v1 =
5
0
0

v2 =
1
1
0

v3 =
-2
0
1

Time taken by function: 29519 microseconds
```

Figure 6.10: *The computation of vector equation of the plane in parametric equations term and then in vector terms. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Vector and Parametric Equations/main.cpp**).*

IV. General Vector Spaces

[SI*] In Euclidean Vector Spaces we developed properties of vectors in $\mathbb{R}^2$ and $\mathbb{R}^3$, we noticed patterns in various formulas that enabled us to extend the notion of a vector to an n-tuple of real numbers. Although n-tuple took us outside the realm of our "visual experience," it gave us a valuable tool for understanding and studying systems of linear equations.

Definition 6.12: Vector Space Axioms

Let V be an arbitrary nonempty set of objects on which two operations are defined: addition, and multiplication by scalars.

- By addition we mean a rule for associating with each pair of objects $\boldsymbol{u}$ and $\boldsymbol{v}$ in V an object $\boldsymbol{u}+\boldsymbol{v}$, called the sum of $\boldsymbol{u}$ and $\boldsymbol{v}$.
- By scalar multiplication we mean a rule for associating with each scalar k and each object $\boldsymbol{u}$ in V an object $k\boldsymbol{u}$, called the scalar multiple of $\boldsymbol{u}$ by k.

If the following axioms are satisfied by all objects $\boldsymbol{u}, \boldsymbol{v}, \boldsymbol{w}$ in V and all scalars k and m, then we call V a vector space and we call the objects in V vectors

1. If $\boldsymbol{u}$ and $\boldsymbol{v}$ are objects in V, then $\boldsymbol{u}+\boldsymbol{v}$ is in V
2. $\boldsymbol{u}+\boldsymbol{v}=\boldsymbol{v}+\boldsymbol{u}$
3. $\boldsymbol{u}+(\boldsymbol{v}+\boldsymbol{w})=(\boldsymbol{u}+\boldsymbol{v})+\boldsymbol{w}$
4. There is an object $\mathbf{0}$ in V, called a zero vector for V, such that
$$\mathbf{0}+\boldsymbol{u}=\boldsymbol{u}+\mathbf{0}=\boldsymbol{u}$$
for all $\boldsymbol{u}$ in V
5. For each $\boldsymbol{u}$ in V, there is an object $-\boldsymbol{u}$ in V, called a negative of $\boldsymbol{u}$, such that
$$\boldsymbol{u}+(-\boldsymbol{u})=(-\boldsymbol{u})+\boldsymbol{u}=\mathbf{0}$$
6. If k is any scalar and $\boldsymbol{u}$ is any object in V, then $k\boldsymbol{u}$ is in V.
7. $k(\boldsymbol{u}+\boldsymbol{v})=k\boldsymbol{u}+k\boldsymbol{v}$
8. $(k+m)\boldsymbol{u}=k\boldsymbol{u}+m\boldsymbol{u}$
9. $k(m\boldsymbol{u})=(km)(\boldsymbol{u})$
10. $1\boldsymbol{u}=\boldsymbol{u}$

Definition 6.13: Subspace

A subset W of a vector space V is called a subspace of V if W is itself a vector space under the addition and scalar multiplication defined on V.

In general, to show that a nonempty set W with two operations is a vector space one must verify the ten vector space axioms.

Definition 6.14: Linear Combination

If $\boldsymbol{w}$ is a vector in a vector space V, then $\boldsymbol{w}$ is said to be a linear combination of the vectors $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_r$ in V if $\boldsymbol{w}$ can be expressed in the form

$$\boldsymbol{w} = k_1\boldsymbol{v}_1 + k_2\boldsymbol{v}_2 + \cdots + k_r\boldsymbol{v}_r \tag{6.24}$$

where $k_1, k_2, \ldots, k_r$ are scalars. These scalars are called the coefficients of the linear combination.

Theorem 6.6: Linear Combination and Subspace

If $S = \{\boldsymbol{w}_1, \boldsymbol{w}_2, \ldots, \boldsymbol{w}_r\}$ is a nonempty set of vectors in a vetor space V, then:

1. The set W of all possible linear combinations of the vectors in S is a subspace of V.
2. The set W in part (a) is the "smallest" subspace of V that contains all of the vectors in S in the sense that any other subspace that contains those vectors contains W.

Definition 6.15: Span of a Subspace

The subspace of a vector space V that is formed from all possible linear combinations of the vectors in a nonempty set S is called the span of S, and we say that the vectors in S span that subspace. If $S = \{\boldsymbol{w}_1, \boldsymbol{w}_2, \ldots, \boldsymbol{w}_r\}$, then we denote the span of S by

$$\text{span}\{\boldsymbol{w}_1, \boldsymbol{w}_2, \ldots, \boldsymbol{w}_r\} \quad \text{or} \quad \text{span}(S)$$

[SI*] The standard unit vectors in $\mathbb{R}^n$ are

$$\boldsymbol{e}_1 = (1, 0, 0, \ldots, 0), \quad \boldsymbol{e}_2 = (0, 1, 0, \ldots, 0), \ldots, \quad \boldsymbol{e}_n = (0, 0, 0, \ldots, 1)$$

these vectors span $\mathbb{R}^n$ since every vector $\boldsymbol{v} = (v_1, v_2, \ldots, v_n)$ in $\mathbb{R}^n$ can be expressed as

$$\boldsymbol{v} = v_1\boldsymbol{e}_1 + v_2\boldsymbol{e}_2 + \cdots + v_n\boldsymbol{e}_n$$

Theorem 6.7: Solution of Homogeneous Linear System

The solution set of a homogeneous linear system $A\boldsymbol{x} = \boldsymbol{0}$ in n unknowns is a subspace of $\mathbb{R}^n$.

[SI*] The solution set of every homogeneous system of m equations in n unknowns is a subspace of $\mathbb{R}^n$. But, the solution set of a nonhomogeneous system of m equations in n unknowns is a

subspace of $\mathbb{R}^n$ is never true, since there are two possible scenarios: first, the system may not have any solution at all, and second, if there are solutions, then the solution set will not be closed under either addition or under scalar multiplication.

Theorem 6.8: Span and Linear Combination

If $S = \{v_1, v_2, \ldots, v_r\}$ and $S' = \{w_1, w_2, \ldots, w_k\}$ are nonempty sets of vectors in a vector space V, then

$$\text{span}\{v_1, v_2, \ldots, v_r\} = \text{span}\{w_1, w_2, \ldots, w_k\}$$

if and only if each vector in S is a linear combination of those in S', and each vector in S' is a linear combination of those in S.

[SI*] In a rectangular xy-coordinate system every vector in the plane can be expressed in exactly one way as a linear combination of the standard unit vectors.

Definition 6.16: Linear Independence

If $S = \{v_1, v_2, \ldots, v_r\}$ is a nonempty set of vectors in a vector space V, then the vector equation

$$k_1v_1 + k_2v_2 + \cdots + k_rv_r = \mathbf{0}$$

has at least one solution, namely

$$k_1 = 0, \quad k_2 = 0, \quad \ldots, \quad k_r = 0$$

we call this the trivial solution. If this is the only solution, then S is said to be a linearly independent set. If there are solutions in addition to the trivial solution, then S is said to be a linearly dependent set.

[SI*] To show that a linear system has a nontrivial solutions you can

1. Solve it directly with Gaussian Elimination, when you have this vector equation

$$k_1v_1 + k_2v_2 + \cdots + k_rv_r = \mathbf{0}$$

then turns it into homogeneous system and make a coefficient matrix out of it.

2. Check the determinant of the coefficient matrix, if the determinant is zero then the coefficient matrix has infinitely many solutions / nontrivial solutions. If the determinant is not zero then the system has only the trivial solution, and the vectors are linearly independent.

Theorem 6.9: Interpretation of Linear Independence

A set S with two or more vectors is

(a) Linearly dependent if and only if at least one of the vectors in S is expressible as a linear combinatino of the other vectors in S.

(b) Linearly independent if and only if no vector in S is expressible as a linear combination of the other vector in S.

Theorem 6.10: Sets With One or Two Vectors

(a) A finite set that contains **0** is linearly dependent.

(b) A set with exactly one vector is exactly independent if and only if that vector is not **0**.

(c) A set with exactly two vectors is linearly independent if and only if neither vector is a scalar multiple of the other.

Theorem 6.11: Geometric Interpretation of Linear Independence

Let $S = \{v_1, v_2, \ldots, v_r\}$ be a set of vectors in $\mathbb{R}^n$. If $r > n$, then S is linearly dependent.

[SI*] Sometimes linear dependence of functions can be deduced from known identities. But, there is no general method that can be used to determine whether a set of functions is linear independent or linear dependent.

Definition 6.17: Wronskian

If $f_1 = f_1(x)$, $f_2 = f_2(x), \ldots, f_n = f_n(x)$ are functions that are $n-1$ times differentiable on the interval $(-\infty, \infty)$, then the determinant

$$\begin{vmatrix} f_1(x) & f_2(x) & \ldots & f_n(x) \\ f_1'(x) & f_2'(x) & \ldots & f_n'(x) \\ \vdots & \vdots & & \vdots \\ f_1^{(n-1)} & f_2^{(n-1)} & \ldots & f_n^{(n-1)} \end{vmatrix}$$

is called the Wronskian of $f_1, f_2, \ldots, f_n$.

Theorem 6.12: Wronskian and Linear Independence of Functions

If the functions $f_1, f_2, \ldots, f_n$ have $n-1$ continuous derivatives on the interval $(-\infty, \infty)$, and if the Wronskian of these functions is not identically zero on $(-\infty, \infty)$, then these functions form a linearly independent set of vectors in $C^{(n-1)}(-\infty, \infty)$

[SI*] Rectangular coordinate systems are common, but they are not essential. There are nonrectangular coordinate systems in 2-space, 3-space, to the n-space with equal spacing, skew axes (the major axis is not orthogonal to each other) with equal spacing or unequal spacing. Our common rectangular coordinate system has perpendicular axes.

Definition 6.18: Basis for a Vector Space

If V is any vector space and $S = \{v_1, v_2, \ldots, v_n\}$ is a finite set of vectors in V, then S is called a basis for V if the following two conditions hold:

(a) S is linearly independent.

(b) S spans V.

[SI*] To prove the vectors $v_1, v_2, \ldots, v_n$ are linearly independent we must show that

$$c_1v_1 + c_2v_2 + \cdots + c_nv_n = \mathbf{0} \tag{6.25}$$

has only the trivial solution.
To prove that the vectors $v_1, v_2, \ldots, v_n$ span $\mathbb{R}^n$, we must show that every vector $\boldsymbol{b} = (b_1, b_2, \ldots, b_n)$ in $\mathbb{R}^n$ can be expressed as

$$c_1v_1 + c_2v_2 + \cdots + c_nv_n = \boldsymbol{b} \tag{6.26}$$

From the two equations above we will have two linear systems, the first is homogeneous (to prove for linear independence) and the second one is nonhomogeneous system (to prove that the vectors span), both systems will have the same coefficient matrix. The nonhomogeneous system is consistent for all values of $b_1, b_2, \ldots, b_n$.

Theorem 6.13: Uniqueness of Basis Representation

If $S = \{v_1, v_2, \ldots, v_n\}$ is a basis for a vector space V, then every vector v in V can be expressed in the form $v = c_1v_1 + c_2v_2 + \cdots + c_nv_n$ in exactly one way.

Definition 6.19: Coordinate Vector

If $S = \{v_1, v_2, \ldots, v_n\}$ is a basis for a vector space V (the order of the vectors in S remains fixed), and

$$v = c_1v_1 + c_2v_2 + \cdots + c_nv_n$$

is the expression for a vector v in terms of the basis S, then the scalars $c_1, c_2, \ldots, c_n$ are called the coordinates of v relative to the basis S. The vector $(c_1, c_2, \ldots, c_n)$ in $\mathbb{R}^n$ constructed from these coordinates is called the coordinate vector of v relative to S; it is denoted by

$$(v)_S = (c_1, c_2, \ldots, c_n)$$

Theorem 6.14: Number of Vectors in a Basis

All bases for a finite-dimensional vector space have the same number of vectors.

Theorem 6.15: Number of Vectors in a Basis 2

Let V be a finite-dimensional vector space, and let $\{v_1, v_2, \ldots, v_n\}$ be any basis

(a) If a set has more than n vectors, then it is linearly dependent.

(b) If a set has fewer than n vectors, then it does not span V.

Definition 6.20: Dimension

The dimension of a finite-dimensional vector space V is denoted by $\dim(V)$ and is defined to be the number of vectors in a basis for V. In addition, the zero vector space is defined to have dimension zero.

Engineers often use the term degrees of freedom as a synonym for dimension.

[SI*] Dimensions of some Vector Spaces:

$$\dim(\mathbb{R}^n) = n$$

$$\dim(P_n) = n + 1$$

$$\dim(M_{mn}) = m \times n$$

e.g. the standard basis for matrix 3×5 / M_{35} has 15 vectors.

Theorem 6.16: Plus/Minus Theorem

Let S be a nonempty set of vectors in a vector space V.

(a) If S is a linearly independent set, and if $\boldsymbol{v}$ is a vector in V that is outside of $span(S$, then the set $S \cup \{\boldsymbol{v}\}$ that results by inserting $\boldsymbol{v}$ into S is still linearly independent.

(b) If $\boldsymbol{v}$ is a vector in S that is expressible as a linear combination of other vectors in S, and if $S - \{\boldsymbol{v}\}$ denotes the set obtained by removing $\boldsymbol{v}$ from S, then S and $S - \{\boldsymbol{v}\}$ span the same space; that is,

$$\text{span}(S) = \text{span}(S - \{\boldsymbol{v}\})$$

[SI*] In general, to show that a set of vectors $\{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n\}$ is a basis for a vector space V, we must show that the vectors are linearly independent and span V. If we happen to know that V has dimension n, so that $\{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n\}$ contains the right number of vectors for a basis, then it suffices to check either linear independence or spanning, the remaining condition will hold automatically.

Theorem 6.17: Basis and Dimension

Let V be an n-dimensional vector space, and let S be a set in V with exactly n vectors. Then S is a basis for V if and only if S spans V or S is linearly independent.

Theorem 6.18: Basis Rules

Let S be a finite set of vectors in a finite-dimensional vector space V.

(a) If S spans V but is not a basis for V, then S can be reduced to a basis for V by removing appropriate vectors from S.

(b) If S is linearly independent set that is not already a basis for V, then S can be enlarged to a basis for V by inserting appropriate vectors in S.

Theorem 6.19: Dimension of a Vector Space and Its Subspaces

If W is a subspace of a finite-dimensional vector space V, then:

(a) W is finite-dimensional.

(b) $\dim(W) \leq \dim(V)$.

(c) $W = V$ if and only if $\dim(W) = \dim(V)$.

[SI*] A basis is the vector space generalization of a coordinate system, changing bases is akin to changing coordinate axes in $\mathbb{R}^2$ and $\mathbb{R}^3$.

[SI*] If $S = \{v_1, v_2, \ldots, v_n\}$ is a basis for a finite-dimensional vector space V, and if

$$(\boldsymbol{v})_S = (c_1, c_2, \ldots, c_n)$$

is the coordinate vector of $\boldsymbol{v}$ relative to S, then the mapping

$$\boldsymbol{v} \to (\boldsymbol{v})_S \tag{6.27}$$

creates a connection (a one-to-one correspondence) between vectors in the general vector space V and vectors in the familiar vector space $\mathbb{R}^n$. We call $\boldsymbol{v} \to (\boldsymbol{v})_S$ the coordinate map from V to $\mathbb{R}^n$.

[SI*] We will find it convenient to express coordinate vectors in the matrix form

$$[\boldsymbol{v}]_S = \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{bmatrix} \tag{6.28}$$

[SI*] If we change the basis for a vector space V from an old basis $B = \{\boldsymbol{u}_1, \boldsymbol{u}_2, \ldots, \boldsymbol{u}_n\}$ to a new basis $B' = \{\boldsymbol{u}'_1, \boldsymbol{u}'_2, \ldots, \boldsymbol{u}'_n\}$, then for each vector $\boldsymbol{v}$ in V, the old coordinate vector $[\boldsymbol{v}]_B$ is related to the new coordinate vector $[\boldsymbol{v}]_{B'}$ by the equation

$$[\boldsymbol{v}]_B = P_{B' \to B}[\boldsymbol{v}]_{B'} \tag{6.29}$$

where the columns of P are the coordinate vectors of the new basis vectors relative to the old basis; that is, the column vectors of P are

$$[\boldsymbol{u}'_1]_B, [\boldsymbol{u}'_2]_B, \ldots, [\boldsymbol{u}'_n]_B \tag{6.30}$$

[SI*] The matrix P is called the transition matrix from B' to B. For emphasis, we will often denote it by

$$P_{B' \to B}$$

This matrix can be expressed in terms of its column vectors as

$$P_{B' \to B} = \left[[\boldsymbol{u}'_1]_B \mid [\boldsymbol{u}'_2]_B \mid \ \ldots \ \mid [\boldsymbol{u}'_n]_B \right] \tag{6.31}$$

A column vector $[\boldsymbol{u}'_1]_B$ is a column vector $\boldsymbol{u}'_1$ represented in old bases $B = \{\boldsymbol{u}_1, \boldsymbol{u}_2, \ldots \boldsymbol{u}_n\}$ terms.

Similarly, the transition matrix from B to B' can be expressed in terms of its column vectors as

$$P_{B \to B'} = \left[[\boldsymbol{u}_1]_{B'} \mid [\boldsymbol{u}_2]_{B'} \mid \ \ldots \ \mid [\boldsymbol{u}_n]_{B'} \right] \tag{6.32}$$

A column vector $[\boldsymbol{u}_1]_{B'}$ is a column vector $\boldsymbol{u}_1$ represented in new bases $B' = \{\boldsymbol{u}'_1, \boldsymbol{u}'_2, \ldots \boldsymbol{u}'_n\}$ terms.

[SI*] Consider this old bases in $\mathbb{R}^n$

$$B = \{\boldsymbol{u}_1, \boldsymbol{u}_2, \ldots \boldsymbol{u}_n\}$$

and the new bases in in $\mathbb{R}^n$

$$B' = \{\boldsymbol{u}'_1, \boldsymbol{u}'_2, \ldots \boldsymbol{u}'_n\}$$

We can use transition matrix $P_{B\to B'}$ to obtain old bases from multiplying $P_{B\to B'}$ with new bases

$$\begin{aligned} P_{B\to B'}[\boldsymbol{u}_1'] &= [\boldsymbol{u}_1] \\ P_{B\to B'}[\boldsymbol{u}_2'] &= [\boldsymbol{u}_2] \\ &\vdots \\ P_{B\to B'}[\boldsymbol{u}_n'] &= [\boldsymbol{u}_n] \end{aligned}$$

in converse, we can use transition matrix $P_{B'\to B}$ to obtain new bases from multiplying $P_{B'\to B}$ with old bases

$$\begin{aligned} P_{B'\to B}[\boldsymbol{u}_1] &= [\boldsymbol{u}_1'] \\ P_{B'\to B}[\boldsymbol{u}_2] &= [\boldsymbol{u}_2'] \\ &\vdots \\ P_{B'\to B}[\boldsymbol{u}_n] &= [\boldsymbol{u}_n'] \end{aligned}$$

[SI*] The columns of the transition matrix from an old basis to a new basis are the coordinate vectors of the old basis relative to the new basis.

[SI*] Let $\boldsymbol{v}$ or $[\boldsymbol{v}]_B$ be any vector in V, the changing of bases will change the vector $\boldsymbol{v}$ as well. We know that vector $\boldsymbol{v}$ can be represented in old bases $B = \{\boldsymbol{u}_1, \boldsymbol{u}_2, \dots \boldsymbol{u}_n\}$ as

$$\boldsymbol{v} = k_1\boldsymbol{u}_1 + k_2\boldsymbol{u}_2 + \cdots + k_n\boldsymbol{u}_n$$

this vector $\boldsymbol{v}$ will change when we use new bases $B' = \{\boldsymbol{u}_1', \boldsymbol{u}_2', \dots \boldsymbol{u}_n'\}$ into $\boldsymbol{v}'$ or $[\boldsymbol{v}]_{B'}$

$$\boldsymbol{v}' = k_1'\boldsymbol{u}_1' + k_2'\boldsymbol{u}_2' + \cdots + k_n'\boldsymbol{u}_n'$$

we can write them in vector notation as

$$\boldsymbol{v} = [\boldsymbol{v}]_B = \begin{bmatrix} k_1 \\ k_2 \\ \vdots \\ k_n \end{bmatrix}, \quad \boldsymbol{v}' = [\boldsymbol{v}]_{B'} = \begin{bmatrix} k_1' \\ k_2' \\ \vdots \\ k_n' \end{bmatrix}$$

We are able to determine $[\boldsymbol{v}]_B$ and $[\boldsymbol{v}]_{B'}$ when we know the new and old bases by using the transition matrix

$$[\boldsymbol{v}]_B = P_{B'\to B}[\boldsymbol{v}]_{B'} \tag{6.33}$$

$$[\boldsymbol{v}]_{B'} = P_{B\to B'}[\boldsymbol{v}]_B \tag{6.34}$$

with

$$(P_{B'\to B})(P_{B\to B'}) = I_n \tag{6.35}$$

Guide 6.1: A procedure for computing $P_{B\to B'}$

1. Form the matrix
$$[B' \mid B]$$
with $B = \{\boldsymbol{u}_1, \boldsymbol{u}_2, \dots \boldsymbol{u}_n\}$ as old bases and $B' = \{\boldsymbol{u}'_1, \boldsymbol{u}'_2, \dots \boldsymbol{u}'_n\}$ as new bases. The bases will be treated as column vectors when constructing the matrix above.
2. Use elementary row operations, e.g. Gaussian elimination / Gauss-Jordan elimination to reduce the matrix to reduced row echelon form.
3. The resulting matrix will be
$$[I \mid P_{B\to B'}]$$

[SI*] Suppose now that B and B' are bases for a finite-dimensional vector space V. Since multiplication by $P_{B'\to B}$ maps coordinate vectors relative to the basis B' into coordinate vectors relative to a basis B, and $P_{B\to B'}$ maps coordinate vectors relative to B into coordinate vectors relative to B', it follows that for every vector $\boldsymbol{v}$ in V we have

$$[\boldsymbol{v}]_B = P_{B'\to B}[\boldsymbol{v}]_{B'} \tag{6.36}$$

$$[\boldsymbol{v}]_{B'} = P_{B\to B'}[\boldsymbol{v}]_B \tag{6.37}$$

Theorem 6.20: Transition to Standard Basis for $\mathbb{R}^n$

Let $B' = \{\boldsymbol{u}_1, \boldsymbol{u}_2, \dots, \boldsymbol{u}_n\}$ be any basis for the vector space $\mathbb{R}^n$ and let $S = \{\boldsymbol{e}_1, \boldsymbol{e}_2, \dots, \boldsymbol{e}_n\}$ be the standard basis for $\mathbb{R}^n$. If the vectors in these bases are written in column form / column vector, then

$$P_{B'\to S} = [\boldsymbol{u}_1 \mid \boldsymbol{u}_2 \mid \dots \mid \boldsymbol{u}_n] \tag{6.38}$$

[SI*] Changing basis allows you to convert a matrix from a complicated form to a simple form. It is often possible to represent a matrix in a basis where the only nonzero elements are on the diagonal, which is exceptionally simple. These diagonal elements will be the eigenvalues of the matrix. This is especially helpful in solving linear systems of differential equations. Often in physics, engineering, logistics, and probably lots of other places, you have a system of differential equations which all depend on each other. In order to solve the system directly, you would have to solve all equations at once, which is hard. We can use matrices to describe this system.

By changing basis, you may be able to make that matrix diagonal, which effectively separates the differential equations from each other, so you can solve just one at a time. This is comparatively easy. Real life situations that uses changing basis are

(a) Quantum mechanics, solving the Schrödinger equation to describe the state of matter on the quantum level.

(b) Electrical engineering, understanding the time-evolution of an electrical circuit.

(c) Mechanical engineering, understanding the motion of a linear mechanical system, such as multiple spring-mass system.

(d) Solid mechanics. Computing the principal stresses/strains in a material involves a change of basis. With the principal stresses, we can compute the maximum normal

stress and maximum shear stress the material experiences. And these maxima tell us whether the material will become damaged or not.

Changing basis allows you to convert a matrix from a complicated form to a simple form. Rotation is essentially a change of basis. If you ever employed a linear coordinate change, you changed bases. Let say you have a cube that is rotated arbitrarily and is not orthogonal the coordinate axes, and you're trying to determine if a point is inside of it. It is a bit tricky to determine directly. If you can change to a basis where the cube is orthogonal to the coordinate axes, determining if the point is inside, then it becomes trivial.

[SI*] Row space, column space, and null space are some important vector spaces that are associated with matrices. Understanding them will make us know the relationships between the solutions of a linear system and properties of its coefficient matrix.

Definition 6.21: Row and Column Vectors

For an $m \times n$ matrix

$$A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}$$

the vectors

$$\begin{aligned} \boldsymbol{r}_1 &= [a_{11} \quad a_{12} \quad \dots \quad a_{1n}] \\ \boldsymbol{r}_2 &= [a_{21} \quad a_{22} \quad \dots \quad a_{2n}] \\ &\vdots \\ \boldsymbol{r}_m &= [a_{m1} \quad a_{m2} \quad \dots \quad a_{mn}] \end{aligned}$$

in $\mathbb{R}^n$ that are formed from the rows of A are called the row vectors of A, and the vectors

$$\boldsymbol{c}_1 = \begin{bmatrix} a_{11} \\ a_{21} \\ \vdots \\ a_{m1} \end{bmatrix}, \quad \boldsymbol{c}_2 = \begin{bmatrix} a_{12} \\ a_{22} \\ \vdots \\ a_{m2} \end{bmatrix}, \quad \boldsymbol{c}_n = \begin{bmatrix} a_{1n} \\ a_{2n} \\ \vdots \\ a_{mn} \end{bmatrix}$$

in $\mathbb{R}^m$ formed from the columns of A are called the column vectors of A.

Definition 6.22: Row Space and Column Space

If A is an $m \times n$ matrix, then the subspace of $\mathbb{R}^n$ spanned by the row vectors of A is called the row space of A.

The subspace of $\mathbb{R}^m$ spanned by the column vectors of A is called the column space of A.

The solution space of the homogeneous system of equations

$$A\boldsymbol{x} = \boldsymbol{0}$$

which is a subspace of $\mathbb{R}^n$, is called the null space of A.

[SI*] Suppose that

$$A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}, \quad \boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

If $\boldsymbol{c}_1, \boldsymbol{c}_2, \dots, \boldsymbol{c}_n$ denote the column vectors of A, then the product $A\boldsymbol{x}$ can be expressed as a linear combination of these vectors with coefficients from $\boldsymbol{x}$

$$A\boldsymbol{x} = x_1\boldsymbol{c}_1 + x_2\boldsymbol{c}_2 + \cdots + x_n\boldsymbol{c}_n \tag{6.39}$$

Thus, a linear system, $A\boldsymbol{x} = \boldsymbol{b}$, of m equations in n unknowns can be written as

$$x_1\boldsymbol{c}_1 + x_2\boldsymbol{c}_2 + \cdots + x_n\boldsymbol{c}_n = \boldsymbol{b} \tag{6.40}$$

from which we conclude that $A\boldsymbol{x} = \boldsymbol{b}$ is consistent if and only if $\boldsymbol{b}$ is expressible as a linear combination of the column vectors of A.

Theorem 6.21: Column Space and Consistent Linear System

A system of linear equations $A\boldsymbol{x} = \boldsymbol{b}$ is consistent if and only if $\boldsymbol{b}$ is in the column space of A.

Theorem 6.22: Null Space and Consistent Linear System

If $\boldsymbol{x}_0$ is any solution of a consistent linear system $A\boldsymbol{x} = \boldsymbol{b}$, and if $S = \{\boldsymbol{v}_1, \boldsymbol{v}_2, \dots, \boldsymbol{v}_k\}$ is a basis for the null space of A, then every solution of $A\boldsymbol{x} = \boldsymbol{b}$ can be expressed in the form

$$\boldsymbol{x} = \boldsymbol{x}_0 + c_1\boldsymbol{v}_1 + c_2\boldsymbol{v}_2 + \cdots + c_k\boldsymbol{v}_k \tag{6.41}$$

Conversely, for all choices of scalars $c_1, c_2, \dots, c_k$, the vector $\boldsymbol{x}$ in this formula is a solution of $A\boldsymbol{x} = \boldsymbol{b}$.

[SI*] The equation

$$\boldsymbol{x} = \boldsymbol{x}_0 + c_1\boldsymbol{v}_1 + c_2\boldsymbol{v}_2 + \cdots + c_k\boldsymbol{v}_k$$

gives a formula for the general solution of $A\boldsymbol{x} = \boldsymbol{b}$.

The vector $\boldsymbol{x}_0$ is called a particular solution of $A\boldsymbol{x} = \boldsymbol{b}$.

The remaining part of the formula $c_1\boldsymbol{v}_1 + c_2\boldsymbol{v}_2 + \cdots + c_k\boldsymbol{v}_k$ is called the general solution of $A\boldsymbol{x} = \boldsymbol{0}$.

The general solution of a consistent linear system can be expressed as the sum of a particular solution of that system and the general solution of the corresponding homogeneous system.

Theorem 6.23: Elementary Row Operations for Null Space, Row Space, and Column Space

Elementary row operations do not change the null space of a matrix.

Elementary row operations do not change the row space of a matrix.

Elementary row operations change the column space of a matrix.

Theorem 6.24: Bases for the Row and Column Spaces

If a matrix R is in row echelon form, then the row vectors with the leading 1's (the nonzero row vectors) form a basis for the row space of R, and the column vectors with the leading 1's of the row vectors form a basis for the column space of R.

Theorem 6.25: Row Equivalent Matrices and Column Vectors

If A and B are row equivalent matrices, then:

(a) A given set of column vectors of A is linearly independent if and only if the corresponding column vectors of B are linearly independent.

(b) A given set of column vectors of A forms a basis for the column space of A if and only if the corresponding column vectors of B form a basis for the column space of B.

[SI*] Vector spaces are used extensively in electrical engineering, particularly in the analysis and design of electrical systems. Here are some examples of applications of vector spaces in electrical engineering:

(a) Analysis of electric circuits: In electric circuits, currents and voltages can be represented as vectors in a vector space. By applying the principles of linear algebra to these vectors, it's possible to analyze the behavior of the circuit and determine the values of circuit variables such as voltage, current, and power.

(b) Design of control systems: Control systems are used to regulate and stabilize the behavior of complex electrical systems. Vector spaces can be used to represent the state of a control system, and linear algebra can be used to design control algorithms that stabilize the system.

(c) Signal processing: In signal processing, signals such as audio, video, and data can be represented as vectors in a vector space. By applying techniques from linear algebra, it's possible to analyze and process these signals, for example, by applying filters or compressing data.

(d) Electromagnetics: Vector spaces are used to describe the behavior of electromagnetic fields, which are represented as vector fields. Maxwell's equations, which govern the behavior of electromagnetic fields, are expressed using vector calculus.

Theorem 6.26: Dimension of Row Space and Column Space

The row space and column space of a matrix A have the same dimension.

[SI*] The number of nonzero rows of a reduced row echelon form matrix R is the same as the number of leading 1's in matrix R.

Definition 6.23: Rank and Nullity

The common dimension of the row space and column space of a matrix A is called the rank of A and is denoted by $\text{rank}(A)$.

The dimension of the null space of A is called the nullity of A and is denoted by $\text{nullity}(A)$.

[SI*] Since the rank of a matrix A of size $m \times n$ is the common dimension of its row and column space, it follows that the rank is at most the smaller of m and n.

$$\text{rank}(A) \leq \min(m, n)$$

Theorem 6.27: Dimension Theorem for Matrices

If A is a matrix with n columns, then

$$\text{rank}(A) + \text{nullity}(A) = n$$

[SI*] The number of leading variables is the same as the number of leading 1's in the reduced row echelon form of matrix A, which is the rank of A.

The number of free variables is the same as the number of parameters in the general solution of

$$A\boldsymbol{x} = \boldsymbol{0}$$

which is the nullity of A.

Theorem 6.28: Rank and Nullity

If A is an $m \times n$ matrix, then

(a) $\text{rank}(A)$ = the number of leading variables in the general solution of $A\boldsymbol{x} = \boldsymbol{0}$.

(b) $\text{nullity}(A)$ = the number of parameters in the general solution of $A\boldsymbol{x} = \boldsymbol{0}$.

[SI*] In many applications the equations in a linear system correspond to physical constraints or conditions that must be satisfied. In general, the most desirable systems are those that have the same number of constraints as unknowns, since such systems often have a unique solution.

Unfortunately, it is not always possible to match the number of constraints and unknowns.

When linear systems have more constraints than unknowns like this

$$\begin{array}{rrrl} x_1 & + x_2 & + x_3 & = b_1 \\ x_1 & + 2x_2 & + 5x_3 & = b_2 \\ -x_1 & + 6x_2 & + 7x_3 & = b_3 \\ -x_1 & - 3x_2 & + x_3 & = b_4 \end{array}$$

it is called overdetermined systems.

When linear systems have fewer constraints than unknowns like this

$$\begin{array}{rrrl} x_1 & + x_2 & + x_3 & = b_1 \\ x_1 & + 2x_2 & + 5x_3 & = b_2 \end{array}$$

it is called underdetermined systems.

Theorem 6.29: Consistent Linear System

If $A\boldsymbol{x} = \boldsymbol{b}$ is a consistent linear system of m equations in n unknowns, and if A has rank r, then the general solution of the system contains $n - r$ parameters.

Theorem 6.30: Overdetermined and Underdetermined Systems

Let A be an $m \times n$ matrix.

(a) **Overdetermined Case**
If $m > n$, then the linear system $A\boldsymbol{x} = \boldsymbol{b}$ is inconsistent for at least one vector $\boldsymbol{b}$ in $\mathbb{R}^n$.

(b) **Underdetermined Case**
If $m < n$, then for each vector $\boldsymbol{b}$ in $\mathbb{R}^n$ the linear system $A\boldsymbol{x} = \boldsymbol{b}$ is either inconsistent or has infinitely many solutions.

[SI*] There are six important vector spaces associated with a matrix A and its transpose A^T:

1. Row space of A
2. Column space of A
3. Null space of A
4. Row space of A^T
5. Column space of A^T
6. Null space of A^T

Transposing a matrix converts row vectors into clumn vectors and conversely, so except for a difference in notation, the row space of A^T is the same as the column space of A, and the column space of A^T is the same as the row space of A.

Out of the six spaces listed above, only the following four are distinct:

1. Row space of A

2. Column space of A
3. Null space of A
4. Null space of A^T

These are called the fundamental spaces of a matrix A.

Theorem 6.31: Rank for Matrix and Its Transpose

If A is any matrix, then

$$\text{rank}(A) = \text{rank}(A^T)$$

[SI*] If A is an $m \times n$ matrix, then

$$\text{rank}(A) + \text{nullity}(A) = m \tag{6.42}$$

It is possible to express the dimensions of all four fundamental spaces in terms of size and rank of A.

$$\begin{aligned} \dim[\text{row}(A)] &= r \\ \dim[\text{col}(A)] &= r \\ \dim[\text{null}(A)] &= n - r \\ \dim[\text{null}(A^T)] &= m - r \end{aligned} \tag{6.43}$$

The four formulas above provide an algebraic relationship between the size of a matrix and the dimension of its fundamental spaces.

[SI*] If A is an $m \times n$ matrix, then the null space of A consists of those vectors that are orthogonal to each of the row vectors of A.

Definition 6.24: Orthogonal Complement

If W is a subspace of $\mathbb{R}^n$, then the set of all vectors in $\mathbb{R}^n$ that are orthogonal to every vector in W is called the orthogonal complement of W and is denoted by the symbol $W^\perp$.

Theorem 6.32: Orthogonal Complement Basic

If W is a subspace of $\mathbb{R}^n$, then:

(a) $W^\perp$ is a subspace of $\mathbb{R}^n$.

(b) The only vector common to W and $W^\perp$ is $\mathbf{0}$.

(c) The orthogonal complement of $W^\perp$ is W.

Theorem 6.33: Orthogonal Complement for Matrix

If A is an $m \times n$ matrix, then:

(a) The null space of A and the row space of A are orthogonal complement in $\mathbb{R}^n$.

(b) The null space of A^T and the column space of A are orthogonal complements in $\mathbb{R}^m$.

[SI*] Digital data are commonly stored in matrix form, and many techniques for improving transmission speed use the rank of a matrix. Rank plays a role to measure the "redundancy" in a matrix in the sense that if A is an $m \times n$ matrix of rank k, then $n - k$ of the column vectors and $m - k$ of the row vectors can be expressed in terms of k linearly independent column or row vectors. The essential idea in many data compression schemes is to approximate the original data set by a data set with smaller rank that conveys nearly the same information, then eliminate redundant vectors in the approximating set.

i. General Vector Spaces Application in Engineering

[SI*] **Row and Column spaces**
Row and column spaces are fundamental in engineering for understanding system constraints, reachability, and data relationship, appearing in robotics (workspace), structural analysis (load distribution), signal processing (data filtering), and data science (dimensionality reduction), representing the span of possible outputs (column space) and system constraints / input dimensions (row space) for linear systems defined by matrices.

Column Space (Image Space): The set of all possible output vectors ($\boldsymbol{b}$) from a transformation $A\boldsymbol{x} = \boldsymbol{b}$, meaning the "reach" or achievable states/positions.

Row Space: The space spanned by the rows of a matrix, revealing inherent constraints or relationships within the system's equations.

Rank: The dimension of both the row and column spaces, indicating the effective number of independent variables or dimensions.

Applications in engineering:

1. **Robotics & Kinematics**
   Workspace determination: The column space defines the reachable workspace for a robot's end-effector, as the final position is a linear combination (span) of arm segment movements.
2. **Structural Analysis & Loads**
   Load paths: The row space helps understand how applied forces (loads) are distributed and constrained within a structure, determining if a structure can support given forces.
3. **Data Analysis & Machine Learning**
   Dimensionality reduction: Identifying the rank of a data matrix helps reduce dimensions (Principal Component Analysis), keeping the most significant components (in the column/row space).
4. **Signal Processing**
   System modeling: Matrices represent filters or transformations. The column space shows what signals can be produced, while the row space indicates filtering properties.
5. **System of Linear Equations (Civil/Electrical Engineering)**
   Solvability: For $A\boldsymbol{x} = \boldsymbol{b}$, the system has solutions if $\boldsymbol{b}$ is in the column space of A (i.e., $\boldsymbol{b}$ can be formed by the columns of A).
   Constraints: The row space defines the constraints that must be met for a solution to exist.

In essence, the column space answers "what can I do or produce?", while the row space answers "what are the inherent rules or limitations?" within a linear engineering model.

[SI*] **Null space**

The solution space of $A\boldsymbol{x} = \mathbf{0}$ is the null space (or kernel), representing all input vectors x that a transformation A maps to the zero vector, crucial in engineering for finding equilibrium states, system redundancies, identifying dependencies (like in structural analysis or circuits), and understanding stability, showing which inputs cause "no output" or collapse, fundamentally linked to rank and nullity in signal processing, control systems, and computer graphics.

For a matrix A, the null space, or kernel, null(A) is the set of all vectors $\boldsymbol{x}$ such that $A\boldsymbol{x} = \mathbf{0}$. The zero vector $\boldsymbol{x} = \mathbf{0}$ is always a trivial solution. If non-zero solutions exist, they form a subspace (a line, a plane, etc., through the origin) with infinite solutions, indicating dependency or redundancy in the system.

Applications in engineering:

1. **Structural Engineering (Statics)**
   Finding a forces in a truss (e.g. $A\boldsymbol{x} = \mathbf{0}$) where A relates forces to equilibrium, $\boldsymbol{x}$ are unknown forces, and $\mathbf{0}$ is zero net force. Non-zero solutions (null space vectors) represent forces that can exist without causing external loads, indicating redundant members or mechanisms where forces can balance in multiple ways (like a slack cable).
2. Electrical Engineering (Circuit Analysis)
   Nodal analysis often leads to $A\boldsymbol{x} = \mathbf{0}$, where $\boldsymbol{x}$ are node voltages, A is the admittance matrix. A non-trivial null space means there are floating nodes or independent loops (Kirchoff's Current Law) where voltages can exist without external sources.
3. **Control Systems & Signal Processing**
   Analyzing system stability or observability ($A\boldsymbol{x} = \mathbf{0}$). A non-trivial null space indicates unobservable states or internal dynamics (like oscilaltions) that aren't detectable from the system's outputs.
4. **Computer Graphics & Robotics**
   Finding transformations (e.g., in 3D rotations, $A\boldsymbol{x} = \mathbf{0}$ can define axes). In kinematics, it can represent null-space motion, allowing for movement (like arm articulation) that doesn't change the end-effector's position, useful for obstacle avoidance.
5. **Data Analysis & Machine Learning (Dimensionality Reduction)**
   Principal Component Analysis (PCA) uses eigenvectors, related to $A\boldsymbol{x} = \lambda\boldsymbol{x}$ (eigenvalue problem), but the null space helps find dimensions where data has zero variance, revealing intrinsic data structure.

ii. Matrix Transformations from $\mathbb{R}^n$ to $\mathbb{R}^m$

[SI*] Functions of the form

$$\boldsymbol{w} = F(\boldsymbol{x})$$

has independent variable $\boldsymbol{x}$ as a vector in $\mathbb{R}^n$ and the dependent variable $\boldsymbol{w}$ as a vector in $\mathbb{R}^m$. The special class of such functions are called matrix transformations. Such transformations are fundamental in the study of linear algebra and have important application in physics, computer graphics, engineering, social sciences, and various branches of mathematics.

Definition 6.25: Transformation

If V and W are vector spaces, and if f is a function with domain V and codomain W, then we say that f is a transformation from V to W or that f maps V to W, which we denote by writing

$$f : V \to W$$

In the special case where $V = W$, the transformation is also called an operator on V.

[SI*] Suppose that $f_1, f_2, \ldots, f_m$ are real-valued functions of n variables

$$\begin{aligned} w_1 &= f_1(x_1, x_2, \ldots, x_n) \\ w_2 &= f_2(x_1, x_2, \ldots, x_n) \\ &\vdots \\ w_m &= f_m(x_1, x_2, \ldots, x_n) \end{aligned} \tag{6.44}$$

These m equations assign a unique point $(w_1, w_2, \ldots, w_m)$ in $\mathbb{R}^m$ to each point $(x_1, x_2, \ldots, x_n)$ in $\mathbb{R}^n$ and thus define a transformation from $\mathbb{R}^n$ to $\mathbb{R}^m$.

If we denote this transformation by T, then

$$T : \mathbb{R}^n \to \mathbb{R}^m$$

and

$$T(x_1, x_2, \ldots, x_n) = (w_1, w_2, \ldots, w_m)$$

[SI*] In the special case where the equations are linear and can be expressed in the form

$$\begin{aligned} w_1 &= a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n \\ w_2 &= a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n \\ &\vdots \\ w_m &= a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n \end{aligned} \tag{6.45}$$

which we can write in matrix notation as

$$\begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_m \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & \ldots & a_{1n} \\ a_{21} & a_{22} & \ldots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \ldots & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \tag{6.46}$$

or more briefly as

$$\boldsymbol{w} = A\boldsymbol{x} \tag{6.47}$$

Although we could view this as a linear system, we will view it instead as a transformation that maps the column vector $\boldsymbol{x}$ in $\mathbb{R}^n$ into the column vector $\boldsymbol{w}$ in $\mathbb{R}^m$.

We call this a matrix transformation (or matrix operator if $m = n$), and we denote it by

$$T_A : \mathbb{R}^n \to \mathbb{R}^m$$

thus

$$\boldsymbol{w} = T_A(\boldsymbol{x})$$

The matrix transformation T_A is called multiplication by A, and the matrix A is called the standard matrix for the transformation.

On occasion, we also use the notation

$$\boldsymbol{x}\vec{T_A}\boldsymbol{w} \tag{6.48}$$

which is read "T_A maps $\boldsymbol{x}$ into $\boldsymbol{w}$."

Theorem 6.34: Properties of Matrix Transformations

For every matrix A the amtrix transformation $T_A : \mathbb{R}^n \to \mathbb{R}^m$ has the following properties for all vectors $\boldsymbol{u}$ and $\boldsymbol{v}$ in $\mathbb{R}^n$ and for every scalar k:

(a) $T_A(\mathbf{0}) = \mathbf{0}$

(b) $T_A(k\boldsymbol{u}) = kT_A(\boldsymbol{u})$ [Homogeneity property]

(c) $T_A(\boldsymbol{u} + \boldsymbol{v}) = T_A(\boldsymbol{u}) + T_A(\boldsymbol{v})$ [Additivity property]

(d) $T_A(\boldsymbol{u} - \boldsymbol{v}) = T_A(\boldsymbol{u}) - T_A(\boldsymbol{v})$

[SI*] A matrix transformation maps linear combinations of vectors in $\mathbb{R}^n$ into the corresponding linear combinations in $\mathbb{R}^m$

$$T_A(k_1\boldsymbol{u}_1 + k_2\boldsymbol{u}_2 + \cdots + k_r\boldsymbol{u}_r) = k_1T_A(\boldsymbol{u}_1) + k_2T_A(\boldsymbol{u}_2) + \cdots + k_rT_A(\boldsymbol{u}_r) \tag{6.49}$$

Depending on whether n-tuples and m-tuples are regarded as vectors or points, the geometric effect of a matrix transformation

$$T_A : \mathbb{R}^n \to \mathbb{R}^m$$

is to map each vector (point) in $\mathbb{R}^n$ into a vector (point) in $\mathbb{R}^m$.

Theorem 6.35: Same Matrix Transformations

If $T_A : \mathbb{R}^n \to \mathbb{R}^m$ and $T_B : \mathbb{R}^n \to \mathbb{R}^m$ are matrix transformations, and if

$$T_A(\boldsymbol{x}) = T_B(\boldsymbol{x})$$

for every vector $\boldsymbol{x}$ in $\mathbb{R}^n$, then $A = B$.

[SI*] For any matrix A with size of $m \times n$ and standard basis vectors $\boldsymbol{e}_1, \boldsymbol{e}_2, \ldots, \boldsymbol{e}_n$ for $\mathbb{R}^n$

$$A\boldsymbol{e}_j, \quad j = 1, 2, \ldots, n$$

will result as the j-th column of A since every entry of $\boldsymbol{e}_j$ is 0 except for the j-th, which is 1.

[SI*] If 0 is the $m \times n$ matrix, then

$$T_0(\boldsymbol{x}) = 0\boldsymbol{x} = \mathbf{0}$$

so multiplication by zero maps every vector in $\mathbb{R}^n$ into the zero vector in $\mathbb{R}^m$. We call T_0 the zero transformation from $\mathbb{R}^n$ to $\mathbb{R}^m$.

[SI*] If I is the $n \times n$ identity matrix, then

$$T_I(\boldsymbol{x}) = I\boldsymbol{x} = \boldsymbol{x}$$

so multiplication by I maps every vector in $\mathbb{R}^n$ into itself. We call T_I the identity operator on $\mathbb{R}^n$.

[SI*] To find the standard matrix for a matrix transformation from $\mathbb{R}^n$ to $\mathbb{R}^m$ by considering the effect of that transformation on the standard basis vectors for $\mathbb{R}^n$.

Suppose that A is unknown and that

$$\boldsymbol{e}_1, \boldsymbol{e}_2, \ldots, \boldsymbol{e}_n$$

are the standard basis vectors for $\mathbb{R}^n$. Suppose also that the images of these vectors under the transformation T_A are

$$T_A(\boldsymbol{e}_1) = A\boldsymbol{e}_1, \quad T_A(\boldsymbol{e}_2) = A\boldsymbol{e}_2, \ldots, \quad T_A(\boldsymbol{e}_n) = A\boldsymbol{e}_n$$

$A\boldsymbol{e}_j$ is a linear combination of the columns of A in which the successive coefficients are the entries of $\boldsymbol{e}_j$, so the product of $A\boldsymbol{e}_j$ is just the j-th column of the matrix A. Thus,

$$A = [T_A(\boldsymbol{e}_1) \mid T_A(\boldsymbol{e}_2) \mid \ldots \mid T_A(\boldsymbol{e}_n)] \tag{6.50}$$

is the standard matrix for a matrix transformation.

[SI*] **Reflection Operators**
Some of the most basic amtrix operators on $\mathbb{R}^2$ and $\mathbb{R}^3$ are those that map each point into its symmetric image about a fixed line or a fixed plane; these are called reflection operators.

[SI*] **Projection Operators**
Matrix operators on $\mathbb{R}^2$ and $\mathbb{R}^3$ that map each point into its orthogonal projection on a fixed line or plane are called projection operators, or more precisely, orthogonal projection operators.

[SI*] **Rotation Operators**
Matrix operators on $\mathbb{R}^2$ and $\mathbb{R}^3$ that move points along circular arcs called rotation operators.

[SI*] **Dilations and Contractions**
If k is a nonnegative scalar, then the operator

$$T(\boldsymbol{x}) = k\boldsymbol{x}$$

on $\mathbb{R}^2$ or $\mathbb{R}^3$ has the effect of increasing or decreasing the length of each vector by a factor of k.

1. If $0 \leq k \leq 1$ the operator is called a contraction with factor k.
2. If $k > 1$ it is called a dilation with factor k.
3. If $k = 1$, then T is the identity operator and can be regarded either as a contraction or a dilation.

[SI*] **Expansion and Compressions**
In a dilation or contraction of $\mathbb{R}^2$ or $\mathbb{R}^3$, all coordinates are multiplied by a factor k. If only one of the coordinates is multiplied by k, then the resulting operator is called an expansion or compression.

[SI*] **Shears**
A matrix operator of the form $T(x, y) = (x + ky, y)$ translates a point (x, y) in the xy-plane parallel to the x-axis by an amount ky that is proportional to the y-coordinate of the point.

[SI*] Now, when we see a square matrix of size 2×2 or 3×3, if it is in reduced row echelon form and fit the standard matrix we can describe it as matrix operator. For example

$$A_1 = \begin{bmatrix} 1 & 3 \\ 0 & 1 \end{bmatrix}$$

A_1 is a matrix corresponds to a shear in the x-direction with factor 3.

$$A_2 = \begin{bmatrix} 1 & 2 \\ -2 & 1 \end{bmatrix}$$

A_2 is a matrix corresponds to a shear in the x-direction with factor 2, and the y-direction with factor -2.

$$A_3 = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$$

A_3 is a matrix corresponds to an expansion in the x-direction with factor 3.

[SI*] Suppose that T_A is a matrix transformation from $\mathbb{R}^n$ to $\mathbb{R}^k$ and T_B is a matrix transformation from $\mathbb{R}^k$ to $\mathbb{R}^m$. If $\boldsymbol{x}$ is a vector in $\mathbb{R}^n$, then T_A maps this vector into a vector $T_A(\boldsymbol{x})$ in $\mathbb{R}^k$, and T_B maps that vector into the vector $T_B(T_A(\boldsymbol{x}))$ in $\mathbb{R}^m$. This process creates a transformation from $\mathbb{R}^n$ to $\mathbb{R}^m$ that we call the composition of T_B with T_A and denote by the symbol

$$T_B \circ T_A$$

which is read "T_B circle T_A."

The transformation T_A is performed first

$$(T_B \circ T_A)(\boldsymbol{x}) = T_B(T_A(\boldsymbol{x})) \tag{6.51}$$

this composition is itself a matrix transformation since

$$(T_B \circ T_A)(\boldsymbol{x}) = T_B(T_A(\boldsymbol{x})) = B(T_A(\boldsymbol{x})) = B(A\boldsymbol{x}) = (BA)\boldsymbol{x}$$

which shows that it is multiplication by BA

$$T_B \circ T_A = T_{BA} \tag{6.52}$$

[SI*] The standard matrix for a composition is the product of the standard matrices in the appropriate order.

[SI*] Composition not always commutative, when we let $T_A : \mathbb{R}^2 \to \mathbb{R}^2$ be the reflection about the line $y = x$, and let $T_B : \mathbb{R}^2 \to \mathbb{R}^2$ be the orthogonal projection on the y-axis. $T_B \circ T_A$ and $T_A \circ T_B$ will have different effects on a vector $\boldsymbol{x}$ graphically.

$$T_B \circ T_A \neq T_A \circ T_B$$

For another case, let $T_A : \mathbb{R}^2 \to \mathbb{R}^2$ be the reflection about the y-axis, and let $T_B : \mathbb{R}^2 \to \mathbb{R}^2$ be the reflection about the x-axis. In this case $T_B \circ T_A$ and $T_A \circ T_B$ are the same; both map every vector $\boldsymbol{x} = (x, y)$ into its negative $-\boldsymbol{x} = (-x, -y)$:

$$(T_A \circ T_B)(x, y) = T_A(x, -y) = (-x, -y)$$
$$(T_B \circ T_A)(x, y) = T_B(-x, y) = (-x, -y)$$

thus

$$T_B \circ T_A = T_A \circ T_B$$

the standard matrices for T_A and T_B also commute

$$[T_B \circ T_A] = [T_A \circ T_B] = \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix}$$

The operator $T(\boldsymbol{x}) = -\boldsymbol{x}$ on $\mathbb{R}^2$ or $\mathbb{R}^3$ is called the reflection about the origin.

Definition 6.26: One-to-One Matrix Transformations

A matrix transformation $T_A : \mathbb{R}^n \to \mathbb{R}^m$ is said to be one-to-one if T_A maps distinct vectors (points) in $\mathbb{R}^n$ into distinct vectors (points) in $\mathbb{R}^m$.

[SI*] Rotation operators on $\mathbb{R}^2$ are one-to-one since distinct vectors that are rotated through the same angle have distinct images.

The orthogonal projection of $\mathbb{R}^3$ on the xy plane is not one-to-one, because if we maps a vertical line, the whole points in that vector / vertical line will have the same image, remember that the image of (x, y, z) for orthogonal projection on xy plane is $(x, y, 0)$.

Theorem 6.36: One-to-One Matrix Transformations

If A is an $n \times n$ matrix and $T_A : \mathbb{R}^n \to \mathbb{R}^n$ is the corresponding matrix operator, then the following statements are equivalent.

(a) A is invertible.

(b) The range of T_A is in $\mathbb{R}^n$.

(c) T_A is one-to-one.

[SI*] If $T_A : \mathbb{R}^n \to \mathbb{R}^n$ is a one-to-one matrix operator, and if $T_{A^{-1}} : \mathbb{R}^n \to \mathbb{R}^n$ is its inverse, then the standard matrices for these operators are related by the equation

$$T_{A^{-1}} = T_A^{-1} \tag{6.53}$$

[SI*] **Standard Matrix for T^{-1}**
Let $T : \mathbb{R}^n \to \mathbb{R}^n$ be the operator that rotates each vector in $\mathbb{R}^2$ through the angle θ, so the standard matrix is

$$[T] = \begin{bmatrix} \cos\,\theta & -\sin\,\theta \\ \sin\,\theta & \cos\,\theta \end{bmatrix}$$

it is evident geometrically that to undo the effect of T, one must rotate each vector in $\mathbb{R}^2$ through the angle $-\theta$. But this is exactly what the operator T^{-1} does, since the standard matrix for T^{-1} is

$$[T^{-1}] = [T]^{-1} = \begin{bmatrix} \cos\,\theta & \sin\,\theta \\ -\sin\,\theta & \cos\,\theta \end{bmatrix} = \begin{bmatrix} \cos\,(-\theta) & -\sin\,(-\theta) \\ \sin\,(-\theta) & \cos\,(-\theta) \end{bmatrix}$$

which is the standard matrix for a rotation through the angle $-\theta$.

[SI*] If $f_1, f_2, \ldots, f_m$ are any functions of the n variables $x_1, x_2, \ldots, x_n$, then the equations

$$\begin{aligned} w_1 &= f_1(x_1, x_2, \ldots, x_n) \\ w_2 &= f_2(x_1, x_2, \ldots, x_n) \\ &\vdots \\ w_m &= f_m(x_1, x_2, \ldots, x_n) \end{aligned}$$

define a transformation $T : \mathbb{R}^n \to \mathbb{R}^m$ that maps the vector $\boldsymbol{x} = (x_1, x_2, \ldots, x_n)$ into the vector $(w_1, w_2, \ldots, w_m)$. But it is only the case where these equations are linear that T is a matrix transformation.

Theorem 6.37: Algebraic Properties of Transformation $T : \mathbb{R}^n \to \mathbb{R}^m$

$T : \mathbb{R}^n \to \mathbb{R}^m$ is a matrix transformation if and only if the following relationships hold for all vectors $\boldsymbol{u}$ and $\boldsymbol{v}$ in $\mathbb{R}^n$ and for every scalar k:

(a) $T(\boldsymbol{u} + \boldsymbol{v}) = T(\boldsymbol{u}) + T(\boldsymbol{v})$ (additivity property)

(b) $T(k\boldsymbol{u}) = kT(\boldsymbol{u})$ (homogeneity property)

Theorem 6.38: Linear Transformation and Matrix Transformation

Every linear transformation from $\mathbb{R}^n$ to $\mathbb{R}^m$ is a matrix transformation, and conversely, every matrix transformation from $\mathbb{R}^n$ to $\mathbb{R}^m$ is a linear transformation.

[SI*] The inverse of reflection in $\mathbb{R}^2$ and $\mathbb{R}^3$ is the reflection itself toward the same axis / plane. The inverse of dilation is contraction. The inverse of rotation through an angle θ is the rotation through an angle $-\theta$ about the same axis (in $\mathbb{R}^2$ or $\mathbb{R}^3$).

[SI*] By inspection

1. The orthogonal projection on the x-axis in $\mathbb{R}^2$ is not one-to-one because the second row is all 0's, so the determinant is 0.
2. The reflection about the y-axis in $\mathbb{R}^2$ is one-to-one because the determinant is -1.
3. The reflection about the line $y = x$ in $\mathbb{R}^2$ is one-to-one because the determinant is -1.
4. The contraction with factor $k > 0$ in $\mathbb{R}^2$ is one-to-one because the determinant is $k^2 \neq 0$.
5. The rotation about the positive z-axis in $\mathbb{R}^3$ is one-to-one because the determinant is 1.
6. The reflection about the xy-plane in $\mathbb{R}^3$ is one-to-one because the determinant is -1.
7. The dilation with factor $k > 0$ in $\mathbb{R}^3$ is one-to-one because the determinant is $k^3 \neq 0$.

If today we can see the application of matrix transformation in photo editing with computer, in the future the automatic matrix transformation in $\mathbb{R}^3$ will be able to create a better and faster optimal engine, machine, computer' hardware, by resizing it to make it smaller but still with great proportion in a system as a whole, or to create a waste plant that convert trashes to electricity that need compact engine, smaller but able to do bigger job, that perhaps can take 10 million tons of trashes a day with a centralized machine as small as 24 inch CRT TV that able to control the gigantic medium to process the trashes.

iii. Dynamical Systems and Markov Chains

[SI*] A dynamical system is a finite set of variables whose values change with time. The value of a variable at a point in time is called the state of the variable at that time, and the vector formed from these states is called the state of the dynamical system at that time.

[SI*] In many dynamical systems the states of the variables are not known with certainty but can be expressed as probabilities; such dynamical systems are called stochastic processes.
If an experiment or observation has n possible outcomes, then the probabilities of those outcomes must be nonnegative fractions whose sum is 1.

In a stochastic process with n possible states, the state vector at each time t has the form

$$\boldsymbol{x}(t) = \begin{bmatrix} x_1(t) \\ x_2(t) \\ \vdots \\ x_n(t) \end{bmatrix}$$

with $x_i(t)$ is the probability that the system is in state i with $i = 1, 2, \ldots, n$. The entries in this vector must add up to 1 since they account for all n possibilities. In general, a vector with nonnegative entries that add up to 1 is called a probability vector.

[SI*] A square matrix, each of whose columns is a probability vector, is called a stochastic matrix. Such matrices commonly occur in formulas that relate successive states of a stochastic process.

For example, the state vectors $\boldsymbol{x}(k+1)$ and $\boldsymbol{x}(k)$ are related by an equation of the form

$$\boldsymbol{x}(k+1) = P\boldsymbol{x}(k) \tag{6.54}$$

in which

$$\begin{bmatrix} P_A & (1-P_B) \\ (1-P_A) & P_B \end{bmatrix}$$

is a stochastic matrix with:
P_A is the probability that event A will occur again at time $k+1$ after it occurs at time k.
$1 - P_A$ is the probability that event A will not occur at time $k+1$ (event B occurs at $k+1$) after event A occurs at time k.
P_B is the probability that event B will occur again at time $k+1$ after it occurs at time k.
$1 - P_B$ is the probability that event B will not occur at time $k+1$ (event A occurs at $k+1$) after event B occurs at time k.

Definition 6.27: Markov Chain

A Markov chain is a dynamical system whose state vectors at a succession of time interval are probability vectors and for which the state vectors at successive time intervals are related by an equation of the form

$$\boldsymbol{x}(k+1) = P\boldsymbol{x}(k)$$

in which $P = [p_{ij}]$ is a stochastic matrix and p_{ij} is the probability that the system will be in state i at time $t = k+1$ if it is in state j at time $t = k$. The matrix P is called the transition matrix for the system.

[SI*] In a Markov chain with an initial state of $\boldsymbol{x}(0)$, the successive state vectors are

$$\boldsymbol{x}(1) = P\boldsymbol{x}(0), \quad \boldsymbol{x}(2) = P\boldsymbol{x}(1), \quad \boldsymbol{x}(3) = P\boldsymbol{x}(2), \quad \boldsymbol{x}(4) = P\boldsymbol{x}(3), \ldots$$

For brevity, it is common to denote $\boldsymbol{x}(k) = \boldsymbol{x}_k$, which allows us to write the successive state vectors more briefly as

$$\boldsymbol{x}_1 = P\boldsymbol{x}_0, \quad \boldsymbol{x}_2 = P\boldsymbol{x}_1, \quad \boldsymbol{x}_3 = P\boldsymbol{x}_2, \quad \boldsymbol{x}_4 = P\boldsymbol{x}_3, \ldots \tag{6.55}$$

Alternatively, these state vectors can be expressed in terms of the initial state vector $\boldsymbol{x}_0$ as

$$\begin{aligned}
\boldsymbol{x}_1 &= P\boldsymbol{x}_0 \\
\boldsymbol{x}_2 &= P(P\boldsymbol{x}_0) = P^2\boldsymbol{x}_0 \\
\boldsymbol{x}_3 &= P(P^2\boldsymbol{x}_0) = P^3\boldsymbol{x}_0 \\
\boldsymbol{x}_4 &= P(P^3\boldsymbol{x}_0) = P^4\boldsymbol{x}_0
\end{aligned}$$

from which it follows that

$$\boldsymbol{x}_k = P^k\boldsymbol{x}_0 \tag{6.56}$$

[SI*] The matrix

$$P = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

is stochastic and hence can be regarded as the transition matrix for a Markov chain. A real life example for this transition matrix is the changing of day and night. It will always happens like that with probability 1, after day we will have night, and vice versa.
A simple calculation shows that

$$P^2 = I$$

from which it follows that

$$I = P^2 = P^4 = P^6 = \ldots$$

and

$$P = P^3 = P^5 = P^7 = \ldots$$

Thus, the successive states in the Markov chain with initial vector $\boldsymbol{x}_0$ are

$$\boldsymbol{x}_0, \ P\boldsymbol{x}_0, \ \boldsymbol{x}_0, \ P\boldsymbol{x}_0, \ \boldsymbol{x}_0, \ldots$$

which oscillate between $\boldsymbol{x}_0$ and $P\boldsymbol{x}_0$. Thus, the Markov chain does not stabilize unless both components of $\boldsymbol{x}_0$ are $\frac{1}{2}$.

Definition 6.28: Regular Markov Chain

A stochastic matrix P is said to be regular if P or some positive power of P has all positive entries, and a Markov chain whose transition matrix is regular is said to be a regular Markov chain.

Theorem 6.39: Long-term Behavior of Markov Chains: Steady-state Vector

If P is the transition matrix for a regular Markov chain, then:

(a) There is a unique probability vector $\boldsymbol{q}$ such that $P\boldsymbol{q} = \boldsymbol{q}$.

(b) For any initial probability vector $\boldsymbol{x}_0$, the sequence of the state vectors

$$\boldsymbol{x}_0,\ P\boldsymbol{x}_0, \ldots,\ P^k\boldsymbol{x}_0$$

converges to $\boldsymbol{q}$.

The vector $\boldsymbol{q}$ is called the steady-state vector of the Markov chain. It can be found by rewriting the equation as

$$(I - P)\boldsymbol{q} = \mathbf{0}$$

and then solving this equation for $\boldsymbol{q}$ subject to the requirement that $\boldsymbol{q}$ be a probability vector.

[SI*] **Example:**
Suppose that at some initial point in time 100,000 people live in a certain city and 25,000 people live in a village on the mountain. The Regional Planning Commission determines that each year 10% of the city population moves to the village for retirement or choose a healthier and more meaningful lifestyle and 3% of the village population moves to the city for study or work purpose.

(a) Assuming that the total population remains constant, make a table that shows the populations of the city and the village over a five-year period (round to the nearest integer).

(b) Over the long term, how will the population be distributed between the city and the village?

Solution:

(a) Let state 1 be living in the city, and state 2 be living in the village on a mountain. Then the transition matrix is

$$P = \begin{bmatrix} 0.9 & 0.03 \\ 0.1 & 0.97 \end{bmatrix}$$

the initial state vector is

$$\boldsymbol{x}_0 = \begin{bmatrix} 100,000 \\ 25,000 \end{bmatrix}$$

The populations over five-year period will be

$$P\boldsymbol{x}_0 = \begin{bmatrix} 90,750 \\ 34,250 \end{bmatrix}, \quad P^2\boldsymbol{x}_0 = \begin{bmatrix} 82,702 \\ 42,298 \end{bmatrix}, \quad P^3\boldsymbol{x}_0 = \begin{bmatrix} 75,701 \\ 49,299 \end{bmatrix}$$

$$P^4\boldsymbol{x}_0 = \begin{bmatrix} 69,610 \\ 55,390 \end{bmatrix}, \quad P^5\boldsymbol{x}_0 = \begin{bmatrix} 64,311 \\ 60,689 \end{bmatrix}$$

(b) The system

$$(I - P)\boldsymbol{q} = \mathbf{0}$$

Year	1	2	3	4	5
City	90.750	82,702	75,701	69,610	64,311
Village	34,250	42,298	49,299	55,390	60,689

Table 6.1: *The populations of the city and the village over a five-year period.*

$$\begin{bmatrix} 0.1 & -0.03 \\ -0.1 & 0.03 \end{bmatrix} \begin{bmatrix} q_1 \\ q_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

The general solution is

$$q_1 = \frac{3}{10}s, \quad q_2 = s$$

since

$$q_1 + q_2 = 1$$

we will have

$$\frac{3}{10}s + s = 1$$
$$\frac{13}{10}s = 1$$
$$s = \frac{10}{13} = 0.7692308$$

and

$$q_1 = \frac{3}{13} = 0.23076922$$

the steady-state vector is

$$\boldsymbol{q} = \begin{bmatrix} \frac{3}{13} \\ \frac{10}{13} \end{bmatrix} = \begin{bmatrix} 0.23076922 \\ 0.7692308 \end{bmatrix}$$

Over the long term, the population of 125,000 will be distributed with $\frac{3}{13}(125,000) = 28,846$ in the city and $\frac{10}{13}(125,000) = 96,154$ in the village.

[SI*] **Example:**
Physical traits are determined by the genes that an offspring receives from its parents. In the simplest case a trait in the offspring is determined by one pair of genes, one member of the pair inherited from the male parent and the other from the female parent. Typically, each gene in a pair can assume one of two forms, called alleles, denoted by A and a. This leads to three possible pairings

$$AA, \quad Aa, \quad aa$$

called genotypes (the pairs Aa and aA determine the same trait and hence are not distinguished from one another). It is shown in the study of heredity that if a parent of known genotype is crossed with a random parent of unknown genotype probabilities given in the following table, which can be viewed as a transition matrix for a Markov process: Thus, for example, the offspring of a parent of genotype AA that is crossed at random with a parent of unknown genotype will have a 50% chance of being AA, a 50% chance of being Aa, and no chance of being aa.

	AA	**Aa**	**aa**
AA	$\frac{1}{2}$	$\frac{1}{4}$	0
Aa	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{2}$
aa	0	$\frac{1}{4}$	$\frac{1}{2}$

Table 6.2: *The row represents the genotype of offspring and the column represents the genotype of parent.*

(a) Show that the transition matrix is regular

(b) Find the steady-state vector, and discuss its physical interpretation.

Solution:

(a) Let state 1 be the genotype being AA, state 2 be the genotype being Aa, and state 3 be the genotype being aa. Then the transition matrix is

$$P = \begin{bmatrix} \frac{1}{2} & \frac{1}{4} & 0 \\ \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ 0 & \frac{1}{4} & \frac{1}{2} \end{bmatrix}$$

the transition matrix is said to be regular if P or some positive power of P has all positive entries, thus

$$P^2 = \begin{bmatrix} \frac{3}{8} & \frac{1}{4} & \frac{1}{8} \\ \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{8} & \frac{1}{4} & \frac{3}{8} \end{bmatrix}$$

since P^2 has positive entries, then the transition matrix P is regular.

(b) Now to find the steady-state vector $\boldsymbol{q}$ we will use the formula

$$(I - P)\boldsymbol{q} = \mathbf{0}$$

then solving this equation for $\boldsymbol{q}$ we will have

$$\begin{bmatrix} \frac{1}{2} & -\frac{1}{4} & 0 \\ -\frac{1}{2} & \frac{1}{2} & -\frac{1}{2} \\ 0 & -\frac{1}{4} & \frac{1}{2} \end{bmatrix} \begin{bmatrix} q_1 \\ q_2 \\ q_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

the general solution of this system is

$$q_1 = s, \quad q_2 = 2s, \quad q_3 = s$$

for $\boldsymbol{q}$ to be a probability vector, we must have

$$\begin{aligned} q_1 + q_2 + q_3 &= 1 \\ s + 2s + s &= 1 \\ 4s &= 1 \\ s &= \frac{1}{4} \end{aligned}$$

the steady-state vector $\boldsymbol{q}$ is

$$\boldsymbol{q} = \begin{bmatrix} q_1 \\ q_2 \\ q_3 \end{bmatrix} = \begin{bmatrix} \frac{1}{4} \\ \frac{1}{2} \\ \frac{1}{4} \end{bmatrix}$$

over long-term, there is a 50% chance the offspring will have genotype Aa, there will be more offspring with genotype being Aa compared to genotype being AA or aa.

iv. Functions in SymIntegration Related to General Vector Spaces

[SI*] We will list the basic functions that can be used related to general vector spaces:

spanningtest(vector<vector<double>> &matrix)
to compute the determinant of a matrix (composed by column vectors), and if the determinant is not equal to 0 then it spans $\mathbb{R}^n$, with n as the dimension of the matrix.

linearindependencetest(vector<vector<double>> &matrix)
to determine whether the column vectors from the coefficient matrix are linearly independent or linearly dependent in $\mathbb{R}^n$, with n is the length / size of the column vectors.

basistest(vector<vector<double>> &matrix)
to determine whether the column vectors from the coefficient matrix are the basis in $\mathbb{R}^n$, with n is the length / size of the column vectors. Basis means that the column vectors span $\mathbb{R}^n$ and are linearly independent.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Test for Basis ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Test for Basis ]# ./main

Matrix:
              1.000000              4.000000              5.000000
              2.000000              9.000000              8.000000
              2.000000              9.000000              9.000000
             -1.000000             -4.000000             -5.000000

 The column vectors span R^{4}

Augmented Matrix:
              1.000000              4.000000              5.000000              0.000000              0.000000
              2.000000              9.000000              8.000000              0.000000              0.000000
              2.000000              9.000000              9.000000              0.000000              0.000000
             -1.000000             -4.000000             -5.000000              0.000000              0.000000
No unique solution or infinite solutions exist.

 The column vectors are the basis in R^{4}

Time taken by function: 1454 microseconds
```

Figure 6.11: *The basis test for coefficient matrix A that is composed of column vectors $\boldsymbol{v}_1, \boldsymbol{v}_2$ and $\boldsymbol{v}_3$ (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Test for Basis/main.cpp**).*

coordinatevector(vector<vector<double>> &A, vector<vector<double>> &b)
to compute the coordinate vector of $\boldsymbol{v}$ relative to the basis $S = \{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n\}$. Vector $\boldsymbol{v}$ will be the input for $\boldsymbol{b}$ and the basis S is consisting of column vectors that will be the input for matrix A.

basistransition_withcoordinatevector(vector<vector<double>> &oldbasis, vector<vector<double>> &newbasis, vector<double> &w))

to compute the transition matrix from B' to B, the transition matrix from B to B', the coordinate vector $[\boldsymbol{w}]_B$, and $[\boldsymbol{w}]_{B'}$.

rowspacebasis(vector<vector<double>> &A)
to obtain the basis for the row space, with A as the matrix, it can be non square matrix as well.

columnspacebasis(vector<vector<double>> &A)
to obtain the basis for the column space, with A as the matrix, it can be non square matrix as well.

homogeneouslinearsystembasis(vector<vector<double>> &A)
to obtain the basis for the solution space of the linear system $A\boldsymbol{x} = \boldsymbol{0}$, with A as the matrix, it can be non square matrix as well.

normalizehomogeneouslinearsystembasis_matrix(vector<vector<double>> &A)
to obtain the normalize basis for the solution space of the linear system $A\boldsymbol{x} = \boldsymbol{0}$, with A as the matrix, it can be non square matrix as well. The output is a matrix with row vector as the basis, you can then transpose it if you want the basis as the column vector.

vector<double> reflection_yaxis(vector<double> &x)
to compute the reflection of vector $\boldsymbol{x}$ about y-axis in $\mathbb{R}^2$.

vector<double> reflection_xaxis(vector<double> &x)
to compute the reflection of vector $\boldsymbol{x}$ about x-axis in $\mathbb{R}^2$.

vector<double> reflection_linex(vector<double> &x)
to compute the reflection of vector $\boldsymbol{x}$ about the line $y = x$ in $\mathbb{R}^2$.

vector<double> orthogonalprojection_xaxis(vector<double> &x)
to compute the orthogonal projection of vector $\boldsymbol{x}$ on the x-axis in $\mathbb{R}^2$.

vector<double> orthogonalprojection_yaxis(vector<double> &x)
to compute the orthogonal projection of vector $\boldsymbol{x}$ on the y-axis in $\mathbb{R}^2$.

vector<double> rotation_ccw(vector<double> &x, double θ)
to compute the image of vector $\boldsymbol{x}$ under a counterclockwise rotation of θ radians in $\mathbb{R}^2$.

vector<double> contractiondilation(vector<double> &x, double k)
to compute the contraction (if $0 \leq k < 1$) or dilation (if $k > 1$) of vector $\boldsymbol{x}$ with factor k in $\mathbb{R}^n$.

vector<double> compressionexpansion_xdirection(vector<double> &x, double k)
to compute the compression (if $0 \leq k < 1$) or expansion (if $k > 1$) of vector $\boldsymbol{x}$ in the x-direction with factor k in $\mathbb{R}^2$.

vector<double> compressionexpansion_ydirection(vector<double> &x, double k)
to compute the compression (if $0 \leq k < 1$) or expansion (if $k > 1$) of vector $\boldsymbol{x}$ in the y-direction with factor k in $\mathbb{R}^2$.

vector<double> shear_xdirection(vector<double> &x, double k)
to compute the shear of vector $\boldsymbol{x}$ in the x-direction with factor k in $\mathbb{R}^2$.

vector<double> shear_ydirection(vector<double> &x, double k)
to compute the shear of vector $\boldsymbol{x}$ in the y-direction with factor k in $\mathbb{R}^2$.

vector<double> reflection_xyplane(vector<double> &x)
to compute the reflection of vector $\boldsymbol{x}$ about xy-plane in $\mathbb{R}^3$.

vector<double> reflection_xzplane(vector<double> &x)
to compute the reflection of vector $\boldsymbol{x}$ about xz-plane in $\mathbb{R}^3$.

vector<double> reflection_yzplane(vector<double> &x)
to compute the reflection of vector $\boldsymbol{x}$ about yz-plane in $\mathbb{R}^3$.

vector<double> orthogonalprojection_xyplane(vector<double> &x)
to compute the orthogonal projection of vector $\boldsymbol{x}$ on the xy-plane in $\mathbb{R}^3$.

vector<double> orthogonalprojection_xzplane(vector<double> &x)
to compute the orthogonal projection of vector $\boldsymbol{x}$ on the xz-plane in $\mathbb{R}^3$.

vector<double> orthogonalprojection_yzplane(vector<double> &x)
to compute the orthogonal projection of vector $\boldsymbol{x}$ on the yz-plane in $\mathbb{R}^3$.

vector<double> rotation3d_xaxis_ccw(vector<double> &x, double θ)
to compute the the image of vector $\boldsymbol{x}$ under a counterclockwise rotation about the x-axis through an angle θ radians in $\mathbb{R}^3$.

vector<double> rotation3d_yaxis_ccw(vector<double> &x, double θ)
to compute the the image of vector $\boldsymbol{x}$ under a counterclockwise rotation about the y-axis through an angle θ radians in $\mathbb{R}^3$.

vector<double> rotation3d_zaxis_ccw(vector<double> &x, double θ)
to compute the the image of vector $\boldsymbol{x}$ under a counterclockwise rotation about the z-axis through an angle θ radians in $\mathbb{R}^3$.

vector<double> compressionexpansion3d_xdirection(vector<double> &x, double k)
to compute the compression (if $0 \leq k < 1$) or expansion (if $k > 1$) of vector $\boldsymbol{x}$ in the x-direction with factor k in $\mathbb{R}^3$.

vector<double> compressionexpansion3d_ydirection(vector<double> &x, double k)
to compute the compression (if $0 \leq k < 1$) or expansion (if $k > 1$) of vector $\boldsymbol{x}$ in the y-direction with factor k in $\mathbb{R}^3$.

vector<double> compressionexpansion3d_zdirection(vector<double> &x, double k)
to compute the compression (if $0 \leq k < 1$) or expansion (if $k > 1$) of vector $\boldsymbol{x}$ in the z-direction with factor k in $\mathbb{R}^3$.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Matrix Transformation ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Matrix Transformation ]# ./main

Vector x:
5
1

Reflection about the y-axis:
-5
1

Reflection about the x-axis:
5
-1

Reflection about the line y=x:
1
5

Orthogonal projection on the x-axis:
5
0

Orthogonal projection on the y-axis:
0
1

Rotation through an angle π/6 :
3.83013
3.36603

Vector x in 3d:
5
1
7

Reflection about the xy-plane:
5
1
-7

Reflection about the xz-plane:
5
-1
7

Reflection about the yz-plane:
-5
1
7
```

Figure 6.12: *The computation for matrix transformation from* $\mathbb{R}^n$ *to* $\mathbb{R}^m$. *(**SymIntegration/Examples/Linear Algebra/Test SymIntegration Matrix Transformation/main.cpp**).*

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Matrix Transformation ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Matrix Transformation ]# ./main

Vector x:
5
1

Reflection about the y-axis:
-5
1

Reflection about the x-axis:
5
-1

Reflection about the line y=x:
1
5

Orthogonal projection on the x-axis:
5
0

Orthogonal projection on the y-axis:
0
1

Rotation through an angle π/6 :
3.83013
3.36603

Vector x in 3d:
5
1
7

Reflection about the xy-plane:
5
1
-7

Reflection about the xz-plane:
5
-1
7

Reflection about the yz-plane:
-5
1
7
```

Figure 6.13: *The computation for matrix transformation from $\mathbb{R}^n$ to $\mathbb{R}^m$. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Matrix Transformation/main.cpp**).*

MarkovChain(vector<vector<double>>& P, vector<double>& x0, int k)
to compute state vector $\boldsymbol{x}_k$ with P as the transition matrix, $\boldsymbol{x}_0$ as the initial state vector, and k is to represents the time $t = k$.

v. Determine Linear Independence in $\mathbb{R}^n$ with SymIntegration

Determine whether the vectors

$$\begin{aligned} \boldsymbol{v}_1 &= (1, 2, 2, -1) \\ \boldsymbol{v}_2 &= (4, 9, 9, -4) \\ \boldsymbol{v}_3 &= (5, 8, 9, -5) \end{aligned}$$

in $\mathbb{R}^4$ are linearly dependent or linearly independent.

Solution:

The linear independence or linear dependence of these vectors is determined by whether there exist nontrivial solutions of the vector equation

$$k_1\boldsymbol{v}_1 + k_2\boldsymbol{v}_2 + k_3\boldsymbol{v}_3 = \mathbf{0}$$

or, equivalently, of

$$k_1(1, 2, 2, -1) + k_2(4, 9, 9, -4) + k_3(5, 8, 9, -5) = (0, 0, 0, 0)$$

Equating coresponding components on the two sides yields the homogeneous linear system

$$\begin{aligned} k_1 + 4k_2 + 5k_3 &= 0 \\ 2k_1 + 9k_2 + 8k_3 &= 0 \\ 2k_1 + 9k_2 + 9k_3 &= 0 \\ -k_1 - 4k_2 - 5k_3 &= 0 \end{aligned}$$

This sytem has only the trivial solution

$$k_1 = 0, \quad k_2 = 0, \quad k_3 = 0$$

from which you can conclude that $\boldsymbol{v}_1, \boldsymbol{v}_2$, and $\boldsymbol{v}_3$ are linearly independent.

```
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Linear Independence ]# ./main
Error: Could not open file b.txt

Matrix:
              1.000000              4.000000              5.000000
              2.000000              9.000000              8.000000
              2.000000              9.000000              9.000000
             -1.000000             -4.000000             -5.000000
No unique solution or infinite solutions exist.

The column vectors are linearly independent.

Time taken by function: 1051 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Linear Independence ]#
```

Figure 6.14: *The computation to determine whether the column vectors $\boldsymbol{v}_1, \boldsymbol{v}_2$, and $\boldsymbol{v}_3$ are linearly independent or linearly dependent in $\mathbb{R}^4$. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Orthogonal Projection/main.cpp**).*

vi. Compute Coordinate Vector Relative to Basis $S = \{v_1, v_2, v_3\}$ with SymIntegration

The vectors

$$v_1 = (1, 2, 1)$$
$$v_2 = (2, 9, 0)$$
$$v_3 = (3, 3, 4)$$

form a basis for $\mathbb{R}^3$. Find the coordinate vector of $v = (5, -1, 9)$ relative to the basis $S = \{v_1, v_2, v_3\}$.

Solution:

To find $(v)_S$ we must first express v as a linear combination of the vectors in S; that is, we must find values of c_1, c_2, and c_3 such that

$$v = c_1 v_1 + c_2 v_2 + c_3 v_3$$

or, in terms of components,

$$(5, -1, 9) = c_1(1, 2, 1) + c_2(2, 9, 0) + c_3(3, 3, 4)$$

Equating corresponding components gives

$$\begin{aligned} c_1 + 2c_2 + 3c_3 &= 5 \\ 2c_1 + 9c_2 + 3c_3 &= -1 \\ c_1 \quad + 4c_3 &= 9 \end{aligned}$$

Solving this system we obtain $c_1 = 1, c_2 = -1, c_3 = 2$. Therefore,

$$(v)_S = (1, -1, 2)$$

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Coordinate Vectors Relative to Standard Bases ]# ./main


Matrix:
               1.000000              2.000000              3.000000
               2.000000              9.000000              3.000000
               1.000000              0.000000              4.000000

(v)_{S}:
1.000000
-1.000000
2.000000

Time taken by function: 1099 microseconds
```

Figure 6.15: *The computation to find the coordinate vector of $v = (5, -1, 9)$ relative to the basis $S = \{v_1, v_2, v_3\}$. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Coordinate Vectors Relative to Standard Bases/main.cpp**).*

vii. Compute Transition Matrix ($P_{B' \to B}$) from Old Basis (B') to New Basis (B) and the Coordinate Vectors $[\boldsymbol{w}]_B$ and $[\boldsymbol{w}]_{B'}$ with SymIntegration

Consider the bases $B = [\boldsymbol{u}_1, \boldsymbol{u}_2]$ and $B' = [\boldsymbol{u}'_1, \boldsymbol{u}'_2]$ for $\mathbb{R}^2$ where

$$\boldsymbol{u}_1 = \begin{bmatrix} 2 \\ 2 \end{bmatrix}, \quad \boldsymbol{u}_2 = \begin{bmatrix} 4 \\ -1 \end{bmatrix}$$

$$\boldsymbol{u}'_1 = \begin{bmatrix} 1 \\ 3 \end{bmatrix}, \quad \boldsymbol{u}'_2 = \begin{bmatrix} -1 \\ -1 \end{bmatrix}$$

(a) Find the transition matrix from B' to B (from old basis to new basis)

(b) Find the transition matrix from B to B' (from new basis to old basis)

(c) Compute the coordinate vector $[\boldsymbol{w}]_B$ where

$$\boldsymbol{w} = \begin{bmatrix} 3 \\ -5 \end{bmatrix}$$

then compute $[\boldsymbol{w}]_{B'}$

Solution:

(a)

$$\left[\begin{array}{cc|cc} 2 & 4 & 1 & -1 \\ 2 & -1 & 3 & -1 \end{array}\right]$$

reduces to

$$\left[\begin{array}{cc|cc} 1 & 0 & \frac{13}{10} & -\frac{1}{2} \\ 0 & 1 & -\frac{2}{5} & 0 \end{array}\right]$$

so

$$P_{B' \to B} = \begin{bmatrix} \frac{13}{10} & -\frac{1}{2} \\ -\frac{2}{5} & 0 \end{bmatrix}$$

(b)

$$\left[\begin{array}{cc|cc} 1 & -1 & 2 & 4 \\ 3 & -1 & 2 & -1 \end{array}\right]$$

reduces to

$$\left[\begin{array}{cc|cc} 1 & 0 & 0 & -\frac{5}{2} \\ 0 & 1 & -2 & -\frac{13}{2} \end{array}\right]$$

so

$$P_{B \to B'} = \begin{bmatrix} 0 & -\frac{5}{2} \\ -2 & -\frac{13}{2} \end{bmatrix}$$

(c)

$$\left[\begin{array}{cc|cc} 2 & 4 & 1 & 0 \\ 2 & -1 & 0 & 1 \end{array}\right]$$

reduces to

$$\left[\begin{array}{cc|cc} 1 & 0 & \frac{1}{10} & \frac{2}{5} \\ 0 & 1 & \frac{1}{5} & -\frac{1}{5} \end{array}\right]$$

so

$$P_{E\to B} = \begin{bmatrix} \frac{1}{10} & \frac{2}{5} \\ \frac{1}{5} & -\frac{1}{5} \end{bmatrix}$$

where E is the standard basis for $\mathbb{R}^2$. Hence

$$[\boldsymbol{w}]_B = P_{E\to B}[\boldsymbol{w}]_E \begin{bmatrix} \frac{1}{10} & \frac{2}{5} \\ \frac{1}{5} & -\frac{1}{5} \end{bmatrix} \begin{bmatrix} 3 \\ -5 \end{bmatrix} = \begin{bmatrix} -\frac{17}{10} \\ \frac{8}{5} \end{bmatrix}$$

and

$$[\boldsymbol{w}]_{B'} = P_{B\to B'}[\boldsymbol{w}]_B \begin{bmatrix} 0 & -\frac{5}{2} \\ -2 & -\frac{13}{2} \end{bmatrix} \begin{bmatrix} -\frac{17}{10} \\ \frac{8}{5} \end{bmatrix} = \begin{bmatrix} -4 \\ -7 \end{bmatrix}$$

If we are to compute $[\boldsymbol{w}]_{B'}$ directly, then we will have

$$\left[\begin{array}{cc|cc} 1 & -1 & 1 & 0 \\ 3 & -1 & 0 & 1 \end{array}\right]$$

reduces to

$$\left[\begin{array}{cc|cc} 1 & 0 & -\frac{1}{2} & \frac{1}{2} \\ 0 & 1 & -\frac{3}{2} & \frac{1}{2} \end{array}\right]$$

so

$$P_{E\to B'} = \begin{bmatrix} -\frac{1}{2} & \frac{1}{2} \\ -\frac{3}{2} & \frac{1}{2} \end{bmatrix}$$

thus

$$[\boldsymbol{w}]_{B'} = P_{E\to B'}[\boldsymbol{w}]_E \begin{bmatrix} -\frac{1}{2} & \frac{1}{2} \\ -\frac{3}{2} & \frac{1}{2} \end{bmatrix} \begin{bmatrix} 3 \\ -5 \end{bmatrix} = \begin{bmatrix} -4 \\ -7 \end{bmatrix}$$

we obtain the same result, it is because $[\boldsymbol{w}]_{B'} = P_{B\to B'}[\boldsymbol{w}]_B = P_{E\to B'}[\boldsymbol{w}]_E$.

In SymIntegration, we can compute the transition matrix from B' to B, the transition matrix from B to B', the coordinate vector $[\boldsymbol{w}]_B$, and $[\boldsymbol{w}]_{B'}$ with this function:
basistransition_withcoordinatevector(vector<vector<double>> &oldbasis, vector<vector<double>> &newbasis, vector<double> &w))

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Transition from Old Basis to New Basis ]# ./main
************************************************************************************************************************

New basis (B):
               2.000000              4.000000
               2.000000             -1.000000

Old basis (B'):
               1.000000             -1.000000
               3.000000             -1.000000

Transition matrix from old basis to new basis:
               2.000000              4.000000              1.000000             -1.000000
               2.000000             -1.000000              3.000000             -1.000000

Transition matrix from elementary basis to new basis:
               2.000000              4.000000              1.000000              0.000000
               2.000000             -1.000000              0.000000              1.000000

Transition matrix from elementary basis to old basis:
               1.000000             -1.000000              1.000000              0.000000
               3.000000             -1.000000              0.000000              1.000000

[ I | transition from B' to B]:
               1.000000              0.000000              1.300000             -0.500000
               0.000000              1.000000             -0.400000             -0.000000

[ I | transition from E to B]:
               1.000000              0.000000              0.100000              0.400000
               0.000000              1.000000              0.200000             -0.200000
```

Figure 6.16: *The computation to compute the transition matrix from B' to B, the transition matrix from B to B', the coordinate vector $[\boldsymbol{w}]_B$, and $[\boldsymbol{w}]_{B'}$. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Transition from Old Basis to New Basis/main.cpp**).*

```
From matrix [ B | B' ],
P_{B' -> B}:
               1.300000             -0.500000
              -0.400000             -0.000000

From matrix [ B' | B ],
P_{B -> B'}:
               0.000000             -2.500000
              -2.000000             -6.500000

P_{E -> B}:
               0.100000              0.400000
               0.200000             -0.200000

P_{E -> B'}:
              -0.500000              0.500000
              -1.500000              0.500000

w:
3.000000
-5.000000

w_{B}:
-1.700000
1.600000

w_{B'}:
-4.000000
-7.000000

************************************************************************************************************************
```

Figure 6.17: *The computation to compute the transition matrix from B' to B, the transition matrix from B to B', the coordinate vector $[\boldsymbol{w}]_B$, and $[\boldsymbol{w}]_{B'}$. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Transition from Old Basis to New Basis/main.cpp**).*

viii. Compute Basis for Row Space and a Column Space of a Matrix with SymIntegration

Find a basis for the row space and the column space of the matrix

$$A = \begin{bmatrix} 1 & -3 & 4 & -2 & 5 & 4 \\ 2 & -6 & 9 & -1 & 8 & 2 \\ 2 & -6 & 9 & -1 & 9 & 7 \\ -1 & 3 & -4 & 2 & -5 & -4 \end{bmatrix}$$

Solution:

Since elementary row operations do not change the row space of a matrix, we can find a basis for the row space of A by finding a basis for the row space of any row echelon form of A. Reducing A to row echelon form, we obtain

$$R = \begin{bmatrix} 1 & -3 & 4.5 & -0.5 & 4 & 1 \\ 0 & 0 & 1 & 3 & -2 & -6 \\ 0 & 0 & 0 & 0 & 1 & 5 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

The nonzero row vectors of R form a basis for the row space of R and hence form a basis for the row space of A. These basis vectors are

$$r_1 = \begin{bmatrix} 1 & -3 & 4.5 & -0.5 & 4 & 1 \end{bmatrix}$$

$$r_2 = \begin{bmatrix} 0 & 0 & 1 & 3 & -2 & -6 \end{bmatrix}$$

$$r_3 = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 5 \end{bmatrix}$$

The problem of finding a basis for the column space of a matrix A is complicated by the fact that an elementary row operation can alter its column space. However, the good news is that elementary row operations do not alter dependence relationships among the column vectors.

The row echelon form of A is R, keeping in mind that A and R can have different column spaces, we cannot find a basis for the column space of A directly from the column vectors of R.

However, if we can find a set of column vectors of R that forms a basis for the column space of R, then the corresponding column vectors of A will form a basis for the column space of A.

Since the first, third, and fifth columns of R contain the leading 1's of the row vectors, the vectors

$$\boldsymbol{c}_1' = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad \boldsymbol{c}_2' = \begin{bmatrix} 4.5 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \quad \boldsymbol{c}_3' = \begin{bmatrix} -0.5 \\ -2 \\ 1 \\ 0 \end{bmatrix}$$

form a basis for the column space of R. Thus, the corresponding column vectors of A, which are

$$\boldsymbol{c}_1 = \begin{bmatrix} 1 \\ 2 \\ 2 \\ -1 \end{bmatrix}, \quad \boldsymbol{c}_2 = \begin{bmatrix} 4 \\ 9 \\ 9 \\ -4 \end{bmatrix}, \quad \boldsymbol{c}_3 = \begin{bmatrix} 5 \\ 8 \\ 9 \\ -5 \end{bmatrix}$$

form a basis for the column space of A.

In SymIntegration, to obtain the basis for the row space we can use this function:
rowspacebasis(vector<vector<double>> &A)

with A as the matrix, it can be non square matrix as well.

To obtain the basis for the column space we can use this function:
columnspacebasis(vector<vector<double>> &A)

with A as the matrix, it can be non square matrix as well.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Row Space Basis and Column Space Basis ]# ./main

************************************************************************************************************************

A:
              1.000000             -3.000000              4.000000             -2.000000              5.000000              4.000000
              2.000000             -6.000000              9.000000             -1.000000              8.000000              2.000000
              2.000000             -6.000000              9.000000             -1.000000              9.000000              7.000000
             -1.000000              3.000000             -4.000000              2.000000             -5.000000             -4.000000

R:
              1.000000             -3.000000              4.500000             -0.500000              4.000000              1.000000
              0.000000              0.000000              1.000000              3.000000             -2.000000             -6.000000
              0.000000              0.000000              0.000000              0.000000              1.000000              5.000000
              0.000000              0.000000              0.000000              0.000000              0.000000              0.000000

Basis for the row space of the matrix A:
              1.000000             -3.000000              4.500000             -0.500000              4.000000              1.000000
              0.000000              0.000000              1.000000              3.000000             -2.000000             -6.000000
              0.000000              0.000000              0.000000              0.000000              1.000000              5.000000

************************************************************************************************************************

************************************************************************************************************************

A:
              1.000000             -3.000000              4.000000             -2.000000              5.000000              4.000000
              2.000000             -6.000000              9.000000             -1.000000              8.000000              2.000000
              2.000000             -6.000000              9.000000             -1.000000              9.000000              7.000000
             -1.000000              3.000000             -4.000000              2.000000             -5.000000             -4.000000

R:
              1.000000             -3.000000              4.500000             -0.500000              4.000000              1.000000
              0.000000              0.000000              1.000000              3.000000             -2.000000             -6.000000
              0.000000              0.000000              0.000000              0.000000              1.000000              5.000000
              0.000000              0.000000              0.000000              0.000000              0.000000              0.000000

Basis for the column space of the matrix A:
              1.000000              4.000000              5.000000
              2.000000              9.000000              8.000000
              2.000000              9.000000              9.000000
             -1.000000             -4.000000             -5.000000

************************************************************************************************************************

Time taken by function: 2870 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Row Space Basis and Column Space Basis ]#
```

Figure 6.18: *The computation to obtain the basis for the row space and column space of matrix A. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Row Space Basis and Column Space Basis/main.cpp**).*

ix. Compute Basis for the Solution Space of the Linear System $A\boldsymbol{x} = \mathbf{0}$ with SymIntegration

Find the rank, nullity, and the basis for the solution space of the linear system $A\boldsymbol{x} = \mathbf{0}$ (homogeneous system), with matrix

$$A = \begin{bmatrix} -1 & 2 & 0 & 4 & 5 & -3 \\ 3 & -7 & 2 & 0 & 1 & 4 \\ 2 & -5 & 2 & 4 & 6 & 1 \\ 4 & -9 & 2 & -4 & -4 & 7 \end{bmatrix}$$

Solution:

The reduced row echelon form of A is

$$R = \begin{bmatrix} 1 & 0 & -4 & -28 & -37 & 13 \\ 0 & 1 & -2 & -12 & -16 & 5 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Since this matrix has two leading 1's, its row and column spaces are two dimensional and

$$\text{rank}(A) = 2$$

To find the nullity of A, we must find the dimension of the solution space of the linear system $A\boldsymbol{x} = \mathbf{0}$. This system can be solved by reducing its augmented matrix to reduced row echelon form. The resulting matrix will be R with an additional last column of zeros, and hence the corresponding system of equations will be

$$\begin{aligned} x_1 - 4x_3 - 28x_4 - 37x_5 + 13x_6 &= 0 \\ x_2 - 2x_3 - 12x_4 - 16x_5 + 5x_6 &= 0 \end{aligned}$$

Solving these equations for the leading variables yields

$$\begin{aligned} x_1 &= 4x_3 + 28x_4 + 37x_5 - 13x_6 \\ x_2 &= 2x_3 + 12x_4 + 16x_5 - 5x_6 \end{aligned}$$

from which we obtain the general solution

$$\begin{aligned} x_1 &= 4r + 28s + 37t - 13u \\ x_2 &= 2r + 12s + 16t - 5u \\ x_3 &= r \\ x_4 &= s \\ x_5 &= t \\ x_6 &= u \end{aligned}$$

or in column vector form

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{bmatrix} = r \begin{bmatrix} 4 \\ 2 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} + s \begin{bmatrix} 28 \\ 12 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} + t \begin{bmatrix} 37 \\ 16 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} + u \begin{bmatrix} -13 \\ -5 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

Because the four vectors form a basis for the solution space,

$$\text{nullity}(A) = 4$$

In SymIntegration, to obtain the basis for the solution space of the linear system $A\boldsymbol{x} = \mathbf{0}$ we can use this function:
homogeneouslinearsystembasis(vector<vector<double>> &A)

with A as the matrix, it can be non square matrix as well.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Homogeneous Linear System Basis for the Solution Space ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Homogeneous Linear System Basis for the Solution Space ]# ./main

*****************************************************************************************************************

A:
             -1.000000              2.000000              0.000000              4.000000              5.000000             -3.000000
              3.000000             -7.000000              2.000000              0.000000              1.000000              4.000000
              2.000000             -5.000000              2.000000              4.000000              6.000000              1.000000
              4.000000             -9.000000              2.000000             -4.000000             -4.000000              7.000000

Rank(A) = 2
nullity(A) = 4
dim(A) = 6

R:
              1.000000              0.000000             -4.000000            -28.000000            -37.000000             13.000000
              0.000000              1.000000             -2.000000            -12.000000            -16.000000              5.000000
              0.000000              0.000000              0.000000              0.000000              0.000000              0.000000
              0.000000              0.000000              0.000000              0.000000              0.000000              0.000000

Basis for the solution space of Ax = 0 :
              4.000000             28.000000             37.000000            -13.000000
              2.000000             12.000000             16.000000             -5.000000
              1.000000              0.000000              0.000000              0.000000
              0.000000              1.000000              0.000000              0.000000
              0.000000              0.000000              1.000000              0.000000
              0.000000              0.000000              0.000000              1.000000

*****************************************************************************************************************

Time taken by function: 1589 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Homogeneous Linear System Basis for the Solution Space ]#
```

Figure 6.19: *The computation to obtain the basis for the solution space of the linear system $A\boldsymbol{x} = \mathbf{0}$. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Homogeneous Linear System Basis for the Solution Space/main.cpp**).*

x. Compute State Vector $\boldsymbol{x}_k$ in a Markov Chain with SymIntegration

Suppose that Glanz is going to find a nice place to do workout, improve her physical, spiritual and mental power, exercise with Tai Chi, FoY, and Ashtanga Yoga. There are 3 places on the mountain she lives on that can accomodate her needs, they are VP (Valhalla Projection), PB (Puncak Bintang), and Kathmandu. Based on data from 2020 Glanz' migration pattern of moving to certain place can be modeled by a Markov chain with transition matrix

$$P = \begin{bmatrix} 0.97 & 0.51 & 0.6 \\ 0.02 & 0.33 & 0.3 \\ 0.01 & 0.16 & 0.1 \end{bmatrix}$$

That is
$p_{11} = 0.97 =$ probability that Glanz will go to VP tomorrow when she already in VP today.
$p_{12} = 0.51 =$ probability that Glanz will go to VP tomorrow when she already in PB today.
$p_{13} = 0.6 =$ probability that Glanz will go to VP tomorrow when she already in Kathmandu today.
$p_{21} = 0.02 =$ probability that Glanz will go to PB tomorrow when she already in VP today.
$p_{22} = 0.33 =$ probability that Glanz will go to PB tomorrow when she already in PB today.
$p_{23} = 0.3 =$ probability that Glanz will go to PB tomorrow when she already in Kathmandu today.
$p_{31} = 0.01 =$ probability that Glanz will go to Kathmandu tomorrow when she already in VP today.
$p_{32} = 0.16 =$ probability that Glanz will go to Kathmandu tomorrow when she already in PB today.
$p_{33} = 0.1 =$ probability that Glanz will go to Kathmandu tomorrow when she already in Kathmandu today.

Assuming that t is in day and Glanz started all her new cycle of becoming Double-Star Hunter, Nature Golden Child and True Earth Rune bearer from Puncak Bintang at time $t = 0$ (in 2020). Track Glanz' probable locations over a 7-days and after 46-days period. Then determine the steady-state vector $\boldsymbol{q}$, if exist.

Solution:

Let $x_1(k), x_2(k)$, and $x_3(k)$ be the probabilities that Glanz is in VP, PB, and Kathmandu, respectively, at time $t = k$, and let

$$\boldsymbol{x}(k) = \begin{bmatrix} x_1(k) \\ x_2(k) \\ x_3(k) \end{bmatrix}$$

be the state vector at that time. Since we know with certainty that Glanz is in PB at time $t = 0$, the initial state vector is

$$\boldsymbol{x}(0) = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

The state vector over a 7-days period and then after 46-days are

$$\boldsymbol{x}(1) = P\boldsymbol{x}(0) = \begin{bmatrix} 0.51 \\ 0.33 \\ 0.16 \end{bmatrix}, \quad \boldsymbol{x}(2) = P\boldsymbol{x}(1) = \begin{bmatrix} 0.759 \\ 0.1671 \\ 0.0739 \end{bmatrix}, \quad \boldsymbol{x}(3) = P\boldsymbol{x}(2) = \begin{bmatrix} 0.8658 \\ 0.0925 \\ 0.0417 \end{bmatrix}$$

$$\boldsymbol{x}(4) = P\boldsymbol{x}(3) = \begin{bmatrix} 0.912 \\ 0.0604 \\ 0.0276 \end{bmatrix}, \quad \boldsymbol{x}(5) = P\boldsymbol{x}(4) = \begin{bmatrix} 0.9320 \\ 0.0464 \\ 0.0215 \end{bmatrix}, \quad \boldsymbol{x}(6) = P\boldsymbol{x}(5) = \begin{bmatrix} 0.9407 \\ 0.0404 \\ 0.0189 \end{bmatrix}$$

$$\boldsymbol{x}(7) = P\boldsymbol{x}(6) = \begin{bmatrix} 0.9444 \\ 0.0378 \\ 0.0178 \end{bmatrix}, \ldots, \boldsymbol{x}(46) = P\boldsymbol{x}(45) = \begin{bmatrix} 0.9473 \\ 0.0358 \\ 0.0169 \end{bmatrix}$$

Now we take a look again at the entries at matrix P

$$P = \begin{bmatrix} 0.97 & 0.51 & 0.6 \\ 0.02 & 0.33 & 0.3 \\ 0.01 & 0.16 & 0.1 \end{bmatrix}$$

Since the entries of P are positive, the Markov chain is regular and hence has a unique steady-state vector $\boldsymbol{q}$. To find $\boldsymbol{q}$ we will solve the system $(I - P)\boldsymbol{q} = \mathbf{0}$, which we can write as

$$(I - P)\boldsymbol{q} = \mathbf{0}$$

$$\begin{bmatrix} 0.03 & -0.51 & -0.6 \\ -0.02 & 0.67 & -0.3 \\ -0.01 & -0.16 & 0.9 \end{bmatrix} \begin{bmatrix} q_1 \\ q_2 \\ q_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

One-way to determine q_1, q_2, q_3 is to reduce the matrix $(I - P)$ into reduced row echelon form then take parameter solution and then use substitution of the knowledge that $q_1 + q_2 + q_3 = 1$.

Since Gauss-Jordan elimination especially for fraction is not an easy task to be done, thus after we retrieve three equations from $(I - P)\boldsymbol{q}$, all equations are homogeneous (equal to 0), then substitute this

$$q_3 = 1 - q_1 - q_2$$

so we have three equations with 2 unknowns. It is easy to be done now, we can just take two out of the three equations (random picking is not a problem) and then we can compute the value for q_1 and q_2 with simple elimination method from algebra, after that we can get the value for q_3. This is the method of thinking of how we solve with C++ as well.

We will obtain

$$q_1 = 0.947260$$
$$q_2 = 0.035842$$
$$q_3 = 0.016897$$

the steady-state vector is

$$\begin{bmatrix} q_1 \\ q_2 \\ q_3 \end{bmatrix} = \begin{bmatrix} 0.947260 \\ 0.035842 \\ 0.016897 \end{bmatrix}$$

which is consistent with the results obtained above after 46-days period ($\boldsymbol{x}(46)$), the 46-days state vector converge to the steady-state vector.

In SymIntegration to compute state vector $\boldsymbol{x}_k$ we can use this function:
MarkovChain(vector<vector<double>>& P, vector<double>& x0, int k)

with P as the transition matrix, $\boldsymbol{x}_0$ as the initial state vector, and k is to represents the time $t = k$.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Compute Steady-State Vector for Markov Chain ]# ./main

P =
              0.970000              0.510000              0.600000
              0.020000              0.330000              0.300000
              0.010000              0.160000              0.100000

x_{0} =
0.000000
1.000000
0.000000

State vector, k = 46 :
0.947261
0.035842
0.016897

Time taken by function: 1977 microseconds
```

Figure 6.20: *The computation to obtain the state vector $\boldsymbol{x}_k$ at time $t = k = 46$ (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Compute Steady-State Vector for Markov Chain/main.cpp**).*

V. Eigenvalues and Eigenvectors

[SI*] The word *eigen* is from the German word, meaning "characteristic". The underlying idea first appeared in the study of rotational motion but was later used to slassify various kinds of surfaces and to desribe solutions to certain differential equations. It has applications in various fields such as computer graphics, mechanical vibrations, heat flow, population dynamics, quantum mechanics, and economics.

[SI*] Let A be a square matrix, $A \in \mathbb{R}^{n\times n}$. Many of the important properties of a matrix are summarized by its eigenvalues and eigenvectors [**?**].

Definition 6.29: Eigenvalue and Eigenvector

Let $\mathbb{C}$ denote the complex plane. $\lambda \in \mathbb{C}$ is an eigenvalue of A (an $n \times n$ matrix) if and only if there is some $\boldsymbol{x} \in \mathbb{C}^n$, $\boldsymbol{x} \neq \mathbf{0}$ (a nonzero vector $\boldsymbol{x}$ in $\mathbb{R}^n$ is called an eigenvector of A / or the matrix operator T_A), such that

$$A\boldsymbol{x} = \lambda\boldsymbol{x} \tag{6.57}$$

for some scalar λ. The scalar λ is called an eigenvalue of A.

Definition 6.30: Spectrum and Spcetral Radius

Let $\sigma(A)$ denote the set of eigenvalues of A; we refer to it as the spectrum of A. If $A\boldsymbol{x} = \lambda\boldsymbol{x}$ for a nonzero $\boldsymbol{x}$, then $\boldsymbol{x}$ is an eigenvector corresponding to $\lambda \in \sigma(A)$. The spectral radius of A is

$$\rho(A) \equiv \max\{|\lambda| : \lambda \in \sigma(A)\} \tag{6.58}$$

If $\boldsymbol{x}$ is an eigenvector, then so is any scalar multiple of $\boldsymbol{x}$; hence eigenvectors are defined only up to a proportionality constant. Also $\lambda \in \sigma(A)$ if and only if

$$\det(A - \lambda I) = 0 \tag{6.59}$$

the characteristic equation of A, implying that $\lambda \in \sigma(A)$ if and only if $A - \lambda I$ is singular.

[SI*] Eigenvalue problems have a very different character from the problems involving square or rectangular linear systems of equation. For a system of equations, the domain of A could be one space and the range could be a different one. For example, matrix A maps n-vectors of polynomial coefficients to m-vectors of sampled polynomial values. To ask about the eigenvalues of such an A would be meaningless.

Eigenvalue problems make sense only when the range adn the domain spaces are the same. This reflects the fact that in applications, eigenvalues are generally use where a matrix is to be compounded iteratively, either explicitly as a power A^k or implicitly in a functional form such as e^{tA}.

Eigenvalues and eigenvectors are useful for two reasons, one algorithmic, the other physical. Algorithmically, eigenvalue analysis can simplify solutions of certain problems by reducing a coupled system to a collection of scalar problems. Physically, eigenvalue analysis can give insight into the behavior of evolving systems governed by linear equations. The most

familiar examples in this latter class are the study of resonance (e.g., of musical instruments when struck or plucked or bowed) and of stability (e.g., of fluid flows subjected to small perturbations). In such cases eigenvalues tend to be particularly useful for analyzing behavior for large times t.

[SI*] In general, the image of a vector $\boldsymbol{x}$ under multiplication by a square matrix A differs from $\boldsymbol{x}$ in both magnitude and direction.

However, in the special case where $\boldsymbol{x}$ is an eigenvector of A, multiplication by A leaves the direction unchanged.

For example, in $\mathbb{R}^2$ or $\mathbb{R}^3$ multiplication by A maps each eigenvector $\boldsymbol{x}$ of A along the same line through the origin as $\boldsymbol{x}$.

Depending on the sign and magnitude of the eigenvalue λ corresponding to $\boldsymbol{x}$, the operation

$$A\boldsymbol{x} = \lambda\boldsymbol{x}$$

compresses or stretches $\boldsymbol{x}$ by a factor of λ, with a reversal of direction in the case where λ is negative.

[SI*] Note first that the equation

$$A\boldsymbol{x} = \lambda\boldsymbol{x}$$

can be rewritten as

$$A\boldsymbol{x} = \lambda I\boldsymbol{x}$$

or equivalently, as

$$(\lambda I - A)\boldsymbol{x} = \boldsymbol{0}$$

For λ to be an eigenvalue of A this equation must have a nonzero solution for $\boldsymbol{x}$.

Theorem 6.40: Characteristic Equation

If A is an $n \times n$ matrix, then λ is an eigenvalue of A if and only if it satisfies the equation

$$\det(\lambda I - A) = 0 \tag{6.60}$$

This is called the characteristic equation of A.

[SI*] When the $\det(\lambda I - A)$ is expanded, the result is a polynomial $p(\lambda)$ of degree n that is called the characteristic polynomial of A.

Since a polynomial of degree n has atmost n distinct roots, it follows that the equation

$$\lambda^n + c_1\lambda^{n-1} + \cdots + c_n = 0 \tag{6.61}$$

has at most n distinct solutions and consequently that an $n \times n$ matrix has at most n distinct eigenvalues. Some of these solutions may be complex numbers, and it is possible for a matrix to have complex eigenvalues, even if that matrix itself has real entries.

Even if a matrix is real, some of its eigenvalues may be complex. Physically, this is related to the phenomenon that real dynamical systems can have motions that oscillate as well as grow

or decay. Algorithmically, it means that even if the input to a matrix eigenvalue problem is real, the output may have to be complex.

Theorem 6.41: Eigenvalues for Triangular Matrix

If A is an $n \times n$ triangular matrix (upper triangular, lower triangular, or diagonal), then the eigenvalues of A are the entries on the main diagonal of A.

Theorem 6.42: Eigenvalues and Eigenvectors

If A is an $n \times n$ matrix, the following statements are equivalent.

(a) λ is an eigenvalue of A.

(b) The system of equations $(\lambda I - A)\boldsymbol{x} = \mathbf{0}$ has nontrivial solutions.

(c) There is nonzero vector $\boldsymbol{x}$ such that $A\boldsymbol{x} = \lambda\boldsymbol{x}$.

(d) λ is a solution of the characteristic equation $\det(\lambda I - A) = 0$

[SI*] Eigenvectors corresponding to an eigenvalue λ of a matrix A are the nonzero vectors that satisfy the equation

$$(\lambda I - A)\boldsymbol{x} = \mathbf{0}$$

these eigenvectors are the nonzero vectors in the null space of the matrix $\lambda I - A$. We call this null space the eigenspace of A corresponding to λ.

The eigenspace of A corresponding to the eigenvalue λ is the solution space of the homogeneous system $(\lambda I - A)\boldsymbol{x} = \mathbf{0}$.

[SI*] Once the eigenvalues and eigenvectors of a matrix A are found, it is a simple matter to find the eigenvalues and eigenvectors of any positive integer power of A.

If λ is an eigenvalue of A and $\boldsymbol{x}$ is a corresponding eigenvector, then

$$A^2\boldsymbol{x} = A(A\boldsymbol{x}) = A(\lambda\boldsymbol{x}) = \lambda(A\boldsymbol{x}) = \lambda(\lambda\boldsymbol{x}) = \lambda^2\boldsymbol{x}$$

which shows that λ^2 is an eigenvalue of A^2 and that $\boldsymbol{x}$ is a corresponding eigenvector.

Theorem 6.43: Eigenvalue for Powers of a Matrix

If k is a positive integer, λ is an eigenvalue of a matrix A, and $\boldsymbol{x}$ is a corresponding eigenvector, then λ^k is an eigenvalue of A^k and $\boldsymbol{x}$ is a corresponding eigenvector.

Theorem 6.44: Eigenvalue and Invertibility

A square matrix A is invertible if and only if $\lambda = 0$ is not an eigenvalue of A.

[SI*] It is often useful to decompose a matrix into the product of other matrices. A particularly useful decomposition is the Jordan decomposition.

Suppose that A is a $n \times n$ nonsingular square matrix and has n distinct eigenvalues. Then

$$A = NDN^{-1} \tag{6.62}$$

where D is a diagonal matrix with the distinct eigenvalues on the diagonal, and the columns of N are right eigenvectors of A.

$$D = \begin{bmatrix} \lambda_1 & 0 & 0 & \dots & 0 \\ 0 & \lambda_2 & 0 & \dots & 0 \\ 0 & 0 & \lambda_3 & \dots & 0 \\ \vdots & 0 & \dots & \ddots & 0 \\ 0 & 0 & 0 & \dots & \lambda_n \end{bmatrix} \tag{6.63}$$

Equivalently we can express

$$A = N^{-1}DN \tag{6.64}$$

where the diagonal elements of D are the left eigenvalues of A and the rows of N are left eigenvectors of A.

[SI*] **Defective Eigenvalues and Matrices**

If a matrix A has the characteristic polynomial $(z-2)^3$, so there is a single eigenvalue $\lambda = 2$ of algebraic multiplicity 3. In the case of this, we can choose three independent eigenvectors, for example $\boldsymbol{e}_1, \boldsymbol{e}_2$, and $\boldsymbol{e}_3$, so the geometric multiplicity is also 3.

An eigenvalue whose algebraic multiplicity exceeds its geometric multiplicity is a defective eigenvalue. A matrix that has one or more defective eigenvalues is a defective matrix. Any diagonal matrix is nondefective. For such a matrix, both the algebraic and the geometric multiplicities of an eigenvalue λ are equal to the number of its occurrences along the diagonal.

[SI*] **Determinant and Trace**

The trace of $A \in \mathbb{C}^{m \times m}$ is the sum of its diagonal elements:

$$\operatorname{tr}(A) = \sum_{i=1}^{m} a_{ii}$$

both the trace and determinant are related simply to the eigenvalues.

Theorem 6.45: Determinant and Trace Relationship with Eigenvalues

The determinant and trace of a matrix A are equal to the product and the sum of the eigenvalues of A, counted with algebraic multiplicity

$$\det(A) = \prod_{i=1}^{m} \lambda_i \tag{6.65}$$

$$\operatorname{tr}(A) = \sum_{i=1}^{m} \lambda_i \tag{6.66}$$

[SI*] **Unitary Diagonalization**

It sometimes happens that not only does an $m \times m$ matrix A have m linearly independent

eigenvectors, but these can be chosen to be orthogonal. In such a case, A is unitarily diagonalizable, that is, there exists a unitary matrix Q such that

$$A = Q\Lambda Q^* \tag{6.67}$$

This factorization is both an eigenvalue decomposition and a singular value decomposition, aside form the matter of the signs (possibly complex) of the entries of A.

Theorem 6.46: Hermitian Matrix and Eigenvalues

A hermitian matrix is unitarily diagonalizable, and its eigenvalues are real.

Theorem 6.47: Normal Matrix and Unitarily Diagonalizable

A matrix is unitarily diagonalizable, if and only if it is normal ($A^*A = AA^*$).

[SI*] **Unitary Triangularization (Schur Factorization)**

One final matrix factorization is actually one that is most useful in numerical analysis, because all matrices, including defective ones, can be factored in this way. A Schur factorization of a matrix A is a factorization

$$A = QTQ^* \tag{6.68}$$

where Q is unitary and T is upper-triangular. Note that since A and T are similar, the eigenvalues of A necessarily appear on the diagonal of T.

Theorem 6.48: Schur Factorization

Every square matrix A has a Schur factorization.

[SI*] **Eigenvalue-Revealing Factorizations**

1. A diagonalization $A = X\Lambda X^{-1}$ exists if and only if A is nondefective.
2. A unitary diagonalization $A = Q\Lambda Q^*$ exists if and only if A is normal.
3. A unitary triangularization (Schur factorization) $A = QTQ^*$ always exists.

To compute eigenvalues, we shall construct one of these factorizations. In general, this will be the Schur factorization, since this applies without restriction to all matrices. Moreover, since unitary transformations are involved, the algorithms that result tend to be numerically stable.

If A is normal, then the Schur form comes out diagonal, and in particular, if A is hermitian, then we cant ake advantage of this symmetry throughout the computation and reduce A to diagonal form with half as much work or less than is required for general A.

[SI*] **Eigenvalue Algorithms**

For computing eigenvalues and eigenvectors most of the available algorithms proceed in two phases: first, a preliminary reduction from full to structured form; then, an iterative process for the final convergence.

The method of computing eigenvalue by using rootfinding technique on characteristic polynomial is a bad strategy [31]. Polynomial rootfinding is an ill-conditioned problem in general, even when the underlying eigenvalue problem is well-conditioned. Polynomial rootfinding

is a mainstream topic in scientific computing-precisely because it is so rarely the best way to solve applied problems.

It is well known that no formula exists for expressing the roots of an arbitrary polynomial, given its coefficients. This impossibility result is one of the crowning achievements of a body of mathematical work carried out by Abel, Galois, and others in the nineteenth century. Abel proved in 1824 that no analogue of the quadratic formula can exist for polynomials of degree 5 or more. There could be no computer program that would produce the exact roots of an arbitrary polynomial in a finite number of steps. It follows that the same conclusion applies to the more general problem of computing eigenvalues of matrices.

An eigenvalue solver cannot be based on the same kind of techniques that we have used so far for solving linear systems. Methods like Householder reflections and Gaussian elimination would solve linear systems of equations exactly in a finite number of steps if they could be implemented in exact arithmetic. By contrast, any eigenvalue solver must be iterative.

The goal of an eigenvalue solver is to produce sequences of numbers that converge rapidly towards eigenvalues. In this respect eigenvalue computations are more representative of scientific computing than solutions of linear systems of equations.

The need to iterate may seem discouraging at first, but the algorithms available in the field converge extraordinary quickly. In most cases it is possible to compute sequences of numbers that double or triple the numbers of digits of accuracy at every step. Thus, although computing eigenvalues is an "unsolvable" problem in principle, in practice it differs from the solution of linear systems by only a small constant factor, typically closer to 1 than 10.

[SI*] **Properties of Eigenvalues**

1. Eigenvectors with distinct eigenvalues are linearly independent.
2. Singular matrices have zero eigenvalues.
3. If A is a square matrix, then $\lambda = 0$ is not an eigenvalue of A.
4. For a scalar multiple of a matrix, if A is a square matrix and λ is an eigenvalue of A then $k\lambda$ is an eigenvalue of kA.
5. For matrix powers, if A is a square matrix and λ is an eigenvalue of A and $n \geq 0$ is an integer, then λ^n is an eigenvalue of A^n.
6. For polynomial of matrix, if A is a square matrix, λ is an eigenvalue of A and $p(x)$ is a polynomial in variable x, then $p(\lambda)$ is the eigenvalue of matrix $p(A)$.
7. If A is a square matrix, λ is an eigenvalue of A, then λ^{-1} is an eigenvalue of A^{-1}.
8. If A is a square matrix, λ is an eigenvalue of A, then λ is an eigenvalue of A^T.

[SI*] **Properties of Eigenvectors**

1. Eigenvectors provide a concise representation of a matrix, as they capture the most important features of the matrix.
2. Eigenvectors can be used to solve linear systems of equations, which arise in many practical applications.
3. Eigenvectors can be used to diagonalize a matrix, which simplifies many computations, e.g. power and exponentials of the matrix.

4. Eigenvector computation can be computationally expensive, particularly for large matrices.
5. Migenvectors may not exist for all matrices, or they may be complex numbers.
6. Eigenvectors may not be unique, as a matrix may have multiple eigenvectors corresponding to the same eigenvalue.

[SI*] **Applications of Eigenvalues and Eigenvectors**

1. **Stability Analysis**
   In engineering, they help analyze the stability of systems.
2. **Quantum Mechanics**
   In physics, they are used to study the states of quantum systems.
3. **Principal Component Analysis (PCA)**
   In machine learning, they are used to reduce the dimensionality of data. The eigenvectors corresponding to the largest eigenvalues capture the most important features of the data, helping reduce complexity while retaining important patterns.
4. **Google's PageRank Algorithm**
   Eigenvalues and eigenvectors play a key role in Google's PageRank algorithm, which determines the importance of web pages based on link structures. In PageRank, each page is represented as a node, and the links between pages form a matrix. By calculating the eigenvalues and eigenvectors of this matrix, the most important pages (with the highest eigenvector values) are identified. These pages are considered more relevant and are ranked higher in search results. The eigenvector corresponding to the eigenvalue of 1 determines the relative importance of each page in the network. This method allows Google to rank pages based on their connectivity, rather than just the number of incoming links.
5. **Markov Processes**
   In a Markov process, eigenvalues are used to find the stationary distribution, which represents the long-term stable state of the system. The eigenvector corresponding to eigenvalue 1 gives the steady-state probabilities. Additionally, the second-largest eigenvalue determines the rate of convergence to equlibrium.
6. **Electrical Engineering**
   In RLC circuits, the analysis of the currents in the loops can be solved using eigenvalues and eigenvectors. The system of differential equations representing the circuit can be transformed into a matrix form. The eigenvalues and eigenvectors of this matrix provide the general solution for the system, which describes how the current in each loop behaves over time. The eigenvectors represent the natural modes of the circuit, while the eigenvalues determine the rates of change. This method helps in understanding the long-term behavior of the RLC circuit.
7. **Mechanical Engineering**
   Eigenvalues and eigenvectors enable us to "decompose" a linear process into smaller, more manageable tasks. When stress is applied to a "plastic" solid, for example, the deformation can be divided into "principle directions," or the directions where the deformation is greatest. The eigenvectors in the principle directions are the eigenvectors, and the associated eigenvalue is the percentage deformation in each principle direction. Also mechanical engineers utilize eigenvalues and eigenvectors to analyze dynamic systems such as rotating machinery and vehicle dynamics. Eigenvalues determine the

stability of rotational speeds or vehicle maneuvers, while eigenvectors represent the modes of motion.

8. **System of Communication**
   Claude Shannon utilized eigenvalues to calculate the theoretical limit of how much information can be carried via a communication channel such as a telephone line or the air. The eigenvectors and eigenvalues of the communication channel (represented as a matrix) are calculated, and then the eigenvalues are waterfilled. The eigenvalues are then essentially the gains of the channel's fundamental modes, which are recorded by the eigenvectors.

9. **Bridge Construction**
   The smallest magnitude eigenvalue of a system that models the bridge is the natural frequency of the bridge. Engineers use this knowledge to guarantee that their structure are stable.

10. **Automobile Stereo System Design**
    Eigenvalue analysis is also employed in the design of car stereo systems, where it aids in the reproduction of car vibration caused by music.

11. **Oil Exploration**
    Eigenvalue analysis is commonly used by oil firms to explore land for oil. Because oil, dirt, and other substances all produce linear systems with varying eigenvalues, eigenvalue analysis can help pinpoint where oil reserves lie. Oil companies set up probes all-around a site to pick up waves created by a massive truck vibrating the ground. The waves are modified when they move through the different substances in the earth. The oil corporations are directed to possible drilling sites based on the study of these waves.

Guide 6.2: Compute Basis for the Eigenspace Corresponding to an Eigenvalue

These are the methods / procedure to compute basis for the eigenspace corresponding to λ_i.

Suppose we have square matrix A with size of $n \times n$, and then we want to compute its basis for the eigenspace.

1. Compute the eigenvalues, with formula

   $$\det(\lambda I - A) = 0$$

   we will obtain characteristic equation that will result in polynomial of degree n, thus with the help of root finding formula (Bisection, Newton-Raphson) or root quadratic formula we will obtain at most n distint eigenvalues.

2. After we obtain $\lambda_1, \lambda_2, \ldots, \lambda_n$, each value of the eigenvalue if substituted into the equation

   $$(\lambda_i I - A)\boldsymbol{x} = \mathbf{0}$$

   can be used to determine the basis for eigenspace corresponding to λ_i, with $i = 1, 2, \ldots, n$ and $\boldsymbol{x}$ is an eigenvector of A corresponding to λ_i.

3. Compute from $i = 1, 2, \ldots, n$ the matrix

   $$(\lambda_i I - A)\boldsymbol{x} = \mathbf{0}$$

   Solve the system using Gaussian elimination to obtain reduced row echelon form for matrix

   $$(\lambda_i I - A)$$

   we will obtain the solution for $\boldsymbol{x}$ in parameter term, for example like this:

   $$\begin{aligned} x_1 &= 2s \\ x_2 &= 3t - s \\ x_3 &= t \\ &\vdots \\ x_n &= s \end{aligned}$$

   with s, t are example of parameter.

Guide 6.3: Compute Basis for the Eigenspace Corresponding to an Eigenvalue

4. The eigenvectors $\boldsymbol{x}$ of A corresponding to λ_i are the nonzero vector of the form

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} 2s \\ 3t - s \\ t \\ \vdots \\ s \end{bmatrix} = \begin{bmatrix} 2s \\ -s \\ 0 \\ \vdots \\ s \end{bmatrix} + \begin{bmatrix} 0 \\ 3t \\ t \\ \vdots \\ 0 \end{bmatrix} = s \begin{bmatrix} 2 \\ -1 \\ 0 \\ \vdots \\ 1 \end{bmatrix} + t \begin{bmatrix} 0 \\ 3 \\ 1 \\ \vdots \\ 0 \end{bmatrix}$$

   thus we obtain that these eigenvectors below

$$\begin{bmatrix} 2 \\ -1 \\ 0 \\ \vdots \\ 1 \end{bmatrix}, \quad \begin{bmatrix} 0 \\ 3 \\ 1 \\ \vdots \\ 0 \end{bmatrix}$$

   are linearly independent and form a basis for the eigenspace corresponding to λ_i.

5. Do the computation till $i = n$.

Guide 6.4: The Power Method for Finding the Largest Eigenvalue and its Eigenvector

Power method is a numerical method to compute the largest eigenvalue along with its eigenvector of a general square matrix A, it does not have to be symmetric.
Remark: this Power method and the two following iterative methods are only working if the matrix has real eigenvalues.

Suppose we have a matrix A with size $n \times n$.

The algorithm for power method:

1. Start with a random initial vector $\boldsymbol{v}$ with size n.
2. Create a **for** loop with highest number of iterations that we can define ourselves ($N = 50, N = 100, N = 200$), in this loop we compute

$$\begin{aligned} \boldsymbol{w} &= A\boldsymbol{v} \\ ||w|| &= \sqrt{\boldsymbol{w} \cdot \boldsymbol{w}} \\ \boldsymbol{w}_{\text{normalize}} &= \frac{\boldsymbol{w}}{||\boldsymbol{w}||} \\ \boldsymbol{v} &= \boldsymbol{w} \end{aligned}$$

3. After the iteration finishes, we will obtain the eigenvector $\boldsymbol{v}$ and we can compute the eigenvalue with this formula

$$\lambda = \frac{\boldsymbol{v}^T A \boldsymbol{v}}{\boldsymbol{v}^T \boldsymbol{v}}$$

   this formula is called the Rayleigh quotient, it produces an approximate eigenvalue when given an estimated eigenvector.

We will proceed with another two methods to approximate all eigenvalues and eigenvectors, these three methods combine work amazingly well for all general square real matrix (not only work for symmetric matrix, e.g. Jacobi eigenvalue algorithm only works for real symmetric matrix).

Guide 6.5: The Deflation Method for Approximating all the Eigenvalues

The method of deflation proceeds after the power method by reducing the matrix A to an $(n-1)\times(n-1)$ matrix for finding the next largest eigenvalue (the second largest eigenvalue) of this matrix, till the smallest eigenvalue.
Suppose we rank the eigenvalues of matrix A like this

$$\lambda_1 > \lambda_2 > \cdots > \lambda_n$$

with the corresponding eigenvector $\boldsymbol{v}_i$ for λ_i, $i = 1, 2, \ldots, n$

1. We perform the power method to obtain $\boldsymbol{v}_1$ and λ_1, multiply $\boldsymbol{v}_1$ by a permutation matrix P which interchanges the largest element and the first element

$$P\boldsymbol{v}_1 = \boldsymbol{w}_1$$

2. We compute the elementary matrix R such that

$$R^{-1} = \begin{bmatrix} \boldsymbol{w}_1 & \boldsymbol{e}_2 & \boldsymbol{e}_3 \end{bmatrix}$$

and R is the inverse of R^{-1} with

$$R\boldsymbol{w}_1 = \boldsymbol{e}_1$$

3. Let

$$B = RPAP^{-1}R^{-1} = RPAPR^{-1}$$

because $P^{-1} = P$, then

$$\begin{aligned} B\boldsymbol{e}_1 &= RPAPRP^{-1}\boldsymbol{e}_1 \\ &= RPAP\boldsymbol{w}_1 \\ &= RPA\boldsymbol{v}_1 \\ &= \lambda_1 RP\boldsymbol{v}_1 \\ &= \lambda_1 \boldsymbol{e}_1 \end{aligned}$$

thus $\boldsymbol{e}_1$ is an eigenvector of B with eigenvalue λ_1 and B must be an upper triangular matrix with λ_1 as the first element on the leading diagonal.

$$B = \begin{bmatrix} \lambda_1 & \cdots & x^1 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & x^n \end{bmatrix}$$

4. Delete the first column and the first row of matrix $B^{k-1} = RPAP^{-1}R^{-1}$ to give an $(n-1)\times(n-1)$ matrix B^k, then perform the power method on this matrix B^k to obtain the next eigenvalue, with k as the indicator for the iteration number ($k = 1, 2, \ldots, n-1$).

5. Repeat till we have computed all the eigenvalues.

Guide 6.6: The Inverse Iteration with Shift for Computing all the Eigenvectors

The method of deflation is able to compute all the eigenvalues, now we want to compute all the eigenvectors correspond to each of the eigenvalue.

The inverse iteration method [25] is a natural generalization of the power iteration method. If A is an invertible matrix with real, nonzero eigenvalues $\{\lambda_1, \ldots, \lambda_n\}$, then the eigenvalues of A^{-1} are $\{\frac{1}{\lambda_1}, \ldots, \frac{1}{\lambda_n}\}$. Thus if

$$|\lambda_1| > |\lambda_2| > \cdots > |\lambda_n|$$

then

$$|\frac{1}{\lambda_1}| < |\frac{1}{\lambda_2}| < \cdots < |\frac{1}{\lambda_n}|$$

and so by applying the power method iteration on A^{-1}, we can obtain the eigenvector $\boldsymbol{x}_n$ and the eigenvalue λ_n (the smallest eigenvalue).

The advantage of inverse iteration is that it can be easily adapted to find any eigenvalue of the matrix A, instead of just the extreme ones. Observe that for any $\mu \in \mathbb{R}$, the matrix $B = A - \mu I$ has eigenvalues $\{\lambda_1 - \mu, \ldots, \lambda_n - \mu\}$. In particular, by choosing μ to be close to an eigenvalue λ_j of A, we can ensure that $\lambda_j - \mu$ is the eigenvalue of B of the smallest magnitude. Then by applying inverse iteration on B, an approximation to $\boldsymbol{x}_j$ and λ_j can be obtained, with $j = 1, 2, \ldots, n$.

This version of the algorithm is known as inverse iteration with shift

1. Pick some μ close to the desired eigenvalue, in this case since we already know all the eigenvalues (from the deflation method) we will write it like this
$$\mu_i = \lambda_i - 0.1$$
2. Generate a random vector of size n, we name it $\boldsymbol{v}^0$, normalize it so that $||\boldsymbol{v}^0|| = 1$
3. For $k = 1, 2, \ldots N$ (N is the maximum number of iterations)
Solve for $\boldsymbol{w}$
$$(A - \mu I)\boldsymbol{w} = \boldsymbol{v}^{(k-1)}$$
then let
$$\boldsymbol{v}^{(k)} = \frac{\boldsymbol{w}}{||\boldsymbol{w}||}$$

The inverse iteration with shift method works really well in cases where the eigenvalues are already known.

Guide 6.7: The Rayleigh Quotient Iteration

We have learned the method for obtaining an eigenvalue estimate from an eigenvector estimate (the Rayleigh quotient), and another method for obtaining an eigenvector estimate from an eigenvalue estimate (inverse iteration). The Rayleigh quotient iteration idea is to use continually improving eigenvalue estimates to increase the rate of convergence of inverse iteration at every step.

The algorithm:

1. We want to compute the eigenvalues of a matrix A with size $n \times n$.
2. Generate random vector with length of n.

   $$\boldsymbol{v}^{(0)}$$

   and normalize it so that $||\boldsymbol{v}^{(0)}|| = 1$.
3. For $k = 1, 2, \ldots, N$ (define N as the maximum number of iterations)
   Solve

   $$(A - \lambda^{(k-1)} I)\boldsymbol{w} = \boldsymbol{v}^{(k-1)}$$

   for $\boldsymbol{w}$, we need to compute $(A - \lambda^{(k-1)} I)^{-1}$.

   Set

   $$\boldsymbol{v}^{(k)} = \frac{\boldsymbol{w}}{||\boldsymbol{w}||}$$

   it normalizes $\boldsymbol{w}$.

   The Rayleigh quotient is

   $$\lambda^{(k)} = (\boldsymbol{v}^{(k)})^T A \boldsymbol{v}^{(k)}$$

The convergence of this algorithm is spectacular: each iteration triples the number of digits of accuracy.

Theorem 6.49: Rayleigh Quotient Iteration

Rayleigh quotient iteration converges to an eigenvalue/eigenvector pair for all except a set of measure zero of starting vectors $\boldsymbol{v}^{(0)}$. When it converges, the convergence is ultimately cubic in the sense that if λ_i is an eigenvalue of A and $\boldsymbol{v}^0$ is sufficiently close to the eigenvector q_i, then

$$||\boldsymbol{v}^{(k+1)} - (\pm q_i)|| = \boldsymbol{O}(||\boldsymbol{v}^{(k)} - (\pm q_i)||^3) \tag{6.69}$$

and

$$|\lambda^{(k+1)} - \lambda_i| = \boldsymbol{O}(|\lambda^{(k)} - \lambda_i|^3) \tag{6.70}$$

as $k \to \infty$. The $\pm$ signs are not necessarily the same on the two sides Eq. (6.62).

Definition 6.31: Compute Real Eigenvalues with QR Algorithm with Shifts

What makes the QR iteration fly is the introduction of shifts $A \to A - \mu I$ at each step. This idea leads to cubic convergence, thanks to its implicit connection with the Rayleigh quotient iteration, the downside is that it can only be used if the matrix has real eigenvalues only, if there is even one complex eigenvalue, then this method will resulting in matrix A that is getting bigger as the iteration goes by.

The Lecture 29 from Trefethen [31] is a great source of learning for this QR algorithm with shifts.

The shifted QR algorithm:

1. Define the number of iterations as N.
2. For $k = 1, 2 \ldots, N$, set $A^{(k-1)} = A$ and compute $A^{(k-1)} - \mu^{(k)} I$, we choose $\mu^{(k)}$ as the a_{nn}, the entry of matrix $A^{(k-1)}$ at the last column and last row, the bottom right entry.
3. Compute the QR decomposition

   $$A^{(k-1)} - \mu^{(k)} I = Q^{(k)} R^{(k)}$$

   we use QR decomposition with Householder reflection instead of Gram-Schmidt.
4. Compute matrix A for the next iteration

   $$A^{(k)} = R^{(k)} Q^{(k)} + \mu^{(k)} I$$

   set $A = A^{(k)}$ and go to the next iteration till finish / converge.

We will obtain all the eigenvalues at the diagonal of matrix $A^{(N)}$, after that we can use inverse iteration with shift to compute the corresponding eigenvector for each eigenvalue. The source code can be seen at **src/linearalgebra.cpp** at the function (vector<complex<double>> QRAlgorithmwithShifts(vector<vector<complex<double>>>& A, int iterations)).

The funny thing why we make the entry and input all in complex term, even if QR algorithm with shifts is said to only work when the matrix has real eigenvalues only, is that one day with new advancement in science it will not become a waste, there is a possibility that this algorithm / iteration can be developed so it can work to compute complex eigenvalues as well.

[SI*] If a real matrix A has complex conjugate eigenvalues, a basic QR algorithm (even with simple real shifts) will fail to converge to a purely upper triangular matrix. However, it will converge to a real quasi-upper triangular matrix (also known as the real Schur form), where complex eigenvalues are represented by 2×2 blocks on the diagonal.

If the initial matrix A is real and you only use real shifts ($s_k \in \mathbb{R}$), all subsequent matrices

A_k in the iteration

$$A_k - s_k I = Q_k R_k$$
$$A_{k+1} = R_k Q_k + s_k I$$

remain real.

A matrix with complex eigenvalues cannot be similar to a real upper triangular matrix. Therefore, the normal QR algorithm cannot "converge" to a diagonal representation of these eigenvalues. The algorithm might cycle, where the matrix A_k switches between different forms without the off-diagonal elements tending to zero.

To handle complex eigenvalues in real matrices, the QR algorithm is modified into Francis double step QR algorithm, this algorithm accelerate convergence for complex pairs, it uses a "double shift" (s_1, s_2) instead of one shift (s). This allows the algorithm to work with real arithmetic while effectively handling complex eigenvalues, avoiding complex numbers altogether. Double shift QR will successfully finds both real and complex eigenvalues by converging to a real schur form, real schur form is a quasi-upper triangular matrix. In this form, complex eigenvalues appear as a 2×2 block on the diagonal

$$A_2 \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}$$

The eigenvalues of this 2×2 block are complex conjugate pairs, solvable via the quadratic formula (solve $|\lambda I - A_2|$)

$$\lambda_1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a}$$
$$\lambda_2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}$$

with

$$a = 1$$
$$b = -a_{11} - a_{22}$$
$$c = (a_{11}a_{22}) - (a_{12}a_{21})$$

[SI*] **Francis Double Step QR Algorithm**
The QR algorithm with double shift, often called the Francis double step / double-shift QR algorithm, is a robust numerical method to compute all eigenvalues (and the Schur form) of a real non-Hermitian (nonsymmetric) matrix. It is an advanced, fast method for finding eigenvalues of real matrices, by applying two shifts simultaneously in each QR iteration to accelerate convergence, particularly for complex conjugate eigenvalues, we can use the eigenvalues of the bottom-right 2×2 block as shifts (the Francis shift) to rapidly converge to the Schur form, allowing deflation and real-arithmetic computation.

Notes: In many textbooks and sources they are mentioning about using the eigenvalues of the bottom-right 2×2 block as shifts, this could be problematic if the eigenvalues are complex, then it will lead to complex arithmetic, thus from our experience and trial in coding for this, we choose to use the algorithm that is using s, t, x, y, z from Lecture 4 ETH Zurich.

How this algorithm works:

1. **Hessenberg Form**
   First, the matrix is reduced to upper Hessenberg form (nearly triangular) for efficiency, and it keeps the eigenvalues of the original matrix invariant.
2. **Shifts**
   Instead of one shift, two shifts (s_1, s_2) are chosen, either the eigenvalues of the trailing 2×2 submatrix of H (bottom-right 2×2 block) as shifts or if we don't want to use eigenvalues at the bottom-right 2×2 matrix, skip this and continue on to the **Initialize "Bulge"** phase.
3. **Simultaneous QR Step**
   A single "double shift" QR step effectively performs two single shifts at once, but keeps the computation in real arithmetic, avoiding complex numbers for real matrices.
4. **Initialize "Bulge"**
   Instead of calculating $QR = (H - s_1 I)(H - s_2 I)$, calculate the first column of this matrix, which only has 3 non-zero entries. Determine a Householder matrix P_0 that maps this column to a multiple of e_1.

   In order to compute the first householder reflector, P_0. Given an upper Hessenberg matrix H, calculate the first column of $G = H^2 - sH + tI$, where

$$
\begin{aligned}
s &= h_{n-1,n-1} + h_{n,n} \\
t &= (h_{n-1,n-1} * h_{n,n}) - (h_{n-1,n} * h_{n,n-1})
\end{aligned}
$$

   The first three components of $Ge_1 = \boldsymbol{x}$ are

$$
\begin{aligned}
x &= h_{1,1}^2 + h_{1,2}h_{2,1} - sh_{1,1} + t \\
y &= h_{2,1}(h_{1,1} + h_{2,2} - s) \\
z &= h_{2,1}h_{3,2}
\end{aligned}
$$

   this will populate the first three entries for the target vector $\boldsymbol{x}$.

   Construct the Householder vector $\boldsymbol{v}$. Compute a reflector P_0 that maps $\boldsymbol{x} = [x, y, z]^T$ to a multiple of $e_1 = [1, 0, 0]^T$ ($P_0\boldsymbol{x} = \alpha \boldsymbol{e}_1$).

   Let

$$
\boldsymbol{x} = \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} h_{1,1}^2 + h_{1,2}h_{2,1} - sh_{1,1} + t \\ h_{2,1}(h_{1,1} + h_{2,2} - s) \\ h_{2,1}h_{3,2} \end{bmatrix}
$$

   Calculate $\boldsymbol{v}$

$$
\boldsymbol{v} = \boldsymbol{x} + \text{sgn}(x_1)||\boldsymbol{x}||_2 e_1
$$

   If the sign of the first component of vector $\boldsymbol{x}$ is negative then $\text{sgn}(x_1) = 1$, if the sign of the first component of vector $\boldsymbol{x}$ is positive then $\text{sgn}(x_1) = -1$. Normalize $\boldsymbol{v}$

$$
\boldsymbol{v} = \frac{\boldsymbol{v}}{||\boldsymbol{v}||_2}
$$

   Define the first Householder reflector P_0

$$
P_0 = I - 2\boldsymbol{v}\boldsymbol{v}^T
$$

given that $\boldsymbol{v}$ is already normalized. If $\boldsymbol{v}$ is not normalized yet, then use this formula

$$P_0 = I - 2\frac{\boldsymbol{v}\boldsymbol{v}^T}{||\boldsymbol{v}||^2}$$

5. **Bulge Chasing**
   When we apply P_0 to H

$$H \leftarrow P_0^T H P_0$$

   This creates a "bulge" (a 3×3 submatrix) below the Hessenberg form, which is then "chased" down the matrix through a series of transformations (e.g. Householder transformations / rotations or Givens rotations) until it disapppears.
   We use a sequence of Householder reflectors $P_1, P_2, \ldots, P_{n-2}$ to eliminate this "bulge," bringing the matrix to quasi-upper triangular form / real Schur form.

6. **Deflation**
   When the subdiagonal elements become near zero, the matrix "deflates," splitting into smaller subproblems, revealing eigenvalues.

7. **Iteration**
   Repeat until the matrix is in real Schur form (quasi-upper triangular)

It is significantly faster than the basic QR algorithm, it handles complex eigenvalues for real matrices efficiently using only real numbers (Francis double shift), and achieve rapid (often quadratic or better) convergence.

Instead of explicitly computing the QR-decomposition of $(H - s_1 I)(H - s_2 I)$, which is expensive, the implicit Q theorem is used to achieve the same result with far less computation. By utilizing the implicit approach and working with Hessenberg matrices, each step costs $O(n^2)$ flops, rather than $O(n^3)$.

There is also another method called Multi-shift QR algorithm, it generalizes the double shift to use multiple simultaneous shifts for even faster convergence, especially on modern parallel hardware.

Algorithm 3.5 The Francis' Double step QR algorithm

Let $H_0 = H \in \mathbb{R}^{n\times n}$ be an upper Hessenberg matrix. This algorithm computes its real Schur form $H = UTU$ using the Francis double step QR algorithm. T is a quasi upper triangular matrix.
$p := n$; /* p indicates the 'actual' matrix size. */
while $p > 2$ **do**
  $q := p - 1$;
  $s := H_{q,q} + H_{p,p}$; $t := H_{q,q}H_{p,p} - H_{q,p}H_{p,q}$;
  /* compute first 3 elements of first column of M */
  $x := H_{1,1}^2 + H_{1,2}H_{2,1} - sH_{1,1} + t$;
  $y := H_{2,1}(H_{1,1} + H_{2,2} - s)$;
  $z := H_{2,1}H_{3,2}$;
  for $k = 0$ to $p - 3$ **do**
    Determine the Householder reflector P with $P^T\,[x; y; z]^T = \alpha\mathbf{e}_1$;
    $r := \max\{1, k\}$;
    $H_{k+1:k+3,r:n} := P^T H_{k+1:k+3,r:n}$;
    $r := \min\{k + 4, p\}$;
    $H_{1:r,k+1:k+3} := H_{1:r,k+1:k+3}P$;
    $x := H_{k+2,k+1}$; $y := H_{k+3,k+1}$;
    if $k < p - 3$ **then**
      $z := H_{k+4,k+1}$;
    end if
  end for
  Determine the Givens rotation P with $P^T\,[x; y]^T = \alpha\mathbf{e}_1$;
  $H_{q:p,p-2:n} := P^T H_{q:p,p-2:n}$;
  $H_{1:p,p-1:p} := H_{1:p,p-1:p}P$;
  /* check for convergence */
  if $|H_{p,q}| < \varepsilon\,(|H_{q,q}| + |H_{p,p}|)$ **then**
    $H_{p,q} := 0$; $p := p - 1$; $q := p - 1$;
  else if $|H_{p-1,q-1}| < \varepsilon\,(|H_{q-1,q-1}| + |H_{q,q}|)$ **then**
    $H_{p-1,q-1} := 0$; $p := p - 2$; $q := p - 1$;
  end if
end while

Figure 6.21: *The Francis double-step QR algorithm from ETH Zürich' lecture 4, March 14th, 2018: the QR algorithm [3].*

i. Diagonalization

[SI*] A basis for $\mathbb{R}^n$ that consists of eigenvectors of an $n \times n$ matrix A can be used to study geometric properties of A and to simplify various numerical computations.

[SI*] The matrix product

$$P^{-1}AP$$

is called a similarity transformation of the matrix A. Such products are important in the study of eigenvectors and eigenvalues.

Definition 6.32: Similarity

If A and B are square matrices, then we say that B is similar to A if there is an invertible matrix P such that

$$B = P^{-1}AP$$

[SI*] Similar matrices have many properties in common. For example, if $B = P^{-1}AP$, then it

follows that A and B have the same determinant, since

$$\begin{aligned}\det(B) &= \det(P^{-1}AP)\\ &= \det(P^{-1})\det(A)\det(P)\\ &= \frac{1}{\det(P)}\det(A)\det(P)\\ &= \det(A)\end{aligned}$$

In general, any property that is shared by all similar matrices is called a similarity invariant. The list of similarity invariants:

(a) A and $P^{-1}AP$ have the same determinant.

(b) A is invertible if and only if $P^{-1}AP$ is invertible.

(c) A and $P^{-1}AP$ have the same rank.

(d) A and $P^{-1}AP$ have the same nullity.

(e) A and $P^{-1}AP$ have the same trace.

(f) A and $P^{-1}AP$ have the same characteristic polynomial.

(g) A and $P^{-1}AP$ have the same eigenvalues.

(h) If λ is an eigenvalue of A and hence of $P^{-1}AP$, then the eigenspace of A corresponding to λ and the eigenspace of $P^{-1}AP$ corresponding to λ have the same dimension.

Definition 6.33: Diagonalizable

A square matrix A is said to be diagonalizable if it is similar to some diagonal matrix; that is, if there exists an invertible matrix P such that

$$P^{-1}AP$$

is diagonal. In this case the matrix P is said to diagonalize A.

Theorem 6.50: Diagonalizable and Eigenvectors

If A is an $n \times n$ matrix, the following statements are equivalent.

(a) A is diagonalizable.

(b) A has n linearly independent eigenvectors.

[SI*] Matrix A cannot always be diagonalized.

After we obtain the eigenvalues for matrix A with size of $n \times n$, we compute these matrices

$$\begin{aligned}A_1 &= \lambda_1 I - A\\ A_2 &= \lambda_2 I - A\\ &\vdots\\ A_n &= \lambda_n I - A\end{aligned}$$

We know that the eigenvectors corresponding to an eigenvalue λ_i of a matrix A are the nonzero vectors that satisfy the equation

$$(\lambda_i I - A)\boldsymbol{x} = \mathbf{0}$$

these eigenvectors are the nonzero vectors in the null space of the matrix $\lambda_i I - A$. With $i, j = 1, 2, \ldots, n$ and $i \neq j$, thus $\lambda_i \neq \lambda_j$, for repeated root / repeated eigenvalue it will share the same set of eigenvectors.

But, instead of computing the eigenvectors for each matrix $A_1, A_2, \ldots, A_n$ we can just compute the rank of each matrix above, then we can obtain the nullity (the dimension of the null space) and find the sum of all nullity.

$$n = \text{rank}(A) + \text{nullity}(A)$$

If

$$\sum_{i=1}^{n} \text{nullity}(A_i) \neq n$$

then there is no matrix P that diagonalizes A.

Theorem 6.51: Linearly Independent Eigenvectors

If $v_1, v_2, \ldots, v_k$ are eigenvectors of a matrix A corresponding to distinct eigenvalues, then $\{v_1, v_2, \ldots, v_k\}$ is a linearly independent set.

Theorem 6.52: Distinct Eigenvalues

If an $n \times n$ matrix A has n distinct eigenvalues, then A is diagonalizable.

[SI*] We can apply diagonalization to compute high powers of a square matrix A.

If A happens to be diagonalizable, then the computations can be simplified by diagonalizing A.

Suppose that A is a diagonalizable $n \times n$ matrix, that P diagonalizes A, and that

$$P^{-1}AP = \begin{bmatrix} \lambda_1 & 0 & \ldots & 0 \\ 0 & \lambda_2 & \ldots & 0 \\ \vdots & \vdots & & \vdots \\ 0 & 0 & \ldots & \lambda_n \end{bmatrix} = D$$

Squaring both sides of this equation yields

$$(P^{-1}AP)^2 = \begin{bmatrix} \lambda_1^2 & 0 & \ldots & 0 \\ 0 & \lambda_2^2 & \ldots & 0 \\ \vdots & \vdots & & \vdots \\ 0 & 0 & \ldots & \lambda_n^2 \end{bmatrix} = D^2$$

We can rewrite the left side of this equation as

$$(P^{-1}AP)^2 = P^{-1}APP^{-1}AP = P^{-1}AIAP = P^{-1}A^2P$$

from which we obtain the relationship

$$P^{-1}A^2P = D^2$$

More generally, if k is a positive integer, then a similar computation will show that

$$P^{-1}A^kP = D^k = \begin{bmatrix} \lambda_1^k & 0 & \dots & 0 \\ 0 & \lambda_2^k & \dots & 0 \\ \vdots & \vdots & & \vdots \\ 0 & 0 & \dots & \lambda_n^k \end{bmatrix}$$

which we can rewrite as

$$A^k = PD^kP^{-1} = P \begin{bmatrix} \lambda_1^k & 0 & \dots & 0 \\ 0 & \lambda_2^k & \dots & 0 \\ \vdots & \vdots & & \vdots \\ 0 & 0 & \dots & \lambda_n^k \end{bmatrix} P^{-1} \tag{6.71}$$

Computing the right side of this formula involves only three matrix multiplication and the powers of the diagonal entries of D. For matrices of large size and high powers of λ, this involves substantially fewer operations than computing A^k directly.

Theorem 6.53: Eigenvalues of Powers of a Matrix

If λ is an eigenvalue of a square matrix A and $\boldsymbol{x}$ is a corresponding eigenvector, and if k is any positive integer, then λ^k is an eigenvalue of A^k and $\boldsymbol{x}$ is a corresponding eigenvector.

[SI*] If we have a matrix of size 3×3 with characteristic polynomial of

$$(\lambda - 1)(\lambda - 2)^2$$

thus, the eigenspace corresponding to $\lambda = 1$ is at most one-dimensional, and the eigenspace corresponding to $\lambda = 2$ is at most two-dimensional.

If λ_0 is an eigenvalue of an $n \times n$ matrix A, then the dimension of the eigenspace corresponding to λ_0 is called the geometric multiplicity of λ_0, and the number of times that $\lambda - \lambda_0$ appears as a factor in the characteristic polynomial of A is called the algebraic multiplicity of λ_0.

Theorem 6.54: Geometric and Algebraic Multiplicity

If A is a square matrix, then:

(a) For every eigenvalue of A, the geometric multiplicity is less than or equal to the algebraic multiplicity.

(b) A is diagonalizable if and only if the geometric multiplicity of every eigenvalue is equal to the algebraic multiplicity.

ii. Complex Vector Spaces

[SI*] The characteristic equation of any square matrix can have complex solutions, the notion of complex eigenvalues and eigenvectors arise naturally, even within the context of matrices with real entries.

[SI*] If $z = a + bi$ is a complex number, then:

- $\text{Re}(z) = a$ and $\text{Im}(z) = b$ are called the real part of z and the imaginary part of z, respectively.
- $|z| = \sqrt{a^2 + b^2}$ is called the modulus (or absolute value) of z.
- $\overline{z} = a - bi$ is called the complex conjugate of z.
- $z\overline{z} = a^2 + b^2 = |z|^2$
- The angle ϕ is called an argument of z.
- $\text{Re}(z) = |z| \cos\ \phi$
- $\text{Im}(z) = |z| \sin\ \phi$
- $e^{i\phi} = \cos\ \phi + i\ \sin\ \phi$
- $z = |z|(\cos\ \phi + i\ \sin\ \phi)$ is called the polar form of z.

[SI*] It is possible for the characteristic equation of a matrix A with real entries to have imaginary solutions, for example

$$A = \begin{bmatrix} -2 & -1 \\ 5 & 2 \end{bmatrix}$$

$$p(\lambda) = \det(\lambda I - A) = \lambda^2 + 1 = 0$$

the characteristic polynomial above has solutions

$$\lambda^2 + 1 = 0$$
$$\lambda = \pm i$$

[SI*] A vector space in which scalars are allowed to be complex numbers is called a complex vector space.

Definition 6.34: Complex Vector Space

If n is a positive integer, then a complex n-tuple is a sequence of n complex numbers $(v_1, v_2, \ldots, v_n)$. The set of all complex n-tuples is called complex n-space and is denoted by $\mathbb{C}^n$. Scalars are complex numbers, and the operations of addition, subtraction, and scalar multiplication are performed componentwise.

[SI*] If $v_1, v_2, \ldots, v_n$ are complex numbers, then we call $\boldsymbol{v} = (v_1, v_2, \ldots, v_n)$ a vector in $\mathbb{C}^n$ and $v_1, v_2, \ldots, v_n$ its components. Some examples of vectors in $\mathbb{C}^3$ are

$$\boldsymbol{u} = (1 + 8i, 5, -3i)$$
$$\boldsymbol{v} = (1, 0, 2i)$$
$$\boldsymbol{w} = (\pi i, 6 - \sqrt{10}i, \frac{1}{2}i)$$

Every vector

$$\boldsymbol{v} = (v_1, v_2, \ldots, v_n) = (a_1 + b_1 i, a_2 + b_2 i, \ldots, a_n + b_n i)$$

in $\mathbb{C}^n$ can be split into real and imaginary parts as

$$\boldsymbol{v} = (a_1, a_2, \ldots, a_n) + i(b_1, b_2, \ldots, b_n)$$

which can be denoted as

$$\boldsymbol{v} = \mathrm{Re}(\boldsymbol{v}) + i\,\mathrm{Im}(\boldsymbol{v})$$

where

$$\mathrm{Re}(\boldsymbol{v}) = (a_1, a_2, \ldots, a_n)$$
$$\mathrm{Im}(\boldsymbol{v}) = (b_1, b_2, \ldots, b_n)$$

The vector

$$\overline{\boldsymbol{v}} = (\overline{v_1}, \overline{v_2}, \ldots, \overline{v_n}) = (a_1 - b_1 i, a_2 - b_2 i, \ldots, a_n - b_n i)$$

is called the complex conjugate of $\boldsymbol{v}$ and can be expressed in terms of $\mathrm{Re}(\boldsymbol{v})$ and $\mathrm{Im}(\boldsymbol{v})$ as

$$\overline{\boldsymbol{v}} = (a_1, a_2, \ldots, a_n) - i(b_1, b_2, \ldots, b_n) = \mathrm{Re}(\boldsymbol{v}) - i\mathrm{Im}(\boldsymbol{v}) \tag{6.72}$$

It follows that the vectors in $\mathbb{R}^n$ can be viewed as those vectors in $\mathbb{C}^n$ whose imaginary part is zero.

A vector $\boldsymbol{v}$ in $\mathbb{C}^n$ is in $\mathbb{R}^n$ if and only if

$$\overline{\boldsymbol{v}} = \boldsymbol{v}$$

[SI*] We call a matrix A a real matrix if its entries are required to be real numbers and a complex matrix if its entries are allowed to be complex numbers. The standard operations on real matrices carry over to complex matrices without change, and all of familiar properties of matrices continue to hold.
If A is a complex matrix, then $\mathrm{Re}(A)$ and $\mathrm{Im}(A)$ are the matrices formed from the real and imaginary parts of the entries of A, and the $\overline{A}$ is the matrix formed by taking the complex conjugate of each entry in A.

Theorem 6.55: Algebraic Properties of the Complex Conjugate for Vectors

If $\boldsymbol{u}$ and $\boldsymbol{v}$ are vectors in $\mathbb{C}^n$, and if k is a scalar, then:

(a) $\overline{\overline{\boldsymbol{u}}} = \boldsymbol{u}$

(b) $\overline{k\boldsymbol{u}} = k\overline{u}$

(c) $\overline{\boldsymbol{u} + \boldsymbol{v}} = \overline{\boldsymbol{u}} + \overline{\boldsymbol{v}}$

(d) $\overline{\boldsymbol{u} - \boldsymbol{v}} = \overline{\boldsymbol{u}} - \overline{\boldsymbol{v}}$

Theorem 6.56: Algebraic Properties of the Complex Conjugate for Matrices

If A is an $m \times k$ complex matrix and B is a $k \times n$ complex matrix, then:

(a) $\overline{\overline{A}} = A$

(b) $(\overline{A^T}) = (\overline{A})^T$

(c) $\overline{AB} = \overline{A}\,\overline{B}$

Definition 6.35: Complex Euclidean Inner Product

If $\boldsymbol{u} = (u_1, u_2, \ldots, u_n)$ and $\boldsymbol{v} = (v_1, v_2, \ldots, v_n)$ are vectors in $\mathbb{C}^n$, then the complex Euclidean inner product of $\boldsymbol{u}$ and $\boldsymbol{v}$ (also called complex dot product) is denoted by $\boldsymbol{u} \cdot \boldsymbol{v}$ and is defined as

$$\boldsymbol{u} \cdot \boldsymbol{u} = u_1\overline{v_1} + u_2\overline{v_2} + \cdots + u_n\overline{v_n} + \tag{6.73}$$

We also define the Euclidean norm on $\mathbb{C}^n$ to be

$$||\boldsymbol{v}|| = \sqrt{\boldsymbol{v} \cdot \boldsymbol{v}} = \sqrt{|v_1|^2 + |v_2|^2 + \cdots + |v_n|^2} \tag{6.74}$$

As in the real case, we call $\boldsymbol{v}$ a unit vector in $\mathbb{C}^n$ if $||\boldsymbol{v}|| = 1$, and we say two vectors $\boldsymbol{u}$ and $\boldsymbol{v}$ are orthogonal if $\boldsymbol{u} \cdot \boldsymbol{v} = 0$.

[SI*] If $\boldsymbol{u}$ and $\boldsymbol{v}$ are column vectors in $\mathbb{R}^n$, then their dot product can be expressed as

$$\boldsymbol{u} \cdot \boldsymbol{v} = \boldsymbol{u}^T\boldsymbol{v} = \boldsymbol{v}^T\boldsymbol{u}$$

The analogous formulas in $\mathbb{C}^n$ are

$$\boldsymbol{u} \cdot \boldsymbol{v} = \boldsymbol{u}^T\overline{\boldsymbol{v}} = \overline{\boldsymbol{v}}^T\boldsymbol{u} \tag{6.75}$$

Theorem 6.57: Properties of Complex Euclidean Inner Product

If $\boldsymbol{u}, \boldsymbol{v}$, and $\boldsymbol{w}$ are vectors in $\mathbb{C}^n$, and if k is a scalar, then the complex Euclidean inner product has the following properties:

(a) $\boldsymbol{u} \cdot \boldsymbol{v} = \overline{\boldsymbol{v} \cdot \boldsymbol{u}}$ (Antisymmetry property)

(b) $\boldsymbol{u} \cdot (\boldsymbol{v} + \boldsymbol{w}) = \boldsymbol{u} \cdot \boldsymbol{v} + \boldsymbol{u} \cdot \boldsymbol{w}$ (Distributive property)

(c) $k(\boldsymbol{u} \cdot \boldsymbol{v}) = (k\boldsymbol{u}) \cdot \boldsymbol{v}$ (Homogeneity property)

(d) $\boldsymbol{u} \cdot k\boldsymbol{v} = \overline{k}(\boldsymbol{u} \cdot \boldsymbol{v})$ (Antihomogeneity property)

(e) $\boldsymbol{v} \cdot \boldsymbol{v} \geq 0$ and $\boldsymbol{v} \cdot \boldsymbol{v} = 0$ if and only if $\boldsymbol{v} = \boldsymbol{0}$ (Positivity property)

[SI*] Except for the use of complex scalars, the notions of linear combination, linear independence, subspace, spanning, basis, and dimension carry over without change to $\mathbb{C}^n$.

[SI*] Eigenvalues and eigenvectors are defined for complex matrices exactly as for real matrices.

If A is an $n \times n$ matrix with complex entries, then the complex roots of the characteristic equation

$$\det(\lambda I - A) = 0$$

are called complex eigenvalues of A.

As in the real case, λ is a complex eigenvalue of A if and only if there exists a nonzero vector $\boldsymbol{x}$ in $\mathbb{C}^n$ such that

$$A\boldsymbol{x} = \lambda\boldsymbol{x}$$

Each such $\boldsymbol{x}$ is called a complex eigenvector of A corresponding to λ.

The complex eigenvectors of A corresponding to λ are the nonzero solutions of the linear system

$$(\lambda I - A)\boldsymbol{x} = \mathbf{0}$$

and the set of all such solutions is a subspace of $\mathbb{C}^n$, called the eigenspace of A corresponding to λ.

Theorem 6.58: Conjugate Pairs in Eigenvectors and Eigenvalues

If λ is an eigenvalue of a real $n \times n$ matrix A, and if $\boldsymbol{x}$ is a corresponding eigenvector, then $\overline{\lambda}$ is also an eigenvalue of A, and $\overline{\boldsymbol{x}}$ is a corresponding eigenvector.

Theorem 6.59: Determining Eigenvalues of a 2×2 Matrix

If A is a 2×2 matrix with real entries, then the characteristic equation of A is

$$\lambda^2 - \operatorname{tr}(A)\lambda + \det(A) = 0 \tag{6.76}$$

and

(a) A has two distinct real eigenvalues if $\operatorname{tr}(A)^2 - 4\det(A) > 0$;

(b) A has one repeated real eigenvalue if $\operatorname{tr}(A)^2 - 4\det(A) = 0$;

(c) A has two complex conjugate eigenvalues if $\operatorname{tr}(A)^2 - 4\det(A) < 0$

Theorem 6.60: Symmetric Matrices Have Real Eigenvalues

If A is a real symmetric matrix, then A has real eigenvalues.

Theorem 6.61: Geometric Interpretation of Complex Eigenvalues

The eigenvalues of the real matrix

$$C = \begin{bmatrix} a & -b \\ b & a \end{bmatrix} \tag{6.77}$$

are $\lambda = a \pm bi$. If a and b are not both zero, then this matrix can be factored as

$$\begin{bmatrix} a & -b \\ b & a \end{bmatrix} = \begin{bmatrix} |\lambda| & 0 \\ 0 & |\lambda| \end{bmatrix} \begin{bmatrix} \cos\phi & -\sin\phi \\ \sin\phi & \cos\phi \end{bmatrix} \tag{6.78}$$

where ϕ is the angle from the positive x-axis to the ray that joins the origin to the point (a, b).

[SI*] Geometrically, a multiplication by a matrix of form

$$C = \begin{bmatrix} a & -b \\ b & a \end{bmatrix} \tag{6.79}$$

can be viewed as a rotation through the angle ϕ followed by a scaling with factor $|\lambda|$. Since the characteristic equation of C is

$$(\lambda - a)^2 + b^2 = 0$$

from which it follows that the eigenvalues of C are

$$\lambda = a \pm bi$$

Assuming that a and b are not both zero, let ϕ be the angle from the positive x-axis to the ray that joins the origin to the point (a, b). The angle ϕ is an argument of the eigenvalue $\lambda = a + bi$, therefore

$$a = |\lambda| \cos \phi$$
$$b = |\lambda| \sin \phi$$

It follows from this that the matrix in (6.68) can be written as

$$\begin{bmatrix} a & -b \\ b & a \end{bmatrix} = \begin{bmatrix} |\lambda| & 0 \\ 0 & |\lambda| \end{bmatrix} \begin{bmatrix} \frac{a}{|\lambda|} & -\frac{b}{|\lambda|} \\ \frac{b}{|\lambda|} & \frac{a}{|\lambda|} \end{bmatrix} = \begin{bmatrix} |\lambda| & 0 \\ 0 & |\lambda| \end{bmatrix} \begin{bmatrix} \cos \phi & -\sin \phi \\ \sin \phi & \cos \phi \end{bmatrix}$$

Theorem 6.62: Similar Matrix to Real Matrix with Complex Eigenvalues

Let A be a real 2×2 matrix with complex eigenvalues $\lambda = a \pm bi$ (where $b \neq 0$). If $\boldsymbol{x}$ is an eigenvector of A corresponding to $\lambda = a - bi$, then the matrix

$$P = [\text{Re}(\boldsymbol{x}) \quad \text{Im}(\boldsymbol{x})]$$

is invertible and

$$A = P \begin{bmatrix} a & -b \\ b & a \end{bmatrix} P^{-1}$$

[SI*] Let

$$A = PCP^{-1} = PSR_\theta P^{-1} = P \begin{bmatrix} |\lambda| & 0 \\ 0 & |\lambda| \end{bmatrix} \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} P^{-1} \tag{6.80}$$

If we now view P as the transition matrix from the basis $B = \{\text{Re}(\boldsymbol{x}), \text{Im}(\boldsymbol{x})\}$ to the standard basis then computing a product $A\boldsymbol{x}_0$ can be broken down into a three-step process:

1. Map $\boldsymbol{x}_0$ from standard coordinates into B-coordinates by forming the product $P^{-1}\boldsymbol{x}_0$.
2. Rotate and scale the vector $P^{-1}\boldsymbol{x}_0$ by forming the product $SR_\theta P^{-1}\boldsymbol{x}_0$.
3. Map the rotated and scaled vector back to standard coordinates to obtain $A\boldsymbol{x_0} = PSR_\theta P^{-1}\boldsymbol{x}_0$.

iii. Differential Equations

[SI*] Differential equations are equations involving functions and their derivatives. Linear algebra, eigenvalues and eigenvectors can be applied to solving systems of differential equations.

[SI*] A differential equation is an equation involving unknown functions and their derivatives. The order of a differential equation is the order of the highest derivative it contains. The simplest differential equations are the first-order equations of the form

$$y' = ay \tag{6.81}$$

where $y = f(x)$ is an unknown differentiable function to be determined, the basic definition of derivative of a function $y = f(x)$ with respect to x is

$$y' = \frac{dy}{dx}$$

where a is a constant. As with most differential equations, this equation has infinitely many solutions; they are the functions of the form

$$y = ce^{ax} \tag{6.82}$$

where c is an arbitrary constant, we call this function $y = ce^{ax}$ as the general solution of $y' = ay$. Below is the step to obtain this general solution

$$\begin{aligned} y' &= ay \\ \frac{dy}{dx} &= ay \\ \frac{dy}{y} &= adx \\ \int \frac{dy}{y} &= \int adx \\ \ln|y| &= ax + c \\ e^{\ln|y|} &= e^{ax+c} \\ y &= ce^{ax} \end{aligned}$$

Thus every function of this form is a solution of $y' = ay$, if we substitute y then

$$y' = cae^{ax} = ay$$

[SI*] Often a physical problem that leads to a differential equation imposes some conditions that enable us to isolate one particular solution from the general solution.

For example, if we have a diferential equation

$$y' = 5y$$

the general solution is

$$y = ce^{5x}$$

now, there is another condition, that is called initial condition that can be represented with this equation

$$y(0) = 6$$

it means $y = 6$ when $x = 0$, then we substitute the initial condition into $y = ce^{5x}$ to obtain the constant c

$$\begin{aligned} y &= ce^{5x} \\ y(0) &= ce^{5(0)} \\ 6 &= ce^{0} \\ c &= 6 \end{aligned}$$

so with this initial condition we are able to obtain a single line, or a single solution for certain condition, instead of infinitely many solutions, that is

$$y = 6e^{5x}$$

the equation above is the only solution for the differential equation $y' = 5y$ with initial condition $y(0) = 6$.

The problem of solving a differential equation subject to an initial condition is called an initial-value problem.

[SI*] We will be concerned with solving systems of differential equations of the form

$$\begin{aligned} y_1' &= a_{11}y_1 + a_{12}y_2 + \cdots + a_{1n}y_n \\ y_2' &= a_{21}y_1 + a_{22}y_2 + \cdots + a_{2n}y_n \\ &\vdots \\ y_n' &= a_{n1}y_1 + a_{n2}y_2 + \cdots + a_{nn}y_n \end{aligned} \tag{6.83}$$

where $y_1 = f_1(x),\ y_2 = f_2(x), \ldots, y_n = f_n(x)$ are functions to be determined, and the a_{ij} are constants. In matrix notation, we can write it as

$$\begin{bmatrix} y_1' \\ y_2' \\ \vdots \\ y_n' \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

or, more briefly as

$$\boldsymbol{y'} = A\boldsymbol{y} \tag{6.84}$$

where the notation $\boldsymbol{y'}$ denotes the vector obtained by differentiating each component of y.

[SI*] The basic idea for solving a system

$$\boldsymbol{y'} = A\boldsymbol{y}$$

whose coefficient matrix is not diagonal is to introduce a new unknown vector $\boldsymbol{u}$ that is related to the unknown vector $\boldsymbol{y}$ by an equation of the form $\boldsymbol{y} = P\boldsymbol{u}$ in which P is an invertible matrix that diagonalizes A. Of course, such a matrix may or may not exist, but if it does then we can rewrite the equation $\boldsymbol{y'} = A\boldsymbol{y}$ as

$$P\boldsymbol{u'} = A(P\boldsymbol{u})$$

[SI*] If A a square matrix of size $n \times n$ is diagonalizable and

$$\boldsymbol{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

is a solution of the system $\boldsymbol{y'} = A\boldsymbol{y}$, then there will exists a matrix P such that

$$P = [\,\boldsymbol{p}_1\ \boldsymbol{p}_2\ \cdots\ \boldsymbol{p}_n\,]$$

where $\boldsymbol{p}_i$ is an eigenvector corresponding to λ_i. Matrix P diagonalizes A, and

$$D = P^{-1}AP = \begin{bmatrix} \lambda_1 & 0 & 0 & \dots & 0 \\ 0 & \lambda_2 & 0 & \dots & 0 \\ 0 & 0 & \lambda_3 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 0 & \lambda_n \end{bmatrix}$$

Then we make the substitution

$$\boldsymbol{y} = P\boldsymbol{u}$$

$$\boldsymbol{y}' = P\boldsymbol{u}'$$

that will yield the diagonal system

$$\boldsymbol{u}' = D\boldsymbol{u}$$

or

$$\begin{aligned} u_1' &= \lambda_1 u_1 \\ u_2' &= \lambda_2 u_2 \\ &= \vdots \\ u_n' &= \lambda_n u_n \end{aligned}$$

the solution of this system is

$$\begin{aligned} u_1 &= c_1 e^{\lambda_1 x} \\ u_2 &= c_2 e^{\lambda_2 x} \\ &= \vdots \\ u_n &= c_n e^{\lambda_n x} \end{aligned}$$

the equation $\boldsymbol{y} = P\boldsymbol{u}$ will be

$$\boldsymbol{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} = [\boldsymbol{p}_1\ \boldsymbol{p}_2\ \dots\ \boldsymbol{p}_n\ ] \begin{bmatrix} c_1 e^{\lambda_1 x} \\ c_2 e^{\lambda_2 x} \\ \vdots \\ c_n e^{\lambda_n x} \end{bmatrix} = \boldsymbol{p}_1 c_1 e^{\lambda_1 x} + \boldsymbol{p}_2 c_2 e^{\lambda_2 x} + \dots + \boldsymbol{p}_n c_n e^{\lambda_n x}$$

so the general solution of the system is a linear combination of $e^{\lambda_1 x}, e^{\lambda_2 x}, \dots, e^{\lambda_n x}$ where $\lambda_1, \lambda_2, \dots, \lambda_n$ are the eigenvalues of A.

A little note here, The vector column for matrix P that we sometimes wrote $\boldsymbol{p}_i$ is another notation for $\boldsymbol{x}_i$ the eigenvector of A corresponding to λ_i.

$$P = [\boldsymbol{p}_1\ \boldsymbol{p}_2\ \dots\ \boldsymbol{p}_n\ = [\boldsymbol{x}_1\ \boldsymbol{x}_2\ \dots\ \boldsymbol{x}_n$$

Theorem 6.63: Diagonalizable Matrix A and the General Solution

If the coefficient matrix A of the system $\boldsymbol{y}' = A\boldsymbol{y}$ is diagonalizable, then the general solution of the system can be expressed as

$$\boldsymbol{y} = c_1 e^{\lambda_1 x}\boldsymbol{x}_1 + c_2 e^{\lambda_2 x}\boldsymbol{x}_2 + \cdots + c_n e^{\lambda_n x}\boldsymbol{x}_n \tag{6.85}$$

where $\lambda_1, \lambda_2, \ldots, \lambda_n$ are the eigenvalues of A, and $\boldsymbol{x}_i$ is an eigenvector of A corresponding to λ_i.

[SI*] **Example:**
Consider the system of differential equation $\boldsymbol{y}' = A\boldsymbol{y}$, where A is a 2×2 matrix. For what values of $a_{11}, a_{12}, a_{21}, a_{22}$ do the component solutions $y_1(t), y_2(t)$ tend to zero as $t \to \infty$? In particular, what must be true about the determinant and the trace of A for this to happen?
Solution:
From the theorem we know that

$$\boldsymbol{y} = c_1 e^{\lambda_1 t}\boldsymbol{t}_1 + c_2 e^{\lambda_2 t}\boldsymbol{t}_2 \tag{6.86}$$

where λ_1, λ_2 are the eigenvalues of A, and $\boldsymbol{t}_i$ is an eigenvector of A corresponding to λ_i. Now if we take a look at the eigenvector components it will be like this

$$\boldsymbol{t}_1 = \begin{bmatrix} t_{11} \\ t_{12} \end{bmatrix}, \quad \boldsymbol{t}_2 = \begin{bmatrix} t_{21} \\ t_{22} \end{bmatrix}$$

thus

$$y_1(t) = c_1 e^{\lambda_1 t} t_{11} + c_2 e^{\lambda_2 t} t_{21}$$
$$y_2(t) = c_1 e^{\lambda_1 t} t_{12} + c_2 e^{\lambda_2 t} t_{22}$$

when $t \to \infty$, $y_1(t)$ and $y_2(t)$ will tend to zero if and only if

$$\lambda_1 < 0 \quad \text{and} \quad \lambda_2 < 0$$

Now, we will check the condition for the eigenvalues λ_1 and λ_2 with the formula

$$\det(\lambda I - A) = 0$$

$$\begin{vmatrix} \lambda - a_{11} & -a_{12} \\ -a_{21} & \lambda - a_{22} \end{vmatrix} = \lambda^2 - (a_{11} + a_{22})\lambda + (a_{11}a_{22} - a_{12}a_{21}) = 0$$

The eigenvalues formula for the characteristic equation above is

$$\lambda_{1,2} = \frac{(a_{11} + a_{22}) \pm \sqrt{(a_{11} + a_{22})^2 - 4(1)(a_{11}a_{22} - a_{12}a_{21})}}{2(1)}$$

we know that

$$\operatorname{tr}(A) = a_{11} + a_{22}$$
$$\det(A) = a_{11}a_{22} - a_{12}a_{21}$$

we can substitute into the eigenvalues formula

$$\lambda_{1,2} = \frac{\text{tr}(A) \pm \sqrt{(\text{tr}(A))^2 - 4(1)(\det(A))}}{2(1)}$$
$$\lambda_{1,2} = \frac{\text{tr}(A) \pm \sqrt{(\text{tr}(A))^2 - 4(\det(A))}}{2}$$

in order to obtain negative eigenvalues then the conditions need to be fulfilled are

$$\begin{aligned} (\text{tr}(A))^2 - 4(\det(A)) &\geq 0 \\ \text{tr}(A) &< 0 \\ \text{tr}(A) + \sqrt{(\text{tr}(A))^2 - 4(\det(A))} &< 0 \\ \text{tr}(A) - \sqrt{(\text{tr}(A))^2 - 4(\det(A))} &< 0 \end{aligned}$$

if the 4 conditions above are met, then as $t \to \infty$, $y_1(t)$ and $y_2(t)$ will go to zero.

iv. Functions in SymIntegration Related to Eigenvalues and Eigenvectors

[SI*] We will list the basic functions that can be used related to eigenvalues and eigenvectors:

GaussJordanComplexMatrix(vector<vector<complex<double>>> &A)
to reduce complex matrix A into reduced row echelon form with the Gauss-Jordan elimination, the input is a complex matrix, so it can handle real matrix (complex matrix input with no imaginary part in the entries) and complex matrix (complex matrix input with imaginary part in the entries) as well.

PermutationMatrixMax(vector<double>& vectorx)
to returns pemutation matrix P that will swap the first row of vector $\boldsymbol{x}$ with the row that contain the maximum / largest value. P initially is an identity matrix, then the first row of P will be swapped with the row of P that has the maximum value in vector $\boldsymbol{x}$. The permutation matrix P multiply vector $\boldsymbol{x}$ will swap two elements in vector $\boldsymbol{x}$ thus making the element with largest value as the first element / first row for $\boldsymbol{x}$.

penrose(vector<double>& vectorx, vector<double>& vectory)
to compute the matrix M such that $M\boldsymbol{x} = \boldsymbol{y}$ with Moore-Penrose pseudoinverse method.

PowerMethod(vector<vector<double>>& A, int N**)**
to compute the largest eigenvalue and its corresponding eigenvector of a real square matrix A, N is the maximum number of iteration for the numerical method that will be used for the computation.

EigenvaluesEigenvectorsApproximation(vector<vector<double>>& A, int N**)**
to compute all the eigenvalues and eigenvectors of real square matrix A, N is the maximum number of iteration for the numerical method that will be used for the computation.

RayleighQuotientIteration(vector<vector<double>>& A, int N**, double a, double b)**
to compute an eigenvalue and the corresponding eigenvector of real square matrix with Rayleigh quotient iteration with A as the real square matrix, N is the number of iteration, a is

the lower bound for uniform distribution, b is the upper bound for the uniform distribution, we are using uniform distribution to generate random μ and σ as the input to generate the initial vector $\boldsymbol{v}^0$ that is normally distributed with mean μ and standard deviation σ, this initial vector is a prerequisite for the Rayleigh quotient iteration. The key here is to choose a and b carefully so we can pinpoint the range where the eigenvalue could be located.

ComplexRayleighQuotientIteration(vector<vector<complex<double>>> &A, int N_{Rayleigh}, int N double a, double b)
to compute all the real or complex eigenvalues and the corresponding eigenvector of real square matrix or complex square matrix with A as the matrix that we want to compute the eigenvalues and the corresponding eigenvector, N_{Rayleigh} is the maximum number of iteration for the Rayleigh quotient iteration, N is the maximum number of the outermost iteration till we obtain all the correct eigenvalues, a is the lower bound for uniform distribution, b is the upper bound for the uniform distribution, we are using uniform distribution to generate random μ and σ as the input to generate the initial complex vector $\boldsymbol{v}^0$ that is normally distributed with mean μ and standard deviation σ, this initial vector is a prerequisite for the Rayleigh quotient iteration. The key here is to choose a and b carefully so we can pinpoint the range where the eigenvalue could be located.

QRAlgorithmwithShifts(vector<vector<complex<double>>>& A, int N)
to compute the eigenvalues and its corresponding eigenvector, given that the matrix has only real eigenvalues with A as the matrix, and N as the number of iterations we want to use till the matrix converge.
diagonalization(vector<vector<double>>& A)
to compute the matrix P that diagonalizes matrix A and also to compute $P^{-1}AP$.

printComplexMatrix(const ComplexMatrix &A)
to print complex matrix A.

printComplexVector(const ComplexVector &x)
to print complex vector $\boldsymbol{x}$.

vector<vector<double>> ComplextoRealMatrix(const ComplexMatrix &A)
to convert complex matrix A into real matrix, we only take the real argument of $A(\text{re}(A))$.

complex<double> complexdivision(complex<double> z, double k)
to divide a complex number z by a scalar k.

vector<complex<double>> complexvecrand_normal(double μ, double σ, int n)
to produce a vector of random complex number of length n that has normal distribution with mean μ and standard deviation σ.

vector<complex<double>> complexvecrand_uniform(double a, double b, int)
to produce a vector of random complex number of length n that has uniform distribution with range $[a, b)$.

complex<double> conjugate(complex<double> y)
to return the conjugate of a complex number $y = a + bi$.

double complexnorm (const ComplexVector &x)
to compute the norm of a complex vector $\boldsymbol{x}$.

complex<double> complexdotproduct (const ComplexVector &x, const ComplexVector &y)
to compute the complex dot product of two complex vectors $\boldsymbol{x}$ and $\boldsymbol{y}$, denoted by $\boldsymbol{x} \cdot \boldsymbol{y}$.

ComplexVector addComplexVectors(const ComplexVector &x, const ComplexVector &y)
to perform add operation on two complex vectors $\boldsymbol{x}$ and $\boldsymbol{y}$.

ComplexVector subtractComplexVectors(const ComplexVector &x, const ComplexVector &y)
to perform subtraction operation on two complex vectors $\boldsymbol{x}$ and $\boldsymbol{y}$.

ComplexVector scalarmultiplicationComplexVector(const ComplexVector &x, double k)
to perform a scalar multiplication on complex vector $\boldsymbol{x}$ with scalar k.

ComplexVector complexnumbermultiplicationComplexVector(const ComplexVector &x, complex<double> z)
to perform a complex number multiplication on complex vector $\boldsymbol{x}$ with $z = a + bi$.

getcolumnComplexMatrix(const ComplexMatrix& A, int *i*)
to obtain the i-th column of complex matrix A.

ComplexMatrix addComplexMatrices(const ComplexMatrix &A, const ComplexMatrix &B)
to perform add operation on two complex matrices A and B.

ComplexMatrix subtractComplexMatrices(const ComplexMatrix &A, const ComplexMatrix &B)
to perform subtraction operation on two complex matrices A and B.

ComplexMatrix multiplyComplexMatrices(const ComplexMatrix &A, const ComplexMatrix &B)
to perform multiplication operation on two complex matrices A and B.

ComplexMatrix scalarmultiplicationComplexMatrix(const ComplexMatrix &A, double k)
to perform a scalar multiplication on a complex matrix A with scalar k.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Complex Vector Spaces ]# ./main
Matrix A:
       (1,2)       (3,-1)
      (0,-1)        (4,5)

Matrix B:
       (2,0)        (1,3)
      (-1,4)       (0,-2)

Vector x:
(1,1)
(0,1)
(3,-1)

Vector y:
(1,1)
(2,0)
(0,4)

x . y = (-2,-10)

y . x = (-2,10)

|| x || = 3.61

|| y || = 4.69

x + y :
(2,2)
(2,1)
(3,3)

x - y :
(0,0)
(-2,1)
(3,-5)

x * 3 :
(3,3)
(0,3)
(9,-3)

A + B:
       (3,2)        (4,2)
      (-1,3)        (4,3)

A - B:
      (-1,2)       (2,-4)
      (1,-5)        (4,7)

A * B:
      (3,17)      (-7,-1)
     (-24,9)      (13,-9)

A * 3:
       (3,6)       (9,-3)
      (0,-3)      (12,15)

Time taken by function: 1382 microseconds
```

Figure 6.22: *The computation related to complex number, complex vectors and matrices. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Complex Vector Spaces/main.cpp**).*

ComplexMatrix complexnumbermultiplicationComplexMatrix(const ComplexMatrix &A, complex<double> z)
to perform a multiplication between a complex matrix A with a complex number $z = a + bi$.

ComplexVector multiplycomplexmatrixvector(const ComplexMatrix &A, const ComplexVector &x)
to perform multiplication operation between a complex matrix A and a complex vector $\boldsymbol{x}$.

ComplexMatrix createSubMatrix(vector<vector<complex<double>>> A , int i_0, int i_1, int j_0, int j_1)
to create a sub matrix from matrix A from row $i_0 + 1$ to row $i_1 + 1$ and column $j_0 + 1$ to

column $j_1 + 1$, always remember that index of matrix and vector in C++ starts from zero.

ComplexMatrix createIdentityComplexMatrix(int n)
to create a size n identity complex matrix.

ComplexMatrix TransposeComplexMatrix(ComplexMatrix &A)
to return the transpose of a complex matrix A.

ComplexMatrix ComplexMatrixInverse(ComplexMatrix &A)
to compute the inverse of a complex matrix complex matrix A.

IVPSolution_firstorderdiffeq(vector<vector<complex<double>>> &A, vector<complex<double>> &x, double a, double b)
to compute the solution satisfying the initial condition for the system of first-order differential equations with A as the matrix that represents the system of first-order differential equations, x as the vector that represents the initial value $y_i(0)$, a is the lower bound for uniform distribution, b is the upper bound for the uniform distribution, we are using uniform distribution to generate random μ and σ as the input to generate the initial complex vector v^0 that is normally distributed with mean μ and standard deviation σ, this initial vector is a prerequisite for the Rayleigh quotient iteration. The key here is to choose a and b carefully so we can pinpoint the range where the eigenvalue could be located.

FrancisDoubleStepQRReal(vector<vector<complex<double>>>& A, int N)
to compute the real and complex eigenvalues from real matrix with Francis double step QR algorithm using real arithmetic in all the iterations.

FrancisDoubleStepQR(vector<vector<complex<double>>>& A, int N)
to compute the real and complex eigenvalues from real (or complex) matrix with Francis double step QR algorithm using complex arithmetic in all the iterations.

v. Compute Moore-Penrose Pseudoinverse with SymIntegration

In eigenvalue and eigenvector computation, when we use power method we can only compute the largest eigenvalue, to compute other eigenvalues, then we need to deflate the matrix, that is why we need to know about this Moore-Penrose pseudoinverse.

Suppose you know vector v and the identity vector e_1, and you want to compute matrix M such that

$$Mv = e_1$$

Solution:

One simple way is to use

$$M = e_1 \cdot v^+$$

where v^+ is the Moore-Penrose pseudoinverse of v (treated as a column vector).

For a non-zero column vector v:

$$v^+ = \frac{v^T}{v^T v}$$

where $v^T v$ is the dot product of v and itself.

So

$$M = \frac{e_1 v^T}{v^T v}$$

The matrix will map v to e_1.

Now, if we want to use an example with vector

$$v = \begin{bmatrix} 5 \\ 6 \\ 7 \end{bmatrix}$$

and

$$e_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

thus

$$M = \frac{e_1 v^T}{v^T v}$$

$$M = \frac{1}{v \cdot v} \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 5 & 6 & 7 \end{bmatrix} = \frac{1}{110} \begin{bmatrix} 5 & 6 & 7 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0.045455 & 0.54545 & 0.063636 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

$(v \cdot v = v^T v)$

In SymIntegration to obtain the matrix M we can use this function:
penrose(vector<double>& vectorx, vector<double>& vectory)

vi. Compute Real Eigenvalues and Its Corresponding Eigenvectors with SymIntegration

Find the eigenvalues and eigenvectors of

$$A = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 4 & -17 & 8 \end{bmatrix}$$

Solution:

The characteristic polynomial of A is

$$\det(\lambda I - A) = \det \begin{bmatrix} \lambda & -1 & 0 \\ 0 & \lambda & -1 \\ -4 & 17 & \lambda - 8 \end{bmatrix} = \lambda^3 - 8\lambda^2 + 17\lambda - 4$$

The eigenvalues of A must therefore satisfy the cubic equations

$$\lambda^3 - 8\lambda^2 + 17\lambda - 4 = 0$$

To solve this equation, we will begin by searching for integer solutions. This task can be simplified by exploiting the fact that all integer solutions (if there are any) of a polynomial equation with integer coefficients

$$\lambda^n + c_1\lambda^{n-1} + \cdots + c_n = 0$$

must be divisors of the constant term, c_n. Thus, the only possible integer solutions are the divisors of -4, that is $\pm 1, \pm 2, \pm 4$. Successively substituting these values in the cubic equations / characteristic equation shows that $\lambda = 4$ is an integer solution. As a consequence $\lambda - 4$ must be a factor of the left side of the characteristic equation. Dividing $\lambda - 4$ into $\lambda^3 - 8\lambda^2 + 17\lambda - 4$ shows that the characteristic equation can be rewritten as

$$(\lambda - 4)(\lambda^2 - 4\lambda + 1) = 0$$

Thus, the remaining solutions satisfy the quadratic equation

$$\lambda^2 - 4\lambda + 1 = 0$$

which can be solved by the quadratic fomula. Thus, the eigenvalues of A are

$$\lambda = 4$$
$$\lambda = 2 + \sqrt{3}$$
$$\lambda = 2 - \sqrt{3}$$

It could be harder if we have square matrix of size 4×4 or more, thus we would opt for computing eigenvalues and eigenvectors with numerical method.

In SymIntegration, this function can be used to compute all the eigenvalues and eigenvectors of real square matrix:
EigenvaluesEigenvectorsApproximation(vector<vector<double>>& A, int N**)**

with A as the matrix that we want to compute the eigenvalues and eigenvectors, N is the number of iterations for the numerical methods that we are going to deploy. The numerical methods are: the power method (as the initial / start to compute the largest eigenvalue and its corresponding eigenvector), the deflation method (to compute all the eigenvalues), and last is the inverse iteration with shift (to compute all the eigenvectors, given the eigenvalues are known).

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Eigenvalues Approximation with Power Method, Deflation Method and Inverse Iterati
on with Shift ]# ./main

A:
              0.000000              1.000000              0.000000
              0.000000              0.000000              1.000000
              4.000000            -17.000000              8.000000

Eigenvalue(1) = 4.008940

Corresponding Eigenvector:
-0.060523
-0.242091
-0.968364

Eigenvalue(2) = 3.721781

Corresponding Eigenvector:
0.069184
0.258199
0.963611

Eigenvalue(3) = 0.267949

Corresponding Eigenvector:
0.963611
0.258199
0.069184

Eigenvectors (in column vectors):
             -0.060523              0.069184              0.963611
             -0.242091              0.258199              0.258199
             -0.968364              0.963611              0.069184

Time taken by function: 3375 microseconds
```

Figure 6.23: *The computation to approximate all eigenvalues and eigenvectors of matrix A. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Eigenvalues Approximation with Power Method, Deflation Method and Inverse Iteration with Shift/main.cpp**).*

As a comparison we try another matrix, suppose we have a matrix A

$$A = \begin{bmatrix} -4 & 12 & -12 \\ 0 & 5 & -6 \\ -2 & -2 & 10 \end{bmatrix}$$

We try to approximate the eigenvalues and eigenvectors with **Eigen** and **SymIntegration** to compare the computation speed / computation cost. Both have the same results, only **Eigen** has faster time due to its only have header library. But the compiling time takes quite a long time on our laptop for **Eigen** compared to **SymIntegration**.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Eigenvalues Approximation with Power Method, Deflation Method and Inverse Iterati
on with Shift ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Eigenvalues Approximation with Power Method, Deflation Method and Inverse Iterati
on with Shift ]# ./main

A:
              -4.000000              12.000000             -12.000000
               0.000000               5.000000              -6.000000
              -2.000000              -2.000000              10.000000

Eigenvalue(1) = 13.678780

Corresponding Eigenvector:
0.686585
0.413453
-0.598045

Eigenvalue(2) = -4.678780

Corresponding Eigenvector:
-0.985014
-0.090874
-0.146591

Eigenvalue(3) = 2.000000

Corresponding Eigenvector:
0.666667
0.666667
0.333333

Eigenvectors (in column vectors):
               0.686585              -0.985014               0.666667
               0.413453              -0.090874               0.666667
              -0.598045              -0.146591               0.333333

Time taken by function: 3433 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Eigenvalues Approximation with Power Method, Deflation Method and Inverse Iterati
on with Shift ]#
```

Figure 6.24: *The computation to approximate all eigenvalues and eigenvectors of matrix A with SymIntegration with SymIntegration. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Eigenvalues Approximation with Power Method, Deflation Method and Inverse Iteration with Shift/main.cpp**).*

```
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Eigenvalues Approximation/Compute Eigenvalues and Eigenvectors with Eigen ]# ./ma
n
Matrix A:
 -4  12 -12
  0   5  -6
 -2  -2  10

Eigenvalues:
λ_1 = -4.67878 + 0i
λ_2 = 13.6788 + 0i
λ_3 = 2 + 0i

Eigenvectors:
x[λ_1] = (-0.985014 + 0i, -0.0908736 + 0i, -0.146591 + 0i )

x[λ_2] = (0.686585 + 0i, 0.413453 + 0i, -0.598045 + 0i )

x[λ_3] = (0.666667 + 0i, 0.666667 + 0i, 0.333333 + 0i )


Time taken by function: 3107 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Eigenvalues Approximation/Compute Eigenvalues and Eigenvectors with Eigen ]#
```

Figure 6.25: *The computation to approximate all eigenvalues and eigenvectors of matrix A with Eigen. (**SymIntegration/Examples/Linear Algebra/Compute Eigenvalues and Eigenvectors with Eigen/main.cpp**).*

We also want to test Rayleigh quotient iteration to compute the eigenvalue and the corresponding eigenvector, this method is said to converge fast, but we need to run it multiple times to obtain

all the eigenvalues, with SymIntegration.

In SymIntegration, this function can be used to compute an eigenvalue and the corresponding eigenvector of real square matrix with Rayleigh quotient iteration:
RayleighQuotientIteration(vector<vector<double>>& A, int N, double a, double b)

with A as the real square matrix, N is the number of iteration, a is the lower bound for uniform distribution, b is the upper bound for the uniform distribution, we are using uniform distribution to generate random μ and σ as the input to generate the initial vector v^0 that is normally distributed with mean μ and standard deviation σ, this initial vector is a prerequisite for the Rayleigh quotient iteration. The key here is to choose a and b carefully so we can pinpoint the range where the eigenvalue could be located.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Eigenvalues Approximation with Rayleigh Quotient Iteration ]
# ./main

v initial vector_{normalize} =
0.372864
-0.386006
0.843784

λ^{0} = 12.0216

Eigenvalue = 4.13413

Corresponding Eigenvector:
-0.0561771
-0.233992
-0.970614

Time taken by function: 4678 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Eigenvalues Approximation with Rayleigh Quotient Iteration ]
# ./main

v initial vector_{normalize} =
0.692503
0.428555
0.580328

λ^{0} = 0.619297

Eigenvalue = 0.267949

Corresponding Eigenvector:
-0.963611
-0.258199
-0.0691842

Time taken by function: 4678 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Eigenvalues Approximation with Rayleigh Quotient Iteration ]
# ./main

v initial vector_{normalize} =
0.687873
0.657233
-0.308019

λ^{0} = 3.60263

Eigenvalue = 3.73205

Corresponding Eigenvector:
0.0691842
0.258199
0.963611

Time taken by function: 4679 microseconds
```

Figure 6.26: *The computation to approximate an eigenvalue and its corresponding eigenvector of matrix A with Rayleigh quotient iteration (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Eigenvalues Approximation with Rayleigh Quotient Iteration/main.cpp**).*

vii. Compute Matrix P that Diagonalizes a Matrix A with SymIntegration

Find a matrix P that diagonalizes

$$A = \begin{bmatrix} 0 & 0 & -2 \\ 1 & 2 & 1 \\ 1 & 0 & 3 \end{bmatrix}$$

Solution:

The characteristic equation of A is

$$(\lambda - 1)(\lambda - 2)^2 = 0$$

and we found the following bases for the eigenspaces:
For $\lambda = 2$

$$\boldsymbol{p}_1 = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix}, \quad \boldsymbol{p}_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

For $\lambda = 1$

$$\boldsymbol{p}_3 = \begin{bmatrix} -2 \\ 1 \\ 1 \end{bmatrix}$$

There are three basis vectors in total, so the matrix

$$P = \begin{bmatrix} -1 & 0 & -2 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix}$$

diagonalizes A. As a check, you should verify that

$$P^{-1}AP = \begin{bmatrix} -1 & 0 & -2 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 0 & -2 \\ 1 & 2 & 1 \\ 21 & 0 & 3 \end{bmatrix} \begin{bmatrix} -1 & 0 & -2 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

In general, there is no preferred order for the columns of P. Since the i-th diagonal entry of $P^{-1}AP$ is an eigenvalue for the i-th column vector of P, changing the order of the columns of P just changes the order of the eigenvalues on the diagonal of $P^{-1}AP$.

In SymIntegration we can use this function to compute the matrix P that diagonalizes matrix A and also to compute $P^{-1}AP$:
diagonalization(vector<vector<double>>& A)

with A as the matrix that we want to diagonalize, we are using the power method, deflation method, and inverse iteration with shift to obtain the matrix P (matrix with eigenvectors as the columns), thus the result here for P is the matrix with normalize eigenvectors, the end result is still the same, we will have the eigenvalues on the diagonal of $P^{-1}AP$.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Diagonalization ]# ./main

A:
               0.000000              0.000000             -2.000000
               1.000000              2.000000              1.000000
               1.000000              0.000000              3.000000

Eigenvalue(1) = 2.000000

Eigenvalue(2) = 2.000000

Eigenvalue(3) = 1.000000

P:
              -0.592140             -0.551064              0.816497
               0.546572              0.626624             -0.408248
               0.592140              0.551064             -0.408248

P^{-1}:
               1.081702             -7.888875             10.052280
               0.652340              8.476911             -7.172231
               2.449490             -0.000000              2.449490

P^{-1}AP:
               2.000000              0.000000              0.000000
               0.000000              2.000000             -0.000000
              -0.000000             -0.000000              1.000000

Time taken by function: 4514 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Diagonalization ]#
```

Figure 6.27: *The computation to obtain matrix P that diagonalizes matrix A, then compute P^{-1} and $P^{-1}AP$. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Diagonalization/main.cpp**).*

viii. Compute Complex Eigenvalues and Its Corresponding Eigenvector using Rayleigh Quotient Iteration with SymIntegration

Find the eigenvalues and bases for the eigenspaces of

$$A = \begin{bmatrix} -2 & -1 \\ 5 & 2 \end{bmatrix}$$

Solution:

The characteristic polynomial of A is

$$\begin{vmatrix} \lambda + 2 & 1 \\ -5 & \lambda - 2 \end{vmatrix} = \lambda^2 + 1 = (\lambda - i)(\lambda + i)$$

so the eigenvalues of A are $\lambda = i$ and $\lambda = -i$. Note that these eigenvalues are complex conjugates. To find the eigenvectors we must solve the system

$$\begin{bmatrix} \lambda + 2 & 1 \\ -5 & \lambda - 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

substitute with $\lambda = i$, the system becomes

$$\begin{bmatrix} i + 2 & 1 \\ -5 & i - 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

We could solve this system by reducing the augmented matrix

$$\begin{bmatrix} i + 2 & 1 & 0 \\ -5 & i - 2 & 0 \end{bmatrix}$$

to reduced row echelon form by Gauss-Jordan elimination. Observe that the reduced row echelon form must have a row of zeros in order to have a basis for the corresponding $\lambda = i$, thus each row of the matrix above must be a scalar multiple of the other.

We can just set one of the row to zero and then do the parameterization. Now we set the first row to zero, so we will have

$$\begin{bmatrix} 0 & 0 & 0 \\ -5 & i - 2 & 0 \end{bmatrix}$$

We divide the second row with $-\frac{1}{5}$ then interchange the rows

$$\begin{bmatrix} 1 & \frac{2}{5} - \frac{i}{5} & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

then do the parameterization to get the general solution

$$x_1 = \left(-\frac{2}{5} + \frac{i}{5} \right) t$$
$$x_2 = t$$

you can check by substituting the values of x_1 and x_2 into the first row

$$\begin{bmatrix} i + 2 & 1 & 0 \\ -5 & i - 2 & 0 \end{bmatrix}$$

The equation will hold for

$$(i+2)x_1 + x_2 = 0$$

you can verify this by hand.

So, the eigenspace corresponding to $\lambda = i$ is one-dimensional and consists of all complex scalar multiples of the basis vector

$$\boldsymbol{x} = \begin{bmatrix} -\frac{2}{5} + \frac{i}{5} \\ 1 \end{bmatrix}$$

As a check, let us confirm that

$$A\boldsymbol{x} = i\boldsymbol{x}$$

$$A\boldsymbol{x} = \begin{bmatrix} -2 & -1 \\ 5 & 2 \end{bmatrix} \begin{bmatrix} -\frac{2}{5} + \frac{i}{5} \\ 1 \end{bmatrix} = \begin{bmatrix} -2\left(-\frac{2}{5} + \frac{i}{5}\right) - 1 \\ 5\left(-\frac{2}{5} + \frac{i}{5}\right) + 2 \end{bmatrix} = \begin{bmatrix} -\frac{1}{5} - \frac{2i}{5} \\ i \end{bmatrix} = i\boldsymbol{x}$$

We could find the basis for the eigenspace corresponding to $\lambda = -i$ in a similar way, but to save time and be more efficient we have a theorem that implies

$$\overline{\boldsymbol{x}} = \begin{bmatrix} -\frac{2}{5} - \frac{i}{5} \\ 1 \end{bmatrix}$$

must be a basis for this eigenspace. The following computations confirm that $\overline{\boldsymbol{x}}$ is an eigenvector of A corresponding to $\lambda = -i$:

$$A\overline{\boldsymbol{x}} = \begin{bmatrix} -2 & -1 \\ 5 & 2 \end{bmatrix} \begin{bmatrix} -\frac{2}{5} - \frac{i}{5} \\ 1 \end{bmatrix} = \begin{bmatrix} -2\left(-\frac{2}{5} - \frac{i}{5}\right) - 1 \\ 5\left(-\frac{2}{5} - \frac{i}{5}\right) + 2 \end{bmatrix} = \begin{bmatrix} -\frac{1}{5} + \frac{2i}{5} \\ -i \end{bmatrix} = -i\overline{\boldsymbol{x}}$$

In SymIntegration, to compute the approximation of a complex eigenvalue we can use Rayleigh quotient iteration that has been modified so it can work with complex number. The function that can be used is:
ComplexRayleighQuotientIteration(vector<vector<complex<double>>> &A, int N_{Rayleigh}, int N double a, double b)

with A as the matrix that we want to compute the eigenvalues and the corresponding eigenvector, N_{Rayleigh} is the maximum number of iteration for the Rayleigh quotient iteration, N is the maximum number of the outermost iteration till we obtain all the correct eigenvalues, a is the lower bound for uniform distribution, b is the upper bound for the uniform distribution, we are using uniform distribution to generate random μ and σ as the input to generate the initial complex vector $\boldsymbol{v}^0$ that is normally distributed with mean μ and standard deviation σ, this initial vector is a prerequisite for the Rayleigh quotient iteration. The key here is to choose a and b carefully so we can pinpoint the range where the eigenvalue could be located.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Complex Eigenvalues with Rayleigh Quotient ]# ./main

Matrix A :
      (-2,0)       (-1,0)
       (5,0)        (2,0)

v initial vector_{normalize} =
(0.195,0.535)
(0.752,0.331)

λ^{0} = (2,2.03)

Eigenvalue = (0,1)

Corresponding Eigenvector:
(0.113,-0.392)
(-0.619,0.671)

Time taken by function: 5184 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Complex Eigenvalues with Rayleigh Quotient ]# ./main

Matrix A :
      (-2,0)       (-1,0)
       (5,0)        (2,0)

v initial vector_{normalize} =
(0.621,-0.0174)
(0.71,0.333)

λ^{0} = (2.2,-1.31)

Eigenvalue = (0,-1)

Corresponding Eigenvector:
(0.359,0.193)
(-0.912,-0.0275)

Time taken by function: 5538 microseconds
```

Figure 6.28: *The computation to obtain complex eigenvalue of matrix A one by one with Rayleigh quotient iteration (the earlier / beta version of the code). (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Complex Eigenvalues with Rayleigh Quotient/main.cpp**).*

We compile it then run it twice and it produces different eigenvalues

$$\lambda_1 = (0,1) = i$$
$$\lambda_2 = (0,-1) = -i$$

which are corrects, and the computed corresponding eigenvector is already normalized.

Rayleigh quotient has a similar atmosphere like bisection method in root finding, in term of choosing a and b when we call the function, the different here is that eigenvalue problem is scientific computing, it is far more complex and deeper than root finding problem.

After some trial and error, to make it more efficient we modify and update the source code so it will produce a vector filled with the correct eigenvalues and a matrix filled with the eigenvectors, that is showing the eigenvector as row vector (depends on the future usage, the matrix can be transposed so it will show the eigenvector as column vector).

```
**********************************************************************

Iter: 2

v initial vector_{normalize} =
(0.66,0.275)
(0.378,0.588)

λ^{0} = (1.6,-1.71)

Eigenvalue = (0,-1)

Corresponding Eigenvector:
(-0.407,-0.0268)
(0.842,-0.354)

Ax:
(-0.0268,0.407)
(-0.354,-0.842)

λx:
(-0.0268,0.407)
(-0.354,-0.842)

Eigenvalue accepted= (0,-1)

Eigenvalues :
(0,1)
(0,-1)

Eigenvectors (represented by row):
(-0.322,-0.251) (0.392,0.824)
(-0.407,-0.0268) (0.842,-0.354)

Time taken by function: 13710 microseconds
```

Figure 6.29: *The computation to obtain all the complex eigenvalues of matrix A along with its corresponding eigenvector, it stops at iteration* 2 *(the latest version of the code made on January 1st, 2026). (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Complex Eigenvalues with Rayleigh Quotient/main.cpp**).*

If we run the compiled code again, we will probably end up with different eigenvectors, yet the eigenvalues are still the same. This proves the eigenvectors' property that says: *eigenvectors may not be unique, as a matrix may have multiple eigenvectors corresponding to the same eigenvalue*.

```
**********************************************************************

Iter: 4

v initial vector_{normalize} =
(0.358,0.744)
(0.16,-0.541)

λ^{0} = (-2.11,1.88)

Eigenvalue = (0,1)

Corresponding Eigenvector:
(-0.356,-0.2)
(0.512,0.756)

Ax:
(0.2,-0.356)
(-0.756,0.512)

λx:
(0.2,-0.356)
(-0.756,0.512)

Eigenvalue accepted= (0,1)

Eigenvalues :
(0,-1)
(0,1)

Eigenvectors (represented by row):
(-0.0169,0.408) (-0.374,-0.833)
(-0.356,-0.2) (0.512,0.756)

Time taken by function: 22872 microseconds
```

Figure 6.30: *The computation to obtain all the complex eigenvalues of matrix A along with its corresponding eigenvector, this time it stops at iteration* 4 *(the latest version of the code made on January 1st, 2026). (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Complex Eigenvalues with Rayleigh Quotient/main.cpp**).*

We modify the source code since we saw the defect when the earlier version of complex Rayleigh quotient iteration resulting in an eigenvalue of $\lambda = (1, -1) = 1 - i$, which is wrong when we check that $A\boldsymbol{x} \neq \lambda\boldsymbol{x}$.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Complex Eigenvalues with Rayleigh Quotient
main

Matrix A :
      (-2,0)       (-1,0)
       (5,0)        (2,0)

v initial vector_{normalize} =
(0.419,0.504)
(0.477,0.585)

λ^{0} = (2.26,-0.0304)

Eigenvalue = (1,-1)

Corresponding Eigenvector:
(-0.282,0.12)
(0.657,-0.689)

Ax:
(-0.0923,0.449)
(-0.0974,-0.778)

λx:
(-0.162,0.402)
(-0.0325,-1.35)

Time taken by function: 5011 microseconds
```

Figure 6.31: *The computation to obtain an eigenvalue of matrix A ended up with the wrong eigenvalue since $A\boldsymbol{x} \neq \lambda \boldsymbol{x}$ (the earlier / beta version of the code). (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Complex Eigenvalues with Rayleigh Quotient/main.cpp**).*

If you want to use the earlier / beta version of the code you just need to delete the outermost **for** loop and adjust a bit the **ComplexRayleighQuotientIteration()** function in **src/linearalgebra.cpp**.

The most robust and generally used method for dense, non-symmetric matrices with complex eigenvalues is the QR algorith with implicit shifts. For finding a specific eigenpair when a good initial guess is available, inverse iteration and Rayleigh quotient iteration are highly effective.

Rayleigh quotient iteration often exhibits cubic convergence for Hermitian matrices and rapid convergence for general matrices, it is a powerful extension of inverse iteration that uses the Rayleigh quotient as the shift μ.

ix. Compute Real Eigenvalues and Its Corresponding Eigenvector using QR Algorithm with Shifts with SymIntegration

Compute the eigenvalues and its corresponding eigenvector for

$$A = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix}$$

Solution:

At iteration $k = 1$, set $A^{(0)} = A$, and we will choose $\mu^1 = a_{22} = 3$, thus

$$\mu^{(1)} I = \begin{bmatrix} 3 & 0 \\ 0 & 3 \end{bmatrix}$$

$$A - \mu^{(1)} I = \begin{bmatrix} 1 & 1 \\ 2 & 0 \end{bmatrix}$$

the QR decomposition of $A - \mu^{(1)} I$ is

$$Q^{(1)} = \begin{bmatrix} -0.447 & -0.894 \\ -0.894 & 0.447 \end{bmatrix}, \quad R^{(1)} = \begin{bmatrix} -2.24 & -0.447 \\ 0 & -0.894 \end{bmatrix}$$

$$R^{(1)} Q^{(1)} = \begin{bmatrix} 1.4 & 1.8 \\ 0.8 & -0.4 \end{bmatrix}$$

$$R^{(1)} Q^{(1)} + \mu^{(1)} I = \begin{bmatrix} 4.4 & 1.8 \\ 0.8 & 2.6 \end{bmatrix}$$

we will set $A^{(1)} = R^{(1)} Q^{(1)} + \mu^{(1)} I$ and proceed to the next iteration $k = 2$, repeat the iteration till $A^{(k)} = R^{(k)} Q^{(k)} + \mu^{(k)} I$ become a matrix with zero below the diagonal, the diagonal will be the eigenvalues of the matrix A.

We use 6 iterations and we obtain

$$A^{(6)} = R^{(6)} Q^{(6)} + \mu^{(6)} I = \begin{bmatrix} 5 & -1 \\ 0 & 2 \end{bmatrix}$$

hence the eigenvalues of matrix A are

$$\lambda_1 = 5$$
$$\lambda_2 = 2$$

the corresponding eigenvector for $\lambda_1 = 5$

$$\boldsymbol{x}_1 = \begin{bmatrix} 0.707 \\ 0.707 \end{bmatrix}$$

and the corresponding eigenvector for $\lambda_2 = 2$

$$\boldsymbol{x}_2 = \begin{bmatrix} 0.447 \\ -0.894 \end{bmatrix}$$

In SymIntegration, to compute the eigenvalues and its corresponding eigenvector, given that the matrix has only real eigenvalues we can use this function:

QRAlgorithmwithShifts(vector<vector<complex<double>>>& A, int N)

with A as the matrix, and N as the number of iterations we want to use till the matrix converge.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Compute Real Eigenvalues and Eigenvectors with QR Algorithm with Shifts ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Compute Real Eigenvalues and Eigenvectors with QR Algorithm with Shifts ]# ./main

Iteration: 1

μ = (3,0)

A:
       (4,0)        (1,0)
       (2,0)        (3,0)

μI =
       (3,0)        (0,0)
       (0,0)        (3,0)

A - μI =
       (1,0)        (1,0)
       (2,0)        (0,0)

Q:
  (-0.447,0)   (-0.894,0)
  (-0.894,0)    (0.447,0)

R:
   (-2.24,0)   (-0.447,0)
       (0,0)   (-0.894,0)

RQ =
     (1.4,0)      (1.8,0)
     (0.8,0)     (-0.4,0)

A_{new} =
     (4.4,0)      (1.8,0)
     (0.8,0)      (2.6,0)

Iteration: 2

μ = (2.6,0)

A:
     (4.4,0)      (1.8,0)
     (0.8,0)      (2.6,0)

μI =
     (2.6,0)        (0,0)
       (0,0)      (2.6,0)
```

Figure 6.32: *The computation to obtain all eigenvalues of matrix A and the corresponding eigenvector for each eigenvalue. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Compute Real Eigenvalues and Eigenvectors with QR Algorithm with Shifts/main.cpp**).*

```
Iteration: 2

μ = (2.6,0)

A:
     (4.4,0)      (1.8,0)
     (0.8,0)      (2.6,0)

μI =
     (2.6,0)        (0,0)
       (0,0)      (2.6,0)

A - μI =
     (1.8,0)      (1.8,0)
     (0.8,0)        (0,0)

Q:
  (-0.914,0)   (-0.406,0)
  (-0.406,0)    (0.914,0)

R:
   (-1.97,0)    (-1.64,0)
       (0,0)   (-0.731,0)

RQ =
    (2.47,0)   (-0.703,0)
   (0.297,0)   (-0.668,0)

A_{new} =
    (5.07,0)   (-0.703,0)
   (0.297,0)     (1.93,0)

Iteration: 3

μ = (1.93,0)

A:
    (5.07,0)   (-0.703,0)
   (0.297,0)     (1.93,0)

μI =
    (1.93,0)        (0,0)
       (0,0)     (1.93,0)

A - μI =
    (3.14,0)   (-0.703,0)
   (0.297,0)        (0,0)

Q:
  (-0.996,0)  (-0.0943,0)
 (-0.0943,0)    (0.996,0)
```

Figure 6.33: *The computation to obtain all eigenvalues of matrix A and the corresponding eigenvector for each eigenvalue. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Compute Real Eigenvalues and Eigenvectors with QR Algorithm with Shifts/main.cpp**).*

```
Iteration: 5

μ = (2,0)

A:
       (5,0)       (-1,0)
 (4.3e-06,0)        (2,0)

μI =
       (2,0)        (0,0)
       (0,0)        (2,0)

A - μI =
       (3,0)       (-1,0)
 (4.3e-06,0)        (0,0)

Q:
      (-1,0) (-1.43e-06,0)
(-1.43e-06,0)        (1,0)

R:
      (-3,0)        (1,0)
       (0,0) (1.43e-06,0)

RQ =
       (3,0)        (1,0)
(-2.05e-12,0) (1.43e-06,0)

A_{new} =
       (5,0)        (1,0)
(-2.05e-12,0)        (2,0)

Iteration: 6

μ = (2,0)

A:
       (5,0)        (1,0)
(-2.05e-12,0)        (2,0)

μI =
       (2,0)        (0,0)
       (0,0)        (2,0)

A - μI =
       (3,0)        (1,0)
(-2.05e-12,0)        (0,0)

Q:
      (-1,0) (6.83e-13,0)
(6.83e-13,0)        (1,0)
```

Figure 6.34: *The computation to obtain all eigenvalues of matrix A and the corresponding eigenvector for each eigenvalue. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Compute Real Eigenvalues and Eigenvectors with QR Algorithm with Shifts/main.cpp**).*

```
A:
       (5,0)        (1,0)
(-2.05e-12,0)        (2,0)

μI =
       (2,0)        (0,0)
       (0,0)        (2,0)

A - μI =
       (3,0)        (1,0)
(-2.05e-12,0)        (0,0)

Q:
      (-1,0) (6.83e-13,0)
(6.83e-13,0)        (1,0)

R:
      (-3,0)       (-1,0)
       (0,0)        (0,0)

RQ =
       (3,0)       (-1,0)
       (0,0)        (0,0)

A_{new} =
       (5,0)       (-1,0)
       (0,0)        (2,0)

********************************************************************************

End of iteration

********************************************************************************

Eigenvalue(1) = (5,0)

Corresponding Eigenvector:
(0.707,0)
(0.707,0)

Eigenvalue(2) = (2,0)

Corresponding Eigenvector:
(0.447,0)
(-0.894,0)

Eigenvectors (in column vectors):
   (0.707,0)    (0.447,0)
   (0.707,0)   (-0.894,0)

Eigenvalues:
(5,0)
(2,0)

Time taken by function: 6656 microseconds
```

Figure 6.35: *The computation to obtain all eigenvalues of matrix A and the corresponding eigenvector for each eigenvalue. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Compute Real Eigenvalues and Eigenvectors with QR Algorithm with Shifts/main.cpp**).*

x. Compute Real and Complex Eigenvalues from Real Matrix using Francis Double Step QR Algorithm with SymIntegration

Compute the eigenvalues of the real matrix

$$A = \begin{bmatrix} 7 & 3 & 4 & -11 & -9 & -2 \\ -6 & 4 & -5 & 7 & 1 & 12 \\ -1 & -9 & 2 & 2 & 9 & 1 \\ -8 & 0 & -1 & 5 & 0 & 8 \\ -4 & 3 & -5 & 7 & 2 & 10 \\ 6 & 1 & 4 & -11 & -7 & -1 \end{bmatrix}$$

Solution:

We will compute the Hessenberg matrix of A

$$H = \begin{bmatrix} 7 & 7.2761 & 5.8120 & -0.1397 & 9.0152 & 7.9363 \\ -12.3693 & 4.1307 & 18.9685 & -1.2071 & 10.6833 & 2.4160 \\ 0 & -7.1603 & 2.4478 & -0.5656 & -4.1814 & -3.2510 \\ 0 & 0 & -8.5988 & 2.9151 & -3.4169 & 5.7230 \\ 0 & 0 & 0 & 1.0464 & -2.8351 & -10.9792 \\ 0 & 0 & 0 & 0 & 1.4143 & 5.3415 \end{bmatrix}$$

Next, we will compute the first householder reflector, P_0. Given an upper Hessenberg matrix H, calculate the first column of $G = H^2 - sH + tI$, where

$$\begin{aligned} s &= h_{n-1,n-1} + h_{n,n} \\ t &= (h_{n-1,n-1} * h_{n,n}) - (h_{n-1,n} * h_{n,n-1}) \end{aligned}$$

The first three components of $Ge_1 = \boldsymbol{x}$ are

$$\begin{aligned} x &= h_{1,1}^2 + h_{1,2}h_{2,1} - sh_{1,1} + t \\ y &= h_{2,1}(h_{1,1} + h_{2,2} - s) \\ z &= h_{2,1}h_{3,2} \end{aligned}$$

this will populate the first three entries for the target vector $\boldsymbol{x}$.

Construct the Householder vector $\boldsymbol{v}$. Compute a reflector P_0 that maps $\boldsymbol{x} = [x, y, z]^T$ to a multiple of $e_1 = [1, 0, 0]^T$ ($P_0\boldsymbol{x} = \alpha \boldsymbol{e}_1$).

Let

$$\boldsymbol{x} = \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} h_{1,1}^2 + h_{1,2}h_{2,1} - sh_{1,1} + t \\ h_{2,1}(h_{1,1} + h_{2,2} - s) \\ h_{2,1}h_{3,2} \end{bmatrix}$$

Calculate $\boldsymbol{v}$

$$\boldsymbol{v} = \boldsymbol{x} + \text{sgn}(x_1)||\boldsymbol{x}||_2 e_1$$

If the sign of the first component of vector $\boldsymbol{x}$ is negative then $\text{sgn}(x_1) = 1$, if the sign of the first component of vector $\boldsymbol{x}$ is positive then $\text{sgn}(x_1) = -1$. Normalize $\boldsymbol{v}$

$$\boldsymbol{v} = \frac{\boldsymbol{v}}{||\boldsymbol{v}||_2}$$

Define the first Householder reflector P_0

$$P_0 = I - 2\boldsymbol{v}\boldsymbol{v}^T$$

given that $\boldsymbol{v}$ is already normalized. If $\boldsymbol{v}$ is not normalized yet, then use this formula

$$P_0 = I - 2\frac{\boldsymbol{v}\boldsymbol{v}^T}{||\boldsymbol{v}||^2}$$

Apply P_0 to the top-left $\times n$ submatrix of H. After P_0 has been applied, H is transformed into

$$H = P_0^T H P_0$$

After computing P_0 to act on the first column, the subsequent reflectors $P_1, P_2, \ldots, P_{n-2}$ are calculated to restore the Hessenberg form (chase the "bulge" created by P_0 down the matrix).

The "bulge" exists in the subdiagonal elements that are now non-zero. The second Householder reflector P_1 is designed to eliminate these new non-zeros.

Identify the "vector" to nullify. Look at the first column of the current matrix H below the subdiagonal. Because of the initial transformation, the elements $h_{3,1}$ and $h_{4,1}$ are now non-zero. Let $\boldsymbol{x}$ be a small vector composed of these elements:

$$\boldsymbol{x} = \begin{bmatrix} h_{2,1} \\ h_{3,1} \\ h_{4,1} \end{bmatrix}$$

In many implementations, the bulge starts at $h_{3,1}, h_{4,1}$, but the reflector P_1 typically operates on a 3×3 submatrix starting from the first non-zero element of the bulde, $h_{2,1}$.

Then define the Householder reflector P_1 formulation. Compute a reflector that maps $\boldsymbol{x}$ to a multiple of $\boldsymbol{e}_1$ ($P_1\boldsymbol{x} = \alpha\boldsymbol{e}_1$)

Let

$$\boldsymbol{x} = \begin{bmatrix} h_{2,1} \\ h_{3,1} \\ h_{4,1} \end{bmatrix}$$

Calculate $\boldsymbol{v}$

$$\boldsymbol{v} = \boldsymbol{x} + \text{sgn}(x_1)||\boldsymbol{x}||_2 e_1$$

If the sign of the first component of vector $\boldsymbol{x}$ is negative then $\text{sgn}(x_1) = 1$, if the sign of the first component of vector $\boldsymbol{x}$ is positive then $\text{sgn}(x_1) = -1$. Normalize $\boldsymbol{v}$

$$\boldsymbol{v} = \frac{\boldsymbol{v}}{||\boldsymbol{v}||_2}$$

Define the first Householder reflector P_1

$$P_1 = I - 2\boldsymbol{v}\boldsymbol{v}^T$$

given that $\boldsymbol{v}$ is already normalized. If $\boldsymbol{v}$ is not normalized yet, then use this formula

$$P_1 = I - 2\frac{\boldsymbol{v}\boldsymbol{v}^T}{||\boldsymbol{v}||^2}$$

Basically, after successfully computing P_0 and updating the matrix H creating the first bulge, iterate for $k = 1$ to $k = n - 2$. Identify the "bulge" (non-zero elements below the subdiagonal) created by the previous step. Compute a new Householder reflector P_k to eliminate these elements in the current column. Apply P_k to H and update the matrix H.

$$H \leftarrow P_k^T H P_k$$

Repeat this iteration to compute $P_0, P_1, \ldots, P_{n-2}$ (to push the bulge off the bottom right of the matrix) for N amount of times, till H converges to the quasi-upper triangular matrix (real Schur form).

The matrix P_k is orthogonal ($P_k = P_k^T$).

In SymIntegration to compute the real and complex eigenvalues from real matrix, we can use this function:
FrancisDoubleStepQRReal(vector<vector<complex<double>>>& A, int N)

with A as the matrix in complex term, even if the entries are all real, no imaginary parts. N is the number of iterations that we want to test.

Another function in SymIntegration that can be used to compute Francis double step QR algorithm with complex arithmetic is:
FrancisDoubleStepQR(vector<vector<complex<double>>>& A, int N)

In this case, we try to put $N = 8$ and it converges to the eigenvalues already. We can also put a stop iteration condition if all the eigenvalues already converges then the iteration will stop, here is an example for the stop criteria

$$|\lambda_i^k| - |\lambda_i^{k-1}| < 10^{-6}$$

for $i = 1, 2, \ldots, n$ and k is the last/ current iteration.

```
************************************************************************

End of 8 iterations

************************************************************************

H updated :
              5.000012             -5.999927              7.326973              6.485041             -8.955880             20.790016
              5.999985              5.000037              4.726162              2.538411            -14.009292              2.792811
              0.000000             -0.000002              0.904127              5.837251             11.286555              1.661152
              0.000000              0.000000             -0.687738              1.079478             -3.042816             10.446256
              0.000000             -0.000000              0.000000              0.011822              4.023762             -4.918176
              0.000000             -0.000000             -0.000000              0.000000             -0.000000              2.961084

Eigenvalues:
(5.000024,-5.999956)
(5.000024,5.999956)
(0.991803,-2.001703)
(0.991803,2.001703)
(4.023762,-0.000000)
(2.961083,0.000000)

Time taken by function: 466193 microseconds
```

Figure 6.36: *The computation to obtain all complex and real eigenvalues of matrix A with real entries, no imaginary parts with N = 8. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Compute Real and Complex Eigenvalues with Francis Double Step QR Algorithm/main.cpp**).*

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Compute Real and Complex Eigenvalues with Francis Double Step QR Algorithm ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Compute Real and Complex Eigenvalues with Francis Double Step QR Algorithm ]# ./main

A =
      (7,0)       (3,0)       (4,0)     (-11,0)      (-9,0)      (-2,0)
     (-6,0)       (4,0)      (-5,0)       (7,0)       (1,0)      (12,0)
     (-1,0)      (-9,0)       (2,0)       (2,0)       (9,0)       (1,0)
     (-8,0)       (0,0)      (-1,0)       (5,0)       (0,0)       (8,0)
     (-4,0)       (3,0)      (-5,0)       (7,0)       (2,0)      (10,0)
      (6,0)       (1,0)       (4,0)     (-11,0)      (-7,0)      (-1,0)

H_{0} =
      (7,0)  (7.27607,0)  (5.81205,0) (-0.139701,0)   (9.0152,0)  (7.93634,0)
 (12.3693,0)  (4.13072,0)  (18.9685,0) (-1.20707,0)  (10.6833,0)  (2.41595,0)
      (0,0) (-7.16034,0)  (2.44776,0) (-0.565594,0)  (-4.1814,0) (-3.25096,0)
      (0,0)       (0,0) (-8.59877,0)   (2.9151,0) (-3.41686,0)  (5.72297,0)
      (0,0)       (0,0)       (0,0)  (1.04644,0)  (-2.8351,0) (-10.9792,0)
      (0,0)       (0,0)       (0,0)       (0,0)  (1.41429,0)  (5.34152,0)

H_{0} =
             7.000000             7.276069             5.812049            -0.139701             9.015201             7.936343
            12.369317             4.130719            18.968509            -1.207073            10.683309             2.415951
             0.000000            -7.160342             2.447765            -0.565594            -4.181396            -3.250955
             0.000000             0.000000            -8.598771             2.915100            -3.416858             5.722969
             0.000000             0.000000             0.000000             1.046436            -2.835101           -10.979178
             0.000000             0.000000             0.000000             0.000000             1.414293             5.341517
```

Figure 6.37: *The computation to obtain all complex and real eigenvalues of matrix A with real entries, no imaginary parts. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Compute Real and Complex Eigenvalues with Francis Double Step QR Algorithm/main.cpp**).*

```
Iteration: 1

H =
             7.000000             7.276069             5.812049            -0.139701             9.015201             7.936343
            12.369317             4.130719            18.968509            -1.207073            10.683309             2.415951
             0.000000            -7.160342             2.447765            -0.565594            -4.181396            -3.250955
             0.000000             0.000000            -8.598771             2.915100            -3.416858             5.722969
             0.000000             0.000000             0.000000             1.046436            -2.835101           -10.979178
             0.000000             0.000000             0.000000             0.000000             1.414293             5.341517

s = 2.506416
t = 0.384040
x = 121.839129
y = 106.676737
z = -88.568536

x =
121.839129
106.676737
-88.568536

w =
-184.578127
-0.000000
-0.000000

u =
-62.738999
106.676737
-88.568536

u =
            -62.738999
            106.676737
            -88.568536

uT =
            -62.738999            106.676737            -88.568536

uTu =
          23160.493693

uuT =
           3936.181938          -6692.791623           5556.701269
          -6692.791623          11379.926135          -9448.202418
           5556.701269          -9448.202418           7844.385621

||u|| = 0.000043
```

Figure 6.38: *The computation to obtain all complex and real eigenvalues of matrix A with real entries, no imaginary parts. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Compute Real and Complex Eigenvalues with Francis Double Step QR Algorithm/main.cpp**).*

```
P_{0}:
              0.660095              0.577949             -0.479843              0.000000              0.000000              0.000000
              0.577949              0.017298              0.815890              0.000000              0.000000              0.000000
             -0.479843              0.815890              0.322606              0.000000              0.000000              0.000000
              0.000000              0.000000              0.000000              1.000000              0.000000              0.000000
              0.000000              0.000000              0.000000              0.000000              1.000000              0.000000
              0.000000              0.000000              0.000000              0.000000              0.000000              1.000000

B_{0}:
              7.372561             18.102312              7.417642             -0.518446             14.131712              8.194987
             -0.820541              7.072521             -1.487345             -0.563083              1.983574              1.976173
             -3.427433             14.845075             -0.866598             -1.100268              3.041574             -2.885829
              4.126061             -7.015647             -2.774018              2.915100             -3.416858              5.722969
              0.000000              0.000000              0.000000              1.046436             -2.835101            -10.979178
              0.000000              0.000000              0.000000              0.000000              1.414293              5.341517

x_{1}:
-0.820541
-3.427433
4.126061

P_{1} reflector:
              1.000000              0.000000              0.000000              0.000000              0.000000              0.000000
              0.000000              0.151215              0.631631             -0.760379              0.000000              0.000000
              0.000000              0.631631              0.529966              0.565843              0.000000              0.000000
              0.000000             -0.760379              0.565843              0.318819              0.000000              0.000000
              0.000000              0.000000              0.000000              0.000000              1.000000              0.000000
              0.000000              0.000000              0.000000              0.000000              0.000000              1.000000

H^{1}:
              7.372561              7.816770             15.071721             -9.732688             14.131712              8.194987
             -5.426321              5.509396              8.980460            -12.198117              4.819205             -5.875578
             -0.000000             -1.150449              4.112520             -7.813504              0.931413              2.957123
              0.000000             -0.594085              0.782781             -0.500893             -0.876574             -1.310975
              0.000000             -0.795688              0.592119              0.333624             -2.835101            -10.979178
              0.000000              0.000000              0.000000              0.000000              1.414293              5.341517
```

Figure 6.39: *The computation to obtain all complex and real eigenvalues of matrix A with real entries, no imaginary parts. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Compute Real and Complex Eigenvalues with Francis Double Step QR Algorithm/main.cpp**).*

```
H^{1}:
              7.372561              7.816770             15.071721             -9.732688             14.131712              8.194987
             -5.426321              5.509396              8.980460            -12.198117              4.819205             -5.875578
             -0.000000             -1.150449              4.112520             -7.813504              0.931413              2.957123
              0.000000             -0.594085              0.782781             -0.500893             -0.876574             -1.310975
              0.000000             -0.795688              0.592119              0.333624             -2.835101            -10.979178
              0.000000              0.000000              0.000000              0.000000              1.414293              5.341517

x_{2}:
-1.150449
-0.594085
-0.795688

P_{2} reflector:
              1.000000              0.000000              0.000000              0.000000              0.000000              0.000000
              0.000000              1.000000              0.000000              0.000000              0.000000              0.000000
              0.000000              0.000000              0.757007              0.390914              0.523571              0.000000
              0.000000              0.000000              0.390914              0.371118             -0.842293              0.000000
              0.000000              0.000000              0.523571             -0.842293             -0.128125              0.000000
              0.000000              0.000000              0.000000              0.000000              0.000000              1.000000

H^{2}:
              7.372561              7.816770             15.003708             -9.623271             14.278258              8.194987
             -5.426321              5.509396              4.553051             -5.075539             14.358833             -5.875578
             -0.000000             -1.519733             -0.084844              0.199860              7.096121             -4.022294
             -0.000000              0.000000              0.953422             -2.803829              3.387732              9.917138
             -0.000000              0.000000              0.454532             -2.161796              3.665199              4.059200
              0.000000              0.000000              0.740483             -1.191249             -0.181207              5.341517

x_{3}:
0.953422
0.454532
0.740483
```

Figure 6.40: *The computation to obtain all complex and real eigenvalues of matrix A with real entries, no imaginary parts. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Compute Real and Complex Eigenvalues with Francis Double Step QR Algorithm/main.cpp**).*

```
P_{3} reflector:
              1.000000              0.000000              0.000000              0.000000              0.000000              0.000000
              0.000000              1.000000              0.000000              0.000000              0.000000              0.000000
              0.000000              0.000000              1.000000              0.000000              0.000000              0.000000
              0.000000              0.000000              0.000000              0.739125              0.352369              0.574047
              0.000000              0.000000              0.000000              0.352369              0.524048             -0.775377
              0.000000              0.000000              0.000000              0.574047             -0.775377             -0.263174

H^{3}:
              7.372561              7.816770             15.003708              2.622721             -2.262645            -18.751949
             -5.426321              5.509396              4.553051             -2.064713             10.292050            -12.500810
             -0.000000             -1.519733             -0.084844              0.339186              6.907930             -4.328878
             -0.000000              0.000000              1.289934              5.489574             -8.475220             -7.994189
             -0.000000              0.000000              0.000000              1.111672              0.136337             -3.600585
              0.000000             -0.000000              0.000000              0.635934             -1.194952              0.576976

x_{4}:
1.111672
0.635934

P_{4} reflector:
              1.000000              0.000000              0.000000              0.000000              0.000000              0.000000
              0.000000              1.000000              0.000000              0.000000              0.000000              0.000000
              0.000000              0.000000              1.000000              0.000000              0.000000              0.000000
              0.000000              0.000000              0.000000              1.000000              0.000000              0.000000
              0.000000              0.000000              0.000000              0.000000              0.868010              0.496546
              0.000000              0.000000              0.000000              0.000000              0.496546             -0.868010

H^{4}:
              7.372561              7.816770             15.003708              2.622721            -11.275212             15.153374
             -5.426321              5.509396              4.553051             -2.064713              2.726372             15.961311
             -0.000000             -1.519733             -0.084844              0.339186              3.846664              7.187618
             -0.000000              0.000000              1.289934              5.489574            -11.326063              2.730697
             -0.000000              0.000000              0.000000              1.280713             -1.821932              2.228287
             -0.000000              0.000000             -0.000000             -0.000000             -0.177346              2.535245
```

Figure 6.41: *The computation to obtain all complex and real eigenvalues of matrix A with real entries, no imaginary parts. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Compute Real and Complex Eigenvalues with Francis Double Step QR Algorithm/main.cpp**).*

```
H^{4}:
              7.372561              7.816770             15.003708              2.622721            -11.275212             15.153374
             -5.426321              5.509396              4.553051             -2.064713              2.726372             15.961311
             -0.000000             -1.519733             -0.084844              0.339186              3.846664              7.187618
             -0.000000              0.000000              1.289934              5.489574            -11.326063              2.730697
             -0.000000              0.000000              0.000000              1.280713             -1.821932              2.228287
             -0.000000              0.000000             -0.000000             -0.000000             -0.177346              2.535245

x_{5}:
-1.821932
-0.177346

P_{5} last:
              0.004704             -0.096882
             -0.096882              1.995296

P_{5} reflector:
              1.000000              0.000000              0.000000              0.000000              0.000000              0.000000
              0.000000              1.000000              0.000000              0.000000              0.000000              0.000000
              0.000000              0.000000              1.000000              0.000000              0.000000              0.000000
              0.000000              0.000000              0.000000              1.000000              0.000000              0.000000
              0.000000              0.000000              0.000000              0.000000              1.000000              0.000000
              0.000000              0.000000              0.000000              0.000000              0.000000              0.995296

H^{5}:
              7.372561              7.816770             15.003708              2.622721            -11.275212             15.082091
             -5.426321              5.509396              4.553051             -2.064713              2.726372             15.886227
             -0.000000             -1.519733             -0.084844              0.339186              3.846664              7.153807
             -0.000000              0.000000              1.289934              5.489574            -11.326063              2.717852
             -0.000000              0.000000              0.000000              1.280713             -1.821932              2.217805
             -0.000000              0.000000             -0.000000             -0.000000             -0.176512              2.511449
```

Figure 6.42: *The computation to obtain all complex and real eigenvalues of matrix A with real entries, no imaginary parts. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Compute Real and Complex Eigenvalues with Francis Double Step QR Algorithm/main.cpp**).*

```
H^{5}:
              7.372561             7.816770            15.003708             2.622721           -11.275212            15.082091
             -5.426321             5.509396             4.553051            -2.064713             2.726372            15.886227
             -0.000000            -1.519733            -0.084844             0.339186             3.846664             7.153807
             -0.000000             0.000000             1.289934             5.489574           -11.326063             2.717852
             -0.000000             0.000000             0.000000             1.280713            -1.821932             2.217805
             -0.000000             0.000000            -0.000000            -0.000000            -0.176512             2.511449

Iteration: 2

H =
              7.372561             7.816770            15.003708             2.622721           -11.275212            15.082091
             -5.426321             5.509396             4.553051            -2.064713             2.726372            15.886227
             -0.000000            -1.519733            -0.084844             0.339186             3.846664             7.153807
             -0.000000             0.000000             1.289934             5.489574           -11.326063             2.717852
             -0.000000             0.000000             0.000000             1.280713            -1.821932             2.217805
             -0.000000             0.000000            -0.000000            -0.000000            -0.176512             2.511449

s = 0.689517
t = -4.184220
x = 2.670627
y = -66.160096
z = 8.246560

x =
2.670627
-66.160096
8.246560

w =
-66.725530
-0.000000
-0.000000

u =
-64.054903
-66.160096
8.246560

u =
            -64.054903
            -66.160096
              8.246560
```

Figure 6.43: *The computation to obtain all complex and real eigenvalues of matrix A with real entries, no imaginary parts. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Compute Real and Complex Eigenvalues with Francis Double Step QR Algorithm/main.cpp**).*

xi. Compute Inverse of a Complex Matrix with Gauss-Jordan Elimination with SymIntegration

Find the inverse of

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 5 & 3 \\ 1 & 0 & 8 \end{bmatrix}$$

Solution:

We want to reduce A to the identity matrix by row operations and simultaneously apply these operations to I to produce A^{-1}. To accomplish this we will adjoin the identity matrix to the right side of A, thereby producing a partitioned matrix of the form

$$[A|I]$$

Then we will apply row operations to this matrix until the left side is reduced to I; these operations will convert the right side to A^{-1}, so the final matrix will have the form.

$$[I|A^{-1}]$$

After the computations we will obtain

$$A^{-1} = \begin{bmatrix} -40 & 16 & 9 \\ 13 & -5 & -3 \\ 5 & -2 & -1 \end{bmatrix}$$

If a matrix A is not invertible, then it will be impossible to reduce A to I_n by elementary row operations. This will be signaled by a row of zeros appearing on the left side of tha partition at some stage of the inversion algorithm.

Reduced row echelon form (rref) is an advanced row echelon form (ref) where pivots are all 1, and there are zeros both above and below each pivot, forming an identity matrix on the coefficient side.

The algorithm steps are as follow:

1. Form an augmented matrix
$$[A|I]$$
2. Perform forward elimination to make matrix A into row echelon form.
   (a) Find the leftmost non-zero column (pivot column).
   (b) Swap rows if needed to get a non-zero entry (pivot) at the top of thsi column.
   (c) Multiply the pivot row by a scalar to make the pivot equal to 1 (leading one).
   (d) Add multiples of this pivot row to rows below it to create zeros in the rest of the pivot column.
   (e) Repeat for the submatrix below the first pivot.
3. Perform backward elimination to make the matrix into reduced row echelon form.

(a) Starting from the last pivot (bottom right), use that pivot row to create zeros above it in its column.

(b) Work upwards and leftwards, clearing entries above each subsequent pivot.

4. Once in reduced row echelon form (rref), given the matrix is invertible, the matrix will look like

$$[I|A^{-1}]$$

it reads as (identity matrix | inverse matrix). Besides finding the inverse of a matrix this can also be used for solving systems of linear equations.

In SymIntegration to reduce complex matrix A into reduced row echelon form with the Gauss-Jordan elimination we can use this function:
GaussJordanComplexMatrix(vector<vector<complex<double>>> &A)

with A as the matrix that we want to compute the inverse, in the source code we are showing with **cout** the process to obtain the inverse, you can comment the process so you can get the final result, A^{-1}, the process can be kept / still shown for learning purpose.

In this case, the matrix A is real, but we can input it in complex matrix term with no imaginary part, so the entries for imaginary parts will be all zeros.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Gauss Jordan Method Finding Inverse for Complex Matrix ]# ./main
A:
       (1,0)        (2,0)        (3,0)
       (2,0)        (5,0)        (3,0)
       (1,0)        (0,0)        (8,0)
Swap row
A_{rref} :
       (2,0)        (5,0)        (3,0)
       (1,0)        (2,0)        (3,0)
       (1,0)        (0,0)        (8,0)
A^{-1} :
       (0,0)        (1,0)        (0,0)
       (1,0)        (0,0)        (0,0)
       (0,0)        (0,0)        (1,0)
Eliminate elements below the pivot, i = 0, k = 1
A_{rref} :
       (2,0)        (5,0)        (3,0)
       (0,0)     (-0.5,0)      (1.5,0)
       (1,0)        (0,0)        (8,0)
A^{-1} :
       (0,0)        (1,0)        (0,0)
       (1,0)     (-0.5,0)        (0,0)
       (0,0)        (0,0)        (1,0)
Eliminate elements below the pivot, i = 0, k = 2
A_{rref} :
       (2,0)        (5,0)        (3,0)
       (0,0)     (-0.5,0)      (1.5,0)
       (0,0)     (-2.5,0)      (6.5,0)
A^{-1} :
       (0,0)        (1,0)        (0,0)
       (1,0)     (-0.5,0)        (0,0)
       (0,0)     (-0.5,0)        (1,0)
Swap row
A_{rref} :
       (2,0)        (5,0)        (3,0)
       (0,0)     (-2.5,0)      (6.5,0)
       (0,0)     (-0.5,0)      (1.5,0)
A^{-1} :
       (0,0)        (1,0)        (0,0)
       (0,0)     (-0.5,0)        (1,0)
       (1,0)     (-0.5,0)        (0,0)
```

Figure 6.44: *The computation to obtain matrix A^{-1} with Gauss-Jordan elimination for matrix A with no imaginary part. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Gauss Jordan Method Finding Inverse for Complex Matrix/main.cpp**).*

```
Eliminate elements below the pivot, i = 1, k = 2
A_{rref} :
       (2,0)        (5,0)        (3,0)
       (0,0)     (-2.5,0)      (6.5,0)
       (0,0)        (0,0)      (0.2,0)
A^{-1} :
       (0,0)        (1,0)        (0,0)
       (0,0)     (-0.5,0)        (1,0)
       (1,0)     (-0.4,0)     (-0.2,0)
Swap row
A_{rref} :
       (2,0)        (5,0)        (3,0)
       (0,0)     (-2.5,0)      (6.5,0)
       (0,0)        (0,0)      (0.2,0)
A^{-1} :
       (0,0)        (1,0)        (0,0)
       (0,0)     (-0.5,0)        (1,0)
       (1,0)     (-0.4,0)     (-0.2,0)
After forward elimination:
A_{rref} :
       (2,0)        (5,0)        (3,0)
       (0,0)     (-2.5,0)      (6.5,0)
       (0,0)        (0,0)      (0.2,0)
A^{-1} :
       (0,0)        (1,0)        (0,0)
       (0,0)     (-0.5,0)        (1,0)
       (1,0)     (-0.4,0)     (-0.2,0)
After leading 1:
A_{rref} :
       (1,0)      (2.5,0)      (1.5,0)
     (-0,-0)       (1,-0)    (-2.6,-0)
       (0,0)        (0,0)        (1,0)
A^{-1} :
       (0,0)      (0.5,0)        (0,0)
     (-0,-0)     (0.2,-0)    (-0.4,-0)
       (5,0)       (-2,0)       (-1,0)
Eliminate elements above the pivot, i = 1, k = 1
A_{rref} :
       (1,0)      (2.5,0)      (1.5,0)
      (-0,0)        (1,0)        (0,0)
       (0,0)        (0,0)        (1,0)
A^{-1} :
       (0,0)      (0.5,0)        (0,0)
      (13,0)      (-5,-0)      (-3,-0)
       (5,0)       (-2,0)       (-1,0)
```

Figure 6.45: *The computation to obtain matrix A^{-1} with Gauss-Jordan elimination for matrix A with no imaginary part. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Gauss Jordan Method Finding Inverse for Complex Matrix/main.cpp**).*

```
      (5,0)       (-2,0)       (-1,0)
Eliminate elements above the pivot, i = 0, k = 1
A_{rref} :
       (1,0)        (0,0)      (1.5,0)
      (-0,0)        (1,0)        (0,0)
       (0,0)        (0,0)        (1,0)
A^{-1} :
   (-32.5,0)       (13,0)      (7.5,0)
      (13,0)      (-5,-0)      (-3,-0)
       (5,0)       (-2,0)       (-1,0)
Eliminate elements above the pivot, i = 0, k = 2
A_{rref} :
       (1,0)        (0,0)        (0,0)
      (-0,0)        (1,0)        (0,0)
       (0,0)        (0,0)        (1,0)
A^{-1} :
     (-40,0)       (16,0)        (9,0)
      (13,0)      (-5,-0)      (-3,-0)
       (5,0)       (-2,0)       (-1,0)
After backward elimination:
A_{rref} :
       (1,0)        (0,0)        (0,0)
      (-0,0)        (1,0)        (0,0)
       (0,0)        (0,0)        (1,0)
A^{-1} :
     (-40,0)       (16,0)        (9,0)
      (13,0)      (-5,-0)      (-3,-0)
       (5,0)       (-2,0)       (-1,0)
AA^{-1} :
       (1,0)        (0,0)        (0,0)
       (0,0)        (1,0)        (0,0)
       (0,0)        (0,0)        (1,0)

Time taken by function: 4610 microseconds
```

Figure 6.46: *The computation to obtain matrix A^{-1} with Gauss-Jordan elimination for matrix A with no imaginary part. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Gauss Jordan Method Finding Inverse for Complex Matrix/main.cpp**).*

xii. Compute Initial Value Problem Solution of First-Order Linear Systems of Differential Equations with SymIntegration

(a) Solve the system

$$y_1' = y_1 + y_2$$
$$y_2' = 4y_1 - 2y_2$$

(b) Find the solution that satisfies the initial conditions $y_1(0) = 1, y_2(0) = 6$.

Solution:

(a) The coefficient matrix for the system is

$$\begin{bmatrix} 1 & 1 \\ 4 & -2 \end{bmatrix}$$

Matrix A will be diagonalized by any matrix P whose columns are linearly independent eigenvectors of A. Since

$$\det(\lambda I - A) = \begin{vmatrix} \lambda - 1 & -1 \\ -4 & \lambda + 2 \end{vmatrix} = \lambda^2 + \lambda - 6 = (\lambda + 3)(\lambda - 2)$$

the eigenvalues of A are

$$\lambda_1 = 2$$
$$\lambda_2 = -3$$

By definition,

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

is an eigenvector of A corresponding to λ if and only if $\boldsymbol{x}$ is a nontrivial solution of

$$\begin{bmatrix} \lambda - 1 & -1 \\ -4 & \lambda + 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

If $\lambda = 2$, this system becomes

$$\begin{bmatrix} 1 & -1 \\ -4 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Solving this system yields

$$x_1 - x_2 = 0$$
$$x_1 = x_2$$
$$x_2 = t$$

so

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} t \\ t \end{bmatrix} = t \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Thus,

$$\boldsymbol{p}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

is a basis for the eigenspace corresponding to $\lambda = 2$.

Similarly, you can show that

$$\boldsymbol{p}_2 = \begin{bmatrix} -\frac{1}{4} \\ 1 \end{bmatrix}$$

is a basis for the eigenspace corresponding to $\lambda = -3$. Thus,

$$P = [\boldsymbol{p}_1 \ \boldsymbol{p}_2] = \begin{bmatrix} 1 & -\frac{1}{4} \\ 1 & 1 \end{bmatrix}$$

diagonalizes A, and

$$D = P^{-1}AP = \begin{bmatrix} 2 & 0 \\ 0 & -3 \end{bmatrix}$$

The substitution

$$\boldsymbol{y} = P\boldsymbol{u}$$
$$\boldsymbol{y}' = P\boldsymbol{u}'$$

yields the "diagonal system"

$$\boldsymbol{u}' = D\boldsymbol{u} = \begin{bmatrix} 2 & 0 \\ 0 & -3 \end{bmatrix} \boldsymbol{u}$$

or

$$u_1' = 2u_1$$
$$u_2' = -3u_2$$

The solution of this system is

$$u_1 = c_1 e^{2x}$$
$$u_2 = c_2 e^{-3x}$$

or

$$\boldsymbol{u} = \begin{bmatrix} c_1 e^{2x} \\ c_2 e^{-3x} \end{bmatrix}$$

now we go back to the equation $\boldsymbol{y} = P\boldsymbol{u}$ and substitute $\boldsymbol{u}$ that we just compute above

$$\boldsymbol{y} = P\boldsymbol{u} = \begin{bmatrix} 1 & -\frac{1}{4} \\ 1 & 1 \end{bmatrix} \begin{bmatrix} c_1 e^{2x} \\ c_2 e^{-3x} \end{bmatrix} = \begin{bmatrix} c_1 e^{2x} - \frac{1}{4} c_2 e^{-3x} \\ c_1 e^{2x} + c_2 e^{-3x} \end{bmatrix}$$

or

$$y_1 = c_1 e^{2x} - \frac{1}{4} c_2 e^{-3x}$$
$$y_2 = c_1 e^{2x} + c_2 e^{-3x}$$

(b) If we substitute the initial conditions $y_1(0) = 1, y_2(0) = 6$ into

$$y_1 = c_1 e^{2x} - \frac{1}{4} c_2 e^{-3x}$$
$$y_2 = c_1 e^{2x} + c_2 e^{-3x}$$

we will obtain

$$c_1 - \frac{1}{4}c_2 = 1$$
$$c_1 + c_2 = 6$$

Solving this system, we obtain

$$c_1 = 2$$
$$c_2 = 4$$

so the solution satisfying the initial condition is

$$y_1 = 2e^{2x} - \frac{1}{4}4e^{-3x}$$
$$y_2 = 2e^{2x} + 4e^{-3x}$$

In SymIntegration, to compute the solution satisfying the initial condition for the system of first-order differential equations is
IVPSolution_firstorderdiffeq(vector<vector<complex<double>>> &A, vector<complex<double>> &x, double a, double b)

with A as the matrix that represents the system of first-order differential equations, x as the vector that represents the initial value $y_i(0)$, a is the lower bound for uniform distribution, b is the upper bound for the uniform distribution, we are using uniform distribution to generate random μ and σ as the input to generate the initial complex vector v^0 that is normally distributed with mean μ and standard deviation σ, this initial vector is a prerequisite for the Rayleigh quotient iteration. The key here is to choose a and b carefully so we can pinpoint the range where the eigenvalue could be located.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Solve Initial Value P
roblem of First Order Differential Equations ]# ./main

Matrix A :
       (1,0)        (1,0)
       (4,0)       (-2,0)

Eigenvalues :
(2,0)
(-3,0)

Eigenvectors (represented bycolumn):
   (0.707,0)   (-0.243,0)
   (0.707,0)     (0.97,0)

Augmented Matrix:
   (0.707,0)   (-0.243,0)        (1,0)
   (0.707,0)     (0.97,0)        (6,0)

After backward elimination:
       (1,0)        (0,0)     (2.83,0)
       (0,0)        (1,0)     (4.12,0)

 The solution satisfying the initial condition is :
[ 2*e^(2*x)-e^(-3*x) ]
[2*e^(2*x)+4*e^(-3*x)]


Time taken by function: 24883 microseconds
```

Figure 6.47: *The computation to obtain the initial value problem solution of system of first-order differential equations. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Solve Initial Value Problem of First Order Differential Equations/main.cpp**).*

VI. Inner Product Spaces

[SI*] We will use the most important properties of the dot product on $\mathbb{R}^n$ as axioms, which, if satisfied by the vectors in a vector space V, will enable us to extend the notions of length, distance, angle, and perpendicularity to general vector spaces.

Definition 6.36: Inner Product

An inner product on a real vector space V is a function that associates a real number $< \boldsymbol{u}, \boldsymbol{v} >$ with each pair of vectors in V in such a way that the following axioms are satisfied for all vectors $\boldsymbol{u}, \boldsymbol{v}$, and $\boldsymbol{w}$ in V and all scalars k.

1. $< \boldsymbol{u}, \boldsymbol{v} >=< \boldsymbol{v}, \boldsymbol{u} >$ [Symmetry axiom]
2. $< \boldsymbol{u} + \boldsymbol{v}, \boldsymbol{w} >=< \boldsymbol{u}, \boldsymbol{w} > + < \boldsymbol{v}, \boldsymbol{w} >$ [Additivity axiom]
3. $< k\boldsymbol{u}, \boldsymbol{v} >= k < \boldsymbol{u}, \boldsymbol{v} >$ [Homogeneity axiom]
4. $< \boldsymbol{v}, \boldsymbol{v} \geq 0$ and $< \boldsymbol{v}, \boldsymbol{v} >= 0$ if and only if $\boldsymbol{v} = \boldsymbol{0}$ [Positivity axiom]

A real vector space with an inner product is called a real inner product space. This definition applies only to real vector spaces (vector spaces with real scalars).

[SI*] The inner product space axioms will be satisfied automatically if we define the inner product of two vectors $\boldsymbol{u}$ and $\boldsymbol{v}$ in $\mathbb{R}^n$ to be

$$< \boldsymbol{u}, \boldsymbol{v} >= \boldsymbol{u} \cdot \boldsymbol{v} = u_1 v_1 + u_2 v_2 + \cdots + u_n v_n$$

This inner product is commonly called the Euclidean inner product (or the standard inner product) on $\mathbb{R}^n$ to distinguish it from other possible inner products that might be defined on $\mathbb{R}^n$. We call $\mathbb{R}^n$ with the Euclidean inner product Euclidean n-space.

[SI*] Inner products can be used to define notions of norm and distance in a general inner product space just as we did with dot products in $\mathbb{R}^n$.

Definition 6.37: Norm and Distance

If V is a real inner product space, then the norm (or length) of a vector $\boldsymbol{v}$ in V is denoted by $||\boldsymbol{v}||$ and is defined by

$$||\boldsymbol{v}|| = \sqrt{< \boldsymbol{v}, \boldsymbol{v} >} = \sqrt{\boldsymbol{v} \cdot \boldsymbol{u}}$$

and the distance between two vectors is denoted by $d(\boldsymbol{u}, \boldsymbol{v})$ and is defined by

$$d(\boldsymbol{u}, \boldsymbol{v}) = ||\boldsymbol{u} - \boldsymbol{v}|| = \sqrt{< \boldsymbol{u} - \boldsymbol{v}, \boldsymbol{u} - \boldsymbol{v} >} = \sqrt{(\boldsymbol{u} - \boldsymbol{v}) \cdot (\boldsymbol{u} - \boldsymbol{v})}$$

A vector of norm 1 is called a unit vector.

Theorem 6.64: Properties of Real Inner Product Spaces

If $\boldsymbol{u}$ and $\boldsymbol{v}$ are vectors in a real inner product space V, and if k is a scalar, then:

(a) $||\boldsymbol{v}|| \geq 0$ with equality if and only if $\boldsymbol{v} = \boldsymbol{0}$.

(b) $||k\boldsymbol{v}|| = |k|||\boldsymbol{v}||$.

(c) $d(\boldsymbol{u}, \boldsymbol{v}) = d(\boldsymbol{v}, \boldsymbol{u})$.

(d) $d(\boldsymbol{u}, \boldsymbol{v}) \geq 0$ with equality if and only if $\boldsymbol{u} = \boldsymbol{v}$.

[SI*] There are various applications when we modify Euclidean inner product by weighting each term differently. If

$$w_1, w_2, \ldots, w_n$$

are positive real numbers, which we will call weights, and if $\boldsymbol{u} = (u_1, u_2, \ldots, u_n)$ and $\boldsymbol{v} = (v_1, v_2, \ldots, v_n)$ are vectors in $\mathbb{R}^n$, then it can be shown that the formula

$$< \boldsymbol{u}, \boldsymbol{v} >= w_1 u_1 v_1 + w_2 u_2 v_2 + \cdots + w_n u_n v_n \tag{6.87}$$

defines an inner product on $\mathbb{R}^n$ that we call the weighted Euclidean inner product with weights $w_1, w_2, \ldots, w_n$.

[SI*] Suppose that some physical experiment has n possible numerical outcomes

$$x_1, x_2, \ldots, x_n$$

and that a series of m repetitions of the experiment yields these values with various frequencies. Specifically, suppose that x_1 occurs f_1 times, x_2 occurs f_2 times, and so forth. Since there are a total of m repetitions of the experiment, it follows that

$$f_1 + f_2 + \cdots + f_n = m$$

Thus, the arithmetic average of the observed numerical values (denoted by $\overline{x}$) is

$$\overline{x} = \frac{f_1 x_1 + f_2 x_2 + \cdots + f_n x_n}{f_1 + f_2 + \cdots + f_n} = \frac{1}{m}(f_1 x_1 + f_2 x_2 + \cdots + f_n x_n) \tag{6.88}$$

if we let

$$\begin{aligned} \boldsymbol{f} &= (f_1, f_2, \ldots, f_n) \\ \boldsymbol{x} &= (x_1, x_2, \ldots, x_n) \\ w_1 = w_2 &= \cdots = w_n = 1/m \end{aligned}$$

then we can express $\overline{x}$ as the weighted Euclidean inner product

$$\overline{x} =< \boldsymbol{f}, \boldsymbol{x} >= w_1 f_1 x_1 + w_2 f_2 x_2 + \cdots + w_n f_n x_n$$

[SI*] Norm and distance depend on the inner product being used. If the inner product is changed, then the norms and distances between vectors also change.

[SI*] If V is an inner product space, then the set of points in V that satisfy

$$||\boldsymbol{u}|| = 1$$

is called the unit sphere or sometimes the unit circle in V.

Case 1:
Let the Euclidean inner product be

$$< \boldsymbol{u}, \boldsymbol{v} >= u_1 v_1 + u_2 v_2$$

If $\boldsymbol{u} = (x, y)$, then

$$||\boldsymbol{u}|| =< \boldsymbol{u}, \boldsymbol{u} >^{1/2}= \sqrt{x^2 + y^2}$$

so the equation of the unit circle is

$$\sqrt{x^2 + y^2} = 1$$
$$x^2 + y^2 = 1$$

the graph of this equation is a circle with radius of 1 centered at the origin.

Case 2:
Let the weighted Euclidean inner product be

$$< \boldsymbol{u}, \boldsymbol{v} >= \frac{1}{9} u_1 v_1 + \frac{1}{4} u_2 v_2$$

If $\boldsymbol{u} = (x, y)$, then

$$||\boldsymbol{u}|| =< \boldsymbol{u}, \boldsymbol{u} >^{1/2}= \sqrt{\frac{1}{9}x^2 + \frac{1}{4}y^2}$$

so the equation of the unit circle is

$$\sqrt{\frac{1}{9}x^2 + \frac{1}{4}y^2} = 1$$
$$\frac{x^2}{9} + \frac{y^2}{4} = 1$$

the graph of this equation is an ellipse centered at the origin.

What we see geometrically here is two different shapes, the first is a circle, the second is an ellipse. Since both share the same norm of $||\boldsymbol{u}|| = 1$, the Euclidean inner product will produces the equation for a circle, while the weighted Euclidean inner product will produces the equation for an ellipse.

[SI*] The Euclidean inner product and the weighted Euclidean inner products are special cases of a general class of inner products on $\mathbb{R}^n$ called matrix inner products.

To define this class of inner products, let $\boldsymbol{u}$ and $\boldsymbol{v}$ be vectors in $\mathbb{R}^n$ that are expressed in column form, and let A be an invertible $n \times n$ matrix. It can be shown that if $\boldsymbol{u} \cdot \boldsymbol{v}$ is the Euclidean inner product on $\mathbb{R}^n$, then the formula

$$< \boldsymbol{u}, \boldsymbol{v} >= A\boldsymbol{u} \cdot A\boldsymbol{v} \tag{6.89}$$

also defines an inner product; it is called the inner product on $\mathbb{R}^n$ generated by A.

If $\boldsymbol{u}$ and $\boldsymbol{v}$ are in column form, then $\boldsymbol{u} \cdot \boldsymbol{v}$ can be written as $\boldsymbol{v}^T\boldsymbol{u}$ from which it follows that

$$< \boldsymbol{u}, \boldsymbol{v} >= (A\boldsymbol{v})^T A\boldsymbol{u}$$

or, equivalently as

$$< \boldsymbol{u}, \boldsymbol{v} >= \boldsymbol{v}^T A^T A\boldsymbol{u} \tag{6.90}$$

[SI*] The standard Euclidean inner product on $\mathbb{R}^n$ is generated by the $n \times n$ identity matrix, since setting $A = I$ yields

$$< \boldsymbol{u}, \boldsymbol{v} >= I\boldsymbol{u} \cdot I\boldsymbol{v} = \boldsymbol{u} \cdot \boldsymbol{v}$$

and the weighted Euclidean inner product

$$< \boldsymbol{u}, \boldsymbol{v} >= w_1u_1v_1 + w_2u_2v_2 + \cdots + w_nu_nv_n$$

is generated by the matrix

$$A = \begin{bmatrix} \sqrt{w_1} & 0 & 0 & \dots & 0 \\ 0 & \sqrt{w_2} & 0 & \dots & 0 \\ \vdots & \vdots & \vdots & & \vdots \\ 0 & 0 & 0 & \dots & \sqrt{w_n} \end{bmatrix}$$

matrix A is a diagonal matrix with entries of $\sqrt{w_i}$ at the row-i-th, column-i-th, and zeros elsewhere.

We can observe that A^TA will become $n \times n$ diagonal matrix whose diagonal entries are the weights $w_1, w_2, \dots, w_n$.

[SI*] The weighted Euclidean inner product $< \boldsymbol{u}, \boldsymbol{v} >= 3u_1v_1 + 2u_2v_2$ is the inner product on $\mathbb{R}^2$ generated by matrix

$$A = \begin{bmatrix} \sqrt{3} & 0 \\ 0 & \sqrt{2} \end{bmatrix}$$

[SI*] If U and V are $n \times n$ matrices, then the formula

$$< U, V >= \text{tr}\left(U^TV\right) \tag{6.91}$$

defines an inner product on the vector space M_{nn}. This can be proved by confirming that the four inner product space axioms are satisfied.

For the 2×2 matrices we will have

$$U = \begin{bmatrix} u_1 & u_2 \\ u_3 & u_4 \end{bmatrix}, \quad \text{and} \quad V = \begin{bmatrix} v_1 & v_2 \\ v_3 & v_4 \end{bmatrix}$$

This yields

$$< U, V >= \text{tr}\left(U^TV\right) = u_1v_1 + u_2v_2 + u_3v_3 + u_4v_4$$

which is, just the dot product of the corresponding entries in the two matrices. The norm of a matrix U relative tot his inner product is

$$||U|| =< U, U >^{1/2}= \sqrt{u_1^2 + u_2^2 + u_3^2 + u_4^2}$$

[SI*] Recall from calculus that the arc length of a curve $y = f(x)$ over an interval $[a, b]$ is given by the formula

$$L = \int_a^b \sqrt{1 + [f'(x)]^2}\, dx$$

the arc length is not the same with $||\boldsymbol{f}||$, which is the length of $\boldsymbol{f}$ when $\boldsymbol{f}$ is viewed as a vector in $C[a, b]$.

Theorem 6.65: Algebraic Properties of Inner Products

If $\boldsymbol{u}, \boldsymbol{v}$, and $\boldsymbol{w}$ are vectors in a real inner product space V, and if k is a scalar, then:

(a) $< \mathbf{0}, \boldsymbol{v} >=< \boldsymbol{v}, \mathbf{0} >= 0$

(b) $< \boldsymbol{u}, \boldsymbol{v} + \boldsymbol{w} >=< \boldsymbol{u}, \boldsymbol{v} > + < \boldsymbol{u}, \boldsymbol{w} >$

(c) $< \boldsymbol{p}, \boldsymbol{v} - \boldsymbol{w} >=< \boldsymbol{u}, \boldsymbol{v} > - < \boldsymbol{u}, \boldsymbol{w} >$

(d) $< \boldsymbol{u} - \boldsymbol{v}, \boldsymbol{w} >=< \boldsymbol{u}, \boldsymbol{w} > - < \boldsymbol{v}, \boldsymbol{w} >$

(e) $k < \boldsymbol{u}, \boldsymbol{v} >=< \boldsymbol{u}, k\boldsymbol{v} >$

[SI*] The angle θ between two vectors $\boldsymbol{u}$ and $\boldsymbol{v}$ in $\mathbb{R}^n$ is

$$\theta = \cos^{-1}\left(\frac{\boldsymbol{u} \cdot \boldsymbol{v}}{||\boldsymbol{u}||||\boldsymbol{v}||}\right) \tag{6.92}$$

it followed from the Cauchy-Schwarz inequality that

$$-1 \leq \frac{\boldsymbol{u} \cdot \boldsymbol{v}}{||\boldsymbol{u}||||\boldsymbol{v}||} \leq 1 \tag{6.93}$$

as required for the inverse cosine to be defined.

Theorem 6.66: Cauchy-Schwarz inequality

If $\boldsymbol{u}$ and $\boldsymbol{v}$ are vectors in a real inner product space V, then

$$| < \boldsymbol{u}, \boldsymbol{v} > | \leq ||\boldsymbol{u}||||\boldsymbol{v}|| \tag{6.94}$$

[SI*] The following two alternative forms of the Cauchy-Schwarz inequality are useful to know:

$$< \boldsymbol{u}, \boldsymbol{v} >^2 \leq < \boldsymbol{u}, \boldsymbol{u} >< \boldsymbol{v}, \boldsymbol{v} > \tag{6.95}$$

$$< \boldsymbol{u}, \boldsymbol{v} >^2 \leq ||\boldsymbol{u}||^2||\boldsymbol{v}||^2 \tag{6.96}$$

[SI*] From the Cauchy-Schwarz inequality

$$-1 \leq \frac{\boldsymbol{u} \cdot \boldsymbol{v}}{||\boldsymbol{u}||||\boldsymbol{v}||} \leq 1$$

we will have

$$-1 \leq \frac{< \boldsymbol{u}, \boldsymbol{v} >}{||\boldsymbol{u}||||\boldsymbol{v}||} \leq 1 \tag{6.97}$$

there is a unique angle θ in radian measure for which

$$\cos\ \theta = \frac{< \boldsymbol{u}, \boldsymbol{v} >}{||\boldsymbol{u}||||\boldsymbol{v}||}, \quad 0 \leq \theta \leq \pi \tag{6.98}$$

this enables us to define the angle θ between $\boldsymbol{u}$ and $\boldsymbol{v}$ to be

$$\theta = \cos^{-1}\left(\frac{< \boldsymbol{u}, \boldsymbol{v} >}{||\boldsymbol{u}||||\boldsymbol{v}||}\right) \tag{6.99}$$

Not to be confused with the angle between two vectors in Euclidean vector spaces that is definitely use Euclidean inner product ($\boldsymbol{u} \cdot \boldsymbol{v}$), the numerator here is the inner product which can be Euclidean inner product, weighted Euclidean inner product, or inner products generated by matrices.

Theorem 6.67: Properties of Length and Distance in General Inner Product Spaces

If $\boldsymbol{u}, \boldsymbol{v}$, and $\boldsymbol{w}$ are vectors in a real inner product space V, and if k is any scalar, then

(a) $||\boldsymbol{u} + \boldsymbol{v}|| \leq ||\boldsymbol{u}|| + ||\boldsymbol{v}||$ (Triangle inequality for vectors)

(b) $d(\boldsymbol{u}, \boldsymbol{v}) \leq d(\boldsymbol{u}, \boldsymbol{w}) + d(\boldsymbol{w}, \boldsymbol{v})$ (Triangle inequality for distances)

[SI*] There is only an occasional need to compute angles in vector spaces other than $\mathbb{R}^2$ and $\mathbb{R}^3$.

A problem of more interest in general vector spaces is ascertaining whether the angle between vectors is $\frac{\pi}{2}$. If $\boldsymbol{u}$ and $\boldsymbol{v}$ are nonzero vectors, then the angle between them is $\theta = \frac{\pi}{2}$ if and only if $< \boldsymbol{u}, \boldsymbol{v} >= 0$.s

Definition 6.38: Orthogonality

Two vectors $\boldsymbol{u}$ and $\boldsymbol{v}$ in an inner product space are called orthogonal if $< \boldsymbol{u}, \boldsymbol{v} >= 0$.

[SI*] **Example:**
The vectors $\boldsymbol{u} = (1, 1)$ and $\boldsymbol{v} = (1, -1)$ are orthogonal with respect to the Euclidean inner product on $\mathbb{R}^2$, since

$$\boldsymbol{u} \cdot \boldsymbol{v} = (1)(1) + (1)(-1) = 0$$

However, they are not orthogonal with respect to the weighted Euclidean inner product $< \boldsymbol{u}, \boldsymbol{v} >= 3u_1v_1 + 5u_2v_2$, since

$$< \boldsymbol{u}, \boldsymbol{v} >= 3(1)(1) + 5(1)(-1) = -2 \neq 0$$

Theorem 6.68: Generalized Theorem of Pythagoras

If $\boldsymbol{u}$ and $\boldsymbol{v}$ are orthogonal vectors in an inner product space, then

$$||\boldsymbol{u} + \boldsymbol{v}||^2 = ||\boldsymbol{u}|| + ||\boldsymbol{v}||^2$$

Definition 6.39: Orthogonal Complements

If W is a subspace of an inner product space V, then the set of all vectors in V that are orthogonal to every vector in W is called the orthogonal complement of W and is denoted by the symbol $W^{\perp}$.

Theorem 6.69: Orthogonality in Subspace of Inner Product Space

If W is a subspace of an inner product space V, then:

(a) $W^{\perp}$ is a subspace of V.

(b) $W \cap W^{\perp} = \{\mathbf{0}\}$

[SI*] The set $W^{\perp}$ contains at least the zero vector (zero vector is perpendicular toward all vectors in $\mathbb{R}^n$), since

$$< \mathbf{0}, \boldsymbol{w} >= 0$$

for every vector $\boldsymbol{w}$ in W. Thus, we need to show that $W^{\perp}$ is closed under addition and scalar multiplication. To do this, suppose that $\boldsymbol{u}$ and $\boldsymbol{v}$ are vectors in $W^{\perp}$, so that for every vector $\boldsymbol{w}$ in W we have $< \boldsymbol{u}, \boldsymbol{w} >= 0$ and $< \boldsymbol{v}, \boldsymbol{w} >= 0$. It follows from the additivity and homogeneity axioms of inner products that

$$< \boldsymbol{u} + \boldsymbol{v}, \boldsymbol{w} > =< \boldsymbol{u}, \boldsymbol{w} > + < \boldsymbol{v}, \boldsymbol{w} >= 0 + 0 = 0$$
$$< k\boldsymbol{u}, \boldsymbol{w} > = k < \boldsymbol{u}, \boldsymbol{w} >= k(0) = 0$$

since $\mathbf{0}$ is in W and $W^{\perp}$, thus it proves that $\boldsymbol{u} + \boldsymbol{v}$ and $k\boldsymbol{u}$ are in $W^{\perp}$.

If there exists a vector $\boldsymbol{z}$ in both W and $W^{\perp}$, then $\boldsymbol{z}$ is orthogonal to itself; that is,

$$< \boldsymbol{z}, \boldsymbol{z} >= 0$$

It follows from the positivity axiom for inner products that

$$\boldsymbol{z} = \mathbf{0}$$

Theorem 6.70: Orthogonal Complement of $W^{\perp}$

If W is a subspace of a finite-dimensional inner product space V, then the orthogonal complement of $W^{\perp}$ is W; that is,

$$(W^{\perp})^{\perp} = W$$

i. Gram-Schmidt Process

[SI*] In many problems involving vector spaces, we are free to choose any basis for the vector space that seems appropriate. In inner product spaces, the solution of a problem is often greatly simplified by choosing a basis in which the vectors are orthogonal to one another.

Definition 6.40: Orthogonal and Orthonormal

A set of two or more vectors in a real inner product space is said to be orthogonal if all pairs of distinct vectors in the set are orthogonal. An orthogonal set in which each vector has norm 1 is said to be orthonormal.

[SI*] If $\boldsymbol{v}$ is a nonzero vector in an inner product space, then with $k = ||\boldsymbol{v}||$

$$\left|\left|\frac{1}{||\boldsymbol{v}||}\boldsymbol{v}\right|\right| = \left|\frac{1}{||\boldsymbol{v}||}\right| ||\boldsymbol{v}|| = \frac{1}{||\boldsymbol{v}||}||\boldsymbol{v}|| = 1$$

from which we see that multiplying a nonzero vector by the reciprocal of its norm produces a vector of norm 1. This process is called normalizing v.
Any orthogonal set of nonzero vectors can be converted to an orthonormal set by normalizing each of its vectors.

[SI*] The set $S = \{v_1, v_2, v_3\}$ is orthonormal if

$$< v_1, v_2 > = < v_1, v_3 > = < v_2, v_3 > = 0$$
$$||v_1|| = ||v_2|| = ||v_3|| = 1$$

[SI*] In $\mathbb{R}^2$ any two nonzero perpendicular vectors are linearly independent because neither is a scalar multiple of the other. In $\mathbb{R}^3$ any three nonzero mutually perpendicular vectors are linearly independent because no one lies in the plane of the other two.

Theorem 6.71: Orthogonal and Linear Independent

If $S = \{v_1, v_2, \ldots, v_n\}$ is an orthogonal set of nonzero vectors in an inner product space, then S is linearly independent.

[SI*] In an inner product space, a basis consisting of orthonormal vectors is called an orthonormal basis, and a basis consisting of orthogonal vectors is called an orthogonal basis. An example of an orthonormal basis is the standard basis for $\mathbb{R}^n$ with the Euclidean inner product.

[SI*] If we want to express a vector u as a linear combination of basis vectors

$$S = \{v_1, v_2, \ldots, v_n\}$$

we need to convert the vector equation

$$u = c_1 v_1 + c_2 v_2 + \cdots + c_n v_n$$

to a linear system and solve for the coefficients $c_1, c_2, \ldots, c_n$.

Theorem 6.72: Coordinates Relative to Orthonormal Bases

(a) If $S = \{v_1, v_2, \ldots, v_n\}$ is an orthogonal basis for an inner product space V, and if u is any vector in V, then

$$u = \frac{< u, v_1 >}{||v_1||^2} v_1 + \frac{< u, v_2 >}{||v_2||^2} v_2 + \cdots + \frac{< u, v_n >}{||v_n||^2} v_n \tag{6.100}$$

(b) If $S = \{v_1, v_2, \ldots, v_n\}$ is an orthonormal basis for an inner product space V, and if u is any vector in V, then

$$u = < u, v_1 > v_1 + < u, v_2 > v_2 + \cdots + < u, v_n > v_n \tag{6.101}$$

[SI*] The coordinate vector of a vector u in V relative to an orthogonal basis $S = \{v_1, v_2, \ldots, v_n\}$ is

$$(u)_S = \left(\frac{< u, v_1 >}{||v_1||^2}, \frac{< u, v_2 >}{||v_2||^2}, \ldots, \frac{< u, v_n >}{||v_n||^2} \right) \tag{6.102}$$

and relative to an orthonormal basis $S = \{v_1, v_2, \ldots, v_n\}$ is

$$u = < u, v_1 >, < u, v_2 >, \ldots, < u, v_n > \tag{6.103}$$

Theorem 6.73: Projection Theorem

If W is a finite-dimensional subspace of an inner product space V, then every vector $\boldsymbol{u}$ in V can be expressed in exactly one way as

$$\boldsymbol{u} = \boldsymbol{w}_1 + \boldsymbol{w}_2 \tag{6.104}$$

where $\boldsymbol{w}_1$ is in W and $\boldsymbol{w}_2$ is in $W^{\perp}$.

[SI*] The vector $\boldsymbol{w}_1$ and $\boldsymbol{w}_2$ are commonly denoted by

$$\begin{aligned} \boldsymbol{w}_1 &= \text{proj}_{\boldsymbol{w}}\boldsymbol{u} \\ \boldsymbol{w}_2 &= \text{proj}_{\boldsymbol{w}^{\perp}}\boldsymbol{u} \end{aligned} \tag{6.105}$$

They are called the orthogonal projection of $\boldsymbol{u}$ on W and the orthogonal projection of $\boldsymbol{u}$ on $W^{\perp}$, respectively.

The vector $\boldsymbol{w}_2$ is also called the component of $\boldsymbol{u}$ orthogonal to W. The formula for $\boldsymbol{u}$ in term of orthogonal projection is

$$\boldsymbol{u} = \text{proj}_{\boldsymbol{w}}\boldsymbol{u} + \text{proj}_{\boldsymbol{w}^{\perp}}\boldsymbol{u} \tag{6.106}$$

or it can also be written as

$$\boldsymbol{u} = \text{proj}_{\boldsymbol{w}}\boldsymbol{u} + (\boldsymbol{u} - \text{proj}_{\boldsymbol{w}}\boldsymbol{u}) \tag{6.107}$$

Theorem 6.74: Formulas for Calculating Orthogonal Projections

Let W be a finite-dimensional subspace of an inner product space V.

(a) If $\{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_r\}$ is an orthogonal basis for W, and $\boldsymbol{u}$ is any vector in V, then

$$\text{proj}_{\boldsymbol{w}}\boldsymbol{u} = \frac{<\boldsymbol{u}, \boldsymbol{v}_1>}{||\boldsymbol{v}_1||^2}\boldsymbol{v}_1 + \frac{<\boldsymbol{u}, \boldsymbol{v}_2>}{||\boldsymbol{v}_2||^2}\boldsymbol{v}_2 + \cdots + \frac{<\boldsymbol{u}, \boldsymbol{v}_r>}{||\boldsymbol{v}_r||^2}\boldsymbol{v}_r \tag{6.108}$$

(b) If $\{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_r\}$ is an orthonormal basis for W, and $\boldsymbol{u}$ is any vector in V, then

$$\text{proj}_{\boldsymbol{w}}\boldsymbol{u} = <\boldsymbol{u}, \boldsymbol{v}_1>\boldsymbol{v}_1 + <\boldsymbol{u}, \boldsymbol{v}_2>\boldsymbol{v}_2 + \cdots + <\boldsymbol{u}, \boldsymbol{v}_r>\boldsymbol{v}_r \tag{6.109}$$

Theorem 6.75: Orthonormal Basis in Inner Product Space

Every nonzero finite-dimensional inner product space has an orthonormal basis.

[SI*] Let W be any nonzero finite-dimensional subspace of an inner product space, and suppose that $\{\boldsymbol{u}_1, \boldsymbol{u}_2, \ldots, \boldsymbol{u}_r\}$ is any basis for W. The following steps will produce an orthogonal basis $\{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_r\}$ for W.

1. Let $\boldsymbol{v}_1 = \boldsymbol{u}_1$
2. We can obtain a vector $\boldsymbol{v}_2$ that is orthogonal to $\boldsymbol{v}_1$ by computing the component $\boldsymbol{u}_2$ that is orthogonal to the space W_1 spanned by $\boldsymbol{v}_1$.

$$\boldsymbol{v}_2 = \boldsymbol{u}_2 - \text{proj}_{W_1}\boldsymbol{u}_2 = \boldsymbol{u}_2 - \frac{<\boldsymbol{u}_2, \boldsymbol{v}_1>}{||\boldsymbol{v}_1||^2}\boldsymbol{v}_1$$

3. To construct a vector v_3 that is orthogonal to both v_1 and v_2, we compute the component of u_3 orthogonal to the space W_2 spanned by v_1 and v_2.

$$v_3 = u_3 - \text{proj}_{W_2} u_3 = u_3 - \frac{< u_3, v_1 >}{||v_1||^2} v_1 - \frac{< u_3, v_2 >}{||v_2||^2} v_2$$

the linear independence of $\{u_1, u_2, \ldots, u_r\}$ ensures that $v_3 \neq \mathbf{0}$. Continuing in this way we will produce an orthogonal set of vectors after r steps. Since orthogonal sets are linearly independet, this set will be an orthogonal basis for the r-dimensinoal space W.

Definition 6.41: The Gram-Schmidt Process

To convert a basis $\{u_1, u_2, \ldots, u_r\}$ into an orthogonal basis $\{v_1, v_2, \ldots, v_r\}$, perform the following computations:

$$\begin{aligned}
v_1 &= u_1 \\
v_2 &= u_2 - \frac{< u_2, v_1 >}{||v_1||^2} v_1 \\
v_3 &= u_3 - \frac{< u_3, v_1 >}{||v_1||^2} v_1 - \frac{< u_3, v_2 >}{||v_2||^2} v_2 \\
v_4 &= u_4 - \frac{< u_4, v_1 >}{||v_1||^2} v_1 - \frac{< u_4, v_2 >}{||v_2||^2} v_2 - \frac{< u_4, v_3 >}{||v_3||^2} v_3 \\
&\vdots \\
v_r &= u_r - \frac{< u_r, v_1 >}{||v_1||^2} v_1 - \frac{< u_r, v_2 >}{||v_2||^2} v_2 - \cdots - \frac{< u_r, v_{r-1} >}{||v_{r-1}||^2} v_{r-1}
\end{aligned} \tag{6.110}$$

To convert the orthogonal basis into an orthonormal basis $\{q_1, q_2, \ldots, q_r\}$, normalize the orthogonal basis vectors.

Theorem 6.76: Extending Orthonormal Sets to Orthonormal Bases

If W is a finite-dimensional inner product space, then

(a) Every orthogonal set of nonzero vectors in W can be enlarged to an orthogonal basis for W.

(b) Every orthonormal set in W can be enlarged to an orthonormal basis for W.

ii. QR-Decomposition

[SI*] The QR-decomposition / factorization is the thread that connects most of the algorithms of numerical linear algebra [31], including methods for least squares, eigenvalue, and singular value problems, as well as iterative methods for all of these and also for systems of equations.

[SI*] QR-decomposition is very important in the field of numerical algorithms, including those for computing eigenvalues of large matrices.

[SI*] If A is an $m \times n$ matrix with linearly independent column vectors, and if Q is the matrix that results by applying Gram-Schmidt process to the column vectors of A, what relationship, if any exist between A and Q?

To solve the problem above, suppose that the column vectors of A are $\boldsymbol{u}_1, \boldsymbol{u}_2, \ldots, \boldsymbol{u}_n$ and the orthonormal column vectors of Q are $\boldsymbol{q}_1, \boldsymbol{q}_2, \ldots, \boldsymbol{q}_n$. Thus, A and Q can be written in partitioned form as

$$A = [\boldsymbol{u}_1 \mid \boldsymbol{u}_2 \mid \ldots \mid \boldsymbol{u}_n]$$

and

$$Q = [\boldsymbol{q}_1 \mid \boldsymbol{q}_2 \mid \ldots \mid \boldsymbol{q}_n]$$

it follows that $\boldsymbol{u}_1, \boldsymbol{u}_2, \ldots, \boldsymbol{u}_n$ are expressible in terms of the vectors $\boldsymbol{q}_1, \boldsymbol{q}_2, \ldots, \boldsymbol{q}_n$ as

$$\begin{aligned}
\boldsymbol{u}_1 &= <\boldsymbol{u}_1, \boldsymbol{q}_1> \boldsymbol{q}_1 + <\boldsymbol{u}_1, \boldsymbol{q}_2> \boldsymbol{q}_2 + \cdots + <\boldsymbol{u}_1, \boldsymbol{q}_n> \boldsymbol{q}_n \\
\boldsymbol{u}_2 &= <\boldsymbol{u}_2, \boldsymbol{q}_1> \boldsymbol{q}_1 + <\boldsymbol{u}_2, \boldsymbol{q}_2> \boldsymbol{q}_2 + \cdots + <\boldsymbol{u}_2, \boldsymbol{q}_n> \boldsymbol{q}_n \\
&\vdots \\
\boldsymbol{u}_n &= <\boldsymbol{u}_n, \boldsymbol{q}_1> \boldsymbol{q}_1 + <\boldsymbol{u}_n, \boldsymbol{q}_2> \boldsymbol{q}_2 + \cdots + <\boldsymbol{u}_n, \boldsymbol{q}_n> \boldsymbol{q}_n
\end{aligned}$$

Recall that the j-th column vector of a matrix product is a linear combination of the column vectors of the first factor with coefficients coming from the j-th column of the second factor, it follows that these relationships can be expressed in matrix form as

$$[\boldsymbol{u}_1 \mid \boldsymbol{u}_2 \mid \ldots \mid \boldsymbol{u}_n] = [\boldsymbol{q}_1 \mid \boldsymbol{q}_2 \mid \ldots \mid \boldsymbol{q}_n] \begin{bmatrix} <\boldsymbol{u}_1, \boldsymbol{q}_1> & <\boldsymbol{u}_2, \boldsymbol{q}_1> & \ldots & <\boldsymbol{u}_n, \boldsymbol{q}_1> \\ <\boldsymbol{u}_1, \boldsymbol{q}_2> & <\boldsymbol{u}_2, \boldsymbol{q}_2> & \ldots & <\boldsymbol{u}_n, \boldsymbol{q}_2> \\ \vdots & \vdots & & \vdots \\ <\boldsymbol{u}_1, \boldsymbol{q}_n> & <\boldsymbol{u}_2, \boldsymbol{q}_n> & \ldots & <\boldsymbol{u}_n, \boldsymbol{q}_n> \end{bmatrix}$$

or more briefly as

$$A = QR \tag{6.111}$$

where R is the second factor in the product. This equation tells that it is a factorization of A into the product of a matrix Q with orthonormal column vectors and an invertible upper triangular matrix R. However, it is a property of the Gram-Schmidt process that for $j \geq 2$, the vector $\boldsymbol{q}_j$ is orthogonal to $\boldsymbol{u}_1, \boldsymbol{u}_2, \ldots, \boldsymbol{u}_{j-1}$. Thus, all entries below the main diagonal of R are zero, and R has the form

$$\begin{bmatrix} <\boldsymbol{u}_1, \boldsymbol{q}_1> & <\boldsymbol{u}_2, \boldsymbol{q}_1> & \ldots & <\boldsymbol{u}_n, \boldsymbol{q}_1> \\ 0 & <\boldsymbol{u}_2, \boldsymbol{q}_2> & \ldots & <\boldsymbol{u}_n, \boldsymbol{q}_2> \\ \vdots & \vdots & & \vdots \\ 0 & 0 & \ldots & <\boldsymbol{u}_n, \boldsymbol{q}_n> \end{bmatrix} \tag{6.112}$$

R is invertible and the diagonal entries are nonzero.

Theorem 6.77: QR-Decomposition

If A is an $m \times n$ matrix with linearly independent column vectors, then A can be factored as

$$A = QR$$

where Q is an $m \times n$ matrix with orthonormal column vectors, and R is an $n \times n$ invertible upper triangular matrix.

[SI*] Some Characteristics of QR-decomposition:

1. QR-decomposition always exists, but may not be unique.
2. If the determinant of matrix A is not 0 and all diagonal entries of $R > 0$, then this decomposition is unique.
3. There are two types of QR-decomposition: full and reduced.

QR-decomposition is also applicable for $m \times n$ matrices, where $m \geq n$. In that case, Q will be a $m \times m$ matrix and R will be an $m \times n$ matrix.

iii. Functions in SymIntegration Related to Inner Product Spaces

[SI*] We will list the basic functions that can be used related to inner product spaces:

gramschmidt(vector<vector<double>> &A)
to compute the orthonormal basis from basis that become the column vector of matrix A.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Gram-Schmidt ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Gram-Schmidt ]# ./main

A:
               1.000000              0.000000              0.000000
               1.000000              1.000000              0.000000
               1.000000              1.000000              1.000000


Orthogonal basis:
               1.000000             -0.666667              0.000000
               1.000000              0.333333             -0.500000
               1.000000              0.333333              0.500000

Orthonormal basis:
               0.577350             -0.816497              0.000000
               0.577350              0.408248             -0.707107
               0.577350              0.408248              0.707107

Time taken by function: 3110 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Gram-Schmidt ]#
```

Figure 6.48: *The computation for Gram-Schmidt orthogonalization method. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Gram-Schmidt/main.cpp**).*

isSymmetric(const vector<vector<double>>& A)
a **bool** function to check whether a square matrix A is symmetric or not.

isPositiveDefinite(const vector<vector<double>>& A)
a **bool** function to check whether matrix A is positive definite using Cholesky decomposition. The Cholesky method is faster and more numerically stable for larger matrices. A matrix is positive definite if it is symmetric and its Cholesky decomposition ($A = LL^T$) succeeds with positive diagonal elements.

computeAInnerProduct(const vector<double>& v_1, const vector<vector<double>>& A, const vector<double>& v_2)
to compute A-inner product

$$< v_1, v_2 >_A = v_1^T A v_2$$

computeAOrthogonal(const vector<vector<double>>& A, vector<vector<double>>& V)
a **vector<vector<double>>** function to compute an A-orthogonal set of vectors from a set

of linearly independent vectors (usually basis vectors or identity matrix) using a positive-definite matrix A with classical Gram-Schmidt method.

ModifiedGramSchmidtA(const vector<vector<double>>& A, vector<vector<double>>& V)
a **void** function to compute an A-orthogonal set of vectors from a set of linearly independent vectors (usually basis vectors or identity matrix) using a positive-definite matrix A with modified Gram-Schmidt method.

QRDecomposition(vector<vector<double>> &A, vector<vector<double>> &Q, vector<vector<double>> &R)
to compute the QR-decomposition of matrix A.

QRDecompositionComplex(ComplexMatrix &, ComplexMatrix &, ComplexMatrix &)
to compute the QR-decomposition of a complex matrix A.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration QR Decomposition Complex Eigenvalues Manual ]# ./main

Matrix A :
       (1,0)        (0,0)        (0,0)
       (1,0)        (1,0)        (0,0)
       (1,0)        (1,0)        (1,0)

Matrix Q :
   (0.577,0)   (-0.816,0) (-2.36e-16,0)
   (0.577,0)    (0.408,0)   (-0.707,0)
   (0.577,0)    (0.408,0)    (0.707,0)

Matrix R :
    (1.73,0)     (1.15,0)    (0.577,0)
       (0,0)    (0.816,0)    (0.408,0)
       (0,0)        (0,0)    (0.707,0)

Time taken by function: 1128 microseconds
```

Figure 6.49: *The QR-decomposition of a complex matrix A. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration QR Decomposition Complex Matrix/main.cpp**).*

QRDecompositionHouseholderReflections(vector<vector<complex<double>>>& A)
to compute the QR-decomposition of a complex matrix A with Householder reflections.

iv. Compute QR-Decomposition Using Gram-Schmidt with SymIntegration

Find the QR-decomposition of

$$A = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

Solution:

The column vectors of A are

$$\boldsymbol{u}_1 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}, \quad \boldsymbol{u}_2 = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}, \quad \boldsymbol{u}_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

Applying the Gram-Schmidt process with normalization to these column vectors yields the orthonormal vectors

$$\boldsymbol{q}_1 = \begin{bmatrix} \frac{1}{\sqrt{3}} \\ \frac{1}{\sqrt{3}} \\ \frac{1}{\sqrt{3}} \end{bmatrix}, \quad \boldsymbol{q}_2 = \begin{bmatrix} -\frac{2}{\sqrt{6}} \\ \frac{1}{\sqrt{6}} \\ \frac{1}{\sqrt{6}} \end{bmatrix}, \quad \boldsymbol{q}_3 = \begin{bmatrix} 0 \\ -\frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{bmatrix}$$

Thus

$$R = \begin{bmatrix} < \boldsymbol{u}_1, \boldsymbol{q}_1 > & < \boldsymbol{u}_2, \boldsymbol{q}_1 > & < \boldsymbol{u}_3, \boldsymbol{q}_1 > \\ 0 & < \boldsymbol{u}_2, \boldsymbol{q}_2 > & < \boldsymbol{u}_3, \boldsymbol{q}_2 > \\ 0 & 0 & < \boldsymbol{u}_3, \boldsymbol{q}_3 > \end{bmatrix} = \begin{bmatrix} \frac{3}{\sqrt{3}} & \frac{2}{\sqrt{3}} & \frac{1}{\sqrt{3}} \\ 0 & \frac{2}{\sqrt{6}} & \frac{1}{\sqrt{6}} \\ 0 & 0 & \frac{1}{\sqrt{2}} \end{bmatrix}$$

It follows that the QR-decomposition of A is

$$\begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} \frac{1}{\sqrt{3}} & -\frac{2}{\sqrt{6}} & 0 \\ \frac{1}{\sqrt{3}} & \frac{1}{\sqrt{6}} & -\frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{3}} & \frac{1}{\sqrt{6}} & \frac{1}{\sqrt{2}} \end{bmatrix} \begin{bmatrix} \frac{3}{\sqrt{3}} & \frac{2}{\sqrt{3}} & \frac{1}{\sqrt{3}} \\ 0 & \frac{2}{\sqrt{6}} & \frac{1}{\sqrt{6}} \\ 0 & 0 & \frac{1}{\sqrt{2}} \end{bmatrix}$$

To compute the QR-decomposition in SymIntegration we use this function:
QRDecomposition(vector<vector<double>> &A, vector<vector<double>> &Q, vector<vector<double>> &R)

With A as the input matrix that we want to decompose, matrices Q and R will be the decomposition result but we need to declare them first as square matrices with same size as A before we call the function.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration QR Decomposition ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration QR Decomposition ]# ./main

A:
               1.000000               0.000000               0.000000
               1.000000               1.000000               0.000000
               1.000000               1.000000               1.000000


Q:
               0.577350              -0.816497               0.000000
               0.577350               0.408248              -0.707107
               0.577350               0.408248               0.707107

R:
               1.732051               1.154701               0.577350
               0.000000               0.816497               0.408248
               0.000000               0.000000               0.707107

Time taken by function: 1341 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration QR Decomposition ]#
```

Figure 6.50: *The computation to obtain the QR-decomposition of matrix A with Gram-Schmidt orthogonalization method. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration QR Decomposition/-main.cpp**).*

Implementing QR-decomposition in this SymIntegration' function is using the Gram-Schmidt orthogonalization, another method is by using Householder transformations. Gram-Schmidt is simpler to implement but less numerically stable. Libraries like LAPACK or Eigen are using Householder reflections for their stability. As a comparison, in another section we will talk about SymIntegration' function to compute QR-decomposition with Householder reflections / transformations.

v. Compute QR-Decomposition Using Householder Reflections with SymInteg-ration

Find the QR-decomposition with Householder reflections of

$$A = \begin{bmatrix} -1 & -1 & 1 \\ 1 & 3 & 3 \\ -1 & -1 & 5 \end{bmatrix}$$

Solution:

Suppose we define matrix A with size fo $m \times n$.

The number of iterations depends on the number of row and the number of column, if $m = n$ then we will have $m - 1$ iterations, if $m < n$ then we will have m iterations.

First, at iteration $j = 1$ we will have

$$\boldsymbol{x} = \begin{bmatrix} -1 \\ 1 \\ -1 \end{bmatrix}$$

(we define vector $\boldsymbol{x}$ as the first column of matrix A with criteria, if $j > 1$ then $x(1 : j - 1) = 0$, e.g. at iteration $j = 2$ we will take the second column of matrix A for vector $\boldsymbol{x}$ and we will make the first entry of vector $\boldsymbol{x}$ to be zero, at iteration $j = 3$ we will take the second column of matrix A for vector $\boldsymbol{x}$ and we will make the first and second entry of vector $\boldsymbol{x}$ to be zero).

then

$$\boldsymbol{w} = \pm||\boldsymbol{x}||\boldsymbol{e}_1 \begin{bmatrix} \sqrt{3} \\ 0 \\ 0 \end{bmatrix}$$

Construct the householder vector $\boldsymbol{v}$

$$\boldsymbol{v} = \boldsymbol{x} - \boldsymbol{w} \begin{bmatrix} -2.7321 \\ 1 \\ -1 \end{bmatrix}$$

Construct the Householder reflector H_j

$$H_1 = I - 2\frac{\boldsymbol{v}\boldsymbol{v}^T}{\boldsymbol{v}^T\boldsymbol{v}} = \begin{bmatrix} -0.5774 & 0.5774 & -0.5774 \\ 0.5774 & 0.7887 & 0.211 \\ -0.5774 & -0.211 & 0.7887 \end{bmatrix}$$

we will obtain

$$R_1 = H_1 A = \begin{bmatrix} 1.73 & 2.89 & -1.73 \\ 0 & 1.58 & 4 \\ 0 & 0.423 & 4 \end{bmatrix}$$

and

$$Q_1 = H_1 = \begin{bmatrix} -0.5774 & 0.5774 & -0.5774 \\ 0.5774 & 0.7887 & 0.211 \\ -0.5774 & -0.211 & 0.7887 \end{bmatrix}$$

we will set $A = R$ for the next iteration.

Continuing at iteration $j = 2$

$$\boldsymbol{x} = \begin{bmatrix} 0 \\ 1.58 \\ 0.423 \end{bmatrix}$$

$$\boldsymbol{w} = \pm||\boldsymbol{x}||\boldsymbol{e}_2 \begin{bmatrix} 0 \\ -1.63 \\ 0 \end{bmatrix}$$

Construct the householder vector $\boldsymbol{v}$

$$\boldsymbol{v} = \boldsymbol{x} - \boldsymbol{w} \begin{bmatrix} 0 \\ 3.21 \\ 0.423 \end{bmatrix}$$

Construct the Householder reflector H_j

$$H_2 = I - 2\frac{\boldsymbol{v}\boldsymbol{v}^T}{\boldsymbol{v}^T\boldsymbol{v}} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -0.966 & -0.259 \\ 0 & -0.259 & 0.966 \end{bmatrix}$$

we will obtain

$$R_2 = H_2 R_1 = \begin{bmatrix} 1.73 & 2.89 & -1.73 \\ 0 & -1.63 & -4.9 \\ 0 & 0 & 2.83 \end{bmatrix}$$

and

$$Q_2 = H_1 H_2 = \begin{bmatrix} -0.5774 & -0.408 & -0.707 \\ 0.5774 & -0.816 & 0 \\ -0.5774 & -0.408 & 0.707 \end{bmatrix}$$

At the end of iteration we will obtain the QR-decomposition:

$$Q = H_1 H_2$$

and

$$R = H_2 H_1 A$$

This method uses a sequence of reflections to zero out the entries below the diagonal of the matrix A. Each reflection is a transformation that flips the matrix across a hyperplane. The sequence of reflections is stored in the orthogonal matrix Q, and the result of these reflections is the upper triangular matrix R. This method is generally more numerically stable than the Gram-Schmidt process.

Implementing Householder transformations is significantly more complex than Gam-Schmidt but yields better results for real-world numerical problems.

In SymIntegration, we can compute the QR-decomposition with Householder reflections with this function:
QRDecompositionHouseholderReflections(vector<vector<complex<double>>>& A)

with A as the matrix that we want to decompose.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration QR Algorithm with Householder Reflections ]# ./main

Matrix A :
      (-1,0)       (-1,0)        (1,0)
       (1,0)        (3,0)        (3,0)
      (-1,0)       (-1,0)        (5,0)

Iteration : 1

x:
(-1,0)
(1,0)
(-1,0)

w:
(1.73,0)
(0,0)
(0,0)

v:
(-2.73,0)
(1,0)
(-1,0)

R:
    (1.73,0)     (2.89,0)    (-1.73,0)
       (0,0)     (1.58,0)        (4,0)
       (0,0)    (0.423,0)        (4,0)

Householder Matrix / Q:
  (-0.577,0)    (0.577,0)   (-0.577,0)
   (0.577,0)    (0.789,0)    (0.211,0)
  (-0.577,0)    (0.211,0)    (0.789,0)
```

Figure 6.51: *The computation to obtain the QR-decomposition of matrix A with Householder reflections method. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration QR Decomposition with Householder Reflections/main.cpp**).*

```
Iteration : 2

X:
(0,0)
(1.58,0)
(0.423,0)

W:
(0,0)
(-1.63,0)
(0,0)

V:
(0,0)
(3.21,0)
(0.423,0)

R:
    (1.73,0)     (2.89,0)    (-1.73,0)
       (0,0)    (-1.63,0)     (-4.9,0)
       (0,0)        (0,0)     (2.83,0)

Householder Matrix / Q:
  (-0.577,0)   (-0.408,0)   (-0.707,0)
   (0.577,0)   (-0.816,0)        (0,0)
  (-0.577,0)   (-0.408,0)    (0.707,0)

*************************************************************************

End of iteration

*************************************************************************

Q:
  (-0.577,0)   (-0.408,0)   (-0.707,0)
   (0.577,0)   (-0.816,0)        (0,0)
  (-0.577,0)   (-0.408,0)    (0.707,0)

R:
    (1.73,0)     (2.89,0)    (-1.73,0)
       (0,0)    (-1.63,0)     (-4.9,0)
       (0,0)        (0,0)     (2.83,0)

Q * R:
      (-1,0)       (-1,0)        (1,0)
       (1,0)        (3,0)        (3,0)
      (-1,0)       (-1,0)        (5,0)

Time taken by function: 2877 microseconds
```

Figure 6.52: *The computation to obtain the QR-decomposition of matrix A with Householder reflections method. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration QR Decomposition with Householder Reflections/main.cpp**).*

At the heart of the Householder method is an idea originally proposed by Alston Householder in 1958. This is an ingenious way of designing the unitary matrices Q_k so that $Q_n \dots Q_2 Q_1 A = R$ is an upper-triangular.

The matrix Q_k is chosen to introduce zeros below the diagonal in the k-th column while preserving all the zeros previously introduced [31].

VII. Diagonalization and Quadratic Forms

[SI*] We have found conditions that guaranteed the diagonalizability of an $n \times n$ matrix, now we will check what class or classes of matrices might actually satisfy those conditions.

Definition 6.42: Orthogonal Matrix

A square matrix A is said to be orthogonal if its transpose is the same as its inverse, that is, if

$$A^{-1} = A^T$$

or, equivalently, if

$$AA^T = A^T A = I$$

[SI*] The standard matrix for the counterclockwise rotation of $\mathbb{R}^2$ through an angle θ is

$$A = \begin{bmatrix} \cos\,\theta & -\sin\,\theta \\ \sin\,\theta & \cos\,\theta \end{bmatrix}$$

This matrix is orthogonal for all choices of θ since

$$A^T A = \begin{bmatrix} \cos\,\theta & \sin\,\theta \\ -\sin\,\theta & \cos\,\theta \end{bmatrix} \begin{bmatrix} \cos\,\theta & -\sin\,\theta \\ \sin\,\theta & \cos\,\theta \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Theorem 6.78: Equivalent Statement for Orthogonal Matrix

The following are equivalent for an $n \times n$ matrix A.

(a) A is orthogonal.

(b) The row vectors of A form an orthonormal set in $\mathbb{R}^n$ with the Euclidean inner product.

(c) The column vectors of A form an orthonormal set in $\mathbb{R}^n$ with the Euclidean inner product.

[SI*] If the row vectors of A are $\boldsymbol{r}_1, \boldsymbol{r}_2, \ldots, \boldsymbol{r}_n$ then AA^T can be expressed as

$$AA^T = \begin{bmatrix} \boldsymbol{r}_1 \\ \boldsymbol{r}_2 \\ \vdots \\ \boldsymbol{r}_n \end{bmatrix} \begin{bmatrix} \boldsymbol{r}_1 & \boldsymbol{r}_2 & \ldots & \boldsymbol{r}_n \end{bmatrix} = \begin{bmatrix} \boldsymbol{r}_1 \cdot \boldsymbol{r}_1 & \boldsymbol{r}_1 \cdot \boldsymbol{r}_2 & \ldots & \boldsymbol{r}_1 \cdot \boldsymbol{r}_n \\ \boldsymbol{r}_2 \cdot \boldsymbol{r}_1 & \boldsymbol{r}_2 \cdot \boldsymbol{r}_2 & \ldots & \boldsymbol{r}_2 \cdot \boldsymbol{r}_n \\ \vdots & \vdots & & \vdots \\ \boldsymbol{r}_n \cdot \boldsymbol{r}_1 & \boldsymbol{r}_n \cdot \boldsymbol{r}_2 & \ldots & \boldsymbol{r}_n \cdot \boldsymbol{r}_n \end{bmatrix}$$

it follows that $AA^T = I$ if and only if

$$\boldsymbol{r}_1 \cdot \boldsymbol{r}_1 = \boldsymbol{r}_2 \cdot \boldsymbol{r}_2 = \cdots = \boldsymbol{r}_n \cdot \boldsymbol{r}_n = 1$$

and

$$\boldsymbol{r}_i \cdot \boldsymbol{r}_j = 0, \quad i \neq j$$

which are true if and only if $\{\boldsymbol{r}_1, \boldsymbol{r}_2, \ldots, \boldsymbol{r}_n\}$ is an orthonormal set in $\mathbb{R}^n$.

Theorem 6.79: Fundamental Properties of Orthogonal Matrices

(a) The inverse of an orthogonal matrix is orthogonal.

(b) A product of orthogonal matrices is orthogonal.

(c) If A is orthogonal, then $\det(A) = 1$ or $\det(A) = -1$.

Theorem 6.80: Orthogonal Matrices as Linear Operators

If A is an $n \times n$ matrix, then the following are equivalent

(a) A is orthogonal.

(b) $||A\boldsymbol{x}|| = ||\boldsymbol{x}||$ for all $\boldsymbol{x}$ in $\mathbb{R}^n$.

(c) $A\boldsymbol{x} \cdot A\boldsymbol{y} = \boldsymbol{x} \cdot \boldsymbol{y}$ for all $\boldsymbol{x}$ and $\boldsymbol{y}$ in $\mathbb{R}^n$.

[SI*] If A is orthogonal matrix and $T_A : \mathbb{R}^n \to \mathbb{R}^n$ is multiplication by A, then we will call T_A orthogonal operator on $\mathbb{R}^n$.

The orthogonal operators on $\mathbb{R}^n$ are precisely those operators that leave the lengths of all vectors unchanged.

[SI*] Orthonormal bases for inner product spaces are convenient because many familiar formulas hold for such bases.

Theorem 6.81: Change of Orthonormal Basis

If S is an orthonormal basis for an n-dimensional inner product space V, and if

$$(\boldsymbol{u})_S = (u_1, u_2, \ldots, u_n)$$
$$(\boldsymbol{v})_S = (v_1, v_2, \ldots, v_n)$$

then:

(a) $||\boldsymbol{u}|| = \sqrt{u_1^2 + u_2^2 + \cdots + u_n^2}$

(b) $d(\boldsymbol{u}, \boldsymbol{v}) = \sqrt{(u_1 - v_1)^2 + (u_2 - v_2)^2 + \cdots + (u_n - v_n)^2}$

(c) $< \boldsymbol{u}, \boldsymbol{v} >= u_1v_1 + u_2v_2 + \cdots + u_nv_n$

[SI*] Transitions between orthonormal bases for an inner product space are of special importance in geometry and various applications.

Theorem 6.82: Transition Matrix for Orthonormal Basis

Let V be a finite-dimensional inner product space. If P is the transition matrix from one orthonormal basis for V to another orthonormal basis for V, then P is an orthogonal matrix.

[SI*] **Example:**

What conditions must a and b satisfy for the matrix

$$\begin{bmatrix} a+b & b-a \\ a-b & b+a \end{bmatrix}$$

to be orthogonal?

Solution:

We will compute A^TA then determine a and b from there

$$A^TA = \begin{bmatrix} 2(a^2+b^2) & 0 \\ 0 & 2(a^2+b^2) \end{bmatrix}$$

To be orthogonal the condition of $A^TA = I$ has to be fulfilled, thus

$$\begin{aligned} 2(a^2+b^2) &= 1 \\ a^2+b^2 &= \frac{1}{2} \end{aligned}$$

so a and b must satisfy $a^2+b^2 = \frac{1}{2}$.

[SI*] **Example:**

Find a, b, and c for which the matrix

$$\begin{bmatrix} a & \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \\ b & \frac{1}{\sqrt{6}} & \frac{1}{\sqrt{6}} \\ c & \frac{1}{\sqrt{3}} & \frac{1}{\sqrt{3}} \end{bmatrix}$$

is orthogonal. Are the values of a, b, and c are unique?

Solution:

The row vectors of an orthogonal matrix are an orthonormal set. Thus

$$\begin{aligned} ||\boldsymbol{r}_1||^2 &= 1 \\ a^2 + \left(\frac{1}{\sqrt{2}}\right)^2 + \left(-\frac{1}{\sqrt{2}}\right)^2 &= 1 \\ a^2 + 1 &= 1 \\ a &= 0 \end{aligned}$$

$$\begin{aligned} ||\boldsymbol{r}_2||^2 &= 1 \\ b^2 + \left(\frac{1}{\sqrt{6}}\right)^2 + \left(\frac{1}{\sqrt{6}}\right)^2 &= 1 \\ b^2 + \frac{2}{6} &= 1 \\ b &= \pm\frac{2}{\sqrt{6}} \end{aligned}$$

$$\begin{aligned} ||\boldsymbol{r}_3||^2 &= 1 \\ c^2 + \left(\frac{1}{\sqrt{3}}\right)^2 + \left(\frac{1}{\sqrt{3}}\right)^2 &= 1 \\ c^2 + \frac{2}{3} &= 1 \\ c &= \pm\frac{1}{\sqrt{3}} \end{aligned}$$

The column vectors of an orthogonal matrix are an orthonormal set, form which it is clear that b and c must have opposite signs. Thus the only possibilities are

$$a = 0, \quad b = \frac{2}{\sqrt{6}}, \quad c = -\frac{1}{\sqrt{3}}$$

or

$$a = 0, \quad b = -\frac{2}{\sqrt{6}}, \quad c = \frac{1}{\sqrt{3}}$$

[SI*] Multiplication by a 3×3 orthogonal matrix A is a rotation about some axis if $\det(A) = 1$ and is a rotation about some axis followed by a reflection about some coordinate plane if $\det(A) = -1$.

[SI*] A linear operator on $\mathbb{R}^2$ is called rigid if it does not change the lengths of vectors, and it is called angle preserving if it does not change the angle between nonzero vectors.

1. Linear operators that are rigid: rotations about the origin, reflections about any line through the origin.
2. Linear operators that are angle preserving: All rigid operators on $\mathbb{R}^2$ are angle preserving. Dilations and contractions are angle preserving operators that are not rigid.

Definition 6.43: Orthogonally Similar

If A and B are suare matrices, then we say that A and B are orthogonally similar if there is an orthogonal matrix P such that $P^T AP = B$.

[SI*] If A is orthogonally similar to some diagonal matrix, say

$$P^T AP = D$$

then we say that A is orthogonally diagonalizable and that P orthogonally diagonalizes A.

Theorem 6.83: Conditions for Orthogonal Diagonalizability

If A is an $n \times n$ matrix, then the following are equivalent

(a) A is orthogonally diagonalizable.

(b) A has an orthonormal set of n eigenvectors.

(c) A is symmetric.

Theorem 6.84: Properties of Symmetric Matrices

If A is a symmetric matrix, then:

(a) The eigenvalues of A are all real numbers.

(b) Eigenvectors from different eigenspaces are orthogonal.

[SI*] Procedure for orthogonally diagonalizing a symmetric matrix.

1. Find a basis for each eigenspace of A.

2. Apply the Gram-Schmidt process to each of these bases to obtain an orthonormal basis for each eigenspace.

3. Form the matrix P whose columns are the vectors constructed in step 2. This matrix will orthogonally diagonalize A, and the eigenvalues on the diagonal of $D = P^TAP$ will be in the same order as their corresponding eigenvectors in P.

Eigenvectors from different eigenspaces are orthogonal, and applying Gram-Schmidt process ensures that the eigenvectors within the same eigenspace are orthonormal. It follows that the entire set of eigenvectors obtained by this procedure will be orthonormal.

[SI*] If A is a symmetric matrix that is orthogonally diagonalized by

$$P = [\boldsymbol{u}_1 \ \boldsymbol{u}_2 \ \ldots \ \boldsymbol{u}_n]$$

and if $\lambda_1, \lambda_2, \ldots, \lambda_n$ are the eigenvalues of A corresponding to the unit eigenvectors $\boldsymbol{u}_1, \boldsymbol{u}_2, \ldots, \boldsymbol{u}_n$, then we know that $D = P^TAP$, where D is a diagonal matrix with the eigenvalues in the diagonal positions. It follows from this that the matrix A can be expressed as

$$A = PDP^T = \begin{bmatrix} \boldsymbol{u}_1 & \boldsymbol{u}_2 & \ldots & \boldsymbol{u}_n \end{bmatrix} \begin{bmatrix} \lambda_1 & 0 & \ldots & 0 \\ 0 & \lambda_2 & \ldots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \ldots & \lambda_n \end{bmatrix} \begin{bmatrix} \boldsymbol{u}_1^T \\ \boldsymbol{u}_2^T \\ \vdots \\ \boldsymbol{u}_n^T \end{bmatrix} = \begin{bmatrix} \lambda_1\boldsymbol{u}_1 & \lambda_2\boldsymbol{u}_2 & \ldots & \lambda_n\boldsymbol{u}_n \end{bmatrix} \begin{bmatrix} \boldsymbol{u}_1^T \\ \boldsymbol{u}_2^T \\ \vdots \\ \boldsymbol{u}_n^T \end{bmatrix}$$

Multiplying out, we obtain the formula

$$A = \lambda_1\boldsymbol{u}_1\boldsymbol{u}_1^T + \lambda_2\boldsymbol{u}_2\boldsymbol{u}_2^T + \cdots + \lambda_n\boldsymbol{u}_n\boldsymbol{u}_n^T \tag{6.113}$$

which is called a spectral decomposition of A.

Note that in each term of the spectral decomposition of A has the form $\lambda\boldsymbol{u}\boldsymbol{u}^T$, where $\boldsymbol{u}$ is a unit eigenvector of A in column form, and λ is an eigenvalue of A correponding to $\boldsymbol{u}$. Since $\boldsymbol{u}$ has size $n \times 1$, it follows that the product $\boldsymbol{u}\boldsymbol{u}^T$ has size $n \times n$.

It can be proved that $\boldsymbol{u}\boldsymbol{u}^T$ is the standard matrix for the orthogonal projection of $\mathbb{R}^n$ on the subspace spanned by the vector $\boldsymbol{u}$.

The spectral decomposition of A tells that the image of a vector $\boldsymbol{x}$ under multiplication by a symmetric matrix A can be obtained by projecting $\boldsymbol{x}$ orthogonally on the lines (one-dimensional subspaces) determined by the eigenvectors of A, then scaling those projections by the eigenvalues, and then adding the scaled projections.

The terminology spectral decomposition is derived from the fact that the set of all eigenvalues of a matrix A is sometimes called the spectrum of A. The terminology eigenvalue decomposition is due to a paper entitled "A Singularly Valuable Decomposition: The SVD of a Matrix," written by Professor Dan Kalman.

[SI*] If A is an $n \times n$ matrix that is not orthogonally diagonalizable, it may still be possible to achieve considerable simplification in the form of P^TAP by choosing the orthogonal matrix P appropriately.

Theorem 6.85: Schur's Theorem

If A is an $n \times n$ matrix with real entries and real eigenvalues, then there is an orthogonal matrix P such that $P^T AP$ is an upper triangular matrix of the form

$$P^T AP = \begin{bmatrix} \lambda_1 & X & X & \dots & X \\ 0 & \lambda_2 & X & \dots & X \\ 0 & 0 & \lambda_3 & \dots & X \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & \lambda_n \end{bmatrix} \tag{6.114}$$

in which $\lambda_1, \lambda_2, \dots, \lambda_n$ are eigenvalues of the matrix A repeated according to multiplicity.

[SI*] It is common to denote the upper triangular matrix in Schur's Theorem by S in which case that the equation from Schur's Theorem can be rewritten as

$$A = PSP^T \tag{6.115}$$

which is called a Schur decomposition of A.

Theorem 6.86: Hessenberg's Theorem

If A is an $n \times n$ matrix, then there is an orthogonal matrix P such that $P^T AP$

$$P^T AP = \begin{bmatrix} X & X & \dots & X & X & X \\ X & X & \dots & X & X & X \\ 0 & X & \ddots & X & X & X \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & \dots & X & X & X \\ 0 & 0 & \dots & 0 & X & X \end{bmatrix} \tag{6.116}$$

or in other words every square matrix with real entries is orthogonally similar to a matrix in which each entry below the first subdiagonal is zero. Such a matrix is said to be in upper Hessenberg form.

* The first subdiagonal is $\{a_{i+1,i}\}$ with $i = 1, 2, \dots, n$.

[SI*] It is common to denote the upper Hessenberg matrix by H, in which case that equation can be rewritten as

$$A = PHP^T \tag{6.117}$$

which is called an upper Hessenberg decomposition of A.

In many numerical algoriths the initial matrix is first converted to upper Hessenberg form to reduce the amount of computation in subsequent parts of the algorithm.

[SI*] **Example:**
Let A be a diagonalizable matrix with the property that eigenvectors from distinct eigenvalues are orthogonal. Must A be symmetric?

Solution:
Suppose A is a matrix with size of $n \times n$, if we want to diagonalize A, then we will have to:

1. Compute the eigenvectors of A:

$$\{\lambda_1, \lambda_2, \ldots, \lambda_n\}$$

2. Find the basis for each eigenspace of A by solving

$$(\lambda_i I - A)\boldsymbol{x} = \mathbf{0}$$

with $i = 1, 2, \ldots, n$. The basis $\{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n\}$ will be called eigenvectors of A.

3. Apply the Gram-Schmidt process to each of the bases / the eigenvectors we obtain from step (2) to convert $\{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n\}$ into an orthogonal set $\{\boldsymbol{u}_1, \boldsymbol{u}_2, \ldots, \boldsymbol{u}_n\}$.

We can then normalize $\{\boldsymbol{u}_1, \boldsymbol{u}_2, \ldots, \boldsymbol{u}_n\}$ to become an orthonormal set.

4. The matrix P that diagonalizes A consist of the orthonormal set $\{\boldsymbol{u}_1, \boldsymbol{u}_2, \ldots, \boldsymbol{u}_n\}$ as the column vectors.

$$P = [\boldsymbol{u}_1 \mid \boldsymbol{u}_2 \mid \ \ldots \ \mid \boldsymbol{u}_n]$$

The Gram-Schmidt process will ensure that columns of P corresponding to the same eigenvalue are an orthonormal set. Since eigenvectors from distinct eigenvalues are orthogonal, this means that P will be an orthogonal matrix. Then since A is orthogonally diagonalizable, it must be symmetric. From Theorem that states if A is an $n \times n$ matrix, whenever A is orthogonally diagonalizable with n orthonormal set of eigenvectors then A is symmetric as well.

Guide 6.8: Hessenberg Decomposition

If A is an $n \times n$ matrix that is not orthogonally diagonalizable, we can use Hessenberg decomposition to decompose every square matrix with real entries into a matrix in which each entry below the first subdiagonal is zero.

$$A = PHP^T$$

and

$$H = P^T AP$$

matrices H and A are orthogonally similar.

The algorithm to obtain matrix H:

1. Set variable:
   $row = 1$;
2. Iterate from first column to the second-to-last column, $j = 0, 1, \ldots, n-2$.

   Extract a sub-column from matrix A and assign it as vector $\boldsymbol{x}$, starting from entry at row $j+1$ down to the bottom. Vector $\boldsymbol{x}$ will have the length of $j - row$.
3. Create vector $\boldsymbol{w}$ from vector $\boldsymbol{x}$,

   $$\boldsymbol{w} = \begin{bmatrix} \pm||\boldsymbol{x}|| \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

   Vector $\boldsymbol{w}$ is a vector with the first entry is the norm of vector $\boldsymbol{x}$ with sign either plus or minus, and the rest entries are zero, the sign is depending on the first entry of vector $\boldsymbol{x}$, if the first entry of vector $\boldsymbol{x}$ is negative then the sign will be positive at vector $\boldsymbol{w}$, if the first entry of vector $\boldsymbol{x}$ is positive then the sign will be negative at vector $\boldsymbol{w}$.
4. Compute vector $\boldsymbol{v}$

   $$\boldsymbol{v} = \boldsymbol{w} - \boldsymbol{x}$$
5. Construct a Householder reflector

   $$\hat{H}_{j+1} = I_{n-row} - 2\frac{\boldsymbol{v}\boldsymbol{v}^T}{\boldsymbol{v}^T\boldsymbol{v}}$$

   thus

   $$H_{j+1} = \begin{bmatrix} 1 & 0 \\ 0 & \hat{H}_{j+1} \end{bmatrix}$$

   $row = row + 1$, $H_{j+1}AH_{j+1} \rightarrowtail A$, we will use this $H_{j+1}AH_{j+1}$ as A at the next iteration, repeat till the last iteration.

After finish we will have

$$P = H_1 H_2 \ldots H_n$$

$$P^T = H_n H_{n-1} \ldots H_1$$

i. Quadratic Forms

[SI*] Expressions of the form

$$a_1x_1 + a_2x_2 + \cdots + a_nx_n$$

occurred in our study of linear equations and linear systems. If $a_1, a_2, \ldots, a_n$ are treated as fixed constants, then this expression is a real-valued function of the n variables $x_1, x_2, \ldots, x_n$ and is called a linear form on $\mathbb{R}^n$.

All variables in a linear form occur to the first power and there are no products of variables.

[SI*] Quadratic forms on $\mathbb{R}^n$ are functions of the form

$$a_1x_1^2 + a_2x_2^2 + \cdots + a_nx_n^2 + \sum a_kx_ix_j$$

The notation of $\sum a_kx_ix_j$ represents all possible terms of $a_kx_ix_j$ in which $x_i \neq x_j$.

The terms of the form $a_kx_ix_j$ are called cross product terms. It is common to combine the cross product terms involving x_ix_j with those involving x_jx_i to avoid duplication.

[SI*] A general quadratic form on $\mathbb{R}^2$ would typically be expressed as

$$a_1x_1^2 + a_2x_2^2 + 2a_3x_1x_2 \tag{6.118}$$

in matrix form it can be expressed as

$$\begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} a_1 & a_3 \\ a_3 & a_2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \boldsymbol{x}^T A\boldsymbol{x}$$

and a general quadratic form on $\mathbb{R}^3$ can be expressed as

$$a_1x_1^2 + a_2x_2^2 + a_3x_3^2 + 2a_4x_1x_2 + 2a_5x_1x_3 + 2a_6x_2x_3 \tag{6.119}$$

or, in matrix form

$$\begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} \begin{bmatrix} a_1 & a_4 & a_5 \\ a_4 & a_2 & a_6 \\ a_5 & a_6 & a_3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \boldsymbol{x}^T A\boldsymbol{x}$$

The matrix A in these formulas is symmetric, that its diagonal entries are the coefficients of the squared terms, and its off-diagonal entries are half the coefficients of the cross product terms.

[SI*] In general, if A is a symmetric $n \times n$ matrix and $\boldsymbol{x}$ is an $n \times 1$ column vector of variables, then we call the function

$$Q_A(\boldsymbol{x}) = \boldsymbol{x}^T A\boldsymbol{x} \tag{6.120}$$

the quadratic form associated with A. It can be expressed in dot product notation as

$$\boldsymbol{x}^T A\boldsymbol{x} = \boldsymbol{x} \cdot A\boldsymbol{x} = A\boldsymbol{x} \cdot \boldsymbol{x} \tag{6.121}$$

In the case where A is a diagonal matrix, the quadratic form $\boldsymbol{x}^T A\boldsymbol{x}$ has no cross product terms; for example, if A has diagonal entries $\lambda_1, \lambda_2, \ldots, \lambda_n$, then

$$\boldsymbol{x}^T A\boldsymbol{x} = \begin{bmatrix} x_1 & x_2 & \ldots & x_n \end{bmatrix} \begin{bmatrix} \lambda_1 & 0 & \ldots & 0 \\ 0 & \lambda_2 & \ldots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \ldots & \lambda_n \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

$$\boldsymbol{x}^T A\boldsymbol{x} = \lambda_1x_1^2 + \lambda_2x_2^2 + \cdots + \lambda_nx_n^2$$

[SI*] There are three kinds of problems that occur in applications of quadratic forms:

1. If $\boldsymbol{x}^T A\boldsymbol{x}$ is a quadratic form on $\mathbb{R}^2$ or $\mathbb{R}^3$, what kind of curve or surface is represented by the equation $\boldsymbol{x}^T A\boldsymbol{x} = k$?
2. If $\boldsymbol{x}^T A\boldsymbol{x}$ is a quadratic form on $\mathbb{R}^n$, what conditions must A satisfy for $\boldsymbol{x}^T A\boldsymbol{x}$ to have positive values for $\boldsymbol{x} \neq \mathbf{0}$?
3. If $\boldsymbol{x}^T A\boldsymbol{x}$ is a quadratic form on $\mathbb{R}^n$, what are its maximum and minimum values if $\boldsymbol{x}$ is constrained to satisfy $||\boldsymbol{x}|| = 1$?

[SI*] Many of the techniques for solving the problems above are based on simplifying the quadratic form $\boldsymbol{x}^T A\boldsymbol{x}$ by making a substitution

$$\boldsymbol{x} = P\boldsymbol{y} \tag{6.122}$$

that expresses the variables $x_1, x_2, \ldots, x_n$ in terms of new variables $y_1, y_2, \ldots, y_n$. If P is invertible, then we call (6.122) a change of variable, and if P is orthogonal, then we call (6.122) an orthogonal change of variable.

If we change the variable $\boldsymbol{x} = P\boldsymbol{y}$ in the quadratic form $\boldsymbol{x}^T A\boldsymbol{x}$, then we obtain

$$\boldsymbol{x}^T A\boldsymbol{x} = (P\boldsymbol{y})^T A(P\boldsymbol{y}) = \boldsymbol{y}^T P^T AP\boldsymbol{y} = \boldsymbol{y}^T (P^T AP)\boldsymbol{y} \tag{6.123}$$

Since the matrix $B = P^T AP$ is symmetric, the effect of the change of variable is to produce a new quadratic form $\boldsymbol{y}^T B\boldsymbol{y}$ in the variables $y_1, y_2, \ldots, y_n$.

In particular, if we choose P to orthogonally diagonalize A, then the new quadratic form will be $\boldsymbol{y}^T D\boldsymbol{y}$, where D is a diagonal matrix with the eigenvalues of A on the main diagonal; that is,

$$\boldsymbol{x}^T A\boldsymbol{x} = \boldsymbol{y}^T D\boldsymbol{y} = \begin{bmatrix} y_1 & y_2 & \ldots & y_n \end{bmatrix} \begin{bmatrix} \lambda_1 & 0 & \ldots & 0 \\ 0 & \lambda_2 & \ldots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \ldots & \lambda_n \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

$$\boldsymbol{x}^T A\boldsymbol{x} = \boldsymbol{y}^T D\boldsymbol{y} = \lambda_1 y_1^2 + \lambda_2 y_2^2 + \cdots + \lambda_n y_n^2$$

Theorem 6.87: The Principal Axes Theorem

If A is a symmetric $n \times n$ matrix, then there is an orthogonal change of variable that transforms the quadratic form $\boldsymbol{x}^T A\boldsymbol{x}$ into a quadratic form $\boldsymbol{y}^T D\boldsymbol{y}$ with no cross product terms. Specifically, if P orthogonally diagonalizes A, then making the change of variable $\boldsymbol{x} = P\boldsymbol{y}$ in the quadratic form $\boldsymbol{x}^T A\boldsymbol{x}$ yields the quadratic form

$$\boldsymbol{x}^T A\boldsymbol{x} = \boldsymbol{y}^T D\boldsymbol{y} = \lambda_1 y_1^2 + \lambda_2 y_2^2 + \cdots + \lambda_n y_n^2 \tag{6.124}$$

in which $\lambda_1, \lambda_2, \ldots, \lambda_n$ are the eigenvalues of A corresponding to the eigenvectors that form the successive columns of P.

[SI*] If A is a symmetric $n \times n$ matrix, then the quadratic form $\boldsymbol{x}^T A\boldsymbol{x}$ is a real-valued function whose range is the set of all possible values for $\boldsymbol{x}^T A\boldsymbol{x}$ as $\boldsymbol{x}$ varies over $\mathbb{R}^n$.

It can be shown that an orthogonal change of variable $\boldsymbol{x} = P\boldsymbol{y}$ does not alter the range of a quadratic form; that is, the set of all values for $\boldsymbol{x}^T A\boldsymbol{x}$ as $\boldsymbol{x}$ varies over $\mathbb{R}^n$ is the same as the set of all values for $\boldsymbol{y}^T (P^T AP)\boldsymbol{y}$ as $\boldsymbol{y}$ varies over $\mathbb{R}^n$.

[SI*] Recall that a conic section or conic is a curve that results by cutting a double-napped cone with a plane. The most important conic sections are ellipses, hyperbolas, and parabolas, which result when the cutting plane does not pass through the vertex.

Circles are special cases of ellipses that results when the cutting plane is perpendicular to the axis of symmetry of the cone. If the cutting plane passes through the vertex, then the resulting intersection is called a degenerate conic. The possibilities are a point, a pair of intersecting lines or a single line.

[SI*] Quadratic forms in $\mathbb{R}^2$ arise naturally in the study of conic sections. For example it is shown in analytic geometry that an equation of the form

$$ax^2 + 2bxy + cy^2 + dx + ey + f = 0 \tag{6.125}$$

in which a, b, and c are not all zero, represents a conic section. If $d = e = 0$ in (6.125) then there are no linear terms, so the equation becomes

$$ax^2 + 2bxy + cy^2 + f = 0 \tag{6.126}$$

and is said to represent central conic. These include circles, ellipses, and hyperbolas, but not parabolas. Furthermore, if $b =$ in, then there is no cross product term, and the equation

$$ax^2 + cy^2 + f = 0 \tag{6.127}$$

is said to represent a central conic in standard position.

If we take the constant f in Equations (6.126) and (6.127) to the right side and let $k = -f$, then we can rewrite these equations in matrix form as

$$\begin{bmatrix} x & y \end{bmatrix} \begin{bmatrix} a & b \\ b & c \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = k \tag{6.128}$$

this one corresponds to Equation (6.126) in which there is a cross product term $2bxy$.

$$\begin{bmatrix} x & y \end{bmatrix} \begin{bmatrix} a & 0 \\ 0 & c \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = k \tag{6.129}$$

This one corresponds to Equation (6.127) in which there is no cross product term. Geometrically, the existence of a cross product term signals that the graph of the quadratic form is rotated about the origin.

The three-dimensional analogs of the Equation (6.128) is

$$\begin{bmatrix} x & y & z \end{bmatrix} \begin{bmatrix} a & d & e \\ d & b & f \\ e & f & c \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = k \tag{6.130}$$

The three-dimensional analogs of the Equation (6.129) is

$$\begin{bmatrix} x & y & z \end{bmatrix} \begin{bmatrix} a & 0 & 0 \\ 0 & b & 0 \\ 0 & 0 & c \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = k \tag{6.131}$$

If a, b, and c are not all zero, then the graphs of these equations in $\mathbb{R}^3$ are called central quadrics in standard position.

[SI*] An equation of the form

$$ax^2 + 2bxy + cy^2 + f = 0 \tag{6.132}$$

represents a central conic. If $b = 0$, then the conic is in standard position, and if $b \neq 0$, it is rotated. It is an easy matter to identify central conics in standard position by matching the equation with one of the standard forms:

Ellipse

$$\frac{x^2}{\alpha^2} + \frac{y^2}{\beta^2} = 1 \tag{6.133}$$

with $\alpha > 0, \beta > 0$.

Hyperbola type 1

$$\frac{x^2}{\alpha^2} - \frac{y^2}{\beta^2} = 1 \tag{6.134}$$

with $\alpha > 0, \beta > 0$.

Hyperbola type 2

$$\frac{y^2}{\beta^2} - \frac{x^2}{\alpha^2} = 1 \tag{6.135}$$

with $\alpha > 0, \beta > 0$.

[SI*] The equation

$$9x^2 + 16y^2 - 144 = 0$$

can be rewritten as

$$\frac{x^2}{16} + \frac{y^2}{9} = 1$$

which is the ellipse with $\alpha = 4, \beta = 3$.

[SI*] If a central conic is rotated out of standard position, then it can be identified by first rotating the coordinate axes to put it in standard position and then matching the resulting equation with one of standard forms.

To find a rotation that eliminates the cross product term in the equation

$$ax^2 + 2bxy + cy^2 = k \tag{6.136}$$

it will be convenient to express the equation in the matrix form

$$\boldsymbol{x}^T A \boldsymbol{x} = \begin{bmatrix} x & y \end{bmatrix} \begin{bmatrix} a & b \\ b & c \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = k \tag{6.137}$$

and look for a change of variable

$$\boldsymbol{x} = P\boldsymbol{x}'$$

that diagonalizes A and for which $\det(P) = 1$. The transition matrix

$$P = \begin{bmatrix} \cos\ \theta & -\sin\ \theta \\ \sin\ \theta & \cos\ \theta \end{bmatrix} \tag{6.138}$$

has the effect of rotating the xy-axes of a rectangular coordinate system through an angle θ, our problem reduces to finding θ that diagonalizes A, thereby eliminating the cross product

term in Equation (6.136). If we make this change of variable, then in the $x'y'$-coordinate system. Equation (6.137) will become

$$\boldsymbol{x}^T A\boldsymbol{x} = (P\boldsymbol{x}')^T AP\boldsymbol{x}' = \boldsymbol{x}'^T P^T AP\boldsymbol{x}' = \boldsymbol{x}'^T D\boldsymbol{x}'$$

$$\boldsymbol{x}'^T D\boldsymbol{x}' = \begin{bmatrix} x' & y' \end{bmatrix} \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} \begin{bmatrix} x' \\ y' \end{bmatrix} = k \tag{6.139}$$

where λ_1 and λ_2 are the eigenvalues of A. The conic can now be identified by writing Equation (6.139) in the form

$$\lambda_1 x'^2 + \lambda_2 y'^2 = k \tag{6.140}$$

and performing the necessary algebra to match it with one of the standard forms.

For example, if λ_1, λ_2, and k are positive, then Equation (6.140) represents an ellipse with an axis of length $2\sqrt{\frac{k}{\lambda_1}}$ in the x'-direction and $2\sqrt{\frac{k}{\lambda_2}}$ in the y'-direction.

The first column vector of P, which is a unit eigenvector corresponding to λ_1, is along the positive x'-axis; and the second column vector of P, which is a unit eigenvector corresponding to λ_2, is a unit vector along the y'-axis. These are called the principal axes of the ellipse.

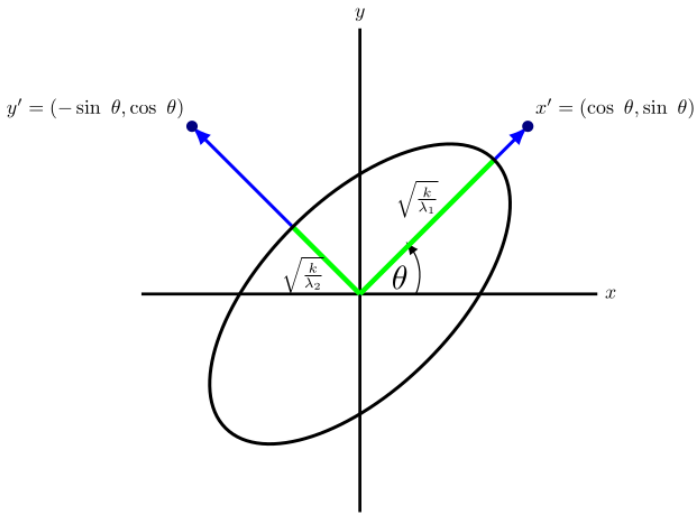

Figure 6.53: *The principal axes of the ellipse, the conic above is of form $\lambda_1 x'^2 + \lambda_2 y'^2 = k$, if λ_1, λ_2, and k are positive, then it is an ellipse with $\alpha = \sqrt{\frac{k}{\lambda_1}}$ and $\beta = \sqrt{\frac{k}{\lambda_2}}$.*

[SI*] If we have a conic with equation

$$ax^2 + 2bxy + cy^2 = k$$

with $b \neq 0$, the cross product term from the equation above ($2bxy$) can be eliminated by a rotation through an angle θ that satisfies

$$\cot\ 2\theta = \frac{a - c}{2b} \tag{6.141}$$

Definition 6.44: Terminology in Quadratic Form

A quadratic form $\boldsymbol{x}^T A\boldsymbol{x}$ is said to be

- **Positive definite** if $\boldsymbol{x}^T A\boldsymbol{x} > 0$ for $\boldsymbol{x} \neq 0$.
- **Negative definite** if $\boldsymbol{x}^T A\boldsymbol{x} < 0$ for $\boldsymbol{x} \neq 0$.
- **Indefinite** if $\boldsymbol{x}^T A\boldsymbol{x}$has both positive and negative values.

Theorem 6.88: Terminology for Known Symmetric Matrix and Eigenvalues

If A is a symmetric matrix, then:

(a) $\boldsymbol{x}^T A\boldsymbol{x}$ is positive definite if and only if all eigenvalues of A are positive.

(b) $\boldsymbol{x}^T A\boldsymbol{x}$ is negative definite if and only if all eigenvalues of A are negative.

(c) $\boldsymbol{x}^T A\boldsymbol{x}$ is indefinite if and only if A has at least one positive eigenvalue and at least one negative eigenvalue.

[SI*] A quadratic form for which $\boldsymbol{x}^T A\boldsymbol{x} \geq 0$ if $\boldsymbol{x} \neq \mathbf{0}$ is called positive semidefinite, and one for which $\boldsymbol{x}^T A\boldsymbol{x} \leq 0$ if $\boldsymbol{x} \neq \mathbf{0}$ is called negative semidefinite.

$\boldsymbol{x}^T A\boldsymbol{x}$ is positive semidefinite if and only if all eigenvalues of A are nonnegative and is negative semidefinite if and only if all eigenvalues of A are nonpositive.

[SI*] **Example:**
It is not usually possible to tell from the signs of the entries in a symmetric matrix A whether that matrix is positive definite, negative definite, or indefinite. For example, the entries of the matrix

$$A = \begin{bmatrix} 3 & 1 & 1 \\ 1 & 0 & 2 \\ 1 & 2 & 0 \end{bmatrix}$$

are nonnegative, but the matrix is indefinite since its eigenvalues are

$$\begin{aligned} \lambda_1 &= 1 \\ \lambda_2 &= 4 \\ \lambda_3 &= -2 \end{aligned}$$

If we write the quadratic form, it will be

$$\boldsymbol{x}^T A\boldsymbol{x} = \begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} \begin{bmatrix} 3 & 1 & 1 \\ 1 & 0 & 2 \\ 1 & 2 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = 3x_1^2 + 2x_1x_2 + 2x_1x_3 + 4x_2x_3$$

now, if we use different input for $\boldsymbol{x}$

$$\boldsymbol{x}^T A\boldsymbol{x} = 4$$

for $x_1 = 0, x_2 = 1$, and $x_3 = 1$.

$$\boldsymbol{x}^T A\boldsymbol{x} = -4$$

for $x_1 = 0, x_2 = 1$, and $x_3 = -1$. Thus the quadratic form is indefinite.

[SI*] If $\boldsymbol{x}^T B\boldsymbol{x} = k$ is the equation of a conic, and if $k \neq 0$, then we can divide through by k and rewrite the equation in the form

$$\boldsymbol{x}^T A\boldsymbol{x} = 1 \tag{6.142}$$

where $A = (1/k)B$. If we now rotate the coordinate axes to eliminate the cross product term (if any) in this equation, then the equation of the conic in the new coordinate system will be of the form

$$\lambda_1 x'^2 + \lambda_2 y'^2 = 1 \tag{6.143}$$

in which λ_1 and λ_2 are the eigenvalues of A. The particular type of conic represented by this equation will depend on the signs of the eigenvalues λ_1 and λ_2

- $\boldsymbol{x}^T A\boldsymbol{x} = 1$ represents an ellipse if $\lambda_1 > 0$ and $\lambda_2 > 0$.
- $\boldsymbol{x}^T A\boldsymbol{x} = 1$ has no graph if $\lambda_1 < 0$ and $\lambda_2 < 0$.
- $\boldsymbol{x}^T A\boldsymbol{x} = 1$ represents a hyperbola if λ_1 and λ_2 have opposite signs.

In the case of the ellipse, Equation (6.143) can be rewritten as

$$\frac{x'^2}{\left(\frac{1}{\sqrt{\lambda_1}}\right)^2} + \frac{y'^2}{\left(\frac{1}{\sqrt{\lambda_2}}\right)^2} = 1 \tag{6.144}$$

so the axes of the ellipse have lengths $2/\sqrt{\lambda_1}$ and $2/\sqrt{\lambda_2}$.

Theorem 6.89: Classifying Conic Sections

If A is a symmetric 2×2 matrix, then:

(a) $\boldsymbol{x}^T A\boldsymbol{x} = 1$ represents an ellipse if A is positive definite.

(b) $\boldsymbol{x}^T A\boldsymbol{x} = 1$ has no graph if A is negative definite.

(c) $\boldsymbol{x}^T A\boldsymbol{x} = 1$ represents an hyperbola if A is indefinite.

[SI*] Positive definite matrices are the most important symmetric matrices in applications, so it will be useful to learn a little more about them. We already know that a symmetric matrix is positive definite if and only if its eigenvalues are all positive.

[SI*] We define the k-th principal submatrix of an $n \times n$ matrix A to be the $k \times k$ submatrix consisting of the first k rows and columns of A.

Theorem 6.90: Determinant Test for Symmetric Matrix

A symmetric matrix A is positive definite if and only if the determinant of every principal submatrix is positive.

ii. Hermitian, Unitary, and Normal Matrices

[SI*] The transpose operation is less important for complex matrices than for real matrices.

Definition 6.45: Conjugate Transpose

If A is a complex matrix, then the conjugate transpose of A, denoted by A^*, is defined by

$$A^* = \overline{A}^T \tag{6.145}$$

[SI*] The order in which the transpose and conjugation operations are performed in computing $A^* = \overline{A}^T$ does not matter

$$(\overline{A^T}) = (\overline{A})^T$$

[SI*] In the case where A has real entries we have

$$A^* = (\overline{A})^T = A^T$$

so A^* is the same as A^T for real matrices.

[SI*] **Example:**
Find the conjugate transpose A^* of the matrix

$$A = \begin{bmatrix} 2+i & -3i & 0 \\ 3 & 1-5i & i \end{bmatrix}$$

Solution:
We have

$$\overline{A} = \begin{bmatrix} 2-i & 3i & 0 \\ 3 & 1+5i & -i \end{bmatrix}$$

and hence

$$A^* = \overline{A}^T = \begin{bmatrix} 2-i & 3 \\ 3i & 1+5i \\ 0 & -i \end{bmatrix}$$

Theorem 6.91: Algebraic Properties of the Conjugate Transpose Operation

If k is a complex scalar, and if A, B, and C are complex matrices whose sizes are such that the stated operations can be performed, then:

(a) $(A^*)^* = A$

(b) $(A+B)^* = A^* + B^*$

(c) $(A-B)^* = A^* - B^*$

(d) $(kA)^* = \overline{k}A^*$

(e) $(AB)^* = B^*A^*$

[SI*] **Remark**
Note that the relationship

$$\boldsymbol{u} \cdot \boldsymbol{v} = \overline{\boldsymbol{v}}^T \boldsymbol{u}$$

can be expressed in terms of the conjugate transpose as

$$\boldsymbol{u} \cdot \boldsymbol{v} = \boldsymbol{v}^* \boldsymbol{u} \tag{6.146}$$

Definition 6.46: Hermitian and Unitary Matrices

A square complex matrix A is said to be unitary if

$$A^{-1} = A^* \tag{6.147}$$

and is said to be Hermitian if

$$A^* = A \tag{6.148}$$

[SI*] If A is a real matrix, then

$$A^* = A^T$$

in which case, the unitary matrix formula for real matrix becomes

$$A^{-1} = A^T$$

and the Hermitian matrix formula for real matrix becomes

$$A^T = A$$

The unitary matrices are complex generalizations of the real orthogonal matrices and Hermitian matrices are complex generalizations of the real symmetric matrices.

[SI*] Hermitian matrices are easy to recognize because their diagonal entries are real, and the entries that are symmetrically positioned across the main diagonal are complex conjugates. For example,

$$A = \begin{bmatrix} 2 & 3i & 2+6i \\ -3i & -6 & 5-7i \\ 2-6i & 5+7i & 1 \end{bmatrix}$$

is Hermitian.

Theorem 6.92: Eigenvalues for Hermitian Matrix

The eigenvalues of a Hermitian matrix are real numbers.

Theorem 6.93: Eigenvectors for Hermitian Matrix

If A is a Hermitian matrix, then eigenvectors from different eigenspaces are orthogonal.

Theorem 6.94: Equivalent Statements for Unitary Matrix

If A is an $n \times n$ matrix with complex entries, then the following are equivalent.

(a) A is unitary.

(b) $||A\boldsymbol{x}|| = ||\boldsymbol{x}||$ for all $\boldsymbol{x}$ in $\mathbb{C}^n$.

(c) $A\boldsymbol{x} \cdot A\boldsymbol{y} = \boldsymbol{x} \cdot \boldsymbol{y}$ for all $\boldsymbol{x}$ and $\boldsymbol{y}$ in $\mathbb{C}^n$.

(d) The column vectors of A form an orthonormal set in $\mathbb{C}^n$ with respect to the complex Euclidean inner product.

(e) The row vectors of A form an orthonormal set in $\mathbb{C}^n$ with respect to the complex Euclidean inner product.

[SI*] Unitary matrices are the complex analogs of the real orthogonal matrices.

Definition 6.47: Unitary Diagonalizability

A square complex matrix is said to be unitarily diagonalizable if there is a unitary matrix P such that $P^*AP = D$ is a complex diagonal matrix. Any such matrix P is said to unitarily diagonalize A.

Theorem 6.95: Unitary Diagonalizability for Hermitian Matrix

Every $n \times n$ Hermitian matrix A has an orthonormal set of n eigenvectors and is unitarily diagonalized by any $n \times n$ matrix P whose column vectors form an orthonormal set of eigenvectors of A.

[SI*] The procedure for unitarily diagonalizing Hermitian matrix A is exactly the same as that for orthogonally diagonalizing a symmetric matrix:

1. Find a basis for each eigenspace of A.
2. Apply the Gram-Schmidt process to each of these bases to obtain orthonormal bases for the eigenspaces.
3. Form the matrix P whose column vectors are the basis vectors obtained in Step 2. This will be a unitary matrix and will unitarily diagonalize A.

[SI*] We defined a square matrix with real entries to be skew-symmetric if

$$A^T = -A$$

A skew-symmetric matrix must have zeros on the main diagonal, and each entry off the main diagonal must be the negative of its mirror image about the main diagonal. Here is an example.

$$A = \begin{bmatrix} 0 & 3 & -5 \\ -3 & 0 & 6 \\ 5 & -6 & 0 \end{bmatrix}$$

[SI*] The complex analogs of the skew-symmetric matrices are the matrices for which

$$A^* = -A$$

Such matrices are said to be skew-Hermitian.

A skew-Hermitian matrix A has the property

$$A^* = \overline{A}^T = -A$$

it must be that A has zeros or pure imaginary numbers on the main diagonal, and that the complex conjugate of each entry off the main diagonal is the negative of the real number of its mirror image about the main diagonal. Here is an example.

$$A = \begin{bmatrix} i & 2-i & 6 \\ -2-1 & 3i & 4i \\ -6 & 4i & 0 \end{bmatrix}$$

[SI*] Hermitian matrices enjoy many, but not all, of the properties of real symmetric matrices. For example, we know that real symmetric matrices are orthogonally diagonalizable and Hermitian matrices are unitarily diagonalizable. However, whereas the real symmetric matrices are the only orthogonally diagonalizable matrices, the Hermitian matrices do not constitute the entire class of unitarily diagonalizable complex matrices; that is, there exist unitarily diagonalizable matrices that are not Hermitian.

Specifically, it can be proved that a square complex matrix A is unitarily diagonalizable if and only if

$$AA^* = A^*A \tag{6.149}$$

Matrices with this property are said to be normal. Normal matrices include the Hermitian, skew-Hermitian, and unitary matrices in the complex case and the symmetric, skew-symmetric, and orthogonal matrices in the real case. The nonzero skew-symmetric matrices are particularly interesting because they are examples of real matrices that are not orthogonally diagonalizable but are unitarily diagonalizable.

[SI*] We have seen that Hermitian matrices have real eigenvalues.

The eigenvalues of skew-Hermitian matrix are either zero or purely imaginary (have real part of zero) and that the eigenvalues of unitary matrices have modulus 1, $|\lambda| = 1$.

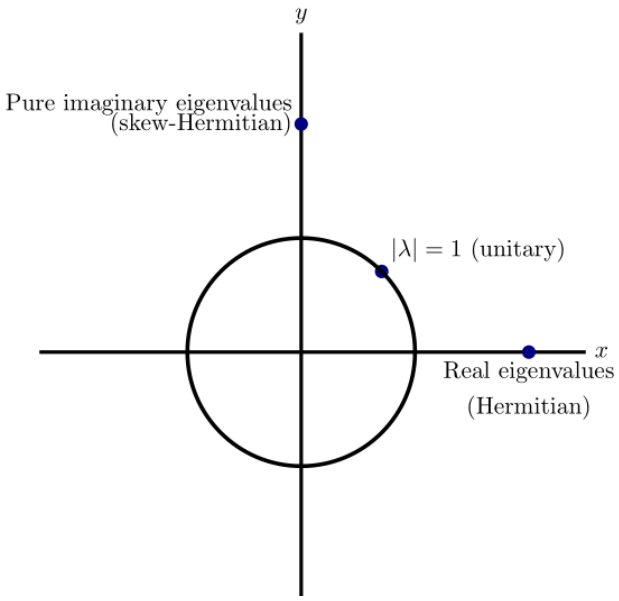

Figure 6.54

iii. Functions in SymIntegration Related to Diagonalization and Quadratic Forms

[SI*] We will list the basic functions that can be used related to diagonalization and quadratic forms:

quadraticmultiplication(vector<vector<double>> &matrixA, vector<double> &vectorx)
to compute the quadratic form of $\boldsymbol{x}^T A\boldsymbol{x}$.

complex<double> complexquadraticmultiplication(ComplexMatrix &A, ComplexVector &x)
to compute the complex quadratic multiplication ($\overline{\boldsymbol{x}}^T A\boldsymbol{x}$) of a square complex matrix A with size $n \times n$ and vector $\boldsymbol{x}$ with size n.
Remember that in real space $\mathbb{R}^n$ the quadratic form is

$$\boldsymbol{x}^T A\boldsymbol{x} = \boldsymbol{x} \cdot A\boldsymbol{x} = A\boldsymbol{x} \cdot \boldsymbol{x}$$

the analogous quadratic form in complex space $\mathbb{C}^n$ is

$$\boldsymbol{x} \cdot A\boldsymbol{x} = A\boldsymbol{x} \cdot \boldsymbol{x} = \overline{\boldsymbol{x}}^T A\boldsymbol{x}$$

SpectralDecomposition(vector<vector<complex<double>>> &A, double a, double b)
to compute the spectral decomposition of matrix A, with a as the lower bound for uniform distribution, b as the upper bound for the uniform distribution, we are using uniform distribution to generate random μ and σ as the input to generate the initial complex vector $\boldsymbol{v}^0$ that is normally distributed with mean μ and standard deviation σ, this initial vector is a prerequisite for the Rayleigh quotient iteration. The key here is to choose a and b carefully so we can pinpoint the range where the eigenvalue could be located.

iv. Compute Spectral Decomposition with SymIntegration

Compute the spectral decomposition of the matrix

$$A = \begin{bmatrix} 1 & 2 \\ 2 & -2 \end{bmatrix}$$

Solution:

The matrix A has eigenvalues $\lambda_1 = -3$ and $\lambda_2 = 2$ with corresponding eigenvectors that already normalized (orthonormal)

$$\boldsymbol{u}_1 = \frac{\boldsymbol{x}_1}{||\boldsymbol{x}_1||} = \begin{bmatrix} \frac{1}{\sqrt{5}} \\ -\frac{2}{\sqrt{5}} \end{bmatrix}, \quad \boldsymbol{u}_2 = \frac{\boldsymbol{x}_2}{||\boldsymbol{x}_2||} = \begin{bmatrix} \frac{2}{\sqrt{5}} \\ \frac{1}{\sqrt{5}} \end{bmatrix}$$

so a spectral decomposition of A is

$$A = \lambda_1 \boldsymbol{u}_1 \boldsymbol{u}_1^T + \lambda_2 \boldsymbol{u}_2 \boldsymbol{u}_2^T$$

$$A = \begin{bmatrix} 1 & 2 \\ 2 & -2 \end{bmatrix} = (-3) \begin{bmatrix} \frac{1}{5} & -\frac{2}{5} \\ -\frac{2}{5} & \frac{4}{5} \end{bmatrix} + (2) \begin{bmatrix} \frac{4}{5} & \frac{2}{5} \\ \frac{2}{5} & \frac{1}{5} \end{bmatrix}$$

where the 2×2 matrices on the right side are the standard matrices for the orthogonal projections onto the eigenspaces corresponding to $\lambda_1 = -3$ and $\lambda_2 = 2$, respectively.

In SymIntegration, to compute spectral decomposition of a matrix we can use this function: **SpectralDecomposition(vector<vector<complex<double>>> &A, double a, double b)** with A as the matrix, a as the lower bound for uniform distribution, b as the upper bound for the uniform distribution, we are using uniform distribution to generate random μ and σ as the input to generate the initial complex vector v^0 that is normally distributed with mean μ and standard deviation σ, this initial vector is a prerequisite for the Rayleigh quotient iteration. The key here is to choose a and b carefully so we can pinpoint the range where the eigenvalue could be located.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Spectral Decomposition ]# ./main

Matrix A :
       (1,0)        (2,0)
       (2,0)       (-2,0)

Eigenvalues :
(-3,0)
(2,0)

Eigenvectors (represented by column):
  (-0.447,0)    (0.894,0)
   (0.894,0)    (0.447,0)

Spectral decomposition for λ_{1} = (-3,0) *
     (0.2,0)     (-0.4,0)
    (-0.4,0)      (0.8,0)

Spectral decomposition for λ_{2} = (2,0) *
     (0.8,0)      (0.4,0)
     (0.4,0)      (0.2,0)

Time taken by function: 10308 microseconds
```

Figure 6.55: *The spectral decomposition of matrix A. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Spectral Decomposition/main.cpp**).*

v. Compute Hessenberg Decomposition with SymIntegration

Compute the Hessenberg decomposition of the matrix

$$A = \begin{bmatrix} 1 & 0 & 2 & 3 \\ -1 & 0 & 5 & 2 \\ 2 & -2 & 0 & 0 \\ 2 & -1 & 2 & 0 \end{bmatrix}$$

Solution:

To compute the upper Hessenberg form H ($H_{ij} = 0$ for $i > j+1$) for Hessenberg decomposition with Householder reflections, you iteratively apply similarity transformations using Householder reflectors to zero out entries below the subdiagonal, preserving eigenvalues. You construct these reflectors (H_k) by reflecting specific vectors (e.g. the first column's tail) towards a desired form, forming

$$A_k = H_k A_{k-1} H_k$$

until the matrix becomes upper Hessenberg (all entries below subdiagonal are zero), with the final

$$H = H_1 H_2 \dots H_{n-2}$$

and

$$H = P^T A P$$

First, at iteration $j = 1$ we will have

$$\boldsymbol{x} = \begin{bmatrix} -1 \\ 2 \\ 2 \end{bmatrix}$$

(we define vector $\boldsymbol{x}$ as the part of column j below the diagonal $A(j+1:n, j)$).

For the vector $\boldsymbol{w}$, the choice of the sign for the first entry is the opposite of the first element of vector $\boldsymbol{x}$.

$$\boldsymbol{w} = \begin{bmatrix} \pm||\boldsymbol{x}|| \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 3 \\ 0 \\ 0 \end{bmatrix}$$

Construct the householder vector $\boldsymbol{v}$

$$\boldsymbol{v} = \boldsymbol{w} - \boldsymbol{x} = \begin{bmatrix} 4 \\ -2 \\ -2 \end{bmatrix}$$

Construct the Householder reflector $\hat{H}_j$

$$\hat{H}_1 = I - 2\frac{\boldsymbol{v}\boldsymbol{v}^T}{\boldsymbol{v}^T\boldsymbol{v}} = \begin{bmatrix} -\frac{1}{3} & \frac{2}{3} & \frac{2}{3} \\ \frac{2}{3} & \frac{2}{3} & -\frac{1}{3} \\ \frac{2}{3} & -\frac{1}{3} & \frac{2}{3} \end{bmatrix}$$

Create $n \times n$ Householder matrix H_j by embedding $\hat{H}_j$ into the lower-right $(n-j) \times (n-j)$ block, with ones on the diagonal and zeros elsewhere in the top-left $j \times j$ block.

$$H_1 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -\frac{1}{3} & \frac{2}{3} & \frac{2}{3} \\ 0 & \frac{2}{3} & \frac{2}{3} & -\frac{1}{3} \\ 0 & \frac{2}{3} & -\frac{1}{3} & \frac{2}{3} \end{bmatrix}$$

Accumulate

$$P = PH_j$$

starts with $P = I$.

Thus

$$H_1 A H_1 = \begin{bmatrix} 1 & \frac{10}{3} & \frac{1}{3} & \frac{4}{3} \\ 3 & 0 & -\frac{4}{3} & -\frac{5}{3} \\ 0 & 3 & \frac{2}{3} & -\frac{2}{3} \\ 0 & 4 & \frac{8}{3} & -\frac{2}{3} \end{bmatrix}$$

Do the transformation at the end of each iteration:

$$A_{new} = H_j A_{old} H_j$$

Repeat the iteration till $j = n - 2$, after $n - 2$ steps the matrix A will be in upper Hessenberg form, and P will hold the accumulated orthogonal transformations.

HessenbergDecomposition(vector<vector<complex<double>>> &A)

with A as the square matrix that we want to compute the upper Hessenberg decomposition of A.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Hessenberg Decomposition ]# ./main

Matrix A :
       (1,0)        (0,0)        (2,0)        (3,0)
      (-1,0)        (0,0)        (5,0)        (2,0)
       (2,0)       (-2,0)        (0,0)        (0,0)
       (2,0)       (-1,0)        (2,0)        (0,0)

x:
(-1,0)
(2,0)
(2,0)

w:
(3,0)
(0,0)
(0,0)

v:
(4,0)
(-2,0)
(-2,0)

vTv:
      (24,0)

vvT:
      (16,0)       (-8,0)       (-8,0)
      (-8,0)        (4,0)        (4,0)
      (-8,0)        (4,0)        (4,0)

P:
    (1.33,0)   (-0.667,0)   (-0.667,0)
  (-0.667,0)    (0.333,0)    (0.333,0)
  (-0.667,0)    (0.333,0)    (0.333,0)

I - 2P:
  (-0.333,0)    (0.667,0)    (0.667,0)
   (0.667,0)    (0.667,0)   (-0.333,0)
   (0.667,0)   (-0.333,0)    (0.667,0)

H_{i}:
       (1,0)        (0,0)        (0,0)        (0,0)
       (0,0)   (-0.333,0)    (0.667,0)    (0.667,0)
       (0,0)    (0.667,0)    (0.667,0)   (-0.333,0)
       (0,0)    (0.667,0)   (-0.333,0)    (0.667,0)
```

Figure 6.56: *The Hessenberg decomposition of matrix A. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Hessenberg Decomposition/main.cpp**).*

```
H_{i}:
       (1,0)        (0,0)        (0,0)        (0,0)
       (0,0)   (-0.333,0)    (0.667,0)    (0.667,0)
       (0,0)    (0.667,0)    (0.667,0)   (-0.333,0)
       (0,0)    (0.667,0)   (-0.333,0)    (0.667,0)

H_{i} A H_{i}:
       (1,0)     (3.33,0)    (0.333,0)     (1.33,0)
       (3,0)        (0,0)    (-1.33,0)    (-1.67,0)
       (0,0)        (3,0)    (0.667,0)   (-0.667,0)
       (0,0)        (4,0)     (2.67,0)   (-0.667,0)

Householder Matrix:
       (1,0)        (0,0)        (0,0)        (0,0)
       (0,0)   (-0.333,0)    (0.667,0)    (0.667,0)
       (0,0)    (0.667,0)    (0.667,0)   (-0.333,0)
       (0,0)    (0.667,0)   (-0.333,0)    (0.667,0)

x:
(3,0)
(4,0)

w:
(-5,0)
(0,0)

v:
(-8,0)
(-4,0)

vTv:
      (80,0)

vvT:
      (64,0)       (32,0)
      (32,0)       (16,0)

P:
     (1.6,0)      (0.8,0)
     (0.8,0)      (0.4,0)

I - 2P:
    (-0.6,0)     (-0.8,0)
    (-0.8,0)      (0.6,0)

H_{i}:
       (1,0)        (0,0)        (0,0)        (0,0)
       (0,0)        (1,0)        (0,0)        (0,0)
       (0,0)        (0,0)     (-0.6,0)     (-0.8,0)
       (0,0)        (0,0)     (-0.8,0)      (0.6,0)
```

Figure 6.57: *The Hessenberg decomposition of matrix A. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Hessenberg Decomposition/main.cpp**).*

```
H_{i}:
       (1,0)        (0,0)        (0,0)        (0,0)
       (0,0)        (1,0)        (0,0)        (0,0)
       (0,0)        (0,0)     (-0.6,0)     (-0.8,0)
       (0,0)        (0,0)     (-0.8,0)      (0.6,0)

H_{i} A H_{i}:
       (1,0)     (3.33,0)    (-1.27,0)    (0.533,0)
       (3,0)        (0,0)     (2.13,0)   (0.0667,0)
       (0,0)       (-5,0)    (0.773,0)     (2.59,0)
       (0,0)        (0,0)   (-0.747,0)   (-0.773,0)

Householder Matrix:
       (1,0)        (0,0)        (0,0)        (0,0)
       (0,0)   (-0.333,0)   (-0.933,0)   (-0.133,0)
       (0,0)    (0.667,0)   (-0.133,0)   (-0.733,0)
       (0,0)    (0.667,0)   (-0.333,0)    (0.667,0)

********************************************************************************

End of iteration

********************************************************************************

P^{T} A P:
       (1,0)     (3.33,0)    (-1.27,0)    (0.533,0)
       (3,0)        (0,0)     (2.13,0)   (0.0667,0)
       (0,0)       (-5,0)    (0.773,0)     (2.59,0)
       (0,0)        (0,0)   (-0.747,0)   (-0.773,0)

Time taken by function: 5124 microseconds
```

Figure 6.58: *The Hessenberg decomposition of matrix A. (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Hessenberg Decomposition/main.cpp**).*

VIII. Linear Transformations

[SI*] A matrix transformation $T_A : \mathbb{R}^n \to \mathbb{R}^m$ is defined as a mapping of the form

$$T_A(\boldsymbol{x}) = A\boldsymbol{x}$$

in which A is an $m \times n$ matrix.

Definition 6.48: Linear Transformation

If $T : V \to W$ is a function from a vector space V to a vector space W, then T is called a linear transformation from V to W if the following two properties hold for all vectors $\boldsymbol{u}$ and $\boldsymbol{v}$ in V and for all scalars k:

(a) $T(k\boldsymbol{u}) = kT(\boldsymbol{u})$ (Homogeneity property)

(b) $T(\boldsymbol{u} + \boldsymbol{v}) = T(\boldsymbol{u}) + T(\boldsymbol{v})$ (Additivity property)

In the special case where $V = W$, the linear transformation T is called a linear operator on the vector space V.

[SI*] The homogeneity and additivity properties of a linear transformation $T : V \to W$ can be used in combination to show that if $\boldsymbol{v}_1$ and $\boldsymbol{v}_2$ are vectors in V and k_1 and k_2 are any scalars, then

$$T(k_1\boldsymbol{v}_1 + k_2\boldsymbol{v}_2) = k_1T(\boldsymbol{v}_1) + k_2T(\boldsymbol{v}_2)$$

More generally, if $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_r$ are vectors in V and $k_1, k_2, \ldots, k_r$ are any scalars, then

$$T(k_1\boldsymbol{v}_1 + k_2\boldsymbol{v}_2 + \cdots + k_r\boldsymbol{v}_r) = k_1T(\boldsymbol{v}_1) + k_2T(\boldsymbol{v}_2) + \cdots + k_rT(\boldsymbol{v}_r) \tag{6.150}$$

Theorem 6.96: Linear Transformation

If $T : V \to W$ is a linear transformation, then:

(a) $T(\boldsymbol{0}) = \boldsymbol{0}$

(b) $T(\boldsymbol{u} - \boldsymbol{v}) = T(\boldsymbol{u}) - T(\boldsymbol{v})$ for all $\boldsymbol{u}$ and $\boldsymbol{v}$ in V

[SI*] **Matrix Transformations**
We have based the definition of a general linear transformation on the homogeneity and additivity properties of matrix transformations, it follows that a matrix transformation $T_A : \mathbb{R}^n \to \mathbb{R}^m$ is also a linear transformation in this more general sense with $V = \mathbb{R}^n$ and $W = \mathbb{R}^m$.

[SI*] **The Zero Transformation**
Let V and W be any two vector spaces. The mapping $T : V \to W$ such that $T(\boldsymbol{v}) = \boldsymbol{0}$ for every $\boldsymbol{v}$ in V is a linear transformation called the zero transformation. To see that T is linear, observe that

$$\begin{aligned} T(\boldsymbol{u} + \boldsymbol{v}) &= \boldsymbol{0} \\ T(\boldsymbol{u}) &= \boldsymbol{0} \\ T(\boldsymbol{v}) &= \boldsymbol{0} \\ T(k\boldsymbol{u}) &= \boldsymbol{0} \end{aligned}$$

Therefore,

$$T(\boldsymbol{u}+\boldsymbol{v}) = T(\boldsymbol{u}) + T(\boldsymbol{v})$$
$$T(k\boldsymbol{u}) = kT(\boldsymbol{u})$$

[SI*] **The Identity Operator**
Let V be any vector space. The mapping $I : V \to V$ defined by $I(\boldsymbol{v}) = \boldsymbol{v}$ is called the identity operator on V. I is linear.

[SI*] **Dilation and Contraction Operators**
If V is a vector space and k is any scalar, then the mapping $T : V \to V$ given by $T(\boldsymbol{x}) = k\boldsymbol{x}$ is a linear operator on V, for if c is any scalar and if $\boldsymbol{u}$ and $\boldsymbol{v}$ are any vectors in V, then

$$T(c\boldsymbol{u}) = k(c\boldsymbol{u}) = c(k\boldsymbol{u}) = cT(\boldsymbol{u})$$
$$T(\boldsymbol{u}+\boldsymbol{v}) = k(\boldsymbol{u}+\boldsymbol{v}) = k\boldsymbol{u} + k\boldsymbol{v} = T(\boldsymbol{u}) + T(\boldsymbol{v})$$

If $0 < k < 1$, then T is called the contraction of V with factor k, and if $k > 1$, it is called the dilation of V with factor k.

[SI*] **A Linear Transformation from P_n to P_{n+1}**
Let $\boldsymbol{p} = p(x) = c_0 + c_1x + \cdots + c_nx^n$ be a polynomial in P_n, and define the transformation $T : P_n \to P_{n+1}$ by

$$T(\boldsymbol{p}) = T(p(x)) = xp(x) = c_0x + c_1x^2 + \cdots + c_nx^{n+1}$$

This transformation is linear because for any scalar k and any polynomials $\boldsymbol{p}_1$ and $\boldsymbol{p}_2$ in P_n we have

$$T(k\boldsymbol{p}) = T(kp(x)) = x(kp(x)) = k(xp(x)) = kT(\boldsymbol{p}))$$

and

$$\begin{aligned} T(\boldsymbol{p}_1 + \boldsymbol{p}_2) &= T(p_1(x) + p_2(x)) \\ &= x(p_1(x) + p_2(x)) \\ &= xp_1(x) + xp_2(x) \\ &= T(\boldsymbol{p}_1) + T(\boldsymbol{p}_2) \end{aligned}$$

[SI*] **A Linear Transformation Using an Inner Product**
Let V be an inner product space, let $\boldsymbol{v}_0$ be any fixed vector in V, and let $T : V \to R$ be the transformation

$$T(\boldsymbol{x}) = < \boldsymbol{x}, \boldsymbol{v}_0 >$$

that maps a vector $\boldsymbol{x}$ into its inner product with $\boldsymbol{v}_0$. This transformation is linear, for if k is any scalar, and if $\boldsymbol{u}$ and $\boldsymbol{v}$ are any vectors in V, then it follows from properties of inner products that

$$T(k\boldsymbol{u}) = < k\boldsymbol{u}, \boldsymbol{v}_0 > = k < \boldsymbol{u}, \boldsymbol{v}_0 > = kT(\boldsymbol{u})$$
$$T(\boldsymbol{u}+\boldsymbol{v}) = < \boldsymbol{u}+\boldsymbol{v}, \boldsymbol{v}_0 > = < \boldsymbol{u}, \boldsymbol{v}_0 > + < \boldsymbol{v}, \boldsymbol{v}_0 > = T(\boldsymbol{u}) + T(\boldsymbol{v})$$

[SI*] **Example: Transformations on Matrix Spaces**
Let M_{nn} be the vector space of $n \times n$ matrices. In each part determine whether the transformation is linear.

(a) $T_1(A) = A^T$

(b) $T_2(A) = \det(A)$

Solution:

(a) It follows that

$$T_1(kA) = (kA)^T = kA^T = kT_1A$$
$$T_1(A+B) = (A+B)^T = A^T + B^T = T_1(A) + T_1(B)$$

so T_1 is linear.

(b) It follows that

$$T_2(kA) = \det(kA) = k^n \det(A) = k^n T_2(A)$$

Thus, T_2 is not homogeneous and hence not linear if $n > 1$. Note that additivity also fails because we showed that $\det(A+B)$ and $\det(A) + \det(B)$ are not generally equal.

[SI*] **Translation Is Not Linear**

A Theorem states that a linear transformation maps $\mathbf{0}$ to $\mathbf{0}$. This property is useful for identifying transformations that are not linear. For example, if $\boldsymbol{x}_0$ is a fixed nonzero vector in $\mathbb{R}^2$, then the transformation

$$T(\boldsymbol{x}) = \boldsymbol{x} + \boldsymbol{x}_0$$

has the geometric effect of translating each point $\boldsymbol{x}$ in a direction parallel to $\boldsymbol{x}_0$ through a distance of $||\boldsymbol{x}_0||$. This cannot be a linear transformation since $T(\mathbf{0}) = \boldsymbol{x}_0$, so T does not map $\mathbf{0}$ to $\mathbf{0}$.

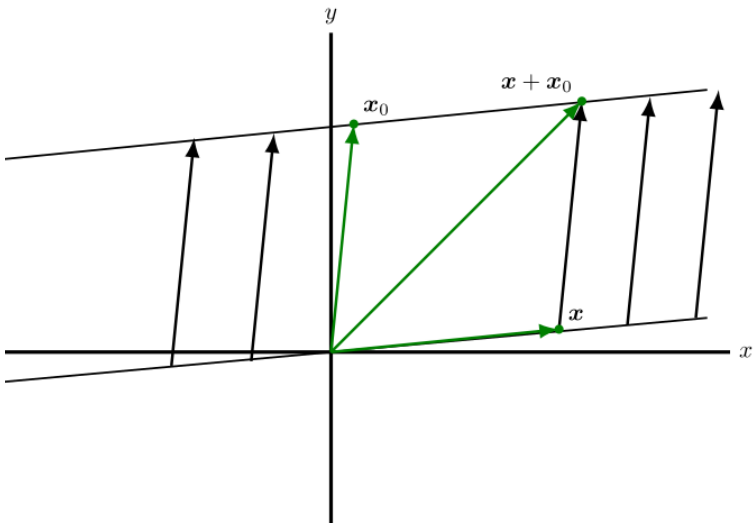

Figure 6.59: *$T(\boldsymbol{x}) = \boldsymbol{x} + \boldsymbol{x}_0$ translates each point $\boldsymbol{x}$ along a line parallel to $\boldsymbol{x}_0$ through a distance $||\boldsymbol{x}_0||$.*

[SI*] **The Evaluation Transformation**

Let V be a subspace of $F(-\infty, \infty)$, let

$$x_1, x_2, \ldots, x_n$$

be distinct real numbers, and let $T : V \to \mathbb{R}^n$ be the transformation

$$T(f) = (f(x_1), f(x_2), \ldots, f(x_n)) \tag{6.151}$$

that associates with f the n-tuple of function values at $x_1, x_2, \ldots, x_n$. We call this the evaluation transformation on V at $x_1, x_2, \ldots, x_n$. Thus, for example, if

$$x_1 = -1, \quad x_2 = 2, \quad x_3 = 4$$

and if $f(x) = x^2 - 1$, then

$$T(f) = (f(x_1), f(x_2), f(x_3)) = (0, 3, 15)$$

The evaluation transformation is linear, for if k is any scalar, and if f and g are any functions in V, then

$$\begin{aligned} T(kf) &= ((kf)(x_1), (kf)(x_2), \ldots, (kf)(x_n)) \\ &= (kf(x_1), kf(x_2), \ldots, kf(x_n)) \\ &= k(f(x_1), f(x_2), \ldots, f(x_n)) \\ &= kT(f) \end{aligned}$$

and

$$\begin{aligned} T(f+g) &= ((f+g)(x_1), (f+g)(x_2), \ldots, (f+g)(x_n)) \\ &= (f(x_1) + g(x_1), f(x_2) + g(x_2), \ldots, f(x_n) + g(x_n)) \\ &= (f(x_1), f(x_2), \ldots, f(x_n)) + (g(x_1), g(x_2), \ldots, g(x_n)) \\ &= T(f) + T(g) \end{aligned}$$

[SI*] If $\mathbb{R}^n \to \mathbb{R}^n$ is a matrix transformation, say multiplication by A, and if $\boldsymbol{e}_1, \boldsymbol{e}_2, \ldots, \boldsymbol{e}_n$ are the standard basis vectors for $\mathbb{R}^n$, then A can be expressed as

$$A = [T(\boldsymbol{e}_1) \mid T(\boldsymbol{e}_2) \mid \ldots \mid T(\boldsymbol{e}_n)]$$

It follows from this that the image of any vector $\boldsymbol{v} = (c_1, c_2, \ldots, c_n)$ in $\mathbb{R}^n$ under multiplication by A can be expressed as

$$T(\boldsymbol{v}) = c_1 T(\boldsymbol{e}_1) + c_2 T(\boldsymbol{e}_2) + \cdots + c_n T(\boldsymbol{e}_n)$$

This formula tells us that for a matrix transformation the image of any vector is expressible as a linear combination of the images of the standard basis vectors.

Theorem 6.97: Linear Transformations from Images of Basis Vectors

Let $T : V \to W$ be a linear transformation, where V is finite dimensional. If $S = \{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n\}$ is a basis for V, then the image of any vector $\boldsymbol{v}$ in V can be expressed as

$$T(\boldsymbol{v}) = c_1 T(\boldsymbol{v}_1) + c_2 T(\boldsymbol{v}_2) + \cdots + c_n T(\boldsymbol{v}_n) \tag{6.152}$$

where $c_1, c_2, \ldots, c_n$ are the coefficients required to express $\boldsymbol{v}$ as a linear combination of the vectors in S.

[SI*] **A Linear Transformation from $C^1(-\infty, \infty)$ to $F(-\infty, \infty)$**
Let $V = C^1(-\infty, \infty)$ be the vector space of functions with continuous first derivatives on $(-\infty, \infty)$, and let $W = F(-\infty, \infty)$ be the vector space of all real-valued functions defined

on $(-\infty, \infty)$. Let $D : V \to W$ be the transformation that maps a function $\boldsymbol{f} = f(x)$ into its derivative, that is

$$D(\boldsymbol{f}) = f'(x)$$

From the properties of differentiation, we have

$$D(\boldsymbol{f} + \boldsymbol{g}) = D(k\boldsymbol{f}) = kD(\boldsymbol{f})$$

and

$$D(\boldsymbol{f} + \boldsymbol{g}) = D(\boldsymbol{f}) + D(\boldsymbol{g})$$

Thus, D is a linear transformation.

[SI*] **An Integral Transformation**

Let $V = C(-\infty, \infty)$ be the vector space of continuous functions on the interval $(-\infty, \infty)$, let $W = C^1(-\infty, \infty)$ be the vector space of functions with continuous first derivatives on $(-\infty, \infty)$, and let $J : V \to W$ be the transformation that maps a function f in V into

$$J(f) = \int_0^x f(t)\, dt$$

For example, if $f(x) = x^2$, then

$$J(f) = \int_0^x t^2\, dt = \frac{t^3}{3}\Big|_0^x = \frac{x^3}{3}$$

The transformation $J : V \to W$ is linear, for if k is any constant, and if f and g are any functions in V, then properties of the integral imply that

$$\begin{aligned} J(kf) &= \int_0^x kf(t)\, dt \\ &= k\int_0^x f(t)\, dt \\ &= kJ(f) \end{aligned}$$

$$\begin{aligned} J(f+g) &= \int_0^x (f(t) + g(t))\, dt \\ &= \int_0^x f(t)\, dt + \int_0^x g(t)\, dt \\ &= J(f) + J(g) \end{aligned}$$

[SI*] If A is an $m \times n$ matrix, then the null space of A consists of all vectors $\boldsymbol{x}$ in $\mathbb{R}^n$ such that $A\boldsymbol{x} = \boldsymbol{0}$, and the column space of A consists of all vectors $\boldsymbol{b}$ in $\mathbb{R}^m$ for which there is at least one vector $\boldsymbol{x}$ in $\mathbb{R}^n$ such that $A\boldsymbol{x} = \boldsymbol{b}$.

From the viewpoint of matrix transformations, the null space of A consists of all vectors in $\mathbb{R}^n$ that multiplication by A maps into $\boldsymbol{0}$, and the column space of A consists of all vectors in $\mathbb{R}^m$ that are images of at least one vector in $\mathbb{R}^n$ under multiplication by A.

Definition 6.49: Kernel and Range

If $T : V \to W$ is a linear transformation, then the set of vectors in V that maps T into $\boldsymbol{0}$ is called the kernel of T, and is denoted by $\ker(T)$.

The set of all vectors in W that are images under T of at least one vector in V is called the range of T and is denoted by $R(T)$.

[SI*] **Kernel and Range of a Matrix Transformation**
If $T_A : \mathbb{R}^n \to \mathbb{R}^m$ is multiplication by the $m \times n$ matrix A, then, the kernel of T_A is the null space of A, and the range of T_A is the column space of A.

[SI*] **Kernel and Range of the Zero Transformation**
Let $T : V \to W$ be the zero transformation. Since T maps every vector in V into $\mathbf{0}$, it follows that $\ker(T) = V$. Moreover, since $\mathbf{0}$ is the only image under T of vectors in V, it folows that $R(T) = \{\mathbf{0}\}$

[SI*] **Kernel and Range of the Identity Transformation**
Let $I : V \to V$ be the identity operator. Since $I(\boldsymbol{v}) = \boldsymbol{v}$ for all vectors in V, every vector in V is the image of some vector (itself); thus $R(I) = V$. Since the only vector that I maps into $\mathbf{0}$ is $\mathbf{0}$, it follows that $\ker(I) = \{\mathbf{0}\}$.

[SI*] **Kernel and Range of an Orthogonal Projection**
The points that T maps into $\mathbf{0} = (0, 0, 0)$ are precisely those on the z-axis, so $\ker(T)$ is the set of points of the form $(0, 0, z)$.

T maps the points in $\mathbb{R}^3$ to the xy-plane, where each point in that plane is the image of each point on the vertical line above it. Thus $R(T)$ is the set of points in the form $(x, y, 0)$.

[SI*] **Kernel and Range of a Rotation**
Let $T : \mathbb{R}^2 \to \mathbb{R}^2$ be the linear operator that rotates each vector in the xy-plane through the angle θ, it follows that $R(T) = R^2$. Moreover, the only vector that rotates into $\mathbf{0}$ is $\mathbf{0}$, so $\ker(T) = \{\mathbf{0}\}$.

[SI*] **Kernel of a Differentiation Transformation**
Let $V = C^1(-\infty, \infty)$ be the vector space of functions with continuous first derivatives on $(-\infty, \infty)$, let $W = F(-\infty, \infty)$ be the vector space of all real-valued functions defined on $(-\infty, \infty)$, and let $D : V \to W$ be the differentiation transformation $D(\boldsymbol{f}) = f'(x)$. The kernel of D is the set of functions in V with derivative zero. From calculus, this is the set of constant functions on $(-\infty, \infty)$.

[SI*] Most of the time, $\ker(T)$ and $R(T)$ turned out to be subspaces. They were either zero subspace or the entire vector space. In another case, the kernal was a line through the origin, and the range was a plane through the origin, both of which are subspaces of $\mathbb{R}^3$.

Theorem 6.98: Kernel and Range of a Linear Transformation

If $T : V \to W$ is a linear transformation, then:

(a) The kernel of T is a subspace of V.

(b) The range of T is a subspace of W.

[SI*] To show that $\ker(T)$ is a subspace, we must show that it contains at least one vector and is closed under addition and scalar multiplication.

[SI*] To show that $R(T)$ is a subspace, we must show that it contains at least one vector and is closed under addition and scalar multiplication.

[SI*] **Application to Differential Equations**
Differential equations of the form

$$y'' + \omega^2 y = 0 \tag{6.153}$$

with ω as a positive constant. The equation above arise in the study of vibrations. The set of all solutions of this equation on the interval $(-\infty, \infty)$ is the kernel of the linear

transformation $D : C^2(-\infty, \infty) \to C(-\infty, \infty)$, given by

$$D(y) = y'' + \omega^2 y$$

It is proved in standard textbooks on differential equations that the kernel is a two-dimensional subspace of $C^2(-\infty, \infty)$, so that if we can find two linearly independent solutions of the differential equation $D(y) = 0$, then all other solutions can be expressed as linear combinations of those two. We will find the general solution of

$$y'' + \omega^2 y = 0$$

the characteristic equation is $r^2 + \omega^2 = 0$ with the roots

$$r = \pm i\omega$$

the roots are conjugate complex numbers, hence the general solution is

$$y = c_1 e^{r_1 x} + c_2 e^{r_2 x}$$

and

$$\begin{aligned} y_1(x) &= e^{i\omega x} \\ y_2(x) &= e^{-i\omega x} \end{aligned}$$

the functions $y_1(x)$ and $y_2(x)$ are solutions of the differential equation. Based on the theorem, if y_1 and y_2 are solutions, then any linear combinations of y_1 and y_2 are also a solution.

$$\begin{aligned} y_1(x) + y_2(x) &= e^{i\omega x} + e^{-i\omega x} \\ &= (\cos\ \omega x + i \sin\ \omega x) + (\cos\ \omega x - i \sin\ \omega x) \\ &= 2 \cos\ \omega x \end{aligned} \tag{6.154}$$

and

$$\begin{aligned} y_1(x) - y_2(x) &= e^{i\omega x} - e^{-i\omega x} \\ &= (\cos\ \omega x + i \sin\ \omega x) - (\cos\ \omega x - i \sin\ \omega x) \\ &= 2i \sin\ \omega x \end{aligned} \tag{6.155}$$

neglecting the constant multipliers 2 and $2i$, respectively, we have obtained a pair of real-valued solutions

$$\begin{aligned} y_1(x) &= \cos\ \omega x \\ y_2(x) &= \sin\ \omega x \end{aligned}$$

they are solutions of the differential equations $D(y) = 0$. These functions are linearly independent since neither is a scalar multiple of the other, and thus

$$y = c_1 \cos\ \omega x + c_2 \sin\ \omega x \tag{6.156}$$

is a "general solution" of $D(y) = 0$ in the sense that every choice of c_1 and c_2 produces a solution, and every solution is of this form.

Definition 6.50: Rank and Nullity of Linear Transformations

Let $T : V \to W$ be a linear transformation. If the range of T is finite-dimensional, then its dimension is called the rank of T; and if the kernel of T is finite-dimensional, then its dimension is called the nullity of T. The rank of T is denoted by $\text{rank}(T)$ and the nullity of T by $\text{nullity}(T)$.

Theorem 6.99: Dimension Theorem for Linear Transformations

If $T : V \to W$ is a linear transformation from an n-dimensional vector space V to a vector sace W, then

$$\text{rank}(T) + \text{nullity}(T) = n \tag{6.157}$$

[SI*] In the special case where A is an $m \times n$ matrix and $T_A : R^n \to R^m$ is multiplication by A, the kernel of T_A is the null space of A, and the range of T_A is the column space of A. Thus, it follows that

$$\text{rank}(T_A) + \text{nullity}(T_A) = n$$

[SI*] **Example:**
Let v_1, v_2, and v_3 be vectors in a vector space V, and let $T : V \to \mathbb{R}^3$ be a linear transformation for which

$$\begin{aligned} T(v_1) &= (2, -1, 4) \\ T(v_2) &= (-3, 2, 1) \\ T(v_3) &= (0, 5, 1) \end{aligned}$$

Find $T(3v_1 - 2v_2 + v_3)$.
Solution:
The computation is quite simple

$$\begin{aligned} T(3v_1 - 2v_2 + v_3) &= 3T(v_1) - 2T(v_2) + T(v_3) \\ &= 3(2, -1, 4) - 2(-3, 2, 1) + (0, 5, 1) \\ &= (6, -3, 12) - (-6, 4, 2) + (0, 5, 1) \\ &= (12, -2, 11) \end{aligned}$$

[SI*] **Example:**
Let $T : \mathbb{R}^2 \to \mathbb{R}^2$ be the linear operator given by the formula

$$T(x, y) = (x - 3y, -2x + 6y)$$

(a) Which of the following vectors are in $R(T) : (1, -2), (3, 1), (-2, 4)$?

(b) Which of the following vectors are in $\ker(T) : (1, -3), (3, 1), (-6, -2)$?

Solution:

(a) We can use this method

$$\begin{aligned} x - 3y &= b_1 \\ -2x + 6y &= b_2 \end{aligned}$$

with (b_1, b_2) is the vector we want to test whether it is in $R(T)$ or not.

$$\begin{aligned} x - 3y &= 1 \\ -2x + 6y &= -2 \end{aligned}$$

the equations are linear combination of each other, thus $(1, -2)$ is in $R(T)$.

$$\begin{aligned} x - 3y &= 3 \\ -2x + 6y &= 1 \end{aligned}$$

the equations are not linear combination of each other, thus $(3, 1)$ is not in $R(T)$.

$$\begin{aligned} x - 3y &= -2 \\ -2x + 6y &= 4 \end{aligned}$$

the equations are linear combination of each other, thus $(-2, 4)$ is in $R(T)$.

(b) We will compute one by one

$$T(1, -3) = (1 - 3(-3), -2(1) + 6(-3)) = (10, -20)$$

$(1, -3)$ is not in $\ker(T)$.

$$T(3, 1) = (3 - 3(1), -2(3) + 6(1)) = (0, 0)$$

$(3, 1)$ is in $\ker(T)$

$$T(-6, -2) = (-6 - 3(-2), -2(-6) + 6(-2)) = (0, 0)$$

$(-6, -2)$ is in $\ker(T)$

[SI*] **Example:**
Let $T : P_2 \to P_3$ be the linear transformation defined by

$$T(p(x)) = xp(x)$$

(a) Which of the following are in $\ker(T)$: $x^2, 0, 1 + x$?

(b) Which of the following are in $R(T)$: $x + x^2, 1 + x, 3 - x^2$?

Solution:

(a) We will compute one by one

$$T(x^2) = x(x^2) = x^3$$

x^2 is not in $\ker(T)$.

$$T(0) = x(0) = 0$$

0 is in $\ker(T)$.

$$T(1 + x) = x(1 + x) = x + x^2$$

$1 + x$ is not in $\ker(T)$. The only vector that will map $T(p(x))$ into $\mathbf{0}$ is 0.

(b) Since $x + x^2 = x(1 + x)$, $x + x^2$ is in $R(T)$.

Neither $1 + x$ nor $3 - x^2$ can be expressed as $xp(x)$ with $p(x)$ in P_2, so neither are in $R(T)$.

[SI*] **Example:**
Find a basis for the kernel of

(a) the linear operator $T : \mathbb{R}^2 \to \mathbb{R}^2$ given by the formula

$$T(x, y) = (x - 3y, -2x + 6y)$$

(b) the linear transformation $T : P_2 \to P_3$ defined by

$$T(p(x)) = xp(x)$$

Solution:

(a) The kernel of $T(x, y)$ is the set of all vectors $(3x, x)$ or the line $y = 3x$.

The vector $(3, 1)$ is a basis for this space. If you input the basis or multiplication of the basis in the $T(x, y) = (x - 3y, -2x + 6y)$ we will obtain $\mathbf{0} = (0, 0)$.

(b) Since we need to obtain $T(x) = 0$, then

$$T(0) = x(0) = 0$$

the basis for the kernel is $\{0, 0, 0\}$ from standard basis of $P_2 : \{1, x, x^2\}$.

[SI*] **Example:**
Find a basis for the range of

(a) the linear operator $T : \mathbb{R}^2 \to \mathbb{R}^2$ given by the formula

$$T(x, y) = (x - 3y, -2x + 6y)$$

(b) the linear transformation $T : P_2 \to P_3$ defined by

$$T(p(x)) = xp(x)$$

(c) the matrix operator $T_A : \mathbb{R}^4 \to \mathbb{R}^3$ is multiplication by the 3×4 matrix A

$$A = \begin{bmatrix} 4 & 1 & -2 & -3 \\ 2 & 1 & 1 & -4 \\ 6 & 0 & -9 & 9 \end{bmatrix}$$

Solution:

(a) Since $x - 3y$ and $-2x + 6y$ can assume any real value, $R(T)$ is the set of all vectors $(x, -2x)$ or the line $y = -2x$. The vector $(1, -2)$ is a basis for this space.

(b) $R(T)$ is the set of all polynomials in P^3 with constant term 0. Thus the basis for the range / $R(T)$ is $\{x, x^2, x^3\}$.

(c) A basis for $R(T)$ is a basis for the column space of A.
First we will transform A into its row echelon form

$$A_{ref} = \begin{bmatrix} 1 & \frac{1}{2} & \frac{1}{2} & -2 \\ 0 & 1 & 4 & -5 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The columns of A corresponding to the columns of A_{ref} containing leading 1s are

$$\begin{bmatrix} 4 \\ 2 \\ 6 \end{bmatrix}, \quad \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}, \quad \begin{bmatrix} -3 \\ -4 \\ 9 \end{bmatrix}$$

Thus, $(4, 2, 6)$, $(1, 1, 0)$, and $(-3, -4, 9)$ form a basis for $R(T)$.

[SI*] **Example:**
Let T be multiplication by the matrix A.

$$\begin{bmatrix} 1 & -1 & 3 \\ 5 & 6 & -4 \\ 7 & 4 & 2 \end{bmatrix}$$

Find

(a) a basis for the range of T.
(b) a basis for the kernel of T.
(c) the rank and nullity of T.
(d) the rank and nullity of A.

Solution:

(a) A row echelon form of A is

$$A_{ref} = \begin{bmatrix} 1 & -1 & 3 \\ 0 & 1 & \frac{19}{11} \\ 0 & 0 & 0 \end{bmatrix}$$

thus columns 1 and 2 of A is

$$\begin{bmatrix} 1 \\ 5 \\ 7 \end{bmatrix}, \quad \begin{bmatrix} -1 \\ 6 \\ 4 \end{bmatrix}$$

form a basis for the column space of A, which is the range of T.

(b) The reduced row echelon form of A is

$$A_{rref} = \begin{bmatrix} 1 & 0 & \frac{14}{11} \\ 0 & 1 & -\frac{19}{11} \\ 0 & 0 & 0 \end{bmatrix}$$

so a general solution of $A\boldsymbol{x} = \mathbf{0}$ is

$$\begin{aligned} x_1 &= -\frac{14}{11}s \\ x_2 &= \frac{19}{11}s \\ x_3 &= s \end{aligned}$$

so

$$\begin{bmatrix} -14 \\ 19 \\ 11 \end{bmatrix}$$

is a basis for the null space of A, which is $\ker(T)$.

(c) $R(T)$ is two-dimensinoal, so $\text{rank}(T) = 2$.

$\ker(T)$ is one-dimensional, so $\text{nullity}(T) = 1$.

(d) The column space of A is two-dimensional, so $\text{rank}(A) = 2$. The null space of A is one-dimensional, so $\text{nullity}(A) = 1$.

[SI*] **Example:**
Describe the kernel and range of

(a) the orthogonal projection on the xz-plane.

(b) the orthogonal projection on the yz-plane.

(c) the orthogonal projection on the plane defined by the equation $y = x$.

Solution:

(a) The kernel is the y-axis, the range is the entire xy-plane.

(b) The kernel is the x-axis, the range is the entire yz-plane.

(c) The kernel is the line through the origin perpendicular to the plane $y = x$, the range is the entire plane $y = x$.

[SI*] **Example:**
In each part, use the given information to find the nullity of the linear transformation T.

(a) $T : \mathbb{R}^5 \to \mathbb{R}^7$ has rank 3.

(b) $T : P_4 \to P_3$ has rank 1.

(c) The range of $T : \mathbb{R}^6 \to \mathbb{R}^3$ is $\mathbb{R}^3$.

(d) $T : M_{22} \to M_{22}$ has rank 3.

Solution:

(a) $\text{nullity}(T) = 5 - \text{rank}(T) = 2$

(b) $\dim(P_4) = 5$, so $\text{nullity}(T) = 5 - \text{rank}(T) = 4$

(c) Since $R(T) = R^3$, T has rank 3.

$$\text{nullity}(T) = 6 - \text{rank}(T) = 3$$

(d) $\text{nullity}(T) = 4 - \text{rank}(T) = 1$

[SI*] **Example:**
Let A be a 5×7 matrix with rank 4.

(a) What is the dimension of the solution space of $A\boldsymbol{x} = \boldsymbol{0}$?

(b) Is $A\boldsymbol{x} = \boldsymbol{b}$ consistent for all vectors $\boldsymbol{b}$ in $\mathbb{R}^5$? Explain.

Solution:

(a) Remember that the dimension of matrix $m \times n$ is n (the number of column), thus

$$\dim(A) = \text{nullity}(A) + \text{rank}(A)$$

$\text{nullity}(A) = 7 - \text{rank}(A) = 3$, so the solution space of $A\boldsymbol{x} = \mathbf{0}$ has dimension 3.

(b) No, since $\text{rank}(A) = 4$ while $\mathbb{R}^5$ has 5 dimension. To be consistent it has to be in $\mathbb{R}^4$.

[SI*] **Example:**
Let $T : V \to \mathbb{R}^3$ be a linear transformation from any vector space to $\mathbb{R}^3$. Give a geometric description of $R(T)$.

Solution:
$R(T)$ must be a subspace of $\mathbb{R}^3$, thus the possibilities are a line through the origin, a plane through the origin, the origin only, or all of $\mathbb{R}^3$. It is the definition of subspaces of $\mathbb{R}^3$ from the General Vector Spaces chapter.

[SI*] **Example:**
Let $T : \mathbb{R}^3 \to \mathbb{R}^3$ be multiplication by

$$A = \begin{bmatrix} 1 & 3 & 4 \\ 3 & 4 & 7 \\ -2 & 2 & 0 \end{bmatrix}$$

(a) Show that the kernel of T is a line through the origin, and find parametric equations for it.

(b) Show that the range of T is a plane through the origin, and find an equation for it.

Solution:

(a) A reduced row echelon form of matrix A is

$$A_{rref} = \begin{bmatrix} 1 & -1 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

A general solution of $A\boldsymbol{x} = \mathbf{0}$ is

$$\begin{aligned} x_1 - x_2 &= 0 \\ x_2 + x_3 &= 0 \end{aligned}$$

$$\begin{aligned} x_1 &= -t \\ x_2 &= -t \\ x_3 &= t \end{aligned}$$

so

$$\begin{bmatrix} -1 \\ -1 \\ 1 \end{bmatrix}$$

is a basis for the null space of A, the kernel of T. Vector $(-1,-1,1)$ is a line through the origin, the parametric equation is

$$(x,y,z) = (-t-t,t) = (-1,-1,1)t$$

with variable t as the parameter.

(b) From the reduced row echelon form of matrix A, the columns of A corresponds to the columns of A_{rref} containing leading 1s are

$$\begin{bmatrix} 1 \\ 3 \\ -2 \end{bmatrix}, \quad \begin{bmatrix} 3 \\ 4 \\ 2 \end{bmatrix}$$

thus, $(1,3,-2)$ and $(3,4,2)$ form a basis for $R(T)$ (both vectors span $\mathbb{R}^3$ and not a linear combination to the other). The geometric description of this range of T is a plane through the origin with parametric equations

$$(x,y,z) = (s,3s,-2s) + (3t,4t,2t) = (1,3,-2)s + (3,4,2)t$$

with s,t as the parameters variables.

[SI*] **Example:**

(a) Show that if a_1, a_2, b_1, and b_2 are any scalars, then the formula

$$F(x,y) = (a_1x + b_1y, a_2x + b_2y)$$

defines a linear operator on $\mathbb{R}^2$.

(b) Does the formula $F(x,y) = (a_1x^2 + b_1y^2, a_2x^2 + b_2y^2)$ define a linear operator on $\mathbb{R}^2$? Explain.

Solution:

(a) We will need to show that $F(x,y)$ follows two properties which are homogeneity and additivity. First, for the homogeneity property

$$\begin{aligned} F(kx,ky) &= (a_1kx + b_1ky, a_2kx + b_2ky) \\ &= k(a_1x + b_1y, a_2x + b_2y) \\ &= kF(x,y) \end{aligned}$$

thus, $F(x,y)$ follows the homogeneity property. Now for the next property,

$$\begin{aligned} F(x+v,y+w) &= (a_1(x+v) + b_1(y+w), a_2(x+v) + b_2(y+w)) \\ &= (a_1x + b_1y, a_2x + b_2y) + (a_1v + b_1w, a_2v + b_2w) \\ &= F(x,y) + F(v,w) \end{aligned}$$

it proven that $F(x,y)$ follows the additivity property. Since $F(x,y)$ follows homogeneity and additivity property then it is a linear operator on $\mathbb{R}^2$.

(b) No, because it does not follow the homogeneity property

$$\begin{aligned} F(kx,ky) &= (a_1k^2x^2 + b_1k^2y^2, a_2k^2x^2 + b_2k^2y^2) \\ &= k^2(a_1x^2 + b_1y^2, a_2x^2 + b_2y^2) \\ &= k^2F(x,y) \\ &\neq kF(x,y) \end{aligned}$$

[SI*] **Example:**
Let $\{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n\}$ be a basis for a vector space V, and let $T : V \to W$ be a linear transformation. Show that if

$$T(\boldsymbol{v}_1) = T(\boldsymbol{v}_2) = T(\boldsymbol{v}_n) = \boldsymbol{0}$$

then T is the zero transformation.
Solution:
The zero transformation will have these properties

$$\begin{aligned} T(\boldsymbol{v}_i + \boldsymbol{v}_j) &= 0 \\ T(\boldsymbol{v}_i) &= 0 \\ T(\boldsymbol{v}_j) &= 0 \\ T(k\boldsymbol{v}_i) &= 0 \end{aligned}$$

for $i, j = 1, 2, \ldots, n$, with $i \neq j$. Since $T : V \to W$ is a linear transformation then it will follow the homogeneity and additivity properties

$$\begin{aligned} T(\boldsymbol{v}_i + \boldsymbol{v}_j) &= T(\boldsymbol{v}_i) + T(\boldsymbol{v}_j) \\ &= 0 + 0 \\ &= 0 \end{aligned}$$

$$\begin{aligned} T(k\boldsymbol{v}_i) &= kT(\boldsymbol{v}_i) \\ &= k(0) \\ &= 0 \end{aligned}$$

thus, T is the zero transformation.

[SI*] **Example:**
Let $\{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n\}$ be a basis for a vector space V, and let $T : V \to V$ be a linear operator. Show that if

$$T(\boldsymbol{v}_1) = \boldsymbol{v}_1, \quad T(\boldsymbol{v}_2) = \boldsymbol{v}_2, \ldots, T(\boldsymbol{v}_n) = \boldsymbol{v}_n$$

then T is the identity transformation on V.

Solution:
Let $\boldsymbol{w} = c_1\boldsymbol{v}_1 + c_2\boldsymbol{v}_2 + \cdots + c_n\boldsymbol{v}_n$ be any vector in V. Then

$$\begin{aligned} T(\boldsymbol{w}) &= c_1T(\boldsymbol{v}_1) + c_2T(\boldsymbol{v}_2) + \cdots + c_nT(\boldsymbol{v}_n) \\ &= c_1\boldsymbol{v}_1 + c_2\boldsymbol{v}_2 + \cdots + c_n\boldsymbol{v}_n \\ &= \boldsymbol{w} \end{aligned}$$

Since $\boldsymbol{w}$ was an arbitrary vector in V, T must be the identity operator.

[SI*] **Example:**
Prove: If $\{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n\}$ is a basis for V and $\boldsymbol{w}_1, \boldsymbol{w}_2, \ldots, \boldsymbol{w}_n$ are vectors in W, not necessarily distinct, then there exists a linear transformation $T : V \to W$ such that

$$T(\boldsymbol{v}_1) = \boldsymbol{w}_1, \quad T(\boldsymbol{v}_2) = \boldsymbol{w}_2, \ldots, T(\boldsymbol{v}_n) = \boldsymbol{w}_n$$

Solution:
If $T : V \to W$ is a linear transformation, then it has to obey these two properties

$$\begin{aligned} T(\boldsymbol{0}) &= \boldsymbol{0} \\ T(\boldsymbol{v}_i - \boldsymbol{v}_j) &= T(\boldsymbol{v}_i) - T(\boldsymbol{v}_j) \end{aligned}$$

for all $i, j = 1, 2, \ldots, n$ and $i \neq j$. Now we know that since $\{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n\}$ is a basis for V, then

$$\boldsymbol{v}_i - \boldsymbol{v}_j \neq 0$$

thus

$$\begin{aligned} T(\boldsymbol{v}_i - \boldsymbol{v}_j) &= T(\boldsymbol{v}_i) - T(\boldsymbol{v}_j) \\ &= \boldsymbol{w}_i - \boldsymbol{w}_j \end{aligned}$$

if we make an assumption that $\boldsymbol{w}_1, \boldsymbol{w}_2, \ldots, \boldsymbol{w}_n$ are vectors in W not necessarily distinct, then we will encounter this (assuming $\boldsymbol{w}_i = \boldsymbol{w}_j$)

$$\begin{aligned} T(\boldsymbol{v}_i - \boldsymbol{v}_j) &= \boldsymbol{w}_i - \boldsymbol{w}_j \\ &= 0 \end{aligned}$$

the consequence is $\boldsymbol{v}_i - \boldsymbol{v}_j$ will be in kernel of T too besides $\mathbf{0}$. Now, let $\boldsymbol{x}$ be any vector in V with $\boldsymbol{x} = c_1\boldsymbol{v}_1 + c_2\boldsymbol{v}_2 + \cdots + \boldsymbol{v}_n$, $c_1, c_2, \ldots, c_n$ are scalars then

$$\begin{aligned} T(\boldsymbol{x}) &= T(c_1\boldsymbol{v}_1 + c_2\boldsymbol{v}_2 + \cdots + c_n\boldsymbol{v}_n) \\ &= c_1T(\boldsymbol{v}_1) + c_2T(\boldsymbol{v}_2) + \cdots + c_nT(\boldsymbol{v}_n) \\ &= c_1\boldsymbol{w}_1 + c_2\boldsymbol{w}_2 + \cdots + c_n\boldsymbol{w}_n \end{aligned}$$

$T(\boldsymbol{x}) \in W$, and for any scalars k

$$\begin{aligned} T(k\boldsymbol{x}) &= T(c_1k\boldsymbol{v}_1 + c_2k\boldsymbol{v}_2 + \cdots + c_nk\boldsymbol{v}_n) \\ &= kc_1T(\boldsymbol{v}_1) + kc_2T(\boldsymbol{v}_2) + \cdots + kc_nT(\boldsymbol{v}_n) \\ &= k(c_1\boldsymbol{w}_1 + c_2\boldsymbol{w}_2 + \cdots + c_n\boldsymbol{w}_n) \end{aligned}$$

T follows the homogeneity and additivity properties, thus the linear transformation T is proven to exist.

[SI*] **Example:**
Let $D : P_3 \to P_2$ be the differentiation transformation $D(\boldsymbol{p}) = p'(x)$. What is the kernel of D?
Solution:
If $p'(x) = 0$, then $p(x)$ is a constant , so $\ker(D)$ consists of all constant polynomials.

[SI*] **Example:**
Let V be the vector space of real-valued functions with continuous derivatives of all orders on the interval $(-\infty, \infty)$, and let $W = F(-\infty, \infty)$ be the vector space of real-valued functions defined on $(-\infty, \infty)$.

(a) Find a linear transformation $T : V \to W$ whose kernel is P_3.

(b) Find a linear transformation $T : V \to W$ whose kernel is P_n.

Solution:

(a) The fourth derivative of any polynomial of degree 3 or less is 0, so

$$T(f(x)) = f^{(4)}(x)$$

has $\ker(T) = P_3$.

(b) The $n+1$-th derivative of any polynomial of degree n or less is 0, so

$$T(f(x)) = f^{(n+1)}(x)$$

has $\ker(T) = P_n$.

[SI*] The vector space $\mathbb{R}^n$ is the "mother" of all real n-dimensional vector spaces in the sense that any such space might differ from $\mathbb{R}^n$ in the notation used to represent vectors, but not in its algebraic structure.

Definition 6.51: One-to-One

If $T : V \to W$ is a linear transformation from a vector space V to a vector space W, then T is said to be one-to-one if T maps distinct vectors in V into distinct vectors in W.

Definition 6.52: Onto

If $T : V \to W$ is a linear transformation from a vector space V to a vector space W, then T is said to be onto (or onto W) if every vector in W is the image of at least one vector in V.

Theorem 6.100: Kernel and One-to-One

If $T : V \to W$ is a linear transformation, then the following statements are equivalent.

(a) T is one-to-one.

(b) $\ker(T) = \{\mathbf{0}\}$

Theorem 6.101: Linear Operator in a Finite-Dimensional Vector Space

If V is a finite-dimensional vector space, and if $T : V \to V$ is a linear operator, then the following statements are equivalent.

(a) T is one-to-one.

(b) $\ker(T) = \{\mathbf{0}\}$.

(c) T is onto [i.e., $R(T) = V$]

[SI*] **Dilations and Contractions Are One-to-One and Onto**
If V is a finite-dimensional vector space and c is any nonzero scalar, then the linear operator $T : V \to V$ defined by $T(\boldsymbol{v}) = c\boldsymbol{v}$ is one-to-one and onto.

We know that the operator T is onto (and hence one-to-one) for if $\boldsymbol{v}$ is any vector in V then that vector is the image of the vector $(1/c)\boldsymbol{v}$.

[SI*] **Matrix Operators**
If $T_A : \mathbb{R}^n \to \mathbb{R}^n$ is the matrix operator $T_A(\boldsymbol{x}) = A\boldsymbol{x}$, then it follows from theorem in the Eigenvalues and Eigenvectors section that

(a) The range of T_A is $\mathbb{R}^n$.

(b) T_A is one-to-one.

so T_A is one-to-one and onto if and only if A is invertible.

[SI*] **Shifting Operators**

Let $V = \mathbb{R}^\infty$ be the sequence and consider the linear "shifting operators" on V defined by

$$T_1(u_1, u_2, \dots, u_n, \dots) = (0, u_1, u_2, \dots, u_n, \dots)$$
$$T_2(u_1, u_2, \dots, u_n, \dots) = (u_2, u_3, \dots, u_n, \dots)$$

T_1 is one-to-one because distinct dequences in $\mathbb{R}^\infty$ obviously have distinct images. This operator is not onto because no vector in $\mathbb{R}^\infty$ maps into the sequence $(1, 0, 0, \dots, 0, \dots)$, for example.

T_2 is not one-to-one because the vectors $(1, 0, 0, \dots, 0, \dots)$ and $(2, 0, 0, \dots, 0, \dots)$ both map into $(0, 0, 0, \dots, 0, \dots)$. This operator is onto because every possible sequence of real numbers can be obtained with an appropriate choice of the numbers $u_2, u3, \dots, u_n, \dots$.

[SI*] **Basic Transformations That Are One-to-One and Onto**

The linear transformation $T_1 : P_3 \to \mathbb{R}^4$ and $T_2 : M_{22} \to \mathbb{R}^4$ defined by

$$T_1(a + bx + cx^2 + dx^3) = (a, b, c, d)$$

$$T_2\left(\begin{bmatrix} a & b \\ c & d \end{bmatrix}\right) = (a, b, c, d)$$

are both one-to-one and onto. Their kernels contain only the zero vector.

[SI*] **A One-to-One Linear Transformation**

Let $T : P_n \to P_{n+1}$ be the linear transformation

$$T(\boldsymbol{p}) = T(p(x)) = xp(x)$$

If

$$\boldsymbol{p} = p(x) = c_0 + c_1 x + \cdots + c_n x^n$$
$$\boldsymbol{q} = q(x) = d_0 + d_1 x + \cdots + d_n x^n$$

are distinct polynomials, then they differ in at least one coefficient. Thus,

$$T(\boldsymbol{p}) = c_0 x + c_1 x^2 + \cdots + c_n x^{n+1}$$
$$T(\boldsymbol{q}) = d_0 x + d_1 x^2 + \cdots + d_n x^{n+1}$$

also differ in at least one coefficient. It follows that T is one-to-one since it maps distinct polynomials $\boldsymbol{p}$ and $\boldsymbol{q}$ into distinct polynomials $T(\boldsymbol{p})$ and $T(\boldsymbol{q})$.

[SI*] **A Transformation That Is Not One-to-One**

Let

$$D : C^1(-\infty, \infty) \to F(-\infty, \infty)$$

be the differentiation transformation. This linear transformation is not one-to-one because it maps functions that differ by constant into the same function. For example,

$$D(x^2) = D(x^2 + 1) = 2x$$

Remember that $F(-\infty, \infty)$ represents the set of functions with derivatives of all orders on $(-\infty, \infty)$, while $C^1(-\infty, \infty)$ represents functions with the first derivatives on $(-\infty, \infty)$ that is continuous, and $C(-\infty, \infty)$ is the set of continuous functions on $(-\infty, \infty)$.

[SI*] Let $T : V \to W$ be a linear transformation, where V and W are finite-dimensional. Then

1. If $\dim(W) < \dim(V)$, then T cannot be one-to-one.
2. If $\dim(V) < \dim(W)$, then T cannot be onto.

These observations tell us, that any linear tansformation from $\mathbb{R}^3$ to $\mathbb{R}^2$ must map some distinct points of $\mathbb{R}^3$ into the same point in $\mathbb{R}^2$, and it also tells us that there is no linear transformation that maps $\mathbb{R}^2$ onto all of $\mathbb{R}^3$.

Definition 6.53: Isomorphism

If a linear transformation $T : V \to W$ is both one-to-one and onto, then T is said to be an isomorphism, and the vector spacesd V and W are said to be isomorphic.

[SI*] Isomorphic vector spaces have the same "algebraic form," even though they may consist of different kinds of objects.

THe isomorphism

$$a_0 + a_1 x + a_2 x^2 \vec{T}(a_0, a_1, a_2)$$

matches up vector operations in P_2 and $\mathbb{R}^3$.

Theorem 6.102: Isomorphism for n-Dimensional Vector Space

Every real n-dimensional vector space is isomorphic to $\mathbb{R}^n$.

[SI*] Let V be a real n-dimensional vector space. To prove that V is isomorphic to $\mathbb{R}^n$ we must find a linear transformation $T : V \to \mathbb{R}^n$ that is one-to-one and onto. For this purpose, let

$$\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n$$

be any basis for V, let

$$\boldsymbol{u} = k_1\boldsymbol{v}_1 + k_2\boldsymbol{v}_2 + \cdots + k_n\boldsymbol{v}_n \tag{6.158}$$

be the representation of a vector $\boldsymbol{u}$ in V as a linear combination of the basis vectors, and define the transformation $T : V \to \mathbb{R}^n$ by

$$T(\boldsymbol{u}) = (k_1, k_2, \ldots, k_n) \tag{6.159}$$

We will show that T is an isomorphism (linear, one-to-one, and onto). To prove the lienarity, let $\boldsymbol{u}$ and $\boldsymbol{v}$ be vectors in V, let c be a scalar, and let

$$\begin{aligned} \boldsymbol{u} &= k_1\boldsymbol{v}_1 + k_2\boldsymbol{v}_2 + \cdots + k_n\boldsymbol{v}_n \\ \boldsymbol{v} &= d_1\boldsymbol{v}_1 + d_2\boldsymbol{v}_2 + \cdots + d_n\boldsymbol{v}_n \end{aligned} \tag{6.160}$$

be the representations of $\boldsymbol{u}$ and $\boldsymbol{v}$ as linear combinations of the basis vectors. Then it follows that

$$\begin{aligned} T(c\boldsymbol{u}) &= T(ck_1\boldsymbol{v}_1 + ck_2\boldsymbol{v}_2 + \cdots + ck_n\boldsymbol{v}_n) \\ &= (ck_1, ck_2, \ldots, ck_n) \\ &= c(k_1, k_2, \ldots, k_n) \\ &= cT(\boldsymbol{u}) \end{aligned}$$

and it folows that

$$\begin{aligned} T(\boldsymbol{u}+\boldsymbol{v}) &= T((k_1+d_1)\boldsymbol{v}_1+(k_2+d_2)\boldsymbol{v}_2+\cdots+(k_n+d_n)\boldsymbol{v}_n) \\ &= (k_1+d_1, k_2+d_2, \ldots, k_n+d_n) \\ &= (k_1, k_2, \ldots, k_n)+(d_1, d_2, \ldots, d_n) \\ &= T(\boldsymbol{u})+T(\boldsymbol{v}) \end{aligned}$$

which shows that T is linear. To show that T is one-to-one, we must show that if $\boldsymbol{u}$ and $\boldsymbol{v}$ are distinct vectors in V, then so are their images in $\mathbb{R}^n$. But if $\boldsymbol{u} \neq \boldsymbol{v}$, and if the representations of these vectors in terms of the basis vectors, then we must have $k_i \neq d_i$ for at least one i. Thus,

$$T(\boldsymbol{u}) = (k_1, k_2, \ldots, k_n) \neq (d_1, d_2, \ldots, d_n) = T(\boldsymbol{v})$$

which shows that $\boldsymbol{u}$ and $\boldsymbol{v}$ have distinct images under T. Finally, the transformation T is onto, for if

$$\boldsymbol{w} = (k_1, k_2, \ldots, k_n)$$

is any vector in $\mathbb{R}^n$, then it folows that $\boldsymbol{w}$ is the image under T of the vector

$$\boldsymbol{u} = k1\boldsymbol{v}_1 + k_2\boldsymbol{v}_2 + \cdots + k_n\boldsymbol{v}_n$$

Remark
Note that the isomorphism T of the foregoing proof is the coordinate map

$$\boldsymbol{u}\vec{T}(k_1, k_2, \ldots, k_n) = (\boldsymbol{u})_S$$

that maps $\boldsymbol{u}$ into its coordinate vector with respect to the basis $S = \{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n\}$. Since there are generally many possible bases for a given vector space V, there are generally many possible isomorphisms between V and $\mathbb{R}^n$, one for each different basis.

[SI*] **The Natural Isomorphism from P_{n-1} to $\mathbb{R}^n$**
The mapping

$$a_0 + a_1x + \cdots + a_{n-1}x^{n-1} \underrightarrow{T} (a_0, a_1, \ldots, a_{n-1})$$

from P_{n-1} to $\mathbb{R}^n$ is one-to-one, onto, and linear. This is called the natural isomorphism from P_{n-1} to $\mathbb{R}^n$ because, as the following computations show, it maps the natural basis $\{1, x, x^2, \ldots, x^{n-1}\}$ for P_{n-1} into the standard basis for $\mathbb{R}^n$:

$$\begin{array}{rcccc} 1 & = & 1+0x+0x^2+\cdots+0x^{n-1} & \underrightarrow{T} & (1,0,0,\ldots,0) \\ x & = & 0+x+0x^2+\cdots+0x^{n-1} & \underrightarrow{T} & (0,1,0,\ldots,0) \\ \vdots & & \vdots & \vdots & \vdots \\ x^{n-1} & = & 0+0x+0x^2+\cdots+0x^{n-2}+x^{n-1} & \underrightarrow{T} & (0,0,0,\ldots,0,1) \end{array}$$

[SI*] **The Natural Isomorphism from M_{22} to $\mathbb{R}^4$**
The matrices

$$E_1 = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, \quad E_2 = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \quad E_3 = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}, \quad E_4 = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$$

form a basis for the vector space M_{22} of 2×2 matrices. An isomorphism $T : M_{22} \to \mathbb{R}^4$ can be constructed by first writing a matrix A in M_{22} in terms of the basis vectors as

$$A = \begin{bmatrix} a_1 & a_2 \\ a_3 & a_4 \end{bmatrix} = a_1 \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} + a_2 \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} + a_3 \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix} + a_4 \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$$

and then defining T as

$$T(A) = (a_1, a_2, a_3, a_4)$$

Thus, for example,

$$\begin{bmatrix} 1 & -3 \\ 4 & 6 \end{bmatrix} \underset{\rightarrow}{T} (1, -3, 4, 6)$$

More generally, this idea can be used to show that the vector space M_{mn} of $m \times n$ matrices with real entries is isomorphic to $\mathbb{R}^{mn}$.

[SI*] **Differentiation by Matrix Multiplication**

Consider the differentiation transformation $D : P_3 \rightarrow P_2$ on the vector space of polynomials of degree three or less. If we map P_3 and P_2 into $\mathbb{R}^4$ and $\mathbb{R}^3$, respectively, by the natural isomorphisms, then the transformation D produces a corresponding matrix transformation from $\mathbb{R}^4$ to $\mathbb{R}^3$. Specifically, the derivative transformation

$$a_0 + a_1 x + a_2 x^2 + a_3 x^3 \underset{\rightarrow}{T} a_1 + 2a_2 x + 3a_3 x^2$$

produces the matrix transformation

$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 3 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} a_1 \\ 2a_2 \\ 3a_3 \end{bmatrix}$$

Thus, for example, the derivative

$$\frac{d}{dx}(2 + x + 4x^2 - x^3) = 1 + 8x - 3x^2$$

can be calculated as the matrix product

$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 3 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \\ 4 \\ -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 8 \\ -3 \end{bmatrix}$$

This idea is useful for constructing numerical algorithms to perform derivative calculations.

[SI*] In the case where V is a real n-dimensional inner product space, both V and $\mathbb{R}^n$ have, in addition to their algebraic structure, a geometric structure arising from their respective inner products. Thus, it is reasonable to inquire if there exists an isomorphism from V to $\mathbb{R}^n$ that preserves the geometric structure as well as the algebraic structure. For example, we would want orthogonal vectors in V to have orthogonal counterparts in $\mathbb{R}^n$, and we would want orthonormal sets in V to correspond to orthonormal sets in $\mathbb{R}^n$.

In order for an isomorphism to preserve geometric structure, it obviously has to preserve inner products, since notions of length, angle, and orthogonality are all based on the inner product. Thus, if V and W are inner product spaces, then we call an isomorphism $T : V \rightarrow W$ an inner product space isomorphism if

$$< T(\boldsymbol{u}), T(\boldsymbol{v}) > = < \boldsymbol{u}, \boldsymbol{v} >$$

It can be proved that if V is any real n-dimensional inner product space and $\mathbb{R}^n$ has the Euclidean inner product (the dot product), then there exists an inner product space

isomorphism from V to $\mathbb{R}^n$. Under such an isomorphism, the inner product space V has the same algebraic and geometric structure as $\mathbb{R}^n$. In this sense, every n-dimensional inner product space is a "carbon copy" of $\mathbb{R}^n$ with the Euclidean inner product that differs only in the notation used to represent vectors.

[SI*] **An Inner Product Space Isomorphism**
Let $\mathbb{R}^n$ be the vector space of real n-tuples in comma-delimited form, let M_n be the vector space of real $n \times 1$ matrices, let $\mathbb{R}^n$ have the Euclidean inner product $< \boldsymbol{u}, \boldsymbol{v} >= \boldsymbol{u} \cdot \boldsymbol{v}$, and let M_n have the inner product $< \boldsymbol{u}, \boldsymbol{v} >= \boldsymbol{u}^T \boldsymbol{v}$ in which $\boldsymbol{u}$ and $\boldsymbol{v}$ are expressed in column form. The mapping $T : \mathbb{R}^n \to M_n$ defined by

$$(v_1, v_2, \ldots, v_n) \xrightarrow{T} \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}$$

is an inner product space isomorphism, so the distinction between the inner product space $\mathbb{R}^n$ and the inner product space M_n is essentially notational, a fact that we have used many times in this text.

i. Functions in SymIntegration Related to Linear Transformations

[SI*] We will list the basic functions that can be used related to linear transformations:

IX. Numerical Methods

[SI*] From the first section on this chapter of Numerical Linear Algebra, we have learned about Gaussian elimination and backward substitution to obtain the solution for systems of linear equations, they are fine for small-scale problems, but Gaussian elimination and Gauss-Jordan elimination are not suitable for large-scale problems in which compute roundoff error, memory usage, and speed are concerns. For large-scale and special case we can use numerical methods to solve systems of linear equations, these are the available methods currently:

1. **LU Decomposition**
   A matrix factorization technique into lower triangular L and upper triangular U, it is used to solve systems of linear equations. It transforms a complex matrix problem into two simpler problems that can be solved sequentially using forward and backward substitution.
2. **QR Decomposition**
   more stable than LU decomposition, but slower to converge.
3. **Cholesky Decomposition**
   It is used if the system is symmetric.
4. **Singular Value Decomposition (SVD)**
   If the system is not square (the number of equations is not the same as the number of variables).

Decomposition has better efficiency, for a given matrix A, the costly decomposition step is performed only once. This is faster for repeated solving of $A\boldsymbol{x} = \boldsymbol{b}$ with different $\boldsymbol{b}$ vectors compared to reinverting the matrix each time.

i. Functions in SymIntegration Related to Numerical Methods

[SI*] We will list the basic functions that can be used related to numerical methods:

LUsolve_nhsystem(vector<vector<double>> &A, vector<vector<double>> &b, vector<double> &x)
to find the solution of a linear system $A\boldsymbol{x} = \boldsymbol{b}$ with LU-decomposition.

LUDecomposition(vector<vector<double>> &A, vector<vector<double>> &L, vector<vector<double>> &U)
to compute the LU-decomposition of matrix A, it is a void function so you have to call the matrix L and U later on in the source code, remember also this function doesn't compute the solution of the linear system.

SVD(vector<vector<double>>& A)
to compute singular value decomposition (SVD) with A as the matrix that we want to apply the SVD, SVD relies heavily on eigenvalues and eigenvectors computation for all the U, Σ, V^T matrices.

ii. LU-Decomposition

The credit for popularizing the matrix formulation of the LU-decomposition is often given to the British mathematician Alan Turing for his work on the subject in 1948. Turing, one of the great geniuses of the twentieth century is the founder of the field of artificial intelligence. Sadly, Turing, a homosexual, was tried and convicted of "gross indecency" in 1952, in violation of the then-existing British statutes. Depressed, he committed suicide at age 41 by eating an apple laced with cyanide. Lately before passed away in the 21st century, Queen Elizabeth II has done public apology for what has happened for Alan Turing, a world war II hero, without him Nazi cannot be stopped, this shows how close minded people in the past are, they read less, comprehend less, but judge more, just like the time when church burned Jeanne D'arc, or burn Galileo Galilei paper works, then apology after hundered of years and made Jeanne D'arc a saint. It is human nature to judge what they don't know, out of fear, people are different just like the world full of color, so by learning science, computer and engineering, hopefully the result will make people have better understanding, compassion, and have higher quality of life.

Definition 6.54: LU-Decomposition

A factorization of a square matrix A as $A = LU$, where L is lower triangular and U is upper triangular is called an LU-decomposition (or LU-factorization) of A.

Not every square matrix has an LU-decomposition.

To systemically solve for entries in L and U from matrix A, the formula for matrix U:

$$\forall j \begin{cases} i = 0 \to U_{ij} = A_{ij} \\ i > 0 \to U_{ij} = A_{ij} - \sum_{k=0}^{i-1} L_{ik} U_{kj} \end{cases} \tag{6.161}$$

the formula for matrix L:

$$\forall i \begin{cases} j = 0 \rightarrow L_{ij} = \dfrac{A_{ij}}{U_{jj}} \\ j > 0 \rightarrow L_{ij} = \dfrac{A_{ij} - \sum_{k=0}^{j-1} L_{ik} U_{kj}}{U_{jj}} \end{cases} \tag{6.162}$$

In LU-decomposition we are using Doolittle algorithm, it is an efficient method for LU-decomposition, enabling various applications in numerical analysis and linear algebra. Applications on Doolittle algorithm:

1. **Solving Linear Equations**
   LU-decomposition simplifies solving $A\boldsymbol{x} = \boldsymbol{b}$ by solving $L\boldsymbol{z} = \boldsymbol{b}$ then $U\boldsymbol{x} = \boldsymbol{y}$.

2. **Matrix Inversion**
   Decompose A into LU, then invert L and U.

3. **Determinant Calculation**
   The determinant of A is the product of the diagonal elements of U. The formula is

   $$\det(A) = \det(L) \times \det(U)$$

   since $\det(L) = 1$, thus we only need to compute $\det(U)$.

With variations like partial pivoting, it creates numerical stability, to made it more robust against numerical errors that can occur during the process.

Applications of LU-decomposition:

1. **Structural Engineering**
   It is used to analyze forces and stresses in bridges and buildings, ensuring efficient and safe designs then model the structure's behavior. It can be used with modal flexibility matrices to identify and locate damage in a structure by analyzing changes in its' structural properties.

2. **Finite Element Analysis (FEA)**
   FEA relies heavily on matrix-based methods. LU-decomposition is used to solve the resulting stiffness matrices, which represent the relationship between forces and displacements in a structure.

3. **Heat Transfer**
   It solves the systems of equations that result from discretizing partial differential equations used to model heat flow. It is used to solve for temperature distribution in objects when modeled using finite difference or finite element methods.

4. **Fluid Dynamics**
   It is applied in computational fluid dynamics (CFD) to solve the pressure and velocity fields in fluid flow, which is often modeled by a system of linear equations.

5. **Geotechnical Engineering**
   It can be applied in solving problems related to soil mechanics and foundation design that involve large systems of equations.

6. **Computer Graphics**
   It helps in transforming 3D objects, like rotating and scaling models, for smoother rendering.

7. **Robotics**
   It assists in solving kinematic equations, enabling real-time movement adjustments for robots.

8. **Weather Prediction**
   It speeds up the solving of climate models and weather simulations for accurate forecasting.

9. **Electrical Engineering**
   It is used in circuit analysis to solve systems of equations for designing and optimizing electrical circuits. *LU*-decomposition is a core component of the numerical iterative algorithms used to solve the power flow problem in power systems. It is used in methods like the Method of Moments for analyzing electromagnetic radiation and scattering, especially when a matrix needs to be decomposed repeatedly for different configurations, saving computation time. It can also be applied to matrix equations that arise in antenna design, particularly when optimizing parameters on a fixed geometry.

10. **Modeling Complex Systems**
    Chemical processes often involve multiple interacting components, leading to a large system of linear equations that describe the process. *LU*-decomposition provides a structured way to solve these systems.

11. **Steady-State and Transient Analysis**
    It is used in analyzing systems at steady-state and in transient simulations where the system evolves over time.

12. **Chemical Reaction Networks**
    It is used for calculating concentrations of reactants and products in a system with multiple parallel or consecutive reactions.

13. **Economics and Finance**
    It helps solve economic models for resource allocation and market predictions efficiently.

*Remember that *LU*-decompositions are not unique.

iii. Solve Nonhomogeneous System of Linear Equation with *LU*-Decomposition in SymIntegration

Solve the linear system

$$\begin{bmatrix} 1 & 1 & 1 \\ 4 & 3 & -1 \\ 3 & 5 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 6 \\ 4 \end{bmatrix}$$

Solution:
We can rewrite this system as

$$\begin{bmatrix} 1 & 0 & 0 \\ 4 & 1 & 0 \\ 3 & -2 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 \\ 0 & -1 & -5 \\ 0 & 0 & -10 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 6 \\ 4 \end{bmatrix}$$

Let us define z_1, z_2, and z_3 by the equation $U\boldsymbol{x} = \boldsymbol{z}$

$$\begin{bmatrix} 1 & 1 & 1 \\ 0 & -1 & -5 \\ 0 & 0 & -10 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix}$$

then we can write the equation $L\boldsymbol{z} = \boldsymbol{b}$

$$\begin{bmatrix} 1 & 0 & 0 \\ 4 & 1 & 0 \\ 3 & -2 & 1 \end{bmatrix} \begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 6 \\ 4 \end{bmatrix}$$

the equation $L\boldsymbol{z} = \boldsymbol{b}$ can be written equivalently as

$$\begin{array}{llll} z_1 & & & = 1 \\ 4z_1 & + z_2 & & = 6 \\ 3z_1 & - 2z_2 & + z_3 & = 4 \end{array}$$

now by using forward substitution we will obtain

$$z_1 = 1,\ z_2 = 2,\ z_3 = 5$$

We go back to the equation $U\boldsymbol{x} = \boldsymbol{z}$ and substitute z into the equation, which yields the linear system

$$\begin{bmatrix} 1 & 1 & 1 \\ 0 & -1 & -5 \\ 0 & 0 & -10 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 5 \end{bmatrix}$$

or equivalently,

$$\begin{array}{llll} x_1 & + x_2 & + x_3 & = 1 \\ & -x_2 & - 5x_3 & = 2 \\ & & -/10x_3 & = 5 \end{array}$$

now by using back substitution we will obtain

$$x_1 = 1,\ x_2 = 0.5,\ x_3 = -0.5$$

In SymIntegration, to make the matrices L and U with LU-decomposition we can use this function:
LUDecomposition(vector<vector<double>> &A, vector<vector<double>> &L, vector<vector<double>> &U)
the main source code for this function is located in **src/numericalmethods.cpp**.

```
#include<bits/stdc++.h>
#include<iostream>
#include "symintegrationc++.h"
#include<vector>
#include <chrono>
#include <algorithm> // For std::next_permutation
#include <string>
using namespace std::chrono;
using namespace std;

// Driver program
int main()
{
       // Get starting timepoint
       auto start = high_resolution_clock::now();

       //string filename = "matrix.txt";

       dmat doubleMatrix = loadMatrixFromFile("matrix.txt");
       dmat vecb = loadMatrixFromFile("vectorb.txt");
       int n = doubleMatrix.size();
       int R = doubleMatrix.size();
       int C = doubleMatrix[0].size();

       // Declare the matrix L and U with the size.
       dmat L(n, vector<double>(n));
       dmat U(n, vector<double>(n));

       printMatrix(doubleMatrix);

       LUDecomposition(doubleMatrix, L, U);

       cout << "\nMatrix A : " << endl;
       printMatrix(doubleMatrix);
       cout << "\nMatrix L : " << endl;
       printMatrix(L);
       cout << "\nMatrix U : " << endl;
       printMatrix(U);

       // Get ending timepoint
       auto stop = high_resolution_clock::now();
       auto duration = duration_cast<microseconds>(stop - start);

       cout << "\nTime taken by function: " << duration.count() << " microseconds"
            << endl;
```

```
	return 0;
}
```

Code 68: *Test SymIntegration LU Decomposition/main.cpp*

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration LU Decomposition ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration LU Decomposition ]# ./main
Data:
        1        1        1
        4        3       -1
        3        5        3

Matrix A :
        1.000000         1.000000         1.000000
        4.000000         3.000000        -1.000000
        3.000000         5.000000         3.000000

Matrix L :
        1.000000         0.000000         0.000000
        4.000000         1.000000         0.000000
        3.000000        -2.000000         1.000000

Matrix U :
        1.000000         1.000000         1.000000
        0.000000        -1.000000        -5.000000
        0.000000         0.000000       -10.000000

Time taken by function: 1485 microseconds
```

Figure 6.60: *The LU-decomposition of matrix A (**SymIntegration/Examples/Linear Algebra/Test SymIntegration LU Decomposition/main.cpp**).*

To find the solution with *LU*-decomposition, we use this function:
LUsolve_nhsystem(vector<vector<double>> &A, vector<vector<double>> &b, vector<double> &x)
the main source code for this function is located in **src/linearalgebra.cpp**.

```
#include<bits/stdc++.h>
#include<iostream>
#include "symintegrationc++.h"
#include<vector>
#include <chrono>
#include <algorithm> // For std::next_permutation
#include <string>

using namespace std::chrono;
using namespace std;

// Driver program
int main()
{
	// Get starting timepoint
	auto start = high_resolution_clock::now();

	dmat doubleMatrix = loadMatrixFromFile("matrix.txt");
	dmat vecb = loadMatrixFromFile("vectorb.txt");
	int n = doubleMatrix.size();

	cout << "Data:" << endl;
	printMatrix(doubleMatrix);
```

```
        dvec x;
        LUsolve_nhsystem(doubleMatrix,vecb,x);
        cout << "\nx : " << endl;
        printVector(x);

        // Get ending timepoint
        auto stop = high_resolution_clock::now();
        auto duration = duration_cast<microseconds>(stop - start);

        cout << "\nTime taken by function: " << duration.count() << " microseconds"
             << endl;

        return 0;
}
```

Code 69: *Test SymIntegration LU Solve/main.cpp*

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration LU Solve ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration LU Solve ]# ./main
Data:
         1.000000         1.000000         1.000000
         4.000000         3.000000        -1.000000
         3.000000         5.000000         3.000000

Solution for Lz = b:
z1 = 1.00000
z2 = 2.00000
z3 = 5.00000


Solution for Ux = z:
x1 = 1.00000
x2 = 0.50000
x3 = -0.50000


x :
1.000000
0.500000
-0.500000

Time taken by function: 1142 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration LU Solve ]#
```

Figure 6.61: *Computing the solution for $A\boldsymbol{x} = \boldsymbol{b}$ with LU-decomposition (**SymIntegration/Examples/Linear Algebra/Test SymIntegration LU Solve/main.cpp**).*

We commented some **cout** commands and only show the solution vector $\boldsymbol{x}$, it decreases the function time making it faster to show the final result.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration LU Solve ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration LU Solve ]# ./main
Data:
         1.000000         1.000000         1.000000
         4.000000         3.000000        -1.000000
         3.000000         5.000000         3.000000

x :
1.000000
0.500000
-0.500000

Time taken by function: 982 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration LU Solve ]#
```

Figure 6.62: *Computing the solution for $A\boldsymbol{x} = \boldsymbol{b}$ with LU-decomposition and only show the solution vector $\boldsymbol{x}$ (**SymIntegration/Examples/Linear Algebra/Test SymIntegration LU Solve/main.cpp**).*

We also compare it with Gaussian elimination function, to see which algorithm could compute the solution vector $\boldsymbol{x}$ faster, *LU*-decomposition is faster than Gaussian elimination.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Gaussian Elimina
tion ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Gaussian Elimination ]# ./main
Data:
         1.000000         1.000000         1.000000
         4.000000         3.000000        -1.000000
         3.000000         5.000000         3.000000
Augmented Matrix:
         1.000000         1.000000         1.000000         1.000000
         4.000000         3.000000        -1.000000         6.000000
         3.000000         5.000000         3.000000         4.000000

Augmented Matrix in reduced row form:
         4.000000         3.000000        -1.000000         6.000000
         0.000000         2.750000         3.750000        -0.500000
         0.000000         0.000000         0.909091        -0.454545

Solution:
x1 = 1.00000
x2 = 0.50000
x3 = -0.50000


x :
1.000000
0.500000
-0.500000

Time taken by function: 1293 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Gaussian Elimination ]#
```

Figure 6.63: *Computing the solution for $A\boldsymbol{x} = \boldsymbol{b}$ with Gaussian elimination (**SymIntegration/Examples/Linear Algebra/Test SymIntegration Gaussian Elimination Function/main.cpp**).*

iv. Cholesky Decomposition

Cholesky decomposition or Cholesky factorization is a decomposition of a Hermitian, positive-definite matrix into the product of a lower triangular matrix and its conjugate transpose. The Cholesky decomposition is roughly twice as efficient as the LU decomposition for solving systems of linear equations. The cholesky decomposition of a Hermitian positive-definite matrix Σ_X is a decomposition of the form

$$\Sigma_X = AA^T$$

where A is a real lower triangular matrix with positive diagonal entries, and A^T denites the conjugate transpose of A. Every Hermitian positive-definite matrix (and thus also every real-valued symmetric positive-definite matrix) has a unique Cholesky decomposition.

$$\begin{bmatrix} \Sigma_{X_{11}} & \Sigma_{X_{12}} & \Sigma_{X_{13}} \\ \Sigma_{X_{21}} & \Sigma_{X_{22}} & \Sigma_{X_{23}} \\ \Sigma_{X_{31}} & \Sigma_{X_{32}} & \Sigma_{X_{33}} \end{bmatrix} = \begin{bmatrix} A_{11} & 0 & 0 \\ A_{21} & A_{22} & 0 \\ A_{31} & A_{32} & A_{33} \end{bmatrix} \begin{bmatrix} A_{11} & A_{21} & A_{31} \\ 0 & A_{22} & A_{32} \\ 0 & 0 & A_{33} \end{bmatrix}$$

Every symmetric, positive definite matrix Σ_X can be decomposed into a product of a unique lower triangular matrix L and its transpose.

Definition 6.55: Cholesky Decomposition Algorithm

The following formulas are obtained by solving above lower triangular matrix and its transpose. These are the basis of Cholesky decomposition algorithm:

$$A_{ii} = \sqrt{\Sigma_{X_{ii}} - \sum_{k=0}^{i-1} (A_{ik})^2}$$

v. Singular Value Decomposition

[SI*] This is an extension of the diagonalization theory for $n \times n$ symmetric matrices to general $m \times n$ matrices.

The results have applications to compression, storage, and transmission of digitized information and form the basis for many of the best computational algorithms that are currently available for solving linear systems.

[SI*] Every symmetric matrix A can be expressed as

$$A = PDP^T \tag{6.163}$$

where P is an $n \times n$ orthogonal matrix of eigenvectors of A, and D is the diagonal matrix whose diagonal entries are the eigenvalues corresponding to the column vectors of P. We will call (6.163) an eigenvalue decomposition of A.

[SI*] If an $n \times n$ matrix A is not symmetric, then it does not have an eigenvalue decomposition, but it does have a Hessenberg decomposition

$$A = PHP^T$$

in which P is an orthogonal matrix and H is in upper Hessenberg form. Moreover, if A has real eigenvalues, then it has a Schur decomposition

$$A = PSP^T$$

in which P is an orthogonal matrix and S is upper triangular.

[SI*] The eigenvalue, Hessenberg, and Schur decompositions are important in numerical algorithms not only because the matrices D, H, and S have simpler forms than A, but also because the orthogonal matrices that appear in these factorizations do not magnify roundoff error. To see why this is so, suppose $\hat{\boldsymbol{x}}$ is a column vector whose entries are known exactly and that

$$\boldsymbol{x} = \hat{\boldsymbol{x}} + \boldsymbol{e}$$

is the vector that results when roundoff error is present in the entries of $\hat{\boldsymbol{x}}$. If P is an orthogonal matrix, then the length-preserving property of orthogonal transformations implies that

$$||P\boldsymbol{x} - P\hat{\boldsymbol{x}} = ||\boldsymbol{x} - \hat{\boldsymbol{x}}|| = ||\boldsymbol{e}||$$

which tells us that the error in approximating $P\hat{\boldsymbol{x}}$ by $P\boldsymbol{x}$ has the same magnitude as the error in approximating $\hat{\boldsymbol{x}}$ by $\boldsymbol{x}$.

[SI*] There are two main paths that one might follow in looking for other kinds of decompositions of a general square matrix A: One might look for decompositions of the form

$$A = PJP^{-1}$$

in which P is invertible but not necessarily orthogonal, or one might look for decompositions of the form

$$A = U\sum V^T$$

in which U and V are orthogonal but not necessarily the same. The first path leads to decompositions in which J is either diagonal or a certain kind of block diagonal matrix, called a Jordan canonical form in honor of the French mathematician Camille Jordan. Jordan canonical forms are important theoretically and in certain applications, but they are of lesser importance numerically because of the roundoff difficulties that result from the lack of orthogonality in P.

Theorem 6.103: Matrices A and A^TA

If A is an $m \times n$ matrix, then:

(a) A and A^TA have the same null space.

(b) A and A^TA have the same row space.

(c) A^T and A^TA have the same column space.

(d) A and A^TA have the same rank.

Definition 6.56: Singular Value of A

If A is an $m \times n$ matrix, and if $\lambda_1, \lambda_2, \ldots, \lambda_n$ are the eigenvalues of $A^T A$, then the numbers

$$\sigma_1 = \sqrt{\lambda_1}, \quad \sigma_2 = \sqrt{\lambda_2}, \ldots \quad \sigma_n = \sqrt{\lambda_n}$$

are called the singular values of A.

Example:
Find the singular values of the matrix

$$\begin{bmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}$$

Solution:
The first step is to find the eigenvalues of the matrix

$$A^T A = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}$$

The characteristic polynomial of $A^T A$ is

$$\lambda^2 - 4\lambda + 3 = (\lambda - 3)(\lambda - 1)$$

so the eigenvalues of $A^T A$ are

$$\lambda_1 = 3$$
$$\lambda_2 = 1$$

and the singular values of A in order of decreasing size are

$$\sigma_1 = \sqrt{\lambda_1} = \sqrt{3}$$
$$\sigma_2 = \sqrt{\lambda_2} = 1$$

[SI*] We will find it useful to extend the notion of a "main diagonal" to matrices that are not square. We define the main diagonal of an $m \times n$ matrix to be the line of entries that starts at the upper left corner and extends diagonally as far as it can go. We will refer to the entries on the main diagonal as the diagonal entries.

[SI*] Factorization of a general $m \times n$ matrix A is called singular value decomposition (abbreviated SVD).

Theorem 6.104: Singular Value Decomposition

If A is an $m \times n$ matrix, then A can be expressed in the form

$$A = U \sum V^T$$

where U and V are orthogonal matrices and $\sum$ is an $m \times n$ matrix whose diagonal entries are the singular values of A and whose other entries are zero.

Theorem 6.105: Singular Value Decomposition (Expanded Form)

If A is an $m \times n$ matrix of rank k, then A can be factored as

$$A = U\sum V^T = \begin{bmatrix} \boldsymbol{u}_1 & \boldsymbol{u}_2 & \dots & \boldsymbol{u}_k & | & \boldsymbol{u}_{k+1} & \dots & \boldsymbol{u}_m \end{bmatrix} \begin{bmatrix} \sigma_1 & 0 & \dots & 0 & | \\ 0 & \sigma_2 & \dots & 0 & | \\ \vdots & \vdots & \ddots & \vdots & | \\ 0 & 0 & \dots & \sigma_k & | \\ & & & & \end{bmatrix}$$

in which $U, \sum$, and V have sizes $m \times m$, $m \times n$, and $n \times n$, respectively, and in which

(a) $V = [\boldsymbol{v}_1\ \boldsymbol{v}_2\ \dots\ \boldsymbol{v}_n]$ orthogonally diagonalizes A^TA.

(b) The nonzero diagonal entries of $\sum$ are $\sigma_1 = \sqrt{\lambda_1}$, $\sigma_2 = \sqrt{\lambda_2}, \dots, \sigma_k = \sqrt{\lambda_k}$, where $\lambda_1, \lambda_2, \dots, \lambda_k$ are the nonzero eigenvalues of A^TA corresponding to the column vectors of V.

(c) The column vector of V are ordered so that $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_k > 0$.

(d) $\boldsymbol{u}_i = \frac{A\boldsymbol{v}_i}{||A\boldsymbol{v}_i||} = \frac{1}{\sigma_i}A\boldsymbol{v}_i \quad (i = 1, 2, \dots, k)$

(e) $\{\boldsymbol{u}_1, \boldsymbol{u}_2, \dots, \boldsymbol{u}_k\}$ is an orthonormal basis for $\text{col}(A)$.

(f) $\{\boldsymbol{u}_1, \boldsymbol{u}_2, \dots, \boldsymbol{u}_k, \boldsymbol{u}_{k+1}, \dots, \boldsymbol{u}_m\}$ is an extension of $\{\boldsymbol{u}_1, \boldsymbol{u}_2, \dots, \boldsymbol{u}_k\}$ to an orthonormal basis for $\mathbb{R}^m$.

vi. Compute Singular Value Decomposition with SymIntegration

Find a singular value decomposition of the matrix

$$A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}$$

Solution:
We will have

$$A^T A = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}$$

The characteristic polynomial of $A^T A$ is

$$\lambda^2 - 4\lambda + 3 = (\lambda - 3)(\lambda - 1)$$

so the eigenvalues of $A^T A$ are

$$\lambda_1 = 3$$
$$\lambda_2 = 1$$

and the corresponding singular values of A in order of decreasing size are

$$\sigma_1 = \sqrt{\lambda_1} = \sqrt{3}$$
$$\sigma_2 = \sqrt{\lambda_2} = 1$$

the eigenvectors corresponding to λ_1 and λ_2 are

$$\boldsymbol{v}_1 = \begin{bmatrix} \frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} \end{bmatrix}, \quad \boldsymbol{v}_2 = \begin{bmatrix} \frac{\sqrt{2}}{2} \\ -\frac{\sqrt{2}}{2} \end{bmatrix}$$

we know that $V = [\boldsymbol{v}_1 | \boldsymbol{v}_2]$ orthogonally diagonalizes $A^T A$. Thus, the vectors

$$\boldsymbol{u}_1 = \frac{1}{\sigma_1} A \boldsymbol{v}_1 = \frac{\sqrt{3}}{3} \begin{bmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} \frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} \end{bmatrix} = \begin{bmatrix} \frac{\sqrt{6}}{3} \\ \frac{\sqrt{6}}{6} \\ \frac{\sqrt{6}}{6} \end{bmatrix}$$

$$\boldsymbol{u}_2 = \frac{1}{\sigma_1} A \boldsymbol{v}_2 = (1) \begin{bmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} \frac{\sqrt{2}}{2} \\ -\frac{\sqrt{2}}{2} \end{bmatrix} = \begin{bmatrix} 0 \\ -\frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} \end{bmatrix}$$

are two of the three column vectors of U. Note that $\boldsymbol{u}_1$ and $\boldsymbol{u}_2$ are orthonormal, as expected.

We could extend the set $\{\boldsymbol{u}_1, \boldsymbol{u}_2\}$ to an orthonormal basis for $\mathbb{R}^3$. However, the computations will be easier if we first remove the messy radicals by multiplying $\boldsymbol{u}_1$ and $\boldsymbol{u}_2$ by appropriate scalars. Thus, we will look for a unit vector $\boldsymbol{u}_3$ that is orthogonal to

$$\sqrt{6}\boldsymbol{u}_1 = \begin{bmatrix} 2 \\ 1 \\ 1 \end{bmatrix}, \quad \sqrt{2}\boldsymbol{u}_2 = \begin{bmatrix} 0 \\ -1 \\ 1 \end{bmatrix}$$

to satisfy these two orthogonality conditions, the vector $\boldsymbol{u}_3$ must be a solution of the homogeneous linear system

$$\begin{bmatrix} 2 & 1 & 1 \\ 0 & -1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

the general solution of this system is

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = t \begin{bmatrix} -1 \\ 1 \\ 1 \end{bmatrix}$$

Normalizing the vector on the right yields

$$\boldsymbol{u}_3 = \begin{bmatrix} -\frac{1}{\sqrt{3}} \\ \frac{1}{\sqrt{3}} \\ \frac{1}{\sqrt{3}} \end{bmatrix}$$

Thus, the singular value decomposition of A is

$$A = U \sum V^T$$

$$\begin{bmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} \frac{\sqrt{6}}{3} & 0 & -\frac{1}{\sqrt{3}} \\ \frac{\sqrt{6}}{6} & -\frac{\sqrt{2}}{2} & \frac{1}{\sqrt{3}} \\ \frac{\sqrt{6}}{6} & \frac{\sqrt{2}}{2} & \frac{1}{\sqrt{3}} \end{bmatrix} \begin{bmatrix} \sqrt{3} & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} \end{bmatrix}$$

In SymIntegration, to compute singular value decomposition (SVD) we can use this function: **SVD(vector<vector<double>>& A)**

with A as the matrix that we want to apply the SVD, SVD relies heavily on eigenvalues and eigenvectors computation for all the U, Σ, V^T matrices.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration SVD ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration SVD ]# ./main

A:
               1.000000               1.000000
               0.000000               1.000000
               1.000000               0.000000

A^{T}A:
               2.000000               1.000000
               1.000000               2.000000

U:
               0.816497               0.000000              -0.577350
               0.408248               0.707107               0.577350
               0.408248              -0.707107               0.577350

Σ:
               1.732051               0.000000
               0.000000               1.000000
               0.000000               0.000000

V^{T}:
               0.707107               0.707107
              -0.707107               0.707107

UΣV^{T}:
               1.000000               1.000000
              -0.000000               1.000000
               1.000000               0.000000

Time taken by function: 3312 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration SVD ]#
```

Figure 6.64: *The computation to obtain the singular value decomposition of matrix A. (**SymIntegration/Examples/-Linear Algebra/Test SymIntegration SVD/main.cpp**).*

Chapter 7

Numerical Analysis

Je n'avais pas peur, je suis né pour faire ça - Jeanne d'Arc

Numerical analysis is the study of algorithms that use numerical approximation (as opposed to symbolic manipulations) for the problems of mathematical analysis (as distinguished from discrete mathematics). It is the study of numerical methods that attempt at finding approximate solutions of problems rather than the exact ones. Numerical analysis finds application in all fields of engineering and the physical sciences, and in the 21st century also the life and social sciences, medicine, business and even the arts.

Modern numerical analysis begins by categorizing equations as belonging to different types and then devises methods of solution for generic problems of each type.

I. Solutions of Equations in One Variable (Root Finding)

[SI*] **Root Finding Methods**
In this section, we consider one of the most basic problems of numerical approximation, the root-finding problem. This process involves finding a root, or solution of an equation of the form

$$f(x) = 0$$

for a given function f.

Definition 7.1: Bisection Method

The Bisection Method is based on the Intermediate Value Theorem. Suppose f is a continuous function defined on the interval $[a, b]$ with $f(a)$ and $f(b)$ of opposite sign. By the Intermediate Value Theorem, there exists a number p in (a, b) with

$$f(p) = 0$$

We assume for simplicity that the root in the interval $[a, b]$ is unique. This method calls for a repeated halving of subintervals of $[a, b]$.

- The bisection method is slow to converge, N may become quite large before $|p - p_N|$ is sufficiently small, and a good intermediate approximation can be inadvertently discarded.
- The bisectionmethod has the important property that it always converges to a solution.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Bisection Method with Function ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Bisection Method with Function ]# ./main
f = -cos(θ0)-2*θ0+2*sin(θ0)+1
iteration             a                     b                     p                     f(p)
     1                1                     2                     1.5                   -0.0757472

     2                1                     1.5                   1.25                  0.0826469

     3                1.25                  1.5                   1.375                 0.0172384

     4                1.375                 1.5                   1.4375                -0.0256436

     5                1.375                 1.4375                1.40625               -0.00331926

     6                1.375                 1.40625               1.39062               0.00717788

     7                1.39062               1.40625               1.39844               0.00198421

     8                1.39844               1.40625               1.40234               -0.000653763

     9                1.39844               1.40234               1.40039               0.000668658

    10                1.40039               1.40234               1.40137               8.30682e-06

    11                1.40137               1.40234               1.40186               -0.000322513

    12                1.40137               1.40186               1.40161               -0.00015705

    13                1.40137               1.40161               1.40149               -7.43579e-05
Procedure completed successfully
solution = 1.40149

Time taken by function: 30471 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Bisection Method with Function ]#
```

Figure 7.1: *The computation for bisection method for $f(\theta_0) = 2\theta_0 - 2\sin\ \theta_0 + \cos\ \theta_0 - 1$ with function (**SymIntegration/Examples/Numerical Methods/Test SymIntegration Bisection Method with Function for Brachistochrone/main.cpp**).*

The computation is using SymIntegration, with this function: **bisectionmethod(**f**,** x**,** a**,** b**,**N**)** that can be called to compute the the solution of a function with one variable, this method doesn't need to compute the derivative so it can be used to almost all functions.

As a comparison we try to measure the time if we apply the bisection method directly in the main source code, turns out it is faster than using the function.

```
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Bisection Method ]# ./main
f = -cos(@0)-2*@0+2*sin(@0)+1

a = 1
b = 2
p = 1.5
f(p) = -0.0757472
f(b) = -0.765258
f(a) = 0.14264
f(a)*f(p) =-0.0108046
iteration               a                       b                       p                       f(p)
     1                  1                       2                       1.5                     -0.0757472

     2                  1                       1.5                     1.25                    0.0826469

     3                  1.25                    1.5                     1.375                   0.0172384

     4                  1.375                   1.5                     1.4375                  -0.0256436

     5                  1.375                   1.4375                  1.40625                 -0.00331926

     6                  1.375                   1.40625                 1.39062                 0.00717788

     7                  1.39062                 1.40625                 1.39844                 0.00198421

     8                  1.39844                 1.40625                 1.40234                 -0.000653763

     9                  1.39844                 1.40234                 1.40039                 0.000668658

    10                  1.40039                 1.40234                 1.40137                 8.30682e-06

    11                  1.40137                 1.40234                 1.40186                 -0.000322513

    12                  1.40137                 1.40186                 1.40161                 -0.00015705

    13                  1.40137                 1.40161                 1.40149                 -7.43579e-05
Procedure completed successfully

Time taken by function: 28906 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Bisection Method ]#
```

Figure 7.2: *The computation for bisection method for* $f(\theta_0) = 2\theta_0 - 2\sin\ \theta_0 + \cos\ \ theta_0 - 1$ *without function (**SymIntegration/Examples/Numerical Methods/Test SymIntegration Bisection Method Manual for Brachistochrone/main.cpp**).*

Definition 7.2: Newton-Raphson Method

The Newton-Raphson Method is one of the most powerful and well known numerical methods for solving a root-finding problem.
Suppose that $f \in \mathbb{C}^2[a,b]$. Let $\bar{x} \in [a,b]$ be an approximate to p such that $f'(\bar{x}) \neq 0$ and $|p - \bar{x}|$ is "small." Consider the first Taylor polynomial for $f(x)$ expanded about $(\bar{x})$,

$$f(x) = f(\bar{x}) + (x - \bar{x})f'(\bar{x}) + \frac{(x-\bar{x})^2}{2}f''(\xi(x))$$

where $\xi(x)$ lies between x and $\bar{x}$. Since $f(p) = 0$, this equation with $x = p$ gives

$$0 = f(\bar{x}) + (p - \bar{x})f'(\bar{x}) + \frac{(p-\bar{x})^2}{2}f''(\xi(p))$$

Newton's method is derived by assuming that since $|p - \bar{x}|$ is small, the term involving $(p - \bar{x})^2$ is much smaller, so

$$0 \approx f(\bar{x}) + (p - \bar{x})f'(\bar{x})$$

Solving for p gives

$$p \approx \bar{x} - \frac{f(\bar{x})}{f'(\bar{x})}$$

Thus, the Newton-Raphson method, which starts with an initial approximation p_0 and generate the sequence $\{p_n\}_{n=0}^{\infty}$, is

$$p_n = p_{n-1} - \frac{f(p_{n-1})}{f'(p_{n-1})}, \quad \text{for } n \geq 1 \tag{7.1}$$

Starting with the initial approximation p_0, the approximation p_1 is the x-intercept of the tangent line to the graph of f at $(p_0, f(p_0))$. The approximation p_2 is the x-intercept of the tangent line to the graph of f at $(p_1, f(p_1))$ and so on.

- Newton-Raphson method is a functional iteration technique of the form

  $$p_n = g(p_{n-1})$$

  for which

  $$g(p_{n-1}) = p_{n-1} - \frac{f(p_{n-1})}{f'(p_{n-1})}, \quad \text{for } n \geq 1 \tag{7.2}$$

  this is the functional iteration technique that was used to give the rapid convergence.
- It is clear that Newton-Raphson method cannot be continued if $f'(p_{n-1}) = 0$ for some n. This method is the most effective when f' is bounded away from zero near p.

The computation is using SymIntegration, with this function: **newtonmethod(**f**,**x**,**x_0**,**N**)** that can be called to compute the the solution of a function with one variable, given that the derivative is continuous and exist.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Newton's Method with function ]# make
g++     -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Newton's Method with function ]# ./main
Newton method :
f(x) = cos(x)-x
f'(x) = -sin(x)-1

     n          p_{n}
     0          0.78539816339742
     1          0.73953613351524
     2          0.73908517810601
     3          0.73908513321516
The procedure was successful.
solution =
0.73908513321516
Time taken by function: 5645 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Newton's Method with function ]#
```

Figure 7.3: *The computation for root finding Newton-Raphson method for $f(x) = \cos(x) - x$. with function (**SymIntegration/Examples/Numerical Methods/Test SymIntegration Newton's Method with function/main.cpp**).*

As a comparison we try to implement Newton-Raphson manually, in the main source code, it turns out calling the function resulting in faster speed.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Newton's Method Manual ]# make
g++     -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Newton's Method Manual ]# ./main
f(x) = cos(x)-x
f(x)[x==pi/4] = -0.0782914

f'(x) = -sin(x)-1
f'(x)[x==pi/4] = -1.70711

     n          p_{n}                          p_{n} in Degree
     0          0.78539816339742               44.999999999999
     1          0.73953613351524               42.372299247846
     2          0.73908517810601               42.346461406149
     3          0.73908513321516               42.346458834093
The procedure was successful.
Time taken by function: 6779 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Newton's Method Manual ]#
```

Figure 7.4: *The manual computation for root finding Newton-Raphson method for $f(x) = \cos(x) - x$. (**SymIntegration/Examples/Numerical Methods/Test SymIntegration Newton's Method Manual/main.cpp**).*

[SI*] Root-finding algorithms for higher order polynomial are Durand-Kerner and Abert-Ehrlich, they are iterative numerical algorithms used to simultaneously find all roots (complex or real) of a polynomial, avoiding the need for deflation techniques. Both require initial complex approximations and offer high efficiency, with Aberth generally converging faster due to its cubic convergence compared to DK' quadratic rate.

Both are efficient at finding all roots of high-degree polynomials simultaneously. But can be sensitive to initial guess (bad guesses can fail to converge) and can be slow for repeated roots.

Definition 7.3: Durand-Kerner Method

The Durand-Kerner method, also known as the Weierstrass method, this algorithm updates all root approximations simultaneously in each iteration.

Durand-Kerner algorithm steps:

1. **Initialize**
   Given a polynomial $P(z)$ of degreen n, choose n distinct initial approximations for the roots: $z_1^{(0)}, z_2^{(0)}, \ldots, z_n^{(0)}$ (typically chosen as complex numbers on a circle).

2. **Iteration**
   For each root $i \in \{1, \ldots, n\}$, calculate the update formula:

$$z_i^{(k+1)} = z_i^{(k)} - \frac{P(z_i^{(k)})}{\prod_{j=1, j\neq i}^{n} \left(z_i^{(k)} - z_j^{(k)}\right)}$$

3. **Terminate**
   Stop when the updates are smaler than the desired tolerance (converges quadratically).

Definition 7.4: Aberth-Ehrlich Method

The Aberth method improves upon Durand-Kerner, it uses a more complex Newton-like formula incorporating derivative $P'(z)$ and a corrective shift to achieve higher efficiency .

Aberth-Ehrlich algorithm steps:

1. **Initialize**
   Choose n distinct initial approximations for the roots: $z_1^{(0)}, z_2^{(0)}, \ldots, z_n^{(0)}$.

2. **Iteration**
   For each root $i \in \{1, \ldots, n\}$, calculate the Newton step and the shift component.
   Newton step:

$$N(z_i) = \frac{P(z_i)}{P'(z_i)}$$

   Shift:

$$S(z_i) = \sum_{j=1, j\neq i}^{n} \frac{1}{z_i - z_j}$$

   Update:

$$z_i^{(new)} = z_i^{(old)} - \frac{N(z_i)}{1 - N(z_i)S(z_i)}$$

3. **Terminate**
   Stop when the updates are within tolerance (converges cubically).

i. Functions in SymIntegration Related to Root Finding

[SI*] We will list the basic functions that can be used for root finding:

AberthEhrlich(const vector<complex<double>> &$\vec{P}$, const vector<complex<double>> &x_0, int N)
to compute the root of higher order polynomial P with Aberth-Ehrlich formula, P is a vector of size n with complex entry, the configuration is like this:

$$\vec{P} = \begin{bmatrix} (a_1, b_1) \\ (a_2, b_2) \\ \vdots \\ (a_n, b_n) \end{bmatrix}$$

$$\text{polynomial}(\vec{P}) = (a_1, b_1)r^{n-1} + (a_2, b_2)r^{n-2} + \cdots + (a_1, b_1)$$

so if we want to compute a polynomial of order 6, then $\vec{P}$ has to be a size of 7. N is the number of maximum iteration for the Abert-Ehrlich method, and x_0 is the initial guess / approximation vector with complex entry and has to be a size of $n-1$, the entry of x_0 is preferable to be distinct and in decimal (e.g. 1.23 or 2.55), not round number to avoid error due to rounding.

This function is written in **src/numericalmethods.cpp**.

II. Interpolation and Polynomial Approximation

i. Interpolation and the Lagrange Polynomial

[SI*] **Interpolation and the Lagrange Polynomial**
Taylor polynomials are not appropriate for interpolation, because the approximations are needed only at points close to x_0. For ordinary computational purposes it is more efficient to use methods that include information at various points. To generalize the concept of linear interpolation, consider construction of a polynomial of degree at most n that passes through $n+1$ points

$$(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n)$$

to approximate that $n+1$ points, we will use the well-established Lagrange Interpolating Polynomial.

Theorem 7.1: nth Lagrange Interpolating Polynomial

If $x_0, x_1, \ldots, x_n$ are $n+1$ distinct numbers and f is a function whose values are given at these numbers, then a unique polynomial $P(x)$ of degreen at most n exists with

$$f(x_k) = P(x_k)$$

for each $k = 0, 1, \ldots, n$ This polynomial is given by

$$P(x) = f(x_0)L_{n,0}(x) + \cdots + f(x_n)L_{n,n}(x) = \sum_{k=0}^{n} f(x_k)L_{n,k}(x) \tag{7.3}$$

where, for each $k = 0, 1, \ldots, n$,

$$\begin{aligned} L_{n,k}(x) &= \frac{(x-x_0)(x-x_1)\ldots(x-x_{k-1})(x-x_{k+1})\ldots(x-x_n)}{(x_k-x_0)(x_k-x_1)\ldots(x_k-x_{k-1})(x_k-x_{k+1})(x_k-x_n)} \\ &= \prod_{i=0,\ i\neq k}^{n} \frac{(x-x_i)}{(x_k-x_i)} \end{aligned} \tag{7.4}$$

Theorem 7.2: Approximating a Function by an Interpolating Polynomial

Suppose $x_0, x_1, \ldots, x_n$ are distinct numbers in the interval $[a, b]$ and $f \in C^{n+1}[a, b]$. Then, for each x in $[a, b]$, a number $\xi(x)$ in (a, b) exists with

$$f(x) = P(x) + \frac{f^{(n+1)(\xi(x))}}{(n+1)!}(x-x_0)(x-x_1)\ldots(x-x_n) \tag{7.5}$$

where $P(x)$ is the interpolating polynomial given in Eq. (7.3).

[SI*] **Example:**
The example is taken from example 1, chapter 3.1 of [8].
Use the numbers (or nodes) $x_0 = 2, x_1 = 2.5$, and $x_2 = 4$ to find the second interpolating polynomial for $f(x) = \frac{1}{x}$.

Solution:
We need to determine the coefficient polynomials $L_0(x), L_1(x)$, and $L_2(x)$. In nested form they are

$$\begin{aligned} L_0(x) &= \frac{(x-2.5)(x-4)}{(2-2.5)(2-4)} = (x-6.5)x + 10 \\ L_1(x) &= \frac{(x-2)(x-4)}{(2.5-2)(2-4)} = \frac{(-4x+24)x - 32}{3} \\ L_2(x) &= \frac{(x-2)(x-2.5}{(4-2)(4-2.5)} = \frac{(x-4.5)x + 5}{3} \end{aligned}$$

Since

$$\begin{aligned} f(x_0) &= f(2) = 0.5 \\ f(x_1) &= f(2.5) = 0.4 \\ f(x_2) &= f(4) = 0.25 \end{aligned}$$

we will have

$$\begin{aligned} P(x) &= \sum_{k=0}^{2} f(x_k) L_k(x) \\ &= 0.5\left((x-6.5)x+10\right) + 0.4\left(\frac{(-4x+24)x-32}{3}\right) + 0.25\left(\frac{(x-4.5)x+5}{3}\right) \\ &= (0.05x - 0.425)x + 1.15 \end{aligned}$$

We can now check with $x = 3$

$$\begin{aligned} f(3) &= \frac{1}{3} \approx 0.33333 \\ P(3) &= 0.325 \end{aligned}$$

it is quite a good approximation.

The C++ code is using SymIntegration and Armadillo to help us compute the polynomial Lagrange $P(x)$, we can input the initial nodes $x_0, x_1, \ldots, x_N$ from textfile **vectorX.txt**, and then the function $f(x)$ can be inputted in the main source code symbolically.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Lagrange Interpolating Polynomial with Armadillo ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -larmadillo -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Lagrange Interpolating Polynomial with Armadillo ]# ./main
Vector x:
   2.0000
   2.5000
   4.0000

f(x) = x^(-1)

Vector f:
   0.5000
   0.4000
   0.2500

L :
[       x^(2)-6.5*x+10       ]
[ -1.33333*x^(2)+8*x-10.6667 ]
[0.333333*x^(2)-1.5*x+1.66667]


P(x) = 0.05*x^(2)-0.425*x+1.15
P(3) = 0.325
f(3) = 1/3

Time taken by function: 39599 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Lagrange Interpolating Polynomial with Armadillo ]#
```

Figure 7.5: *The computation to approximate $f(x) = \frac{1}{x}$ with nodes $x_0 = 2, x_1 = 2.5$, and $x_2 = 4$ using Lagrange interpolating polynomial with SymIntegration and Armadillo (**SymIntegration/Examples/Numerical Methods/Test SymIntegration Lagrange Interpolating Polynomial with Armadillo/main.cpp**).*

Another C++ code that we create is using SymIntegration only, after doing work on linear algebra we add more functions so it can have better matrix and vector operations. The result is amazing, the speed almost twice faster than when we are using Armadillo.

```
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Numerical Methods/Test SymIntegration Lag
range Interpolating Polynomial ]# ./main
Vector x:
2
2.5
4

f(x) = x^(-1)

Vector f:
0.5
0.4
0.25

L :
[       x^(2)-6.5*x+10       ]
[ -1.33333*x^(2)+8*x-10.6667 ]
[0.333333*x^(2)-1.5*x+1.66667]


P(x) = 0.05*x^(2)-0.425*x+1.15
P(3) = 0.325
f(3) = 1/3

Time taken by function: 28588 microseconds
```

Figure 7.6: *The computation to approximate* $f(x) = \frac{1}{x}$ *with nodes* $x_0 = 2, x_1 = 2.5$, *and* $x_2 = 4$ *using Lagrange interpolating polynomial with SymIntegration only(**SymIntegration/Examples/Numerical Methods/Test SymIntegration Lagrange Interpolating Polynomial/main.cpp**).*

ii. Divided Differences

[SI*]

iii. Hermite Interpolation

[SI*]

iv. Cubic Spline Interpolation

[SI*]

III. Numerical Methods for Optimization

[SI*] **Minimizing a single-variable function**

[SI*] The numerical methods for minimizing a single-variable function can be broadly categorized into interval reduction methods and descent methods.

Interval reduction methods, like Golden Section search, iteratively narrow down an interval known to contain the minimum, while descent methods, such as gradient descent, use derivative information to iteratively move towards the minimum.

[SI*] The interval reduction methods:

1. **Golden Section Search**
   This method is a bracketing method that works by iteratively narrowing down the interval containing the minimum. It uses the golden ratio to determine two points within the interval, evaluates the function at these points, and then discards a portion of the interval based on the function values. This process is repeated until the interval is sufficiently small.
2. **Fibonacci Search**
   Similar to Golden Section Search, Fibonacci Search uses the Fibonacci sequence to

determine the points at which to evaluate the function. It offers optimal performance for a fixed number of function evaluations.

[SI*] The descent methods:

1. **Gradient Descent**
   This method uses the gradient (derivative) of the function to determine the direction of steepest descent and iteratively moves towards the minimum. At each step, the algorithm moves in the opposite direction of the gradient.
2. **Newton's Method**
   This method uses both the first and second derivatives (gradient and Hessian) to find the minimum. It offers faster convergence than gradient descent but requires the calculation of the Hessian.

   Many algorithms for geometry optimization are based on some variant of the Newton-Raphson (NR) scheme. In order to find the minima, the first derivative $f'(x) = 0$ (must be zero) and the second derivative $f''(x) > 0$ (must be positive). To find the maxima, the first derivative $f'(x) = 0$ (must be zero) and the second derivative $f''(x) < 0$ (must be negative).

[SI*] The other methods:

1. **Brent's Method**
   A hybrid method that combines the bracketing approach of Golden Section Search with the local quadratic approximation of Newton's method, offering a good balance of robustness and efficiency.
2. **Downhill Simplex Method (Nelder-Mead)**
   A direct search method that does not require derivative information. It works by iteratively moving and transforming a simplex (a geometric shape with $n + 1$ vertices in n-dimensional space) in the search space to find the minimum.

[SI*] The best method for minimizing a single-variable function depends on the specific characteristics of the function and the desired level of accuracy. For well-behaved functions, gradient-based methods like Newton's method can offer fast convergence. For functions whose derivatives are difficult or impossible to compute, direct search methods like Brent's method or the downhill simplex method may be more suitable.

[SI*] **Gradient Descent**
Gradient Descent (GD) is an iterative first-order optimisation algorithm, used to find a local maximum/minimum of a given function. This method is commonly used in machine learning (ML) and deep learning (DL) to minimize a cost/loss function (e.g. in a linear regression). Due to its importance and ease of implementation, this algorithm is usually taught at the beginning of almost all machine learning courses. It is also widely used in areas like control engineering (robotics, chemical), computer games, mechanical engineering.

- This method was proposed long before the era of modern computers by Augustin-Louis Cauchy in 1847.
- Gradient descent algorithm does not work for all functions. There are two specific requirements. A function has to be:
  1. Differentiable
     Differentiable means a function has a derivative for each point in its domain. Typical non-differentiable functions have a step or a cusp or a discontinuity, such as $f(x) = \frac{1}{x}, f(x) = \sqrt{|x|}$

2. Convex
   For a univariate function, this means that the line segment connecting two function's points lays on or above its curve (it does not cross it). If it does, then it means that it has a local minimum which is not a global one.
   In mathematics notation, for two points x_1 and x_2 laying on the function's curve this condition is expressed as:

$$f(\lambda x_1 + (1-\lambda)x_2) \leq \lambda f(x_1) + (1-\lambda)f(x_2) \tag{7.6}$$

   where λ denotes a point's location on a section line and its value has to be between 0 (left point) and 1 (right point), e.g. $\lambda = 0.5$ means a location in the middle.

   Another way to check mathematically if a univariate function is convex is to calculate the second derivative and check if its value is always bigger than 0.

$$\frac{d^2 f(x)}{dx^2} > 0$$

- A gradient of an n-dimensional function $f(x)$ at a given point p is defined as

$$\nabla f(p) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(p) \\ \frac{\partial f}{\partial x_2}(p) \\ \vdots \\ \frac{\partial f}{\partial x_n}(p) \end{bmatrix} \tag{7.7}$$

  The upside-down triangle is a notation called nabla, or you can also call it del operator.

- **Gradient Descent Algorithm**
  Gradient Descent Algorithm iteratively calculates the next point using gradient at the current position, scales it (by a learning rate) and subtracts obtained value from the current position (makes a step). It subtracts the value because we want to minimize the function (to maximize it would be adding). This process can be written as:

$$p_{n+1} = p_n - \gamma \nabla f(p_n) \tag{7.8}$$

  There's an important parameter γ (read: gamma) which scales the gradient and thus controls the step size. In machine learning, gamma is called the learning rate and have a strong influence on performance.

  The smaller the learning rate, the longer the GD converges, or may reach maximum iteration before reaching the optimum point.

  If the learning rate is too big the algorithm may not converge to the optimal point (jump around) or even diverge completely.

  The Gradient Descent method's steps are:
  1. Choose a starting point (initialisation).
  2. Calculate gradient at this point.
  3. Make a scaled step in the opposite direction to the gradient (objective: minimize).

4. Repeat steps number 2 and 3 until one of the criteria is met:
   - Maximum number of iterations reached.
   - Step size is smaller than the tolerance (due to scaling or a small gradient)

- **Example:**
  Find the global minimum of a function that is given by

$$f(x) = x^2 - x + 3$$

Solution:
We will compute the first and second derivative of $f(x)$.

$$\frac{df(x)}{dx} = 2x - 1$$
$$\frac{d^2f(x)}{dx^2} = 2$$

Because the second derivative is always bigger than 0, $f(x)$ is strictly convex.

Now we examine the first derivative and equate it to zero.

$$\frac{df(x)}{dx} = 0$$
$$2x - 1 = 0$$
$$x = 0.5$$

$x = 0.5$ is the critical point.

Now, we will check with the gradient descent method, first we will choose the initial value and the parameter γ (the learning rate):

$$x_{old} = 2, \quad \gamma = 0.1$$

then we will have

$$x_{new} = x_{old} - \gamma \nabla f(x_{old})$$
$$x_{new} = 2 - 0.1(2(2) - 1) = 1.7 \quad (i = 1)$$
$$x_{new} = 1.7 - 0.1(2(1.7) - 1) = 1.46 \quad (i = 2)$$
$$x_{new} = 1.46 - 0.1(2(1.46) - 1) = 1.268 \quad (i = 3)$$
$$\vdots$$
$$x_{new} = 0.500027 - 0.1(2(0.500027) - 1) = 0.500021 \quad (i = 50)$$

It is proven then with the gradient descent method that the critical point is at $x = 0.5$. The computation can be done with **SymIntegration**, as long as the function $f(x)$ is differentiable and the derivative can be computed then we can use gradient descent method. Polynomials without the combination of trigonometry and transcendentals functions can be differentiated easily, even the fraction polynomial.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Gradient Descent
 ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Gradient Descent
 ]# ./main
F(x) = x^(2)-x+3
i = 1
xnew  = 1.7
f(xnew)  = 4.19
i = 2
xnew  = 1.46
f(xnew)  = 3.6716
i = 3
xnew  = 1.268
f(xnew)  = 3.33982
i = 4
xnew  = 1.1144
f(xnew)  = 3.12749
i = 5
xnew  = 0.99152
f(xnew)  = 2.99159
i = 6
xnew  = 0.893216
f(xnew)  = 2.90462
i = 7
xnew  = 0.814573
f(xnew)  = 2.84896
```

Figure 7.7: *The computation for minimization problem for function $f(x) = x^2 - x + 3$ with gradient descent method. (**SymIntegration/Examples/Operations Research/Test SymIntegration Gradient Descent/-main.cpp**).*

```
i = 45
xnew  = 0.500065
f(xnew)  = 2.75
i = 46
xnew  = 0.500052
f(xnew)  = 2.75
i = 47
xnew  = 0.500042
f(xnew)  = 2.75
i = 48
xnew  = 0.500033
f(xnew)  = 2.75
i = 49
xnew  = 0.500027
f(xnew)  = 2.75
i = 50
xnew  = 0.500021
f(xnew)  = 2.75
The gradient descent converges.
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Gradient Descent ]#
```

Figure 7.8: *The gradient descent method took 50 iteration to converges with $\gamma = 0.1$. (**SymIntegration/Examples/-Operations Research/Test SymIntegration Gradient Descent/main.cpp**).*

- **Example:**
  Find the global minimum of a function that is given by

$$f(x,y) = \frac{1}{2}x^2 + 2x + y^2 + y + 3$$

  Solution:
  In the case of a multivariate function, we need to compute the vector of derivatives in each main direction (along variable axes). Because we are interested only in a slope along one axis and we don't care about others, these derivatives are called partial derivatives.

  We will have

$$\begin{aligned} z = f(x,y) &= \frac{1}{2}x^2 + 2x + y^2 + y + 3 \\ \frac{\partial f}{\partial x} &= x + 2 \\ \frac{\partial f}{\partial y} &= 2y + 1 \end{aligned}$$

then the nabla is

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix} = \begin{bmatrix} x+2 \\ 2y+1 \end{bmatrix}$$

at initialization we will input the initial value and the parameter γ

$$\begin{aligned} x_{old} &= -5 \\ y_{old} &= 21 \\ \gamma &= 0.1 \end{aligned}$$

thus the gradient descent method

$$\begin{aligned} x_{new} &= x_{old} - \gamma \nabla f(x_{old}) \\ x_{new} &= -5 - 0.1((-5)+2) = -4.7 \quad (i=1) \\ x_{new} &= -4.7 - 0.1((-4.7)+2) = -4.43 \quad (i=2) \\ x_{new} &= -4.43 - 0.1((-4.43)+2) = -4.187 \quad (i=3) \\ &\vdots \\ x_{new} &= -2.01718 - 0.1((-2.01718)+2) = -2.01546 \quad (i=50) \end{aligned}$$

$$\begin{aligned} y_{new} &= y_{old} - \gamma \nabla f(y_{old}) \\ y_{new} &= 21 - 0.1(2(21)+1) = 16.7 \quad (i=1) \\ y_{new} &= 16.7 - 0.1(2(16.7)+1) = 13.26 \quad (i=2) \\ y_{new} &= 13.26 - 0.1(2(13.26)+1) = 10.508 \quad (i=3) \\ &\vdots \\ y_{new} &= -0.499616 - 0.1(2(-0.499616)+1) = -0.499693 \quad (i=50) \end{aligned}$$

and the value for function $f(x,y)$ at every iteration

$$\begin{aligned} f(x_{new}, y_{new}) &= 300.235 \quad (i=1) \\ f(x_{new}, y_{new}) &= 193.04 \quad (i=2) \\ f(x_{new}, y_{new}) &= 124.318 \quad (i=3) \\ &\vdots \\ f(x_{new}, y_{new}) &= 0.75012 \quad (i=50) \end{aligned}$$

so the function $f(x,y) = \frac{1}{2}x^2 + 2x + y^2 + y + 3$ reaches the minimum of 0.75012 at $(-2.01546, -0.499693)$.

```
xterm
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Gradient Descent 2 Variables ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Gradient Descent 2 Variables ]# ./main
F(x,y) = 0.5*x^(2)+2*x+y^(2)+y+3
i = 1
xnew  = -4.7
ynew  = 16.7
F(xnew,ynew)  = 300.235
i = 2
xnew  = -4.43
ynew  = 13.26
F(xnew,ynew)  = 193.04
i = 3
xnew  = -4.187
ynew  = 10.508
F(xnew,ynew)  = 124.318
i = 4
xnew  = -3.9683
ynew  = 8.3064
F(xnew,ynew)  = 80.2398
i = 5
xnew  = -3.77147
ynew  = 6.54512
F(xnew,ynew)  = 51.9528
i = 6
xnew  = -3.59432
ynew  = 5.1361
F(xnew,ynew)  = 33.7865
i = 7
xnew  = -3.43489
ynew  = 4.00888
F(xnew,ynew)  = 22.1094
```

Figure 7.9: *The computation for minimization problem for function $f(x,y) = \frac{1}{2}x^2 + 2x + y^2 + y + 3$ with gradient descent method. (**SymIntegration/Examples/Operations Research/Test SymIntegration Gradient Descent 2 Variables/main.cpp**).*

```
i = 45
xnew  = -2.02618
ynew  = -0.499064
F(xnew,ynew)  = 0.750344
i = 46
xnew  = -2.02357
ynew  = -0.499251
F(xnew,ynew)  = 0.750278
i = 47
xnew  = -2.02121
ynew  = -0.499401
F(xnew,ynew)  = 0.750225
i = 48
xnew  = -2.01909
ynew  = -0.499521
F(xnew,ynew)  = 0.750182
i = 49
xnew  = -2.01718
ynew  = -0.499616
F(xnew,ynew)  = 0.750148
i = 50
xnew  = -2.01546
ynew  = -0.499693
F(xnew,ynew)  = 0.75012
Maximum number of iteration reached.
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Gradient Descent 2 Variables ]#
```

Figure 7.10: *The gradient descent method took 50 iteration to converges with $\gamma = 0.1$. (**SymIntegration/Examples/-Operations Research/Test SymIntegration Gradient Descent 2 Variables/main.cpp**).*

[SI*] **Downhill Simplex**

The downhill simplex, introduced by Nelder and Mead, also known as the Nelder-Mead' method [23], it is a method for solving multidimensional, unconstrained, numerical optimization problems. This type of problem is one of the most common in engineering and science, and this method is one of the simplest and fairly effective to solve the problem.

[SI*] We define a minimization problem as

$$\arg\ \min f(\boldsymbol{x}) \tag{7.9}$$

$$\boldsymbol{x} \in \mathbb{R}^N,\ f : \mathbb{R}^N \rightarrowtail \mathbb{R}^1$$

[SI*] **Method and simplex transforms**

A simplex is a collection of connected points in $\mathbb{R}^N$. We call it a k-simplex where k

is the degree that indicates how many points it contains. A k-simplex contains $k+1$ points, so a triangle which has 3 points is a 2-simplex,a tetrahedron which has 4 points is a 3-simplex and so on.

When optimizing a function with N parameters we will use a simplex with $N+1$ points. Meaning the degree k of the simplex is the same as the dimension of the parameter space. So for a problem with 2 parameters, $N = 2$, we use a 2-simplex, a triangle, that has three points. A 1 parameter problem would use a line segment in 1D and a 3 parameter problem would use a tetrahedron in 3D.

Each point on the simplex, denoted by $\boldsymbol{x}_i, 0 \leq i \leq N$, corresponds to a complete set of parameters and therefore to a value of the objective function $f_i = f(\boldsymbol{x}_i)$.

The core of the method is to keep the points sorted so that $f(\boldsymbol{x}_0)$ is the current lowest value, i.e. the best point, and $\boldsymbol{x}_N$ is the worst. We then calculate the center of mass of all points except $\boldsymbol{x}_N$:

$$\boldsymbol{c} = \frac{1}{N} \sum_{i=0}^{N-1} \boldsymbol{x}_i \tag{7.10}$$

Afterwards we attempt to replace the point $\boldsymbol{x}_N$ with one along the line formed by $\overrightarrow{\boldsymbol{x}_N \boldsymbol{c}}$ according to a set of transformations called reflection, expansion, and contraction. A shrink transform that moves all points closer to the current best can also be performed. We associate each transform with a parameter, α for reflection, β for contraction, γ for expansion and δ for shrink, that should satisfy the following constraints:

- $\alpha > 0$
- $0 < \beta < 1$
- $\gamma > \alpha$, $\gamma > 1$
- $0 < \delta < 1$

[SI*] **Reflection**
The following illustrations are a 2D problem with a triangle simplex (can be used for function with two independent variable such as $f(x, y)$). The point $\boldsymbol{x}_0$ with blue color is the current best, the point $\boldsymbol{x}_1$ with green color is the second best, and the point $\boldsymbol{x}_2$ with light red color is the third best / the one being transformed except for when the simplex performs a shrink and all but the best are changed. The blue lines indicate the current simplex and the light red lines are the new one.

For every iteration the reflected point $\boldsymbol{x}_r$ is the one tried first

$$\boldsymbol{x}_r = \boldsymbol{c} + \alpha(\boldsymbol{c} - \boldsymbol{x}_2) \tag{7.11}$$

If this point is better than the second best but not better than the current best such that

$$f(\boldsymbol{x}_0) \leq f(\boldsymbol{x}_r) < f(\boldsymbol{x}_1)$$

we replace $\boldsymbol{x}_2$ with $\boldsymbol{x}_r$, otherwise we do either expansion, contraction or shrink.

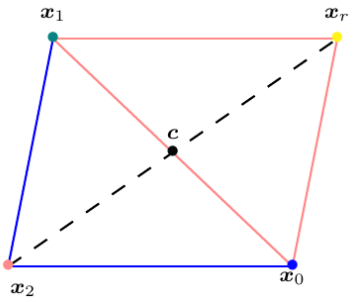

Figure 7.11: *x_2 is mirrored in c and creates a reflected point x_r.*

[SI*] **Expansion**
If x_r is better than the current best,

$$f(x_r) < f(x_0)$$

we continue to explore and compute the expansion point x_e

$$x_e = c + \gamma(x_r - c) \tag{7.12}$$

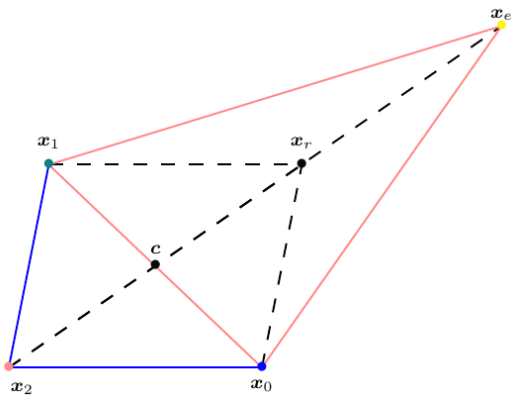

Figure 7.12: *The expansion point x_e is computed by moving further away from c.*

Then one of two approaches can be used:

- Greedy minimization
  in which the reflected and expanded points are compared and the better of the two is picked such that if

  $$f(x_e) < f(x_r) < f(x_0)$$

  we accept x_e ($x_2 = x_e$) and terminate the iteration.

  Otherwise, if

  $$f(x_r) \leq f(x_e)$$

  then x_r is accepted ($x_2 = x_r$).

- Greedy expansion
  we will accept $\boldsymbol{x}_e$ if
  $$f(\boldsymbol{x}_e) < f(\boldsymbol{x}_1)$$
  and
  $$f(\boldsymbol{x}_r) < f(\boldsymbol{x}_0)$$
  and skip comparing $f(\boldsymbol{x}_e)$ to $f(\boldsymbol{x}_r)$. This will keep the simplex as large as possible which can have some advantages for non-smooth functions.

[SI*] **Contraction**
If $\boldsymbol{x}_r$ is worse than the current second worst point, in 2D case it is $\boldsymbol{x}_1$, which means

$$f(\boldsymbol{x}_r) \geq f(\boldsymbol{x}_1)$$

we compute a contraction point using $\boldsymbol{x}_2$ and $\boldsymbol{x}_r$.

It will be

- Outside
  if
  $$f(\boldsymbol{x}_1) \leq f(\boldsymbol{x}_r) < f(\boldsymbol{x}_2)$$
  We compute the contraction point as
  $$\boldsymbol{x}_c^{out} = \boldsymbol{c} + \beta(\boldsymbol{x}_r - \boldsymbol{c}) \tag{7.13}$$
  and if
  $$f(\boldsymbol{x}_c^{out}) \leq f(\boldsymbol{x}_r)$$
  then $\boldsymbol{x}_c^{out}$ is accepted ($\boldsymbol{x}_2 = \boldsymbol{x}_c^{out}$). Otherwise, a shrink of the simplex is performed.

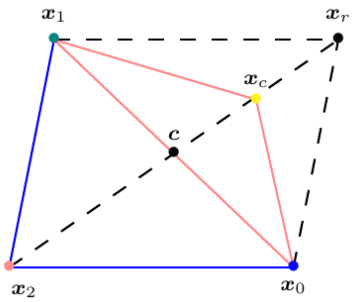

Figure 7.13: *The contraction point $\boldsymbol{x}_c^{out}$ is computed outside the old simplex.*

- Inside
  if
  $$f(\boldsymbol{x}_r) \geq f(\boldsymbol{x}_2)$$
  We compute the contraction point as
  $$\boldsymbol{x}_c^{in} = \boldsymbol{c} + \beta(\boldsymbol{x}_2 - \boldsymbol{c}) \tag{7.14}$$
  and if
  $$f(\boldsymbol{x}_c^{in}) \leq f(\boldsymbol{x}_2)$$

then $\boldsymbol{x}_c^{in}$ is accepted ($\boldsymbol{x}_2 = \boldsymbol{x}_c^{in}$). Otherwise, a shrink of the simplex is performed.

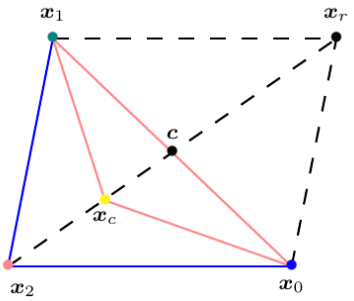

Figure 7.14: *The contraction point $\boldsymbol{x}_c^{in}$ is computed inside the old simplex.*

[SI*] **Shrink**
the last resort that will be performed if none of the criteria so far have been met is to shrink the entire simplex by moving all points closer to the current best points.

$$\boldsymbol{s}_{i-1} = \boldsymbol{x}_0 + \delta(\boldsymbol{x}_i - \boldsymbol{x}_0), \quad i = \{1, 2, \ldots, N\} \tag{7.15}$$

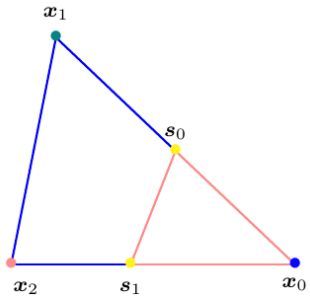

Figure 7.15: *Shrink the simplex by moving every point closer to the current best.*

[SI*] **Initialization and termination**
When initializing the simplex, usually one point $\boldsymbol{x}_0$ is to be given by the user (as the initial value / input), then the other points are created by placing them at certain distance μ from the initial point along the coordinate axes in the parameter space $\mathbb{R}^N$ of the objective function.

$$\boldsymbol{x}_i = \boldsymbol{x}_0 + \mu \hat{\boldsymbol{e}}_i, \quad i = \{1, 2, \ldots, N\} \tag{7.16}$$

The algorithm terminates in one of three different ways. Either by having the simplex become sufficiently small, such that the active search space is thought to not have any considerable variation in function value. Another criteria is for all the function values of the points of the simplex to have a similar enough value. And finally the algorithm must be set to run for a maximum number of iterations or function evaluations.

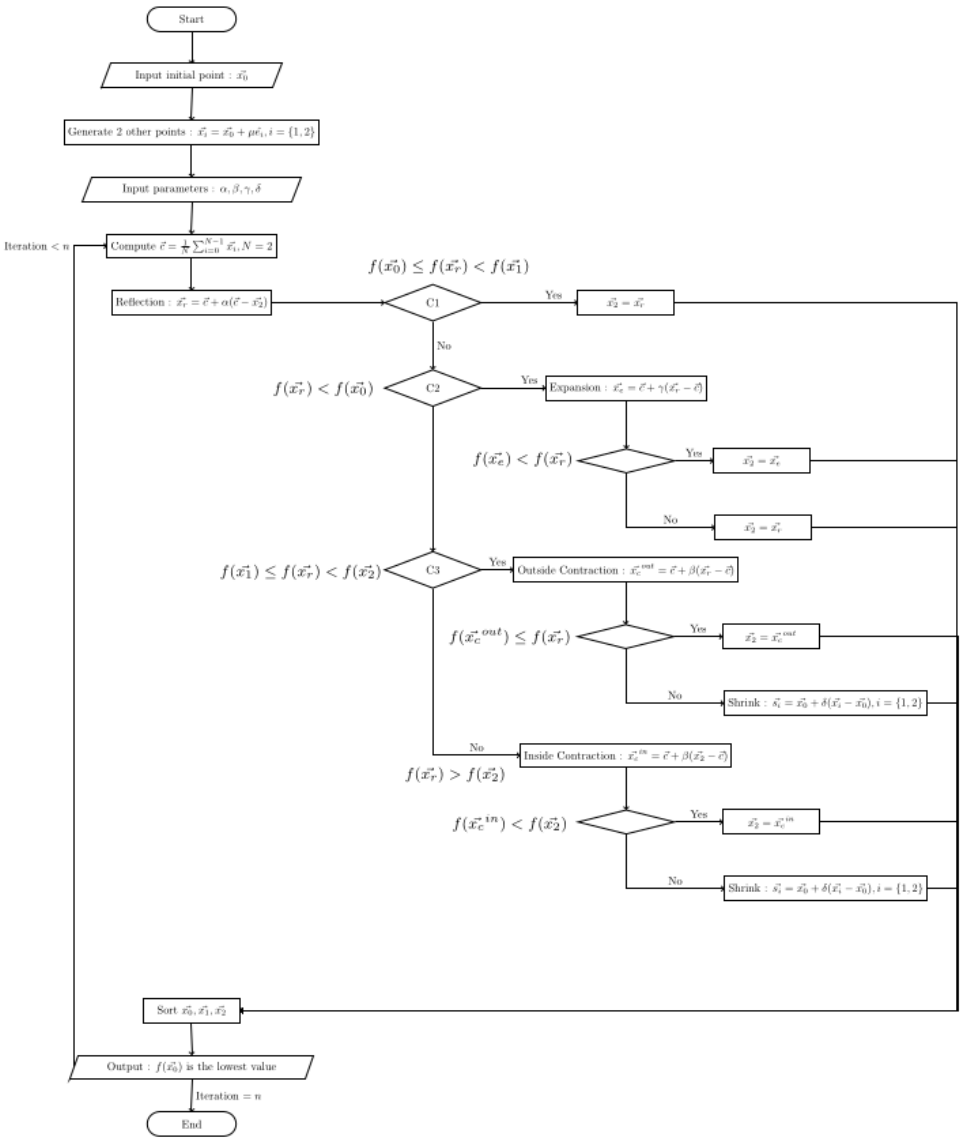

Figure 7.16: *The flowchart for downhill simplex algorithm that is used to make the C++ code for computing the solution.*

[SI*] The downhill simplex method has some pros and cons. Its major strength is that it does not require the gradient / derivative computation of f. the gradient of the objective function is often not known as an analytical expression and can therefore be very expensive to evaluate. If so it has to be done using some form of finite difference scheme. This computational cost increases with the dimensionality of the problem and is also subject to potential numerical stability prolems if some care is not taken. Since downhill simplex does not require to compute the gradient of f, this makes this method applicable to non-smooth functions where the gradient may not exist on the whole domain.

This method has a memory footprint on the order of about $(N + 2) \times N$ since we always

have to store $N+2$ sets of N parameters. Since we don't have to compute any gradient, then this method converges slowly compared to gradient based methods, generally because it requires more function evaluations.

[SI*] **Example:**
Find the value x and y that minimizes

$$f(\boldsymbol{x}) = f(x,y) = e^{-\frac{x^2+y^2}{2s^2}} - e^{-\frac{x^2+y^2}{2t^2}} + \frac{1}{10}x^2 + \frac{1}{10}y^2, \quad s = 0.75,\ t = 1 \tag{7.17}$$

Solution:
First, we choose the initial point:

$$\boldsymbol{x}_0 = \begin{bmatrix} 0.1 \\ 0.5 \end{bmatrix}$$

The other two points can be generated from $\boldsymbol{x}_0$ by placing a certain distance $\mu = 0.05$ that we choose, thus

$$\boldsymbol{x}_1 = \boldsymbol{x}_0 + \mu\hat{e}_1 = \begin{bmatrix} 0.1 \\ 0.5 \end{bmatrix} + 0.05 \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.15 \\ 0.5 \end{bmatrix}$$

$$\boldsymbol{x}_2 = \boldsymbol{x}_0 + \mu\hat{e}_2 = \begin{bmatrix} 0.1 \\ 0.5 \end{bmatrix} + 0.05 \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.1 \\ 0.55 \end{bmatrix}$$

We will have

$$f(\boldsymbol{x}_0) = -0.058442$$
$$f(\boldsymbol{x}_1) = -0.0604927$$
$$f(\boldsymbol{x}_2) = -0.066630$$

(It is unsorted, but we try to compute with unsorted vector first) From here we can compute $\boldsymbol{c}$

$$\boldsymbol{c} = \frac{1}{2}\sum_{i=0}^{1} \boldsymbol{x}_i = \begin{bmatrix} 0.125 \\ 0.5 \end{bmatrix}$$

Now, we need to input the parameters as

$$\alpha = 1, \quad \beta = 0.5, \quad \gamma = 2, \quad \delta = 0.5$$

Then we will perform the reflection operation first,

$$\boldsymbol{x}_r = \boldsymbol{c} + \alpha(\boldsymbol{c} - \boldsymbol{x}_2) = \begin{bmatrix} 0.15 \\ 0.45 \end{bmatrix}$$

From here we will compare the value of $f(\boldsymbol{x}_0), f(\boldsymbol{x}_1), f(\boldsymbol{x}_2)$ and $f(\boldsymbol{x}_r)$.

If $f(\boldsymbol{x}_0) < f(\boldsymbol{x}_r) < f(\boldsymbol{x}_1)$ then

$$\boldsymbol{x}_2 = \boldsymbol{x}_r$$

and we go to iteration 2.

If $f(\boldsymbol{x}_r) < f(\boldsymbol{x}_0)$, then go to the expansion step.

If $f(\boldsymbol{x}_r) \geq f(\boldsymbol{x}_1)$, then go to the contraction step.

In this case we have

$$\begin{aligned} f(\boldsymbol{x}_0) &= -0.058442 \\ f(\boldsymbol{x}_1) &= -0.0604927 \\ f(\boldsymbol{x}_2) &= -0.0666302 \\ f(\boldsymbol{x}_r) &= -0.0523666 \end{aligned}$$

$f(\boldsymbol{x}_r) \geq f(\boldsymbol{x}_1)$ so we go and compute the contraction point, since $f(\boldsymbol{x}_r) \geq f(\boldsymbol{x}_2)$ we compute the inside contraction point

$$\boldsymbol{x}_c^{in} = \boldsymbol{c} + \beta(\boldsymbol{x}_0 - \boldsymbol{c}) = \begin{bmatrix} 0.1125 \\ 0.5 \end{bmatrix}$$

then we will have

$$\begin{aligned} f(\boldsymbol{x}_0) &= -0.058442 \\ f(\boldsymbol{x}_1) &= -0.0604927 \\ f(\boldsymbol{x}_2) &= -0.0666302 \\ f(\boldsymbol{x}_c) &= -0.0588848 \end{aligned}$$

since $f(\boldsymbol{x}_c) > f(\boldsymbol{x}_2)$ then we perform shrink that will replace $\boldsymbol{x}_1$ with $\boldsymbol{s}_0$ and $\boldsymbol{x}_2$ with $\boldsymbol{s}_1$

$$\boldsymbol{s}_0 = \boldsymbol{x}_0 + \delta(\boldsymbol{x}_1 - \boldsymbol{x}_0) = \begin{bmatrix} 0.1250 \\ 0.5250 \end{bmatrix}$$

$$\boldsymbol{s}_1 = \boldsymbol{x}_0 + \delta(\boldsymbol{x}_2 - \boldsymbol{x}_0) = \begin{bmatrix} 0.1375 \\ 0.5625 \end{bmatrix}$$

So we have reached the end of the first iteration, which is using the shrink and we get these values at the end of this iteration:

$$\begin{aligned} f(\boldsymbol{x}_0) &= -0.058442 \\ f(\boldsymbol{x}_1) &= -0.0604927 \\ f(\boldsymbol{x}_2) &= -0.0666302 \end{aligned}$$

Continue till the last number of iteration. We use 6 iterations and we use C++ to help us in computing, thus we obtain the best vector $\boldsymbol{x}_0$ that minimizes the function $f(x, y)$ is

$$\boldsymbol{x}_0 = \begin{bmatrix} 0.1916 \\ 0.5 \end{bmatrix}$$

with $f(\boldsymbol{x}_0) = -0.063016$.

Since it is only using 6 iterations, thus the real optimal solution can be different if we add more iterations, we also sort the vectors $\boldsymbol{x}_0, \boldsymbol{x}_1, \boldsymbol{x}_2$ at the end of iteration 1, if it is moved before iteration 1 then we will get faster convergence. We try the code with sorted vector before iteration 1 and obtain $f(\boldsymbol{x}_0) = -0.086404$ at the end of iteration 6.

We create the computation with **SymIntegration** and **Armadillo** and another one is with **SymIntegration** and **Eigen** for comparison, both produce the same answer but the one with **Eigen** has faster compiling time, we use 6 iterations with unsorted initial vectors then we sort the initial vectors and use 37 iterations, and then for the plotting in 3D we use Hamzstlab Mathematics with backend **ImGui** and **ImPlot3D** so we can change the value of s and t and the surface plot will change interactively as well.

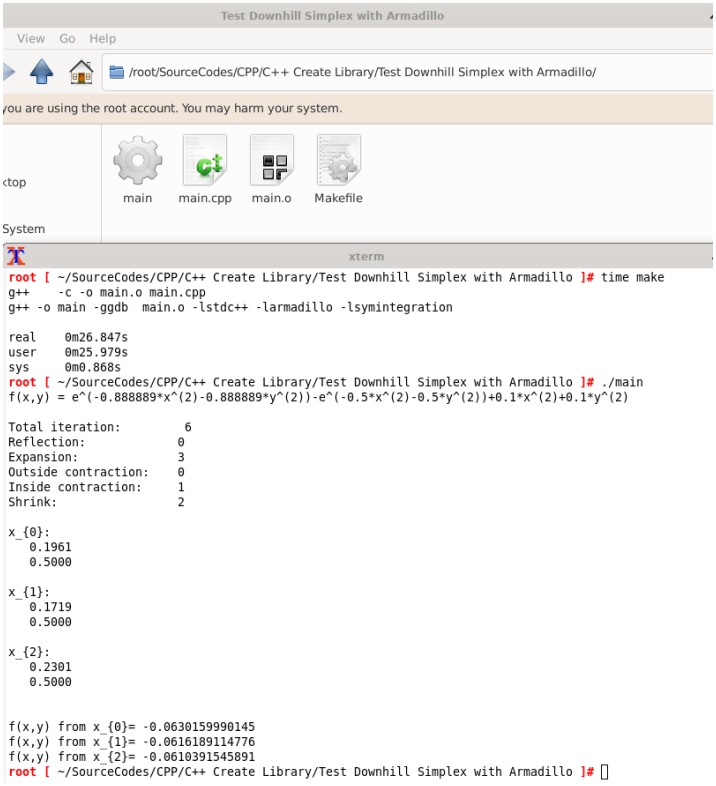

Figure 7.17: *The computation of downhill simplex for* $f(x,y) = e^{-\frac{x^2+y^2}{2s^2}} - e^{-\frac{x^2+y^2}{2t^2}} + \frac{1}{10}x^2 + \frac{1}{10}y^2$, $s = 0.75$, $t = 1$ *with SymIntegration and Armadillo (**SymIntegration/Examples/Operations Research/Test Downhill Simplex with Armadillo/main.cpp**).*

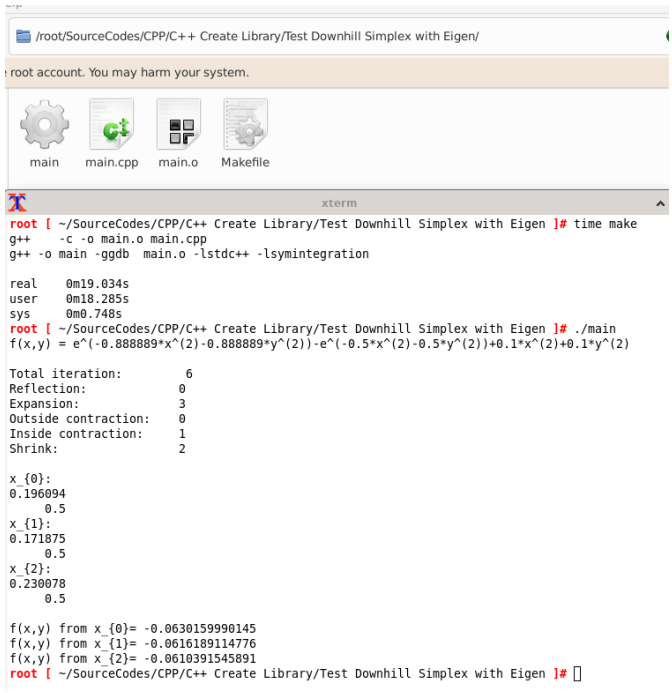

Figure 7.18: *The computation of downhill simplex for* $f(x,y) = e^{-\frac{x^2+y^2}{2s^2}} - e^{-\frac{x^2+y^2}{2t^2}} + \frac{1}{10}x^2 + \frac{1}{10}y^2$, $s = 0.75$, $t = 1$ *with SymIntegration and Eigen (**SymIntegration/Examples/Operations Research/Test Downhill Simplex with Eigen/main.cpp**).*

```
g++ -o main -ggdb  main.o -lstdc++ -larmadillo -lsymintegration

real    0m29.108s
user    0m28.112s
sys     0m0.893s
root [ ~/SourceCodes/CPP/C++ Create Library/Test Downhill Simplex with Armadillo ]# ./main
Vector x_{0}:
   0.1000
   0.5000

Vector x_{1}:
   0.1500
   0.5000

Vector x_{2}:
   0.1000
   0.5500

After sorting,
x_{0}:
   0.1000
   0.5500

x_{1}:
   0.1500
   0.5000

x_{2}:
   0.1000
   0.5000

f(x,y) = e^(-0.888889*x^(2)-0.888889*y^(2))-e^(-0.5*x^(2)-0.5*y^(2))+0.1*x^(2)+0.1*y^(2)

Total iteration:         37
Reflection:              5
Expansion:               14
Outside contraction:     0
Inside contraction:      18
Shrink:                  0

x_{0}:
   0.0973
   0.8914

x_{1}:
   0.0852
   0.8891

x_{2}:
   0.0629
   0.8977


f(x,y) from x_{0}= -0.0992255745363
f(x,y) from x_{1}= -0.0992180473087
f(x,y) from x_{2}= -0.0992086382243
root [ ~/SourceCodes/CPP/C++ Create Library/Test Downhill Simplex with Armadillo ]#
```

Figure 7.19: *The computation of downhill simplex for* $f(x,y) = e^{-\frac{x^2+y^2}{2s^2}} - e^{-\frac{x^2+y^2}{2t^2}} + \frac{1}{10}x^2 + \frac{1}{10}y^2$, $s = 0.75$, $t = 1$ *with 37 iterations (**SymIntegration/Examples/Operations Research/Test Downhill Simplex with Armadillo/main.cpp**).*

```
troot [ ~/SourceCodes/CPP/C++ Create Library/Test Downhill Simplex with Eigen ]#
time make
g++    -c -o main.o main.cpp
g++ -o main -ggdb  main.o -lstdc++ -lsymintegration

real    0m19.921s
user    0m19.108s
sys     0m0.760s
root [ ~/SourceCodes/CPP/C++ Create Library/Test Downhill Simplex with Eigen ]# ./main
Vector x_{0}:
0.1
0.5
Vector x_{1}:
0.15
 0.5
Vector x_{2}:
 0.1
0.55
After sorting,
x_{0}:
 0.1
0.55
x_{1}:
0.15
 0.5
x_{2}:
0.1
0.5
f(x,y) = e^(-0.888889*x^(2)-0.888889*y^(2))-e^(-0.5*x^(2)-0.5*y^(2))+0.1*x^(2)+0.1*y^(2)

Total iteration:         37
Reflection:              5
Expansion:               14
Outside contraction:     0
Inside contraction:      18
Shrink:                  0

x_{0}:
0.0972656
 0.891406
x_{1}:
0.0851562
 0.889063
x_{2}:
0.0628906
 0.897656

f(x,y) from x_{0}= -0.0992255745363
f(x,y) from x_{1}= -0.0992180473087
f(x,y) from x_{2}= -0.0992086382243
root [ ~/SourceCodes/CPP/C++ Create Library/Test Downhill Simplex with Eigen ]#
```

Figure 7.20: *The computation of downhill simplex for* $f(x,y) = e^{-\frac{x^2+y^2}{2s^2}} - e^{-\frac{x^2+y^2}{2t^2}} + \frac{1}{10}x^2 + \frac{1}{10}y^2$, $s = 0.75$, $t = 1$ *with 37 iterations (**SymIntegration/Examples/Operations Research/Test Downhill Simplex with Eigen/main.cpp**).*

* the C++ codes are already updated so the initial vectors will be sorted before the **for** looping.

[SI*] **Global Optimization**

Global optimization is used for finding all global minima or maxima of a given function on a given domain disregarding any local optima. Global optimization is a large field with many applications, and many deterministic and stochastic optimization methods have been developed.

Deterministic methods include:

1. Branch-and-bound methods
2. Cutting-plane methods
3. Inner-and-outer approximation
4. Interval methods

Stochastic methods, on the other hand, are methods whose results depend on random numbers drawn while running the algorithm; the function to be optimized is still deterministic. Stochastic methods include:

1. Direct Monte Carlo sampling
2. Stochastic tunneling
3. Parallel tempering

Heuristic methods are strategies for searching the domain in an intelligent manner.

They include:

1. Differential evolution
2. Evolutionary computation (including, e.g., evolution strategies, genetic algorithms, genetic programming)
3. Graduated optimization
4. Simulated annealing
5. Swarm based optimization (including, e.g., particle-swarm optimization and ant colony optimization)
6. Taboo search

Other methods are Bayesian optimization and memetic algorithms.

i. Deterministic Methods for Optimization

[SI*] Deterministic optimization methods are mathematical algorithms used to find the optimal solution (minimum or maximum) of a problem without relying on randomness or probabilistic elements. Given the same initial conditions and input parameters, a deterministic method will always follow the exact same execution path and yield the identical output.
Unlike stochastic algorithms (such as genetic algorithms or simulated annealing) which explore search spaces using random probability, deterministic techniques exploit the underlying analytical properties of the problem-like gradient information, convexity, and linearity-to converge toward the solution.

ii. Stochastic Optimization

[SI*] Stochastic optimization is a collection of optimization methods that incorporate randomness to find the global minimum or maximum of a system. Unlike deterministic methods (like gradient descent or standard Nelder-Mead), which can easily get trapped in the nearest local valley, stochastic algorithms use controlled random choices to "jump" out of local traps and explore the broader landscape to find the global optimum.

1. Direct Monte Carlo Sampling
   Direct Monte Carlo Sampling is a brute-force exploratory technique that evaluates a function at completely random points chosen across the search space. It generates independent random coordinate sets within defined boundaries, calculates the objective function value for each set, and tracks the best result found so far.
   It requires absolutely no gradient information or setup rules. However, because it treats every step independently without "learning" from previous successes, it suffers heavily from the curse of dimensionality and is highly inefficient for complex systems.
   Direct Monte Carlo optimization uses independent random sampling across the parameter space. It is non-adaptive, meaning sample $i+1$ does not depend on sample i.

   Mathematical Formulation
   Let $f(\boldsymbol{x})$ be the objective function to minimize, where $\boldsymbol{x} \in \mathbb{R}^n$ is bounded by a hypercube defined by lower bounds $\boldsymbol{a}$ and upper bounds $\boldsymbol{b}$ ($\boldsymbol{a} \leq \boldsymbol{x} \leq \boldsymbol{b}$).
   Each component j of the random vector $\boldsymbol{x}^{(i)}$ is generated using a uniform continuous distribution

$$x_j^{(i)} \; U(a_j, b_j) = a_j + (b_j - a_j) \cdot u_j$$

where u_j $U(0,1)$ is a standard uniformly distributed random number.
The optimization search tracks is

$$\boldsymbol{x}_{best} = \arg\min_{\boldsymbol{x}^{(i)}} \{f(\boldsymbol{x}^{(i)}) | i = 1, 2, \ldots, M\}$$

The Algorithm

(a) **Define** the objective function $f(\boldsymbol{x})$, bounds $[\boldsymbol{a}, \boldsymbol{b}]$ and total samples M.
(b) **Initialize** $f_{best} = \infty$ and an empty vector $\boldsymbol{x}_{best}$.
(c) **Loop** from $i = 1$ to M
Generate a random vector $\boldsymbol{x}^{(i)}$ where each element $x_j^{(i)} = a_j + (b_j - a_j) \times rand(0,1)$.
Calculate $f_{current} = f(\boldsymbol{x}^{(i)})$.
If $f_{current} < f_{best}$ then set $f_{best} = fcurrent$ and $\boldsymbol{x}_{best} = \boldsymbol{x}^{(i)}$.
(d) **Return** $\boldsymbol{x}_{best}$ and f_{best}.

2. Stochastic Tunneling (STUN)
Stochastic Tunneling prevents an optimizer from getting stuck in local valleys by dynamically flattening the math landscpae using a non-linear transformation function. When an algorithm finds a local minimum with an energy value E_0, a transformation function maps the objective function into a new landscape. All structures with energy higher than E_0 are flattened or suppressed, while regions with energy lower thatn E_0 are preserved.
To the algorithm, the local barriers literally vanish. It can "tunnel" straight through what used to be a massive hill to explore only the regions that promise a superior global outcome.

Stochastic Tunneling maps the original objective function to a transformed function $f_{STUN}(\boldsymbol{x})$. This transformation eliminates energy barriers higher than the best local minimum found so far (f_0), allowing the algorithm to tunnel through hills.

The STUN transformation function is defined as

$$f_{STUN}(\boldsymbol{x}) = 1 - e^{-\gamma \cdot (f(\boldsymbol{x})) - f_0} \cdot \Theta(f(\boldsymbol{x}) - f_0)$$

where

- f_0 is the lowest functional energy value found so far.
- $\gamma > 0$ is a tuning parameter dictating the steepness of the tunnel valley.
- $\Theta(z)$ is the Heaviside step function

$$\Theta(z) = \begin{cases} 1, & \text{if } z \geq 0 \\ 0, & \text{if } z < 0 \end{cases}$$

When $f(\boldsymbol{x}) \gg f_0, f_{STUN}(\boldsymbol{x}) \to 1$ the landscape becomes completely flat, by passing local barriers. When $f(\boldsymbol{x}) < f_0, \Theta(z) = 0$ causing $f_{STUN}(\boldsymbol{x}) < 0$, which accelerates exploration into new deeper valleys.
Monte Carlo moves on f_{STUN} are accepted using the standard Metropolis criterion at a virtual temperature T

$$P_{accept} = \min\left(1, e^{-\frac{\triangle f_{STUN}}{T}}\right)$$

The Algorithm

(a) **Initialize** a starting position $\boldsymbol{x}$, set $f_0 = f(\boldsymbol{x})$, and pick constants γ and T.

(b) **Loop** for a set number of steps
Generate a trial position $\boldsymbol{x}_{new} = \boldsymbol{x} + \epsilon$, where ϵ $N(0, \sigma^2)$.
Calculate the true objective value $f(\boldsymbol{x}_{new})$.
Calculate the tunneling transformation values

$$f_{STUN}(\boldsymbol{x}) = \begin{cases} 1 - e^{-\gamma(f(\boldsymbol{x}) - f_0)}, & \text{if } f(\boldsymbol{x}) \geq f_0 \\ 0, \quad f(\boldsymbol{x}) < f_0 \end{cases}$$

$$f_{STUN}(\boldsymbol{x}_{new}) = \begin{cases} 1 - e^{-\gamma(f(\boldsymbol{x}_{new}) - f_0)}, & \text{if } f(\boldsymbol{x}_{new}) \geq f_0 \\ f(\boldsymbol{x}_{new}) - f_0, & \text{if } f(\boldsymbol{x}_{new}) < f_0 \end{cases}$$

Calculate $\triangle f_{STUN} = f(\boldsymbol{x}_{new}) - f(\boldsymbol{x})$.
If $\triangle f_{STUN} \leq 0$ or $rand(0,1) < e^{-\frac{\triangle f_{STUN}}{T}}$ then $\boldsymbol{x} = \boldsymbol{x}_{new}$.
If $f(\boldsymbol{x}_{new}) < f_0$ then update the tunneling baseline $f_0 = f(\boldsymbol{x}_{new})$

(c) **Return** the globally lowest configuration parameters found.

3. Parallel Tempering (Replica Exchange)
Parallel Tempering is an advanced Markov Chain Monte Carlo (MCMC) technique that runs multiple versions (replicas) of a system simultaneously at different virtual "temperatures."

- **Hot replicas** have high thermal energy. They easily hop over huge energy barriers to broadly map out the entire search space.
- **Cold replicas** have low energy. They refine parameters and precisely drop into the lowest local valleys nearby.

Periodically, the algorithm tests adjacent replicas. If a hot replica finds a promizing low-energy valley, it swaps states with a colder replica so the colder model can descend into it and optimize the exact coordinates.
Parallel Tempering runs K independent replicas of the system simultaneously, each at a distinct temperature $T_1 < T_2 < \cdots < T_K$. Replicas periodically attempt to swap configurations based on a thermal equilibrium probability criterion.

Mathematical Formulation
The state of the composite system is defined by the set of configurations $\{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_K\}$. Each replica i evaluates its local state using its designated temperature T_i. The standard Metropolis acceptance probability for a spatial step within replica i is

$$P(\boldsymbol{x}_i \to \boldsymbol{x}_i') = \min\left(1, e^{-\frac{f(\boldsymbol{x}_i') - f(\boldsymbol{x}_i)}{T_i}}\right)$$

The Swap Formula
Periodically, a swap of configurations between adjacent replicas i and j (where $j = i + 1$) is attempted. To maintain a detailed balance of the thermal states, the swap acceptance probability P_{swap} is calculated using the Metropolis-Hastings criterion

$$P_{swap} = \min\left(1, e^{\triangle\beta \cdot \triangle f}\right)$$

with

$$\triangle\beta = \frac{1}{T_i} - \frac{1}{T_j}$$
$$\triangle f = f(\boldsymbol{x}_i) - f(\boldsymbol{x}_j)$$

Expanding the exponent gives

$$e^{\left(\frac{1}{T_i}-\frac{1}{T_j}\right)\cdot(f(\boldsymbol{x}_i)-f(\boldsymbol{x}_j))} = e^{\frac{f(\boldsymbol{x}_j)-f(\boldsymbol{x}_i)}{T_i}+\frac{f(\boldsymbol{x}_i)-f(\boldsymbol{x}_j)}{T_i}}$$

If a high-temperature replica (large T_j) discovers a deep local minimum (low $f(\boldsymbol{x}_j)$), this formula yields a high probability of swapping that state down to a colder replica (small T_i) for local optimization.

The Algorithm

(a) Initialize K replica positions $\{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_K\}$ and a geometric distribution of temperatures $\{T_1, \ldots, T_k\}$.

(b) **Loop** for a set number of global iterations:
Phase 1: Local Space Exploration
For each replica i from 1 to K:
Propose a local step: $\boldsymbol{x}_i' = \boldsymbol{x}_i + \epsilon$.
Compute $\triangle f = f(\boldsymbol{x}_i') - f(\boldsymbol{x}_i)$.
Accept $\boldsymbol{x}_i = \boldsymbol{x}_i'$ if $\triangle f \leq 0$ or if $rand(0,1) < e^{-\frac{\triangle f}{T_i}}$.
Phase 2: Replica Swap Exchange
Pick a random replica index i (where $i < K$) and select its neighbor $j = i + 1$.
Compute the swap delta

$$\triangle = \left(\frac{1}{T_i} - \frac{1}{T_j}\right) \cdot (f(\boldsymbol{x}_i) - f(\boldsymbol{x}_j))$$

If $\triangle \geq 0$ or $rand(0,1) < e^{\triangle}$ then swap the configurations $swap(\boldsymbol{x}_i, \boldsymbol{x}_j)$.

(c) **Track** the absolute lowest function value evaluated by any replica, and return its coordinates from the cold baseline state (T_1).

iii. Particle Swarm Optimization for Optimization

[SI*]

iv. Compute and Simulate a Minimization of Function of Two Variables with Direct Monte Carlo, Stochastic Tunneling, and Parallel Tempering

We will compute the minimum value of a function

$$f(x,y) = e^{-\frac{x^2+y^2}{2s^2}} - e^{-\frac{x^2+y^2}{2t^2}} + 0.1x^2 + 0.1y^2$$

with all the three stochastic optimization methods. The benchmark harness tests all three algorithms against this identical 2-dimensional doule-exponential potential energy landscape paired with a quadratic penalty framework. The input argument itself is a 2-dimensional vector $\boldsymbol{v} = [x,y]^T$. The shapes of the hills and troughs are dictated by scaling factors set to $s = 0.75$ and $t = 1.0$. The local minimum is located at the origin (0,0) where the baseline value settles precisely to 0.000000, but it is not the global minimum. The equation only depends on the radius $r^2 = x^2 + y^2$, the true global minimum is not a single point, but a perfectly circular hyperspherical valley (a ring canyon) around the origin.

Direct Monte Carlo

DMC translates standard uniformly distributed random numbers bounded by $[0,1]$ into a continuous uniform random spread across the specific lower boundary limits (a) and upper boundary limits (b) for each dimension:

$$x_j = a_j + (b_j - a_j) \cdot u_j, \quad \text{where } u_j \; U(0,1)$$

it is wrapped inside the standard initialization.

The coordinates are found through an algorithmic selection formula. The framework creates thousands of guess vectors ($\boldsymbol{x}_{proposal}$) using random numbers, and filters them

$$\boldsymbol{x}_{best} = \arg \min_{\boldsymbol{x} \in \text{Search Space}} f(\boldsymbol{x})$$

Once the algorithm selects the best coordinate vector, the energy value is strictly computed by evaluating the objective function at that specific coordinate vector

$$E_{best} = f(\boldsymbol{x}_{best})$$

Direct Monte Carlo throws darts blindly. Its discovered energy will be close, but noticeably higher than the true minimum because it doesn't "walk down" into the valley.

Stochastic Tunneling (STUN)

The STUN transformation equation changes the andscape dynamically by calculating a new virtual energy function f_{STUN} relative to the lowest true function value found so far (f_0):

$$f_{STUN}(\boldsymbol{x}) = \begin{cases} 1 - e^{-\gamma(f(\boldsymbol{x}_{new}) - f_0)}, & \text{if } f(\boldsymbol{x}) \geq f_0 \\ f(\boldsymbol{x}) - f_0, & \text{if } f(\boldsymbol{x}) < f_0 \end{cases}$$

The algorithm then applies a standard Metropolis validation pass using the change in transformed values ($\triangle f_{STUN} = f_{STUN}(\boldsymbol{x}_{new}) - f_{STUN}(\boldsymbol{x})$)

$$P_{accept} = \min\left(1, e^{-\frac{\triangle f_{STUN}}{T}}\right)$$

The minimum energy value found (**results.best_energy**) represents the absolute lowest structural value achieved by the solver throughout the random walk. It is direcly calculated by execucuting the physical objective function at the best discovered coordinates

$$E_{best} = f(\boldsymbol{x}_{best})$$

The best coordinate vector array (**results.best_x**) is the location vector containing the precise spatial parameters matching the minimum energy point. This is mathematically modeled as the continuous global argmin selection

$$\boldsymbol{x}_{best} = \arg \min_{\boldsymbol{x} \in \text{Domain}} f(\boldsymbol{x})$$

it uses an exploratory stochastic vector step formula

$$\boldsymbol{x}_{proposal} = \boldsymbol{x}_{current} + N(0, \sigma)$$

where $N(0, \sigma)$ represents a vector of random mutations generated from a normal distribution curve (**rand_n(gen)**).

This method uses local updates, after finding a slope it will step downward systematically and will achieve an E_{best} that matches the true analytical minimum to many decimal places. Hitting a perfectly continuous infinite decimal coordinate at random is statistically impossible.

Standard optimizers frequently get stuck inside the central origin crater because stepping back toward the outside requires climbing an energy barrier. This is where Stochastic Tunneling alters the game. Instead of comparing the true energies directly, STUN passes the values through an exponential tunneling mapping transformation formula

$$\tilde{f}(\boldsymbol{x}) = 1 - e^{-\gamma(f(\boldsymbol{x}) - f_0)}$$

if there is a physical surface, a high energy barrier (hilltop) with $f(\boldsymbol{x}) \gg f_0$ then we will use the STUN transformed surface, a flattened tunneling plateau for easy pass-through with $\tilde{f}(\boldsymbol{x}) \to 1$ (asymptotic limit) because the value of $e^{-\gamma(f(\boldsymbol{x}) - f_0)}$ vanishes to 0, making the transformed potential value freeze near 1.0.
The high energy hilltops are dynamically flattened out into an easy-to-cross plateau. The random walker can "tunnel" through high-energy barriers seamlessly to drop directly into the true continuous global minimum canyon ring (≈ -0.09923).

Parallel Tempering
Each replica i first runs local spatial exploration using a standard Metropolis validation test at its own assigned temperature T_i with its change in true energy $\triangle E = f(\boldsymbol{x}_{new}) - f(\boldsymbol{x})$

$$P_{local} = \min\left(1, e^{-\frac{\triangle E}{T_i}}\right)$$

Replicas then attempt a swap check between neighboring replicas r_1 and r_2 (where $\beta = \frac{1}{T}$)

$$\triangle_{swap} = (\beta_1 - \beta_2) \cdot (E_1 - E_2)$$

$$P_{swap} = \min(1, e^{\triangle_{swap}})$$

Similar to DMC, the minimum energy value formula is calculated by executing the objective function programmatically at the best discovered coordinates:

$$E_{best} = f(\boldsymbol{x}_{best})$$

Inside the core local exploration loops of **run_parallel_tempering()** function, whenever a replica satisfies a transition step or a swap argument that produces a lower value than the current tracking variable **metrics.best_energy** is updated.

The best coordinate vector array (**metrics.best_x**) is the precise directional position within the N-dimensional space where that lowest energy altitude exists

$$\boldsymbol{x}_{best} = \arg \min_{\boldsymbol{x} \in \text{Domain}} f(\boldsymbol{x})$$

The algorithm doesn't use a closed-form inverse calculus formula. Instead, it tests hundreds of thousands of candidate coordinate arrays through a stochastic transition proposal

$$\boldsymbol{x}_{proposal} = \boldsymbol{x}_{current} + \boldsymbol{Z}$$

where $\boldsymbol{Z}$ is a vector of random deviations generated from the normal distribution windown (**rand_n(gen)**). If the resulting evaluation passes the Metropolis-Hastings acceptance probability, the coordinate states transition forward.
Blind search strategies like Direct Monte Carlo throw guesses wildly and might settle at -0.095. However, because Parallel Tempering utilizes hot replicas to jump across the central hilltop barrier and cold replicas to step incrementally down the inner slopes, the **best_energy** output will typically match the true analytical minimum to 5 or 6 decimal places.

Core conceptual relationships:

- **The Objective Function ($f(x)$):** This is the mathematical equation that defines the "landscape" or "terrain" of our problem. In optimization and physical chemistry, this function is called the Energy Landscape.
- **Minimum Energy Value Found (E_{best}):** This is the lowest altitude (value) the solvers managed to find on the landscape after testing different points.
- **Best Coordinate Vector Array ($\boldsymbol{x}_{best}$):** This is the exact location (the N-dimensional map coordinate) where that lowest energy value was discovered.
- **True Global Minimum:** This is the absolute lowest possible point that mathematically exists on the entire landscape.

The objective function provided in the C++ code is a generalized, N-dimensional variation of a classic multi-modal system

$$f(\boldsymbol{x}) = e^{-\frac{||\boldsymbol{x}||}{2s^2}} - e^{-\frac{||\boldsymbol{x}||}{2t^2}} + 0.1||\boldsymbol{x}||^2$$

where $||\boldsymbol{x}|| = x_1^2 + x_2^2 + \cdots + x_N^2$ (the squared distance from the origin). Let's break down how these terms fight each other to create the terrain:

1. **The First Exponential $e^{-\frac{r^2}{2s^2}}$:** A sharp, narrow bell curve centered at the origin. Since $s = 0.75$, it decays quickly as you move away from the center.

2. **The Second Exponential $-e^{-\frac{r^2}{2t^2}}$:** A wider bell curve centered at the origin ($t = 1.0$), subtracted from the total. This drags the center down, creating a crater.

3. **The Quadratic Penalty ($0.1r^2$):** A bowl that frows infinitely large as you move far away from the center, forcing the optimizer to stay inside a tight radius.

If we differentiate $f(r)$ with respect to r ($r^2 = x^2 + y^2$) and find where the slope is zero, the continuous true global minimum settles at a radius of $r \approx 0.8979$, where the theoretical minimum energy reaches ≈ -0.09923. Because the function relies purely on the radius r, the global minimum is not a single coordinate point. It is an infinite family of points forming a perfect circular hyper-spherical valley (a ring canyon) surrounding the origin at a distance of 0.8979.

Application Code Architecture

The C++ codes have been structured, the core library has been fully decoupled from the mathematical objective functions, so we can swap out custom functions or topologies right from **main.cpp** using a **std::function** pass structure.

- **include/symintegral/numericalmethods.h**
  Standardized structural definitions for algorithm performance tracking metrics, ececution benchmarking arrays, and N-dimensional function hooks.
- **src/numericalmethods.cpp**
  Algorithmic implementaiton code optimizing spatial updates across any dimension parameter scale for Direct Monte Carlo, STUN, and Parallel Tempering.
- **Examples/Numerical Methods/Test SymIntegration Direct Monte Carlo/main.cpp**
  Compute the solution with Direct Monte Carlo method.
- **Examples/Numerical Methods/Test SymIntegration Parallel Tempering/main.cpp**
  Compute the solution with Parallel Tempering method.
- **Examples/Numerical Methods/Test SymIntegration Stochastic Tunneling/main.cpp**
  Compute the solution with Stochastic Tunneling method.

```
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Numerical Methods/Test SymIntegration Direct Monte Carlo ]# ./main
======================================================================
       N-DIMENSIONAL DIRECT MONTE CARLO OPTIMIZATION ENGINE
======================================================================
Target Space Resolution Size: 5 Dimensions
Search Envelope Matrix Bounds:  [-2.0, 2.0] across all dimensions
Running Execution Loop for 500000 random trial allocations...

Optimization Search Completed.
Total Random Samples Evaluated:         500000

--- Absolute Best Global Minimum Discovered ---
Minimum Energy Value Found:             -0.0992260903225
Best Coordinate Vector Array:   [ 0.470643 0.656146 -0.300255 -0.173976 -0.184382 ]
======================================================================

Time taken by function: 2090616 microseconds
```

Figure 7.21: *The computation of the minimum value of a function* $f(x,y) = e^{-\frac{x^2+y^2}{2s^2}} - e^{-\frac{x^2+y^2}{2t^2}} + 0.1x^2 + 0.1y^2$ *with Direct Monte Carlo, a stochastic optimization method. (**SymIntegration/Examples/Numerical Methods/Test SymIntegration Direct Monte Carlo/main.cpp**).*

```
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Numerical Methods/Test SymIntegration Parallel Tempering ]# ./main
======================================================================
         N-DIMENSIONAL PARALLEL TEMPERING OPTIMIZATION ENGINE
======================================================================
Target Space Resolution Size: 5 Dimensions
Active Replica Chain Count:     4 system nodes
Running Execution Loop for 5000 global swap configurations...

Optimization Search Completed.
Total Swap Attempts Checked:    15000
Successful Swap Transactions:   1949
Swap Acceptance Ratio:                  12.99%

--- Absolute Best Global Minimum Discovered ---
Minimum Energy Value Found:             -0.099226094868
Best Coordinate Vector Array:   [ 0.733922 0.087418 0.113729 0.229781 -0.440715 ]
======================================================================

Time taken by function: 1492389 microseconds
```

Figure 7.22: *The computation of the minimum value of a function* $f(x,y) = e^{-\frac{x^2+y^2}{2s^2}} - e^{-\frac{x^2+y^2}{2l^2}} + 0.1x^2 + 0.1y^2$ *with Parallel Tempering, a stochastic optimization method. (**SymIntegration/Examples/Numerical Methods/Test SymIntegration Parallel Tempering/main.cpp**).*

```
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Numerical Methods/Test SymIntegration Stochastic Tunneling ]# ./main
======================================================================
       N-DIMENSIONAL STOCHASTIC TUNNELING OPTIMIZATION ENGINE
======================================================================
Target Space Resolution Size: 5 Dimensions
Initial Base Coordinates:       [ 1.00 1.00 1.00 1.00 1.00 ]
Running Execution Loop for 100000 tunneling step attempts...

Optimization Search Completed.
Total Steps Executed:           100000
Accepted Move Transitions:      89994

--- Absolute Best Global Minimum Discovered ---
Minimum Energy Value Found:             -0.099226094865
Best Coordinate Vector Array:   [ -0.641869 0.105137 0.578232 -0.194442 0.105025 ]
======================================================================

Time taken by function: 679614 microseconds
```

Figure 7.23: *The computation of the minimum value of a function* $f(x,y) = e^{-\frac{x^2+y^2}{2s^2}} - e^{-\frac{x^2+y^2}{2l^2}} + 0.1x^2 + 0.1y^2$ *with Stochastic Tunneling, a stochastic optimization method. (**SymIntegration/Examples/Numerical Methods/Test SymIntegration Stochastic Tunneling/main.cpp**).*

In addition, we also create a simulation with SFML 2.5.1 for the visualization of the stochastic optimizers for the objective function in this case.

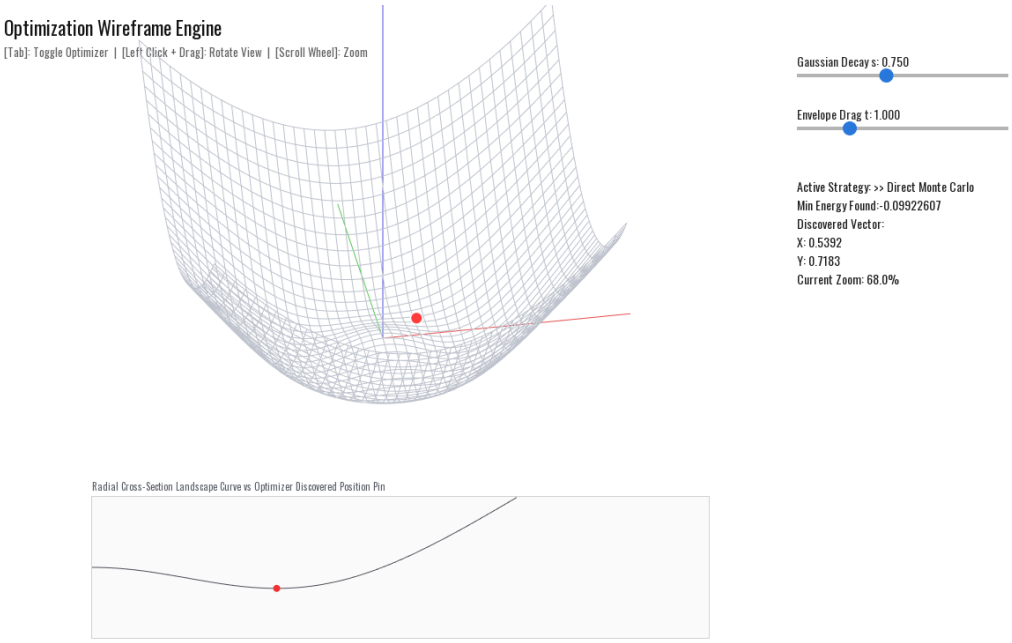

Figure 7.24: *The simulation to compute the minimum value of a function* $f(x,y) = e^{-\frac{x^2+y^2}{2s^2}} - e^{-\frac{x^2+y^2}{2t^2}} + 0.1x^2 + 0.1y^2$ *with Direct Monte Carlo, Stochastic Tunneling, Parallel Tempering (changeable by clicking Tab button), the parameters* s *and* t *are also changeable with the slider. (**SymIntegration/Examples/Numerical Methods/Test SymIntegration Stochastic Optimizer for 3D Function of two Variables with SFML/main.cpp**).*

v. Simulate a Minimization of Function of Two Variables with Particle Swarm Optimization

With the help of SFML 2.5.1 as the visualization tool we will create a full decoupled particle swarm optimization framework visualization.

We want to optimize against the multimodal landscape function

$$f(x, y) = \sin(x)\cos(y) + 0.1(x^2 + y^2)$$

this function provides an excellent optimization benchmark. It creates multiple local minima (valleys) across the landscape due to the trigonometric functions, while the quadratic term $0.1(x^2 + y^2)$ forces a global minimum near the center origin $(0, 0)$.

1. **Mathematical Formulas & Kinematics**
   The algorithm operates by updating the kinematic states of individual particles across discrete time steps. Each particle i has a position vector $\boldsymbol{x}_i$ and a velocity vector $\boldsymbol{v}_i$. In the 2D workspace, these are represented as (x, y) and (vx, vy). The core mechanics rely on two fundamental equations updated every frame, first it is the velocity update equation

   $$\boldsymbol{v}_i^{(t+1)} = w \cdot \boldsymbol{v}_i^{(t)} + c_1 \cdot r_1 \cdot (\boldsymbol{p}_i - \boldsymbol{x}_i^{(t)}) + c_2 \cdot r_2 \cdot (\boldsymbol{g} - \boldsymbol{x}_i^{(t)})$$

   where

   - $\boldsymbol{v}_i^{(t)}$ is the current velocity vector of particle i.
   - w is the inertia weight coefficient. It dictates how much of the particle's prior trajectory momentum is conserved. Higher values promote global exploration; lower values enforce local exploitation.
   - c_1 is the cognitive (personal) acceleration coefficient. It scales the behavioral pull toward the individual particles's own historical best location $\boldsymbol{p}_i$.
   - c_2 is the social (global) acceleration coefficient. It scales the behavioral pull toward the entire swarm's collective historical best location $\boldsymbol{g}$.
   - r_1, r_2 $U(0, 1)$ are independent uniformly distributed random numbers generated at every step, providing stochastic exploration characteristics.

   Second, we have the position translation equation

   $$\boldsymbol{x}_i^{(t+1)} = \boldsymbol{x}_i^{(t)} + \boldsymbol{v}_i^{(t+1)}$$

   The particle advances its current position coordinates by adding its freshly computed velocity components.

   When a particle attempts to step outside the specified bounding box limits $(X_{min}, X_{max}, Y_{min}, Y_{max})$, a "wall clamping" boundary condition is enforced, thus the absolute kinematic boundary constraints is

   $$\text{if } x_i > X_{max} \rightarrow x_i = X_{max}, \quad vx_i = 0$$

   Setting the corresponding velocity component to zero stops the particle from continuing to push wastefully outward against the system boundaries.

2. **Comprehensive Algorithmic Workflow**
   The logic flow executed across the **initSwarm()** and **updateSwarmMath()** functions follows this structured pipeline:

(a) **Step 1: Initialization Module**
Set globalBestFitness=$+\infty$, distribute positions via uniform distribution, generate minor randomized initial velocities, evaluate initial objective function costs.

(b) **Step 2: Main Physics Loop**
Generate unique stochastically decoupled r_1, r_2 pairs.

(c) **Step 3: Kinematics Vector Updates**
Update components: vx, vy (using $w, c1, c2, r1, r2$), shift workspace positions $x = x + vx$, $y = y + vy$, enforce boundary constraints (clamp and zero-out v).

(d) **Step 4: Fitness Evaluation Phase**
Calculate currentFitness = $f(x, y)$.

(e) **Step 5: Memory & Topology Updates**
If currentFitness < personalBestFitness then update individual particle history boundaries.
If currentFitness < globalBestFitness then update shared swarm-wide best target metadata.

Repeat step 2 until terminated.

3. **Application Code Architecture**
The C++ visualization program separates the implementation into decoupled mathematical engine files and an interactive visualization file.

- **include/symintegral/sfml-visualizations.h**
Structure definitions (particle, SwarmState, PSOHyperparams).
- **src/sfml-visualizations.cpp**
Pure numerical routies (initSwarm, updateSwarmMath).
- **Examples/Numerical Methods/Test SymIntegration PSO 2 Variables with SFML/-main.cpp**
SFML 2.5.1 graphic pipeline, handles the vector canvas drawings, samples runtime interaction states, evaluates optimization landscapes. TO visualize the terration, evaluation scores are converted into pixels matching the classic Jet Color Map topology. The values are normalized between 0.0 and 1.0. The mathematical curves transition linearly across the visible spectrum. Deep blue/cyan represents low function evaluations (global minima / valleys), green/yellow represents mid-range transitional landscape plateau elevations, vibrant red represents high function values (peaks/computational walls). The particles use this visual gradient to coordinate their flight path, dynamically adjusting their settings via the user interface panel.

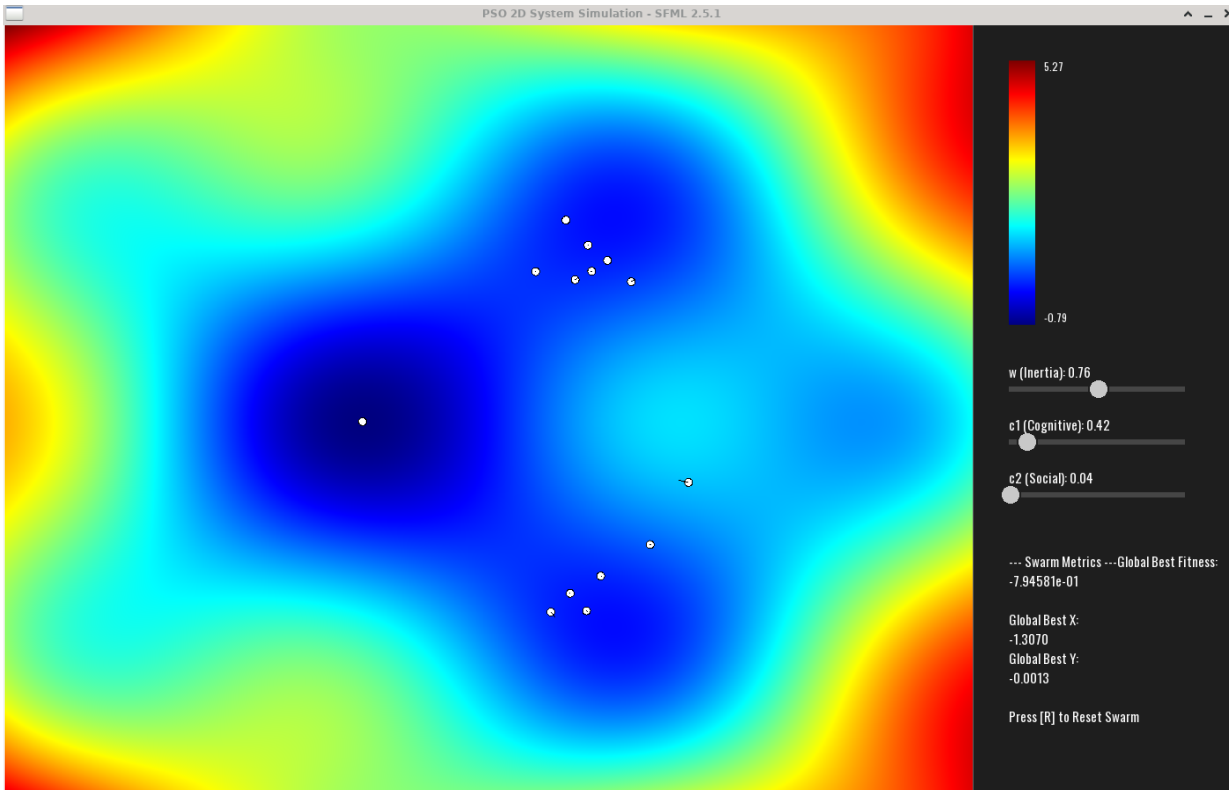

Figure 7.25: *The computation of particle swarm optimization for minimizing function* $w = 0.76, c1 = 0.42, and c2 = 0.04$*, there are some particles who haven't reached the global best due to the very small social acceleration coefficient. (**SymIntegration/Examples/Numerical Methods/Test SymIntegration PSO 2 Variables with SFML/main.cpp**).*

vi. Functions in SymIntegration Related to Numerical Methods for Optimization

[SI*]

IV. Numerical Differentiation

[SI*] The derivative of the function f at x_0 is

$$f'(x_0) = \lim_{h \to 0} \frac{f(x_0 + h) - f(x_0)}{h} \tag{7.18}$$

This formula gives an obvious way to generate an approximation to $f'(x)$; simply compute

$$\frac{f(x_0 + h) - f(x_0)}{h}$$

for small values of h. Although this may be obvious, it is not very successful, die to roundoff error.

To approximate $f'(x_0)$, suppose first that $x_0 \in (a, b)$ where $f \in C^2[a, b]$, and that $x_1 = x_0 + h$ for some $h \neq 0$ that is sufficiently small to ensure that $x_1 \in [a, b]$. We construct the first Lagrange polynomial $P_{0,1}(x)$ for f determined by x_0 and x_1, with its error term:

$$\begin{aligned} f(x) &= P_{0,1}(x) + \frac{(x - x_0)(x - x_1)}{2!} f''(\xi(x)) \\ &= \frac{f(x_0)(x - x_0 - h)}{-h} + \frac{f(x_0 + h)(x - x_0)}{h} + \frac{(x - x_0)(x - x_0 - h)}{2} f''(\xi(x)) \end{aligned}$$

for some $\xi(x)$ in $[a, b]$. Differentiating gives

$$\begin{aligned} f'(x) &= \frac{f(x_0 + h) - f(x_0)}{h} + D_x \left[\frac{(x - x_0)(x - x_0 - h)}{2} f''(\xi(x)) \right] \\ &= \frac{f(x_0 + h) - f(x_0)}{h} + \frac{2(x - x_0) - h}{2} f''(\xi(x)) + \frac{(x - x_0)(x - x_0 - h)}{2} D_x(f''(\xi(x))) \end{aligned}$$

so

$$f'(x) \approx \frac{f(x_0 + h) - f(x_0)}{h}$$

One difficulty with this formula is that we have no information about $D_x f''(\xi(x))$, so the truncation error cannot be estimated. When x is x_0, however, the coefficent of $D_x f''(\xi(x))$ is 0, and the formula simplies to

$$f'(x_0) = \frac{f(x_0 + h) - f(x - 0)}{h} - \frac{h}{2} f''(\xi) \tag{7.19}$$

for small values of h, the difference quotient $\frac{f(x_0+h)-f(x-0)}{h}$ can be used to approximate $f'(x_0)$ with an error bounded by $M\frac{|h|}{2}$, where M is a bound on $|f''(\xi)|$ for $x \in [a, b]$. This formula is known as the forward-difference formula if $h > 0$ and the backward-difference formula if $h < 0$.

[SI*] To obtain general derivative approximation formulas, suppose that $\{x_0, x_1, \ldots, x_n\}$ are $(n+1)$ distinct numbers in some interval I and that $f \in C^{n+1}(I)$. From Theorem 11.2 we will have,

$$f(x) = \sum_{k=0}^{n} f(x_k) L_k(x) + \frac{(x - x_0) \ldots (x - x_n)}{(n+1)!} f^{(n+1)}(\xi(x))$$

for some $\xi(x)$ in I, where $L_k(x)$ denotes the kth Lagrange coefficient polynomial for f at $x_0, x_1, \ldots, x_n$. Differentiating this expression gives

$$f'(x) = \sum_{k=0}^{n} f(x_k)L_k'(x) + D_x\left[\frac{(x-x_0)\ldots(x-x_n)}{(n+1)!}\right] f^{(n+1)}(\xi(x)) + \frac{(x-x_0)\ldots(x-x_n)}{(n+1)!} D_x[f^{(n+1)}(\xi(x))]$$

Again, we have a problem estimating the truncation error unless x is one of the numbers x_j. In this case, the term multiplying $D_x[f^{(n+1)}(\xi(x))]$ is 0, and the formula becomes

$$f'(x_j) = \sum_{k=0}^{n} f(x_k)L_k'(x_j) + \frac{f^{(n+1)}(\xi(x_j))}{(n+1)!} \prod_{k=0,\, k\neq j}^{n} (x_j - x_k) \tag{7.20}$$

which is called an $(n+1)$-point formula to approximate $f'(x_j)$.
In general, using more evaluation points in Eq. (7.20) produces greater accuracy, although the number of functional evaluations and growth of roundoff error discourages this somewhat. The most common formulas are those involving three and five evaluation points.

We first derive some useful three-point formulas and consider aspects of their errors. Since

$$L_0(x) = \frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)}$$

we have

$$L_0'(x) = \frac{2x-x_1-x_2}{(x_0-x_1)(x_0-x_2)}$$

Similarly,

$$L_1'(x) = \frac{2x-x_0-x_2}{(x_1-x_0)(x_1-x_2)}$$
$$L_2'(x) = \frac{2x-x_0-x_1}{(x_2-x_0)(x_2-x_1)}$$

Hence, from Eq. (7.20),

$$f'(x_j) = f(x_0)\left[\frac{2x_j-x_1-x_2}{(x_0-x_1)(x_0-x_2)}\right] + f(x_1)\left[\frac{2x_j-x_0-x_2}{(x_1-x_0)(x_1-x_2)}\right] + f(x_2)\left[\frac{2x_j-x_0-x_1}{(x_2-x_0)(x_2-x_1)}\right] + \frac{1}{6}f^{(3)}(\xi_j) \prod_{k=0,k\neq j}^{2} (x_j - x_k) \tag{7.21}$$

for each $j = 0, 1, 2$, where the notation ξ_j indicates that this point depends on x_j.

The three formulas from Eq. (7.21) become especially useful if the nodes are equally spaced, that is, when

$$x_1 = x_0 + h$$
$$x_2 = x_0 + 2h$$

for some $h \neq 0$.

Using Eq. (7.21) with $x_j = x_0,\ x_1 = x_0 + h$, and $x_2 = x_0 + 2h$ gives

$$f'(x_0) = \frac{1}{h}\left[-\frac{3}{2}f(x_0) + 2f(x_1) - \frac{1}{2}f(x_2)\right] + \frac{h^2}{3}f^{(3)}(\xi_0)$$

Doing the same for $x_j = x_1$ gives

$$f'(x_1) = \frac{1}{h}\left[-\frac{1}{2}f(x_0) + \frac{1}{2}f(x_2)\right] - \frac{h^2}{6}f^{(3)}(\xi_1)$$

and for $x_j = x_2$,

$$f'(x_2) = \frac{1}{h}\left[\frac{1}{2}f(x_0) - 2f(x_1) + \frac{3}{2}f(x_2)\right] + \frac{h^2}{3}f^{(3)}(\xi_2)$$

Since $x_1 = x_0 + h$ and $x_2 = x_0 + 2h$, these formulas can also be expressed as

$$\begin{aligned}
f'(x_0) &= \frac{1}{h}\left[-\frac{3}{2}f(x_0) + 2f(x_0+h) - \frac{1}{2}f(x_0+2h)\right] + \frac{h^2}{3}f^{(3)}(\xi_0)\\
f'(x_0+h) &= \frac{1}{h}\left[-\frac{1}{2}f(x_0) + \frac{1}{2}f(x_0+2h)\right] - \frac{h^2}{6}f^{(3)}(\xi_1)\\
f'(x_0+2h) &= \frac{1}{h}\left[\frac{1}{2}f(x_0) - 2f(x_0+h) + \frac{3}{2}f(x_0+2h)\right] + \frac{h^2}{3}f^{(3)}(\xi_2)
\end{aligned}$$

As a matter of convenience, the variable substitution x_0 for $x_0 + h$ is used in the middle equation to change this formula to an approximation for $f'(x_0)$. A similar change, x_0 for $x_0 + 2h$, is used in the last equation. This gives three formulas for approximating $f'(x_0)$:

$$\begin{aligned}
f'(x_0) &= \frac{1}{2h}\left[-3f(x_0) + 4f(x_0+h) - f(x_0+2h)\right] + \frac{h^2}{3}f^{(3)}(\xi_0)\\
f'(x_0) &= \frac{1}{2h}\left[-f(x_0-h) + f(x_0+h)\right] - \frac{h^2}{6}f^{(3)}(\xi_1)\\
f'(x_0) &= \frac{1}{2h}\left[f(x_0-2h) - 4f(x_0-h) + 3f(x_0)\right] + \frac{h^2}{3}f^{(3)}(\xi_2)
\end{aligned}$$

Finally, note that since the last of these equations can be obtained from the first by simply replacing h with $-h$, there are actually only two formulas:

$$f'(x_0) = \frac{1}{2h}\left[-3f(x_0) + 4f(x_0+h) - f(x_0+2h)\right] + \frac{h^2}{3}f^{(3)}(\xi_0) \tag{7.22}$$

where ξ_0 lies between x_0 and $x_0 + 2h$, and

$$f'(x_0) = \frac{1}{2h}\left[f(x_0+h) - f(x_0-h)\right] - \frac{h^2}{6}f^{(3)}(\xi_1) \tag{7.23}$$

where ξ_1 lies between $(x_0 - h)$ and $(x_0 + h)$.

Although the error in both Eqs. (7.22) and (7.23) are $O(h^2)$, the error in Eq. (7.23) are approximately half the error in Eq. (7.22). This is because Eq. (7.23) uses data on both sides of x_0 and Eq. (7.22) uses data on only one side. Note also that f needs to be evaluated at

only two points in Eq. (7.23), whereas in Eq. (7.22) three evaluations are needed.

Similarly, the five-point formulas that involve evaluating the function at two more points is

$$f'(x_0) = \frac{1}{12h}[f(x_0 - 2h) - 8f(x_0 - h) + 8f(x_0 + h) - f(x_0 + 2h)] + \frac{h^4}{30}f^{(5)}(\xi) \tag{7.24}$$

where ξ lies between $x_0 - 2h$ and $x_0 + 2h$.

The other five-point formula is useful for end-point approximations, particularly with regard to the clamped cubic spline interpolation. It is

$$\begin{aligned} f'(x_0) = \frac{1}{12h}[&-25f(x_0) + 48f(x_0 + h) - 36f(x_0 + 2h) \\ &+ 16f(x_0 + 3h) - 3f(x_0 + 4h)] + \frac{h^4}{5}f^{(5)}(\xi) \end{aligned} \tag{7.25}$$

where ξ lies between x_0 and $x_0 + 4h$. Left-endpoint approximations are found using this formula with $h > 0$ and right-endpoint approximations with $h < 0$.

[SI*] When we use three-point formula that use both sides of point, the optimal choice for h theoretically can be found when we find the minimum of function $e(h)$ (the roundoff errors) and solve it for h [8]

$$e(h) = \frac{\epsilon}{h} + \frac{h^2}{6}M \tag{7.26}$$

where

$$M = \max_{x \in [x_0 - a, x_0 + a]} |f'''(x)| \tag{7.27}$$

with a is a small number such as 0.1. With this formula we can determine the optimal choice of h only for certain cases. In practice, we cannot compute an optimal h to use in approximating the derivative, since we have no knowledge of the third derivative of the function or it will cost more.

Guide 7.1: Approximating Second Order Derivative

Methods can also be derived to find approximations to higher derivatives of a function using only tabulated values of the function at various points. The derivation is algebraically tedious, however, so only a representative procedure will be presented. Expand a function f in a third Taylor polynomial about a point x_0 and evaluate at $x_0 + h$ and $x_0 - h$. Then

$$f(x_0 + h) = f(x_0) + f'(x_0)h + \frac{1}{2}f''(x_0)h^2 + \frac{1}{6}f'''(x_0)h^3 + \frac{1}{24}f^{(4)}(\xi_1)h^4$$

and

$$f(x_0 - h) = f(x_0) - f'(x_0)h + \frac{1}{2}f''(x_0)h^2 - \frac{1}{6}f'''(x_0)h^3 + \frac{1}{24}f^{(4)}(\xi_{-1})h^4$$

where $x_0 - h < \xi_{-1} < x_0 < \xi_1 < x_0 + h$.

If we add these equations, the term involving $f'(x_0)$ cancels and we obtain

$$f(x_0 + h) + f(x_0 - h) = 2f(x_0) + f''(x_0)h^2 + \frac{1}{24}[f^{(4)}(\xi_1) + f^{(4)}(\xi_{-1})]h^4$$

Solving this equaiton for $f''(x_0)$ gives

$$f''(x_0) = \frac{1}{h^2}[f(x_0 - h) - 2f(x_0) + f(x_0 + h)] - \frac{h^2}{24}[f^{(4)}(\xi_1) + f^{(4)}(\xi_{-1})] \tag{7.28}$$

Suppose $f^{(4)}$ is continuous on $[x_0 - h, x_0 + h]$. Since $\frac{1}{2}[f^{(4)}(\xi_1) + f^{(4)}(\xi_{-1})]$ is between $f^{(4)}(\xi_1)$ and $f^{(4)}(\xi_{-1})$, the Intermediate Value Theorem implies that a number ξ exists between ξ_1 and ξ_{-1}, and hence in $(x_0 - h, x_0 + h)$, with

$$f^{(4)}(\xi) = \frac{1}{2}[f^{(4)}(\xi_1) + f^{(4)}(\xi_{-1})]$$

This permits us to rewrite the approximation equation Eq. (7.28) as

$$f''(x_0) = \frac{1}{h^2}[f(x_0 - h) - 2f(x_0) + f(x_0 + h)] - \frac{h^2}{12}f^{(4)}(\xi) \tag{7.29}$$

for some ξ, where $x_0 - h < \xi < x_0 + h$.

[SI*] **Example:**
Let $f(x) = \ln(x)$ and $x_0 = 1.8$, use forward-difference formula to approximate $f'(1.8)$ with $h = 0.1, 0.01, 0.001$.

Then, let $f_2(x) = xe^x$ and $x_0 = 2.0$, use three-point formulas and five-point formulas to approximate $f'(2.0)$ with $h = 0.1$ and $h = -0.1$.

Solution:
Since $f'(x) = \frac{1}{x}$, the exact value of $f'(1.8)$ is 0.55555.

For f_2, since $f_2'(x) = (x+1)e^x$, the exact value of $f_2'(x)$ is 22.167168.

In SymIntegration we can approximate the numerical differentiation problem with forward-difference formula:
numericaldifferentiation(const Symbolic &f, const Symbolic &x, double x_0, double h)
where f is the function of x, x_0 is the value that we want to approximate, h is the step size.

For three-point formulas, the functions in SymIntegration are:
numericaldifferentiation3pointoneside(const Symbolic &f, const Symbolic &x, double x_0, double h)
numericaldifferentiation3pointbothsides(const Symbolic &f, const Symbolic &x, double x_0, double h)

For five-point formulas, the functions in SymIntegration are:
numericaldifferentiation5pointoneside(const Symbolic &f, const Symbolic &x, double x_0, double h)
numericaldifferentiation5pointbothsides(const Symbolic &f, const Symbolic &x, double x_0, double h)

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Numerical Differentiation ]# make
g++     -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Numerical Differentiation ]# ./main
For f(x)=ln(x)
f'(1.8) with h=0.001 : 0.5554012917
f'(1.8) with h=0.01 : 0.5540180376
f'(1.8) with h=0.1 : 0.5406722127

For f(x)=x*exp(x)
Three Point Formulas one side:
f'(2.0) with h=0.1 : 22.03230487
f'(2.0) with h=-0.1 : 22.05452134
Three Point Formulas both sides:
f'(2.0) with h=0.1 : 22.22878688
f'(2.0) with h=0.2 : 22.41416066
Five Point Formulas one side:
f'(2.0) with h=0.1 : 22.16699562
Five Point Formulas both sides:
f'(2.0) with h=0.1 : 22.16591457

Time taken by function: 71978 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Numerical Differentiation ]#
```

Figure 7.26: *The computation of numerical differentiation for $f(x) = \ln(x)$ with forward-difference formula and for $f(x) = xe^x$ with three-point and five-point formulas (**SymIntegration/Examples/Numerical Methods/Test SymIntegration Numerical Differentiation//main.cpp**).*

[SI*] **Richardson's Extrapolation**
Richardson's extrapolation is used to generate high-accuracy results while using low-order formulas. Extrapolation can be applied whenever it is known that an approximation technique has an error term with a predictable form, one that depends on a parameter, usually the step size h.

Suppose that for each number $h \neq 0$ we have a formula $N(h)$ that approximates an unknown value M and that the truncation error involved with the approximation has the form

$$M - N(h) = K_1 h + K_2 h^2 + K_3 h^3 + \dots,$$

for some collection of unknown constants $K_1, K_2, K_3, \dots$.

Since the truncation error is $O(h)$, we would expect, for example, that

$$M - N(0.1) \approx 0.1K_1$$
$$M - N(0.01) \approx 0.01K_1$$

and, in general, $M - N(h) \approx K_1 h$, unless there was a large variation in magnitude among the constants $K_1, K_2, K_3, \ldots$.

The object of extrapolation is to find an easy way to combine the rather inaccurate $O(h)$ approximations in an appropriate way to produce formulas with a higher-order truncation error. Suppose, for example, we could combine the $N(h)$ formulas so as to produce an $O(h^2)$ approximation formula, $\hat{N}(h)$, for M with

$$M - \hat{N}(h) = \hat{K}_2 h^2 + \hat{K}_3 h^3 + \ldots,$$

for some, again unknown, collection of constants $\hat{K}_1, \hat{K}_2, \ldots$. Then we would have

$$M - \hat{N}(0.1) \approx 0.01\hat{K}_2$$
$$M - \hat{N}(0.01) \approx 0.0001\hat{K}_2$$

and so on. If the constants K_1 and $\hat{K}_2$ are roughly of the same magnitude, then the $\hat{N}(h)$ approximations would be much better than the corresponding $N(h)$ approximations. The extrapolation continues by combining the $\hat{N}(h)$ approximations in a manner that produces formulas with $O(h^3)$ truncation error, and so on.
To see specifically how we can generate these higher-order formulas, let us consider the formula for approximating M of the form

$$M = N(h) + K_1 h + K_2 h^2 + K_3 h^3 + \ldots \tag{7.30}$$

Since the formula is assumed to hold for all positive h, consider the result when we replace the parameter h by half its value. Then we have the formula

$$M = N\left(\frac{h}{2}\right) + K_1\frac{h}{2} + K_2\frac{h^2}{4} + K_3\frac{h^3}{8} + \ldots$$

Subtracting Eq. (7.30) from twice this equation eliminates the term involving K_1 and gives

$$M = \left[N\left(\frac{h}{2}\right) + \left(N\left(\frac{h}{2}\right) - N(h)\right)\right] + K_2\left(\frac{h^2}{2} - h^2\right) + K_3\left(\frac{h^3}{4} - h^3\right) + \ldots$$

to facilitate this discussion, we define $N_1(h) \equiv N(h)$ and

$$N_2(h) = N_1\left(\frac{h}{2}\right) + \left[N_1\left(\frac{h}{2}\right) - N_1(h)\right]$$

Then we have the $O(h^2)$ approximation formula for M:

$$M = N_2(h) - \frac{K_2}{2}h^2 - \frac{3K_3}{4}h^3 - \ldots \tag{7.31}$$

If we now replace h by $\frac{h}{2}$ in this formula, we have

$$M = N_2\left(\frac{h}{2}\right) - \frac{K_2}{8}h^2 - \frac{3K_3}{32}h^3 - \ldots \tag{7.32}$$

this can be combined with Eq. (7.31) to eliminate the h^2 term. Specifically, subtracting Eq. (7.31) from 4 times Eq. (7.32) gives

$$3M = 4N_2\left(\frac{h}{2}\right) - N_2(h) + \frac{3K_3}{8}h^3 + \dots$$

and dividing by 3 gives an $O(h^3)$ formula for approximating M:

$$M = \left[N_2\left(\frac{h}{2}\right) + \frac{N_2\left(\frac{h}{2}\right) - N_2(h)}{3}\right] + \frac{K_3}{8}h^3 + \dots$$

By defining

$$N_3(h) \equiv N_2\left(\frac{h}{2}\right) + \frac{N_2\left(\frac{h}{2}\right) - N_2(h)}{3}$$

we have the $O(h^3)$ formula:

$$M = N_3(h) + \frac{K_3}{8}h^3 + \dots$$

The process is continued by constructing an $O(h^4)$ approximation

$$N_4(h) = N_3\left(\frac{h}{2}\right) + \frac{N_3\left(\frac{h}{2}\right) - N_3(h)}{7}$$

an $O(h^5)$ approximation

$$N_5(h) = N_4\left(\frac{h}{2}\right) + \frac{N_4\left(\frac{h}{2}\right) - N_4(h)}{15}$$

and so on. In general, if M can be written in the form

$$M = N(h) + \sum_{j=1}^{m-1} K_j h^j + O(h^m) \tag{7.33}$$

then for each $j = 2, 3, \dots, m$, we have an $O(h^j)$ approximation of the form

$$N_j(h) = N_{j-1}\left(\frac{h}{2}\right) + \frac{N_{j-1}\left(\frac{h}{2}\right) - N_{j-1}(h)}{2^{j-1} - 1} \tag{7.34}$$

These approximations are generated by rows in the order indicated by the numbered entries. This is done to take best advantage of the highest-order formulas. The extrapolation can be applied whenever the truncation error for a formula has the form

$$\sum_{j=1}^{m-1} K_j h^{\alpha_j} + O(h^{\alpha_m})$$

for a collection of constants K_j and when $\alpha_1 < \alpha_2 < \alpha_3 < \cdots < \alpha_m$.

$O(h)$	$O(h^2)$	$O(h^3)$	$O(h^4)$
1 : $N_1(h) \equiv N(h)$			
2 : $N_1\left(\frac{h}{2}\right) \equiv N\left(\frac{h}{2}\right)$	**3 :** $N_2(h)$		
4 : $N_1\left(\frac{h}{4}\right) \equiv N\left(\frac{h}{4}\right)$	**5 :** $N_2\left(\frac{h}{2}\right)$	**6 :** $N_3(h)$	
7 : $N_1\left(\frac{h}{8}\right) \equiv N\left(\frac{h}{8}\right)$	**8 :** $N_2\left(\frac{h}{4}\right)$	**9 :** $N_3\left(\frac{h}{4}\right)$	**10 :** $N_4(h)$

Table 7.1: *The extrapolation table*

[SI*] The centered difference formula in Eq. (7.23) to approximate $f'(x_0)$ can be expressed with an error formula

$$f'(x_0) = \frac{1}{2h}[f(x_0+h) - f(x_0-h)] - \frac{h^2}{6}f'''(x_0) - \frac{h^4}{120}f^{(5)}(x_0) - \dots$$

Since this error formula contains only even powers of h, extrapolation is more effective than as outlined in the opening discussin. In this case, we have the $O(h^2)$ approximation

$$f'(x_0) = N_1(h) - \frac{h^2}{6}f'''(x_0) - \frac{h^4}{120}f^{(5)}(x_0) - \dots \tag{7.35}$$

where

$$N_1(h) \equiv N(h) = \frac{1}{2h}[f(x_0+h) - f(x_0-h)]$$

Replacing h by $\frac{h}{2}$ in this formula gives the approximation

$$f'(x_0) = N_1\left(\frac{h}{2}\right) - \frac{h^2}{24}f'''(x_0) - \frac{h^4}{1920}f^{(5)}(x_0) - \dots$$

Subtracting Eq. (7.35) from 4 times this equation eliminates the $O(h^2)$ term that involves $f'''(x_0)$ and gives

$$3f'(x_0) = 4N_1\left(\frac{h}{2}\right) - N_1(h) + \frac{h^4}{160}f^{(5)}(x_0) + \dots$$

Dividing by 3 provides an $O(h^4)$ formula

$$f'(x_0) = N_2(h) + \frac{h^4}{480}f^{(5)}(x_0) + \dots$$

where

$$N_2(h) = N_1\left(\frac{h}{2}\right) + \frac{N_1\left(\frac{h}{2}\right) - N_1(h)}{3}$$

Continuing this procedure gives, for each $j = 2, 3, \dots$, and $O(h^{2j})$ approximation

$$N_j(h) = N_{j-1}\left(\frac{h}{2}\right) + \frac{N_{j-1}\left(\frac{h}{2}\right) - N_{j-1}(h)}{4^{j-1}-1} \tag{7.36}$$

Notice that the denominator of the quotient is $4^{j-1}-1$ instead of $2^{j-1}-1$ because we are now eliminating powers of h^2 instead of powers of h. Since $\left(\frac{h}{2}\right)^2 = \frac{h^2}{4}$, the multipliers used to eliminate the powers of h^2 are powers of 4 instead of 2.

[SI*] **Example:**
Suppose that $x_0 = 2.0, h = 0.2$, and $f(x = xe^x)$, create the extrapolation table with Richardson's extrapolation.

Solution:

$$N_1(0.2) = N(0.2) = \frac{1}{0.4}[f(2.2) - 1.8] = 22.414160$$
$$N_1(0.1) = N(0.1) = 22.228786$$
$$N_1(0.05) = N(0.05) = 22.182564$$

The exact value of $f'(x) = xe^x + e^x$ at $x_0 = 2.0$ to six decimal places is 22.167168. We see that $N_1(0.05)$ has only one decimal place of accuracy, while $N_3(0.2)$ are exact up to 6 decimal places.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Richardson's Extrapolation ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Richardson's Extrapolation ]# ./main
For f(x) = x *exp(x), x0 = 2, h = 0.2, order or error = O(h^4), level =3
Richardson's Extrapolation table:

          22.41416066                    0                    0
          22.22878688           22.16699562                    0
          22.18256486           22.16715752          22.16716831

                  888

Time taken by function: 7573 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Richardson's Extrapolation ]#
```

Figure 7.27: *The computation of Richardson's extrapolation for $x_0 = 2.0, h = 0.2$, and $f(x = xe^x)$ (**SymIntegration/Examples/Numerical Methods/Test SymIntegration Richardson's Extrapolation/main.cpp**).*

We use the Richardson's extrapolation formula of this (with centered difference formula to approximate $f'(x_0)$):

$$N_j(h) = N_{j-1}\left(\frac{h}{2}\right) + \frac{N_{j-1}\left(\frac{h}{2}\right) - N_{j-1}(h)}{m^{j-1}-1} \tag{7.37}$$

with m as the order of error as in $O(h^m)$.

In SymIntegration we can use this function to obtain the extrapolation table that is using the Eq. (7.37):
richardsonextrapolation(const Symbolic &f, const Symbolic &x, double x_0, double h, int l, int m)
where f is the function of x, x_0 is the value that we want to approximate, h is the step size, l is the level / size of the square matrix, and m is the order of error.

V. Numerical Integration

[SI*] The need often arises for evaluating the definite integral of a function that has no explicit antiderivative or whose antiderivative is not easy to obtain. The basic method involved in

approximating $\int_a^b f(x)\ dx$ is called numerical quadrature. It uses a sum

$$\sum_{i=0}^{n} a_i f(x_i)$$

to approximate $\int_a^b f(x)\ dx$.

The methods of quadrature are based on the interpolation polynomials. We first select a set of distinct nodes $\{x_0, x_1, \ldots, x_n\}$ from the interval $[a, b]$. Then we integrate the Lagrange interpolating polynomial

$$P_n(x) = \sum_{i=0}^{n} f(x_i) L_i(x)$$

and its truncation error term over $[a, b]$ to obtain

$$\begin{aligned}\int_a^b f(x)\ dx &= \int_a^b \sum_{i=0}^{n} f(x_i) L_i(x)\ dx + \int_a^b \prod_{i=0}^{n} (x - x_i) \frac{f^{(n+1)}(\xi(x))}{(n+1)!}\ dx \\ &= \sum_{i=0}^{n} a_i f(x_i) + \frac{1}{(n+1)!} \int_a^b \prod_{i=0}^{n} n(x - x_i) f^{(n+1)}(\xi(x))\ dx\end{aligned}$$

where $\xi(x)$ is in $[a, b]$ for each x and

$$a_i = \int_a^b L_i(x)\ dx$$

for each $i = 0, 1, 2, \ldots, n$.

The quadrature formula is, therefore,

$$\int_a^b f(x)\ dx \approx \sum_{i=0}^{n} a_i f(x_i)$$

with error given by

$$E(f) = \frac{1}{(n+1)!} \int_a^b \prod_{i=0}^{n} (x - x_i) f^{(n+1)}(\xi(x))\ dx$$

[SI*] To derive the Trapezoidal rule for approximating $\int_a^b f(x)\ dx$. let $x_0 = a,\ x_1 = b,\ h = b - a$ and use the linear Lagrange polynomial:

$$P_1(x) = \frac{(x - x_1)}{(x_0 - x_1)} f(x_0) + \frac{(x - x_0)}{(x_1 - x_0)} f(x_1)$$

Then,

$$\begin{aligned}\int_a^b f(x)\ dx = \int_{x_0}^{x_1} \left[\frac{(x - x_1)}{(x_0 - x_1)} f(x_0) + \frac{(x - x_0)}{(x_1 - x_0)} f(x_1) \right]\ dx \\ + \frac{1}{2} \int_{x_0}^{x_1} f''(\xi(x))(x - x_0)(x - x_1)\ dx\end{aligned} \tag{7.38}$$

Since $(x - x_0)(x - x_1)$ does not change sign on $[x_0, x_1]$, the Weighted Mean Value Theorem for Integrals can be applied to the error term to give, for some ξ in (x_0, x_1).

$$\begin{aligned}\int_{x_0}^{x_1} f''(\xi(x))(x - x_0)(x - x_1)\, dx &= f''(\xi(x)) \int_{x_0}^{x_1} (x - x_0)(x - x_1)\, dx \\ &= f''(\xi(x)) \left[\frac{x^3}{3} - \frac{(x_1 + x_0)}{2}x^2 + x_0 x_1 x\right]_{x_0}^{x_1} \\ &= -\frac{h^3}{6} f''(\xi)\end{aligned}$$

Consequently, Eq. (7.38) implies that

$$\begin{aligned}\int_a^b f(x)\, dx &= \left[\frac{(x - x_1)^2}{2(x_0 - x_1)} f(x_0) + \frac{(x - x_0)^2}{2(x_1 - x_0)} f(x_1)\right] - \frac{h^3}{12} f''(\xi) \\ &= \frac{(x_1 - x_0)}{2}[f(x_0) + f(x_1)] - \frac{h^3}{12} f''(\xi)\end{aligned}$$

Since $h = x_1 - x_0$, we have the following rule:

$$\int_a^b f(x)\, dx = \frac{h}{2}[f(x_0) + f(x_1)] - \frac{h^3}{12} f''(\xi) \tag{7.39}$$

this is called the Trapezoidal rule because when f is a function with positive values, $\int_a^b f(x)\, dx$ is approximated by the area in a trapezoid.

Since the error term for the Trapezoidal rule involves f'', the rule gives the exact result when applied to any function whose second derivative is identically zero, that is, any polynomial of degree one or less.

[SI*] Simpson's rule results from integrating over $[a, b]$ the second Lagrange polynomial with nodes $x_0 = a$, $x_2 = b$, and $x_1 = a + h$, where $h = \frac{b-a}{2}$.

Therefore,

$$\begin{aligned}\int_a^b f(x)\, dx = \int_{x_0}^{x_2} \Big[&\frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)} f(x_0) + \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)} f(x_1) \\ &+ \frac{(x - x_0)(x - x_1)}{(x_2 - x_0)(x_2 - x_1)} f(x_2)\Big]\, dx \\ &+ \int_{x_0}^{x_2} \frac{(x - x_0)(x - x_1)(x - x_2)}{6} f^{(3)}(\xi(x))\, dx\end{aligned}$$

Deriving Simpson's rule in this manner, however, provides only an $O(h^4)$ error term involving $f^{(3)}$. By approaching this problem in another way, a higher-order term involving $f^{(4)}$ can be derived.

To illustrate this alternative formula, suppose that f is expanded in the third Taylor polynomial about x_1. Then for each x in $[x_0, x_2]$, a number $\xi(x)$ in (x_0, x_2) exists with

$$f(x) = f(x_1) + f'(x_1)(x - x_1) + \frac{f''(x_1)}{2}(x - x_1)^2 + \frac{f'''(x_1)}{6}(x - x_1)^3 + \frac{f^{(4)}(\xi(x))}{24}(x - x_1)^4$$

and

$$\int_{x_0}^{x_2} f(x)\,dx = \Big[f(x_1)(x-x_0) + \frac{f'(x_1)}{2}(x-x_1)^2 + \frac{f''(x_1)}{6}(x-x_1)^3 + \frac{f'''(x_1)}{24}(x-x_1)^4\Big]_{x_0}^{x_2} + \frac{1}{24}\int_{x_0}^{x_2} f^{(4)}(\xi(x))(x-x_1)^4\,dx \tag{7.40}$$

since $(x-x_1)^4$ is never negative on $[x_0, x_2]$, the Weighted Mean Value Theorem for Integrals implies that

$$\frac{1}{24}\int_{x_0}^{x_2} f^{(4)}(\xi(x))(x-x_1)^4\,dx = \frac{f^{(4)}(\xi_1)}{24}\int_{x_0}^{x_2}(x-x_1)^4\,dx = \frac{f^{(4)}(\xi_1)}{120}(x-x_1)^5\Big]_{x_0}^{x_2}$$

for some number ξ_1 in (x_0, x_2).

However, $h = x_2 - x_1 = x_1 - x_0$, so

$$(x_2-x_1)^2 - (x_0-x_1)^2 = (x_2-x_1)^4 - (x_0-x_1)^4 = 0$$

whereas

$$(x_2-x_1)^3 - (x_0-x_1)^3 = 2h^3$$
$$(x_2-x_1)^5 - (x_0-x_1)^5 = 2h^5$$

Consequently, Eq. (7.40) can be rewritten as

$$\int_{x_0}^{x_2} f(x)\,dx = 2hf(x_1) + \frac{h^3}{3}f''(x_1) + \frac{f^{(4)}(\xi_1)}{60}h^5$$

If we now replace $f'''(x_1)$ by the approximation given in Eq. (7.29), we have

$$\begin{aligned}\int_{x_0}^{x_2} f(x)\,dx &= 2hf(x_1) + \frac{h^3}{3}\left[\frac{1}{h^2}[f(x_0) - 2f(x_1) + f(x_2)] - \frac{h^2}{12}f^{(4)}(\xi_2)\right] + \frac{f^{(4)}(\xi_1)}{60}h^5 \\ &= \frac{h}{3}[f(x_0) + 4f(x_1) + f(x_2)] - \frac{h^5}{12}\left[\frac{1}{3}f^{(4)}(\xi_2) - \frac{1}{5}f^{(4)}(\xi_1)\right]\end{aligned}$$

it can be shown by alternative methods that the values ξ_1 and ξ_2 in this expression can be replaced by a common value ξ in (x_0, x_2). This gives Simpson's rule:

$$\int_{x_0}^{x_2} f(x)\,dx = \frac{h}{3}[f(x_0) + 4f(x_1) + f(x_2)] - \frac{h^5}{90}f^{(4)}(\xi) \tag{7.41}$$

Since the error term involves the fourth derivative of f, Simpson's rule gives exact results when applied to any polynomial of degree three or less.

[SI*] **Example:**
Compute the definite integral for $x^2, x^4, \frac{1}{x+1}, \sqrt{1+x^2}, \sin\ x, e^x$ using the Trapezoidal rule and Simpson's rule on the interval $[0, 2]$.

Solution:

VI. Iterative Techniques in Matrix Algebra

[SI*] **Iterative Techniques for Solving Linear Systems**
Iterative techniques are seldom used for solving linear systems of small dimension since the time required for sufficient accuracy exceeds that requried for direct techniques such as Gaussian elimination. For large systems with a high percentage of 0 entries (sparse matrix), however, these techniques are efficient in terms of both computer storage and computation. Systems of this type arise frequently in circuit analysis and in the numerical solution of boundary-value problems and partial differential equations.
An iterative technique to solve the $n \times n$ linear system

$$A\boldsymbol{x} = \boldsymbol{b}$$

starts with an initial approximation $\boldsymbol{x}^{(0)}$ to the solution $\boldsymbol{x}$ and generates a sequence of vectors $\{\boldsymbol{x}^{(k)}\}_{k=0}^{\infty}$ that converges to $\boldsymbol{x}$. Iterative techniques involve a process that converts the system $A\boldsymbol{x} = \boldsymbol{b}$ into an equivalent system of the form $\boldsymbol{x} = T\boldsymbol{x} + \boldsymbol{c}$ for some fixed matrix T and vector $\boldsymbol{c}$.
After the initial vector $\boldsymbol{x}^{(0)}$ is selected, the sequence of approximate solution vectors is generated by computing

$$\boldsymbol{x}^{(k)} = T\boldsymbol{x}^{(k-1)} + \boldsymbol{c}$$

for each $k = 1, 2, 3, \ldots$. This result should be reminiscent of the fixed-point iteration.

i. Jacobi Iterative Method

[SI*] The Jacobi iterative method is an algorithm used in numerical linear algebra to approximate the solution of a system of linear equations

$$A\boldsymbol{x} = \boldsymbol{b}$$

It works best and is guaranteed to converge if the coefficient matrix (matrix A) is strictly diagonally dominant system of linear equations / matrices (meaning the absolute value of each diagonal element must be greater than the sum of the absolute values of the other elements in that row).
The Jacobi method solves a system of linear equations by breaking down a complex set into simpler isolated parts. Unlike the Gauss-Seidel method which uses updated values immediately, the Jacobi method updates all variables simultaneously using only the values from the previous iteration.
Each diagonal element is solved for, and an approximate value is plugged in. The process is then iterated until it converges. This algorithm is a stripped-down version of the Jacobi transformation method of matrix diagonalization. The method is named after Carl Gustav Jacob Jacobi.

[SI*] Let $A\boldsymbol{x} = \boldsymbol{b}$ be a square system of n linear equations, where:

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix}, \quad \boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad \boldsymbol{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

When A and $\boldsymbol{b}$ are known, and $\boldsymbol{x}$ is unknown, we can use the Jacobi method to approximate $\boldsymbol{x}$. The vector $\boldsymbol{x}^{(0)}$ denotes our initial guess for $\boldsymbol{x}$. We denote $\boldsymbol{x}^{(k)}$ as the k-th approximation or iteration of $\boldsymbol{x}$, and $\boldsymbol{x}^{(k-1)}$ is the previous (or $k-1$) iteration of $\boldsymbol{x}$.

[SI*] Matrix A can be decomposed into a diagonal component D, a lower triangular part L and an upper triangular part U:

$$A = D + L + U$$

where

$$A = \begin{bmatrix} a_{11} & 0 & \dots & 0 \\ 0 & a_{22} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & a_{nn} \end{bmatrix}, \quad L + U = \begin{bmatrix} 0 & a_{12} & \dots & a_{1n} \\ a_{21} & 0 & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & 0 \end{bmatrix},$$

The solution is then obtained iteratively via

$$\begin{aligned} \boldsymbol{x}^{(k)} &= D^{-1}(\boldsymbol{b} - (L + U)\boldsymbol{x}^{(k-1)}) \\ \boldsymbol{x}^{(k)} &= T\boldsymbol{x}^{(k-1)} + \boldsymbol{c} \end{aligned}$$

with $T = -D^{-1}(L + U)$ and $\boldsymbol{c} = D^{-1}\boldsymbol{b}$.
The element-based formula for each row i is thus

$$x_i^{(k)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1, j \neq i}^{n} a_{ij} x_j^{(k-1)} \right)$$

with $i = 1, 2, \dots, n$.

[SI*] **Convergence**
The standard convergence condition (for any iterative method) is when the spectral radius of the iteration matrix is less than 1:

$$\rho(D^{-1}(L + U)) < 1$$

A sufficient (but not necessary) condition for the method to converge is that the matrix A is strictly or irreducibly diagonally dominant. Strict row diagonal dominance means that for each row, the absolute value of the diagonal term is greater than the sum of absolute values of other terms:

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|$$

The Jacobi method sometimes converges even if these conditions are not satisfied. Note that the Jacobi method does not converge for every symmetric positive-definite matrix.

Definition 7.5: The Jacobi Iterative Method

To solve $A\boldsymbol{x} = \boldsymbol{b}$, we will use an initial approximation $\boldsymbol{x}^{(0)}$.
The Jacobi iterative algorithm steps:

1. **Rearrange Equations**
   Rewrite each i-th equation in the system (matrix A of size $n \times n$) to isolate the diagonal variable x_i, $i = 1, 2, \ldots, n$.
   We will convert $A\boldsymbol{x} = \boldsymbol{b}$ to the form $\boldsymbol{x}^{(k)} = T\boldsymbol{x}^{k-1} + \boldsymbol{c}$ with
   $$T = -D^{-1}(L + U)$$
   $$\boldsymbol{c} = D^{-1}\boldsymbol{b}$$
   D is the diagonal matrix whose diagonal entries are those of A, L is the strictly lower-triangular part of A, and U is the strictly upper-triangular part of A.

2. **Make an Initial Guess**
   Choose starting values for all variables (usually zeros if no other information is available).

3. **Iterate**
   Substitute the current guesses into the right-hand side of the rearranged equations to calculate a new set of approximations.
   We will use the Jacobi iterative formula, for each variable x_i at iteration k, the new value is calculated using the values from the previous iteration $(k-1)$:
   $$x_i^{(k)} = \frac{1}{a_{ii}}\left(b_i - \sum_{j=1, j\neq i}^{n} a_{ij}x_j^{(k-1)}\right)$$
   for $k \geq 1$.

4. **Check Convergence**
   Repeat the substitution process until the changes between iterations fall below your desired error tolerance (ϵ)
   $$\frac{||\boldsymbol{x}^{(k)} - \boldsymbol{x}^{(k-1)}||}{||\boldsymbol{x}^{(k)}||}$$

Remark:
The algorithm works best and is guaranteed to converge if the coefficient matrix is strictly diagonal dominant
$$|a_{ii}| > \sum_{j\neq i} |a_{ij}|$$

ii. Gauss-Seidel Method

[SI*] The Gauss-Seidel method is an iterative technique used to solve a linear system $A\boldsymbol{x} = \boldsymbol{b}$. Unlike the Jacobi method, Gauss-Seidel updates the solution vector immediately within the same iteration, allowing for faster convergence.
For the algorithm to converge reliably, the matrix A should ideally be diagonally dominant

(i.e., the magnitude of the diagonal element in each row is greater than or equal to the sum of the magnitudes of the other elements in that row).

Lemma 7.1: Spectral Radius

If the spectral radius $\rho(T)$ satisfies $\rho(T) < 1$, then $(I - T)^{-1}$ exists, and

$$(I - T)^{-1} = I + T + T^2 + \cdots = \sum_{j=0}^{\infty} T^j$$

Theorem 7.3: Strictly Diagonal Dominant

If A is strictly diagonal dominant, then for any choice of $\boldsymbol{x}^{(0)}$, both the Jacobi and Gauss-Seidel methods give sequences $\{\boldsymbol{x}^{(k)}\}_{k=0}^{\infty}$ that converge to the unique solution of $A\boldsymbol{x} = \boldsymbol{b}$.

Theorem 7.4: Stein-Rosenberg

If $a_{ij} \leq 0$, for each $i \neq j$ and $a_{ii} > 0$, for each $i = 1, 2, \ldots, n$, then one and only one of the following statements holds:

(a) $0 \leq \rho(T_g) < \rho(T_j) < 1$;

(b) $1 < \rho(T_j) < \rho(T_g)$;

(c) $\rho(T_j) = \rho(T_g) = 0$;

(d) $\rho(T_j) = \rho(T_g) = 1$;

when one method gives convergence (Jacobi or Gauss-Seidel), then both give convergence, and the Gauss-Seidel method converges faster than the Jacobi method.
Since the rate of convergence od a procedure depends on the spectral radius of the matrix associated with the method, one way to select a procedure to accelerate convergence is to choose a method whose associated matrix has minimal spectral radius.

Definition 7.6: The Gauss-Seidel Iterative Method

To solve $A\boldsymbol{x} = \boldsymbol{b}$, we will use an initial approximation $\boldsymbol{x}^{(0)}$.
The Gauss-Seidel iterative algorithm steps:

1. **Rearrange Equations**
   Rewrite each i-th equation in the system (matrix A of size $n \times n$) to isolate the diagonal variable x_i, $i = 1, 2, \ldots, n$.
   We will convert $A\boldsymbol{x} = \boldsymbol{b}$ to the form $\boldsymbol{x}^{(k)} = T\boldsymbol{x}^{k-1} + \boldsymbol{c}$ with
   $$T = -D^{-1}(L + U)$$
   $$\boldsymbol{c} = D^{-1}\boldsymbol{b}$$
   D is the diagonal matrix whose diagonal entries are those of A, L is the strictly lower-triangular part of A, and U is the strictly upper-triangular part of A.

2. **Make an Initial Guess**
   Choose starting values for all variables (usually zeros if no other information is available).

3. **Iterate**
   Substitute the current guesses into the right-hand side of the rearranged equations to calculate a new set of approximations.
   We will use the Gauss-Seidel iterative formula, for each variable x_i at iteration k, the new value is calculated using the values from the previous iteration $(k-1)$:
   $$x_i^{(k)} = \frac{1}{a_{ii}}\left(b_i - \sum_{j<i} a_{ij}x_j^{(k)} - \sum_{j>i} a_{ij}x_j^{(k-1)}\right)$$
   for $k \geq 1$.

4. **Check Convergence**
   Repeat the substitution process until the changes between iterations fall below your desired error tolerance (ϵ)
   $$\frac{||\boldsymbol{x}^{(k)} - \boldsymbol{x}^{(k-1)}||}{||\boldsymbol{x}^{(k)}||}$$

Remark:
For the method to guarantee convergence, the matrix A should ideally be strictly diagonally dominant
$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|$$

iii. Successive Over-Relaxation (SOR) Method

[SI*] The Successive Over-Relaxation (SOR) method is an iterative algorithm used to solve a system of linear equations $A\boldsymbol{x} = \boldsymbol{b}$. It is an optimized variant of the Gauss-Seidel method, that accelerates convergence by introducing a relaxation factor, ω (omega).

Feature	Jacobi Method	Gauss-Seidel Method
Value Updates	Simultaneous (uses array from the previous step).	Immediate (uses the newest available values).
Memory cost	Higher (requires holding two arrays: $\boldsymbol{x}^{(k+1)}$ and $\boldsymbol{x}^{(k)}$).	Lower (overwrites a single array in place).
Convergence Speed	Slower.	Generally twice as fast.
Parallelization	Excellent. Elements are entirely independent during loops	Difficult. Dependencies exist between elements within a single loop.

Table 7.2: *Comparison between Jacobi method and Gauss-Seidel method.*

Definition 7.7: Residual Vector

Suppose $\tilde{\boldsymbol{x}} \in \mathbb{R}^n$ is an approximation to the solution of the linear system defined by $A\boldsymbol{x} = \boldsymbol{b}$. The residual vector for $\tilde{\boldsymbol{x}}$ with respect to this system is $\boldsymbol{r} = \boldsymbol{b} - A\tilde{\boldsymbol{x}}$.

In procedures such as Jacobi or Gauss-Seidel methods, a residual vector is associated with each calculation of an approximation component to the solution vector. The object is to generate a sequence of approximations that will cause the residual vectors to converge rapidly to zero. Suppose we let

$$\boldsymbol{r}_i^{(k)} = (r_{1i}^{(k)}, r_{2i}^{(k)}, \ldots, r_{ni}^{(k)})^T$$

denote the residual vector for the Gauss-Seidel method corresponding to the approximate solution vector $\boldsymbol{x}_i^{(k)}$ defined by

$$\boldsymbol{x}_i^{(k)} = (x_1^{(k)}, x_2^{(k)}, \ldots, x_{i-1}^{(k)}, x_i^{(k-1)}, \ldots, x_n^{(k-1)})^T$$

The m-th component of $\boldsymbol{r}_i^{(k)}$ is

$$r_{mi}^{(k)} = b_m - \sum_{j=1}^{i-1} a_{mj}\boldsymbol{x}_j^{(k)} - \sum_{j=i}^{n} a_{mj}\boldsymbol{x}_j^{(k-1)} \tag{7.42}$$

for each $m = 1, 2, \ldots, n$.
In particular, the i-th component of $\boldsymbol{r}_i^{(k)}$ is

$$r_{ii}^{(k)} = b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k)} - \sum_{j=i+1}^{n} a_{ij}x_j^{(k-1)} - a_{ii}x_i^{(k-1)}$$

so

$$a_{ii}x_i^{(k-1)} + r_{ii}^{(k)} = b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k)} - \sum_{j=i+1}^{n} a_{ij}x_j^{(k-1)} \tag{7.43}$$

Recall, however, that in the Gauss-Seidel method, $x_i^{(k)}$ is chosen to be

$$x_i^{(k)} = \frac{1}{a_{ii}}\left[b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k)} - \sum_{j=i+1}^{n} a_{ij}x_j^{(k-1)}\right] \tag{7.44}$$

so Eq. (7.43) can be rewritten as

$$a_{ii}x_i^{(k-1)} + r_{ii}^{(k)} = a_{ii}x_i^{(k)}$$

Consequently, the Gauss-Seidel method can be characterized as choosing $x_i^{(k)}$ to satisfy

$$x_i^{(k)} = x_i^{(k-1)} + \frac{r_{ii}^{(k)}}{a_{ii}} \tag{7.45}$$

We can derive another connection between the residual vectors and the Gauss-Seidel technique. Consider the residual vector $\boldsymbol{r}_{i+1}^{(k)}$, associated with the vectors $\boldsymbol{x}_{i+1}^{(k)} = (x_1^{(k)}, \ldots, x_i^{(k)}, x_{i+1}^{(k-1)}, \ldots x_n^{(k-1)})^T$. By Eq. (7.42), the i-th component of $\boldsymbol{r}_{i+1}^{(k)}$ is

$$\begin{aligned} r_{i,i+1}^{(k)} &= b_i - \sum_{j=1}^{i} a_{ij}x_j^{(k)} - \sum_{j=i+1}^{n} a_{ij}x_j^{(k-1)} \\ &= b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k)} - \sum_{j=i+1}^{n} a_{ij}x_j^{(k-1)} - a_{ii}x_i^{(k)} \end{aligned}$$

Equation (7.44) implies that $r_{i,i+1}^{(k)} = 0$. In a sense, then, the Gauss-Seidel technique is also characterized choosing $x_{i+1}^{(k)}$ in such way that the i-th component of $\boldsymbol{r}_{i+1}^{(k)}$ is zero.

Choosing $x_{i+1}^{(k)}$ so that one coordinate of the residual vector is zero, however, is not the most efficient way to reduce the norm of the vector $\boldsymbol{r}_{i+1}^{(k)}$. If we modify the Gauss-Seidel procedure, as given by Eq. (7.45), to

$$x_i^{(k)} = x_i^{(k-1)} + \omega\frac{r_{ii}^{(k)}}{a_{ii}} \tag{7.46}$$

for certain choices of positive ω, we can reduce the norm of the residual vector and obtain significantly faster convergence.

Methods involving Eq. (7.46) are called relaxation methods. For choices of ω with $0 < \omega < 1$, the procedures are called under-relaxation methods and can be used to obtain convergence of some systems that are not convergent by the Gauss-Seidel method. For choices of ω with $1 < \omega < 2$, the procedures are called over-relaxation methods, which are used to accelerate the convergence for systems that are convergent by the Gauss-Seidel technique. These methods are abbreviated SOR, for Successive Over-Relaxation, and are particularly useful for solving the linear systems that occur in the numerical solution of certain partial-differential equations.

We note that by using Eq. (7.43), Eq. (7.46) can be reformulated for calculation purposes to

$$x_i^{(k)} = (1-\omega)x_i^{(k-1)} + \frac{\omega}{a_{ii}}\left[b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k)} - \sum_{j=i+1}^{n} a_{ij}x_j^{(k-1)}\right]$$

To determine the matrix of the SOR method, we rewrite this as

$$a_{ii}x_i^{(k)} + \omega\sum_{j=1}^{i-1} a_{ij}x_j^{(k)} = (1-\omega)a_{ii}x_i^{(k-1)} - \omega\sum_{j=i+1}^{n} a_{ij}x_j^{(k-1)} + \omega b_i$$

so that in vector form, we have

$$(D - \omega L)\boldsymbol{x}^{(k)} = [(1-\omega)D + \omega U]\boldsymbol{x}^{(k-1)} + \omega\boldsymbol{b}$$

or

$$\boldsymbol{x}^{(k)} = (D - \omega L)^{-1}[(1-\omega)D + \omega U]\boldsymbol{x}^{(k-1)} + \omega(D - \omega L)^{-1}\boldsymbol{b} \tag{7.47}$$

If we let $T_\omega = (D - \omega L)^{-1}[(1-\omega)D + \omega U]$ and $\boldsymbol{c}_\omega = \omega(D - \omega L)^{-1}\boldsymbol{b}$, the SOR technique has the form

$$\boldsymbol{x}^{(k)} = T_\omega \boldsymbol{x}^{(k-1)} + \boldsymbol{c}_\omega \tag{7.48}$$

Theorem 7.5: Kahan

If $a_{ii} \neq 0$, for each $i = 1, 2, \ldots, n$, then $\rho(T_\omega) \geq |\omega - 1|$. This implies that the SOR method can converge only if $0 < \omega < 2$.

Theorem 7.6: Ostrowski-Reich

If A is a positive definite matrix and $0 < \omega < 2$, then the SOR method converges for any choice of initial approximate vector $\boldsymbol{x}^{(0)}$.

Theorem 7.7: Positive Definite and Tridiagonal

If A is positive definite and tridiagonal, then $\rho(T_g) = [\rho(T_j)]^2 < 1$, and the optimal choice of ω for the SOR method is

$$\omega = \frac{2}{1 + \sqrt{1 - [\rho(T_j)]^2}}$$

With this choice of ω, we have $\rho(T_\omega) = \omega - 1$.

Definition 7.8: The Successive Over-Relaxation (SOR) Iterative Method

To solve $A\boldsymbol{x} = \boldsymbol{b}$, we will use an initial approximation $\boldsymbol{x}^{(0)}$.
The Successive Over-Relaxation iterative algorithm steps:

1. **Rearrange Equations**
   Rewrite each i-th equation in the system (matrix A of size $n \times n$) to isolate the diagonal variable x_i, $i = 1, 2, \ldots, n$.
   We will convert $A\boldsymbol{x} = \boldsymbol{b}$ to the form $\boldsymbol{x}^{(k)} = T\boldsymbol{x}^{k-1} + \boldsymbol{c}$ with

   $$T = -D^{-1}(L + U)$$
   $$\boldsymbol{c} = D^{-1}\boldsymbol{b}$$

   D is the diagonal matrix whose diagonal entries are those of A, L is the strictly lower-triangular part of A, and U is the strictly upper-triangular part of A.

2. **Make an Initial Guess**
   Choose starting values for all variables (usually zeros if no other information is available).

3. **Choose a Relaxation Factor, ω**
   Kahan implies that SOR method can converge only if $0 < \omega < 2$, we can choose $\omega = 1.05$ or $\omega = 1.25$ and compare the results (this is over-relaxation and it is used to accelerate convergence) while $0 < \omega < 1$ choice is under relaxation, it is used to stabilize non-convergent system. $\omega = 1$ is equivalent to the standard Gauss-Seidel method.

4. **Iterate**
   Substitute the current guesses into the right-hand side of the rearranged equations to calculate a new set of approximations.
   We will use the SOR iterative formula, for each variable x_i at iteration k:

   $$x_i^{(k)} = (1 - \omega)x_i^{(k-1)} + \frac{\omega}{a_{ii}}\left(b_i - \sum_{j<i} a_{ij}x_j^{(k)} - \sum_{j>i} a_{ij}x_j^{(k-1)}\right)$$

   for $k \geq 1$.

5. **Check Convergence**
   Repeat the substitution process until the changes between iterations fall below your desired error tolerance (ϵ)

   $$\frac{||\boldsymbol{x}^{(k)} - \boldsymbol{x}^{(k-1)}||}{||\boldsymbol{x}^{(k)}||}$$

Remark:
When ω is chosen well it significantly reduces the total number of iterations needed to reach a solution compared to Jacobi or Gauss-Seidel methods. This algorithm is highly effective for large, sparse linear systems that commonly arise from the numerical solution of partial differential equations.

iv. The Conjugate Gradient Method

[SI*] In mathematics, the conjugate gradient method is an algorithm for the numerical methods of particular systems of linear equations, namely those whose matrix is positive-semidefinite. The conjugate gradient method is often implemented as an iterative algorithm, applicable to sparse systems that are too large to be handled by a direct implementation or other direct methods such as the Cholesky decomposition. Large sparse systems often arise when numerically solving partial differential equations or optimization problems.
The conjugate gradient method can also be used to solve unconstrained optimization problems such as energy minimization. It is commonly attributed to Magnus Hestenes and Eduard Stiefel, who programmed it on the Z4, and extensively researched it.
The biconjugate gradient method provides a generalization to non-symmetric matrices. Various nonlinear conjugate gradient methods seek minima of nonlinear optimization problems.
The conjugate gradient method of Hestenes and Stiefel was originally developed as a direct method designed to solve an $n \times n$ positive definite linear system. As a direct method it is generally inferior to Gaussian elimination with pivoting since both methods require n steps to determine a solution, and the steps of the conjugate gradient method are more computationally expensive than those in Gaussian elimination.
However, the conjugate gradient method is very useful when employed as an iterative approximation method for solving large sparse systems with nonzero entries occurring in predictable patterns. When the matrix has been preconditioned to make the calculations more effective, good results are obtained in only about $\sqrt{n}$ steps. Employed in this way, the method is preferred over Gaussian elimination and the previously-discussed iterative methods.

[SI*] We assume that the matrix A is positive definite, we will use the inner product notation

$$< \boldsymbol{x}, \boldsymbol{y} >= \boldsymbol{x}^T \boldsymbol{y}$$

where $\boldsymbol{x}$ and $\boldsymbol{y}$ are n-dimensional vectors.

Theorem 7.8: Solution to Positive Definite Linear System

The vector $\boldsymbol{x}*$ is a solution to the positive definite linear system $A\boldsymbol{x} = \boldsymbol{b}$ if and only if $\boldsymbol{x}*$ minimizes

$$g(\boldsymbol{x}) =< \boldsymbol{x}, A\boldsymbol{x} > -2 < \boldsymbol{x}, \boldsymbol{b} >$$

[SI*] To begin the conjugate gradient method, we choose $\boldsymbol{x}$, an approximate solution to $A\boldsymbol{x}^* = \boldsymbol{b}$, and $\boldsymbol{p} \neq \boldsymbol{0}$, which gives a search direction in which to move away from $\boldsymbol{x}$ to improve the approximation. Let $\boldsymbol{r} = \boldsymbol{b} - A\boldsymbol{x}$ be the residual vector associated with $\boldsymbol{x}$ and

$$\alpha = \frac{< \boldsymbol{p}, \boldsymbol{b} - A\boldsymbol{x} >}{< \boldsymbol{p}, A\boldsymbol{p} >} = \frac{< \boldsymbol{p}, \boldsymbol{r} >}{< \boldsymbol{p}, A\boldsymbol{p} >}$$

If $\boldsymbol{r} \neq \boldsymbol{0}$ and if $\boldsymbol{p}$ and $\boldsymbol{r}$ are not orthogonal, then $\boldsymbol{x} + \alpha\boldsymbol{p}$ gives a smaller value for g than $g(\boldsymbol{x})$ and is presumably closer to $\boldsymbol{x}^*$ than is $\boldsymbol{x}$. This suggests the following method.
Let $\boldsymbol{x}^{(0)}$ be an initial approximation to $\boldsymbol{x}^*$, and let $\boldsymbol{p}^1 \neq \boldsymbol{0}$ be an initial search direction. For

$k = 1, 2, 3, \ldots$, we compute

$$\alpha_k = \frac{< \boldsymbol{p}^{(k)}, \boldsymbol{b} - A\boldsymbol{x}^{(k-1)} >}{< \boldsymbol{p}^{(k)}, A\boldsymbol{p}^{(k)} >}$$
$$\boldsymbol{x}^{(k)} = \boldsymbol{x}^{(k-1)} + \alpha_k \boldsymbol{p}^{(k)}$$

and choose a new search direction $\boldsymbol{p}^{(k+1)}$. The object is to make this selection so that the sequence of approximation $\{\boldsymbol{x}^{(k)}\}$ converges rapidly to $\boldsymbol{x}^*$.
To choose the search diections, we view g as a function of the components of $\boldsymbol{x} = (x_1, x_2, \ldots, x_n)^T$. Thus,

$$g(x_1, x_2, \ldots, x_n) =< \boldsymbol{x}, A\boldsymbol{x} > -2 < \boldsymbol{x}, \boldsymbol{b} >= \sum_{i=1}^{n} \sum_{j=1}^{n} a_{ij} x_i x_j - 2 \sum_{i=1}^{n} x_i b_i$$

Taking partial derivatives with respect to the component variables x_k gives

$$\frac{\partial g}{\partial x_k}(\boldsymbol{x}) = 2 \sum_{i=1}^{n} a_{ki} x_i - 2 b_k$$

Therefore, the gradient of g is

$$\nabla g(\boldsymbol{x}) = \left(\frac{\partial g}{\partial x_1}(\boldsymbol{x}), \frac{\partial g}{\partial x_2}(\boldsymbol{x}), \ldots, \frac{\partial g}{\partial x_n}(\boldsymbol{x}) \right)^T = 2(A\boldsymbol{x} - \boldsymbol{b}) = -2\boldsymbol{r}$$

where the vector $\boldsymbol{r}$ is the residual vector for $\boldsymbol{x}$.
From multivariable calculus, we know that the direction of greatest decrease in the value of $g(\boldsymbol{x})$ is the direction given by $-\nabla g(\boldsymbol{x})$; that is, in the direction of the residual $\boldsymbol{r}$. The method that chooses

$$\boldsymbol{p}^{(k+1)} = \boldsymbol{r}^{(k)} = \boldsymbol{b} - A\boldsymbol{x}^{(k)}$$

is called the method of steepest descent. Although this method has merit for nonlinear systems and optimization problems, it is not used for linear systems because of slow convergence.
An alternative approach uses a set of nonzero direction vectors $\{\boldsymbol{p}^{(1)}, \ldots, \boldsymbol{p}^{(n)}\}$ that satisfy

$$< \boldsymbol{p}^{(i)}, A\boldsymbol{p}^{(j)} >= 0, \quad \text{if } i \neq j$$

This is called an A-orthogonality condition, and the set of vectors $\{\boldsymbol{p}^{(1)}, \ldots, \boldsymbol{p}^{(n)}\}$ is said to be A-orthogonal. It is not difficult to show that a set of A-orthogonal vectors associated with the positive definite matrix A is linearly independent. This set of search directions gives

$$\alpha_k = \frac{< \boldsymbol{p}^{(k)}, \boldsymbol{b} - A\boldsymbol{x}^{(k-1)} >}{< \boldsymbol{p}^{(k)}, A\boldsymbol{p}^{(k)} >} = \frac{< \boldsymbol{p}^{(k)}, \boldsymbol{r}^{(k-1)} >}{< \boldsymbol{p}^{(k)}, A\boldsymbol{p}^{(k)} >}$$

and $\boldsymbol{x}^{(k)} = \boldsymbol{x}^{(k-1)} + \alpha_k \boldsymbol{p}^{(k)}$.

Theorem 7.9: Search Directions Gives Convergence

Let $\{\boldsymbol{p}^{(1)}, \ldots, \boldsymbol{p}^{(n)}\}$ be an A-orthogonal set of nonzero vectors associated with the positive definite matrix A, and let $\boldsymbol{x}^{(0)}$ be arbitrary. Define

$$\alpha_k = \frac{< \boldsymbol{p}^{(k)}, \boldsymbol{b} - A\boldsymbol{x}^{(k-1)} >}{< \boldsymbol{p}^{(k)}, A\boldsymbol{p}^{(k)} >}$$
$$\boldsymbol{x}^{(k)} = \boldsymbol{x}^{(k-1)} + \alpha_k \boldsymbol{p}^{(k)}$$

for $k = 1, 2, \ldots, n$. Then, assuming exact arithmetic, $A\boldsymbol{x}^{(n)} = \boldsymbol{b}$. This choice of search directions gives convergence in at most n-steps, as as a direct method it produces the exact solution. The use of an A-orthogonal set $\{\boldsymbol{p}^{(1)}, \ldots, \boldsymbol{p}^{(n)}\}$ of direction vectors gives what is called a conjugate direction method.

Theorem 7.10: The Residual Vectors

The residual vectors $\boldsymbol{r}^{(k)}$, where $k = 1, 2, \ldots, n$, for a conjugate direction method, satisfy the equations

$$< \boldsymbol{r}^{(k)}, \boldsymbol{p}^{(j)} >= 0, \quad \text{for each } j = 1, 2, \ldots, k$$

The conjugate gradient method of Hestenes and Stiefel chooses the search directions $\{\boldsymbol{p}^{(k)}\}$ during the iterative process so that the residual vectors $\{\boldsymbol{r}^{(k)}\}$ are mutually orthogonal. To construct the direction vectors $\{\boldsymbol{p}^{(1)}, \boldsymbol{p}^{(2)}, \ldots\}$ and the approximations $\{\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \ldots\}$, we start with an initial approximation $\boldsymbol{x}^{(0)}$ and use the steepest descent direction $\boldsymbol{r}^{(0)} = \boldsymbol{b} - A\boldsymbol{x}^{(0)}$ as the first search direction $\boldsymbol{p}^{(1)}$.
Assume that the conjugate directions $\boldsymbol{p}^{(1)}, \ldots, \boldsymbol{p}^{(k-1)}$ and the approximations $\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(k-1)}$ have been computed with

$$\boldsymbol{x}^{(k-1)} = \boldsymbol{x}^{(k-2)} + \alpha_{k-1} \boldsymbol{p}^{(k-1)}$$

where

$$< \boldsymbol{p}^{(i)}, A\boldsymbol{p}^{(j)} > = 0$$
$$< \boldsymbol{r}^{(i)}, \boldsymbol{r}^{(j)} > = 0, \quad \text{for } i \neq j$$

If $\boldsymbol{x}^{(k-1)}$ is the solution to $A\boldsymbol{x} = \boldsymbol{b}$, we are done. Otherwise, $\boldsymbol{r}^{(k-1)} = \boldsymbol{b} - A\boldsymbol{x}^{(k-1)} \neq \boldsymbol{0}$ and it implies that $< \boldsymbol{r}^{(k-1)}, \boldsymbol{p}^{(i)} >= 0$, for $i = 1, 2, \ldots, k-1$. We then use $\boldsymbol{r}^{(k-1)}$ to generate $\boldsymbol{p}^{(k)}$ by setting

$$\boldsymbol{p}^{(k)} = \boldsymbol{r}^{(k-1)} + \beta_{k-1} \boldsymbol{p}^{(k-1)}$$

We want to choose β_{k-1} so that

$$< \boldsymbol{p}^{(k-1)}, A\boldsymbol{p}^{(k)} >= 0$$

Since

$$A\boldsymbol{p}^{(k)} = A\boldsymbol{r}^{(k-1)} + \beta_{k-1} A\boldsymbol{p}^{(k-1)}$$

and

$$< \boldsymbol{p}^{(k-1)}, A\boldsymbol{p}^{(k)} >=< \boldsymbol{p}^{(k-1)}, A\boldsymbol{r}^{(k-1)} > + \beta_{k-1} < \boldsymbol{p}^{(k-1)}, A\boldsymbol{p}^{(k-1)} >$$

we will have $< \boldsymbol{p}^{(k-1)}, A\boldsymbol{p}^{(k)} >= 0$ when

$$\beta_{k-1} = -\frac{< \boldsymbol{p}^{(k-1)}, A\boldsymbol{r}^{(k-1)} >}{< \boldsymbol{p}^{(k-1)}, A\boldsymbol{p}^{(k-1)} >}$$

It can also be shown that with this choice of β_{k-1} we have $< \boldsymbol{p}^{(k)}, A\boldsymbol{p}^{(i)} >= 0$, for each $i = 1, 2, \ldots, k-2$. Thus, $\{\boldsymbol{p}^{(1)}, \ldots, \boldsymbol{p}^{(k)}\}$ is an A-orthogonal set.
Having chosen $\boldsymbol{p}^{(k)}$, we compute

$$\begin{aligned}
\alpha_k &= \frac{< \boldsymbol{p}^{(k)}, \boldsymbol{r}^{(k-1)} >}{< \boldsymbol{p}^{(k)}, A\boldsymbol{p}^{(k)} >} \\
&= \frac{< \boldsymbol{r}^{(k-1)} + \beta_{k-1}\boldsymbol{p}^{(k-1)}, \boldsymbol{r}^{(k-1)} >}{< \boldsymbol{p}^{(k)}, A\boldsymbol{p}^{(k)} >} \\
&= \frac{< \boldsymbol{r}^{(k-1)}, \boldsymbol{r}^{(k-1)} >}{< \boldsymbol{p}^{(k)}, A\boldsymbol{p}^{(k)} >} + \beta_{k-1}\frac{< \boldsymbol{p}^{(k-1)}, \boldsymbol{r}^{(k-1)} >}{< \boldsymbol{p}^{(k)}, A\boldsymbol{p}^{(k)} >}
\end{aligned}$$

By Theorem, $< \boldsymbol{p}^{(k-1)}, \boldsymbol{r}^{(k-1)} >= 0$, so

$$\alpha_k = \frac{< \boldsymbol{r}^{(k-1)}, \boldsymbol{r}^{(k-1)} >}{< \boldsymbol{p}^{(k)}, A\boldsymbol{p}^{(k)} >} \tag{7.49}$$

Thus,

$$\boldsymbol{x}^{(k)} = \boldsymbol{x}^{(k-1)} + \alpha_k \boldsymbol{p}^{(k)}$$

To compute $\boldsymbol{r}^{(k)}$, we multiply by A and subtract $\boldsymbol{b}$ to obtain

$$A\boldsymbol{x}^{(k)} - \boldsymbol{b} = A\boldsymbol{x}^{(k-1)} - \boldsymbol{b} + \alpha_k A\boldsymbol{p}^{(k)}$$

or

$$\boldsymbol{r}^{(k)} = \boldsymbol{r}^{(k-1)} - \alpha_k A\boldsymbol{p}^{(k)}$$

Thus,

$$< \boldsymbol{r}^{(k)}, \boldsymbol{r}^{(k)} >=< \boldsymbol{r}^{(k-1)}, \boldsymbol{r}^{(k)} > -\alpha_k < A\boldsymbol{p}^{(k)}, \boldsymbol{r}^{(k)} >= -\alpha_k < \boldsymbol{r}^{(k)}, A\boldsymbol{p}^{(k)} >$$

Further, from $\alpha_k = \frac{<\boldsymbol{r}^{(k-1)}, \boldsymbol{r}^{(k-1)}>}{<\boldsymbol{p}^{(k)}, A\boldsymbol{p}^{(k)}>}$,

$$< \boldsymbol{r}^{(k-1)}, \boldsymbol{r}^{(k-1)} >= \alpha_k < \boldsymbol{p}^{(k)}, A\boldsymbol{p}^{(k)} >$$

so

$$\begin{aligned}
\beta_k &= -\frac{< \boldsymbol{p}^{(k)}, A\boldsymbol{r}^{(k)} >}{< \boldsymbol{p}^{(k)}, A\boldsymbol{p}^{(k)} >} \\
&= -\frac{< \boldsymbol{r}^{(k)}, A\boldsymbol{p}^{(k)} >}{< \boldsymbol{p}^{(k)}, A\boldsymbol{p}^{(k)} >} \\
&= \frac{\left(\frac{1}{\alpha_k}\right) < \boldsymbol{r}^{(k)}, \boldsymbol{r}^{(k)} >}{\left(\frac{1}{\alpha_k}\right) < \boldsymbol{r}^{(k-1)}, \boldsymbol{r}^{(k-1)} >} \\
&= \frac{< \boldsymbol{r}^{(k)}, \boldsymbol{r}^{(k)} >}{< \boldsymbol{r}^{(k-1)}, \boldsymbol{r}^{(k-1)} >}
\end{aligned}$$

In summary, we have the formulas:

$$\begin{aligned} \boldsymbol{r}^{(0)} &= \boldsymbol{b} - A\boldsymbol{x}^{(0)} \\ \boldsymbol{p}^{(1)} &= \boldsymbol{r}^{(0)} \end{aligned}$$

and, for $k = 1, 2, \ldots, n$,

$$\begin{aligned} \alpha_k &= \frac{< \boldsymbol{r}^{(k-1)}, \boldsymbol{r}^{(k-1)} >}{< \boldsymbol{p}^{(k)}, A\boldsymbol{p}^{(k)} >} \\ \boldsymbol{x}^{(k)} &= \boldsymbol{x}^{(k-1)} + \alpha_k \boldsymbol{p}^{(k)} \\ \boldsymbol{r}^{(k)} &= \boldsymbol{r}^{(k-1)} - \alpha_k A\boldsymbol{p}^{(k)} \\ \beta_k &= \frac{< \boldsymbol{r}^{(k)}, \boldsymbol{r}^{(k)} >}{< \boldsymbol{r}^{(k-1)}, \boldsymbol{r}^{(k-1)} >} \\ \boldsymbol{p}^{(k+1)} &= \boldsymbol{r}^{(k)} + \beta_k \boldsymbol{p}^{(k)} \end{aligned}$$

If the matrix A is ill-conditioned, the conjugate gradient method is highly susceptible to rounding errors. So, although the exact answer should be obtained in n steps, this is not usually the case. As a direct method the conjugate gradient method is not as good as Gaussian elimination with pivoting. The main use of the conjugate gradient method is as an iterative method applied to a better-conditioned system. In this case an acceptable approximate solution is often obtained in about $\sqrt{n}$ steps.

[SI*] **Preconditioned Conjugate Gradient**

To apply the conjugate gradient method to a better-conditioned system, we want to select a nonsingular conditioning matrix C so that

$$\tilde{A} = C^{-1}A(C^{-1})^T$$

is better conditioned. To simplify the notation, we will use the matrix C^{-t} to refer to $(C^{-1})^T$. Consider the linear system

$$\tilde{A}\tilde{\boldsymbol{x}} = \tilde{\boldsymbol{b}}$$

where $\tilde{\boldsymbol{x}} = C^T\boldsymbol{x}$ and $\tilde{\boldsymbol{b}} = C^{-1}\boldsymbol{b}$. Then

$$\tilde{A}\tilde{\boldsymbol{x}} = (C^{-1}AC^{-t})(C^T\boldsymbol{x}) = C^{-1}A\boldsymbol{x}$$

Thus, we could solve $\tilde{A}\tilde{\boldsymbol{x}} = \tilde{\boldsymbol{b}}$ for $\tilde{\boldsymbol{x}}$ and then obtain $\boldsymbol{x}$ by multiplyinh by C^{-t}. However, instead of rewriting conjugate gradient equations using $\tilde{\boldsymbol{r}}^{(k)}, \tilde{\boldsymbol{p}}^{(k)}, \tilde{\alpha}_k, \tilde{\boldsymbol{x}}^{(k)}$, and $\tilde{\beta_k}$, we incorporate the preconditioning implicitly.

Since

$$\tilde{\boldsymbol{x}}^{(k)} = C^T\boldsymbol{x}^{(k)}$$

we have

$$\tilde{\boldsymbol{r}}^{(k)} = \tilde{\boldsymbol{b}} - A\tilde{\boldsymbol{x}}^{(k)} = C^{-1}\boldsymbol{b} - (C^{-1}AC^{-t})C^T\boldsymbol{x}^{(k)} = C^{-1}(\boldsymbol{b} - A\boldsymbol{x}^{(k)}) = C^{-1}\boldsymbol{r}^{(k)}$$

Let $\tilde{\boldsymbol{p}}^{(k)} = C^T\boldsymbol{p}^{(k)}$ and $\boldsymbol{w}^{(k)} = C^{-1}\boldsymbol{r}^{(k)}$. Then

$$\tilde{\beta_k} = \frac{< \tilde{\boldsymbol{r}}^{(k)}, \tilde{\boldsymbol{r}}^{(k)} >}{< \tilde{\boldsymbol{r}}^{(k-1)}, \tilde{\boldsymbol{r}}^{(k-1)} >} = \frac{< C^{-1}\boldsymbol{r}^{(k)}, C^{-1}\boldsymbol{r}^{(k)} >}{< C^{-1}\boldsymbol{r}^{(k-1)}, C^{-1}\boldsymbol{r}^{(k-1)} >}$$

so

$$\tilde{\beta_k} = \frac{< \boldsymbol{w}^{(k)}, \boldsymbol{w}^{(k)} >}{< \boldsymbol{w}^{(k-1)}, \boldsymbol{w}^{(k-1)} >} \tag{7.50}$$

Thus,

$$\tilde{\alpha_k} = \frac{< \tilde{\boldsymbol{r}}^{(k-1)}, \tilde{\boldsymbol{r}}^{(k-1)} >}{< \tilde{\boldsymbol{p}}^{(k)}, \tilde{A}\tilde{\boldsymbol{p}}^{(k)} >} = \frac{< C^{-1}\boldsymbol{r}^{(k-1)}, C^{-1}\boldsymbol{r}^{(k-1)} >}{< C^T\boldsymbol{p}^{(k)}, C^{-1}AC^{-t}C^T\boldsymbol{p}^{(k)} >} = \frac{< \boldsymbol{w}^{(k-1)}, \boldsymbol{w}^{(k-1)} >}{< C^T\boldsymbol{p}^{(k)}, C^{-1}A\boldsymbol{p}^{(k)} >}$$

and

$$\tilde{\alpha_k} = \frac{< \boldsymbol{w}^{(k-1)}, \boldsymbol{w}^{(k-1)} >}{< \boldsymbol{p}^{(k)}, A\boldsymbol{p}^{(k)} >} \tag{7.51}$$

Further,

$$\tilde{\boldsymbol{x}}^{(k)} = \tilde{\boldsymbol{x}}^{(k-1)} + \tilde{\alpha_k}\tilde{\boldsymbol{p}}^{(k)}$$
$$C^T\boldsymbol{x}^{(k)} = C^T\boldsymbol{x}^{(k-1)} + \tilde{\alpha_k}C^T\boldsymbol{p}^{(k)}$$

and

$$\boldsymbol{x}^{(k)} = \boldsymbol{x}^{(k-1)} + \tilde{\alpha_k}\boldsymbol{p}^{(k)} \tag{7.52}$$

Continuing,

$$\tilde{\boldsymbol{r}}^{(k)} = \tilde{\boldsymbol{r}}^{(k-1)} - \tilde{\alpha_k}\tilde{A}\tilde{\boldsymbol{p}}^{(k)}$$

so

$$C^{-1}\boldsymbol{r}^{(k)} = C^{-1}\boldsymbol{r}^{(k-1)} - \tilde{\alpha_k}C^{-1}AC^{-t}\tilde{\boldsymbol{p}}^{(k)}$$
$$\boldsymbol{r}^{(k)} = \boldsymbol{r}^{(k-1)} - \tilde{\alpha_k}AC^{-T}C^T\tilde{\boldsymbol{p}}^{(k)}$$

and

$$\boldsymbol{r}^{(k)} = \boldsymbol{r}^{(k-1)} - \tilde{\alpha_k}A\boldsymbol{p}^{(k)} \tag{7.53}$$

Finally,

$$\tilde{\boldsymbol{p}}^{(k+1)} = \tilde{\boldsymbol{r}}^{(k)} + \tilde{\beta_k}\tilde{\boldsymbol{p}}^{(k)}$$
$$C^T\boldsymbol{p}^{(k+1)} = C^{-1}\boldsymbol{r}^{(k)} + \tilde{\beta_k}C^T\boldsymbol{p}^{(k)}$$

so

$$\boldsymbol{p}^{(k+1)} = C^{-t}C^{-1}\boldsymbol{r}^{(k)} + \tilde{\beta_k}\boldsymbol{p}^{(k)} = C^{-t}\boldsymbol{w}^{(k)} + \tilde{\beta_k}\boldsymbol{p}^{(k)} \tag{7.54}$$

Definition 7.9: The Conjugate Gradient Method

To solve $A\boldsymbol{x} = \boldsymbol{b}$ with Conjugate Gradient, we will use an initial guess $\boldsymbol{x}_0$, initial residual vector $\boldsymbol{r}_0$ and initial search direction vector $\boldsymbol{p}_0 = \boldsymbol{r}_0$.
The Conjugate Gradient iterative algorithm steps:

1. **Initialize Parameters**
   Compute the initial residual vector $\boldsymbol{r}_0 = \boldsymbol{b} - A\boldsymbol{x}_0$ and set the initial search direction vector $\boldsymbol{p}_0 = \boldsymbol{r}_0$.

2. **Calculate Step Size (α_k)**
   Determine how far to move along the search direction:

$$\alpha_k = \frac{\boldsymbol{r}_k^T \boldsymbol{r}_k}{\boldsymbol{p}_k^T A \boldsymbol{p}_k}$$

3. **Update State**
   Compute the updated solution candidate

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \alpha_k \boldsymbol{p}_k$$

   and the new residual

$$\boldsymbol{r}_{k+1} = \boldsymbol{r}_k - \alpha_k A \boldsymbol{p}_k$$

4. **Evaluate Convergence**
   If the norm of the residual $||\boldsymbol{r}_{k+1}|| = \sqrt{\boldsymbol{r}_{k+1}^T \boldsymbol{r}_{k+1}}$ falls below the tolerance (ϵ), exit.

5. **Update Direction Coefficient (β_k)**
   Adjust the next search direction to maintain A-orthogonality (conjugacy):

$$\beta_k = \frac{\boldsymbol{r}_{k+1}^T \boldsymbol{r}_{k+1}}{\boldsymbol{r}_k^T \boldsymbol{r}_k}$$

   this is called Gram-Schmidt coefficient.

6. **Compute Next Search Direction ($\boldsymbol{p}_{k+1}$)**
   Combine the new residual with the old search direction

$$\boldsymbol{p}_{k+1} = \boldsymbol{r}_{k+1} + \beta_k \boldsymbol{p}_k$$

The memory storage is $O(n)$ space complexity, each iteration we have one matrix-vector multiplication $A\boldsymbol{p}_k$.
Remark:
Conjugate Gradient requires matrix A to be real, symmetric ($A = A^T$) and positive-definite ($\boldsymbol{x}^T A \boldsymbol{x} > 0$, for all non-zero $\boldsymbol{x}$), otherwise standard Conjugate Gradient will stall or diverge if A does not meet these criteria.

v. Compute the Solution of a Linear System with Jacobi Iterative Method

Compute the unique solution of a linear system $A\boldsymbol{x} = \boldsymbol{b}$ given by

$$\begin{aligned} 10x_1 - x_2 + 2x_3 &= 6 \\ -x_1 + 11x_2 - x_3 + 3x_4 &= 25 \\ 2x_1 - x_2 + 10x_3 - x_4 &= -11 \\ 3x_2 - x_3 + 8x_4 &= 15 \end{aligned}$$

Solution:
To convert $A\boldsymbol{x} = \boldsymbol{b}$ to the form $\boldsymbol{x}^{(k)} = T\boldsymbol{x}^{(k-1)} + \boldsymbol{c}$, solve equation E_i for x_i for each $i = 1, 2, 3, 4$, to obtain

$$\begin{aligned} x_1 &= \frac{1}{10}x_2 - \frac{1}{5}x_3 + \frac{3}{5} \\ x_2 &= \frac{1}{11}x_1 + \frac{1}{11}x_3 - \frac{3}{11}x_4 + \frac{25}{11} \\ x_3 &= -\frac{1}{5}x_1 + \frac{1}{10}x_2 \quad \frac{1}{10}x_4 - \frac{11}{10} \\ x_4 &= -\frac{3}{8}x_2 + \frac{1}{8}x_3 + \frac{15}{8} \end{aligned}$$

Then $A\boldsymbol{x} = \boldsymbol{b}$ can be rewritten in the form $\boldsymbol{x}^{(k)} = T\boldsymbol{x}^{(k-1)} + \boldsymbol{c}$ with

$$T = \begin{bmatrix} 0 & \frac{1}{10} & -\frac{1}{5} & 0 \\ \frac{1}{11} & 0 & \frac{1}{11} & -\frac{3}{11} \\ -\frac{1}{5} & \frac{1}{10} & 0 & \frac{1}{10} \\ 0 & -\frac{3}{8} & \frac{1}{8} & 0 \end{bmatrix}, \quad \boldsymbol{c} = \begin{bmatrix} \frac{3}{5} \\ \frac{25}{11} \\ -\frac{11}{10} \\ \frac{15}{8} \end{bmatrix}$$

For an initial approximation, we let $\boldsymbol{x}^{(0)} = (0, 0, 0, 0)^T$. Then $\boldsymbol{x}^{(1)}$ is given by

$$\begin{aligned} x_1^{(1)} &= \frac{1}{10}x_2^{(0)} - \frac{1}{5}x_3^{(0)} + \frac{3}{5} = 0.6000 \\ x_2^{(1)} &= \frac{1}{11}x_1^{(0)} + \frac{1}{11}x_3^{(0)} - \frac{3}{11}x_4^{(0)} + \frac{25}{11} = 2.272727 \\ x_3^{(1)} &= -\frac{1}{5}x_1^{(0)} + \frac{1}{10}x_2^{(0)} \quad \frac{1}{10}x_4^{(0)} - \frac{11}{10} = -1.1000 \\ x_4^{(1)} &= -\frac{3}{8}x_2^{(0)} + \frac{1}{8}x_3^{(0)} + \frac{15}{8} = 1.8750 \end{aligned}$$

This process is repeated until convergence, we set the $\epsilon = 10^{-3}$ (the tolerance), the stop criteria is $||A\boldsymbol{x}^{(n)} - \boldsymbol{b}|| < \epsilon$. The solution is obtained after 11 iterations. The exact solution of the system is $(1, 2, -1, 1)$.

In SymIntegration, to approximate the solution of $A\boldsymbol{x} = \boldsymbol{b}$ with Jacobi iterative method we can use this function:
JacobiMethod(const vector<vector<double>>& A, const vector<double>& b, int N, double ϵ)

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Jacobi Iterative Method ]# ./main
Starting Jacobi Iteration...

Iteration 1: x[0]=0.600000       x[1]=2.272727          x[2]=-1.100000         x[3]=1.875000          (Error: 3.201705)
Iteration 2: x[0]=1.047273       x[1]=1.715909          x[2]=-0.805227         x[3]=0.885227          (Error: 1.255643)
Iteration 3: x[0]=0.932636       x[1]=2.053306          x[2]=-1.049341         x[3]=1.130881          (Error: 0.496906)
Iteration 4: x[0]=1.015199       x[1]=1.953696          x[2]=-0.968109         x[3]=0.973843          (Error: 0.219085)
Iteration 5: x[0]=0.988991       x[1]=2.011415          x[2]=-1.010286         x[3]=1.021351          (Error: 0.089745)
Iteration 6: x[0]=1.003199       x[1]=1.992241          x[2]=-0.994522         x[3]=0.994434          (Error: 0.039275)
Iteration 7: x[0]=0.998128       x[1]=2.002307          x[2]=-1.001972         x[3]=1.003594          (Error: 0.016323)
Iteration 8: x[0]=1.000625       x[1]=1.998670          x[2]=-0.999036         x[3]=0.998888          (Error: 0.007087)
Iteration 9: x[0]=0.999674       x[1]=2.000448          x[2]=-1.000369         x[3]=1.000619          (Error: 0.002973)
Iteration 10: x[0]=1.000119      x[1]=1.999768          x[2]=-0.999828         x[3]=0.999786          (Error: 0.001283)
Iteration 11: x[0]=0.999942      x[1]=2.000085          x[2]=-1.000068         x[3]=1.000109          (Error: 0.000541)

Converged in 11 iterations.
```

Figure 7.28: *The approximated solutions after* 11 *iterations with Jacobi iterative method (**SymIntegration/Examples/Numerical Methods/Test SymIntegration Jacobi Iterative Method/main.cpp**).*

For comparison, we also try to solve this linear system with Gauss-Seidel and SOR method

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Gauss-Seidel Method ]# ./main
Starting Gauss-Seidel Method


Iterative Steps:
Iteration 1: x[0]=0.600000       x[1]=2.327273          x[2]=-0.987273         x[3]=0.878864          (Error: 2.742865)

Iteration 2: x[0]=1.030182       x[1]=2.036938          x[2]=-1.014456         x[3]=0.984341          (Error: 0.530297)

Iteration 3: x[0]=1.006585       x[1]=2.003555          x[2]=-1.002527         x[3]=0.998351          (Error: 0.044831)

Iteration 4: x[0]=1.000861       x[1]=2.000298          x[2]=-1.000307         x[3]=0.999850          (Error: 0.007110)

Iteration 5: x[0]=1.000091       x[1]=2.000021          x[2]=-1.000031         x[3]=0.999988          (Error: 0.000874)


Converged successfully in 5 iterations.

Final Solution:
x[1] = 1.000091
x[2] = 2.000021
x[3] = -1.000031
x[4] = 0.999988

Time taken by function: 1072 microseconds
```

Figure 7.29: *The approximated solutions after* 5 *iterations with Gauss-Seidel method (**SymIntegration/Examples/Numerical Methods/Test SymIntegration Gauss-Seidel Method/main.cpp**).*

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration SOR Method ]# ./main
Starting SOR Method with omega = 1.050000

Iteration 1: x[0]=0.630000       x[1]=2.446500          x[2]=-1.030417         x[3]=0.870198          (Error: 2.863786)
Iteration 2: x[0]=1.071770       x[1]=2.018793          x[2]=-1.025207         x[3]=0.995782          (Error: 0.627609)
Iteration 3: x[0]=1.003678       x[1]=1.998213          x[2]=-1.000143         x[3]=1.000896          (Error: 0.075594)
Iteration 4: x[0]=0.999658       x[1]=1.999787          x[2]=-0.999849         x[3]=1.000059          (Error: 0.004407)
Iteration 5: x[0]=0.999963       x[1]=2.000005          x[2]=-0.999993         x[3]=0.999996          (Error: 0.000406)

Converged in 5 iterations.
Solution vector x:
x[0] = 0.999963
x[1] = 2.000005
x[2] = -0.999993
x[3] = 0.999996

Time taken by function: 1203 microseconds
```

Figure 7.30: *The approximated solutions after* 5 *iterations with SOR method with* $\omega = 1.05$ *(**SymIntegration/Examples/Numerical Methods/Test SymIntegration SOR Method/main.cpp**).*

vi. Compute the Solution of a Linear System with Gauss-Seidel Method and SOR Method

Compute the solution of the linear system $A\boldsymbol{x} = \boldsymbol{b}$ given by

$$\begin{aligned} 4x_1 + 3x_2 &= 24 \\ 3x_1 + 4x_2 - x_3 &= 30 \\ -x_2 + 4x_3 &= -24 \end{aligned}$$

with Gauss-Seidel method and SOR method with $\omega = 1.25$.

Solution:
The linear system above has the solution $(3, 4, -5)^T$. We will use $\boldsymbol{x}^{(0)} = (1, 1, 1)^T$ for both methods. For each $k = 1, 2, \ldots$, the equations for the Gauss-Seidel method are

$$\begin{aligned} x_1^{(k)} &= -0.75x_2^{(k-1)} + 6 \\ x_2^{(k)} &= -0.75x_1^{(k)} + 0.25x_3^{(k-1)} + 7.5 \\ x_3^{(k)} &= 0.25x_2^{(k)} - 6 \end{aligned}$$

and the equations for the SOR method with $\omega = 1.25$ are

$$\begin{aligned} x_1^{(k)} &= -0.25x_1^{(k-1)} - 0.9375x_2^{(k-1)} + 7.5 \\ x_2^{(k)} &= -0.9375x_1^{(k)} - 0.25x_2^{(k-1)} + 0.3125x_3^{(k-1)} + 9.375 \\ x_3^{(k)} &= 0.3125x_2^{(k)} - 0.25x_3^{(k-1)} - 7.5 \end{aligned}$$

For $\epsilon = 10^{-3}$ the Gauss-Seidel method requires 12 iterations, while SOR method only needs 8 iterations with $\omega = 1.25$. If we use $\epsilon = 10^{-7}$ then the Gauss-Seidel method requires 32 iterations, while SOR method only needs 15 iterations with $\omega = 1.25$.

In SymIntegration, to approximate the solution of $A\boldsymbol{x} = \boldsymbol{b}$ with Gauss-Seidel method we can use this function:
GaussSeidel(const vector<vector<double>>& A, const vector<double>& b, const vector<double>& x, double ϵ, int N)

In SymIntegration, to approximate the solution of $A\boldsymbol{x} = \boldsymbol{b}$ with SOR method we can use this function:
SORIterativeMethod(const vector<vector<double>>& A, const vector<double>& b, const vector<double>& x, double ω, double ϵ, int N)

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Gauss-Seidel Method ]# ./main
Starting Gauss-Seidel Method


Iterative Steps:
Iteration 1: x[0]=5.250000       x[1]=3.812500          x[2]=-5.046875         (Error: 7.908056)

Iteration 2: x[0]=3.140625       x[1]=3.882812          x[2]=-5.029297         (Error: 2.110620)

Iteration 3: x[0]=3.087891       x[1]=3.926758          x[2]=-5.018311         (Error: 0.069518)

Iteration 4: x[0]=3.054932       x[1]=3.954224          x[2]=-5.011444         (Error: 0.043449)

Iteration 5: x[0]=3.034332       x[1]=3.971390          x[2]=-5.007153         (Error: 0.027156)

Iteration 6: x[0]=3.021458       x[1]=3.982119          x[2]=-5.004470         (Error: 0.016972)

Iteration 7: x[0]=3.013411       x[1]=3.988824          x[2]=-5.002794         (Error: 0.010608)

Iteration 8: x[0]=3.008382       x[1]=3.993015          x[2]=-5.001746         (Error: 0.006630)

Iteration 9: x[0]=3.005239       x[1]=3.995634          x[2]=-5.001091         (Error: 0.004144)

Iteration 10: x[0]=3.003274      x[1]=3.997272          x[2]=-5.000682         (Error: 0.002590)

Iteration 11: x[0]=3.002046      x[1]=3.998295          x[2]=-5.000426         (Error: 0.001619)

Iteration 12: x[0]=3.001279      x[1]=3.998934          x[2]=-5.000266         (Error: 0.001012)


Converged successfully in 12 iterations.

Final Solution:
x[1] = 3.001279
x[2] = 3.998934
x[3] = -5.000266

Time taken by function: 1283 microseconds
```

Figure 7.31: *The approximated solutions after* 12 *iterations with Gauss-Seidel method (**SymIntegration/Examples/Numerical Methods/Test SymIntegration Gauss-Seidel Method/main.cpp**).*

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration SOR Method ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration SOR Method ]# ./main
Starting SOR Method with omega = 1.250000

Iteration 1: x[0]=6.312500       x[1]=3.519531          x[2]=-6.650146         (Error: 9.648598)
Iteration 2: x[0]=2.622314       x[1]=3.958527          x[2]=-4.600424         (Error: 4.244002)
Iteration 3: x[0]=3.133303       x[1]=4.010265          x[2]=-5.096686         (Error: 0.714186)
Iteration 4: x[0]=2.957051       x[1]=4.007484          x[2]=-4.973490         (Error: 0.215057)
Iteration 5: x[0]=3.003721       x[1]=4.002925          x[2]=-5.005714         (Error: 0.056897)
Iteration 6: x[0]=2.996328       x[1]=4.000926          x[2]=-4.998282         (Error: 0.010672)
Iteration 7: x[0]=3.000050       x[1]=4.000259          x[2]=-5.000349         (Error: 0.004309)
Iteration 8: x[0]=2.999745       x[1]=4.000065          x[2]=-4.999892         (Error: 0.000582)

Converged in 8 iterations.
Solution vector x:
x[0] = 2.999745
x[1] = 4.000065
x[2] = -4.999892

Time taken by function: 1393 microseconds
```

Figure 7.32: *The approximated solutions after* 8 *iterations with SOR method with* $\omega = 1.25$ *(**SymIntegration/Examples/Numerical Methods/Test SymIntegration SOR Method/main.cpp**).*

vii. Compute A-Orthogonal Vectors with Modified Gram-Schmidt

In conjugate gradient method, we need to compute a set of nonzero direction vectors that satisfy the A-orthogonality condition.

We will recall again the Gram-Schmidt process from elementary linear algebra, but this time we will use modified Gram-Schmidt to compute an A-orthogonal (or A-conjugate) set of vectors from a set of linearly independent vectors using a positive definite matrix A. Using the modified Gram-Schmidt (MGS) approach is numerically more stable than the Classical Gram-Schmidt method.

Two vectors $\boldsymbol{u}$ and $\boldsymbol{v}$ are A-orthogonal if

$$< \boldsymbol{u}, \boldsymbol{v} >_A = \boldsymbol{u}^T A \boldsymbol{v} = 0$$

with $< \boldsymbol{u}, \boldsymbol{v} >_A$ as the A-inner product.

In MGS, every standard dot product $\boldsymbol{u} \cdot \boldsymbol{v}$ is replaced by $\boldsymbol{u}^T A \boldsymbol{v}$. The vector magnitude is determined by its A-norm

$$||\boldsymbol{v}||_A = \sqrt{\boldsymbol{v}^T A \boldsymbol{v}}$$

Given a set of linearly independent vectors $\{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_n\}$, we can generate and A-orthogonal set $\{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n\}$ using these steps:

1. **First Vector:**

$$\boldsymbol{v}_1 = \boldsymbol{x}_1$$

2. **Subsequent Vectors:**
   For $k = 2$ to n:

$$\boldsymbol{v}_k = \boldsymbol{x}_k - \sum_{i=1}^{k-1} \frac{\boldsymbol{x}_k^T A \boldsymbol{v}_i}{\boldsymbol{v}_i^T A \boldsymbol{v}_i} \boldsymbol{v}_i$$

In SymIntegration we can use this function to compute an A-orthogonal set of vectors from a set of linearly independent vectors (usually basis vectors or identity matrix) using a positive-definite matrix A with modified Gram-Schmidt method:
ModifiedGramSchmidtA(const vector<vector<double>>& A, vector<vector<double>>& V)

Suppose we have this matrix

$$A = \begin{bmatrix} 4 & 3 & 0 \\ 3 & 4 & -1 \\ 0 & -1 & 4 \end{bmatrix}$$

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Modified Gram-Schmidt ]# ./main
Matrix A:
              4.000000              3.000000              0.000000
              3.000000              4.000000             -1.000000
              0.000000             -1.000000              4.000000

A-orthogonalized (and A-normalized) vectors:
Vector 0: [ 0.500000 0.000000 0.000000 ]
Vector 1: [ -0.566947 0.755929 0.000000 ]
Vector 2: [ -0.231455 0.308607 0.540062 ]

Verification check (V[0]^T * A * V[1]): -0.000000 (Should be near 0)

Time taken by function: 993 microseconds
```

Figure 7.33: *The A-orthogonal set of vectors (**SymIntegration/Examples/Linear Algebra/Test SymIntegration ModifiedGramSchmidtA/main.cpp**).*

viii. Compute the Solution of a Linear System with Conjugate Gradient Method and Preconditioned Conjugate Gradient Method

The linear system $A\boldsymbol{x} = \boldsymbol{b}$ with

$$A = \begin{bmatrix} 0.2 & 0.1 & 1 & 1 & 0 \\ 0.1 & 4 & -1 & 1 & -1 \\ 1 & -1 & 60 & 0 & -2 \\ 1 & 1 & 0 & 8 & 4 \\ 0 & -1 & -2 & 4 & 700 \end{bmatrix}, \quad \boldsymbol{b} = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{bmatrix}$$

has the solution

$$\boldsymbol{x}^* = \begin{bmatrix} 7.859713071 \\ 0.4229264082 \\ -0.07359223906 \\ -0.5406430164 \\ 0.01062616286 \end{bmatrix}$$

The matrix A is symmetrix and positive definite but is ill-conditioned with condition number $K_\infty(A) = 13961.71$.

We will use Conjugate Gradient and Preconditioned Conjugate Gradient (with Jacobi preconditioner) with tolerance of 0.001 and compare the results obtained from the Jacobi, Gauss-Seidel, and SOR (with $\omega = 1.25$) iterative methods.

In SymIntegration, to approximate the solution of $A\boldsymbol{x} = \boldsymbol{b}$ with conjugate gradient we can use this function:
vector<double> ConjugateGradient(const CRSMatrix& A, const vector<double>& b, double ϵ, int max_iterations)

Moreover, to approximate the solution of $A\boldsymbol{x} = \boldsymbol{b}$ with preconditioned conjugate gradient we can use this function:
vector<double> PreconditionedConjugateGradient(const CRSMatrix& A, const vector<double>& b, double ϵ, int max_iterations)

The input matrix A for both functions above are in CRS format. In C++, an efficient implementation of the Conjugate Gradient (CG) method requires compressing a sparse matrix A into the Compressed Row Storage (CRS) format, also known as Compressed Sparse Row (CSR), to save memory and speed up matrix-vector multiplications.

In SymIntegration we can load the matrix A with type of **vector<vector<double>>** easily, if we have a matrix A that contain mostly zeros, it is called a sparse matrix, converting this dense matrix into CRS format and solve the linear system $A\boldsymbol{x} = \boldsymbol{b}$ with CRS format we can speed up the matrix vector multiplication and it will be very helpful in mission-critical task.

We create a struct that is named as **CRSMatrix** and we define the struct inside a header file in **include/symintegral/numericalmethods.h**. We use include guards **#ifndef** to prevent duplicate inclusion.

To convert a dense 2D vector (**vector<vector<double>>**) into a CRS struct, we need to iterate through the original matrix A, identify non-zero elements, and track their column indices and row starting positions.

```
...

#ifndef STRUCT_CRS_MATRIX
#define STRUCT_CRS_MATRIX

// Defining struct in .h file (header file) will make it work when we call it from
     .cpp file from anywhere that use SymIntegration library
// Structure to store a sparse matrix in Compressed Row Storage (CRS) format
struct CRSMatrix {
       vector<double> values; // Non-zero elements
       vector<int> col_indices; // Column indices of non-zero elements
       vector<int> row_ptr; // Row pointers
       int num_rows;
       int num_cols;
};

CRSMatrix denseToCRS(const vector<vector<double>>& dense);
vector<double> spmv(const CRSMatrix& A, const std::vector<double>& x);
double dotProduct(const vector<double>& u, const vector<double>& v);
vector<double> conjugateGradient(const CRSMatrix& A, const vector<double>& b,
    double tolerance, int max_iterations);

...
```

Code 70: *include/symintegral/numericalmethods.h*

In the source file **src/numericalmethods.cpp** we write the logic for any functions declared related with the struct **CRSMatrix** and the conjugate gradient computation.

```
...

CRSMatrix denseToCRS(const vector<vector<double>>& dense)
{
       CRSMatrix crs;
       crs.num_rows = dense.size();
       crs.num_cols = crs.num_rows > 0 ? dense[0].size() : 0;

       crs.row_ptr.push_back(0); // First element is always 0

       for (int i = 0; i < crs.num_rows; ++i)
       {
              for (int j = 0; j < crs.num_cols; ++j)
              {
                     if (dense[i][j] != 0.0)
                     {
                            crs.values.push_back(dense[i][j]);
```

```
                        crs.col_indices.push_back(j);
                    }
                }
                crs.row_ptr.push_back(crs.values.size());
        }
        return crs;
}

// Helper function: Sparse Matrix—Vector Multiplication (y = A * x)
vector<double> spmv(const CRSMatrix& A, const std::vector<double>& x)
{
        vector<double> y(A.num_rows, 0.0);
        for (int i = 0; i < A.num_rows; ++i)
        {
                double sum = 0.0;
                int row_start = A.row_ptr[i];
                int row_end = A.row_ptr[i + 1];
                for (int k = row_start; k < row_end; ++k)
                {
                        sum += A.values[k] * x[A.col_indices[k]];
                }
                y[i] = sum;
        }
        return y;
}

// Helper function: Vector dot product (u . v)
double dotProduct(const vector<double>& u, const vector<double>& v)
{
        double dot = 0.0;
        for (size_t i = 0; i < u.size(); ++i)
        {
                dot += u[i] * v[i];
        }
        return dot;
}

// Conjugate Gradient Solver for CRS Matrix
vector<double> ConjugateGradient(const CRSMatrix& A, const vector<double>& b,
    double tolerance, int max_iterations)
{
        cout << "Starting Conjugate Gradient Method.\n\n";

        int n = A.num_rows;
        vector<double> x(n, 0.0); // Initial guess x_0 = 0

        // r = b — A * x
        vector<double> Ax = spmv(A, x);
```

```
vector<double> r(n);
for (int i = 0; i < n; ++i)
{
	r[i] = b[i] — Ax[i];
}

// p = r
vector<double> p = r;
double rsold = dotProduct(r, r);

if (std::sqrt(rsold) < tolerance)
{
	return x; // Initial guess is already close enough
}

for (int iter = 0; iter < max_iterations; ++iter)
{
	vector<double> Ap = spmv(A, p);
	double alpha = rsold / dotProduct(p, Ap);

	// Update x and r
	for (int i = 0; i < n; ++i)
	{
		x[i] += alpha * p[i];
		r[i] —= alpha * Ap[i];
	}

	double rsnew = dotProduct(r, r);

	// Print progress (optional)
	cout << "Iteration " << iter + 1 << ": ";
	for (int i = 0; i < n; ++i)
	{
		cout << "x[" << i << "]=" << x[i] << " \t ";
	}
	cout << "( || r || : " << rsnew << ")\n";

	// Check convergence
	if (std::sqrt(rsnew) < tolerance)
	{
		cout << "Converged in " << iter + 1 << " iterations.\n";
		return x;
	}

	// Update direction p
	double beta = rsnew / rsold;
	for (int i = 0; i < n; ++i)
	{
```

```
                    p[i] = r[i] + beta * p[i];
                }
                rsold = rsnew;
        }

        cout << "Warning: Maximum iterations reached without full convergence.\n";
        return x;
}
...
```

Code 71: *src/numericalmethods.cpp*

The preconditioned conjugate gradient method is often used in the solution of large linear systems in which the matrix is sparse and positive definite. These systems must be solved to approximate solutions to boundary-value problems in ordinary-differential equations. The larger the system, the more impressive the conjugate gradient method becomes since it significantly reduces the number of iterations required.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Conjugate Gradient to Solve Linear System ]# ./main
Starting Conjugate Gradient Method.

Iteration 1: x[0]=0.00300832     x[1]=0.00601665         x[2]=0.00902497         x[3]=0.0120333          x[4]=0.0150416          ( || r || : 56.6566)
Iteration 2: x[0]=0.0464709      x[1]=0.0936386          x[2]=0.129853           x[3]=0.18423    x[4]=0.00632678         ( || r || : 30.9131)
Iteration 3: x[0]=0.127921       x[1]=0.268358           x[2]=0.0519631          x[3]=0.493859           x[4]=0.00484866         ( || r || : 0.52409)
Iteration 4: x[0]=0.305993       x[1]=0.491477           x[2]=0.053518           x[3]=0.389512           x[4]=0.00577334         ( || r || : 0.310433)
Iteration 5: x[0]=7.85971        x[1]=0.422926           x[2]=-0.0735922         x[3]=-0.540643          x[4]=0.0106262          ( || r || : 6.55823e-14)
Converged in 5 iterations.

Solution vector x:
x[0] = 7.85971308
x[1] = 0.42292641
x[2] = -0.07359224
x[3] = -0.54064302
x[4] = 0.01062616

Time taken by function: 1460 microseconds
```

Figure 7.34: *The approximated solutions after* 5 *iterations with conjugate gradient method with tolerance of* 0.001 *(**SymIntegration/Examples/Numerical Methods/Test SymIntegration Conjugate Gradient to Solve Linear System/main.cpp**).*

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Preconditioned Conjugate Gradient to Solve Linear System ]# ./main
Starting Preconditioned Conjugate Gradient.

Iteration 1: x[0]=2.79267        x[1]=0.279267           x[2]=0.0279267          x[3]=0.279267           x[4]=0.00398953         ( || r || : 2.3049)
Iteration 2: x[0]=5.43426        x[1]=0.672458           x[2]=-0.0355021         x[3]=-0.24152           x[4]=0.012351           ( || r || : 2.3598)
Iteration 3: x[0]=7.77114        x[1]=0.437064           x[2]=-0.0556134         x[3]=-0.54481           x[4]=0.0114452          ( || r || : 1.10344)
Iteration 4: x[0]=7.85969        x[1]=0.422883           x[2]=-0.0735988         x[3]=-0.540632          x[4]=0.0106434          ( || r || : 0.012201)
Iteration 5: x[0]=7.85971        x[1]=0.422926           x[2]=-0.0735922         x[3]=-0.540643          x[4]=0.0106262          ( || r || : 3.66808e-15)
Converged in 5 iterations.

Solution vector x:
x[0] = 7.85971308
x[1] = 0.42292641
x[2] = -0.07359224
x[3] = -0.54064302
x[4] = 0.01062616

Time taken by function: 1282 microseconds
```

Figure 7.35: *The approximated solutions after* 5 *iterations with preconditioned conjugate gradient method with tolerance of* 0.001 *(**SymIntegration/Examples/Numerical Methods/Test SymIntegration Preconditioned Conjugate Gradient to Solve Linear System/main.cpp**).*

```
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Numerical Methods/Test SymIntegration Jacobi Iterative Method ]# make
g++     -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Numerical Methods/Test SymIntegration Jacobi Iterative Method ]# ./main
Starting Jacobi Iteration...

Iteration 1: x[0]=5.000000      x[1]=0.500000          x[2]=0.050000          x[3]=0.500000          x[4]=0.007143          (Error: 5.050005)
Iteration 2: x[0]=2.000000      x[1]=0.264286          x[2]=-0.024762         x[3]=-0.191071         x[4]=0.005143          (Error: 3.088484)
Iteration 3: x[0]=5.947024      x[1]=0.492863          x[2]=0.021243          x[3]=0.214393          x[4]=0.008541          (Error: 3.974641)
Iteration 4: x[0]=3.575390      x[1]=0.305172          x[2]=-0.040618         x[3]=-0.309257         x[4]=0.006683          (Error: 2.436784)
Iteration 5: x[0]=6.596787      x[1]=0.479446          x[2]=-0.004281         x[3]=0.011588          x[4]=0.009230          (Error: 3.043596)
Iteration 6: x[0]=4.723739      x[1]=0.333420          x[2]=-0.051648         x[3]=-0.389144         x[4]=0.007749          (Error: 1.921578)
Iteration 7: x[0]=7.037250      x[1]=0.468218          x[2]=-0.022914         x[3]=-0.136020         x[4]=0.009695          (Error: 2.331395)
Iteration 8: x[0]=5.560558      x[1]=0.354769          x[2]=-0.059161         x[3]=-0.443031         x[4]=0.008524          (Error: 1.512965)
Iteration 9: x[0]=7.333574      x[1]=0.459085          x[2]=-0.036479         x[3]=-0.243678         x[4]=0.010012          (Error: 1.787381)
Iteration 10: x[0]=6.171241     x[1]=0.370963          x[2]=-0.064241         x[3]=-0.479089         x[4]=0.009087          (Error: 1.189527)
Iteration 11: x[0]=7.531166     x[1]=0.451703          x[2]=-0.046368         x[3]=-0.322319         x[4]=0.010227          (Error: 1.371428)
Iteration 12: x[0]=6.617586     x[1]=0.383265          x[2]=-0.067650         x[3]=-0.502972         x[4]=0.009497          (Error: 0.934025)
Iteration 13: x[0]=7.661479     x[1]=0.445765          x[2]=-0.053589         x[3]=-0.379855         x[4]=0.010371          (Error: 1.053079)
Iteration 14: x[0]=6.944337     x[1]=0.392622          x[2]=-0.069916         x[3]=-0.518591         x[4]=0.009797          (Error: 0.732551)
Iteration 15: x[0]=7.746225     x[1]=0.441010          x[2]=-0.058869         x[3]=-0.422018         x[4]=0.010467          (Error: 0.809206)
Iteration 16: x[0]=7.183931     x[1]=0.399749          x[2]=-0.071405         x[3]=-0.528638         x[4]=0.010016          (Error: 0.573936)
Iteration 17: x[0]=7.800339     x[1]=0.437214          x[2]=-0.062736         x[3]=-0.452968         x[4]=0.010531          (Error: 0.622225)
Iteration 18: x[0]=7.359912     x[1]=0.405182          x[2]=-0.072368         x[3]=-0.534960         x[4]=0.010177          (Error: 0.449241)
Iteration 19: x[0]=7.834045     x[1]=0.434194          x[2]=-0.065573         x[3]=-0.475725         x[4]=0.010572          (Error: 0.478747)
Iteration 20: x[0]=7.489393     x[1]=0.409330          x[2]=-0.072978         x[3]=-0.538816         x[4]=0.010294          (Error: 0.351338)
Iteration 21: x[0]=7.854306     x[1]=0.431798          x[2]=-0.067658         x[3]=-0.492487         x[4]=0.010598          (Error: 0.368567)
Iteration 22: x[0]=7.584828     x[1]=0.412499          x[2]=-0.073355         x[3]=-0.541062         x[4]=0.010381          (Error: 0.274560)
Iteration 23: x[0]=7.865837     x[1]=0.429901          x[2]=-0.069193         x[3]=-0.504856         x[4]=0.010614          (Error: 0.283896)
Iteration 24: x[0]=7.655294     x[1]=0.414924          x[2]=-0.073578         x[3]=-0.542274         x[4]=0.010444          (Error: 0.214410)
Iteration 25: x[0]=7.871803     x[1]=0.428403          x[2]=-0.070325         x[3]=-0.513999         x[4]=0.010624          (Error: 0.218787)
Iteration 26: x[0]=7.707419     x[1]=0.416780          x[2]=-0.073703         x[3]=-0.542838         x[4]=0.010491          (Error: 0.167333)
Iteration 27: x[0]=7.874311     x[1]=0.427221          x[2]=-0.071161         x[3]=-0.520770         x[4]=0.010630          (Error: 0.168688)
Iteration 28: x[0]=7.746046     x[1]=0.418202          x[2]=-0.073764         x[3]=-0.543006         x[4]=0.010526          (Error: 0.130517)
Iteration 29: x[0]=7.874750     x[1]=0.426291          x[2]=-0.071780         x[3]=-0.525794         x[4]=0.010632          (Error: 0.130117)
Iteration 30: x[0]=7.774723     x[1]=0.419293          x[2]=-0.073787         x[3]=-0.542946         x[4]=0.010551          (Error: 0.101748)
Iteration 31: x[0]=7.874018     x[1]=0.425560          x[2]=-0.072239         x[3]=-0.529528         x[4]=0.010634          (Error: 0.100405)
Iteration 32: x[0]=7.796052     x[1]=0.420130          x[2]=-0.073787         x[3]=-0.542764         x[4]=0.010570          (Error: 0.079283)
Iteration 33: x[0]=7.872688     x[1]=0.424986          x[2]=-0.072580         x[3]=-0.532308         x[4]=0.010634          (Error: 0.077507)
Iteration 34: x[0]=7.811945     x[1]=0.420773          x[2]=-0.073774         x[3]=-0.542526         x[4]=0.010584          (Error: 0.061751)
Iteration 35: x[0]=7.871113     x[1]=0.424535          x[2]=-0.072833         x[3]=-0.534382         x[4]=0.010633          (Error: 0.059851)
Iteration 36: x[0]=7.823809     x[1]=0.421268          x[2]=-0.073755         x[3]=-0.542273         x[4]=0.010595          (Error: 0.048077)
Iteration 37: x[0]=7.869506     x[1]=0.424183          x[2]=-0.073023         x[3]=-0.535932         x[4]=0.010633          (Error: 0.046232)
```

Figure 7.36: *The approximated solutions after* 67 *iterations with Jacobi method with tolerance of* 0.001 *(**SymIntegration/Examples/Numerical Methods/Test SymIntegration Jacobi Iterative Method/main.cpp**).*

```
Iteration 37: x[0]=7.869506     x[1]=0.424183          x[2]=-0.073023         x[3]=-0.535932         x[4]=0.010633          (Error: 0.046232)
Iteration 38: x[0]=7.832681     x[1]=0.421648          x[2]=-0.073734         x[3]=-0.542027         x[4]=0.010603          (Error: 0.037418)
Iteration 39: x[0]=7.867984     x[1]=0.423907          x[2]=-0.073164         x[3]=-0.537093         x[4]=0.010632          (Error: 0.035722)
Iteration 40: x[0]=7.839328     x[1]=0.421941          x[2]=-0.073714         x[3]=-0.541802         x[4]=0.010608          (Error: 0.029113)
Iteration 41: x[0]=7.866609     x[1]=0.423691          x[2]=-0.073270         x[3]=-0.537963         x[4]=0.010631          (Error: 0.027609)
Iteration 42: x[0]=7.844316     x[1]=0.422166          x[2]=-0.073694         x[3]=-0.541603         x[4]=0.010613          (Error: 0.022644)
Iteration 43: x[0]=7.865404     x[1]=0.423523          x[2]=-0.073349         x[3]=-0.538617         x[4]=0.010630          (Error: 0.021344)
Iteration 44: x[0]=7.848066     x[1]=0.422339          x[2]=-0.073677         x[3]=-0.541431         x[4]=0.010616          (Error: 0.017608)
Iteration 45: x[0]=7.864370     x[1]=0.423391          x[2]=-0.073408         x[3]=-0.539109         x[4]=0.010630          (Error: 0.016504)
Iteration 46: x[0]=7.850889     x[1]=0.422473          x[2]=-0.073662         x[3]=-0.541285         x[4]=0.010619          (Error: 0.013688)
Iteration 47: x[0]=7.863498     x[1]=0.423288          x[2]=-0.073453         x[3]=-0.539480         x[4]=0.010629          (Error: 0.012765)
Iteration 48: x[0]=7.853019     x[1]=0.422576          x[2]=-0.073649         x[3]=-0.541163         x[4]=0.010620          (Error: 0.010639)
Iteration 49: x[0]=7.862771     x[1]=0.423208          x[2]=-0.073487         x[3]=-0.539760         x[4]=0.010628          (Error: 0.009874)
Iteration 50: x[0]=7.854628     x[1]=0.422656          x[2]=-0.073638         x[3]=-0.541062         x[4]=0.010622          (Error: 0.008267)
Iteration 51: x[0]=7.862172     x[1]=0.423146          x[2]=-0.073512         x[3]=-0.539971         x[4]=0.010628          (Error: 0.007640)
Iteration 52: x[0]=7.855845     x[1]=0.422718          x[2]=-0.073630         x[3]=-0.540979         x[4]=0.010623          (Error: 0.006423)
Iteration 53: x[0]=7.861683     x[1]=0.423097          x[2]=-0.073531         x[3]=-0.540132         x[4]=0.010628          (Error: 0.005912)
Iteration 54: x[0]=7.856767     x[1]=0.422765          x[2]=-0.073622         x[3]=-0.540911         x[4]=0.010624          (Error: 0.004989)
Iteration 55: x[0]=7.861285     x[1]=0.423059          x[2]=-0.073546         x[3]=-0.540253         x[4]=0.010627          (Error: 0.004576)
Iteration 56: x[0]=7.857467     x[1]=0.422802          x[2]=-0.073616         x[3]=-0.540857         x[4]=0.010624          (Error: 0.003875)
Iteration 57: x[0]=7.860963     x[1]=0.423030          x[2]=-0.073557         x[3]=-0.540346         x[4]=0.010627          (Error: 0.003542)
Iteration 58: x[0]=7.857998     x[1]=0.422830          x[2]=-0.073611         x[3]=-0.540813         x[4]=0.010625          (Error: 0.003009)
Iteration 59: x[0]=7.860705     x[1]=0.423007          x[2]=-0.073565         x[3]=-0.540416         x[4]=0.010627          (Error: 0.002742)
Iteration 60: x[0]=7.858403     x[1]=0.422852          x[2]=-0.073607         x[3]=-0.540777         x[4]=0.010625          (Error: 0.002336)
Iteration 61: x[0]=7.860498     x[1]=0.422989          x[2]=-0.073572         x[3]=-0.540469         x[4]=0.010627          (Error: 0.002123)
Iteration 62: x[0]=7.858711     x[1]=0.422869          x[2]=-0.073604         x[3]=-0.540749         x[4]=0.010625          (Error: 0.001814)
Iteration 63: x[0]=7.860333     x[1]=0.422975          x[2]=-0.073577         x[3]=-0.540510         x[4]=0.010627          (Error: 0.001644)
Iteration 64: x[0]=7.858946     x[1]=0.422882          x[2]=-0.073602         x[3]=-0.540727         x[4]=0.010626          (Error: 0.001408)
Iteration 65: x[0]=7.860202     x[1]=0.422964          x[2]=-0.073580         x[3]=-0.540541         x[4]=0.010627          (Error: 0.001273)
Iteration 66: x[0]=7.859125     x[1]=0.422892          x[2]=-0.073600         x[3]=-0.540709         x[4]=0.010626          (Error: 0.001093)
Iteration 67: x[0]=7.860098     x[1]=0.422956          x[2]=-0.073583         x[3]=-0.540565         x[4]=0.010626          (Error: 0.000986)

Converged in 67 iterations.

Time taken by function: 4196 microseconds
```

Figure 7.37: *The approximated solutions after* 67 *iterations with Jacobi method with tolerance of* 0.001 *(**SymIntegration/Examples/Numerical Methods/Test SymIntegration Jacobi Iterative Method/main.cpp**).*

```
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Numerical Methods/Test SymIntegration Gauss-Seidel Method ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Numerical Methods/Test SymIntegration Gauss-Seidel Method ]# ./main
Starting Gauss-Seidel Method
Iterative Steps:
Iteration 1: x[0]=5.000000       x[1]=0.375000          x[2]=-0.027083         x[3]=-0.171875         x[4]=0.008583          (Error: 5.017068)
Iteration 2: x[0]=5.807292       x[1]=0.393161          x[2]=-0.039949         x[3]=-0.279348         x[4]=0.009187          (Error: 0.814718)
Iteration 3: x[0]=6.399908       x[1]=0.402149          x[2]=-0.049656         x[3]=-0.354850         x[4]=0.009603          (Error: 0.597553)
Iteration 4: x[0]=6.821460       x[1]=0.408163          x[2]=-0.056568         x[3]=-0.408504         x[4]=0.009899          (Error: 0.425052)
Iteration 5: x[0]=7.121282       x[1]=0.412427          x[2]=-0.061484         x[3]=-0.446663         x[4]=0.010109          (Error: 0.302310)
Iteration 6: x[0]=7.334522       x[1]=0.415459          x[2]=-0.064981         x[3]=-0.473802         x[4]=0.010258          (Error: 0.215011)
Iteration 7: x[0]=7.486184       x[1]=0.417615          x[2]=-0.067468         x[3]=-0.493104         x[4]=0.010364          (Error: 0.152921)
Iteration 8: x[0]=7.594050       x[1]=0.419149          x[2]=-0.069236         x[3]=-0.506832         x[4]=0.010440          (Error: 0.108761)
Iteration 9: x[0]=7.670767       x[1]=0.420240          x[2]=-0.070494         x[3]=-0.516596         x[4]=0.010494          (Error: 0.077354)
Iteration 10: x[0]=7.725330      x[1]=0.421016          x[2]=-0.071389         x[3]=-0.523540         x[4]=0.010532          (Error: 0.055016)
Iteration 11: x[0]=7.764137      x[1]=0.421567          x[2]=-0.072025         x[3]=-0.528479         x[4]=0.010559          (Error: 0.039129)
Iteration 12: x[0]=7.791737      x[1]=0.421960          x[2]=-0.072478         x[3]=-0.531992         x[4]=0.010579          (Error: 0.027829)
Iteration 13: x[0]=7.811367      x[1]=0.422239          x[2]=-0.072800         x[3]=-0.534490         x[4]=0.010592          (Error: 0.019793)
Iteration 14: x[0]=7.825328      x[1]=0.422437          x[2]=-0.073028         x[3]=-0.536267         x[4]=0.010602          (Error: 0.014077)
Iteration 15: x[0]=7.835257      x[1]=0.422579          x[2]=-0.073191         x[3]=-0.537531         x[4]=0.010609          (Error: 0.010012)
Iteration 16: x[0]=7.842320      x[1]=0.422679          x[2]=-0.073307         x[3]=-0.538429         x[4]=0.010614          (Error: 0.007121)
Iteration 17: x[0]=7.847342      x[1]=0.422751          x[2]=-0.073389         x[3]=-0.539069         x[4]=0.010617          (Error: 0.005064)
Iteration 18: x[0]=7.850915      x[1]=0.422801          x[2]=-0.073448         x[3]=-0.539523         x[4]=0.010620          (Error: 0.003602)
Iteration 19: x[0]=7.853456      x[1]=0.422837          x[2]=-0.073490         x[3]=-0.539847         x[4]=0.010622          (Error: 0.002562)
Iteration 20: x[0]=7.855263      x[1]=0.422863          x[2]=-0.073519         x[3]=-0.540077         x[4]=0.010623          (Error: 0.001822)
Iteration 21: x[0]=7.856548      x[1]=0.422881          x[2]=-0.073540         x[3]=-0.540240         x[4]=0.010624          (Error: 0.001296)
Iteration 22: x[0]=7.857462      x[1]=0.422894          x[2]=-0.073555         x[3]=-0.540356         x[4]=0.010625          (Error: 0.000922)

Converged successfully in 22 iterations.

Final Solution:
x[1] = 7.857462
x[2] = 0.422894
x[3] = -0.073555
x[4] = -0.540356
x[5] = 0.010625

Time taken by function: 2050 microseconds
```

Figure 7.38: *The approximated solutions after* 22 *iterations with Gauss-Seidel method with tolerance of* 0.001 *(**SymIntegration/Examples/Numerical Methods/Test SymIntegration Gauss-Seidel Method/main.cpp**).*

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration SOR Method ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration SOR Method ]# ./main
Starting SOR Method with omega = 1.250000

Iteration 1: x[0]=6.312500       x[1]=3.519531          x[2]=-6.650146         (Error: 9.648598)
Iteration 2: x[0]=2.622314       x[1]=3.958527          x[2]=-4.600424         (Error: 4.244002)
Iteration 3: x[0]=3.133303       x[1]=4.010265          x[2]=-5.096686         (Error: 0.714186)
Iteration 4: x[0]=2.957051       x[1]=4.007484          x[2]=-4.973490         (Error: 0.215057)
Iteration 5: x[0]=3.003721       x[1]=4.002925          x[2]=-5.005714         (Error: 0.056897)
Iteration 6: x[0]=2.996328       x[1]=4.000926          x[2]=-4.998282         (Error: 0.010672)
Iteration 7: x[0]=3.000050       x[1]=4.000259          x[2]=-5.000349         (Error: 0.004309)
Iteration 8: x[0]=2.999745       x[1]=4.000065          x[2]=-4.999892         (Error: 0.000582)

Converged in 8 iterations.
Solution vector x:
x[0] = 2.999745
x[1] = 4.000065
x[2] = -4.999892

Time taken by function: 1393 microseconds
```

Figure 7.39: *The approximated solutions after* 10 *iterations with SOR method with tolerance of* 0.001 *and* $\omega = 1.25$ *(**SymIntegration/Examples/Numerical Methods/Test SymIntegration SOR Method/main.cpp**).*

ix. Functions in SymIntegration Related to Iterative Techniques in Matrix Algebra

[SI*] We will list the basic functions, struct and classes that can be used in iterative techniques in matrix algebra:

JacobiMethod(const vector<vector<double>>& A, const vector<double>& b, int N, double ϵ)
this **void** function computes the Jacobi iterative method to approximate the solution of a linear system $A\boldsymbol{x} = \boldsymbol{b}$ with maximum iterations N and the tolerance is ϵ. If the matrix A is not strictly diagonally dominant, the method might diverge. The error formula that is used here

$$\text{error}^{(k)} = \sqrt{\sum_{i=1}^{n} (x_i^{(k)} - x_i^{(k-1)})^2}$$

GaussSeidel(const vector<vector<double>>& A, const vector<double>& b, const vector<double>& x, double ϵ, int N)
this **bool** function approximates the solution of $A\boldsymbol{x} = \boldsymbol{b}$ with Gauss-Seidel method with maximum iterations N and the tolerance is ϵ. The algorithm updates elements directly into x_i, when the inner loop progresses to calculating x_{i+1}, it automatically fetches the newly calculated x_i from memory. The formula divides directly by a_{ii}, always ensure the main diagonal components of matrix A values do not contain zeros. If zeros occur on the diagonal, rearrange the system rows beforehand. The error formula that is used here

$$\text{error}^{(k)} = \sqrt{\sum_{i=1}^{n} (x_i^{(k)} - x_i^{(k-1)})^2}$$

SORIterativeMethod(const vector<vector<double>>& A, const vector<double>& b, const vector<double>& x, double ω, double ϵ, int N)
this **bool** function approximates the solution of $A\boldsymbol{x} = \boldsymbol{b}$ with Successive Over-Relaxation (SOR) method with ω as the parameter, maximum iterations N and the tolerance is ϵ. The error formula that is used here

$$\text{error}^{(k)} = \sqrt{\sum_{i=1}^{n} (x_i^{(k)} - x_i^{(k-1)})^2}$$

An alternative for the error formula is by using L_∞ norm, the maximum absolute difference between current and previous values.
Same case like Jacobi and Gauss-Seidel, the matrix A for SOR method should ideally be diagonally dominant or symmetric positive-definite to guarantee convergence. If the diagonal entries are weak $\left(|a_{ii}| \leq \sum_{j \neq i} |a_{ij}|\right)$, the code might diverge.
Finding the optimal ω usually requires empirical testing or computing the spectral radius of the Jacobi iteration matrix.
For massive engineering workflows (such as 2D/3D heat equations or grid computations), it is best to avoid 2D vectors and store coefficients as sparse arrays or band-diagonal formats.

struct CRSMatrix
a structure to store a sparse matrix in Compressed Row Storage (CRS) format, the elements are:

- **vector<double>** values (the non-zero elements, read row by row)

- **vector<int>** col_indices (column indices of non-zero elements)
- **vector<int>** row_ptr (row pointers, stores the starting index in the **values** array for each row. It has a size of **num_rows + 1**, with the last element equal to the total number of non-zero values. Instead of scanning across an entire 2D matrix row, which contains wasted zeros, row_ptr[i] and row_ptr[i+1] isolate only the meaningful indexes inside the packed arrays)
- **int** num_rows
- **int** num_cols

Some functions related to conjugate gradient and preconditioned conjugate gradient using **struct CRSMatrix**:

- **CRSMatrix denseToCRS(const vector<vector<double>>&** A**)**
  to convert / compress a dense matrix A into CRS format. This routine strips out zero entries. It maps rows into sequential memory chunks inside **values** and **col_indices**, bound by pointers in **row_ptr**.
- **vector<double> spmv(const CRSMatrix&** A**, const vector<double>&** $\boldsymbol{x}$**)**
  to perform sparse matrix-vector multiplication by looping through each row, retrieve the column indexes and non-zero values corresponding to that row via the row pointer array, multiply those values by the matching vector entries, and accumulate the result. The **for** loop here guarantees that we process one row at a time. This keeps memory access localized to one row of output element $y[i]$. This is the primary computational bottleneck of the CG algorithmm, it utilizes the CRS layout to bypass zeros entirely. It drops runtime complexity $O(N^2)$ to $O(nnz)$, where nnz is the number of non-zero entries.
- **double dotProduct(const vector<double>&** $\boldsymbol{u}$**, const vector<double>&** $\boldsymbol{v}$**)**
- **vector<double> ConjugateGradient(const CRSMatrix&** A**, const vector<double>&** $\boldsymbol{b}$**, double** ϵ**, int max_iterations)**
  to compute the solution of a linear system $A\boldsymbol{x} = \boldsymbol{b}$ with conjugate gradient method. Conjugate gradient requires matrix A to be symmetric and positive-definite. We set the initial approximation vector to be zeros vector.
- **vector<double> PreconditionedConjugateGradient(const CRSMatrix&** A**, const vector<double>&** $\boldsymbol{b}$**, double** ϵ**, int max_iterations)**
  to compute the solution of a linear system $A\boldsymbol{x} = \boldsymbol{b}$ with preconditioned conjugate gradient method. The preconditioner is Jacobi ($M = \text{diag}(A)$), the inverse diagonal elements M^{-1} are cached beforehand in **get_jacobi_preconditioner** to transform the linear correction vector via simple vector scalar multiplication : $z[i] = \text{inv}(M[i]) * r[i]$. We set the initial approximation vector to be zeros vector.

CRS format is efficient to help computing sparse matrix, since zero elements are never allocated or processed (memory savings), and the elements of a single row are laid out sequentially next to each other in memory (**values** array), making ideal use of CPU cache lines.

In C++, defining a **struct** function (member function) inside a header (**.h**) file makes it implicitly inline, whereas defining it inside a source file (**.cpp**) file treats it as a standard outlined function.
Calling a function degined in a header (**.h**) file is usually faster because the compiler can inline the code. However, it is best to declare a **struct** and its function signatures in a header (**.h**) file, and write the actual function code (implementations) in a source (**.cpp**) file for larger

programs.
Eigen compiles the program very slow, due to its being a header only library. They put all the definition in the (**.h**) files.
The reasons why we need to use a header (**.h**) file to put a **struct**:

- **Sharing:** Place the **struct** definition in a **.h** file so multiple **.cpp** files can use the same **struct**.
- **Inline Functions:** Write very short functions directly inside the **struct** in the header (**.h**) file. The compiler can expand these quickly.
- **Templates:** Keep template **struct** functions in the header (**.h**) file because the compiler needs to see the full code to build them.

The reasons why we need to use a source (**.cpp**) file to put the functions' definitions / implementations related to a **struct**:

- **Hiding Details:** Put long function code in a **.cpp** file to keep your header clean and easy to read.
- **Faster Builds:** Change the code inside a **.cpp** file without forcing every other file that includes the header to recompile.
- **Clean Design:** Separate what the **struct** does (in the **.h**) from how it does it (in the **.cpp**).

Put **declarations** (the blueprint) in the **.h** file, and put **definitions** (the heavy code) in the **.cpp** file, unless the function is very short. Speed differences for large functions do not matter.

The reason why putting the definition of functions in the header file is faster:

- **Implicit Inlining:** Functions written directly inside a **struct** inside a **.h** file are inline by default.
- **No Call Overhead:** The compiler replaces the function call directly with the function's code. This removes the time needed to jump to a different memory address.
- **Visibility:** The compiler must see the function body when it compiles the **.cpp** file to inline it. A **.h** file makes this code visible.

The reason why putting the definition of functions in the source file is slower:

- **Separate Compilation:** The compiler only sees the function declaration in the header, not the body in the **.cpp** file.
- **Standard Call:** It makes a normal function call, which takes a tiny bit more time.
- **Link-Time Optimization:** Modern compilers can sometimes inline **.cpp** functions if we turn on Link-Time Optimization(LTO), but this is not automatic.

We will create a comparison between defining a function in **.h** and defining a function in **.cpp**, the case here for SymIntegration is that we define the long functions that include some **for** loop when computing conjugate gradient with CRS matrix in **.cpp** for a reason: neat coding. It is just a style of coding for SymIntegration, and it won't take long to compile the code. If we want to see the comparison that only use **.h** or header only library, it is **Eigen**, and it is called the gold standard and already used by NASA, CERN, and other prestigious institutions, but based on our experiment compiling a code that is using **Eigen** can take quite a long time (the build times), even if the computing and execution time is very fast.

- **Use the .h file** for small, simple, 1-to-3 line functions (like getters, setters, or basic initializations) where the speed boost of eliminating call overhead outweighs build times.

Feature	Defining in .h (Inline)	Defining in .cpp (Outlined)
Inlining	**Implicitly Inlined.** The compiler attempts to copy-paste the function code directly into the caller site to eliminate function call overhead	**Not Inlined.** The program jumps to a specific, single memory address where the function lives.
Compilation Speed	**Slower builds.** If we change code inside the function, every single **.cpp** file that includes this header must be recompiled.	**Faster builds.** If you change code inside the **.cpp** function, only that specific **.cpp** file recompiles.
Binary Size	**Can cause code bloat.** If a large function is inlined in many places, the final executable size increases.	**Optimized size.** The function exists in only one place in the compiled binary.
One Definition Rule (ODR)	Allowed in multiple translation units because inline functions bypass standard ODR duplication errors.	Standard behavior. If defined in a **.h** without the **inline** keyword, it triggers a "**duplicate symbol**" linker error.

Table 7.3: *Comparison of Inline and Outlined Coding in C++.*

- **Use the .cpp file** for complex logic, functions containing loops, heavy object creation, or code that we expect to modify frequently during development.

VII. Aproximation Theory

[SI*]

VIII. Numerical Solutions of Nonlinear Systems of Equations

[SI*] Finding approximations to the zeros or roots of nonlinear functions is a task that is often required in various applications in different fields of science and engineering. A system of nonlinear equations has the form

$$\begin{aligned} f_1(x_1, x_2, \ldots, x_n) &= 0 \\ f_2(x_1, x_2, \ldots, x_n) &= 0 \\ &\vdots \\ f_n(x_1, x_2, \ldots, x_n) &= 0 \end{aligned} \tag{7.55}$$

where each function f_i can be thought of as mapping a vector $\boldsymbol{x} = (x_1, x_2, \ldots, x_n)^T$ of the n-dimensional space $\mathbb{R}^n$ into the real line $\mathbb{R}$.
This system of n nonlinear equations in n unknowns can also be represented by defining a function $\boldsymbol{F}$ mapping $\mathbb{R}^n$ into $\mathbb{R}^n$ as

$$\boldsymbol{F}(x_1, x_2, \ldots, x_n) = (f_1(x_1, x_2, \ldots, x_n), f_2(x_1, x_2, \ldots, x_n), \ldots, f_n(x_1, x_2, \ldots, x_n))^T$$

If vector notation is used to represent the variables $x_1, x_2, \ldots, x_n$, system (7.55) assumes the form

$$\boldsymbol{F}\boldsymbol{x} = \boldsymbol{0} \tag{7.56}$$

i. Newton's Method

[SI*] To construct the algorithm that led to an appropriate fixed-point method in the one-dimensional case, we found a matching function ϕ with the property that

$$g(x) = x - \phi(x)f(x)$$

gives quadratic convergence to the fixed point p of the function g. From this condition Newton's method evolved by choosing $\phi(x) = \frac{1}{f'(x)}$, assuming that $f'(x) \neq 0$.
Using a similar approach in the n-dimensional case involves a matrix

$$A(\boldsymbol{x}) = \begin{bmatrix} a_{11}(\boldsymbol{x}) & a_{12}(\boldsymbol{x}) & \dots & a_{1n}(\boldsymbol{x}) \\ a_{21}(\boldsymbol{x}) & a_{22}(\boldsymbol{x}) & \dots & a_{2n}(\boldsymbol{x}) \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1}(\boldsymbol{x}) & a_{n2}(\boldsymbol{x}) & \dots & a_{nn}(\boldsymbol{x}) \end{bmatrix} \tag{7.57}$$

where each of the entries $a_{ij}(\boldsymbol{x})$ is a function from $\mathbb{R}^n$ into $\mathbb{R}$. This requires that $A(\boldsymbol{x})$ be found so that

$$\boldsymbol{G}(\boldsymbol{x}) = \boldsymbol{x} - A(\boldsymbol{x})^{-1}\boldsymbol{F}(\boldsymbol{x})$$

gives quadratic convergence to the solution of $\boldsymbol{F}(\boldsymbol{x}) = \boldsymbol{0}$, assuming that $A(\boldsymbol{x})$ is nonsingular at the fixed point $\boldsymbol{p}$ of $\boldsymbol{G}$.

Theorem 7.11: Solution for Function G

Let $\boldsymbol{p}$ be a solution of $\boldsymbol{G}(\boldsymbol{x}) = \boldsymbol{x}$. Suppose a number $\delta > 0$ exists with

1. $\frac{\partial g_i}{\partial x_j}$ is continuous on $N_\delta = \{\boldsymbol{x} | ||\boldsymbol{x} - \boldsymbol{p}|| < \delta\}$, for each $i = 1, 2, \dots, n$ and $j = 1, 2, \dots, n$;
2. $\frac{\partial^2 g_i(\boldsymbol{x})}{\partial x_j \partial x_k}$ is continuous, and $\left|\frac{\partial^2 g_i(\boldsymbol{x})}{\partial x_j \partial x_k}\right| \leq M$ for some constant M, whenever $\boldsymbol{x} \in N_\delta$, for each $i = 1, 2, \dots, n$, $j = 1, 2, \dots, n$, and $k = 1, 2, \dots, n$;
3. $\frac{\partial g_i(\boldsymbol{p})}{\partial x_k} = 0$, for each $i = 1, 2, \dots, n$ and $k = 1, 2, \dots, n$.

Then a number $\hat{\delta} \leq \delta$ exists such that the sequence generated by $\boldsymbol{x}^{(k)} = \boldsymbol{G}(\boldsymbol{x}^{(k-1)})$ converges quadratically to $\boldsymbol{p}$ for any choice of $\boldsymbol{x}^0$, provided that $||\boldsymbol{x}^{(0)} - \boldsymbol{p}|| < \hat{\delta}$. Moreover,

$$||\boldsymbol{x}^{(k)} - \boldsymbol{p}||_\infty \leq \frac{n^2 M}{2} ||\boldsymbol{x}^{(k-1)} - \boldsymbol{p}||_\infty^2, \quad \text{for each } k \geq 1$$

To use Theorem 7.11, suppose that $A(\boldsymbol{x})$ is an $n \times n$ matrix of functions from $\mathbb{R}^n$ into $\mathbb{R}$ in the form of Eq. (7.57), where the specific entries will be chosen later. Assume, moreover, that $A(\boldsymbol{x})$ is nonsingular near a solution $\boldsymbol{p}$ of $\boldsymbol{F}(\boldsymbol{x}) = \boldsymbol{0}$, and let $b_{ij}(\boldsymbol{x})$ denote the entry of $A(\boldsymbol{x})^{-1}$ in the ith row and jth column.
Since $\boldsymbol{G}(\boldsymbol{x}) = \boldsymbol{x} - A(\boldsymbol{x})^{-1}\boldsymbol{F}(\boldsymbol{x})$, we have $g_i(\boldsymbol{x}) = x_i - \sum_{j=1}^n b_{ij}(\boldsymbol{x}) f_j(\boldsymbol{x})$ and

$$\frac{\partial g_i}{\partial x_k}(\boldsymbol{x}) = \begin{cases} 1 - \sum_{j=1}^n \left(b_{ij}(\boldsymbol{x}) \frac{\partial f_j}{\partial x_k}(\boldsymbol{x}) + \frac{\partial b_{ij}}{\partial x_k}(\boldsymbol{x}) f_j(\boldsymbol{x}) \right), & \text{if } i = k \\ - \sum_{j=1}^n \left(b_{ij}(\boldsymbol{x}) \frac{\partial f_j}{\partial x_k}(\boldsymbol{x}) + \frac{\partial b_{ij}}{\partial x_k}(\boldsymbol{x}) f_j(\boldsymbol{x}) \right), & \text{if } i \neq k \end{cases}$$

Theorem 7.11 implies that we need $\frac{\partial g_i(\boldsymbol{p})}{\partial x_k} = 0$, for each $i = 1, 2, \ldots, n$ and $k = 1, 2, \ldots, n$. This means that for $i = k$,

$$0 = 1 - \sum_{j=1}^{n} b_{ij}(\boldsymbol{p}) \frac{\partial f_j}{\partial x_i}(\boldsymbol{p})$$

so

$$\sum_{j=1}^{n} b_{ij}(\boldsymbol{p}) \frac{\partial f_j}{\partial x_i}(\boldsymbol{p}) = 1 \tag{7.58}$$

When $k \neq i$,

$$0 = -\sum_{j=1}^{n} b_{ij}(\boldsymbol{p}) \frac{\partial f_j}{\partial x_k}(\boldsymbol{p})$$

so

$$\sum_{j=1}^{n} b_{ij}(\boldsymbol{p}) \frac{\partial f_j}{\partial x_k}(\boldsymbol{p}) = 0 \tag{7.59}$$

Defining the matrix $J(\boldsymbol{x})$ by

$$J(\boldsymbol{x}) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1}(\boldsymbol{x}) & \frac{\partial f_1}{\partial x_2}(\boldsymbol{x}) & \ldots & \frac{\partial f_1}{\partial x_n}(\boldsymbol{x}) \\ \frac{\partial f_2}{\partial x_1}(\boldsymbol{x}) & \frac{\partial f_2}{\partial x_2}(\boldsymbol{x}) & \ldots & \frac{\partial f_2}{\partial x_n}(\boldsymbol{x}) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1}(\boldsymbol{x}) & \frac{\partial f_n}{\partial x_2}(\boldsymbol{x}) & \ldots & \frac{\partial f_n}{\partial x_n}(\boldsymbol{x}) \end{bmatrix} \tag{7.60}$$

we see that conditions (7.58) and (7.59) require

$$A(\boldsymbol{p})^{-1} J(\boldsymbol{p}) = I$$

so

$$A(\boldsymbol{p}) = J(\boldsymbol{p})$$

An appropriate choice for $A(\boldsymbol{x})$ is, consequently, $A(\boldsymbol{x}) = J(\boldsymbol{x})$ since this satisfies condition (3) in Theorem 7.11.

The function $\boldsymbol{G}$ is defined by

$$\boldsymbol{G}(\boldsymbol{x}) = \boldsymbol{x} - J(\boldsymbol{x})^{-1} \boldsymbol{F}(\boldsymbol{x})$$

and the functional iteration procedure evolves from selecting $\boldsymbol{x}^0$ and generating, for $k \geq 1$,

$$\boldsymbol{x}^{(k)} = \boldsymbol{G}(\boldsymbol{x}^{(k-1)}) = \boldsymbol{x}^{(k-1)} - J(\boldsymbol{x}^{(k-1)})^{-1} \boldsymbol{F}(\boldsymbol{x}^{(k-1)}) \tag{7.61}$$

This is called Newton's method for nonlinear systems, and it is generally expected to give quadratic convergence, provided that a sufficiently accurate starting value is known and $J(\boldsymbol{p})^{-1}$ exists.

The matrix $J(\boldsymbol{x})$ is called the Jacobian matrix and has a number of applications in analysis. It might, in particular, be familiar to the reader due to its applicaiton in the multiple integration of a function of several variables over a region that requires a change of variables to be peformed.

The weakness in Newton's method arises from the need to compute and invert the matrix $J(\boldsymbol{x})$ at each step. In practice, explicit computation of $J(\boldsymbol{x})^{-1}$ is avoided by performing the operation in a two-step manner. First, a vector $\boldsymbol{y}$ is found that satisfies $J(\boldsymbol{x}^{(k-1)})\boldsymbol{y} = -\boldsymbol{F}(\boldsymbol{x}^{(k-1)})$. Then the new approximation, $\boldsymbol{x}^{(k)}$, is obtained by adding $\boldsymbol{y}$ to $\boldsymbol{x}^{(k-1)}$.

ii. Quasi-Newton Method

[SI*] A significant weakness of Newton's method for solving systems of nonlinear equations is that, for each iteration, a Jacobian matrix must be computed and an $n \times n$ linear system has to be solved that involves this matrix. Consider the amount of computation associated with one iteration of Newton's method. The Jacobian matrix associated with a system of n nonlinear equations written in the form $\boldsymbol{F}(\boldsymbol{x}) = \boldsymbol{0}$ requires that the n^2 partial derivatives of the n component functions of $\boldsymbol{F}$ be determined and evaluated. In most situations, the exact evaluation of the partial derivatives is inconvenient, although the problem has been made more tractable with the widespread use of symbolic computation systems.
When the exact evaluation is not practical, we can use finite difference approximations to the partial derivatives. For example,

$$\frac{\partial f_j}{\partial x_k}(\boldsymbol{x}^{(i)}) \approx \frac{f_j(\boldsymbol{x}^{(i)} + \boldsymbol{e}_k h) - f_j(\boldsymbol{x}^{(i)})}{h} \tag{7.62}$$

where h is small in absolute value and $\boldsymbol{e}_k$ is the vector whose only nonzero entry is a 1 in the kth coordinate. This approximation, however, still requires that at least n^2 scalar functional evaluations be performed to approximate the Jacobian and does not decrease the amount of calculation, in general $O(n^3)$, required for solving the linear system involving this approximate Jacobian. The total computational effort for just one iteration of Newton's method is consequently at least $n^2 + n$ scalar functional evaluations (n^2 for the evaluation of the Jacobian matrix and n for the evaluation of $\boldsymbol{F}$) together with $O(n^3)$ arithmetic operations to solve the linear system. This amount of computational effort is expensive, except for relatively small values of n and easily evaluated scalar functions.

[SI*] Now we consider a generalization of the Secant method to systems of nonlinear equations, a technique known as Broyden's method. The method requires only n scalar functional evaluation per iteration and also reduces the number of arithmetic calculations to $O(n^2)$. It belongs to a class of methods known as *least-change secant updates* that produce algorithms called quasi-Newton. These methods replace the Jacobian matrix in Newton's method with an approximation matrix that is updated at each iteration. The disadvantage of the methods is that the quadratic convergence of Newton's method is lost, being replaced, in general, by a convergence called superlinear, which implies that

$$\lim_{i \to \infty} \frac{||\boldsymbol{x}^{(i+1)} - \boldsymbol{p}}{\boldsymbol{x}^{(i)} - \boldsymbol{p}} = 0$$

where $\boldsymbol{p}$ denotes the solution to $\boldsymbol{F}(\boldsymbol{x}) = \boldsymbol{0}$ and $\boldsymbol{x}^{(i)}$ and $\boldsymbol{x}^{(i+1)}$ are consecutive approximations to $\boldsymbol{p}$.
In most applications, the reduction to superlinear convergence is a more than acceptable trade-off for the decrease in the amount of computation. An additional disadvantage of quasi-Newton method is that, unlike Newton's method, they are not self-correcting. Newton's method will generally correct for roundoff error with successive iterations, but unless special safeguards are incorporated, Broyden's method will not.
To describe Broyden's method, suppose that an initial approximation $\boldsymbol{x}^{(0)}$ is given to the solution $\boldsymbol{p}$ of $\boldsymbol{F}(\boldsymbol{x}) = \boldsymbol{0}$. We calculate the next approximation $\boldsymbol{x}^{(1)}$ in the same manner as Newton's method, or, if it is inconvenient to determine $J(\boldsymbol{x}^{(0)})$ exactly, we use the difference equations given by Eq. (7.62) to approximate the partial derivatives. To compute $\boldsymbol{x}^{(2)}$,

however, we depart from Newton's method and examine the Secant method for a single nonlinear equation. The Secant method uses the approximation

$$f'(x_1) \approx \frac{f(x_1) - f(x_0)}{x_1 - x_0}$$

as a replacement for $f'(x_1)$ in Newton's method. For nonlinear systems, $\boldsymbol{x}^{(1)} - \boldsymbol{x}^{(0)}$ is a vector, and the corresponding quotient is undefined. However, the method proceeds similarly in that we replace the matrix $J(\boldsymbol{x}^{(1)})$ in Newton's method for systems by a matrix A_1 with the property that

$$A_1(\boldsymbol{x}^{(1)} - \boldsymbol{x}^{(0)}) = \boldsymbol{F}(\boldsymbol{x}^{(1)}) - \boldsymbol{F}(\boldsymbol{x}^{(0)}) \tag{7.63}$$

Any nonzero vector in $\mathbb{R}^n$ can be written as the sum of a multiple of $\boldsymbol{x}^{(1)} - \boldsymbol{x}^{(0)}$ and a multiple of a vector in the orthogonal complement of $\boldsymbol{x}^{(1)} - \boldsymbol{x}^{(0)}$. So, to uniquely define the matrix A_1, we need to specify how it acts on the orthogonal complement of $\boldsymbol{x}^{(1)} - \boldsymbol{x}^{(0)}$. Since no information is available about the change in $\boldsymbol{F}$ in a direction orthogonal to $\boldsymbol{x}^{(1)} - \boldsymbol{x}^{(0)}$, we require that

$$A_1 \boldsymbol{z} = J(\boldsymbol{x}^{(0)})\boldsymbol{z}, \quad \text{whenever } (\boldsymbol{x}^{(1)} - \boldsymbol{x}^{(0)})^T \boldsymbol{z} = 0 \tag{7.64}$$

Thus, any vector orthogonal to $\boldsymbol{x}^{(1)} - \boldsymbol{x}^{(0)}$ is unaffected by the update from $J(\boldsymbol{x}^{(0)})$, which was used to compute $\boldsymbol{x}^{(1)}$, to A_1, which is used in the determination of $\boldsymbol{x}^{(2)}$.
Conditions (7.63) and (7.64) uniwuely define A_1 as

$$A_1 = J\left(\boldsymbol{x}^{(0)}\right) + \frac{\left[\boldsymbol{F}\left(\boldsymbol{x}^{(1)}\right) - \boldsymbol{F}\left(\boldsymbol{x}^{(0)}\right) - J\left(\boldsymbol{x}^{(0)}\right)\left(\boldsymbol{x}^{(1)} - \boldsymbol{x}^{(0)}\right)\right]\left(\boldsymbol{x}^{(1)} - \boldsymbol{x}^{(0)}\right)^T}{||\boldsymbol{x}^{(1)} - \boldsymbol{x}^{(0)}||_2^2}$$

It is this matrix that is used in place of $J(\boldsymbol{x}^{(1)})$ to determine $\boldsymbol{x}^{(2)}$ as

$$\boldsymbol{x}^{(2)} = \boldsymbol{x}^{(1)} - A_1^{-1}\boldsymbol{F}\left(\boldsymbol{x}^{(1)}\right)$$

Once $\boldsymbol{x}^{(2)}$ has been determined, the method is repeated to determine $\boldsymbol{x}^{(3)}$, using A_1 in place of $A_0 \equiv J\left(\boldsymbol{x}^{(0)}\right)$ and with $\boldsymbol{x}^{(2)}$ and $\boldsymbol{x}^{(1)}$ in place of $\boldsymbol{x}^{(1)}$ and $\boldsymbol{x}^{(0)}$. In general, once $\boldsymbol{x}^{(i)}$ has been determined, $\boldsymbol{x}^{(i+1)}$ is computed by

$$A_i = A_{i-1} + \frac{\boldsymbol{y}_i - A_{i-1}\boldsymbol{s}_i}{||\boldsymbol{s}_i||_2^2}\boldsymbol{s}_i^T \tag{7.65}$$

and

$$\boldsymbol{x}^{(i+1)} = \boldsymbol{x}^{(i)} - A_i^{-1}\boldsymbol{F}\left(\boldsymbol{x}^{(i)}\right) \tag{7.66}$$

where the notation $\boldsymbol{y}_i = \boldsymbol{F}\left(\boldsymbol{x}^{(i)}\right) - \boldsymbol{F}\left(\boldsymbol{x}^{(i-1)}\right)$ and $\boldsymbol{s}_i = \boldsymbol{x}^{(i)} - \boldsymbol{x}^{(i-1)}$ is introduced to simplify the equations.
If the method is performed as outlined in Equations (7.65) and (7.66), the number of scalar functional evaluations is reduced from $n^2 + n$ to n (those required for evaluating $\boldsymbol{F}\left(\boldsymbol{x}^{(i)}\right)$), but $O(n^3)$ calculations are still required to solve the associated $n \times n$ linear system

$$A_i \boldsymbol{s}_{i+1} = -\boldsymbol{F}\left(\boldsymbol{x}^{(i)}\right) \tag{7.67}$$

Employing the method in this form would not be justified because of the reduction to superlinear convergence from the quadratic convergence of Newton's method.

Theorem 7.12: Sherman-Morrison Formula

If A is a nonsingular matrix and $\boldsymbol{x}$ and $\boldsymbol{y}$ are vectors, then $A + \boldsymbol{x}\boldsymbol{y}^T$ is nonsingular, provided that $\boldsymbol{y}^T A^{-1} \neq -1$, and

$$(A + \boldsymbol{x}\boldsymbol{y}^T)^{-1} = A^{-1} - \frac{A^{-1}\boldsymbol{x}\boldsymbol{y}^T A^{-1}}{1 + \boldsymbol{y}^T A^{-1}\boldsymbol{x}}$$

The Sherman-Morrison formula permits A_i^{-1} to be computed directly from A_{i-1}^{-1}, eliminating the need for a matrix inversion with each iteration. By letting $A = A_{i-1}, \boldsymbol{x} = \frac{\boldsymbol{y}_i - A_{i-1}\boldsymbol{s}_i}{||\boldsymbol{s}_i||_2^2}$, and $\boldsymbol{y} = \boldsymbol{s}_i$, Eq. (7.65) together with Sherman-Morrison formula Theorem imply that

$$\begin{aligned} A_i^{-1} &= \left(A_{i-1} + \frac{\boldsymbol{y}_i - A_{i-1}\boldsymbol{s}_i}{||\boldsymbol{s}_i||_2^2} (\boldsymbol{s}_i)^T \right)^{-1} \\ &= A_{i-1}^{-1} - \frac{A_{i-1}^{-1} \left(\frac{\boldsymbol{y}_i - A_{i-1}\boldsymbol{s}_i}{||\boldsymbol{s}_i||_2^2} (\boldsymbol{s}_i)^T \right) A_{i-1}^{-1}}{1 + (\boldsymbol{s}_i)^T A_{i-1}^{-1} \left(\frac{\boldsymbol{y}_i - A_{i-1}\boldsymbol{s}_i}{||\boldsymbol{s}_i||_2^2} \right)} \\ &= A_{i-1}^{-1} - \frac{(A_{i-1}\boldsymbol{y}_i - \boldsymbol{s}_i)(\boldsymbol{s}_i)^T A_{i-1}^T}{||\boldsymbol{s}_i||_2^2 + (\boldsymbol{s}_i)^T A_{i-1}^{-1}\boldsymbol{y}_i - ||\boldsymbol{s}_i||_2^2} \end{aligned}$$

so

$$A_i^{-1} = A_{i-1}^{-1} + \frac{(\boldsymbol{s}_i - A_{i-1}^{-1}\boldsymbol{y}_i)(\boldsymbol{s}_i)^T A_{i-1}^{-1}}{(\boldsymbol{s}_i)^T A_{i-1}^{-1}\boldsymbol{y}_i} \tag{7.68}$$

This computation involves only matrix-vector multiplication at each step and therefore requires only $O(n^2)$ arithmetic calculations. The calculation of A_i is bypassed, as is the necessity of solving the linear system (7.67).

[SI*] Key advantages:

- **No derivative evaluation:** Eliminates the need to analytically derive or compute n^2 partial derivatives for the Jacobian at each step.
- **Lower cost:** Reduces computational work per iteration significantly compared to standard Newton-Raphson, especially for large systems when paired with matrix factorizations.
- **Superlinear convergence:** While slower than true quadratic Newton convergence, it achieves local superlinear convergence near the root.

Guide 7.2: Overview's of Broyden's Method

A Quasi-Newton algorithm for a multivariable nonlinear system $F(\boldsymbol{x}) = 0$ finds roots by iteratively approximating the Jacobian matrix or its inverse rather than computing exact partial derivatives at every step.
The most common algorithm for general nonlinear systems of equations is Broyden's method.
Instead of computing the exact Jacobian matrix $J(\boldsymbol{x})$ at high computational costs ($O(n^3)$), Broyden's method updates an approximation $B^{(k)}$ (or directly its inverse $H^{(k)}$) using only the change in the function values (F) and the change in the variable vectors ($\boldsymbol{x}$).
The algorithm:

1. **Initialization**
   - Choose an initial guess vectors $\boldsymbol{x}^{(0)} \in \mathbb{R}^n$.
   - Choose an initial $n \times n$ approximation of the Jacobian matrix, $B^{(0)}$ (often set to the identity matrix I or calculated via finite differences).
   - Set toleance ϵ and maximum iterations N.
   - Set $k = 0$.
2. **Iteration Loop**
   - Solve the linear system for the step vector $\boldsymbol{s}^{(k)}$:
   $$B^{(k)}\boldsymbol{s}^{(k)} = -\boldsymbol{F}(\boldsymbol{x}^{(k)})$$
   - Update the position
   $$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} + \boldsymbol{s}^{(k)}$$
   - Evaluate the function at the new point
   $$\boldsymbol{F}^{(k+1)} = \boldsymbol{F}(\boldsymbol{x}^{(k+1)})$$
   - Check convergence, if $||\boldsymbol{F}(\boldsymbol{x}^{(k+1)})|| < \epsilon$, stop and return $\boldsymbol{x}^{(k+1)}$ as the solution.
   - Calculate the change in function value
   $$\boldsymbol{y}^{(k)} = \boldsymbol{F}(\boldsymbol{x}^{(k+1)}) - \boldsymbol{F}(\boldsymbol{x}^{(k)})$$
   - Update the Jacobian approximation ($B^{(k)} \to B^{(k+1)}$) using Broyden's rank-one update formula:
   $$B^{(k+1)} = B^{(k)} + \frac{(\boldsymbol{y}^{(k)} - B^{(k)}\boldsymbol{s}^{(k)})^T(\boldsymbol{s}^{(k)})^T}{(\boldsymbol{s}^{(k)})^T\boldsymbol{s}^{(k)}}$$
   - Increment counter: $k = k + 1$. Repeat until convergence or $k = N$.

In this Numerical Analysis chapter, the writing of $\boldsymbol{s}_k$ has the same meaning with $\boldsymbol{s}^{(k)}$, although we use both interchangeably we prefer to use the latter because lower index sometimes is interpreted as the k-th member / coefficient of vector $\boldsymbol{s}$, while $\boldsymbol{s}^{(k)}$ means the vector $\boldsymbol{s}$ at iteration k; not to be confused with $y^{(k)}$ in differential equations it means the k-th derivative of y, and the y is not bolded.

iii. Steepest Descent Techniques

[SI*] The advantage of the Newton and Quasi-Newton methods for solving systems of nonlinear equations is their speed of convergence once a sufficiently accurate approximation is known. A weakness of these methods is that an accurate initial approximation to the solution is needed to ensure convergence. The Steepest Descent method converges only linearly to the solution, but it will usually converge even for poor initial approximations. As a consequence, this method is used to find sufficiently accurate starting approximations for the Newton-based techniques in the same way the Bisection method is used for a single equation.

[SI*] The method of Steepest Descent determines a local minimum for a multivariable function of the form $g : \mathbb{R}^n \to \mathbb{R}$. The method is valuable quite apart from the application as a starting method for solving nonlinear systems.
The connection between the minimization of a function from $\mathbb{R}^n$ to $\mathbb{R}$ and the solution of a system of nonlinear equations is due to the fact that a system of the form

$$\begin{aligned} f_1(x_1, x_2, \ldots, x_n) &= 0 \\ f_1(x_1, x_2, \ldots, x_n) &= 0 \\ &\vdots \\ f_n(x_1, x_2, \ldots, x_n) &= 0 \end{aligned}$$

has a solution at $\boldsymbol{x} = (x_1, x_2, \ldots, x_n)^T$ precisely when the function g defined by

$$g(x_1, x_2, \ldots, x_n) = \sum_{i=1}^{n} [f_i(x_1, x_2, \ldots, x_n)]$$

has the minimal value 0.
The method of Steepest Descent for finding a local minimum for an arbitrary function g from $\mathbb{R}^n$ into $\mathbb{R}$ can be intuitively described as follows:

1. Evaluate g at an initial approximation $\boldsymbol{x}^{(0)} = \left(x_1^{(0)}, x_2^{(0)}, \ldots, x_n^{(0)}\right)^T$.
2. Determine a direction from $\boldsymbol{x}^{(0)}$ that results in a decrease in the value of g.
3. Move an appropriate amount in this direction and call the new value $\boldsymbol{x}^{(1)}$.
4. Repeat steps 1 through 3 with $\boldsymbol{x}^{(0)}$ replaced by $\boldsymbol{x}^{(1)}$

Theorem 7.13: Extreme Value Theorem for Multivariable Functions

For $g : \mathbb{R}^n \to \mathbb{R}$, the gradient of g at $\boldsymbol{x} = (x_1, x_2, \ldots, x_n)^T$ is denoted by $\nabla g(\boldsymbol{x})$ and defined by

$$\nabla g(\boldsymbol{x}) = \left(\frac{\partial g}{\partial x_1}(\boldsymbol{x}), \frac{\partial g}{\partial x_2}(\boldsymbol{x}), \ldots, \frac{\partial g}{\partial x_n}(\boldsymbol{x})\right)^T$$

The gradient for a multivariable function is analogous to the derivative of a single-variable function in the sense that a differentiable multivariable function can have a relative minimum at $\boldsymbol{x}$ only when the gradient at $\boldsymbol{x}$ is the zero vector.

The gradient has another important propety connected with the minimization of multivariable functions. Suppose that $\boldsymbol{v} = (v_1, v_2, \ldots, v_n)^T$ is a unit vector in $\mathbb{R}^n$; that is,

$$||\boldsymbol{v}||_2^2 = \sum_{i=1}^{n} v_i^2 = 1$$

The directional derivative of g at $\boldsymbol{x}$ in the direction of $\boldsymbol{v}$ is defined by

$$D_v g(\boldsymbol{x}) = \lim_{h \to 0} \frac{1}{h}[g(\boldsymbol{x} + h\boldsymbol{v}) - g(\boldsymbol{x})] = \boldsymbol{v}^T \cdot \nabla g(\boldsymbol{x})$$

The directional derivative of g at $\boldsymbol{x}$ in the direction of $\boldsymbol{v}$ measures the change in the value of the function g relative to the change in the variable in the direction of $\boldsymbol{v}$.
A standard result from the calculus of multivariable functions states that, when g is differentiable, the direction that produces the maximum value for the directional derivative occurs when $\boldsymbol{v}$ is chosen to be parallel to $\nabla g(\boldsymbol{x})$, provided that $\nabla g(\boldsymbol{x}) \neq \boldsymbol{0}$. As a consequence, the direction of greatest decrease in the value of g at $\boldsymbol{x}$ is the direction given by $-\nabla g(\boldsymbol{x})$.
The objective is to reduce $g(\boldsymbol{x})$ to its minimal value of zero, so an appropriate choice for $\boldsymbol{x}^{(1)}$ is

$$\boldsymbol{x}^{(1)} = \boldsymbol{x}^{(0)} - \alpha \nabla g\left(\boldsymbol{x}^{(0)}\right), \quad \text{for some constant } \alpha > 0 \tag{7.69}$$

The problem now reduces to choosing α so that $g\left(\boldsymbol{x}^{(1)}\right)$ will be significantly less than $g\left(\boldsymbol{x}^{(0)}\right)$. To determine appropriate choice for the value α, we consider the single-variable function

$$h(\alpha) = g\left(\boldsymbol{x}^{(0)} - \alpha \nabla g\left(\boldsymbol{x}^{(0)}\right)\right) \tag{7.70}$$

the value of α that minimizes h is the value needed for Eq. (7.69).
Finding a minimal value for h directly would require differentiating h and then solving a root-finding problem to determine the critical points of h. This procedure is generally too costly. Instead, we choose three number $\alpha_1 < \alpha_2 < \alpha_3$ that, we hope, are close to where the minimum value of $h(\alpha)$ occurs. Then we construct the quadratic polynomial $P(x)$ that interpolates h at α_1, α_2, and α_3. We define $\hat{\alpha}$ in $[\alpha_1, \alpha_3]$ so that $P(\hat{\alpha})$ is a minimum in $[\alpha_1, \alpha_3]$ and use $P(\hat{\alpha})$ to approximate the minimal value of $h(\alpha)$. Then $\hat{\alpha}$ is used to determine the new iterate for approximating the minimal value of g:

$$\boldsymbol{x}^{(1)} = \boldsymbol{x}^{(0)} - \hat{\alpha} \nabla g\left(\boldsymbol{x}^{(0)}\right)$$

Since $g\left(\boldsymbol{x}^{(0)}\right)$ is available, we first choose $\alpha_1 = 0$ to minimize the computation. Next, a number α_3 is found with $h(\alpha_3) < h(\alpha_1)$. Since α_1 does not minimize h, such a number α_3 does exist. Finally, α_2 si chosen to be $\frac{\alpha_3}{2}$.
The minimum value of P on $[\alpha_1, \alpha_3]$ occurs either at the only critical point of P or at the right endpoint α_3 because, by assumption, $P(\alpha_3) = h(\alpha_3) < h(\alpha_1) = P(\alpha_1)$. The critical point is easily determined since P is a quadratic polynomial.

[SI*] There are many variations of the method of Steepest Descent, some of which involve more intricate methods for determining the value of α that will produce a minimum for the single-variable function h defined in Eq. (7.70). Other techniques use a multidimensional Taylor polynomial to replace the original multivariable function g and minimize the polynomial instead of g. Although there are advantage to some of these methods, all the Steepest Descent methods are, in general, linearly convergent and converge independent of the

starting approximation. In some instances, however, the methods may converge to something other than the absolute minimum of the function g.

Guide 7.3: Steepest Descent Algorithm

The Steepest Descent algorithm is a first-order iterative numerical method used to find a local minimum of a multivariable nonlinear objective function $f(\boldsymbol{x})$ where $\boldsymbol{x} \in \mathbb{R}^n$. It moves iteratively along the direction of the negative gradient, which represents the local direction of maximum decrease.
At any current point $\boldsymbol{x}^{(k)}$, the function decreases most rapidly in the direction of the negative gradient vector, $-\nabla f(\boldsymbol{x}^{(k)})$.
The iterative update formula is

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} - \alpha^{(k)} \nabla f(\boldsymbol{x}^{(k)})$$

with $\boldsymbol{x}^{(k)}$ as the current position vector (n-variables) at step k, $\nabla f(\boldsymbol{x}^{(k)})$ is the gradient vector of partial derivatives, and $\alpha^{(k)}$ is the step size (or learning rate) determined via line search to minimize $f(\boldsymbol{x}^{(k)} - \alpha^{(k)} \nabla f(\boldsymbol{x}^{(k)}))$.
The algorithm:

1. **Initialize:** Choose a starting point $\boldsymbol{x}^{(0)} \in \mathbb{R}^n$, a tolerance $\epsilon > 0$ for convergence, and set iteration counter $k = 0$.
2. **Compute Gradient:** Evaluate the gradient vector $\boldsymbol{g}^{(k)} = \nabla f(\boldsymbol{x}^{(k)})$.
3. **Check Stopping Criterion:** If $||\boldsymbol{g}^{(k)}|| < \epsilon$, the algorithm has converged to a stationary point; stop and output $\boldsymbol{x}^{(k)}$. Otherwise, continue.
4. **Determine Search Direction:** Set the descent direction $\boldsymbol{d}^{(k)} = -\boldsymbol{g}^{(k)}$.
5. **Find Step Size (Line Search):** Solve the one-dimensional optimization problem to find $\alpha^{(k)} > 0$ that minimizes $f(\boldsymbol{x}^{(k)} + \alpha^{(k)} \boldsymbol{d}^{(k)})$ or $f(\boldsymbol{x}^{(k)} - \alpha^{(k)} \boldsymbol{g}^{(k)})$ (using methods like backtracking, Armijo rule, or exact line search).
6. **Update Position:** Compute $\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} + \alpha^{(k)} \boldsymbol{d}^{(k)}$
7. **Increment:** Set $k = k + 1$ and return to Step 2.

Characteristics:

- Simple to implement, robust, requires low memory/storage since it only uses first-order derivatives.
- Exhibits linear convergence and tends to zig-zag (successive search directions are orthogonal $(\boldsymbol{d}^{(k)})^T \boldsymbol{d}^{(k+1)} = 0$) when navigating narrow, ill-conditioned valleys where the Hessian matrix has a high condition number.

iv. Homotopy and Continuation Method

[SI*] Homotopy, or continuation, methods for nonlinear systems embed the problem to be solved within a collection of problems. Specifically, to solve a problem of the form

$$\boldsymbol{F}(\boldsymbol{x}) = \boldsymbol{0}$$

which has the unknown solution $\boldsymbol{x}^*$, we consider a family of problems described using a parameter λ taht assumes values in $[0,1]$. A problem with a known solution $\boldsymbol{x}(0)$ corresponds to $\lambda = 0$, and the problem with the unknown solution $\boldsymbol{x}(1) \equiv \boldsymbol{x}^*$ corresponds to $\lambda = 1$.
For example, suppose $\boldsymbol{x}(0)$ is an initial approximation to the solution of $\boldsymbol{F}(\boldsymbol{x}^*) = \boldsymbol{0}$. Define

$$\boldsymbol{G} : [0,1] \times \mathbb{R}^n \to \mathbb{R}^n$$

by

$$\boldsymbol{G}(\lambda, \boldsymbol{x}) = \lambda \boldsymbol{F}(\boldsymbol{x}) + (1-\lambda)[\boldsymbol{F}(\boldsymbol{x}) - \boldsymbol{F}(\boldsymbol{x}(0))] = \boldsymbol{F}(\boldsymbol{x}) + (\lambda - 1)\boldsymbol{F}(\boldsymbol{x}(0)) \tag{7.71}$$

We will determine, for various values of λ, a solution to

$$\boldsymbol{G}(\lambda, \boldsymbol{x}) = \boldsymbol{0}$$

When $\lambda = 0$, this equation assumes the form

$$\boldsymbol{0} = \boldsymbol{G}(0, \boldsymbol{x}) = \boldsymbol{F}(\boldsymbol{x}) - \boldsymbol{F}(\boldsymbol{x}(0))$$

and $\boldsymbol{x}(0)$ is a solution. When $\lambda = 1$, the equation assumes the form

$$\boldsymbol{0} = \boldsymbol{G}(1, \boldsymbol{x}) = \boldsymbol{F}(\boldsymbol{x})$$

and $\boldsymbol{x}(1) = \boldsymbol{x}^*$ is a solution.
The function $\boldsymbol{G}$ with tha parameter λ, provides us with a family of functions that can lead from the known value $\boldsymbol{x}(0)$ to the solution $\boldsymbol{x}(1) = \boldsymbol{x}^*$. The function $\boldsymbol{G}$ is called a homotopy between the function $\boldsymbol{G}(0, \boldsymbol{x}) = \boldsymbol{F}(\boldsymbol{x}) - \boldsymbol{F}(\boldsymbol{x}(0))$ and the function $\boldsymbol{G}(1, \boldsymbol{x}) = \boldsymbol{F}(\boldsymbol{x})$.
The continuation problem is to:
Determine a way to proceed from the known solution $\boldsymbol{x}(0)$ of $\boldsymbol{G}(0, \boldsymbol{x})$ to the unknown solution $\boldsymbol{x}(1) = \boldsymbol{x}^*$ of $\boldsymbol{G}(1, \boldsymbol{x}) = \boldsymbol{0}$ that solves $\boldsymbol{F}(\boldsymbol{x}) = \boldsymbol{0}$.
We first assume that $\boldsymbol{x}(\lambda)$ is the unique solution to the equation

$$\boldsymbol{G}(\lambda, \boldsymbol{x}) = \boldsymbol{0} \tag{7.72}$$

For each $\lambda \in [0,1]$. The set $\{\boldsymbol{x}(\lambda) | 0 \leq \lambda \leq 1\}$ can be viewed as a curve in $\mathbb{R}^n$ from $\boldsymbol{x}(0)$ to $\boldsymbol{x}(1) = \boldsymbol{x}^*$ parameterized by λ. A continuation method that finds a sequence of steps along this curve corresponding to $\{\boldsymbol{x}(\lambda_k)\}_{k=0}^m$, where $\lambda_0 = 0 < \lambda_1 < \cdots < \lambda_m = 1$.
If the functions $\lambda \to \boldsymbol{x}(\lambda)$ and $\boldsymbol{G}$ are differentiable, then differentiating Eq. (7.72) with respect to λ gives

$$\boldsymbol{0} = \frac{\partial \boldsymbol{G}(\lambda, \boldsymbol{x}(\lambda))}{\partial \lambda} + \frac{\partial \boldsymbol{G}(\lambda, \boldsymbol{x}(\lambda))}{\partial \boldsymbol{x}} \boldsymbol{x}'(\lambda)$$

and solving for $\boldsymbol{x}'(\lambda)$ gives

$$\boldsymbol{x}'(\lambda) = -\left[\frac{\partial \boldsymbol{G}(\lambda, \boldsymbol{x}(\lambda))}{\partial \boldsymbol{x}}\right]^{-1} \frac{\partial \boldsymbol{G}(\lambda, \boldsymbol{x}(\lambda))}{\partial \lambda}$$

This is a system of differential equations with the initial condition $\boldsymbol{x}(0)$.
Since

$$\boldsymbol{G}(\lambda, \boldsymbol{x}(\lambda)) = \boldsymbol{F}(\boldsymbol{x}(\lambda)) + (\lambda - 1)\boldsymbol{F}(\boldsymbol{x}(0))$$

we can determine both

$$\frac{\partial \boldsymbol{G}}{\partial \boldsymbol{x}}(\lambda, \boldsymbol{x}(\lambda)) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1}(\boldsymbol{x}(\lambda)) & \frac{\partial f_1}{\partial x_2}(\boldsymbol{x}(\lambda)) & \dots & \frac{\partial f_1}{\partial x_n}(\boldsymbol{x}(\lambda)) \\ \frac{\partial f_2}{\partial x_1}(\boldsymbol{x}(\lambda)) & \frac{\partial f_2}{\partial x_2}(\boldsymbol{x}(\lambda)) & \dots & \frac{\partial f_2}{\partial x_n}(\boldsymbol{x}(\lambda)) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1}(\boldsymbol{x}(\lambda)) & \frac{\partial f_n}{\partial x_2}(\boldsymbol{x}(\lambda)) & \dots & \frac{\partial f_n}{\partial x_n}(\boldsymbol{x}(\lambda)) \end{bmatrix} = J(\boldsymbol{x}(\lambda))$$

the Jacobian matrix, and

$$\frac{\partial \boldsymbol{G}(\lambda, \boldsymbol{x}(\lambda))}{\partial \lambda} = \boldsymbol{F}(\boldsymbol{x}(0))$$

Therefore, the system of differential equations becomes

$$\boldsymbol{x}'(\lambda) = -\left[J(\boldsymbol{x}(\lambda))\right]^{-1} \boldsymbol{F}(\boldsymbol{x}(0)), \quad \text{for } 0 \leq \lambda \leq 1 \tag{7.73}$$

with the initial condition $\boldsymbol{x}(0)$.

Theorem 7.14: Conditions for the Continuation Method

Let $\boldsymbol{F}(\boldsymbol{x})$ be continuously differentiable for $\boldsymbol{x} \in \mathbb{R}^n$. Suppose that the Jacobian matrix $J(\boldsymbol{x})$ is nonsingular for all $\boldsymbol{x} \in \mathbb{R}^n$ and that a constant M exists with $||J(\boldsymbol{x})^{-1}|| \leq M$, for all $\boldsymbol{x} \in \mathbb{R}^n$. Then, for any $\boldsymbol{x}(0)$ in $\mathbb{R}^n$, there exists a unique solution $\boldsymbol{x}(\lambda)$, such that

$$\boldsymbol{G}(\lambda, \boldsymbol{x}(\lambda)) = \boldsymbol{0}$$

for all λ in $[0, 1]$. Moreover, $\boldsymbol{x}(\lambda)$ is continuously differentiable and

$$\boldsymbol{x}'(\lambda) = -J(\boldsymbol{x}(\lambda))^{-1}\boldsymbol{F}(\boldsymbol{x}(0)), \quad \text{for each } \lambda \in [0, 1]$$

[SI*] Consider the nonlinear system

$$\begin{aligned} f_1(x_1, x_2, x_3) &= 0 \\ f_2(x_1, x_2, x_3) &= 0 \\ &\vdots \\ f_n(x_1, x_2, x_3) &= 0 \end{aligned}$$

Then we can compute the Jacobian matrix $J(\boldsymbol{x})$, and let $\boldsymbol{x}(0) = (0, 0, \ldots, 0)^T$ to compute $\boldsymbol{F}(\boldsymbol{x}(0))$.
The system of differential equation is

$$\begin{bmatrix} x_1'(\lambda) \\ x_2'(\lambda) \\ \vdots \\ x_n'(\lambda) \end{bmatrix} = J(\boldsymbol{x})^{-1}\boldsymbol{F}(\boldsymbol{x}(0))$$

In general, the system of differential equations that we need to solve for our continuation problem has the form

$$\begin{aligned} \frac{dx_1}{d\lambda} &= \phi_1(\lambda, x_1, x_2, \ldots, x_n) \\ \frac{dx_2}{d\lambda} &= \phi_2(\lambda, x_1, x_2, \ldots, x_n) \\ &\vdots \\ \frac{dx_n}{d\lambda} &= \phi_n(\lambda, x_1, x_2, \ldots, x_n) \end{aligned}$$

where

$$\begin{bmatrix} \phi_1(\lambda, x_1, x_2, \ldots, x_n) \\ \phi_2(\lambda, x_1, x_2, \ldots, x_n) \\ \vdots \\ \phi_n(\lambda, x_1, x_2, \ldots, x_n) \end{bmatrix} = -J(x_1, x_2, \ldots, x_n)^{-1} \begin{bmatrix} f_1(\boldsymbol{x}(0)) \\ f_2(\boldsymbol{x}(0)) \\ \vdots \\ f_n(\boldsymbol{x}(0)) \end{bmatrix} \tag{7.74}$$

To use the Runge-Kutta method of order four to solve this system, we first choose an integer $N > 0$ and let $h = \frac{1-0}{N}$. Partition the interval $[0, 1]$ into N subintervals with the mesh points

$$\lambda_j = jh, \quad \text{for each } j = 0, 1, \ldots, N$$

We use the notation w_{ij}, for each $j = 0, 1, \ldots, N$ and $i = 1, 2, \ldots, n$, to denote an approximation to $x_i(\lambda_j)$. For the initial conditions, set

$$\begin{aligned} w_{1,0} &= x_1(0) \\ w_{2,0} &= x_2(0) \\ &\vdots \\ w_{n,0} &= x_n(0) \end{aligned}$$

Suppose $w_{1,j}, w_{2,j}, \ldots w_{n,j}$ have been computed. We obtain $w_{1,j+1}, w_{2,j+1}, \ldots w_{n,j+1}$ using the equations

$$\begin{aligned} k_{1,i} &= h\phi_i(\lambda_j, w_{1,j}, w_{2,j}, \ldots, w_{n,j}) \\ k_{2,i} &= h\phi_i\left(\lambda_j + \frac{h}{2}, w_{1,j} + \frac{1}{2}k_{1,1}, w_{2,j} + \frac{1}{2}k_{1,2}, \ldots, w_{n,j} + \frac{1}{2}k_{1,n}\right) \\ k_{3,i} &= h\phi_i\left(\lambda_j + \frac{h}{2}, w_{1,j} + \frac{1}{2}k_{2,1}, w_{2,j} + \frac{1}{2}k_{2,2}, \ldots, w_{n,j} + \frac{1}{2}k_{2,n}\right) \\ k_{4,i} &= h\phi_i\left(\lambda_j + h, w_{1,j} + k_{3,1}, w_{2,j} + k_{3,2}, \ldots, w_{n,j} + k_{3,n}\right) \end{aligned}$$

for each $i = 1, 2, \ldots, n$.
Finally,

$$w_{i,j+1} = w_{i,j} + \frac{1}{6}(k_{1,i} + 2k_{2,i} + 2k_{3,i} + k_{4,i})$$

for each $i = 1, 2, \ldots, n$.
We use the vector notation

$$\boldsymbol{k}_1 = \begin{bmatrix} k_{1,1} \\ k_{1,2} \\ \vdots \\ k_{1,n} \end{bmatrix}, \ \boldsymbol{k}_2 = \begin{bmatrix} k_{2,1} \\ k_{2,2} \\ \vdots \\ k_{2,n} \end{bmatrix}, \ \boldsymbol{k}_3 = \begin{bmatrix} k_{3,1} \\ k_{3,2} \\ \vdots \\ k_{3,n} \end{bmatrix}, \ \boldsymbol{k}_4 = \begin{bmatrix} k_{4,1} \\ k_{4,2} \\ \vdots \\ k_{4,n} \end{bmatrix}$$

and

$$\boldsymbol{w}_j = \begin{bmatrix} w_{1,j} \\ w_{2,j} \\ \vdots \\ w_{n,j} \end{bmatrix}$$

Eq. (7.74) gives us $\boldsymbol{x}(0) = \boldsymbol{x}(\lambda_0) = \boldsymbol{w}_0$, and for each $j = 0, 2, \ldots, N$,

$$\begin{aligned}
\boldsymbol{k}_1 &= h \begin{bmatrix} \phi_1(\lambda_j, w_{1,j}, \ldots, w_{n,j}) \\ \phi_2(\lambda_j, w_{1,j}, \ldots, w_{n,j}) \\ \vdots \\ \phi_n(\lambda_j, w_{1,j}, \ldots, w_{n,j}) \end{bmatrix} \\
&= h \left[-J(w_{1,j}, w_{2,j}, \ldots, w_{n,j}) \right]^{-1} \boldsymbol{F}(\boldsymbol{x}(0)) \\
&= h \left[-J(\boldsymbol{w}_j) \right]^{-1} \boldsymbol{F}(\boldsymbol{x}(0)) \\
\boldsymbol{k}_2 &= h \left[-J \left(\boldsymbol{w}_j + \frac{1}{2}\boldsymbol{k}_1 \right) \right]^{-1} \boldsymbol{F}(\boldsymbol{x}(0)) \\
\boldsymbol{k}_3 &= h \left[-J \left(\boldsymbol{w}_j + \frac{1}{2}\boldsymbol{k}_1 \right) \right]^{-1} \boldsymbol{F}(\boldsymbol{x}(0)) \\
\boldsymbol{k}_4 &= h \left[-J \left(\boldsymbol{w}_j + \boldsymbol{k}_3 \right) \right]^{-1} \boldsymbol{F}(\boldsymbol{x}(0))
\end{aligned}$$

and

$$\begin{aligned}
\boldsymbol{x}(\lambda_{j+1}) &= \boldsymbol{x}(\lambda_j) + \frac{1}{6}(\boldsymbol{k}_1 + 2\boldsymbol{k}_2 + 2\boldsymbol{k}_3 + \boldsymbol{k}_4) \\
&= \boldsymbol{w}_j + \frac{1}{6}(\boldsymbol{k}_1 + 2\boldsymbol{k}_2 + 2\boldsymbol{k}_3 + \boldsymbol{k}_4)
\end{aligned}$$

Finally, $\boldsymbol{x}(\lambda_n) = \boldsymbol{x}(1)$ is our approximation to $\boldsymbol{x}^*$.

v. Particle Swarm Optimization

[SI*] Swarm intelligence refers to a kind of problem-solving ability that emerges in the interactions of simple information-processing units. The concept of a swarm suggests multiplicity, stochasticity, randomness, and messiness, and the concept of intelligence suggests that the problem-solving method is somehow successful. The information-processing units that compose a swarm can be animate, mechanical, computational, or mathematical; they can be insects, birds, or human beings; they can be array elements, robots, or standalone workstations; they can be real or imaginary.

Particle swarm optimization is a stochastic, population-based optimization metaheuristic inspired by the social behavior of bird flocking or fish schooling. The system coordinates multiple candidate solutions (called particles) that traverse a multi-dimensional search space to locate global minima. It is based on a certain insight regarding human behavior and cognition. The not-very-technical observation enables us to design a family of computer algorithms that encode some population of individuals who propose solutions to a problem, and then are able to refine those solutions by interacting with their "peers," picking up suggestions from their neighbors, and adjusting their own patterns of variables. This technique has been used for binary problems, multiobjective problems, dynamic problems that keep changing, and many other tough kinds of problems. It has been used for a variety of engineering problems, from the maintenance of electrical grids to the classification of physiological variables in early diagnosis of disease.

[SI*] Particle Swarm Optimization (PSO) is a computer algorithm that finds the best solution to a problem by letting a group of candidate solutions, called particles, move around a search space based on simple mathematical rules inspired by bird flocking.

In computational science, it is a computational method that optimizes a problem by iteratively trying to improve a population of candidate solutions with regard to a given measure of quality. It solves a problem through interactions among a population of candidate solutions, dubbed particles, moving the particles around in the search-space according to simple mathematical formula that adjust each particle's position and velocity. Each particle's movement is influenced by its own best known position so far, and by the ebst known position in its topological neighborhood (which may include the entire population if so specified); vectors are updated as better positions are found. This is expected to move the swarm toward good solutions.
PSO is a metaheuristic as it makes few or no assumptions about the problem being optimized and can search very large spaces of candidate solutions. Also, PSO does not use the gradient of the problem being optimized, which means PSO does not require that the optimization problem be differentiable as is required by classic optimization methods such as gradient descent and Quasi-Newton methods. However, metaheuristics such as PSO do not guarantee an optimal solution is ever found.

[SI*] How PSO works

- **Initialization:** A swarm of random particles is spread across a problem's search space. Each particle has a random position and speed (velocity)
- **Evaluation:** An objective function measures how good each particle's position is (its fitness value).
- **Tracking Bests:** Each particle remembers the best position it has ever found, and the entire swarm knows the best overall position found by any particle.
- **Updating Motion:** Particles update their velocity and move closer to a mix of their own best spot and the swarm's best spot, plus a bit of randomness to explore new areas.

Key parameters:

- **Inertia Weight (w):** Keeps the particle moving in its current direction.
- **Cognitive Coefficient (c_1):** Pulls the particle back toward its own personal best position.
- **Social Coefficient (c_2):** Pulls the particle toward the global best position found by the swarm.

[SI*] Genetic Algorithms (GA) and Particle Swarm Optimization (PSO) are used to explore the entire domain and identify approximate root locations, it is called the global search phase. Afterwards, we go to the local refinement phase, where we use the approximate locations from the global search as initial guesses for efficient local methods (like derivative-free iterative methods or Steffensen-type methods) to achieve high accuracy.

[SI*] When using Particle Swarm Optimization (PSO) for multi-root finding (locating multiple roots of a single function $f(x) = 0$ or a nonlinear system of equations simultaneously), a standard PSO must be modified. A traditional PSO converges onto a single global minimum, but multi-root problems require niching (sub-swarming) techniques or fitness landscape modifications to capture multiple roots across the search space.

1. **The Core PSO Velocity and Position Formulas**
   In any PSO variant, the movement of each particle i in a D-dimensional space is governed by two main updates:

$$\boldsymbol{v}_i^{(t+1)} = w\boldsymbol{v}_i^{(t)} + c_1 r_1(\boldsymbol{p}_i^{(t)} - \boldsymbol{x}_i^{(t)}) + c_2 r_2(\boldsymbol{g}_i^{(t)} - \boldsymbol{x}_i^{(t)})$$
$$\boldsymbol{x}_i^{(t+1)} = \boldsymbol{x}_i^{(t)} + \boldsymbol{v}_i^{(t+1)}$$

   with:

- $\boldsymbol{x}_i^{(t)}$ as the current position vector of particle i at iteration t.
- $\boldsymbol{v}_i^{(t)}$ as the current velocity vector of particle i at iteration t.
- $\boldsymbol{p}_i^{(t)}$ as the personal best ($\boldsymbol{p}_{best}$) position discovered by particle i at iteration t.
- $\boldsymbol{g}_i^{(t)}$ as the neighborhood best position found within particle i's niche/ring group or a local topology leader to prevent all particles from collapsing onto one single root.
- w as the inertia weight (dictates how much of the previous velocity is kept).
- c_1 and c_2 are the cognitive and social acceleration coefficients.
- r_1, r_2 are random matrices/vectors uniformly distributed in $[0, 1]$.

2. **Multi-Root Objective (Fitness) Function Transformation**
To find roots, the problem is framed as a minimization task where the target value is exactly 0. For a system of nonlinear equations

$$\boldsymbol{F}(\boldsymbol{x}) = [f_1(\boldsymbol{x}), f_2(\boldsymbol{x}), \ldots, f_m(\boldsymbol{x})]^T = \boldsymbol{0}$$

the basic fitness function $F(\boldsymbol{x})$ is defined using the Mean Squared Error (MSE):

$$F(\boldsymbol{x}) = \sum_{j=1}^{m} [f_j(\boldsymbol{x})]^2$$

or the absolute sum

$$F(\boldsymbol{x}) = \sum_{j=1}^{m} |f_j(\boldsymbol{x})|$$

A perfect root yields $F(\boldsymbol{x}) = 0$.

3. **Adapting the Formula for Multiple Roots (Niching Methods)**
Because a standard PSO will converge onto just one root, multi-root finders replace or modify the standard guidance framework using one of the following methods:

   (a) **Ring Topology / Local Neighborhood PSO (LPSO)**
   Instead of communicating globally, particles are arranged structually in a ring network. The velocity gormula swaps the absolute $\boldsymbol{g}_i^{(t)}$ with the neighborhood best ($\boldsymbol{n}_i^{(t)}$), calculated among its immediate topological neighbors (e.g., particles $i-1, i$, and $i+1$):

$$\boldsymbol{v}_i^{(t+1)} = w\boldsymbol{v}_i^{(t)} + c_1 r_1(\boldsymbol{p}_i^{(t)} - \boldsymbol{x}_i^{(t)}) + c_2 r_2(\boldsymbol{n}_i^{(t)} - \boldsymbol{x}_i^{(t)})$$

   (b) **Deflation / Repulsion Method (Sequential Root Finding)**
   If you want to find roots one after another, you can alter the fitness function dynamically once a root is found. When a root $\boldsymbol{r}^{(k)}$ is isolated, the objective function is deflated by adding a penalty barrier around it. The modified fitness function becomes:

$$F_{new}(\boldsymbol{x}) = F(\boldsymbol{x}) \cdot \prod_{k=1}^{R} \left(1 + \frac{\lambda}{||\boldsymbol{x} - \boldsymbol{r}^{(k)}||^2}\right)$$

   with $\boldsymbol{r}^{(k)}$ as the k-th previously discovered root, λ is the scaling penalty parameter, $||\boldsymbol{x} - \boldsymbol{r}^{(k)}||$ is the Euclidean distance between the particle and the known root. When a particle gets close to an already discovered root, the denominator goes to 0, driving $F_{new}(\boldsymbol{x})$ to infinity, forcing the swarm to fly away and locate undiscovered roots.

(c) **NichePSO (Dynamic Multi-Swarming)**
Algorithms like NichePSO dynamically split the main swarm into distinct sub-swarms when a region shows potential. A sub-swarm S_k focuses strictly on optimizing its own localized landscape using its local sub-swarm best $\boldsymbol{g}_k^{(t)}$:

$$\boldsymbol{v}_i^{(t+1)} = w\boldsymbol{v}_i^{(t)} + c_1 r_1(\boldsymbol{p}_i^{(t)} - \boldsymbol{x}_i^{(t)}) + c_2 r_2(\boldsymbol{g}_k^{(t)} - \boldsymbol{x}_i^{(t)}), \quad \text{for } i \in S_k$$

vi. Compute the Solution of a Nonlinear System with Newton's Method

Compute the solution of a system of nonlinear equations given by

$$\begin{aligned} 3x_1 - \cos(x_2x_3) - \frac{1}{2} &= 0 \\ x_1^2 - 81(x_2 + 0.1)^2 + \sin(x_3) + 1.06 &= 0 \\ e^{-x_1x_2} + 20x_3 + \frac{10\pi - 3}{3} &= 0 \end{aligned}$$

Solution:

We will use Newton's method to obtain the approximation of the solution for the nonlinear system with the initial approximation

$$\boldsymbol{x}^{(0)} = (0.1, 0.1, -0.1)^T$$

and

$$\boldsymbol{F}(x_1, x_2, x_3) = (f_1(x_1, x_2, x_3), f_2(x_1, x_2, x_3), f_3(x_1, x_2, x_3))$$

where

$$\begin{aligned} f_1(x_1, x_2, x_3) &= 3x_1 - \cos(x_2x_3) - \frac{1}{2} \\ f_2(x_1, x_2, x_3) &= x_1^2 - 81(x_2 + 0.1)^2 + \sin(x_3) + 1.06 \\ f_3(x_1, x_2, x_3) &= e^{-x_1x_2} + 20x_3 + \frac{10\pi - 3}{3} \end{aligned}$$

The Jacobian matrix $J(\boldsymbol{x})$ for this system is

$$J(x_1, x_2, x_3) = \begin{bmatrix} 3 & x_3\sin(x_2x_3) & x_2\sin(x_2x_3) \\ 2x_1 & -162(x_2 + 0.1) & \cos(x_3) \\ -x_2e^{-x_1x_2} & -x_1e^{-x_1x_2} & 20 \end{bmatrix}$$

and

$$\begin{bmatrix} x_1^{(k)} \\ x_2^{(k)} \\ x_3^{(k)} \end{bmatrix} = \begin{bmatrix} x_1^{(k-1)} \\ x_2^{(k-1)} \\ x_3^{(k-1)} \end{bmatrix} + \begin{bmatrix} y_1^{(k-1)} \\ y_2^{(k-1)} \\ y_3^{(k-1)} \end{bmatrix}$$

where

$$\begin{bmatrix} y_1^{(k-1)} \\ y_2^{(k-1)} \\ y_3^{(k-1)} \end{bmatrix} = -\left(J\left(x_1^{(k-1)}, x_2^{(k-1)}, x_3^{(k-1)}\right)\right)^{-1} \boldsymbol{F}\left(x_1^{(k-1)}, x_2^{(k-1)}, x_3^{(k-1)}\right)$$

Thus, at the kth step, the linear system $J(\boldsymbol{x}^{(k-1)})\boldsymbol{y}^{(k-1)} = -\boldsymbol{F}(\boldsymbol{x}^{(k-1)})$ must be solved, where

$$J(\boldsymbol{x}^{(k-1)}) = \begin{bmatrix} 3 & x_3^{(k-1)}\sin(x_2^{(k-1)}x_3^{(k-1)}) & x_2^{(k-1)}\sin(x_2^{(k-1)}x_3^{(k-1)}) \\ 2x_1^{(k-1)} & -162(x_2^{(k-1)} + 0.1) & \cos(x_3^{(k-1)}) \\ -x_2^{(k-1)}e^{-x_1^{(k-1)}x_2^{(k-1)}} & -x_1^{(k-1)}e^{-x_1^{(k-1)}x_2^{(k-1)}} & 20 \end{bmatrix}$$

$$\boldsymbol{y}^{(k-1)} = \begin{bmatrix} y_1^{(k-1)} \\ y_2^{(k-1)} \\ y_3^{(k-1)} \end{bmatrix}$$

and

$$\boldsymbol{F}(\boldsymbol{x}^{(k-1)}) = \begin{bmatrix} 3x_1^{(k-1)} - \cos(x_2^{(k-1)} x_3^{(k-1)}) - \frac{1}{2} \\ (x_1^{(k-1)})^2 - 81(x_2^{(k-1)} + 0.1)^2 + \sin(x_3^{(k-1)}) + 1.06 \\ e^{-x_1^{(k-1)} x_2^{(k-1)}} + 20x_3^{(k-1)} + \frac{10\pi - 3}{3} \end{bmatrix}$$

In SymIntegration we have created a class that is able to solve nonlinear system with Newton's method, it is **NonlinearNewtonSolver**, in this class we are using **std::function** and **Dual** class that can help us to compute the Jacobian matrix.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Solutions of Nonlinear Systems of Equations with Newton's Metho
Initial Guess:
x1 = (0.1,0)
x2 = (0.1,0)
x3 = (-0.1,0)

=======================================================
  Newton' Method Multivariable Complex Root Solver
=======================================================
Iter 0 | Residual Norm: 8.8429574631 | Update Norm: 0.5865718917
Iter 1 | Residual Norm: 0.3458544333 | Update Norm: 0.0179944843
Iter 2 | Residual Norm: 0.0258894796 | Update Norm: 0.0015767007
Iter 3 | Residual Norm: 0.0002012114 | Update Norm: 0.0000124474
Iter 4 | Residual Norm: 0.0000000125 | Update Norm: 0.0000000008

[Success] Converged in 5 iterations.

Computed Roots:
-------------------------------------------------------------
x1 = 0.5000000000
x2 = -0.0000000000
x3 = -0.5235987756
-------------------------------------------------------------

Time taken by function: 1351 microseconds
```

Figure 7.40: *The approximated solutions after* 5 *iterations with Newton's method (**SymIntegration/Examples/Numerical Methods/Test SymIntegration Solutions of Nonlinear Systems of Equations with Newton's Method/main.cpp**).*

The Newton's method can converge very rapidly once an approximation is obtained that is near the true solution. However, it is not always easy to determine starting values that will lead to a solution, and the method is comparatively expensive to employ (computing the Jacobian matrix).

vii. Compute the Solution of a Nonlinear System with Quasi-Newton

Compute the solution of a system of nonlinear equations given by

$$\begin{aligned} 3x_1 - \cos(x_2x_3) - \frac{1}{2} &= 0 \\ x_1^2 - 81(x_2 + 0.1)^2 + \sin(x_3) + 1.06 &= 0 \\ e^{-x_1x_2} + 20x_3 + \frac{10\pi - 3}{3} &= 0 \end{aligned}$$

Solution:

We will use Quasi-Newton method to obtain the approximation of the solution for the nonlinear system with the initial approximation

$$\boldsymbol{x}^{(0)} = (0.1, 0.1, -0.1)^T$$

and

$$\boldsymbol{F}(x_1, x_2, x_3) = (f_1(x_1, x_2, x_3), f_2(x_1, x_2, x_3), f_3(x_1, x_2, x_3))$$

where

$$\begin{aligned} f_1(x_1, x_2, x_3) &= 3x_1 - \cos(x_2x_3) - \frac{1}{2} \\ f_2(x_1, x_2, x_3) &= x_1^2 - 81(x_2 + 0.1)^2 + \sin(x_3) + 1.06 \\ f_3(x_1, x_2, x_3) &= e^{-x_1x_2} + 20x_3 + \frac{10\pi - 3}{3} \end{aligned}$$

The Jacobian matrix $J(\boldsymbol{x})$ for this system is

$$J(x_1, x_2, x_3) = \begin{bmatrix} 3 & x_3\sin(x_2x_3) & x_2\sin(x_2x_3) \\ 2x_1 & -162(x_2 + 0.1) & \cos(x_3) \\ -x_2e^{-x_1x_2} & -x_1e^{-x_1x_2} & 20 \end{bmatrix}$$

Then

$$\boldsymbol{F}\left(\boldsymbol{x}^{(0)}\right) = \begin{bmatrix} -1.199950 \\ -2.269833 \\ 8.462025 \end{bmatrix}$$

and since

$$\begin{aligned} A_0 &= J(x_1^{(0)}, x_2^{(0)}, x_3^{(0)}) \\ &= \begin{bmatrix} 3 & 9.999833 \times 10^{-4} & -9.999833 \times 10^{-4} \\ 0.2 & -32.4 & 0.9950042 \\ -9.900498 \times 10^{-2} & -9.900498 \times 10^{-2} & 20 \end{bmatrix} \end{aligned}$$

we have

$$\begin{aligned} A_0^{-1} &= J(x_1^{(0)}, x_2^{(0)}, x_3^{(0)})^{-1} \\ &= \begin{bmatrix} 0.3333332 & 1.023852 \times 10^{-5} & 1.615701 \times 10^{-5} \\ 2.108607 \times 10^{-3} & -3.086883 \times 10^{-2} & 1.535836 \times 10^{-3} \\ 1.660520 \times 10^{-3} & -1.527577 \times 10^{-4} & 5.000768 \times 10^{-2} \end{bmatrix} \end{aligned}$$

So

$$\boldsymbol{x}^{(1)} = \boldsymbol{x}^{(0)} - A_0^{-1}\boldsymbol{F}\left(\boldsymbol{x}^{(0)}\right) = \begin{bmatrix} 0.4998697 \\ 1.946685 \times 10^{-2} \\ -0.5215205 \end{bmatrix}$$

$$\boldsymbol{F}\left(\boldsymbol{x}^{(1)}\right) = \begin{bmatrix} -3.394465 \times 10^{-4} \\ -0.3443879 \\ 3.188238 \times 10^{-2} \end{bmatrix}$$

$$\boldsymbol{F}\left(\boldsymbol{x}^{(1)}\right) = \begin{bmatrix} -3.394465 \times 10^{-4} \\ -0.3443879 \\ 3.188238 \times 10^{-2} \end{bmatrix}$$

$$\boldsymbol{y}_1 = \boldsymbol{F}\left(\boldsymbol{x}^{(1)}\right) - \boldsymbol{F}\left(\boldsymbol{x}^{(0)}\right) = \begin{bmatrix} 1.199611 \\ 1.925445 \\ -8.430143 \end{bmatrix}$$

$$\boldsymbol{s}_1 = \begin{bmatrix} 0.3998697 \\ -8.053315 \times 10^{-2} \\ -0.4215204 \end{bmatrix}$$

$$\boldsymbol{s}_1^T A_0^{-1} \boldsymbol{y}_1 = 0.3424604$$

$$\begin{aligned} A_1^{-1} &= A_0^{-1} + \left(\frac{1}{0.3424604}\right)[(\boldsymbol{s}_1 - A_0^{-1}\boldsymbol{y}_1)\boldsymbol{s}_1^T A_0^{-1}] \\ &= \begin{bmatrix} 0.3333781 & 1.11050 \times 10^{-5} & 8.967344 \times 10^{-6} \\ -2.021270 \times 10^{-3} & -3.094849 \times 10^{-2} & 2.196906 \times 10^{-3} \\ 1.022214 \times 10^{-3} & -1.650709 \times 10^{-4} & 5.010986 \times 10^{-2} \end{bmatrix} \end{aligned}$$

$$\boldsymbol{x}^{(2)} = \boldsymbol{x}^{(1)} - A_1\boldsymbol{F}(\boldsymbol{x}^{(1)}) = \begin{bmatrix} 0.4999863 \\ 8.737833 \times 10^{-3} \\ -0.5231746 \end{bmatrix}$$

In SymIntegration we have created a class that is able to solve nonlinear system with Newton's method, it is **QuasiNewtonSolver**, in this class we are using the **Dual** class that can help us to compute the Jacobian matrix.

```
Initial Guess coordinates:
x1 = (0.1,0)
x2 = (0.1,0)
x3 = (-0.1,0)


============================================================
  Quasi-Newton Multivariable Complex Root Solver
============================================================
Iter 0 | Residual Norm: 8.8429574631 | Update Norm: 0.3440665841
Iter 1 | Residual Norm: 0.3458544333 | Update Norm: 0.0001178553
Iter 2 | Residual Norm: 0.1474442176 | Update Norm: 0.0000621104
Iter 3 | Residual Norm: 0.0140805722 | Update Norm: 0.0000006858
Iter 4 | Residual Norm: 0.0006378613 | Update Norm: 0.0000000015
Iter 5 | Residual Norm: 0.0000031178 | Update Norm: 0.0000000000

Converged successfully in 6 iterations.

Computed Roots:
------------------------------------------------------------
x1 = 0.5000000000
x2 = 0.0000000000
x3 = -0.5235987756
------------------------------------------------------------

Time taken by function: 1501 microseconds
```

Figure 7.41: *The approximated solutions after* 6 *iterations with Quasi Newton method (**SymIntegration/Examples/Numerical Methods/Test SymIntegration Solutions of Nonlinear Systems of Equations with Quasi Newton/main.cpp**).*

viii. Compute the Solution of a Nonlinear System with Steepest Descent

Compute the solution of a system of nonlinear equations given by

$$\begin{aligned} 3x_1 - \cos(x_2 x_3) - \frac{1}{2} &= 0 \\ x_1^2 - 81(x_2 + 0.1)^2 + \sin(x_3) + 1.06 &= 0 \\ e^{-x_1 x_2} + 20x_3 + \frac{10\pi - 3}{3} &= 0 \end{aligned}$$

Solution:

To find a reasonable starting approximation to the solution of the nonlinear system

$$\begin{aligned} f_1(x_1, x_2, x_3) &= 3x_1 - \cos(x_2 x_3) - \frac{1}{2} = 0 \\ f_2(x_1, x_2, x_3) &= x_1^2 - 81(x_2 + 0.1)^2 + \sin(x_3) + 1.06 = 0 \\ f_3(x_1, x_2, x_3) &= e^{-x_1 x_2} + 20x_3 + \frac{10\pi - 3}{3} = 0 \end{aligned}$$

we use the Steepest Descent method with $\boldsymbol{x}^{(0)} = (0, 0, 0)^T$.
Let $g(x_1, x_2, x_3) = [f_1(x_1, x_2, x_3)]^2 + [f_2(x_1, x_2, x_3)]^2 + [f_3(x_1, x_2, x_3)]^2$. Then

$$\begin{aligned} \nabla g(x_1, x_2, x_3) \equiv \nabla g(\boldsymbol{x}) = \big(&2f_1(\boldsymbol{x})\frac{\partial f_1}{\partial x_1}(\boldsymbol{x}) + 2f_2(\boldsymbol{x})\frac{\partial f_2}{\partial x_1}(\boldsymbol{x}) + 2f_3(\boldsymbol{x})\frac{\partial f_3}{\partial x_1}(\boldsymbol{x}), \\ &2f_1(\boldsymbol{x})\frac{\partial f_1}{\partial x_2}(\boldsymbol{x}) + 2f_2(\boldsymbol{x})\frac{\partial f_2}{\partial x_{12}}(\boldsymbol{x}) + 2f_3(\boldsymbol{x})\frac{\partial f_3}{\partial x_2}(\boldsymbol{x}), \\ &2f_1(\boldsymbol{x})\frac{\partial f_1}{\partial x_3}(\boldsymbol{x}) + 2f_2(\boldsymbol{x})\frac{\partial f_2}{\partial x_3}(\boldsymbol{x}) + 2f_3(\boldsymbol{x})\frac{\partial f_3}{\partial x_3}(\boldsymbol{x})\big) \\ &= 2\boldsymbol{J}(\boldsymbol{x})^T \boldsymbol{F}(\boldsymbol{x}) \end{aligned}$$

For $\boldsymbol{x}^{(0)} = (0, 0, 0)^T$, we have

$$g\left(\boldsymbol{x}^{(0)}\right) = 111.975 \quad \text{and} \quad z_0 = ||\nabla g\left(\boldsymbol{x}^{(0)}\right)||_2 = 419.554$$

Let

$$\boldsymbol{z} = \frac{1}{z_0}\nabla g\left(\boldsymbol{x}^{(0)}\right) = (-0.0214514, -0.0193062, 0.999583)^T$$

With $\alpha_1 = 0$, we have $g_1 = g\left(\boldsymbol{x}^{(0)} - \alpha_1 \boldsymbol{z}\right) = g\left(\boldsymbol{x}^{(0)}\right) = 111.975$. We arbitrarily let $\alpha_3 = 1$ so that

$$g_3 = g\left(\boldsymbol{x}^{(0)} - \alpha_3 \boldsymbol{z}\right) = 93.5649$$

Since $g_3 < g_1$, we accept α_3 and set $\alpha_2 = 0.5$. Thus,

$$g_2 = g\left(\boldsymbol{x}^{(0)} - \alpha_2 \boldsymbol{z}\right) = 2.53557$$

We now form the Newton's forward divided-difference interpolating polynomial

$$P(\alpha) = g_1 + h_1\alpha + h_3\alpha(\alpha - \alpha_2)$$

that interpolates

$$g\left(\boldsymbol{x}^{(0)} - \alpha \nabla g\left(\boldsymbol{x}^{(0)}\right)\right) = g\left(\boldsymbol{x}^{(0)} - \alpha \boldsymbol{z}\right)$$

at $\alpha_1 = 0, \alpha_2 = 0.5$, and $\alpha_3 = 1$ as follows:

$$\begin{aligned}
\alpha_1 = 0, & g_1 = 111.975 \\
\alpha_2 = 0.5, & g_2 = 2.53557, \quad h_1 = \frac{g_2 - g_1}{\alpha_2 - \alpha_1} = -218.878 \\
\alpha_3 = 1, & g_3 = 93.5649, \quad h_2 = \frac{g_3 - g_2}{\alpha_3 - \alpha_2} = 182.059, \quad h_3 = \frac{h_2 - h_1}{\alpha_3 - \alpha_1} = 400.937
\end{aligned}$$

Thus,

$$P(\alpha) = 111.975 - 218.878\alpha + 400.937\alpha(\alpha - 0.5)$$

We have $P'(\alpha) = 0$ when $\alpha = \alpha_0 = 0.522959$. Since $g_0 = g\left(\boldsymbol{x}^{(0)} - \alpha_0 \boldsymbol{z}\right) = 2.32762$ is smalle than g_1 and g_3, we set

$$\boldsymbol{x}^{(1)} = \boldsymbol{x}^{(0)} - \alpha_0 \boldsymbol{z} = \boldsymbol{x}^{(0)} - 0.522959\boldsymbol{z} = (0.0112182, 0.0100964, -0.522741)^T$$

and

$$g\left(\boldsymbol{x}^{(1)}\right) = 2.32762$$

A true solution to the nonlinear system is $(0.5, 0, -0.5235988)^T$, so $\boldsymbol{x}^2$ would likely be adequate as an initial approximation for Newton's method or Broyden's method. One of these quicker converging techniques would be appropriate at this stage, since 70 iterations of the Steepest Descent method are required to find $||\boldsymbol{x}^{(k)} - \boldsymbol{x}||_\infty < 0.01$.

To begin an iteration when we use the method of Steepest Descent, the value 0 is assigned to α_1 and the value 1 is assigned to α_3. If $h(\alpha_3) \geq h(\alpha_1)$, then successive divisions of α_3 by 2 are performed and the value of α_3 is reassigned until $h(\alpha_3) < h(\alpha_1)$ and $\alpha_3 = 2^{-k}$ for some value of k. To employ the method to approximate the solution to the system

$$\begin{aligned}
f_1(x_1, x_2, x_3) &= 0 \\
f_2(x_1, x_2, x_3) &= 0 \\
&\vdots \\
f_n(x_1, x_2, x_3) &= 0
\end{aligned}$$

we simply replace the function g with $\sum_{i=1}^{n} f_i^2$.

```
======================================================
  Steepest Descent Nonlinear Solver
======================================================

Tolerance required: 0.01
Iter 0 | g(x): 55.9874
Iter 1 | g(x): 18.7994
Iter 2 | g(x): 6.73204
Iter 3 | g(x): 2.76369
Iter 4 | g(x): 1.44311
Iter 5 | g(x): 0.981409
Iter 6 | g(x): 0.796993
Iter 7 | g(x): 0.704335
Iter 8 | g(x): 0.683034
Iter 9 | g(x): 0.58033
Iter 10 | g(x): 0.522274
Iter 11 | g(x): 0.487164
Iter 12 | g(x): 0.426861
Iter 13 | g(x): 0.422178
Iter 14 | g(x): 0.353369
Iter 15 | g(x): 0.316067
Iter 16 | g(x): 0.299135
Iter 17 | g(x): 0.259209
Iter 18 | g(x): 0.235038
Iter 19 | g(x): 0.215885
Iter 20 | g(x): 0.191564
Iter 21 | g(x): 0.184732
Iter 22 | g(x): 0.157805
Iter 23 | g(x): 0.14227
Iter 24 | g(x): 0.132489
Iter 25 | g(x): 0.116327
Iter 26 | g(x): 0.114957
Iter 27 | g(x): 0.0963877
Iter 28 | g(x): 0.0862424
Iter 29 | g(x): 0.0817791
Iter 30 | g(x): 0.0708165
```

Figure 7.42: *The approximated solutions after* 51 *iterations with Steepest Descent method (**SymIntegration/Examples/Numerical Methods/Test SymIntegration Solutions of Nonlinear Systems of Equations with Steepest Descent/main.cpp**).*

```
Iter 30 | g(x): 0.0708165
Iter 31 | g(x): 0.064177
Iter 32 | g(x): 0.0591319
Iter 33 | g(x): 0.0523719
Iter 34 | g(x): 0.05086
Iter 35 | g(x): 0.0432463
Iter 36 | g(x): 0.0389064
Iter 37 | g(x): 0.0364829
Iter 38 | g(x): 0.0318798
Iter 39 | g(x): 0.0289895
Iter 40 | g(x): 0.0265232
Iter 41 | g(x): 0.0236267
Iter 42 | g(x): 0.0226785
Iter 43 | g(x): 0.0194654
Iter 44 | g(x): 0.0175753
Iter 45 | g(x): 0.0163584
Iter 46 | g(x): 0.0143809
Iter 47 | g(x): 0.0142369
Iter 48 | g(x): 0.0119365
Iter 49 | g(x): 0.0106732
Iter 50 | g(x): 0.0101662
Converged in 51 iterations.

Computed Roots:
x1 = 0.4567991332 + 0.0000000000i
x2 = -0.0024840452 + 0.0000000000i
x3 = -0.5250301420 + 0.0000000000i

Time taken by function: 27648 microseconds
```

Figure 7.43: *The approximated solutions after* 51 *iterations with Steepest Descent method (**SymIntegration/Examples/Numerical Methods/Test SymIntegration Solutions of Nonlinear Systems of Equations with Steepest Descent/main.cpp**).*

Steepest Descent method is the slowest of all the methods that are used in this section since this method is prone to zig-zagging when computing the solution. Nevertheless, even if we use a bad initial approximation it will still converge. Moreoever, in Convolutional Neural Network and Feed-Forward Neural Network this method is often used to compute the weights of the neurons.

ix. Compute the Solution of a Nonlinear System with RK-4

Compute the solution of a system of nonlinear equations given by

$$f_1(x_1, x_2, x_3) = 3x_1 - \cos(x_2 x_3) - \frac{1}{2} = 0$$
$$f_2(x_1, x_2, x_3) = x_1^2 - 81(x_2 + 0.1)^2 + \sin(x_3) + 1.06 = 0$$
$$f_3(x_1, x_2, x_3) = e^{-x_1 x_2} + 20x_3 + \frac{10\pi - 3}{3} = 0$$

Solution:

The Jacobian matrix is

$$J(\boldsymbol{x}) = \begin{bmatrix} 3 & x_3 \sin(x_2 x_3) & x_2 \sin(x_2 x_3) \\ 2x_1 & -162(x_2 + 0.1) & \cos(x_3) \\ -x_2 e^{-x_1 x_2} & -x_1 e^{-x_1 x_2} & 20 \end{bmatrix}$$

Let $\boldsymbol{x}(0) = (0, 0, 0)^T$, so that

$$\boldsymbol{F}(\boldsymbol{x}(0)) = (-1.5, 0.25, 10\pi/3)^T$$

with $N = 4$ and $h = 0.25$, we have

$$\begin{aligned}
\boldsymbol{k}_1 &= h\left[-J(\boldsymbol{x}^{(0)})\right]^{-1} F(\boldsymbol{x}(0)) \\
&= 0.25 \begin{bmatrix} 3 & 0 & 0 \\ 0 & -16.2 & 1 \\ 0 & 0 & 20 \end{bmatrix}^{-1} \begin{bmatrix} -1.5 \\ 0.25 \\ \frac{10\pi}{3} \end{bmatrix} \\
&= \begin{bmatrix} 0.125 \\ -0.004222203325 \\ -0.1308996939 \end{bmatrix} \\
\boldsymbol{k}_2 &= h[-J(0.0625, -0.002111101663, -0.06544984695)]^{-1}(-1.5, 0.25, 10\pi/3)^T \\
&= 0.25 \begin{bmatrix} 3 & -0.9043289149 \times 10^{-5} & -0.2916936196 \times 10^{-6} \\ 0.125 & -15.85800153 & 0.9978589232 \\ 0.002111380229 & -0.06250824706 & 20 \end{bmatrix}^{-1} \begin{bmatrix} -1.5 \\ 0.25 \\ \frac{10\pi}{3} \end{bmatrix} \\
&= (0.1249999773, -0.003311761993, -0.1309232406)^T \\
\boldsymbol{k}_3 &= h\left[-J(0.06249998865, -0.001655880997, -0.0654616203)\right]^{-1}(-1.5, 0.25, 10\pi/3)^T \\
&= (0.1249999844, -0.003296244825, -0.130920346)^T \\
\boldsymbol{k}_4 &= h\left[-J(0.1249999844, -0.003296244825, -0.130920346)\right]^{-1}(-1.5, 0.25, 10\pi/3)^T \\
&= (0.1249998945, -0.00230206762, -0.1309346977)^T
\end{aligned}$$

and

$$\begin{aligned}
\boldsymbol{x}(\lambda_1) = \boldsymbol{w}_1 &= \boldsymbol{w}_0 + \frac{1}{6}(\boldsymbol{k}_1 + 2\boldsymbol{k}_2 + 2\boldsymbol{k}_3 + \boldsymbol{k}_4) \\
&= \begin{bmatrix} 0.1249999697 \\ -0.00329004743 \\ -0.1309202608 \end{bmatrix}
\end{aligned}$$

Continuing, we have

$$\begin{aligned}
\boldsymbol{x}(\lambda_2) &= \boldsymbol{w}_2 \\
&= \begin{bmatrix} 0.2499997679 \\ -0.004507400128 \\ -0.2618557619 \end{bmatrix} \\
\boldsymbol{x}(\lambda_3) &= \boldsymbol{w}_3 \\
&= \begin{bmatrix} 0.3749996956 \\ -0.003430352103 \\ -0.3927634423 \end{bmatrix} \\
\boldsymbol{x}(\lambda_4) &= \boldsymbol{x}(1) = \boldsymbol{w}_4 \\
&= \begin{bmatrix} 0.4999999954 \\ 0.126782 \times 10^{-7} \\ -0.5235987758 \end{bmatrix}
\end{aligned}$$

The results obtained here are very accurate, since the actual solution is approximately $(0.5, 0, -0.52359877)^T$.

```
============================================================
  RK4 Multiroot Solver
============================================================

Step size: 150

Initial Guess 1:
  x1 = (0.1000000000,0.0000000000)
  x2 = (0.1000000000,0.0000000000)
  x3 = (-0.1000000000,0.0000000000)
Computed Convergence Root:
  x1 = (0.4999998776, 0.0000000000)
  x2 = (0.0000000423, 0.0000000000)
  x3 = (-0.5235986451, 0.0000000000)
------------------------------------------
Initial Guess 2:
  x1 = (1.0000000000,1.0000000000)
  x2 = (-1.0000000000,1.0000000000)
  x3 = (0.0000000000,-1.0000000000)
Computed Convergence Root:
  x1 = (0.4983522714, 0.0000003493)
  x2 = (-0.1996182757, 0.0000027702)
  x3 = (-0.5288283488, -0.0000002335)
------------------------------------------
Initial Guess 3:
  x1 = (-0.5000000000,-0.5000000000)
  x2 = (0.5000000000,0.5000000000)
  x3 = (1.0000000000,1.0000000000)
Computed Convergence Root:
  x1 = (0.4999996811, -0.0000001530)
  x2 = (0.0000001290, 0.0000009048)
  x3 = (-0.5235983082, 0.0000003359)
------------------------------------------

Time taken by function: 249331 microseconds
```

Figure 7.44: *The approximated solutions with three different initial appoximations $\boldsymbol{x}(0)$ with Runge-Kutta method of order four, we set the step size to be* 150*, each initial approximation returns a different solution (**SymIntegration/Examples/Numerical Methods/Test SymIntegration Solutions of Nonlinear Systems of Equations with RK4/main.cpp**).*

In the Runge-Kutta method of order four, the calculation of each $\boldsymbol{w}_j$ requires four matrix inversions, one each when computing $\boldsymbol{k}_1, \boldsymbol{k}_2, \boldsymbol{k}_3$, and $\boldsymbol{k}_4$.Thus, using N steps requires $4N$ matrix inversions. By comparison, Newton's method requires one matrix inversion per iteration. Therefore, the work involved for the Runge-Kutta method is roughly equivalent to $4N$ iterations of Newton's method.
An alternative is to use a Runge-Kutta of order two, such as the modified Euler method or even Euler's method, to decrease the number of inversions. Another possibility is to use smaller values of N.

x. Compute the Solution of a Nonlinear System with Particle Swarm Optimization

Compute the solution of a system of nonlinear equations given by

$$\begin{aligned} 3x_1 - \cos(x_2 x_3) - \frac{1}{2} &= 0 \\ x_1^2 - 81(x_2 + 0.1)^2 + \sin(x_3) + 1.06 &= 0 \\ e^{-x_1 x_2} + 20x_3 + \frac{10\pi - 3}{3} &= 0 \end{aligned}$$

Solution:
Particle Swarm Optimization (PSO) multi-root finding transforms a nonlinear system of equations into a global minimization problem, combined with a niching or deflation technique to locate multiple distinct roots.

1. **Step 1: Define the Objective (Fitness) Function**
   To apply PSO, the nonlinear system must be converted into a single minimization objectice. Define the individual equations as

   $$\begin{aligned} f_1(x_1, x_2, x_3) &= 3x_1 - \cos(x_2 x_3) - \frac{1}{2} \\ f_2(x_1, x_2, x_3) &= x_1^2 - 81(x_2 + 0.1)^2 + \sin(x_3) + 1.06 \\ f_3(x_1, x_2, x_3) &= e^{-x_1 x_2} + 20x_3 + \frac{10\pi - 3}{3} \end{aligned}$$

   Construct the aggregate objective function $\boldsymbol{F}(\boldsymbol{x})$ using the Sum of Squared Errors (SSE), where $\boldsymbol{x} = \{x_1, x_2, x_3\}$:

   $$\boldsymbol{F}(\boldsymbol{x}) = [f_1(\boldsymbol{x})]^2 + [f_2(\boldsymbol{x})]^2 + [f_3(\boldsymbol{x})]^2$$

   A root is perfectly resolved when $\boldsymbol{F}(\boldsymbol{x}) = 0$.

2. **Step 2: Initialize PSO Parameters**
   Set the algorithmic control boundaries before starting the optimization loop:

   - **Swarm Size (N):** Typically between 30 and 100 particles.
   - **Search Space Bounds:** Define the lower bounds (x_{min}) and upper bounds (x_{max}) for x_1, x_2, x_3.
   -
   - **Inertia Weight (w):** Controls momentum (typically dynamically scaled from 0.9 to 0.4).
   - **Acceleration Coeffcients (c_1, c_2):** Cognitive and social parameters (typically set to 2.0).
   - **Stopping Criteria:** Define a maximum number of iterations (t_{max}) and a root tolerance threshold (e.g., $\epsilon = 10^{-6}$).

3. **Step 3: Initialize Particle Positions and Velocities**
   For each particle i (where $i = 1, 2, \ldots, N$):

   (a) Randomly initialize the position vector $\boldsymbol{x}_i = \{x_{i1}, x_{i2}, x_{i3}\}$ uniformly within the search space bounds.

   (b) Randomly initialize the velocity vector $\boldsymbol{v} = \{v_{i1}, v_{i2}, v_{i3}\}$, usually set to zero or small random values within a fraction of the search range.

4. **Step 4: Evaluate Initial Fitness and Memory Matrices**
   (a) Calculate the fitness value $\boldsymbol{F}(\boldsymbol{x}_i)$ for every particle.
   (b) Initialize the personal best position ($\boldsymbol{p}_{best,i}$) for each particle as its current position: $\boldsymbol{p}_{best,i} = \boldsymbol{x}_i$.
   (c) Identify the particle with the lowest fitness value across the entire swarm and assign its position as the global best ($\boldsymbol{g}_{best}$).

5. **Step 5: Execute the Particle Optimization Loop**
   For each iteration $t = 1$ to $t = t_{max}$:
   (a) **Update Velocity:** Compute the new velocity for each particle using the standard PSO formula:
   $$\boldsymbol{v}_i^{(t+1)} = w\boldsymbol{v}_i^{(t)} + c_1 r_1(\boldsymbol{p}_{best,i}^{(t)} - \boldsymbol{x}_i^{(t)}) + c_2 r_2((\boldsymbol{g}_{best}^{(t)} - \boldsymbol{x}_i^{(t)}))$$
   where r_1 and r_2 are random numbers uniformly distributed between 0 and 1.
   (b) **Apply Velocity Clamping:** Constrain $\boldsymbol{v}_i^{(t+1)}$ within a predefined maximum limit $[-v_{max}, v_{max}]$ to prevent chaotic divergence.
   (c) **Update Position:** Calculate the new coordinates:
   $$\boldsymbol{x}_i^{(t+1)} = \boldsymbol{x}_i^{(t)} + \boldsymbol{v}_i^{(t+1)}$$
   (d) **Boundary Control:** If a particle steps outside $[x_{min}, x_{max}]$, snap it back to the boundary or re-initialize its position.
   (e) **Update $\boldsymbol{p}_{best}$ and $\boldsymbol{p}_{best}$**
   If $\boldsymbol{F}(\boldsymbol{x}_i^{(t+1)}) < \boldsymbol{F}(\boldsymbol{p}_{best,i})$, then $\boldsymbol{p}_{best,i} = \boldsymbol{x}_i^{(t+1)}$.
   If $\boldsymbol{F}(\boldsymbol{x}_i^{(t+1)}) < \boldsymbol{F}(\boldsymbol{g}_{best})$, then $\boldsymbol{g}_{best} = \boldsymbol{x}_i^{(t+1)}$.
   (f) **Check Convergence:** If $\boldsymbol{F}(\boldsymbol{g}_{best}) < \epsilon$, pause the loop. The current $\boldsymbol{g}_{best}$ vector is saved as a verified root $\boldsymbol{r}^*$.

6. **Step 6: Handle Multi-Root Finding via Deflation**
   Standard PSO naturally zeroes in on only one global minimum. To find multiple distinct roots, apply a deflation technique to modify the fitness landscape once a root is found:
   (a) Store the discovered root $\boldsymbol{r}^* = \boldsymbol{g}_{best}$.
   (b) Reformulate the objective function into a deflated landscape function $\boldsymbol{F}_{deflated}(\boldsymbol{x})$ so that the previously found root becomes a severe penalty peak instead of a valley:
   $$\boldsymbol{F}_{deflated}(\boldsymbol{x}) = \boldsymbol{F}(\boldsymbol{x}) \prod_{k=1}^{m} \left(1 + \frac{\lambda}{||\boldsymbol{x} - \boldsymbol{r}_k^*||}\right)$$
   where m is the number of found roots, $||\cdot||$ is the Euclidean distance, and λ is a large scaling penalty constant.
   (c) Reset the swarm partices' positions and velocities randomly.
   (d) Restart **Step 5** using $\boldsymbol{F}_{deflated}(\boldsymbol{x})$ as the new fitness evaluation standard to force the swarm to discover alternative coordinate roots.

7. **Step 7: Termination**
   Repeat the deflation loop until no new roots are discovered within the maximum designated global run time, or when the entire designated search space has been swept. The algorithm outputs the comprehensive list of unique coordinate vectors $\{x_1, x_2, x_3\}$ that satisfy the original system.

In SymIntegration we have created a class that is able to solve nonlinear system with PSO method, it is **PSOMultiRootFinder**.

```
=========================================================
  PSO Multivariable Complex Root Finder
=========================================================

Swarm population: 100
Maximum iteration: 1000
Lower bound: -4
Upper bound: 4

Found 2 unique root(s):
----------------------------------------
Root #1:
x1 = 0.4981446846 + 0.0000000000i
x2 = -0.1996058955 - 0.0000000000i
x3 = -0.5288259776 - 0.0000000000i
Residual Norm = 0.0000000000
----------------------------------------
Root #2:
x1 = 0.5000000000 - 0.0000000000i
x2 = -0.0000000000 - 0.0000000000i
x3 = -0.5235987756 - 0.0000000000i
Residual Norm = 0.0000000000
----------------------------------------

Time taken by function: 40036626 microseconds
```

Figure 7.45: *The approximated solutions with Particle Swarm Optimization method, we set the number of particles to be $N = 100$, with maximum number of iterations $t_{max} = 1000$, the lower bound is $x_{min} = -4$, and the upper bound is $x_{max} = 4$. (**SymIntegration/Examples/Numerical Methods/Test SymIntegration Solutions of Nonlinear Systems of Equations with PSO/main.cpp**).*

xi. Functions in SymIntegration Related to Numerical Solutions of Nonlinear Systems of Equations

[SI*] We will list the basic functions, struct and classes that can be used for solving nonlinear systems of equations:

class NonlinearNewtonSolver
this class computes the solution of a system of nonlinear equations using multivariable Newton's method with complex coefficients, we iteratively update the root approximation using the equation

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} - J^{-1}(\boldsymbol{x}^{(k)})\boldsymbol{F}(\boldsymbol{x}^{(k)})$$

where $\boldsymbol{F}(\boldsymbol{x})$ represents the residual vector of our functions, and $J(\boldsymbol{x})$ is the Jacobian matrix containing all partial derivatives. Instead of manually deriving or approximating these derivatives with finite differences (prone to error in approximating), we implement forward-mode Automatic-Differentiation (AD) using a templated **Dual** Number class over **std::complex<double>**.
Functions like **std::cos**, **std::sin**, and **std::exp** are fully overloaded in **complex<double>** to smoothly accept complex arguments and handle complex-plane mappings automatically.
We are using **DualClass** for this class, it operates on Dual number mechanics. For each variable, it holds a scalar value (**val**) and a vector array of partial derivatives (**der**). Overloading the fundamental mathematical rules ensures that evaluating the equations simultaneously computes the numerical values alongside exact algebraic analytical derivatives via the chain and product rules ($d(uv) = u\,dv + v\,du$).

To call the class in **main.cpp**, we need to create the object for the class :
NonlinearNewtonSolver solver1(100, 1e-9)
with $N = 100$ as the maximum number of iterations and $1e-9$ (10^{-9}) is the tolerance level (ϵ). We name the object for this class as **solver1**.

The exact system equations are fed into **main()** via a lambda wrapper, adhering cleanly to standard functional layout parameters.

```
int main()
{
       // Get starting timepoint
       auto start = high_resolution_clock::now();

       // Input the complex multivariable nonlinear system inside main()
       SystemFunction nonlinearSystem = [PI](const vector<DualClass>& x) ->
           vector<DualClass>
       {
              DualClass x1 = x[0];
              DualClass x2 = x[1];
              DualClass x3 = x[2];

              vector<DualClass> eq(3);
```

```
            // eq1 = 3*x1 - cos(x1*x2) - 0.5 = 0
            eq[0] = x1 * 3.0 - cos(x1 * x2) - 0.5;

            // eq2 = x1^2 - 81*(x2 + 0.1)^2 + sin(x3) + 1.06 = 0
            DualClass inner_eq2 = x2 + 0.1;
            eq[1] = x1 * x1 - (inner_eq2 * inner_eq2) * 81.0 + sin(x3) +
                1.06;

            // eq3 = exp(-x1*x2) + 20*x3 + (10*pi - 3)/3 = 0
            eq[2] = exp(-x1 * x2) + x3 * 20.0 + (10.0 * PI - 3.0) / 3.0;

            return eq;
      };

      // Define complex initial guess: {x1, x2, x3}
      vector<complex<double>> initial_guess = {
            complex<double>(0.1, 0.0),
            complex<double>(0.1, 0.0),
            complex<double>(-0.1, 0.0)
      };

      cout << "Initial Guess:" << endl;
      for(size_t i = 0; i < initial_guess.size(); ++i)
      {
            cout << "x" << i+1 << " = " << initial_guess[i] << endl;
      }

      // Solve the system
      NonlinearNewtonSolver solver1(100, 1e-9);
      // solve and print calculated complex roots
      solver1.solve_printSolution(nonlinearSystem, initial_guess);

...

      return 0;
}
```

Code 72: *SymIntegration/Examples/Numerical Methods/Test SymIntegration Solutions of Nonlinear Systems of Equations with Newton's Method/main.cpp*

The functions related to this class are:

- **solveLinearSystem(vector<vector<complex<double>>> J, vector<complex<double>> F)**
  a **vector<complex<double>>** function to solve the linear system of

$$J(\boldsymbol{x})\boldsymbol{y} = -\boldsymbol{F}(\boldsymbol{x})$$

  via Gaussian elimination with partial pivoting, thus

$$\boldsymbol{y} = -J(\boldsymbol{x})^{-1}\boldsymbol{F}(\boldsymbol{x})$$

we will obtain vector $\boldsymbol{y}$.

- **solve(SystemFunction func, const vector<complex<double>>& $\boldsymbol{x}^{(0)}$)**
  a **vector<complex<double>>** function to compute the solution with Newton's method, it will return the roots only as **vector<complex<double>>** and not showing the roots automatically.
- **solve_printSolution(SystemFunction func, const vector<complex<double>>& $\boldsymbol{x}^{(0)}$)**
  a **void** function to compute the solution with Newton's method within N iterations, but if the roots already converge with this criteria $||\boldsymbol{y}|| < \epsilon$ (the norm of $\boldsymbol{y}$ is under the tolerance level) then the iteration will stop. This function starts with computing the matrix $\boldsymbol{F}(\boldsymbol{x}^{(k)})$ (the evaluation vector) and $J(\boldsymbol{x}^{(k)})$ (Jacobian matrix) with **Dual**, then it will call:
  solveLinearSystem(vector<vector<complex<double>>> J, vector<complex<double>> $\boldsymbol{F}$)
  it returns a vector $\boldsymbol{y}$ that will be used to generate the next sequence for the approximated solution
  $$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} - J(\boldsymbol{x})^{-1}\boldsymbol{F}(\boldsymbol{x})$$
  Afterwards, it will show the computed roots, if any of the roots is complex then it will be shown as complex number.

class QuasiNewtonSolver
this class computes the solution of a system of nonlinear equations using multivariable Quasi-Newton's method with complex coefficients, we iteratively update the root approximation using the equations

$$A_i = A_{i-1} + \frac{\boldsymbol{y}_i - A_{i-1}\boldsymbol{s}_i}{||\boldsymbol{s}_i||_2^2}\boldsymbol{s}_i^T$$

and

$$\boldsymbol{x}^{(i+1)} = \boldsymbol{x}^{(i)} - \triangle\boldsymbol{x}^{(i)} = \boldsymbol{x}^{(i)} - A_i^{-1}\boldsymbol{F}\left(\boldsymbol{x}^{(i)}\right)$$

with $A_1 = J(\boldsymbol{x}^0)$, $\boldsymbol{y}_i = \boldsymbol{F}\left(\boldsymbol{x}^{(i)}\right) - \boldsymbol{F}\left(\boldsymbol{x}^{(i-1)}\right)$ and $\boldsymbol{s}_i = \boldsymbol{x}^{(i)} - \boldsymbol{x}^{(i-1)}$.
Instead of manually deriving or approximating these derivatives with finite differences (prone to error when approximating), we implement forward-mode Automatic-Differentiation (AD) using a templated **Dual** Number class over **std::complex<double>**, just like the **NonlinearNewtonMethod** class.
We use rank-1 Broyden update instead of recalculating costly numerical finite-difference Jacobians at each iteration. It updates an implicit directional step matrix using the identity mapping baseline, optimizing performance. It is better than parsing heavy evaluation loops at every step.

To call the class in **main.cpp**, we need to create the object for the class :
QuasiNewtonSolver solver1(100, 1e-9)
with $N = 100$ as the maximum number of iterations and $1e-9$ (10^{-9}) is the tolerance level (ϵ). We name the object for this class as **solver1**.

The functions related to this class are:

- **solveLinearSystem(vector<vector<complex<double>>> A, vector<complex<double>> $\boldsymbol{b}$, vector<complex<double>> $\boldsymbol{x}$)**
  a **bool** function to solve the linear system of
  $$A\boldsymbol{x} = \boldsymbol{b}$$

via Gaussian elimination with partial pivoting wrapper designed for handling complex matrix systems during directional step computations, with $A = J$, $\boldsymbol{b} = -\boldsymbol{F}$, and $\boldsymbol{x} = \triangle \boldsymbol{x}^i = \boldsymbol{x}^{i+1} - \boldsymbol{x}^i$; we will obtain vector $\triangle \boldsymbol{x}^i$.

- **solve(NonlinearSystem system, const vector<complex<double>>& $x^{(0)}$)**
  a **void>** function to compute the solution with quasi-Newton's method. At each iteration $i = 1, 2, \ldots, N$ it will show $||F(\boldsymbol{x}^i)||$, the residual norm, and $||\triangle \boldsymbol{x}^i||$.
- **solve_getvector(NonlinearSystem system, const vector<complex<double>>& $x^{(0)}$)**
  a **vector<complex<double>>** function to compute the solution with Quasi-Newton's method, it will return the roots only as **vector<complex<double>>** and not showing the roots automatically.
- **printSolution() const**
  a **void** function to print the solution / roots after we call the function **solve(NonlinearSystem system, const vector<complex<double>>& $x^{(0)}$)**.

class SteepestDescentSolver
this class computes the roots of the multivariable nonlinear system using **std::complex<double>**. Because Steepest Descent minimizes a scalar objective function, we define the objective function as

$$g(\boldsymbol{x}) = \frac{1}{2} \sum_{i=1}^{n} ||f_i(\boldsymbol{x})||^2$$

Since $\boldsymbol{x} \in \mathbb{C}$, the Steepest Descent update is driven by the complex gradient vector $\nabla g(\boldsymbol{x}) = J^+ F(\boldsymbol{x})$, where J^+ represents the conjugate transpose (Hermitian adjoint) of the Jacobian matrix.

To call the class in **main.cpp**, we need to create the object for the class :
SteepestDescentSolver solver1(1000, 1e-2)
with $N = 1000$ as the maximum number of iterations and $1e-2$ (10^{-2}) is the tolerance level (ϵ). We name the object for this class as **solver1**. We feed the nonlinear system directly through **main()** cleanly via a C++ polymorphic lambda expression passed to a generic template runtime (lambda capsule input), so we can change our system anytime from **main.cpp**, while the backend to compute the solution in **numericalmethods.cpp** can be left alone.

To handle complex variables seamlessly without assuming analytical derivatives, the gradient is computed with **Dual** class for both the real and imaginary components of each variable, the **Dual** class carries an internal vector **der** tracking $\frac{\partial}{\partial x_i}$. Whenever analytical operations $(\sin, \cos, \exp)$ are evaluated, the chain rule updates the derivatives flawlessly inside the complex space. The Dual number supporting **std::complex<double>** to compute the exact analytical Jacobian matrix via automatic differentiation. The nonlinear system $(f_1, f_2, \ldots, f_n)$ is encapsulated in a class structure and passed directly inside **main()**.

We use Armijo backtracking line search because a constant descent step size can easily overstep local minimum valleys and diverge. The built-in backtracking checks at each loop whether the updated system coordinates satisfy sufficient functional reduction before confirming a positional movement.

The functions related to this class are:

- **solve(SystemFunction system, const vector<complex<double>>& $x^{(0)}$)**
  A **vector<complex<double>>** function to return the roots from solving the nonlinear

system with Steepest Descent method.

- **solve_printSolution(SystemFunction system, const vector<complex<double>>& $x^{(0)}$)** A **void** function to compute the roots from solving the nonlinear system with Steepest Descent method, afterwards the roots will be printed out.

class RK4MultiRootSolver
this class computes the solution of the given nonlinear system in the complex domain using the continuous Newton's method via a 4th-order Runge-Kutta (RK4) integrator. To avoid manual derivation errors, a custom multivariable **Dual** number class computes the exact Jacobian matrix automatically using automatic differentiation (AD).
The root-finding problem $\boldsymbol{F}(\boldsymbol{x})$ is converted into an autonomous system of differential equations (Davidenko's method):

$$\frac{dx}{dt} = -J(\boldsymbol{x})^{-1}\boldsymbol{F}(\boldsymbol{x})$$

Integrating this system from $t = 0$ using RK4 drives the system towards the roots exponentially ($\boldsymbol{F}(\boldsymbol{x}) \approx 0$) as $t \to \infty$. Multiple roots are discovered by launching trajectories from a grid of initial guesses in the complex plane.

To call the class in **main.cpp**, we need to create the object for the class :
RK4MultiRootSolver solver1(initial_guesses, targetSystem, 150)
with $h = 150$ as the step size, *targetSystem* is a **vector<DualClassTemplate<complex<double>>>** (a matrix with Dual number entries) that define the nonlinear system $\boldsymbol{F}(\boldsymbol{x})$, and *initial_guesses* is a **vector<vector<complex<double>>>** (a matrix with complex entries) that store initial guesses like this $X^{(0)} = [\boldsymbol{x}_1^{(0)}|\boldsymbol{x}_2^{(0)}|\boldsymbol{x}_3^{(0)}]$, in this example we create 3 initial guesses thus $\boldsymbol{x}_i^{(0)}$ represents the i-th initial guess with $i = 1, 2, 3$. We name the object for this class as **solver1**.

Key design details:

1. For this class we are using **DualClassTemplate** class that is using **template<typename T>**. This class overrides basic operations dynamically. When the problem functions are executed with it, standard math constructs output both the calculated functional scalar values (**val**) and the automated exact derivatives (**der**).
2. **Dynamic Jacobian Mapping:** The **Dual** matrix operators automatically calculate partial derivatives via the chain rule during the computation of the equations. The **der** vector stores $\left[\frac{\partial}{\partial x_1}, \frac{\partial}{\partial x_2}, \frac{\partial}{\partial x_3}\right]$.
3. **Complex Numbers Stability:** Using **std::complex<double>** throughout ensures functions like $\cos, \sin$, and exp automatically look up the correct complex definitions via Argument-Dependent Lookup (ADL).
4. **Termination Criteria:** Stop iterating when the Euclidean norm of the functions falls below the tolerance $||\boldsymbol{F}(\boldsymbol{x}^{(n+1)})|| < \epsilon$. If $n > N_{max}$ or if the Jacobian becomes singular $\det(J) \approx$, abandon the current path as divergent.
5. **Homotopy Continuation via RK4:** The system doesn't just step blindly like typical Newton methods. It transforms the root optimization task into a continuous system of differential equations:
$$\frac{dx}{dt} = -J(\boldsymbol{x})^{-1}\boldsymbol{F}(\boldsymbol{x})$$
solving this with explicit RK4 ensures an ultra-stable, smooth approach radius toward complex/real root basins.

The functions related to this class are:

- **solveLinearSystem(vector<vector<complex<double>>>& A, vector<complex<double>>& b, vector<complex<double>>& x)**
  A **bool** function to compute the solution $\boldsymbol{x}$ from the linear system $A\boldsymbol{x} = \boldsymbol{b}$ with Gaussian elimination.
- **computeDerivatives(const vector<complex<double>>& x, SystemFunc system)**
  A **vector<complex<double>>** function to compute the derivative $\frac{dx}{dt} = -J(\boldsymbol{x})^{-1}\boldsymbol{F}(\boldsymbol{x})$ with the help of **Dual** number class.
- **solve(vector<complex<double>>& $x^{(0)}$, SystemFunc system)**
  A **vector<complex<double>>** function to solve a system of n coupled nonlinear differential equations written in vector form as $\frac{dx}{dt}\boldsymbol{F}(t, \boldsymbol{x})$, the state vector $\boldsymbol{x}$ is advanced from time step t_n to $t_{n+1} = t + h$ (where h is the step size) using the following iterative formulas:

$$\begin{aligned}
\boldsymbol{k}_1 &= \boldsymbol{F}(t_n, \boldsymbol{x}^{(n)}) \\
\boldsymbol{k}_2 &= \boldsymbol{F}\left(t_n + \frac{h}{2}, \boldsymbol{x}^{(n)} + \frac{h}{2}\boldsymbol{k}_1\right) \\
\boldsymbol{k}_3 &= \boldsymbol{F}\left(t_n + \frac{h}{2}, \boldsymbol{x}^{(n)} + \frac{h}{2}\boldsymbol{k}_2\right) \\
\boldsymbol{k}_4 &= \boldsymbol{F}(t_n + h, \boldsymbol{x}^{(n)} + h\boldsymbol{k}_3) \\
\boldsymbol{x}^{(n+1)} &= \boldsymbol{x}^{(n)} + \frac{h}{6}(\boldsymbol{k}_1 + 2\boldsymbol{k}_2 + 2\boldsymbol{k}_3 + \boldsymbol{k}_4)
\end{aligned}$$

  with $\boldsymbol{x}, \boldsymbol{F}$, and $\boldsymbol{k}$ are vectors with complex entries with same size.
- **printSolution()**
  A **void** function that show the computed roots, we don't need to call **solve(vector<complex<double>>& $x^{(0)}$, SystemFunc system)** in the **main.cpp** because it is already called inside the backend code of the **printSolution()** function inside the **numericalmethods.cpp**.

class PSOMultiRootFinder
this class finds multiple complex roots of a multivariable nonlinear system using Particle Swarm Optimization (PSO), it converts the root-finding problem into a global minimization task.
The nonlinear system $\boldsymbol{F}(\boldsymbol{x}) = \boldsymbol{0}$ is converted into an optimization objective, in this class the objective function minimizes the squared Euclidean norm / the sum of the squared magnitudes of the system equations(we minimize the norm $||\boldsymbol{F}(\boldsymbol{x})||$ / $|f_1(\boldsymbol{x})|^2 + |f_2(\boldsymbol{x})|^2 + \cdots + |f_n(\boldsymbol{x})|^2$).

Different than the previous methods mentioned above (Newton's method, Quasi-Newton, RK4), this technique does not need initial guesses. Particles are scattered randomly across the search space, meaning we do not need prior knowledge of where the roots are located. Moreover, PSO does not require derivative (Jacobian) calculations.

Standard PSO operations (like position and velocity updates) rely on ordered vector spaces, the solution treats each **std::complex<double>** variable as two separate dimensions. Since complex numbers $z \in \mathbb{C}$ consist of a real and imaginary part ($z = x + iy$), a system of N multivariable complex equations can be remapped to a system of $2N$ real variables. The objective (fitness) function to minimize is the sum of squared absolute errors across all equations. To discover multiple roots in a single execution or consecutive runs, we apply a Deflation / Repulsion technique that penalizes the swarm if it approaches an already

discovered root (a mechanism to prevent the swarm from repeatedly collapsing into the same global minimum). Deflation technique, also known as repelling or penalty function, creates an "energy barrier" around it, forcing subsequent optimization phases to discover entirely new roots.

The PSO treats the real component and imaginary component of the **std::complex<double>** vectors as separate continuous spatial dimensions during velocity and position transitions.

To call the class in **main.cpp**, we need to create the object for the class :
PSOMultiRootFinder finder(3, 100, 1000, -4.0, 4.0)
with $n = 3$ as the number of variables, $pop = 100$ as the number of population, $N = 1000$ as the maximum iteration number, $lb = -4$ as the lower bound, and $ub = 4$ as the upper bound. We name the object for this class as **finder**.

The functions related to this class are:

- **toComplexVector(const vector<double>& real_vec)**
  A **vector<complex<double>>** function to convert the internal real representation (*real_vec*) back to **std::vector<complex<double>>**.
- **evaluateFitness(const vector<double>& real_vec, const ComplexVectorFunction& system)**
  A **double** function to evaluate the *system*, the nonlinear system of equations , and returns the total residual magnitude squared, with
  ComplexVectorFunction = function<vector<complex<double>>(const vector<complex<double>>&)>
- **findOneRoot(const ComplexVectorFunction& system, mt19937& rng)**
  A **vector<complex<double>>** function to locate a single global minimm spot, *system* is the nonlinear system of equations, *rng* is the object created by Mersenne twister engine to generate random numbers for r_1 and r_2.
- **findMultipleRoots(const ComplexVectorFunction& system, int num_runs)**
  A **vector<vector<complex<double>>>** function to compute all the unique roots in the search space under *num_runs* number of running. It tracks unique coordinates across distinct initial seeds and filters out duplicates via an adjustable absolute tolerance factor $(1e-3)$.
- **findMultipleRoots_printSolution(const ComplexVectorFunction& system, int num_runs)**
  A **void** function to compute all the unique roots in the search space under *num_runs* number of running. In the end, it will show all the roots that have been found.

After learning all the methods and techniques above, we can try the hybrid refinement approach. If we have obtained rough coordinates found by PSO, then pass it into a fast local convergence method like Newton's method to achieve machine-precision accuracy.

In the **include/symintegral/numericalmethods.h** we make use of **using** and **typedef**

```
using SystemFunction = std::function<vector<DualClass>(const vector<DualClass
    >&)>;

#ifndef NONLINEAR_NEWTONSOLVER_H
#define NONLINEAR_NEWTONSOLVER_H

class NonlinearNewtonSolver {
```

```
        private:
        vector<complex<double>> roots;
        SystemFunction system;
        int max_iter ;
        double tol ;

        public:
        NonlinearNewtonSolver(int max_iter_in, double tol_in)
        : max_iter(max_iter_in), tol(tol_in)
        {}

        // Gaussian Elimination with partial pivoting for complex systems
        vector<complex<double>> solveLinearSystem(vector<vector<complex<double
            >>> J, vector<complex<double>> F) ;
        vector<complex<double>> solve(SystemFunction func, const vector<
            complex<double>>& initial_guess) ;
        void solve_printSolution(SystemFunction func, const vector<complex<
            double>>& initial_guess) ;
};


#endif

typedef vector<DualClass> (*NonlinearSystem)(const vector<DualClass>&);

#ifndef QUASINEWTONSOLVER_H
#define QUASINEWTONSOLVER_H

class QuasiNewtonSolver {
        private:
        vector<complex<double>> roots;
        int max_iter ;
        double tol ;

        public:
        QuasiNewtonSolver(int max_iter_in, double tol_in)
        : max_iter(max_iter_in), tol(tol_in)
        {}

        // Gaussian Elimination with partial pivoting for complex systems
        bool solveLinearSystem(vector<vector<complex<double>>> A, vector<
            complex<double>> b, vector<complex<double>>& x) ;
        void solve(NonlinearSystem system, const vector<complex<double>>& x0)
            ;
        vector<complex<double>> solve_getvector(NonlinearSystem system, const
            vector<complex<double>>& x0) ;
        void printSolution() const;
};
```

```
#endif

#ifndef STEEPESTDESCENTSOLVER_H
#define STEEPESTDESCENTSOLVER_H

class SteepestDescentSolver {
       private:
       vector<complex<double>> roots;
       int max_iter ;
       double tol ;

       public:
       using SystemFunction = std::function<vector<DualClass>(const vector<
           DualClass>&)>;

       SteepestDescentSolver(int max_iter_in, double tol_in)
       : max_iter(max_iter_in), tol(tol_in)
       {}

       vector<complex<double>> solve(SystemFunction system, const vector<
           complex<double>>& x0) ;
       void solve_printSolution(SystemFunction system, const vector<complex<
           double>>& x0) ;
};


#endif

#ifndef RK4MULTIROOTSOLVER_H
#define RK4MULTIROOTSOLVER_H
// Typedef for your system function wrapper using DualClassTemplate numbers
typedef vector<DualClassTemplate<complex<double>>> (*SystemFunc)(const vector
    <DualClassTemplate<complex<double>>>&);

class RK4MultiRootSolver {
       private:
       vector<vector<complex<double>>> initial_guesses;
       SystemFunc targetSystem;
       int steps;

       public:
       RK4MultiRootSolver(vector<vector<complex<double>>> initial_guesses_in,
            SystemFunc targetSystem_in, int steps_in)
       : initial_guesses(initial_guesses_in), targetSystem(targetSystem_in),
           steps(steps_in){}
```

```
        bool solveLinearSystem(vector<vector<complex<double>>>& A, vector<
            complex<double>>& b, vector<complex<double>>& x) ;
        vector<complex<double>> computeDerivatives(const vector<complex<double
            >>& x, SystemFunc system);
        vector<complex<double>> solve(vector<complex<double>> x0, SystemFunc
            system);

        void printSolution();
};


#endif
```

Code 73: *include/symintegral/numericalmethods.h*

Both lines of code **using SystemFunction = std::function<vector<DualClass>(const vector<DualClass>&)>** and **typedef vector<DualClass> (*NonlinearSystem)(const vector<DualClass>&)** define a custom type alias for a function that takes a **const vector<DualClass>&** as input and returns a **vector<DualClass>**. However, they use different language features (**std::function** and a raw function pointer) which changes what kinds of functions they can store and how much performance overhead they have. The **using SystemFunction**

Feature	SystemFunction (std::function)	NonlinearSystem (Typedef Pointer)
Type	Modern C++ Object Wrapper	Traditional C Funciton Pointer
Can store regular functions?	Yes	Yes
Can store lambdas with captures?	Yes (e.g., **[factor](const vector<DualClass>&) {...}**)	No (Only lambdas with empty **[]** captures)
Can store member functions?	Yes (via **std::bind** or a lambda wrapper)	No
Performance Overhead	Small overhead (potential heap allocation/virtual call)	None (Maximum performance) (Direct jump in memory); ideal for high-performance mathematical solvers
Syntax	Modern, readable, left-to-right (**using**)	Traditional, complex, right-to-left (**typedef**)

Table 7.4: *Comparison of Inline and Outlined Coding in C++.*

= std::function<vector<DualClass>(const vector<DualClass>&)> alias uses modern C++ (C++11 and newer) type aliasing (**using**) to wrap a **std::function** object. **std::function** is a polymorphic function wrapper (a highly adaptable container for anything that behaves like a function), it can hold almost anything callable. We can use it because it has increadible flexibility, it can store standard global/static functions, store the lambdas capture variables from the surrounding scope, store functors (classes that overload the **()** operator), and store

class member functions (when bound using **std::bind**). The downside is it comes with a small performance cost. It can introduce dynamic memory allocation (if the captured data is large) and uses indirect calls (virtual function-like overhead), which might slow down tight mathematical loops. We can use this type if the equations depend on external configuration variables (like time-dependent parameters or solver settings) that need to be captured inside a lambda function.

The **typedef vector<DualClass> (*NonlinearSystem)(const vector<DualClass>&)** uses a traditional **C**-style function pointer (which can only point to regular, standalone functions). This raw pointer can be very useful when we are building a high-performance numerical solver (like Quasi-Newton for the **DualClass** automatic differentiation) where we only need to pass pure, stateless mathematical equations. It works by storing a literal memory address pointing directly to a specific compiled block of executable code. The downside is that it is highly restrictive, cannot hold C++ lambdas that capture external variables, and it cannot directly hold class member functions.

In the **include/symintegral/symintegrationc++.h** we have defined a global variable **const double PI = acos(-1.0)**, the static and global variables live for the entire lifetime of the program, they have a fixed memory location. The lambda can access them directly without capturing them. Thus we use this **ComplexVectorFunction nonlinear_system = [](const ComplexVector& x) -> ComplexVector** instead of this **ComplexVectorFunction nonlinear_system = [PI](const ComplexVector& x) -> ComplexVector**. This is why we can use **PI** directly inside the lambda body when defining the equations.

[SI*] We have created a different **Dual** classes, such as **DualClass, DualClassTemplate, Dual, Dual1,** etc. They are all defined in **include/symintegral/dual.h**.
When computing for power series solution of differential equations with Frobenius method we also need **Dual** number, that is why we created varieties of **Dual** class.

The downside is, because SymIntegration has a lot of **.cpp** files in **src/**, there is an occurence of a multiple definition error. This error occurs because the library header **dual.h** defines non-member functions (sin, cos, exp, *pow*) directly in the header file without marking them as inline.

When we include **symintegrationc++.h** into multiple **.cpp** files (e.g. **functions.cpp, dsolve.cpp, symintegrationc++.cpp**), each compilation unit creates its own standalone copy of these functions. When the linker (**ld**) tries to merge them into the final executable, it sees the same function defined multiple times and throws a multiple definition error.

By adding the **inline** keyword before non-member functions (sin, cos, exp, *pow*), this keyword tells the compiler and linker that this function is allowed to be defined in multiple translation units, and the linker will safely merge them into a single definition.

```
root [ ~/SourceCodes/CPP/C++ Create Library/SymIntegration1.51/src ]# make
g++ -fPIC -g -Wall -Wformat -c -o functions.o ../src/functions.cpp
g++ -shared -o libsymintegration.so array.o artificialneuralnetworks.o cloning.o
 complexanalysis.o constants.o derive.o differentialequations.o divgradcurl.o ds
olve.o dynamicalsystem.o equation.o functions.o geneticalgorithms.o geometryandv
ectors.o geompack.o identity.o infiniteseries.o integrate.o linearalgebra.o mach
inelearning.o mathematicalfinance.o matnorm.o matrix.o multinomial.o navierstoke
s2d.o number.o numericalmethods.o operationsresearch.o polynomial.o product.o qu
atern.o rational.o simplify.o solve.o specialfunctions.o statistics.o stbimage.o
 sum.o symbol.o symbolic.o symerror.o symintegrationc++.o symmatrix.o vecnorm.o
vector.o vectormatrixoperations.o verylong.o asa241.o statistics-mle.o biology-d
na.o -fPIC -g -Wall -Wformat
/bin/ld: functions.o: in function `__gnu_cxx::new_allocator<std::complex<double>
 >::~new_allocator()':
/usr/include/symintegral/dual.h:246: multiple definition of `cos(DualClass const
&)'; dsolve.o:/usr/include/symintegral/dual.h:246: first defined here
/bin/ld: functions.o: in function `sin(DualClass const&)':
/usr/include/symintegral/dual.h:256: multiple definition of `sin(DualClass const
&)'; dsolve.o:/usr/include/symintegral/dual.h:256: first defined here
/bin/ld: functions.o: in function `exp(DualClass const&)':
/usr/include/symintegral/dual.h:267: multiple definition of `exp(DualClass const
&)'; dsolve.o:/usr/include/symintegral/dual.h:267: first defined here
/bin/ld: functions.o: in function `pow(DualClass const&, int)':
/usr/include/symintegral/dual.h:276: multiple definition of `pow(DualClass const
&, int)'; dsolve.o:/usr/include/symintegral/dual.h:276: first defined here
/bin/ld: symintegrationc++.o: in function `__gnu_cxx::new_allocator<std::complex
<double> >::~new_allocator()':
/usr/include/symintegral/dual.h:246: multiple definition of `cos(DualClass const
&)'; dsolve.o:/usr/include/symintegral/dual.h:246: first defined here
/bin/ld: symintegrationc++.o: in function `sin(DualClass const&)':
/usr/include/symintegral/dual.h:256: multiple definition of `sin(DualClass const
&)'; dsolve.o:/usr/include/symintegral/dual.h:256: first defined here
/bin/ld: symintegrationc++.o: in function `exp(DualClass const&)':
/usr/include/symintegral/dual.h:267: multiple definition of `exp(DualClass const
&)'; dsolve.o:/usr/include/symintegral/dual.h:267: first defined here
/bin/ld: symintegrationc++.o: in function `pow(DualClass const&, int)':
/usr/include/symintegral/dual.h:276: multiple definition of `pow(DualClass const
&, int)'; dsolve.o:/usr/include/symintegral/dual.h:276: first defined here
collect2: error: ld returned 1 exit status
make: *** [Makefile:43: libsymintegration.so] Error 1
```

Figure 7.46: *The multiple definition error.*

IX. Boundary-Value Problems for Ordinary Differential Equations

[SI*]

X. Numerical Solutions to Partial Differential Equations

[SI*] Below is an elliptic equation known as the Poisson equation

$$\frac{\partial^2 u}{\partial x^2}(x,y) + \frac{\partial^2 u}{\partial y^2}(x,y) = f(x,y)$$

In this partial differential equation we assume that f describes the input to the problem on a plane region R with boundary S. Equations of this type arise in the study of various time-independent physical problems such as the steady-state distribution of heat in a plane region, the potential energy of a point in a plane acted on by gravitational forces in the plane, and two-dimensional steady-state problems involving incompressible fluids.
Additional constraints must be imposed to obtain a unique solution to the Poisson equation. For example, the study of the steady-state distribution of heat in a plane region requires that $f(x,y) = 0$, resulting in simplification to

$$\frac{\partial^2 u}{\partial x^2}(x,y) + \frac{\partial^2 u}{\partial y^2}(x,y) = 0$$

which is called Laplace's equation. If the temperature within the region is determined by the temperature distribution on the boundary of the region, the constraints are called the Dirichlet boundary conditions.

Chapter 8

Molecular Biology

"I'm the most beautiful dog in the world bih, I'm too good to be true bih." - ***Sine Bam Bam***

This section is made to honor the science field of biology that had helped a lot of people, e.g. doctor who helps cure diseases thus make people live longer and be more productive. By learning biology we will know about certain plants or herbs that are able to cure deadly disease, for example Drontal is a medicine that costs USD 3 per tablet to cure worms for dogs, if you have no such USD 3 and you love your furry friend, then you have to know that mixing turmeric (blended turmeric) in your dog foods will eliminate the worms (ringworms, roundworms, and all kind of worms in your dog' intestine). How do we know that if we don't have search engine that can tell us turmeric is able to do such thing? By learning about the molecular / structure of tumeric and simulate or interact it with all kind of parasites, worms that can be simulated and computed with computer, not only we will be smarter but we will also find better and cheaper cure for all kinds of disease.

I. DNA Transcription and Translation

[SI*] A DNA (deoxyribonucleic acid) molecule is the carrier of genetic information, structured as a double helix. It is composed of two strands, each with a backbone of alternating sugar and phosphate groups. The two strands are connected by four chemical bases: adenine (A), thymine (T), cytosine (C), and guanine (G), which form rungs on the "twisted ladder."

[SI*] Structures and components of DNA:

1. **Double helix**
   DNA has the shape of a twisted ladder, formed by two strands that coil around each other.
2. **Backbone**
   The sides of the ladder are made of a repeating sugar-phosphate backbone.
3. **Bases**
   The "rungs" are made of pairs of four nitrogenous bases:
   - Adenine (A) always pairs with Thymine (T).
   - Cytosine (C) always pairs with Guanine (G).
4. **Nucleotides**
   Each strand is a polymer of nucleotides, which are the basic units of DNA. Each

nucleotide consists of a sugar (deoxyribose), a phosphate group, and one of the four bases.

[SI*] Functions of DNA:

- **Genetic Information**
  The specific order of sequence of the bases along the DNA backbone encodes the instructions for building and maintaining an organism.
- **Heredity**
  This sequence of bases allows genetic information to be passed down from one generation to the next.
- **Protein Synthesis**
  DNA's instructions are used to create proteins through a two-step process of transcription and translation.
  1. **Transcription**
     Enzymes "read" the DNA and create a copy in the form of a messenger RNA (mRNA) molecule.
  2. **Translation**
     The mRNA is then used as a template to link amino acids in the correct order to form a specific protein.

[SI*] DNA transcription and translation are the two core processes of protein synthesis that convert genetic information into functional proteins. Transcription is the first step, where a segment of DNA is copied into a messenger RNA (mRNA) molecule. The second step, translation, occurs on ribosomes, where the mRNA sequence is read and used to assemble a specific chain of amino acids, when then folds into a protein.

[SI*] Transcription is a process of creating an RNA copy from a DNA template. The purpose of transcription is to make a mobile copy of a gene that can be used to build a protein. Transcription happens in the nucleus in eukaryotes; in prokaryotes, it takes place in the cytoplasm.

The enzyme RNA polymerase reads the DNA and builds the new RNA strand. The end product is a messenger RNA (mRNA) molecule.

[SI*] Translation is the process of synthesizing a protein using the mRNA template. The purpose of translation is to convert the genetic code from mRNA into a sequence of amino acids that form a protein. Translation happens on ribosomes, which are located in the cytoplasm (or on the endoplasmic reticulum in eukaryotes).

Ribosomes is the cellular machinery where translation takes place. They are made of ribosomal RNA (rRNA) and proteins and hold the mRNA and tRNA in the correct positions. Transfer RNA (tRNA) acts as an adapter molecule, with one end carrying a specific amino acid and the other having an anticodon that matches a complementary codon on the mRNA.

Ribosomes read the mRNA sequence in three-nucleotide units called codons. Each codon specifies a particular amino acid, which is delivered to the ribosome by a transfer RNA (tRNA) molecule. The ribosome then links these amino acids together, forming a chain that grows until a stop codon is reached, at which point the completed polypeptide is released.

Key players:

1. **Ribosomes**
   The molecular machies that read the mRNA

2. **Transfer RNA (tRNA)**
   Molecules that bring the correct amino acid to the ribosome for each codon on the mRNA.
3. **Codons**
   Three-nucleotide sequences on mRNA that specify which amino acid should be added next.
4. **Anticodons**
   The three-nucleotide sequence on a tRNA that matches a specific mRNA codon.

The end product of translation is a chain of amino acids that folds into a functional protein.

The mRNA Genetic Code:

- **Codons**
  Each codon is a sequence of three bases (Adenine, (A), Guanine (G), Cytosine (C), and Urasil (U), which replaces Thymine found in DNA). There are 64 possible codon combinations (permutation of 3 different letters from A,G,C,U $4 \times 4 \times 4$).
- **Reading Frame**
  The sequence is always read in the $5'$ to $3'$ direction, starting from a specific start codon and ending at a stop codon.
- **Start Codon**
  The universally recognized start codon is AUG, which codes for the amino acid Methionine (Met) and establishes the reading frame for the entire sequence.
- **Stop Codons**
  There are three stop codons that signal the termination of translation: UAA, UAG, UGA. They do not code for any amino acid.
- **Degeneracy**
  Most amino acids are coded by more than one codon, which is known as the degeneracy of the genetic code.

The process of translation:

1. **Initiation**
   The ribosome assembles around the mRNA molecule. The first tRNA, carrying the amino acid methionine, binds to the start codon (AUG).
2. **Elongation**
   The ribosome moves along the mRNA strand one codon at a time. A tRNA with an anticodon complementary to the next codon enters the ribosome. The amino acid on this new tRNA is added to the growing polypeptide chain via a peptide bond. The ribosome then translocates to the next codon to continue the process.
3. **Termination**
   The ribosome reaches a stop codon (UAA, UAG, or UGA). A release factor binds to the stop codon instead of a tRNA. This triggers the release of the completed polypeptide chain from the ribosome. The ribosomal complex and the mRNA then dissociate.

[SI*] Together, transcription and translation are the two main steps of the "central dogma" of molecular biology: DNA $\rightarrow$ RNA $\rightarrow$ Protein.

The genetic information is first transcribed from the permanent DNA "blueprint" into a temporary mRNA "message." This message is then translated into the "machinery" of the cell, which are proteins.

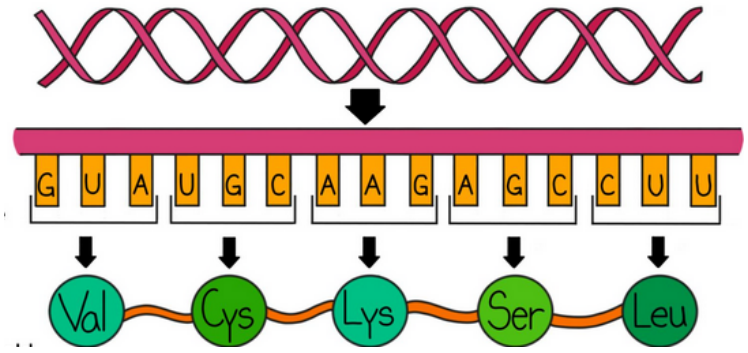

Figure 8.1: *DNA is transcribed into mRNA, mRNA is translated into amino acids. Each codon corresponds to one amino acid. Amino acids form polypeptide chains. Polypeptide chains fold into proteins.*

Second letter

First letter	U	C	A	G	Third letter
U	UUU Phe UUC Phe UUA Leu UUG Leu	UCU Ser UCC Ser UCA Ser UCG Ser	UAU Tyr UAC Tyr **UAA Stop** **UAG Stop**	UGU Cys UGC Cys **UGA Stop** UGG Trp	U C A G
C	CUU Leu CUC Leu CUA Leu CUG Leu	CCU Pro CCC Pro CCA Pro CCG Pro	CAU His CAC His CAA Gln CAG Gln	CGU Arg CGC Arg CGA Arg CGG Arg	U C A G
A	AUU Ile AUC Ile AUA Ile **AUG Met**	ACU Thr ACC Thr ACA Thr ACG Thr	AAU Asn AAC Asn AAA Lys AAG Lys	AGU Ser AGC Ser AGA Arg AGG Arg	U C A G
G	GUU Val GUC Val GUA Val GUG Val	GCU Ala GCC Ala GCA Ala GCG Ala	GAU Asp GAC Asp GAA Glu GAG Glu	GGU Gly GGC Gly GGA Gly GGG Gly	U C A G

Figure 8.2: *The 64 possible codon combinations for mRNA.*

[SI*] The 20 standard amino acids are the building blocks for poteins. these are categorized as either essential (must be obtained from the diet) or non-essential (can be synthesized by the body).

1. Alanine (Ala, A)
2. Arginine (Arg, R)
3. Asparagine (Asn, N)

4. Aspartic Acid (Asp, D)
5. Cysteine (Cys, C)
6. Glutamic Acid (Glu, E)
7. Glutamine (Gln, Q)
8. Glycine (Gly, G)
9. Histidine (His, H)
10. Isoleucine (Ile, I)
11. Leucine (Leu, L)
12. Lysine (Lys, K)
13. Methionine (Met, M)
14. Phenylalanine (Phe, F)
15. Proline (Pro, P)
16. Serine (Ser, S)
17. Threonine (Thr, T)
18. Trytophan (Trp, W)
19. Tyrosine (Tyr, Y)
20. Valine (Val, V)

i. Transcription and translation from DNA to mRNA to Polypeptide with SymIntegration

Suppose we have a 12 DNA code that we want to translate into polypeptide, it is :

$$\begin{bmatrix} A & C & G & T & C & A & G & C & A & G & C & T \end{bmatrix}$$

DNA is a double helix, so the pair will be

$$\begin{bmatrix} A & C & G & T & C & A & G & C & A & G & C & T \\ T & G & C & A & G & T & C & G & T & C & G & A \end{bmatrix}$$

Then the mRNA that carries the genetic code from the DNA to the ribosome

$$\begin{bmatrix} A & C & G & T & C & A & G & C & A & G & C & T \\ T & G & C & A & G & T & C & G & T & C & G & A \\ A & C & G & U & C & A & G & C & A & G & C & U \end{bmatrix}$$

Then we convert RNA into a polypeptide, ribosomes reading the mRNA sequence in three-nucleotide units called codons, thus

$$\begin{bmatrix} ACG & UCA & GCA & GCU \\ \text{Thr} & \text{Ser} & \text{Ala} & \text{Ala} \end{bmatrix}$$

In SymIntegration we can use this function to perform the transcription and translation:
polypeptide(vector<string> DNA)

DNA is a data type of **vector<string>**, we load a textfile **DNA.txt** that is populated with the first half pair of DNA, it is treated as a vector in linear algebra, then after we make it into double helix DNA (becoming two strands) it become a 12×2 matrix, 12 is the number of the sequence of the nitrogenous bases (Adenine, Thymine, Guanine, Cytosine).

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DNA to mRNA to Polypeptide ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration DNA to mRNA to Polypeptide ]# ./main
DNA double helix:
A C G T C A G C A G C T
T G C A G T C G T C G A
A C G U C A G C A G C U

Polypeptide:
ACG UCA GCA GCU
Thr Ser Ala Ala

Time taken by function: 883 microseconds
```

Figure 8.3: *The computation to obtain polypeptide from first half pair of DNA (**SymIntegration/Examples/Molecular Biology/Test SymIntegration DNA to mRNA to Polypeptide/main.cpp**).*

The coding till we obtain the polypeptide takes place in **src/biology-dna.cpp** and **include/symintegral/biology-dna.h**. We are open for any kind of suggestions and improvements, it is only an example to demonstrate how transcription and translation from DNA to polypeptide works, we don't really take into account the real life situation where we have to start from the start codon (AUG) and stop at stop codon (UAA, UAG, UGA).

Chapter 9

Evolutionary Algorithms

"Walnut is here. Krem is here. We are kampung dogs from Pogor." - ***Walnut and Krem***

Evolution algorithms are computer science-oriented techniques which imitate nature according to Darwin's principle of "survival of the fittest." A population of solutions is treated with genetic operations, namely selection, crossover and mutation in order to produce improved generations. This evolution approach can be applied in fluid dynamics, electrical engineering, engineering design, electromagnetic, scheduling, pattern recognition and signal processing.

I. Genetic Algorithms

[SI*] Genetic algorithms are search algorithms on the mechanics of natural selection and natural genetics. They combina survival of the fittest among string structures with a structured yet randomized information exchange to form a search algorithm with some of the innovative flair of human search. In every generation, a new set of artificial creatures (strings) is created using bits and pieces of the fittest of the old; an occasional new part is tried for good measure. While randomized, genetic algorithms are no simple random walk. They efficiently exploit historical information to speculate on new search points with expected impoved performance.

[SI*] Genetic algorithms can be used for optimization. Optimization aims for efficient allocation of scarce resources. You find examples of optimization in any scientific or engineering discipline. For example, the minimum principle of physics produces many kinds of optimization problems. Agriculturists want to improve their cattle so that the cows are more healthy and produce better milk. Engineers are interested in making better paper machines, or in optimizing the fuel consumption of engines. As another type of example, consider designing optimal schedules for school classes or aircraft crews, or minimizing the lengths of wires in an electrical device [21].

The bottleneck of optimization is finding an algorithm which solves a given class of problems. There are no methods which can solve all optimization problems. A good solution now is better than a perfect solution in the future, any method which can be adapted to a variety of problems is of great interest and of great practical use.

[SI*] Usually, we do not have a ready-made optimization problem waiting for a solution. Rather, we have a phenomenon we are interested in. The phenomenon may be a physical process, a

financial situation or an abstract mathematical construct.

The first step is to explore the phenomenon using mathematical tools. After we have a model, we may proceed to simulate the phenomenon. We often must simplify the model to make simulations. Furthermore, we may have to design new computational methods to solve the problem in a reasonable time. When we have found a solution, we have to compare the results with the actual phenomenon to verify our model and solution.

When we have an optimization problem, we have to find a solution method. Here we have a wide variety of choices to make. First, we have to find out what kind of problem we have: Are the functions smooth, non-smooth or even non-continuous? Do we have discrete or continuous variables? Is the problem convex or not? Do we have many local maxima or there is just one maximum? Can we find a good initial guess for the solution?

In many real-wold cases it is not possible to formulate a standard problem. Also, there may be so many local maxima that local optimization methods are useless. Optimization is hard, but finding the global maximum is much harder.

In these cases it is worthwhile to try genetic algorithms (GA). However, tailoring a GA to the problem may be far from nontrivial. It may be reasonable to look at other methods also: simulated annealing, tabu search and other heuristics, or local optimization with several different initial guesses.

[SI*] Evolution is a collective phenomenon of adapting to the environment and passing genes to next generations. If a combination of genes gives a reasonable chance of survival and of finding a mate, this set of genes may survive to the next generation, even if the combination is non-optimal.

Evolution, unfortunately, is not optimization. Consider the human eye, where the light sensors are "backwards." Or consider the appendix, or the way we are eating and breathing through our mouths. These nonoptimal solutions are due to the genes which our distant ancestors to have.

Genetic algorithms (GA) are based on principles from genetics and evolution. GA can be used to solve optimization problems, although the greatest promise may be in machine learning and adaptive systems.

The power of mathematics is in technology transfer: we have models and methods which can describe many different phenomena and solve a wide variety of problems. GAs are one example of mathematical technology transfer: by simulating evolution we are actually able to solve optimization problems from a variety of sources.

Instead of a single sample from the solution space. GAs maintain a population of vectors. New solution vectors are generated using selection, recombination and mutation. The new vectors are evaluated for their fitness, and the old population is replaced with a new one. This is repeated until the algorithm converges or we run out of time or patience.

Often a good enough solution will do. If the search space of all possible solution is big enough, we may be satisfied with a solution which is better than a random guess.

Genetic algorithms are adaptable to different situations. If you can find a good coding for the problem, GAs can be quite efficient and reliable. However, there are cases where other methods are more appropriate. For example, if the problem is smooth and derivatives are available, combining a gradient-based local optimization method with a global strategy will be hard to beat.

[SI*] One of the problems of GAs is the optimal coding of a particular optimization task. There are many choices to make: are we using binary or real coding, what are the crossover, mutation, and selection operators, and how the values of the object function are mapped into function values.

The traditional genetic algorithms don't process real numbers. Instead, a binary representation is used. In practice, using binary coding for real variables is wasteful. Often the performance of binary coded genetic algorithms is not satisfactory either.

i. Functions in SymIntegration Related to Genetic Algorithms

[SI*] We will list the basic functions that can be used related to genetic algorithms:

GA_maximizationproblem_functionwithtwovariables_N_generations(const Symbolic &F, const Symbolic &x_1, const Symbolic &x_2, int $n_population$, double a_j1, double b_j1, double a_j2, double b_j2, int N)
this function compute the unconstrained optimization problem for function of two variables with this specifications

$$\max f(x_1, x_2) \text{ or } \min f(x_1, x_2)$$

with $F = f(x_1, x_2)$

$$a_j1 \leq x_1 \leq b_j1$$
$$a_j2 \leq x_2 \leq b_j2$$

$n_population$ is the number of the chromosome population, N is the number of generations / iterations we want to compute.

ii. Optimization Problem with Simple Genetic Algorithms

Consider this example of unconstrained optimization problem :

$$\max f(x_1, x_2) = 21.5 + x_1 \sin(4\pi x_1) + x_2 \sin(20\pi x_2)$$

with

$$-3.0 \leq x_1 \leq 12.1$$
$$4.1 \leq x_2 \leq 5.8$$

We want to use genetic algorithms to find the solution.

Representation
First, we need to encode decision variables into binary strings. The length of the string depends on the required precision. For example, the domain of variable x_j is $[a_j, b_j]$ and the required precision is five places after the decimal point. The precision requirement implies that the range of domain of each variable should be divided into at least $(b_j - a_j) \times 10^5$ size ranges. The required bits (denoted with m_j) for a variable is calculated as follows:

$$2^{m_j - 1} \leq (b_j - a_j) \times 10^5 \leq 2^{m_j} - 1$$
$$(m_j - 1)\ln(2) \leq \ln((b_j - a_j) \times 10^5) \leq m_j \ln(2) - \ln(1)$$
$$(m_j - 1) \leq \frac{\ln((b_j - a_j) \times 10^5)}{\ln(2)} \leq m_j$$

The mapping from a binary string to a real number for variable x_j is straightforward and completed as follows:

$$x_j = a_j + \text{decimal}(\text{substring}_j) \times \frac{b_j - a_j}{2^{m_j} - 1} \tag{9.1}$$

where $\text{decimal}(\text{substring}_j)$ represents the decimal value of substring_j for decision variable x_j.

Suppose that the precision is set as five places after the decimal point. The required bits for variables x_1 and x_2 is calculated as follows:

$$(12.1 - (-3.0)) \times 100,000 = 1,510,000$$
$$2^{20} < 1,510,000 \leq 2^{21}, \quad m_1 = 21$$
$$(5.8 - (4.1)) \times 100,000 = 170,000$$
$$2^{17} < 1,510,000 \leq 2^{18}, \quad m_2 = 18$$
$$m = m_1 + m_2 = 21 + 18 = 39$$

The total length of a chromosome is 39 bits which can be represented as follows:

000000101000100110100	101101011100101011
21bits	18bits

The corresponding values for variables x_1 and x_2 are given below: In standard C++ template you are able to convert binary number that is represented as **string** data type into decimal value with **stoi(...)** function.

	Binary Number	Decimal Number
x_1	000000101000100110100	20788
x_2	101101011100101011	186155

Table 9.1: *The variable x_1 and x_2 with its corresponding binary and decimal number.*

Initial Population

Initial population is randomly generated as follows:

$$\begin{aligned}
\boldsymbol{v}_1 &= [11011010101110001000101011010010 01001] \\
\boldsymbol{v}_2 &= [10001001000010111001000000111101011 1010] \\
\boldsymbol{v}_3 &= [10100101000100110010111100110100001 0100] \\
\boldsymbol{v}_4 &= [01110001110100001011001010100110001 0110] \\
\boldsymbol{v}_5 &= [01111001111110110110011100101001001 1100] \\
\boldsymbol{v}_6 &= [11101110111001100010010010100010000 0010] \\
\boldsymbol{v}_7 &= [00100110110101001000111001001010000 1000] \\
\boldsymbol{v}_8 &= [00011011001100011001000011110110101 0010] \\
\boldsymbol{v}_9 &= [01110111111101001001111110000101010 1100] \\
\boldsymbol{v}_{10} &= [10100101100001110100101000011001001 0111]
\end{aligned}$$

The corresponding values are

$$\begin{aligned}
\boldsymbol{v}_1 &= [x_1, x_2] = [9.90112, 4.69979] \\
\boldsymbol{v}_2 &= [x_1, x_2] = [5.08353, 4.15101] \\
\boldsymbol{v}_3 &= [x_1, x_2] = [6.73684, 5.6308] \\
\boldsymbol{v}_4 &= [x_1, x_2] = [3.71332, 4.66294] \\
\boldsymbol{v}_5 &= [x_1, x_2] = [4.19503, 5.62172] \\
\boldsymbol{v}_6 &= [x_1, x_2] = [11.0913, 5.08449] \\
\boldsymbol{v}_7 &= [x_1, x_2] = [-0.709624, 5.43648] \\
\boldsymbol{v}_8 &= [x_1, x_2] = [-1.396, 4.30473] \\
\boldsymbol{v}_9 &= [x_1, x_2] = [4.0755, 5.69819] \\
\boldsymbol{v}_{10} &= [x_1, x_2] = [6.7636, 4.5459]
\end{aligned}$$

Evaluation

The process of evaluating the fitness of a chromosome consists of the following three steps:

1. **Step 1**
   Convert the chromosome's genotype to its phenotype. Here, this means converting binary string into relative real values $\boldsymbol{x}^k = (x_1^k, x_2^k), k = 1, 2, \ldots pop_size$.

2. **Step 2**
   Evaluate the objective function $f(\boldsymbol{x}^k)$.

3. **Step 3**
   Convert the value of objective function into fitness. For the maximization problem, the fitness is simply equal to the value of objective function $eval(\boldsymbol{v}_k) = f(\boldsymbol{x}^k), k = 1, 2, \ldots, pop_size$.

An evaluation function plays the role of the environment, and it rates chromosomes in terms of their fitness.

The fitness function values of above chromosomes are as follows:

$$\begin{aligned}
eval(\boldsymbol{v}_1) &= f(9.90112, 4.69979) = 10.9558 \\
eval(\boldsymbol{v}_2) &= f(5.08353, 4.15101) = 29.9485 \\
eval(\boldsymbol{v}_3) &= f(6.73684, 5.6308) = 20.0021 \\
eval(\boldsymbol{v}_4) &= f(3.71332, 4.66294) = 19.2558 \\
eval(\boldsymbol{v}_5) &= f(4.19503, 5.62172) = 20.713 \\
eval(\boldsymbol{v}_6) &= f(11.0913, 5.08449) = 36.2712 \\
eval(\boldsymbol{v}_7) &= f(-0.709624, 5.43648) = 26.9615 \\
eval(\boldsymbol{v}_8) &= f(-1.396, 4.30473) = 16.0388 \\
eval(\boldsymbol{v}_9) &= f(4.0755, 5.69819) = 29.1546 \\
eval(\boldsymbol{v}_{10}) &= f(6.7636, 4.5459) = 21.2002
\end{aligned}$$

It is clear that chromosome $\boldsymbol{v}_6$ is the strongest one and that chromosome $\boldsymbol{v}_1$ is the weakest one.

Selection

In most practices, a roulette wheel approach is adopted as the selection procedure; it belongs to the fitness-proportional selection and can select a new population with respect to the probability distribution based on fitness values. The roulette wheel can be constructed as follows:

1. Calculate the fitness value $eval(\boldsymbol{v}_k)$ for each chromosome $\boldsymbol{v}_k$:

$$eval(\boldsymbol{v}_k) = f(\boldsymbol{x}^k), \quad k = 1, 2, \ldots, pop_size$$

2. Calculate the total fitness for the population:

$$F = \sum_{k=1}^{pop_size} eval(\boldsymbol{v}_k)$$

3. Calculate selection probability p_k for each chromosome $\boldsymbol{v}_k$:

$$p_k = \frac{eval(\boldsymbol{v}_k)}{F}, \quad k = 1, 2, \ldots, pop_size$$

4. Calculate cumulative probability q_k for each chromosome $\boldsymbol{v}_k$:

$$q_k = \sum_{j=1}^{k} p_j, \quad k = 1, 2, \ldots, pop_size$$

The selection process begins by spinning the roulette wheel *pop_size* times: each time, a single chromosome is selected for a new population in the following way:

1. **Step 1**
   Generate a random number r (*pop_size* times) from the range $[0, 1]$.
2. **Step 2**
   If $r \leq q_1$, then select the first chromosome $\boldsymbol{v}_1$; otherwise, select the kth chromosome $\boldsymbol{v}_k (2 \leq k \leq pop_size)$ such that $q_{k-1} < r \leq q_k$.

The total fitness F of the population is

$$F = \sum_{k=1}^{10} eval(\boldsymbol{v}_k) = 230.501$$

The probability of a selection p_k for each chromosome $v_k (k = 1, 2, \ldots, 10)$ is as follows:

$$\begin{array}{ccc} p_1 = 0.0475303 & p_2 = 0.129928 & p_3 = 0.0867765 \\ p_4 = 0.0835386 & p_5 = 0.0898606 & p_6 = 0.157358 \\ p_7 = 0.116969 & p_8 = 0.0695822 & p_9 = 0.126483 \\ p_{10} = 0.919742 & & \end{array}$$

The cumulative probabilities q_k for each chromosome $v_k (k = 1, 2, \ldots, 10)$ is as follows:

$$\begin{array}{ccc} q_1 = 0.0475303 & q_2 = 0.177458 & q_3 = 0.264234 \\ q_4 = 0.347773 & q_5 = 0.437633 & q_6 = 0.594991 \\ q_7 = 0.71196 & q_8 = 0.781543 & q_9 = 0.908026 \\ q_{10} = 1 & & \end{array}$$

Now we are ready to spin the roulette wheel 10 times, and each time we select a single chromosome for a new population. Let us assume that a random sequence of 10 numbers from the range $[0, 1]$ is as follows:

$$\begin{array}{ccc} 0.438817 & 0.585455 & 0.35496 \\ 0.303427 & 0.323658 & 0.915456 \\ 0.661517 & 0.417606 & 0.586972 \\ 0.947615 & & \end{array}$$

The first number $r_1 = 0.438817$ is greater than q_5 and smaller than q_6, meaning that the chromosome v_6 is selected for the new population: the second number $r_2 = 0.585455$ is greater than q_5 and smaller than q_6, meaning that the chromosome v_6 is again selected for the new population; and so on. Finally, the new population consists of the following chromosomes:

$$\begin{array}{rl} v'_1 & = [11101110111001100010010010101000100000010]\ (v_6) \\ v'_2 & = [11101110111001100010010010101000100000010]\ (v_6) \\ v'_3 & = [01111001111110110110011100101001001100]\ (v_5) \\ v'_4 & = [01110001110100001011001010100110001010]\ (v_4) \\ v'_5 & = [01110001110100001011001010100110001010]\ (v_4) \\ v'_6 & = [10100101100001110100101000011001001011]\ (v_{10}) \\ v'_7 & = [00100110110101001000111001001010000100]\ (v_7) \\ v'_8 & = [01111001111110110110011100101001001100]\ (v_5) \\ v'_9 & = [11101110111001100010010010101000100000010]\ (v_6) \\ v'_{10} & = [10100101100001110100101000011001001011]\ (v_{10}) \end{array}$$

Crossover

Crossover used here is one-cut-point method, which randomly selects one cut-point and exchanges the right parts of two parents to generate offspring. Consider two chromosomes as follows, and the cut point is randomly selected after the 21st gene:

$$\begin{array}{rl} v_1 & = [111011101110011000100|100101000100000010] \\ v_2 & = [101001011000011101001|010000110010010111] \end{array}$$

The resulting offspring by exchanging the right parts of their parents would be as follows:

$$\begin{array}{rl} v'_1 & = [111011101110011000100|010000110010010111] \\ v'_2 & = [101001011000011101001|100101000100000010] \end{array}$$

The probability of crossover is set as $p_c = 0.25$, so we expect that, on average, 25% of chromosomes undergo crossover.

Now, we will generate a random number r (pop_size times) from the range $[0,1]$, assume that the sequence of random numbers is

0.941361	0.522253	0.729774
0.41996	0.453728	0.601589
0.67799	0.764918	0.363573
0.0676645		

This means that the chromosome v_{10} is the only chromosome selected for crossover, since we don't have the couple here, in this computation example, we will proceed to the next step -> mutation.

Otherwise, if we have at least 2 chromosomes that were chosen for crossover. We will then generate a random integer number pos from the range $[1,38]$ (because 39 is the total length of a chromosome) as cutting point or in other wods, the position of the crossover point. Assume that the generated number pos equals 1, then the two chromosomes are cut after the first bit, and offspring are generated by exchanging the right parts of them.

In the C++ code in SymIntegration, we assume that if the chosen chromosomes are odd number and more than 2, then the last one will not undergo any crossover, the chromosomes that undegone the crossover are the first even number only. In other words, we assume the mating is loyal only to another one, and not mating with another one.

Mutation

Mutation alters one or more genes with a probability equal to the mutation rate. Assume that the 23rd gene of the chromosome v_1 is selected for a mutation. Since the gene is 1, it would be flipped into 0. Thus the chromosome after mutation would be

$$\begin{aligned} v_1 &= [111011101110011000100|010000110010010111] \\ &\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\downarrow \\ v_1' &= [111011101110011000100|000000110010010111] \end{aligned}$$

The probability of mutation is set as $p_m = 0.01$, so we expect that, on average, 1% of total bit of population would undergo mutation. There are $m \times pop_size = 39 \times 10 = 390$ bits in the whole population; we expect 3.9 mutations per generation. Every bit has an equal chance to be mutated. Thus we need to generate a sequence of random numbers $r_k (k = 1,2,\ldots,390)$ from the range $[0,1]$, or we generate a pair sequence of random numbers of r_{row}, $row = 1,2,\ldots,m$ and r_{column}, $column = 1,2,\ldots,pop_size$ for the choice of bit that will undergo mutation.

After the mutation we will obtain the corresponding decimal values of variables $[x_1, x_2]$ and the new fitness are as follows:

$$
\begin{aligned}
f(10.6194, 5.08449) &= 36.7526 \\
f(11.0914, 5.08449) &= 36.2746 \\
f(4.19503, 5.62172) &= 20.713 \\
f(3.71332, 4.66294) &= 19.2558 \\
f(3.71378, 4.66294) &= 19.2367 \\
f(6.7636, 4.5459) &= 21.2002 \\
f(-0.709624, 5.43648) &= 26.9615 \\
f(4.19503, 5.62172) &= 20.713 \\
f(11.0913, 5.08449) &= 36.2712 \\
f(6.70461, 4.5459) &= 25.9702
\end{aligned}
$$

Now we just completed one iteration of genetic algorithm. The test run is terminated after 5 generations. We have obtained the best chromosome in the 5th generation as follows:

$$
\begin{aligned}
\boldsymbol{v}^* &= (111111111111111100101111100010111101011) \\
eval(\boldsymbol{v}^*) &= f(12.0998, 5.70358) = 37.7174
\end{aligned}
$$

Compared to the result of [14] that will have $eval(\boldsymbol{v}^*) = f(11.631407, 5.724824) = 38.818208$ at the 419th generation, we don't use a lot of generations, instead we only use 5 and re-initialize it again to see in the 5th generation the highest $eval(\boldsymbol{v}^*)$ will reach which value.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Genetic Algorithms for Mutivariable Functions Optimization
# ./main

Maximize this function: x1*sin(12.5664*x1)+x2*sin(20*x2)+21.5

-3 <= x_{1} <= 12.1

4.1 <= x_{2} <= 5.8

m_{1} = 21

m_{2} = 18

Initial population for x_{1}:
11011010101100010001
10001001000010111010
10100101000100110101
01110001101000010110
01111001111101101100
11101110111001000100
00100110110101001001
00011011001100011010
01110111111010010011
10100101100001101001

Corresponding Decimal for x_{1}:
1.79176e+06
1.12267e+06
1.35229e+06
932374
999276
1.95706e+06
318097
222770
982675
1.35601e+06

Corresponding value for x_{1}:
9.90112
5.08353
6.73684
3.71332
4.19503
11.0913
-0.709624
-1.396
4.0755
6.7636
```

Figure 9.1: *The computation of the simple genetic algorithms for the unconstrained optimization problem (maximization) (**SymIntegration/Examples/Genetic Algorithms/Test SymIntegration Genetic Algorithms for Mutivariable Functions Optimization/main.cpp**).*

```
Initial population for x_{2}:
0101101001010001001
0000011111010111010
1110011010000010100
0101010011000010110
1110010100100111100
1001010000100000010
1100100101000001000
0001111011010010010
1111000010101011100
0100001100100010111

Corresponding Decimal for x_{2}:
92489
7866
236052
86806
234652
151810
206088
31570
246444
68759

Corresponding value for x_{2}:
4.69979
4.15101
5.6308
4.66294
5.62172
5.08449
5.43648
4.30473
5.69819
4.5459

Initial population for x_{1} and x_{2}:
1101101010111000100010101101001010011001
1000100100001011100100000011110101110 10
1010010100010011001011110011010000010100
0111000111010000010110010101001100010110
0111100111111101101100111001010010011100
1110111011100110001001001010001000000010
0010011011010100100011100100101000001000
0001101100110001100100001111011010100 10
0111011111111010010011111100001010101100
1010010110000111010010100011010010111
```

Figure 9.2: *The computation of the simple genetic algorithms for the unconstrained optimization problem (maximization) (**SymIntegration/Examples/Genetic Algorithms/Test SymIntegration Genetic Algorithms for Mutivariable Functions Optimization/main.cpp**).*

```
************************************************************

Procedure: Evaluation

Generation: 1

************************************************************

F(x_{1}, x_{2}):
10.9558
29.9485
20.0021
19.2558
20.713
36.2712
26.9615
16.0388
29.1546
21.2002
```

Figure 9.3: *The computation of the simple genetic algorithms for the unconstrained optimization problem (maximization) (**SymIntegration/Examples/Genetic Algorithms/Test SymIntegration Genetic Algorithms for Mutivariable Functions Optimization/main.cpp**).*

```
************************************************************

Procedure: Selection

Generation: 1

************************************************************

Total fitness for the population = 230.501

Selection probability, p_{k}:
0.0475303
0.129928
0.0867765
0.0835386
0.0898606
0.157358
0.116969
0.0695822
0.126483
0.0919742

Cumulative probability, q_{k}:
0.0475303
0.177458
0.264234
0.347773
0.437633
0.594991
0.71196
0.781543
0.908026
1

Roulette Wheel, random sequence of 10 numbers from the range [0,1]:
0.438817
0.585455
0.35496
0.303427
0.323658
0.915456
0.661517
0.417606
0.586972
0.947615
```

Figure 9.4: *The computation of the simple genetic algorithms for the unconstrained optimization problem (maximization) (**SymIntegration/Examples/Genetic Algorithms/Test SymIntegration Genetic Algorithms for Mutivariable Functions Optimization/main.cpp**).*

```
New population :
11101110111001100010010010000100000010
11101110111001100010010010000100000010
01111001111110110110011100101001011100
01110001110100001011001010100110010110
01110001110100001011001010100110010110
10100101100001110100101000011001001011
00100110110101001000111001001010000100
01111001111110110110011100101001011100
11101110111001100010010010000100000010
10100101100001110100101000011001001011
```

Figure 9.5: *The computation of the simple genetic algorithms for the unconstrained optimization problem (maximization) (**SymIntegration/Examples/Genetic Algorithms/Test SymIntegration Genetic Algorithms for Mutivariable Functions Optimization/main.cpp**).*

```
************************************************************

Procedure: Crossover

Generation: 1

************************************************************

Random sequence of 10 numbers from the range [0,1]:
0.941361
0.522253
0.729774
0.41996
0.453728
0.601589
0.67799
0.764918
0.363573
0.0676645

Assume that the probability of crossover is 0.25

Crossover population :
1010010110000111010010100001100100101111

Bits per chromosome = 39

Crossover position = 17

Crossover population final :

Crossover population index:
9

Population after crossover:
1110111011100110001001001010001000000010
1110111011100110001001001010001000000010
0111100111111011011001110010100100111100
0111000111010000101100101010011000010110
0111000111010000101100101010011000010110
1010010110000111010010100001100100101111
0010011011010100100011100100101000001000
0111100111111011011001110010100100111100
1110111011100110001001001010001000000010
1010010110000111010010100001100100101111
```

Figure 9.6: *The computation of the simple genetic algorithms for the unconstrained optimization problem (maximization) (**SymIntegration/Examples/Genetic Algorithms/Test SymIntegration Genetic Algorithms for Mutivariable Functions Optimization/main.cpp**).*

```
************************************************************

Procedure: Mutation

Generation: 1

************************************************************

Population size = 390

Assume that the probability of mutation is : 0.01

We expect 4 mutations per generation.
Every bit has an equal chance to be mutated.

Generating a sequence of random numbers for the row and column....

Mutated row:
1
2
5
10

Mutated column:
5
18
15
8

Population before mutation:
11101110111001100010010010100010000010
11101110111001100010010010100010000010
01111001111111011011001110010100100111100
01110001110100000101100101010011000010110
01110001110100000101100101010011000010110
10100101100001110100101000011001001011 1
00100110110101001000111001001010000 1000
01111001111111011011001110010100100111100
11101110111001100010010010100010000010
10100101100001110100101000011001001011 1

Population after mutation:
11100110111001100010010010100010000010
11101110111001100110010010100010000010
01111001111111011011001110010100100111100
01110001110100000101100101010011000010110
01110001110100101011001010100110000 10110
10100101100001110100101000011001001011 1
00100110110101001000111001001010000 1000
01111001111111011011001110010100100111100
11101110111001100010010010100010000010
10100100100001110100101000011001001011 1
```

Figure 9.7: *The computation of the simple genetic algorithms for the unconstrained optimization problem (maximization) (**SymIntegration/Examples/Genetic Algorithms/Test SymIntegration Genetic Algorithms for Mutivariable Functions Optimization/main.cpp**).*

```
************************************************************

Procedure: Evaluating New Generation

Generation: 1

************************************************************

New binary x1:
11100110111001100 0100
11101110111001100 1100
01111001111110110 1100
01110001110100001 0110
01110001110100001 0110
10100101100001110 1001
00100110110101001 0001
01111001111110110 1100
11101110111001100 0100
10100100100001110 1001

New binary x2:
10010100010000001 0
10010100010000001 0
11100101001001110 0
01010100110001011 0
01010100110001011 0
01000011001001011 1
11001001010000100 0
11100101001001110 0
10010100010000001 0
01000011001001011 1
```

Figure 9.8: *The computation of the simple genetic algorithms for the unconstrained optimization problem (maximization) (**SymIntegration/Examples/Genetic Algorithms/Test SymIntegration Genetic Algorithms for Mutivariable Functions Optimization/main.cpp**).*

```
New x1 :
10.6194
11.0914
4.19503
3.71332
3.71378
6.7636
-0.709624
4.19503
11.0913
6.70461

New x2:
5.08449
5.08449
5.62172
4.66294
4.66294
4.5459
5.43648
5.62172
5.08449
4.5459

New F(x_{1}, x_{2}) evaluation:
36.7526
36.2746
20.713
19.2558
19.2367
21.2002
26.9615
20.713
36.2712
25.9702
```

Figure 9.9: *The computation of the simple genetic algorithms for the unconstrained optimization problem (maximization) (**SymIntegration/Examples/Genetic Algorithms/Test SymIntegration Genetic Algorithms for Mutivariable Functions Optimization/main.cpp**).*

iii. Visualization of Optimization Problem with Genetic Algorithms with SFML

We want to visualize the genetic algorithms process in finding the global minimum value of four functions. These functions are standard benchmarks used in computer science to evaluate how well optimization algorithms (like Genetic Algorithms) can find global minima without getting trapped in complex mathematical landscapes. All the four functions are structured so that their global minimum value is exactly 0.

1. **The Optimization Test Functions**

   (a) **Rosenbrock Function (Valley)**

   $$f(x,y) = (1-x)^2 + 100(y-x^2)^2$$

   The landscape of this function is a long, narrow, curved parabolic valley. Finding the valley is easy, but converging to the absolute global minimum at $(x,y) = (1,1)$ is difficult because the floor of the valley is extremely flat.
   For the visualization trick in code since the value spikes exponentially away from the valley. The code uses log-scaling $(\ln(1+f(x,y)))$ so the vertical walls don't shoot out of the screen.

   (b) **Rastrigin Function (Highly Multimodal)**

   $$f(x,y) = 20 + (x^2 10\cos(2\pi x)) + (y^2 - 10\cos(2\pi y))$$

   The landscape is a massive grid of frequent, undulating hills and deep holes. The global minimum is dead center at $(0,0)$.
   It contains hundreds of local minima traps. It perfectly tests whether the genetic algorithm's mutation rate is high enough to escape local valleys.

   (c) **Booth Function (Plateau)**

   $$f(x,y) = (x+2y-7)^2 + (2x+y-5)^2$$

   The landscape is a long, gentle, oblong plateau that slopes down smoothly toward a single local trough at $(1,3)$. It tests steady directional optimization convegence.

   (d) **Ackley Function (Deep Central Hole)**

   $$f(x,y) = -2e^{-0.2\sqrt{0.5(x^2+y^2)}} - e^{0.5(\cos(2\pi x)+\cos(2\pi y))} + 20 + e^1$$

   The landscape is a nearly flat outer terrain that suddenly drops into a steep, narrow hole at absolute center $(0,0)$. It tests whether algorithms can search wide areas and drop precisely into hidden targets.

2. **The Genetic Algorithm Pipeline**
   A genetic algorithm mimics natural selection to solve optimization problems through an evolutionary loop. The basicalgorithm of GA are:

   (a) Initial Population

   (b) Evaluate Fitness

   (c) Selection

(d) Crossover

(e) Mutation

(f) Loop to the next generation

- **Chromosome Representation:** Each agent (individual) carries a coordinate pair vector (x, y) as its genetic sequence code.
- **Fitness Evaluation:** GAs are naturally designed to maximize a value, but our test functions are minimization problems(looking for 0). The trick is the code transforms the score using a reciprocal mapping function

$$Fitness = \frac{1}{1 + f(x, y)}$$

  As the function value $f(x, y)$ approaches 0, the fitness score approaches its maximum value of 1.0.
- **Elitism (Preservation):** The code sorts the population by fitness score and explicitly duplicates the top 2 individuals directly into the next generation. This guarantees the algorithm never accidentally forgets its best discovery.
- **Tournament Selection:** To pick parents for breeding, the engine pulls two random individuals from the population and pits them against each other

$$Winner = \max(Fitness(I_1), Fitness(I_2))$$

  This balances randomness with selective pressure.
- **Blend Crossover (BLX-α):** Rather than swapping hard chunks of code, the algorithm blends parent coordinates linearly based on the crossover bias slider (α):

$$x_{child} = \alpha \cdot x_{parent1} + (1 - \alpha) \cdot x_{parent2}$$
$$y_{child} = \alpha \cdot y_{parent1} + (1 - \alpha) \cdot y_{parent2}$$

- **Uniform Boundary Mutation:** With a probability governed by the mutation rate slider, a random value is injected into the child coordinates to simulate genetic mutations. The mutation step size scales adaptively based on the dimensions of the active landscape domain:

$$x_{mutated} = x_{child} + rand(-5\%, +5\%) \times (Domain_{Max} - Domain_{Min})$$

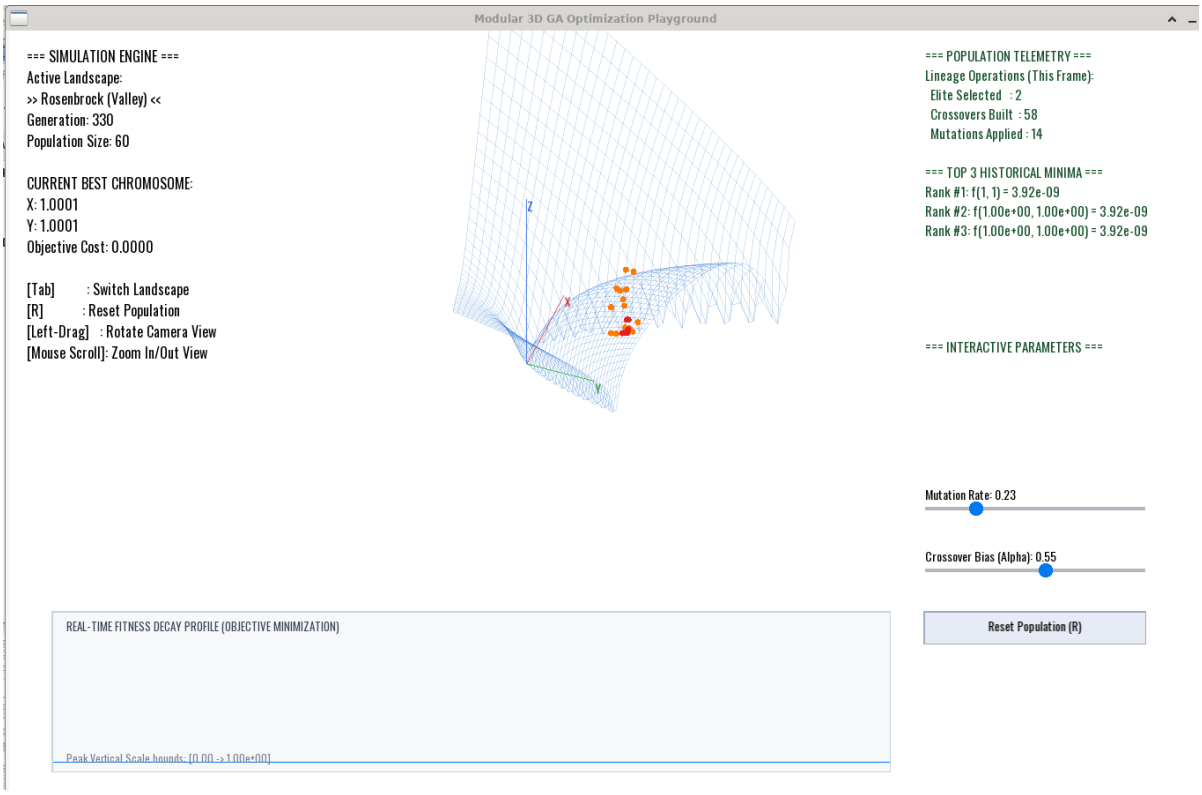

Figure 9.10: *The visualization of the simple genetic algorithms for the optimization problem of the Rosenbrock function (**SymIntegration/Examples/Genetic Algorithms/Test SymIntegration Genetic Algorithms Simulation and Optimization of Function with 2 Variables with SFML/main.cpp**).*

The background of the visualization is pure white for better sight, all HUD text and UI elements are made to be dark or black tones for clean readability, we also have full mouse controls with left-click and drag to rotate the camera and mouse wheel to zoom in or out.

Application Code Architecture

- **include/symintegral/sfml-visualizations.h**
  Declares foundational mathematical optimization parameters, chromosome structures, boundary domain restrictions, and core evaluation routines.
- **src/sfml-visualizations.cpp**
  Contains the fixed evaluation math formulas (Ackley string anomalies corrected), tournamend parent selection logic, crossover engines, boundary clipping, and sorting.
- **Examples/Genetic Algorithms/Test SymIntegration Genetic Algorithms Simulation and Optimization of Function with 2 Variables with SFML/main.cpp**
  Controls the structural SFML interactive environment context. Coordinates rendering pipelines for coordinate mesh maps, agent positions, rolling tracking graphs, and color-coded XYZ axes elements.

Chapter 10

Machine Learning

"Krem is not being naughty, Krem is only being Krem-self." - ***Krem***

Machine learning is a subfield of artificial intelligence (AI) that allows machines to learn from data without being explicitly programmed.

Artificial neural networks are a subset or technique within machine learning. ANNs were first conceived decades ago and have been an important part of machine learning and artificial intelligence ever since. The basic idea behind neural networks is to combine linear combinations and nonlinear functions into layers such that they can approximate arbitrary functions. Neural networks have been used to great effect for example in image classification and recognition once the computational power and suitable algorithms to train large neural networks had become available [18].

Neural networks can be used for pattern recognition, e.g. digit recognition, letters recognition, image recognition. The problem of searching patterns in data is a fundamental one and has a long and successful history. For instance, the extensive astronomical observations of Tycho Brahe in the 16^{th} century allowed Johannes Kepler to discover the empirical laws of planetary motion, which in turn provided a springboard for the development of classical mechanics. Similarly, the discovery of regularities inatomic spectra played a key role in the development and verification of quantum physics in the early twentieth century. The field of pattern recognition is concerned with the automatic discovery of regularities in data through the use of computer algorithms and with the use of these regularities to take actions such as classifying the data into different categories.

The use of artificial neural networks as a tool to help computers recognize patterns and simulate human intelligence had been introduced in the 1980s, by the early 2000s, Yann LeCun, Geoffrey Hinton, and Yoshua Bengio (recipients of the 2018 ACM A.M. Turing Award) were among a small group who remained committed to this approach. Deep learning methods (e.g., CNN) have been responsible for astonishing breakthroughs in computer vision, speech recognition, natural languange processing, and robotics.

Artificial intelligence is now one of the fastest-growing areas in all of science and one of the most talked-about topics in society. The growth of and interest in AI is due to the recent advances in deep learning. These technologies are used by billions of people. Anyone who has a smartphone in their pocket can tangibly experience advances in natural language processing and computer

vision. In addition to the products we use every day, new advances in deep learning have given scientists powerful new tools-in areas ranging from medicine, to astronomy, to materials science.

I. Introduction

[SI*] The idea underlying an artificial neural network is to define a function taking a vector as the input and yielding a vector as the output. The analogy of artificial neural networks obviously stems from the neurons and axons in the nervous system and the brain, although the dependencies or conenctions implemented by axons are in general much more complicated (and still mostly unknown) than the ones betwwn adjacent layers indicated.

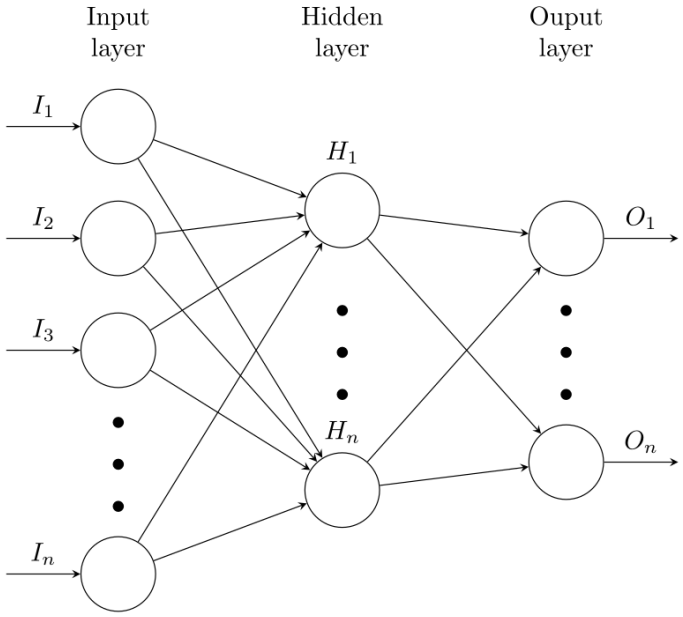

Figure 10.1: *The schematic diagram of an artificial neural network. Each neuron influences each neuron in the next layer.*

It may be unsurprising that large enough neural networks with a certain number and structure of the hidden layers have been used very successfully in image-classification and image-recognition tasks.

In engineering, our aim is not to understand the brain per se, but to build better computer systems. The brain is an information processing device that has some incredible abilities and surpasses current engineering products in many domains, e.g., vision, speech recognition, and learning. These applications have evident economic utility if implemented on machines. If we can understand how the brain performs these functions, we can define solutions to these tasks as formal algorithms and implement them on computers.

The human brain is quite different from a computer. Whereas a computer generally has one processor, the brain is composed of a very large (10^{11}) number of processing units, namely, neurons, operating in parallel. The processing units are believed to be much simpler and slower than a processor in a computer. You can just compile a code and it will be able to

give you the initial value problem of a fourth order linear differential equation solution in few seconds, while on hands it will take probably around 10 minutes or more for human to compute the solution.

What makes the brain different, and is believed to provide its computational power, is the large connectivity. Neurons in the brain have connections, called synapses, to around 10^4 other neurons, all operating in parallel. In a computer, the processor is active and the memory is separate and passive, but it is believed that in the brain, both the processing and memory are distributed together over the network; processing is done by the neurons, and the memory is in the synapses between the neurons.

The difference between natural and artifical neural network might be the same as the classic example of the natural and artificial flying machines. An eagle flaps its wings; while a commercial airplane does not flap its wings but uses jet engines. The eagle and the airplane are two hardware implementations built for different purposes, satisfying different constraints. But both implement the same theory, which is aerodynamics.

[SI*] **Activation Functions**

Activation functions are critical in neural networks because they introduce non-linearity, allowing the model to learn complex patterns in data. Without them, a neural network would be equivalent to a linear regression model, no matter how many layers it had.
The activation function is applied to the weighted sum of inputs before producing the final output of a neuron. Activation functions decide whether a neuron should be activated based on the weighted sum of inputs and a bias term. They also make backpropagation possible by providing gradients for weight updates.

Suppose we have input vector $\boldsymbol{x} = \{x_1, x_2, \ldots, x_n\}$ with its corresponding weights $\boldsymbol{w} = \{w_1, w_2, \ldots, w_n\}$, thus the inputs will go to certain node and we will have to compute the sum formula first

$$\sum_{i=1}^{n} x_i w_i + b$$

with b as the bias term. Then we will put this sum formula into a certain activation function (sigmoid, tanh, ReLU, etc) that will be denoted as $f(\cdot)$ thus

$$f\left(\sum_{i=1}^{n} x_i w_i + b\right)$$

The activation function $f(\cdot)$ will be the output from a node that has the input vector $\boldsymbol{x}$ and its corresponding weight vector $\boldsymbol{w}$.

Now, we will explain some activation functions that can be chosen depend on the case or application of the artificial neural networks:

ReLU (Rectified Linear Unit)

This activation function is a widely used activation function in deep learning that outputs the input directly if it is positive and returns zero otherwise. Its simplicity and efficiency make it a default choice in many neural network architectures, helping models learn complex patterns while reducing issues like the vanishing gradient problem (a common issue with sigmoid and tanh activation functions). It allows positive values to pass unchanged and sets

negative values to zero.
The output range: $(0, \infty)$.
It is defined as follow:

$$\text{ReLU}(x) = \max(0, x) \tag{10.1}$$

it retuns x if x is greater than 0, if x is less than or equal to 0, the function returns 0.
ReLU's ability to output zero for negative inputs introduces sparsity in the network, meaning only a fraction of neurons activate at any given time. This can lead to more efficient and faster computation.
ReLU is computationally efficient as it involves only a thresholding operation. This simplicity makes it easy to implement and compute, which is important when training deep neural netowrks with millions of parameters. Although it seems like a piecewise linear function, ReLU is still a non-linear function.

Sigmoid
This activation function compresses input values into a range between 0 and 1, making it suitable for binary classification. However, sigmoid suffers from vanishing gradients, especially with very high or very low inputs.
The output range: $(0, 1)$.
It is defined as follow:

$$S(x) = \frac{1}{1 + e^{-x}} \tag{10.2}$$

The derivative of sigmoid function is known as the density of the logistic distribution

$$\frac{d}{dx} S(x) = f(x)(1 - f(x)) \tag{10.3}$$

Softmax
The softmax activation function converts raw output scores (logits) from a neural network into a probability distribution, where each value is between 0 and 1, and all values sum to exactly 1. Primarily used in the final layer of multi-class classification models, it makes output interpretable by highlighting the most likely class. It is a $\mathbb{R}^n \to \mathbb{R}^n$ function, it takes a vector of n size as the input and produce a vector of n size as the output.
The output range: $(0, 1)$.
For a vector of raw outputs (logits) $\boldsymbol{x} = \{x_1, x_2, \ldots, x_n\}$, the softmax function is defined as follows:

$$s_i = \sigma(\boldsymbol{x})_i = \frac{e^{x_i - \max(\boldsymbol{x})}}{\sum_{j=1}^{n} e^{x_j - \max(\boldsymbol{x})}} \tag{10.4}$$

the exponentiation makes all inputs positive and accentuates differences, then dividing by $\sum e^{x_j - \max(\boldsymbol{x})}$ ensures the sum of all output is 1 (normalization process). A direct implementation of e^{x_i} can easily cause an overflow error if x_i is large, to prevent this we often use $e^{x_i - \max(\boldsymbol{x})}$.
This will shift the inputs to a range close to zero, assuming the inputs themselves are not too far from each other. Crucially, it shifts them all to be negative (except the maximal x_j which turns into a zero). Negatives with large exponents "saturate" to zero rather than infinity, so we have a better chance of avoiding NaNs result.

The resulting probabilities indicates the confidence in each class i. This activation function is typically used at the end of Convolutional Neural Networks or Deep Neural Networks

for classifying inputs into one of n mutually exclusive categories, it also works best with Categorical Cross-Entropy Loss to calculate the difference between predicted and actual probability distributions.

Categorical Cross-Entropy loss measures the dissimilarity between the probability distributions predicted by a machine learning model and the true probability distributions of the target classes. It is the standard loss function for multi-class classification.

When using softmax as the output activation combined with Categorical Cross-Entropy loss, the derivative simplifies into a remarkably elegant formula:

$$\delta_i = \hat{y}_i - y_i$$

where $\hat{y}_i$ is the predicted probability from the softmax function for class i, y_i is the actual target for class i, which is typically 1 for correct class and 0 for incorrect class (one-hot encoded). To understand why this happens, we must apply the chain rule. The delta for the output layer is the derivative of the loss (L) with respect to the pre-activation input (x_i).

1. **Categorical Cross-Entropy Loss**
   For a multi-class classification task, the loss function is defined as:

   $$L = -\sum_{j=1}^{n} y_j \log(\hat{y}_j)$$

   with n as the total number of classes, y_j is the actual output (usually a one-hot encoded vector), and $\hat{y}_j$ is the predicted probability for class j.
   Because the true labels y_j are one-hot encoded, only the term corresponding to the true class ($y_{true} = 1$) remains, simplifying the formula to

   $$L = -\log(\hat{y}_{true})$$

   The multi-class classification is a classification problem where each sample belongs to only one class (mutually exclusive), it is different than multi-label classification where each sample may belong to multiple classes (or to no class).
2. **The Softmax Derivative (Jacobian Matrix)**
   Because a change in a single pre-activation x_j affects all probabilities $\hat{y}_i$ (due to the normalization division). Since softmax has multiple inputs, with respect to which input element the partial derivative is computed the derivative of softmax requires a Jacobian matrix:

   $$\frac{\partial \hat{y}_i}{\partial x_j} = \hat{y}_i(\delta_{ij} - \hat{y}_j) = \begin{cases} \hat{y}_i(1 - \hat{y}_i), & \text{if } i = j \\ -\hat{y}_i\hat{y}_j, & \text{if } i \neq j \end{cases}$$

   where δ_{ij} is the Kronecker delta (equal to 1 if $i = j$, and 0 if $i \neq j$), thus

   $$J_{softmax} = \begin{bmatrix} \frac{\partial s_1}{\partial x_1} & \frac{\partial s_1}{\partial x_2} * & \cdots & \frac{\partial s_1}{\partial x_n} \\ \frac{\partial s_2}{\partial x_1} & \frac{\partial s_2}{\partial x_2} & \cdots & \frac{\partial s_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial s_n}{\partial x_1} & \frac{\partial s_n}{\partial x_2} & \cdots & \frac{\partial s_n}{\partial x_n} \end{bmatrix}$$

   where n is the number of classification. When talking about the derivative of the softmax function, we actually talk about its Jacobian matrix, which is the matrix of all

first-order derivatives.
For vector functions like softmax it is imprecise to talk about a "gradient"; the Jacobian is the fully general derivative of a vector function.

3. **Combining the Chain Rule**
To get the final delta, you take the derivative of the loss with respect to the pre-activation x_i:

$$\delta_i = \frac{\partial L}{\partial x_i} = \sum_j \frac{\partial L}{\partial \hat{y}_j} \frac{\partial \hat{y}_j}{\partial x_i}$$

By substituting the derivatives and evaluating summation for one-hot encoded targets, the complex derivatives collapses into the simple difference between prediction and actual output:

$$: \delta_i = \hat{y}_i - y_i$$

where $\hat{y}_i$ is the predicted probability from the softmax function for class i, y_i is the actual target for class i, which is typically 1 for correct class and 0 for incorrect class (one-hot encoded).

We note that the formula of $\delta_i = \hat{y}_i - y_i$ does not give a satisfactory result when we compute for **Feed-Forward Neural Network for Iris Dataset Classification with 1 Hidden Layer** subsection under the feed-forward neural network section, thus we use this formula instead:

$$\delta_j^L = f'(s_j^L)\left(y_j - \hat{y}_j\right)$$

with $f'(s_j^L)$ as the Jacobian matrix and the $y_j - \hat{y}_j$ as the error for class j, L is the last layer, the output layer. This formula gives amazing accuracy between 97% to 99%.

Tanh **(Hyperbolic Tangent)**
This activation function is often used in hidden layers of neural networks.
The output range: $(-1, 1)$.
It is defined as follow:

$$\tanh(x) = \frac{\sinh(x)}{\cosh(x)} = \frac{e^x - e^{-x}}{e^x + e^{-x}} \tag{10.5}$$

The derivative of *Tanh* is defined as

$$\frac{d}{dx}\left(\frac{e^x - e^{-x}}{e^x + e^{-x}}\right) = 1 - \tanh(x)^2 \tag{10.6}$$

Leaky ReLU
This activation function addresses the dying ReLU problem by allowing a small gradient for negative inputs, defined as

$$f(x) = \begin{cases} x, & \text{if } x > 0 \\ \alpha x, & \text{if } x \leq 0 \end{cases} \tag{10.7}$$

where α is a small positive constant (e.g., 0.01). This retains computational simplicity while ensuring neurons remain active. It is a useful alternative to ReLU when dying neurons are a concern.

In Parametric ReLU, it extends ReLU by making the slope α a learnable parameter during training. This flexibility allows the network to adaptively adjust the negative slope for

improved performance. It is often used in deeper architectures where a more adaptable activation is beneficial.

Exponential Linar Unit (ELU)
This activation function introduces smooth transitions for negative values. It mitigates the vanishing gradient problem and impoves gradient flow, particularly in deep networks. However, it has slightly higher computational costs and is used where faster convergence is desired. It is defined as

$$f(x) = \begin{cases} x, & \text{if } x > 0 \\ \alpha(e^x - 1), & \text{if } x \leq 0 \end{cases} \tag{10.8}$$

where $\alpha > 0$.

Swish
This activation function is a smooth, non-monotonic activation function that often outperforms ReLU in deep architectures like EfficientNet. It enhances gradient flow but comes with a slightly higher computation cost. Swish is commonly used in modern CNNs and transformer-based models. It is defined as

$$f(x) = x \cdot \sigma(x) = x \cdot \frac{1}{1 + e^{-x}} \tag{10.9}$$

Function	Range	Gradient	Advantages	Disadvantages	Use Cases
Sigmoid	$(0, 1)$	Small for extremes	Smooth, probabilistic output	Vanishing gradients	Output layer for binary tasks
Tanh	$(-1, 1)$	Small for extremes	Zero centered	Vanishing gradients	RNNs
ReLU	$[0, \infty)$	Constant (1 for $x > 0$)	Sparse, efficient	Dying ReLU problems	CNNs, general DNNs
Leaky ReLU	$(-\infty, \infty)$	Constant	Mitigates dying ReLU problem	Requires tuning of α	Variants of CNNs, DNNs

Table 10.1: *Comparison of activation functions.*

[SI*] **Backpropagation**
Backward propagation of errors (backpropagation) is a key algorithm used to train neural networks by minimizing the difference between predicted and actual outputs. It works by propagating errors backward through the network, using the chain rule of calculus to compute gradients and then iteratively updating the weights and biases. Combined with optimization techniques like gradient descent, backpropagation enables the model to reduce loss across epochs and effectively learn complex patterns from data.

Mathematicaly, "training" means minimizing an error function, the sum of the squaes of the errors in the outputs. Backpropagation corresponds to using a steepest-descent method for minimizing the error function. While useful for training networks, backpropagation has the

disadvantages that convergence is slow compared to Conjugate Gradient.

Minimization of a function of many variables is a mature field; there are numerous algorithms and more numerous variations of them. All are iterative, generating a sequence of sets of weights.

Backpropagation plays a critical role in how neural networks improve over time.

1. **Efficient Weight Update**
   It computes the gradient of the loss function with respect to each weight using the chain rule making it possible to update weights efficiently.
2. **Scalability**
   The backpropagation algorithm scales well to networks with multiple layers and complex architectures making deep learning feasible.
3. **Automated Learning**
   With backpropagation the learning process becomes automated and the model can adjust itself to optimize its performance.

The working of backpropagation:

1. **Forward Pass Work**
   In forward pass the input data is fed into the input layer. These inputs combined with their respective weights are passed to hidden layers. For example in a network with two hidden layers (h_1 and h_2) the output from h_1 serves as the input to h_2. Before applying an activation function, a bias is added to the weighted inputs.
   Each hidden layer computes the weighted sum of the inputs ($\sum_{i=1^n} x_i w_i$) then applies an activation function like ReLU (Rectified Linear Unit) to obtain the output. The output is passed to the next layer till reaches the output layer where an activation function such as softmax convers the weighted outputs into probabilities for classification.
2. **Error Calculation**
   The network's prediction is compared against the actual target value using a loss function (e.g., Mean Squared Error) to determine the "cost" or error.
3. **Backward Pass (Backpropagation)**
   The algorithm calculates the gradient of the loss function with respect to every weight in the network. It uses the chain rule to propagate the error from the output layer back to the input layer.
   In the backward pass the error (the difference between the predicted and actual output) is propagated back through the network to adjust the weights and biases. One common method for error calculation is the Mean Squared Error given by

   $$MSE = (y_i - \hat{y}_i)^2$$

   with y_i as the actual output, $\hat{y}_i$ is the predicted output, $i = 1, 2, \ldots, n$ and n is the number of classification of the outputs / possible outcome for the output.
   Once the eror is calculated the network adjust weights using gradients which are computed with the chain rule. These gradients indicate how much each weight and bias should be adjusted to minimize the error in the next iteration. The backward pass continues layer by layer ensuring that the network learns and improves its performance. The activation function through its derivative plays a crucial role in computing these gradients during backpropagation.

4. **Weight Update**
   An optimization algorithm, usually a gradient descent, adjusts the weights by adding or subtracting (depends on the loss function that is used) a portion of the gradient (scaled by a learning rate) to minimize the error

[SI*] **Gradient in Backpropagation**
In a neural network, the gradient is the partial derivative of the loss function (error) with respect to each individual weight and bias using the multivariable chain rule. We use the gradient descent algorithm to find the best parameters that minimize the error. Gradient represents the direction and magnitude of the change needed to reduce the model's prediction error.

The algorithm iteratively moves, step by step, in the direction of steepest descent, toward the minimum point, the step size is the learning rate (η). During propagation, the network calculates how much each weight contributed to the total error.

- **Direction**
  The sign of the gradient tells you whether to increase or decrease the weight to reduce loss.
- **Steepness**
  The magnitude (absolute value) of the gradient indicates how sensitive the error is to that specific weight.

After the gradient is calculated, weights are updated using an optimization algorithm, most commonly Gradient Descent. The standard weight update formula for layer L using the gradient descent rule is

$$w_{new}^{(L)} = w_{old}^{(L)} - (\eta \times \frac{\partial C}{\partial w^{(L)}})$$

with η as the learning rate, it is a positive scalar, we can use fraction of real number like 0.01, remember that learning rate controls the step size. Gradient is denoted by $\frac{\partial C}{\partial w}$, C is the cost function, it is calculated using the chain rule to propagate error, we start from the output layer back to the input layer.

The gradient for a weight connecting neuron i to neuron j is typically computed as

$$\textbf{Weight Gradient}: \frac{\partial C}{\partial w_{ij}^{(L)}} = \delta_j^{(L)} \times y_i^{(L-1)}$$

and for the bias gradient at layer L

$$\textbf{Bias Gradient}: \frac{\partial C}{\partial b_j^{(L)}} = \delta_j^{(L)}$$

$$z_j^{(L)} = \sum_i w_{ij}^{(L)} y_i^{(L-1)} + b_j^{(L)}$$

$$y_j^{(L)} = \sigma(z_j^{(L)})$$

We compute the δ from the output layer with this formula

$$\delta_j^{(L)} = \frac{\partial C}{\partial y_j^{(L)}} \oplus \sigma'(z_j^{(L)})$$

$\oplus$ represents the hadamard product, $\delta_j^{(L)}$ is the error term that represents the error of the receiving neuron (neuron j at layer L), which includes the derivative of its activation function, $\sigma'(z_j^{(L)})$. $y_i^{(L-1)}$ is the post-activation output value of the sending neuron (neuron i at layer $L-1$), while $y_j^{(L)}$ is the post-activation output value of the receiving neuron (neuron j at layer L).

For any previous hidden layer ($l < L$), we calculate the gradients by propagating the error backwards from the subsequent layer $(l+1)$, thus we compute δ for the hidden layer/s with this formula

$$\delta_i^{(l)} = \frac{\partial C}{\partial y_i^{(l)}} \oplus \sigma'(z_i^{(l)}) = \sum_j \left(w_{ij}^{(l+1)} \delta_j^{(l+1)} \right) \oplus \sigma'(z_i^l)$$

and the formula to compute the weight and bias gradient connecting neuron k to neuron i to neuron j for hidden layer/s to input layer

$$\textbf{Weight Gradient}: \frac{\partial C}{\partial w_{ki}^{(l)}} = \delta_i^{(l)} \times y_k^{(l-1)}$$

$$\textbf{Bias Gradient}: \frac{\partial C}{\partial b_i^{(l)}} = \delta_i^{(l)}$$

with $\delta_i^{(l)}$ as the error term that represents the error of the neuron i at layer l, which includes the derivative of its activation function, $\sigma'(z_i^{(l)})$. $y_k^{(l-1)}$ is the post-activation output value of the sending neuron (neuron k at layer $l-1$), while $y_i^{(l)}$ is the post-activation output value of the receiving neuron (neuron i at layer l). If l represents the first hidden layer then $l-1$ is the input layer and $y_k^{(l-1)}$ is the k-th input variable.

In a neural network, the "delta" (δ) represents the error gradient with respect to the pre-activation (logit) of a layer.

If gradients become too small, the network suffers from the vanishing gradient problem, where early layers stop learning because their weights are no longer updated.

The delta output layer of a feed-forward neural network uses the post-activation neuron value to calculate the error. We start from the output layer to compute delta.

Calculate Post-Activation Output (y_j)
First we calculate the weighted sum (pre-activation, z_j) and then apply the activation function σ:

$$z_j = \sum_i (w_{ij} \cdot x_i) + b_j$$

$$y_j = \sigma(z_j)$$

Calculate the Error (e_j)
After forward pass, determine the error of the final output using a loss function, the first one is the mean squared error that uses the error formula as the difference between the actual post-activation output y_j and the actual output t_j:

$$e_j = t_j - y_j$$

thus

$$L = (t_j - y_j)^2$$

The second one is Categorical Cross-Entropy, if the output layer uses softmax as the activation function (e.g., multi-class problem) then the loss function / error is

$$L = -\sum_{j=1}^{C} t_j \log(y_j)$$

with C as the total number of classes, t_j is the true value (either 1 or 0 for one-hot encoded labels), y_j is the predicted probability for j-th class. For this loss function because the true values are 0 for incorrect classes, the formula simplifies to the negative natural logarithm of the predicted probability for the correct class, so if the prediction (y_j value) is high then the loss is low, vice versa.

Formula to Compute the Output Delta (δ_j)
The delta at the output layer is the first delta we compute in backpropagation, and it is computed by multiplying the error by the derivative of the activation function with respect to the pre-activation sum z_j:

$$\delta_j^{Output\ Layer} = (t_j - y_j) \cdot \sigma'(z_j)$$

The pre-activation (z_j) is needed for calculating the derivative of the activation function $\sigma'(z_j)$, which adjusts the gradient to account for the specific shape of the activation curve. The post-activation output (y_j) is essential for calculating the actual error signal because it represents the network's final prediction.

Afterwards, the error term for a hidden unit is calculated by backpropagating the error from the next layer. This process relies on the chain rule to determine how much a hidden unit contributed to the total error at the output layer.

Formula to Compute the Hidden layer' Delta (δ_j)
For a hidden unit j in layer L (assuming L is not the output layer), the neuron-specific delta δ_j^L for hidden neuron j relies on the deltas of all the neurons in the next layer that it connects to. It is defined mathematically as the derivative of the activation function multiplied by the weighted sum of the subsequent layer's deltas, and it is defined by the following formula:

$$\delta_j^L = \sigma'(z_j^L) \sum_k \left(w_{kj}^{L+1} \cdot \delta_k^{L+1} \right) \tag{10.10}$$

with z_j^L as the pre-activation of the hidden neuron j at layer L, δ_k^{L+1} as the error term of unit k in the next layer $(L+1)$, w_{kj}^{L+1} as the weight connecting the hidden unit j (current layer) to unit k (next layer), $\sigma'(z_j^L)$ as the derivative of the activation function evaluated at the pre-activation, z_j^L, and $\sum_k \left(w_{kj}^{L+1} \cdot \delta_k^{L+1} \right)$ is a summation over all neurons k in the subsequent layer that receive input from neuron j, we can also say that this is the backpropagated error signal from the next layer. Remember that the pre-activation, z_j, is the weighted sum from the previous layer plus the bias

$$z_j^L = \sum_i (w_{ij}^{L-1} \cdot x_i^{L-1}) + b_j^L$$

x_i^{L-1} is the neurons of the previous layer.

The post-activation value of the hidden neuron is used as an input to the weights of the next layer. Therefore, it dictates how much influence neuron j has on the network's final error. While the pre-activation and derivative $\sigma'(z_j)$ evaluates how sensitive the hidden neuron's output is to changes in its pre-activation. Multiplying by this derivative ensures the gradient descent updates only adjust weights in a way that respects the non-linear boundaries of the activation function.

[SI*] **Train Data and Test Data**

Training data is used to teach a model, while test data is used to evaluate its performance on unseen information, ensuring it generalizes than just memorizing. Training data is generally larger (e.g., 80%), used for fitting model parameters, whereas test data (e.g., 20%) remains isolated until final assessment.

Training data teaches the model patterns, weights, or features, it happens first. Test data checks the model's accuracy on new data, preventing overfitting, it is the final step to determine model reliability.

Basically the whole dataset will be divided into training data with large portion and into testing data with smaller portion (80/20 or 70/30).

[SI*] **Epoch in a Neural Network**

An epoch in a neural network represents one complete, single pass of the entire training dataset through the algorithm-both forward and backward. It is a hyperparameter determining how many times the model updates its internal weights using all training samples. Multiple epochs are usually required to minimize error and improve learning.

The different from batch, is that a batch is a subset of the dataset used to update weights, an iteration is a one update step using a single batch. The relationship formula is

$$1 \text{ Epoch} = \frac{\text{Total Dataset Size}}{\text{Batch Size}} \text{ iterations}$$

An iteration here refers to a single step of training where the model processes one batch of data and updates its parameters (weights and biases). Each iteration includes two main parts:

1. **Forward Propagation**
   The input data in the batch is passed through the network to generate predictions, and the loss (error) is calculated.
2. **Backward Propagation**
   The gradients are calculated, and the model's parameters are updated to reduce the loss.

We usually use multiple epochs because a single epoch is rarely sufficient to capture complex patterns, so the model re-reads the data, often updating weights after every batch to refine its understanding. Too many epochs can cause the model to learn the noise in the training data rather than general patterns, leading to high training accuracy but low validation accuracy (overfitting). The optimal number of epochs is the balance between overfitting and underfitting. An example is if you have 60,000 images of handwriting for digits 0-9 and a batch size of 500, it takes 120 iterations $\left(\frac{60{,}000}{500}\right)$ to complete one epoch.

[SI*] **Confusion Matrix**

A confusion matrix for a neural network is a table used to evaluate classification model performance by comparing predicted labels against true labels. It highlights True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN), enabling identification of misclassifications, particularly for binary or multi-class classification problems.
The diagonal elements of the confusion matrix represent correct predictions (actual=predicted) while off-diagonal elements indicate errors.
The formula to compute the accuracy of the neural network is

$$\frac{TP + TN}{TP + TN + FP + FN}$$

The formula to compute the precision of the neural network is

$$\frac{TP}{TP + FP}$$

The formula to compute the recall/sensitivity of the neural network is

$$\frac{TP}{TP + FN}$$

The formula to compute the F1-score we need to compute the harmonic mean of precision and recall.

Table (10.2) is an example of a model analyzing 100 Kohanku Koi fishes:

- Actual Aeromonas (the number of fishes that is infected with Aeromonas Hydrophilia): 50
- Actual Not Aeromonas (the number of fishes that is not infected with Aeromonas Hydrophilia): 50

	Predicted: Aeromonas	Predicted: Not Aeromonas
Actual: Aeromonas	40(TP)	10(FN)
Actual: Not Aeromonas	5(FP)	45(FN)

Table 10.2: *Binary confusion matrix example.*

The confusion matrix is essential for unbalanced datasets (e.g., detecting a disease where 99% of fishes are healthy), as it reveals if the model is ignoring the minor class, which high accuracy alone might hide.

[SI*] **Conjugate Gradient in Backpropagation**

Previously we are talking about steepest descent / gradient descent method for finding the optimal weights in neural network, there is another method called Conjugate Gradient and it is known to train the neural networks faster than the steepest descent method. The Conjugate Gradient is a line-search optimization technique that improves upon steepest descent by generating conjugate search directions, reducing the "zigzagging" behavior and ensuring faster convergence. In the context of training a neural network using the Conjugate Gradient (CG) method, the algorithm minimizes the error (cost function) by treating it as

a local quadratic approximation. It solves for the optimal weights using the linear system $A\boldsymbol{x} = \boldsymbol{b}$.

1. Vector $\boldsymbol{x}$ (The Weights)
   This is the vector of all trainable parameters (weights and biases) in the neural network, flattened into a single one-dimensional vector, typically denoted as w or θ in literatire. The goal is to find the optimal x that minimizes the error function $E(x)$.
2. Matrix A (The Hessian)
   The Hessian matrix (H or $\nabla^2 E$) of the neural network's cost function, which contains all partial derivatives of the second order (second-order curvature). It describes the shape of the error surface, determining how the gradient changes. For conjugate gradient to work ideally, A must be symmetric and positive-definite (locally true near a minimum). In neural network training, the full Hessian is rarely explicitly computed due to its massive size ($N \times N$, where N is the number of parameters). Instead, the algorithm uses products of the Hessian with a vector, such as $Ap \approx \nabla^2 \cdot p$.
3. Vector $\boldsymbol{b}$ (The Gradient-related term)
   A vector related to the gradient of the error function, frequently derived from the gradient $\nabla E(x)$.
   If using a pure linear system $A\boldsymbol{x} = \boldsymbol{b}$ for a quadratic error, b is a constant vector. In nonlinear optimization (neural network training), this represents components of the gradient, and the residual $\boldsymbol{r} = \boldsymbol{b} - A\boldsymbol{x}$ is replaced by the negative gradient of the error function ($-\nabla E(x)$).

Since neural network error surfaces are not strictly quadratic, non-linear conjugate gradient methods are used instead of solving the strict linear system.
Instead of calculating A (the Hessian), these algorithms compute the new search direction d_k as a linear combination of the current negative gradient and the previous search direction.

$$\boldsymbol{d}_k = -\nabla E(k) + \beta_k \boldsymbol{d}_{k-1}$$

where β_k is a scalar parameter, from now on we will denote $\nabla E(k)$ as $\boldsymbol{g}_k$

$$\beta_k^{PRP} = \frac{\boldsymbol{g}_k^T(\boldsymbol{g}_k - \boldsymbol{g}_{k-1})}{||\boldsymbol{g}_{k-1}||^2}, \quad \text{(Polak-Ribiere)}$$

if $\beta_k^{PRP} < 0$, it is often set to 0 to restart the algorithm, which helps avoid slow convergence. The Polak-Ribiere is very useful for unconstrained optimization and neural network training. It is often more efficient than Fletcher-Reeves method, because it has automatic restart that avoid jamming while optimizing.

$$\beta_k^{FR} = \frac{\boldsymbol{g}_k^T \boldsymbol{g}_k}{||\boldsymbol{g}_{k-1}||^2}, \quad \text{(Fletcher-Reeves)}$$

$$\beta_k^{DY} = \frac{\boldsymbol{g}_k^T \boldsymbol{g}_k}{\boldsymbol{d}_{k-1}^T(\boldsymbol{g}_k - \boldsymbol{g}_{k-1})}, \quad \text{(Dai-Yuan)}$$

$$\beta_k^{CD} = -\frac{\boldsymbol{g}_k^T \boldsymbol{g}_k}{\boldsymbol{d}_{k-1}^T \boldsymbol{g}_{k-1}}, \quad \text{(Conjugate Descent)}$$

The Conjugate Gradient loop / algorithm:

1. Initialize weights and biases randomly.
2. Calculate the gradient $\frac{\partial E}{\partial w}$ for all weights and bias, concatenated into a single vector $\boldsymbol{g}$. The formula is
$$\boldsymbol{g}_k = \nabla f(\boldsymbol{w}_k)$$
it is calculated using backpropagation on the feed-forward neural network, where f is the loss function.
3. Initialize $\boldsymbol{d}_0 = -\boldsymbol{g}_0$, the first step is steepest descent.
4. Iterate:
   - Find optimal η_k / the learning rate with strong Wolfe line search. The line search method determines the optimal step size (η_k) along the search direction (d_k) to minimize loss function $f(w_k + \eta_k d_k)$ and it is crucial for ensuring stability. It is determined by minimizing $f(\boldsymbol{w}_k + \eta_k \boldsymbol{d}_k)$.
   $$f(\boldsymbol{w}_k + \eta_k \boldsymbol{d}_k) = \min_{\eta \geq 0} f(\boldsymbol{w}_k + \eta \boldsymbol{d}_k)$$
   it means that the orthogonality condition
   $$\boldsymbol{g}_k^T \boldsymbol{d}_{k-1} = 0$$
   is satisfied, where $\boldsymbol{g}_k$ represents the gradient of the objective function f at the value $\boldsymbol{w}_k$. Since it is difficult in practice to compute η_k, the inexact line search is introduced to compute approximate values for η_k, thus we will use strong Wolfe line search. The strong Wolfe conditions requires η_k to satisfy the following two conditions [24]:
   $$f(\boldsymbol{w}_k + \eta_k \boldsymbol{d}_k) \leq f(\boldsymbol{w}_k) + \delta \eta_k \boldsymbol{g}_k^T \boldsymbol{d}_k \quad \text{(sufficient decrease)}$$
   $$|g(\boldsymbol{w}_k + \eta_k \boldsymbol{d}_k)^T \boldsymbol{d}_k| \leq \sigma |\boldsymbol{g}_k^T \boldsymbol{d}_k| \quad \text{(strong curvature condition)}$$
   where $0 < \delta < \sigma < 1$, and $\boldsymbol{d}_k$ is the search direction, which is given by
   $$\boldsymbol{d}_k = \begin{cases} -\boldsymbol{g}_k, & \text{if } k = 0 \\ -\boldsymbol{g}_k + \beta_k \boldsymbol{d}_{k-1}, & \text{if } k \geq 1 \end{cases}$$
   The factor β_k determines how the CG methods differ, it is calculated by the algorithm to make d_k and d_{k-1} conjugate, or orthogonal. The β_k can be an indication if there is numerical instability or a poor search direction / small steps (the new gradient g_k almost similar to the old gradient g_{k-1}) occurs while iterating, if β_k produces large values then there might be something wrong and we have to change our β_k formula since it will cause the training to fail or stall. Periodically resetting β_k to zero ensures the algorithm does not stay trapped in a poor, large-gradient region, this is called reverting to the steepest descent direction ($d_{k=-g_k}$).
   - Update weights
   $$\boldsymbol{w}_{k+1} = \boldsymbol{w}_k + \eta_k \boldsymbol{d}_k$$
   - Calculate new gradient $\boldsymbol{g}_{k+1}$.
   - Compute update coefficient β_k (e.g. with Polak-Ribiere or Fletcher-Reeves).
   - Calculate new direction
   $$\boldsymbol{d}_{k+1} = -\boldsymbol{g}_{k+1} + \beta_k \boldsymbol{d}_k, \quad k = 0, 1, \ldots$$

5. Run until maximum iterations / last epoch or target error is reached.

Note: Moller uses a (temporary) small value for η_k, does a function evaluation in order to approximate the second derivative in the search direction, then uses the approximate second derivative to select a final η_k; this uses two function calls. (If the new weights result in an increased error, they are rejected and some parameters are adjusted until the new weights reduce the error. In this case, a successful iteration takes more than two function calls. [16]).

Instead of backtracking like gradient descent, Conjugate Gradient creates "conjugate" directions that do not "undo" previous minimization steps, resulting in faster convergence. And unlike Newton's method, Conjugate Gradient does not require storing the Hessian matrix, making it efficient for large-scale feed-forward neural networks.

When the conjugate gradient method in a neural network stalls, it is typically due to a stuck search direction, a suboptimal line search, or a vanishing gradient problem. Unlike gradient descent, conjugate gradient needs precise line searches, is highly sensitive to the line search algorithm used to calculate the learning rate / step size η_k. If the loss is flat, it stops converging.

When a feed-forward neural network trained with a conjugate gradient algorithm consistently predicts only one class (for multi-class problem), it usually means the network is trapped in a local minimum, the learning rate is mismatched (check the line search algorithm, you should use the Wolfe conditions, if the learning rate is too large the CG method will diverge or get stuck, if it is too small the CG algorithm will make negligible progress and might just predict the majority class), or the architecture is incorrectly configured for multi-class classification.

To fix the local minimum trap, we need to make sure that the output layer is using softmax as the activation function and the loss function being used is categorical cross-entropy rather than binary cross-entropy. Don't forget for multi-class prolem we need to use one-hot encoding for the actual output. Avoid using sigmoid activation function, which causes vanishing gradients, use ReLU activation function instead in the hidden layers.

Most common causes and solutions for conjugate gradient stagnation:

- **Invalid or Inaccurate Line Search**
  Conjugate gradient relies on finding an optimal step size (η_k) to move along the conjugate direction (d_k). If the line search method (e.g. backtracking, secant) failes to find a good η_k, the algorithm stalls. If the step size is too large, the "conjugate" property of the search directions is lost, and the algorithm may oscillate or diverge just like standard stochastic gradient descent.
  To fix this, use a more robust line search method that satisfies the Wolfe conditions to ensure sufficient decrease.
- **Stuck in Saddle Points**
  The optimizer may have reached a saddle point where the gradient is zero or near-zero, but it is not a local minimum.
  Normalize the inputs for weights and bias to each layer to have a mean of 0 and variance of 1. This keeps gradients in a healthy range and speeds up convergence.
- **Vanishing Gradients**
  Deep networks can cause gradients to become negligible, resulting in tiny or no weight updates.

If the network is very deep, utilize residuals (skip connections) to mitigate vanishing gradients or avoid using sigmoid or tanh activation function and used LeakyReLU or ReLU instead.

- **Loss of Conjugacy**
  Over many iterations, or due to non-convexity, the directions might lose their conjugacy. Resetting the conjugate gradient direction to the negative gradient (steepest descent) periodically can resolve this.
  Reset the search direction to the steepest descent direction every N iterations (N is typically the number of parameters, the weights and bias).
- **Oscillating Loss**
  An oscillating loss during epoch training with a conjugate gradient optimizer and ReLU activation function is usually caused by line search failures, non-convex gradient discontinuities (dying ReLUs), or an excessive learning rate.

  Conjugate gradient heavily relies on exact or inexact line searches to determine the optimal step size. Because ReLU introduces sharp, non-convex boundaries, the function is piecewise linear. Standard line searches often fail to bracket the minimum, causing erratic jumps.
  Ensure the line search algorithm satisfies the strong Wolfe conditions. If already does, tighten the tolerance parameters (e.g., lower the σ parameter in the curvature condition) to prevent the CG method from overshooting the optimal step in the parameter space.

  The standard ReLU derivative is discontinuous. When a large batch of neurons "dies" (outputs 0), the gradient drops abruptly, causing the CG direction vectors to break orthogonality. The optimizer then "bounces" back and forth across these kinks. Use leaky ReLU or GELU instead to solve this. They provide a continuous, non-zero gradient across negative inputs, smoothing out the loss landscape and stabilizing the conjugate search directions.

  If the initial learning rate or step bound is too large, CG will step outside the valid quadratic approximation of the loss surface. It then tries to correct itself backward on the next step, resulting in a zig-zagging or oscillating epoch loss.
  Manually decrease the learning rate or the maximum step size. Test a value 10 times to 100 times smaller than the current setup to see if the oscillation smooths out.
- **Impoper Initialization**
  Poorly initialized weights and bias may land the model in a flat region of the loss landscape immediately.
  Use Xavier/Glorot if you use tanh or sigmoid activation function or He initialization if you use ReLU activation function to ensure gradients are health.

The conjugate gradient that we are talking here is the nonlinear conjugate gradient that use inexact line search (strong Wolfe conditions), it is best for large-scale neural network training where exact search is too slow. Another type of conjugate gradient is scaled conjugate gradient with Levenberg-Marquardt approach to compute the step size, it is highly efficient for neural network training without manual tuning.

We use conjugate gradient when we have small to medium-size networks where rapid convergence is needed and the loss function is well-behaved (not extremely noisy). If we have very large datasets where calculating exact gradient at every step is too costly, then we

use steepest descent.

Applications of conjugate gradient:

1. Solving large sparse linear systems in computational physics and engineering.

2. Image reconstruction for medical imaging (CT scan, MRI).

3. Neural network training.

4. Structural analysis with finite element methods.

5. Computational fluid dynamics.

Moreover, their favorable properties, such as global convergence and low memory requirements, have qualified them to solve problems in diverse areas of science, such as data estimation, image restoration, and signal processing.

II. DataFrame in C++ for Neural Networks

[SI*] Neural networks have become a vast field with many applications in supervised machine learning. Supervised learning means that the input cases and the output cases are known and that their functional relationship is to be learned while the data may be noisy.

As the first stepping stone, we will need to be able to load the data, in this book we will use a simple **.csv** file that separates the column with comma (",") and the row with line break. The data in the **.csv** file is what we call dataframe. The popular programming language like Python, JULIA and R have packages to handle dataframe easily, with C++ we are not only gaining more speed, but also more power and flexibility that are future-proof for the future Artificial Intelligence, Machine Learning, and beyond this.

i. Loading Dataframe with Iris Dataset

Suppose we have a CSV-formatted data, with name of the file as "iris.csv" that classify iris flowers, this is the iris dataset. It contains 150 samples of iris flowers, with 4 features each

- Sepal length (in cm)
- Sepal width (in cm)
- Petal length (in cm)
- Petal width (in cm)

Each sample is labeled with one of three species

- Iris Setosa
- Iris Versicolor
- Iris Virginica

```
5.1,3.5,1.4,.2,"Setosa"
4.9,3,1.4,.2,"Setosa"
4.7,3.2,1.3,.2,"Setosa"
4.6,3.1,1.5,.2,"Setosa"
5,3.6,1.4,.2,"Setosa"
5.4,3.9,1.7,.4,"Setosa"
4.6,3.4,1.4,.3,"Setosa"
5,3.4,1.5,.2,"Setosa"
4.4,2.9,1.4,.2,"Setosa"
4.9,3.1,1.5,.1,"Setosa"
5.4,3.7,1.5,.2,"Setosa"
4.8,3.4,1.6,.2,"Setosa"
4.8,3,1.4,.1,"Setosa"
4.3,3,1.1,.1,"Setosa"
5.8,4,1.2,.2,"Setosa"
5.7,4.4,1.5,.4,"Setosa"
5.4,3.9,1.3,.4,"Setosa"
5.1,3.5,1.4,.3,"Setosa"
5.7,3.8,1.7,.3,"Setosa"
5.1,3.8,1.5,.3,"Setosa"
5.4,3.4,1.7,.2,"Setosa"
```

```
5.1,3.7,1.5,.4,"Setosa"
4.6,3.6,1,.2,"Setosa"
5.1,3.3,1.7,.5,"Setosa"
4.8,3.4,1.9,.2,"Setosa"
5,3,1.6,.2,"Setosa"
...
...
6.7,3.1,5.6,2.4,"Virginica"
6.9,3.1,5.1,2.3,"Virginica"
5.8,2.7,5.1,1.9,"Virginica"
6.8,3.2,5.9,2.3,"Virginica"
6.7,3.3,5.7,2.5,"Virginica"
6.7,3,5.2,2.3,"Virginica"
6.3,2.5,5,1.9,"Virginica"
6.5,3,5.2,2,"Virginica"
6.2,3.4,5.4,2.3,"Virginica"
5.9,3,5.1,1.8,"Virginica"
```

Code 74: *iris.csv*

From the data "iris.csv" above we can categorize the variables into 5 different variables / 5 columns of a matrix. We will have the variables in column vector-term like this:

$$\vec{x_1} \quad \vec{x_2} \quad \vec{x_3} \quad \vec{x_4} \quad \vec{x_5}$$

and

$$\begin{aligned} x_1 &= \text{sepal_length} \\ x_2 &= \text{sepal_width} \\ x_3 &= \text{petal_length} \\ x_4 &= \text{petal_width} \\ x_5 &= \text{species} \end{aligned}$$

The variables x_1, x_2, x_3, and x_4 are defined as **double** data type in C++, while the variable x_5 is defined as **string** data type.

The code below shows how to load a CSV with variables of these:

```
#include <iostream>
#include "symintegrationc++.h"
#include <bits/stdc++.h>
#include <cmath>
#include <chrono>

using namespace std::chrono;
using namespace std;
using namespace SymbolicConstant;

#include <iostream>
#include <fstream>
```

```
#include <vector>
#include <string>
#include <sstream>

struct IrisRecord
{
	double sepal_length, sepal_width, petal_length, petal_width;
	std::string species;
};

std::vector<IrisRecord> loadIris(const std::string& filename)
{
	std::vector<IrisRecord> data;
	std::ifstream file(filename);
	std::string line;

	while (std::getline(file, line))
	{
		if (line.empty()) continue;
		std::stringstream ss(line);
		std::string val;
		IrisRecord record;

		std::getline(ss, val, ','); record.sepal_length = std::stod(
		    val);
		std::getline(ss, val, ','); record.sepal_width = std::stod(
		    val);
		std::getline(ss, val, ','); record.petal_length = std::stod(
		    val);
		std::getline(ss, val, ','); record.petal_width = std::stod(
		    val);
		std::getline(ss, record.species, ',');

		data.push_back(record);
	}
	return data;
}

void showIrisVector(const std::vector<IrisRecord>& data)
{
	cout << "Species \t | Sepal length " << setw(10) << "| Sepal width "
	     << setw(10) << " | Petal length " << setw(10) << " | Petal
	    width " << endl;
	for (const auto& record : data)
	{
		cout << record.species
		<< " \t | \t " << record.sepal_length << " \t | \t " <<
		    record.sepal_width
```

```
                << " \t | \t " << record.petal_length << " \t | \t" << record
                    .petal_width
                << endl;
        }
    }


    // Driver code
    int main(int argc, char** argv)
    {
        // Get starting timepoint
        auto start = high_resolution_clock::now();

        std::vector<IrisRecord> data = loadIris("iris.csv");

        int n = data.size();
        showIrisVector(data);
        cout << "\nNumber of data: " << n << endl;

        // Get ending timepoint
        auto stop = high_resolution_clock::now();
        auto duration = duration_cast<microseconds>(stop - start);

        cout << "\nTime taken by function: " << duration.count() << "
            microseconds" << endl;

        return 0;
    }
```

Code 75: *Neural Networks/Test Iris DataFrame/main.cpp*

The basic needs when coding this is we know that each column represents which variable (and each variable is assigned to a certain suitable data type available in C++), we don't want to interchange the variable of petal length with sepal width, so we need to be sure which column represents which variable.

If we are able to load the data that contain not only data types of double, string, char, datetime, then we can then process it further with machine learning algorithms to recognize a pattern, or gain knowledge of the data.

- To "show" or print the contents of a std::vector<IrisRecord>, use a range-based **for** loop, which is the most concise method in modern C++.
- The most common method uses std::ifstream and std::getline to read the CSV-formatted data into vectors or arrays.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test Iris DataFrame ]# ./main
Species          | Sepal length  | Sepal width   | Petal length  | Petal width
"Setosa"         |       5.1     |       3.5     |       1.4     |       0.2
"Setosa"         |       4.9     |       3       |       1.4     |       0.2
"Setosa"         |       4.7     |       3.2     |       1.3     |       0.2
"Setosa"         |       4.6     |       3.1     |       1.5     |       0.2
"Setosa"         |       5       |       3.6     |       1.4     |       0.2
"Setosa"         |       5.4     |       3.9     |       1.7     |       0.4
"Setosa"         |       4.6     |       3.4     |       1.4     |       0.3
"Setosa"         |       5       |       3.4     |       1.5     |       0.2
"Setosa"         |       4.4     |       2.9     |       1.4     |       0.2
"Setosa"         |       4.9     |       3.1     |       1.5     |       0.1
"Setosa"         |       5.4     |       3.7     |       1.5     |       0.2
"Setosa"         |       4.8     |       3.4     |       1.6     |       0.2
"Setosa"         |       4.8     |       3       |       1.4     |       0.1
"Setosa"         |       4.3     |       3       |       1.1     |       0.1
"Setosa"         |       5.8     |       4       |       1.2     |       0.2
"Setosa"         |       5.7     |       4.4     |       1.5     |       0.4
"Setosa"         |       5.4     |       3.9     |       1.3     |       0.4
"Setosa"         |       5.1     |       3.5     |       1.4     |       0.3
"Setosa"         |       5.7     |       3.8     |       1.7     |       0.3
"Setosa"         |       5.1     |       3.8     |       1.5     |       0.3
"Setosa"         |       5.4     |       3.4     |       1.7     |       0.2
"Setosa"         |       5.1     |       3.7     |       1.5     |       0.4
"Setosa"         |       4.6     |       3.6     |       1       |       0.2
"Setosa"         |       5.1     |       3.3     |       1.7     |       0.5
"Setosa"         |       4.8     |       3.4     |       1.9     |       0.2
"Setosa"         |       5       |       3       |       1.6     |       0.2
"Setosa"         |       5       |       3.4     |       1.6     |       0.4
"Setosa"         |       5.2     |       3.5     |       1.5     |       0.2
"Setosa"         |       5.2     |       3.4     |       1.4     |       0.2
"Setosa"         |       4.7     |       3.2     |       1.6     |       0.2
"Setosa"         |       4.8     |       3.1     |       1.6     |       0.2
"Setosa"         |       5.4     |       3.4     |       1.5     |       0.4
"Setosa"         |       5.2     |       4.1     |       1.5     |       0.1
```

Figure 10.2: *The loading of "iris.csv" data and show it as a dataframe (**SymIntegration/Examples/Neural Networks/Test Iris DataFrame/main.cpp**).*

```
"Virginica"      |       6       |       2.2     |       5       |       1.5
"Virginica"      |       6.9     |       3.2     |       5.7     |       2.3
"Virginica"      |       5.6     |       2.8     |       4.9     |       2
"Virginica"      |       7.7     |       2.8     |       6.7     |       2
"Virginica"      |       6.3     |       2.7     |       4.9     |       1.8
"Virginica"      |       6.7     |       3.3     |       5.7     |       2.1
"Virginica"      |       7.2     |       3.2     |       6       |       1.8
"Virginica"      |       6.2     |       2.8     |       4.8     |       1.8
"Virginica"      |       6.1     |       3       |       4.9     |       1.8
"Virginica"      |       6.4     |       2.8     |       5.6     |       2.1
"Virginica"      |       7.2     |       3       |       5.8     |       1.6
"Virginica"      |       7.4     |       2.8     |       6.1     |       1.9
"Virginica"      |       7.9     |       3.8     |       6.4     |       2
"Virginica"      |       6.4     |       2.8     |       5.6     |       2.2
"Virginica"      |       6.3     |       2.8     |       5.1     |       1.5
"Virginica"      |       6.1     |       2.6     |       5.6     |       1.4
"Virginica"      |       7.7     |       3       |       6.1     |       2.3
"Virginica"      |       6.3     |       3.4     |       5.6     |       2.4
"Virginica"      |       6.4     |       3.1     |       5.5     |       1.8
"Virginica"      |       6       |       3       |       4.8     |       1.8
"Virginica"      |       6.9     |       3.1     |       5.4     |       2.1
"Virginica"      |       6.7     |       3.1     |       5.6     |       2.4
"Virginica"      |       6.9     |       3.1     |       5.1     |       2.3
"Virginica"      |       5.8     |       2.7     |       5.1     |       1.9
"Virginica"      |       6.8     |       3.2     |       5.9     |       2.3
"Virginica"      |       6.7     |       3.3     |       5.7     |       2.5
"Virginica"      |       6.7     |       3       |       5.2     |       2.3
"Virginica"      |       6.3     |       2.5     |       5       |       1.9
"Virginica"      |       6.5     |       3       |       5.2     |       2
"Virginica"      |       6.2     |       3.4     |       5.4     |       2.3
"Virginica"      |       5.9     |       3       |       5.1     |       1.8

Number of data: 150

Time taken by function: 6610 microseconds
```

Figure 10.3: *The loading of "iris.csv" data and show it as a dataframe (**SymIntegration/Examples/Neural Networks/Test Iris DataFrame/main.cpp**).*

III. Supervised Learning: Feed-Forward Neural Network (Multilayer Perceptron)

[SI*] This is the first category of machine learning, the supervised learning which uses labeled data to train models that can predict outcomes for new data (e.g., classification or regression). A feed-forward neural network (FNN) is the simplest type of artificial neural networks where information moves in only one direction (forward flow) from input nodes, through hidden nodes (if any), to output nodes. It has no loops or cycles, the data never moves backward. It is used for pattern recognition, classification, and regression tasks like credit scoring or speech recognition. Other notable examples of the FNN usage are image recognition, financial forecasting, data transformation.

The feed-forward neutral network has a structured layered design where data flows sequentially through each layer.

1. **Input Layer**
   The input layer consists of neurons that receive the input data. Each neuron in the input layer represents a feature of the input data.
2. **Hidden Layers**
   One or more hidden layers are placed between the input and output layers. These layers are responsible for learning the complex patterns in the data. Each neuron in a hidden layer applies a weighted sum of inputs followed by a non-linear activation function.
3. **Output Layer**
   The output layer provides the final output of the network. The number of neurons in this layer corresponds to the number of classes in a classification problem or the number of outputs in a regression problem.
   For classification problem, we use Softmax as the activation function from the output layer to determine the classification result.

In FNN, the neurons are connected by the corresponding weights, and non-linear activation functions (like Sigmoid or ReLU) are used in hidden layers. The weights are updated / adjusted to reduce prediction error using techniques like gradient descent, often through backpropagation. The process for training a feed-forward neural network:

1. **Forward Propagation**
   During forward propagation the input data passes through the network and the output is calculated to generate a prediction. For classification, the output layer typically produces a set of scores or probabilities that represent the likelihood of the input belonging to specific classes.

   The core mechanism of forward propagation:

   - **Weighted Sum (z)**
     We define the weighted sum with the notation of z and the formula is

     $$\boldsymbol{z} = W^L \boldsymbol{a}^{(L-1)} + \boldsymbol{b}^{(L)}$$

     with $\boldsymbol{z}$ as the weighted sum that is already added with bias $\boldsymbol{b}$ at layer L, we define $\boldsymbol{z}, \boldsymbol{a}, \boldsymbol{b}$ as vectors and W as a matrix of size $m \times n$ that represents the weights from the n nodes of the current layer to the m nodes of the next layer, the size of the matrix and vectors have to match because they will become the next layer' weighted sum vector that will be activated with activation function.

- **Activation(a)**
  The activation function is denoted with this formula

$$\boldsymbol{a}^{(L)} = \sigma(\boldsymbol{z}^{(L)})$$

  where σ is a non-linear activation function like sigmoid, tanh, ReLU, or softmax.

The final layer's configuration depends on the type of classification:

- **Binary Classification**
  Uses a single neuron with a sigmoid activation. An output of greater than 0.5 typically indicates the positive class.
- **Multi-class Classification**
  Uses multiple neurons (one per class) with a softmax activation. Softmax converts raw scores into a probability distribution that sums to 1.
- **Result Interpretation**
  The predicted class is usually the index of the neuron with the highest probability score.

2. **Loss Calculation**
   The loss (or error) is calculated using a loss function such as Mean Squared Error (MSE) for regression tasks or Cross-Entropy Loss for classification tasks.
3. **Backpropagation**
   In backpropagation the error is propagated back through the network to update the weights. The gradient of the loss function with respect to each weight is calculated and the weights are adjusted using gradient descent.

Gradient descent is an optimization algorithm used to minimize the loss function by iteratively updating the weights in the direction of the negative gradient. Common variants of gradient descent include:

1. **Batch Gradient Descent**
   Updates weights after computing the gradient over the entire dataset.
2. **Stochastic Gradient Descent**
   Updates weights for each training example individually.
3. **Mini-batch Gradient Descent**
   It updates weights after computing the gradient over a small batch of training examples.

Evaluating the performance of the feed-forward neural network trained model involves several metrics:

1. **Accuracy**
   The proportion of correctly classified instances out of the total instances.
2. **Precision**
   The ratio of true positive predictions to the total predicted positives.
3. **Recall**
   The ratio of true positive predictions to the actual positives.
4. **F1 Score**
   The harmonic mean of precision and recall, providing a balance between the two.
5. **Confusion Matrix**
   A table used to describe the performance of a classification model, showing the true positives, true negatives, false positives and false negatives.

Improving Convergence

Gradient descent has various advantages. It is simple, local; namely, the change in a weight uses only the values of the presynaptic and postsynaptic units and the error (suitably backpropagated). When on-line training is used, it does not need to store the training set and can adapt as the task to be learned changes. But gradient descent converges slowly, there are other methods that has faster convergence, e.g. conjugate gradient and second-order methods to the training of multilayer perceptrons.

Overtraining

A feed-forward neural network (multilayer perceptron) with d inputs, H neurons in hidden layer (hidden units), and K outputs has $H(d+1)$ weights in the first layer (the $+1$ is for bias) and $K(H+1)$ weights in the second layer. Both the space and time complexity of an MLP is $O(H \cdot (K+d))$. When e denotes the number of training epochs, training time complexity is $O(e \cdot H \cdot (K+d))$.

In an application, d and K are predefined and H is the parameter that we play with to tune the complexity of the model. An overcomplex model memorizes the noise in the training set and does not generalize to the validation set. For example, in the case of polynomial regression we notices that in the presence of noise or small samples, increasing the polynomial order leads to worse generalization. Similarly in an MLP, when the number of hidden units is large, the generalization accuracy deteriorates, and the bias/variance dillema also holds for the MLP, as it does for any statistical estimator [2].

Structuring the Network

In some applications we know that the input has a local structure. For example, in computer vision we know that nearby pixels are correlated and there are local features like edges and corners; any object, for example, a handwritten digit, may be defined as a combination of such primitives. Similarly, in speech recognition, locality is in time and inputs close in time can be grouped as speech primitives. By combining these primitives, longer utterances, for example, speech phonemes, may be defined. In such a case when designing the MLP, hidden units are not connected to all input units because not all inputs are correlated. Instead, we define hidden units that define a window over the input space and are connected to only a small local subset of the inputs. This decreases the number of connections and therefore the number of free parameters.

We can repeat this in successive layers where each layer is conencted to a small number of local units below and checks for a more complicated feature by combining the features below in a larger part of the input space until we get to the output layer.

For example, the input may be pixels. By looking at pixels, the first hidden layer units may learn to check for edges of various orientations. Then by combining edges, the second hidden layer units can learn to check for combinations of edges-for example, arcs, corners, line ends-and then combining them in upper layers, the units can look for semi-circles, rectangles, or in the case of a face recognition application, eyes, mouth, and so forth. This is the example of a hierarchical cone where features get more comples, abstract, and fewer in number as we go up the network until we get to classes. Such an architecture is called a convolutional neural network where the work of each hidden unit is considered to be a convolutional of its input with its weight vector.

[SI*] The feed-forward neural network is the most successful model for searching patterns in data [5]. This model comprises multiple layers of logistic regression models rather than multiple perceptrons. For many applications, the resulting model can be significantly more compact, and hence faster to evaluate, than a support vector machine having the same generalization performance. the price to be paid for this compactness, is that the likelihood

function, which forms the basis for network training, is no longer a convex function of the model parameters.

[SI*] The linear model that will be the basic for the feed-forward neural network has this form

$$y(\boldsymbol{x}, \boldsymbol{w}) = f\left(\sum_{j=1}^{M} w_j \phi_j(\boldsymbol{x})\right) \tag{10.11}$$

where $f(\cdot)$ is a nonlinear activation function in the case of classification and is the identity in the case of regression. The difference between a linear model is that we will make the basic functions $\phi_j(\boldsymbol{x})$ depend on parameters and then to allow these parameters to be adjusted, along with the coefficients $\{w_j\}$, during training.

This leads to the basic neural network model, which can be described a series of functional transformations. First we construct M linear combinations of the input variables $x_1, \ldots, x_D$ in the form

$$a_j = \sum_{i=1}^{D} w_{ji}^{(1)} x_i + w_{j0}^{(1)} \tag{10.12}$$

where $j = 1, 2, \ldots, M$, and the superscript (1) indicates that the corresponding parameters are in the first 'layer' of the network. With $w_{ji}^{(1)}$ as weights and the parameters $w_{j0}^{(1)}$ as biases. The quantities a_j are known as activations. Each of them is then transformed using a differentiable, nonlinear activation function $h(\cdot)$ to give

$$z_j = h(a_j) \tag{10.13}$$

These quantities correspond to the outputs of the basis functions in (10.11) that, in the context of neural networks, are called hidden units. The nonlinear functions $h(\cdot)$ are generally chosen to be sigmoidal functions such as the logistic sigmoid or the tanh function. Following (10.11), these values are again linearly combined to give output unit activations

$$a_k = \sum_{j=1}^{M} w_{kj}^{(2)} z_j + w_{k0}^{(2)} \tag{10.14}$$

where $k = 1, 2, \ldots, K$, and K is the total number of outputs. This transformation corresponds to the second layer of the network, and gain the $w_{k0}^{(2)}$ are bias parameters. Finally, the output unit activations are transformed using an appropriate activation function to give a set of network outputs y_k. The choice of activation function is determined by the nature of data and the assumed distribution of target variables. Thus for standard regression problems, the activation function is the identity so that $y_k = a_k$. Similarly, for multiple binary classification problems, each output unit activation is transformed using a logistic sigmoid function so that

$$y_k = \sigma(a_k) \tag{10.15}$$

where

$$\sigma(a) = \frac{1}{1 + e^{-a}} \tag{10.16}$$

For multiclass problem, a softmax activation function is used.

We can combine these various stages to give the overall network function that, for sigmoidal output unit activation, takes the form

$$y_k(\boldsymbol{x}, \boldsymbol{w}) = \sigma\left(\sum_{j=1}^{M} w_{kj}^{(2)} h\left(\sum_{i=1}^{D} w_{ji}^{(1)} x_i + w_{j0}^{(1)}\right) + w_{k0}^{(2)}\right) \tag{10.17}$$

where the set of all weight and bias parameters have been grouped together into a vector $\boldsymbol{w}$. Thus the neural network model is simply a nonlinear function from a set of inpute variables $\{x_i\}$ to a set of output variables $\{y_k\}$ controlled by a vector $\boldsymbol{w}$ of adjustable parameters.
The process of evaluating (10.17) can then be interpreted as a forward propagation or information through the network.

The bias parameters in (10.12) can be absorbed into the set of weight parameters by defining an additional input variable x_0 whose value is clamped at $x_0 = 1$, so that (10.12) takes the form

$$a_j = \sum_{i=0}^{D} w_{ji}^{(1)} x_i \tag{10.18}$$

We can similarly absorb the second-layer biases into the second-layer weights, so that the overall network function becomes

$$y_k(\boldsymbol{x}, \boldsymbol{w}) = \sigma\left(\sum_{j=0}^{M} w_{kj}^{(2)} h\left(\sum_{i=0}^{D} w_{ji}^{(1)} x_i\right)\right) \tag{10.19}$$

The neural network model comprises two stages of processing, each of which resembles the perceptron model, and for this reason the feed-forward neural network is also known as the multilayer perceptron (MLP). A key difference compared to the perceptron, however, is that the neural network uses continuous sigmoidal nonlinearities in the hidden units, whereas the perceptron uses step-function nonlinearities. This means that the neural network function is differentiable with respect to the network parameters, and this property will play a central role in network training.
If the activation functions of all the hidden units in a network are taken to be lienar, then for any such network we can always find an equivalent network without hidden units. This follows from the fact that the composition of successive linear transformations is itself a linear transformation. However, if the number of hidden units is smaller than either the number of input or output units, then the transformations that the network can generate are not the most general possible linear transformations from inputs to outputs because information is lost in the dimensionality reduction at the hidden units.

[SI*] **Network Training**

We have viewed neural networks as a general class of parametric nonlinear functions from a vector $\boldsymbol{x}$ of input variables to a vector $\boldsymbol{y}$ of output variables. A simple approach to the problem of determining the network parameters is to make an analogy with the discussion of polynomial curve fitting. Given a training set comprising a set of input vectors $\{\boldsymbol{x}_n\}$, where $n = 1, 2, \ldots, N$, together with a corresponding set of target vectors $\{\boldsymbol{t}_n\}$, we minimize the error function

$$E(\boldsymbol{w}) = \frac{1}{2} \sum_{n=1}^{N} ||\boldsymbol{y}(\boldsymbol{x}_n, \boldsymbol{w}) - \boldsymbol{t}_n||^2 \tag{10.20}$$

However, we can provide a much more general view of network training by first giving a probabilistic interpretation to the network outputs.

Consider a single target variable t that can take any real value. We assume that t has Gaussian distribution with an $\boldsymbol{x}$-dependent mean, which is given by the output of the neural network, so that

$$p(t|\boldsymbol{x}, \boldsymbol{w}) = N\left(t|y(\boldsymbol{x}, \boldsymbol{w}), \beta^{-1}\right) \tag{10.21}$$

where β is the precision (inverse variance) of the Gaussian noise. For the conditional distribution given by (10.21), it is sufficient to take the output unit activation function to be the identity, because such a network can approximate any continuous function from $\boldsymbol{x}$ to y. Given a data set of N independent, identically distributed observations $\boldsymbol{X} = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_N\}$, along with corresponding target values $\boldsymbol{t} = \{t_1, t_2, \ldots, t_N\}$, we can construct the corresponding likelihood function

$$p(\boldsymbol{t}|\boldsymbol{X}, \boldsymbol{w}, \beta) = \prod_{n=1}^{N} p(t_n|\boldsymbol{x}_n, \boldsymbol{w}, \beta)$$

Taking the negative logarithm, we obtain the error function

$$\frac{\beta}{2}\sum_{n=1}^{N}\{y(\boldsymbol{x}_n, \boldsymbol{w}) - t_n\}^2 - \frac{N}{2}\ln\beta + \frac{N}{2}\ln(2\pi) \tag{10.22}$$

which can be used to learn the parameters $\boldsymbol{w}$ and β. Here we consider the maximum likelihood approach of neural networks, it is usual to consider the minimization of an error function rather than the maximization of the (log) likelihood.

Now, consider first the determination of $\boldsymbol{w}$. Maximizing the likelihood function is equivalent to minimizing the sum-of-squares error function given by

$$E(\boldsymbol{w}) = \frac{1}{2}\sum_{n=1}^{N}\{y(\boldsymbol{x}_n, \boldsymbol{w}) - t_n\}^2 \tag{10.23}$$

where we have discarded additive and multiplicative constants. The value of $\boldsymbol{w}$ found by minimizing $E(\boldsymbol{w})$ will be denoted by $\boldsymbol{w}_{ML}$ because it corresponds to the maximum likelihood solution. In practice, the nonlinearity of the network function $y(\boldsymbol{x}_n, \boldsymbol{w})$ causes the error $E(\boldsymbol{w})$ to be nonconvex, and so in practice local maxima of the likelihood may be found, corresponding to local minima of the error function.
Having found $\boldsymbol{w}_{ML}$, the value of β can be found by minimizing the negative log likelihood to give

$$\frac{1}{\beta_{ML}} = \frac{1}{N}\sum_{n=1}^{N}\{y(\boldsymbol{x}_n, \boldsymbol{w}_{ML}) - t_n\}^2 \tag{10.24}$$

Note that this can be evaluated once the iterative optimization required to find $\boldsymbol{w}_{ML}$ is completed. If we have multiple target variables, and we assume that they are independent conditional on $\boldsymbol{x}$ and $\boldsymbol{w}$ with shared noise precision β, then the conditional distribution of the target values is given by

$$p(\boldsymbol{t}|\boldsymbol{x}, \boldsymbol{w}) = N(\boldsymbol{t}|\boldsymbol{y}(\boldsymbol{x}, \boldsymbol{w}), \beta^{-1}\boldsymbol{I}) \tag{10.25}$$

Following the same argument as for a single target variable, we see that the maximum likelihood weights are determined by minimizing the sum-of-squares error function (10.20). The noise precision is then given by

$$\frac{1}{\beta_{ML}} = \frac{1}{NK}\sum_{n=1}^{N}||\boldsymbol{y}(\boldsymbol{x}_n, \boldsymbol{w}_{ML}) - \boldsymbol{t}_n||^2 \tag{10.26}$$

where K is the number of target variables. The assumption of independence can be dropped at the expense of a slightly more complex optimization problem.
There is a natural pairing of the error function (given by the negative log likelihood) and the output unit activation function. In the regression case, we can view the network as having an output activation function that is the identity, so that $y_k = a_k$. The corresponding sum-of-squares error function has the property

$$\frac{\partial E}{\partial a_k} = y_k - t_k \tag{10.27}$$

which we shall make use when discussing error backpropagation.
Now consider the case of binary classification in which we have a single target variable t such that $t = 1$ denotes class C_1 and $t = 0$ denotes class C_2. We consider a network having a single output whose activation function is a logistic sigmoid

$$y = \sigma(a) = \frac{1}{1 + e^{-a}} \tag{10.28}$$

so that $0 \leq y(\boldsymbol{x}, \boldsymbol{w}) \leq 1$. We can interpret $y(\boldsymbol{x}, \boldsymbol{w})$ as the conditional probability $p(C_1|\boldsymbol{x})$, with $p(C_2|\boldsymbol{x})$ given by $1 - y(\boldsymbol{x}, \boldsymbol{w})$. The conditional distribution of targets given inputs is then a Bernoulli distribution of the form

$$p(t|\boldsymbol{x}, \boldsymbol{w}) = y(\boldsymbol{x}, \boldsymbol{w})^t \{1 - y(\boldsymbol{x}, \boldsymbol{w})\}^{1-t} \tag{10.29}$$

If we consider a training set of independent observations, then the error function, which is given by the negative log likelihood, is then a cross-entropy error function of the form

$$E(\boldsymbol{w}) = -\sum_{n=1}^{N} \{t_n \ln(y_n) + (1 - t_n) \ln(1 - y_n)\} \tag{10.30}$$

where y_n denotes $y(\boldsymbol{x}_n, \boldsymbol{w})$. Note that there is no analogue of the noise precision β because the target values are assumed to be correctly labelled. However, the model is easily extended to allow for labelling errors. Simard *et al.* (2003) found that using the cross-entropy error function instead of the sum-of-squares for a classification problem leads to faster training as well as improved generalization.
If we have K separate binary classifications to perform, then we can use a network having K outputs each of which has a logistic sigmoid activation function. Associated with each output is a binary class label $t_k \in \{0, 1\}$), where $k = 1, 2, \ldots, K$. If we assume that the class labels are independent, given the input vector, then the conditional distribution of the targets is

$$p(\boldsymbol{t}|\boldsymbol{x}, \boldsymbol{w}) = \prod_{k=1}^{K} y_k(\boldsymbol{x}, \boldsymbol{w})^{t_k} \left[1 - y_k(\boldsymbol{x}, \boldsymbol{w})\right]^{1-t_k} \tag{10.31}$$

Taking the negative logarithm of the corresponding likelihood function then gives the following error function

$$E(\boldsymbol{w}) = -\sum_{n=1}^{N} \sum_{k=1}^{K} \{t_{nk} \ln(y_{nk}) + (1 - t_{nk}) \ln(1 - y_{nk})\} \tag{10.32}$$

where y_{nk} denotes $y_k(\boldsymbol{x}_n, \boldsymbol{w})$. Again, the derivative of the error function with respect to the activation for a particular output unit takes the form (10.27) just as in the regression case.

It is interesting to contrast the neural network solution to this problem with the corresponding approach based on a linear classification model. Suppose that we are using a standard two-layer network. We see that the weight parameters in the first layer of the network are shared between the various outputs, whereas in the linear model each classification problem is solved independently. The first layer of the network can be viewed as performing a nonlinear feature extraction, and the sharing of features between the different outputs can save on computation and can also lead to improved generalization.
Finally, we consider the standard multiclass classification problem in which each input is assigned to one of K mutually exclusive classes. The binary target variables $t_k \in \{0, 1\}$ have a 1-of-K coding scheme indicating the class, and the network outputs are interpreted as $y_k(\boldsymbol{x}, \boldsymbol{w}) = p(t_k = 1|\boldsymbol{x})$, leading to the following error function

$$E(\boldsymbol{w}) = -\sum_{n=1}^{N}\sum_{k=1}^{K} t_{kn} \ln(y_k(\boldsymbol{x}_n, \boldsymbol{w})) \tag{10.33}$$

We see that the output unit activation function, which corresponds to the canonical link, is given by the softmax function

$$y_k(\boldsymbol{x}, \boldsymbol{w}) = \frac{e^{a_k(\boldsymbol{x},\boldsymbol{w})}}{\sum_j e^{a_j(\boldsymbol{x},\boldsymbol{w})}} \tag{10.34}$$

which satisfies $0 \leq y_k \leq 1$ and $\sum_k y_k = 1$. Note that the $y_k(\boldsymbol{x}, \boldsymbol{w})$ are unchanged if a constant is added to all of the $a_k(\boldsymbol{x}, \boldsymbol{w})$, causing the error function to be constant for some directions in weight space. This degeneracy is removed if an appropriate regularization term is added to the error function.
Once again, the derivative of the error function with respect to the activation for a particular output unit takes the familiar form (10.27).
In summary, there is a natural choice of both output unit activation function and matching error function, according to the type of problem being solved. For regression we use linear outputs and a sum-of-squares error, for (multiple independent) binary classifications we use logistic sigmoid outputs and a cross-entropy error function, and for multiclass classification we use softmax outputs with the corresponding multiclass cross-entropy error function. For classification problems involving two classes, we can use a single logistic sigmoid output, or alternatively we can use a network with two outputs having a softmax output activation function.

Parameter Optimization
We turn next to the task of finding a weight vector $\boldsymbol{w}$, which minimizes the chosen function $E(\boldsymbol{w})$. First note that if we make a small step in weight space from $\boldsymbol{w}$ to $\boldsymbol{w} + \delta\boldsymbol{w}$ then the change in the error function is $\delta E \cong \delta\boldsymbol{w}^T \nabla E(\boldsymbol{w})$ points in the direction of greatest rate of increase of the error function. Because the error $E(\boldsymbol{w})$ is a smooth continuous function of $\boldsymbol{w}$, its smallest value will occur at a point in weight space such that the gradient of the error function vanishes, so that

$$\nabla E(\boldsymbol{w}) = 0 \tag{10.35}$$

as otherwise we could make a small step in the direction of $-\nabla E(\boldsymbol{w})$ and thereby further reduce the error. Points at which the gradient vanishes are called stationary points, and may be futher classified into minima, maxima, and saddle points.
Our goal is to find a vector $\boldsymbol{w}$ such that $E(\boldsymbol{w})$ takes its smallest value. However, the error function typically has a highly nonlinear dependence on the weights and bias parameters,

and so there will be many points in weight space at which the gradient vanishes (or is numerically very small). Indeed, we see that for any point $\boldsymbol{w}$ that is a local minimum, there will be other points in weight space that are equivalent minima. For instance, in a two-layer network with M hidden neurons/units, each point in weight space is a member of a family of $M!2^M$ equivalent points.
Furthermore, there will typically be multiple inequivalent stationary points and in particular multiple inequivalent minima. A minimum that corresponds to the smallest value of the error function for any weight vector is said to be a global minimum. Any other minima corresponding to higher values of the error function are said to be local minima. For a successful application of neural networks, it may not be necessary to find the global minimum (and in general it will not be known whether the global minimum has been found) but it may be necessary to compare several local minima in order to find a sufficiently good solution.
Because there is clearly no hope of finding an analytical solution to the equation $\nabla E(\boldsymbol{w}) = 0$ we resort to iterative numerical procedures. The optimization of continuous nonlinear functions is a widely studied problem and there exists an extensive literature on how to solve it efficiently. Most techniques involve choosing some initial value $\boldsymbol{w}^{(0)}$ for the weight vector and then moving through weight space in a succession of steps of the form

$$\boldsymbol{w}^{(\tau+1)} = \boldsymbol{w}^{(\tau)} + \triangle \boldsymbol{w}^{(\tau)} \tag{10.36}$$

where τ labels the iteration step. Different algorithms involve different choices for the weight vector update $\triangle \boldsymbol{w}^{(\tau)}$. Many algorithms make use of gradient information and therefore require that, after each update, the value of $\nabla E(\boldsymbol{w})$ is evaluated at the new weight vector $\boldsymbol{w}^{(\tau+1)}$. In order to understand the importance of gradient information, it is useful to consider a local approximation to the error function based on a Taylor expansion.

Local Quadratic Approximation
Insight into the optimization problem, and into the varios techniques for solving it, can be obtained by considering a local quadratic approximation to the error function.
Consider the Taylor expansion of $E(\boldsymbol{w})$ around some point $\hat{\boldsymbol{w}}$ in weight space

$$E(\boldsymbol{w}) \approxeq E(\hat{\boldsymbol{w}}) + (\boldsymbol{w} - \hat{\boldsymbol{w}})^T \boldsymbol{b} + \frac{1}{2}(\boldsymbol{w} - \hat{\boldsymbol{w}})^T \boldsymbol{H} (\boldsymbol{w} - \hat{\boldsymbol{w}}) \tag{10.37}$$

where cubic and higher terms have been omitted. Here $\boldsymbol{b}$ is defined to be the gradient of E evaluated at $\hat{\boldsymbol{w}}$

$$\boldsymbol{b} \equiv \nabla E|_{\boldsymbol{w}=\hat{\boldsymbol{w}}} \tag{10.38}$$

and the Hessian matrix $\boldsymbol{H} = \nabla\nabla E$ has elements

$$(\boldsymbol{H})_{ij} \equiv \frac{\partial E}{\partial w_i \partial w_j}\Big|_{\boldsymbol{w}=\hat{\boldsymbol{w}}} \tag{10.39}$$

From Eq. (10.37), the corresponding local approximation to the gradient is given by

$$\nabla E \approxeq \boldsymbol{b} + \boldsymbol{H}(\boldsymbol{w} - \hat{\boldsymbol{w}}) \tag{10.40}$$

For points $\boldsymbol{w}$ that are sufficiently close to $\hat{\boldsymbol{w}}$, these expressions will give reasonable approximations for the error and its gradient.
Consider the particular case of a local quadratic approximation around a point $\boldsymbol{w}^*$ that is a

minimum of the error function. In this case there is no linear term, because $\nabla E = 0$ at $\boldsymbol{w}^*$, and Eq. (10.37) becomes

$$E(\boldsymbol{w}) = E(\boldsymbol{w}^*) + \frac{1}{2}(\boldsymbol{w} - \boldsymbol{w}^*)^T \boldsymbol{H}(\boldsymbol{w} - \boldsymbol{w}^*) \tag{10.41}$$

where the Hessian $\boldsymbol{H}$ is evaluated at $\boldsymbol{w}^*$. In order to interpret this geometrically, consider the eigenvalue equation for the Hessian matrix

$$\boldsymbol{H}\boldsymbol{u}_i = \lambda_i \boldsymbol{u}_i \tag{10.42}$$

where the eigenvectors $\boldsymbol{u}_i$ form a complete orthonormal set so that

$$\boldsymbol{u}_i^T \boldsymbol{u}_j = \delta_{ij} \tag{10.43}$$

We now expand $(\boldsymbol{w} - \boldsymbol{w}^*)$ as a linear combination of the eigenvectors in the form

$$\boldsymbol{w} - \boldsymbol{w}^* = \sum_i \alpha_i \boldsymbol{u}_i \tag{10.44}$$

This can be regarded as a transformation of the coordinate system in which the origin is translated to the point $\boldsymbol{w}^*$, and the axes are rotated to align with the eigenvectors (through the orthogonal matrix whose columns are the $\boldsymbol{u}_i$). Substituting Eq. (10.44) into Eq. (10.41), and using Eqs. (10.42) and (10.43), allows the error function to be written in the form

$$E(\boldsymbol{w}) = E(\boldsymbol{w}^*) + \frac{1}{2}\sum_i \lambda_i \alpha_i^2 \tag{10.45}$$

A matrix $\boldsymbol{H}$ is said to be positive definite if, and only if,

$$\boldsymbol{v}^T \boldsymbol{H}\boldsymbol{v} > 0, \quad \text{for all } \boldsymbol{v} \tag{10.46}$$

Because the eigenvectors $\{\boldsymbol{u}_i\}$ form a complete set, an arbitrary vector $\boldsymbol{v}$ can be written in the form

$$\boldsymbol{v} = \sum_i c_i \boldsymbol{u}_i \tag{10.47}$$

From Eqs (10.42) and (10.43), we then have

$$\boldsymbol{v}^T \boldsymbol{H}\boldsymbol{v} = \sum_i c_i^2 \lambda_i \tag{10.48}$$

and so $\boldsymbol{H}$ will be positive definite, if, and only if, all of its eigenvalues are positive. In the new coordinate system, whose basis vectors are given by the eigenvectors $\{\boldsymbol{u}_i\}$, the contours of constant E are ellpises centred on the origin. For a one-dimensional weight space, a stationary point w^* will be a minimum if

$$\frac{\partial^2 E}{\partial w^2}\Big|_{w^*} > 0 \tag{10.49}$$

The corresponding result in D-dimensions is that the Hessian matrix, evaluated at w^*, should be positive definite.

Use of Gradient Information

It is possible to evaluate the gradient of an error function efficiently by means of the backpropagation procedure. The use of this gradient information can lead to significant improvements

in the speed with which the minima of the error function can be located.
In the quadratic approximation to the error function, given in Eq. (10.37), the error surface is specified by the quantities $\boldsymbol{b}$ and $\boldsymbol{H}$, which contain a total of $W\frac{W+3}{2}$ independent elements (because the matrix $\boldsymbol{H}$ is symmetric), where W is the dimensionality of $\boldsymbol{w}$ (i.e., the total number of adaptive parameters in the network). The location of the minimum of this quadratic approximation therefore depends on $O(W^2)$ parameters, and we should not expect to be able to locate the minimum until we have gathered $O(W^2)$ independent pieces of information. If we do not make use of gradient information, we would expect to have to perform $O(W^2)$ function evaluations, each of which would require $O(W)$ steps. Thus, the computational effort needed to find the minimum using such an approach would be $O(W^3)$.
Now compare this with an algorithm that makes use of the gradient information. Because each evaluation of ∇E brings W items of information, we might hope to find the minimum of the function in $O(W)$ gradient evaluations. As we shall see, by using error backpropagation, each such evaluation takes only $O(W)$ steps and so the minimum can now be found in $O(W^2)$ steps. For this reason, the use of gradient information forms the basis of practical algorithms for training neural networks.

Gradient Descent Optimization
The simplest approach to using gradient information is to choose the weight update in Eq. (10.36) to comprise a small step in the direction of the negative gradient, so that

$$\boldsymbol{w}^{(\tau+1)} = \boldsymbol{w}^{(\tau)} - \eta\nabla E(\boldsymbol{w}^{(\tau)}) \tag{10.50}$$

where the parameter $\eta > 0$ is known as the learning rate. After each such update, the gradient is re-evaluated for the new weight vector and the process repeated. Note that the error function is defined with respect to a training set, and so each step requires that the entire training set be processed in order to evaluate ∇E. Techniques that use the whole data set at once are called batch methods. At each step the weight vector is moved in the direction of the greatest of decrease of the error function, and so this approach is known as gradient descent or steepest descent. Although such an approach might intuitively seem reasonable, in fact it turns out to be a poor algorithm.
For batch optimization, there are more efficient methods, such as conjugate gradients and quasi-Newton methods, which are much more robust and much faster than simple gradient descent [24]. Unlike gradient descent, these algorithms have the property that the error function always decreases at each iteration unless the weight vector has arrived at a local or global minimum.
In order to find a sufficiently good minimum, it may be necessary to run a gradient-based algorithm multiple times, each time using a differently randomly chosen starting point, and comparing the resulting performance on an independent validation set.
There is, however, an on-line version of gradient descent that has proved useful in practice for training neural networks on large data sets. Error functions based on maximum likelihood for a set of independent observations comprise a sum of terms, one for each data

$$E(\boldsymbol{w}) = \sum_{n=1}^{N} E_n(\boldsymbol{w}) \tag{10.51}$$

On-line gradient descent, also known as sequentiel gradient descent or stochastic gradient descent, makes an update to the weight vector based on one data point at a time, so that

$$\boldsymbol{w}^{(\tau+1)} = \boldsymbol{w}^{(\tau)} - \eta\nabla E_n(\boldsymbol{w}^{(\tau)}) \tag{10.52}$$

This update is repeated by cycling through the data either in sequence or by selecting points at random with replacement. There are of course intermediate scenarios in which the updates are based on batches of data points.
One advantage of on-line methods compared to batch methods is that the former handle redundancy in the data much more efficiently. To see, this consider an extreme example in which we take a data set and double its size by duplicating every data point. Note that this simply multiplies the error function by a factor of 2 and so is equivalent to using the original error function. Batch methods will require double the computational effort to evaluate the batch error function gradient, whereas on-line methods will be unaffected. Another property of on-line gradient descent is the possibility of escaping from local minima, since a stationary point with respect to the error function for the whole data set will generally not be a stationary point for each data point individually.

[SI*] **Error Backpropagation**

The multilayer perceptron architecture is sometimes called a backpropagation network. The term backpropagation is also used to describe the training of a multilayer perceptron using gradient descent applied to a sum-of-squares error function. Most training algorithms involve an iterative procedure for minimization of an error function, with adjustments to the weights being made in a sequence of steps. At each such step, we can distinguish between two distinct stages. In the first stage, the derivatives of the error function with respect to the weights must be evaluated. The important contribution of the backpropagation technique is in providing a computationally efficient method for evaluating such derivatives. Because it is at this stage that errors are propagated backwards through the network. In the second stage, the derivatives are then used to compute the adjustments to be made to the weights. The simplest such technique involves gradient descent. Thus, the first stage, namely the propagation of errors backwards through the network in order to evaluate derivatives, can be applied to many other kinds of network and not just the multilayer perceptron. It can also be applied to error functions other than just the simple sum-of-squares, and to the evaluation of other derivatives such as the Jacobian and Hessian matrices.

Evaluation of Error-Function Derivatives

We now derive the backpropagation algorithm for a general network having arbitrary feed-forward topology, arbitrary differentiable nonlinear activation functions, and a broad class of error function.
Many error functions of practical interest, for instance those defined by maximum likelihood for a set of i.i.d. data, comprise a sum of terms, one for each data point in the training set, so that

$$E(\boldsymbol{w}) = \sum_{n=1}^{N} E_n(\boldsymbol{w}) \tag{10.53}$$

Here we shall consider the problem of evaluating $\nabla E_n(\boldsymbol{w})$ for one such term in the error function. This may be used directly for sequential optimization, or the results can be accumulated over the training set in the case of batch methods.
Consider first a simple linear model in which the outputs y_k are linear combinations of the input variables x_i, so that

$$y_k = \sum_i w_{ki} x_i \tag{10.54}$$

together with an error function that, for a paticular input pattern n, takes the form

$$E_n = \frac{1}{2}\sum_k (y_{nk} - t_{nk})^2 \tag{10.55}$$

where $y_{nk} = y_k(\boldsymbol{x}_n, \boldsymbol{w})$. The gradient of this error function with respect to a weight w_{ji} is given by

$$\frac{\partial E_n}{\partial w_{ji}} = (y_{nj} - t_{nj})x_{ni} \tag{10.56}$$

which can be interpreted as a 'local' computation involving the product of an 'error signal' $y_{nj} - t_{nj}$ associated with the output end of the link w_{ji} and the variable x_{ni} associated with the input end of the link.

In a general feed-forward network, each unit computes a weighted sum of its inputs of the form

$$a_j = \sum_i w_{ji} z_i \tag{10.57}$$

where z_i is the activation of a unit, or input, that sends a connection to unit j, and w_{ji} is the weight associated with that connection. We saw that biases can be included in this sum by introducing an extra unit, or input, with activation fixed at $+1$. We therefore do not need to deal with biases explicitly. The sum in (10.57) is transformed by a nonlinear activation function $h(\cdot)$ to give the activation z_j of unit j in the form

$$z_j = h(a_j) \tag{10.58}$$

Note that one or more variables z_i in the sum in (10.57) could be an input, and similarly, the unit j in (10.58) could be an output.

For each pattern in the training set, we shall suppose that we have supplied the corresponding input vector to the network and calculated the activations of all of the hidden and output units in the network by successive application of (10.57) and (10.58). This process is often called forward propagation because it can be regarded as a forward flow of information through the network.

Now consider the evaluation of the derivative of E_n with respect to a weight w_{ji}. The outputs of the various units will depend on the particular input pattern n. However, in order to keep the notation uncluttered, we shall omit the subscript n from the network variables. First we note that E_n depends on the weight w_{ji} only via the summed input a_j to unit j. We can therefore apply the chain rule for partial derivatives to give

$$\frac{\partial E_n}{\partial w_{ji}} = \frac{\partial E_n}{\partial a_j}\frac{\partial a_j}{\partial w_{ji}} \tag{10.59}$$

We now introduce a useful notation

$$\delta_j = \frac{\partial E_n}{\partial a_[j]} \tag{10.60}$$

where the δ's are often referred to as errors for reasons. Using (10.57), we can write

$$\frac{\partial a_j}{\partial w_{ji}} = z_i \tag{10.61}$$

Substituting (10.60) and (10.61) into (10.59), we then obtain

$$\frac{\partial E_n}{\partial w_{ji}} = \delta_j z_i \tag{10.62}$$

Equation (10.62) tells us that the required derivative is obtained simply by mutiplying the value of δ for the unit at the output end of the weight by the value of z for the unit at the input end of the weight (where $z = 1$ in the case of a bias). Note that this takes the same form as for the simple linear model considered at the start of this section. Thus, in order to evaluate the derivatives, we need only to calculate the value of δ_j for each hidden and output unit in the network, and then apply (10.62).
As we have seen already, for the output units, we have

$$\delta_k = y_k - t_k \tag{10.63}$$

provided we are using the canonical link as the output-unit activation function. To evaluate the δ's for hidden units, we again make use of the chain rule for partial derivatives,

$$\delta_j = \frac{\partial E_n}{\partial a_j} = \sum_k \frac{\partial E_n}{\partial a_k} \frac{\partial a_k}{\partial a_j} \tag{10.64}$$

where the sum runs over all units k to which unit j sends connections. Note that the labelled k could include other hidden units and/or output units. In writing down (10.64), we are making use of the fact that variations in a_j give rise to variations in the error function only through variations in the variables a_k. If we now substitute the definition of δ given by (10.60) into (10.64), and make use of (10.57) and (10.58), we obtain the following backpropagation formula

$$\delta_j = h'(a_j) \sum_k w_{kj} \delta_k \tag{10.65}$$

which tells us that the value of δ fo a particular hidden unit can be obtained by propagating the δ's backwards from units higher up in the network. Note that the summation in (10.65) is taken over the first index on w_{kj} (corresponding to backward propagation of information through the network), whereas in the forward propagation

$$z_k = h\left(\sum_j w_{kj} z_j\right)$$

where the sum runs over all units that send connections to unit k (and a bias parameter is included in the summation). It is taken over the second index. Because we already know the values of the δ's for the output units, it follows that recursively applying (10.65) we can evaluate the δ's for all of the hidden units in a feed-forward network, regardless of its topology.
For batch methods, the derivative of the total error E can then be obtained by repeating the above steps for each pattern in the training set and then summing over all patterns:

$$\frac{\partial E}{\partial w_{ji}} = \sum_n \frac{\partial E_n}{\partial w_{ji}} \tag{10.66}$$

In the above derivation we have implicitly assumed that each hidden or output unit in the network has the same activation function $h(\cdot)$.

i. Backpropagation for Feed-Forward Neural Network

Consider a feed-forward neural network with 2 inputs, and one hidden layer with 2 neurons in the hidden layer, and 2 outputs. Assume that the neurons use sigmoid activation function for the forward and backward pass and there is no bias.

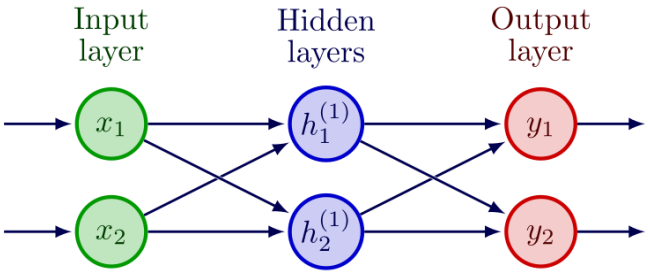

Figure 10.4: *The schematic diagram of this example feed-forward neural network.*

We will compute and simulate the backpropagation for this feed-forward neural network (FFN).

Let the input be

$$\boldsymbol{x} = \begin{bmatrix} 0.35 \\ 0.7 \end{bmatrix}$$

The initial weights matrix from the input to the hidden layer

$$W0 = \begin{bmatrix} 0.2 & 0.2 \\ 0.3 & 0.3 \end{bmatrix}$$

The initial weights matrix from the hidden layer to the output

$$W1 = \begin{bmatrix} 0.3 & 0.9 \\ 0.1 & 0.5 \end{bmatrix}$$

and the output

$$\boldsymbol{y} = \begin{bmatrix} 0.5 \\ 0.37 \end{bmatrix}$$

The initial calculation will compute the weighted sum at each node with the sum formula

$$a_j = f\left(\sum_{i=1}^{2} x_i w_{ij} + b_j\right)$$

x_i represents the value of the i-th input, a_j represents the weighted sum of all the inputs and weights at each node, w_{ij} represents the weights between the i-th input and the j-th neuron, there is no bias, so $b_j = 0$.

After applying the activation function to a_j, we get the output of the neuron

$$o_j = \text{activation function}(a_j) = f\left(\sum_{i=1}^{2} x_i w_{ij} + b_j\right)$$

The activation function is using the sigmoid function that returns a value between 0 and 1, introducing non-linearity into the model, thus

$$o_j = \frac{1}{1 + e^{-a_j}}$$

At h_1 node we will have

$$\begin{aligned} a_1 &= (w0_{11}x_1) + (w0_{12}x_2) \\ &= (0.2 * 0.35) + (0.2 * 0.7) \\ &= 0.21 \end{aligned}$$

Once we calculated the a_1 value, we can now proceed to find the $f(a_1)$ value (activation function computation)

$$\begin{aligned} f(a_1) &= \frac{1}{1 + e^{-0.21}} \\ &= 0.5523 \end{aligned}$$

Similarly, for the the second node in the hidden layer h_2 we will have

$$\begin{aligned} a_2 &= (w0_{21}x_1) + (w0_{22}x_2) \\ &= (0.3 * 0.35) + (0.3 * 0.7) \\ &= 0.315 \end{aligned}$$

$$\begin{aligned} f(a_2) &= \frac{1}{1 + e^{-0.315}} \\ &= 0.5781 \end{aligned}$$

The predicted output will be

$$\begin{aligned} o_1 &= (w1_{11}h_1) + (w1_{12}h_2) \\ &= (0.3 * 0.35) + (0.9 * 0.5781) \\ &= 0.68598 \end{aligned}$$

$$\begin{aligned} f(o_1) &= \frac{1}{1 + e^{-0.68598}} \\ &= 0.66507 \end{aligned}$$

and

$$\begin{aligned} o_2 &= (w1_{21}h_1) + (w1_{22}h_2) \\ &= (0.3 * 0.35) + (0.9 * 0.5781) \\ &= 0.34428 \end{aligned}$$

$$\begin{aligned} f(o_2) &= \frac{1}{1 + e^{-0.34428}} \\ &= 0.58523 \end{aligned}$$

thus we will have

$$\hat{y} = \begin{bmatrix} 0.66507 \\ 0.585231 \end{bmatrix}$$

The error calculation can be obtained by computing with this formula

$$\epsilon = \boldsymbol{y} - \hat{y}$$

$$\epsilon = \begin{bmatrix} -0.165074 \\ -0.215231 \end{bmatrix}$$

The norm is

$$||\epsilon|| = 0.271244$$

the target is to minimize the norm of ϵ, to make it near zero. From here, we will do backpropagating process because the weights $W0$ and $W1$ that we have used are not sufficient to predict the output.

Backpropagation

First we will calculate the gradients with this formula to compute the change in each weight

$$\triangle w_{ij} = \eta \times \delta_j^{(L)} \times O_j^{(L-1)}$$

with η as the learning rate (we set it to 0.51), $\delta_j^{(L)}$ as the error term of the j-th neuron in the next layer L, and $O_j^{(L-1)}$ is the output value of the current layer / the sending neuron (layer $L-1$). For this example, we will assume that layer L denotes the output layer.

To compute the error term, we can start from the first node in the last layer, the output layer

$$\begin{aligned} \delta y_1 &= f'(O_1^{(L)}) \left(\delta_k^{(L)} \right) \\ &= \hat{y}_1(1-\hat{y}_1)(y_1-\hat{y}_1) \\ &= 0.66507(1-0.66507)(-0.165074) \\ &= -0.03677 \end{aligned}$$

the second nodes in the output layer

$$\begin{aligned} \delta y_2 &= f'(O_1^{(L)}) \left(\delta_k^{(L)} \right) \\ &= \hat{y}_2(1-\hat{y}_2)(y_2-\hat{y}_2) \\ &= 0.585231(1-0.585231)(-0.215231) \\ &= -0.052244 \end{aligned}$$

now to the first node in the hidden layer

$$\begin{aligned} \delta h_1 &= f'(O_1^{(L-1)}) \sum_{k=1}^{2} \left(w_{k,1} \cdot \delta_k^L \right) \\ &= 0.5523(1-0.5523)((0.3 \times -0.03677) + (0.1 \times -0.052244)) \\ &= -0.004019 \end{aligned}$$

the second node in the hidden layer

$$\begin{aligned} \delta h_2 &= f'(O_2^{(L-1)}) \sum_{k=1}^{2} \left(w_{k,2} \cdot \delta_k^L \right) \\ &= 0.578105(1-0.578105)((0.9 \times -0.03677) + (0.5 \times -0.052244)) \\ &= -0.014443 \end{aligned}$$

After we have computed all the δ we will have the δ vectors

$$\delta_{output} = \begin{bmatrix} -0.03677 \\ -0.052244 \end{bmatrix}$$

$$\delta_{hidden} = \begin{bmatrix} -0.004019 \\ -0.014443 \end{bmatrix}$$

Now, we can do weights updates computation, for the weights from the hidden layer to the output layer

$$\begin{aligned} \triangle w1_{11} &= \eta \times (\delta y_1) \times O_1^{(L-1)} \\ &= 0.51 \times -0.03677 \times 0.552308 \\ &= -0.010357 \end{aligned}$$

$$\begin{aligned} \triangle w1_{12} &= \eta \times (\delta y_1) \times O_2^{(L-1)} \\ &= 0.51 \times -0.03677 \times 0.578105 \\ &= -0.010841 \end{aligned}$$

$$\begin{aligned} \triangle w1_{21} &= \eta \times (\delta y_2) \times O_1^{(L-1)} \\ &= 0.51 \times -0.052244 \times 0.552308 \\ &= -0.0147159 \end{aligned}$$

$$\begin{aligned} \triangle w1_{22} &= \eta \times (\delta y_2) \times O_2^{(L-1)} \\ &= 0.51 \times -0.052244 \times 0.578105 \\ &= -0.015403 \end{aligned}$$

for the weights from the input layer to the hidden layer

$$\begin{aligned} \triangle w0_{11} &= \eta \times (\delta h_1) \times O_1^{L-2} \\ &= 0.51 \times -0.004019 \times 0.35 \\ &= -0.00071739 \end{aligned}$$

$$\begin{aligned} \triangle w0_{12} &= \eta \times (\delta h_1) \times O_2^{L-2} \\ &= 0.51 \times -0.004019 \times 0.7 \\ &= -0.001434783 \end{aligned}$$

$$\begin{aligned} \triangle w0_{21} &= \eta \times (\delta h_2) \times O_1^{(L-2)} \\ &= 0.51 \times -0.01443 \times 0.35 \\ &= -0.002575755 \end{aligned}$$

$$\begin{aligned} \triangle w0_{22} &= \eta \times (\delta h_2) \times O_2^{(L-2)} \\ &= 0.51 \times -0.01443 \times 0.7 \\ &= -0.00515151 \end{aligned}$$

Thus, the new weights matrix for the iteration 1 is

$$W0^{(1)} = \begin{bmatrix} 0.199283 & 0.198565 \\ 0.297422 & 0.294844 \end{bmatrix}$$

and the new hidden weights matrix is

$$W1^{(1)} = \begin{bmatrix} 0.289643 & 0.889159 \\ 0.085284 & 0.484597 \end{bmatrix}$$

We will use the weights matrix $W0^{(1)}$ and the hidden weights matrix $W1^{(1)}$ for the forward propagation to obtain new prediction for $\boldsymbol{y}$, after iteration 1, we will have

$$\hat{y} = \begin{bmatrix} 0.662159 \\ 0.580954 \end{bmatrix}$$

The error calculation

$$\epsilon = \boldsymbol{y} - \hat{y}$$

$$\epsilon = \begin{bmatrix} -0.162159 \\ -0.210954 \end{bmatrix}$$

The norm is

$$||\epsilon|| = 0.266077$$

Since the predicted vector $\hat{y}$ is still not converge to $\boldsymbol{y}$ (the target output vector), the process of calculating the error (ϵ) and backpropagating continues until the desired output is reached and we will obtain the final weights matrix and the final hidden weights matrix that can be used for testing.

This process demonstrates how backpropagation iteratively updates weights by minimizing errors until the network acurately predicts the output.

For the C++ code, we use a lot of matrix and vector multiplication and we have to remember the rule of the multiplication for them, thus when we create the weights matrix, we transpose the weights matrix that represents $W0$ with the entries $w0_{ij}$ that represents the weights between the i-th input and the j-th neuron. That is why the initial weights matrix is like this

$$W0 = \begin{bmatrix} 0.2 & 0.2 \\ 0.3 & 0.3 \end{bmatrix}$$

instead of

$$W0 = \begin{bmatrix} 0.2 & 0.3 \\ 0.2 & 0.3 \end{bmatrix}$$

You will comprehend it when you remember the result of $W0\boldsymbol{x}$ corresponds to this neural network example, see Figure (10.4).

```
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Neural Networks/Test Backpropagation FNN ]# ./main
Input:
0.35
0.7

Initial Weights :
               0.200000               0.200000
               0.300000               0.300000

Initial Hidden Layer Weights :
               0.300000               0.900000
               0.100000               0.500000

Biases:
0.000000
0.000000

Actual Output:
0.500000
0.370000

Feed-forward Neural Network with 1 hidden layer:

Hidden layer:
0.552308
0.578105

Predicted Output:
0.665074
0.585231

Error vector:
-0.165074
-0.215231

||Error|| = 0.271244
```

Figure 10.5: *The computation of backpropagation process for the feed-forward neural network with* ***FNN_1hiddenlayer()*** *function (****SymIntegration/Examples/Neural Networks/Test Backpropagation FNN/main.cpp****).*

```
*******************************************************************************

                        Backpropagation iteration

*******************************************************************************

Learning rate: 0.510000

Iteration = 1

Delta output vector:
-0.036770
-0.052244

Delta hidden layer vector:
-0.004019
-0.014443

New weights matrix:
               0.199283               0.198565
               0.297422               0.294844

New hiddenweights matrix:
               0.289643               0.889159
               0.085284               0.484597

Hidden layer:
0.551997
0.577004

Predicted Output:
0.662159
0.580954

Error vector:
-0.162159
-0.210954

||Error|| = 0.266077
```

Figure 10.6: *The computation of backpropagation process for the feed-forward neural network with* ***FNN_1hiddenlayer()*** *function (****SymIntegration/Examples/Neural Networks/Test Backpropagation FNN/main.cpp****).*

```
Iteration = 5

Delta output vector:
-0.034246
-0.048222

Delta hidden layer vector:
-0.002712
-0.012377

New weights matrix:
               0.197011               0.194022
               0.288050               0.276100

New hiddenweights matrix:
               0.250058               0.847861
               0.029354               0.426246

Hidden layer:
0.551014
0.572996

Predicted Output:
0.651039
0.564738

Error vector:
-0.151039
-0.194738

||Error|| = 0.246445
```

Figure 10.7: *The computation of backpropagation process for the feed-forward neural network with* ***FNN_1hiddenlayer()*** *function (****SymIntegration/Examples/Neural Networks/Test Backpropagation FNN/main.cpp****).*

ii. Feed-Forward Neural Network for Iris Dataset Classification with 1 Hidden Layer

Suppose we have data that are classified into n categories and each category is described by m variables. We want to create a neural network that can learn about the pattern of the data, so if we input a random number for the m variables we will know which category it belongs to based on the data that we have.

For this purpose, we will use the Iris dataset, it is a famous data of Iris flower species first introduced by the statistician Ronald Fisher in 1936, the Iris Setosa, Iris Versicolor, and Iris Virginica, there are 150 data that are classified into the three Iris species that we have mentioned, each iris has their corresponding features, represented by variables sepal length, sepal width, petal length, and petal width.

1. **Data Preparation**
   We have a CSV file of the iris dataset, each line contains the measurements of one iris species.

2. **Training**
   Training is where our network learns from its mistakes and gradually improves with backpropagation.
   We feed some data into the neural network, letting it make predictions based on that data. We choose to feed 80% training data, which is 120 data, and we will have to shuffle the index when we code it before we train it, otherwise if we don't shuffle the index before we train the data, the computer will only learn about the first 120 data in the CSV file and it will never learn anything to be better.

3. **Testing / Evaluation**
   After training we will obtain 120 bias for hidden layer, bias for output layer, weights from input to hiddenlayer and weights from hiddenlayer to output matrices that represent the model, we can use 20% of testing data, the leftover after we shuffle the index take the first 80% data for training the last 20% will be used to check which model can predict with highest accuracy.

   Of course we can do that 20% testing, but in this book instead of using 20% we will test it on all the original data, all the 150 data with the best weights model we have obtained. Of course C++ can compute faster than Python and JULIA, it is a powerful programming language for implementing machine learning algorithms, so we have nothing to worry about, just run the code after long time of writing the code, we can learn more by doing this. Food for thoughts is like this, what if the last 20% of data all belong to Iris Setosa species only? and the model able to predict Iris Setosa 100% correctly without a doubt? Then it will probably make a false prediction in the end if we want the model to predict a flower with measurement of Iris Vesicolor.

   Predict is where our network makes the prediction about the flower' species based on our input variables or testing data.

In this example we will create a Feed-forward neural network (multilayer peceptron) with 1 hidden layer to learn about the pattern of the iris dataset, it will have the architecture and configuration like this:

- **Input Layer:** 4 Neurons (representing sepal length, sepal width, petal length, petal width).
- **Hidden Layer(s):** 1 hidden layer is sufficient with 5-10 neurons in the hidden layer. Activation function in the hidden layer: Sigmoid, or tanh, it is strong and stable. Another alternative is ReLU because of the fast training and avoiding vanishing gradient issues.
- **Output Layer:** 3 Neurons (representing Setosa, Versicolor, and Virginica). Activation function in the output layer: Softmax, it is required for multi-class classification to output probabilities that sum to 1.
- **Loss Function:** Categorical Cross-Entropy.
- **Optimizer:** SGD (Stochastic Gradient Descent). Stochastic Gradient Descent for feed-forward neural networks updates model weights iteratively by calculating the gradient of the loss function based on a single sample or a small mini-batch, ather than the entire dataset. This approach allows for faster iteration and lower memory usage, which is ideal for large datasets, though it introduces high variance (noise) in parameter updates, which can help avoid local minima.

Small data size (150 data) doesn't need a lot of hidden layers, 2 is more than enough for iris dataset, but we only use 1 here.

Initialization
We initialize weights from input layer to hidden layer, it will become a matrix $W^{(0)}$ with size of 4×5, then we initialize weights from hidden layer to the output layer, it is a matrix $W^{(1)}$ with size of 5×3, then we initialize bias in the hidden layer, it is a vector $\boldsymbol{b}_0$ of size 5, then we initialize bias in the output layer, a vector $\boldsymbol{b}_1$ of size 3.
The matrix $W^{(0)}$ means that it represents the weights connecting the input neurons to the neurons in the hidden layer, we number it as 0, while matrix $W^{(1)}$ means that it represents the weights connecting the neurons in the hidden layer to the neurons in the output layer, we number it as 1, the superscript doesn't represents the updated value, it represents the layer location for the weights.

By the meaning of initialize, we generate random number (with double data type) with Mersenne Twister engine with range $[-1, 1]$.

The input for the training and testing function is:

1. Training matrix T that has the size of 120×4, the order of the data is different with the original CSV data, because we have shuffled it first before it becomes an input to the function.
2. Vector $\boldsymbol{t}$ for the classification of the training matrix with corresponding order.
3. Testing matrix A that has the size of 150×4, the order of the data is the same as the original CSV data, it is to test whether the model obtained in the training phase able to predict all the 150 data accurately.
4. Vector $\boldsymbol{y}$ that has the classification of the matrix A, it has the size of 150, the order of the data is the same as the original CSV data.

The first **for** loop is from 1 to 120, training phase, we check vector $\boldsymbol{t}$, the first entry, if it is Setosa then we will create a new vector for this iteration

$$\boldsymbol{t}_i = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

if it is Versicolor then we will create a new vector for this iteration

$$\boldsymbol{t}_i = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

if it is Virginica then we will create a new vector for this iteration

$$\boldsymbol{t}_i = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

This process of creating an identity vector / binary vector based on each classification / type is called one-hot encoding. The forward propagation phase will have a result of predicted probability by using Softmax activation function at the output layer, consider this output example $[0.2\ 0.5\ 0.3]$, it shows that the result leans toward Versicolor, the neural network predicts that based on the inputs the probability is 20% Setosa, 50% Versicolor, 30% Virginica.

Forward Propagation
In this phase, data flows from input to output, calculating the prediction based on the current weights.

A network with 4 input neurons, 1 hidden layer of 5 neurons, and an output layer of 3 neurons has:
Weights: $(4 \times 5) + (5 \times 3) = 35$
Biases: $5 + 3 = 8$
Total Parameters: 43

the hidden layer plays as the intermediate layer where transformations occur, to learn the complex pattern, while learning rate η is the hyperparameter that regulates the speed of weight updates.

The first general formula for the forward propagation from the input layer to the hidden layer is

$$h = f(W^{(0)}\boldsymbol{x} + \boldsymbol{b})$$

with $\boldsymbol{x}$ as the input vector that represents the sepal length, sepal width, petal length, and petal width (4×1 vector), W is the weight matrix at the between the input layer and the hidden layer, $\boldsymbol{b}$ is the bias vector in the hidden layer, and f is the activation function (tanh), you can also try Sigmoid or ReLU for educational purpose.

The second general formula for the forward propagation from the hidden layer to the output layer is

$$y = f(W^{(1)}\boldsymbol{h} + \boldsymbol{b})$$

with $\boldsymbol{h}$ as the input vector that represents the neurons in the hidden layer (5×1 vector), W is the weight matrix at the between the hidden layer and the output layer, $\boldsymbol{b}$ is the bias vector in the output layer, and f is the activation function (Softmax).

Loss Function(L)
The network computes the difference between predicted output $\hat{\boldsymbol{y}}$ and the actual output $\boldsymbol{y}$, often

using Mean Squared Error (MSE) or Cross-Entropy loss.

The error calculation can be computed with this formula:

$$\boldsymbol{e} = \boldsymbol{y} - \hat{\boldsymbol{y}}$$

or if the output layer is using softmax then we will use cross-entropy loss that is defined as:

$$L = -\sum y \log(\hat{y})$$

Backpropagation
The error is sent backward through the network, using the chain rule to compute the gradient of the loss function with respect to each weight.
The backpropagation is the practice of fine-tuning the weights of a neural network based on the error rate obtained in the previous epoch. Proper tuning of the weights ensures lower error rates, making the model reliable by increasing its generalisation.

Weight Update
Weights are updated iteratively to reduce the loss / minimize the error (the error usually using Cross-Entropy Loss), by using an optimizer Stochastic Gradient Descent (SGD) formula, we will have the weight update formula

$$\begin{aligned} w_{ij}^{(L)} &= w_{ij}^{(L)} + \left(\eta \times \frac{\partial C}{\partial w^L}\right) \\ &= w_{ij}^{(L)} + \left(\delta_j^{(L)} \times O_i^{(L-1)}\right) \\ \delta_j^{(L)} &= f'(O_j^{(L)}) e_j \end{aligned}$$

with $\delta_j^{(L)}$ as the error term that represents the error of the receiving neuron (layer L), which includes the derivative of its activation function. O_i is the output value of the sending neuron (layer $L-1$), e_j is the difference between the actual output of neuron j with the predicted output for neuron j ($y_j - \hat{y}_j$).
Gradient is denoted by $\frac{\partial C}{\partial w}$, C is the cost function, it is calculated using the chain rule to propagate error from the output layer back to the input.

The iterative process of feeding data and refining weights continues until the model's loss is minimized, resulting in optimized weights.

Bias Update
We update the bias after we compute the δ_j with this formula

$$b_j = b_j + \eta \cdot \delta_j$$

Classification Process
We do the normalization, scaling the input data, often using min-max scaling to a $[0,1]$ range (with Sigmoid activation function) or $[-1,1]$ range (with Tanh activation function), after the input signals pass through the optimal weighted connections, summed, and activated in the forward propagation phase we will reach the output layer, the final layer uses a Softmax activation function to produce a probability distribution across the three iris' species. The species with the highest probability is chosen.

In the end, this feed-forward neural network can learn to classify objects or flowers with impressive accuracy, if it is trained well, we choose the optimal number of hidden layer, the activation function in each layer, and the optimal learning rate.

```
Epoch loss:6.315158

Epoch:994

Epoch loss:7.147247

Epoch:995

Epoch loss:6.726554

Epoch:996

Epoch loss:6.972265

Epoch:997

Epoch loss:6.941392

Epoch:998

Epoch loss:7.031226

Epoch:999

Epoch loss:6.588238

Epoch:1000

Epoch loss:4.846478

Model W0 matrix (from input to hidden layer):
              0.393350              3.921721              6.454273              1.608165             12.071175
             -2.648544             11.946345              6.058942              0.180194              9.000676
              2.442088            -11.245547            -10.595181             -2.257611            -18.769742
              0.583343             -6.368492             -6.489850             -1.133309            -10.643930

Model W1 matrix (from hidden layer to output):
             -3.357494              4.587249              5.600502
              3.581425              1.134718             -4.240889
              2.204437              1.369549             -1.116935
              2.784585             -0.994479             -3.142601
              1.424090              2.531305             -1.759481

Testing Feed-forward Neural Network with 1 hidden layer for Iris dataset:

Time taken by function: 18947334 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Neural Networks/Test FNN with Iris DataFrame 1 Hidden layer with Steepest Descent ]#
```

Figure 10.8: *The training process computes the optimal $W^{(0)}$ and $W^{(1)}$ using* 120 *training data (*80% *of the original data), the updated weights with minimal loss from the backpropagation process with* ***FNN_1hiddenlayer_irisdataset_training*** *function, the epoch loss is zig-zagging due to the steepest descent formula, the loss cannot decrease continuously (****SymIntegration/Examples/Neural Networks/Test FNN with Iris DataFrame 1 Hidden layer with Steepest Descent/main.cpp****).*

```
********************************************************************************

                                    Next data

********************************************************************************

Testing sample:30

Input layer:
5.900000
3.000000
5.100000
1.800000

Output layer:
0.000000
0.000000
1.000000

Predicted Output:
0.000004
0.020944
0.979052

Error vector:
-0.000004
-0.020944
0.020948

||Error|| = 0.029623

Average Loss over a batch of 30 samples : 0.194586

Correct prediction : 28
Wrong prediction : 2
Correct matrix:
          Total Correct                Setosa            Versicolor             Virginica                 Model
              28.000000              9.000000             11.000000              8.000000              1.000000

Time taken by function: 24928 microseconds
```

Figure 10.9: *The testing process with the optimal $W^{(0)}$ and $W^{(1)}$ for* 30 *testing sample (*20% *of the original data) with* ***FNN_1hiddenlayer_irisdataset_testing*** *function (****SymIntegration/Examples/Neural Networks/Test FNN with Iris DataFrame 1 Hidden layer with Steepest Descent/main.cpp****).*

```
********************************************************************************

                                    Next data

********************************************************************************

Testing sample:150

Input layer:
5.900000
3.000000
5.100000
1.800000

Output layer:
0.000000
0.000000
1.000000

Predicted Output:
0.000004
0.020944
0.979052

Error vector:
-0.000004
-0.020944
0.020948

||Error|| = 0.029623

Average Loss over a batch of 150 samples : 0.161075

Correct prediction : 142
Wrong prediction : 8
Correct matrix:
          Total Correct                Setosa            Versicolor             Virginica                 Model
             142.000000             50.000000             49.000000             43.000000              1.000000

Time taken by function: 1181561 microseconds
```

Figure 10.10: *The testing process with the optimal $W^{(0)}$ and $W^{(1)}$ for* 150 *testing sample (*100% *of the original data) with* ***FNN_1hiddenlayer_irisdataset_testing*** *function (****SymIntegration/Examples/Neural Networks/Test FNN with Iris DataFrame 1 Hidden layer with Steepest Descent/main.cpp****).*

The **FNN_1hiddenlayer_irisdataset_training** and **FNN_1hiddenlayer_irisdataset_testing** functions are an educative way to learn about neural network forward propagation, backpropagation

and the weights update algorithm. When we have finish till the last epoch, we know why we obtain the final weight that has the least error, you can see in the **src/artificialneuralnetworks.cpp** source code and you can uncomment or add something to print out the weights matrix, the output vector for the logit or output vector after the Softmax activation function.

It takes longer to code the training and testing in **vector<double>** and the **vector<vector<double>>** terms as a function for training and testing the neural network.

We want to compare it with the code that is using **class**, instead of function. This takes less codes and simpler, the structure is also better with **class**.

We have found a really good C++ codes with **class** from:
https://github.com/HimanshuMohanty-Git24/Neural-Network-CPP.git
it is made by Himanshu Mohanty with MIT License, and we use it then modify it, so the reader can compare the differences between the original code that we use, and the modified one that we then put into SymIntegration.

We will talk about the theory first: in C++, a **class** is a user-defined data type that serves as a blueprint or template for creating objects. It encapsulates data (attributes) and functions (methods) into a single unit, which is the core principle of Object-Oriented Programming.

A class is defined using the **class** keyword followed by the class name. The body is enclosed in curly braces and must end with a semicolon.

Key components:

- **Attributes**
  Variables declared inside a class to hold data.
- **Methods**
  Functions that belong to the class and define its behavior.
- **Access Specifiers**
  public: Members are accessible from outside the class.
  private: Members can only be accessed withing the class itself (default for class).
  protected: Members are accessible in inherited classes.
- **Constructors & Destructors**
  Special methods used to initialize objects when they are created and clean up resources when they are destroyed.

An object is an instance of a class. You create one by specifying the class name followed by the object name. The difference between class and struct is that class members are private by default while struct members are public by default and structs are used for simple data structure. You can use classes for complex objects with methods and inheritance, if you have tried to play computer games, or complex simulator, or physics engine, they are all using class so you can see the realistic details, effects, how far a person will fall back after hit by a shotgun at a certain distance, the lighting from sunlight that creates shadow of an object, almost like real life.

To create a C++ class across multiple files, you split it into a header file (**.h**) for the class blueprint and a source file (**.cpp**) for the logic. This modular approach speeds up compilation and keeps your project organized.

The header file contains the class declaration, which includes member variables like arrays and function prototypes.

The source file provides the actual logic for the functions declared in the header. You must include the include file **.h** to connect the definition to the declaration, in this book we compile all include files inside **symintegrationc++.h**, so we only need to put this one include file on top of the main source code. Use the class name and **::** before each function name to tell the compiler these functions belong to that specific class.

When compiling, you must include both the **main.cpp** and the **class.cpp** files (e.g., **g++ main.cpp class.cpp -o main**). We have created Makefile for this feed-forward neural network for iris dataset with **class**, so you only need to type **make** to compile it without the need to write the long **g++** compiling formula.

Now, we have learned the basic theory for **class**. Let see the code for the feed-forward neural network for iris dataset with **class**, first in the **include/artificialneuralnetworks.h** we will have

```
...
#ifndef FNN_NOHIDDENLAYER_IRIS_H
#define FNN_NOHIDDENLAYER_IRIS_H

class FNN_NoHiddenLayer_Iris {
       private:
       double weights[4][3];
       double bias[3]; // Fixed-size array member
       double learning_rate;

       public:
       FNN_NoHiddenLayer_Iris(); // Constructor

       vector<double> predict(const vector<double> &inputs);

       void train(vector<vector<double>> &X, vector<int> &y, int epochs);

       void save_model(const string &filename) ; // Void function declaration
       void load_model(const string& filename);
};

#endif

...
```

Code 76: *include/artificialneuralnetworks.h*

We include guards to prevent the file from being included multiple times, which causes errors.

Now for the **src/artificialneuralnetworks.cpp** source file

```
...
```

```
FNN_NoHiddenLayer_Iris::FNN_NoHiddenLayer_Iris()
{
	for (int i = 0; i < 4; ++i)
	{
		for (int j = 0; j < 3; ++j)
		{
			weights[i][j] = random_double(-0.5, 0.5);
		}
	}
	for (int j = 0; j < 3; ++j)
	{
		bias[j] = random_double(-0.5, 0.5);
	}
	learning_rate = 0.01;
}

double FNN_NoHiddenLayer_Iris::sigmoid(double x)
{
	return 1.0 / (1.0 + std::exp(-x));
}

double FNN_NoHiddenLayer_Iris::sigmoid_derivative(double x)
{
	return x * (1.0 - x);
}

vector<double> FNN_NoHiddenLayer_Iris::predict(const vector<double> &inputs)
{
	vector<double> outputs(3, 0.0);
	for (int j = 0; j < 3; ++j)
	{
		for (int i = 0; i < 4; ++i)
		{
			outputs[j] += inputs[i] * weights[i][j];
		}
		outputs[j] += bias[j];
		outputs[j] = SigmoidActivationfunction(outputs[j]);
	}
	return outputs;
}

...
```

Code 77: *src/artificialneuralnetworks.cpp*

The name of the class is **FNN_NoHiddenLayer_Iris** that initializes array for the weights and bias with fix size, and as we can see in the source file we have created functions of **predict, train, load_model, save_model** under this class.

After we have created the class, now we want to create the **main.cpp**, but in this case we will create two and name it as **train.cpp** and **predict.cpp**, the **train.cpp** has to be run first, and it will be used to obtain the best weights with least error to predict the iris species, while the **predict.cpp** will be used as a prediction cli (command line interface) where we can input the four variables for the irises (the sepal length, sepal width, petal length, and petal width) then the best model obtained from **train.cpp** will predict whether the inputted values belong to either iris Setosa, iris Versicolor or iris Virginica.

```
g++    -c -o predict.o predict.cpp
g++ -o predict -ggdb  predict.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Neural Networks/Test FNN Iris Dataset Predictor using Class with Steepest Descent ]# ./train

======== Iris Classification with Neural Network ========
Loading dataset...
DEBUG: Dataset loaded. Size: 150 samples, 4 features
DEBUG: Number of labels: 150
DEBUG: Label names:
DEBUG:  - Iris-setosa
DEBUG:  - Iris-versicolor
DEBUG:  - Iris-virginica
DEBUG: Train set size: 120
DEBUG: Test set size: 150

Initializing Neural Network...

Starting Training Process

Training Progress:

[==================================================] 100% Epoch 1000/1000 - Loss: 0.2084

Training completed in 1.0358 seconds.

Evaluating the model on the test set:
[==================================================] 100%

Accuracy: 96.67%

Confusion Matrix:

      Predicted >      Iris-setosa  Iris-versicolor   Iris-virginica
      Iris-setosa               50                0                0
  Iris-versicolor                0               47                3
   Iris-virginica                0                2               48
```

Figure 10.11: *The training process for the iris dataset without hidden layer and the activation function at the output layer is sigmoid with **FNN_NoHiddenLayer_Iris** class (**SymIntegration/Examples/Neural Networks/Test FNN Iris Dataset Predictor using Class with Steepest Descent/train.cpp**).*

```
Confusion Matrix:

     Predicted >      Iris-setosa  Iris-versicolor   Iris-virginica
      Iris-setosa              50                0                0
  Iris-versicolor               0               47                3
   Iris-virginica               0                2               48

Precision, Recall, and F1-score:

Iris-setosa:
  Precision: 1.00
  Recall:    1.00
  F1-score:  1.00

Iris-versicolor:
  Precision: 0.96
  Recall:    0.94
  F1-score:  0.95

Iris-virginica:
  Precision: 0.94
  Recall:    0.96
  F1-score:  0.95


The model weights:
              0.187405              0.959035             -2.161684
              1.850920             -2.421174             -2.644875
             -2.292653              0.714200              3.410055
             -1.058507             -2.371125              3.601225

The model bias:
0.378021
1.410220
-1.612257
```

Figure 10.12: *The evaluation of the model obtained from the training process for the iris dataset without hidden layer and the activation function at the output layer is sigmoid with* ***FNN_NoHiddenLayer_Iris*** *class (**SymIntegration/Examples/Neural Networks/Test FNN Iris Dataset Predictor using Class with Steepest Descent/train.cpp**).*

```
===========Iris Species Predictor===========


Enter four input features (or type 'exit' to quit):
1. Sepal length (cm): 4.5
2. Sepal width (cm):  5.1
3. Petal length (cm): 5.3
4. Petal width (cm):  4.6

Predicting...
[==============================] 100%

Prediction Results:
-------------------
Predicted Species: Iris-virginica

Probabilities:
 0.17%          Iris-setosa
 0.00%          Iris-versicolor
 99.99%         Iris-virginica


Enter four input features (or type 'exit' to quit):
1. Sepal length (cm):
```

Figure 10.13: *The prediction of which iris species belong to based on the given input of sepal length, sepal width, petal length, and petal width that user will give in the cli / terminal / xterm with* ***FNN_NoHiddenLayer_Iris*** *class (**SymIntegration/Examples/Neural Networks/Test FNN Iris Dataset Predictor using Class with Steepest Descent/predict.cpp**).*

We create another class as comparison, **FNN_1HiddenLayer_Iris**, we set the hidden layer to have 5 neurons (it can be changed by user for experimentation purpose) and the activation function at the hidden layer is sigmoid, then the activation function at the output layer is using softmax. The input neurons stay the same, 4 neurons, and the output neurons still 3 neurons / 3 classifications.

```
[==================================================] 99% Epoch 990/1000 - Loss:
[==================================================] 100% Epoch 1000/1000 - Loss
: 0.0393

Training completed in 3.1363 seconds.

Evaluating the model on the test set:
[==================================================] 100%

Accuracy: 99.33%

Confusion Matrix:

      Predicted >      Iris-setosa  Iris-versicolor   Iris-virginica
      Iris-setosa               50                0                0
  Iris-versicolor                0               49                1
   Iris-virginica                0                0               50

Precision, Recall, and F1-score:

Iris-setosa:
  Precision: 1.00
  Recall:    1.00
  F1-score:  1.00

Iris-versicolor:
  Precision: 1.00
  Recall:    0.98
  F1-score:  0.99

Iris-virginica:
  Precision: 0.98
  Recall:    1.00
  F1-score:  0.99
```

Figure 10.14: *The training process for the iris dataset with 1 hidden layer consisting of* 5 *neurons with sigmoid activation function at the hidden layer and the activation function at the output layer is softmax with* ***FNN_1HiddenLayer_Iris*** *class (****SymIntegration/Examples/Neural Networks/Test FNN Iris Dataset Predictor 1 Hidden Layer using Class with Steepest Descent/train.cpp****).*

```
The model weights:
              2.873544               1.181453              -1.193326
    7.713539              -0.435190
              4.772574               0.408036               3.434799
   -1.376258               9.565981
             -4.499668               0.315779              -0.710437
   -7.864477              -4.905031
             -6.072253              -0.013974              -1.957176
   -7.206076              -4.734083

The model hiddenweights:
              1.779509               1.400948              -0.717926
             -0.608578               2.371403               4.947981
              7.136587              -3.220675              -2.177980
              2.291308               3.554297              -3.039231
              2.814101               2.133686              -2.283374

The model bias in the hidden layer:
4.320484
0.342381
0.773372
4.789821
4.492461

The model bias in the output layer:
-0.669905
2.417736
4.461967
```

Figure 10.15: *The training process for the iris dataset with 1 hidden layer consisting of* 5 *neurons with sigmoid activation function at the hidden layer and the activation function at the output layer is softmax with* ***FNN_1HiddenLayer_Iris*** *class (****SymIntegration/Examples/Neural Networks/Test FNN Iris Dataset Predictor 1 Hidden Layer using Class with Steepest Descent/train.cpp****).*

```
Accuracy: 98.67%

Confusion Matrix:

      Predicted >      Iris-setosa  Iris-versicolor   Iris-virginica
       Iris-setosa              50                0                0
  Iris-versicolor               0               48                2
   Iris-virginica               0                0               50

Precision, Recall, and F1-score:

Iris-setosa:
  Precision: 1.00
  Recall:    1.00
  F1-score:  1.00

Iris-versicolor:
  Precision: 1.00
  Recall:    0.96
  F1-score:  0.98

Iris-virginica:
  Precision: 0.96
  Recall:    1.00
  F1-score:  0.98


The model weights:
              0.613851              1.612227             -1.453502              5.644846             -1.401964
              0.938255              7.728365              4.515118              1.287156              9.714031
             -1.532171             -7.595027             -1.284246             -7.360822             -5.407066
             -1.808880             -3.781532             -1.591036             -6.282293             -2.025311

The model hiddenweights:
              5.011217             -2.101302             -1.622205
              1.588085              2.883951             -2.936433
              4.346169             -0.099619             -1.798401
              1.526841              3.492190             -3.579990
              2.273748              1.714440             -2.228758

The model bias in the hidden layer:
0.430315
8.906203
1.564328
6.755378
6.497567

The model bias in the output layer:
-2.732761
2.047286
6.477377
```

Figure 10.16: *The training process for the iris dataset with 1 hidden layer consisting of* 5 *neurons with sigmoid activation function at the hidden layer and the activation function at the output layer is softmax with* ***FNN_1HiddenLayer_Iris*** *class (****SymIntegration/Examples/Neural Networks/Test FNN Iris Dataset Predictor 1 Hidden Layer using Class with Steepest Descent/train.cpp****).*

We have used standard gradient descent for training the neural network, now we want to compare if we are using a Conjugate Gradient (CG) algorithm for a feed-forward neural network. For a small dataset like Iris, it makes the neural network to converge faster than standard gradient descent. The approach involves representing the network weights as a vector, using backpropagation to compute the gradient, and using a line search to update weights along a conjugate direction, reducing MSE (Mean Squared Error).

```
Starting Training Process

Training Progress:

[==================================================] 100% Epoch 200/200 - Loss: 0.1882

Training completed in 16.8021 seconds.

Evaluating the model on the test set:
[==================================================] 100%

Accuracy: 98.67%

Confusion Matrix:

      Predicted >      Iris-setosa  Iris-versicolor   Iris-virginica
      Iris-setosa               50                0                0
  Iris-versicolor                0               48                2
   Iris-virginica                0                0               50

Precision, Recall, and F1-score:

Iris-setosa:
  Precision: 1.00
  Recall:    1.00
  F1-score:  1.00

Iris-versicolor:
  Precision: 1.00
  Recall:    0.96
  F1-score:  0.98

Iris-virginica:
  Precision: 0.96
  Recall:    1.00
  F1-score:  0.98


The model weights:
             -1.062172              0.314579             -0.529218             -1.157496             -0.764458
              0.164349             -0.401029              0.405046             -0.871413             -0.656090
             -0.529516             -0.368303             -0.055132              1.942797             -0.536916
              0.861717             -0.085202             -0.250408              2.666763             -0.354715

The model hiddenweights:
             -0.762174             -0.906450              0.233996
             -0.583184             -0.151056              0.851803
              0.150078              0.674797              0.686892
             -2.535796              0.608768              1.583251
              1.014366              0.083956             -0.385530
```

Figure 10.17: *The training process for the iris dataset with 1 hidden layer consisting of* 5 *neurons with ReLU activation function at the hidden layer and the activation function at the output layer is softmax using conjugate gradient method with* ***FNN_1HiddenLayer_Iris_ConjugateGradient*** *class (****SymIntegration/Examples/Neural Networks/Test FNN Iris Dataset Predictor 1 Hidden Layer using Class with Conjugate Gradient/train.cpp****).*

```
     Predicted >      Iris-setosa  Iris-versicolor   Iris-virginica
      Iris-setosa              50                0                0
  Iris-versicolor               0               48                2
   Iris-virginica               0                0               50

Precision, Recall, and F1-score:

Iris-setosa:
  Precision: 1.00
  Recall:    1.00
  F1-score:  1.00

Iris-versicolor:
  Precision: 1.00
  Recall:    0.96
  F1-score:  0.98

Iris-virginica:
  Precision: 0.96
  Recall:    1.00
  F1-score:  0.98


The model weights:
             -1.062172              0.314579             -0.529218             -1.157496             -0.764458
              0.164349             -0.401029              0.405046             -0.871413             -0.656090
             -0.529516             -0.368303             -0.055132              1.942797             -0.536916
              0.861717             -0.085202             -0.250408              2.666763             -0.354715

The model hiddenweights:
             -0.762174             -0.906450              0.233996
             -0.583184             -0.151056              0.851803
              0.150078              0.674797              0.686892
             -2.535796              0.608768              1.583251
              1.014366              0.083956             -0.385530

The model bias in the hidden layer:
-0.234676
-0.074763
0.776133
0.430983
-0.022482

The model bias in the output layer:
2.603302
0.581319
-3.850410

Model trained and saved successfully!
```

Figure 10.18: *The training process for the iris dataset with 1 hidden layer consisting of* 5 *neurons with ReLU activation function at the hidden layer and the activation function at the output layer is softmax using conjugate gradient method with* ***FNN_1HiddenLayer_Iris_ConjugateGradient*** *class (****SymIntegration/Examples/Neural Networks/Test FNN Iris Dataset Predictor 1 Hidden Layer using Class with Conjugate Gradient/train.cpp****).*

The conjugate gradient method for training neural network on this iris dataset able to achieve 98.67% of accuracy with loss of 0.1882 within 200 epochs for training of 80% of data (120 samples) and the testing is 150 samples, 100% of data. It is sometimes said that vey high accuracy is overfitting and 80% of accuracy is the best because it is not too high nor too low, for some cases perhaps it is overfitting, but it depends on the case or problem that we are working on.

```
Initializing Neural Network...

Starting Training Process

Training Progress:

[==================================================] 100% Epoch 200/200 - Loss: 0.0957

Training completed in 16.1711 seconds.

Evaluating the model on the test set:
[==================================================] 100%

Accuracy: 98.00%

Confusion Matrix:

      Predicted >      Iris-setosa  Iris-versicolor   Iris-virginica
      Iris-setosa               50                0                0
  Iris-versicolor                0               49                1
   Iris-virginica                0                2               48

Precision, Recall, and F1-score:

Iris-setosa:
  Precision: 1.00
  Recall:    1.00
  F1-score:  1.00

Iris-versicolor:
  Precision: 0.96
  Recall:    0.98
  F1-score:  0.97

Iris-virginica:
  Precision: 0.98
  Recall:    0.96
  F1-score:  0.97


The model weights:
              -2.246316              0.032038             -0.108140             -0.698284              0.110260
              -1.998872             -1.095444              1.460190              0.234610             -0.311328
               3.769575             -0.062567             -1.270523             -0.873604             -0.323579
               2.649634             -0.119740              0.108883             -1.249114              0.210217

The model hiddenweights:
              -1.363625             -0.001630              1.513059
               0.440795             -1.212376              1.172248
               1.322513             -1.098335             -0.673634
              -0.451245              1.459457             -0.940929
              -0.701461             -1.498091             -0.666894
```

Figure 10.19: *The training process for the iris dataset with 1 hidden layer consisting of* 5 *neurons with ReLU activation function at the hidden layer and the activation function at the output layer is softmax using conjugate gradient method with* ***FNN_1HiddenLayer_Iris_ConjugateGradient*** *class (****SymIntegration/Examples/Neural Networks/Test FNN Iris Dataset Predictor 1 Hidden Layer using Class with Conjugate Gradient/train.cpp****).*

```
Confusion Matrix:

      Predicted >      Iris-setosa  Iris-versicolor   Iris-virginica
       Iris-setosa              50                0                0
  Iris-versicolor               0               49                1
   Iris-virginica               0                2               48

Precision, Recall, and F1-score:

Iris-setosa:
  Precision: 1.00
  Recall:    1.00
  F1-score:  1.00

Iris-versicolor:
  Precision: 0.96
  Recall:    0.98
  F1-score:  0.97

Iris-virginica:
  Precision: 0.98
  Recall:    0.96
  F1-score:  0.97


The model weights:
             -2.246316              0.032038             -0.108140             -0.698284              0.110260
             -1.998872             -1.095444              1.460190              0.234610             -0.311328
              3.769575             -0.062567             -1.270523             -0.873604             -0.323579
              2.649634             -0.119740              0.108883             -1.249114              0.210217

The model hiddenweights:
             -1.363625             -0.001630              1.513059
              0.440795             -1.212376              1.172248
              1.322513             -1.098335             -0.673634
             -0.451245              1.459457             -0.940929
             -0.701461             -1.498091             -0.666894

The model bias in the hidden layer:
-0.653320
0.002011
1.277995
-1.167769
0.554871

The model bias in the output layer:
-0.650087
2.571865
-1.626273

Model trained and saved successfully!
```

Figure 10.20: *The training process for the iris dataset with 1 hidden layer consisting of* 5 *neurons with ReLU activation function at the hidden layer and the activation function at the output layer is softmax using conjugate gradient method with* ***FNN_1HiddenLayer_Iris_ConjugateGradient*** *class (**SymIntegration/Examples/Neural Networks/Test FNN Iris Dataset Predictor 1 Hidden Layer using Class with Conjugate Gradient/train.cpp**).*

In this C++ class code we are using a beta that we create on our own, since Polak-Ribiere, Fletcher Reeves or even OFR [34], they cannot converge and only lead to flat loss or oscillating at high loss or can only predict one class / stuck at local minimum even after a lot of re-training.

The beta formula that we are using is:

$$\beta_k^G = \begin{cases} \frac{||g_k||}{||g_{k-1}||}, & \text{if } \beta_{k-1} \geq \frac{||g_k||}{||g_{k-1}||} \\ 0.01, & \text{if } \beta_{k-1} < \frac{||g_k||}{||g_{k-1}||} \end{cases}$$

The 0.01 is the smaller decimal value of the input variables, when we see the sepal length, sepal width, petal length and petal width, they are all have 1 decimal number. This beta perhaps only work for iris dataset and need more improvements to be generalized for other dataset. This beta also sometimes stuck or has flat loss, but if you call the code again for another training (call **./train** again) then it will converge and has an amazing small loss ± 0.1 after 200 epochs. The code can be seen in **src/artificialneuralnetworks.cpp**

```
...

FNN_1HiddenLayer_Iris_ConjugateGradient::FNN_1HiddenLayer_Iris_ConjugateGradient()
```

```
{
	int n_classification = 3;
	int n_hiddenlayer = 5;
	int n_input = 4;
	for (int i = 0; i < n_input; ++i)
	{
		for (int j = 0; j < n_hiddenlayer; ++j)
		{
			weights[i][j] = He_initialization(n_input);
		}
	}
	for (int j = 0; j < n_hiddenlayer; ++j)
	{
		for (int k = 0; k < n_classification; ++k)
		{
			hiddenweights[j][k] = He_initialization(n_input);
		}
	}
	for (int j = 0; j < n_hiddenlayer; ++j)
	{
		biashidden[j] = He_initialization(n_input);
	}
	for (int k = 0; k < n_classification; ++k)
	{
		biasoutput[k] = He_initialization(n_input);
	}
	learning_rate = 0.51;
}

vector<double> FNN_1HiddenLayer_Iris_ConjugateGradient::predict(const vector<
    double> &inputs)
{
	int n_classification = 3;
	int n_hiddenlayer = 5;
	int n_input = 4;
	vector<double> hiddenlayer(n_hiddenlayer, 0.0);
	vector<double> outputs(n_classification, 0.0);
	for (int j = 0; j < n_hiddenlayer; ++j)
	{
		for (int i = 0; i < n_input; ++i)
		{
			hiddenlayer[j] += inputs[i] * weights[i][j];
		}
		hiddenlayer[j] += biashidden[j];
		hiddenlayer[j] = ReLUActivationfunction(hiddenlayer[j]);
		//hiddenlayer[j] = TanhActivationfunction(hiddenlayer[j]);
	}
	for (int k = 0; k < n_classification; ++k)
```

```
        {
                for (int j = 0; j < n_hiddenlayer; ++j)
                {
                        outputs[k] += hiddenlayer[j] * hiddenweights[j][k];
                }
                outputs[k] += biasoutput[k];
        }
        vector<double>predicted_output_softmax =
            SoftMax_vectorresult_activationfunction(outputs);
        return predicted_output_softmax;
}

void FNN_1HiddenLayer_Iris_ConjugateGradient::train(vector<vector<double>> &X,
    vector<int> &y, int epochs)
{
...
...
}
```

Code 78: *src/artificialneuralnetworks.cpp*

The class that handles the training and testing for the iris dataset specifically with 1 hidden layer, we initialize the weights and bias with He initialization because we are using ReLU as the activation function on the hidden layer.

iii. Functions in SymIntegration Related to Feed-Forward Neural Network

[SI*] We will list the basic functions that can be used for computing feed-forward neural networks:
FNN_1hiddenlayer(vector<double>& x, vector<vector<double>>& W, vector<vector<double>>& W_{hidden}, vector<double>& b, vector<double>& y)
this function computes and updates the weights parameter from input to output layer, with only 1 hidden layer that has m nodes, we need the input of the vector $\boldsymbol{x}$ as the vector of input layer with size of n, vector $\boldsymbol{b}$ as the bias with size of m, vector $\boldsymbol{y}$ as the vector of output layer, the matrix W as the $m \times n$ matrix that represents the weights from the input layer to the hidden layer, the matrix W_{hidden} as the $k \times m$ matrix from the hidden layer to the output layer. The activation function is using sigmoid, the parameter η (learning rate) is set to 0.51, and the iteration is set to 5, and these parameters can be modified in **src/artificialneuralnetwork.cpp**.

FNN_1hiddenlayer(vector<double>& x, vector<double>& y, int n_{hidden})
this function computes the same thing as the previous function, but we only need to give the input of $\boldsymbol{x}$ the input vector, and $\boldsymbol{y}$ the output vector, and n_{hidden} as the number of nodes in the hidden layer. The weight matrix W and the hiddenweight matrix W_{hidden} will be generated with Mersenne Twister engine with uniform distribution from 0.1 to 1. The activation function is using sigmoid, the parameter η (learning rate) is set to 0.51, and the iteration is set to 5.

```
Feed-forward Neural Network with 1 hidden layer type 2:

Weights from input to hidden layer:
               0.253954               0.490407
               0.606950               0.590103

Weights from hidden layer to output layer:
               0.219816               0.417713
               0.720708               0.139265

Hidden layer:
0.606391
0.651469

Predicted Output:
0.599990
0.628960

Error vector:
-0.099990
-0.258960

||Error|| = 0.277594

********************************************************************************

                        Backpropagation iteration

********************************************************************************

Learning rate: 0.510000

Iteration = 1

Delta output vector:
-0.023998
-0.060433

Delta hidden layer vector:
-0.011655
-0.004187

New weights matrix:
               0.251873               0.486246
               0.606203               0.588608

New hiddenweights matrix:
               0.212395               0.409740
               0.702018               0.119186

Hidden layer:
0.605522
0.651172
```

Figure 10.21: *The computation of backpropagation process for the feed-forward neural network with **FNN_1hiddenlayer()** function (**SymIntegration/Examples/Neural Networks/Test Backpropagation FNN/main.cpp**).*

```
Iteration = 5

Delta output vector:
-0.021779
-0.055120

Delta hidden layer vector:
-0.009554
-0.002689

New weights matrix:
               0.244511               0.471521
               0.603901               0.584005

New hiddenweights matrix:
               0.184541               0.379744
               0.631648               0.043407

Hidden layer:
0.602440
0.650257

Predicted Output:
0.588582
0.600790

Error vector:
-0.088582
-0.230790

||Error|| = 0.247206

Time taken by function: 6397 microseconds
```

Figure 10.22: *The computation of backpropagation process for the feed-forward neural network with* ***FNN_1hiddenlayer()*** *function (**SymIntegration/Examples/Neural Networks/Test Backpropagation FNN/main.cpp**).*

FNN_nohiddenlayer_irisdataset_training(vector<vector<double>>& X, vector<string>& y)
this function trains the neural network for iris dataset, with no hidden layer, so the input matrix X is a matrix of size $n \times 4$, 4 is the input variables (sepal length, sepal width, petal length, petal width) and n is the number of data that will be trained, normally 80% of total data, while vector y is a vector of string of the classification for the input variables, the classification consists of Iris Setosa, Iris Versicolor, and Iris Virginica. The activation function at the output layer is set to sigmoid, and can be changed by the user in **src/artificialneuralnetworks.cpp**.

FNN_nohiddenlayer_irisdataset_testing(vector<vector<double>>& X, vector<string>& y)
this function tests the optimal weight obtained from the function:
FNN_nohiddenlayer_irisdataset_training(vector<vector<double>>& X, vector<string>& y)
for iris dataset, with no hidden layer, so the input matrix X is a matrix of size $m \times 4$, 4 is the input variables (sepal length, sepal width, petal length, petal width) and m is the number of data that will be tested, normally 20% of total data, it is the data that is not trained that will be tested, while vector y is a vector of string of the classification for the input variables, the classification consists of Iris Setosa, Iris Versicolor, and Iris Virginica. The activation function at the output layer is set to sigmoid, and can be changed by the user in **src/artificialneuralnetworks.cpp**.

FNN_1hiddenlayer_irisdataset_training(vector<vector<double>>& X, vector<string>& y)
this function trains the neural network for iris dataset, with 1 hidden layer and 5 number of neurons in the hidden layer by default, so the input matrix X is a matrix of size $n \times 4$, 4 is the input variables (sepal length, sepal width, petal length, petal width) and n is the number of data that will be trained, normally 80% of total data, while vector y is a vector of string of the classification for the input variables, the classification consists of Iris Setosa, Iris Versicolor, and Iris Virginica. The activation function at the output layer is softmax,

the activation function at the hidden layer is sigmoid, and can be changed by the user in **src/artificialneuralnetworks.cpp**.

FNN_1hiddenlayer_irisdataset_testing(vector<vector<double>>& X, vector<string>& y) this function tests the optimal weight obtained from the function:
FNN_1hiddenlayer_irisdataset_training(vector<vector<double>>& X, vector<string>& y) for iris dataset, with 1 hidden layer and 5 number of neurons in the hidden layer by default, so the input matrix X is a matrix of size $m \times 4$, 4 is the input variables (sepal length, sepal width, petal length, petal width) and m is the number of data that will be tested, normally 20% of total data, it is the data that is not trained that will be tested, while vector y is a vector of string of the classification for the input variables, the classification consists of Iris Setosa, Iris Versicolor, and Iris Virginica. The activation function at the output layer is softmax, the activation function at the hidden layer is sigmoid, and can be changed by the user in **src/artificialneuralnetworks.cpp**.

IV. Supervised Learning: Support Vector Machines (SVM)

[SI*] Support vector machines is a classification method based on maximum margin linear discriminants, that is, the goal is to find the optimal hyperplane (a line in 2D or a flat plane in 3D) that maximizes the gap or margin between the classes. Further, we can use the kernel trick to find the optimal nonlinear decision boundary between classes, which corresponds to a hyperplane in some high-dimensional "nonlinear" space.

V. Unsupervised Learning: Principal Component Analysis (PCA)

[SI*] Unsupervised learning is the second type of machine learning which finds patterns in unlabeled data, grouping similar items (clustering) or reducing data dimensions.
Principla Component Analysis reduces the complexity (dimensionality) of the data by combining highly correlated features into fewer, uncorrelated "principal components" that capture the most variance.

VI. Hybrid Learning: Kernel Methods

[SI*] A mathematical technique that takes non-linearly separable data and implicitly projects it into a higher-dimensional space where a flat line or plane can separate them. Crucially, it does this without the heavy computations of explicitly mapping the data.

[SI*] It is important to find a suitable data representation that facilitates data analysis. For example, for complex data such as text, sequences, images, and so on, we must typically extract or construct a set of attributes or features, so that we can represent the data instances as multivariate vectors.
That is, given a data instance vector $\boldsymbol{x}$ (e.g., a sequence), we need to find a mapping ϕ, so that $\phi(\boldsymbol{x})$ is the feature representation of $\boldsymbol{x}$. Even when the input data is a numeric data matrix, if we wish to discover nonlinear relationships among the attributes, then a nonlinear mapping ϕ may be used, so that $\phi(\boldsymbol{x})$ represents a vector in the corresponding high-dimensional space comprising nonlinear attributes.
We use the term input space to refer to the data space for the input data $\boldsymbol{x}$ and feature space

to refer to the space of mapped vectors $\phi(\boldsymbol{x})$. Thus, given a set of data objects or instances $\boldsymbol{x}_i$, and given a mapping function ϕ, we can transform them into feature vectors $\phi(\boldsymbol{x}_i)$, which then allows us to analyze complex data instances via numeric analysis methods.

[SI*] **Sequence-based Features Example**

Consider a dataset of DNA sequences over the alphabet $\Sigma = \{A, C, G, T\}$. One simple feature space is to represent each sequence in terms of the probability distribution over symbols in Σ. That is, given a sequence $\boldsymbol{x}$ with length $|\boldsymbol{x}| = m$, the mapping into feature space is given as

$$\phi(\boldsymbol{x}) = \{P(A), P(C), P(G), P(T)\}$$

where $P(s) = \frac{n_s}{m}$ is the probability of observing symbol $s \in \Sigma$, and n_s is the number of times s appears in sequence $\boldsymbol{x}$. Here the input space is the set of sequence Σ^*, and the feature space is $\mathbb{R}^4$. For example, if $\boldsymbol{x} = ACAGCAGTA$, with $m = |\boldsymbol{x}| = 9$, since A occurs four times, C and G occur twice, and T occurs once, we have

$$\phi(\boldsymbol{x}) = \left(\frac{4}{9}, \frac{2}{9}, \frac{2}{9}, \frac{1}{9}\right) = (0.44, 0.22, 0.22, 0.11)$$

The mapping ϕ now allows one to compute the statistics over the data smaple to make inferences about the population. For example, we may compute the mean symbol composition. We can also define the distance between any two sequences, for example

$$\begin{aligned}\delta(\boldsymbol{x}, \boldsymbol{y}) &= ||\phi(\boldsymbol{x}) - \phi(\boldsymbol{y})|| \\ &= \sqrt{(0.44 - 0.4)^2 + (0.22 - 0.2)^2 + (0.22 - 0.4)^2 + (0.11 - 0)^2} \\ &= 0.22\end{aligned}$$

We can compute larger feature spaces by considering, for exmaple, the probability distribution over all substrings or words of size up to k over the alphabet Σ, and so on.

[SI*] **Nonlinear Features Example**

Consider the mapping ϕ that takes as input a vector $\boldsymbol{x} = (x_1, x_2) \in \mathbb{R}^2$ and maps it to a "quadratic" feature space via the nonlinear mapping

$$\phi(\boldsymbol{x}) = (x_1^2, x_2^2, \sqrt{2}x_1x_2)$$

with $\phi \in \mathbb{R}^3$. For example, the point $\boldsymbol{x} = (5.9, 3)$ is mapped to the vector

$$\phi(\boldsymbol{x}) = (5.9^2, 3^2, \sqrt{2}(5.9)(3)) = (34.81, 9, 25.03)$$

The main benefit of this transformation is that we may apply well-known linear analysis methods in the feature space. However, because the features are nonlinear combinations of the original attributs, this allows us to extract / mine the nonlinear patterns and relationships. For someone aware enough and have read Linear Algebra book this is actually a linear transformation from $\mathbb{R}^n to \mathbb{R}^m$.

[SI*] Whereas mapping into feature space allows one to analyze the data via algebraic and probabilistic modeling, the resulting feature space is usually very high-dimensional; it may even be infinite dimensional. Thus, transforming all the input points into feature space can be very expensive.

Kernel methods avoid explicitly transforming each points $\boldsymbol{x}$ in the input space into the mapped point $\phi(\boldsymbol{x})$ in the feature space. Instead, the input objects are represented via their $n \times n$ pairwise similarity values. The similarity function, called a kernel, is chosen so that it

represents a dot product in some high-dimensional feature space, yet it can be computed without directly constructing $\phi(\boldsymbol{x})$.

Let $\mathbb{I}$ denote the input space, qwhich can comprise any arbitrary set of objects, and let $\boldsymbol{d} = \{\boldsymbol{x}_i\} \in \mathbb{I}, i = 1, \ldots, n$ be a dataset comprising n objects in the input space. We can represent the pairwise similarity values between points in $\boldsymbol{d}$ via the $n \times n$ kernel matrix, defined as

$$K = \begin{bmatrix} K(\boldsymbol{x}_1, \boldsymbol{x}_1) & K(\boldsymbol{x}_1, \boldsymbol{x}_2) & \ldots & K(\boldsymbol{x}_1, \boldsymbol{x}_n) \\ K(\boldsymbol{x}_2, \boldsymbol{x}_1) & K(\boldsymbol{x}_2, \boldsymbol{x}_2) & \ldots & K(\boldsymbol{x}_2, \boldsymbol{x}_n) \\ \vdots & \vdots & \ddots & \vdots \\ K(\boldsymbol{x}_n, \boldsymbol{x}_1) & K(\boldsymbol{x}_n, \boldsymbol{x}_2) & \ldots & K(\boldsymbol{x}_n, \boldsymbol{x}_n) \end{bmatrix}$$

where $K : \mathbb{I} \times \mathbb{I} \to \mathbb{R}$ is a kernel function on any two points in input space. However, we require that K corresponds to a dot product in some feature space. That is, for any $\boldsymbol{x}_i, \boldsymbol{x}_j \in \mathbb{I}$, the kernel function should satisfy the condition

$$K(\boldsymbol{x}_i, \boldsymbol{x}_j) = \phi(\boldsymbol{x}_i)^T \phi(\boldsymbol{x}_j) \tag{10.67}$$

where $\phi : \mathbb{I} \to \mathbb{F}$ is a mapping from the input space $\mathbb{I}$ to the feature space $\mathbb{F}$. Intuitively, this means that we should be able to compute the value of the dot product using the original input representation $\boldsymbol{x}$, without having recourse to the mapping $\phi(\boldsymbol{x})$. Obviously, not just any arbitrary function can be used as a kernel; a valid kernel function must satisfy certain conditions so that Eq. (10.67) remains valid.

It is important to remark that the transpose operator for the dot product applies only when $\mathbb{F}$ is an abstract vector space with an inner product, the kernel is written as $K(\boldsymbol{x}_i, \boldsymbol{x}_j) = < \phi(\boldsymbol{x}_i), \phi(\boldsymbol{x}_j) >$. However, for convenience we use the transpose operator, when $\mathbb{F}$ is an inner product space it should be understood that

$$\phi(\boldsymbol{x}_i)^T \phi(\boldsymbol{x}_j) = < \phi(\boldsymbol{x}_i), \phi(\boldsymbol{x}_j) >$$

[SI*] **Linear Kernel Example**

Consider the identity mapping, $\phi(\boldsymbol{x}) \to \boldsymbol{x}$. This naturally leads to the linear kernel, which is simply the dot product between two input vectors, and thus satisfies Eq. (10.67):

$$\phi(\boldsymbol{x})^T \phi(\boldsymbol{y}) = \boldsymbol{x}^T \boldsymbol{y} = K(\boldsymbol{x}, \boldsymbol{y})$$

For example, consider the six points from the Iris dataset:

$$\boldsymbol{x}_1 = \begin{bmatrix} 5.1 \\ 3.5 \\ 1.4 \\ 0.2 \end{bmatrix}, \quad \boldsymbol{x}_2 = \begin{bmatrix} 4.9 \\ 3.0 \\ 1.4 \\ 0.2 \end{bmatrix}, \quad \boldsymbol{x}_3 = \begin{bmatrix} 4.7 \\ 3.2 \\ 1.3 \\ 0.2 \end{bmatrix}, \quad \boldsymbol{x}_4 = \begin{bmatrix} 4.6 \\ 3.1 \\ 1.5 \\ 0.2 \end{bmatrix}, \quad \boldsymbol{x}_5 = \begin{bmatrix} 5.0 \\ 3.6 \\ 1.4 \\ 0.2 \end{bmatrix}, \quad \boldsymbol{x}_6 = \begin{bmatrix} 5.4 \\ 3.9 \\ 1.7 \\ 0.4 \end{bmatrix}$$

The formula to compute the entry for the i-th row and j-th column in the kernel matrix is

$$K(\boldsymbol{x}_i, \boldsymbol{x}_j) = \boldsymbol{x}_i^T \boldsymbol{x}_j$$

The kernel matrix for the linear kernel is

$$K = \begin{bmatrix} 40.26 & 37.49 & 37.03 & 36.45 & 40.10 & 43.65 \\ 37.49 & 35.01 & 34.49 & 33.98 & 37.30 & 40.62 \\ 37.03 & 34.49 & 34.06 & 33.53 & 36.88 & 40.15 \\ 36.45 & 33.98 & 33.53 & 33.06 & 36.30 & 39.56 \\ 40.10 & 37.30 & 36.88 & 36.30 & 39.96 & 43.50 \\ 43.65 & 40.62 & 40.15 & 39.56 & 43.50 & 47.42 \end{bmatrix}$$

VII. Reinforcement Learning

[SI*] Reinforcement learning is the third categories in machine learning, the algorithms learns to make decisions through trial and error in an interactive environment, aiming to maximize some notion of cumulative reward.

VIII. Deep Learning: Convolutional Neural Network

[SI*] The convolutional neural network (CNN) is a type of artificial neural network that is effective for processing spatial relationships in complex images. CNN scans the image (or sequence of pixels) to learn spatial patterns, stroke thickness, curves, and angles. It is perfect when you are building an isolated character or digit classifier (e.g. recognizing single digits $0-9$ or single alphabet letters).
The convolutional neural networks (CNNs) are mainly designed for image data they use convolution operations to reduce the number of parameters needed, it is one of two categories in deep learning (the other one is RNN / recurrent neural networks), deep learning is a subset of machine learning that involves neural networks with multiple layers. By mimicking the human visual system, CNNs automatically learn to detect spatial patterns-such as edges, shapes, and textures-without requiring manual feature extraction.
CNNs are specifically designed to process data with a grid-like topology, such as images, represented as grids of pixels. CNNs serve as a cornerstone technology in the field of computer vision and posses robust capabilities for processing multimedia data. Their applications beside digit recognition ranging from facial recognition (for security and surveillance), object classification to medical image analysis (assisting professionals by detecting anomalies in X-rays, MRIs, and CT scans).

[SI*] **Key Components of CNN**
A standard CNN architecture consists of three main types of layers:

- **Convolutional Layer**
  The core building block. It applies learnable filters (kernels) that slide over the input data to calculate dot products, generating "feature maps" that highlighting specific patterns like lines or curves..
- **Pooling Layer**
  Applied after convolutional layers to reduce the spatial dimensions of the data (down-sampling). This minimizes computational load, limits memory usage, and helps prevent the model from overfitting.
- **Fully Connected (Dense) Layer**
  Flattens the multi-dimensional feature maps into a 1D vector and connects every neuron to every neuron in the preceding and succeeding layers. It performs high-level reasoning and outputs the final classification scores or probabilities.

Between these layers, activation functions (most commonly ReLU) are used to introduce non-linearity into the network. This allows the model to learn complex patterns and relationships rather than just simple linear combinations.

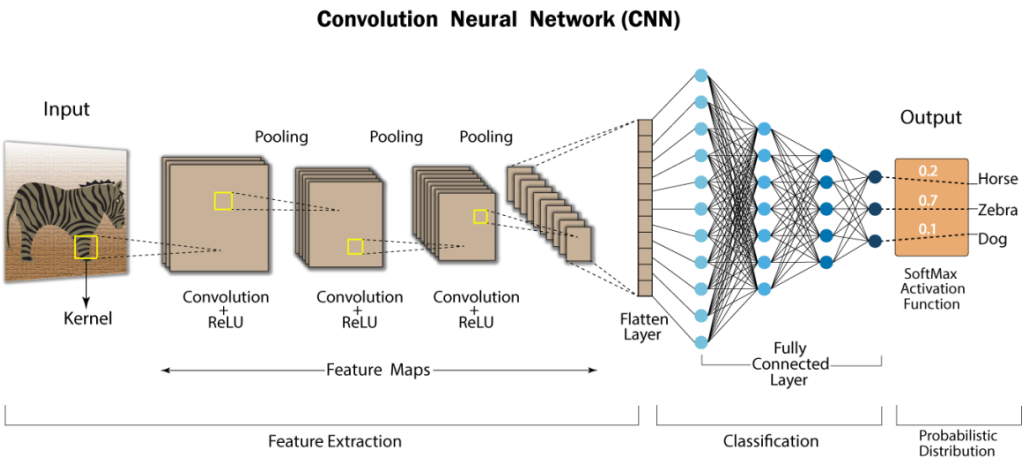

Figure 10.23: *The illustration of convolutional neural network pipeline for image classification.*

[SI*] **CNN: How Computers See Images**

When we have an image saved in a computer, the computer sees the image as a matrix of numbers $[0, 255]$, it can be represented as 3D matrix of size $1080 \times 1920 \times 3$, 1080 represents the width of the image, 1920 represents the height of the image and 3 is the depth to represents the RGB color for each pixel.

RGB can be represented as vector of size 3, with domain $[0, 255] \in \mathbb{N}$, for example:

$$
\begin{aligned}
[255, 0, 0] &\rightarrow \text{Red} \\
[0, 255, 0] &\rightarrow \text{Green} \\
[0, 0, 255] &\rightarrow \text{Blue} \\
[0, 0, 0] &\rightarrow \text{Black} \\
[255, 255, 255] &\rightarrow \text{White} \\
[0, 255, 255] &\rightarrow \text{Cyan}
\end{aligned}
$$

If we are having a matrix with depth of 1, e.g. a picture of $1080 \times 1920 \times 1$, then we are dealing with a grayscale image, it means each pixel can take a value of $[0, 255]$, with 0 represents black, and 255 represents white, the image will not be colorful as the $3D$ matrix, because it is grayscale, it is easier and faster for CNN computation with grayscale matrix.

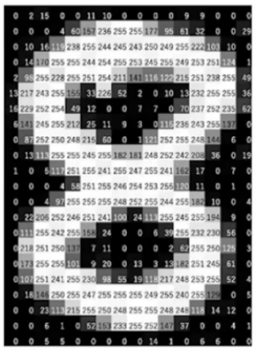

Figure 10.24: *An example of a grayscale image.*

[SI*] **How Convolutional Neural Network Works**

Suppose we have an image of size $6 \times 6 \times 1$, a yellowscale image with the yellowscale representation of the image like this:

$$I = \begin{bmatrix} 0 & 0 & 0 & 200 & 100 & 0 \\ 0 & 200 & 100 & 0 & 0 & 0 \\ 0 & 200 & 100 & 100 & 0 & 0 \\ 0 & 0 & 0 & 100 & 200 & 0 \\ 0 & 0 & 200 & 100 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

to extract a feature of that image we will use a kernel, M_{kernel}, a square matrix of size k, with $k < w$ and $k < h$, w is the width of the input image, and h is the height of the input image.

Thus we will take submatrix of $k \times k$ from the input image starting from the top left moving to the right and then down till finishes at the bottom right and each time we perform the operation with the kernel matrix M_{kernel} by using hadamard product (represented by $\oplus$), after the hadamard product operation we add all the number in the resulting matrix to obtain a single number that will be the input for the output map.

Suppose we will use the kernel of vertical edges of size 3:

$$M_{kernel} = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

We will start to show the example from the top left of matrix I, we will have

$$I_1 = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 200 & 100 \\ 0 & 200 & 100 \end{bmatrix}$$

multiply it with the kernel we will obtain

$$I_1 \oplus M_{kernel} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 200 & 100 \\ 0 & 200 & 100 \end{bmatrix} \oplus \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & -100 \\ 0 & 0 & -100 \end{bmatrix}$$

we add up all the entries in the resulting matrix thus we will obtain

$$o_{11} = \sum_{ij} I_1 \oplus M_{kernel} = -200$$

then we will move to the right 1 pixel and working on the next submatrix

$$I_2 = \begin{bmatrix} 0 & 0 & 200 \\ 200 & 100 & 0 \\ 200 & 100 & 100 \end{bmatrix}$$

multiply it with the kernel we will obtain

$$I_2 \oplus M_{kernel} = \begin{bmatrix} 0 & 0 & 200 \\ 200 & 100 & 0 \\ 200 & 100 & 100 \end{bmatrix} \oplus \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & -200 \\ 200 & 0 & 0 \\ 200 & 0 & -100 \end{bmatrix}$$

we add up all the entries in the resulting matrix thus we will obtain

$$o_{12} = \sum_{ij} I_2 \oplus M_{kernel} = 100$$

we iterate until the last bottom right submatrix of the input matrix, then we will have the output map of size $o_w \times o_h$ with

$$o_w = w - k + 1$$
$$o_h = h - k + 1$$

thus we will obtain the output map like this:

$$O = \begin{bmatrix} o_{11} & o_{12} & \cdots & o_{1,w-k+1} \\ o_{21} & o_{22} & \cdots & o_{2,w-k+1} \\ \vdots & \vdots & \ddots & \vdots \\ o_{h-k+1,1} & o_{h-k+1,2} & \cdots & o_{h-k+1,w-k+1} \end{bmatrix} = \begin{bmatrix} -200 & 100 & 100 & 300 \\ -200 & 200 & 0 & 200 \\ -300 & -100 & 100 & 300 \\ -200 & -200 & 0 & 200 \end{bmatrix}$$

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration CNN Compute Output Map from Convolution Operation ]# ./main

Matrix A :
               0.000000               0.000000               0.000000             200.000000             100.000000               0.000000
               0.000000             200.000000             100.000000               0.000000               0.000000               0.000000
               0.000000             200.000000             100.000000             100.000000               0.000000               0.000000
               0.000000               0.000000               0.000000             100.000000             200.000000               0.000000
               0.000000               0.000000             200.000000             100.000000               0.000000               0.000000
               0.000000               0.000000               0.000000               0.000000               0.000000               0.000000

Kernel :
               1.000000               0.000000              -1.000000
               1.000000               0.000000              -1.000000
               1.000000               0.000000              -1.000000

i: 0

Submatrix at j: 0
               0.000000               0.000000               0.000000
               0.000000             200.000000             100.000000
               0.000000             200.000000             100.000000

Hadamard product:
               0.000000               0.000000              -0.000000
               0.000000               0.000000            -100.000000
               0.000000               0.000000            -100.000000

Submatrix at j: 1
               0.000000               0.000000             200.000000
             200.000000             100.000000               0.000000
             200.000000             100.000000             100.000000

Hadamard product:
               0.000000               0.000000            -200.000000
             200.000000               0.000000              -0.000000
             200.000000               0.000000            -100.000000

Submatrix at j: 2
               0.000000             200.000000             100.000000
             100.000000               0.000000               0.000000
             100.000000             100.000000               0.000000

Hadamard product:
               0.000000               0.000000            -100.000000
             100.000000               0.000000              -0.000000
             100.000000               0.000000              -0.000000
```

Figure 10.25: *The computation of an output map from an input image that is represented by matrix A and the kernel to detect vertical edges with* ***CNN_2DConvolutionOperation()*** *function (****SymIntegration/Examples/Neural Networks/Test CNN Compute Output Map from Convolution Operation/main.cpp****).*

```
i: 3

Submatrix at j: 0
              0.000000              0.000000              0.000000
              0.000000              0.000000            200.000000
              0.000000              0.000000              0.000000

Hadamard product:
              0.000000              0.000000             -0.000000
              0.000000              0.000000           -200.000000
              0.000000              0.000000             -0.000000

Submatrix at j: 1
              0.000000              0.000000            100.000000
              0.000000            200.000000            100.000000
              0.000000              0.000000              0.000000

Hadamard product:
              0.000000              0.000000           -100.000000
              0.000000              0.000000           -100.000000
              0.000000              0.000000             -0.000000

Submatrix at j: 2
              0.000000            100.000000            200.000000
            200.000000            100.000000              0.000000
              0.000000              0.000000              0.000000

Hadamard product:
              0.000000              0.000000           -200.000000
            200.000000              0.000000             -0.000000
              0.000000              0.000000             -0.000000

Submatrix at j: 3
            100.000000            200.000000              0.000000
            100.000000              0.000000              0.000000
              0.000000              0.000000              0.000000

Hadamard product:
            100.000000              0.000000             -0.000000
            100.000000              0.000000             -0.000000
              0.000000              0.000000             -0.000000

Output map:
           -200.000000            100.000000            100.000000            300.000000
           -200.000000            200.000000              0.000000            200.000000
           -300.000000           -100.000000            100.000000            300.000000
           -200.000000           -200.000000              0.000000            200.000000

Time taken by function: 5537 microseconds
```

Figure 10.26: *The computation of an output map from an input image that is represented by matrix A and the kernel to detect vertical edges with* ***CNN_2DConvolutionOperation()*** *function (****SymIntegration/Examples/Neural Networks/Test CNN Compute Output Map from Convolution Operation/main.cpp****).*

To detect many different features of an image we usually use multiple kernels, such as vertical edges kernel, horizontal edges kernel, sharpening kernel. When we use multiple kernels, we multiply each kernel matrix by the corresponding input image matrix from top left to bottom right. Each kernel matrix will produce an output map, thus with n number of kernels we will obtain $o_w \times o_h \times n$ output maps.

When we use a full color RGB image, we will have a 3D matrix multiplication, and the kernel will also be a 3D matrix because real images have depth. The resulting output map will still be a 2D matrix.

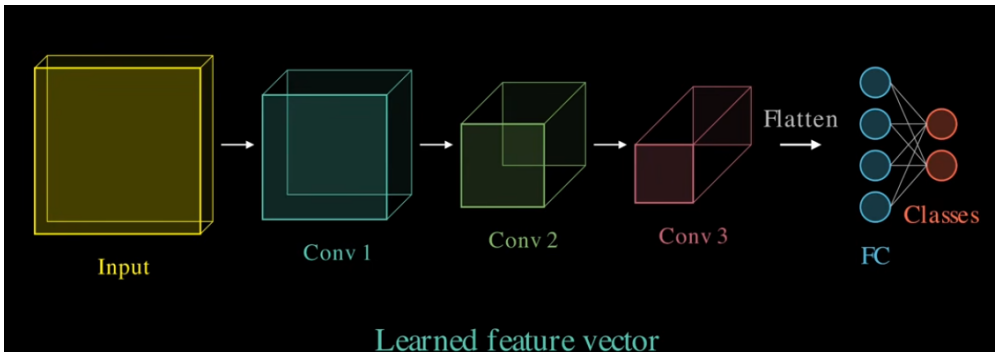

Figure 10.27: *After the last convolution layer, we will flatten the 3D matrix into a vector then process it again till the output layer that produces the probability for the image classification.*

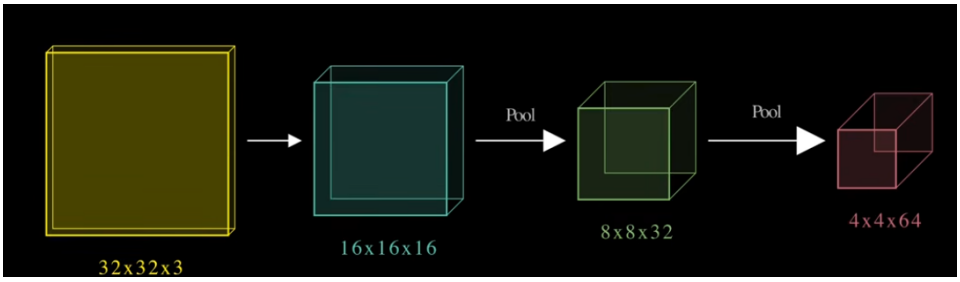

Figure 10.28: *The spatial dimensions gradually get smaller while the depth / the number of channels grows. The early layers work with large spatial maps and few channels, while the deeper layers work with small spatial maps and many channels.*

The flattened vector after the last convolution layer is a distillation of the image's most important information or feature, compressed into a format that is useful for making predictions.

[SI*] **Kernels in Convolution Operation**

Convolution operation is performed by sliding a kernel (a filter) matrix over the input matrix, performing a Hadamard (element wise, represented by $\oplus$) product of the kernel with the underlying part of input matrix, summing up the result and recording it into the output matrix. When kernels slide over images, they apply operations like blurring, sharpening, and edge detection, depends on the kernel that is used. Each kernel type has a unique function, altering the image in specific ways.

The formula for Hadamard product, if we multiply 2 matrices A and B with same size the result is matrix C with the same size

$$c_{ij} = a_{ij} * b_{ij}$$

it is not the same as the usual matrix multiplication from linear algebra, so be careful.

The convolution operation purpose is to extract certain 'features' from an input image by using a 'kernel' that will generate an output feature map.

We will identify different type of kernel that can be used in standard image processing (e.g., Sobel X, Sobel Y, Laplacian, etc), but for the convolution neural network a convolution layer

uses filters / kernels that are essentially small grids of numbers e.g., (3×3 or 5×5) that slide across the image. These numbers are the weights. The network in CNN automatically learns the best values for these weights through backpropagation to detect things like edges, shapes, and textures.

The core features of convolutional layer weights:

- **Parameter Sharing:** Unlike standard fully connected layers where every pixel has its own unique weight, a convolutional layer reuses the exact same filter (weights) across the entire image.
- **Translation Invariance:** Because the same weights are shared, the network can detect an object regardless of where it appears in the image.
- **Fewer Parameters:** This sharing drastically reduces the total number of weights the network needs to learn, preventing overfitting.

Remember the fact that images are stored as matrices of pixel values in computer. Grayscale images are simple 2D matrices while colored images are a collection of 3 such 2D matrices used for storing each color value R, G, and B.

Convolution can be done on multiple dimensions including 1D, 2D, and 3D. 1D convolutions usually are done on linear time series. A popular example of 2D convolution is image processing as well as the filters that we use. 3D convolution has its use case in video event recognition and medical imaging.

In a CNNs, a 'feature' is a specific pattern or characteristic extracted from an image using convolutional filters, essentially representing a key aspect of the image like edges, lines, textures, or shapes, which are identified by the network during the process of feature extraction that is carried out by kernels.

Types of Kernels
Kernels can be of many types. In fact, any combination of values in a matrix can act as a kernel, but a good rule of thumb is to make sure the values add up to one to avoid distorting the overall values.
Basic convolution kernes are foundational in image processing, designed to perform simple yet essential operations on images. Each type of kernel has a specific structure and function, enabling it to highlight or modify particular aspects of the input data.

1. **Identity Kernel**
   The simplest type of convolutional kernel. It just leave the input image as it is.

$$M_{kernel} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

2. **Sharpen Kernel**
   It enhances the details and edges of an image. It does so by increasing the difference of pixel intensities, making the image appear sharper and clearer.

$$M_{kernel} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

3. **Laplacian Kernel**
   It is a second-order derivative operator used for edge detection. It highlights regions of rapid intensity change and is particularly useful for finding edges in noisy images.

$$M_{kernel} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

4. **Box-blur Kernel**
   It makes the image blurrier. It does so by averaging out the values of neighboring pixels. It smoothens out the image and reduces noise as well.

$$M_{kernel} = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

5. **Gaussian-blur Kernel**
   It makes the image blurrier with a weighted Gaussian distribution. This kernel assigns higher weights to the central pixels and progressively lower weights to the surrounding pixels, resulting in a more natural blur. This creating a smooth, blurred effect that minimizes the high-frequency noise in the image. The matrix can have different values based on different σ values (standard deviation).

$$M_{kernel} = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

6. **Sobel Kernels**
   They are used for detecting edges in images by emphasizing the gradients. They calculate the first derivative of the image intensity in both horizontal and vertical directions, making them effective for edge detection.
   Sobel X Kernel (Horizontal)
   This kernel detects horizontal edges by emphasizing vertical gradients.

$$M_{kernel} = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

   Sobel Y Kernel (Vertical)
   This kernel detects vertical edges by emphasizing horizontal gradients.

$$M_{kernel} = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

7. **Prewitt Kernels**
   Similar to Sobel kernels, they are used for detecting edges in images by emphasizing the gradients. These kernels emphasize regions with high intensity changes, which correspond to the edges. They compute the gradients in the horizontal and vertical directions, providing a simple way to detect edges in images.

Prewitt X Kernel (Horizontal)
This kernel emphasizes horizontal changes in intensity.

$$M_{kernel} = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

Prewitt Y Kernel (Vertical)
This kernel emphasizes vertical changes in intensity.

$$M_{kernel} = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

8. **Specialized Convolution Kernels**
These kernels are designed for specific tasks and applications in image processing and computer vision. These kernels go beyond basic and advanced operations to provide more focused and sophisticated feature extraction, texture analysis, and edge detection capabilities.

Gabor Kernels
Gabor kernels are used for texture analysis and feature extraction. They are particularly effective in edge detection and texture discrimination. Gabor kernels combine a Gaussian function and a sinuisoidal wave, making them capable of capturing both spatial and frequency information.
The Gabor kernel is defined by the following equation:

$$G(x, y; \lambda, \theta, \phi, \sigma, \gamma) = e^{-\frac{x'^2 + \gamma^2 y'^2}{2\sigma^2}} \cos\left(2\pi \frac{x'}{\lambda} + \phi\right)$$

Where: $x' = x\cos(\theta) + y\sin(\theta)$
$y' = -x\sin(\theta) + y\cos(\theta)$
λ is the wavelength of the sinusoidal factor.
θ is the orientation of the normal to the parallel stripes of a Gabor function.
ϕ is the phase offset.
σ is the standard deviation of the Gaussian envelope.
γ is the spatial aspect ratio.

Scharr Kernels
Scharr kernels are similar to Sobel and Prewitt kernels but offer better rotational symmetry and more accurate edge detection. They are used for detecting edges with higher precision, making them suitable for applications requiring detailed edge information.

Scharr Kernel (Horizontal)
This kernel emphasizes horizontal changes in intensity with greater accuracy.

$$M_{kernel} = \begin{bmatrix} -3 & 0 & 3 \\ -10 & 0 & 10 \\ -3 & 0 & 3 \end{bmatrix}$$

Scharr Kernel (Vertical)
This kernel emphasizes vertical changes in intensity with greater accuracy.

$$M_{kernel} = \begin{bmatrix} -3 & -10 & -3 \\ 0 & 0 & 0 \\ 3 & 10 & 3 \end{bmatrix}$$

9. **Kirsch Compass Kernel**
   This kernel is a non-linear edge detector that finds the maximum edge strength in a few predetermined directions. The operator takes a single kernel mask and rotates it in 45 degree increments through all 8 compass directions: N, NE, E, SE, S, SW, W, and NW.

   Kirsch Kernel (North)

$$M_{kernel} = \begin{bmatrix} 5 & 5 & 5 \\ -3 & 0 & -3 \\ -3 & -3 & -3 \end{bmatrix}$$

   Kirsch Kernel (East)

$$M_{kernel} = \begin{bmatrix} -3 & -3 & 5 \\ -3 & 0 & 5 \\ -3 & -3 & 5 \end{bmatrix}$$

   Kirsch Maximum Response (All Directions)
   This kernel also accomplishes total edge detection by using the maximum of all kernel directions.

$$M_{kernel} = \max\Big(\begin{bmatrix} 5 & 5 & 5 \\ -3 & 0 & -3 \\ -3 & -3 & -3 \end{bmatrix}, \begin{bmatrix} -3 & 5 & 5 \\ -3 & 0 & 5 \\ -3 & -3 & -3 \end{bmatrix}, \begin{bmatrix} -3 & -3 & 5 \\ -3 & 0 & 5 \\ -3 & -3 & 5 \end{bmatrix}, \begin{bmatrix} -3 & -3 & -3 \\ -3 & 0 & 5 \\ -3 & 5 & 5 \end{bmatrix}$$
$$\begin{bmatrix} -3 & -3 & -3 \\ -3 & 0 & -3 \\ 5 & 5 & 5 \end{bmatrix}, \begin{bmatrix} -3 & -3 & -3 \\ 5 & 0 & -3 \\ 5 & 5 & -3 \end{bmatrix}, \begin{bmatrix} 5 & -3 & -3 \\ 5 & 0 & -3 \\ 5 & -3 & -3 \end{bmatrix}, \begin{bmatrix} 5 & 5 & -3 \\ 5 & 0 & -3 \\ -3 & -3 & -3 \end{bmatrix} \Big)$$

Applications of convolution kernels:

1. Image processing
   Enhancing images, detecting edges, and removing noise.
2. Computer vision
   Object detection, facial recognition, and scene understanding.
3. Medical imaging
   Analyzing medical images for disease detection and diagnosis.
4. Robotics
   Enabling robots to perceive and understand their environment.

In machine learning, during training the values inside the kernels get trained which act as feature detection for the images, in order to obtain more information from the images. During the training of CNNs, the values (weights) inside the kernels are adjusted through the process of backpropagationand optimization (such as gradient descent). These kernels are now trained feature detectors for images and can perform tasks like categorization and identification.

[SI*] **Stride**
The distance the kernel window moves at each step across the image is called the stride. We need to specify the stride in both dimensions of the image: one for moving left to right and one for moving top to bottom.

Whenever the stride in either direction is greater than 1, the sliding window will skip over some of the pixels in the input at each step. But, because we want high-quality features to use for classification, it is almost always set to one. Stride values are only increased when we wish to reduce the resolution of the image which occurs during the pooling stage of convolution.

[SI*] **Padding**
When performing the sliding window computation, we don't know what to do with the boundaries of the input. Staying entirely inside the input image means the kernel will never sit squarely over these boundary pixels like it does for every other pixel in the input. This could lead to a problem as all pixels aren't being treated the same.

We can leave it alone and force the kernel to stay within the image. But the image loses pixels: precisely $\frac{n-1}{2}$ for ever odd$-n$ sized kernel. This isn't ideal for large kernels and small images.

This is why we use padding. Here, we also have options. We can pad the input boundaries with 0's just enough to make the output the original size. This can; however, dilute the influence of the border pixels. This method is called same padding. Another method used is duplicate padding where instead of zeros, the borders are padded with values duplicating the respective border values. This helps preserve features near the edges of the image.

[SI*] **Convolutional Networks**
Each input image comprises a set of pixel intensity values, and the desired output is a posterior probability distribution over the ten digit classes. We know that the identity of the digit is invariant under translations and scaling as well as (small) rotations. Furthermore, the network must also exhibit invariance to more subtle transformations such as elastic deformations.

Key property of images: nearby pixels are more strongly correlated than more distant pixels. Many of the modern approaches to computer vision exploit this property by extracting local features that depend only on small subregions of the image. Information from such features can then be merged in later stages of processing in order to detect higher-order features and ultimately to yield information about the image as whole. Also, local features that are useful in one region of the image are likely to be useful in other regions of the image, for instance if the object of interest is translated.

These notions are incorporated into convolutional neural networks through three mechanisms:

- Local receptive fields
- Weight sharing
- Subsampling

In the convolutional layer the units are organized into planes, each of which is called a feature map. Units in a feature map each take inputs only from a small subregion of the image, and all of the units in a feature map are constrained to share the same weight values. For instance, a feature map might consists of 100 units arranged in a 10×10 grid, with each

unit taking inputs from a 5×5 pixel patch of the image. The whole feature map therefore has 25 adjustable weight parameters plus one adjustable bias parameter. Input values from a patch are linearly combined using the weights and the bias, and the result transformed by a sigmoidal nonlinearity. If we think of the units as feature detectors, then all of the units in a feature map detect the same pattern but at different locations in the input image. Due to the weight sharing, the evaluation of the activations of these units is equivalent to a convolution of the image pixel intensities with a 'kernel' comprising the weight parameters. If the input image is shifted, the activations of the feature map will be shifted by the same amount but will otherwise be unchanged. This provides the basis for the (approximate) invariance of the network outputs to translations and distortions of the input image. Because we will typically need to detect multiple features in order to build an effective model, there will generally be multiple feature maps in the convolutional layer, each having its own set of weight and bias parameters.

The outputs of the convolutional units form the inputs to the subsampling layer of the network. For each feature map in the convolutional layer, there is a plane of units in the subsampling layer and each unit takes inputs from a small receptive field in the corresponding feature map of the convolutional layer. These units perform subsampling. For instance, each subsampling unit might take inputs from a 2×2 unit region in the corresponding feature map and would compute the average of those inputs, multiplied by an adaptive weight with the addition of an adaptive bias parameter, and then transformed using a sigmoidal nonlinear activation function. The recetive fields are chosen to be continuous and nonoverlapping so that there are half the number of rows and columns in the subsampling layer compared with the convolutional layer. In this way, the response of a unit in the subsampling layer will be relatively insensitive to small shifts of the image in the corresponding regions of the input space.

In a practical architecture, there may be several pairs of convolutional and subsampling layers. At each stage there is a larger degree of invariance to input transformations compared to the previous layer. There may be several feature maps in a given convolutional layer for each plane of units in the previous subsampling layer, so that the gradual reduction in spatial resolution is then compensated by an increasing number of features. The final layer of the network would typically be a fully connected, fully adaptive layer, with a softmax output nonlinearity in the case of multiclass classification.

The whole network can be trained by error minimization using backpropagation to evaluate the gradient of the error function. This involves a slight modification of the usual backpropagation algorithm to ensure that the shared-weight constraints are satisfied. Due to the use of local receptive fields, the number of weights in the network is smaller than if the network were fully connected. Furthermore, the number of independent parameters to be learned from the data is much smaller still, due to the substantial numbers of constraints on the weights.

[SI*] **Operational Principles of CNNs**

Within the architecture of convolutional neural networks, image data is initially input into CNN through the input layer. Following this, the network employs a sequence of layers, including the convolutional layer, batch normalization layer, ReLU layer, and pooling layer, to systematically extract and learn features from the input images [17].

After features are extracted, the final layers (often fully connected layers) classify the image based on the detected features. Once the network produces an output, a loss function measures the difference between the predicted output and the actual labels. Common loss functions include cross-entropy for classification tasks and mean squared error (MSE) or

mean absolute error (MAE) for regression tasks.
Then, perform backpropagation and optimization. During this process, training begins with the computation of gradients, utilizing the chain rule to ascertain the gradient of the loss with respect to the network's weights. This is followed by a weight adjustment phase, where optimization algorithms, such as stochastic gradient descent (SGD) or its advanced counterparts like Adam and RMSprop, refine the weights to minimize the loss. This intricate dance of forward (from input to loss determination) and backward (involving backpropagation and optimization) passes is iteratively executed over several epochs, continuing until the network reaches an optimal or satisfactory performance threshold.
CNNs are trained using backpropagation with gradient descent optimization. The goal is to adjust the weights to minimize the difference between the predicted output and the actual labels, typically represented by a loss function. Regularization techniques, such as dropout and weight decay, are often employed to prevent overfitting.

[SI*] **Input Layer**
The input layer in a convolutional neural network (CNN) serves as the initial stage where raw data is introduced into the model for subsequent processing. Typically, the input layer accepts multidimensional data, such as images represented by width, height, and channel dimensions. This layer plays a critical role in defining the structure of the data pipeline, as it must accommodate the specific dimensions and format of the dataset under consideration. For instance, an input layer for grayscale images wold require a single channel, whereas for RGB iamges, it would require three channels corresponding to red, green, and blue. The input layer neither performs computation nor alters the data but serves as a conduit to prepare and structure the input for downstream convolutional layers, ensuring compatibility and facilitating efficient feature extraction in subsequent stages.

[SI*] **Convolutional Layer**
This is the core building block of a CNN. The layer's primary function is to detect local patterns using learned kernels (or filters). When processing an image, these kernels slide (or convolve) around the input data (such as an image) to produce a feature map, effectively transforming the input data into a form that makes it easier for the network to understand. The result of this convolution operation is a set of feature maps that give the spatial arrangement for the detected feature at different locations in the input.
The core concepts of convolutional layer:

- **Kernels/Filters**
  Kernels are small, trainable matrices that slide across the input data to extract features such as edges, textures, or more abstract patterns. Each kernel is applied to the input data via convolution, producing a feature map as output. For example, a 3×3 kernel slides over an image, performing element-wise multiplication and summing the results.
- **Stride**
  Stride determines how much the kernel shifts at each step during the convolution operation. Larger strides reduce the size of the output feature map but may lose fine-grained details. Typical strides: $S = 1$(no skip) or $S = 2$ (downsampling, faster computation but less accurate).
- **Padding**
  Padding is a technique used in convolutional operations to add extra rows and columns around the edges of the input tensor, enabling the preservation or modification of spatial dimensions after the operation. Common types of padding are
  1. Valid Padding (no padding): In this approach, no additional rows or columns are added, resulting in an output tensor smaller than the input tensor due to the

reduction caused by the kernel size.

2. Same Padding: Here, padding is added such that the spatial dimensions of the output tensor match those of the input tensor. This is particulaly useful for maintaining consistency in the feature map size across layers.
3. Full Padding: This type extends the input tensor by padding to ensure that every element of the input is fully convolved, leading to an output tensor larger than the input tensor.

Padding choices significantly impact the receptive field, computational efficiency, and the overall architecture of convolutional neural networks.

Formula for output size with stride S and padding P:

$$\text{Output size} = \frac{N + 2P - F}{S} + 1 \tag{10.68}$$

with N as the input size and F as the filter/kernel size.

The convolutional operation, a fundamental process in deep learning, involves applying a kernel K to an input tensor X to produce an output tensor Y. Mathematically, this is expressed as

$$Y[i \cdot j] = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} X[i+m, j+n] \cdot K[m, n] \tag{10.69}$$

where the kernel slides over the input tensor, performing element-wise multiplication and summing the results.
Additionally, convolutional layers benefit from weight sharing, wherein a single set of kernel parameters is applied across all spatial locations of the input tensor. This parameter-sharing mechanism significantly reduces the number of trainable parameters, thereby improving the model's computational efficiency and mitigating the risk of overfitting.

Feature maps are the outputs generated by applying a kernel to an input tensor, effectively highlighting regions where the kernel's learned pattern is most prominent. These maps serve as representations of specific features within the input data, facilitating the detection of patterns such as edges, textures, or more complex structures.
Each value in a feature map corresponds to a receptive field, which represents the specific region of the input image that influences that value. As the network depth increases, the receptive fields expand, allowing deeper layers to capture high-level, abstract features by aggregating information from larger regions of the input. This hierarchical representation is essential for learning complex patterns and achieving strong performance int asks such as object recognition and semantic segmentation.

Convolutions are versatile operations in deep learning, tailored to different types of data and tasks.
1D convolutions are used for sequential data such as time series, signals, or text, with applications in tasks like speech recognition.
2D convolutions, the most common type, are applied to image data, processing 2D grayscale o RGB images to extract spatial features. For processing 3D data, such as volumetric scans (e.g., CT or MRI) or video frames.
3D convolutions introduce depth as a third dimension, capturing spatial-temporal information.

Dilated (atrous) convolutions expand the receptive field by introducing gaps between kernel elements, enabling models to capture broader contextual information without increasing computational parameters, which is particularly useful in semantic segmentation.
Lastly, depthwise separable convolutions decompose standard convolution into two steps: depthwise convolution, which applies a separate kernel to each input channel, and pointwise convolution, which uses a 1×1 kernel to combine channel outputs. This approach significantly reduces computational cost while maintaining accuracy, making it a key component in efficient architectures like MobileNet.

[SI*] **Pooling Layer**

Pooling layers are an essential component of convolutional neural networks (CNNs) used to reduce the spatial dimensions of feature maps.
Pooling layers serve several critical purposes in convolutional neural networks (CNNs). First, they facilitate dimensionality reduction by reducing the size of feature maps, thereby decreasing computational costs and memory requirements in subsequent layers. This ensures the network remains efficient, even with large input images. Second, pooling enhances translation invariance by summarizing features within a region, making the model robust to small shifts or distortions in the input. Lastly, pooling contributes to preventing overfitting by reducing spatial dimensions, acting as a form of implicit regularization that improves generalization.

Let $x^L \in \mathbb{R}^{H^L \times W^L \times D^L}$ be the input to the L-th layer, which is now a pooling layer. The pooling operation requires no parameter (i.e. w^i is null; hence, parameter leaning is not needed for this layer). The spatial extent of the pooling ($H \times W$) is specified in the design of the CNN structure. Assume that H divides H^L and W divides W^L and the stride equals the pooling spatial extent, the output of pooling will be an order three tensor of size $H^{L+1} \times W^{L+1} \times D^{L+1}$, with

$$H^{L+1} = \frac{H^L}{H}, \quad W^{L+1} = \frac{W^L}{W}, \quad D^{L+1} = D^L \tag{10.70}$$

This formula ensures precise control over the spatial dimensions of the feature maps after pooling, which is essential for designing and understanding CNN architectures.

Pooling layers often use stride and padding to control the operation's behavior and output dimensions. The stride specifies the step size of the pooling window as it moves across the feature map. A stride of 2, for example, reduces the spatial dimensions by approximately half, enabling efficient dimensionality reduction while summarizing features.

Padding add extra rows and columns around the edges of the feature map, similar to its role in convolution. This ensures that the pooling operation includes the boundary regions of the feature map, preserving spatial alignment and avoiding loss of information at the edges, especially in cases where maintaining the feature map's structure is crucial.

Pooling operations in convolutional neural networks (CNNs) come in various forms, each designed to summarize feature maps in different ways while reducing dimensions,

1. **Max Pooling:** This method retains the maximum value from a pooling region, effectively highlighting the most prominent feature. It is defined as

$$Y[i, j] = \max(X[i : i + F, j : j + F]) \tag{10.71}$$

where F is the window size.
Max pooling takes the maximum value from each $F \times F$ window of the input and assigns it to the output position (i, j).

2. **Average Pooling:** In contrast, this method computes the average of values within the pooling region, capturing the general trend of features. It is defined as

$$Y[i,j] = \frac{1}{F^2} \sum_{k=0}^{F-1} \sum_{l=0}^{F-1} X[i+k, j+l] \tag{10.72}$$

This formula defines average pooling in a neural network. Here, X is the input feature map, and $Y[i, j]$ is the output at position (i, j). A squae window of size $F \times F$ slides over the input, and the values within the window are averaged. The result is assigned to the corresponding position in the output Y.

3. **Global Pooling:** It reduces each feature map to a single value, either by taking the maximum (global max pooling) or the average (global average pooling) over all spatial dimensions. This approach is commonly used in architectures like ResNet to replace fully connected layers, simplifying the model while maintaining performance.
4. **Adaptive Pooling:** It dynamically adjusts the output size to a fixed dimension, regardless of input size, making it particularly useful for handling variable-sized inputs.

An example of max pooling with filters: 2×2 and stride : 2

$$\begin{bmatrix} 1 & 1 & 2 & 5 \\ 5 & 6 & 7 & 8 \\ 3 & 2 & 1 & 0 \\ 1 & 2 & 3 & 4 \end{bmatrix} \xrightarrow[Max\ Pooling]{} \begin{bmatrix} 6 & 8 \\ 3 & 4 \end{bmatrix}$$

Advanced pooling methods extend the capabilities of standard pooling to adress specific challenges in computer vision tasks:

- Region of Interest (ROI) Pooling: This technique is commonly used in object detection models like Faster-R-CNN to dynamically pool features from regions corresponding to detected objects, enabling precise localization and classification.
- Feature Pyramid Pooling: Employed in networks such as PSPNet, this method extracts multi-scale features by pooling at different scales, enhancing the model's ability to recognize objects of varying sizes and improving performance in tasks like semantic segmentation.
- Fractional Pooling: Designed to handle non-integer downsampling ratios, fractional pooling interpolates between neighboring values, allowing finer control over output dimensions and adapting to diverse input sizes more flexibly.

These techniques enhance pooling's adaptability, improving the performance of CNNs in complex tasks like object detection, segmentation, and multi-scale feature extraction.

[SI*] **Normalization Layer**

Normalization layers are essential for stabilizing and accelerating the training of deep neural networks. Normalization in neural netowrks involves adjusting the distribution of data, such as activations or inputs, to improve training and performance. It ensures stabilized learning by maintaining consistent distributions across layers, allowing gradient to flow predictably and reducing the risk of issues like exploding or vanishing gradients. Additionally, normalization facilitates faster convergence by minimizing internal covariate shift, which leads to

smoother and more efficient optimization. Finally, it enhances generalization, enabling the model to perform bettern on unseen data by encouraging robust and consistent learning of features.

1. **Batch Normalization (BN):** Batch normalization normalizes activations within a mini-batch by adjusting their distribution to have a mean of zero and a variance of one. Specificallt, it computes the batch mean

$$\mu = \frac{1}{m}\sum_{i=1}^{m} x_i \tag{10.73}$$

and variance

$$\sigma^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu)^2 \tag{10.74}$$

where m is the mini-batch size. Each activation is then normalized as

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} \tag{10.75}$$

with a small constant ϵ added for numerical stability. Finally, learnable parameters γ (scale) and β (shift) are applied to allow the network to recover useful representations:

$$y_i = \gamma \hat{x}_i + \beta \tag{10.76}$$

Batch normalization reduces internal covariate shift, accelerates convergence, and provides implicit regularization. However, it may perform poorly with very small batch sizes and introduces computational overhead during training.

2. **Layer Normalization (LN):** Layer normalization normalizes activations across all features of a single input instance, making it independent of batch size. For an input with H features, the mean and variance are computed as

$$\mu = \frac{1}{H}\sum_{j=1}^{H} x_j \tag{10.77}$$

and

$$\sigma^2 = \frac{1}{H}\sum_{j=1}^{H}(x_j - \mu)^2 \tag{10.78}$$

Each feature is normalized using

$$\hat{x}_j = \frac{x_j - \mu}{\sqrt{\sigma^2 + \epsilon}} \tag{10.79}$$

followed by scaling and shifting with learnable parameters $y_j = \gamma \hat{x}_j + \beta$. This method works well for tasks like NLP and reinforcement learning, where batch dependencies are less relevant. Layer normalization is particularly useful in transformer architectures and recurrent neural networks, offering stable learning regardless of batch size.

3. **Instance Normalization (IN):** Instance normalization focuses on normalizing the spatial dimensions of each channel independently, making it effective for style transfer and

image generation tasks. For a feature map of height H and width W, the mean and variance for each channel c are calculated as

$$\mu_c = \frac{1}{HW} \sum_{i=1}^{H} \sum_{j=1}^{W} x_{i,j,c} \tag{10.80}$$

and

$$\sigma_c^2 \frac{1}{HW} \sum_{i=1}^{H} \sum_{j=1}^{W} (x_{i,j,c} - \mu_c)^2 \tag{10.81}$$

The spatial dimensions are then normalized using

$$\hat{x}_{i,j,c} = \frac{x_{i,j,c} - \mu_c}{\sqrt{\sigma_c^2 + \epsilon}} \tag{10.82}$$

followed by scaling and shifting:

$$y_{i,j,c} = \gamma \hat{x}_{i,j,c} + \beta \tag{10.83}$$

Instance normalization normalizes each feature map independently, removing instance-specific variations while preserving structural features. This makes it especially effective for tasks involving stylized outputs.

In addressing the inherent limitations of conventional normalization strategies, advanced normalization techniques and their alternatives have emerged, presenting novel approaches to significantly enhance neural network performance. Batch renormalization stands as a notable extension of batch normalization (BN), specifically engineered to mitigate the instability encountered with smaller batch sizes. This method introduces adaptive scaling and shifting parameters, which serve to stabilize training by ensuring consistent statistical properties across a wider range of batch sizes, thereby improving the reliability and generalization of models trained under such conditions.

Furthermore, switchable normalization represents a dynamic approach wherein the network learns to optimally select from a repertoire of normalization techniques, including BN, layer normalization (LN), and instance normalization (IN), on a per-layer basis during training. This adaptive mechanism allows for a tailored normalization strategy, optimizing performance across diverse tasks and architectural configurations. Concurrently, a burgeoning area of research explores the feasibility of normalization-free networks, which entirely circumvent the need for explicit normalization layers. A prominent example is FixUp initialization, which employs specialized parameter initialization schemes and concomitant training adjustments. This methodology enables the effective training of deep neural networks without recourse to normalization, consequently reducing computational overhead and simplifying the overall model architecture.

[SI*] **Activation Layer**

Activation layers are a critical component of neural networks, introducing nonlinearity to the model, enabling it to learn complex patterns, and transforming raw inputs into useful features.

Activation functions also perform transformation, converting raw weighted sums into meaningful outputs for subsequent layers to process. Additionally, they influence gradient flow

during backpropagation, playing a vital role in convergence and stability by ensuring gradients are neither too large nor too small. These functions make activation layers essential for effective learning and performance.

We have describes the types of activation functions in the feed-forward neural network section, at the beginning of this chapter, thus we won't repeat again, readers can read that section if they are not well informed about activation functions.

Choosing an activation function depends on the specific requirements of the task and architecture. ReLU is the most commonly used activation function due to its simplicity, efficiency, and ability to mitigate the vanishing gradient problem, making it a default choice for many networks like CNNs and DNNs. Variants of ReLU, such as Leaky ReLU and Parametric ReLU, address issues like the dying ReLU" problem. For outputs requiring bounded ranges, functions like Tanh or sigmoid are more appropriate. In modern architectures, Swish has shown improved performance in certain scenarios, especially when its higher computational cost is acceptable. Ultimately, selecting the best activation function often relies on experimentation, as empirical results can vary based on the dataset and application.

[SI*] **Fully Connected Layer**

Fully connected (FC) layers are fundamental components of neural networks, especially in traditional architectures. They connect every input neuron to every output neuron, enabling the network to learn global patterns and make predictions.

A fully connected layer is a dense neural layer where every neuron is connected to all neurons in the previous layer, playing a critical role in aggregating and utilizing learned features. FC layers combine spatially distributed features extracted by convolutional layers into a compact representation, enabling high-level decision-making tasks such as classification or regression. They act as a transition point by flattening feature maps into a 1D vector before further processing. FC layers provide a global representation by integrating information from all preceding neurons, making them essential for tasks requiring comprehensive feature aggregation. However, they are computationally expensive due to the large number of parameters, proportional to the product of the input size and the number of neurons, which increases the risk of overfitting. Additionally, FC layers are vital for dimensionality transformation, reshaping data to suit downstream tasks effectively.

The output of a fully connected layer is computed as

$$y = f(Wx + b) \tag{10.84}$$

where W is the weight matrix of size [output_neurons $\times$ input_neurons], x is the input vector, b is the bias vector, and f is the activation function.

For an input vector x of size N and a fully connected layer with M neurons, the layer's functionality is defined by its weights and biases, output computation, and parameter count. The weight matrix W has dimensions $M \times N$ representing the connections between each input and neuron, while the bias vector b has size M, adding a trainable offset to each neuron. The output is computed in two steps: first, a linear transformation is applied, $z = Wx + b$, which aggregates the input features weighted by the learned parameters. Then, the activation function f is applied to the result, yielding the final output $y = f(z)$.

The total number of parameters in the FC layer, which includes $N \times M$ weights and M biases, is calculated as parameters $= (N \times M) + M$. For example, with an input vector of size 128 and an FC layer comprising 64 neurons, tha parameter count would be $(128 \times 64) + 64 = 8256$. This high parameter count highlights the computation expense and potential overfitting risk associated with fully connected layers, making them critical yet resource-intensive component of neural networks.

Fully connected layers play a crucial role in aggregating features and making predictions. Despite their simplicity, they are computationally intensive and prove to overfitting. Modern architectures often replace or minimize their use with techniques like global pooling, attention mechanisms, or sparse connections.

[SI*] **Classification Layer**

In the end, a CNN often has a classification layer to make a prediction. If, for instance, the task is to classify an image among ten categories, this layer would output ten values, each one representing the probability of the image belonging to that class. In CNNs, fully connected layers serve as a crucial bridge between the feature extraction process and high-level decision-making tasks such as classification or regression. The transition to FC layers involves flattening the multidimensional feature maps produced by convolutional layers intoa a one-dimensional vector. For example, a $7 \times 7 \times 64$ feature map is reshaped into a 3136-dimensional vector, enabling the network to process spatially features, preparing them for classification or other downstream tasks.

FC layers are frequently employed as the final layers in CNNs, particularly in image classification tasks, where they act as output layers. In these scenarios, the final FC layer typically consists of C neurons, where C represents the number of target classes. A softmax activation function is then applied to this layer to produce class probabilities, ensuring the outputs are interpretable and suitable for classification. By integrating spatially learned features from convolutional layers into a global representation, FC layers enable CNNs to make accurate predictions, highlighting their essential role in combining feature extraction and task-specific decision-making.

[SI*] **Typical Model Structure of CNNs**

The fundamental principle underpinning convolutional neural networks lies in their utilization of shared local receptive fields and weights. This core concept enables efficient feature extraction from spatially correlated data. The seminal work in this field, LeNet, introduced by LeCun et al. in 1998, is widely recognized as the archetype of CNN architectures.

Over the past devade, convolutional neural networks have undergone significant evolution, driven by the need for improved performance, scalability, and efficiency in computer vision tasks. Each advancement in CNN architecture has introduced novel design principles, addressing critical challenges such as vanishing gradients, computational cost, and overfitting. Starting with AlexNet which demonstrated the power of deep learning on large-scale image datasets, modern architectures like ResNet, DenseNet, and EfficientNet have pushed the boundaries of depth, parameter efficiency, and computational optimization. This section explores the innovations and design philosophies of key CNN architectures, highlighting their contributions to the field and their practical applications in solving complex visual tasks.

1. **LeNet (1998)**

   The original LeNet model featured a straightforward architecture comprising five

convolutional layers directly linked to corresponding pooling layers. Its initial application focused on the recognition of handwritten and printed characters, demonstrating remarkable performance for its time. It is worth noting that contemporary implementations of LeNet often incorporate modifications from the original design to suit a broader range of applications.

2. **AlexNet (2012)**
   AlexNet marked a milestone in deep learning by winning the 2012 ImageNet competition, demonstrating the potential of deep CNNs for image classification. It introduced ReLU activation to accelerate training, dropout to reduce overfitting, and leveraged GPUs for efficient computation. The architecture consists of five convolutional layers and three fully connected layers, using large filters such as 11×11 and 5×5. Despite its breakthrough, AlexNet's large parameter count (60 M) and computational requirements were significant drawbacks.
3. **VGG (2014)**
   The VGG network demonstrated that deeper networks with smaller filters (3×3) could significantly improve performance. VGG-16 and VGG-19 are the most common configurations, comprising 13-16 convolutional layers followed by three fully connected layers. Max pooling is used for spatial downsampling. While VGG is simple and effective, its parameter count is exceedingly high (e.g., 138M for VGG-16), making it resource-intensive in terms of computation and memory.
4. **Inception (GoogleLeNet, 2014)**
   Inception networks introduced inception modules, which apply convolutions of varying kernel sizes in parallel to capture multi-scale features. The architecture reduces computation through 1×1 convolutions for dimensionality reduction. Auxiliary classifiers improve gradient flow in deeper networks. Despite its efficiency and strong performance, the complexity of its design poses implementation challenges.
5. **ResNet (2015)**
   ResNet introduced residual connections (skip connections) to address the vanishing gradient problem, enabling the training of extremely deep networks, such as ResNet-50 and ResNet-152. Residual connections allow gradients to flow directly through layers by learning residual mappings ($F(x) + x$), significantly improving training dynamics and performance. However, the increased depth of ResNet comes at the cost of high computational complexity.
6. **MobileNet (2017)**
   MobileNet was designed for mobile and embedded systems, introducing depth-wise separable convolutions to reduce computation. Depthwise convolution applies a single filter per input channel, followed by pointwise convolution to combine channels. This lightweight design significantly reduces model size and complexity, albeit with a trade-off in accuracy for highly constrained model. Variants like MobileNetV2 and V3 further enhance performance and efficiency.
7. **ShuffleNet (2017)**
   ShuffleNet targeted efficient computation on mobile devices by introducing grouped convolutions and channel shuffle mechanisms. Grouped convolutions divide channels into subsets for parallel computation, while channel shuffle ensures information exchange across groups. This design achieves a balance between accuracy and computational efficiency, though its implementation is more complex compared to simpler architectures.

8. **GhostNet (2020)**
   GhostNet optimized lightweight CNNs by introducing ghost modules, which generate additional feature maps through inexpensive linear transformations instead of full convolutions. This reduces computational cost and parameter count while maintaining competitive accuracy. GhostNet is highly effective for resource-constrained devices, though extreme compression may lead to slight performance degradation.

[SI*] **Applications of CNNs**
Convolutional neural networks have revolutionized numerous fields by offering unparalleled performance in processing visual and spatial data. Their ability to learn hierarchical features and extract intricate patterns has enabled groundbreaking applications across industries.

Image classification is a fundamental task in computer vision where convolutional neural networks assign labels to images based on learned features. By extracting hierarchical patterns from raw image data, CNNs enable accurate categorization and identification across a wide range of applications. In medical imaging, CNNs classify X-rays, MRIs, and CT scans to assist in diagnosing diseases such as pneumonia or detecting tumors. For autonomous vehicles, they identify road signs, pedestrians, and objects to support safe navigation. In retail and commerce, CNNs categorize products based on visual attributes, enahncing search and recommendation systems.

Object detection is a key task in computer vision that combines classification and localization to identify objects in an image and draw bounding boxes around them. This task extends beyond merely recognizing what is in an image by also determining where the objects are located. It has diverse applications across various domains. In surveillance, object detection helps identify suspicious activities or unauthorized access in security footage. For autonomous vehicles, it is essential for real-time identification of vehicles, pedestrians, and obstacles to ensure safe navigation. In agriculture, object detection aids in identifying pests and diseases or estimating crop yield, improving efficiency and productivity. A significant milestone in object detection is the development of YOLO (You Only Look Once), a single-stage detection pipeline that revolutionized real-time object detection.
By combining detection and classification into a single step, YOLO dramatically reduced inference time while maintaining high accuracy, making it suitable for applications requiring low-latency detection. Its efficiency and reliability have made YOLO a widely adopted framework in real-world scenarios demanding real-time performance.

Face recognition systems are designed to identify or verify individuals by anayzing facial features from images or video streams. These systems have become integral in various domains. In security, face recognition is used for access control and surveillance, ensuring only authorized individuals can enter restricted areas. In consumer electronics, facial unlocking features enhance convenience and security in smartphones and laptops. In retail, face recognition enables personalized customer experiences by analyzing facial data to udnerstand preferences and behaviors.

A notable advancement in this field is FaceNet, which employs deep embeddings to map faces into a high-dimensional feature space. This approach allows for hightly accurate face verification and clustering of identities, enabling robust authentication systems and efficient video analytics. FaceNet's deployment in applications like secure authentication and surveillance highlights its effectiveness in real-world face recognition tasks.

Autonomous vehicles rely extensively on convolutional neural network (CNNs) for real-time scene understanding, integrating multiple vision tasks to ensure safe and efficient operation. Core tasks include object detection using models like YOLO for identifying vehicles, pedestrians, and obstacles in real-time, semantic segmentation with architectures like DeepLab to classify road scenes into meaningful regions and monocular depth estimation using CNNs to predict depth information from single-camera inputs. These capabilities enable autonomous systems to interpret complex environments dynamically. A prominent example is Tesla's Autopilot, which uses deep CNNs for multi-camera vision processing. This system powers advanced driver assistance features such as lane keeping, obstacle detection, and adaptive cruise control, forming the foundation for achieving full vehicle autonomy. By combining these techniques, autonomous vehicles demonstrate the potential of CNNs in transforming transporation. pg 42

i. Convolution Operation on an Image with OpenGL and STB Image

We will perform a convolution operation by sliding a kernel matrix over the input matrix (the input image). It is very handy to use OpenGL for this kind of task, since it is the fastest around, even if we need a steeper learning curve and need to code more it is worth the time and sweat.

A lot of people out there are coding with OpenCV for image manipulation, loading, or converting an image into greyscale, OpenCV requires at least 280 MB storage, and if we can do such image manipulation with smaller but faster library and then develop it ourselves from that, it is better that way.

So we are using STB image for the image loading part and showing the image on a new GUI window is thanks to OpenGL + vertex shaders + fragment shaders and the interactivity for peforming the convolution operation with keyboard pressed events is thanks to GLFW.

The whole size for the source code is 17 KB, no OpenCV or any heavy dependencies, but we need OpenGL and GLFW which will be very useful in the future if we want to create physics simulation like Bullet or Project Chrono, or deep learning project like video recognition.

This code depends on shaders a lot to make the convolution operation works, shaders are little programs that rest on the GPU. These programs are run for each specific section of the graphics pipeline. In a basic sense, shaders are nothing more than programs transforming inputs to outputs. Shaders are also very isolated programs in that they're not allowed to communicate with each other; the only communication they have is via their inputs and outputs.

Shaders are witten in the C-like language GLSL, GLSL is tailored for use with graphics and contains useful features specifically targeted at vector and matrix manipulation.

Shaders always begin with a version declaration, e.g. **# version 330** (meaning it uses OpenGL version 3.3), followed by a list of input and output variables, uniforms and its main function. Each shader's entry point is at its main function where we process any input variables and output the results in its output variables. If you want to comprehend about shaders more, you can read the comprehensive book by Joey de Vries [32].

Now we are going to talk about loading the image with an image-loading library, in this example we are using STB Image, it supports several popular formats like **.png** and **.jpg**. **stb_image.h** is a very popular single header image loading library by Sean Barrett that is able to load most popular file formats and is easy to integrate in your project. You just need to define it (**#define STB_IMAGE_IMPLEMENTATION**) then include the header (**#include "stb_image.h"**). By defining **STB_IMAGE_IMPLEMENTATION** the preprocessor modifies the header file such that it only contains the relevant definition source code, effectively turning the header file into a **.cpp** file.

In the computer world, OpenGL is considered as an API (Application Programming Interface) that provides us with a large set of functions that we can use to manipulate graphics and images. However, OpenGL by itself is not an API, but merely a specification, it specifies exactly what the result/output of each function should be and how it should perform. To perform kernel convolution on an image we cannot use OpenGL only, we need another libraries such as GLFW (it handles keyboard or mouse input events and gives us the bare necessities required for rendering

images/graphics to the screen), GLAD (this library is the manager library to handle the location of most OpenGL functions that we need, because most of its functions' location is not known at compile-time, thus this library retrieve the location of the functions that we need and store them in function pointers for later use).

Some definitions that are related to OpenGL:

- Shader: a small program that runs on the graphics card. Several stages of the graphics pipeline can use user-made shaders to replace existing functionality.
- Vertex: a collection of data that represent a single point.
- Vertex Buffer Object (VBO): a buffer object that allocates memory on the GPU and stores all the vertex data there for the graphics card to se.
- Vertex Array Object (VAO): stores buffer and vertex attribute state information.
- Element Buffer Object (EBO): a buffer object that stores indices on the GPU for indexed drawing.
- Uniform: a special type of GLSL variable that is global (each shader in a shader program can access this uniform variable) and only has to be set once.
- Texture: a special type of image used in shaders and usually wrapped around objects, giving the illusion an object is extremely detailed.
- Texture Wrapping: defines the mode that specifies how OpenGL should sample textures when texture coordinates are outside the range: $(0, 1)$.
- Texture Filtering: defines the mode that specifies how OpenGL should sample the texture when there are several texels (texture pixels) to choose from. This usually occurs when a texture is magnified.

A kernel (convolution matrix) is a small matrix-like array of values centered on the current pixel that multiplies surrounding pixel values by its kernel values and adds them all together to form a single value.

Kernels are an extremely useful tool for post-processing since they're quite easy to use and experiment with.

We will perform some convolution operations on this picture:

Figure 10.29: *This is a female Gerberian Shepsky dog who is about 1 month old, the one and only.*

We will perform the laplacian operation with this kernel / convolution matrix:

$$M_{kernel} = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

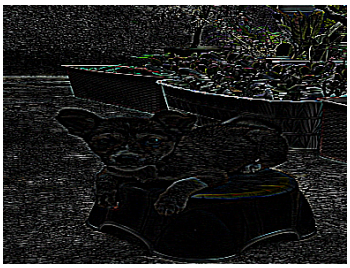

Figure 10.30: *The laplacian kernel operation on an image (**SymIntegration/Examples/Neural Networks/Test Kernel Convolution Operation on an Image with OpenGL GLFW GLAD and STB Image/main.cpp**).*

Next, we will perform the Sobel X operation with this kernel / convolution matrix:

$$M_{kernel} = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

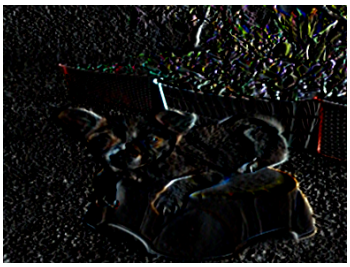

Figure 10.31: *The Sobel X kernel operation on an image (**SymIntegration/Examples/Neural Networks/Test Kernel Convolution Operation on an Image with OpenGL GLFW GLAD and STB Image/main.cpp**).*

Next, we will perform the Sobel Y operation with this kernel / convolution matrix:

$$M_{kernel} = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

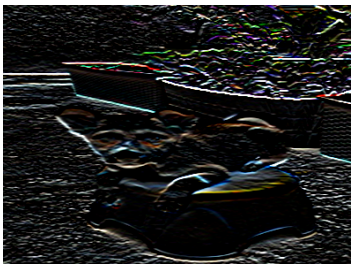

Figure 10.32: *The Sobel Y kernel operation on an image (**SymIntegration/Examples/Neural Networks/Test Kernel Convolution Operation on an Image with OpenGL GLFW GLAD and STB Image/main.cpp**).*

Next, we will perform the Schar horizontal operation with this kernel / convolution matrix:

$$M_{kernel} = \begin{bmatrix} -3 & 0 & 3 \\ -10 & 0 & 10 \\ -3 & 0 & 3 \end{bmatrix}$$

Figure 10.33: *The Schar horizontal operation on an image (**SymIntegration/Examples/Neural Networks/Test Kernel Convolution Operation on an Image with OpenGL GLFW GLAD and STB Image/main.cpp**).*

Next, we will perform the Schar vertical operation with this kernel / convolution matrix:

$$M_{kernel} = \begin{bmatrix} -3 & -10 & -3 \\ 0 & 0 & 0 \\ 3 & 10 & 3 \end{bmatrix}$$

Figure 10.34: *The Schar vertical operation on an image (**SymIntegration/Examples/Neural Networks/Test Kernel Convolution Operation on an Image with OpenGL GLFW GLAD and STB Image/main.cpp**).*

Next, we will perform the Kirsch west operation with this kernel / convolution matrix:

$$M_{kernel} = \begin{bmatrix} 5 & -3 & -3 \\ 5 & 0 & -3 \\ 5 & -3 & -3 \end{bmatrix}$$

Figure 10.35: *The Kisch west operation on an image (**SymIntegration/Examples/Neural Networks/Test Kernel Convolution Operation on an Image with OpenGL GLFW GLAD and STB Image/main.cpp**).*

The C++ source code is modified from chapter 7 of the Learn OpenGL book [32], if we do not want to change the logic in the C++ code then we only need to play around in the **fragment.fs** (the fragment shader), this file handles the color output for the rendered image depends on the kernel / convolution matrix that we want to apply on the image or even just some nice RGB color interpolation or weighted grayscale. We don't need to recompile just call again **./main** and the image will be transformed / convoluted.

```
...

#version 330 core
out vec4 FragColor;

in vec3 ourColor;
in vec2 TexCoord;

// texture samplers
uniform sampler2D texture1;
uniform sampler2D texture2;
const float offset = 1.0 / 300.0;

void main()
{
       // linearly interpolate between both textures (10% texture1, 90% texture2)
       FragColor = mix(texture(texture1, TexCoord), texture(texture2, TexCoord),
           0.9);

       /* // Grayscale
       FragColor = texture(texture2, TexCoord);
       float average = (FragColor.r + FragColor.g + FragColor.b) / 3.0 ;
       FragColor = vec4(average, average, average, 1.0);
       */

       // /*
       // Weighted grayscale
       FragColor = texture(texture2, TexCoord);
       float average = 0.2126*FragColor.r + 0.7152*FragColor.g + 0.0722*FragColor.
           b ;
```

```
        FragColor = vec4(average, average, average, 1.0);
        // */
...
```

Code 79: *Neural Network/Test Kernel Convolution Operation on an Image with OpenGL GLFW GLAD and STB Image/fragment.fs*

We have uncommented the weighted grayscale, so the image will be grayscaled like below when we do so, don't forget to keep all other convolution operations in comment.

Figure 10.36: *The weighted grayscale of an image (**SymIntegration/Examples/Neural Networks/Test Kernel Convolution Operation on an Image with OpenGL GLFW GLAD and STB Image/main.cpp**).*

In the current updated code, we create a new logic so it is more efficient and easier to see the convoluted image while the GLFW GUI window still active.

In this new logic, we use **void processInput(GLFWwindow *window, Shader* activeShader)** instead of **void processInput(GLFWwindow *window)**, so the input variables become 2, so we can interchange the shader in use easily while the GLFW window is still open. This **void** function evaluates key presses in the main render loop to compile/bind the desired shader program on demand. Before starting the render loop, we initialize the base shader with this:

Shader *activeShader = new Shader("vertex.vs", "fragment.fs");
We are not using memory assignment (&m); instead we dynamically allocate it with **new** so it can be cleanly deleted and re-instantiated.

Then in the render loop we use the dynamically managed shader with **activeShader->use();**.

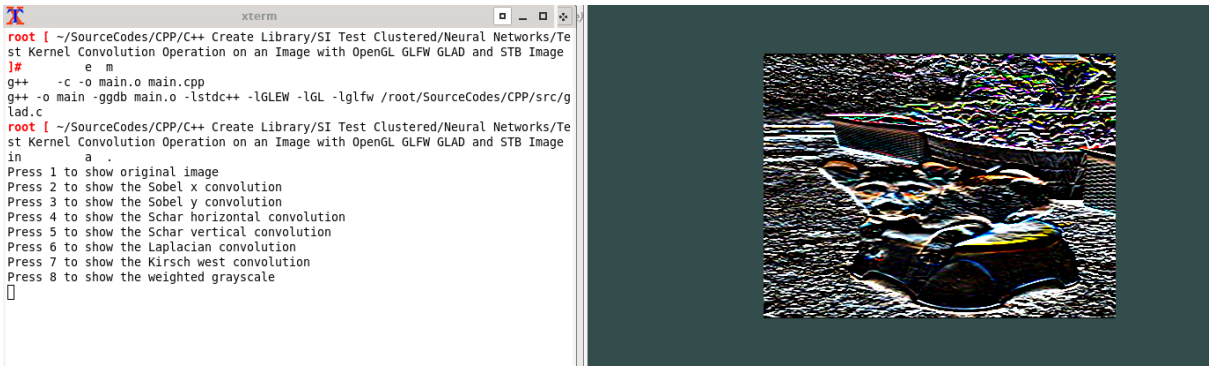

Figure 10.37: *By pressing keyboard keys from 1 to 8, we can alternate and see the convoluted image (**SymIntegration/Examples/Neural Networks/Test Kernel Convolution Operation on an Image with OpenGL GLFW GLAD and STB Image/main.cpp**).*

This is the prelude / introduction before we get into the real CNNs, even if in convolutional neural networks feature-extracting kernels are not pedefined, so we will not be using any of the Sobel X, or Sobel Y or Laplacian kernel. In CNNs we will have to perform a lot of matrix computation with tons of randomly initialized kernels / weights (when training from scratch) to detect the pattern in an image, and we will still need to do the backpropagation to adjusting the weights and biases. The network learns the optimal weight matrices / kernel matrices during the training process to detect complex features like eyes in a face or wheels on a car.

In the image example above we can even create an image classification CNN to detect whether it is a certain race of a dog, determine the age of the dog, or even the gender of the dog.

Implementing a convolutional neural network (CNN) convolution operation in OpenGL involves utilizing the GPU's parallel processing capabilities. We typically execute convolutions by loading input feature maps and kernels as textures, and using an OpenGL Shading Language (GLSL) **fragment shader** attached to a Frame Buffer Object (FBO) to perform a sliding-window dot product.

OpenGL is designed for image rendering, it natively handles 2D textures, vector arithmetic, and linear interpolation-which map directly onto CNN components:

- **Input Feature Map:** Bound as a GLSL **sampler2D** containing the input values.
- **Filter/Kernel:** Passed as shader uniforms, often structured as **vec4** arrays or a 1D/2D texture depending on size.
- **Dot Product:** Calculated per pixel inside the fragment shader, utilizing functions like **dot()** or nested **for** loops.

The fragment shader is responsible for reading the neighboring pixels of the current coordinate, multiplying them by the kernel weights, and outputting the convolved value.

ii. Extract RGBA Matrices from an Image and then Create an Image from the RGBA Matrices with SymIntegration, OpenGL and STB Image

Remember that a computer sees an image as a matrix of numbers $[0, 255]$? it can be represented as a 3D matrix with 3 channels for RGB image, or with 4 channels for RGBA image, often the matrix A will have the same entries of 255.

We will use an example of Sine Bam Bam picture again, she is the best, one and only Gerberian Shepsky dog who is also a Mini Pincher mix, a very rare breed, 1 in a billion.

First we will add a few lines of code before the render loop **while (!glfwWindowShouldClose(window)){}**

```
...

vector<vector<int>> matrixR(height, vector<int>(width, 0.0));
vector<vector<int>> matrixG(height, vector<int>(width, 0.0));
vector<vector<int>> matrixB(height, vector<int>(width, 0.0));
vector<vector<int>> matrixA(height, vector<int>(width, 0.0));
// Save the RGBA matrix into 4 matrices
for (int y = 0; y < height; y++)
{
       for (int x = 0; x < width; x++)
       {
              // 1. Calculate the starting memory address of the pixel
              unsigned char* pixel_ptr = data + (y * width + x) * nrChannels;

              // 2. Extract components based on available channels (e.g., RGB or
                  RGBA)
              // Cast to int to print numerical values rather than ASCII
                  characters
              int r = pixel_ptr[0];
              int g = (nrChannels >= 2) ? pixel_ptr[1] : 0;
              int b = (nrChannels >= 3) ? pixel_ptr[2] : 0;
              int a = (nrChannels >= 4) ? pixel_ptr[3] : 255;
              matrixR[y][x] = r;
              matrixG[y][x] = g;
              matrixB[y][x] = b;
              matrixA[y][x] = a;
       }
}
saveMatrixint(matrixR,"matrixR.txt");
saveMatrixint(matrixG,"matrixG.txt");
saveMatrixint(matrixB,"matrixB.txt");
saveMatrixint(matrixA,"matrixA.txt");
stbi_image_free(data);

...
```

Code 80: *Neural Network/Test Create RGBA Matrices with SymIntegration OpenGL GLFW GLAD and STB Image/main.cpp*

those lines of codes will extract the RGBA matrices into the current working directory.

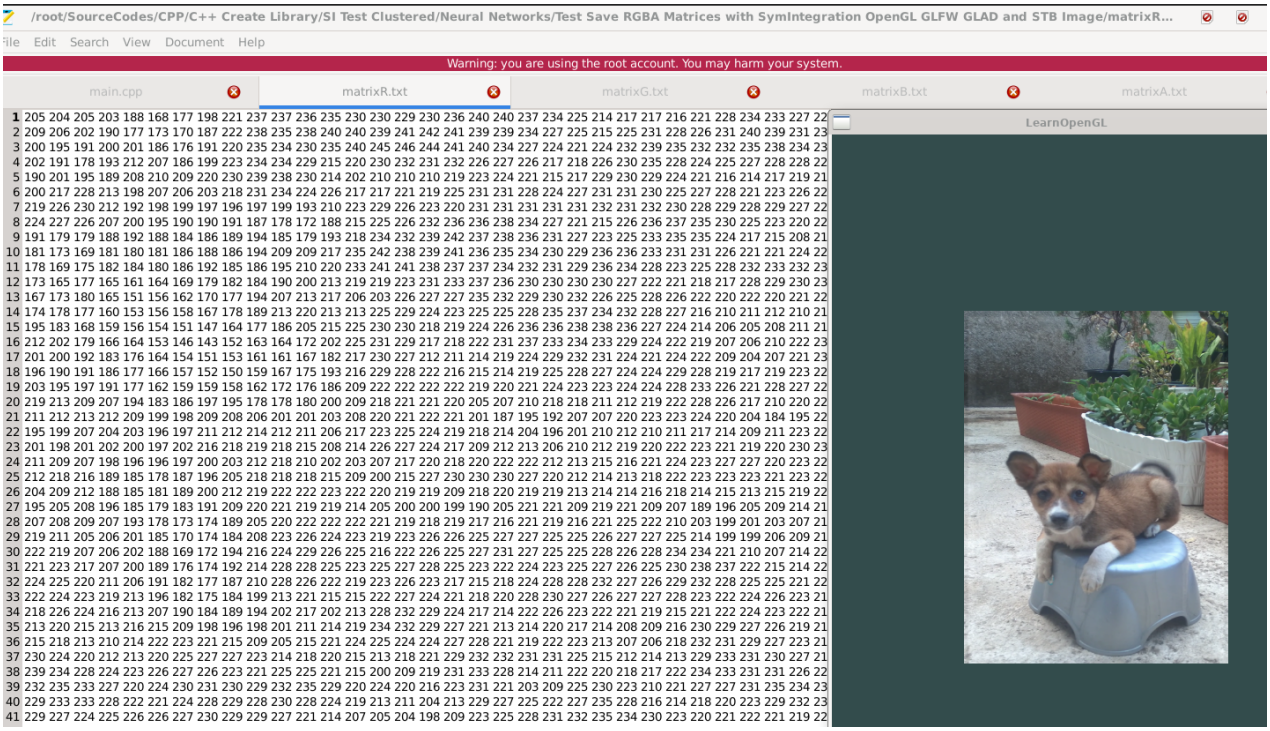

Figure 10.38: *When the program starts it will save separatedly each of the RGBA matrices from the second texture that we load (**/sources/Hamzstlab/LearnOpenGL/textures/6.png**) as **.txt** file in the current working directory (**SymIntegration/Examples/Neural Networks/Test Create RGBA Matrices with SymIntegration OpenGL GLFW GLAD and STB Image/main.cpp**).*

After this we can copy the **matrixR.txt, matrixG.txt, matrixB.txt, matrixA.txt** into the **SymIntegration/Examples/Neural Networks/Test Create Image from RGBA Matrices with STB Image/** folder, if there are previous RGBA matrices then they will be overwritten.

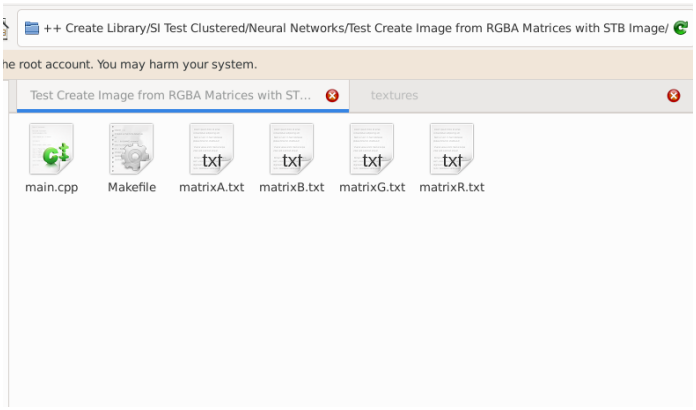

Figure 10.39: *Before we compile the program this is the current working directory (**SymIntegration/Examples/Neural Networks/Test Create Image from RGBA Matrices with STB Image/**).*

open xterm/terminal in the current working directory and compile then run the program:
make
./main

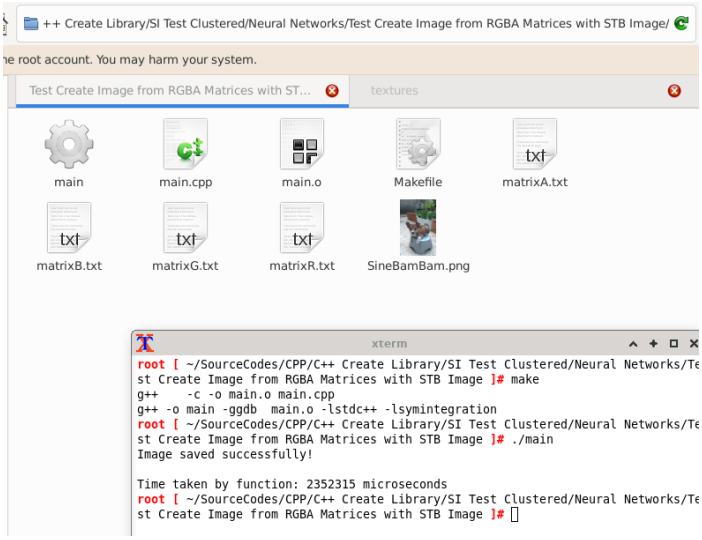

Figure 10.40: *After running the program we obtain the image, same as the image from **/sources/Hamzst-lab/LearnOpenGL/textures/6.png** with 522 KB of size, this one has higher size of 721 KB, better resolution, sharper perhaps. (**SymIntegration/Examples/Neural Networks/Test Create Image from RGBA Matrices with STB Image/main.cpp**).*

```
#include <iostream>
#include "symintegrationc++.h"
#include <bits/stdc++.h>
#include <cmath>
#include <chrono>

using namespace std::chrono;
```

```
using namespace std;
using namespace SymbolicConstant;

#define STB_IMAGE_WRITE_IMPLEMENTATION
#include "stb_image_write.h"


int main() {
	int width = 434;
	int height = 679;
	int channels = 4; // RGBA

	// Get starting timepoint
	auto start = high_resolution_clock::now();
	// Allocate memory for the pixel array (width * height * 4 bytes)
	uint8_t* pixels = (uint8_t*)malloc(width * height * channels);
	//int pixels[width * height * channels];
	if (pixels == NULL)
	{
		printf("Memory allocation failed!\n");
		return 1;
	}

	vector<vector<int>> matrixR = loadIntMatrixFromFile("matrixR.txt");
	vector<vector<int>> matrixG = loadIntMatrixFromFile("matrixG.txt");
	vector<vector<int>> matrixB = loadIntMatrixFromFile("matrixB.txt");
	vector<vector<int>> matrixA = loadIntMatrixFromFile("matrixA.txt");

	// Generate the image from RGBA matrices
	for (int y = 0; y < height; y++)
	{
		for (int x = 0; x < width; x++)
		{
			int index = (y * width + x) * channels;
			pixels[index + 0] = (uint8_t)matrixR[y][x]; // Red channel
			pixels[index + 1] = (uint8_t)matrixG[y][x]; // Green channel
			pixels[index + 2] = (uint8_t)matrixB[y][x]; // Blue channel
			pixels[index + 3] = (uint8_t)matrixA[y][x]; // A channel
		}
	}

	// Calculate stride: distance in bytes between the start of consecutive
	    rows
	int stride = width * channels;
	stbi_flip_vertically_on_write(true); // tell stb_image.h to flip the saved
	    image's on the y-axis.
	// Save the pixel data to a PNG file
	if (stbi_write_png("SineBamBam.png", width, height, channels, pixels,
```

```
            stride))
        {
            printf("Image saved successfully!\n");
        }
        else
        {
            printf("Failed to save image.\n");
        }
        // stbi_write_jpg("output.jpg", width, height, channels, pixels, 100); //
            100 is quality (1—100)
        // Get ending timepoint
        auto stop = high_resolution_clock::now();
        auto duration = duration_cast<microseconds>(stop — start);

        cout << "\nTime taken by function: " << duration.count() << " microseconds"
             << endl;
        // Always clean up allocated memory
        free(pixels);
        return 0;
}
```

Code 81: *Neural Network/Test Create Image from RGBA Matrices with STB Image/main.cpp*

We need to define the correct width and height of the original image at **/sources/Hamzstlab/LearnOpenGL/textures/6.png**, we are also using **malloc()**, it is inherited from C language and always paired with **free()**, notice that this is the alternative of **new** operator that we have used in the previous project when we perform kernel / convolution matrix on an image, we delete current activeShader to create a new Shader that will become the currect activeShader based on keyboard press.

Some things need to remember when using **malloc()**:

- If you use **malloc()** on a C++ class or struct, the constructor will not run. This means internal elements (like **std::string** fields) will remain uninitialized, leading to immediate program crashes.
- Memory allocated via **malloc()** lives forever until the program closes or until you call **free()** this is called memory leaks problem.
- It allocates raw bytes onlye, while **new** operator calls object constructors.
- It returns **void*** (requires cast), while **new** operator automatically returns correct type.
- It returns **NULL** when it has failure, while **new** operator throws **std::bad_alloc** exception.

The main point of this project example is to extract the RGB matrices, we can leave alone the matrix A, since it is sufficient with RGB / 3 channels only for CNN to do image classification.

Suppose we want to classify image based on the images that we have with label on each image as well (so we are not only depend on the MNIST dataset / images), we will need to learn from the two projects (the previous one and this current project), resize all of the images so the input layer will be uniform, then extract the RGB matrices of each image then perform the CNN operation to get the best model weights and biases that has high accuracy to predict the next image.

iii. Convolutional Neural Networks for Handwriting Recognition: Digit Classifier with SymIntegration

[SI*] Handwriting recognition is a classic, nontrivial task in artificial intelligence and machine learning and well-suited for neural networks, yet recognizing hand-written digits is still feasible as a first problem.
Fortunately, a large database of hand-written digits is available as the MNIST (modified National Institute of Standards and Technology) database, which contains 60,000 training images and 10,000 test images. The grayscale images have a size of 28×28 pixels and were digitized from digits written by employees of the United States Census Bureau and American high-school students.

The classification pipeline for the MNIST digit recognition with LeNet-5 architecture:

1. **Data Preparation:** Load the dataset (MNIST) and normalize pixel values from $[0, 255]$ to $[-1, 1]$ so the model converges faster during training.
2. **Compilation:** Select an optimizer between Adam or SGD (Stochastic Gradient Descent), a loss function (e.g. sparse categorical cross-entropy), and metrics.
3. **Input Layer:** Determine the input data, the standard MNIST digit recognition uses 28×28 images, we can choose whether to flattenind the image into a vector with 784 elements (using the **CSV** file format) or to still use the images as 3D matrix (using the **idx3-ubyte** format), we will break down the differences between these two below.
4. **Convolutional Layers:** Apply filters (kernels) to slide over the image and extract distinct visual features like lines and loops.
5. **Activation function:** Use ReLU / tanh / sigmoid / scaled hyperbolic tangent activation function, it introduces non-linearity so the network can learn complex patterns.
6. **Pooling Layers:** Downsample the data to reduce spatial dimensions and computation while keeping the most critical features.
7. **Flattening:** Converts the multidimensional feature maps into a 1-dimensional array.
8. **Fully Connected Layers (Dense):** Interpret the extracted features and assign probabilities to each digit class (0 through 9).
9. **Output Layer:** Uses a softmax activation function to output the final prediction

$$\boldsymbol{z}_i = w_i \cdot \boldsymbol{a}_i + \boldsymbol{b}_i$$

$$a_2 = \text{Softmax}(z_2)$$

The class with the highest probability is chosen as the predicted label.

Instead of using MNIST dataset, we create our own handwritten digits about 100 images thus in this section we will use our own handwritten digits as the input image, the explanation that is using MNIST dataset is just a model benchmark.

We will feed the training images data into the model in batches across multiple epochs to continuously adjust the weights and improve accuracy, once done we will test the model on unseen data (the test images) to calculate the accuracy and then using SFML (a multimedia C++ library) we can create a GUI to create a canvas where we can write a digit $0 - 9$ and then the model will predict our hand-drawn digits.

At the start of making a C++ code for digit recognition with CNN we will have to choose whether to use **idx3-ubyte** file or a **CSV** file. The primary difference is how the data is structured and stored: **idx3-ubyte** is the original compressed binary format, while **CSV** is a plain-text representation created for human readablity and easier parsing.

Feature	**idx3-ubyte** Binary Format	MNIST **CSV** Format
File Type	Raw Binary	Plain Text (ASCII)
Data Separation	Separated (Images and labels are stored in distinct files)	Unified (The label and its 784 pixels are stored in the same row)
Structure	3D Matrix ($60000 \times 28 \times 28$)	2D Tabular Row (1 label + 784 columns)
File Size	Very Small ($\pm$ 47 MB unzipped)	Large ($\pm$ 100 MB unzipped)
Readability	Requires a custom binary parser	Readable with Excel or Text Editors
I/O Speed	Blazing fast to load	Much slower due to text-to-number conversion

Table 10.3: *The differences between the MNIST' **idx3-byte** file and an MNIST' **CSV** file.*

The explanation of **idx3-ubyte** format:

- **The Name: idx** means indexed, **3** denotes it is a 3-dimensional tensor (number of images $\times$ rows $\times$ columns), and **ubyte** means "unsigned byte" (values from 0 to 255).
- **Separation:** To use it, you must download two separate files: **train-images-idx3-ubyte** (the raw pixels) and **train-labels-idx1-ubyte** (the correct digit label).

The explanation of MNIST' **CSV** format:

- **The Structure:** Each row in CSV represents exactly one handwritten digit.
- **Layout:** The very first column / the last column contains the integer **label**. The remaining 784 columns represent the flattened pixel values on the 28×28 image.
- **How to open:** It is highly accessible. It can be opened with a simple text editor like mousepad.

If you want to perform quick data exploration with small datasets you can use **CSV** format, otherwise if you are training heavy Deep Learning networks then **idx3-ubyte** format is the best as it minimizes storage overhead and optimizes GPU data-loading pipelines.

After we are done with the format for the input layer, then we are going to explain about the model architecture for this CNN digit recognition example. A model achitecture is the design of the neural network with which we train the parameters in training process. We are using LeNet-5 architecture It is pretty popular for its minimal structure and easy to train nature.

LeNet-5, a gradient-based learning model, developed in 1998 by Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner [20] was developed for the purpose of document recognition. LeNet-5 is a small network and very efficient convolutional neural network for handwritten character recognition and classification of different classes of objects in small resolution images, it contains the basic modules of deep learning: convolutional layer, pooling layer, and full link layer. It is the basis of other deep learning models. LeNet-5 has

a total of seven-layer, that do not comprise an input, each containing trainable parameters; each layer has a plurality of the pap, the feature, a characteristic of each of the input feature map extracted by means of a convolution filter, and then each feature map has multiple neurons.
This arrangement of layers was designed to automatically learn the features from the input images, rather than relying on hand-engineered features, setting a new standard for machine learning applications.

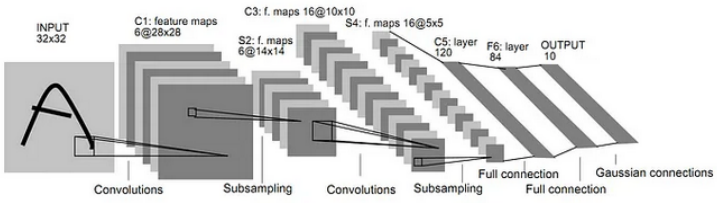

Figure 10.41: *The architecture of LeNet-5, a convolutional neural network for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.*

The model comprises of total 1 input layer + 6 processing layers + 1 output layer.

1. **Input Layer**
   In the data input layer, we make sure that the size of the input image is uniformly normalized to $32 \times 32 \times 1$, the "1" signifies that it is grayscaled (single-channel, no color). The original MNIST dataset consists of 28×28 but were padded to 32×32 to ensure that the features at the edges of the image could be captured by the network's filters.
   Grayscale image still has a pixel value of $[0, 255]$, thus normalization (scales pixel values to a range of $[0, 1]$ or $[-1, 1]$), is also necessary for stable and faster training.
   This layer does not count as the network structure of LeNet-5. Traditionally, the input layer is not considered as one of the network hierarchy.

2. **C1 Convolutional Layer**
   In LeNet-5, the first layer C1 performs the first convolutional operation on the input image (using 6 convolution kernels of size 5×5) to obtain 6 C1 feature maps. A filter, or kernel, is a small matrix of weights that slides over the image, performing a dot product (hadamard product) at each position, this process is also known as convolution, it allows the network to detect low-level features like horizontal line, edges, corners, and curves.
   Input picture: $32 \times 32 \times 1$
   it means we analyze one image at a time.
   Convolutional kernel size: 5×5
   Number of convolutional kernels: 6
   Stride: 1
   with a stride of 1, the filters/kernels move one pixel at a time.
   Padding: 0

   Output feature map size: 28×28
   the output spatial dimension formula: $O = \frac{I-K+2P}{S} + 1$, with $I = 32, K = 5, P = 0, S = 1$
   we will obtain $\frac{32-5+2(0)}{1} + 1 = 28$.

Number of neurons: $28 \times 28 \times 6 = 4,704$
Trainable parameters: $(5 \times 5 + 1) \times 6 = 156$
there are 25 unit parameters (the size of the convolutional kernel is $5 \times 5 = 25$) and one bias parameter per kernel/filter($+1$ indicates a kernel has a bias), a total of 6 filters.
Number of connections: $(5 \times 5 + 1) \times 28 \times 28 \times 6 = 122,304$
each pixel in C1 is connected to 5×5 pixels and 1 bias in the input image, so there are $122,304$ connections in total, but we only need to learn 156 parameters, mainly through weight sharing.
Output: $28 \times 28 \times 6$ feature maps

The mathematical formula for the C1 layer, for filter / kernel k ($k \in \{1, 2, \ldots, 6\}$), the output position (i, j) is defined as:

$$y_{i,j}^k = f\left(\sum_{m=0}^{K-1}\sum_{n=0}^{K-1} w_{m,n}^k \cdot x_{i+m,j+n} + b^k\right)$$

where:

- x is the 2D input matrix or previous layer's feature map.
- w^k is the k-th filter/kernel of size $K \times K$ (in LeNet-5, $K = 5$).
- b^k is the learnable bias term for the k-th filter.
- $f(\cdot)$ is the activation function (LeNet-5 originally uses the scaled hyperbolic tangent function).
- $y_{i,j}^k$is the pixel value at output coordinate (i, j) for the k-th feature map.

3. **S2 Pooling Layer**
The next layer is a pooling layer, which is used to reduce the spatial dimensions of the feature maps. Unlike a convolutional layer, a pooling layer does not learn a parameters. It is purely a deterministic mathematical function, we only train the parameters after the pooling operation (e.g., max pooling, average pooling, original LeNet-5 pooling). This has two key benefits: it reduces the number of parameters and computations in the network, and it heps to make the learned features more robust to small translations in the input image.
In LeNet-5, the S2 layer uses average pooling. It performs downsampling by taking the average of 2×2 non-overlapping windows from the previous layer (C1), multiplying by a trainable weight, adding a trainable bias, and applying a sigmoid activation function.
Input: $28 \times 28 \times 6$ feature maps
number of neurons from C1.
Input feature map size: 28×28
Sampling area: 2×2
Stride: 2
Sampling method: 4 inputs are added, multiplied by a trainable parameter, plus a trainable offset.
Number of feature maps / kernels: 6

Output feature map size: 14×14
pooling is performed using a 2×2 kernels, the output feature map size for a pooling (subsampling) layer in the LeNet-5 architecture is $O = \frac{I-K}{S} + 1 = \frac{28-2}{2} + 1 = 14$, K is the kernel size, if it is 2×2 then $K = 2$, S is the stride size there is no padding applied during LeNet-5 pooling stages, thus 6 feature maps of 14×14 are obtained. Each unit

connected to a 2×2 neighborhood in C1.
Number of neurons: $14 \times 14 \times 6 = 1,176$
Trainable parameters: $(1 + 1) \times 6 = 12$ (one weight and one bias per feature map)
(trainable weight / coefficient + bias) $\times$ filters
Number of connection: $(2 \times 2 + 1) \times 14 \times 14 \times 6 = 5,880$
Output: $14 \times 14 \times 6$ feature maps

The mathematical formula for each of the S2 layer's output feature map y_k, where $k \in \{1, 2, \ldots, 6\}$, the operation is computed as:

$$y_k = f\left(w_k \cdot \frac{1}{4} \sum_{i=1}^{4} x_i + b_k\right)$$

where:

- x_i represents the activations in the 2×2 receptive field of the previous C1 layer map.
- w_k is the trainable weight for that specific feature map.
- b_k is the trainable bias for that feature map.
- $f(\cdot)$ is the activation function (sigmoid or scaled hyperbolic tangent function)

4. **C3 Convolutional Layer**
The third layer for LeNet-5. The C3 layer is a convolutional layer that performs cross-correlation using 5×5 kernels. Its defining mathematical feature is a custom connectivity map where not every output map connects to every input map, reducing computation and enforcing varied feature extraction. Each feature map in C3 connected to all 6 or several feature maps in S2, indicating that the feature map of this layer is a different combination of the feature maps extracted from the previous layer, these are the combinations :

   - **First 6 feature maps:** Connect to any **3 contiguous** subset of S2.
   - **Next 6 feature maps:** Connect to any **4 contiguous** subset of S2.
   - **Next 3 feature maps:** Connect to any **4 discontinuous** subset of S2.
   - **Last 1 feature map:** Connects to all **6** input maps from S2.

   for those who wonder why we obtain $10 \times 10 \times 16$ feature maps (16 feature maps of size 10×10) in C3 from $14 \times 14 \times 6$ feature maps in S2. The C3 layer's 16 feature maps are not fully connected to all 6 input feature maps from the previous layer (S2). This was originally done to limit computation and break symmetry in order to learn diverse, higher-level combinations of basic shapes found in C1 layer.

   Thus, instead of simply applying every kernel/filter to every previous feature map, this layer combines subsets of the input feature maps from S2 / it uses sparse (partial) connectivity. This means individual kernels in C3 are only connected to specific combinations of feature maps from the previous layer.

   In C3, the 16 kernels / filters scan the $14 \times 14 \times 6$ output from the preceding pooling layer (S2), capturing more complex visual representations by combining the basic features.

To find all the contiguous and discontiguous combinations of k elements from n available elements you can check the function **contiguousCombinations(int** n**, int** k**, int** *start_idx*) and **discontiguousCombinations(int** n**, int** k**, int** *start_idx*), these functions are located in **src/operationsresearch.cpp** and explained in this book at chapter 5 Operations Research Computation with SymIntegration.

The first 12 feature maps are fixed because the contiguous combinations for choosing 3 elements from 6 available elements are 6 in total, the same as the contiguous combinations for choosing 4 elements from 6 available elements are also 6 in total, while the discontiguous combinations for choosing 4 elements from 6 available elements are 9 in total, and we only need 3, thus we will use a roulette wheel method, generating 4 random integers from 0 to 8 that are unique to each other. This roulette wheel method is adapted from genetic algorithms. The last feature map is also fix since it is a combination of all the 6 previous feature maps from S2.

Thus, in order to generate 4 unique integer random numbers that have uniform distribution with range $[0, 8]$ (C++ index starts from 0; 9 elements means the index will ends at 8) we can use this function in SymIntegration: **vrandn_uniqueinteger(0, 8, 4)**

```
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Operations Research/Test SymIntegration Unique Random Number Integer ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Operations Research/Test SymIntegration Unique Random Number Integer ]# ./main
3
8
0

Time taken by function: 610 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Operations Research/Test SymIntegration Unique Random Number Integer ]# ./main
5
2
7

Time taken by function: 563 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Operations Research/Test SymIntegration Unique Random Number Integer ]# ./main
3
7
0

Time taken by function: 575 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Operations Research/Test SymIntegration Unique Random Number Integer ]#
```

Figure 10.42: *The 3 unique integer generated with* ***vrandn_uniqueinteger(0, 8, 3)*** *that can be used to determine the next 3 feature maps (after the first 12) in C3 that connect to any 4 discontiguous subset of S2, it is using a modified roulette wheel method. (**SymIntegration/Examples/Operations Research/Test SymIntegration Unique Random Number Integer/main.cpp**).*

The roulette wheel method that is able to produce a set of unique integers is using **std::unordered_set<int>**, this is a standard C++ associative container that stores unique elements in no particular order. So if the set already contains the number 1, it cannot be inserted another 1 that is generated by the **std::uniform_int_distribution<>** with Mersenne Twister engine, it will generate again another integer random number until it obtains a number that is unique to all the numbers that are already in the set.

All discontiguous index combinations of 4 out of 6 are

$$
\begin{array}{cccc}
0 & 1 & 2 & 4 \quad \rightarrow \mathbf{0} \\
0 & 1 & 3 & 4 \quad \rightarrow \mathbf{1} \\
0 & 1 & 3 & 5 \quad \rightarrow \mathbf{2} \\
0 & 2 & 3 & 4 \quad \rightarrow \mathbf{3} \\
0 & 2 & 3 & 5 \quad \rightarrow \mathbf{4} \\
0 & 2 & 4 & 5 \quad \rightarrow \mathbf{5} \\
1 & 2 & 3 & 5 \quad \rightarrow \mathbf{6} \\
1 & 2 & 4 & 5 \quad \rightarrow \mathbf{7} \\
1 & 3 & 4 & 5 \quad \rightarrow \mathbf{8}
\end{array}
$$

So if we choose $3, 7, 0$ from the roulette wheel method the index of feature maps in S2 that are choosen for the next 3 feature maps (after the first 12) in C3 are

$$
\begin{array}{cccc}
0 & 1 & 2 & 4 \quad \rightarrow \mathbf{0} \\
0 & 2 & 3 & 4 \quad \rightarrow \mathbf{3} \\
1 & 2 & 4 & 5 \quad \rightarrow \mathbf{7}
\end{array}
$$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	X				X	X	X			X	X	X	X		X	X
1	X	X				X	X	X			X	X	X	X		X
2	X	X	X				X	X	X			X		X	X	X
3		X	X	X			X	X	X	X			X		X	X
4			X	X	X			X	X	X	X		X	X		X
5				X	X	X			X	X	X	X		X	X	X

Figure 10.43: *The combinations to create feature maps in C3, each column indicates which feature map in S2 are combined to create a particular feature map in C3.*

Output Maps (C3)	Inputs Connected (S2 Maps)	Number of Input Maps
Maps 0 to 5	Indices $\{0,1,2\}, \{1,2,3\}, \{2,3,4\}, \{3,4,5\}, \{4,5,0\}, \{5,0,1\}$	3
Maps 6 to 11	Indices $\{0,1,2,3\}, \{1,2,3,4\}, \{2,3,4,5\}, \{3,4,5,0\}, \{4,5,0,1\}, \{5,0,1,2\}$	4
Maps 12 to 14	Indices $\{0,1,3,4\}, \{1,2,4,5\}, \{2,3,5,0\}$	4
Maps 15	All indices $\{0,1,2,3,4,5\}$	6

Table 10.4: *The input combinations for creating the feature maps in C3 layer, this is the C3 connectivity map. In the original LeNet-5 paper, the 16 output maps of C3 take input from the 6 maps of S2 using a custom subset table to form varying depths.*

Input size: $14 \times 14 \times 6$
Input feature map size: 14×14
Convolutional kernel size: 5×5

Number of convolutional kernels: 16
Stride: 1
with a stride of 1, the filters/kernels move one pixel at a time.
Padding: 0

Output feature map size: 10×10
the output spatial dimension formula: $O = \frac{I-K+2P}{S} + 1$, with $I = 14, K = 5, P = 0, S = 1$ we will obtain $\frac{14-5+2(0)}{1} + 1 = 10$.
Number of neurons: $10 \times 10 \times 16 = 1,600$
Trainable parameters: $(5 \times 5) \times (6 \times 10) + 16 = 1,516$
the number of trainable parameters varies since C3 is not connected to every feature map of the previous layer. Instead, each feature map of C3 is connected to a subset of the previous maps. There are 16 kernels' receptive fields that are divided into 6 of 3 S2' features, 9 of 4 S2' features, and 1 of 6' S2 features (the size of the C3 convolutional kernel is $5 \times 5 = 25$) and one bias parameter per kernel/filter($+16$ indicates that one kernel has only one bias).
Number of connections: $(10 \times 10) \times 1,516 = 151,600$
the partial connectivity reduces parameters and encourages diverse feature learning.
Output: $10 \times 10 \times 16$ feature maps

The mathematical formula for each of the C3 layer's output feature map y_k, where $k \in \{1, 2, \ldots, 16\}$, the operation is computed as:

$$y_k = f\left(\sum_{m \in M_c} x_m * w_{k,m} + b_k\right)$$

where:

- y_k is the output feature map of the k-th channel in the C3 layer.
- $*$ denotes the 2D spatial cross-correlation operator.
- x_m is the m-th input feature map from the previous layer (S2).
- $w_{k,m}$ is the learnable 5×5 convolution kernel connecting the m-th input map to the k-th output map.
- b_k is the trainable bias for the k-th output map.
- $f(\cdot)$ is the activation function (typically the scaled hyperbolic tangent function).
- M_k is the subset of input maps from the S2 layer that are connected to the k-th output map.

5. **S4 Pooling Layer**
The next layer is another pooling layer. In LeNet-5, the S4 layer is a subsampling (average pooling) layer. It takes the 10×10 feature maps from the C3 layer and reduces them. For each of the 16 feature maps, S4 computes the average of 2×2 non-overlapping blocks, multiplies it by a trainable weight, adds a bias, and passes it through an activation function.
Input: $10 \times 10 \times 16$
Input feature map size: 10×10
Sampling area: 2×2
Stride: 2
Sampling method: 4 inputs are added, multiplied by a trainable parameter, plus a

trainable offset.
Number of feature maps / kernels: 16

Output feature map size: $\frac{10}{2} \times \frac{10}{2} = 5 \times 5$
pooling is performed using a 2×2 kernels, 16 feature maps of 10×10 are obtained. Each unit connected to a 2×2 neighborhood in C3.
Number of neurons: $5 \times 5 \times 16 = 400$
Trainable parameters: $(1 + 1) \times 16 = 32$ (one weight and one bias per feature map)
(coefficient + bias) $\times$ filters
Number of connection: $(2 \times 2 + 1) \times 5 \times 5 \times 16 = 2,000$
output size = $5 \times 5 \times 16$, the size of each feature map in S4 is $\frac{1}{2}$ of the size of the feature map in C3.
Output: $5 \times 5 \times 16$ feature maps

The mathematical formula for each of the S4 layer's output feature map y_k, where $k \in \{1, 2, \ldots, 16\}$, the operation is computed as:

$$y_k = f\left(w_k \cdot \frac{1}{4} \sum_{i=1}^{4} x_i + b_k\right)$$

where:

- y_k is the output of the subsampled feature map.
- x_i is the four input values from the 2×2 receptive field in the preceding C3 layer.
- w_k is a trainable weight, specific to the feature map.
- b_k is a trainable bias, specific to the feature map.
- f is the activation function (historically a scaled hyperbolic tangent, or tanh, in the original LeNet-5).

We can also write the forward pass equation for an average pooled unit from C3 to S4 k-th feature map as:

$$x_{i,j,k}^{S4} = f\left(w_k^{C3} \cdot \frac{1}{4} \sum_{m=1}^{2} \sum_{n=1}^{2} x_{2i-2+m,2j-2+n}^{C3} + b_k^{C3}\right)$$

with $i, j = 0, 1, 2, 3, 4$.

6. **C5 Convolutional Layer**
The next layer is a convolutional layer again. In LeNet-5, the C5 layer is a fully connected convolutional layer. It acts as the bridge between the convolutional/pooling features and the traditional dense neural network. The C5 layer has 120 featured maps, with each feature map measuring 1×1. It connects to the output of the previous layer, S4, which consists of 16 feature maps, each measuring 5×5, it then convolves them with 120 filters of size 5×5. Since the size of the preceding layer' feature map (S4) precisely matches the size of the filter/kernel in C5 (5×5), there is no spatial reduction, then the output will be a feature map of size 1×1 / exactly 1 scalar value is produced per filter.

Input: $5 \times 5 \times 16$
Input feature map size: 5×5
Convolutional kernel size: 5×5

Output feature map size: 120×1

Trainable parameters: $((5 \times 5 \times 16) \times 120) + 120 = 48,120$
(input size $\times$ output feature map size + bias)

Number of connection: Parameters $\times$ output area = $48,120 \times 1 \times 1 = 48,120$ fully connected to all previous feature maps.
Output: $1 \times 1 \times 120$ a vector

The fundamental mathematical formula for the C5 layer's forward pass consists of two operations: **convolution** and **activation**.

(a) **The Convolution Operation**
For each output feature map o_k where $k \in \{1, 2, \ldots, 120\}$, the operation is computed as:

$$o_k = \sum_{c=1}^{16} (w_{k,c} * x_c) + b_k$$

where:

- $*$ denotes the 2D discrete convolution operator.
- x_c is the c-th input feature map from the 16 feature maps of layer S4.
- $w_{k,c}$ is the 5×5 convolutional kernel connecting the c-th input map to the k-th output map.
- b_k is the learnable bias term for the k-th filter.

(b) **The Activation Function**
The resulting pre-activation from the convolution step is passed through a non-linear activation function (f). LeNet-5 originally used a scaled hyperbolic tangent (tanh) activation:

$$y_k = f(o_k) = 1.7159 \tanh\left(\frac{2}{3} o_k\right)$$

7. **F6 Fully Connected Layer**
The next layer is a fully connected layer, which might reminds you to feed-forward neural network. The Lenet-5'F6 layer processes the 120-dimensional output vector from the preceding C5 layer and outputs an 84-dimensional vector, this 84 nodes corresponding to a 7×12 ASCII bitmap matrix used to decode and recognize stylistically similar alphanumeric characters, -1 means white, 1 means black, so the black and white of the bitmap of each symbol corresponds to a code.
Input size: 120
Output size: 84

Trainable parameters: $(120 \times 84) + 84 = 10,164$
(input size $\times$ output size + bias)

Number of connection: fully connected to all previous neurons in the previous layer C5.

The mathematical formula for each of the F6 layer's output y_k, where $k \in \{1, 2, \ldots, 84\}$,

the operation is computed as:

$$y_k = f\left(\sum_{j=0}^{119} w_{kj} x_j + b_k\right)$$

where:

- w_{kj} is the weight connecting the j-th input from C5 layer to the k-th neuron in the F6 layer.
- x_j is the j-th element of the 120-dimensional input vector from the C5 layer.
- b_k is the trainable bias for the k-th neuron in the F6 layer.
- f is the activation function, in the original LeNet-5 formulation, the F6 layer uses a scaled hyperbolic tangent function to introduce non-linearity and keep the network's values within a specific range:

  $$f(x) = A \tanh(Sx)$$

  with $A = 1.7159$ and $S = \frac{2}{3}$.

8. **Output Layer**
   The output layer is also a fully connected layer, with a total of 10 nodes, which respectively represent the numbers 0 to 9, and if the value of node i is 0, the result of network recognition is the number i (if we are using RBF / radial basis function) For the case of digit recognition we will have the output layer of size 10.

$$\vec{y} = \begin{bmatrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \\ 8 \\ 9 \end{bmatrix}$$

Input size: 84
Output size: 10
Trainable parameters: (84 $\times$ 10) + 10 = 850
Number of connection: 850 (fully connected to all previous neurons in the previous layer F6)

In the original LeNet-5 architecture, the output layer utilizes Radial Basis Function (RBF) units. Modern implementations, however, usually substitute this with a softmax layer for multi-class classification.

Radial Basis Function
Each class is represented by a Radial Basis Function (RBF) unit, where the output depends on the Euclidean distance between the input and its parameter vector, with larger distances indicating poorer fit. In the 1998 paper, each of the $i = 10$ output nodes (representing digits 0 through 9) evaluates how closely the 84-dimensional feature

vector x from the previous layer matches the predefined, hand-coded target bitmaps vector of that digit class.

The formula to compute the output of each RBF unit y_i is the squared Euclidean distance:

$$y_i = \sum_{j=1}^{84} (x_j - w_{ij})^2 \tag{10.85}$$

where:

- x_j represents the j-th element of the input vector from layer F6.
- w_{ij} is a fixe, non-trainable weight representing the j-th pixel value of a stylized 7×12 binary bitmap for character class i.
- The target class selected by the network is $\min_i(y_i)$, meaning the class with the minimum distance is the predicted digit.

Softmax
Most modern deep learning frameworks adapt LeNet-5 by replacing RBF layer with a standard Dense/Fully connected layer followed by a softmax activation function.

The output vector $\hat{y}$ is defined as

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{k=0}^{9} e^{z_k}}$$

where the raw logit z_i for class i is calculated via standard matrix multiplication:

$$z_i = \sum_{j=1}^{84} x_j w_{ij} + b_i$$

where:

- $x_j$is the j-th element of the input vector from layer F6.
- w_{ij} are the learnable weigts of the j-th input vector from layer F6 to the i-th output vector in the output layer.
- b_i is the learnable bias of the output layer.
- z_k represents the logits for all 10 possible classes.

Backpropagation and Weights Update for LeNet-5
Backpropagation is a fundamental training algorithm in CNNs that propagates errors backwards through the network to optimise learning. It enables the model to adjust internal parameters and improve performance over time. In CNN convolutional layer, backpropagation updates the filter weights by calculating the gradient of the loss function with respect to each weight element using the cross-correlation between the input feature map and the upstream gradient map.
To define the formulas clearly, we track a single forward and backward pass with these variables:

- X as the input feature map to the convolutional layer.
- W as the filter (kernel) representing the shared weights.
- Z as the output feature map before activation ($Z = X * W + b$).
- L as the final scalar loss function value.

- $\frac{\partial L}{\partial Z}$ as the upstream gradient matrix received from the subsequent layer.

For a standard 2D convolution (mathematically implemented as valid cross-correlation), the value at position (i, j) of the output map Z is calculated as:

$$z_{i,j} = \left(\sum_m \sum_n x_{i+m,j+n} \cdot w_{m,n} \right) + b$$

During the backpropagation, we apply the calculus chain rule. Because a single weight $w_{m,n}$ is reused across every receptive field during the forward pass, its total impact on the loss is the sum of its impacts across all output positions:

$$\frac{\partial L}{\partial w_{m,n}} = \sum_i \sum_j \frac{\partial L}{\partial z_{i,j}} \cdot \frac{\partial z_{i,j}}{\partial w_{m,n}}$$

Since $\frac{\partial Z_{i,j}}{\partial w_{m,n}} = x_{i+m,j+n}$, substituting this yields the final Weight Gradient Formula:

$$\frac{\partial L}{\partial w_{m,n}} = \sum_i \sum_j \frac{\partial L}{\partial z_{i,j}} \cdot x_{i+m,j+n}$$

The explicit sum describes a valid cross-correlation operation where the input feature map X acts as the larger matrix and the upstream gradient $\frac{\partial L}{\partial Z}$ acts as the sliding filter:

$$\frac{\partial L}{\partial W} = X * \frac{\partial L}{\partial Z}$$

To continue passing gradients to earlier layers, you must also compute how the loss changes with respect to the input matrix X. This requires a full convolution (padding the upstream gradient matrix) using a 180-degree rotated version of the weight filter:

$$\frac{\partial W}{\partial X} = \text{pad}\left(\frac{\partial L}{\partial Z} \right) * \text{rot}_{180}(W)$$

Once the weight gradient matrix $\frac{\partial L}{\partial W}$ is isolated, the shared weights are updated using gradient descent:

$$W^{new} = W^{old} - \eta \cdot \frac{\partial L}{\partial W}$$

with η as the learning rate and $\frac{\partial L}{\partial W}$ as the gradient with respect to weights.

In a convolutional neural network like LeNet-5, computing the weight gradient does not involve rotating the weight matrix. Instead, we compute the gradient with respect to the weights by performing a valid 2D convolution between the input features and the error gradients from the subsequent layer.

1. **Weight Gradient (Filter/Kernel Update)**
   Operation: Valid 2D convolution (for C1, C3, C5 layer).
   Method: To calculate the gradient for a specific filter, you perform a standard convolution (or cross-correlation) using the input patch as the feature map and the upstream error gradient (δ^{L+1}) from the next layer as the kernel.
   Rotation Required: No rotation is required. We do not rotate the weight matrix to find the weight gradient. We simply correlate the incoming error with the layer's input.

2. **Input Gradient / Error Propagation (δ) to Previous Layers**
   Operation: Full 2D convolution (for S2 and S4 layer).
   Method: When passing error gradients backwards to the previous layers, we need to convolve the upstream error gradient (δ) with the layer's weights.
   Rotation Required: Yes. To make the convolution mathematically align during this backward pass, the weight matrix (filter) must be rotated (or flipped) by 180^0 (flipped vertically and horizontally). We only perform full convolution on S2 and S4 layer because when we perform forward pass from S2 to C3 or S4 to C5 we use weights kernel of size $k \times k$.

Feature	Valid Convolution	Full Convolution
Padding	None	Implicitly pads the input
Output Size	Smaller than the input	Larger than the input
Formula	Output = $N - K + 1$	Output = $N + K - 1$
Pros	Highly accurate because all calculations are performed on real input pixels, not artificial zeros	Every pixel-including the original corner pixels-is visited exactly the same number of times by the filter.
Cons	The feature map shrinks with every layer. If your network is very deep, the data can shrink down too fast to extract meaningful hierarchical features.	The output includes elements that are calculated partially over "fake" zero-padded values, which can introduce artifacts along the edges.
Use Case	Standard CNN layers to extract features without "fake" padded data	Signal processing, polynomial multiplication, or inverting operations.

Table 10.5: *Key differences between 2D valid convolution and 2D full convolution.*

Now we will start the backpropagation / training the parameters from the last layer / output layer to the C1 layer in LeNet-5 architecture:

1. **Output Layer**
   In LeNet-5, if we are using softmax in the output layer for classification tasks, then we will use cross-entropy loss to calculate the Loss:

$$L = -\sum y \log(\hat{y})$$

   with y as the true label and $\hat{y}$ as the predicted label.

   We can compute the delta (δ) at this output layer with this formula:

$$\delta_j^{output\ layer} = \frac{\partial L}{\partial a_j^{output\ layer}} f'(z_j^{output\ layer}) = f'(s_j^{output\ layer}) \left(y_j - \hat{y}_j\right)$$

   with $\delta_j^{output\ layer}$ as the delta error term for the j-th neuron in the output layer, L is the loss or cost function (MSE or cross-entropy), $a_j^{output\ layer}$ is the actual output / activation

of the j-th neuron, $z_j^{output\ layer}$ is the weighted sum of inputs for the j-th neuron before activation ($z = \sum_i w_{ij}x_i + b_j$), $f'(z_j^{output\ layer})$ is the derivative of the activation function for the output layer, $f'(s_j^{output\ layer})$ as the derivative of softmax, which is a Jacobian matrix, $j = 1, 2, \ldots, n_{classification}$.

MSE & Linear Activation
For an output node with a linear activation function, the delta combines the prediction error with the derivative of the identity function:

$$\delta_j^{output\ layer} = (\hat{y}_j^{output\ layer} - y_j)1$$

with y_j as the actual target value. This simplification is often used in frameworks because it is computationally cheaper and more numerically stable than calculating the full Jacobian matrix separately.

Cross-Entropy Loss & Softmax Activation
LeNet-5's original design uses Euclidean Radial Basis Functions (RBF) in the output layer, but modern implementations of LeNet-5 frequently swap this for a softmax output layer combined with cross-entropy loss. For the j-th neuron:

$$\delta_j^{output\ layer} = a_j^{output\ layer} - y_j$$

with $y_j = 1$ if the true class is j, and 0 otherwise.

After we have the $\delta_j^{output\ layer}$ then we can compute the weight update for the output layer:

$$w_{ij}^{new} = w_{ij}^{old} - \eta\delta_j^{output\ layer} y_i^{F6}$$

with η as the learning rate, $\delta_j^{output\ layer}$ as the j-th error term in the output layer, y_i^{F6} is the activated output from i-th neuron in F6 layer, $i = 1, 2, \ldots, 84,\ j = 1, 2, \ldots, 10$.

The bias update formula for the output layer is

$$b_j^{new} = b_j^{old} - \eta\delta_j^{output\ layer}$$

2. **F6 (10,164 trainable parameters)**
In LeNet-5, F6 layer is a fully connected layer with 84 neurons that uses a scaled hyperbolic tangent activation function. Since F6 layer has no weight sharing, every single parameter acts as a distinct connection between the source nodes (120 outputs of C5) and the destination nodes (84 neurons in F6). The weight update that is connecting the i-th neuron from C5 layer to the j-th neuron in the F6 layer:

$$w_{ij}^{new} = w_{ij}^{old} - \eta\frac{\partial L}{\partial w_{ij}} = w_{ij}^{old} - \eta\delta_j^{F6}a_i^{C5}$$

with η as the learning rate, δ_j^{F6} as the error gradient calculated for the j-th neuron in the F6 layer, a_i^{C5} is the activated output of the i-th neuron from the C5 layer,

$i = 1, 2, \ldots, 120,\ j = 1, 2, \ldots, 84.$

The bias update formula for the F6 layer is

$$b_j^{new} = b_j^{old} - \eta \delta_j^{F6}$$

Once the output delta is calculated, you can propagate the error backward to the preceding fully connected layer (F6).

The formula to compute the delta error term (δ_j^{F6}) during backpropagation is using the chain rule, which multiplies the downstream gradients by the derivative of the activation function, the formula is

$$\delta_j^{F6} = \left(\sum_k^{10} \delta_k^{output\ layer} w_{jk} \right) f'(z_j^{F6})$$

with δ_j^{F6} as the delta error term for the j-th neuron in F6 layer, $\sum_k^{10} \delta_k^{output\ layer} w_{jk}$ is the backpropagated error from the next layer (the output layer), this is the sum of the delta terms of all subsequent neurons k, weighted by their respective connection weights (w_{jk}) to neuron j in F6 layer, $f'(z_j^{F6})$ is the derivative of the scaled hyperbolic tangent activation function with respect to the pre-activation z_j.
The original LeNet-5 utilizes the following activation function in F6 layer

$$f(z) = 1.7159 \tanh\left(\frac{2}{3}z\right)$$

its derivative with respect to the pre-activation z is

$$f'(z) = \frac{(1.7159)(2)}{3} \operatorname{sech}^2\left(\frac{2}{3}z\right) \approx 1.1439 \left[1 - \tanh^2\left(\frac{2}{3}z\right)\right]$$

3. **C5 (48,120 trainable parameters)**
In LeNet-5, the C5 layer is functionally treated as the first fully connected layer during the weight update process. Weights (W) are updated during the backward pass using standard gradient descent and the chain rule.

To update any weight w_{ij} in the C5 layer, the formula is

$$w_{ij}^{new} = w_{ij}^{old} - \eta \frac{\partial L}{\partial w_{ij}}$$

with η as the learning rate and $\frac{\partial L}{\partial w_{ij}}$ as the gradient of the loss function L (cross-entropy) with respect to that specific weight, $i = 1, 2, \ldots, 400,\ j = 1, 2, \ldots, 84$.

The bias update formula for the C5 layer is

$$b_j^{new} = b_j^{old} - \eta \delta_j^{C5}$$

Since C5 is a fully connected layer, a weight's gradient depends on the downstream error (from the F6 layer) and the upstream activation (from the S4 layer). The formula is

$$\frac{\partial L}{\partial w_{ij}} = \frac{\partial L}{\partial z_j} \frac{\partial z_j}{\partial w_{ij}} = \delta_j^{C5} a_i^{S4}$$

with w_{ij} as the weight connecting i-th neuron in S4 layer to the j-th neuron in C5 layer, δ_j^{C5} as the j-th neuron error term in C5 layer, and a_i^{S4} is the i-th neuron activation output in the previous S4 layer, z_j is the forward pass calculation from S4 layer to C5 layer $z_j = f\left(\sum_i w_{ij}x_i + b_j\right)$.
Remember that S4 layer has $16 \times 5 \times 5$ feature maps / 400 inputs that is fully connected to the 120 neurons in C5, the number of weights is $400 \times 120 = 48,000$ and the number of biases is 120.
Knowing that C5 layer acting as a convolutional layer with 5×5 kernels mapped over the entire S4 feature map. It contains 120 feature maps that collapse to a 1D vector of length 120.

For backpropagation when we arrive in layer C5, even if it is a convolutional layer, we do not use a full convolution operation between the error gradient δ^{F6} and the weight kernel in C5 to compute the input gradient (delta) for the C5 layer. Instead, we compute a standard matrix multiplication (vector-matrix product). Full convolutions are only used during backpropagation when we are passing gradients backward through a spatial convolutional layer (where the stride is 1) to restore the original dimensions of the input feature maps. Because the spatial dimensions of C5 and F6 are already reduced to 1×1, there is no spatial structure to convolve over, making full convolution unnecessary.

The delta error term (δ_j^{C5}) for neuron / node j in the C5 layer represents the derivative of the activation function with respect to the layer's inactivated weighted input (z_j^{C5}). It is calculated by taking the sum of the propagated delta terms from the subsequent layer (F6).

Since C5 is fully connected to the subsequent layer F6, the error is propagated backward using matrix multiplication:

$$\delta_j^{C5} = \left(\sum_k \delta_k^{F6} w_{kj}^{F6}\right) f'(z_j^{C5})$$

where:

- δ_j^{C5} is the delta error term for the j-th neuron in layer C5.
- k is the indices of the nodes in the next layer (F6) connected to the j-th neuron in C5 layer.
- δ_k^{F6} is the delta error term of the k-th neuron in layer F6.
- w_{kj}^{F6} is the weight connecting the j-th neuron in C5 to the k-th neuron in F6.
- $f'(z_j^{C5})$ is the derivative of the pre-activated j-th neuron in C5 layer. Depending on the activation function that is used in C5 layer, if the activation function is a sigmoid then $f'(z_j^{C5}) = a_j(1 - a_j)$, with a_j as the activated j-th neuron in C5.
  If the activation function is hyperbolic tangent then $f'(z_j^{C5}) = 1 - \tanh^2(z_j^{C5}) = 1 - a_j^2$, with a_j as the activated j-th neuron in C5.
  $a_j = f(z_j)$ is the actual output / activation of the j-th neuron in C5 layer.

Once you compute and obtain 120 δ_j^{C5}, it is directly used to calculate how much the weights and biases in layer C5 contributed to the overall network error, allowing gradient descent to adjust them. With valid convolution the 120 δ^{C5} will become 48,000

δ^{C5} that will be used for computing weights gradient.

Gradient for weights

The mathematical formula to calculate the gradient of the loss with respect to a weight kernel tensor element is a convolution between the forward-pass input (from S4 to C5) and the error sensitivity (δ) from the subsequent layer:

$$\frac{\partial L}{\partial w} = 2Dvalidconv(X_k^{S4}, \delta_j^{C5})$$

Where:

- X_k^{S4} is the 5×5 input feature map from layer S4, $k = 1, 2, \ldots, 16$.
- δ_j^{C5} is the 1×1 error gradient map / delta at layer C5 with $j = 1, 2, \ldots, 120$.
- $2Dvalidconv()$ represents the valid convolution operation, the result will be the required 5×5 weight gradient matrix for C5 ($N - K + 1 = 5 - 1 + 1 = 5$).

Gradient for biases

The bias gradient is very simple, it is the sum of all error gradient in the corresponding feature map, we have 120 feature maps in C5, and the size is 1×1 for each feature map, thus

$$\frac{\partial L}{\partial b_j} = \delta_j^{C5}$$

with $j = 1, 2, \ldots, 120$.

The computation for δ and weight gradient until layer C5 are still easy and quite similar with FNN. After this, the CNN technique begins, it gets more difficult to comprehend on how to compute δ and weight gradient for layer S4 to layer C1.

4. **S4 (32 trainable parameters)**
   In LeNet-5 architecture, S4 layer is a trainable average pooling layer. Its weights are updated using the standard gradient descent backpropagation rule, multiplied by a learning rate η.

   Because S4 has 16 feature maps, this gives exactly 32 trainable parameters. The 32 trainable parameters consists of 16 weights for each feature map, and 16 bias for each feature map in S4 layer.

   There are exactly 400 deltas (or gradients) in the flattened S4 layer of LeNet-5, we will compute this 400 deltas first. Remember that the S4 layer is a subsampling (pooling) layer that consists of 16 feature maps, each with a spatial dimension of 5×5, after flattened it becomes 400 neurons.

   In backpropagation, there is exactly one delta (error gradient) δ computed for every activation or neuron in the layer. Computing delta in CNN is special, especially in the S4 pooling (subsampling) layer that is connected from C3 layer, and will be connected to C5 layer (C3 and C5 are convolutional layer).

   To compute the delta (the error gradient, δ) for a layer during backpropagation in a neural network, you backpropagate the error from the layer above it. In the LeNet-5 architecture, S4 is an average subsampling (pooling) layer, the mathematical formulation

for computing its delta relies solely on upsampling the deltas of the following layer (C5) and applying the derivative of S4's activation function.

The exact mathematical formulation relies on whether you are analyzing LeNet-5 based on its original Yann LeCun publication specifications or adapting it for modern deep learning practices.

The δ (delta) error term represents the local gradient, or the derivative of the loss with respect to a neuron's pre-activation. For the subsampling/pooling layer (like S4 or S2), the delta is computed by backpropagating the errors from the downstream convolutional layer (C5) and upscaling them using a kronecker product (or upscale matrix).

Original LeNet-5 Specification
In the original leNet-5 architecture, the S4 average pooling operation is defined as a linear function with trainable coefficients, followed by a scaled hyperbolic tangent activation function.
The delta error term (δ_j^{S4}) for the j-th neuron in the S4 layer that is flattened is computed as

$$\delta_{flattened,j}^{S4} = \left[\sum_k \delta_k^{C5} w_{jk}^{S4 \to C5} \right] f'(z_j^{S4})$$

where:

- $\delta_{flattened,j}^{S4}$ is the delta error term of the j-th feature map in the S4 layer, $j = 0, 1, \ldots, 399$.
- δ_k^{C5} is the delta error term of the k-th neuron in the C5 layer, $k = 0, 1, 2, \ldots, 119$.
- w_{jk} is the weights connecting the j-th neuron in the pooling layer S4 to the k-th neuron in the next layer C5 (C5 is fully connected to S4, meaning each S4 output feeds into every C5 neuron).
- z_j^{S4} is the pre-activation input of the S4 layer.
- $f'(z_j^{S4})$ is the derivative of the activation function with respect to the pre-activated j-th neuron in S4 layer.

Computing δ^{S4} with SymIntegration
There are a lot of ways to compute the δ^{S4}, in SymIntegration we have created 2 arrays that represents the trainable weights for S4 layer:
s4_weights_kernel[16] (to represent the trainable weights for each feature map)
s4_bias[16] (to represent the trainable biases for each feature map)

Since S4 is connected to C5 layer, and in order to compute the delta (error gradient, δ) for the S4 layer we will need to propagate the gradients backward from the C5 layer. To connect S4 layer to C5 layer we create this array that represents the deltas of the flattened S4 layer and the C5 deltas:
c5_weights_kernel[400][120]
Flattening the $16 \times 5 \times 5$ S4 layer into a 400×1 vector does not change the 32 trainable parameters, the 32 parameters belong to the S4 downsampling operation, not the flattening process.

Before computing δ^{S4}, we must already have computed the deltas for the C5 layer (120 δ^{C5}). Since C5 is effectively fully connected to S4 (each C5 neuron takes input from all 400 neurons from flattened S4 layer), the error propagates backwards from the 120 C5 neurons.

To compute the 400 δ^{S4} we are using this formula:

$$\delta^{S4}_{k,x,y} = \left[\sum_{j=1}^{120} 2Dfullconv(\delta^{C5}_j, \text{rot180}(w^{S4\to C5}_{jk}))\right] \oplus f'(z^{S4}_{k,x,y})$$

Where:

- δ^{C5}_j is the delta error term for the j-th neuron in layer C5.
- $\text{rot180}(w^{S4\to C5}_{jk})$ is the rotated 5×5 filter/kernel connecting the k-th map in S4 layer to the j-th map of C5 layer, we have total of 48,000 trainable weights, which means in C5 there are 1,920 distinct weights kernels.
- $f'(z^{S4}_{k,x,y})$ is the derivative of the activation function in S4 layer with respect to the pre-activated neurons in S4 k-th map at spatial position (x, y), $k = 1, 2, \ldots, 16,\ x, y = 1, 2, \ldots, 5$.
- $2Dfullconv(\ldots)$ is the full convolution operation across all 120 feature maps of C5 layer for the padded δ^{C5}_j with the rotated weight kernel $\text{rot180}(w^{S4\to C5}_{jk})$, we didn't write it in the formula explicitly, but this is the equivalent:

  $$2Dfullconv(\delta^{C5}_j, \text{rot180}(w^{S4\to C5}_{jk})) = 2Dvalidconv(\text{pad}(\delta^{C5}_j, 4), \text{rot180}(w^{S4\to C5}_{jk}))$$

  pad(...,n) is the notation to pad each 1×1 C5 delta map with 4 rows and columns of zeros on all sides (making it 9×9). This step ensures the backward convolution outputs an array the same size as the S4 map.
- $\oplus$ represents the element-wise multiplication (Hadamard product).

When propagating error, computing δ^{L-1} from δ^L for layer S4 and S2 we do need to rotate the weight matrix, the rotated weight matrix is used to convolve with the error gradients to pass the error signal back to the previous layer. Propagating error requires a mathematical operation that reverses the spatial dimensions to match the error gradient's direction back to the input space (which is why a 180^0 rotation is needed).

About Rotated 180^0 **Matrix** rot180 is the notation for 180 degree 2D matrix rotation, in backpropagation the weights must be rotated 180^0 for proper convolution gradient alignment. To perform a 180^0 rotation on a 2D matrix you reverse the rows and reverse the columns of the matrix. For a given weight matrix W, the rotated matrix W′ satisfies $w'_{m,n} = w_{k-m,k-n},\ k = 5,\ m, n = 1, 2, 3, 4, 5$.

$$W = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$$

$$W' = \text{rot180}(W) = \begin{bmatrix} i & h & g \\ f & e & d \\ c & b & a \end{bmatrix}$$

After we have obtained 400 δ^{S4} from backpropagating 120 δ^{C5} we can then shrink this 400 error gradient into 16 δ^{S4} that represents each feature map in S4 pooling layer.

In the original pooling layer S4 we have 16 features map with size of 5×5 each, thus if it is flattened then the first 25 entries will represents the first feature map, till the last 25 entries that represents the last feature map. In the flattened vector of size 400 we will partition every 25 δ^{S4} to represents each feature map and it will be used to compute the weight updates in S4 layer.

Gradient for weights

$$\frac{\partial L}{\partial w_{ji}} = \sum_{pixels} \frac{\partial L}{\partial z^{S4}} \frac{\partial z^{S4}}{\partial u^{S4}} \frac{\partial u^{S4}}{\partial w_{ji}}$$

with z^{C3} as the pre-activated feature map values from the previous layer (C3) corresponding to the receptive field, δ^{S4} is the error term propagated backward from the C5 layer to the S4 layer, x and y are the spatial coordinates within the feature map. $\frac{\partial L}{\partial z^{S4}}$ is the gradient propagated from the C5 layer, $\frac{\partial z^{S4}}{\partial u^{S4}} = f'(u)$ is the derivative of the activation function with respect to the pre-activated u^{S4}, $\frac{\partial u^{S4}}{\partial w_{ji}} = \frac{1}{4} \sum_{l=1}^{4} x_l^{C3}$ is the sum of the 4 inputs in the 2×2 receptive field from C3 layer, multiplied by $\frac{1}{4}$ for average pooling.

Once the 400 delta δ^{S4} are computed for S4 layer, the gradients for the S4 layer's trainable weights are computed by taking the sum over the batch of the element-wise product between the delta and the C3 layer's output feature maps.

Thus the formula to compute the total gradient for the k-th feature map' weight become:

$$\frac{\partial L}{\partial w_k} = \sum_{i=1}^{5} \sum_{j=1}^{5} \delta_{k,i,j}^{S4} \left(\frac{1}{4} \sum_{m=1}^{2} \sum_{n=1}^{2} x_{k,2i-2+m,2j-2+n}^{C3} \right)$$

with $i, j = 1, 2, 3, 4, 5$ as the pixel that represents the spatial position for the 5×5 size of each feature map in S4 layer.

Once the gradients are computed, the weight update in the S4 layer using standard gradient descent formula is

$$\begin{aligned} w_k^{new} &= w_k^{old} - \eta \frac{\partial L}{\partial w_k} \\ &= w_k^{old} - \eta \frac{\partial L}{\partial a_{k,i,j}} \frac{\partial a_{k,i,j}}{\partial w_k} \\ &= w_k^{old} - \eta \sum_{i,j \in \text{map } k} \delta_{k,i,j}^{S4} \left(\frac{1}{4} \sum_{l=1}^{4} x_l^{C3} \right) \end{aligned}$$

with η as the learning rate and $\frac{\partial L}{\partial w_j}$ as the gradient of the loss function L (cross-entropy) with respect to that specific weight for the k-th feature map in S4 layer, $i, j = 1, 2, \ldots, 5,\ k = 1, 2, \ldots, 16$, a_i^{S4} is the output activation for the i-th neuron in the S4

layer $\frac{\partial a_{k,i,j}^{S4}}{\partial w_k} = f'(w_k \cdot \frac{1}{4}\sum_{l=1}^{4} x_l^{C3} + b_k) \cdot \frac{1}{4}\sum_{i=1}^{4} x_i^{C3}$, and $\frac{\partial a_{k,i,j}^{S4}}{\partial w_k} = \frac{1}{4}\sum_{l=1}^{4} x_l^{C3}$, x_l are the four activations in the 2×2 pooling window from the preceding C3 convolutional layer.

Thus

$$\frac{\partial L}{\partial w_k} = \sum_{i=1}^{5}\sum_{j=1}^{5} \delta_{k,i,j}^{S4} \cdot x_{k,i,j}^{S4} = \sum_{i=1}^{5}\sum_{j=1}^{5} \delta_{k,i,j}^{S4} \left(\frac{1}{4} \sum_{m=1}^{2}\sum_{n=1}^{2} x_{k,2i-2+m,2j-2+n}^{C3} \right)$$

where:

- k is the index of the feature map for S4 layer, $k = 1, 2, \ldots, 16$.
- (i, j) is the spatial position for each feature map in S4, $i, j = 1, 2, \ldots, 5$.
- $x_{k,i,j}^{S4}$ is the original forward-pass activations of the k-th feature map in S4 at spatial position (i, j), $a_{k,i,j}^{S4} = f\left(w_k * \left(\sum_{m=1}^{2}\sum_{n=1}^{2} x_{m,2i-2+m,2j-2+n}^{C3}\right) + b_k\right)$, $x_{k,i,j}^{S4} = \left(\sum_{m=1}^{2}\sum_{n=1}^{2} x_{m,2i-2+m,2j-2+n}^{C3}\right)$.

For weight gradients we do not need to rotate or flip any matrices, we simply compute the unrotated convolution of the input volume (from the previous layer) and the incoming error gradients δ (from the next layer). Weight updating merely measures how much each specific weight contributed to the loss. This is achieved by element-wise multiplication (dot product) of the incoming gradient matrix and the exact receptive field it interacted with during the forward pass. Because we are doing cross-correlation to compute the weight gradient, the weight kernel stays in the original orientation. Cross-correlation is a standard valid convolution without rotating any matrices / filter matrix.

Gradient for biases
Thus the formula to compute the total gradient for the k-th feature map' bias become:

$$\frac{\partial L}{\partial b_k} = \sum_{i=1}^{5}\sum_{j=1}^{5} \delta_k^{S4}$$

with $i, j = 1, 2, 3, 4, 5$ as the pixel that represents the spatial position for the 5×5 size of each feature map in S4 layer.

The bias update formula for the S4 layer is

$$b_k^{new} = b_k^{old} - \eta \left(\sum_{i=1}^{5}\sum_{j=1}^{5} \delta_k^{S4} \right)$$

with η as the learning rate and $k = 1, 2, \ldots, 16$. Since the bias is added uniformly as a scalar constant to the pre-activation sum before passing through the activation function, its direct partial derivative is the sum of the gradients flowing back from the downstream activation. Because the bias term b is additive, its rate of change relative to a_j is exactly 1, $\frac{\partial a_j^{S4}}{\partial b_j} = 1$.

Important variables and dimensions when computing delta in S4:

- **S4 Output:** 16 feature maps of size 5×5, flattened into a 400-element array.

- **C5 Layer:** 120 fully-connected neurons.
- w_{ji} is the weight connecting the j-th neuron in S4 to the i-th neuron in the C5 layer (dimension: 400×120).
- δ_i^{C5} is the error term of the i-th neuron in the C5 layer.
- w_j^{S4}, b_j^{S4} are the trainable weights and bias for the S4 average pooling layer (32 total parameters: 16 weights and 16 biases).
- a_j^{S4} is the activation (output) of the j-th node in layer S4.

5. **C3 (1,516 trainable parameters)**
In LeNet-5, the C3 layer's weight change formula relies on gradient descent and backpropagation. The weight update formula formula is

$$w^{new} = w^{old} - \eta \frac{\partial L}{\partial w}$$

with η as the learning rate, $\frac{\partial L}{\partial w}$ as the gradient of the error with respect to the specific weight.
Using the chain rule, the gradient for a specific weight in C3 is the sum over all pixels in the C3 feature map of the local error gradient (delta, δ^{C3}) multiplied by the corresponding input pixel from the previous layer (S2).
The C3 layer produces exactly 16 feature maps. During backpropagation, one delta (error gradient) matrix is generated for each feature map. Therefore, the C3 layer produces 16 delta matrices, each with dimensions of 10×10.

For a C3 feature map k and S2 feature map j:

$$\frac{\partial L}{\partial w_{kj}} = \sum_x \sum_y \delta_k(x,y) \cdot a_j(x+m, y+l)$$

where:

- $\delta_k(x,y)$ is the error gradient at output coordinate (x,y) of C3 feature map k.
- a_j is the activation / output of the S2 feature map j that is connected to C3 feature map k.
- m, l are the spatial offsets of the 5×5 kernel mask.

Weight Sharing in C3
Because C3 is a convolutional layer, it utilizes weight sharing. A single weight value is applied to multiple connections across the feature map. Therefore, the total gradient for a single weight is the sum of all individual gradients calculated for every spatial position where that weight is shared.
Unlike a standard modern convolutional layer, in LeNet-5 C3 is only partially connected. Out of $16 \times 6 = 96$ possible combinations between the 6 S2 feature maps and 16 C3 feature maps, only 60 connections are active (the rest are zeroed out). The gradient is only calculated and updated for the 1,516 active weights and 16 biases in this layer.

Computing the delta (δ) for the C3 layer in LeNet-5 involves calculating the error gradient $\frac{\partial L}{\partial z}$ with respect to the pre-activation value z. This is done via backpropagation, where error gradients are passed backward from the next layer (S4) to C3. Because C3 is a convolutional layer, this requires propagating the deltas backward from the subsequent S4 pooling layer, adjusted for C3's unique partially-connected feature map

scheme.

Passing δ^{S4} to C3 Layer
Because S4 layer performs average pooling, the delta from the m-th feature map of S4 (δ_m^{S4}) gets upsampled to match the C3 dimension before being propagated backwards. If S4 has a 2×2 pooling window, we distribute (broadcast) the error to the four corresponding units in the C3 feature map.

Subsampling for S4 Delta
Since S4 is an average pooling layer with trainable weights and biases. During backpropagation, its delta is mapped as follows:

- **Error Upsampling:** The errors flowing back from S4 (which has a 5×5 spatial dimension) must be mapped back to the 10×10 dimensions of C3. Since pooling condenses the image by taking the average (e.g., a 2×2 pool is unrolled into 4 identical values), the incoming delta is upsampled.

  Because S4 uses average pooling with a 2×2 window and stride of 2, forward pooling takes an average of 2×2 block. During backpropagation when we compute delta for C3, to reverse this, we must distribute the error term from a single S4 neuron equally across the 2×2 block (a technique often called Kronecker product or "upsampling" the error gradient).

  For a 2D feature map matrix x of shape $H \times W$ and a subsampling (pooling) factor t, the upsampled matrix upsample(x) is computed via the Kronecker product:

  $$\text{upsample}(x) \equiv \frac{1}{t * t} \cdot x \otimes \mathbf{1}_{t \times t}$$

  with $\mathbf{1}_{t \times t}$ as a $t \times t$ matrix filled with 1s.

  For example if we have

  $$\delta_k^{S4} = 2.3$$

  then the Kronecker product is

  $$2.3 \quad \xrightarrow{upsampling} \frac{1}{4} \begin{bmatrix} 2.3 & 2.3 \\ 2.3 & 2.3 \end{bmatrix}$$

  Applying the Kronecker product upsamples it back by mapping each individual pixel into a 2×2 block of identical values (given that the current poolng layer is an average pooling).
- **Scaling / Apply the Layer's Scale Weight**
  After upsampling the delta from S4 layer, multiply them with the corresponding weight from S4 (the 16 trainable weights for each feature map in S4) at that current feature map.
- **Gradient Application:** The upscaled delta will be 1,600, and it will be multiplied (element-wise / Hadamard product) by the derivative of the C3 layer's activation function.

Formula to Compute δ^{C3}
During backpropagation, the delta for a specific neuron or spatial position (i, j) in a

feature map k of the C3 layer is computed using the chain rule. It is the sum of the upsampled upstream deltas from all connected feature maps in the next layer (S4) multiplied by the derivative of the activation function:

$$\delta^{C3}_{k,i,j} = w^{S4}_k \cdot f'(z^{C3}_{k,i,j}) \oplus \text{upsampled}(\delta^{S4}_{k,i,j})$$

where:

- $\delta^{C3}_{k,i,j}$ is the error gradient for the k-th feature map in layer C3 in feature map k with spatial position i, j.
- $f'(z^{C3}_{k,i,j})$ is the derivative of the activation function with respect to the pre-activation z^{C3}_k at layer C3 in feature map k with spatial position i, j.
- $\delta^{S4}_{k,i,j}$ is the upsampled error gradient of the k-th S4 feature map with spatial position i, j.
- w^{S4}_k is the trainable weight from S4 layer, $k = 1, 2 \ldots, 16$.
- $\oplus$ is the Hadamard (element-wise) product.

To get the final delta for layer C3 and C1, that has pooling layer afterwards, we only need to multiply the upsampled δ^{S4} (for C3) or δ^{S2} (for C1), then multiply it by the trainable weight w^{S4}_k (for C3) or w^{S2}_k (for C1). Because a single S4 value is derived from four C3 values, distributing the gradient backward requires scaling it by the exact same shared weight used during the forward pass.

We do not add the S4 bias during the delta computation in backpropagation. In neural networks, biases are strictly used during the forward pass to shift activations. During backpropagation, the derivative of a constant bias with respect to the loss is zero, meaning it does not contribute to the error gradient flowing backward to the previous layer.

The Special Case of C3 Sparse Connections

The C3 layer in LeNet-5 is not fully connected to the previous layer, S2. Instead each of the 16 feature maps in C3 selectively receives inputs from a subset of S2's 6 feature maps (sparse connections). So when backpropagating, the error for an S2 feature map is the sum of the gradients flowing back through all filters in C3 that were connected to it (the gradients from upstream layers are strictly routed backward only through these pre-defined mapped connections).

When updating $w^{S2 \to C3}_{ij}$ (weights for the convolution kernel from S2 to C3), you only calculate and update the weights for the valid, allowed connections as defined by the classic LeNet-5 adjacency table (Table 10.4), setting non-existent connections to zero.

Gradient for weights

Once you have the delta δ^{C3}_j for every node, you can calculate the gradients to update the C3 layer's trainable parameters (the weights):

$$\frac{\partial L}{\partial w_{ij}} = \sum_{x,y} \delta^{C3}_j * x^{S2}_i$$

To calculate the weight gradients for the C3 convolutional layer in LeNet-5, you use the standard backpropagation chain rule. The gradient for any specific weight is the

sum of the partial derivatives of the loss with respect to that weight across all spatial positions where the weight was applied (due to weight sharing).

To perform this calculation, you need the following standard variables from both the forward and backward passes:

- m, n as the spatial dimensions of the C3 feature maps (10×10).
- δ_k^{C3} as the error gradient matrix (delta) for the k-th feature map in layer C3.
- X_j^{S2} as the input feature map from the previous layer (S2) into the C3 convolutional filter.
- W_{kj} as the weight tensor connecting the j-th feature map in S2 to the k-th feature map in C3(5×5).
- L as the loss or cost function.

Because C3 layer in LeNet-5 is a convolutional layer, the same set of weights is slid across the entire image. Therefore, the total gradient for a specific weight w is the sum of its gradients over all coordinates (x, y) in the feature map:

$$\frac{\partial L}{\partial W_{kj}} = \sum_{x=1}^{m} \sum_{y=1}^{n} \frac{\partial L}{\partial z_{k,x,y}^{C3}} \cdot \frac{\partial z_{k,x,y}^{C3}}{\partial W_{kj}}$$

where:

- $\frac{\partial L}{\partial z_{k,x,y}^{C3}}$ is the delta $\delta_{k,x,y}^{C3}$ for the specific neuron at map k and position (x, y).
- $\frac{\partial z_{k,x,y}^{C3}}{\partial W_{kj}}$ is the local receptive field of the previous layer's activations at that position.

In practice, you do not calculate the gradient for each pixel individually. Instead, you compute it as a full convolution between the input activations and the error deltas:

(a) **Calculate the 1600 deltas for C3**
First, backpropagate the errors from the next layer (S4) into C3. This gives you the error matrix δ^{C3}, there will be $16 \times 10 \times 10$ matrix of deltas.
To backpropagate the error delta from an average pooling layer S4 to a convolutional layer C3 in LeNet-5, you must reverse the forward-pass operations by: **upsample the S4 deltas** and pass them through the derivative of the activation function used in C3.

i. **Upscale the S4 Deltas**
Layer S4 has 16 feature maps of size 5×5. The pooling operation in S4 is an average pooling, which downsampled the C3 layer (S4 pools a 2×2 neighborhood into a single pixel). During the backward pass, each individual delta in the 5×5 S4 maps is "unpooled" (upsampled) into a 2×2 region. This spreads the error back out/ distribute the error uniformly backward to match the original C3 shape, expanding the resolution to 10×10. Each delta value in S4 is scaled by $\frac{1}{M+N}$ and broadcast to a 2×2 block in the corresponding C3 feature map.

The formula is:

$$\text{upsample}(\delta_i^{S4}) = \delta_i^{S4} \times \frac{1}{4}$$

Because S4 layer is an average pooling layer, its deltas are scaled by $\frac{1}{4}$ as they are mapped backwards, the scale formula is $\frac{1}{M+N}$ for 2×2 pooling $M = 2, N = 2$.

The result will be $16 \times 10 \times 10$ matrices filled with upsampled δ^{S4}.

ii. **Multiply by S4 Trainable Weight**
Multiply the upsampled error matrix by the S4 layer's specific weight w_k^{S4}.

iii. **Apply the Activation Function Derivative**
In LeNet-5, the activation function for C3 is the hyperbolic tangent (tanh). The gradient (derivative) must be applied to the upscaled deltas for each neuron.

$$\tanh'(x) = 1 - \tanh^2(x)$$

The backpropagation computation is to multiply the upscaled delta maps element-wise by the derivative of the C3 layer output evaluated during the forward pass (pre-activated output).

$$\delta_{k,x,y}^{C3} = w_k^{S4} \cdot f'(z_{k,x,y}^{C3}) \oplus \text{upsampled}(\delta_{k,x,y}^{S4})$$

the 1,6000 deltas in C3 can be represented with the above formula, at feature map k, spatial position (x, y), notice that the **upsampled** is different than **upsample** notation, the **upsampled** notation means that the 400 $delta^{S4}$ is already upsampled from $16 \times 5 \times 5$ into $16 \times 10 \times 10$ size that match C3 layer / the 400 deltas of S4 are map directly onto the 16 corresponding feature maps of C3.

After upsampling δ^{S4} from 16 feature maps of 5×5 dimensions to 10×10 dimensions, scale / multiply with the trainable weight in S4 layer (16 trainable weights) and multiply the upscaled delta maps with derivative of the pre-activated output in C3 layer we will obtain 1,600 backpropagated deltas for C3 (δ^{C3}) / error delta matrix for C3 of size $16 \times 10 \times 10$. Upsample is also called Kronecker product operation.

(b) **2D Convolution**
Perform a valid convolution operation on the input activations X_j^{S2} with the C3 layer's unrotated deltas δ_k^{C3} with $k = 1, 2, \ldots, 16$, a matrix delta for each feature map in C3.

The convolution of the unrotated 180 degree delta map with the corresponding 14×14 input feature map from S2 is used to compute the spatial gradient correctly to obtain the weight gradient for the 1,516 trainable parameters. The resulting 5×5 matrix of the convolution operation will represent the gradient $\frac{\partial L}{\partial w_{kj}}$ for that specific kernel. In S4 layer we perform a **rot180** towards the weight kernel $W^{S4 \to C5}$, thus in this C3 layer when computing δ^{C3} we do not need to rotate the matrix delta, instead we use upsample operation to obtain δ^{C3}.

Alternatively, you can calculate this efficiently as a cross-correlation: Keep the delta map unflipped, but slide the 14×14 S2 input matrix over the 10×10 delta map, computing the sum of element-wise products.

(c) **Compute the Weight Gradient**
Now, we can compute the 1,516 weight gradients by convolution operation formula:

$$\frac{\partial L}{\partial W_k} = 2Dvalidconv(X_j^{S2}, \delta_k^{C3})$$

$2Dvalidconv$ is a valid convolution operation which don't add artificial zeros for the matrices, no padding. The kernel that will slide over the matrix X_j^{S2} is the δ_k^{C3} matrix. The resulting matrix will be of size $N - k + 1 = 14 - 10 + 1 = 5$ thus we will obtain sixteen 5×5 weight gradients matrix.

The weight update in the C3 layer using standard gradient descent formula is

$$w_{kj}^{new} = w_{kj}^{old} - \eta \left(2Dvalidconv(X_j^{S2}, \delta_k^{C3}) \right)$$

Gradient for biases
The bias gradient for a specific feature map is simply the sum of all deltas in that feature map:

$$\frac{\partial L}{\partial b_k} = \sum_{x=1}^{10} \sum_{y=1}^{10} \delta_{k,x,y}^{C3}$$

x, y is the spatial position in the k-th feature map.

The bias update in the C3 layer using standard gradient descent formula is

$$b_k^{new} = b_k^{old} - \eta \left(\sum_{x=1}^{10} \sum_{y=1}^{10} \delta_{k,x,y}^{C3} \right)$$

6. **S2 (12 trainable parameters)**
Layer S2 has 6 feature maps, and each feature map has exactly 1 trainable weight (coefficient) for the sum of the 4 inputs (2×2 receptive field) and 1 trainable bias (offset). An average pooling layer does not use complex, multi-dimensional trainable filter weights. Layer S2 and S4 are average pooling layers in the original LeNet-5. Many modern variations replace this with max pooling, which generally yields sharper edge detection features and speeds up training convergence. Average pooling layer on the other side assigns equal weight to all features in the window, avoiding the loss of background or texture information. Average pooling was chosen due to its feature smoothing, it acts as a low-pass filter, creating smoother feature maps that help the network recognize general shapes without over-indexing on localized sharp edges.
While average pooling is foundational to the original design, most modern CNNs prefer max pooling.
   - Max pooling selects the maximum activation within the window, effectively isolating the strongest features(e.g., the most distinct stroke of a handwritten digit).
   - Because max pooling selects the most prominent feature, it produces larger gradients during back-propagation, which can help the network train faster and achieve better classification accuracy.

   For the S2 average pooling layer, we compute the delta by propagating the gradients backwards from the upstream layer. The mathematical operations required to compute and propagate the delta depend on the type of pooling (max / average / sum only pooling), the sum only pooling adds the four inputs in a 2×2 window, multiplies them by a trainable weight, adds a trainable bias, and passes the result through tanh activation function, the average pooling will take the average of the four inputs in a 2×2 window, and the max pooling will take the largest value as the output of the four inputs in a 2×2 window.

Delta Calculation

From S2 to C3 layer, the forward pass performs a 2D convolution using a set of weights / kernels. During the backward pass, to calculate the error gradients for the previous layer, you must perform a cross-correlation. The mathematical equivalent of doing this is running a standard convolution using a flipped (180-degree rotated) version of the weight matrix. Rotating the weights allows the network to smoothly propagate the derivative of the error backward through the exact spatial reverse of the forward pass.

S2 layer has 1,176 neurons from its $6 \times 14 \times 14$ feature map, thus we will compute 1,176 delta first. To find the delta for a specific S2 map, sum the full 2D convolution of the C3 deltas with the flipped C3 weights kernel. Do this only for the C3 maps that are connected to that S2 map.

This is just a food for thought: if you take a look at the S2 to C3 connectivity map, excluding the feature map 13-15 in C3 layer that connect to 4 discontiguous feature maps in S2 layer, you will realize then that each feature map in S2 from $k = 1, 2 \ldots, 6$ will appear 8 times each in the first 12 feature maps and the last feature map of C3 layer.

The formula for the delta of the j-th feature map in S2 layer is

$$\delta^{S2}_{k,x,y} = f'(z^{S2}_{k,x,y}) \oplus \left(\sum_{k \in \text{Connected}(j)} 2Dvalidconv(\text{pad}(\delta^{C3}_{j}, 4), \text{rot180}(W^{S2 \to C3}_{jk})) \right)$$

where:

- $\delta^{C3}_{k,x,y}$ is the delta map (size 10×10) of the k-th feature map in C3.
- W_{jk} is the 5×5 weight kernel connecting the k-th map in S2 to the j-th map in C3 layer.
- **rot180(...)** is the notation to rotate the 5×5 weight matrix 180 degree to align it for the backward pass.
- **2Dvalidconv** is the notation to perform 2D valid convolution between two matrices, the padded δ^{C3} and the flipped W_{ij}, the flipped / rotated weight will be the kernel matrix that will slide over the delta matrix. If we do not want to pad the delta manually first we can use the **2Dfullconv(...)** it will pad the delta matrix automatically while running the operation.
- **pad(...,n)** is the notation to pad each 10×10 C3 delta map with 4 rows and columns of zeros on all sides (making it 18×18). This step ensures the backward convolution outputs an array the same size as the S2 map.
- $\sum_{k \in \text{Connected}(j)}$ / sum maps means for each S2 map, sum the results of all convolutions coming from its connected C3 maps. This gives you exactly 6 S2 delta feature maps, ready to be passed to the S2 to C1 backward pass.
- $\oplus$ is the element-wise (Hadamard) multiplication.
- $f'(z^{S2}_{k,x,y})$ is the derivative of the activation function with respect to the pre-activated neuron in k-th feature map of S2 layer at spatial position (x, y), $k = 1, 2, \ldots, 6,\ x, y = 1, 2, \ldots, 14$.

Gradient for weights
During training, the change in the weight $\triangle W$ is proportional to the negative gradient of the loss function (L) with respect to W, scaled by the learning rate (η).

$$\triangle W = -\eta \frac{\partial L}{\partial W}$$

Using the chain rule, $\frac{\partial L}{\partial W}$ expands to:

$$\frac{\partial L}{\partial W} = \sum \delta^{S2} \frac{\partial a^{S2}}{\partial W}$$

where:

- δ^{S2} is the error term propagated from the next layer.
- $\frac{\partial a^{S2}}{\partial W}$ is the derivative of the S2 activation.

$$\frac{\partial a^{S2}}{\partial W} = f'(W \cdot \text{avg}(a^{C1}) + b) \cdot \text{avg}(a^{C1})$$

Because the S2 layer uses average pooling, the gradient of the loss function L with respect to a specific weight w_k (for k-th feature map, $k = 1, 2, \ldots, 6$) requires you to reverse the pooling operation

(a) **Sum of the Gradients:** For each of the 6 feature maps, sum all the $14 \times 14 = 196$ corresponding delta values (δ_k^{S2}) in the S2 layer.

(b) **Weight Gradient** $\left(\frac{\partial L}{\partial w_k}\right)$**:** Multiply this sum by the average of the 2×2 pixel region from the previous layer (C1).

Mathematically, for a single feature map, the weight gradient is:

$$\frac{\partial L}{\partial w_k^{S2}} = \sum_{i=1}^{14} \sum_{j=1}^{14} \delta_{k,i,j}^{S2} \left(\frac{1}{4} \sum_{m=1}^{2} \sum_{n=1}^{2} x_{k,2i-2+m,2j-2+n}^{C1} \right)$$

with $k = 1, 2, \ldots, 6$.

To apply the weight for the next iteration, we apply standard gradient descent. The weight update in the S2 layer using standard gradient descent formula is:

$$\begin{aligned} W^{new} &= W^{old} - \eta \frac{\partial L}{\partial W} \\ &= W^{old} - \eta \left(\sum_{i=1}^{14} \sum_{j=1}^{14} \delta_{k,i,j}^{S2} \left(\frac{1}{4} \sum_{m=1}^{2} \sum_{n=1}^{2} x_{k,2i-2+m,2j-2+n}^{C1} \right) \right) \end{aligned}$$

Gradient for biases
The gradient for the bias is simply the sum of all deltas in that specific feature map:

$$\frac{\partial L}{\partial b_k^{S2}} = \sum_{i=1}^{14} \sum_{j=1}^{14} \delta_{k,i,j}^{S2}$$

The bias update in the S2 layer using standard gradient descent formula is

$$b_k^{new} = b_k^{old} - \eta \left(\sum_{i=1}^{14} \sum_{j=1}^{14} \delta_{k,i,j}^{S2} \right)$$

7. **C1 (156 trainable parameters)**
   To backpropagate the error delta from an average pooling layer (S2) to a convolutional layer (C1) in LeNet-5, you must reverse the forward-pass operations: upsample the S2 deltas and pass them through the derivative of the activation function used in C3.
   (a) **Upsample the S2 Deltas**
   Layer S2 has 6 feature maps of size 14×14. The pooling operation in S2 is an average pooling, which downsampled the C1 layer. During the backward pass, each individual delta in the 14×14 maps is "unpooled" (upsampled) into a 2×2 region. This spreads the error back out, expanding the resolution to 28×28.

$$\text{upsample}(\delta_i^{S2}) = \frac{1}{4}\delta_i^{S2}$$

   the resulting array will be six 28×28 matrices.
   (b) **Scale / Multiply by S2 Trainable Weight**
   Multiply the upsampled error matrix by the S2 layer's specific weight w_k^{S2}.
   (c) **Apply the Activation Function Derivative**
   In LeNet-5, the activation function for C1 is the hyperbolic tangent (tanh). The gradient (derivative) must be applied to the upscaled deltas for each neuron.

$$\tanh'(x) = 1 - \tanh^2(x)$$

   Backpropagation: Multiply the upscaled delta maps element-wise by the derivative of the C1 layer output evaluated during the forward pass.
   (d) **Result: 4,704 Deltas**
   This upsampling of 6 feature maps of 14×14 dimensions to 28×28 dimensions perfectly accounts for the $4,704$ values across C1 (6 maps $\times$28 rows $\times$28 columns). Thus the formula to compute the 4,704 δ^{C1} is

$$\delta_{k,i,j}^{C1} = (\text{upsample}(\delta_{k,x,y}^{S2})) \cdot w_k^{S2} \cdot f'(z_{k,i,j}^{C1})$$

Gradient for weights
The weight update in the C1 layer using standard gradient descent formula is

$$W^{new} = W^{old} - \eta \frac{\partial L}{\partial W}$$

Generally, the delta (error term) for any convolutional neuron i is calculated by multiplying the upstream error by the derivative of its activation function:

$$\delta_i = \frac{\partial L}{\partial u_i} = \frac{\partial L}{\partial a_i}\frac{\partial a_i}{\partial u_i}$$

where:

- $\frac{\partial L}{\partial a_i}$ is the summed gradient flowing backwards from all upstreams neurons connected to neuron i.
- $\frac{\partial a_i}{\partial u_i} = f'(u_i)$ is the derivative of the activation function with respect to the pre-activation u_i

The weight gradient for C1 layer is computed by performing a valid 2D convolution between the input image pixels (at C1) and the delta map (δ^{C1}).

$$\frac{\partial L}{\partial w_{ij}^{C1}} = 2Dvalidconv(X^{input}, \delta_k^{C1})$$

$2Dvalidconv(X^{input}, \delta_k^{C1})$ will produce a 6 matrices with size $N - K + 1 = 32 - 28 + 1 = 5$ / six 5×5 weight gradient matrix.

Thus the weight update formula for C1 layer is

$$w_{k,i,j}^{new} = w_{k,i,j}^{old} - \eta \left(2Dvalidconv(X^{input}, \delta_k^{C1}) \right)$$

Gradient for biases
Once you have the delta (δ^{C1}), you use it to find the gradients for C1's biases, the gradient of the loss with respect to a bias for a specific feature map in C1 is the sum of all deltas in that corresponding feature map.

$$\frac{\partial L}{\partial b_k^{C1}} = \sum_{i=1}^{28} \sum_{j=1}^{28} \delta_{k,i,j}^{C1}$$

with $k = 1, 2, \ldots, 6$.

The bias update in the C1 layer using standard gradient descent formula is

$$b_k^{new} = b_k^{old} - \eta \left(\frac{\partial L}{\partial b_k^{C1}} \right) = b_k^{old} - \eta \left(\sum_{i=1}^{28} \sum_{j=1}^{28} \delta_{k,i,j}^{C1} \right)$$

We have summarized forward pass, delta backpropagation computation, the weight gradient, and bias gradient formula in tables (10.7), (10.8), (10.9), and (10.10).

LeNet-5 has seven layer in total, does not comprise an input layer, each containing a trainable parameters; each layer has a plurality of the feature map, a characteristic of each of the input feature map extracted by means of convolution filter, and then in each there are multiple neurons.

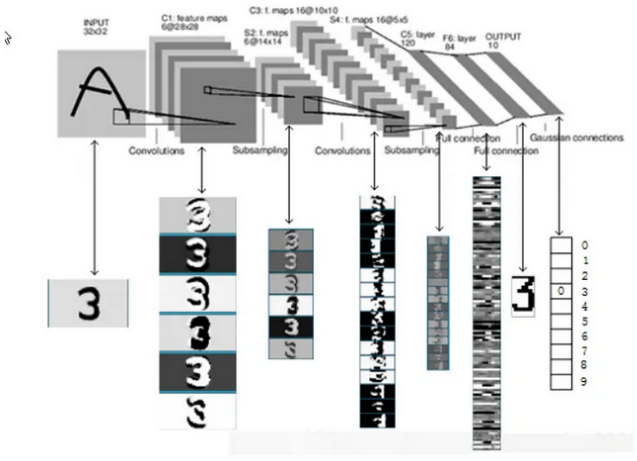

Figure 10.44: *The LeNet-5 architecture & pipeline for digit recognition. It is a very efficient convolutional neural network for handwritten character recognition.*

Layer	Feature Maps	Size	Kernel	Stride	Activation
Input	1	32×32	-	-	-
Conv1 (C1)	6	28×28	5×5	1	Leaky ReLU ($\alpha = 0.8$)
Avg Pool (S2)	6	14×14	2×2	2	Leaky ReLU ($\alpha = 0.8$)
Conv2 (C3)	16	10×10	5×5	1	Leaky ReLU ($\alpha = 0.8$)
Avg Pool (S4)	16	5×5	2×2	2	Leaky ReLU ($\alpha = 0.8$)
Conv3 (C5)	120	1×1	5×5	1	Leaky ReLU ($\alpha = 0.8$)
FC (F6)	84	-	-	-	Leaky ReLU ($\alpha = 0.8$)
Output	10	-	-	-	softmax

Table 10.6: *Summary of LeNet-5 architecture.*

LeNet-5's significance in deep learning cannot be overstated. It was one of the first demonstrations that convolutional neural networks could be successfully applied to visual pattern recognition. LeNet-5 introduced several key concepts that are now standard in CNN architectures, including the use of multiple convolutional and pooling layers, local receptive fields, shared weights, and the backpropagation algorithm for training the network. The development of LeNet was also driven by the need for automation in industries that dealt with large volumes of handwritten documents, such as banks and postal services, where automatic digit recognition could significantly speed up processing times and reduce errors. These innovations have paved the way for the development of more complex and deeper networks, which are the backbone of modern artificial intelligence systems in various applications ranging from autonomous vehicles to medical diagnosis. This architecture leads to the development of more sophisticated deep learning frameworks that continue to push the boundaries of what machines can learn and achieve.

The concept of using localized receptive fields, shared weights, and spatial hierarchies in neural networks was inspired by studies of the visual cortex in animals, suggesting that these biological processes could be mimicked to improve machine perception.

Layer	δ	Trainable Parameters	Forward Pass Formula
Conv1 (C1)	4,704	156 (150w + 6b)	$a_k^{C1} = f\left(conv2D(W_k^{C1}, X^{input}) + b_k^{C1}\right);\ k = 1, 2, \ldots, 6$
Avg Pool (S2)	1,176	12 (6w + 6b)	$a_{i,j,k}^{S2} = f\left(w_k^{S2} \cdot \frac{1}{4} \sum_{m=1}^{2} \sum_{n=1}^{2} a_{2i-2+m,2j-2+n}^{C1} + b_k^{S2}\right);\ i, j = 1, 2, \ldots, 28;\ k = 1, 2, \ldots 6$
Conv2 (C3)	1,600	1,516 (1,500w + 16b)	$a_k^{C3} = f\left(\sum_{i \in \text{connection}} conv2D(X_i^{S2}, W_i^{S2 \to C3}) + b_k^{C3}\right);\ k = 1, 2, \ldots 16$
Avg Pool (S4)	400	32 (16w + 16b)	$a_{i,j,k}^{S4} = f\left(w_k^{S4} \cdot \frac{1}{4} \sum_{m=1}^{2} \sum_{n=1}^{2} x_{2i-2+m,2j-2+n}^{C3} + b_k^{S4}\right);\ i, j = 1, 2, \ldots, 10;\ k = 1, 2, \ldots 16$
Conv3 (C5)	120	48,120 (48,000w + 120b)	$a_k^{C5} = f\left(\sum_{j=1}^{16} conv2D(W_{j,k}^{C5}, X_j^{S4}) + b_k^{C5}\right);\ k = 1, 2, \ldots, 120,\ j = 1, 2, \ldots, 16$
FC (F6)	84	10,164 (10,080w + 84b)	$a_k^{F6} = f\left(\sum_{j=1}^{120} (W_{j,k}^{F6}, X_j^{C5}) + b_k^{F6}\right);\ k = 1, 2, \ldots, 84,\ j = 1, 2, \ldots, 120$
Output	10	850 (840w + 10b)	$\hat{y}_i = \frac{e^{z_i}}{\sum_{k=0}^{9} e^{z_k}};\ z_i = \sum_{j=1}^{84} x_j w_{ij} + b_i$

Table 10.7: *Summary of LeNet-5 architecture forward pass formula.*

Layer	δ	δ Backpropagation Formula
Output	10	$\delta_j^{output} = \hat{y}_j - y_j; j = 1, 2, \ldots, 10$
FC (F6)	84	$\delta_j^{F6} = f'(z_j^{F6}) \left(\sum_{k=1}^{10} \delta_k^{output} w_{kj}^{F6 \to output} \right); j = 1, 2, \ldots, 84$
Conv3 (C5)	120	$\delta_j^{C5} = f'(z_j^{C5}) \left(\sum_{k=1}^{84} \delta_k^{F6} w_{kj}^{C5 \to F6} \right); j = 1, 2, \ldots, 120$
Avg Pool (S4)	400	$\delta_{k,x,y}^{S4} = \left[\sum_{j=1}^{120} 2Dfullconv(\delta_j^{C5}, \text{rot180}(w_{jk}^{S4 \to C5})) \right] \oplus f'(z_{k,x,y}^{S4}); \; k = 1, 2, \ldots, 16, \; x, y = 1, 2, \ldots, 5$
Conv2 (C3)	1,600	$\delta_{k,i,j}^{C3} = w_k^{S4} \cdot f'(z_{k,i,j}^{C3}) \oplus \text{upsample}(\delta_{k,x,y}^{S4}); \; k = 1, 2, \ldots 16; \; x, y = 1, 2, \ldots, 5; \; i, j = 1, 2, \ldots, 10$
Avg Pool (S2)	1,176	$\delta_{k,x,y}^{S2} = f'(z_{k,x,y}^{S2}) \oplus \left(\sum_{k \in \text{Connected}(j)} 2Dfullconv(\delta_j^{C3}, \text{rot180}(W_{jk}^{S2 \to C3})) \right); \; x, y = 1, 2, \ldots, 14; \; k = 1, 2, \ldots 6$
Conv1 (C1)	4,704	$\delta_{k,i,j}^{C1} = w_k^{S2} \cdot f'(z_{k,i,j}^{C1}) \oplus \text{upsample}(\delta_{k,x,y}^{S2}); \; k = 1, 2, \ldots 6; \; x, y = 1, 2, \ldots, 14; \; i, j = 1, 2, \ldots, 28$

Table 10.8: *Summary of LeNet-5 architecture backpropagation formula, z_j^L means pre-activated neuron in layer L.*

Layer	Trainable Parameters	Weight Gradient Formula
Output	840w + 10b	$\frac{\partial L}{\partial w^{output}} = \delta_j^{output} * f(z_i^{F6}); \; i = 1, 2, \ldots 84$
FC (F6)	10,080w + 84b	$\frac{\partial L}{\partial w^{F6}} = \delta_j^{F6} * f(z_i^{C5}); \; i = 1, 2, \ldots 120$
Conv3 (C5)	48,000w + 120b	$\frac{\partial L}{\partial w^{C5}} = 2Dvalidconv(X_k^{S4}, \delta_j^{C5}); \; j = 1, 2, \ldots 120 \;\; k = 1, 2, \ldots, 16$
Avg Pool (S4)	16w + 16b	$\frac{\partial L}{\partial w_k^{S4}} = \sum_{i=1}^{5} \sum_{j=1}^{5} \delta_{k,i,j}^{S4} \left(\frac{1}{4} \sum_{m=1}^{2} \sum_{n=1}^{2} x_{k,2i-2+m,2j-2+n}^{C3} \right); \; k = 1, 2, \ldots, 16$
Conv2 (C3)	1,500w + 16b	$\frac{\partial L}{\partial W_{jk}^{C3}} = 2Dvalidconv(X_j^{S2}, \delta_k^{C3}); \; j = 1, 2, \ldots, 6, \; k = 1, 2, \ldots, 16$
Avg Pool (S2)	6w + 6b	$\frac{\partial L}{\partial w_k^{S2}} = \sum_{i=1}^{14} \sum_{j=1}^{14} \delta_{k,i,j}^{S2} \left(\frac{1}{4} \sum_{m=1}^{2} \sum_{n=1}^{2} x_{k,2i-2+m,2j-2+n}^{C1} \right); \; k = 1, 2, \ldots, 6$
Conv1 (C1)	150w + 6b	$\frac{\partial L}{\partial w_{ij}^{C1}} = 2Dvalidconv(X^{input}, \delta_k^{C1}); k = 1, 2, \ldots, 6, \; i, j = 1, 2, \ldots 28$

Table 10.9: *Summary of LeNet-5 architecture weight update formula.*

Layer	Trainable Parameters	Bias Gradient
Output	840w + 10b	$\frac{\partial L}{\partial b^{output}} = \delta_j^{output}; j = 1, 2, \ldots 10$
FC (F6)	10,080w + 84b	$\frac{\partial L}{\partial b^{F6}} = \delta_j^{F6}; j = 1, 2, \ldots 84$
Conv3 (C5)	48,000w + 120b	$\frac{\partial L}{\partial b^{C5}} = \delta_j^{C5}; j = 1, 2, \ldots 120$
Avg Pool (S4)	16w + 16b	$\frac{\partial L}{\partial b_k^{S4}} = \sum_{i=1}^{5} \sum_{j=1}^{5} \delta_{k,i,j}^{S4}$
Conv2 (C3)	1,500w + 16b	$\frac{\partial L}{\partial b^{C3}} = \sum_{x=1}^{10} \sum_{y=1}^{10} \delta_{k,x,y}^{C3}; k = 1, 2, \ldots, 16$
Avg Pool (S2)	6w + 6b	$\frac{\partial L}{\partial b_k^{S2}} = \sum_{i=1}^{14} \sum_{j=1}^{14} \delta_{k,i,j}^{S2}; k = 1, 2, \ldots, 6$
Conv1 (C1)	150w + 6b	$\frac{\partial L}{\partial b_k^{C1}} = \sum_{i=1}^{28} \sum_{j=1}^{28} \delta_{k,i,j}^{C1}; k = 1, 2, \ldots, 6$

Table 10.10: *Summary of LeNet-5 architecture bias update formula.*

[SI*] We have learned a nice C++ code for digit recognition using SFML from here:
https://github.com/AdamYuan/SimpleNN
The input vector is flattened to become a vector of 784 elements.

The good thing is this code has a feature to make user interface handwriting with SFML library, so the neural network that is being used will be trained and create around 300,000 parameters, that is way too many compared to LeNet-5 architecture, afterwards with the trained parameters / weights and biases it will be the model to predict our handwritten digits from 0 to 9.

The downside of that code is the documentation / the book that cover the formula being used is not complete or established nicely, thus instead of spending time to figure out the formula behind that code, we create our own code based on LeNet-5 architecture, that we will explain below.

[SI*] We have created a CNN C++ code for digit recognition, modifying from the one that we have used in FNN, this code can predict the classification with softmax output and train the training images with steepest descent by using **CNN_LeNet5** class.

When we finished with the code, we learned that the loss keeps increasing, this is due to the learning rate that we set at first is too high at $\eta = 0.5$, for CNN with LeNet-5 architecture $\eta = 0.01$ is the highest / the cap for starter, but since we are using our own digit handwriting with 60 training images thus we set $\eta = 0.1$ then the loss is decreasing, if any readers facing the same problem, these are the cause that makes the loss increases when backpropagating / training in CNN:

1. **The Learning Rate is Too High**
   If the learning rate is set too high, the gradients calculated during backpropagation will adjust the network's weights by massive amounts. This causes the model to step completely over the lowest point (optimum) of the loss function, making the loss get progressively worse with each training step. To fix this try lowering the learning rate by an order of magnitude (e.g., reduce from 0.1 to 0.01 or 0.0001) or implement a learning

scheduler to dynamically decrease the rate during training.

2. **Exploding Gradients**
   During the backward pass in deep CNNs, error gradients can multiply exponentially as they travel backward through the layers. This causes weight updates to become overwhelmingly large, rendering the network unstable. To fix this try to implement gradient clipping, a technique that caps the maximum value of gradients during backpropagation.

3. **Vanishing Gradients**
   Conversely, the gradients can become infinitesimally small as they propagate backward if we are using tanh or sigmoid activation function. This halts the network's ability to update its weights in the earlier, feature-extracting convolutional layers, while later layers may overfit. To fix this try to use ReLU (or its variants) in the hidden layers to mitigate this issue. If the network is very deep, consider incorporating residual connections (like those used in ResNet).

4. **Incorrect Loss Function Implementation**
   The loss function quantifies how wrong the CNN predictions are. If the custom loss function or the framework's loss criterion is misaligned with the final activation function, the backpropagated gradients will push the network in the wrong direction. To fix this ensure that the final layer's activation matches the loss function. For example if the output' activation is softmax then the loss function is categorical cross-entropy.

5. **Incorrect Data or Label Mismatch**
   If the gradients and loss calculations are mathematically sound, the issue may be in the data pipeline. If the training labels are noisy, misaligned with the images, or the input data pipeline is unnormalized, the model cannot learn meaningful patterns. To fix this double-check the data augmentation logic, ensure batches are properly shuffled, and verify that the target labels match the expected ground truth.

We have written our own handwriting digits, about 100 handwriting digits from 0 to 9. It is not like MNIST that has 60,000 handwriting digits, the main purpose is for educational only, how CNN LeNet-5 architecture works, will it works with our own handwritten digits even if it is only 100? Based on theory, it should work.

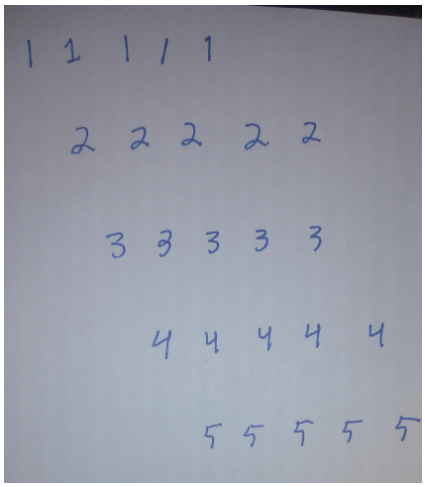

Figure 10.45: *The handwritten digits that we have created on a paper.*

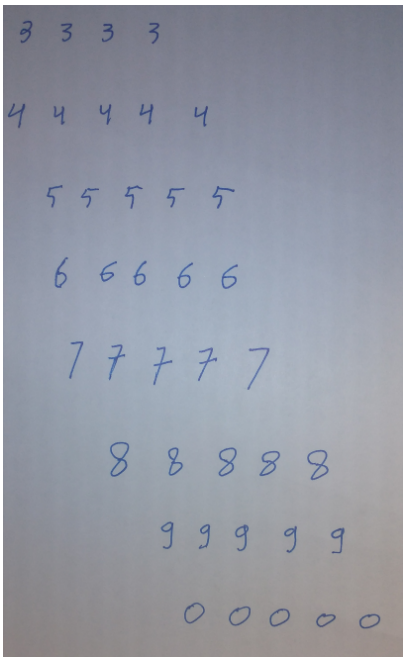

Figure 10.46: *We process this by uploading the photo to computer, crop each digit written, naming each of them nicely so it is well clustered (give unique label to each), then convert it to* ***idx3-ubyte*** *file and* ***idx1-ubyte*** *file.*

1. **Preparing the Input Images and Convert Them to IDX3-Ubyte**
   The reader can use their own handwriting or the one that we have created.

   First crop each digit and name the file that match the label for the resizing, grayscaling and converting purpose till the CNN phase.
   Then all we have to do is to resize all the images and turn them into grayscale, **STB Image** is very handy for this purpose. You can take a loot at **SymIntegration/Exam-**

ples/Neural Networks/Test Resize and Convert to Grayscale Images with STB Image/ put all the images there and compile the **main.cpp**, from the current working directory open xterm / terminal then type:
make
./main
All the images will be resized to 32×32 and become grayscale images, the name is also written with **gray_(originalname)**.

Then we move all the resized and grayscaled images to a new directory that will process them and convert the images into **idx3-ubyte** file, it is in **SymIntegration/Examples/Neural Networks/Test Create idx3-ubyte Files with STB Image/**, after you copy the images you can open terminal there and type:
bash createimageslist
(this will create **images.txt** that will store the images name)

You still need to create manually **labels.txt** to store the label name for each images, but if you can do it automatically it is good then. Afterwards, type again
make
./main
The images have been successfully converted to **idx3-ubyte** file along with the corresponding labels to **idx1-ubyte** file.

The function:
convert_to_idx(images, labels, "train-images-idx3-ubyte", "train-labels-idx1-ubyte", target_width, target_height)
will create the **idx3-ubyte** to represents the images and **idx1-ubyte** to represents the corresponding labels for the images.

We have finished preparing for the input, move the **train-images-idx3-ubyte** and **train-labels-idx1-ubyte** to the directory for the CNN training in **SymIntegration/Examples/Neural Networks/Test CNN using Class with Steepest Descent**.

2. **Normalize the Input Image to the $[-1, 1]$ Range**
After we moved the **train-images-idx3-ubyte** and **train-labels-idx1-ubyte** files to this directory **SymIntegration/Examples/Neural Networks/Test CNN using Class with Steepest Descent** all we have to do is normalize the grayscale input image. Normalizing grayscale images to the $[0, 1]$ range is significantly better for training a LeNet-5 CNN than leaving the raw pixel values at $[0, 255]$. But since we are using the LeNet-5 with tanh activation functions we will proceed to normalizing into $[-1, 1]$ range, it is slightly better than $[0, 1]$ because the inputs are strictly positive with $[0, 1]$ this forces the first layer of tanh activations to operate entirely in its upper quadrant, introducing a massive bias, causing earlier gradient saturation, and slowing down the overall optimization process. Here is why centering the data around zero helps the neural network train faster and more accurately:

   - **Faster Convergence:** Neural networks optimize weights by calculating gradients. When input data is centered around zero, the gradients flow more evenly. This helps the model find the optimal weights in fewer training steps. Large input values (like 255) cause large weight updates, making the training process unstable

or causing the model to diverge.

- **Better Gradient Flow:** Prevents numerical instability and ensures that gradients propagate properly to the earlier layers of the network during backpropagation. Large, positive-only inputs like $[0, 255]$ can cause weight updates to push neurons into saturation, particularly with sigmoid and tanh activation functions. Saturated neurons have near-zero gradients, which effectively stops the network from learning. The derivative of $\tanh(x)$ is highest near 0 ($f'(0) = 1$) and saturates (flattens out) as values approach $+1$ or -1. Inputs scaled to $[-1, 1]$ fall neatly into high-sensitivity zone, preventing the vanishing gradient problem in early training stages.
- **Symmetry:** LeNet-5 traditionally uses tanh activation functions in its hidden layer. Since tanh outputs values between $[-1, 1]$, having inputs on the same scale helps the network map features much more effectively. Leaky ReLU activation function can also works with normalized input that has range $[-1, 1]$.
- **Zero-Centered Input:** The tanh function has inflection point at exactly 0. Normalizing the inputs to $[-1, 1]$ creates a zero-centered dataset. This prevent gradients from systematically pushing in the same direction during backpropagation, resulting in faster and more stable convergence.

The formula to normalize to $[-1, 1]$:

$$\text{Normalized Pixel} = \frac{\text{Pixel Value} - 127.5}{127.5}$$

Under standard training conditions on the 60,000 images MNIST dataset, this configuration with normalized grayscale images to a $[-1, 1]$ typically yields a test accuracy between 98.5% and 99.0%.

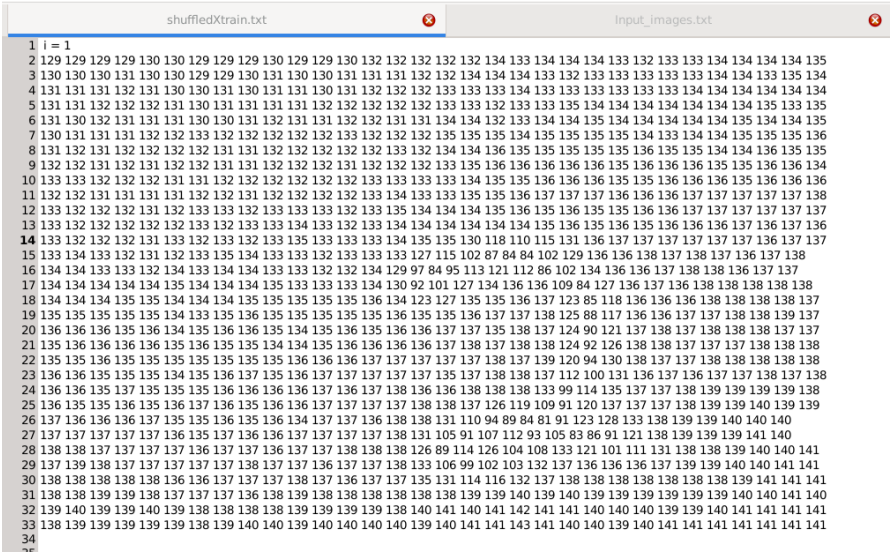

Figure 10.47: *The original input image is in standard grayscale range* $[0, 255]$.

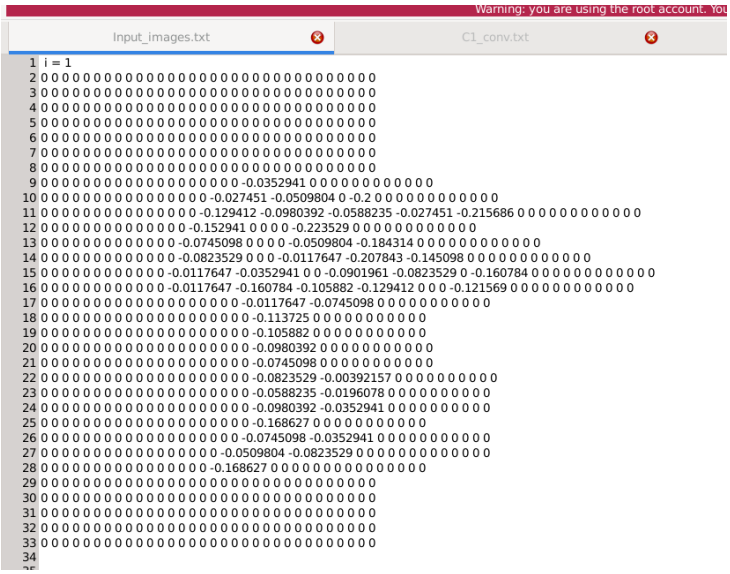

Figure 10.48: *The normalized input image with range* $[-1, 1]$ *and we also notice that the background paper has positive value while the pen stroke has negative value, thus we zeroes all the positive values and let the negative values be.*

3. **Weight and Bias Initialization**
   Since the original LeNet-5 predates many of the optimal weight initialization schemes such as Xavier or He initialization, in SymIntegration for **CNN_LeNet5** class the weights were initialized with He initialization.

4. **Training the Convolutional Neural Network for Digit Recognition with LeNet-5 Architecture**
   After we have normalized the input image to be trained with CNN in this directory **SymIntegration/Examples/Neural Networks/Test CNN using Class with Steepest Descent** all we have to do is open terminal there and type:
   make
   ./train

   In SymIntegration, we have put the normalizing to $[-1, 1]$ of the input image at the start of the training at **src/artificialneuralnetworks.cpp**

```
              ...

void CNN_LeNet5::train(vector<vector<vector<int>>> &X_input, vector<int>
    &y, int epochs)
{
...
int n_images = X_input.size();
int n_input = X_input[0].size();
vector<vector<vector<double>>> X(n_images, vector<vector<double>>(n_input
    , vector<double>(n_input, 0.0)));;

for (int epoch = 0; epoch < epochs; ++epoch)
```

```
{
       double epoch_loss = 0.0;
       for (size_t iter = 0; iter < X.size(); ++iter)
       {

              // Normalize the input image
              for(int i = 0; i < n_input; ++i)
              {
                     for(int j = 0; j < n_input; ++j)
                     {
                            X[iter][i][j] = divisiond(X_input[iter][i][j
                                ]-127.5,127.5);
                     }
              }
       ...
       }
...
}
```

Code 82: *src/artificialneuralnetworks.cpp*

The convolutional neural networks training will be starting, it takes more time compared to FNN training, because we are having a lot of matrices operation, including convolutional operation, pooling operation, forward pass and backpropagation, all requires more cost compared to plain FNN with iris dataset.

We will explain the code that is located in **src/artificialneuralnetworks.cpp** so the reader can comprehend and able to modify it to create their own CNN network, or replicate other famous network like MobileNet or VGG.

In the **include/symintegral/artificialneuralnetworks.h** we will define the class **CNN_LeNet5**, in **private** we will declare the variables that will be used as weights (array), and we declare a **vector<int> vec_uniqueinteger** to store the integers that will be generated randomly to choose the 3 feature maps in S3 layer that connect to any 4 discontiguous feature maps of S2 layer.

```
...

#ifndef CNN_LENET5_H
#define CNN_LENET5_H

class CNN_LeNet5 {
       private:

       double c1_weights_kernel[6][5][5]; // first convolutional layer, 6
            feature maps
       double s2_weights_kernel[6];
       double c3_weights_kernel_first6[450]; // second convolutional
           layer, first 6 feature maps
```

```
        double c3_weights_kernel_next6[600]; // second convolutional layer
            , next 6 feature maps
        double c3_weights_kernel_next3[300]; // second convolutional layer
            , next 3 feature maps
        double c3_weights_kernel_last1[150]; // second convolutional layer
            , last 1 feature map
        double s4_weights_kernel[16];
        double c5_weights_kernel[400][120]; // third convolutional layer,
            120 feature maps, kernel size of 5x5
        double f6_weights_kernel[120][84]; // the sixth layer F6
        double output_weights[84][10];

        double c1_bias[6];
        double s2_bias[6];
        double c3_bias[16];
        double s4_bias[16];
        double c5_bias[120];
        double f6_bias[84];
        double output_bias[10];

        vector<int> vec_uniqueinteger;

        double learning_rate;

        public:
        CNN_LeNet5(); // Constructor

        vector<double> predict(const vector<vector<double>> &inputs);

        void train(vector<vector<vector<int>>> &X, vector<int> &y, int
            epochs);

        void save_model(const string &filename) ; // Void function
            declaration
        void load_model(const string& filename);
};

#endif

...
```

Code 83: *include/symintegral/artificialneuralnetworks.h*

In the **src/artificialneuralnetworks.cpp** we define the **vector<double> CNN_LeNet5::predict(const vector<vector<double>> &inputs)** function that is used to produce the output vector (activated by softmax) from the input of **vector<vector<double>>** that is a normalized input image with range of $[-1, 1]$.

```
vector<double> CNN_LeNet5::predict(const vector<vector<double>> &inputs)
    // become vector<vector<double>>
```

```
{
	int n_kernelSize = 5;
	int n_classification = 10;
	int pooling_kernelSize = 2;
	int C1_stride = 1, C3_stride = 1, C5_stride = 1;
	int S2_stride = 2, S4_stride = 2;
	int padding = 0;
	vector<vector<vector<double>>> C1_conv, C1_Matrices, S2_Matrices,
	    C3_Matrices, S4_Matrices;

	int n_input = inputs.size();
	int r_C1 = (n_input - n_kernelSize + 2*padding)/(C1_stride) + 1 ;
	int c_C1 = r_C1;
	int r_S2 = (r_C1 - pooling_kernelSize)/(S2_stride) + 1;
	int c_S2 = r_S2;
	double alpha = 0.8;

	// First convolutional layer
	for (int k = 0 ; k < 6; ++k)
	{
		vector<vector<double>> c1_weights(n_kernelSize,vector<
		    double>(n_kernelSize,0.0));
		for (int i = 0; i < n_kernelSize; ++i)
		{
			for (int j = 0; j < n_kernelSize; ++j)
			{
				c1_weights[i][j] = c1_weights_kernel[k][i][j]
				    ;
			}
		}

		vector<vector<double>> C1_result =
		    CNN_2DConvolutionOperation(inputs,c1_weights, C1_stride
		    );
		C1_conv.push_back(C1_result);
		//cout <<"\nc1 k: "<< k << endl;

		for (int i = 0; i < r_C1; ++i)
		{
			for (int j = 0; j < c_C1; ++j)
			{
				if(C1_result[i][i] != 0)
				{
					C1_result[i][j] += c1_bias[k] ; // add
					    a bias
				}
				C1_result[i][j] = LeakyReLUActivationfunction
				    (C1_result[i][j], alpha); // apply the
```

```
                                activation function
                        }
                }

                // The pooling layer S2
                vector<vector<double>> S2_result = CNN_2DaveragePooling(
                    C1_result,pooling_kernelSize, S2_stride) ; // average
                    pooling

                for (int i = 0; i < r_S2; ++i)
                {
                        for (int j = 0; j < c_S2; ++j)
                        {
                                S2_result[i][j] *= s2_weights_kernel[k] ; //
                                    multiply with a weight
                                if(S2_result[i][j] != 0)
                                {
                                        S2_result[i][j] += s2_bias[k] ; // add
                                             a bias
                                }
                                S2_result[i][j] = LeakyReLUActivationfunction
                                    (S2_result[i][j], alpha); // apply the
                                    activation function
                        }
                }

                C1_Matrices.push_back(C1_result);
                S2_Matrices.push_back(S2_result);
        }

        // Continuing on the second convolutional layer C3 then S4
        vector<vector<int>> contiguous_combinations35 =
            contiguousCombinations(5,3,0);
        vector<vector<int>> contiguous_combinations45 =
            contiguousCombinations(5,4,0);
        vector<vector<int>> discontiguous_combinations45 =
            discontiguousCombinations(5,4,0);

...
...

vector<double> f6_output;
for (int i = 0; i < 84; ++i)
{
        double f6_sum = 0;

        for (int j = 0; j < 120; ++j)
        {
```

```
                f6_sum += f6_weights_kernel[j][i]*c5_output[j] ;
        }

        f6_output.push_back(f6_sum);
        if(f6_output[i] != 0)
        {
                f6_output[i] += f6_bias[i]; // add a bias
        }
        //f6_output[i] = TanhActivationfunction(f6_output[i]); // apply
            the activation function
        f6_output[i] = LeakyReLUActivationfunction(f6_output[i], alpha);
            // apply the activation function
}


// The output layer in LeNet-5, we use softmax instead of RBF
vector<double> outputs; // to compute the raw logit z_{i}
for (int i = 0; i < n_classification; ++i)
{
        double output_sum = 0;

        for (int j = 0; j < 84; ++j)
        {
                output_sum += output_weights[j][i]*f6_output[j] ;
        }

        outputs.push_back(output_sum);
        outputs[i] += output_bias[i];
}
// apply the activation function
vector<double>predicted_output_softmax =
    SoftMax_vectorresult_activationfunction(outputs);

return predicted_output_softmax;
}
```

Code 84: *src/artificialneuralnetworks.cpp*

It can be read fully and understood by the reader when reading the source code and matching it with the forward pass formula from Table (10.7).

For the training of the convolutional neural networks, we use a function:
void CNN_LeNet5::train(vector<vector<vector<int>>> &X_input, vector<int> &y, int epochs)

The input here is a 3D integer matrix to represent the input images with order that are shuffled first before training (**vector<vector<vector<int>>> &X_input**). At each epoch, each of the input image will be normalized to have the range of $[-1, 1]$.

```
void CNN_LeNet5::train(vector<vector<vector<int>>> &X_input, vector<int>
    &y, int epochs)
{

       cout << BLUE << BOLD << "\nTraining Progress:\n" << RESET << endl;

       int bar_width = 50;
       int n_classification = 10;
       int n_F6 = 84;
       int n_C5 = 120;
       //int n_C2 = 6;
       int n_kernelSize = 5;
       int pooling_kernelSize = 2;
       int C1_stride = 1, C3_stride = 1, C5_stride = 1;
       int S2_stride = 2, S4_stride = 2;
       int padding = 0;

...
...

for (int epoch = 0; epoch < epochs; ++epoch)
{
       double epoch_loss = 0.0;
       for (size_t iter = 0; iter < X_input.size(); ++iter)
       {

              // Normalize the input image
              for(int i = 0; i < n_input; ++i)
              {
                     for(int j = 0; j < n_input; ++j)
                     {
                            X[iter][i][j] = divisiond(X_input[iter][i][j
                                ]-127.5,127.5);
                            if (X[iter][i][j] > 0)
                            {
                                   X[iter][i][j] = 0;
                            }
                     }
              }

              vector<vector<vector<double>>> C1_Matrices, S2_Matrices,
                  S2_Matrices_xi, C3_Matrices, S4_Matrices,
                  S4_Matrices_xi;
              vector<vector<vector<double>>> C3_Distinct_Kernels;
              vector<double> delta_outputlayer(n_classification,0.0);
              vector<double> outputs = predict(X[iter]);
              vector<double> target(n_classification, 0.0);
              target[y[iter]] = 1.0;
```

```
            std::vector<double> errors(n_classification, 0.0);
            for (int k = 0; k < n_classification; ++k)
            {
                  errors[k] = -target[k] * log(outputs[k]); //
                      Categorical Cross-Entropy (CCE) Loss
                  epoch_loss += errors[k];
            }

            ...
```

Code 85: *src/artificialneuralnetworks.cpp*

We will do the forward pass once again in the train code in order to compute δ for all layers from the output layer back to the C1 layer.

```
...

// We will do forward pass once again here to help computing delta from
    F6 back to C1
// First convolutional layer
for (int k = 0 ; k < 6; ++k)
{
      vector<vector<double>> C1_weights(n_kernelSize,vector<double>(
          n_kernelSize,0.0));
      for (int i = 0; i < n_kernelSize; ++i)
      {
            for (int j = 0; j < n_kernelSize; ++j)
            {
                  C1_weights[i][j] = c1_weights_kernel[k][i][j] ;
            }
      }

      vector<vector<double>> C1_result = CNN_2DConvolutionOperation(X[
          iter],C1_weights, C1_stride);

      for (int i = 0; i < r_C1; ++i)
      {
            for (int j = 0; j < c_C1; ++j)
            {
                  if(C1_result[i][i] != 0)
                  {
                        C1_result[i][j] += c1_bias[k] ; // add a bias
                  }
                  C1_result[i][j] = LeakyReLUActivationfunction(
                      C1_result[i][j], alpha); // apply the activation
                       function
            }
      }
```

```
        // The pooling layer S2

        ...
}
// Continuing on the second convolutional layer C3 then S4
vector<vector<int>> contiguous_combinations35 = contiguousCombinations
    (5,3,0);
vector<vector<int>> contiguous_combinations45 = contiguousCombinations
    (5,4,0);
vector<vector<int>> discontiguous_combinations45 =
    discontiguousCombinations(5,4,0);

// The first 6: connect to any 3 contiguous feature maps of S2
for (int k = 0 ; k < 6; ++k)
{
        vector<vector<double>> C3_result(r_C3, vector<double>(c_C3,0.0));

        for (int k3 = 0 ; k3 < 3 ; ++k3)
        {
                vector<vector<double>> c3_weights(n_kernelSize,vector<
                    double>(n_kernelSize,0.0));
                for (int i = 0; i < n_kernelSize; ++i)
                {
                        for (int j = 0; j < n_kernelSize; ++j)
                        {
                                c3_weights[i][j] = c3_weights_kernel_first6
                                    [75*k + 25*k3 + 5*i + j] ; // The first
                                    three contiguous feature maps will cover
                                    75 neurons, each feature map from S2 will
                                     cover 25 neurons
                        }
                }

                int n_comb1 = contiguous_combinations35[k][k3];

                vector<vector<double>> S2(r_S2, vector<double>(c_S2, 0.0));
                for (int i = 0; i < r_S2; ++i)
                {
                        for (int j = 0; j < c_S2; ++j)
                        {
                                S2[i][j] = S2_Matrices[n_comb1][i][j];
                        }
                }
                vector<vector<double>> C3_result_temp =
                    CNN_2DConvolutionOperation(S2,c3_weights, C3_stride);

                for (int i = 0; i < r_C3; ++i)
                {
```

```
                    for (int j = 0; j < c_C3; ++j)
                    {
                           C3_result[i][j] += C3_result_temp[i][j]; //
                               sum of the indices combination
                    }
             }
      }

      for (int i = 0; i < r_C3; ++i)
      {
             for (int j = 0; j < c_C3; ++j)
             {
                    if(C3_result[i][j] != 0)
                    {
                           C3_result[i][j] += c3_bias[k] ; // add a bias
                    }
                    C3_result[i][j] = LeakyReLUActivationfunction(
                        C3_result[i][j], alpha); // apply the activation
                         function
             }
      }
      C3_Matrices.push_back(C3_result);

      vector<vector<double>> S4_result = CNN_2DaveragePooling(C3_result,
           pooling_kernelSize, S4_stride) ; // average pooling
      vector<vector<double>> S4_xi(r_S4, vector<double>(c_S4,0.0));
      for (int i = 0; i < r_S4; ++i)
      {
             for (int j = 0; j < c_S4; ++j)
             {
                    S4_xi[i][j] = S4_result[i][j];
                    S4_result[i][j] *= s4_weights_kernel[k] ; //
                        multiply with a weight
                    if(S4_result[i][j] != 0)
                    {
                           S4_result[i][j] += s4_bias[k] ; // add a bias
                    }
                    S4_result[i][j] = LeakyReLUActivationfunction(
                        S4_result[i][j], alpha); // apply the activation
                         function
             }
      }
      S4_Matrices_xi.push_back(S4_xi);
      S4_Matrices.push_back(S4_result);
}

...
```

Code 86: *src/artificialneuralnetworks.cpp*

Afterwards we will compute the δ starting from the output layer, backpropagating to C1 layer. The formula to compute the δ can be seen in Table (10.8).

```
...

for (int k_row = 0; k_row < n_classification; ++k_row)
{
       delta_outputlayer[k_row] = outputs[k_row] — target[k_row];
}

// Delta for F6 layer, output_weights connects the F6 to output layer
vector<double> delta_F6(n_F6, 0.0);
for (int j = 0; j < n_F6; ++j)
{
       for (int i = 0; i < n_classification; ++i)
       {
              delta_F6[j] += delta_outputlayer[i] * output_weights[j][i];
       }
       delta_F6[j] *= LeakyReLUDerivative(f6_output[j], alpha);
}
// Delta for C5 layer, f6_weights_kernel connects the C5 to F6 layer
vector<double> delta_C5(n_C5, 0.0);
for (int j = 0; j < n_C5; ++j)
{
       for (int i = 0; i < n_F6; ++i)
       {
              delta_C5[j] += delta_F6[i] * f6_weights_kernel[j][i];
       }
       delta_C5[j] *= LeakyReLUDerivative(c5_output[j], alpha);
}
vector<vector<vector<double>>> S4_deltamap;

// Compute delta for S4, 1920 distinct kernels
for (int k = 0; k <16; ++k) // 16 x 5 x 5
{
       vector<vector<double>> S4_deltamap_k(5,vector<double>(5,0.0));
       for (int k5 = 0; k5 < 120; ++k5)
       {
              int iter25 = 0;
              vector<vector<double>> c5_weights(5,vector<double>(5,0.0));
              for (int i = 0; i < n_kernelSize; ++i)
              {
                     for (int j = 0; j < n_kernelSize; ++j)
                     {
                            c5_weights[i][j] = c5_weights_kernel[25*k +
                                iter25][k5] ;
                            iter25 += 1;
                     }
              }
```

```
            vector<vector<double>> C5_M(1, vector<double>(1, delta_C5[
                k5]));
            vector<vector<double>> kernel_C5_rotated = rotate180(
                c5_weights); // rotate 180 = transpose weight matrix
                from C5 to F6 or F6 to output to compute delta
            vector<vector<double>> delta_C5_padded = CNN_2DpadBorder(
                C5_M,4);

            vector<vector<double>> fullconv_S4_result =
                CNN_2DConvolutionOperation(delta_C5_padded,
                kernel_C5_rotated, C5_stride);
            for (int i = 0; i < n_kernelSize; ++i)
            {
                  for (int j = 0; j < n_kernelSize; ++j)
                  {
                        S4_deltamap_k[i][j] += fullconv_S4_result[i][
                            j];
                  }
            }
      }
      for (int i = 0; i < n_kernelSize; ++i)
      {
            for (int j = 0; j < n_kernelSize; ++j)
            {
                  S4_deltamap_k[i][j] *= LeakyReLUDerivative(
                      S4_Matrices[k][i][j], alpha);
            }
      }
      S4_deltamap.push_back(S4_deltamap_k);
}

vector<double> delta_S4_final(16, 0.0);
vector<double> delta_S4_final_bias(16, 0.0);
for (int k = 0; k < 16; ++k)
{
      for (int i = 0; i < n_kernelSize; ++i)
      {
            for (int j = 0; j < n_kernelSize; ++j)
            {
                  delta_S4_final[k] += S4_deltamap[k][i][j] *
                      S4_Matrices_xi[k][i][j];
                  delta_S4_final_bias[j] += S4_deltamap[k][i][j];
            }
      }
}

vector<vector<vector<double>>> upsampled_delta_S4 =
    CNN_2DaverageUpsample3DMatrix(S4_deltamap,2,2);
```

```
vector<vector<vector<double>>> delta_C3(16,vector<vector<double>>(10,
    vector<double>(10,0.0)));
for (int k = 0 ; k < 16; ++k)
{
       for (int i = 0; i < 10 ; ++i)
       {
              for (int j = 0; j < 10; ++j)
              {
                     //delta_C3[k][i][j] = upsampled_delta_S4[k][i][j] *
                         s4_weights_kernel[k] * TanhDerivative(
                         C3_Matrices[k][i][j]);
                     delta_C3[k][i][j] = upsampled_delta_S4[k][i][j] *
                         s4_weights_kernel[k] * LeakyReLUDerivative(
                         C3_Matrices[k][i][j], alpha);
              }
       }
}

for (int k = 0 ; k < 6; ++k)
{
       for (int k3 = 0 ; k3 < 3 ; ++k3)
       {
              int n_comb1 = contiguous_combinations35[k][k3];

              vector<vector<double>> S2(r_S2, vector<double>(c_S2, 0.0));
              for (int i = 0; i < r_S2; ++i)
              {
                     for (int j = 0; j < c_S2; ++j)
                     {
                            S2[i][j] = S2_Matrices[n_comb1][i][j];
                     }
              }
              vector<vector<double>> delta_C3_matrix(10, vector<double
                  >(10, 0.0));
              for (int i = 0; i < 10; ++i)
              {
                     for (int j = 0; j < 10; ++j)
                     {
                            delta_C3_matrix[i][j] = delta_C3[k][i][j];
                     }
              }
              vector<vector<double>> distinct_kernels =
                  CNN_2DConvolutionOperation(S2,delta_C3_matrix,
                  C3_stride);

              C3_Distinct_Kernels.push_back(distinct_kernels);
       }
```

```
}
k1 = 0;
for (int k = 6 ; k < 12; ++k)// The next 24 distinct kernels that connect
     to any 4 contiguous feature maps of S2
{
       for (int k3 = 0 ; k3 < 4 ; ++k3)
       {
              int n_comb1 = contiguous_combinations45[k1][k3];

              vector<vector<double>> S2(r_S2, vector<double>(c_S2, 0.0));
              for (int i = 0; i < r_S2; ++i)
              {
                     for (int j = 0; j < c_S2; ++j)
                     {
                            S2[i][j] = S2_Matrices[n_comb1][i][j];
                     }
              }
              vector<vector<double>> delta_C3_matrix(10, vector<double
                  >(10, 0.0));
              for (int i = 0; i < 10; ++i)
              {
                     for (int j = 0; j < 10; ++j)
                     {
                            delta_C3_matrix[i][j] = delta_C3[k][i][j];
                     }
              }
              vector<vector<double>> distinct_kernels =
                  CNN_2DConvolutionOperation(S2,delta_C3_matrix,
                  C3_stride);

              C3_Distinct_Kernels.push_back(distinct_kernels);
       }
       k1 += 1;
}

k1 = 0;
for (int k = 12 ; k < 15; ++k) // The next 12 distinct kernels that
    connect to any 4 discontiguous feature maps of S2
{
       for (int k3 = 0 ; k3 < 4 ; ++k3)
       {
              int n1 = vec_uniqueinteger[k1];

              int n_comb1 = discontiguous_combinations45[n1][k3];
              vector<vector<double>> S2(r_S2, vector<double>(c_S2, 0.0));
              for (int i = 0; i < r_S2; ++i)
              {
```

```
                    for (int j = 0; j < c_S2; ++j)
                    {
                           S2[i][j] = S2_Matrices[n_comb1][i][j];
                    }
             }
             vector<vector<double>> delta_C3_matrix(10, vector<double
                 >(10, 0.0));
             for (int i = 0; i < 10; ++i)
             {
                    for (int j = 0; j < 10; ++j)
                    {
                           delta_C3_matrix[i][j] = delta_C3[k][i][j];
                    }
             }
             vector<vector<double>> distinct_kernels =
                 CNN_2DConvolutionOperation(S2,delta_C3_matrix,
                 C3_stride);

             C3_Distinct_Kernels.push_back(distinct_kernels);
       }
       k1 +=1;
}

for (int k = 15 ; k < 16; ++k) // The last 6 distinct kernels that
    connect to all 6 feature maps in S2
{
       for (int k3 = 0 ; k3 < 6 ; ++k3)
       {
             vector<vector<double>> S2(r_S2, vector<double>(c_S2, 0.0));
             for (int i = 0; i < r_S2; ++i)
             {
                    for (int j = 0; j < c_S2; ++j)
                    {
                           S2[i][j] = S2_Matrices[k3][i][j];
                    }
             }
             vector<vector<double>> delta_C3_matrix(10, vector<double
                 >(10, 0.0));
             for (int i = 0; i < 10; ++i)
             {
                    for (int j = 0; j < 10; ++j)
                    {
                           delta_C3_matrix[i][j] = delta_C3[k][i][j];
                    }
             }
             vector<vector<double>> distinct_kernels =
                 CNN_2DConvolutionOperation(S2,delta_C3_matrix,
                 C3_stride);
```

```
                C3_Distinct_Kernels.push_back(distinct_kernels);
        }
}

...
```

Code 87: *src/artificialneuralnetworks.cpp*

In the C3 layer, we have to compute 1600 δ for the 16 feature maps of size 10×10 then we will use the deltas to compute the weight update for the 1,500 trainable weights and 16 trainable biases in C3 layer. The 1,500 trainable weights can be updated if we know the vector gradients for the corresponding trainable weights, thus we will have to compute 60 distinct kernels that corresponds to the 16 feature maps in C3 layer: the first 18 distinct kernels connect to any 3 contiguous feature maps of S2 layer (the first 6 feature maps in C3), then the next 24 distinct kernels connect to any 4 contiguous feature maps in S2 layer (the next 6 feature maps in C3), then the next 12 distinct kernels connect to any 4 discontiguous feature maps in S2 layer (the next 3 feature maps in C3), finally the last 6 distinct kernels connect to all feature maps in S2 (the last feature map in C3).

We store 60 distinct kernels that will be responsible for the weight update for C3 layer in a 3D matrix **vector<vector<vector<double>>>C3_Distinct_Kernels**.

- **c3_weights_kernel_first6[450]** to represent the weights for the first 6 feature maps will use the first 18 distinct kernels for the weight update.
- **c3_weights_kernel_next6[600]** to represent the weights for the next 6 feature maps will use the next 24 distinct kernels for the weight update.
- **c3_weights_kernel_next3[300]** to represent the weights for the next 3 feature maps will use the next 12 distinct kernels for the weight update.
- **c3_weights_kernel_last1[150]** to represent the weights for the last feature map will use the last 6 distinct kernels for the weight update.

To compute the δ for the S2 layer it is as complex as the C3 layer since it is partially connected thus we need longer code. The formula to compute the δ^{S2} can be seen in table (10.8) in which we have to rotate weight matrix from S2 to C3 180^0 and perform full convolution with the delta matrix (to perform full convoltion: pad the delta matrix of C3 layer of size 10×10 with 4 rows and columns of zeos on all sides making it 18×18, afterwards do the convolution with the rotated weight matrix).

First we will declare / create 3D matrices to represent the delta map for S2 from 1 to 6, since we are using sparse connectivity then we perform the full convolution between the padded δ^{C3} matrix and the rotated weight matrix from S2 to C3 from the first 6 feature maps in C3, to the next 6 feature maps in C3, to the next 3 feature maps in C3, and the last feature map in C3.

Each time we perform convolution operation we know which feature map in S2 that is used from the functions **contiguous_combinations35 contiguous_combinations45**, **discontiguous_combinations45**, thus we create **if.. else if..** conditional to store the result of the full convolution operation to the 3D matrix **vector<vector<vector<double>>>**

S2_deltamap_i ($i = 1, 2, \ldots, 6$) that represents the feature map in S2 layer that is used for the current full convolution operation.

In the end, we will obtain 60 results from the full convolution operation between the rotated weight matrix from S2 to C3 and the padded δ^{C3}, each of these 60 matrices will represent feature map of S2, each could belong to feature map 1,2,3,4,5, or 6. After we already have them clustered from the operations above, then we will use a class **Matrix3Dto2D** to sum over the depth in 3D matrix at index $[i][j]$ into a 2D matrix. These 2D matrices are the delta map (1,176 δ^{S2} that represents $6 \times 14 \times 14$ feature maps in S2) that will be used to compute the weight gradients for S2 weights update.

```
...

vector<vector<vector<double>>> S2_deltamap_1, S2_deltamap_2,
    S2_deltamap_3, S2_deltamap_4, S2_deltamap_5, S2_deltamap_6; // Store
     delta map for S2 with sparse connectivity
// Computation for delta map for S2 layer:
// The first 6 feature maps in S3 that connect to any 3 contiguous
    feature maps of S2
for (int k = 0 ; k < 6; ++k)
{
       for (int k3 = 0 ; k3 < 3 ; ++k3)
       {
              vector<vector<double>> delta_C3_matrix(10, vector<double
                  >(10, 0.0));
              vector<vector<double>> S2_C3_kernelmatrix(5, vector<double
                  >(5, 0.0));
              for (int i = 0; i < 10; ++i)
              {
                     for (int j = 0; j < 10; ++j)
                     {
                            delta_C3_matrix[i][j] = delta_C3[k][i][j];
                     }
              }
              vector<vector<double>> padded_delta = CNN_2DpadBorder(
                  delta_C3_matrix,4);
              for (int i = 0; i < 5; ++i)
              {
                     for (int j = 0; j < 5; ++j)
                     {
                            S2_C3_kernelmatrix[i][j] =
                                c3_weights_kernel_first6[75*k + 25*k3 +
                                5*i + j] ;
                     }
              }
              vector<vector<double>> rotated_W = rotate180(
                  S2_C3_kernelmatrix);

              int n_comb1 = contiguous_combinations35[k][k3];
```

```
            vector<vector<double>> deltamap_kernel =
                CNN_2DConvolutionOperation(padded_delta, rotated_W,
                C3_stride);

            if(n_comb1 == 0)
            {
                  S2_deltamap_1.push_back(deltamap_kernel);
            }
            else if(n_comb1 == 1)
            {
                  S2_deltamap_2.push_back(deltamap_kernel);
            }
            else if(n_comb1 == 2)
            {
                  S2_deltamap_3.push_back(deltamap_kernel);
            }
            else if(n_comb1 == 3)
            {
                  S2_deltamap_4.push_back(deltamap_kernel);
            }
            else if(n_comb1 == 4)
            {
                  S2_deltamap_5.push_back(deltamap_kernel);
            }
            else if(n_comb1 == 5)
            {
                  S2_deltamap_6.push_back(deltamap_kernel);
            }
      }

}

// The next 6 feature maps in S3 that connect to any 4 contiguous feature
     maps of S2
k1 = 0;
for (int k = 6 ; k < 12; ++k)
{
      for (int k3 = 0 ; k3 < 4 ; ++k3)
      {
            vector<vector<double>> delta_C3_matrix(10, vector<double
                >(10, 0.0));
            vector<vector<double>> S2_C3_kernelmatrix(5, vector<double
                >(5, 0.0));
            for (int i = 0; i < 10; ++i)
            {
                  for (int j = 0; j < 10; ++j)
                  {
```

```
                        delta_C3_matrix[i][j] = delta_C3[k][i][j];
                }
        }
        vector<vector<double>> padded_delta = CNN_2DpadBorder(
            delta_C3_matrix,4);
        for (int i = 0; i < 5; ++i)
        {
                for (int j = 0; j < 5; ++j)
                {
                        S2_C3_kernelmatrix[i][j] =
                            c3_weights_kernel_next6[100*k1 +25*k3 +
                            5*i + j] ;
                }
        }
        vector<vector<double>> rotated_W = rotate180(
            S2_C3_kernelmatrix);

        int n_comb1 = contiguous_combinations45[k1][k3];

        vector<vector<double>> deltamap_kernel =
            CNN_2DConvolutionOperation(padded_delta, rotated_W,
            C3_stride);

        if(n_comb1 == 0)
        {
                S2_deltamap_1.push_back(deltamap_kernel);
        }
        else if(n_comb1 == 1)
        {
                S2_deltamap_2.push_back(deltamap_kernel);
        }
        else if(n_comb1 == 2)
        {
                S2_deltamap_3.push_back(deltamap_kernel);
        }
        else if(n_comb1 == 3)
        {
                S2_deltamap_4.push_back(deltamap_kernel);
        }
        else if(n_comb1 == 4)
        {
                S2_deltamap_5.push_back(deltamap_kernel);
        }
        else if(n_comb1 == 5)
        {
                S2_deltamap_6.push_back(deltamap_kernel);
        }
}
```

```
       k1 += 1;
}

// The next 3 feature maps in S3 that connect to any 4 discontiguous
    feature maps of S2
k1 = 0;
for (int k = 12 ; k < 15; ++k)
{
       for (int k3 = 0 ; k3 < 4 ; ++k3)
       {
              vector<vector<double>> delta_C3_matrix(10, vector<double
                  >(10, 0.0));
              vector<vector<double>> S2_C3_kernelmatrix(5, vector<double
                  >(5, 0.0));
              for (int i = 0; i < 10; ++i)
              {
                     for (int j = 0; j < 10; ++j)
                     {
                            delta_C3_matrix[i][j] = delta_C3[k][i][j];
                     }
              }
              vector<vector<double>> padded_delta = CNN_2DpadBorder(
                  delta_C3_matrix,4);
              for (int i = 0; i < 5; ++i)
              {
                     for (int j = 0; j < 5; ++j)
                     {
                            S2_C3_kernelmatrix[i][j] =
                                c3_weights_kernel_next3[100*k1 + 25*k3+
                                5*i + j] ;
                     }
              }
              vector<vector<double>> rotated_W = rotate180(
                  S2_C3_kernelmatrix); // rotate 180 = transpose weight
                  matrix from C5 to F6 or F6 to output to compute delta
              int n1 = vec_uniqueinteger[k1];

              int n_comb1 = discontiguous_combinations45[n1][k3];

              vector<vector<double>> deltamap_kernel =
                  CNN_2DConvolutionOperation(padded_delta, rotated_W,
                  C3_stride);

              if(n_comb1 == 0)
              {
                     S2_deltamap_1.push_back(deltamap_kernel);
              }
              else if(n_comb1 == 1)
```

```
                {
                        S2_deltamap_2.push_back(deltamap_kernel);
                }
                else if(n_comb1 == 2)
                {
                        S2_deltamap_3.push_back(deltamap_kernel);
                }
                else if(n_comb1 == 3)
                {
                        S2_deltamap_4.push_back(deltamap_kernel);
                }
                else if(n_comb1 == 4)
                {
                        S2_deltamap_5.push_back(deltamap_kernel);
                }
                else if(n_comb1 == 5)
                {
                        S2_deltamap_6.push_back(deltamap_kernel);
                }
        }
        k1 += 1;
}

// The last feature map in S3 that connect to all 6 discontiguous feature
     maps of S2
for (int k = 15 ; k < 16; ++k)
{
        for (int k3 = 0 ; k3 < 6 ; ++k3)
        {
                vector<vector<double>> delta_C3_matrix(10, vector<double
                    >(10, 0.0));
                vector<vector<double>> S2_C3_kernelmatrix(5, vector<double
                    >(5, 0.0));
                for (int i = 0; i < 10; ++i)
                {
                        for (int j = 0; j < 10; ++j)
                        {
                                delta_C3_matrix[i][j] = delta_C3[k][i][j];
                        }
                }
                vector<vector<double>> padded_delta = CNN_2DpadBorder(
                    delta_C3_matrix,4);
                for (int i = 0; i < 5; ++i)
                {
                        for (int j = 0; j < 5; ++j)
                        {
                                S2_C3_kernelmatrix[i][j] =
                                    c3_weights_kernel_last1[25*k3+ 5*i + j] ;
```

```
                    }
                }
                vector<vector<double>> rotated_W = rotate180(
                    S2_C3_kernelmatrix); // rotate 180 = transpose weight
                    matrix from C5 to F6 or F6 to output to compute delta

                vector<vector<double>> deltamap_kernel =
                    CNN_2DConvolutionOperation(padded_delta, rotated_W,
                    C3_stride);

                if(k3 == 0)
                {
                       S2_deltamap_1.push_back(deltamap_kernel);
                }
                else if(k3 == 1)
                {
                       S2_deltamap_2.push_back(deltamap_kernel);
                }
                else if(k3 == 2)
                {
                       S2_deltamap_3.push_back(deltamap_kernel);
                }
                else if(k3 == 3)
                {
                       S2_deltamap_4.push_back(deltamap_kernel);
                }
                else if(k3 == 4)
                {
                       S2_deltamap_5.push_back(deltamap_kernel);
                }
                else if(k3 == 5)
                {
                       S2_deltamap_6.push_back(deltamap_kernel);
                }
       }
}

int n_S2_map1 = S2_deltamap_1.size();
int n_S2_map2 = S2_deltamap_2.size();
int n_S2_map3 = S2_deltamap_3.size();
int n_S2_map4 = S2_deltamap_4.size();
int n_S2_map5 = S2_deltamap_5.size();
int n_S2_map6 = S2_deltamap_6.size();

// Sum over the depth at [i][j] into a 2D matrix with class Matrix3DTo2D
// 14 rows, 14 columns, n_S2_map_k depth layers
Matrix3DTo2D S2_deltamap_1_final(14, 14, n_S2_map1);
Matrix3DTo2D S2_deltamap_2_final(14, 14, n_S2_map2);
```

```
Matrix3DTo2D S2_deltamap_3_final(14, 14, n_S2_map3);
Matrix3DTo2D S2_deltamap_4_final(14, 14, n_S2_map4);
Matrix3DTo2D S2_deltamap_5_final(14, 14, n_S2_map5);
Matrix3DTo2D S2_deltamap_6_final(14, 14, n_S2_map6);

// To compute the 1,176 deltas for S2 layer (6 x 14 x 14)
for (int k = 0; k < n_S2_map1; ++k)
{
	for (int i = 0; i < 14; ++i)
	{
		for (int j = 0; j < 14; ++j)
		{
			double index = S2_deltamap_1[k][i][j];
			S2_deltamap_1_final.set(i, j, k, index);
		}
	}
}
for (int k = 0; k < n_S2_map2; ++k)
{
	for (int i = 0; i < 14; ++i)
	{
		for (int j = 0; j < 14; ++j)
		{
			double index = S2_deltamap_2[k][i][j];
			S2_deltamap_2_final.set(i, j, k, index);
		}
	}
}
for (int k = 0; k < n_S2_map3; ++k)
{
	for (int i = 0; i < 14; ++i)
	{
		for (int j = 0; j < 14; ++j)
		{
			double index = S2_deltamap_3[k][i][j];
			S2_deltamap_3_final.set(i, j, k, index);
		}
	}
}
for (int k = 0; k < n_S2_map4; ++k)
{
	for (int i = 0; i < 14; ++i)
	{
		for (int j = 0; j < 14; ++j)
		{
			double index = S2_deltamap_4[k][i][j];
			S2_deltamap_4_final.set(i, j, k, index);
		}
```

```
        }
}
for (int k = 0; k < n_S2_map5; ++k)
{
        for (int i = 0; i < 14; ++i)
        {
                for (int j = 0; j < 14; ++j)
                {
                        double index = S2_deltamap_5[k][i][j];
                        S2_deltamap_5_final.set(i, j, k, index);
                }
        }
}
for (int k = 0; k < n_S2_map6; ++k)
{
        for (int i = 0; i < 14; ++i)
        {
                for (int j = 0; j < 14; ++j)
                {
                        double index = S2_deltamap_6[k][i][j];
                        S2_deltamap_6_final.set(i, j, k, index);
                }
        }
}
vector<double> S2_Delta_Final1 = S2_deltamap_1_final.collapseTo2D();
vector<double> S2_Delta_Final2 = S2_deltamap_2_final.collapseTo2D();
vector<double> S2_Delta_Final3 = S2_deltamap_3_final.collapseTo2D();
vector<double> S2_Delta_Final4 = S2_deltamap_4_final.collapseTo2D();
vector<double> S2_Delta_Final5 = S2_deltamap_5_final.collapseTo2D();
vector<double> S2_Delta_Final6 = S2_deltamap_6_final.collapseTo2D();
vector<vector<vector<double>>> S2_deltamap(6,vector<vector<double>>(14,
    vector<double>(14,0.0)));
for (int i = 0; i < 14; ++i)
{
        for (int j = 0; j < 14; ++j)
        {
                S2_deltamap[0][i][j] = S2_Delta_Final1[i*14 + j];
                S2_deltamap[1][i][j] = S2_Delta_Final2[i*14 + j];
                S2_deltamap[2][i][j] = S2_Delta_Final3[i*14 + j];
                S2_deltamap[3][i][j] = S2_Delta_Final4[i*14 + j];
                S2_deltamap[4][i][j] = S2_Delta_Final5[i*14 + j];
                S2_deltamap[5][i][j] = S2_Delta_Final6[i*14 + j];
        }
}

// to compute the delta for each feature map in S2 layer
vector<double> delta_S2_final(6, 0.0);
vector<double> delta_S2_final_bias(6, 0.0);
```

```
for (int k = 0; k < 6; ++k)
{
       double delta_sum = 0;
       for (int i = 0; i < 14; ++i)
       {
              for (int j = 0; j < 14; ++j)
              {
                     delta_S2_final[k] += S2_deltamap[k][i][j] *
                         S2_Matrices_xi[k][i][j] * LeakyReLUDerivative(
                         S2_Matrices[k][i][j],alpha);
                     delta_sum += S2_deltamap[k][i][j];
              }
       }
       delta_S2_final_bias[k] = delta_sum;
}
```

Code 88: *src/artificialneuralnetworks.cpp*

Finally, we will compute the δ^{C1}, we follow the formula from Table (10.8) and we will obtain 4,704 δ^{C1} that represents $6 \times 28 \times 28$ feature maps in C1. The weight update for C1 is performing valid convolution between the input image (that is normalized) and the δ_k^{C1} with $k = 1, 2, \ldots, 6$.

```
...

// To Compute the delta for each feature map in C1 layer
vector<vector<vector<double>>> upsampled_delta_S2 =
    CNN_2DaverageUpsample3DMatrix(S2_deltamap,2,2);

vector<vector<vector<double>>> delta_C1(6,vector<vector<double>>(28,
    vector<double>(28,0.0)));
for (int k = 0 ; k < 6; ++k)
{
       for (int i = 0; i < 28 ; ++i)
       {
              for (int j = 0; j < 28; ++j)
              {
                     //delta_C1[k][i][j] = upsampled_delta_S2[k][i][j] *
                         s2_weights_kernel[k] * TanhDerivative(
                         C1_Matrices[k][i][j]);
                     delta_C1[k][i][j] = upsampled_delta_S2[k][i][j] *
                         s2_weights_kernel[k] * LeakyReLUDerivative(
                         C1_Matrices[k][i][j], alpha);
              }
       }
}
// Now we have obtained 4704 deltas in each neuron in 6x28x28 feature map
     for C1 layer.
vector<vector<vector<double>>> C1_weights_update;
for (int k = 0 ; k < 6; ++k)
```

```
{
       vector<vector<double>> weights_update = CNN_2DConvolutionOperation
           (X[iter],delta_C1[k], C1_stride);
       C1_weights_update.push_back(weights_update);
}

...
```

Code 89: *src/artificialneuralnetworks.cpp*

Adaptive Moment Estimation (Adam) Optimizer An alternative in neural networks besides using steepest descent / gradient descent method is using Adam optimizer. The Adam optimizer is a highly efficient, first-order gradient-based optimization algorithm used to train deep learning models. It dynamically adjusts the learning rate for each network weight individually by combining the principles of Momentum and RMSprop. Adam is favored as the default optimizer for many machine learning and deep learning tasks because it requires minimal tuning. Default values typically perform well across a wide variety of datasets. To helps LeNet-5 converge faster and avoid getting stuck in local minima we will use Adam optimizer. We will apply Adam optimizer to the weights of the feature maps in the convolutional layers (C1, C3, and C5), and the fully connected layers (F6 and output) and their respective biases. We will maintaining separate, adaptive learning rates for each layer's parameters.

Adam optimizer is applied after the backpropagation phase calculates the gradients of the loss function with respect to the network's weights and biases. Adam combines Momentum (first moment) and RMSProp (second moment) to dynamically adjust the learning rate for each specific weight and bias in the convolutional and fully connected layers.

During backpropagation at time step t, we fist compute the gradient of the loss L with respect to a specific parameter (weight w, or bias b), represented here as $g_t = \nabla_\theta L_t$.

(a) **Calculate the First and Second Moments**
Adam calculates the exponential moving averages of the gradient (first moment) and the squared gradient (second moment).

The first moment vector (mean / momentum) tracks the average of past gradients / acts like a moving average of the past gradients to maintain velocity, it provides direction and smooths out the path of descent, providing "momentum" to speed up training:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

The second moment vector (uncentered variance) acts like RMSProp , calculating the exponential moving average of the squared gradients to scale the learning rate, increasing it (taking larger steps) for sparse features (noisy gradients) and decreasing it (more cautious steps) for frequent ones (complex areas):

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Where:

- β_1 is the exponential decay rate for the first moment (usually 0.9).
- β_2 is the exponential decay rate for the second moment (usually 0.999).
- g_t is the gradient at time t.
- m_t is the first moment vector.
- v_t is the second moment vector.

(b) **Apply Bias Correction**
Because m_t and v_t are initialized to zero, they are biased toward zero early in training. Bias correction is used to adjust for this.

Bias-corrected first moment:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

Bias-corrected second moment:

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

(c) **Update the Parameters**
Finally, the weights and biases of the LeNet-5 network are updated using the bias-corrected moments.

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon}\hat{m}_t$$

Where:

- θ represents the parameter being updated (a weight w or a bias b).
- η is the learning rate.
- ϵ is a very small number added to prevent division by zero (e.g., $\epsilon 10^{-8}$)

(d) **Update Exponential Decay Rates (β_1 and β_2)**
In standard Adam, β_1 and β_2 are static hyperparameters. However, for specialized training (like fine-tuning a LeNet-5), you can update or decay them over time using the following techniques:

- **Exponential Decay (for Schedule):** You can decay the decay rate itself over time to narrow down the search space as training converges. For instance, at each epoch t, you can update β_1 as:

  $$\beta_1^t = \beta_1^0 \times \lambda^t$$

  and update β_2 as:

  $$\beta_2^t = \beta_2^0 \times \lambda^t$$

  with λ as a decay rate constant (e.g., $\lambda = 0.99$), and t is the epoch number.
- **Decay Rate for the Learning Rate**
  If you wish to decay the learning rate instead to prevent oscillation near the minimum, you can dynamically adjust η per epoch:

  $$\eta_t = \frac{\eta_0}{\sqrt{t}}$$

Each weight in the network gets its own specific learning rate, which is incredibly useful for processing sparse data. Adam corrects for early-stage biases during the initial

iterations to ensure precise gradient approximations. It requires very little memory and works incredibly well on a wide range of architectures, including convolutional neural networks and recurrent neural networks.

Adam optimizer can also gets stuck in local minima primarily because its learning rate diminishes too quickly, its momentum becomes trapped by directional shifts, or the batch size removes the necessary exploration noise. While Adam combines momentum and adaptive learning rates to escape shallow traps, specific mathematical dynamics can lock it into suboptimal basins.

(a) **Bad Weight Initialization**
Adam may converge to the neares poor local minimum.
The Mechanism: Poor weights initialization will place the starting weights far from optima. Zero-initialization breaks symmetry and stalls learning entirely.

(b) **Vanishing Learning Rate via the Second Moment (v_t)**
Adam tracks the moving average of squared gradients (v_t) to scale updates for each parameter.
The Mechanism: In a deep or wide local minimum, gradients drop significantly.
The Trap: The historical accumulation of v_t forces individual parameter learning rates to shrink exponentially. This leaves the optimizer with too little "energy" to step back up the slope and escape.

(c) **Situational Oscillations and Momentum Cancellation**
Adam utilizes a first-moment vector (m_t) to act like physics-based momentum.
The Mechanism: When entering a local minimum, the optimizer relies on past inertia to carry it through.
The Trap: If the true loss landscape requires a sharp change in direction to find the exit, the calculated step stalls. The opposing forces of historical momentum and the new uphill gradient can cancel each other out, causing Adam to oscillate back and forth at the basin floor until it settles there completely.

(d) **Too Large of a Batch Size (Lack of Stochastic Noise)**
Adam is heavily reliant on stochasticity (randomness) to jump out of local wells.
The Mechanism: Small batch training creates a high-variance, "noisy" loss landscape that naturally bounces the optimizer out of narrow local traps.
The Trap: If your batch size is too large, the loss landscape becomes incredibly smooth and deterministic. Without random perturbations, Adam follows the exact mathematical gradient straight to the bottom of the nearest local minimum and freezes.

(e) **Poor Learning Rate Scheduling**
The Mechanism: Models often start with a high learning rate to explore the landscape and lower it over time to fine-tune.
The Trap: If the initial learning rate (η) is set too low, Adam never develops the speed required to clear initial shallow valleys. Conversely, if the learning rate schedule decays too aggressively, it cuts off Adam's stepping power before it can escape early traps.

(f) **Saturated Activation Functions**
Original LeNet-5 uses sigmoid or tanh activation functions.
The Mechanism: Deep layers can cause vanishing gradients problem.
The Trap: The weights stop updating because gradients approach zero.

(g) **Small Batch Sizes**
Tiny batches introduce extreme gradient noise.
The Mechanism: The Adam optimizer moves erratically across the loss landscape.
The Trap: It can get trapped in sharp, sub-optimal valleys.

(h) **Loss Landscape of LeNet-5**
LeNet-5 has 7 layers in total, it is a shallow, classic architecture.
The Mechanism: It processes an image through 3 convolutional layers and 2 pooling layers then a fully dense connected layer and an output.
The Trap: Shallow networks like LeNet-5 have more high-loss local minima.

How to help Adam escape local minima traps:

(a) **Use Activation Function that Fix Vanishing Gradients Problem:** Replace sigmoid or tanh to ReLU or Leaky ReLU activation functions. The original LeNet-5 uses tanh, which suffer heavily from vanishing gradients. If using standard ReLU, then dead neurons could still stall the updates. Swap out ReLU for LeakyReLU($\alpha = 0.01$) to ensure backpropagation always maintains a small, healthy gradient flow.

(b) **Implement Batch Normalization:** LeNet-5 was originally built without Batch Normalization (BN), but modern implementations benefit greatly from it. Without it, vanishing or exploding internal covariate shifts can stall the gradients. Insert a BN layer on the input image that still have vale in the range of $[0, 255]$ to become $[-1, 1]$.

(c) **Incease Batch Size:** Increase the number of input images processed together in a batch during training to 64 or 128.

(d) **Implement a Learning Rate Warmup and Decay Schedule:** Start with a learning rate then scale it down under certain conditions to prevent v_t from locking up prematurely. High learning rate cause Adam to overshoot, thus making the algorithm skips over the global minimum.
In LeNet-5 particularly with MNIST dataset, we can tune the learning rate within these ranges:

- **0.001 (Recommended baseline):** The default learning rate for Adam optimizes in major architecture. It provides the best balance between fast convergence and stable training.
  Adam's default learning rate (0.001) can sometimes be too large for a simple architecture like LeNet-5, causing the weights to oscillate back and forth over a narrow minimum valley. To fix this drop the learning rate to 0.0001 or 0.0005.
- **0.0005 to 0.0001 (Conservative / Fine-Tuning):** Use this if your loss curve oscillates wildly or if you notice overfitting happening too early.
- **0.002 to 0.005 (Aggressive):** Use this only if you are using large batch sizes (e.g., 256 or higher) and want to speed up training.

For LeNet-5 a learning rate of 0.01 or higher will cause Adam to overshoot, resulting in unstable training or failing to converge entirely.

(e) **Reduce Batch Size:** Force more stochastic noise into the training routine to shake the optimizer out of shallow basins.

(f) **Switch to Cosine Annealing with Restarts:** Periodically spike the learning rate back up during training to forcefully push the model out of local minima.

(g) **Use He/Xavier Initialization:** To initialize weights that is near the optima. **Use a Modern Architecture:** If accuracy plateaus, try to use modern CNN architecture such as ResNet, GoogleNet, EfficientNet, DenseNet, or ConvNeXt.

When LeNet-5 model that is using Adam optimizer hits a stagnant training loss, it means the parameter updates have dropped too low to escape a local minimum, a saddle point, or a flat plateau. The stagnated loss in LeNet-5 with Adam is primarily caused by uncalibrated learning rates or vanishing gradient trajectories.

5. **Results**
   The hyperparameters for the implementation of LeNet-5 architecture for digit recognition are as follows:

   - Data = 100 images
     Training data 60%, testing data 100%
   - Epochs = 20
   - Learning rate

$$\eta = \begin{cases} 0.1, & \text{if } i = 1 \\ 0.06, & \text{if } Loss[i] > 2 \ \& \ |Loss[i] - Loss[i-1]| < 0.1 \\ 0.025, & \text{if } 1 \leq Loss[i] < 2 \\ 0.01, & \text{if } Loss[i] < 1 \end{cases}$$

   with $i = 1, 2, \ldots, 20$. We start with $\eta = 0.1$ and after the first epoch if the loss difference is less than 0.1 then η will be 0.06 and keep decreasing as the loss decreases at certain point. This scheme of learning rate decreasing as the loss decreases really helps and it still depends on the initial weights that are initialized because it can sometimes overshoot and the loss become very high. If the loss become high, we reset and train again.

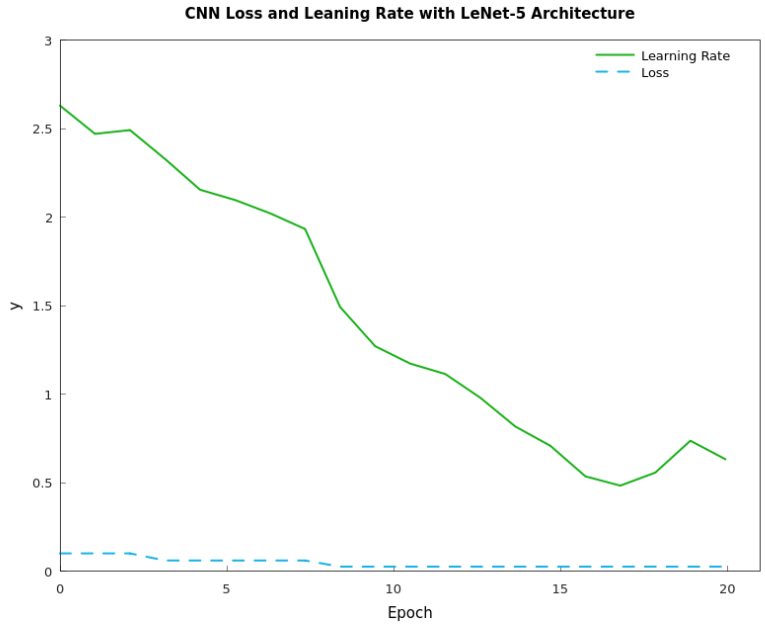

Figure 10.49: *The training loss of LeNet-5 as plotted over 20 epochs along with the learning rate at each epoch. The training loss is monotonically decreasing indicating the network is effectively learning to differentiate between the ten classes in the dataset (**Hamzstplot/Examples with Makefile/Neural Networks/Plot 2D Line for Loss and Learning rate for CNN/main.cpp**).*

Here are two results of training the CNN with LeNet-5 architecture:

```
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Neural Networks/Test CNN using Class with Steepest Descent ]# ./

======== Convolutional Neural Network ========
Loading dataset...
Successfully loaded 100 images!
Dimension of Image : 32x32
DEBUG: Number of labels: 100
DEBUG: Label names:
DEBUG:  - 0
DEBUG:  - 1
DEBUG:  - 2
DEBUG:  - 3
DEBUG:  - 4
DEBUG:  - 5
DEBUG:  - 6
DEBUG:  - 7
DEBUG:  - 8
DEBUG:  - 9
DEBUG: Dataset loaded. Size: 100 images.
DEBUG: Number of labels: 100
DEBUG: Train set size: 60
DEBUG: Test set size: 40

Initializing Convolutional Neural Network with LeNet5 Architecture for Digit Recognition...

Starting Training Process

Training Progress:

[==================================================] 100% Epoch 20/20 - Loss: 0.6317 - Learning Rate: 0.025000000
```

Figure 10.50: *Training the convolutional neural networks with LeNet-5 architecture for handwritten digit recognition, we are using configuration in Table (10.6), all activation functions are Leaky ReLU ($\alpha = 0.8$) and the output layer is using softmax. The whole data is 100 images, we train 60% of the data, we use 20 epochs and it took 1093 seconds till it finishes, we haven't optimize or make the code faster and more efficient by replacing some vectors and matrices operations with either* ***class*** *or* ***struct*** *(**SymIntegration/Examples/Neural Networks/Test CNN using Class with Steepest Descent/main.cpp**).*

```
DEBUG:  - 7
DEBUG:  - 8
DEBUG:  - 9
DEBUG: Dataset loaded. Size: 100 images.
DEBUG: Number of labels: 100
DEBUG: Train set size: 60
DEBUG: Test set size: 40

Initializing Convolutional Neural Network with LeNet5 Architecture for Digit Recognition...

Starting Training Process

Training Progress:

[==================================================] 100% Epoch 20/20 - Loss: 0.6317 - Learning Rate: 0.025000000

Training completed in 1093.48092652 seconds.

Evaluating the model on the test set:
[==================================================] 100%

Accuracy: 73.00%

Confusion Matrix:

      Predicted >       0       1       2       3       4       5       6       7       8       9
                0       8       0       0       0       1       0       0       0       1       0
                1       0       9       0       0       1       0       0       0       0       0
                2       0       0       5       1       2       0       1       1       0       0
                3       0       0       0       7       1       0       0       1       1       0
                4       0       0       0       0       6       0       1       1       0       2
                5       0       0       0       1       0       9       0       0       0       0
                6       0       0       0       0       1       2       6       0       1       0
                7       0       0       0       0       1       0       0       9       0       0
                8       0       0       0       0       0       0       1       0       8       1
                9       0       0       0       1       1       0       1       0       1       6

Precision, Recall, and F1-score:
```

Figure 10.51: *The accuracy after training the convolutional neural networks with LeNet-5 architecture for handwritten digit recognition is 73%, the confusion matrix is also generated after the training is finished, the testing data for confusion matrix is the whole 100 images of handwritten digit (**SymIntegration/Examples/Neural Networks/Test CNN using Class with Steepest Descent/main.cpp**).*

```
Initializing Convolutional Neural Network with LeNet5 Architecture for Digit Recognition...

Starting Training Process

Training Progress:

[==================================================] 100% Epoch 20/20 - Loss: 0.1597 - Learning Rate: 0.02500000

Training completed in 589.67249413 seconds.

Evaluating the model on the test set:
[==================================================] 100%

Accuracy: 76.00%

Confusion Matrix:

      Predicted >              0              1              2              3              4              5              6              7              8              9
                0              9              1              0              0              0              0              0              0              0              0
                1              0             10              0              0              0              0              0              0              0              0
                2              0              1              5              1              2              1              0              0              0              0
                3              0              0              0              8              1              0              1              0              0              0
                4              0              0              1              0              5              0              0              2              2              0
                5              0              0              0              0              0              9              0              0              1              0
                6              0              0              0              0              1              0              9              0              0              0
                7              0              0              0              0              0              0              0             10              0              0
                8              0              0              0              1              0              0              0              0              9              0
                9              0              1              0              0              2              3              0              0              2              2

Precision, Recall, and F1-score:

0:
  Precision: 1.00
  Recall:    0.90
  F1-score:  0.95
```

Figure 10.52: *We try to perform another training for the convolutional neural networks with LeNet-5 architecture for handwritten digit recognition, we obtain the accracy of 76 %. The digit 9 is wrongly predicted as 4, some is predicted as 5 or as 8, we can see this happens becase the shape is almost similar. Confusion matrix plotting is another way of model evalation. It is used for graphical representation of performance of the model. It shows the performance of model in predicting every class (digit 0 to 9) (**SymIntegration/Examples/Neural Networks/Test CNN using Class with Steepest Descent/main.cpp**).*

In a convolutional neural network, accuracy measures how often the model is correct, while loss measures the penalty for incorrect predictions. We use the optimizer like Adam to minimize the loss during training.

We have obtained accuracy of 73% and 76%, when we searched and read on internet and paper, they claim to have 98% of accuracy or even 99%, it is quite misleading. An accuracy of 70-80% on unseen data is almost always better than hitting almost 100%, because the latter usually indicates overfitting, meaning the model has simply memorized the training data rather than actually learning to recognize patterns.

When a CNN achieves near 100% training accuracy but fails in the real world, it has usually memorized the specific noise, lighting conditions, or artifacts of the training dataset. When presented with new, unseen data, it struggles to adapt. While 70-80% accuracy indicates the model has learned general, underlying patterns and can successfully generalize to new situations. Pushing the accuracy to 99% or 100% requires training the model on edge cases, outliers, and dataset anomalies, which actively hurts its ability to perform well on standard, everyday inputs. Rather than obsessing over a raw accuracy percentage, it is often more important to analyze a confusion matrix to understand where the model is making mistakes.

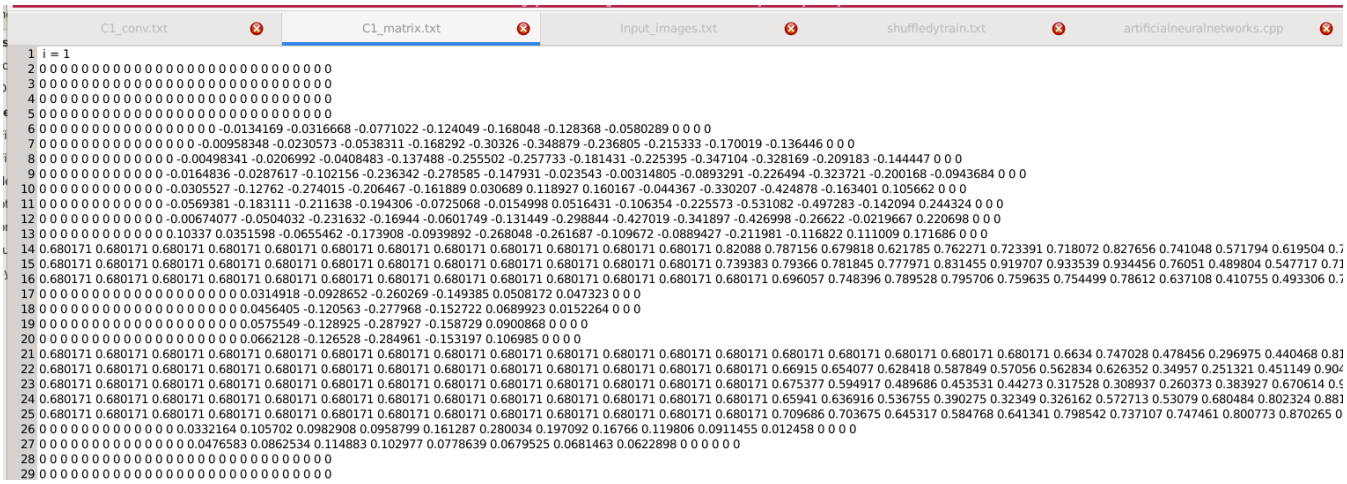

Figure 10.53: *The C1 matrix that represents the first feature map of C1 at* $i = 1$.

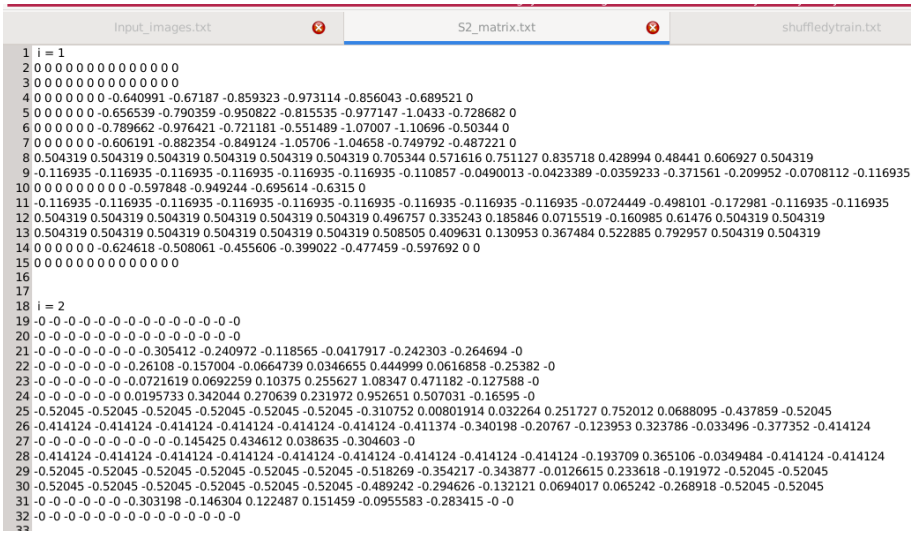

Figure 10.54: *The S2 feature maps after performing average pooling on C1 layer.*

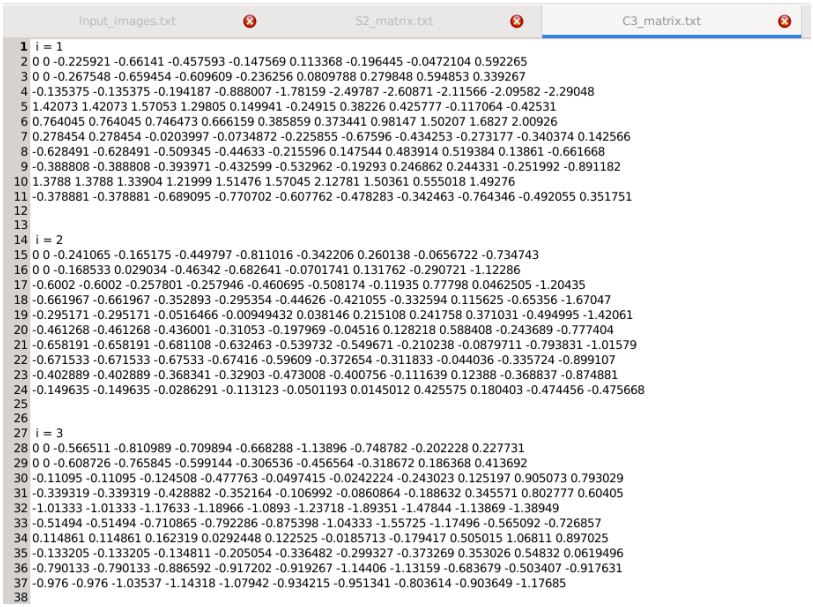

Figure 10.55: *The C3 feature maps after performing convolution operation with sparse connectivity with S2 layer.*

Figure 10.56: *The output vector after we use softmax activation function.*

All the results from C1 to output layer can be saved or choose not to be saved by commenting them in the **src/artificialneuralnetworks.cpp** or commenting them out. The δ computation while training / backpropagating can also be seen and saved as **.txt** files so we can figure out whether the CNNs are training well or have vanishing gradients problem or others.

output_vector.txt | C5_final_vector.txt | F6_delta.txt

```
4.36339e-05
-0.000129459
9.54815e-06
4.08578e-05
9.4857e-05
3.87842e-05
-2.83785e-05
2.22647e-05
3.65675e-05
8.26934e-05
-1.97492e-05
-1.78704e-05
-9.2854e-06
3.92866e-06
-7.68817e-05
-2.00093e-05
3.76887e-05
-1.22074e-05
2.76977e-05
-1.09376e-05
-0.00010593
2.34821e-05
3.73629e-05
-8.27391e-05
9.95176e-05
6.11887e-05
-2.67277e-05
6.37315e-06
-6.71053e-05
-9.90484e-05
8.02027e-05
```

Figure 10.57: *The* δ^{F6}.

S4_deltamap_final.txt | Input_images.txt

```
i = 1
-6.32029e-06 8.95793e-05 -0.00010703 -0.000173552 1.60147e-05
0.000182448 5.66686e-05 -4.33069e-07 7.4924e-05 3.225e-05
4.20137e-05 -3.48339e-05 7.88585e-05 -4.42406e-05 -0.000181813
-9.03876e-05 -3.00092e-05 7.60825e-05 0.000150513 9.20057e-05
0.000203196 -0.00022227 -0.000144309 7.47829e-05 -9.90843e-05


i = 2
-5.42072e-05 5.24281e-06 3.56729e-05 0.000193734 -3.63205e-05
3.73557e-05 -0.000124641 2.59496e-05 -1.64415e-05 -7.37138e-05
0.00011741 1.27629e-05 2.36809e-05 4.49779e-05 -0.000113744
3.74159e-05 -0.000131021 5.12295e-05 -5.45604e-06 -6.24822e-05
-7.59466e-05 4.85688e-05 -6.52174e-05 -1.4011e-05 0.000118302


i = 3
2.17248e-05 2.96477e-05 7.5694e-05 3.765e-05 9.6928e-05
-5.69775e-05 -5.69638e-05 0.00012873 -1.5534e-05 5.12692e-05
-2.56431e-05 1.68554e-05 9.07676e-06 1.55472e-06 3.37963e-05
-3.42228e-05 -7.01426e-06 1.21498e-05 8.4814e-05 2.64127e-05
-6.47918e-05 -2.5804e-05 -2.54201e-05 5.58348e-05 -5.11937e-06


i = 4
-0.000210311 -0.00014297 -0.000246277 -0.000206578 -0.000101722
-2.98123e-05 9.21134e-06 4.89934e-05 8.53895e-05 -7.73975e-05
-7.64518e-05 0.000132129 -1.19056e-05 -7.53743e-05 4.84581e-05
4.51766e-05 0.00023555 7.20006e-05 8.09797e-05 6.49746e-05
8.84252e-05 6.09046e-05 -9.68885e-05 3.98228e-05 3.48768e-06


i = 5
-0.00012443 -0.000219234 -8.10631e-05 -8.22525e-05 -6.02224e-05
-7.71172e-05 3.4899e-06 2.40098e-05 -2.01917e-05 1.46021e-05
-4.3721e-05 -5.73209e-05 4.61834e-05 0.000111961 3.25934e-05
-8.29863e-05 -4.0672e-05 -9.72248e-06 2.3081e-05 -4.80831e-05
-4.34243e-05 -5.76859e-05 -2.32531e-05 9.0909e-06 5.03772e-05

```

Figure 10.58: *The* δ^{S4}.

iv. Functions in SymIntegration Related to Convolutional Neural Network

[SI*] We will list the basic functions that can be used for computing convolutional neural networks:

CNN_2DConvolutionOperation(vector<vector<int>>& *input*, vector<vector<double>>& *kernel*)
this void function will perform a convolution operation of the *input* matrix of size $R \times C$ with *kernel* matrix of size $k \times k, k < \min(R, C)$, the stride is set as 1.

CNN_2DConvolutionOperation(vector<vector<int>>& *input*, vector<vector<double>>& *kernel*, int *stride*)
this function will perform a convolution operation of the *input* matrix of size $R \times C$ with *kernel* matrix of size $k \times k, k < \min(R, C)$, the stride number can be set in this function, the domain for stride is $\in [1, M]$ with M as the $\min(R, C)$.

CNN_2DpadBorder(const vector<vector<int>>& *matrix*, int *pad_size*)
this function will return a 2D matrix of **integer** data type from the padding operation toward *matrix*, we can set the padding size here at *pad_size*.

CNN_2DmaxPooling(const vector<vector<int>>& *input*, int *kernelSize*, int *stride*)
this function returns a 2D matrix of **integer** data type from performing max pooling operation on *input* matrix, we can define the kernel size for the max pooling here at *kernelSize* and the number of stride at *stride*.

CNN_2DaveragePooling(const vector<vector<int>>& *input*, int *kernelSize*, int *stride*)
this function returns a 2D matrix of **integer** data type from performing average pooling operation on *input* matrix, we can define the kernel size for the max pooling here at *kernelSize* and the number of stride at *stride*.

```
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Neural Networks/Te
st CNN Max Pooling, Average Pooling, and Padding Operation ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/SI Test Clustered/Neural Networks/Test CNN Max Pooling, Average Pooling, and Padding Operation
]# ./main

Matrix A :
                    10                    15                    20                    25
                    30                    35                    40                    45
                     5                    10                    15                    20
                    25                    30                    35                    40

Max Pooling :
                    35                    45
                    30                    40

Average Pooling :
             22.500000             32.500000
             17.500000             27.500000

Padding Matrix A :
                     0                     0                     0                     0                     0                     0
                     0                    10                    15                    20                    25                     0
                     0                    30                    35                    40                    45                     0
                     0                     5                    10                    15                    20                     0
                     0                    25                    30                    35                    40                     0
                     0                     0                     0                     0                     0                     0

Time taken by function: 1841 microseconds
```

Figure 10.59: *Performing max pooling, average pooling, and padding operation on input matrix (**SymIntegration/Examples/Neural Networks/Test CNN Max Pooling, Average Pooling, and Padding Operation/main.cpp**).*

CNN_2DUpsample3DMatrix(const vector<vector<vector<double>>>& *input*, int *M*, int *N*)

this function returns a 3D upsampled matrix of **double** data type from *input* matrix.

CNN_2DaverageUpsample3DMatrix(const vector<vector<vector<double>>>& *input*, int *M*, int *N*)
this function returns a 3D average upsampled matrix of **double** data type from *input* matrix and then multiply each entry by $\frac{1}{M*N}$.

```
Matrix A :
--- Slice (Depth) 0 ---
1       2       3
4       5       6
7       8       9

--- Slice (Depth) 1 ---
10      11      12
13      14      15
16      17      18


Upsample Matrix A :
--- Slice (Depth) 0 ---
1       1       2       2       3       3
1       1       2       2       3       3
4       4       5       5       6       6
4       4       5       5       6       6
7       7       8       8       9       9
7       7       8       8       9       9

--- Slice (Depth) 1 ---
10      10      11      11      12      12
10      10      11      11      12      12
13      13      14      14      15      15
13      13      14      14      15      15
16      16      17      17      18      18
16      16      17      17      18      18


Average Upsample Matrix A :
--- Slice (Depth) 0 ---
0.25    0.25    0.5     0.5     0.75    0.75
0.25    0.25    0.5     0.5     0.75    0.75
1       1       1.25    1.25    1.5     1.5
1       1       1.25    1.25    1.5     1.5
1.75    1.75    2       2       2.25    2.25
1.75    1.75    2       2       2.25    2.25

--- Slice (Depth) 1 ---
2.5     2.5     2.75    2.75    3       3
2.5     2.5     2.75    2.75    3       3
3.25    3.25    3.5     3.5     3.75    3.75
3.25    3.25    3.5     3.5     3.75    3.75
4       4       4.25    4.25    4.5     4.5
4       4       4.25    4.25    4.5     4.5


Time taken by function: 2034 microseconds
```

Figure 10.60: *Performing upsampling and average upsampling on input matrix (**SymIntegration/Examples/Neural Networks/Test Upsample 3D Matrix/main.cpp**).*

v. Functions in SymIntegration Related to Image Manipulation with STB Image

The source code in SymIntegration that handles these functions is in **src/stbimage.cpp** with its corresponding header **include/symintegral/stbimage.h**.

[SI*] We will list the basic functions that can be used for image manipulation that will be very useful in convolutional neural network (CNN), the engine behind this is the STB image library (it is a header only library so it is quite light) :

ResizeAndGrayscale(const std::string& inputPath, const std::string& outputPath, int newWidth, int newHeight)
this **void** function resize the images that we have listed as **string** vector then turn them into grayscale. This is the basic to create MNIST dataset for digit recognition, if we would like to recreate 60,000 train images of digits from 0 to 9 with our own handwriting or our peers handwriting this function will be very handy.

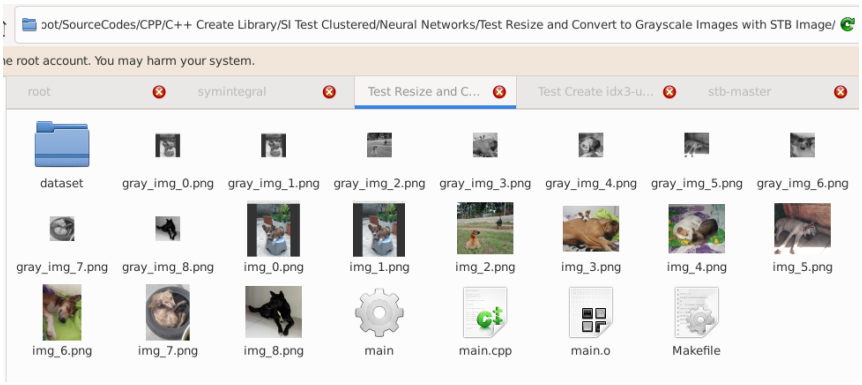

Figure 10.61: *Converting several images with different sizes to uniform size of* 28×28 *then convert them to grayscale (**SymIntegration/Examples/Neural Networks/Test Resize and Convert to Grayscale Images with STB Image/main.cpp**).*

swap_endian(uint32_t val)
this function convert little-endian to big-endian, it is required to create **idx3-ubyte** or **idx1-ubyte** format.
To create an **idx3-ubyte** file (the standard format for MNIST-style image datasets) in C++, you must write a specific binary header followed by the raw pixel data. The format uses big-endian byte ordering, which requires reversing the byte order of integers on most modern (little-endian) systems.
convert_to_idx(const vector<string>& image_paths, const vector<uint8_t>& labels, const string& images_out_path, const string& labels_out_path, int width, int height)
this function will create the **idx3-ubyte** format for the training images, and **idx1-ubyte** for the labels of the images, it corresponds one-to-one, order matters. The input is a **string** vector to represents the path to the images, one by one, we can call them from a **.txt** file or manually write them in the **main.cpp**, then the labels is a **uint8_t** vector to represents the label, for digit recognition we can use the label directly as 0 to 0, 1 to 1, but if it is an image classification that we are creating, then we can represents 0 as "cat", 1 as "dog", 2 as "horse", 3 as "monkey", 4 as "dragon", 5 as "snake", 6 as "rooster", etc.

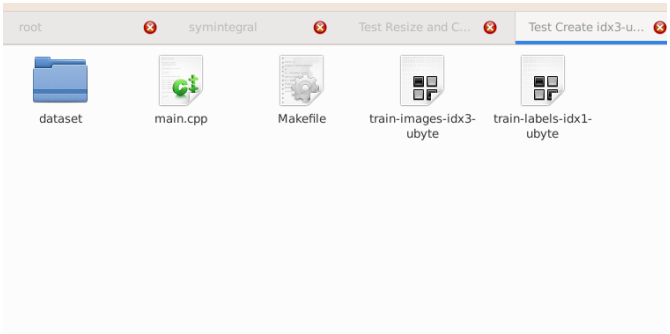

Figure 10.62: *An example to create **idx3-ubyte** and **idx1-ubyte** files from two grayscale images of size* 28×28 *(**SymIntegration/Examples/Neural Networks/Test Create idx3-ubyte Files with STB Image/-main.cpp**).*

An **idx3-ubyte** file consists of:

- **Magic Number (4 bytes):** Always **0x00000803** for 3D unsigned byte tensors.
- **Number of Items (4 bytes):** The total count of images.
- **Number of Rows (4 bytes):** Height of each image.
- **Number of Columns (4 bytes):** Width of each image.
- **Data:** Raw pixel bytes (0-255) for all images, ordered sequentially.

* Pixel values should be unsigned 8-bit integers (**uint8_t**) ranging from 0 to 255.

the **idx** file format is a method for storing numerical data in the form of vectors and multi-dimensional matrices.

loadIDX3(const std::string& filename)
this function will load idx3-ubyte format and convert it to become 3D matrix of **integer** data type (**vector<vector<vector<int>>>**).

loadIDX1(const std::string& filename)
this function will load idx1-ubyte format and convert it to become a vector of **integer** data type (**vector<int>**).

images.txt | labels.txt

```
1 i = 1
2 229 235 232 233 234 232 231 231 233 234 234 232 231 234 230 233 231 231 234 232 232 230 235 240 242 239 236 237
3 154 157 155 155 159 177 181 184 185 185 184 179 177 170 171 167 168 176 183 187 189 170 167 161 161 159 157 174
4 50 50 50 50 55 101 112 115 113 110 115 114 107 97 103 80 84 98 114 128 141 98 60 50 50 50 48 85
5 50 50 50 51 56 96 106 108 110 107 109 105 97 104 98 98 101 104 115 117 126 102 68 50 50 50 48 87
6 50 50 50 50 55 85 92 93 89 90 95 95 99 111 95 106 113 111 118 117 104 99 55 49 50 50 48 86
7 50 50 50 50 53 71 74 73 72 76 79 80 86 91 81 88 94 93 92 89 78 66 62 52 50 50 48 87
8 50 50 50 50 51 57 58 58 58 61 64 66 79 66 54 67 76 82 77 61 70 60 79 55 50 50 48 87
9 50 50 50 50 51 52 50 51 62 72 73 73 87 84 84 83 87 81 90 118 105 87 93 54 50 51 48 87
10 50 49 50 50 54 75 86 88 84 79 81 84 92 102 100 94 75 69 97 101 88 89 78 53 50 50 48 87
11 50 50 50 50 58 118 133 139 120 78 73 73 83 133 92 86 76 68 92 73 75 76 65 52 50 50 48 87
12 50 50 50 50 64 156 182 182 157 83 70 75 79 143 160 142 116 95 124 84 84 94 88 55 50 50 48 87
13 50 50 50 50 64 156 181 184 169 141 107 86 77 127 179 176 176 173 171 145 132 112 94 54 50 50 48 87
14 50 50 50 50 65 163 189 179 100 117 163 151 112 80 131 171 174 159 162 167 141 102 81 53 50 50 48 87
15 50 50 50 50 68 186 209 196 102 71 110 109 103 66 80 140 148 127 118 153 138 106 91 53 50 50 48 87
16 50 50 50 50 70 196 215 211 164 80 88 89 93 72 77 99 128 133 140 142 132 102 90 53 50 50 48 87
17 50 50 50 50 70 198 217 216 216 147 81 114 86 103 111 109 119 103 102 117 113 92 85 53 50 50 48 87
18 49 50 50 50 70 198 223 226 224 199 138 142 119 111 98 98 91 85 70 109 133 103 83 53 50 50 48 87
19 50 50 50 50 70 199 226 226 224 220 152 95 111 80 87 89 80 87 100 173 200 174 125 55 50 50 48 87
20 50 50 49 50 72 213 233 220 226 222 163 114 113 84 119 124 98 99 148 204 208 198 161 58 50 50 48 87
21 50 50 50 50 72 213 234 224 225 223 141 84 75 101 192 155 140 177 207 210 205 217 176 60 50 50 48 87
22 50 50 51 50 71 207 233 231 230 215 135 108 112 143 179 180 201 207 199 164 184 214 178 61 50 50 48 87
23 50 50 49 50 70 200 227 227 222 199 129 118 127 153 144 132 143 144 138 131 165 217 179 61 50 50 48 87
24 50 50 50 50 68 186 210 215 215 179 129 124 126 128 116 120 130 133 135 132 147 211 193 62 50 50 48 87
25 50 50 50 50 67 190 218 208 204 164 106 116 123 120 116 121 132 132 134 129 137 193 178 62 50 50 48 87
26 50 50 50 50 69 199 221 212 199 180 116 83 115 122 120 121 131 128 113 87 132 198 180 61 50 50 48 87
27 51 50 50 50 70 204 223 218 202 190 170 115 97 117 119 123 125 106 88 109 166 209 182 61 50 50 48 87
28 48 48 47 47 67 199 222 220 217 203 188 171 131 119 113 117 117 115 143 170 193 205 183 58 48 48 47 85
29 93 93 93 92 104 205 233 230 231 227 215 208 199 185 170 164 169 181 199 202 209 207 196 101 92 93 91 121
30
31
32 i = 2
33 227 235 231 232 234 232 231 231 232 234 234 233 230 231 231 231 231 232 234 231 232 229 239 241 243 237 238
34 123 127 125 125 129 154 162 166 166 165 165 163 160 149 149 145 143 147 162 165 177 165 141 135 128 128 128 127
35 48 48 48 48 51 94 110 112 112 107 111 113 106 96 102 88 80 86 106 115 135 132 76 56 48 47 49 48
36 50 50 50 50 53 89 105 106 109 105 106 106 96 104 103 93 107 100 115 112 122 116 94 57 50 50 50 50
37 50 50 50 51 53 80 89 90 86 85 91 93 94 106 107 91 119 104 116 115 111 98 80 46 50 50 50 50
38 51 50 50 50 52 67 71 71 70 73 77 77 81 87 81 73 91 86 94 79 82 68 63 63 50 50 50 50
39 50 50 50 50 51 55 55 56 56 59 61 63 75 76 56 64 75 79 83 70 74 67 71 78 50 50 50 50
40 50 50 50 50 50 53 51 51 61 74 76 78 83 91 86 85 87 82 82 107 122 95 94 79 49 50 50 50
41 50 50 50 50 52 80 93 97 91 80 78 81 84 104 101 99 84 67 81 102 91 84 88 69 50 50 50 50
```

Figure 10.63: *An example to create a 3D integer matrix from **idx3-ubyte** format and an integer vector from **idx1-ubyte** format then save them as separated text files (**SymIntegration/Examples/Neural Networks/Test Create 3D Matrix from idx3-ubyte File and Vector from idx1-ubyte/main.cpp**).*

Chapter 11

Computational Fluid Dynamics

"Om Mani Padme Hum." - ***Tibetan Buddhist Chanting***

Mankind has been seeking to harness the power of fluids since the early ages in its attempts to utilize it for various utilities such as irrigation in the form of the aqueducts built by the Romans, windmills for milling, ballistics in artillery, and so on. However, engineering these systems relied on rudimentary techniques, which were often trial-and-error experiments based on highly simplified models and empirical observations. Advancing to the 18th century the mathematical model for fluid flow was conceived by David Bernoulli, laying the foundation for studying fluid flows in a systematic and quantitative manner.
This led to various innovations in engineering, helping build hydraulic systems such as pumps and turbines, designing ventilation systems, early aircraft systems, etc. Although designing such systems was now feasible, they relied heavily on manual calculations, experimental data, and simplified assumptions. This required lots of tedious work and manpower, leading to inefficiencies and multiple points of vulnerability. The Wright brothers required wind tunnel testing due to the idealized noninviscid assumption of Bernoulli's equations, whereas flow analysis for pipe networks in the real world required empirical formulas such as the Darcy-Weisback equations, since Bernoulli's equations neglected losses due to friction. Until the mid-20th century, many such drawbacks pushed back on innovation, which was, until the advent of computer technology. The rise of computing technology combined with mathematical models describing the underlying physical phenomena of fluid flows, kick-started the field of computational fluid dynamics (CFD) [1].

I. Introduction

[SI*] CFD is an amalgamation of various disciplines such as numerical methods, computational algorithms, and principles of physics, for the study and simulation of fluid flows. Over the past few decades, complex systems such as jet turbines and internal combustion engines greatly benefited from the enhancements in this multidisciplinary field, leading to increased efficiencies in performance and energy utilization. Early models, such as the finite difference method (FDM), were used to discretize the governing equations into solvable numerical grids in various applications. The finite element method (FEM) was later developed for solving fluid problems and structural mechanics in complex geometries, where the structure under study was subdivided into simpler elements like triangular or quadrilateral sections. In the 1960s the finite volume method (FVM) transformed the field of CFD by subdividing

the problem into small cells and applying the laws of conservation across these volumes. With complex models such as Reynolds-averaged Navier-Stokes equations, chaotic turbulent models could be comprehensively studied.

Leaps in this domain helped humanity in achieving significant milestones: in 1960s NASA uses CFD in modeling parts of the Apollo spacecraft; in 1970s Boeing adopted CFD for aerodynamic analysis and design of aircraft, decreasing their reliance on wind tunnel systems; and supersonic and hypersonic flows were modeled by CFD to aid in the development of Lockheed Martin's SR-71 aircraft, etc. Evidently, CFD has been crucial in scientific breakthroughs, and these systems are still extensively used in the analysis and design of fluid systems. With advances in parallel computing in the 1990s, large-scale simulations could be tackled with methods such as large eddy simulation (LES) and direct numerical simulation (DNS). This led to applications not only in aerospace but in the automotive industry for combustion simulation and engine cooling, medicine for simulation of blood flow in arteries and design of microfluid devices, environmental science for weather pattern prediction and climate change modeling, etc.

Employing CFD, industries have been able to optimize the design workflows, enhance performance, and streamline costs across automotive, aerospace, energy, and environmental sectors. However, as the simulation models increase in complexity and the demand for higher computational power grows, traditional methods become inefficient. Therefore data-driven techniques, or so-called artificial intelligence (AI) methods, have entered into play by providing faster and often accurate solutions.

II. Navier-Stokes

[SI*] The Navier-Stokes equations describe how fluids (liquids and gases) move. They are essentially Newton's second law of motion ($F = ma$) applied to fluids, balancing inertia, pressure, viscosity, and external forces like gravity. It was developed in the 19th century by Claude-Louis Navier and George Gabriel Stokes. The equations are paired with the continuity equation, which ensures that fluid mass is neither created nor destroyed. Because the equations are non-linear and produce complex, chaotic turbulence, mathematicians have never found a general analytical solution.

The key components:

- **Inertia/Acceleration:** How the fluid speeds up or slows down.
- **Pressure Gradients:** Fluids flow naturally from high-pressure areas to low-pressure areas.
- **Viscosity (Internal Friction):** How "thick" or sticky a fluid is (e.g., water flows easily; honey resists flow).
- **Body Forces:** External forces like gravity or electromagnetism.

For an incompressible Newtonian fluid (like water with constant density), the momentum Navier-Stokes equation is written as:

$$\rho\left(\frac{\partial \boldsymbol{u}}{\partial t} + \boldsymbol{u}\cdot\nabla\boldsymbol{u}\right) = -\nabla p + \mu\nabla^2\boldsymbol{u} + \boldsymbol{f} \tag{11.1}$$

paired with the continuity equation (conservation of mass):

$$\nabla\cdot\boldsymbol{u} = 0 \tag{11.2}$$

with

- ρ as the fluid density.
- $\rho\frac{\partial \boldsymbol{u}}{\partial t}$ as the local acceleration (how fluid velocity changes over time at a specific point).
- $\rho(\boldsymbol{u} \cdot \nabla \boldsymbol{u})$ as the convective acceleration (how fluid velocity changes because the fluid is moving to a new position).
- ρg as the body forces, such as gravity pulling the fluid downward.
- $\boldsymbol{u}$ as the velocity vector of the fluid.
- t as the time.
- $-\nabla p$ as the pressure gradient force (pushes fluid from high-pressure areas to low-pressure areas).
- $\mu\nabla^2\boldsymbol{u}$ as the viscous force term (friction within the fluid layers, where μ is the dynamic viscosity).
- $\boldsymbol{f}$ as the external body forces (such as gravity).

The term $\rho\boldsymbol{u} \cdot \nabla \boldsymbol{u}$ represents convective acceleration, which makes the equations non-linear and difficult to solve.

[SI*] Applications of Navier-Stokes equations:

- **Aerodynamics:** Calculating airflow and drag over airplane wings, cars, and rockets.
- **Weather Forecasting:** Modeling global wind patterns, ocean currents, and storm movements.
- **Medicine & Biology:** Simulating blood flow through arteries, veins, and cardiac devices.
- **Engineering & Industry:** Designing oil pipelines, water supply networks, HVAC airflow, and chemical reactors.
- **Computer Graphics:** Animators and game developers use simplified versions via Computational Fluid Dynamics (CFD) to render realistic water splashes, smoke, and fire in movies.

i. Couette Flow

[SI*] A classic and simplest example of using the Navier-Stokes is Couette flow where fluid trapped between two infinite parallel flat plates, where the bottom plate is stationary and the top plate moves at a constant velocity U.

1. **Identify the geometry and direction:** The flow is 1D, moving in the x-direction driven by the moving top plate ($y = h$).
2. List of assumptions:
   Steady-state flow: time derivative $\frac{\partial u}{\partial t} = 0$.
   No pressure gradient in the x-direction: $\frac{\partial p}{\partial x} = 0$.
   No gravity or external body forces in the x-direction.
   The flow only varies vertically: velocity $\boldsymbol{u}$ depende only on $y(u = u(y))$, and vertical velocity $\boldsymbol{v} = \boldsymbol{0}$.
3. **Simplify the Navier-Stokes Equation:** The x-component momentum equation reduces down drastically:

$$\mu\frac{d^2u}{dy^2} = 0$$

4. **Integrate Twice:**

$$\frac{du}{dy} = C_1$$
$$u(y) = C_1 y + C_2$$

5. **Apply Boundary Conditions:**
   At the bottom wall ($y = 0$), velocity $u = 0 \rightarrow C_2 = 0$.
   At the top wall ($y = h$), velocity $u = U \rightarrow C_1 = \frac{U}{h}$.
6. **Final Solution (Velocity Profile):**

$$u(y) = \frac{U}{h} y$$

This linear profile shows that the fluid speed increases steadily from zero at the bottom plate to U at the top plate. For complex real-world geometries (like airplane wings or weather systems), these equations cannot be solved by hand and are instead approximated using Computational Fluid Dynamics (CFD) software.

ii. Poiseuille Flow

[SI*] Poiseuille flow (Hagen-Poiseuille flow)is a classic exact solution to the Navier-Stokes equations for steady, laminar incompressible, and axisymmetric viscous flow through a long cylindrical pipe.

iii. Couette and Poiseuille Flow Solver Simulation Using a Second-Orderr Runge-Kutta (RK2) with SFML 2.5.1

We create a simulation that models the fluid between two plates using the 1D Navier-Stokes momentum equation simplified for parallel flow:

$$\frac{\partial u}{\partial t} = \nu \frac{\partial^2 u}{\partial y^2}$$

We optimize the numerical accuracy of the solver, by using a second-order accurate Runge-Kutta 2nd Order (RK2 / Heun's method) scheme. This substantially reduces numerical diffusion errors.

To solve the transient Couette flow problem using a second-order accurate Runge-Kutta(RK2) method, we combine spatial discretization (Method of Lines) with an RK2 time-stepping scheme.

1. **Governing Equation & Discretization**
   The transient 1D Couette flow is governed by the viscous diffusion equation:

$$\frac{\partial u}{\partial t} = \nu \frac{\partial^2 u}{\partial y^2}$$

   where $u(y, t)$ is the fluid velocity, t is time, y is the spatial coordinate perpendicular to the walls, and ν is the kinematic viscosity.

Spatial Discretization
Using a second-order central difference scheme on a uniform grid with spacing $\triangle y$:

$$\frac{\partial^2 u_i}{\partial y^2} \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{(\triangle y)^2}$$

This transforms the PDE into a system of Ordinary Differential Equations (ODEs) for each internal grid point i:

$$\frac{du_i}{dt} = f_i(u) = \frac{\nu}{(\triangle y)^2}(u_{i+1} - 2u_i + u_{i-1})$$

2. **RK2 Formulas**
There are two common variations of the second-order Runge-Kutta method: Heun's method(explicit trapezoidal) and the Midpoint method. Both are second order accurate $O(\triangle t^2)$.

Heun's Method (Predictor-Corrector)
Predictor step (Euler step to $t + \triangle t$):

$$k_1 = f(u^n)$$
$$u^* = u^n + \triangle t \cdot k_1$$

Corrector step (averages slopes at t and $t + \triangle t$):

$$k_2 = f(u^*)$$
$$u^{n+1} = u^n + \frac{\triangle t}{2}(k_1 + k_2)$$

Midpoint Method
Predictor step (Euler step to the midpoint $t + \frac{\triangle t}{2}$):

$$k_1 = f(u^n)$$
$$u^{n+\frac{1}{2}} = u^n + \frac{\triangle t}{2} \cdot k_1$$

Corrector step (evaluates slopes at the midpoint):

$$k_2 = f(u^{n+\frac{1}{2}})$$
$$u^{n+1} = u^n + \triangle t \cdot k_2$$

For stability, the time step must satisfy the diffusion CFL condition: $\triangle t \leq \frac{(\triangle y)^2}{2\nu}$ (though RK2 has slightly stricter stability limits than forward Euler, keeping $\triangle t$ well below this limit ensures stability).

The algorithm workflow to implement Couette flow in C++ code:

1. **Initialization**
Define domain length H, kinematic viscosity ν, and grid size N (number of points). Then calculate spatial step size

$$\triangle y = \frac{H}{N-1}$$

Choose a stable time step $\triangle t$ and total simulation time T.
Set up the spatial grid vector y from 0 to H.
Set boundary conditions:

(a) Bottom wall ($y = 0$): $u_0 = 0$ (Stationary)

(b) Top wall ($y = H$):$u_{N-1} = U_{wall}$ (Moving plate velocity)

Set initial condition: $u_i = 0$ for all internal nodes ($i = 1$ to $N - 2$).

2. **Time-Stepping Loop**
For each time step from $t = 0$ to T with increment $\triangle t$:

(a) **Calculate k_1**
For each internal node $i = 1$ to $N - 2$:

$$k_{1,i} = \frac{\nu}{(\triangle y)^2}(u_{i+1}^n - 2u_i^n + u_{i-1}^n)$$

(b) **Predict intermediate velocities (u^* or $u^{n+\frac{1}{2}}$)**
Using Heun's method:

$$u_i^* = u_i^n + \triangle t \cdot k_{1,i}$$

apply boundary conditions to the intermediate array

$$u_0^* = 0, \quad u_{N-1} = U_{wall}$$

(c) **Calculate k_2**
Evaluate the spatial derivatives using the intermediate array:

$$k_{2,i} = \frac{\nu}{(\triangle y)^2}(u_{i+1}^* - 2u_i^* + u_{i-1}^*)$$

(d) **Update to next time step (u^{n+1})**
For each internal node $i = 1$ to $N - 2$ we will have

$$u_i^{n+1} = u_i^n + \frac{\triangle t}{2}(k_{1,i} + k_{2,i})$$

enforce boundary conditions explicitly with

$$u_0^{n+1} = 0, \quad u_{N-1}^{n+1} = U_{wall}$$

(e) **Advance time**
Set $t = t + \triangle t$. Check for steady state if desired (e.g., when $\max|u^{n+1} - u^n| < \epsilon$).

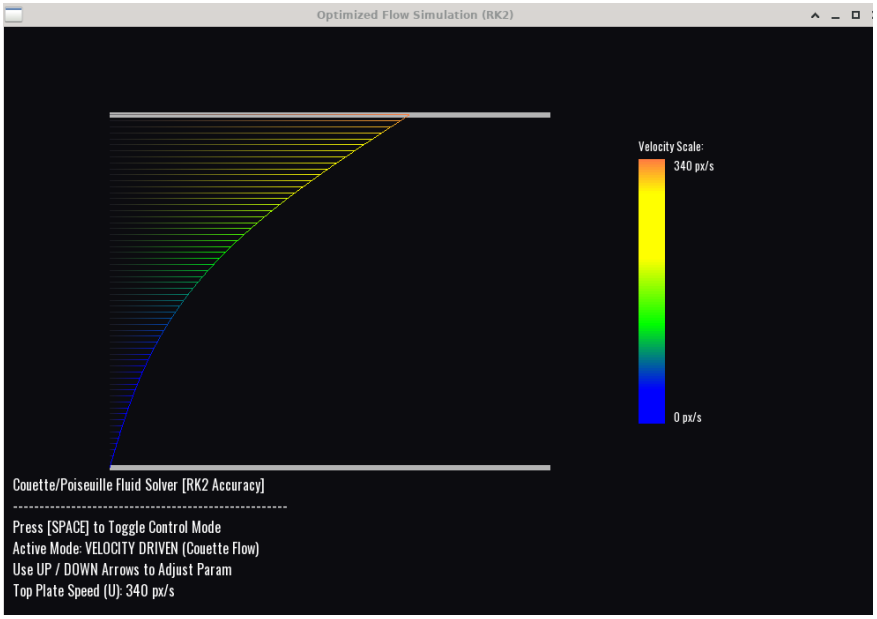

Figure 11.1: *The simulation of Couette flow, we can use the up and down arrow keys to dynamically change the top plate's velocity for the Couette flow. By pressing spacebar the simulation mode will change between pure veloccity (Couette flow) and an interactive Pressure Gradient ($\triangle P$) indicator mode. In the pressure mode, the flow profile changes into an archetypal parabolic curve (Poiseuille-like channel configuration) while maintaining the plate velocity conditions. (**SymIntegration/Examples/Computational Fluid Dynamics/Navier Stokes Couette Poiseuille Flow 2D with RK2 with SFML/main.cpp**).*

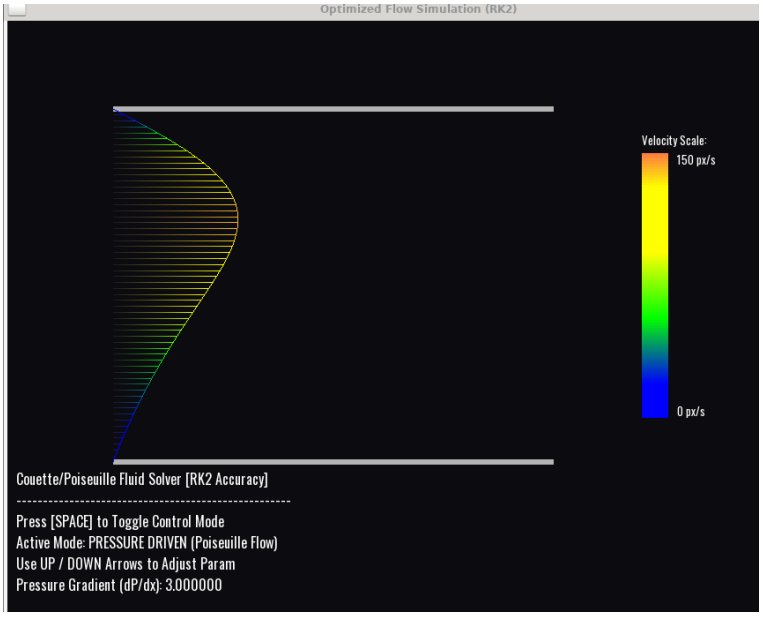

Figure 11.2: *The simulation of Poiseuille flow. (**SymIntegration/Examples/Computational Fluid Dynamics/-Navier Stokes Couette Poiseuille Flow 2D with RK2 with SFML/main.cpp**).*

Chapter 12

Introduction to Theory of Interest and Financial Mathematics

*"But in my world, everything what would be.. it wouldn't be, and what wouldn't be.. it would." (Alice in Wonderland 1951) - **Alice***

Mathematical finance, also known as quantitative finance and financial mathematics, is a field of applied mathematics, concerned with mathematical modeling of financial markets.

In general, there exist two separate branches of finance that require advanced quantitative techniques: derivatives pricing on the one hand, and risk and portfolio management on the other. Mathematical finance overlaps heavily with the fields of computational finance and financial engineering. The latter focuses on applications and modeling, often by help of stochastic asset models, while the former focuses, in addition to analysis, on building tools of implementation for the models. Also related is quantitative investing, which relies on statistical and numerical models (and lately machine learning) as opposed to traditional fundamental analysis when managing portfolios.

French mathematician Louis Bachelier's doctoral thesis, defended in 1900, is considered the first scholarly work on mathematical finance. But mathematical finance emerged as a discipline in the 1970s, following the work of Fischer Black, Myron Scholes and Robert Merton on option pricing theory. Mathematical investing originated from the research of mathematician Edward Thorp who used statistical methods to first invent card counting in blackjack and then applied its principles to modern systematic investing.

I. Bond Pricing

[SI*] A bond is a debt instrument requiring the issuer (also called the debtor or borrower) to repat to the lender/investor the amount borowed plus interest over a specified period of time.

A typical ("plain vanilla") bond issued in the United States specifies:

1. A fixed date when the amount borrowed (the principal) is due.
2. The contractual amount of interest, which typically is paid every six months.

The date on which the principal is required to be repaid is called the maturity date. Assuming that the issuer does not default or redeem the issue prior to the maturity date, an investor holding this bond until the maturity date is assured of a known cash flow pattern [11].

[SI*] For a variety of reasons, since 1980s there was a wide range of bond structures being developed. In the residential mortgage market particularly, new types of mortgage designs were introduced. The practice of pooling of individual mortgages to form mortgage pass-through securities grew dramatically. Using the basic instruments in the mortgage market (mortgages and mortgage pass-through securities), issuers created derivative instruments such as collateralized mortgage obligations and stripped mortgage-backed securities that met specific investment needs of a broadening range of institutional investors.

[SI*] The U.S. bond market is the largest bond market in the world. The market is divided into six sectors:

1. **U.S. Treasury sector**
   This sector includes securities issued by the U.S. government. These securities include Treasury bills, notes, and bonds. The U.S. Treasury is the largest issuer of securities in the world. This sector plays a key role in the valuation of securities and the determination of interest rates throughout the world.
2. **The agency sector**
   This sector includes securities issued by federally related institutions and government sponsored enterprises. The securities issued are not backed by any collateral and are referred to as agency debenture securities.
3. **The municipal sector**
   This sector is where the state and local governments and their authorities raise funds. The two major sectors within the municipal sector are general obligation sector and the revenue sector. Bonds issued in this sector typically are exempt from federal income taxes.
4. **The corporate sector**
   It includes securities issued by U.S. corporations and non U.S. corporations issued in the United States. The latter securities are referred to as Yankee bonds. The corporate sector is divided into the investment grade and noninvestment grade sectors.
5. **Asset-backed securities sector**
   Int his sector, a corporate issuer pools loans or receivables and uses the pool of assets of collateral for the issuance of a security.
6. **The mortgage sector**
   It is the sector where securities are backed by mortgage loans. These are loans obtained by borrowers in order to purchase residential property or an entity to purchase commercial propety.

[SI*] These are some important features of bonds:

1. **Type of issuer**
   A key feature of a bond is the nature of the issuer. Each issuer has different abilities to satisfy their contractual obligation to lenders.
2. **Term to maturity**
   The term to maturity of a bond is the number of years over which the issuer has promised to meet the conditions of the obligation.
   There are three reasons why the term to maturity of a bond is important. The most obvious is that it indicates the time period over which the holder of the bond can expect

to receive the coupon payments and the number of years before the principal will be paid in full. The second reason is that the yield on a bond depends on it. Finally, the price of a bond will fluctuate over its life as yields in the market change, the volatility of a bond's price is dependent on its maturity.

3. **Principal and coupon rate**
   The principal value of a bond is the amount that the issuer agrees to repay the bondholder at the maturity date. This amount is also referred to as the redemption value, maturity value, par value, or face value.
   The coupon rate, also called the nominal rate, is the interest that the issuer agrees to pay each year. The annual amount of the interest payment made to owners during the term of the bond is called the coupon. The coupon rate multiplied by the principal of the bond provides the dollar amount of the coupon.
4. **Amortization feature**
   The principal repayment of a bond issue can call for either the total principal to be repaid at maturity or the principal repaid over the life of the bond. In the latter case, there is a schedule of principal repayments. This schedule is called an amortization schedule. Loans that have this feature are automobile loans and home mortgage loans.
5. **Embedded options**
   It is common for a bond issue to include a provision in the indenture that gives either the bondholder and/or the issuer an option to take some action against the other party. The most common type of option embedded in a bond is a call feature. This provision grants the issuer the right to retire the debt, fully or partially, before the scheduled maturity date. Inclusion of a call feature benefits bond issuers by allowing them to replace an old bond issue with a lower-interest cost issue if interest rates in the market decline. A call provision effectively allows the issuer to alter the maturity of a bond. A call provision is detrimental to the bondholder's interests.

   An issue may also include a provision that allows the bondholder to change the maturity of a bond. An issue with a put provision included in the indenture grants the bondholder the right to sell the issue back to the issuer at par value on designated dates. Here the advantage to the investor is that if interest rates rise after the issue date, thereby reducing a bond's price, the investor can force the issuer to redeem the bond at par value.

   A convertible bond is an issue giving the bondholder the right to exchange the bond for a specified number of shares of common stock. Such a feature allows the bondholder to take advantage of favorable movements in the price of the issuer's common stock. An exchangeable bond allows the bondholder to exchange the issue for a specified number of common stock shares of a corporation different from the issuer of the bond.

[SI*] Risks associated with investing in bonds:

1. **Interest-rate risk**
   The price of a typical bond will change in the opposite direction from a change in interest rates: As interest rates rise, the price of a bond will fall; as interest rates fall, the price of a bond will rise. If an investor has to sell a bond prior to the maturity date, an increase in interest rates will mean the realization of a capital loss (selling the bond below the purchase price). This risk is referred to as interest-rate risk or market risk. This risk is by far the major risk faced by an investor in the bond market.

2. **Reinvestment risk**
   Calculation of the yield of a bond assumes that the cash flows received are reinvested. The additional income from such reinvestment, sometimes called interest-on-interest, depends on the prevailing interest-rate levels at the time of reinvestment, as well as on the reinvestment strategy. Variability in the reinvestment rate of a given strategy because of changes in market interest rates is called reinvestment risk. Reinvestment risk is greater for longer holding periods, as well as for bonds with large, early, cash flows, such as high-coupon bonds.

3. **Call risk**
   Many bonds include a provision that allows the issuer to retire or "call" all or part of the issue before the maturity date. The issuer usually retains this right in order to have flexibility to refinance the bond in the future if the market interest rate drops below the coupon rate.
   From the investor's perspective, there are three disadvantages to call provisions. First, the cash flow pattern of a callable bond is not known with certainty. Second, because the issuer will call the bonds when interest rates have dropped, the investor is exposed to reinvestment risk. Finally, the capital appreciation potential of a bond will be reduced, because the price of a callable bond may not rise much above the price at which the issuer will call the bond.

4. **Default risk**
   Also referred as credit risk, refers to the risk that the issuer of a bond may default (i.e., will be unable to make timely principal and interest patments on the issue). Default risk is gauged by quality ratings assigned by four nationally recognized rating companies: Moody's Investors Service, Standard & Poor's Corporation, Duff & Phelps Credit Rating Company, and Fitch IBCA, as well as the credit research staffs of securities firms.
   Because of this risk, bonds with default risk trade in the market at a price that is lower than comparable U.S. Treasury securities, which are considered free of default risk.

5. **Inflation risk**
   This purchasing-power risk arises because of the variation in the value of cash flows from a security due to inflation, as measured in terms of purchasing power. For example, if investors purchase a bond on which they can realize a coupon rate of 6% but the rate of inflation is 10%, the purchasing power of the cash flow actually has declined.

6. **Exchange-rate risk**
   A non-dollar-denominated bond (i.e., a bond whose payments occur in a foreign currency) has unknown U.S. dollar cash flows. The dollar cash flows are dependent on the exchange rateat the time the payments are received. For example, suppose that an investor purchases a bond whose payments are in Japanese Yen / JPY. If the JPY depreciates relative to the USD, fewer dollars will be received. The risk of this occurring is referred to as exchange-rate or currency risk. Of course, should the JPY appreciate relative to the USD, the investor will benefit by receiving more dollars.

7. **Liquidity risk**
   Liquidity or marketability risk depends on the case with which an issue can be sold at or near its value. The primary measure of liquidity is the size of the spread between the bid and the ask price quoted by a dealer. The wider the dealer spread, the more the liquidity risk. For an investor who plans to hold the bond until the maturity date, liquidity risk is less important.

8. **Volatility risk**
   The risk that a change in volatility will affect the price of a bond adversely. The price of a bond with certain types of embedded options depends on the level of interest rates and factors that influence the value of the embedded option. One of these factors is the expected volatiliy of interest rates. Specifically, the value of an option rises when expected interest-rate volatility increases. In the case of a bond that is callable or a mortgage-backed security, in which the investor has granted the borrower an option, the price of the security falls, because the investor has given away a more valuable option.

[SI*] The notion that money has a time value is one of the basic concepts in the analysis of any financial instrument. Money has time value because of the opportunity to invest it at some interest rate.

Future Value
To determine the future value of any sum of money invested today, we can use this equation

$$P_n = P_0(1+r)^n \tag{12.1}$$

where n is the number of periods, P_n is the future value, P_0 is the original principal, and r is the interest rate per period.

Present Value
This time we will show how to determine the amount of money that must be invested today in order to realize a specific future value. This amount is called the present value. The formula is

$$PV = P_n \left[\frac{1}{(1+r)^n} \right] \tag{12.2}$$

the term in brackets is the present value of USD 1.

In most applications in portfolio management a financial instrument will offer a series of future values. To determine the present value of a series of future values, the present value of each future value must first be computed. Then these present values are added together to obtain the present value of the entire series of future values. The formula can be expressed as follows:

$$PV = \sum_{t=1}^{n} \frac{P_t}{(1+r)^t} \tag{12.3}$$

Present Value of an Ordinary Annuity
When the same dollar amount of money is received each period or paid each year, the series is referred to as an annuity. When the first payment is received one period from now, the annuity is called an ordinary annuity. When the first payment is immediate, the annuity is called an annuity due. For bond, we shall deal with ordinary annuities.

To compute the present value of an ordinary annuity, the present value of each future value can be computed and then summed. Alternatively, a formula fo the present value of an ordinary annuity can be used

$$PV = A \left[\frac{1 - \frac{1}{(1+r)^n}}{r} \right] \tag{12.4}$$

[SI*] **Pricing a Bond**

The price of any financial instrument is equal to the present value of the expected cash flows from the financial instrument. Therefore determining the price requires

1. An estimate of the expected cash flows
2. An estimate of the appropriate required yield

The expected cash flows for some financial instruments are simple to compute; for others, the task is more difficult. The required yield reflects the yield for financial instruments with comparable risk, or alternative (or substitute) investments.

The first step in determining the price of a bond is to determine its cash flows. The cash flows for a bond that the issuer cannot retire prior to its stated maturity date (i.e., a noncallable bond) consist of

1. Periodic coupon interest payments to the maturity date
2. The par (or maturity) value at maturity

The required yield is determined by investigating the yields offered on comparable bonds in the market. By comparable, we mean noncallable bonds of the same credit quality and the same maturity. The required yield typically is expressed as an annual interest rate. When the cash flows occur semiannually, the market convention is to use one-half the annual interest rate as the periodic interest rate with which to discount the cash flows.

The price of a bond is the present value of the cash flows, it is determined by adding these two present values:

1. The present value of the semiannual coupon payments
2. The present value of the par or maturity value at the maturity date

In general, the price of a bond can be computed using the following formula:

$$P = \frac{C}{1+r} + \frac{C}{(1+r)^2} + \frac{C}{(1+r)^3} + \cdots + \frac{C}{(1+r)^n} + \frac{M}{(1+r)^n} \tag{12.5}$$

or

$$P = \sum_{t=1}^{n} \frac{C}{(1+r)^t} + \frac{M}{(1+r)^n} = C\left[\frac{1-\frac{1}{(1+r)^n}}{r}\right] + \frac{M}{(1+r)^n} \tag{12.6}$$

where
$P =$ the bond price
$n =$ the number of periods
$C =$ the coupon paid every period
$r =$ the interest rate per period
$M =$ maturity value
$t =$ time period when the payment is to be received

[SI*] As yields in the marketplace change, the only variable that can change to compensate an investor for the new required yield in the market is the price of the bond. When the coupon rate is equal to the required yield, the price of the bond will be equal to its par value.

When yields in the marketplace rise above the coupon rate at a given point in time, the price of the bond adjusts so that an investor cintemplating the purchase of the bond can

realize some additional interest. If it did not, investors would not buy the issue because it offers a below market yield; the resulting lack of demand would cause the price to fall and thus the yield on the bond to increase. This is how a bond's price fall below its par value.

The capital appreciation realized by holding the bond to maturity represents a form of interest to a new investor to compensate for a coupon rate that is lower than the required yield. When a bond sells below its par value, it is said to be selling at a discount. When the required yield is greater than the coupon rate, the price of the bond is always lower than the par value.

When the required yield in the market is below the coupon rate, the bond must sell above its par value. This is because investors who have the opportunity to purchase the bond at par would be getting a coupon rate in excess of what the market requires. As a result, investors would bid up the price of the bond because its yield is so attractive.the price would eventually be bid up to a level where the bond offers the required yield in the market. A bond whose price is above its par value is said to be selling at a premium.

Discount Bond: coupon rate < required yield $\iff$ price < par
Par Bond: coupon rate = required yield $\iff$ price = par
Premium Bond: coupon rate > required yield $\iff$ price > par

i. Bond Pricing Computation

Compute the bond price for a 20-year coupon bond paid semiannually with a 10% coupon rate and a par or maturity value of USD 1,000, for the required yield of 11% coupon rate.

Solution:

The cash flows for this bond are as follows:

1. 40 semiannual coupon payments of USD 50.

$$C = \frac{100}{2} = 50$$

2. A USD 1,000 cash flow 40 six-months periods from now.

$$M = 1000$$

The semianuual or periodic interest rate (or periodic require yield) is $\frac{11\%}{2} = 5.5\%$.

The present value of the 40 semiannual coupon payments of USD 50 discounted 5.5% is

$$\begin{aligned} PV_{coupon} &= C\left[\frac{1-\frac{1}{(1+r)^n}}{r}\right] \\ &= 50\left[\frac{1-\frac{1}{(1.055)^{40}}}{0.055}\right] \\ &= 50\left[\frac{1-\frac{1}{(8.51332)}}{0.055}\right] \\ &= 50\left[\frac{1-0.117463}{0.055}\right] \\ &= 50[16.04613] \\ &= 802.31 \end{aligned}$$

The present value of the par or maturity value of USD 1,000 received 40 six-months periods from now, discounted at 5.5% is

$$PV_M = \frac{M}{(1+r)^n} = \frac{1000}{(1.055)^{40}} = 117.46$$

the price of the bond is then equal to the sum of the two present values above

$$P = PV_{coupon} + PV_M = 919.77$$

The bond price with 11% required yield is USD 919.77.

The computation can be done with SymIntegration, it has the function **bondpricing(C,M,r,n)** to compute the bond price with formula from Eq. (12.6).

We also include a table of price-yield relationship for the 20-year bond with 10% coupon bond paid semiannually.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Present Value and Bond Pricing ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Present Value and Bond Pricing ]# ./main

A 20-year 10% coupon bond with yield 11% and semiannual coupon payments
Present Value of Coupon payments = 802.306
Present value of par (maturity value) = 117.463
Bond Price = 919.769

Price-Yield Relationship for a 20-year 10% coupon bond

Yield            Price
0.045            1720.32
0.05             1627.57
0.055            1541.76
0.06             1462.3
0.065            1388.65
0.07             1320.33
0.075            1256.89
0.08             1197.93
0.085            1143.08
0.09             1092.01
0.095            1044.41
0.1              1000
0.105            958.531
0.11             919.769
0.115            883.503
0.12             849.537
0.125            817.696
0.13             787.817
0.135            759.752
0.14             733.366
0.145            708.533
0.15             685.14
0.155            663.081
0.16             642.262
0.165            622.592

Time taken by function: 976 microseconds
```

Figure 12.1: *The computation for the price of a 20-year coupon bond paid semiannually with a 10% coupon rate and a par or maturity value of USD 1,000, for the required yield of 11% coupon rate (**SymIntegration/Examples/Financial Mathematics/Test SymIntegration Present Value and Bond Pricing/main.cpp**).*

II. Mortgage Loans

[SI*] A mortgage is a loan secured by the colalteral of specified real estate property, which obliges the borrower to make a predetermined series of payments. The mortgage gives the lender (the mortgage) the right of foreclosure on the loan if the borrower (the mortgager) defaults. That is, if the borrower fails to make the contracted payments, the lender can seize the property eo ensure that the debt is paid off.

[SI*] A potential homeowner who wants to borrow funds to purchase a home will apply for a loan from a mortgage originator. Upon completion of the application form, which provides financial information about the applicant, and payment of an application fee, the mortgage originator will perform a credit evaluation of the applicant. The two primary factors in determining whether the funds will be lect are the payment-to-income (PTI) ratio and the loan-to-value (LTV) ratio.

The PTI, the ratio of monthly payments (both mortgage and real estate tax payments) to monthly income, is a measure of the ability of the applicant to make monthly payments. The lower this ratio, the greater the likelihood that the applicant will be able to meet the required payments. The difference between the purchase price of the property and the amount borrowed is the borrower's down payment.

The LTV is the ratio of the amount of the loan to the market (or appraised) value of the property. The lower this ratio, the more protection the lender has if the applicant defaults and the property must be repossessed and sold.

[SI*] Every mortgage loan must be serviced. Servicing of a mortgage loan involves collecting monthly payments and forwarding proceeds to ownersof the loan, sending payment notices to mortgagors; reminding mortgagors when payments are overdue, maintaining records of principal balances; administering an escrow balance for real estate taxes and insurance purposes, intiating foreclosure proceedings if necessary, and furnishing tax information to mortgagors when applicable.

Mortgage servicers include bank-related entities, thrift-related entities, and mortgage bankers.

[SI*] To compute the mortgage payment every period, we use the amortization method. The amortization method is the method where the borrower repays the lender by means of installemnt payments at periodic intervals.

The formula to compute mortgage payment is derived from the present value formula:

$$MP = MB_0 \left[\frac{i(1+i)^n}{(i+1)^n - 1} \right] \tag{12.7}$$

where:
MP = mortgage payment every period
n = number of periods (months, years)
MB_0 = original mortgage balance
i = periodic interest rate

The term in brackets is called the payment factor or annuity factor.

We will represent the present value by v with formula

$$v = \frac{1}{1+i} \tag{12.8}$$

The formula to compute the interest that has to be paid at period-jth is:

$$interest_j = (1 - v^{n-j+1})MP \tag{12.9}$$

with $j = 1, 2, \ldots, n$.
The formula to compute the principal that has to be paid at period-jth is:

$$principal_j = (v^{n-j+1})MP \tag{12.10}$$

with $j = 1, 2, \ldots, n$.

i. Mortgage Loan Computation

Compute the amortization table for a 30-year (360-month) of USD 100,000 monthly mortgage (fixed-rate mortgage) with an 8.125% mortgage rate.

Solution:

Each monthly mortgage payment for a level-payment mortgage is due on the first of each month and consists of

1. Interest of one-twelfth of the fixed annual interest rate times the amount of the outstanding mortgage balance at the beginning of the previous month.
2. A repayment of a portion of the outstanding mortgage balance (principal).

The difference between the monthly mortgage payment adn the portion of the payment that represents interest equals the amount that is applied to reduce the outstanding mortgage balance. The monthly mortgage payment is designed so that after the last scheduled monthly payment of the loan is made, the amount of the outstanding mortgage balance is zero (the mortgage is fully repaid).

At the beginning of month 1, the mortgage balance is USD 100,000, the amount of the original loan. The mortgage payment for month 1 includes interest on the USD 100,000 borrowed for the month.

The monthly payment is

$$MP = MB_0 \left[\frac{i(1+i)^n}{(i+1)^n - 1} \right] = 100000 \left[\frac{0.0067708(1.0067708)^{360}}{(1.0067708)^{360} - 1} \right] = 742.50$$

Because the interest is 8.125%, the monthly interest rate is $\frac{0.08125}{12} = 0.0067708$. Interest for month 1 is therefore

$$interest_1 = (1 - v^{n-1+1})MP = \left(1 - \left(\frac{1}{1.0067708} \right)^{360} \right) 742.50 = 677.08$$

the difference between the monthly mortgage payment of USD 742.50 and the interest of USD 677.08 is the portion of the monthly mortgage payment that represent repayment of principal.

$$principal_1 = (v^{n-1+1})MP = \left(\left(\frac{1}{1.0067708} \right)^{360} \right) 742.50 = 65.41$$

We can also use $principal_1 = MP - interest_1$ formula. This USD 65.41 in month 1 reduces the mortgage balance.

In SymIntegration we have a function that will show the amortization table till the last period, it is:
amortization(L,r,n)

with L as the total loan / mortgage, r as the interest rate per period, and n is the number of periods.

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Amortization ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Amortization ]# ./main

Amortization

Period               Payment              Interest             Principal      Outstanding Loan Balance
1                    742.497               677.083               65.4138              99934.6
2                    742.497                676.64               65.8568              99868.7
3                    742.497               676.195               66.3027              99802.4
4                    742.497               675.746               66.7516              99735.7
5                    742.497               675.294               67.2035              99668.5
6                    742.497               674.839               67.6586              99600.8
7                    742.497               674.381               68.1167              99532.7
8                    742.497               673.919               68.5779              99464.1
9                    742.497               673.455               69.0422              99395.1
10                    742.497               672.987               69.5097               99325.6
11                    742.497               672.517               69.9803               99255.6
12                    742.497               672.043               70.4541               99185.1
13                    742.497               671.566               70.9312               99114.2
14                    742.497               671.086               71.4114               99042.8
15                    742.497               670.602                71.895               98970.9
16                    742.497               670.115               72.3817               98898.5
17                    742.497               669.625               72.8718               98825.6
18                    742.497               669.132               73.3652               98752.3
19                    742.497               668.635                73.862               98678.4
20                    742.497               668.135               74.3621               98604.1
21                    742.497               667.632               74.8656               98529.2
22                    742.497               667.125               75.3725               98453.8
23                    742.497               666.614               75.8828               98377.9
24                    742.497               666.101               76.3966               98301.5
25                    742.497               665.583               76.9139               98224.6
26                    742.497               665.063               77.4346               98147.2
27                    742.497               664.538               77.9589               98069.2
28                    742.497                664.01               78.4868               97990.7
29                    742.497               663.479               79.0182               97911.7
30                    742.497               662.944               79.5532               97832.2
```

Figure 12.2: *The amortization table for a* 30*-year (*360*-month) of USD 100,000 monthly mortgage (fixed-rate mortgage) with an* 8.125% *mortgage rate(**SymIntegration/Examples/Financial Mathematics/Test SymIntegration Amortization for Mortgage/main.cpp**).*

```
328                   742.497               148.227               594.271               21297.7
329                   742.497               144.203               598.294               20699.4
330                   742.497               140.152               602.345                 20097
331                   742.497               136.074               606.424               19490.6
332                   742.497               131.968                610.53               18880.1
333                   742.497               127.834               614.663               18265.4
334                   742.497               123.672               618.825               17646.6
335                   742.497               119.482               623.015               17023.6
336                   742.497               115.264               627.233               16396.3
337                   742.497               111.017                631.48               15764.9
338                   742.497               106.741               635.756               15129.1
339                   742.497               102.437               640.061                 14489
340                   742.497               98.1029               644.394               13844.6
341                   742.497               93.7398               648.757               13195.9
342                   742.497               89.3472                653.15               12542.7
343                   742.497               84.9248               657.572               11885.2
344                   742.497               80.4725               662.025               11223.1
345                   742.497                 75.99               666.507               10556.6
346                   742.497               71.4772                671.02               9885.62
347                   742.497               66.9339               675.563               9210.05
348                   742.497               62.3597               680.137               8529.92
349                   742.497               57.7546               684.743               7845.17
350                   742.497               53.1184               689.379                7155.8
351                   742.497               48.4507               694.046               6461.75
352                   742.497               43.7514               698.746                  5763
353                   742.497               39.0203               703.477               5059.53
354                   742.497               34.2572                708.24               4351.29
355                   742.497               29.4618               713.035               3638.25
356                   742.497                24.634               717.863               2920.39
357                   742.497               19.7735               722.724               2197.66
358                   742.497                 14.88               727.617               1470.05
359                   742.497               9.95345               732.544               737.504
360                   742.497               4.99351               737.504                     0

Total payment:
267299

Time taken by function: 17046 microseconds
```

Figure 12.3: *The amortization table for a* 30*-year (*360*-month) of USD 100,000 monthly mortgage (fixed-rate mortgage) with an* 8.125% *mortgage rate(**SymIntegration/Examples/Financial Mathematics/Test SymIntegration Amortization for Mortgage/main.cpp**).*

III. Financial Derivative: Forwards, Futures, Swaps, and Options

[SI*] A financial derivative is an instrument that is related tos ome other asset and whose value is derived from that asset. There are many types of derivatives. Forward/futures and option

contracts are two of the most widely used financial derivatives.

[SI*] A forward or futures contract on an underlying asset is an agreement between two parties to sell and buy that asset at a specified future date for a specified price.

An option on an asset is a contract that allows, but does not require, the holder of the option to buy the asset (call option) or sell the asset (put option) on or before a specified date at a specified price.

[SI*] A financial position is any combination of investments, including long and short positions in any assets, or in risk-free investing or borrowing. When describing the outcome at time T of a financial position created at time 0 we make a distinction between the payoff at time T of the financial position and the profit from time 0 to time 4t4 on the financial position.

The payoff is simply the value of the asset at time T, say S_T. If the cost of the asset at time 0 was S_0, then the accumulated cost to time T of creating the long position at time 0 is $S_0(1+i)^T$, where i, is the (annual effective) risk-free rate of interest. The profit at time T, is

$$S_T - S_0(1+i)^T$$

Definition 12.1: Forward Contract

A forward contract is an agreement to buy or sell a certain asset at a specified future date called the delivery date, for a specific price called the delivery price. Both parties of the forward contract are bound by the contract terms at the time the contract is made, with the contract being settled on the delivery date.

[SI*] In general, the forward contract is constructed at time 0 so that no money changes hands at time 0, and the only exchange takes place at time T.

We will denote by S_0 the value of the asset at time 0, and S_T denote its value at time T. The value at time 0 (current price) is also referred to as the spot price. The spot price is the price quoted for imemdiate settlement, payment and delivery of a security or commodity. For a forward contract arranged at time 0 for delivery to take place at time T, we will denote the delivery price by $F_{0,T}$. The payoff on the long position at the time of delivery is

$$S_T - F_{0,T}$$

and the payoff on the short position at the time of delivery is

$$F_{0,T} - S_T$$

Since no money is initially invested for a forward contract, the profit on the delivery date is the same as the payoff on the delivery date. As with any buy/sell transaction, there are two positions that are taken on a forward contract. The party that will take delivery of the asset and pay the delivery price at time T has a long position on the forward contract, and the party that will deliver the asset and be paid at time t has a short position on the contract.

i. Options

[SI*] Options are derivative financial instruments in which two parties contractually agree to transact an asset at the strike price before or on an expiration date. The option contract

gives the buyer the right, but not the obligation, to buy or sell the underlying asset. Options constitute a binding contract with strictly defined terms and properties including the strike price, options premium, and expiration date.
An option is a derivative because its price is intrinsically linked to the price of the underlying asset. In trading, context options can be purchased and traded via brokerage investment accounts. These can be used for various hedging strategies, to help generate income, for speculative purposes and as executive compensation. In a corporate context, executive options are written by the company to their executive team. Options do not require the investor to buy or sell the underlying asset if they do not wish to. Having the obligation to buy or sell assets in the future is termed 'futures.'

[SI*] Options are typically classified by style into European, American, ermudan, or by type into Call and Put options. A 'Call' option is the right to buy the underlying asset at a certain price. A 'Put' option is the right to sell the underlying asset at a certain price.
At a given time before the expiration date an option can be 'Out of the Money' (OTM), 'At the Money' (ATM) or 'In the Money' (ITM). Options can be traded using various long or short strategies, a short position without ownership of the underlying can lead to unlimited losses.

[SI*] European, American or Bermudan options all have different obligations. European options can only be exercised on the action expiration date, American options can be exercised on any trading day on or before expiration date, and Bermudan options are effectively a hybrid, as they can be exercised on specified days or before expiration date.

- **Call Options**
  a call is an agreement that gives the buyer the right, but not the obligation to buy a stock or any other security at the strike price on or before the expiration date. The risk to the call option is limited to the premium paid for the option, which doesn't include the price of the underlying asset.
  Call option buyers believe the share price will rise above the strike price before the option's expiry (hence purchasing the 'right' to buy it at a set price in the future). If this happens, the investor can exercise the option, buy the stock at the strike price K. Subject to trading costs, they could then immediately sell the stock at the current market price for a profit. The profit on this trade is the market share price minus the strike share price minus the premium and any brokerage commission to place the orders.
- **Put Options**
  A put option is an option contract giving the buyer the right, but not the obligation to sell a specified amount of an underlying security at the strike price on or before expiration date. The buyer believes the stock's market price will fall below the strike price (hence buying the 'right' to sell it at a set price in the future). The put option is profitable when the underlying stock's price is below the strike price. If the prevailing market price is less than the strike price at expiry, the investor can exercise the put by selling shares at the option's higher strike price. The profit on this trade is the strike price subtracted with the current market price, the premium and any brokerage commission.

- If the strike price is identical to the price of the underlying security, the option is at the money (ATM).
- If the call strike price is below and a put strike price above the underlying security, the option is in the money (ITM).

- If the call strike price is above and the put strike price is below the price of the underlying security, the option is out of the money (OTM).

[SI*] With options, buying or holding a call or put option is considered a 'long' position. Conversely, selling or writing a call or put option is a 'short' position; the writer must sell to or buy from the long position holder or buyer of the option. Covered options are contracts sold by traders who actually own the underlying shares. In contrast, in the case of uncovered or "naked" options the writer does not own the underlying assets. Writers of naked options are unprotected and open themselves up for unlimited loss.

- **Long Call:** You buy call options, bullish strategy.
- **Short Call:** You sell call options, bearish strategy.
- **Long Put:** You buy put options, bearish strategy.
- **Short Put:** You sell put options, bullish strategy.

IV. Econometric Models

[SI*] Econometric models are statistical and mathematical frameworks that use real-world data to test economic theories, measure causal relationship, and forecast future economic trends. Unlike purely theoretical or deterministic economic models, econometric models incorporate a stochastic (error) term to account for real-world randomness, unmeasured influences, and data noise.

Key types of econometric models:

- **Linear Regression (OLS):** The foundational model used to measure a direct linear relationship between a dependent variable (y) and one or more independent variable (x) using Ordinary Least Squares.
- **Generalized Linear Models (GLM) &Binary Choice Models (Probit / Logit):** Used when the outcome variable is categorical or binary (e.g., whether a person is employed or a loan defaults).
- **Time Series Models (ARIMA, VAR):** Designed for sequential data collected over time (such as GDP, inflation, or stocks prices) to capture trends, seasonality, and dynamic feedback loops.
- **Panel Data Models:** Tracks the same cross-section of entities (individuals, firms, or countries) across multiple time periods, combining time-series and cross-sectional data advantages.
- **Simultaneous Equations & Instrumental Variable (IV):** Used to solve problems like endogeneity, feedback loops, and reverse causality where explanatory variables are correlated with the error term.

Common use cases:

- **Policy Evaluation:** Central banks and governments use structural macro-econometric models to simulate the impact of interest rate changes or fiscal stimulus.
- **Demand Forecasting:** Retail and corporate sectors analyze historical sales, pricing, and consumer behavior to predict future demand and inventory needs.
- **Causal Inference:** Researchers use techniques like Propensity Score Matching or instrumental variables to isolate the true causal effect of a program (e.g., the effect of education years on future wages).

[SI*] Core components of an econometric model:

1. **Economic Theory / Hypothesis:** The initial framework proposing how variables interact (e.g., consumer spending depends on disposable income).
2. **Mathematical Formulation:** The structural equation defining the deterministic relationship (e.g. $y = \beta_1 + \beta_2 x$).
3. **Stochastic Error Term (u):**The random disturbance added to form the econometric model ($y = \beta_1 + \beta_2 + u$), epresenting omitted or unmeasurable factors.
4. **Data Estimation:** Applying software packages to estimate parameter values (β) from sample data.
5. **Statistical Inference & Testing:** Evaluating the validity of the results using metrics like R-squared, p-value, t-tests, and null-hypothesis testing.

i. Time Series Models

[SI*] **ARMA(p,q)**

An ARMA(p,q) (Autoregressive Moving Average) model is a tool used in time series analysis and forecasting that combines two parts:

1. **AR(p)-Autoregressive part of order p:** The current value depends linearly on its own past values (lags).
2. **MA(q)-Moving Average part of order q:** The current value depends linearly on past white noise error terms (shocks).

In standard notaiton, an ARMA(p,q) model for a time series X_t is written as

$$X_t = \mu + \sum_{i=1}^{p} \phi_i X_{t-i} + \sum_{j=1}^{q} \theta_j \epsilon_{t-j} + \epsilon_t \tag{12.11}$$

with

- X_t is the value of the time series at time t.
- μ is a constant or intercept term (often related to the mean of the series).
- $\phi_1, \phi_2, \ldots, \phi_p$ are the parameters (coefficients) of the Autoregressive (AR) component.
- p is the order of the AR part, representing how many past time steps (X_{t-i}) affect the current value.
- $\theta_1, \theta_2, \ldots, \theta_1$ are the parameters (coefficients) of the Moving Average (MA) component.
- q is the order of the MA part, representing how many past error terms (ϵ_{t-j}) affect the current value.
- ϵ_t is the white noise error term at time t, assumed to be independent and identically distributed (i.i.d.) with mean 0 and variance σ^2.

ARMA(p,q) model written using backshift operator (L) notation

$$\Phi(L)X_t = c + \Theta(L)\epsilon_t \tag{12.12}$$

where

- $\Phi(L)X_t = 1 - \phi_1 L - \phi_2 L^2 - \cdots - \phi_p L^p$ is the autoregressive polynomial.
- $\Theta(L) = 1 + \theta_1 L + \theta_2 L^2 + \cdots + \theta_q L^q$ is the moving average polynomial.
- $L^k X_t = X_{t-k}$

Key requirements and properties:

1. **Stationarity:** The time series must be weakly stationary (its mean, variance, and autocorelation do not change over time). If the data has a trend, it must be differenced first (leading to an ARIMA model).
2. **Causality (Stationarity condition for AR part):** The roots of the AR polynomial $\phi(z) = 0$ must lie outside the unit circle ($|z| > 1$).
3. **Invertibility (MA condition):** The roots of the MA polynomial $\theta(z) = 0$ must lie outside the unit circle ($|z| > 1$), ensuring a unique representation and that remote past errors have diminishing weights.
4. **Model Identification:** Analysts use the ACF (Autocorrelation Function) and PACF (Partial Autocorrelation Function) plots to help determine the optimal orders of p and q.

[SI*] **ARIMA(p,q)**

[SI*] **GARCH(p,q)**

ii. Predicting Crude Oil and Natural Gas with ARMA(1,1), ARIMA(1,1,1), and GARCH(1,1) Visualized by SFML

We will use historical monthly spot prices data for Henry Hub natural gas and West Texas Intermediate (WTI) crude oil starting from January 2000 to October 2026. Grounded in economic indicators from the FRED Data Series.
The file outlines over two decades of global shifts in the energy sector:

- **The 9/11 Terror Attacks & War on Terror (2001):** The catalyst for a paradigm shift in international security, launching the War on Terror and subsequent U.S. interventions in Afghanistan (2001) and Iraq (2003), structural risk premiums became embedded in Middle East energy assets.
- **The China Demand Super-Cycle (2003-2008):** Following China's late-2001 WTO accession, rapid industrialization created massive commodity demand. This collided with tight global spare capacity, driving WTI crude oil from roughly USD 30 in 2003 to an all-time record peak of USD 145.31 per barrel in July 2008.
- **The Hurrican Katrina/Rita Volatility (2005):** Captures the massive infrastructure disruption that sent monthly spot averages of natural gas to an early peak of USD 13.42 per MMBtu in October 2005. It is one of the costliest natural disasters in the U.S. history, Katrina devastated offshore production infrastructure and refining hubs in the Gulf Coast, knocking out 1.38 million barrels per day of oil production and forcing coordinated emergency releases from strategic reserves.
- **The Great Recession & Global Financial Crisis (2007-2009):** The structural collapse of the U.S. subprime mortgage market triggered the bankcruptcy of Lehman Brothers in September 2008. The ensuing financial contagion caused global asset values to crash; crude oil plummeted from its USD 145 peak back down to under USD 40 in a matter of months as economic demand collapsed.
- **The Arab Spring & Libyan Civil War (2010-2011):** Mass popular uprisings across the Middle East and North Africa reshaped regional political layouts. The eruption of the Libyan Civil War in 2011 completely halted its light sweet crude exports, driving Brent crude oil back above USD 120 per barrel.

- **The Shale Boom & Excess Supply (2012-2016):** Tracks the structural decline domestic prices as horizontal drilling expanded production and hydraulic fracturing unlocked vast tight oil reserves in the U.S. Permian Basin, sending the natural gas price down to the USD 1.50 - USD 2.50 range. This massive structural supply surge broke OPEC's pricing leverage, creating a global oversupply "glut" that crashed oil prices from over USD 100 down to a USD 26 per barrel by early 2016.
- **The Brexit Referendum & Geopolitical Friction (2016):** The United Kingdom's unexpected vote to leave the European Union introduced a long-term trade frictions and regulatory uncertainty across the European markets.
- **The COVID-19 Energy Collapse (April 2020):** Lockdowns, travel bans, and economic standstills caused global demand to fall off a cliff. In an unprecedented market anomaly on April 20, 2020, WTI futures crashed into negative territory, hitting -USD 37.63 per barrel because the physical storage infrastructure was full.
- **The Russian Invasion of Ukraine & Euopean Energy Crisis (February 2022):** The invasion fundamentally disrupted European energy security. Aggressive sanctions and pipeline blocks triggered a massive price spike, driving global oil past USD 120 and forcing Europe to rapidly structurally shift its long-term natural gas reliance away from Russia. It maps the sharp international supply shocks that pushed the natural gas price up over USD 8.80 pe MMBtu in late summer 2022.
- **The Middle East Crisis & Strait Hormuz Block (February 2026):** Military conflict directly involved Iran and targeted regional infrastructure. The crisis effectively blocked maritime transit through the critical Strait of Hormuz-the chink point controlling 25% of global seaborne crude and 20% of liquefied natural gas (LNG). This structural disruption pushed Brent oil back to USD 100 per barrel and renewed international inflation risks.
- **Recent 2026 Resurgence:** Details the major winter squeeze from early 2026 (peaking at a monthly average of USD 7.72 in January) followed by a correction to USD 3.12 per MMBtu as of mid-October 2026 due to comfortable storage limits for natural gas.

The interactive macro-economic dashboard displaying WTI oil and Henry Hub gas price charts synced onto a unified timeline from 2000 to 2026. The implementaiton maps out 9 structural world events (including 9/11, the 2008 Lehman Brothers collapse, the COVID-19 drop, and the 2026 Strait of Hormuz conflict) as responsive dashed crosshair markers that hide or display dynamically according to the zoom scale that is chosen.

We will use three time series model to predict the WTI oil and Henry Hub gas, they are ARMA(1,1), ARIMA(1,1,1), and GARCH(1,1):

1. **ARMA(1,1)**
   This model is a standard time series model that forecasts or explains a variable (X_t) by combining a single past value of the variable itself with a single past forecast error (shock). The formula for an ARMA(1,1) process is

   $$X_t = \mu + \phi X_{t-1} + \epsilon_t + \theta \epsilon_{t-1}$$

   or using the Lag operator (L)

   $$(1 - \phi L) X_t = \mu + (1 + \theta L) \epsilon_t$$

   with

   - X_t is the value of the time series at the current time step t.

- μ is a constant term representing the baseline level of the proecss (the long-term mean is $\frac{\mu}{1-\phi}$, provided $\varphi \neq 1$).
- φ is the Autoregressive coefficient, AR(1), it measures the dependence of the current value X_t on its imemdiate predecessor X_{t-1} (lag 1).
- X_{t-1} is the value of the time series at the previous time step $t-1$.
- ϵ_t is the white noise error, the current random shock or error term at time t, assumed to be white noise with mean zero and variance $\sigma_\epsilon^2 (\epsilon\ N(0, \sigma_\epsilon^2))$.
- θ is the moving average coefficient, MA(1), it measures the impact of the past random shock ϵ_{t-1} on the current value X_t.
- ϵ_{t-1} is the random shock or error term from the previous time step $t-1$.

For an ARMA(1,1) model to behave properly in time series analysis, it must satisfy two main theoretical conditions:

(a) **Stationarity (Causality)**
It requires that $|\varphi| < 1$, this ensures that the effects of past shocks die out over time and the series has a stable mean and variance.

(b) **Invertibility**
It requires that $|\theta| < 1$, this ensures that the model can be uniquely inverted and represented as an infinite autoregressive process.

Chapter 13

Dynamical System

"Chaos would suit me just fine." - ***Yuber (Suikoden III)***

I. Double Pendulum

[SI*] In physics and mathematics, a double pendulum, also known as a chaotic pendulum, is a pendulum with another pendulum attached to its end, forming a complex physical system that exhibits rich dynamics behavior with a strong sensitivity to initial conditions. The motion of a double pendulum is governed by a pair of coupled ordinary differential equations and is chaotic.

[SI*] There are several variants of the double pendulum; the two limbs may be of equal or unequal length and masses, they may be simple pendulums or compound pendulums (complex pendulums) and the motion may be in three dimensions or restricted to one vertical plane.

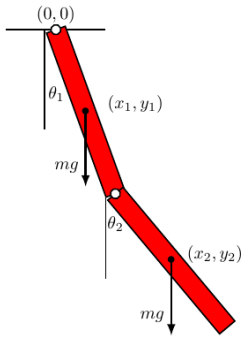

Figure 13.1: *The illustration of double compound pendulum .*

In the following analysis the limbs are taken to be identical compound pendulums of length l and mass m, and the motion is restricted to two dimensions.

In a compound pendulum, the mass is distributed along its length. If the double pendulum mass is evenly distributed, then the center of mass of each limb is at its midpoint, and the limb has a moment of inertia of

$$I = \frac{1}{12} m l^2$$

about that point.

Although it is possible to derive the equations of a double pendulum with Newtonian mechanics, it is considered to be cumbersome to work with as it would require resolving vectors with respect to constraint forces. So it is more convenient to use the angles between each limb and the vertical as the generalized coordinates defining the configuration of the system. These angles are denoted θ_1 and θ_2. The position of the center of mass of each rod may be written in terms of these two coordinates. If the origin of the Cartesian coordinate system is taken to be at the point of suspension of the first pendulum, then the center of mass of this pendulum is at:

$$x_1 = \frac{1}{2} l \sin\theta_1$$
$$y_1 = -\frac{1}{2} l \cos\theta_1$$

and the center of mass of the second pendulum is at

$$x_2 = l\left(\sin\theta_1 + \frac{1}{2}\sin\theta_2\right)$$
$$y_2 = -l\left(\cos\theta_1 + \frac{1}{2}\cos\theta_2\right)$$

This is enough information to write out the Lagrangian.

[SI*] **Lagrangian for Double Compound Pendulum**

The Lagrangian is given by

$$\begin{aligned} L &= \text{kinetic energy} - \text{potential energy} \\ &= \frac{1}{2} m(v_1^2 + v_2^2) + \frac{1}{2} I(\dot{\theta}_1^2 + \dot{\theta}_2^2) - mg(y_1 + y_2) \\ &= \frac{1}{2} m(\dot{x}_1^2 + \dot{y}_1^2 + \dot{x}_2^2 + \dot{y}_2^2) + \frac{1}{2} I(\dot{\theta}_1^2 + \dot{\theta}_2^2) - mg(y_1 + y_2) \end{aligned} \tag{13.1}$$

The first term is the linear kinetic energy of the center of mass of the bodies and the second term is the rotational kinetic energy around the center of mass of each rod. The last term is the potential energy of the bodies in a uniform gravitational field. The dot-notation indicates the time derivative of the variable in question.

Using the values of x_1 and y_1 defined above, we have

$$\dot{x}_1 = \dot{\theta}_1\left(\frac{1}{2} l \cos\theta_1\right)$$
$$\dot{y}_1 = \dot{\theta}_1\left(\frac{1}{2} l \sin\theta_1\right)$$

which leads to

$$v_1^2 = \dot{x}_1^2 + \dot{y}_1^2 = \frac{1}{4}\dot{\theta}_1^2 l^2\left(\cos^2\theta_1 + \sin^2\theta_1\right) = \frac{1}{4} l^2 \dot{\theta}_1^2$$

Similarly, for x_2 and y_2 we have

$$\dot{x}_2 = l\left(\dot{\theta}_1\cos\theta_1 + \frac{1}{2}\dot{\theta}_2\cos\theta_2\right)$$
$$\dot{y}_2 = l\left(\dot{\theta}_1\sin\theta_1 + \frac{1}{2}\dot{\theta}_2\sin\theta_2\right)$$

and therefore

$$\begin{aligned} v_2^2 &= \dot{x}_2^2 + \dot{y}_2^2 \\ &= l^2 \left(\dot{\theta}_1^2 \cos^2\theta_1 + \dot{\theta}_1^2 \sin^2\theta_1 + \frac{1}{4}\dot{\theta}_2^2 \cos^2\theta_2 + \frac{1}{4}\dot{\theta}_2^2 \sin^2\theta_2 + \dot{\theta}_1\dot{\theta}_2 \cos\theta_1 \cos\theta_2 + \dot{\theta}_1\dot{\theta}_2 \sin\theta_1 \sin\theta_2 \right) \\ &= l^2 \left(\dot{\theta}_1^2 + \frac{1}{4}\dot{\theta}_2^2 + \dot{\theta}_1\dot{\theta}_2 \cos(\theta_1 - \theta_2) \right) \end{aligned}$$

Substituting the coordinates above into the definition of the Lagrangian, and rearranging the equation, gives

$$\begin{aligned} L &= \frac{1}{2}m(v_1^2 + v_2^2) + \frac{1}{2}I(\dot{\theta}_1^2 + \dot{\theta}_2^2) - mg(y_1 + y_2) \\ &= \frac{1}{2}ml^2 \left(\frac{1}{4}\dot{\theta}_1^2 + \dot{\theta}_1^2 + \frac{1}{4}\dot{\theta}_2^2 + \dot{\theta}_1\dot{\theta}_2 \cos(\theta_1 - \theta_2) \right) + \frac{1}{24}ml^2 \left(\dot{\theta}_1^2 + \dot{\theta}_2^2 \right) \\ &- mgl \left(-\frac{1}{2}\cos\theta_1 - \cos\theta_1 - \frac{1}{2}\cos\theta_2 \right) \\ &= \frac{1}{6}ml^2 \left(\dot{\theta}_2^2 + 4\dot{\theta}_1^2 + 3\dot{\theta}_1\dot{\theta}_2 \cos(\theta_1 - \theta_2) \right) + \frac{1}{2}mgl(3\cos\theta_1 + \cos\theta_2) \end{aligned}$$

The equations of motion can now be derived using the Euler-Lagrange equations, which are given by

$$\frac{d}{dt}\frac{\partial L}{\partial \dot{\theta}_i} - \frac{\partial L}{\partial \theta_i} = 0, \quad i = 1, 2. \tag{13.2}$$

Equations of motion describe how an object's position, velocity, and acceleration change over time. We begin with the equation of motion for θ_1. The derivatives of the Lagrangian are given by

$$\frac{\partial L}{\partial \theta_1} = -\frac{1}{2}ml^2\dot{\theta}_1\dot{\theta}_2 \sin(\theta_1 - \theta_2) - \frac{3}{2}mgl \sin\theta_1$$

and

$$\frac{\partial L}{\partial \dot{\theta}_1} = \frac{4}{3}ml^2\dot{\theta}_1 + \frac{1}{2}ml^2\dot{\theta}_2 \cos(\theta_1 - \theta_2)$$

Thus

$$\frac{d}{dt}\frac{\partial L}{\partial \dot{\theta}_i} = \frac{4}{3}ml^2\ddot{\theta}_1 + \frac{1}{2}ml^2\ddot{\theta}_2 \cos(\theta_1 - \theta_2) - \frac{1}{2}ml^2\dot{\theta}_2(\dot{\theta}_1 - \dot{\theta}_2) \sin(\theta_1 - \theta_2)$$

Combining these results and simplifying yields the first equation of motion,

$$\frac{4}{3}l\ddot{\theta}_1 + \frac{1}{2}l\ddot{\theta}_2 \cos(\theta_1 - \theta_2) + \frac{1}{2}l\dot{\theta}_2^2 \sin(\theta_1 - \theta_2) + \frac{3}{2}g \sin\theta_1 = 0 \tag{13.3}$$

Similarly, the derivatives of the Lagrangian with respect to θ_2 and $\dot{\theta}_2$ are given by

$$\frac{\partial L}{\partial \theta_2} = -\frac{1}{2}ml^2\dot{\theta}_1\dot{\theta}_2 \sin(\theta_1 - \theta_2) - \frac{1}{2}mgl \sin\theta_2$$

and

$$\frac{\partial L}{\partial \dot{\theta}_2} = \frac{1}{3}ml^2\dot{\theta}_2 + \frac{1}{2}ml^2\dot{\theta}_1 \cos(\theta_1 - \theta_2)$$

Thus

$$\frac{d}{dt}\frac{\partial L}{\partial \dot{\theta}_2} = \frac{1}{3}ml^2\ddot{\theta}_2 + \frac{1}{2}ml^2\ddot{\theta}_1 \cos(\theta_1 - \theta_2) - \frac{1}{2}ml^2\dot{\theta}_1(\dot{\theta}_1 - \dot{\theta}_2) \sin(\theta_1 - \theta_2)$$

Plugging these results into the Euler-Lagrange equation and simplifying yields the second equation of motion,

$$\frac{1}{3}l\ddot{\theta}_2 + \frac{1}{2}l\ddot{\theta}_1\cos(\theta_1 - \theta_2) - \frac{1}{2}l\dot{\theta}_1^2\sin(\theta_1 - \theta_2) + \frac{1}{2}g\sin\theta_2 = 0 \tag{13.4}$$

No closed form solutions for θ_1 and θ_2 as functions of time are known, therefore the system can only be solved numerically, using the Runge Kutta method or similar techniques.

[SI*] **Lagrangian for Double Simple Pendulum**

The knowledge of this writing is taken from Wolfram Mathematica webpage written by Eric W. Weisstein.

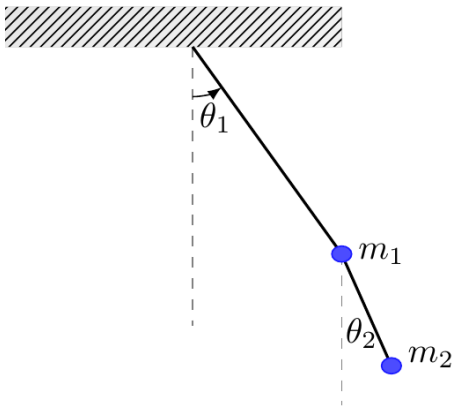

Figure 13.2: *The illustration of double simple pendulum .*

Consider a double bob pendulum with masses m_1 and m_2 attached by rigid massless wires of lengths l_1 and l_2. Further, let the angles of the two wires with the vertical be denoted by θ_1 and θ_2. Finally, let gravity be given by g. Then the positions of the bobs are given by

$$\begin{aligned} x_1 &= l_1\sin\theta_1 \\ y_1 &= -l_1\cos\theta_1 \\ x_2 &= l_1\sin\theta_1 + l_2\sin\theta_2 \\ y_2 &= -l_1\cos\theta_1 - l_2\cos\theta_2 \end{aligned}$$

the potential energy of the system is given by

$$\begin{aligned} V &= m_1gy_1 + m_2gy_2 \\ &= -(m_1 + m_2)gl_1\cos\theta_1 - m_2gl_2\cos\theta_2 \end{aligned}$$

the potential energy is negative because it is going downward, and the first potential energy for m_1 is the sum of $m_1 + m_2$ because the first bob is carrying the mass of the second bob pendulum.

The kinetic energy is given by

$$\begin{aligned} T &= \frac{1}{2}m_1v_1^2 + \frac{1}{2}m_2v_2^2 \\ &= \frac{1}{2}m_1l_1^2\dot{\theta}_1^2 + \frac{1}{2}m_2\left[l_1^2\dot{\theta}_1^2 + l_2^2\dot{\theta}_2^2 + 2l_1l_2\dot{\theta}_1\dot{\theta}_2\cos(\theta_1 - \theta_2)\right] \end{aligned}$$

The Lagrangian is then

$$\begin{aligned}
L &= T - V \\
&= \frac{1}{2}(m_1 + m_2) l_1^2 \dot{\theta}_1^2 + \frac{1}{2} m_2 l_2^2 \dot{\theta}_2^2 + m_2 l_1 l_2 \dot{\theta}_1 \dot{\theta}_2 \cos(\theta_1 - \theta_2) - (-(m_1 + m_2) g l_1 \cos\theta_1 - m_2 g l_2 \cos\theta_2) \\
&= \frac{1}{2}(m_1 + m_2) l_1^2 \dot{\theta}_1^2 + \frac{1}{2} m_2 l_2^2 \dot{\theta}_2^2 + m_2 l_1 l_2 \dot{\theta}_1 \dot{\theta}_2 \cos(\theta_1 - \theta_2) + ((m_1 + m_2) g l_1 \cos\theta_1 + m_2 g l_2 \cos\theta_2)
\end{aligned}$$

Therefore, the equations of motion can now be derived using the Euler-Lagrange equations, for $\dot{\theta}_1$

$$\begin{aligned}
\frac{\partial L}{\partial \dot{\theta}_1} &= m_1 l_1^2 \dot{\theta}_1 + m_2 l_1^2 \dot{\theta}_1 + m_2 l_1 l_2 \dot{\theta}_2 \cos(\theta_1 - \theta_2) \\
\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{\theta}_1}\right) &= (m_1 + m_2) l_1^2 \ddot{\theta}_1 + m_2 l_1 l_2 \ddot{\theta}_2 \cos(\theta_1 - \theta_2) - m_2 l_1 l_2 \dot{\theta}_2 \sin(\theta_1 - \theta_2)(\dot{\theta}_1 - \dot{\theta}_2)
\end{aligned}$$

and for θ_1

$$\frac{\partial L}{\partial \theta_1} = -l_1 g (m_1 + m_2) \sin\theta_1 - m_2 l_1 l_2 \dot{\theta}_1 \dot{\theta}_2 \sin(\theta_1 - \theta_2)$$

so the Euler-Lagrange differential equation for θ_1 / the first equation of motion becomes

$$\begin{aligned}
\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{\theta}_1}\right) - \frac{\partial L}{\partial \theta_1} &= 0 \\
(m_1 + m_2) l_1^2 \ddot{\theta}_1 + m_2 l_1 l_2 \ddot{\theta}_2 \cos(\theta_1 - \theta_2) + m_2 l_1 l_2 \dot{\theta}_2^2 \sin(\theta_1 - \theta_2) + l_1 g (m_1 + m_2) \sin\theta_1 &= 0
\end{aligned} \tag{13.5}$$

Similarly, the equations of motion for $\dot{\theta}_2$

$$\begin{aligned}
\frac{\partial L}{\partial \dot{\theta}_2} &= m_2 l_2^2 \dot{\theta}_2 + m_2 l_1 l_2 \dot{\theta}_1 \cos(\theta_1 - \theta_2) \\
\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{\theta}_2}\right) &= m_2 l_2^2 \ddot{\theta}_2 + m_2 l_1 l_2 \ddot{\theta}_1 \cos(\theta_1 - \theta_2) - m_2 l_1 l_2 \dot{\theta}_1 \sin(\theta_1 - \theta_2)(\dot{\theta}_1 - \dot{\theta}_2)
\end{aligned}$$

and for θ_2

$$\frac{\partial L}{\partial \theta_2} = m_2 l_1 l_2 \dot{\theta}_1 \dot{\theta}_2 \sin(\theta_1 - \theta_2) - l_2 m_2 g \sin\theta_2$$

so the Euler-Lagrange differential equation for θ_2 / the second equation of motion becomes

$$\begin{aligned}
\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{\theta}_2}\right) - \frac{\partial L}{\partial \theta_2} &= 0 \\
m_2 l_2^2 \ddot{\theta}_2 + m_2 l_1 l_2 \ddot{\theta}_1 \cos(\theta_1 - \theta_2) - m_2 l_1 l_2 \dot{\theta}_1^2 \sin(\theta_1 - \theta_2) + l_2 m_2 g \sin\theta_2 &= 0
\end{aligned} \tag{13.6}$$

[SI*] **Chaotic Motion**
The double pendulum undergoes chaotic motion, and clearly shows a sensitive dependence on initial conditions.

i. Double Simple Pendulum Computation and Simulation

In SymIntegration there is a function to compute the lagrangian and the equations of motion, **lagrangian(**$x_1, y_1, x_2, y_2, \theta_1, \dot{\theta}_1, \ddot{\theta}_1, \theta_2, \dot{\theta}_2, \ddot{\theta}_2$**)**

We input them all as symbolic to obtain the equations of motion formula symbolically, and then if we want to plot or compute numerically at certain time t we can substitute the symbolic, e.g. θ_1 into numeric, including defining / substituting the mass for both bob pendulum (m_1, m_2), the length of the wire from the ceiling to the first bob pendulum (l_1), the length of the wire between the first and the second bob pendulum (l_2) and the gravity (g).

```
#include <iostream>
#include "symintegrationc++.h"
#include <bits/stdc++.h>
#include <cmath>

#include <chrono>

using namespace std::chrono;
using namespace std;
using namespace SymbolicConstant;

double division(double x, double y)
{
	return x/y;
}

int main(void)
{
	// Get starting timepoint
	auto start = high_resolution_clock::now();

	Symbolic l1("l1"), l2("l2"), t1("t1"), t2("t2"), dt1("t1'"), dt2("t2
	    '"), ddt1("t1''"),ddt2("t2''");
	Symbolic x1, y1, x2, y2;

	x1 = l1*sin(t1);
	y1 = -l1*cos(t1);
	x2 = l1*sin(t1) + l2*sin(t2);
	y2 = -l1*cos(t1) - l2*cos(t2);

	cout << "x1 = " << x1 << endl;
	cout << "y1 = " << y1 << endl;
	cout << "x2 = " << x2 << endl;
	cout << "y2 = " << y2 << endl;

	cout << "Here we go..." << lagrangian(x1,y1,x2,y2,t1, dt1, dt1, t2,
	    dt2,ddt2) << endl;
```

```
            // Get ending timepoint
            auto stop = high_resolution_clock::now();
            auto duration = duration_cast<microseconds>(stop - start);

            cout << "\nTime taken by function: " << duration.count() << "
                microseconds" << endl;

            return 0;
        }
```

Code 90: *SymIntegration/Examples/Dynamical System/Test SymIntegration Double Pendulum Problem with Lagrangian Function/main.cpp*

```
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Double Pendulum Problem with Lagrangian Function ]# make
g++    -c -o main.o main.cpp
g++ -o main -ggdb main.o -lstdc++ -lsymintegration
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Double Pendulum Problem with Lagrangian Function ]# ./main
x1 = l1*sin(θ1)
y1 = -l1*cos(θ1)
x2 = l1*sin(θ1)+l2*sin(θ2)
y2 = -l1*cos(θ1)-l2*cos(θ2)
Here we go...
Potential energy = -m1*g*l1*cos(θ1)-m2*g*l1*cos(θ1)-m2*g*l2*cos(θ2)
Kinetic energy = 0.5*m1*l1^(2)*θ1'^(2)+0.5*m2*l1^(2)*θ1'^(2)+m2*l1*cos(θ1)*θ1'*l2*cos(θ2)*θ2'+0.5*m2*l2^(2)*θ2'^(2)+m2*l1*sin(θ1)*θ1'*l2*sin(θ2)*θ2'
Lagrangian = 0.5*m1*l1^(2)*θ1'^(2)+0.5*m2*l1^(2)*θ1'^(2)+m2*l1*cos(θ1)*θ1'*l2*cos(θ2)*θ2'+0.5*m2*l2^(2)*θ2'^(2)+m2*l1*sin(θ1)*θ1'*l2*sin(θ2)*θ2'+m1*g*l1*cos(θ1)+m2
*g*l1*cos(θ1)+m2*g*l2*cos(θ2)

dL / dθ1 = -m2*l1*sin(θ1)*θ1'*l2*cos(θ2)*θ2'+m2*l1*cos(θ1)*θ1'*l2*sin(θ2)*θ2'-m1*g*l1*sin(θ1)-m2*g*l1*sin(θ1)

dL / dθ1' = m1*l1^(2)*θ1'+m2*l1^(2)*θ1'+m2*l1*cos(θ1)*l2*cos(θ2)*θ2'+m2*l1*sin(θ1)*l2*sin(θ2)*θ2'

d/dt (dL / dθ1') = -m2*l1*sin(θ1)*l2*cos(θ2)*θ2'*θ1'+m2*l1*cos(θ1)*l2*sin(θ2)*θ2'*θ1'-m2*l1*cos(θ1)*l2*sin(θ2)*θ2'^(2)+m2*l1*sin(θ1)*l2*cos(θ2)*θ2'^(2)+m1*l1^(2)*θ
1''+m2*l1^(2)*θ1''+m2*l1*cos(θ1)*l2*cos(θ2)*θ2''+m2*l1*sin(θ1)*l2*sin(θ2)*θ2''

dL / dθ2 = -m2*l1*cos(θ1)*θ1'*l2*sin(θ2)*θ2'+m2*l1*sin(θ1)*θ1'*l2*cos(θ2)*θ2'-m2*g*l2*sin(θ2)

dL / dθ2' = m2*l1*cos(θ1)*θ1'*l2*cos(θ2)+m2*l2^(2)*θ2'+m2*l1*sin(θ1)*θ1'*l2*sin(θ2)

d/dt (dL / dθ2') = -m2*l1*sin(θ1)*θ1'^(2)*l2*cos(θ2)+m2*l1*cos(θ1)*θ1'^(2)*l2*sin(θ2)-m2*l1*cos(θ1)*θ1'*l2*sin(θ2)*θ2'+m2*l1*sin(θ1)*θ1'*l2*cos(θ2)*θ2'+m2*l1*cos(θ
1)*l2*cos(θ2)*θ1''+m2*l1*sin(θ1)*l2*sin(θ2)*θ1''+m2*l2^(2)*θ2''


First Equation of motion =
-m2*l1*cos(θ1)*l2*sin(θ2)*θ2'^(2)+m2*l1*sin(θ1)*l2*cos(θ2)*θ2'^(2)+m1*l1^(2)*θ1''+m2*l1^(2)*θ1''+m2*l1*cos(θ1)*l2*cos(θ2)*θ2''+m2*l1*sin(θ1)*l2*sin(θ2)*θ2''+m1*g*l
1*sin(θ1)+m2*g*l1*sin(θ1)

Second Equation of motion =
-m2*l1*sin(θ1)*θ1'^(2)*l2*cos(θ2)+m2*l1*cos(θ1)*θ1'^(2)*l2*sin(θ2)+m2*l1*cos(θ1)*l2*cos(θ2)*θ1''+m2*l1*sin(θ1)*l2*sin(θ2)*θ1''+m2*l2^(2)*θ2''+m2*g*l2*sin(θ2)

Time taken by function: 9332191 microseconds
root [ ~/SourceCodes/CPP/C++ Create Library/Test SymIntegration Double Pendulum Problem with Lagrangian Function ]#
```

Figure 13.3: *The computation to obtain the Lagrangian and the equations of motion for double simple pendulum (**Dynamical System/Test SymIntegration Double Pendulum Problem with Lagrangian Function/-main.cpp**).*

We are using Hamzstlab Physics 2D to create a double simple pendulum simulation, we can vary the length from the wall above to the first body and the length from the first body to the second body.

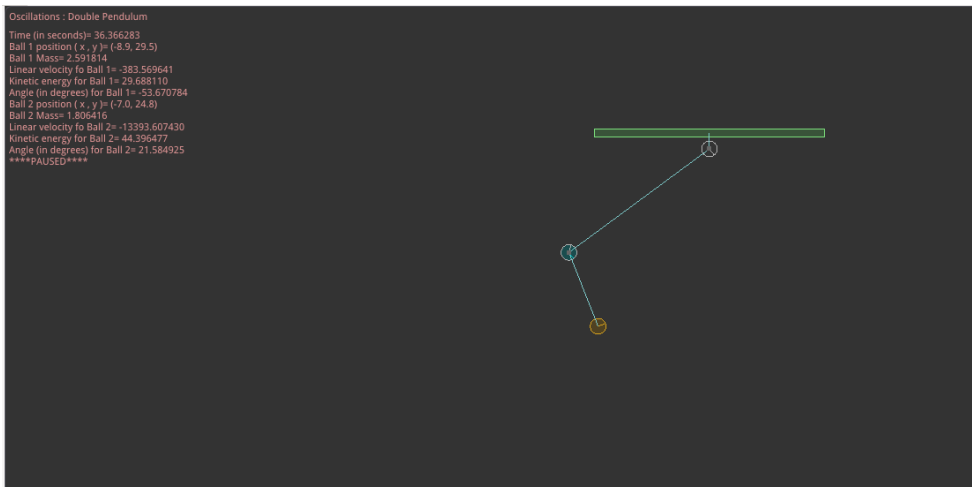

Figure 13.4: *The 2D simulation of a double simple pendulum (**Hamzstlab-Physics2D/tests/double_pendulum.cpp**).*

Box2D has created a 2D circle that can provide us with the angle of the rotation of the circle, in this pendulum case when the body is swinging to the right then the angle will be positive (counterclockwise rotation), and if the body is swinging to the left, the angle will be negative (clockwise rotation). The angle of the rotation is the same as the angle between the vertical axis and the rope that is connected to the circle / the body / bob.

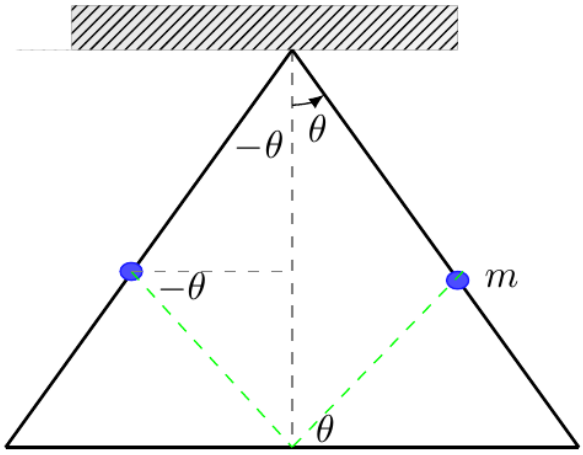

Figure 13.5: *The angle for the pendulum can be obtained easily from the rotation of the circle.*

Chapter 14

Modern Control System

"I see trees are green, red roses are beautiful, and I think to myself what a wonderful world..." - ***Future AI***

"How to control a system is just like how a Pluto person control someone with Opposite Venus or in Conjunct with their Pluto" - ***Merlot***

To reduce greenhouse gases and minimize pollution, it is necessary to improve both the quality and quantity of our environmental monitoring systems. Green engineering is needed to solve global issues such as climate change, clean water, sustainability, waste management, emissions reduction, and minimizing raw material and energy use.

An example is the wind energy conversion to electric power by wind energy turbines connected to electric generators. The intermittency characteristic of the wind makes smart grid development essential to bring the energy to the power grid when it is available and to provide energy from other sources when the wind dies down.

A smart grid can be viewed as a system comprised of hardware and software that routes power more reliably and efficiently to homes, businesses, schools and other users of power in the presence of intermittency or other disturbances.

I. Introduction to Control System

- **Introduction**

[SI*] An open-loop control system uses a controller and an actuator to obtain the desired response. An open-loop system is a system without feedback.

In contrast to an open-loop control system, a closed-loop control system utilizes an additional measure of the actual output to compare the actual output with the desired output response. The measure of the output is called the feedback signal.

[SI*] A feedback control system often uses a function of a prescribed relationship between the output and reference input to control the process. Often the difference between the output of the process under control and the reference input is amplified and used to control the process so that the difference is continually reduced.

[SI*] Closed-loop control has many advantages over open-loop control including the ability to reject external disturbances and improve measurement noise attenuation.

[SI*] A common example of an open-loop control system is a microwave oven set to operate for a fixed time. An example of a closed-loop control system is a person steering an automobile (assuming his or her eyes are open) by looking at the auto's location on the road and making the appropriate adjustments.

- **Brief History of Automatic Control**

[SI*] The first automatic feedback controller used in an industrial process is generally agreed to be James Watt's flyball governor, developed in 1769 for controlling the speed of a steam engine.

The all-mechanical device measured the speed of the output shaft and utilized the movement of the flyball to control the steam valve and therefore the amount of steam entering the engine.

In 1868, J. C. Maxwell formulated a mathematical theory related to control theory using a differential equation model of a governor.

[SI*] Prior to World War II, control theory and practice developed differently in the United States and western Europe than in Russia and eastern Europe. The main impetus for the use of feedback in the United States was the development of the telephone system and electronic feedback amplifiers by Bode, Nyquist, and Black at Bell Telephone Laboratories.

[SI*] The control of an industrial process (manufacturing, production, and so on) by automatic rather than manual means is often called automation. Automation is prevalent in the chemical, electric power, paper, automobile, and steel industries, among others. The concept of automation is central to our industrial society.

[SI*] Automatic machines are used to increase the production of a plant per worker in order to offset rising wages and inflationary costs. Thus industries are concerned with the productivity per worker of their plants. Productivity is defined as the ratio of physical output to physical input. In this case, we are referring to labor productivity, which is real output per hour of work.

The transformation of the U.S. labor force in the country's brief history follows the progressive mechanization of work that attended the evolution of the agrarian republic into an industrial world power. In 1820, more than 70 percent of the labor force worked on the farm. By 1900, less than 40 percent were engaged in agriculture. Today, less than 5 percent works in agriculture.

[SI*] In 1925, some 588,000 people—about 1.3 percent of the nation's labor force were needed to mine 520 million tons of bituminous coal and lignite, almost all of it from underground. By 1980, production was up to 774 million tons, but the work force had been reduced to 208,000. Furthermore, only 136,000 of that number were employed in underground mining operations. The highly mechanized and highly productive surface mines, with just 72,000 workers, produced 482 million tons, or 62 percent of the total.

[SI*] Frequency-domain techniques continued to dominate the field of control following World War II with the increased use of the Laplace transform and the complex frequency plane. During the 1950s, the emphasis in control engineering theory was on the development and use of the s-plane methods and, particularly, the root locus approach.

Furthermore, during the 1980s, the use of digital computers for control components became routine. The technology of these new control elements to perform accurate and rapid calculations was formerly unavailable to control engineers. There are now over 400,000 digital process control computers installed in the United States.

[SI*] With the advent of Sputnik and the space age, another new impetus was imparted to control engineering. It became necessary to design complex, highly accurate control systems for missiles and space probes.

Furthermore, the necessity to minimize the weight of satellites and to control them very accurately has spawned the important field of optimal control. Due to these requirements, the time-domain methods devel oped by Liapunov, Minorsky, and others have been met with great interest in the last two decades. Recent theories of optimal control developed by L. S. Pontryagin in the former Soviet Union and R. Bellman in the United States, as well as recent studies of robust systems, have contributed to the interest in time-domain methods. It now is clear that control engineering must consider both the time-domain and the frequency-domain approaches simultaneously in the analysis and design of control systems.

[SI*] With the advent of Sputnik and the space age, it became known that radio signals from satellites could be used to navigate on the ground by observing the Doppler shift of the received radio signals.

- **Examples of Control Systems**

[SI*] Modern control theory is concerned with systems that have self-organizing, adaptive, robust, learning, and optimum qualities.

[SI*] Machines that automatically load and unload, cut, weld, or cast are used by industry to obtain accuracy, safety, economy, and productivity.

[SI*] A robot is a computer-controlled machine and involves technology closely associated with automation.

[SI*] Today, the most common humanlike characteristic is a mechanical manipulator that is patterned somewhat after the human arm and wrist. Some de vices even have anthropomorphic mechanisms, including what we might recognize as mechanical arms,

wrists, and hands.

[SI*] It is critical that computer control be increasingly applied to the power industry in order to improve the efficient use of energy resources. Also, the control of power plants for minimum waste emission has become increasingly important. The modern, large-capacity plants, which exceed several hundred megawatts, require automatic control systems that account for the interrelationship of the process variables and optimum power production. It is common to have 90 or more manipulated variables under coordinated control.

[SI*] Rapidly rising energy costs coupled with threats of energy curtailment are resulting in new efforts for efficient automatic energy management. Computer controls are used to control energy use in industry and to stabilize and connect loads evenly to gain fuel economy.

[SI*] The ongoing area of research and development of unmanned aerial vehicles (UAVs) is full of potential for the application of control systems.

UAVs are unmanned but are usually controlled by ground operators. Typically they do not operate autonomously and their inability to provide the level of safety of a manned plane keeps them from flying freely in the commercial airspace. One significant challenge is to develop control systems that will avoid in-air collisions.

Ultimately, the goal is to employ the UAV autonomously in such applications as aerial photography to assist in disaster mitigation, survey work to assist in construction projects, crop monitoring, and continuous weather monitoring. In a military setting, UAVs can perform intelligence, surveillance, and reconnaissance missions.

- **Engineering Design**

[SI*] Design is the process of conceiving or inventing the forms, parts, and details of a system to achieve a specified purpose.

Design activity can be thought of as planning for the emergence of a particular product or system. Design is an innovative act whereby the engineer creatively uses knowledge and materials to specify the shape, function, and material content of a system. The design steps are

1. to determine a need arising from the values of various groups, covering the spectrum from public policy makers to the consumer;
2. to specify in detail what the solution to that need must be and to embody these values;
3. to develop and evaluate various alternative solutions to meet these specifications;
4. to decide which one is to be designed in detail and fabricated

[SI*] Specifications are statements that explicitly state what the device or product is to be and do. The design of technical systems aims to provide appropriate design specifications and rests on four characteristics: complexity, trade-offs, design gaps, and risk.

[SI*] Design is a process that may proceed in many directions before the desired one is found. It is a deliberate process by which a designer creates something new in response to a recognized need while recognizing realistic constraints.

The main approach to the most effective engineering design is parameter analysis and optimization. Parameter analysis is based on

1. identification of the key parameters,
2. generation of the system configuration, and
3. evaluation of how well the configuration meets the needs.

[SI*] These three steps form an iterative loop. Once the key parameters are identified and the configuration synthesized, the designer can optimize the parameters.

- **Control System Design**

[SI*] The first step in the design process consists of establishing the system goals. For example, we may state that our goal is to control the velocity of a motor accurately.

The second step is to identify the variables that we desire to control (for example, the velocity of the motor).

The third step is to write the specifications in terms of the accuracy we must attain.

This required accuracy of control will then lead to the identification of a sensor to measure the controlled variable. The performance specifications will describe how the closed-loop system should perform and will include

1. good regulation against disturbances,
2. desirable responses to commands,
3. realistic actuator signals,
4. low sensitivities, and
5. robustness.

[SI*] As designers, we proceed to the first attempt to configure a system that will result in the desired control performance. This system configuration will normally consist of a sensor, the process under control, an actuator, and a controller.

The next step consists of identifying a candidate for the actuator. This will, of course, depend on the process, but the actuation chosen must be capable of effectively adjusting the performance of the process. For example, if we wish to control the speed of a rotating flywheel, we will select a motor as the actuator.

The sensor, in this case, must be capable of accurately measuring the speed. We then obtain a model for each of these elements.

The next step is the selection of a controller, which often consists of a summing amplifier that will compare the desired response and the actual response and then forward this error-measurement signal to an amplifier.

The final step in the design process is the adjustment of the parameters of the system to achieve the desired performance. If we can achieve the desired performance by adjusting the parameters, we will finalize the design and proceed to document the results. If not, we will need to establish an improved system configuration and perhaps select an enhanced actuator and sensor.

[SI*] The design process has been dramatically affected by the advent of powerful and inexpensive computers and effective control design and analysis software.

For example, the Boeing 777, which incorporates the most advanced flight avionics of any U.S. commercial aircraft, was almost entirely computer-designed.

[SI*] Verification of final designs in high-fidelity computer simulations is essential. In many applications, the certification of the control system in realistic simulations represents a significant cost in terms of money and time. The Boeing 777 test pilots flew about 2400 flights in high-fidelity simulations before the first aircraft was even built.

Another notable example of computer-aided design and analysis is the McDonnell Douglas Delta Clipper experimental vehicle DC-X, which was designed, built, and flown in 24 months. Computer-aided design tools and automated code-generation contributed to an estimated 80 percent cost savings and 30 percent time savings.

[SI*] In summary, the controller design problem is as follows: Given a model of the system to be controlled (including its sensors and actuators) and a set of design goals, find a suitable controller, or determine that none exists. As with most of engineering design, the design of a feedback control system is an iterative and nonlinear process. A successful designer must consider the underlying physics of the plant under control, the control design strategy, the controller design architecture (that is, what type of controller will be employed), and effective controller tuning strategies.

- **Mechatronic Systems**

[SI*] A natural stage in the evolutionary process of modern engineering design is encompassed in the area known as mechatronics.

Mechatronics is the synergistic integration of mechanical, electrical, and computer systems and has evolved over the past 30 years, leading to a new breed of intelligent products. Feedback control is an integral aspect of modern mechatronics systems. One can understand the extent that mechatronics reaches into various disciplines by considering the components that make up mechatronics.
The key elements of mechatronics are

1. physical systems modeling,
2. sensors and actuators,
3. signals and systems,
4. computers and logic systems, and
5. software and data acquisition.

[SI*] Feedback control encompasses aspects of all five key elements of mechatronics, but is associated primarily with the element of signals and systems.

[SI*] Advancements in traditional disciplines are fueling the growth of mechatronics systems by providing "enabling technologies." A critical enabling technology was the microprocessor which has had a profound effect on the design of consumer products.

We should expect continued advancements in cost-effective microprocessors and microcontrollers, novel sensors and actuators enabled by advancements in applications of microelectromechanical systems (MEMS), advanced control methodologies and

real-time programming methods, networking and wireless technologies, and mature computer-aided engineering (CAE) technologies for advanced system modeling, virtual prototyping, and testing.

[SI*] Before the 1960s, the radio was the only significant electronic device in an automobile. Today, many automobiles have 30-60 microcontrollers, up to 100 electric motors, about 200 pounds of wiring, a multitude of sensors, and thousands of lines of software code. A modern automobile can no longer be classified as a strictly mechanical machine—it has been transformed into a comprehensive mechatronic system.

[SI*] The hybrid fuel vehicle utilizes a conventional internal combustion engine in combination with a battery (or other energy storage device such as a fuel cell or flywheel) and an electric motor to provide a propulsion system capable of doubling the fuel economy over conventional automobiles. Although these hybrid vehicles will never be zero emission vehicles (since they have internal combustion engines), they can reduce the level of harmful emissions by one-third to one-half, and with future improvements, these emissions may reduce even further.

Wind Power

[SI*] Many nations in the world today are faced with unstable energy supplies, often leading to rising fuel prices and energy shortages. Additionally, the negative effects of fossil fuel utilization on the quality of our air are well documented. Many nations have an imbalance in the supply and demand of energy, consuming more than they produce.

To address this imbalance, many engineers are considering developing advanced systems to access other sources of energy, such as wind energy. In fact, wind energy is one of the fastest-growing forms of energy generation in the United States and in other locations around the world.

[SI*] In 2006. the installed global wind energy capacity was over 59,000 MW. In the United States, there was enough energy derived from wind to power over 2.5 million homes, according to the American Wind Energy Association.

For the past 35 years, researchers have concentrated on developing technologies that work well in high wind areas (defined to be areas with a wind speed of at least 6.7 m/s at a height of 10 m).

[SI*] Most of the easily accessible high wind sites in the United States are now utilized, and improved technology must be developed to make lower wind areas more cost effective. New developments are required in materials and aerodynamics so that longer turbine rotors can operate efficiently in the lower winds, and in a related problem, the towers that support the turbine must be made taller without increasing the overall costs.

Embedded Computers

[SI*] Embedded control systems employ on-board special-purpose digital computers as integral components of the feedback loop.

[SI*] There is a student who built a rover constructed around the Compact RIO by National Instruments, Inc. that serves as the on-board embedded computer. In the rover design, the sensors include an optical encoder for measuring engine speed, a rate gyro and accelerometer to measure turns, and a Global Positioning System (GPS) unit to obtain

position and velocity estimates of the vehicle.

The actuators include two linear actuators to turn the front wheels and to brake and accelerate. The communications device permits the rover to stay in contact with the ground station.

[SI*] Advances in sensors, actuators, and communication devices are leading to a new class of embedded control systems that are networked using wireless technology, thereby enabling spatially-distributed control. Embedded control system designers must be able to understand and work with various network protocols, diverse operating systems and programming languages. While the theory of systems and controls serves as the foundation for the modern control system design, the design process is rapidly expanding into a multi-disciplinary enterprise encompassing multiple engineering areas, as well as information technology and computer science.

[SI*] There are numerous other examples of intelligent systems poised to enter our everyday life, including autonomous rovers, smart home appliances (e.g., dishwashers, vacuum cleaners, and microwave ovens), wireless network-enabled devices, "human-friendly machines" that perform robot-assisted surgery, and implantable sensors and actuators.

- **Green Engineering**

[SI*] Global issues such as climate change, clean water, sustainability, waste management, emissions reduction, and minimizing raw material and energy use have caused many engineers to re-think existing approaches to engineering design in critical areas.

One outcome of the evolving design strategy is to consider an approach that has come to be known as "green engineering." The goal of green engineering is to design products that will minimize pollution, reduce the risk to human health, and improve the environment. The basic principles of green engineering are :

1. Engineer processes and products holistically, use systems analysis, and integrate environmental impact assessment tools.
2. Conserve and improve natural ecosystems while protecting human health and well-being.
3. Use life-cycle thinking in all engineering activities.
4. Ensure that all material and energy inputs and outputs are as inherently safe and benign as possible.
5. Minimize depletion of natural resources.
6. Strive to prevent waste.
7. Develop and apply engineering solutions, while being cognizant of local geography, aspirations, and cultures.
8. Create engineering solutions beyond current or dominant technologies; improve, innovate, and invent technologies to achieve sustainability.
9. Actively engage communities and stakeholders in development of engineering solutions.

[SI*] Green engineering applications can be classified into one of five categories:

1. Environmental Monitoring
2. Energy Storage Systems
3. Power Quality Monitoring

4. Solar Energy
5. Wind Energy

[SI*] Energy storage systems are critical technologies for green engineering. There are many types of energy storage systems. The energy storage system we are most familiar with is the battery. Batteries are used to power most of the electronic devices in use today; some batteries are rechargeable and some are single-use throwaways.

[SI*] The problems associated with power quality monitoring are varied and can include leading and lagging power, voltage variations, and waveform harmonics. Many of the green engineering systems and components require careful monitoring of current and voltages. An interesting example would be the modeling of current transformers that are used in various capacities for measuring and monitoring within the power grid network of interconnected systems used to deliver electricity.

[SI*] Efficiently converting solar energy into electricity is an engineering challenge. Two technologies for generation of electricity using sunshine are solar photovoltaic and solar thermal. With photovoltaic systems the sunlight is converted directly to electricity, and with solar thermal the sun heats water to create steam that is used to power steam engines. Designing and deploying solar photovoltaic systems for solar power generation is one approach employing green engineering principles to utilize the sun's energy to power our homes, offices, and businesses.

[SI*] Power derived from wind is an important source of renewable energy around the world. Wind energy conversion to electric power is achieved by wind energy turbines connected to electric generators.

- **The Future Evolution of Control Systems**

[SI*] Research areas concentrating on artificial intelligence, sensor integration, computer vision, and off-line CAD/CAM programming will make systems more universal and economical. Control systems are moving toward autonomous operation as an enhancement to human control. Research in supervisory control, human-machine interface methods, and computer database management are intended to reduce operator burden and improve operator efficiency.

The acceleration in the pace of technological innovation inaugurated by the Industrial Revolution has until recently resulted mainly in the displacement of human muscle power from the tasks of production.

[SI*] Automation is used to improve productivity and obtain high-quality products. Automation is the automatic operation or control of a process, device, or system. We use automatic control of machines and processes to produce a product reliably and with high precision.

- **Design Examples**

[SI*] **Smart grid control systems**
A smart grid is as much a concept as it is a physical system. In essence, the concept is to deliver power more reliably and efficiently while remaining environmentally friendly, economical, and safe. A smart grid can be viewed as a system comprised of hardware and software that routes power more reliably and efficiently to homes, businesses, schools, and other users of power.

One interesting aspect of the smart grid is real-time demand side management requiring a two-way flow of information between the user and the power generation system.

The five key technologies required to implement a successful modern smart grid include

1. integrated communications,
2. sensing and measurements,
3. advanced components,
4. advanced control methods, and
5. improved interfaces and decision support

[SI*] A smart grid will use sensors, controllers, the Internet, and communication systems to improve the reliability and efficiency of the grid. It is estimated that deployment of smart grids could reduce emissions globally of CO_2 due to power systems by 14 percent by 2020.

[SI*] Transmission of power is called power flow and the improved control of power will increase its security and efficiency. Transmission lines have inductive, capacitive, and resistive effects that result in dynamic impacts or disturbances. The smart grid must anticipate and respond to system disturbances rapidly. This is referred to as self-healing. In other words, a smart grid should be capable of managing significant disturbances occurring on very short time scales. **Rotating disk speed control**

[SI*] Many modern devices employ a rotating disk held at a constant speed. For example, a CD player requires a constant speed of rotation in spite of motor wear and variation and other component changes. Our goal is to design a system for rotating disk speed control that will ensure that the actual speed of rotation is within a specified percentage of the desired speed.

[SI*] The open-loop system (without feedback) uses a battery source to provide a voltage that is proportional to the desired speed. This voltage is amplified and applied to the motor.

To obtain a feedback system, we need to select a sensor. One useful sensor is a tachometer that provides an output voltage proportional to the speed of its shaft.
Insulin delivery control system

[SI*] Control systems have been utilized in the biomedical field to create implanted automatic drug-delivery systems to patients.

[SI*] Automatic systems can be used to regulate blood pressure, blood sugar level, and heart rate, A common application of control engineering is in the field of open-loop system drug delivery, in which mathematical models of the dose-effect relationship of the drugs are used.

[SI*] A drug-delivery system implanted in the body uses an open-loop system, since miniaturized glucose sensors are not yet available.The best solutions rely on individually programmable, pocket-sized insulin pumps that can deliver insulin according to a preset time history. More complicated systems will use closed-loop control for the measured blood glucose levels.

II. Mathematical Models of Systems

- **Introduction**
  Mathematical models of physical systems are key elements in the design and analysis of control systems. The dynamic behavior is generally described by ordinary differential equations. We will consider a wide range of systems, including mechanical, hydraulic, and electrical. Since most physical systems are nonlinear, we will discuss linearization approximations, which allow us to use Laplace transform methods. We will then proceed to obtain the input-output relationship for components and subsystems in the form of transfer functions.

[SI*] To understand and control complex systems, one must obtain quantitative mathematical models of these systems. It is necessary therefore to analyze the relationships between the system variables and to obtain a mathematical model.
Because the systems under consideration are dynamic in nature, the descriptive equations are usually differential equations. Furthermore, if these equations can be linearized, then the Laplace transform can be used to simplify the method of solution.

[SI*] In practice, the complexity of systems and our ignorance of all the relevant factors necessitate the introduction of assumptions concerning the system operation.
The approach to dynamic system modeling can be listed as follows:

1. Define the system and its components.
2. Formulate the mathematical model and fundamental necessary assumptions based on basic principles.
3. Obtain the differential equations representing the mathematical model.
4. Solve the equations for the desired output variables.
5. Examine the solutions and the assumptions.
6. If necessary, reanalyze or redesign the system.

- **Differential Equations of Physical Systems**

[SI*] Consider the torsional spring-mass system with applied torque $T_a(t)$. Assume the torsional spring element is massless. Suppose we want to measure the torque $T_s(t)$ transmitted to the mass m. Since the spring is massless, the sum of the torques acting on the spring itself must be zero, or

$$T_a(t) - T_s(t) = 0$$

We see immediately that the external torque $T_a(t)$ applied at the end of the spring is transmitted through the torsional spring. Because of this, we refer to the torque as a through-variable. In a similar manner, the angular rate difference associated with the torsional spring element is

$$\omega(t) = \omega_s(t) - \omega_a(t)$$

Thus, the angular rate difference is measured across the torsional spring element and is referred to as an across-variable. These same types of arguments can be made for most common physical variables (such as force, current, volume, flow rate, etc.

[SI*] **Nomenclature**

* Through-variable: F = force, T = torque, i = current, Q = fluid volumetric flow rate, q = heat flow rate.
* Across-variable: v = translational velocity, ω = angular velocity, v = voltage, P = pressure,T = temperature.
* Inductive storage: L = inductance, $\frac{1}{k}$ - reciprocal translational or rotational stiffness, I = fluid inertance.
* Capacitive storage: C = capacitance, M - mass, J = moment of inertia, C_f = fluid capacitance, C_t = thermal capacitance.
* Energy dissipators: R = resistance, b = viscous friction, R_f = fluid resistance, R_t = thermal resistance.

The symbol v is used for both voltage in electrical circuits and velocity in translational mechanical systems and is distinguished within the context of each differential equation.

[SI*] For mechanical systems, one uses Newton's laws; for electrical systems, Kirchhoff s voltage laws.

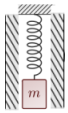

Figure 14.1: *An illustration of spring-mass system*

In this spring-mass-damper example, we model with the wall surrounding on the left and right side of the mass, and treat the wall friction as a viscous damper, that is, the friction force is linearly proportional to the velocity of the mass. In reality the friction force may behave in a more complicated fashion. For example, the wall friction may behave as a Coulomb damper. Coulomb friction, also known as dry friction, is a nonlinear function of the mass velocity and possesses a discontinuity around zero velocity.

For a well-lubricated, sliding surface, the viscous friction is appropriate and will be used here and in subsequent spring-mass-damper examples. Summing the forces acting on M and utilizing Newton's second law yields the second order linear constant-coefficient differential equation:

$$M\frac{d^2y(t)}{dt^2} + b\frac{dy(t)}{dt} + ky(t) = r(t)$$

where k is the spring constant of the ideal spring and b is the friction constant.

When the mass is initially displaced a distance v(0) = y 0 and released, the dynamic response of the system can be represented by an equation of the form

$$y(t) = K_1 e^{-\alpha_1 t}\sin(\beta_1 t + \theta_1) \tag{14.1}$$

[SI*] RLC Circuit

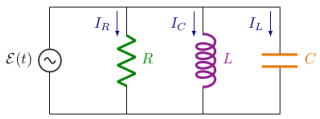

Figure 14.2: *An illustration of RLC circuit*

the following differential equation describe the electrical RLC circuit by utilizing Kirchhoff's current law:

$$\frac{v(t)}{R} + C\frac{dv(t)}{dt} + \frac{1}{L}\int_0^t v(t)\,dt = r(t) \tag{14.2}$$

The solution of the differential equation describing the process may be obtained by classical methods such as the use of integrating factors and the method of undetermined coefficients.

A solution is obtained for the voltage of the RLC circuit when the circuit is subjected to a constant current $r(t) = I$. Then the voltage is

$$v(t) = K_2 e^{-\alpha_2 t}\sin(\beta_2 t + \theta_2) \tag{14.3}$$

To reveal further the close similarity between the differential equations for the mechanical and electrical systems

$$v(t) = \frac{dy(t)}{dt} \tag{14.4}$$

Then we have

$$M\frac{dv(t)}{dt} + bv(t) + k\int_0^t v(t)dt = r(t) \tag{14.5}$$

The concept of analogous systems is a very useful and powerful technique for system modeling. The voltage-velocity analogy, often called the force-current analogy, is a natural one because it relates the analogous through- and across-variables of the electrical and mechanical systems. Another analogy that relates the velocity and current variables is often used and is called the force-voltage analogy.

Analogous systems with similar solutions exist for electrical, mechanical, thermal, and fluid systems. The existence of analogous systems and solutions provides the analyst with the ability to extend the solution of one system to all analogous systems with the same describing differential equations. Therefore what one learns about the analysis and design of electrical systems is immediately extended to an understanding of fluid, thermal, and mechanical systems.

- **Linear Approximations of Physical Systems**

[SI*] A great majority of physical systems are linear within some range of the variables. In general, systems ultimately become nonlinear as the variables are increased without limit.

A system is defined as linear in terms of the system excitation and response. In the case of the electrical network, the excitation is the input current $r(t)$ and the response is the voltage $v(t)$.

In general, a necessary condition for a linear system can be determined in terms of an excitation $x(t)$ and a response $y(t)$.

When the system at rest is subjected to an excitation $x_1(t)$, it provides a response $y_1(t)$.

Furthermore, when the system is subjected to an excitation $x_2(t)$, it provides a corresponding response $y_2(t)$. For a linear system, it is necessary that the excitation $x_1(t) + x_2(t)$ result in a response $y_1(t) + y_2(t)$. This is usually called the principle of superposition.

Furthermore, the magnitude scale factor must be preserved in a linear system.

Again, consider a system with an input $x(t)$ that results in an output $y(t)$. Then the response of a linear system to a constant multiple β of an input x must be equal to the response to the input multiplied by the same constant so that the output is equal to βy. This is called the property of homogeneity.

*** A linear system satisfies the properties of superposition and homogeneity.**

[SI*] Furthermore,

1. A system characterized by the relation

   $$y = x^2$$

   is not linear, because the superposition property is not satisfied.
2. A system represented by the relation

   $$y = mx + b$$

   is not linear, because it does not satisfy the homogeneity property.
   However, this second system may be considered linear about an operating point (x_0, y_0) for small changes δx and δy. When $x = x_0 + \delta x$ and $y = y_0 + \delta y$, we have

   $$y = mx + b$$

   or

   $$y_0 + \delta y = mx_0 + m\delta x + b$$

   Therefore, $\delta y = m\delta x$, which satisfies the necessary conditions.
   The linearity of many mechanical and electrical elements can be assumed over a reasonably large range of the variables.
   This is not usually the case for thermal and fluid elements, which are more frequently nonlinear in character. Fortunately, however, one can often linearize nonlinear elements assuming small-signal conditions. This is the normal approach used to obtain a linear equivalent circuit for electronic circuits and transistors.

Consider a general element with an excitation (through-) variable $x(t)$ and a response (across-) variable $y(t)$.
The relationship of the two variables is written as

$$y(t) = g(x(t))$$

where $g(x(t))$ indicates $y(t)$ is a function of $x(t)$.
The normal operating point is designated by x_0. Because the curve (function) is continuous over the range of interest, a Taylor series expansion about the operating point may be utilized. Then we have

$$y = g(x) = g(x_0) + \frac{dg}{dx}\bigg|_{x=x_0} \frac{(x-x_0)}{1!} + \frac{d^2g}{dx^2}\bigg|_{x=x_0} \frac{(x-x_0)^2}{2!} + \dots \tag{14.6}$$

The slope at the operating point,

$$\frac{dg}{dx}|_{x=x_0}$$

is a good approximation to the curve over a small range of $(x - x_0)$, the deviation from the operating point.

Thus

$$y = g(x_0) + \frac{dg}{dx}|_{x=x_0}\,(x-x_0) = y_0 + m(x-x_0) \tag{14.7}$$

where m is the slope at the operating point. The equation above can be rewritten as the linear equation

$$(y - y_0) = m(x - x_0) \tag{14.8}$$

or

$$\delta y = m\delta x \tag{14.9}$$

Example 1 *Pendulum Oscillator Model A*

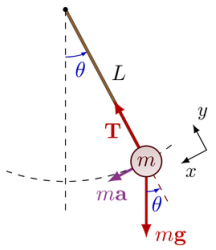

Figure 14.3: *An illustration of a pendulum*

Consider the pendulum oscillator with the torque on the mass is

$$T = MgL\sin\theta(t) \tag{14.10}$$

where g is the gravity constant. The equilibrium condition for the mass is $\theta_0 = 0^0$. The nonlinear relation between $T(t)$ and $\theta(t)$ is like a sin *function.*
The first derivative evaluated at equilibrium provides the linear approximation, which is

$$T(t) - T_0 = MgL\frac{\partial \sin\ \theta}{\partial \theta}|_{\theta=\theta_0}\ (\theta - \theta_0) \tag{14.11}$$

where $T_0 = 0$. Then, we have

$$T(t) = MgL(\cos\ 0^0)(\theta(t) - 0^0) = MgL\theta(t) \tag{14.12}$$

This approximation is reasonably accurate for $-\pi/4 \leq \theta \leq \pi/4$. The response of the linear model for the swing through $\pm 30^0$ is within 5% of the actual nonlinear pendulum response.

- **The Laplace Transform**

[SI*] The ability to obtain linear approximations of physical systems allows the analyst to consider the use of the Laplace transformation.

The Laplace transform is a powerful tool to solve differential equations. It transforms an Initial Value Problem in Ordinary Differential Equation to algebraic equations. Function of a real variable t to a function of a complex variable s. The transform has many application in science and engineering.

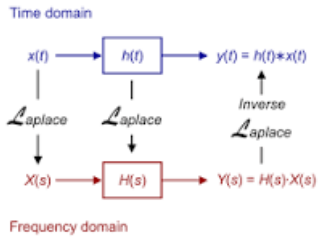

Figure 14.4: *The Laplace Transform is a powerful tool that is very useful in Electrical Engineering.*

The Laplace transform method substitutes relatively easily solved algebraic equations for the more difficult differential equations.

The advantage of using the Laplace transform is that it converts an ODE into an algebraic equation of the same order that is simpler to solve, even though it is a function of a complex variable.

The time-response solution is obtained by the following operations:

1. Obtain the linearized differential equations
2. Obtain the Laplace transformation of the differential equation
3. Solve the resulting algebraic equation for the transform of the variable of interest

[SI*] The Laplace transform exists for linear differential equations for which the transformation integral converges.

For $f(t)$ to be transformable, it is sufficient that

$$\int_{0^-}^{\infty} f(t)e^{-st}\,dt = \mathcal{L}\{f(t)\} \tag{14.13}$$

The inverse Laplace transform is written as

$$f(t) = \frac{1}{2\pi j}\int_{\sigma-j\infty}^{\sigma+j\infty} F(s)e^{st}\,ds \tag{14.14}$$

[SI*] Alternatively, one may describe the electrical RLC circuit of Figure above by utilizing Kirchhoff's current law. Then we obtain the following differential equation

$$\frac{v(t)}{R} + C\frac{dv(t)}{dt} + \frac{1}{L}\int_0^t v(t)\,dt = r(t) \tag{14.15}$$

The solution of the differential equation describing the process may be obtained by classical methods such as the use of integrating factors and the method of undetermined coefficients.

For example, when the mass is initially displaced a distance $v(0) = y_0$ and released, the dynamic response of the system can be represented by an equation of the form

$$y(t) = K_1 e^{-\alpha_1 t} \sin(\beta_1 t + \theta_1) \tag{14.16}$$

A similar solution is obtained for the voltage of the RLC circuit when the circuit is subjected to a constant current $r(t) = I$. Then the voltage is

$$v(t) = K_2 e^{-\alpha_2 t} \sin(\beta_2 t + \theta_2) \tag{14.17}$$

To reveal further the close similarity between the differential equations for the mechanical and electrical systems

$$v(t) = \frac{dy(t)}{dt} \tag{14.18}$$

Then we have

$$M\frac{dv(t)}{dt} + bv(t) + k\int_0^t v(t)\, dt = r(t) \tag{14.19}$$

$f(t)$	$F(s)$
Step function, $u(t)$	$\frac{1}{s}$
e^{-at}	$\frac{1}{s+a}$
$\sin \omega t$	$\frac{\omega}{s^2+\omega^2}$
$\cos \omega t$	$\frac{s}{s^2+\omega^2}$
t^n	$\frac{n!}{s^{n+1}}$
$f^{(k)}(t) = \frac{d^k f(t)}{dt^k}$	$s^k F(s) - s^{k-1}f(0^-) - s^{k-2}f'(0^-)$ $-\ldots - f^{(k-1)}(0^-)$
$\int_{-\infty}^{t} f(t)\, dt$	$\frac{F(s)}{s} + \frac{1}{s}\int_{-\infty}^{0} f(t)\, dt$
Impulse function $\delta(t)$	1
$e^{-at} \sin \omega t$	$\frac{\omega}{(s+a)^2+\omega^2}$
$e^{-at} \cos \omega t$	$\frac{s+a}{(s+a)^2+\omega^2}$
$\frac{1}{\omega}[(\alpha-a)^2+\omega^2]^{1/2}e^{-at}\sin(\omega t+\phi)$, $\phi = \tan^{-1}\frac{\omega}{\alpha-a}$	$\frac{s+\alpha}{(s+a)^2+\omega^2}$

Figure 14.5: *A table of Laplace Transform*

[SI*] Alternatively, the Laplace variable s can be considered to be the differential operator so that

$$s = \frac{d}{dt} \tag{14.20}$$

Then we also have the integral operator

$$\frac{1}{s} = \int_{0^-}^{t} dt \tag{14.21}$$

The inverse Laplace transformation is usually obtained by using the Heaviside partial fraction expansion.

*** This approach is particularly useful for systems analysis and design because the effect of each characteristic root or eigenvalue can be clearly observed.**

[SI*] Laplace Transform Formulas

$f(t)$	$F(s)$
Step function, $u(t)$	$\frac{1}{s}$
e^{-at}	$\frac{1}{s+a}$
$\sin\ \omega t$	$\frac{\omega}{s^2+\omega^2}$
$\cos\ \omega t$	$\frac{s}{s^2+\omega^2}$
t^n	$\frac{n!}{s^{n+1}}$
$f^{(k)}(t) = \frac{d^k f(t)}{dt^k}$	$s^k F(s) - s^{k-1} f(0^-) - s^{k-2} f$
$\int_{-\infty}^{t} f(t)\ dt$	$\frac{F(s)}{s} + \frac{1}{s}\int_{-\infty}^{0} f(t)\ dt$
Impulse function $\delta(t)$	1
$e^{-at}\ \sin\ \omega t$	$\frac{\omega}{(s+a)^2+\omega^2}$
$e^{-at}\ \cos\ \omega t$	$\frac{s+a}{(s+a)^2+\omega^2}$
$\frac{1}{\omega}[(\alpha-a)^2+\omega^2]^{1/2} e^{-at}\sin(\omega t+\phi),\ \phi = \tan^{-1}\frac{\omega}{\alpha-a}$	$\frac{s+\alpha}{(s+a)^2+\omega^2}$
$\frac{\omega_n}{\sqrt{1-\zeta^2}} e^{-\zeta\omega_n t}\ \sin\ \omega_n\sqrt{1-\zeta^2}t,\ \zeta < 1$	$\frac{\omega_n^2}{s^2+2\zeta\omega_n s+\omega_n^2}$
$\frac{1}{a^2+\omega^2} + \frac{1}{\omega\sqrt{a^2+\omega^2}} e^{-at}\ \sin(\omega t-\phi),\ \phi = \tan^{-1}\frac{\omega}{-a}$	$\frac{1}{s[(s+a)^2+\omega^2]}$
$1 - \frac{1}{\sqrt{1-\zeta^2}} e^{-\zeta\omega_n t}\ \sin(\omega_n\sqrt{1-\zeta^2}t+\phi),\ \phi = \cos^{-1}\zeta,\ \zeta < 1$	$\frac{\omega_n^2}{s(s^2+2\zeta\omega_n s+\omega_n^2)}$
$\frac{\alpha}{a^2+\omega^2} + \frac{1}{\omega}\left[\frac{(\alpha-a)^2+\omega^2}{a^2+\omega^2}\right]^{1/2} e^{-at}\ \sin(\omega t+\phi),\ \phi = \tan^{-1}\frac{\omega}{\alpha-a} - \tan^{-1}\frac{\omega}{-a}$	$\frac{s+\alpha}{s[(s+a)^2+\omega^2]}$

Example 2 *The Laplace Transform A Consider this differential equation*

$$M\frac{d^2y}{dt^2} + b\frac{dy}{dt} + ky = r(t) \tag{14.22}$$

The Laplace transform of the equation above is

$$M\left(s^2Y(s) - sy(0^-) - \frac{dy}{dt}(0^-)\right) + b(sY(s) - y(0^-)) + kY(s) = R(s) \tag{14.23}$$

when

$$r(t) = 0 \quad and \quad y(0^-) = y_0, \quad and \quad \frac{dy}{dt}|_{t=0^+} = 0 \tag{14.24}$$

we have

$$Ms^2Y(s) - Msy_0 + bsY(s) - by_0 + kY(s) = 0 \tag{14.25}$$

Solving for $Y(s)$, we obtain

$$Y(s) = \frac{(Ms+b)y_0}{Ms^2 + bs + k} = \frac{p(s)}{q(s)} \tag{14.26}$$

[SI*] The denominator polynomial $q(s)$, when set equal to zero, is called the characteristic equation. The roots of this equation determine the character of the time response.

* The roots of this characteristic equation are also called poles of the system.
* The root of the numerator polynomial $p(s)$ are called the zeros of the system, for example, $s = -b/M$ is a zero of $p(s)$.

[SI*] Poles and zeros are critical frequencies.

* At the poles, the function $Y(s)$ becomes infinite.
* At the zeros, the function becomes zero.

The complex frequency s-plane plot of the poles and zeros graphically potrays the character of the natural transient response of the system.

Example 3 (The poles and zeros of $Y(s)$) *Consider the system when $k/M = 2$ and $b/M = 3$. Then $Y(s)$ becomes*

$$Y(s) = \frac{(s+3)y_0}{(s+1)(s+2)} \tag{14.27}$$

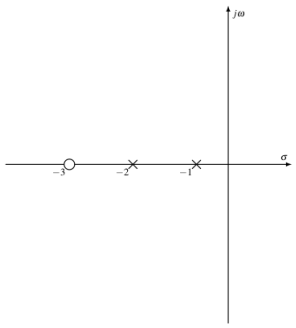

Figure 14.6: *An s-plane pole and zero plot*

The poles and zeros of $Y(s)$ are shown on the s-plane in the figure above.

Expanding the equation above in a partial fraction expansion, we obtain

$$Y(s) = \frac{k_1}{s+1} + \frac{k_2}{s+2} \tag{14.28}$$

*where k_1 and k_2 are the coefficients of the expansion. The coefficients k_i are called **residues** and are evaluated by multiplying through by the denominator factor corresponding to k_i and setting s equal to the root. Evaluating k_1 when $y_0 = 1$, we have*

$$\begin{aligned} k_1 &= \frac{(s-s_1)p(s)}{q(s)}\Big|_{s=s_1} \\ &= (s-s_1)\frac{(s+3)y_0}{(s+1)(s+2)}\Big|_{s_1=-1} \\ &= \frac{(s+1)(s+3)}{(s+1)(s+2)}\Big|_{s_1=-1} \\ &= \frac{s+3}{s+2}\Big|_{s_1=-1} = 2 \end{aligned} \tag{14.29}$$

Now for k_2

$$\begin{aligned} \frac{k_1}{s+1} + \frac{k_2}{s+2} &= \frac{(s+3)y_0}{(s+1)(s+2)} \\ \frac{2(s+2)+k_2(s+1)}{(s+1)(s+2)} &= \frac{(s+3)y_0}{(s+1)(s+2)} \\ 2(s+2)+k_2(s+1) &= s+3 \\ k_2(s+1) &= -s-1 = -(s+1) \\ k_2 &= -1 \end{aligned} \tag{14.30}$$

and we obtain $k_2 = -1$. Alternatively, the residues of $Y(s)$ at the respective poles may be

evaluated graphically on the s-plane plot, since equation k_1 may be written as

$$\begin{aligned} k_1 &= \frac{s+3}{s+2}|_{s=s_1=-1} \\ &= \frac{s_1+3}{s_1+2}|_{s_1=-1} = 2 \end{aligned} \tag{14.31}$$

The inverse Laplace transform of equation $Y(s) = \frac{(s+3)y_0}{(s+1)(s+2)}$ is

$$y(t) = \mathcal{L}^{-1}\left\{\frac{2}{s+1}\right\} + \mathcal{L}^{-1}\left\{\frac{-1}{s+2}\right\} \tag{14.32}$$

By using the Laplace transform table we find that

$$y(t) = 2e^{-t} - 1e^{-2t} \tag{14.33}$$

[SI*] It is usually desired to determine the **steady-state** or final value of the response of $y(t)$. For example, the final or steady-state rest position of the spring-mass-damper system may be calculated.

The **final value theorem** states that

$$\lim_{t\to\infty} y(t) = \lim_{s\to 0} sY(s) \tag{14.34}$$

where a simple pole of $Y(s)$ at the origin is permitted, but poles on the imaginary axis and in the right half-plane and repeated poles at the origin are excluded. Therefore, for the specific case of the spring-mass-damper, we find that

$$\lim_{t\to\infty} y(t) = \lim_{s\to 0} sY(s) = 0 \tag{14.35}$$

Hence the final position for the mass is the normal equilibrium position $y = 0$.

[SI*] Reconsider the spring-mass-damper system. The equation for $Y(s)$ may be written as

$$Y(s) = \frac{(s+b/M)y_0}{s^2 + (b/M)s + k/M} = \frac{(s+2\zeta\omega_n)y_0}{s^2 + 2\zeta\omega_n s + \omega_n^2} \tag{14.36}$$

where ζ is the dimensionless **damping ratio**, and ω_n is the **natural frequency** of the system. The roots of the characteristic equation are

$$s_1, s_2 = -\zeta\omega_n \pm \omega_n\sqrt{\zeta^2 - 1} \tag{14.37}$$

where $\omega_n = \sqrt{k/M}$ and $\zeta = b/(2\sqrt{kM})$.

Here are the conditions for ζ:

* When $\zeta > 1$, the roots are real and the system is **overdamped**.
* When $\zeta < 1$, the roots are complex and the system is **underdamped**.
* When $\zeta = 1$, the roots are repeated and real and the condition is called **critical damping**.

When $\zeta < 1$, the response is underdamped, and

$$s_{1,2} = -\zeta\omega_n \pm j\omega_n\sqrt{1-\zeta^2} \tag{14.38}$$

The s-plane plot of the poles and zeros of $Y(s)$

- **The Transfer Function of Linear Systems**

[SI*] The first
[SI*]

- **Block Diagram Models**

[SI*] The first
[SI*]

- **Signal-Flow Graph Models**

[SI*] The first
[SI*]

Bibliography

[1] Ahmad, Kamarul Arifin, Jawaid, Mohammad, Singh, Balbir (2026) Artificial Intelligence for Computational Fluid Dynamics, Academic Press, London, England.

[2] Alpaydin, Ethem (2014) Introduction to Machine Learning 3rd Edition, MIT Press, London, England.

[3] Arbenz, Peter (2018) Lecture Notes on Solving Large Scale Eigenvalue Problems, https://people.inf.ethz.ch/arbenz/ewp/Lnotes/lsevp.pdf, ETH Zürich, Switzerland.

[4] Bazaraa, Mokhtar S., Jarvis, John J., Sherali, Hanif D. (1990) Linear Programming and Network Flows, John Wiley & Sons, New Jersey, United States.

[5] Bishop, Christopher M. (2006) Pattern Recognition and Machine Learning, Springer, New York, United States.

[6] Borse, G.J. (1997) Numerical Methods with MATLAB: A Resouce for Scientists and Engineers, PWS Publishing Company, Worcester, United Kingdom.

[7] Boyce, William E., DiPrima, Richard C. (2010) Elementary Differential Equations and Boundary Value Problems 9th Edition, John Wiley & Sons, Hoboken, New Jersey, United States.

[8] Burden, Richard L., Faires, J. Douglas (2001) Numerical Analysis 7th Edition, Brooks/Cole, Pacific Grove, California, United States.

[9] Colley, Susan Jane (2012) Vector Calculus 4th Edition, Pearson, Boston, MA, United States.

[10] Dorf, Richard C., Bishop, Robert H. (2010) Modern Control Systems 12th Edition, Pearson, New Jersey, United States.

[11] Fabozzi, Frank J. (1999) Bond Markets, Analysis and Strategies 4th Edition, Prentice Hall, New Jersey, United States.

[12] Freitag, Eberhard. Busam, Rolf (2005) Complex Analysis Second Edition, Springer, Berlin, Germany.

[13] Freya, Glanzsche, DS (2025) DianFreya Math Physics Simulator, Berlin-Sentinel Academy of Science, AliceGard.

[14] Gen, Mitsuo, Cheng, Runwei (1997) Genetic Algorithms and Engineering Design, John Wiley & Sons, New York, United States.

[15] Goldberg, David E. (2025) Genetic Algorithms in Search, Optimization and Machine Learning, Addison-Wesley, Canada.

[16] Grother, Patrick J. Blue, James L. (1992) Training Feed Forward Neural Networks Using Conjugate Gradients, Gaithersburg, United States.

[17] Halak, Basel (2026) Convolutional Neural Network Accelerators, Springer, Switzerland.

[18] Heitzinger, Clemens (2022) Algorithms with JULIA, Springer, Switzerland.

[19] Lasthrim, Glanzsche, DS, Freya (2025) Lasthrim Projection 1st Edition, Berlin-Sentinel Academy of Science, AliceGard.

[20] LeCun, Y., Bottou, L., Haffner, P., Bengio, Y. (1998). Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11), 2278–2324.

[21] Miettinen, K., Neittaanmäki, P., Mäkelä, M.M. Periaux, J., Evolutionary Algorithms in Engineering and Computer Science, John Wiley & Sons, West Sussex, England,1999.

[22] Nazarathy, Yoni, Klok, Hayden, Statistics with Julia, Springer, Cham, Switzerland, 2021.

[23] Nelder J.A, Mead R. (1965) A simplex method for function minimization The computer journal, 7(4), 308-313.

[24] Nocedal, Jorge, Wright, Stephen J. (1999) Numerical Optimization, Springer, New York, United States.

[25] Panju, Maysum, Iterative Methods for Computing Eigenvalues and Eigenvectors, The Waterloo Mathematics Review, University of Waterloo, Canada.

[26] Purcell, Rigdon, Varberg (2007) Calculus 9th Edition, Pearson, Upper Saddle River, New Jersey, United States.

[27] Rao, SS (2006) Optimization Theory and Applications 2nd Edition, Wiley Eastern Limited, New Delhi, India.

[28] Kenneth H. Rosen (2006) Discrete Mathematics and its Applications 6th Edition, McGraw-Hill, Boston, United States.

[29] Anton, Rorres (2006) Elementary Linear Algebra with Supplemental Applications 10th Edition, Wiley, Boston, United States.

[30] Anton, Howard, Rorres, Chris, Elementary Linear Algebra with Supplemental Applications 12th edition, Wiley, Hoboken, New Jersey, United States.

[31] Trefethen, Lloyd N., III, David Bau (1997) Numerical Linear Algebra, SIAM, 3600 University City Science Center, Philadelphia, United States.

[32] Vries, Joey de (2020) Learn OpenGL - Graphics Programming, Kendall & Welling, United States.

[33] Walpole, Ronald E., Myers, Raymond H., Myers, Sharon L., Ye, Keying (2012) Probability & Statistics 9th Edition, Pearson, Boston, United States.

[34] Yousif, O. O. O. , Ziadi, R., Almaymuni, A. Z., Saleh, M.A. (2025) Decent Directions Generator Conjugate Gradient Method With Its Application To Train A Two-Layer Neural Network Model, AIMS Mathematics 2025, Volume 10, Issue 11: 26844-26866. doi: 10.3934/math.20251180 (https://www.aimspress.com/article/doi/10.3934/math.20251180)

[35] Zaki, Mohammed J., Meira Jr., Wagner (2006) Data Mining and Analysis: Fundamental Concepts and Algorithms, Wiley Eastern Limited, New Delhi, India.